Voblint

A Verified Goblint-Style Static Analysis Pipeline in Isabelle/HOL

Manuel Lerchner

October 19, 2026

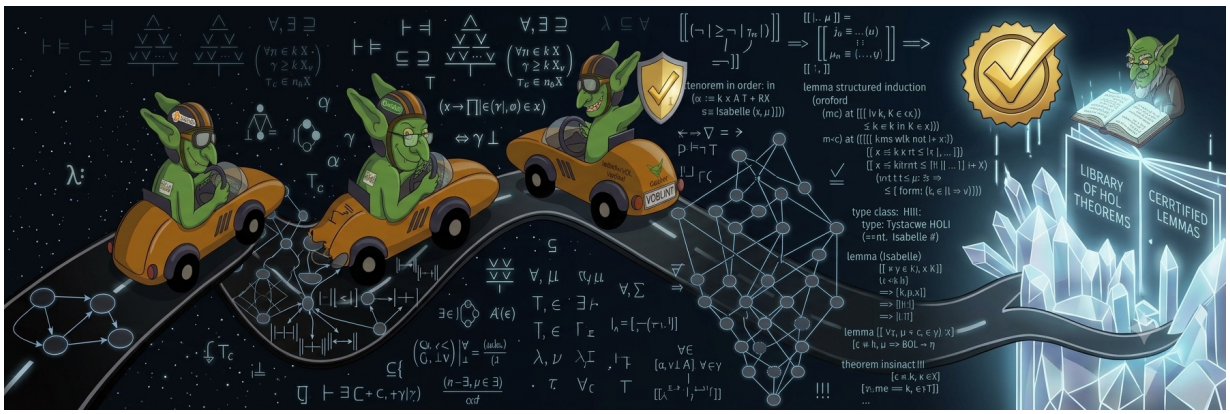
Abstract

Voblint formalizes an executable interprocedural static analysis pipeline in Isabelle/HOL, following Goblint's D/G architecture. The development connects programs in VIMP, a procedural imperative language, to sound abstract results through a verified control-flow graph compiler, an equation generator, and a verified side-effecting top-down solver. The soundness argument applies to the result computed by the analyzer.

The framework separates program semantics, abstract domains, transfer operations, and solver integration. Its D/G interface distinguishes local analysis states from shared information communicated through side effects. Reusable soundness results support context-insensitive and context-sensitive analyses. Concrete instances include sign, interval, parity, and congruence analyses, together with a reduced product of these four domains.

The main soundness theorem relates every finite source-execution prefix to a corresponding control-flow graph state, establishes coverage by the computed abstract result, and proves the correctness of reported check verdicts. It applies to accepted programs and configurations under a solver-termination assumption; source-program termination is unnecessary. The verified pipeline begins with a VIMP abstract syntax tree. Parsing, code generation, and the external compilation and runtime toolchain remain outside the proved guarantees. This document presents the formalization by session, together with an executable end-to-end certificate and the code-export boundary.

The source code and build instructions are available in the Voblint GitHub repository.



Voblint

A Verified Goblint-Style Static Analysis Pipeline in Isabelle/HOL

Master's thesis. Manuel Lerchner, supervised by @AlexandraGrass

CI Ask DeepWiki Isabelle DOI

What is verified?

Voblint proves end-to-end soundness of its executable analyzer. Whenever `run_voblint` returns a report, every configuration a run of the program reaches is simulated at a CFG node, its store is collected there, some state the report holds there describes it, and every definite verdict there holds for it.

```
\underbrace{(\mathit{main}, s_0, [\,]) \to_p^{*} (r, s, \mathit{fs})}_{\text{run of the program}}
\;\Longrightarrow\;
\exists v.\;
\underbrace{(r, s, \mathit{fs}) \approx v}_{\text{simulated at a node}}
\;\wedge\;
\underbrace{s \in \mathcal{C}(v)}_{\text{collecting semantics}}
\;\wedge\;
\underbrace{s \in \gamma_{\mathit{res}}(v)}_{\text{analysis soundness}}
\;\wedge\;
\underbrace{s \models \mathit{verdicts}_{\mathit{res}}(v)}_{\text{verdict soundness}}
```

Here `$\gamma_{res}(v)$` stands for the stores some state the report lists at v describes, and `$s \models verdicts_{res}(v)$` for "every `PROVED` check at v is true in s, every `REFUTED` one false, v is not `DEAD`, and if v is `SAFE` every arithmetic obligation there holds". These are README shorthands; the headline theorem gives the Isabelle statement.

The run is of the program's core form. The parser builds surface VIMP; elaboration (`elab_ptable`, an Isabelle definition) resolves names to variable identities, computes every operation at its machine kind and makes every conversion explicit; `pstep` executes the result. A surface program means the core semantics of its elaboration. That meaning is part of the adequacy argument, not a proved transformation.

Each step is an Isabelle theorem: `source_reaches_node_collect` (a run is simulated at a node whose collected stores contain its store), `report_state_sound` and `report_check_sound` (the report covers every collected store and its verdicts hold there), read at `run_voblint`'s answer by `run_voblint_sound`. `run_voblint_source_sound` states the whole chain for every run that returns a report.

The context-indexed sets $\mathcal{A}(v, c)$ (`activation_collect`) split the collecting semantics by context without loss: $\mathcal{C}(v) = \bigcup_c \mathcal{A}(v, c)$ for each shipped context policy. `run_voblint_spine` states the chain through them for any policy that owes this, and `run_voblint_call_string_chain` and `run_voblint_entry_state_chain` discharge it for call strings and entry states. The guarantees are one-way: the report may describe stores that no run reaches.

What is Voblint?

Voblint is a machine-checked Isabelle/HOL framework for building, running and verifying interprocedural abstract interpreters, modelled on Goblint's D/G architecture. It chains verified CFG compilation, activation-trace operational semantics, executable equation generation, and a vendored verified top-down solver into one soundness statement about the analyzer's own output.

The solver runs inside Isabelle/HOL and computes an abstract **post-solution**. With widening and narrowing in play that is not necessarily a least fixpoint. The end-to-end analyzer corollaries apply to that computed solution rather than to an externally supplied one; the generic framework theorems below them are stated for an arbitrary abstract bound.

Running Voblint

Voblint programs are written in VIMP, a small IMP-style language with `__voblint_check(cond)` assertions:

```
fun main() {
    i32 x = 0;
    while (x < 10) {
        x = x + 1;
    }
    __voblint_check(0 < x);
}
```

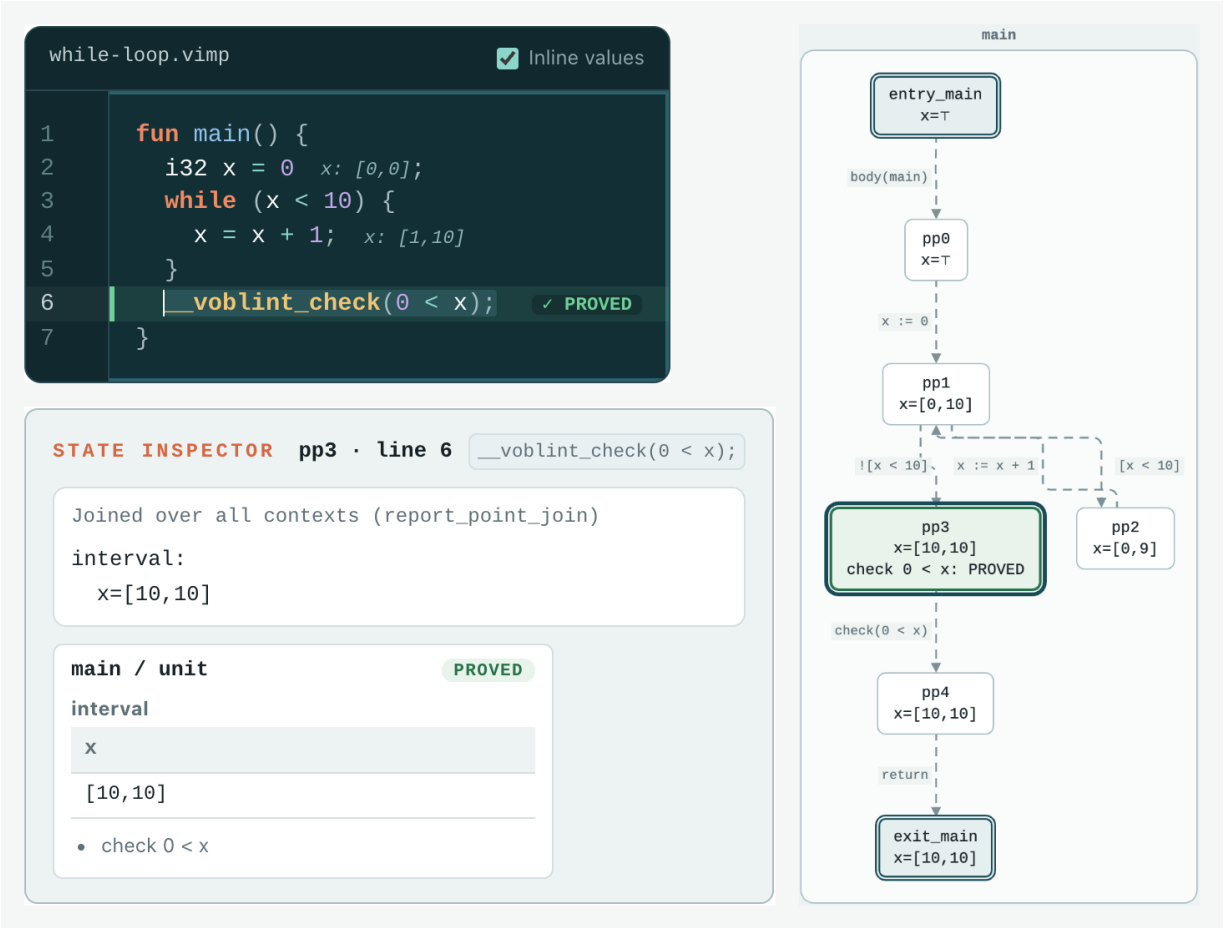
```
$ pixi run voblint --analysis interval tests/regression/02-control-flow/precision/02-while_loop.vimp
tests/regression/02-control-flow/precision/02-while_loop.vimp [interval]

Arithmetic diagnostics
None

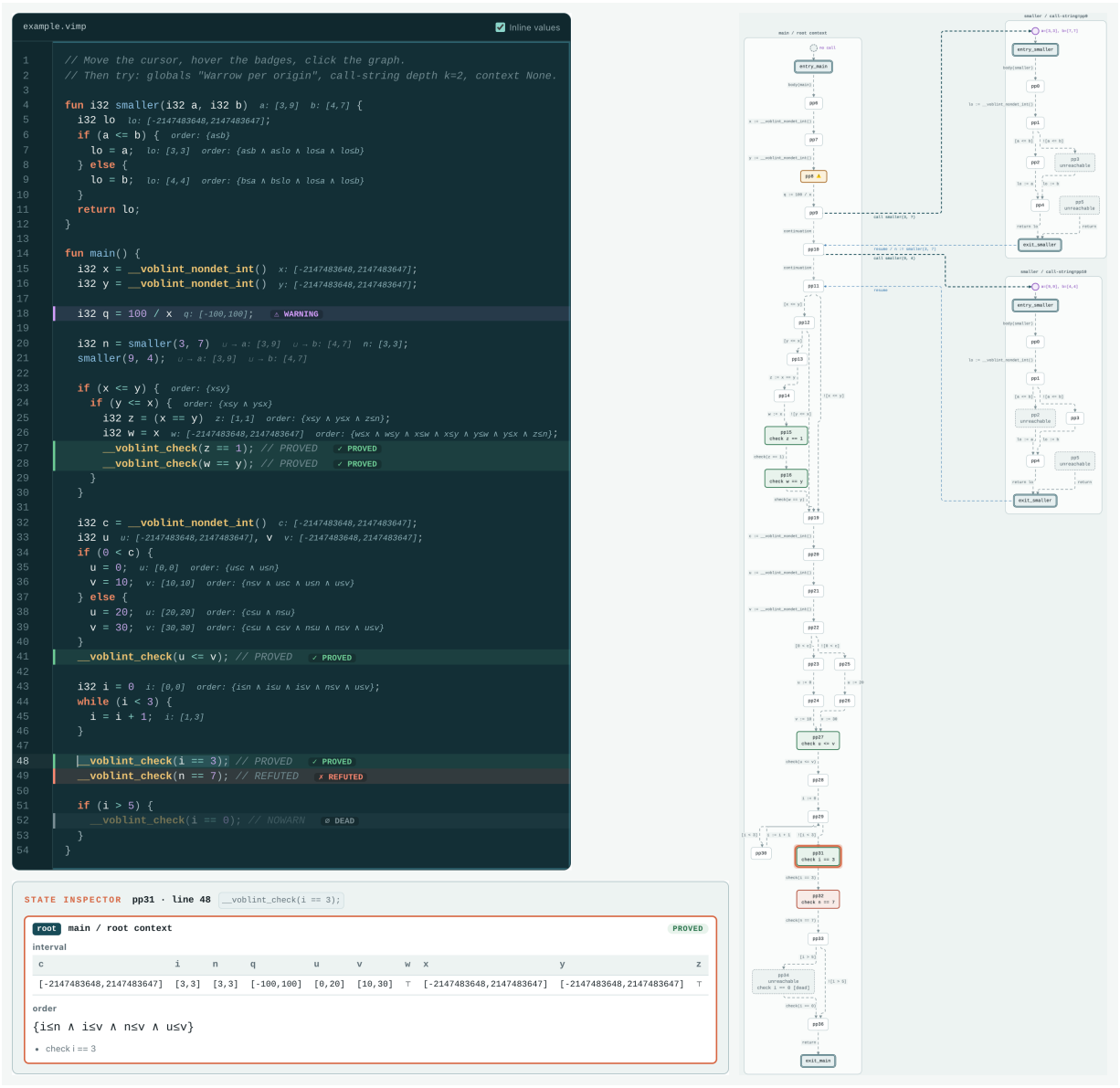
Assertion checks
Location  Point  Condition  Verdict  State
--------  -----  ---------  -------  ------------------
8:3       pp3    0 < x      PROVED   interval: x=[10,10]
```

The browser playground runs the same generated analyzer on a program you edit, with no server involved. Verdicts and value hints appear in the source, the cursor's statement shows its state in every context, and the solved graph draws one box per procedure and context. To explore a local program there, add `--playground`: it opens the playground with the file and the given settings, or those of its `// PARAM:` header, instead of analyzing it locally.

```
pixi run voblint --analysis interval --context entry-state my_program.vimp --playground
```



The program above in the playground. Open this run.



Calls, contexts, a relational domain, verdicts and an arithmetic warning in one run. Open this run.

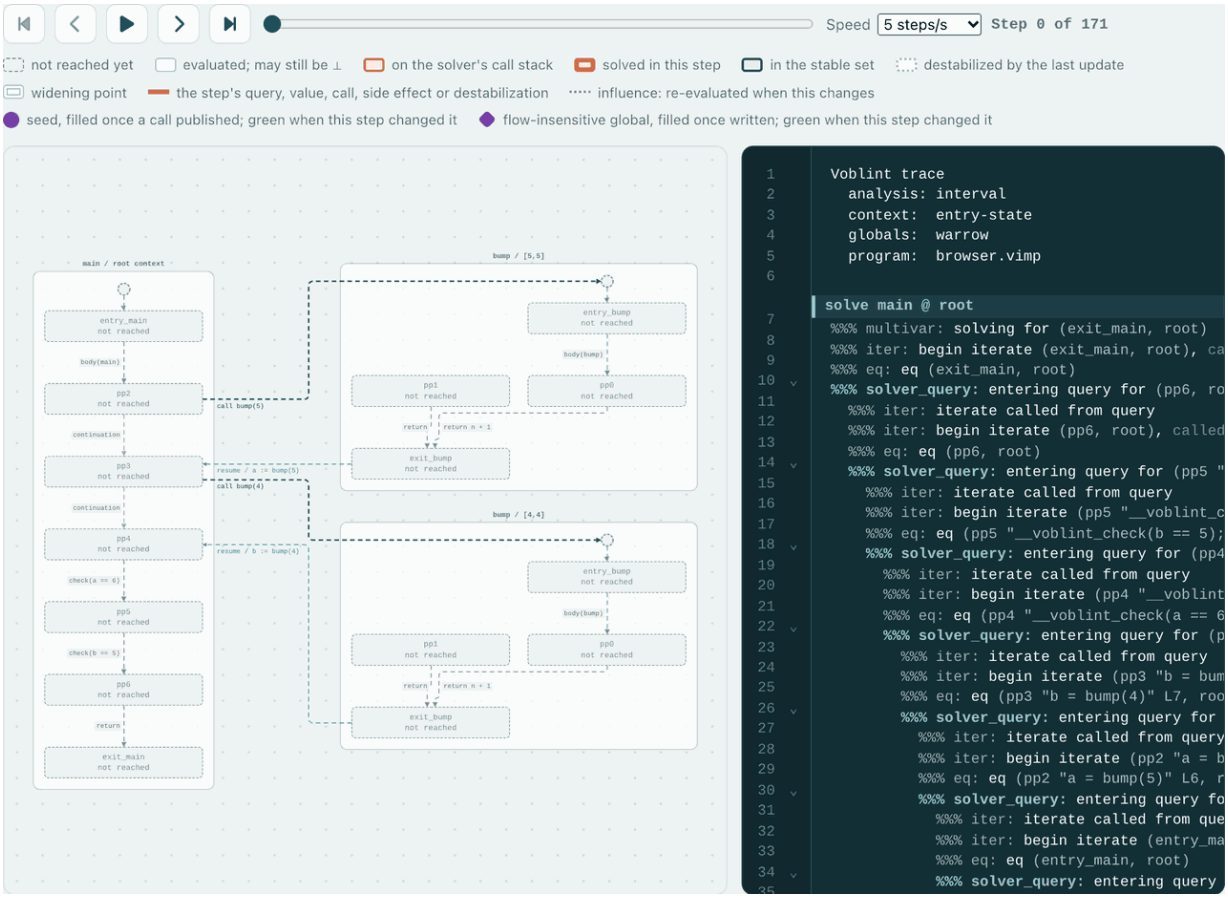
`--context none|entry-state|call-string` selects the analysis context, `--globals` the update rule for side-effected globals, and `--program-globals flow-sensitive|flow-insensitive` whether a program's globals travel with each point's state or each live at its own global unknown, read only by the points that mention it; both placements carry the same soundness theorem. `pixi run voblint --help` lists every analyzer flag; `--playground` is an option of the `pixi run voblint` wrapper. `--graph-snapshot` prints the solved graph as deterministic text and `--parse-only` checks syntax. `docs/CLI_DESIGN.md` describes the CLI trust boundary and `src/Executable_Surface/CLI/README.md` how a run becomes a report.

`--trace` also writes the solver's steps to stderr: per call, the context it is routed to, what the callee's entry reads from its seed, and which publications restart the caller. `--verbose` lists every step in a Goblint-aligned tracing vocabulary (`%%% iter: begin iterate ...`; the mapping table in `docs/CLI_DESIGN.md` records every difference), `--trace-sys iter,side` selects subsystems, `--format jsonl` emits JSON Lines and `--output FILE` writes to a file; standard output stays the same. The playground's **run_voblint: call and answer** panel shows the full (`--verbose`) trace under the call and its answer, colored by step and folded by query, with downloads of the whole text and of its JSON Lines form. Three layers: the solver result is the exported computation of proved equations, and the trace calls inside it come from code equations proved equal to the untraced ones; the trace is an unverified observation of that computation, through a `code_printing` mapping of `trace_event` to an OCaml hook that returns unit, swallows exceptions and never touches solver state, so it cannot feed back; the replay and animation are an unverified visualization of the trace (`docs/CLI_DESIGN.md`).

```
pixi run voblint --analysis interval --context entry-state --trace docs/readme-figures/contexts.vimp
```

The playground's **Solve replay** section steps through the same solve on the graph, with the verbose trace beside

it: the value each unknown holds, the seed each call publishes its entry state into, the stack of open queries and the edge each query follows backward from the exit, with the stable set, destabilization cascades, widening points and influence edges drawn on the graph and the routes and counters below it. One reducer folds the run's JSON Lines trace into every state it shows. `node scripts/capture_readme_figures.mjs solve-replay` regenerates the animation below from the playground.



The context example solved step by step. Open this run.

Arithmetic diagnostics

Every analysis also checks the arithmetic of each evaluated expression against the solved state before the statement: division and remainder by zero, signed `+`, `-`, `*` and `/` whose result may leave its kind, signed `%` whose quotient may leave it (`INT_MIN % -1`), and conversions to a narrower kind that may not fit, including a returned value converted to the caller's destination and a `min`/`max` result converted to its destination. An obligation that may fail is a `warning`; one that fails in every reaching store is an `error`. Safe operations and unreachable points report nothing, and diagnostics never change the exit code.

division-definite.vimp

Inline values

```
1  fun main() {
2    i32 divisor = 0  divisor: [0,0];
3    i32 quotient = 7 / divisor  quotient: [0,0];   ✗ ERROR
4    i32 remainder = 7 % divisor  remainder: [7,7];   ✗ ERROR
5    __voblint_check(quotient == 0);   ✓ PROVED
6    __voblint_check(remainder == 7);   ✓ PROVED
7  }
```

STATE INSPECTOR **pp2 · line 4** `remainder = 7 % divisor`

Joined over all contexts (report_point_join)

interval:
  divisor=[0,0]
  quotient=[0,0]
  remainder=⊤

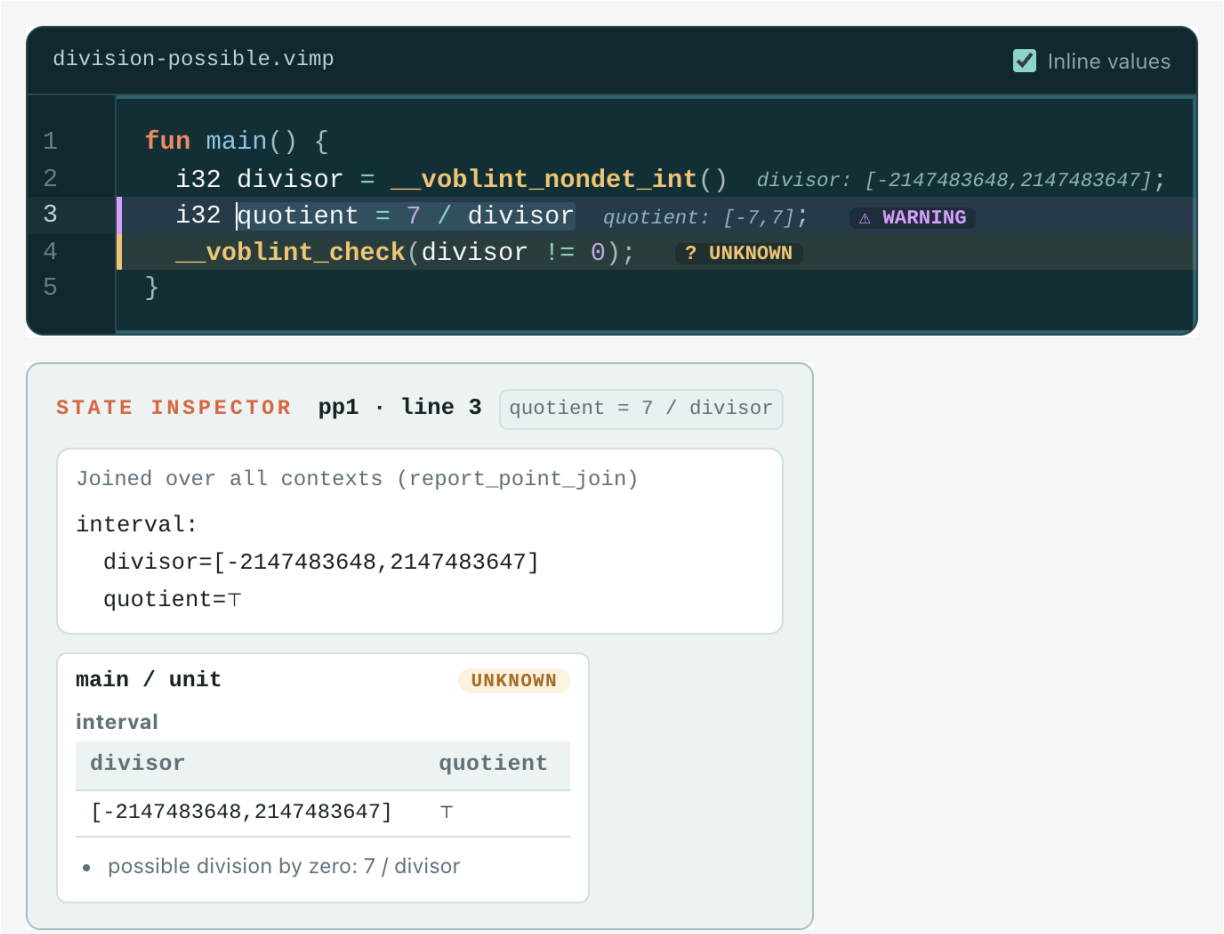
main / unit REFUTED

interval

divisor	quotient	remainder
[0,0]	[0,0]	⊤

- remainder by zero whenever this operation is evaluated: 7 % divisor

Definite: errors, while both checks still prove. Open.



Possible: the state cannot exclude zero. Open.

The traversal covers assignments, guards, call arguments, returns, `min`/`max` and checks, nested arithmetic included, and both operands of `&&` and `||`. It is a diagnostic policy over VIMP's total expressions: execution still uses `a / 0 = 0` and `a % 0 = a`, and wraps every result into its kind. "Possible" means the abstraction cannot exclude the hazard, not that a violating execution exists; an `error` does not prove its point reachable. The arithmetic fixtures and signed-overflow fixtures pin the findings.

Machine integers

Every VIMP value is a machine integer of some kind (`i8` to `u64`). Every variable declares its kind C-style: on a `global` line, on a formal, and on locals, declared anywhere in a body with C's block scoping; a procedure that returns a value declares its return kind. The parser rejects a name that no declaration in scope introduces. A local's initializer is an expression or a call (`u8 r = f(n);`); a local declared without one starts at an arbitrary value of its kind. `(k) e` converts explicitly:

```
global u8 g;
fun u8 f(u8 n) { return n + 1; }
fun main() {
  g = f(300);
  __voblint_check(g == 45); // PROVED: 300 converts to 44 at u8, then 44 + 1
}
```

Elaboration (`Core_Exp.thy`, `Core_Com.thy`) resolves every name to a static identity, computes every operation at its C kind (integer promotion and the usual arithmetic conversions), and inserts a conversion wherever a value is written to a place of another kind. Every result wraps into its kind, signed overflow included (the `-fwrapv` reading, which C leaves undefined); the signed-overflow diagnostics report where a C program could overflow. How a domain treats a possible signed overflow is a run option, Goblint's `sem.int.signed_overflow: assume_top` (the default) or `assume_wraparound`; the theorems hold under both. A literal outside `i64` is rejected. The machine-kind fixtures have the examples.

Three steps before Isabelle are trusted: the parser's block-scope resolution (a declaration that shadows a name gets a fresh name, and a name without a declaration in scope is rejected), the rewriting of a declaration without

initializer into `x = __voblint_nondet_int()`, and the source positions it records.

What Voblint proves

For every selection of analyses, globals rule, context policy, placement of program globals and overflow policy, the report `run_voblint` returns is sound for every execution of the program: whenever an execution reaches a program point, some state the report holds there describes its store, and every `PROVED` or `REFUTED` verdict there is correct for it. The theorems are stated about the same `run_voblint` that is exported to OCaml and runs in the CLI and the playground.

"Every execution" is made precise by an activation-trace collecting semantics. A `valid_activation_trace` records one procedure activation's view of a run, and `node_collect` gathers every store such traces hold at each CFG node, adapting thread-modular local traces (see Foundations) from threads to procedure activations. The public theorems read it as `reaching ds p v`, the stores that reach node `v` of the elaborated graph `elab_cfg p ds` from `init_stores` (globals zero, every value in its kind).

The headline theorem

`run_voblint_source_sound` relates any finite run of the program to the report:

```
theorem run_voblint_source_sound:
  assumes run: "run_voblint conf ds p = Analysed res"
    and s0: "s0 ∈ init_stores (kenv_of ds)"
    and exec: "star (pstep (kenv_of ds) (elab_ptable p ds))
                   (main_body (elab_ptable p ds), s0, []) (residual, s, frs)"
  shows "∃v stk. elab_ptable p ds, elab_cfg p ds, prog_main_name
                     ⊢ (residual, s, frs) ≈ (v, s, stk)
           ∧ s ∈ reaching ds p v
           ∧ (∃d. Lifted d ∈ report_states_at res v
                  ∧ s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d)
           ∧ ¬ DEAD res v
           ∧ (∀c. PROVED res v c ⟶ truthy (eval c s))
           ∧ (∀c. REFUTED res v c ⟶ ¬ truthy (eval c s))
           ∧ (SAFE res v ⟶ (∀ob ∈ obligations_at ds p v. s ⊨ ob))"
```

Premise	Meaning
`run`	`run_voblint` accepted the configuration and the program, and its solve finished.
`s0`	Initial store: globals start at `0`, every identity holds a value of its kind.
`exec`	Any finite execution prefix of the elaborated program from `main`; the program need not terminate.

Conclusion	Guarantee
`... ≈ (v, s, stk)`	The reached configuration is simulated at CFG node `v` by the compiler's simulation relation `csim`, with the same store.
`s ∈ reaching ds p v`	A valid activation trace reaches `v` with store `s`.
`∃d. ...`	Some state the report holds at `v` describes `s`, through the concretization of the activated analyses.
`¬ DEAD res v`	`v` is not reported dead.
`PROVED, REFUTED`	Every definite verdict at `v` holds for `s`.
`SAFE`	If `v` has no arithmetic diagnostic, every obligation decided there holds of `s`.

`ds` are the kind declarations the parser hands over; `conf` names a nonempty list of distinct analyses among Interval, Congruence, Int (at a refinement mode), Order, Sign and Parity, run together, one of the globals rules, a context policy (call strings of any length, length `0` being context-insensitive, or entry states), the placement of program globals, and the signed-overflow policy.

`run_voblint_sound` states the same guarantees at a node, for every store in `reaching ds p v`, without a run.

Checks, dead code and arithmetic

Results in the same theory specialise the guarantee to what the report shows:

- run_voblint_check_sound: when an execution is about to run a check, the report lists a row under that check's label at a node reached with the current store, the row is not Dead, and a definite verdict is correct. run_voblint_labelled_check_sound adds that, when no two rows share a label, every row carrying the label is that row.
- run_voblint_dead_unreached: no store reaches a DEAD point.
- run_voblint_arithmetic_safe: at a point with no arithmetic diagnostic, every obligation of every expression an edge or call there evaluates holds in every store that reaches it, and every value returned there fits the kind a caller converts it to. run_voblint_no_signed_overflow, run_voblint_no_remainder_overflow and run_voblint_no_conversion_overflow read it for signed operations, signed remainders and conversions.
- run_voblint_arithmetic_refuted: the obligation of an error fails in every store that reaches its point.

Exact statements

```
theorem run_voblint_report_contract:
  assumes "run_voblint conf ds p = Analysed res"
  shows "valid_activation (conf_analyses conf)" and "wf_program_compile_input_exec p"
    and "report_config res = conf" and "report_cfg res = elab_cfg p ds"

theorem run_voblint_check_sound:
  assumes run: "run_voblint conf ds p = Analysed res"
    and s0: "s0 ∈ init_stores (kenv_of ds)"
    and exec: "star (pstep (kenv_of ds) (elab_ptable p ds))
                 (main_body (elab_ptable p ds), s0, []) (residual, s, frs)"
    and chk: "next_check residual = Some (l, e)"
  shows "∃c ∈ set (report_checks res). check_label c = l ∧ check_exp c = e
           ∧ s ∈ reaching ds p (check_point c)
           ∧ check_verdict c ≠ Dead
           ∧ (check_verdict c = Decided Check_Proved ⟶ truthy (eval e s))
           ∧ (check_verdict c = Decided Check_Refuted ⟶ ¬ truthy (eval e s))"

corollary run_voblint_dead_unreached:
  "run_voblint conf ds p = Analysed res ⟹ DEAD res v ⟹ reaching ds p v = {}"

theorem run_voblint_arithmetic_refuted:
  assumes run: "run_voblint conf ds p = Analysed res"
    and d: "d ∈ set (report_diagnostics res)" and r: "diagnostic_verdict d = Check_Refuted"
    and s: "s ∈ reaching ds p (diagnostic_point d)"
  shows "¬ s ⊨ diagnostic_obligation d"
```

Limits

The result is partial correctness. run_voblint runs the executable solver, which answers only where the solve terminates, so the theorems need no termination premise; nothing proves that the solve terminates for every program, and where it does not, the analyzer gives no answer. Neither PROVED nor REFUTED establishes that its point is reachable, so REFUTED is not a verified counterexample. DEAD is the only reachability claim: no covered execution reaches the point. The converse does not hold: a point no execution reaches need not be DEAD.

Covered by the proof	Outside the proof
Execution of the elaborated core program (pstep on elab_ptable p ds)	Solver termination for arbitrary programs
Compilation to the procedure-aware CFG (elab_cfg) and its simulation	Lexing, parsing, block-scope resolution, and the rewriting of a declaration without initializer
Equation generation and the computed post-solution	Isabelle code generation, the OCaml compiler and runtime
The report's states, check rows under their labels, arithmetic safety at quiet points, failing obligations at errors	That a check's label is its source position; diagnostic positions; rendered graphs and state strings; the playground
	That elaboration and pstep model C's meaning of the source (an adequacy argument, not a theorem)
	Completeness and precision

`Example_Non_Vacuity` evaluates a concrete answer and discharges the headline theorem's premises for it, so the theorem is not vacuous. `docs/THEOREM_MAP.md` lists the lower-level statements.

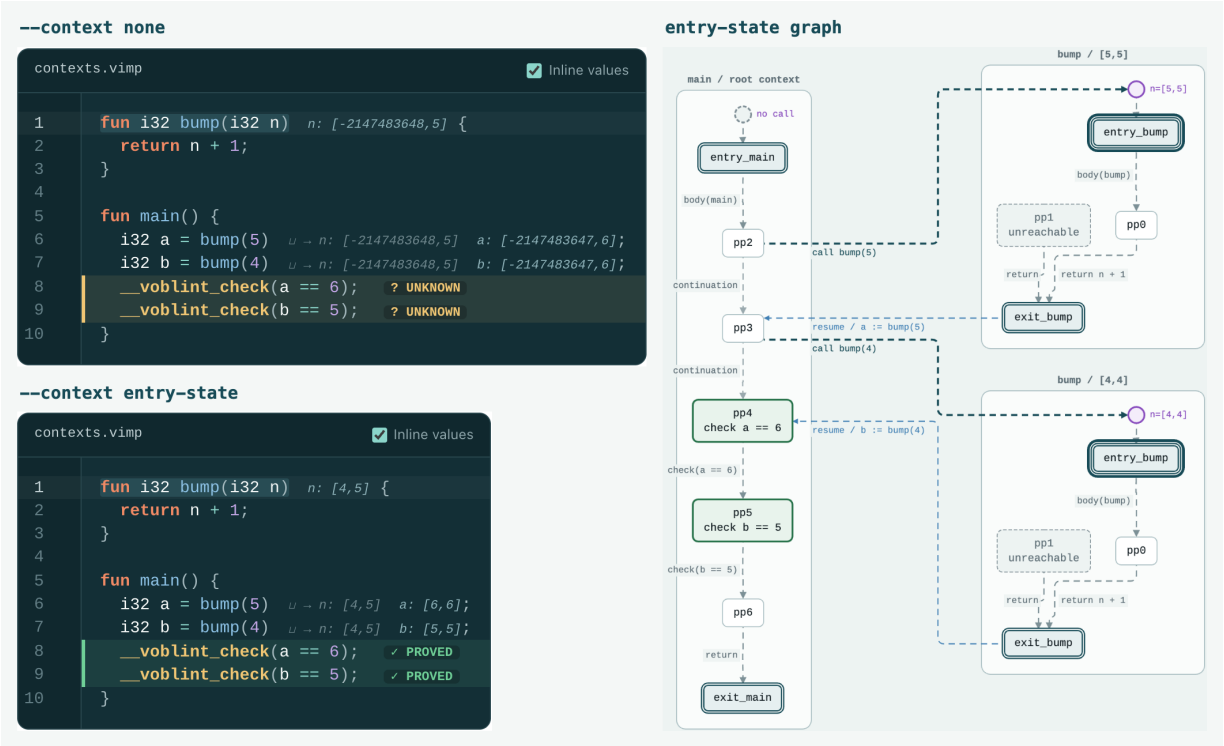
Notation

Theorem statements use a few symbols the theories declare. Each name links to its declaration.

Symbol	Reads as	Isabelle	Meaning
,Π c →p c' / ,Π c →p* c'	c steps to c	`pstep` / `psteps`	one small step of a core VIMP configuration (command, store, frame stack) under kind environment and procedure table Π; every value it writes is converted to its destination's kind; the starred arrow is its reflexive-transitive closure
Π,g,p c c'	c is matched with c	`csim`	the compiler's simulation relation between a source configuration c whose outermost activation runs procedure p and a configuration c of the compiled graph g
	the kind environment	`kenv_of`	the machine kind of every variable identity; a program supplies it from its kind declarations, an undeclared identity being I32, and the theorems take it as a parameter
,g,S	the valid activation traces of g from S	`valid_activation_trace`	the activation traces of graph g from initial stores S
,g,S v	the stores collected at v	`node_collect`	the stores valid activation traces reach at node v
,adm,c ,g,S v c	the stores collected at v in context c	`activation_collect`	the stores valid activation traces carrying context c reach at node v; *adm* admits callee contexts and the root runs in c
carries t cIn `activation_coverage`, short for activation_context_rel adm c0 g	t carries context c	`carries`	the context relation of the locale's policy, between a valid activation trace and the contexts it may run in

Reading a result

Each figure is a playground run; the link under it opens the same program and settings.



Context sensitivity, Interval

Without contexts, `bump(5)` and `bump(4)` share one state for `n`, so neither result is exact and both checks are UNKNOWN. `--context entry-state` keeps a separa

Open without contexts · open with entry-state contexts

The generic D/G framework

Analyses are factored the way Goblint factors them: **D** holds the abstract facts attached to program points, **G** the information published and consumed across points. The framework's interface is `dg_spec`, the Isabelle counterpart of Goblint's `Spec` (transfer, entry, combine and communication). Equation generation with context routing, solver integration and the lifting to source-level soundness are proved once, for any `dg_spec` with a `analysis_contract` relating it to concrete stores; see `src/Abstract_Interpreter/Framework/README.md`.

Every analysis selectable through `run_voblint` enters through a smaller interface, a `local_spec`: one transfer per edge kind plus entry and combine, over local state only, with the soundness condition `sound_local_spec`. `dg_spec_of` lifts it to a `dg_spec`, and `dg_spec_of_contract` supplies the contract. The activated analyses run as one state, one field each, as in Goblint's MCP (`MCP_Registry`, `MCP_Carrier`), and answer each other's queries. With flow-insensitive program globals, `keyed_split_spec` wraps that specification so each edge reads only the globals its expressions mention and publishes only the globals it may assign, each at its own unknown, and `keyed_split_contract` carries the contract over.

A numeric domain enters as an instance of the class `kind_int_domain`: values at a machine kind with kind-aware arithmetic, conversion, comparison, backward refinement and top, each with its soundness assumption. The non-relational transfer `nonrel_spec` is written once over any such domain and proved sound once (`nonrel_spec_sound`); its executable carrier is `nonrel_exec_spec` (`Nonrelational_Exec`). Sign, Interval, Parity, Congruence and the Interval-Congruence product `int_tuple` are instances. The relational Order analysis writes its `local_spec` by hand (`ord_spec`). Deriving transfer functions from certified value operations follows Nipkow's Isabelle abstract interpreter (see Foundations).

Building

Requirements: **Isabelle2025-2**, an **AFP** checkout, **pixi**, and OCaml via opam for the CLI.

```
./scripts/setup.sh                              # pixi, OCaml and developer tooling
pixi run vendor-init                            # fetch the TD solver submodule
AFP=/path/to/afp/thys pixi run isabelle-build   # check the formalization
```

On a fresh clone without parent heaps, run `pixi run isabelle-bootstrap` first, with the same `AFP` setting. `pixi run isar-check` checks ROOT entries, unfinished proofs, theory syntax and formatting with isar-tools, without Isabelle or the AFP. `pixi task list` lists everything else.

The vendored solver is pinned to a private fork of stilscher/td-verification; CI needs a 'SUBMODULES_TOKEN' secret to clone it. Making that fork public is the outstanding step for a fully reproducible artifact.

Foundations

- **Goblint** (GitHub): modular interprocedural abstract interpretation and the D/G architecture.
- **Abstract Interpretation of Annotated Commands** (ITP 2012): reusable abstract interpretation in Isabelle/HOL.
- **The Top-Down Solver Verified** (CAV 2024): the vendored executable verified solver.
- **Mixed Flow-Sensitive Static Analysis: Engineering Modularity** (FM 2026).
- **Data Race Detection by Digest-Driven Abstract Interpretation** (arXiv:2511.11055): thread-modular local-trace semantics of a multithreaded program. Voblint's activation traces (`src/Program_Model/CFG/Collecting/`) adapt its shape to procedure activations of a sequential run.

Documentation

Document	Contents
`AGENTS.md`	Project contract, style rules, working agreements
`docs/PROOF_OVERVIEW.md`	Proof architecture and intended claims
`docs/THEOREM_MAP.md`	Thesis claims mapped to checked Isabelle theorems
`docs/GLOSSARY.md`	Terminology and defining layers
`docs/VERIFICATION_CHAIN_AND_TRUST_BOUNDARY.md`	What is proved, what is trusted
`docs/GOBLINT_ALIGNMENT_REGISTER.md`	Where this formalization differs from upstream Goblint, and why
`docs/ISABELLE_AGENT_NOTES.md`	Build commands, agent-assisted development (I/Q, I/R), `./scripts/setup.sh`
`docs/ROADMAP.md`, `docs/NON_GOALS.md`	Scope and priorities

License

Voblint is available under the `BSD-3-Clause` license. Vendored dependencies retain their own licenses.

Contents

1 Voblint_VIMP **1**
- 1.1 Machine_Kind . . . 2
- 1.2 VIMP_Syntax . . . 6
- 1.3 VIMP_Globals . . . 7
- 1.4 VIMP_Expr . . . 8
- 1.5 VIMP_Special . . . 10
- 1.6 VIMP_Proc . . . 11
- 1.7 VIMP_Program . . . 16
- 1.8 Var_Id . . . 18
- 1.9 Core_Exp . . . 20
- 1.10 Core_Com . . . 25
- 1.11 VIMP_Semantics . . . 28
- 1.12 VIMP_Grammar_Generated . . . 33
- 1.13 VIMP_Notation . . . 37

2 Voblint_Domain **40**
- 2.1 Abstract_Domain . . . 41
- 2.2 Kind_Lifted . . . 43
- 2.3 Reachability_Lift . . . 46
- 2.4 Three_Valued . . . 52
- 2.5 Query_Lift . . . 53
- 2.6 State_Concretization . . . 55
- 2.7 Nonrelational_State . . . 56
- 2.8 Nonrelational_Reachability . . . 59
- 2.9 Forward_Domain . . . 60
- 2.10 Backward_Domain . . . 61
- 2.11 Backward_Domain_Mono . . . 63
- 2.12 Numeric_Queries . . . 64
- 2.13 Backward_Numeric_Queries . . . 66
- 2.14 Interval_Lattice . . . 68
- 2.15 Sign_Lattice . . . 75
- 2.16 Parity_Lattice . . . 79
- 2.17 Congruence_Lattice . . . 82
- 2.18 Int_Lattice . . . 96

3 Voblint_Solver **107**
- 3.1 Strategy_Program_Fold . . . 108
- 3.2 Strategy_Tree_Program . . . 113
- 3.3 Strategy_Tree_Side_Buffering . . . 118
- 3.4 Strategy_Tree_Pair . . . 124
- 3.5 TD_Solver_Bridge . . . 126
- 3.6 Globals_Rule . . . 127
- 3.7 Solver_Trace . . . 130

4 Voblint_CFG **133**
- 4.1 CFG_Def . . . 134
- 4.2 CFG_Transfer . . . 139
- 4.3 CFG_Prune . . . 141
- 4.4 CFG_Exec . . . 143

4.5 Activation_Trace_Def . . . 145
4.6 Activation_Trace_Context . . . 150
4.7 Activation_Trace_Collect . . . 156
4.8 Activation_Trace_Abstract . . . 160

5 Voblint_Compile **165**
5.1 VIMP_Proc_to_CFG . . . 166
5.2 Compile_Wellformed . . . 173
5.3 Compile_Invariants . . . 182
5.4 Residual_Location . . . 188
5.5 Residual_Edges . . . 191
5.6 Simulation_Relation . . . 203
5.7 Simulation_Preservation . . . 211
5.8 Source_Progress . . . 223
5.9 Procedure_Ownership . . . 227
5.10 Live_Nodes . . . 236
5.11 Source_To_Trace . . . 243
5.12 Kind_Invariants . . . 248

6 Voblint_Framework **252**
6.1 CFG_Enumeration . . . 253
6.2 DG_State . . . 261
6.3 DG_Constraint_Programs . . . 263
6.4 DG_Manager . . . 268
6.5 DG_Spec . . . 271
6.6 DG_Spec_Sound . . . 278
6.7 Transfer_Algebra . . . 283
6.8 State_Restriction . . . 285
6.9 DG_Ownership_Split_Spec . . . 286
6.10 DG_Indexed_Generator . . . 287
6.11 DG_Reader_Transport . . . 301
6.12 Activation_Backbone . . . 303
6.13 DG_Ctx_Activation . . . 304
6.14 Routed_Call_Programs . . . 308
6.15 Routed_Context . . . 316
6.16 DG_Keyed_Split_Spec . . . 326
6.17 Analysis_Query . . . 332
6.18 Local_Spec_Product . . . 334
6.19 MCP_Spec . . . 336
6.20 Call_String_Context . . . 351
6.21 Solved_Table . . . 353
6.22 Check_Result . . . 356
6.23 Check_Report . . . 357
6.24 Contextual_Check_Report . . . 359
6.25 Check_Answer . . . 363
6.26 MCP_Registry . . . 364
6.27 Bool_Refine . . . 367
6.28 DG_Analysis_Adapter . . . 369
6.29 Routed_Analysis_Sound . . . 374

7 Voblint_Exec **376**
7.1 Default_St_Base . . . 377
7.2 Default_St_Algebra . . . 383
7.3 Default_St_Transfer . . . 387
7.4 Default_St_Reachability . . . 394
7.5 Routed_Exec_Refinement . . . 395

8 Voblint_Routing **397**
8.1 Compiled_Routed_Equations . . . 398
8.2 Context_Space_Finite . . . 400

9 Voblint_Result **402**
9.1 Source_Activation_Sound . . . 403
9.2 DG_Result_Construction . . . 405
9.3 Analysis_Surface . . . 406
9.4 DG_Analysis . . . 408
9.5 DG_Live_Unknowns . . . 427

10 Voblint_Nonrelational **438**
10.1 Nonrelational_Domain . . . 439
10.2 Nonrelational_Transfer . . . 445
10.3 Nonrelational_Exec . . . 455
10.4 Nonrelational_Analysis . . . 461

11 Voblint_Analysis_Sign **464**
11.1 Sign_Warrowing . . . 465
11.2 Sign_Arithmetic . . . 466
11.3 Sign_Backward . . . 469
11.4 Sign_Kind . . . 470

12 Voblint_Analysis_Interval **474**
12.1 Interval_Warrowing . . . 475
12.2 Interval_Arithmetic . . . 479
12.3 Interval_Numeric_Queries . . . 486
12.4 Interval_Kind . . . 490
12.5 Interval_Regression . . . 495

13 Voblint_Analysis_Parity **497**
13.1 Parity_Warrowing . . . 498
13.2 Parity_Domain . . . 499
13.3 Parity_Kind . . . 501

14 Voblint_Analysis_Congruence **503**
14.1 Congruence_Warrowing . . . 504
14.2 Congruence_Arithmetic . . . 506
14.3 Congruence_Kind . . . 512
14.4 Congruence_Regression . . . 517

15 Voblint_Analysis_Order **519**
15.1 Order_Analysis . . . 520
15.2 Order_Regression . . . 526

16 Voblint_Analysis_Int **527**
16.1 Int_Warrowing . . . 528
16.2 Int_Refinement . . . 531
16.3 Int_Refinement_Control . . . 549
16.4 Int_Tuple_Kind . . . 554
16.5 Int_Regression . . . 561

17 Voblint_CLI **563**
17.1 Analysis_Config . . . 564
17.2 MCP_Carrier . . . 565
17.3 MCP . . . 569
17.4 Dispatch_Carrier . . . 572
17.5 MCP_Split . . . 574
17.6 Analysis_Run . . . 584
17.7 Analysis_Render . . . 588
17.8 Analysis_Report . . . 593
17.9 Run_Report . . . 596
17.10 Run_Voblint . . . 609
17.11 Trace_Run . . . 617
17.12 Arithmetic_Diagnostics . . . 620

18 Voblint_Codegen **623**
18.1 Voblint_Codegen 624

A End-to-end certificate **627**
A.1 Example_Non_Vacuity 628

Chapter 1

Voblint_VIMP

1.1 Machine_Kind

```
theory Machine_Kind
  imports Main "HOL-Library.Countable" "Deriving.Compare_Order_Instances"
begin
```

Machine-integer kinds

A kind fixes a bit width and a signedness, in the role of Goblint's `ikind`. It determines a representable range, and `ik_norm` maps a mathematical integer onto that range by two's-complement wraparound. Kinds are fixed-width, so nothing plays the part of C's platform-dependent type sizes.

Wraparound is the defined behaviour for signed kinds too. C leaves signed overflow undefined; this semantics instead gives it the meaning `-fwrapv` gives it, which keeps evaluation total. A claim proved against it is a claim about wrapping arithmetic, not about C programs whose signed arithmetic overflows.

The carrier stays `int`: a store holds one integer per variable whatever its kind, and the kind is read off the syntax.

```
datatype ikind = I8 | U8 | I16 | U16 | I32 | U32 | I64 | U64

instance ikind :: countable
  by countable_datatype

derive linorder ikind

fun ik_bits :: "ikind ⇒ nat" where
  "ik_bits I8 = 8" | "ik_bits U8 = 8"
| "ik_bits I16 = 16" | "ik_bits U16 = 16"
| "ik_bits I32 = 32" | "ik_bits U32 = 32"
| "ik_bits I64 = 64" | "ik_bits U64 = 64"

fun ik_signed :: "ikind ⇒ bool" where
  "ik_signed I8 = True" | "ik_signed U8 = False"
| "ik_signed I16 = True" | "ik_signed U16 = False"
| "ik_signed I32 = True" | "ik_signed U32 = False"
| "ik_signed I64 = True" | "ik_signed U64 = False"

lemma ik_bits_pos [simp]: "0 < ik_bits ik"
  by (cases ik) simp_all
```

Range and wraparound

A kind's range is the interval its values occupy, and `ik_norm` retracts every integer onto it.

```
definition ik_mod :: "ikind ⇒ int" where
  "ik_mod ik = 2 ^ ik_bits ik"

definition ik_half :: "ikind ⇒ int" where
  "ik_half ik = 2 ^ (ik_bits ik - 1)"

definition ik_min :: "ikind ⇒ int" where
  "ik_min ik = (if ik_signed ik then - ik_half ik else 0)"

definition ik_max :: "ikind ⇒ int" where
  "ik_max ik = (if ik_signed ik then ik_half ik - 1 else ik_mod ik - 1)"

definition ik_range :: "ikind ⇒ int set" where
  "ik_range ik = {ik_min ik..ik_max ik}"
```

`ik_norm` is the truncation primitive a bit-vector type would use, applied to an integer: `take_bit` for an unsigned kind, `signed_take_bit` for a signed one. Their library lemmas carry the arithmetic congruences below.

```
definition ik_norm :: "ikind ⇒ int ⇒ int" where
  "ik_norm ik n =
     (if ik_signed ik then signed_take_bit (ik_bits ik - 1) n else take_bit (ik_bits ik) n)"

lemma ik_range_iff [simp, code_unfold]:
  "n ∈ ik_range ik ⟷ ik_min ik ≤ n ∧ n ≤ ik_max ik"
  unfolding ik_range_def by simp

lemma zero_in_ik_range [simp, intro]: "0 ∈ ik_range ik"
  by (cases ik) (simp_all add: ik_min_def ik_max_def ik_half_def ik_mod_def)

lemma one_in_ik_range [simp, intro]: "1 ∈ ik_range ik"
  by (cases ik) (simp_all add: ik_min_def ik_max_def ik_half_def ik_mod_def)
```

```
lemma ik_norm_in_range [simp, intro]: "ik_norm ik n ∈ ik_range ik"
proof (cases "ik_signed ik")
  case True
  have "- (2 ^ (ik_bits ik - 1)) ≤ signed_take_bit (ik_bits ik - 1) n"
    by (rule signed_take_bit_int_greater_eq_minus_exp)
  moreover have "signed_take_bit (ik_bits ik - 1) n < 2 ^ (ik_bits ik - 1)"
    by (rule signed_take_bit_int_less_exp)
  ultimately show ?thesis
    using True by (simp add: ik_norm_def ik_min_def ik_max_def ik_half_def)
next
  case False
  have "0 ≤ take_bit (ik_bits ik) n"
    by (rule take_bit_nonnegative)
  moreover have "take_bit (ik_bits ik) n < 2 ^ ik_bits ik"
    by (rule take_bit_int_less_exp)
  ultimately show ?thesis
    using False by (simp add: ik_norm_def ik_min_def ik_max_def ik_mod_def)
qed

lemma ik_norm_ge_min [simp]: "ik_min ik ≤ ik_norm ik n"
  and ik_norm_le_max [simp]: "ik_norm ik n ≤ ik_max ik"
  using ik_norm_in_range [of ik n] by simp_all

lemma ik_min_le_zero [simp]: "ik_min ik ≤ 0" and zero_le_ik_max [simp]: "0 ≤ ik_max ik"
  and ik_min_le_one [simp]: "ik_min ik ≤ 1" and one_le_ik_max [simp]: "1 ≤ ik_max ik"
  by (cases ik; simp add: ik_min_def ik_max_def ik_half_def ik_mod_def)+

lemma ik_norm_id [simp]:
  assumes "n ∈ ik_range ik"
  shows "ik_norm ik n = n"
proof (cases "ik_signed ik")
  case True
  with assms have "- (2 ^ (ik_bits ik - 1)) ≤ n" and "n < 2 ^ (ik_bits ik - 1)"
    by (simp_all add: ik_min_def ik_max_def ik_half_def)
  from signed_take_bit_int_eq_self [OF this] True show ?thesis
    by (simp add: ik_norm_def)
next
  case False
  with assms have "0 ≤ n" and "n < 2 ^ ik_bits ik"
    by (simp_all add: ik_min_def ik_max_def ik_mod_def)
  from take_bit_int_eq_self [OF this] False show ?thesis
    by (simp add: ik_norm_def)
qed

lemma ik_norm_idem [simp]: "ik_norm ik (ik_norm ik n) = ik_norm ik n"
  by (rule ik_norm_id [OF ik_norm_in_range])

lemma ik_norm_nonneg_unsigned: "¬ ik_signed ik ⟹ 0 ≤ ik_norm ik n"
  by (simp add: ik_norm_def)
```

Wraparound subtracts a multiple of an even modulus, so it preserves parity.

```
lemma even_ik_norm_iff [simp]: "even (ik_norm ik n) ⟷ even n"
  by (cases ik) (simp_all add: ik_norm_def even_take_bit_eq even_signed_take_bit_iff)

lemma ik_norm_zero [simp]: "ik_norm ik 0 = 0"
  by (rule ik_norm_id [OF zero_in_ik_range])

lemma ik_norm_one [simp]: "ik_norm ik 1 = 1"
  by (rule ik_norm_id [OF one_in_ik_range])
```

`ik_norm` as modular arithmetic, the form the interval argument below needs.

```
lemma ik_mod_pos [simp]: "0 < ik_mod ik"
  unfolding ik_mod_def by simp

lemma ik_norm_mod: "ik_norm ik n = (n - ik_min ik) mod ik_mod ik + ik_min ik"
proof (cases "ik_signed ik")
  case True
  have suc: "Suc (ik_bits ik - 1) = ik_bits ik" using ik_bits_pos [of ik] by simp
  show ?thesis
    using signed_take_bit_eq_take_bit_shift [of "ik_bits ik - 1" n]
    by (simp add: ik_norm_def ik_min_def ik_max_def ik_half_def ik_mod_def
                  take_bit_eq_mod suc True algebra_simps)
next
  case False
```

```
  then show ?thesis
    by (simp add: ik_norm_def ik_min_def ik_max_def ik_mod_def take_bit_eq_mod)
qed
```

Wrapping the endpoints of an interval no wider than the kind's range and finding them still ordered certifies that wraparound shifts the whole interval by one amount, so every value in between keeps its distance to the endpoints. Goblint's `IntervalDomain.norm ~cast:true` relies on the same fact.

```
lemma ik_norm_interval_shift:
  assumes width: "h - l ≤ ik_max ik - ik_min ik"
    and ordered: "ik_norm ik l ≤ ik_norm ik h"
    and lv: "l ≤ v" and vh: "v ≤ h"
  shows "ik_norm ik v = ik_norm ik l + (v - l)"
proof -
  define M where "M = ik_mod ik"
  define m where "m = ik_min ik"
  have Mpos: "0 < M" unfolding M_def by simp
  have bits2: "2 * (2::int) ^ (ik_bits ik - 1) = 2 ^ ik_bits ik"
    using ik_bits_pos [of ik]
    by (metis One_nat_def Suc_pred power_Suc)
  have max_min_mod: "ik_max ik - ik_min ik + 1 = M"
    unfolding M_def ik_mod_def ik_max_def ik_min_def ik_half_def
    using bits2 by (simp split: if_splits)
  have width': "h - l < M" using width max_min_mod by linarith
  have wrap: "ik_norm ik x = (x - m) mod M + m" for x
    unfolding M_def m_def by (rule ik_norm_mod)
  have dist: "(x - m) mod M = ((l - m) mod M + (x - l)) mod M" if "l ≤ x" for x
  proof -
    have "x - m = (l - m) + (x - l)" by simp
    then show ?thesis by (metis mod_add_left_eq)
  qed
  have hv: "(h - m) mod M = ((l - m) mod M + (h - l)) mod M" using dist[OF order_refl[of l]] lv vh
    by (metis dist order_trans)
  have vv: "(v - m) mod M = ((l - m) mod M + (v - l)) mod M" using dist[OF lv] .
  have lmM: "0 ≤ (l - m) mod M" using Mpos by simp
  have not_caseB: "¬ (l - m) mod M + (h - l) ≥ M"
  proof
    assume ge0: "(l - m) mod M + (h - l) ≥ M"
    have ge: "(l - m) mod M + (h - l) - M ≥ 0" using ge0 by linarith
    have lt: "(l - m) mod M + (h - l) - M < M"
      using pos_mod_bound[OF Mpos, of "l - m"] width' by linarith
    have "((l - m) mod M + (h - l)) mod M = ((l - m) mod M + (h - l) - M + M) mod M"
      by simp
    also have "... = ((l - m) mod M + (h - l) - M) mod M" by (rule mod_add_self2)
    also have "... = (l - m) mod M + (h - l) - M" using ge lt by (rule mod_pos_pos_trivial)
    finally have "((l - m) mod M + (h - l)) mod M = (l - m) mod M + (h - l) - M" .
    with hv have "ik_norm ik h = ik_norm ik l + ((h - l) - M)" by (simp add: wrap)
    then show False using ordered width' by linarith
  qed
  have caseA_h: "(h - m) mod M = (l - m) mod M + (h - l)"
    using not_caseB hv Mpos lmM lv vh
    by (simp add: mod_pos_pos_trivial)
  have range_v: "0 ≤ v - l" "v - l ≤ h - l" using lv vh by simp_all
  have not_caseB_v: "¬ (l - m) mod M + (v - l) ≥ M" using not_caseB range_v by linarith
  have caseA_v: "(v - m) mod M = (l - m) mod M + (v - l)"
    using vv not_caseB_v Mpos lmM range_v
    by (simp add: mod_pos_pos_trivial)
  show ?thesis using caseA_v by (simp add: wrap)
qed
```

Integer promotion and the usual arithmetic conversions

`ik_promote` is C's integer promotion (ISO/IEC 9899 6.3.1.1p2): a kind narrower than `I32` becomes `I32`, which represents every narrower value, so the "promote to unsigned int" branch of the C rule never applies.

```
definition ik_promote :: "ikind ⇒ ikind" where
  "ik_promote ik = (if ik_bits ik < ik_bits I32 then I32 else ik)"

lemma ik_promote_pins [simp]:
  "ik_promote I8 = I32" "ik_promote U8 = I32" "ik_promote I16 = I32" "ik_promote U16 = I32"
  "ik_promote I32 = I32" "ik_promote U32 = U32" "ik_promote I64 = I64" "ik_promote U64 = U64"
  by (simp_all add: ik_promote_def)
```

`ik_unsigned_of` is C's corresponding unsigned type: same width, unsigned.

```
fun ik_unsigned_of :: "ikind ⇒ ikind" where
```

```
  "ik_unsigned_of I8 = U8" | "ik_unsigned_of U8 = U8"
| "ik_unsigned_of I16 = U16" | "ik_unsigned_of U16 = U16"
| "ik_unsigned_of I32 = U32" | "ik_unsigned_of U32 = U32"
| "ik_unsigned_of I64 = U64" | "ik_unsigned_of U64 = U64"
```

`usual_kind` is the usual arithmetic conversions (ISO/IEC 9899 6.3.1.8): promote both operands, then pick the common kind by signedness and width. Width stands in for conversion rank, which is faithful because every rank class here has one width. The final branch of the standard's rule is kept although no promoted pair reaches it, so the definition reads like the standard.

```
definition usual_kind :: "ikind ⇒ ikind ⇒ ikind" where
  "usual_kind a0 b0 =
     (let a = ik_promote a0; b = ik_promote b0 in
      if a = b then a
      else if ik_signed a = ik_signed b then (if ik_bits a < ik_bits b then b else a)
      else
        (let s = (if ik_signed a then a else b); u = (if ik_signed a then b else a)
         in if ik_bits s ≤ ik_bits u then u
            else if ik_max u ≤ ik_max s then s
            else ik_unsigned_of s))"


end
```

1.2 VIMP_Syntax

```
theory VIMP_Syntax
  imports Machine_Kind
begin
```

Source-language expressions

One integer-valued expression language, as in C: comparisons and logical operators are ordinary constructors evaluating to `0/1`, and source `true/false` lower to `N 1/N 0` in the grammar. `Cast ik e` converts the value of `e` to the machine kind `ik` by wraparound. A program without machine kinds contains none, and its expressions compute over mathematical integers.

```
type_synonym vname = String.literal
type_synonym src_store = "vname ⇒ int"

datatype src_exp =
    N (exp_lit: int)
  | V (exp_var: vname)
  | Plus  (exp_lhs: src_exp) (exp_rhs: src_exp)
  | Minus (exp_lhs: src_exp) (exp_rhs: src_exp)
  | Times (exp_lhs: src_exp) (exp_rhs: src_exp)
  | Div (exp_lhs: src_exp) (exp_rhs: src_exp)
  | Mod (exp_lhs: src_exp) (exp_rhs: src_exp)
  | Less (cmp_lhs: src_exp) (cmp_rhs: src_exp)
  | LessEq (cmp_lhs: src_exp) (cmp_rhs: src_exp)
  | Greater (cmp_lhs: src_exp) (cmp_rhs: src_exp)
  | GreaterEq (cmp_lhs: src_exp) (cmp_rhs: src_exp)
  | NotEq (cmp_lhs: src_exp) (cmp_rhs: src_exp)
  | Eq (cmp_lhs: src_exp) (cmp_rhs: src_exp)
  | Not (exp_arg: src_exp)
  | And (exp_lhs: src_exp) (exp_rhs: src_exp)
  | Or (exp_lhs: src_exp) (exp_rhs: src_exp)
  | Cast (cast_kind: ikind) (cast_arg: src_exp)

instance src_exp :: countable
  by countable_datatype
```

The executable linear order gives `sorted_list_of_set` a deterministic representation of CFG edge sets; `compare_order` on `String.literal` makes it a usable leaf for the derivation.

```
derive (linorder) compare_order String.literal
derive linorder src_exp

end
```

1.3 VIMP_Globals

```
theory VIMP_Globals
  imports VIMP_Syntax
begin
```

Locals and globals over the scalar store

The store is one `vname ⇒ int` function; the locals/globals split is a classifier `𝒢 :: vname ⇒ bool` on top of it. A call needs both halves of that split: `enter_state` keeps the globals and resets the locals on entry, `combine_env caller callee` restores the caller's locals and keeps the callee's globals on exit.

`combine_env` is the single pointwise selector, generic in both key and codomain. `enter_frame` specializes it with a constant reset map. The concrete semantics uses these operations at `store = vname ⇒ int`, while abstract states reuse them at their own codomain.

```
type_synonym pname = String.literal

definition combine_env ::
    "('k ⇒ bool) ⇒ ('k ⇒ 'a) ⇒ ('k ⇒ 'a) ⇒ ('k ⇒ 'a)" where
  "combine_env 𝒢 s t = (λn. if 𝒢 n then t n else s n)"

definition enter_frame :: "('k ⇒ bool) ⇒ 'a ⇒ ('k ⇒ 'a) ⇒ ('k ⇒ 'a)" where
  "enter_frame 𝒢 reset_val s = combine_env 𝒢 (λ_. reset_val) s"

definition enter_state :: "('k ⇒ bool) ⇒ ('k ⇒ int) ⇒ ('k ⇒ int)" where
  "enter_state 𝒢 s = enter_frame 𝒢 0 s"

lemma combine_query [simp]:
  "combine_env 𝒢 s t n = (if 𝒢 n then t n else s n)"
  unfolding combine_env_def by simp

lemma enter_frame_apply [simp]:
  "enter_frame 𝒢 reset_val s n = (if 𝒢 n then s n else reset_val)"
  unfolding enter_frame_def by simp

lemma enter_state_apply [simp]:
  "enter_state 𝒢 s n = (if 𝒢 n then s n else 0)"
  unfolding enter_state_def by simp

lemma combine_collapse [simp]:
  "combine_env 𝒢 s s = s"
  by (rule ext) simp

lemma combine_nest_left [simp]:
  "combine_env 𝒢 (combine_env 𝒢 s t) u = combine_env 𝒢 s u"
  by (rule ext) simp

lemma combine_nest_right [simp]:
  "combine_env 𝒢 s (combine_env 𝒢 t u) = combine_env 𝒢 s u"
  by (rule ext) simp
```

C-faithful initial store set

VIMP approximates C startup by initializing every global with no explicit initializer to 0 and leaving every other name of the initial store unconstrained. `cinit_stores` is the corresponding set of concrete stores: those where every global is 0, so `main`'s locals may hold any integer. It constrains only the initial store. A callee's locals do not come from it: `enter_state` resets them to 0 on every entry, which defines what C leaves indeterminate, as `c_div` defines division by zero. Every local of a parsed program is declared, and a declaration forgets the value it finds, so no run of a parsed program observes this 0; a core program built inside Isabelle without declarations can. Neither convention is a model of C's indeterminate-value or undefined-behaviour rules, and VIMP has no global initializer syntax for a declaration such as `int x = 42;` to diverge from.

```
definition cinit_stores :: "(vname ⇒ bool) ⇒ src_store set" where
  "cinit_stores 𝒢 = {s. ∀x. 𝒢 x ⟶ s x = 0}"

end
```

1.4 VIMP_Expr

```
theory VIMP_Expr
  imports VIMP_Syntax
begin
```

Expression evaluation

The integer operations every VIMP evaluator shares. Every VIMP expression is integer-valued, so there is no separate Boolean type. Conditions use C-style truthiness (non-zero), while comparisons and logical operators yield `1` or `0`. Evaluation is total and has no effects, which is why a guard can be re-evaluated freely by the transfer functions.

Division and remainder truncate toward zero for non-zero divisors. VIMP keeps expressions total by defining division by zero as zero and remainder by zero as the dividend. These zero-divisor cases are VIMP conventions; they do not model C's undefined behavior.

```
definition c_div :: "int ⇒ int ⇒ int" where
  "c_div a b =
    (if b = 0 then 0 else sgn a * sgn b * (abs a div abs b))"

definition c_mod :: "int ⇒ int ⇒ int" where
  "c_mod a b = (if b = 0 then a else a - c_div a b * b)"

lemma c_div_zero [simp]: "c_div a 0 = 0" "c_div 0 b = 0"
  by (simp_all add: c_div_def)

lemma c_mod_zero [simp]: "c_mod a 0 = a" "c_mod 0 b = 0"
  by (simp_all add: c_mod_def)

lemma c_div_one [simp]: "c_div a 1 = a" "c_div a (-1) = -a"
  by (simp_all add: c_div_def sgn_mult_abs)

lemma c_mod_one [simp]: "c_mod a 1 = 0" "c_mod a (-1) = 0"
  by (simp_all add: c_mod_def)

lemma c_div_mod_reconstruct:
  "c_div a b * b + c_mod a b = a"
  by (simp add: c_mod_def)

lemma c_mod_abs:
  "c_mod a b = sgn a * (abs a mod abs b)"
proof (cases "b = 0")
  case True
  then show ?thesis by (simp add: sgn_mult_abs)
next
  case False
  have product: "c_div a b * b = sgn a * (abs a div abs b * abs b)"
    using False unfolding c_div_def
    by (simp add: abs_sgn mult_ac)
  have "sgn a * (abs a div abs b * abs b) + sgn a * (abs a mod abs b) = a"
    by (simp only: distrib_left[symmetric] div_mult_mod_eq sgn_mult_abs)
  with False product show ?thesis unfolding c_mod_def by (simp only: False if_False; linarith)
qed

lemma c_div_exact:
  assumes "b dvd a"
  shows "c_div a b = a div b"
proof (cases "b = 0")
  case True
  with assms show ?thesis by simp
next
  case False
  have rem: "c_mod a b = 0"
    using assms by (simp add: c_mod_abs)
  have product: "c_div a b * b = a"
    using c_div_mod_reconstruct[of a b] rem by simp
  have "c_div a b = (c_div a b * b) div b"
    using False by simp
  also have "... = a div b" using product by simp
  finally show ?thesis .
qed

lemma c_div_nonneg:
  "0 ≤ a ⟹ 0 ≤ b ⟹ 0 ≤ c_div a b"
```

```
  "a ≤ 0 ⟹ b ≤ 0 ⟹ 0 ≤ c_div a b"
  by (auto simp: c_div_def sgn_if pos_imp_zdiv_nonneg_iff neg_imp_zdiv_nonneg_iff)

lemma c_div_nonpos:
  "a ≤ 0 ⟹ 0 ≤ b ⟹ c_div a b ≤ 0"
  "0 ≤ a ⟹ b ≤ 0 ⟹ c_div a b ≤ 0"
  by (auto simp: c_div_def sgn_if pos_imp_zdiv_nonneg_iff neg_imp_zdiv_nonneg_iff)

lemma c_mod_nonneg:
  "0 ≤ a ⟹ 0 ≤ c_mod a b"
  by (cases "b = 0") (auto simp: c_mod_abs sgn_if intro: pos_mod_sign)

lemma c_mod_nonpos:
  "a ≤ 0 ⟹ c_mod a b ≤ 0"
  by (cases "b = 0") (auto simp: c_mod_abs sgn_if intro: pos_mod_sign)

lemma c_div_neg_divisor:
  "c_div a (-b) = - c_div a b"
  by (simp add: c_div_def)

lemma c_div_neg_dividend:
  "c_div (-a) b = - c_div a b"
  by (simp add: c_div_def)

definition truthy :: "int ⇒ bool" where
  [simp]: "truthy n ⟷ n ≠ 0"
```

Finite expression footprints

The variables an expression mentions. A transfer that evaluates the expression needs no other variable, which is what lets a keyed analysis read only the globals an edge mentions.

```
fun exp_vnames :: "src_exp ⇒ vname set" where
  "exp_vnames (N _) = {}"
| "exp_vnames (V x) = {x}"
| "exp_vnames (Plus a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (Minus a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (Times a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (Div a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (Mod a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (Less a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (LessEq a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (Greater a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (GreaterEq a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (NotEq a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (Eq a b) = exp_vnames a ∪ exp_vnames b"
| "exp_vnames (Not b) = exp_vnames b"
| "exp_vnames (And b1 b2) = exp_vnames b1 ∪ exp_vnames b2"
| "exp_vnames (Or b1 b2) = exp_vnames b1 ∪ exp_vnames b2"
| "exp_vnames (Cast _ a) = exp_vnames a"

lemma finite_exp_vnames [simp]: "finite (exp_vnames a)"
  by (induction a) auto
```

Every literal of an expression fits `I64`. C gives an unsuffixed decimal constant no type beyond it (ISO/IEC 9899 6.4.4.1p5), so a well-formed program writes no other.

```
fun lits_fit :: "src_exp ⇒ bool" where
  "lits_fit (N n) ⟷ ik_min I64 ≤ n ∧ n ≤ ik_max I64"
| "lits_fit (V _) ⟷ True"
| "lits_fit (Plus a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Minus a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Times a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Div a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Mod a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Less a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (LessEq a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Greater a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (GreaterEq a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (NotEq a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Eq a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Not a) ⟷ lits_fit a"
| "lits_fit (And a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Or a b) ⟷ lits_fit a ∧ lits_fit b"
| "lits_fit (Cast _ a) ⟷ lits_fit a"

end
```

1.5 VIMP_Special

```
theory VIMP_Special
  imports VIMP_Expr VIMP_Globals
begin
```

Special calls

VIMP's analogue of Goblint's library-function dispatch, as a closed enumeration rather than an open table. Classification is two-level, as in Goblint: `special_table` resolves a callee name to a shape descriptor and `classify_special` applies the descriptor to the actuals, so no consumer re-checks arity. A special call never enters an activation.

```
datatype special_desc = SD_Nondet_Int | SD_Min | SD_Max
```

A classified call is generic in its argument expressions, so a typed CFG carries typed arguments; `special_call` is the instance at `src_exp`.

```
datatype 'e gspecial_call =
    Nondet_Int
  | Min 'e 'e
  | Max 'e 'e

type_synonym special_call = "src_exp gspecial_call"

instance gspecial_call :: (countable) countable
  by countable_datatype

fun classify_special :: "special_desc ⇒ 'e list ⇒ 'e gspecial_call option" where
  "classify_special SD_Nondet_Int [] = Some Nondet_Int"
| "classify_special SD_Min [a, b] = Some (Min a b)"
| "classify_special SD_Max [a, b] = Some (Max a b)"
| "classify_special _ _ = None"
```

Shared by `pstep`'s `Special` rule and the CFG's `special_step`, so the two concrete semantics cannot drift. `Nondet_Int` admits every integer. The arguments are evaluated by `ev`, the evaluator of the language the call is written in.

```
fun special_result :: "('e ⇒ 's ⇒ int) ⇒ 'e gspecial_call ⇒ 's ⇒ int ⇒ bool" where
  "special_result ev Nondet_Int s v = True"
| "special_result ev (Min a b) s v = (v = min (ev a s) (ev b s))"
| "special_result ev (Max a b) s v = (v = max (ev a s) (ev b s))"

lemma special_result_ex [simp]: "∃v. special_result ev sc s v"
  by (cases sc) auto
```

Every `special_pname_*` is an ordinary identifier, so a source program could declare a colliding procedure; `wf_source_program` rejects that instead of letting special-call semantics shadow the declaration.

```
definition special_pname_nondet_int :: pname where
  "special_pname_nondet_int = STR ''__voblint_nondet_int''"

definition special_pname_min :: pname where
  "special_pname_min = STR ''min''"

definition special_pname_max :: pname where
  "special_pname_max = STR ''max''"

definition special_table :: "pname ⇒ special_desc option" where
  "special_table p =
     (if      p = special_pname_nondet_int then Some SD_Nondet_Int
      else if p = special_pname_min       then Some SD_Min
      else if p = special_pname_max       then Some SD_Max
      else None)"

end
```

1.6 VIMP_Proc

```
theory VIMP_Proc
  imports VIMP_Special "HOL-IMP.Star"
begin
```

Procedure commands and activation frames

A call evaluates its actuals in the caller store, binds them in a fresh activation, and pushes a frame holding the caller store and the optional destination. `Restore` and `Unwind` never occur in source programs: `Restore` marks the activation boundary inside sequential syntax, and `Unwind` is the state after a `Return` has published its value through `ret_var`, discarding pending commands up to the nearest `Restore`.

A check carries a label naming where it was written. The semantics never reads it; compilation copies it onto the check's edge, and the analysis result lists each check under it, so a report can be matched to its source check by label instead of by position in a list. The CLI parser writes the line and column of the check's `__voblint_check` keyword; a program written in Isabelle notation carries the placeholder `(0, 0)`.

```
type_synonym check_label = "nat × nat"
```

Every statement carries an annotation `'a`. Source programs written by hand or produced by the parser carry `()`; elaboration replaces it with a statement identity. The semantics never reads it: a step copies the annotation of the command it reduces onto what it leaves behind, so a residual still names the statement it came from. The variable and expression payloads are parameters too, so a later resolved or typed front end instantiates the same command syntax.

```
datatype ('a, 'v, 'e) gcom =
    is_SKIP: SKIP 'a
  | Assign 'a (assign_var: 'v) (assign_rhs: 'e)
  | Check  'a check_label (check_cond: 'e)
  | Seq    (seq_first: "('a, 'v, 'e) gcom") (seq_second: "('a, 'v, 'e) gcom")
  | If     'a (if_cond: 'e) (if_then: "('a, 'v, 'e) gcom") (if_else: "('a, 'v, 'e) gcom")
  | While  'a (while_cond: 'e) (while_body: "('a, 'v, 'e) gcom")
  | Call   'a (call_dest: "'v option") (call_proc: pname) (call_args: "'e list")
  | Return 'a (return_val: "'e option")
  | is_Restore: Restore 'a
  | Unwind 'a

type_synonym 'a acom = "('a, vname, src_exp) gcom"
type_synonym src_com = "unit acom"
```

The expressions a command evaluates, as a list for the executable checks.

```
fun com_exps :: "('a, 'v, 'e) gcom ⇒ 'e list" where
  "com_exps (SKIP _) = []"
| "com_exps (Assign _ _ e) = [e]"
| "com_exps (Check _ _ e) = [e]"
| "com_exps (Seq c1 c2) = com_exps c1 @ com_exps c2"
| "com_exps (If _ b c1 c2) = b # com_exps c1 @ com_exps c2"
| "com_exps (While _ b c) = b # com_exps c"
| "com_exps (Call _ _ _ es) = es"
| "com_exps (Return _ e) = (case e of Some e ⇒ [e] | None ⇒ [])"
| "com_exps (Restore _) = []"
| "com_exps (Unwind _) = []"

lemma set_com_exps [simp]: "set (com_exps c) = set3_gcom c"
  by (induction c) (auto split: option.splits)
```

The discriminators read as the equations they replace, so automation that used to substitute `c = SKIP` still finds the constructor.

```
lemma is_SKIP_iff [simp]: "is_SKIP c ⟷ (∃i. c = SKIP i)"
  by (cases c) auto

lemma is_Restore_iff [simp]: "is_Restore c ⟷ (∃i. c = Restore i)"
  by (cases c) auto
```

Activation – fresh locals and formal binding – happens at the call site, so a declaration records only what a call site looks up by name.

```
record ('a, 'v, 'e) gproc_decl =
  formals :: "'v list"
  body    :: "('a, 'v, 'e) gcom"

type_synonym proc_decl = "(unit, vname, src_exp) gproc_decl"
```

The reserved local `ret_var` carries a published result to the restore boundary. It is absent from source programs, so a value-less completion leaves it at `enter_state`'s zero.

```
definition ret_var :: vname where
  "ret_var = STR ''#ret''"

type_synonym ('a, 'v, 'e) gptable = "pname ⇒ ('a, 'v, 'e) gproc_decl option"
type_synonym 'a ptable = "('a, vname, src_exp) gptable"
type_synonym proc_table = "unit ptable"
```

Formal binding is the same fold over the concrete store, over every abstract state, and over a typed store keyed by variable identities, so it is one polymorphic abbreviation.

```
abbreviation bind_formals :: "'k list ⇒ 'v list ⇒ ('k ⇒ 'v) ⇒ 'k ⇒ 'v" where
  "bind_formals xs vs s ≡ fold (λ(x, v) st. st(x := v)) (zip xs vs) s"

lemma bind_formals_other:
  "x ∉ set xs ⟹ bind_formals xs vs s x = s x"
proof (induction xs arbitrary: vs s)
  case Nil then show ?case by simp
next
  case (Cons y ys)
  then show ?case by (cases vs) simp_all
qed
```

Whole procedure entry, for the same reason: reset every local to a chosen value, keep the globals, then bind the formals to the actuals evaluated in the caller's own state. Which state and which evaluator is all that separates the concrete entry from an abstract one – the concrete semantics enters with its evaluator and the reset value `0`, an abstract domain with its own evaluator and its own `top` – so it is one definition, not a concrete one with an `_abs` copy beside it.

```
definition enter_binding ::
  "('k ⇒ bool) ⇒ 'a ⇒ ('e ⇒ ('k ⇒ 'a) ⇒ 'a)
   ⇒ 'k list ⇒ 'e list ⇒ ('k ⇒ 'a) ⇒ ('k ⇒ 'a)"
where
  "enter_binding 𝒢 reset_val ev xs es s =
     bind_formals xs (map (λe. ev e s) es) (enter_frame 𝒢 reset_val s)"
```

The return-slot write is a pure per-variable update, generic in the codomain: the concrete semantics uses it at a store of integers, and every abstract domain's state of abstract values reuses the same definition rather than restating an `_abs` copy of it.

```
fun combine_assign :: "'k option ⇒ 'a ⇒ ('k ⇒ 'a) ⇒ ('k ⇒ 'a)" where
  "combine_assign None _ s = s"
| "combine_assign (Some x) v s = s(x := v)"
```

Source-program well-formedness

Source syntax excludes the runtime markers `Restore` and `Unwind`. The reserved `ret_var` belongs to the call mechanism, so source expressions, assignments, destinations, and formal parameters cannot mention it.

Finite syntactic variable sets

```
fun com_vnames :: "'a acom ⇒ vname set" where
  "com_vnames (SKIP _) = {}"
| "com_vnames (Assign _ x a) = insert x (exp_vnames a)"
| "com_vnames (Check _ l c) = exp_vnames c"
| "com_vnames (Seq c1 c2) = com_vnames c1 ∪ com_vnames c2"
| "com_vnames (If _ b c1 c2) =
    exp_vnames b ∪ com_vnames c1 ∪ com_vnames c2"
| "com_vnames (While _ b c) = exp_vnames b ∪ com_vnames c"
| "com_vnames (Call _ dst _ actuals) =
    (case dst of None ⇒ {} | Some x ⇒ {x}) ∪
    ⋃ (set (map exp_vnames actuals))"
| "com_vnames (Return _ e) = (case e of None ⇒ {} | Some a ⇒ exp_vnames a)"
| "com_vnames (Restore _) = {}"
| "com_vnames (Unwind _) = {}"

lemma finite_com_vnames [simp]: "finite (com_vnames c)"
  by (induction c) (auto split: option.splits)

fun source_com :: "('a, 'v, 'e) gcom ⇒ bool" where
  "source_com (SKIP _) = True"
| "source_com (Assign _ x a) = True"
```

```
| "source_com (Check _ l c) = True"
| "source_com (Seq c1 c2) = (source_com c1 ∧ source_com c2)"
| "source_com (If _ b c1 c2) = (source_com c1 ∧ source_com c2)"
| "source_com (While _ b c) = source_com c"
| "source_com (Call _ dst p actuals) = True"
| "source_com (Return _ e) = True"
| "source_com (Restore _) = False"
| "source_com (Unwind _) = False"

definition source_pi :: "('a, 'v, 'e) gptable ⇒ bool" where
  "source_pi Π = (∀p decl. Π p = Some decl ⟶ source_com (body decl))"
```

Well-formedness is stated once for any language: `vars` gives the variables an expression reads, and `rv` is the reserved return slot. The surface predicates are the instances at `exp_vnames` and `ret_var`.

```
definition gsource_exp :: "('e ⇒ 'v set) ⇒ 'v ⇒ 'e ⇒ bool" where
  "gsource_exp vars rv a ⟷ rv ∉ vars a"

abbreviation source_exp :: "src_exp ⇒ bool" where
  "source_exp ≡ gsource_exp exp_vnames ret_var"

definition gvalid_formal :: "'v ⇒ ('v ⇒ bool) ⇒ 'v ⇒ bool" where
  "gvalid_formal rv 𝒢 x ⟷ ¬ 𝒢 x ∧ x ≠ rv"

abbreviation valid_formal :: "(vname ⇒ bool) ⇒ vname ⇒ bool" where
  "valid_formal ≡ gvalid_formal ret_var"
```

Procedure result contract

Declarations carry no return-kind annotation. A body is a value provider when every syntactic way to finish the body reaches `Return` with a value. The analysis is conservative: a loop may fall through because its guard may be false, and a call resumes normally after its callee completes.

```
fun may_fallthrough :: "('a, 'v, 'e) gcom ⇒ bool" where
  "may_fallthrough (SKIP _) = True"
| "may_fallthrough (Assign _ _ _) = True"
| "may_fallthrough (Check _ _ _) = True"
| "may_fallthrough (Seq c1 c2) = (may_fallthrough c1 ∧ may_fallthrough c2)"
| "may_fallthrough (If _ _ c1 c2) = (may_fallthrough c1 ∨ may_fallthrough c2)"
| "may_fallthrough (While _ _ _) = True"
| "may_fallthrough (Call _ _ _ _) = True"
| "may_fallthrough (Return _ _) = False"
| "may_fallthrough (Restore _) = False"
| "may_fallthrough (Unwind _) = False"

fun may_return_none :: "('a, 'v, 'e) gcom ⇒ bool" where
  "may_return_none (Seq c1 c2) =
     (may_return_none c1 ∨ (may_fallthrough c1 ∧ may_return_none c2))"
| "may_return_none (If _ _ c1 c2) = (may_return_none c1 ∨ may_return_none c2)"
| "may_return_none (While _ _ c) = may_return_none c"
| "may_return_none (Return _ e) = (e = None)"
| "may_return_none _ = False"

fun may_return_value :: "('a, 'v, 'e) gcom ⇒ bool" where
  "may_return_value (Seq c1 c2) =
     (may_return_value c1 ∨ (may_fallthrough c1 ∧ may_return_value c2))"
| "may_return_value (If _ _ c1 c2) = (may_return_value c1 ∨ may_return_value c2)"
| "may_return_value (While _ _ c) = may_return_value c"
| "may_return_value (Return _ e) = (e ≠ None)"
| "may_return_value _ = False"

definition value_providing :: "('a, 'v, 'e) gcom ⇒ bool" where
  "value_providing c ⟷
     source_com c ∧ ¬ may_fallthrough c ∧
     ¬ may_return_none c ∧ may_return_value c"
```

Commands, declarations, and programs

Every call names a declared procedure and supplies one actual per formal. A destination requires a value-providing callee; a destination-less call may discard a published value. Procedure formals are distinct local names, and the distinguished root command contains no return.

```
fun gwf_source_com ::
    "('e ⇒ 'v set) ⇒ 'v ⇒ ('a, 'v, 'e) gptable ⇒ ('a, 'v, 'e) gcom ⇒ bool" where
  "gwf_source_com vars rv Π (SKIP _) = True"
| "gwf_source_com vars rv Π (Assign _ x a) = (x ≠ rv ∧ gsource_exp vars rv a)"
```

```
| "gwf_source_com vars rv Π (Check _ l c) = gsource_exp vars rv c"
| "gwf_source_com vars rv Π (Seq c1 c2) = (gwf_source_com vars rv Π c1 ∧ gwf_source_com vars rv Π c2)"
| "gwf_source_com vars rv Π (If _ b c1 c2) =
     (gsource_exp vars rv b ∧ gwf_source_com vars rv Π c1 ∧ gwf_source_com vars rv Π c2)"
| "gwf_source_com vars rv Π (While _ b c) = (gsource_exp vars rv b ∧ gwf_source_com vars rv Π c)"
| "gwf_source_com vars rv Π (Call _ dst p actuals) =
     (case special_table p of
        Some desc ⇒
          classify_special desc actuals ≠ None ∧ list_all (gsource_exp vars rv) actuals ∧
          (case dst of None ⇒ False | Some x ⇒ x ≠ rv)
      | None ⇒
          (case Π p of
             None ⇒ False
           | Some decl ⇒
               length actuals = length (formals decl) ∧
               list_all (gsource_exp vars rv) actuals ∧
               (case dst of
                  None ⇒ True
                | Some x ⇒ x ≠ rv ∧ value_providing (body decl))))"
| "gwf_source_com vars rv Π (Return _ e) = (case e of None ⇒ True | Some a ⇒ gsource_exp vars rv a)"
| "gwf_source_com vars rv Π (Restore _) = False"
| "gwf_source_com vars rv Π (Unwind _) = False"

abbreviation wf_source_com :: "'a ptable ⇒ 'a acom ⇒ bool" where
  "wf_source_com ≡ gwf_source_com exp_vnames ret_var"

fun no_return :: "('a, 'v, 'e) gcom ⇒ bool" where
  "no_return (Seq c1 c2) = (no_return c1 ∧ no_return c2)"
| "no_return (If _ _ c1 c2) = (no_return c1 ∧ no_return c2)"
| "no_return (While _ _ c) = no_return c"
| "no_return (Return _ _) = False"
| "no_return _ = True"

definition gwf_proc_decl ::
    "('e ⇒ 'v set) ⇒ 'v ⇒ ('v ⇒ bool) ⇒ ('a, 'v, 'e) gptable
       ⇒ ('a, 'v, 'e) gproc_decl ⇒ bool" where
  "gwf_proc_decl vars rv 𝒢 Π decl ⟷
     distinct (formals decl) ∧
     list_all (gvalid_formal rv 𝒢) (formals decl) ∧
     gwf_source_com vars rv Π (body decl)"

abbreviation wf_proc_decl :: "(vname ⇒ bool) ⇒ 'a ptable ⇒ ('a, vname, src_exp) gproc_decl ⇒ bool"
  where
  "wf_proc_decl ≡ gwf_proc_decl exp_vnames ret_var"

definition greserved_ret_var :: "'v ⇒ ('v ⇒ bool) ⇒ bool" where
  "greserved_ret_var rv 𝒢 ⟷ ¬ 𝒢 rv"

abbreviation reserved_ret_var :: "(vname ⇒ bool) ⇒ bool" where
  "reserved_ret_var ≡ greserved_ret_var ret_var"
```

The entry procedure is fixed: a program has exactly one, it is called `main`, and the parser rejects formals on it. Naming it here rather than threading it as a parameter means the compiler and its contract take a table and a callee list and nothing else.

```
definition prog_main_name :: pname where
  "prog_main_name = STR ''main''"
```

The entry body, looked up rather than passed alongside the table. The lookup cannot fail where `wf_source_program` holds — its second conjunct is exactly that the entry is declared — so an undeclared entry is an invariant violation, and aborts in generated code rather than compiling to a plausible empty program.

```
definition main_body :: "('a, 'v, 'e) gptable ⇒ ('a, 'v, 'e) gcom" where
  "main_body Π =
     (case Π prog_main_name of
        Some decl ⇒ body decl
      | None ⇒ Code.abort (STR ''main_body: entry procedure not declared'') (λ_. SKIP undefined))"
```

Deliberately not `[simp]`: `wf_source_program`'s entry conjunct has the shape `Π prog_main_name = Some (⦇formals = [], body = main_body Π⦈)`, against which this rule would rewrite `main_body Π` to itself.

```
definition gwf_source_program ::
    "('e ⇒ 'v set) ⇒ 'v ⇒ ('v ⇒ bool) ⇒ ('a, 'v, 'e) gptable ⇒ bool" where
  "gwf_source_program vars rv 𝒢 Π ⟷
     greserved_ret_var rv 𝒢 ∧
     Π prog_main_name = Some (⦇formals = [], body = main_body Π⦈) ∧
```

```
    gwf_source_com vars rv Π (main_body Π) ∧ no_return (main_body Π) ∧
    (∀p decl. Π p = Some decl ⟶ gwf_proc_decl vars rv 𝒢 Π decl) ∧
    (∀p. Π p ≠ None ⟶ special_table p = None)"

abbreviation wf_source_program :: "(vname ⇒ bool) ⇒ 'a ptable ⇒ bool" where
  "wf_source_program ≡ gwf_source_program exp_vnames ret_var"
```

The compiler-input well-formedness defined in the compiler session unfolds through this pair before reaching `wf_source_com`; every site discharging that obligation shares the unfold skeleton, so it is collected here and the downstream definition adds itself to the same collection.

```
named_theorems wf_compile_input_simps

declare
  gwf_proc_decl_def [wf_compile_input_simps]
  gwf_source_program_def [wf_compile_input_simps]

lemma wf_source_com_source_com:
  "gwf_source_com vars rv Π c ⟹ source_com c"
  by (induction c) (auto split: option.splits)

lemma wf_source_programD:
  assumes "gwf_source_program vars rv 𝒢 Π"
  shows "greserved_ret_var rv 𝒢"
    and "Π prog_main_name = Some (|formals = [], body = main_body Π|)"
    and "gwf_source_com vars rv Π (main_body Π)"
    and "no_return (main_body Π)"
    and "Π p = Some decl ⟹ gwf_proc_decl vars rv 𝒢 Π decl"
    and "Π p = Some decl ⟹ special_table p = None"
    and "source_pi Π"
    and "source_com (main_body Π)"
  using assms wf_source_com_source_com [of vars rv Π]
  unfolding gwf_source_program_def source_pi_def gwf_proc_decl_def by blast+

end
```

1.7 VIMP_Program

```
theory VIMP_Program
  imports VIMP_Proc
begin
```

Whole programs

`declared_global_vars` is the declared-global list in source order; a list, so finiteness is by type. `declared_global` is the derived classifier every program-level operation uses; procedure-level interfaces take the classifier as an explicit `vname ⇒ bool` argument instead.

```
record imp_prog =
  proc_rep :: "(pname * proc_decl) list"
  declared_global_vars :: "vname list"

definition declared_global :: "imp_prog ⇒ vname ⇒ bool" where
  "declared_global p x ⟷ x ∈ set (declared_global_vars p)"

lemma declared_global_iff [simp]:
  "declared_global p x ⟷ x ∈ set (declared_global_vars p)"
  by (simp add: declared_global_def)
```

The `program` parser fixes the entry name and rejects formals on it, so `main` is an ordinary `proc_rep` entry. `prog_main` is total through `main_body`'s aborting fallback; `wf_source_program`'s entry conjunct is what makes the lookup meaningful.

```
definition prog_table :: "imp_prog ⇒ proc_table" where
  "prog_table p = map_of (proc_rep p)"

definition prog_procs :: "imp_prog ⇒ pname list" where
  "prog_procs p = filter (λn. n ≠ prog_main_name) (map fst (proc_rep p))"

definition prog_main :: "imp_prog ⇒ src_com" where
  "prog_main p = main_body (prog_table p)"

lemma prog_main_eq_main_body [simp]: "main_body (prog_table p) = prog_main p"
  by (simp add: prog_main_def)
```

`mk_program` conses the entry onto `proc_rep`, so a program built through it satisfies `wf_source_program`'s entry conjunct by construction.

```
definition mk_program :: "(pname * proc_decl) list ⇒ src_com ⇒ vname list ⇒ imp_prog" where
  "mk_program ps m gv = imp_prog.make ((prog_main_name, (|formals = [], body = m|)) # ps) gv"

lemma mk_program_simps [simp]:
  "prog_table (mk_program ps m gv) = (map_of ps)(prog_main_name ↦ (|formals = [], body = m|))"
  "prog_main (mk_program ps m gv) = m"
  "declared_global_vars (mk_program ps m gv) = gv"
  "prog_main_name ∉ set (map fst ps) ⟹ prog_procs (mk_program ps m gv) = map fst ps"
  by (force simp: prog_table_def prog_main_def main_body_def prog_procs_def mk_program_def
      imp_prog.make_def filter_id_conv)+
```

The finite variable scope of an activation: every declared global, the activation's formals and body occurrences, and the reserved return location.

```
definition scope_vnames :: "imp_prog ⇒ pname ⇒ vname set" where
  "scope_vnames p owner =
    set (declared_global_vars p) ∪ {ret_var} ∪
    (case prog_table p owner of
      None ⇒ {} |
      Some decl ⇒ set (formals decl) ∪ com_vnames (body decl))"

lemma finite_scope_vnames [simp]: "finite (scope_vnames p owner)"
  unfolding scope_vnames_def by (auto split: option.splits)

definition scope_vnames_list :: "imp_prog ⇒ pname ⇒ vname list" where
  "scope_vnames_list p owner = sorted_list_of_set (scope_vnames p owner)"

lemma set_scope_vnames_list [simp]:
  "set (scope_vnames_list p owner) = scope_vnames p owner"
  unfolding scope_vnames_list_def by simp
```

Every variable some procedure of the program declares: a program-wide superset of any one activation's scope.

```
definition program_vars :: "imp_prog ⇒ vname list" where
```

```
  "program_vars p =
     remdups (concat (map (scope_vnames_list p) (prog_main_name # prog_procs p)))"

end
```

1.8 Var_Id

```
theory Var_Id
  imports VIMP_Proc
begin
```

Static variable identities

A name is not an identity: two procedures may both call a formal `n`, and the two are different variables that may carry different kinds. A `var_id` names the scope a binding belongs to, in the role of CIL's `varinfo`. `ReturnId` is the return slot of one procedure. It is a constructor rather than a spelling, so no source program can name it, and each procedure's slot can carry its own kind.

```
datatype var_id =
    GlobalId vname
  | ScopedId pname vname
  | ReturnId pname
  | ReturnVar

derive linorder var_id
```

`ReturnVar` is the one slot a return writes and a call result is read from, Goblint's `return_varinfo`: activation-local, belonging to no single procedure. `ReturnId p` names the kind procedure `p` returns, which elaboration converts a returned value to.

```
fun var_scope :: "var_id ⇒ pname option" where
  "var_scope (GlobalId _) = None"
| "var_scope (ScopedId q _) = Some q"
| "var_scope (ReturnId q) = Some q"
| "var_scope ReturnVar = None"
```

The source name of an identity, which a report shows and erasure restores. The return slot keeps the reserved source name `ret_var`.

```
fun var_name :: "var_id ⇒ vname" where
  "var_name (GlobalId x) = x"
| "var_name (ScopedId _ x) = x"
| "var_name (ReturnId _) = ret_var"
| "var_name ReturnVar = ret_var"
```

Resolution

VIMP has no block scopes and no shadowing: a name the program declares global is that global in every procedure, and every other name is local to the procedure that mentions it (`enter_state` resets exactly those at a call). Resolution reads that rule off the globals classifier $\mathcal{G}$.

```
definition resolve_var :: "(vname ⇒ bool) ⇒ pname ⇒ vname ⇒ var_id" where
  "resolve_var 𝒢 q x = (if 𝒢 x then GlobalId x else ScopedId q x)"

lemma var_name_resolve_var [simp]: "var_name (resolve_var 𝒢 q x) = x"
  by (simp add: resolve_var_def)

lemma var_scope_resolve_var [simp]:
  "var_scope (resolve_var 𝒢 q x) = (if 𝒢 x then None else Some q)"
  by (simp add: resolve_var_def)

lemma resolve_var_not_ReturnId [simp]: "resolve_var 𝒢 q x ≠ ReturnId r"
  by (simp add: resolve_var_def)

lemma resolve_var_not_ReturnVar [simp]: "resolve_var 𝒢 q x ≠ ReturnVar"
  by (simp add: resolve_var_def)
```

Resolution is unique: within one procedure distinct names get distinct identities, and two procedures share an identity only for a global.

```
theorem resolve_var_eq_iff:
  "resolve_var 𝒢 q x = resolve_var 𝒢 r y ⟷ x = y ∧ (𝒢 x ∨ q = r)"
  by (auto simp: resolve_var_def)
```

Kinds and typed stores

A kind environment answers a kind for every identity. VIMP's grammar declares no kinds yet, so an environment is an input to elaboration; `int_kenv` is the one a program without declarations means, every variable a C `int`.

```
type_synonym kenv = "var_id ⇒ ikind"
```

```
type_synonym store = "var_id ⇒ int"

definition int_kenv :: kenv where
  "int_kenv = (λ_. I32)"
```

A store is typed when every identity holds a value of its kind.

```
definition all_in_kinds :: "kenv ⇒ store ⇒ bool" where
  "all_in_kinds κ t ⟷ (∀v. t v ∈ ik_range (κ v))"

lemma all_in_kindsD: "all_in_kinds κ t ⟹ t v ∈ ik_range (κ v)"
  by (simp add: all_in_kinds_def)

lemma all_in_kinds_zero [simp, intro]: "all_in_kinds κ (λ_. 0)"
  by (simp add: all_in_kinds_def del: ik_range_iff)

lemma all_in_kinds_update [simp, intro]:
  "all_in_kinds κ t ⟹ all_in_kinds κ (t(v := ik_norm (κ v) n))"
  by (simp add: all_in_kinds_def del: ik_range_iff)

end
```

1.9 Core_Exp

```
theory Core_Exp
  imports Var_Id
begin
```

Typed expressions

Under machine kinds an expression's value depends on the kind each operation computes at: `200 + 100 < 250` holds over `int` but fails when the sum wraps at an 8-bit kind. This theory states that meaning twice. `src_eval` reads it off a source expression and a kind for every name, in the shape the C standard describes it. `eval` evaluates a typed expression whose nodes already carry their kinds, which is what an analyzer consumes. Elaboration translates the first into the second, and `eval_elaborate` shows that it preserves the meaning.

Operators

```
datatype aop = AAdd | ASub | AMul | ADiv | AMod
datatype cop = CLt | CLe | CGt | CGe | CEq | CNe
```

Division and remainder keep VIMP's total conventions for a zero divisor (`c_div`, `c_mod`). On a nonzero divisor they truncate toward zero, as C does, and on the nonnegative values of an unsigned kind they are floor division.

```
fun aop_eval :: "aop ⇒ int ⇒ int ⇒ int" where
  "aop_eval AAdd a b = a + b"
| "aop_eval ASub a b = a - b"
| "aop_eval AMul a b = a * b"
| "aop_eval ADiv a b = c_div a b"
| "aop_eval AMod a b = c_mod a b"

fun cop_eval :: "cop ⇒ int ⇒ int ⇒ bool" where
  "cop_eval CLt a b = (a < b)"
| "cop_eval CLe a b = (a ≤ b)"
| "cop_eval CGt a b = (a > b)"
| "cop_eval CGe a b = (a ≥ b)"
| "cop_eval CEq a b = (a = b)"
| "cop_eval CNe a b = (a ≠ b)"

lemma ik_norm_bool [simp]: "ik_norm ik (if P then 1 else 0) = (if P then 1 else 0)"
  by simp
```

The source-level typed semantics

An unsuffixed decimal constant takes the first of `I32` and `I64` that represents its magnitude (ISO/IEC 9899 6.4.4.1p5); the frontend folds a sign into the literal, and C types the unsigned token. A literal outside `I64` has no type in that list, and well-formedness rejects it (`lits_fit`). `esyn` is the kind an expression has of its own accord: a variable's kind, a literal's kind, the usual arithmetic conversion of an arithmetic operator's operands, and `I32` for the truth value of a comparison or a logical operator.

```
definition ik_of_lit :: "int ⇒ ikind" where
  "ik_of_lit n = (if |n| ≤ ik_max I32 then I32 else I64)"

fun esyn :: "(vname ⇒ ikind) ⇒ src_exp ⇒ ikind" where
  "esyn Γ (N n) = ik_of_lit n"
| "esyn Γ (V x) = Γ x"
| "esyn Γ (Plus a b) = usual_kind (esyn Γ a) (esyn Γ b)"
| "esyn Γ (Minus a b) = usual_kind (esyn Γ a) (esyn Γ b)"
| "esyn Γ (Times a b) = usual_kind (esyn Γ a) (esyn Γ b)"
| "esyn Γ (Div a b) = usual_kind (esyn Γ a) (esyn Γ b)"
| "esyn Γ (Mod a b) = usual_kind (esyn Γ a) (esyn Γ b)"
| "esyn Γ (Less a b) = I32"
| "esyn Γ (LessEq a b) = I32"
| "esyn Γ (Greater a b) = I32"
| "esyn Γ (GreaterEq a b) = I32"
| "esyn Γ (NotEq a b) = I32"
| "esyn Γ (Eq a b) = I32"
| "esyn Γ (Not a) = I32"
| "esyn Γ (And a b) = I32"
| "esyn Γ (Or a b) = I32"
| "esyn Γ (Cast ik a) = ik"
```

`src_eval Γ ik e s` is the value of `e` converted to kind `ik`. An arithmetic operator converts both operands to their common kind `k`, computes there with wraparound, and only then converts to `ik`. A comparison converts its operands the same way and yields `0` or `1`. A logical operator tests each operand at its own kind. Computing

an inner operation at the context's kind instead would give `4294967295 < u + 1` the wrong value for an unsigned 32-bit `u`: the comparison agrees on 64 bits, but the sum wraps at 32 bits before the comparison sees it.

```
definition arith_at :: "ikind ⇒ ikind ⇒ aop ⇒ int ⇒ int ⇒ int" where
  "arith_at ik k op a b = ik_norm ik (ik_norm k (aop_eval op a b))"

fun src_eval :: "(vname ⇒ ikind) ⇒ ikind ⇒ src_exp ⇒ src_store ⇒ int" where
  "src_eval Γ ik (N n) s = ik_norm ik n"
| "src_eval Γ ik (V x) s = ik_norm ik (ik_norm (Γ x) (s x))"
| "src_eval Γ ik (Plus a b) s = (let k = esyn Γ (Plus a b) in
     arith_at ik k AAdd (src_eval Γ k a s) (src_eval Γ k b s))"
| "src_eval Γ ik (Minus a b) s = (let k = esyn Γ (Minus a b) in
     arith_at ik k ASub (src_eval Γ k a s) (src_eval Γ k b s))"
| "src_eval Γ ik (Times a b) s = (let k = esyn Γ (Times a b) in
     arith_at ik k AMul (src_eval Γ k a s) (src_eval Γ k b s))"
| "src_eval Γ ik (Div a b) s = (let k = esyn Γ (Div a b) in
     arith_at ik k ADiv (src_eval Γ k a s) (src_eval Γ k b s))"
| "src_eval Γ ik (Mod a b) s = (let k = esyn Γ (Mod a b) in
     arith_at ik k AMod (src_eval Γ k a s) (src_eval Γ k b s))"
| "src_eval Γ ik (Less a b) s = (let k = usual_kind (esyn Γ a) (esyn Γ b) in
     if cop_eval CLt (src_eval Γ k a s) (src_eval Γ k b s) then 1 else 0)"
| "src_eval Γ ik (LessEq a b) s = (let k = usual_kind (esyn Γ a) (esyn Γ b) in
     if cop_eval CLe (src_eval Γ k a s) (src_eval Γ k b s) then 1 else 0)"
| "src_eval Γ ik (Greater a b) s = (let k = usual_kind (esyn Γ a) (esyn Γ b) in
     if cop_eval CGt (src_eval Γ k a s) (src_eval Γ k b s) then 1 else 0)"
| "src_eval Γ ik (GreaterEq a b) s = (let k = usual_kind (esyn Γ a) (esyn Γ b) in
     if cop_eval CGe (src_eval Γ k a s) (src_eval Γ k b s) then 1 else 0)"
| "src_eval Γ ik (NotEq a b) s = (let k = usual_kind (esyn Γ a) (esyn Γ b) in
     if cop_eval CNe (src_eval Γ k a s) (src_eval Γ k b s) then 1 else 0)"
| "src_eval Γ ik (Eq a b) s = (let k = usual_kind (esyn Γ a) (esyn Γ b) in
     if cop_eval CEq (src_eval Γ k a s) (src_eval Γ k b s) then 1 else 0)"
| "src_eval Γ ik (Not a) s = (if truthy (src_eval Γ (esyn Γ a) a s) then 0 else 1)"
| "src_eval Γ ik (And a b) s =
     (if truthy (src_eval Γ (esyn Γ a) a s) ∧ truthy (src_eval Γ (esyn Γ b) b s) then 1 else 0)"
| "src_eval Γ ik (Or a b) s =
     (if truthy (src_eval Γ (esyn Γ a) a s) ∨ truthy (src_eval Γ (esyn Γ b) b s) then 1 else 0)"
| "src_eval Γ ik (Cast k a) s = ik_norm ik (ik_norm k (src_eval Γ (esyn Γ a) a s))"
```

Typed expressions and their evaluator

A typed expression carries the kind each arithmetic node computes at and names variables by identity. `TCast`, the one node without a source counterpart, is a conversion, in the role of CIL's `CastE`. `eval` needs no kind environment: every kind it uses is on the node. A variable read does not convert, because a typed store already holds a value of the variable's kind.

```
datatype exp =
    TN ikind int
  | TVar ikind var_id
  | TArith ikind aop exp exp
  | TCast ikind exp
  | TCmp cop exp exp
  | TNot exp
  | TAnd exp exp
  | TOr exp exp

derive linorder aop
derive linorder cop
derive linorder exp

fun eval :: "exp ⇒ store ⇒ int" where
  "eval (TN ik n) t = ik_norm ik n"
| "eval (TVar ik v) t = t v"
| "eval (TArith ik op a b) t = ik_norm ik (aop_eval op (eval a t) (eval b t))"
| "eval (TCast ik a) t = ik_norm ik (eval a t)"
| "eval (TCmp op a b) t = (if cop_eval op (eval a t) (eval b t) then 1 else 0)"
| "eval (TNot a) t = (if truthy (eval a t) then 0 else 1)"
| "eval (TAnd a b) t = (if truthy (eval a t) ∧ truthy (eval b t) then 1 else 0)"
| "eval (TOr a b) t = (if truthy (eval a t) ∨ truthy (eval b t) then 1 else 0)"
```

The kind a node produces: the one an arithmetic or literal node carries, the target of a conversion, and `I32` for a truth value.

```
fun exp_kind :: "exp ⇒ ikind" where
  "exp_kind (TN ik _) = ik"
| "exp_kind (TVar ik _) = ik"
| "exp_kind (TArith ik _ _ _) = ik"
```

```
| "exp_kind (TCast ik _) = ik"
| "exp_kind (TCmp _ _ _) = I32"
| "exp_kind (TNot _) = I32"
| "exp_kind (TAnd _ _) = I32"
| "exp_kind (TOr _ _) = I32"
```

A typed expression is well-typed when its kinds agree with each other and with the environment: an arithmetic node's operands carry its kind, a comparison's two operands share one, and a variable read carries the variable's kind. Abstract transfers that read an operand's kind off the operand rely on exactly this.

```
fun wt_exp :: "kenv ⇒ exp ⇒ bool" where
  "wt_exp κ (TN ik n) ⟷ True"
| "wt_exp κ (TVar ik v) ⟷ κ v = ik"
| "wt_exp κ (TArith ik op a b) ⟷
     exp_kind a = ik ∧ exp_kind b = ik ∧ wt_exp κ a ∧ wt_exp κ b"
| "wt_exp κ (TCast ik a) ⟷ wt_exp κ a"
| "wt_exp κ (TCmp op a b) ⟷ exp_kind a = exp_kind b ∧ wt_exp κ a ∧ wt_exp κ b"
| "wt_exp κ (TNot a) ⟷ wt_exp κ a"
| "wt_exp κ (TAnd a b) ⟷ wt_exp κ a ∧ wt_exp κ b"
| "wt_exp κ (TOr a b) ⟷ wt_exp κ a ∧ wt_exp κ b"

theorem eval_in_range:
  assumes "wt_exp κ e" and "all_in_kinds κ t"
  shows "eval e t ∈ ik_range (exp_kind e)"
  using assms by (cases e) (auto dest: all_in_kindsD simp del: ik_range_iff)
```

Elaboration

`elaborate ϱ κ ik e` translates a source expression of one procedure, whose names ϱ resolves, into a typed expression whose value has kind `ik`. It follows `src_eval` node for node and inserts a conversion only where a value reaches a context at another kind. A conversion to the kind a value already has is left out: it would change no value, but a domain without magnitudes, such as Sign, could only answer top for it.

```
definition cast_to :: "ikind ⇒ ikind ⇒ exp ⇒ exp" where
  "cast_to ik k a = (if ik = k then a else TCast ik a)"

fun elaborate :: "(vname ⇒ var_id) ⇒ kenv ⇒ ikind ⇒ src_exp ⇒ exp" where
  "elaborate ϱ κ ik (N n) = TN ik n"
| "elaborate ϱ κ ik (V x) = cast_to ik (κ (ϱ x)) (TVar (κ (ϱ x)) (ϱ x))"
| "elaborate ϱ κ ik (Plus a b) = (let k = esyn (κ ∘ ϱ) (Plus a b) in
     cast_to ik k (TArith k AAdd (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (Minus a b) = (let k = esyn (κ ∘ ϱ) (Minus a b) in
     cast_to ik k (TArith k ASub (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (Times a b) = (let k = esyn (κ ∘ ϱ) (Times a b) in
     cast_to ik k (TArith k AMul (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (Div a b) = (let k = esyn (κ ∘ ϱ) (Div a b) in
     cast_to ik k (TArith k ADiv (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (Mod a b) = (let k = esyn (κ ∘ ϱ) (Mod a b) in
     cast_to ik k (TArith k AMod (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (Less a b) = (let k = usual_kind (esyn (κ ∘ ϱ) a) (esyn (κ ∘ ϱ) b) in
     cast_to ik I32 (TCmp CLt (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (LessEq a b) = (let k = usual_kind (esyn (κ ∘ ϱ) a) (esyn (κ ∘ ϱ) b) in
     cast_to ik I32 (TCmp CLe (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (Greater a b) = (let k = usual_kind (esyn (κ ∘ ϱ) a) (esyn (κ ∘ ϱ) b) in
     cast_to ik I32 (TCmp CGt (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (GreaterEq a b) = (let k = usual_kind (esyn (κ ∘ ϱ) a) (esyn (κ ∘ ϱ) b) in
     cast_to ik I32 (TCmp CGe (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (NotEq a b) = (let k = usual_kind (esyn (κ ∘ ϱ) a) (esyn (κ ∘ ϱ) b) in
     cast_to ik I32 (TCmp CNe (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (Eq a b) = (let k = usual_kind (esyn (κ ∘ ϱ) a) (esyn (κ ∘ ϱ) b) in
     cast_to ik I32 (TCmp CEq (elaborate ϱ κ k a) (elaborate ϱ κ k b)))"
| "elaborate ϱ κ ik (Not a) =
     cast_to ik I32 (TNot (elaborate ϱ κ (esyn (κ ∘ ϱ) a) a))"
| "elaborate ϱ κ ik (And a b) = cast_to ik I32
     (TAnd (elaborate ϱ κ (esyn (κ ∘ ϱ) a) a) (elaborate ϱ κ (esyn (κ ∘ ϱ) b) b))"
| "elaborate ϱ κ ik (Or a b) = cast_to ik I32
     (TOr (elaborate ϱ κ (esyn (κ ∘ ϱ) a) a) (elaborate ϱ κ (esyn (κ ∘ ϱ) b) b))"
| "elaborate ϱ κ ik (Cast k a) =
     cast_to ik k (TCast k (elaborate ϱ κ (esyn (κ ∘ ϱ) a) a))"
```

A context without a kind of its own, such as a branch condition, evaluates an expression at the kind it synthesizes.

```
definition elaborate_syn :: "(vname ⇒ var_id) ⇒ kenv ⇒ src_exp ⇒ exp" where
  "elaborate_syn ϱ κ e = elaborate ϱ κ (esyn (κ ∘ ϱ) e) e"

lemma exp_kind_cast_to [simp]:
```

```
  "exp_kind (cast_to ik k a) = (if ik = k then exp_kind a else ik)"
  by (simp add: cast_to_def)

lemma exp_kind_elaborate [simp]: "exp_kind (elaborate ϱ κ ik e) = ik"
  by (cases e) (simp_all add: Let_def)

theorem wt_exp_elaborate: "wt_exp κ (elaborate ϱ κ ik e)"
  by (induction e arbitrary: ik) (simp_all add: Let_def cast_to_def)
```

Elaboration preserves the meaning. The typed store `t` holds a value for every identity, and the procedure sees it through its resolution, as $t \circ \varrho$. The premise that `t` is typed is used at one place: `src_eval` reads a variable at its own kind, `eval` reads the stored value directly, and the two agree because the stored value already lies in that kind's range.

```
theorem eval_elaborate:
  assumes "all_in_kinds κ t"
  shows "eval (elaborate ϱ κ ik e) t = src_eval (κ ∘ ϱ) ik e (t ∘ ϱ)"
proof -
  have var: "ik_norm (κ v) (t v) = t v" for v
    by (rule ik_norm_id [OF all_in_kindsD [OF assms]])
  show ?thesis
    by (induction e arbitrary: ik) (simp_all add: Let_def cast_to_def arith_at_def var)
qed

corollary eval_elaborate_syn:
  assumes "all_in_kinds κ t"
  shows "eval (elaborate_syn ϱ κ e) t = src_eval (κ ∘ ϱ) (esyn (κ ∘ ϱ) e) e (t ∘ ϱ)"
  using eval_elaborate [OF assms] by (simp add: elaborate_syn_def)
```

The identities a typed expression reads.

```
fun exp_vars :: "exp ⇒ var_id set" where
  "exp_vars (TN _ _) = {}"
| "exp_vars (TVar _ v) = {v}"
| "exp_vars (TArith _ _ a b) = exp_vars a ∪ exp_vars b"
| "exp_vars (TCast _ a) = exp_vars a"
| "exp_vars (TCmp _ a b) = exp_vars a ∪ exp_vars b"
| "exp_vars (TNot a) = exp_vars a"
| "exp_vars (TAnd a b) = exp_vars a ∪ exp_vars b"
| "exp_vars (TOr a b) = exp_vars a ∪ exp_vars b"

lemma exp_vars_elaborate: "exp_vars (elaborate ϱ κ ik e) ⊆ range ϱ"
  by (induction e arbitrary: ik) (auto simp: Let_def cast_to_def)
```

A conversion to a kind whose range contains the source kind's range changes no value.

```
definition ik_widens :: "ikind ⇒ ikind ⇒ bool" where
  "ik_widens k ik ⟷ ik_min ik ≤ ik_min k ∧ ik_max k ≤ ik_max ik"
```

Erasure

Erasure drops the kinds and the conversions and names every identity by its source name. It recovers the expression elaboration started from. It belongs to no semantics: the erased expression has lost the kinds that `eval` wraps at, so read over unbounded integers it can mean something other than the original.

```
fun aop_exp :: "aop ⇒ src_exp ⇒ src_exp ⇒ src_exp" where
  "aop_exp AAdd = Plus" | "aop_exp ASub = Minus" | "aop_exp AMul = Times"
| "aop_exp ADiv = Div" | "aop_exp AMod = Mod"

fun cop_exp :: "cop ⇒ src_exp ⇒ src_exp ⇒ src_exp" where
  "cop_exp CLt = Less" | "cop_exp CLe = LessEq" | "cop_exp CGt = Greater"
| "cop_exp CGe = GreaterEq" | "cop_exp CEq = Eq" | "cop_exp CNe = NotEq"

fun exp_erase :: "exp ⇒ src_exp" where
  "exp_erase (TN _ n) = N n"
| "exp_erase (TVar _ v) = V (var_name v)"
| "exp_erase (TArith _ op a b) = aop_exp op (exp_erase a) (exp_erase b)"
| "exp_erase (TCast _ a) = exp_erase a"
| "exp_erase (TCmp op a b) = cop_exp op (exp_erase a) (exp_erase b)"
| "exp_erase (TNot a) = Not (exp_erase a)"
| "exp_erase (TAnd a b) = And (exp_erase a) (exp_erase b)"
| "exp_erase (TOr a b) = Or (exp_erase a) (exp_erase b)"
```

Erasure drops every conversion, including one the source wrote, so it recovers an expression without explicit casts.

```
fun cast_free :: "src_exp ⇒ bool" where
```

```
  "cast_free (Cast _ _) ⟷ False"
| "cast_free (Plus a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (Minus a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (Times a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (Div a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (Mod a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (Less a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (LessEq a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (Greater a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (GreaterEq a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (NotEq a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (Eq a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (Not a) ⟷ cast_free a"
| "cast_free (And a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free (Or a b) ⟷ cast_free a ∧ cast_free b"
| "cast_free _ ⟷ True"

theorem exp_erase_elaborate:
  assumes "⋀x. var_name (ϱ x) = x" and "cast_free e"
  shows "exp_erase (elaborate ϱ κ ik e) = e"
  using assms(2) by (induction e arbitrary: ik) (simp_all add: Let_def cast_to_def assms(1))

end
```

1.10 Core_Com

```
theory Core_Com
  imports Core_Exp VIMP_Program
begin
```

Typed commands

A typed command is the generic command type at identities and typed expressions, with the annotation left free. Elaboration resolves every name of a procedure and inserts the conversions C performs where a value is written: to the variable's kind at an assignment, to the formal's kind at an actual argument, and to the procedure's return kind at a return. A condition is evaluated at its own kind. The call destination is only resolved here, since converting the returned value to it happens when the frame is popped, which is the business of a typed semantics.

```
type_synonym 'a com = "('a, var_id, exp) gcom"
```

An actual argument converts to the kind of the formal it binds. The arguments without a formal, which only a library call has, convert to their common kind, as the operands of an arithmetic operator do: C has no integer `min` or `max`, and the macro `(a) < (b) ? (a) : (b)` that stands for one compares and selects at the usual arithmetic conversion of its arguments. `I32` is a neutral start: the common kind of `I32` and a kind is that kind promoted.

```
definition args_kind :: "(vname ⇒ ikind) ⇒ src_exp list ⇒ ikind" where
  "args_kind Γ es = foldr (λe. usual_kind (esyn Γ e)) es I32"

lemma args_kind_pair: "args_kind Γ [a, b] = usual_kind (esyn Γ a) (esyn Γ b)"
  unfolding args_kind_def
  by (cases "esyn Γ a"; cases "esyn Γ b") (simp_all add: usual_kind_def)

fun elab_args :: "(vname ⇒ var_id) ⇒ kenv ⇒ var_id list ⇒ src_exp list ⇒ exp list" where
  "elab_args ϱ κ (f # fs) (e # es) = elaborate ϱ κ (κ f) e # elab_args ϱ κ fs es"
| "elab_args ϱ κ [] es = map (elaborate ϱ κ (args_kind (κ ∘ ϱ) es)) es"
| "elab_args ϱ κ fs [] = []"

lemma exp_vars_elab_args: "e ∈ set (elab_args ϱ κ fs es) ⟹ exp_vars e ⊆ range ϱ"
  by (induction ϱ κ fs es rule: elab_args.induct)
     (auto dest: subsetD [OF exp_vars_elaborate])
```

`fm p` lists the formals of procedure `p`, empty for a library call, and `q` is the procedure whose body is elaborated.

```
fun elab_com ::
  "(vname ⇒ bool) ⇒ (pname ⇒ vname list) ⇒ kenv ⇒ pname
     ⇒ ('a, vname, src_exp) gcom ⇒ 'a com" where
  "elab_com 𝒢 fm κ q (SKIP i) = SKIP i"
| "elab_com 𝒢 fm κ q (Assign i x e) =
     Assign i (resolve_var 𝒢 q x) (elaborate (resolve_var 𝒢 q) κ (κ (resolve_var 𝒢 q x)) e)"
| "elab_com 𝒢 fm κ q (Check i l c) = Check i l (elaborate_syn (resolve_var 𝒢 q) κ c)"
| "elab_com 𝒢 fm κ q (Seq c1 c2) = Seq (elab_com 𝒢 fm κ q c1) (elab_com 𝒢 fm κ q c2)"
| "elab_com 𝒢 fm κ q (If i b c1 c2) =
     If i (elaborate_syn (resolve_var 𝒢 q) κ b) (elab_com 𝒢 fm κ q c1) (elab_com 𝒢 fm κ q c2)"
| "elab_com 𝒢 fm κ q (While i b c) =
     While i (elaborate_syn (resolve_var 𝒢 q) κ b) (elab_com 𝒢 fm κ q c)"
| "elab_com 𝒢 fm κ q (Call i dst p es) =
     Call i (map_option (resolve_var 𝒢 q) dst) p
       (elab_args (resolve_var 𝒢 q) κ (map (resolve_var 𝒢 p) (fm p)) es)"
| "elab_com 𝒢 fm κ q (Return i e) =
     Return i (map_option (elaborate (resolve_var 𝒢 q) κ (κ (ReturnId q))) e)"
| "elab_com 𝒢 fm κ q (Restore i) = Restore i"
| "elab_com 𝒢 fm κ q (Unwind i) = Unwind i"
```

Well-typed commands

A typed command is well-typed when its expressions are, and every written value already has the kind of the place it is written to. `fids p` lists the identities of the formals of `p`.

```
fun args_ok :: "kenv ⇒ var_id list ⇒ exp list ⇒ bool" where
  "args_ok κ (f # fs) (e # es) ⟷ exp_kind e = κ f ∧ args_ok κ fs es"
| "args_ok κ _ _ ⟷ True"

fun wt_com :: "kenv ⇒ (pname ⇒ var_id list) ⇒ pname ⇒ 'a com ⇒ bool" where
  "wt_com κ fids q (SKIP i) ⟷ True"
| "wt_com κ fids q (Assign i v e) ⟷ wt_exp κ e ∧ exp_kind e = κ v"
| "wt_com κ fids q (Check i l c) ⟷ wt_exp κ c"
| "wt_com κ fids q (Seq c1 c2) ⟷ wt_com κ fids q c1 ∧ wt_com κ fids q c2"
```

```
| "wt_com κ fids q (If i b c1 c2) ⟷
     wt_exp κ b ∧ wt_com κ fids q c1 ∧ wt_com κ fids q c2"
| "wt_com κ fids q (While i b c) ⟷ wt_exp κ b ∧ wt_com κ fids q c"
| "wt_com κ fids q (Call i dst p es) ⟷ list_all (wt_exp κ) es ∧ args_ok κ (fids p) es"
| "wt_com κ fids q (Return i e) ⟷
     pred_option (λa. wt_exp κ a ∧ exp_kind a = κ (ReturnId q)) e"
| "wt_com κ fids q (Restore i) ⟷ True"
| "wt_com κ fids q (Unwind i) ⟷ True"

lemma elab_args_wt:
  "list_all (wt_exp κ) (elab_args ϱ κ fs es) ∧ args_ok κ fs (elab_args ϱ κ fs es)"
  by (induction ϱ κ fs es rule: elab_args.induct) (simp_all add: wt_exp_elaborate list_all_iff)

theorem wt_com_elab_com:
  "wt_com κ (λp. map (resolve_var 𝒢 p) (fm p)) q (elab_com 𝒢 fm κ q c)"
  by (induction c)
     (simp_all add: wt_exp_elaborate elaborate_syn_def elab_args_wt option.pred_map comp_def
       option.pred_True)
```

Erasure and shape

Erasing a typed command names every identity by its source name and drops kinds and conversions. It returns the command elaboration started from. The statement is syntactic: a typed command and its erasure can execute differently wherever a kind wraps, so it says nothing about preserving behaviour.

```
abbreviation com_erase :: "'a com ⇒ ('a, vname, src_exp) gcom" where
  "com_erase ≡ map_gcom id var_name exp_erase"

lemma map_exp_erase_elab_args [simp]:
  assumes "⋀x. var_name (ϱ x) = x" and "list_all cast_free es"
  shows "map exp_erase (elab_args ϱ κ fs es) = es"
  using assms(2)
proof (induction es arbitrary: fs)
  case Nil
  show ?case by (cases fs) simp_all
next
  case (Cons e es)
  have syn: "list_all cast_free xs ⟹ map (λa. exp_erase (elaborate ϱ κ k a)) xs = xs"
    for xs k
    by (induction xs) (simp_all add: exp_erase_elaborate [OF assms(1)])
  show ?case
    using Cons syn
    by (cases fs) (simp_all add: exp_erase_elaborate [OF assms(1)] comp_def)
qed

theorem com_erase_elab_com:
  assumes "∀e ∈ set3_gcom c. cast_free e"
  shows "com_erase (elab_com 𝒢 fm κ q c) = c"
  using assms
proof (induction c)
  case (Return i e)
  then show ?case by (cases e) (simp_all add: exp_erase_elaborate [OF var_name_resolve_var])
qed (auto simp: exp_erase_elaborate [OF var_name_resolve_var] elaborate_syn_def
       map_exp_erase_elab_args [OF var_name_resolve_var] option.map_comp comp_def
       option.map_ident list_all_iff)
```

Elaboration keeps a command's skeleton: the constructors, the annotations and the number of arguments of every call. Only the variables and expressions at its leaves change.

```
lemma length_elab_args [simp]: "length (elab_args ϱ κ fs es) = length es"
  by (induction ϱ κ fs es rule: elab_args.induct) simp_all

theorem skeleton_elab_com:
  "map_gcom id (λ_. ()) (λ_. ()) (elab_com 𝒢 fm κ q c) = map_gcom id (λ_. ()) (λ_. ()) c"
  by (induction c) (simp_all add: option.map_comp comp_def map_replicate_const)
```

Programs

Kinds are declared per identity. The parser declares every variable of a program, so the default `I32` only makes the environment total, for identities such as the return slot that no declaration names.

```
type_synonym kind_decls = "(var_id × ikind) list"

definition kenv_of :: "kind_decls ⇒ kenv" where
  "kenv_of ds v = (case map_of ds v of Some k ⇒ k | None ⇒ I32)"
```

```
definition prog_formals :: "imp_prog ⇒ pname ⇒ vname list" where
  "prog_formals p q = (case prog_table p q of Some d ⇒ formals d | None ⇒ [])"

end
```

1.11 VIMP_Semantics

```
theory VIMP_Semantics
  imports Core_Com
begin
```

Typed source semantics

A typed program runs on typed stores, which hold one value per variable identity. Every written value has the kind of the identity it is written to: elaboration already converts an assigned value, an actual argument and a returned value, and the semantics converts the two values it writes itself, a call result and a library result, to the destination's kind. This theory fixes the globals and the initial stores of that semantics and defines it.

The globals are the `GlobalId` identities. A call keeps them and resets every other identity, the shared return slot included, as `enter_state` keeps the declared globals.

```
fun is_global :: "var_id ⇒ bool" where
  "is_global (GlobalId _) = True"
| "is_global _ = False"
```

A program starts with its globals at zero and every other identity holding some value of its kind.

```
definition init_stores :: "kenv ⇒ store set" where
  "init_stores κ = {t. all_in_kinds κ t ∧ (∀x. t (GlobalId x) = 0)}"
```

Frame-stack small-step semantics

A written value is converted to the kind of its destination, as C converts on assignment. `kind_conv` takes the destination as an option because a call without a destination converts nothing. A variable read carries no conversion: an identity holds a value of its own kind, since every write converts.

```
definition kind_conv :: "kenv ⇒ var_id option ⇒ int ⇒ int" where
  "kind_conv κ dst n = (case dst of None ⇒ n | Some v ⇒ ik_norm (κ v) n)"
```

Not `[simp]`: the semantics' rules are stated with the conversion folded, and unfolding it in a goal would keep them from matching.

```
lemma kind_conv_simps:
  "kind_conv κ None n = n"
  "kind_conv κ (Some v) n = ik_norm (κ v) n"
  by (simp_all add: kind_conv_def)
```

A frame records the caller's store and the destination of the pending call.

```
datatype frame = Frame (frame_store: store) (frame_dest: "var_id option")
```

The reference semantics every soundness claim is ultimately about. A configuration is a command, a store, and a stack of caller frames; a call pushes one frame and continues in the callee body followed by `Restore`, a return writes the return slot and hands control to `Unwind`, which discards the rest of the callee body until that `Restore` pops the frame. `Restore` and `Unwind` exist only here, in configurations the semantics builds; no source program contains them.

`eval` evaluates an expression, `is_global` classifies the variables a call keeps, `ReturnVar` is the return slot (Goblint's single `return_varinfo`), and `kind_conv` converts a value written to a destination, as Goblint converts a stored value to its target's declared type. A call converts each actual to its formal, as an assignment converts to its destination, so every entry store holds each formal at its own kind. Elaboration has already converted an actual it produced, as CIL does, and there the conversion changes nothing. A returned value is converted once, to the caller's destination.

```
inductive pstep :: "kenv ⇒ ('a, var_id, exp) gptable
    ⇒ 'a com × store × frame list ⇒ 'a com × store × frame list ⇒ bool"
    ("(_,_ ⊢/ _ →p/ _)" [51, 51, 51, 51] 50)
  for κ :: kenv and Π :: "('a, var_id, exp) gptable"
where
  Assign:  "κ,Π ⊢ (Assign i x a, s, frs) →p (SKIP i, s(x := kind_conv κ (Some x) (eval a s)), frs)"
| Check:   "κ,Π ⊢ (Check i l c, s, frs) →p (SKIP i, s, frs)"
| Seq1:    "κ,Π ⊢ (Seq (SKIP i) c2, s, frs) →p (c2, s, frs)"
| Seq2:    "κ,Π ⊢ (c1, s, frs) →p (c1', s', frs')
             ⟹ κ,Π ⊢ (Seq c1 c2, s, frs) →p (Seq c1' c2, s', frs')"
| IfTrue:  "truthy (eval b s) ⟹ κ,Π ⊢ (If i b c1 c2, s, frs) →p (c1, s, frs)"
| IfFalse: "¬ truthy (eval b s) ⟹ κ,Π ⊢ (If i b c1 c2, s, frs) →p (c2, s, frs)"
| While:   "κ,Π ⊢ (While i b c, s, frs) →p (If i b (Seq c (While i b c)) (SKIP i), s, frs)"
| Call:    "Π p = Some decl
```

```
            ⟹ length actuals = length (formals decl)
            ⟹ distinct (formals decl)
            ⟹ vals = map (λe. eval e s) actuals
            ⟹ callee = bind_formals (formals decl)
                        (map2 (λx. kind_conv κ (Some x)) (formals decl) vals)
                        (enter_state is_global s)
            ⟹ κ,Π ⊢ (Call i dst p actuals, s, frs)
                  →p (Seq (body decl) (Restore i), callee, Frame s dst # frs)"
| Special: "special_table p = Some desc
            ⟹ classify_special desc actuals = Some sc
            ⟹ special_result eval sc s v
            ⟹ κ,Π ⊢ (Call i (Some x) p actuals, s, frs) →p (SKIP i, s(x := kind_conv κ (Some x) v), frs)"
| RestoreStep:
    "κ,Π ⊢ (Restore i, s, Frame fr dst # frs)
       →p (SKIP i, combine_assign dst (kind_conv κ dst (s ReturnVar)) (combine_env is_global fr s), frs)"
| ReturnSome:
    "κ,Π ⊢ (Return i (Some e), s, frs) →p (Unwind i, s(ReturnVar := eval e s), frs)"
| ReturnNone:
    "κ,Π ⊢ (Return i None, s, frs) →p (Unwind i, s, frs)"
| UnwindDead:
    "¬ is_Restore c2 ⟹ κ,Π ⊢ (Seq (Unwind i) c2, s, frs) →p (Unwind i, s, frs)"
| UnwindAct:
    "κ,Π ⊢ (Seq (Unwind j) (Restore i), s, Frame fr dst # frs)
       →p (SKIP i, combine_assign dst (kind_conv κ dst (s ReturnVar)) (combine_env is_global fr s), frs)"

abbreviation psteps :: "kenv ⇒ ('a, var_id, exp) gptable
    ⇒ 'a com × store × frame list ⇒ 'a com × store × frame list ⇒ bool"
    ("(_,_ ⊢/ _ →p*/ _)" [51, 51, 51, 51] 50) where
  "psteps κ Π x y ≡ star (pstep κ Π) x y"

inductive_cases SkipSE[elim!]: "κ,Π ⊢ (SKIP i, s, frs) →p cfg"
inductive_cases AssignSE[elim!]: "κ,Π ⊢ (Assign i x a, s, frs) →p cfg"
inductive_cases CheckSE[elim!]: "κ,Π ⊢ (Check i l c, s, frs) →p cfg"
inductive_cases SeqSE[elim]: "κ,Π ⊢ (Seq c1 c2, s, frs) →p cfg"
inductive_cases IfSE[elim!]: "κ,Π ⊢ (If i b c1 c2, s, frs) →p cfg"
inductive_cases WhileSE[elim]: "κ,Π ⊢ (While i b c, s, frs) →p cfg"
inductive_cases CallSE[elim]: "κ,Π ⊢ (Call i dst p actuals, s, frs) →p cfg"
inductive_cases RestoreSE[elim!]: "κ,Π ⊢ (Restore i, s, frs) →p cfg"
inductive_cases ReturnSE[elim!]: "κ,Π ⊢ (Return i e, s, frs) →p cfg"
inductive_cases UnwindSE[elim!]: "κ,Π ⊢ (Unwind i, s, frs) →p cfg"

declare pstep.intros [simp, intro]

lemma combine_env_rv_irrelevant [simp]:
  "combine_env is_global fr (t(ReturnVar := v)) = combine_env is_global fr t"
  by (rule ext) simp

lemma psteps_Seq2:
  "κ,Π ⊢ (c1, s, frs) →p* (c1', s', frs')
   ⟹ κ,Π ⊢ (Seq c1 c2, s, frs) →p* (Seq c1' c2, s', frs')"
  by (induction rule: star.induct[of "pstep κ Π", split_format(complete)])
     (auto intro: star.step)
```

Frame-stack extension

A step never inspects the frames below the one it works on, so any run stays valid with more callers underneath. This is what lets a proof about a procedure in isolation be reused at an arbitrary call depth.

```
lemma pstep_frame_extend:
  "κ,Π ⊢ (c, s, frs) →p (c', s', frs') ⟹
   κ,Π ⊢ (c, s, frs @ extra) →p (c', s', frs' @ extra)"
  by (induction "(c, s, frs)" "(c', s', frs')"
        arbitrary: c s frs c' s' frs' rule: pstep.induct)
     auto
```

Call completion

Running a whole call in one step of reasoning: a body that finishes under its own frame lets the caller resume with the frame popped and the return slot written. Callers use these instead of replaying the `Restore` bookkeeping at every call site.

```
lemma pstep_Call:
  assumes "Π p = Some decl"
      and "length actuals = length (formals decl)"
      and "distinct (formals decl)"
  shows "κ,Π ⊢ (Call i dst p actuals, s, frs) →p (Seq (body decl) (Restore i),
```

```
           bind_formals (formals decl)
             (map2 (λx. kind_conv κ (Some x)) (formals decl) (map (λe. eval e s) actuals))
             (enter_state is_global s),
           Frame s dst # frs)"
  using assms by (rule pstep.Call) (rule refl)+

lemma pstep_Call_parameterless [intro]:
  assumes "Π p = Some (⦇formals = [], body = c⦈)"
  shows
    "κ,Π ⊢ (Call i dst p [], s, frs) →p (Seq c (Restore i), enter_state is_global s, Frame s dst # frs)"
proof -
  have "κ,Π ⊢ (Call i dst p [], s, frs) →p (Seq (body (⦇formals = [], body = c⦈)) (Restore i),
          bind_formals (formals (⦇formals = [], body = c⦈))
            (map2 (λx. kind_conv κ (Some x)) (formals (⦇formals = [], body = c⦈))
              (map (λe. eval e s) []))
            (enter_state is_global s),
          Frame s dst # frs)"
    using assms by (rule pstep_Call) simp_all
  then show ?thesis by simp
qed
```

A declaration without an initializer

The frontend writes a local's declaration without an initializer as the library call `x = __voblint_nondet_int()` at the declaration's position, so it runs on every activation and every loop iteration that reaches it. That call may leave any value of `x`'s kind in `x` and changes nothing else. Its outcomes do not depend on what `x` held before: the `0` a call entry wrote, or the value of an earlier iteration or activation. That the frontend places every use of a local after its declaration is trusted.

```
lemma pstep_declare:
  assumes "Π special_pname_nondet_int = None"
  shows "κ,Π ⊢ (gcom.Call i (Some x) special_pname_nondet_int [], s, frs) →p (c', s', frs')
         ⟷ c' = SKIP i ∧ frs' = frs ∧ (∃n ∈ ik_range (κ x). s' = s(x := n))"
proof
  assume "κ,Π ⊢ (gcom.Call i (Some x) special_pname_nondet_int [], s, frs) →p (c', s', frs')"
  then show "c' = SKIP i ∧ frs' = frs ∧ (∃n ∈ ik_range (κ x). s' = s(x := n))"
    using assms by (auto elim!: CallSE simp: special_table_def kind_conv_simps)
next
  assume "c' = SKIP i ∧ frs' = frs ∧ (∃n ∈ ik_range (κ x). s' = s(x := n))"
  then obtain n where n: "n ∈ ik_range (κ x)" and eq: "c' = SKIP i" "frs' = frs" "s' = s(x := n)"
    by blast
  have "κ,Π ⊢ (gcom.Call i (Some x) special_pname_nondet_int [], s, frs) →p (SKIP i, s(x := kind_conv κ (Some x)
n), frs)"
    by (rule pstep.Special[where desc = SD_Nondet_Int and sc = "Nondet_Int :: exp gspecial_call"])
       (simp_all add: special_table_def)
  with n eq
  show "κ,Π ⊢ (gcom.Call i (Some x) special_pname_nondet_int [], s, frs) →p (c', s', frs')"
    by (simp add: kind_conv_simps del: ik_range_iff)
qed

corollary pstep_declare_forgets:
  assumes "Π special_pname_nondet_int = None"
  shows "κ,Π ⊢ (gcom.Call i (Some x) special_pname_nondet_int [], s(x := m), frs) →p cfg
         ⟷ κ,Π ⊢ (gcom.Call i (Some x) special_pname_nondet_int [], s, frs) →p cfg"
  by (cases cfg) (simp add: pstep_declare[where Π = Π, OF assms])
```

The typed program

A procedure's typed formals are the identities its formals resolve to, and its typed body is its elaboration.

```
definition elab_ptable :: "imp_prog ⇒ kind_decls ⇒ (unit, var_id, exp) gptable" where
  "elab_ptable p ds q =
     map_option (λd. ⦇formals = map (resolve_var (declared_global p) q) (formals d),
                      body = elab_com (declared_global p) (prog_formals p) (kenv_of ds) q (body d)⦈)
       (prog_table p q)"

lemma elab_ptable_None_iff [simp]: "elab_ptable p ds q = None ⟷ prog_table p q = None"
  by (simp add: elab_ptable_def)
```

Elaboration keeps the control structure

Elaboration rewrites expressions only, so every predicate on a command's control structure is the same before and after it.

```
lemma elab_com_control [simp]:
```

```
  "source_com (elab_com 𝒢 fm κ q c) = source_com c"
  "may_fallthrough (elab_com 𝒢 fm κ q c) = may_fallthrough c"
  "may_return_none (elab_com 𝒢 fm κ q c) = may_return_none c"
  "may_return_value (elab_com 𝒢 fm κ q c) = may_return_value c"
  "no_return (elab_com 𝒢 fm κ q c) = no_return c"
  by (induction c) auto

lemma value_providing_elab_com [simp]:
  "value_providing (elab_com 𝒢 fm κ q c) = value_providing c"
  by (simp add: value_providing_def)

lemma classify_special_map_None_iff [simp]:
  "classify_special d (map f xs) = None ⟷ classify_special d xs = None"
  by (cases "(d, xs)" rule: classify_special.cases) auto
```

Well-formedness carries over

Elaborated expressions never mention the return slot, so a well-formed source program elaborates to a well-formed typed program.

```
lemma ReturnVar_notin_range_resolve_var [simp]: "ReturnVar ∉ range (resolve_var 𝒢 q)"
  by (auto simp: resolve_var_def)

lemma ReturnVar_notin_elaborate [simp]:
  "ReturnVar ∉ exp_vars (elaborate (resolve_var 𝒢 q) κ ik e)"
  using exp_vars_elaborate [of "resolve_var 𝒢 q" κ ik e]
    ReturnVar_notin_range_resolve_var [of 𝒢 q] by blast

lemma gsource_exp_elaborate [simp]:
  "gsource_exp exp_vars ReturnVar (elaborate (resolve_var 𝒢 q) κ ik e)"
  "gsource_exp exp_vars ReturnVar (elaborate_syn (resolve_var 𝒢 q) κ e)"
  by (simp_all add: gsource_exp_def elaborate_syn_def)

lemma gsource_exp_elab_args:
  "list_all (gsource_exp exp_vars ReturnVar) (elab_args (resolve_var 𝒢 q) κ fs es)"
  unfolding list_all_iff gsource_exp_def
  using exp_vars_elab_args ReturnVar_notin_range_resolve_var by blast

lemma gwf_source_com_elab_com:
  assumes wf: "wf_source_program (declared_global p) (prog_table p)"
    and c: "wf_source_com (prog_table p) c"
  shows "gwf_source_com exp_vars ReturnVar (elab_ptable p ds)
           (elab_com (declared_global p) (prog_formals p) (kenv_of ds) q c)"
  using c
proof (induction c)
  case (Call i dst r es)
  show ?case
  proof (cases "special_table r")
    case (Some desc)
    with wf have "prog_table p r = None"
      by (auto simp: gwf_source_program_def)
    then have "prog_formals p r = []" by (simp add: prog_formals_def)
    with Call Some show ?thesis
      by (auto simp: list_all_iff split: option.splits)
  next
    case None
    with Call show ?thesis
      by (auto simp: elab_ptable_def prog_formals_def gsource_exp_elab_args
               split: option.splits)
  qed
qed (auto split: option.splits)

lemma elab_ptable_main:
  assumes wf: "wf_source_program (declared_global p) (prog_table p)"
  shows "elab_ptable p ds prog_main_name
           = Some (|formals = [], body = main_body (elab_ptable p ds)|)"
    and "main_body (elab_ptable p ds)
           = elab_com (declared_global p) (prog_formals p) (kenv_of ds) prog_main_name
               (main_body (prog_table p))"
proof -
  from wf have "prog_table p prog_main_name = Some (|formals = [], body = main_body (prog_table p)|)"
    by (simp add: gwf_source_program_def)
  then have t: "elab_ptable p ds prog_main_name
     = Some (|formals = [], body = elab_com (declared_global p) (prog_formals p) (kenv_of ds)
                                     prog_main_name (main_body (prog_table p))|)"
    by (simp add: elab_ptable_def)
```

```
  then show "main_body (elab_ptable p ds)
      = elab_com (declared_global p) (prog_formals p) (kenv_of ds) prog_main_name
          (main_body (prog_table p))"
    by (simp add: main_body_def)
  with t show "elab_ptable p ds prog_main_name
      = Some (|formals = [], body = main_body (elab_ptable p ds)|)"
    by simp
qed

lemma gwf_proc_decl_elab:
  assumes wf: "wf_source_program (declared_global p) (prog_table p)"
    and d: "prog_table p q = Some d"
  shows "gwf_proc_decl exp_vars ReturnVar is_global (elab_ptable p ds)
           (|formals = map (resolve_var (declared_global p) q) (formals d),
            body = elab_com (declared_global p) (prog_formals p) (kenv_of ds) q (body d)|)"
proof -
  from wf d have pd: "wf_proc_decl (declared_global p) (prog_table p) d"
    by (simp add: gwf_source_program_def)
  then have local: "∀x ∈ set (formals d). ¬ declared_global p x"
    by (auto simp: gwf_proc_decl_def gvalid_formal_def list_all_iff)
  then have "inj_on (resolve_var (declared_global p) q) (set (formals d))"
    by (auto simp: inj_on_def resolve_var_eq_iff)
  with pd have "distinct (map (resolve_var (declared_global p) q) (formals d))"
    by (simp add: gwf_proc_decl_def distinct_map)
  moreover have "list_all (gvalid_formal ReturnVar is_global)
                   (map (resolve_var (declared_global p) q) (formals d))"
    using local by (auto simp: list_all_iff gvalid_formal_def resolve_var_def)
  moreover have "gwf_source_com exp_vars ReturnVar (elab_ptable p ds)
                   (elab_com (declared_global p) (prog_formals p) (kenv_of ds) q (body d))"
    using pd by (intro gwf_source_com_elab_com [OF wf]) (simp add: gwf_proc_decl_def)
  ultimately show ?thesis by (simp add: gwf_proc_decl_def)
qed

end
```

1.12 VIMP_Grammar_Generated

```
theory VIMP_Grammar_Generated
  imports VIMP_Proc
begin
```

GENERATED FILE. Source: `manifests/vimp-grammar.yaml`; generator: `scripts/gen_vimp_isabelle.py`. Regenerate with the generator rather than hand-editing; a CI drift check compares regenerated output against this file.

Owns the grammar-level nonterminals, syntax, and lowering (`Vimp_Grammar_Tr`) shared by every VIMP quotation: expressions, statements, statement lists, actuals, and the two identifier-list shapes (`formals`/`ids`). Does not register `imp ⟦ ... ⟧` or the whole-program `program { ... }` quotation itself, and has no notion of `imp_prog` – those are `VIMP_Notation`'s concern (main-selection, `proc_rep` construction, and everything else specific to that record shape), which imports this theory and calls into `Vimp_Grammar_Tr` for the pieces that are the same regardless of what the surrounding quotation ultimately builds.

Two lowering rules stay hand-written inside `Vimp_Grammar_Tr` rather than being generated from `manifests/vimp-grammar.yaml` directly: Isabelle's binary `Num` literal decoding (`dest_num`/`read_num_const`, no Menhir analogue) and the compositional unary-minus rule (folds a numeral operand into a negative `N`, otherwise `Minus (N 0) x` – deliberately does NOT cancel a nested unary minus; `--x` lowers to `Minus (N 0) (Minus (N 0) (V x))`, matching the shipped CLI frontend, not to `x`). `_exp_zero`/`_exp_one` and `_stmt_call0`/`_stmt_callret0` are Isabelle-target realizations of a canonical rule that Isabelle's own mixfix grammar cannot express directly: `num_const` has no derivation for the literal `0` or `1` (`Num.num`'s `One`/`Bit0`/`Bit1` encoding has no zero, and `1` is the unwrapped base case), and mixfix lists have no empty derivation at all – both confirmed empirically, not just by inspection: an isolation load test with only the generic `actuals` production failed to parse `f()`.

```
nonterminal imp2_exp
nonterminal imp2_stmt
nonterminal imp2_if_stmt
nonterminal imp2_stmts
nonterminal imp2_stmts_opt
nonterminal imp2_actuals
nonterminal imp2_formals
nonterminal imp2_ids
nonterminal imp2_kind

syntax
  "_exp_var" :: "id_position => imp2_exp" ("_" 1000)
  "_exp_num" :: "num_const => imp2_exp" ("_" 1000)
  "_exp_uminus" :: "imp2_exp => imp2_exp" ("- _" [90] 90)
  "_exp_plus" :: "imp2_exp => imp2_exp => imp2_exp" ("_ + _" [70, 71] 70)
  "_exp_minus" :: "imp2_exp => imp2_exp => imp2_exp" ("_ - _" [70, 71] 70)
  "_exp_times" :: "imp2_exp => imp2_exp => imp2_exp" ("_ * _" [80, 81] 80)
  "_exp_div" :: "imp2_exp => imp2_exp => imp2_exp" ("_ '/ _" [80, 81] 80)
  "_exp_mod" :: "imp2_exp => imp2_exp => imp2_exp" ("_ % _" [80, 81] 80)
  "_exp_true" :: imp2_exp ("true" 1000)
  "_exp_false" :: imp2_exp ("false" 1000)
  "_exp_less" :: "imp2_exp => imp2_exp => imp2_exp" ("_ < _" [61, 61] 60)
  "_exp_less_eq" :: "imp2_exp => imp2_exp => imp2_exp" ("_ <= _" [61, 61] 60)
  "_exp_greater" :: "imp2_exp => imp2_exp => imp2_exp" ("_ > _" [61, 61] 60)
  "_exp_greater_eq" :: "imp2_exp => imp2_exp => imp2_exp" ("_ >= _" [61, 61] 60)
  "_exp_eq" :: "imp2_exp => imp2_exp => imp2_exp" ("_ == _" [51, 51] 50)
  "_exp_not_eq" :: "imp2_exp => imp2_exp => imp2_exp" ("_ != _" [51, 51] 50)
  "_exp_not" :: "imp2_exp => imp2_exp" ("! _" [90] 90)
  "_exp_and" :: "imp2_exp => imp2_exp => imp2_exp" ("_ && _" [40, 41] 40)
  "_exp_or" :: "imp2_exp => imp2_exp => imp2_exp" ("_ || _" [30, 31] 30)
  "_exp_paren" :: "imp2_exp => imp2_exp" ("'( _ ')" [0] 1000)
  "_exp_cast" :: "imp2_kind => imp2_exp => imp2_exp" ("'( _ ') _" 90)
  "_kind_i8" :: imp2_kind ("i8")
  "_kind_u8" :: imp2_kind ("u8")
  "_kind_i16" :: imp2_kind ("i16")
  "_kind_u16" :: imp2_kind ("u16")
  "_kind_i32" :: imp2_kind ("i32")
  "_kind_u32" :: imp2_kind ("u32")
  "_kind_i64" :: imp2_kind ("i64")
  "_kind_u64" :: imp2_kind ("u64")
  "_stmt_skip" :: imp2_stmt ("skip ;" 61)
  "_stmt_assign" :: "id_position => imp2_exp => imp2_stmt" ("_ = _ ;" [900, 0] 61)
  "_stmt_return" :: "imp2_exp => imp2_stmt" ("return _ ;" [0] 61)
  "_stmt_return0" :: imp2_stmt ("return ;" 61)
  "_stmt_check" :: "imp2_exp => imp2_stmt" ("'_'_voblint'_check '( _ ') ;" [0] 61)
  "_stmt_if" :: "imp2_exp => imp2_stmts_opt => imp2_stmts_opt => imp2_if_stmt" ("if '( _ ') { _ } else { _ }" [0, 0,
0] 61)
```

```
  "_stmt_if_then" :: "imp2_exp => imp2_stmts_opt => imp2_if_stmt" ("if '( _ ') { _ }" [0, 0] 61)
  "_stmt_if_chain" :: "imp2_exp => imp2_stmts_opt => imp2_if_stmt => imp2_if_stmt" ("if '( _ ') { _ } else _" [0, 0,
0] 61)
  "_stmt_conditional" :: "imp2_if_stmt => imp2_stmt" ("_" [0] 61)
  "_stmt_while" :: "imp2_exp => imp2_stmts_opt => imp2_stmt" ("while '( _ ') { _ }" [0, 0] 61)
  "_stmt_call" :: "id_position => imp2_actuals => imp2_stmt" ("_'( _ ') ;" [1000, 0] 61)
  "_stmt_callret" :: "id_position => id_position => imp2_actuals => imp2_stmt" ("_ = _'( _ ') ;" [900, 1000, 0] 61)
  "_stmts_one" :: "imp2_stmt => imp2_stmts" ("_" 61)
  "_stmts_seq" :: "imp2_stmts => imp2_stmt => imp2_stmts" ("_ _" [61, 61] 61)
  "_stmts_opt_none" :: imp2_stmts_opt ("")
  "_stmts_opt_some" :: "imp2_stmts => imp2_stmts_opt" ("_")
  "_actuals_one" :: "imp2_exp => imp2_actuals" ("_")
  "_actuals_cons" :: "imp2_exp => imp2_actuals => imp2_actuals" ("_, _")
  "_formals_one" :: "id_position => imp2_formals" ("_")
  "_formals_cons" :: "id_position => imp2_formals => imp2_formals" ("_, _")
  "_ids_one" :: "id_position => imp2_ids" ("_")
  "_ids_cons" :: "id_position => imp2_ids => imp2_ids" ("_, _")
  "_exp_zero" :: imp2_exp ("0" 1000)
  "_exp_one" :: imp2_exp ("1" 1000)
  "_stmt_call0" :: "id_position => imp2_stmt" ("_'(') ;" [1000] 61)
  "_stmt_callret0" :: "id_position => id_position => imp2_stmt" ("_ = _'(') ;" [900, 1000] 61)
```

ML ‹

```
structure Vimp_Grammar_Tr =
struct
  val c_N      = "VIMP_Syntax.N"
  val c_V      = "VIMP_Syntax.V"
  val c_Plus   = "VIMP_Syntax.src_exp.Plus"
  val c_Minus  = "VIMP_Syntax.src_exp.Minus"
  val c_Times  = "VIMP_Syntax.src_exp.Times"
  val c_Div = "VIMP_Syntax.src_exp.Div"
  val c_Mod = "VIMP_Syntax.src_exp.Mod"

  val c_Less   = "VIMP_Syntax.src_exp.Less"
  val c_LessEq = "VIMP_Syntax.src_exp.LessEq"
  val c_Greater = "VIMP_Syntax.src_exp.Greater"
  val c_GreaterEq = "VIMP_Syntax.src_exp.GreaterEq"
  val c_NotEq = "VIMP_Syntax.src_exp.NotEq"
  val c_Eq     = "VIMP_Syntax.src_exp.Eq"
  val c_Not    = "VIMP_Syntax.src_exp.Not"
  val c_And    = "VIMP_Syntax.src_exp.And"
  val c_Or     = "VIMP_Syntax.src_exp.Or"
  val c_Cast   = "VIMP_Syntax.src_exp.Cast"

  val c_I8  = "Machine_Kind.ikind.I8"
  val c_U8  = "Machine_Kind.ikind.U8"
  val c_I16 = "Machine_Kind.ikind.I16"
  val c_U16 = "Machine_Kind.ikind.U16"
  val c_I32 = "Machine_Kind.ikind.I32"
  val c_U32 = "Machine_Kind.ikind.U32"
  val c_I64 = "Machine_Kind.ikind.I64"
  val c_U64 = "Machine_Kind.ikind.U64"

  val c_SKIP   = "VIMP_Proc.gcom.SKIP"
  val c_Assign = "VIMP_Proc.gcom.Assign"
  val c_Seq    = "VIMP_Proc.gcom.Seq"
  val c_If     = "VIMP_Proc.gcom.If"
  val c_While  = "VIMP_Proc.gcom.While"
  val c_Call   = "VIMP_Proc.gcom.Call"
  val c_Return = "VIMP_Proc.gcom.Return"
  val c_Check  = "VIMP_Proc.gcom.Check"

  val c_None   = "Option.option.None"
  val c_Some   = "Option.option.Some"
  val c_Cons   = "List.list.Cons"
  val c_Nil    = "List.list.Nil"

  (* Parsed statements carry the unit annotation; elaboration assigns identities later. *)
  val annotated_ctors = [c_SKIP, c_Assign, c_If, c_While, c_Call, c_Return, c_Check]

  fun K name =
    if member (op =) annotated_ctors name
    then Const (name, dummyT) $ Const ("Product_Type.Unity", dummyT)
    else Const (name, dummyT)

  (* Decode Isabelle's Num binary structure: One=1, Bit0 n=2n, Bit1 n=2n+1.
```

```
   Leaf may also be a decimal-string Const (e.g. Const("20",_)) from the
   raw lexer. *)
fun dest_num (Const (c, _)) =
      let val name = Long_Name.base_name c
      in if name = "One" then 1
         else case Int.fromString name of
                SOME n => n
              | NONE => raise TERM ("Vimp_Grammar_Tr: not a num leaf", [Const (c, dummyT)])
      end
  | dest_num (Const (c, _) $ t) =
      let val name = Long_Name.base_name c
          val n    = dest_num t
      in if name = "Bit0" then 2 * n
         else if name = "Bit1" then 2 * n + 1
         else raise TERM ("Vimp_Grammar_Tr: not a num constructor", [Const (c, dummyT) $ t])
      end
  | dest_num t = raise TERM ("Vimp_Grammar_Tr: dest_num catchall", [t])

fun read_num_const (Const ("_constify", _) $ t) = read_num_const t
  | read_num_const (Const ("_position", _) $ t) = read_num_const t
  | read_num_const ((Const ("_constrain", _) $ t) $ _) = read_num_const t
  | read_num_const (Free (s, _)) =
      (case Int.fromString s of
         SOME n => n
       | NONE => raise TERM ("Vimp_Grammar_Tr: not a numeral", [Free (s, dummyT)]))
  | read_num_const (Const (s, _)) =
      (case Int.fromString (Long_Name.base_name s) of
         SOME n => n
       | NONE => raise TERM ("Vimp_Grammar_Tr: not a numeral", [Const (s, dummyT)]))
  | read_num_const t = dest_num t

fun neg_num n = K c_N $ HOLogic.mk_number HOLogic.intT (~ n)

fun dest_id_position report_markup ctxt (Const ("_constrain", _) $ Free (s, _) $ m) =
      (case (report_markup, Term_Position.decode_position m) of
         (SOME markup, SOME (ps, _)) =>
           (List.app (fn p => Context_Position.report ctxt (#pos p) markup) ps; s)
       | _ => s)
  | dest_id_position _ _ (Free (s, _)) = s
  | dest_id_position _ _ t = raise TERM ("Vimp_Grammar_Tr: dest_id_position", [t])

fun kind_tr ctxt t =
      (case Term.strip_comb t of
         (Const ("_kind_i8", _), []) => K c_I8
       | (Const ("_kind_u8", _), []) => K c_U8
       | (Const ("_kind_i16", _), []) => K c_I16
       | (Const ("_kind_u16", _), []) => K c_U16
       | (Const ("_kind_i32", _), []) => K c_I32
       | (Const ("_kind_u32", _), []) => K c_U32
       | (Const ("_kind_i64", _), []) => K c_I64
       | (Const ("_kind_u64", _), []) => K c_U64
       | _ => raise TERM ("Vimp_Grammar_Tr: kind_tr", [t]))

fun exp_tr ctxt t =
      (case Term.strip_comb t of
         (Const ("_exp_num", _), [n]) =>
           K c_N $ HOLogic.mk_number HOLogic.intT (read_num_const n)
       | (Const ("_exp_uminus", _), [a]) =>
           (case Term.strip_comb a of
              (Const ("_exp_num", _), [n]) => neg_num (read_num_const n)
            | (Const ("_exp_zero", _), []) =>
              K c_N $ HOLogic.mk_number HOLogic.intT 0
            | (Const ("_exp_one", _), []) => neg_num 1
            | _ =>
              K c_Minus $ (K c_N $ HOLogic.mk_number HOLogic.intT 0) $ exp_tr ctxt a)
       | (Const ("_exp_zero", _), []) => K c_N $ HOLogic.mk_number HOLogic.intT 0
       | (Const ("_exp_one", _), []) => K c_N $ HOLogic.mk_number HOLogic.intT 1
       | (Const ("_exp_var", _), [x0]) => K c_V $ (HOLogic.mk_literal (dest_id_position (SOME Markup.free) ctxt x0))
       | (Const ("_exp_plus", _), [a0, a2]) => K c_Plus $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
       | (Const ("_exp_minus", _), [a0, a2]) => K c_Minus $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
       | (Const ("_exp_times", _), [a0, a2]) => K c_Times $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
       | (Const ("_exp_div", _), [a0, a2]) => K c_Div $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
       | (Const ("_exp_mod", _), [a0, a2]) => K c_Mod $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
       | (Const ("_exp_true", _), []) => K c_N $ (HOLogic.mk_number HOLogic.intT 1)
       | (Const ("_exp_false", _), []) => K c_N $ (HOLogic.mk_number HOLogic.intT 0)
       | (Const ("_exp_less", _), [a0, a2]) => K c_Less $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
```

```
        | (Const ("_exp_less_eq", _), [a0, a2]) => K c_LessEq $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
        | (Const ("_exp_greater", _), [a0, a2]) => K c_Greater $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
        | (Const ("_exp_greater_eq", _), [a0, a2]) => K c_GreaterEq $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
        | (Const ("_exp_eq", _), [a0, a2]) => K c_Eq $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
        | (Const ("_exp_not_eq", _), [a0, a2]) => K c_NotEq $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
        | (Const ("_exp_not", _), [a1]) => K c_Not $ (exp_tr ctxt a1)
        | (Const ("_exp_and", _), [a0, a2]) => K c_And $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
        | (Const ("_exp_or", _), [a0, a2]) => K c_Or $ (exp_tr ctxt a0) $ (exp_tr ctxt a2)
        | (Const ("_exp_paren", _), [a1]) => exp_tr ctxt a1
        | (Const ("_exp_cast", _), [a1, a3]) => K c_Cast $ (kind_tr ctxt a1) $ (exp_tr ctxt a3)
        | _ => raise TERM ("Vimp_Grammar_Tr: exp_tr", [t]))

  fun actuals_tr ctxt (Const ("_actuals_one", _) $ x) = K c_Cons $ (exp_tr ctxt x) $ K c_Nil
    | actuals_tr ctxt (Const ("_actuals_cons", _) $ x $ rest) = K c_Cons $ (exp_tr ctxt x) $ (actuals_tr ctxt rest)
    | actuals_tr _ t = raise TERM ("Vimp_Grammar_Tr: actuals_tr", [t])

  fun stmts_tr ctxt (Const ("_stmts_one", _) $ x) = stmt_tr ctxt x
    | stmts_tr ctxt (Const ("_stmts_seq", _) $ xs $ x) = K c_Seq $ (stmts_tr ctxt xs) $ (stmt_tr ctxt x)
    | stmts_tr _ t = raise TERM ("Vimp_Grammar_Tr: stmts_tr", [t])
  and stmts_opt_tr ctxt (Const ("_stmts_opt_none", _)) = K c_SKIP
    | stmts_opt_tr ctxt (Const ("_stmts_opt_some", _) $ s) = stmts_tr ctxt s
    | stmts_opt_tr _ t = raise TERM ("Vimp_Grammar_Tr: stmts_opt_tr", [t])
  and stmt_tr ctxt t =
        (case Term.strip_comb t of
           (Const ("_stmt_call0", _), [x0]) => K c_Call $ (K c_None) $ (HOLogic.mk_literal (dest_id_position (SOME
Markup.skolem) ctxt x0)) $ K c_Nil
         | (Const ("_stmt_callret0", _), [x0, x2]) => K c_Call $ ((K c_Some $ (HOLogic.mk_literal (dest_id_position
(SOME Markup.free) ctxt x0)))) $ (HOLogic.mk_literal (dest_id_position (SOME Markup.skolem) ctxt x2)) $ K c_Nil
         | (Const ("_stmt_skip", _), []) => K c_SKIP
         | (Const ("_stmt_assign", _), [x0, a2]) => K c_Assign $ (HOLogic.mk_literal (dest_id_position (SOME Markup.free)
ctxt x0)) $ (exp_tr ctxt a2)
         | (Const ("_stmt_return", _), [a1]) => K c_Return $ ((K c_Some $ (exp_tr ctxt a1)))
         | (Const ("_stmt_return0", _), []) => K c_Return $ (K c_None)
         | (Const ("_stmt_check", _), [a2]) => K c_Check $ (HOLogic.mk_prod (HOLogic.mk_nat 0, HOLogic.mk_nat 0)) $
(exp_tr ctxt a2)
         | (Const ("_stmt_conditional", _), [a0]) => if_stmt_tr ctxt a0
         | (Const ("_stmt_while", _), [a2, a5]) => K c_While $ (exp_tr ctxt a2) $ (stmts_opt_tr ctxt a5)
         | (Const ("_stmt_call", _), [x0, a2]) => K c_Call $ (K c_None) $ (HOLogic.mk_literal (dest_id_position (SOME
Markup.skolem) ctxt x0)) $ (actuals_tr ctxt a2)
         | (Const ("_stmt_callret", _), [x0, x2, a4]) => K c_Call $ ((K c_Some $ (HOLogic.mk_literal (dest_id_position
(SOME Markup.free) ctxt x0)))) $ (HOLogic.mk_literal (dest_id_position (SOME Markup.skolem) ctxt x2)) $ (actuals_tr
ctxt a4)
         | _ => raise TERM ("Vimp_Grammar_Tr: stmt_tr", [t]))
  and if_stmt_tr ctxt t =
        (case Term.strip_comb t of
           (Const ("_stmt_if", _), [a2, a5, a9]) => K c_If $ (exp_tr ctxt a2) $ (stmts_opt_tr ctxt a5) $ (stmts_opt_tr
ctxt a9)
         | (Const ("_stmt_if_then", _), [a2, a5]) => K c_If $ (exp_tr ctxt a2) $ (stmts_opt_tr ctxt a5) $ (K c_SKIP)
         | (Const ("_stmt_if_chain", _), [a2, a5, a8]) => K c_If $ (exp_tr ctxt a2) $ (stmts_opt_tr ctxt a5) $ (if_stmt_tr
ctxt a8)
         | _ => raise TERM ("Vimp_Grammar_Tr: if_stmt_tr", [t]))

  fun formals_of ctxt (Const ("_formals_one", _) $ x) = [dest_id_position (SOME Markup.free) ctxt x]
    | formals_of ctxt (Const ("_formals_cons", _) $ x $ rest) = dest_id_position (SOME Markup.free) ctxt x :: formals_of
ctxt rest
    | formals_of _ t = raise TERM ("Vimp_Grammar_Tr: formals_of", [t])

  fun names_of ctxt (Const ("_ids_one", _) $ x) = [dest_id_position (SOME Markup.bound) ctxt x]
    | names_of ctxt (Const ("_ids_cons", _) $ x $ rest) = dest_id_position (SOME Markup.bound) ctxt x :: names_of ctxt
rest
    | names_of _ t = raise TERM ("Vimp_Grammar_Tr: names_of", [t])
end
›
```

end

1.13 VIMP_Notation

theory VIMP_Notation
  imports VIMP_Program VIMP_Grammar_Generated
begin

Concrete syntax

The `imp ⟦ ... ⟧` quotation produces an `src_com` without HOL string quotes or qualified constructor names. The `imp` keyword distinguishes command quotation from Pure premises; whole programs use the explicit `program` prefix.

Design is inspired by: https://awslabs.github.io/AutoCorrode/Unsorted/AutoCorrode/Micro_Rust_Examples.Basic_Micro_Rust.html and https://github.com/awslabs/AutoCorrode/blob/e234addc5e67f78cbff63defd24199578e8e1af3/Micro_Rust_Parsing_Frontend/Micro_Rust_Syntax.thy#L5

Inside the bracket: - bare identifiers become `V` literals (HOL string literals via `vname`) - numerals become `N` - arithmetic: +, -, *, /, `Div`, `Mod` unary minus `-n` on numerals becomes `N` with a negative int - comparisons: <, <=, >, >=, ==, != evaluate to an `int` 0/1 - constants: true/false lower to `N` 1/`N` 0

Example:

```
definition loop_prog :: VIMP_Proc.com where
      "loop_prog = imp \<lbrakk>
         x = 0;
         while (x < 20) { x = x + 1; }
       \<rbrakk>"
```

Whole-program syntax and its `imp_prog` lowering stay hand-written here, matching the grammar's `special: program_structure` boundary; every other nonterminal and lowering function comes from the generated `Vimp_Grammar_Tr` structure.

nonterminal imp2_funcs

syntax
```
  "_IMP2"        :: "imp2_stmts_opt ⇒ VIMP_Proc.src_com"        ("imp ⟦ _ ⟧" [0] 1000)

  "_PROGKW0"    :: "imp2_funcs ⇒ imp_prog"                ("program { _ }" [0] 1000)
  "_PROGKW"     :: "imp2_ids ⇒ imp2_funcs ⇒ imp_prog"      ("program { global _ ; _ }" [0, 0] 1000)
  "_funcs_nil"   :: imp2_funcs                                ("")
  "_funcs_cons0" :: "id_position ⇒ imp2_stmts_opt ⇒ imp2_funcs ⇒ imp2_funcs"
    ("fun _'(') { _ } _" [1000, 0, 0] 1000)
  "_funcs_cons"  :: "id_position ⇒ imp2_formals ⇒ imp2_stmts_opt ⇒ imp2_funcs ⇒ imp2_funcs"
    ("fun _'( _ ') { _ } _" [1000, 0, 0, 0] 1000)
```

parse_translation ‹
```
  let
    val c_proc_decl_ext = "VIMP_Proc.gproc_decl.gproc_decl_ext"
    val c_Unity   = "Product_Type.Unity"
    val c_imp_prog = "VIMP_Program.mk_program"
    val c_Pair    = "Product_Type.Pair"

    val K = Vimp_Grammar_Tr.K
    val c_SKIP   = Vimp_Grammar_Tr.c_SKIP
    val c_Seq    = Vimp_Grammar_Tr.c_Seq
    val c_Return = Vimp_Grammar_Tr.c_Return
    val c_Some   = Vimp_Grammar_Tr.c_Some
    val c_None   = Vimp_Grammar_Tr.c_None
    val c_Cons   = Vimp_Grammar_Tr.c_Cons
    val c_Nil    = Vimp_Grammar_Tr.c_Nil

    (* Bodies stay raw parse trees until has_return has inspected them;
       mk_body_ret translates afterwards. The procedure name reports
       Markup.skolem, the same category a call site to it reports. *)
    fun funcs_tr ctxt (Const ("_funcs_nil", _)) = []
      | funcs_tr ctxt (Const ("_funcs_cons0", _) $ f $ pbody $ rest) =
          (Vimp_Grammar_Tr.dest_id_position (SOME Markup.skolem) ctxt f,
           [], SOME pbody, NONE) :: funcs_tr ctxt rest
      | funcs_tr ctxt (Const ("_funcs_cons", _) $ f $ formals $ pbody $ rest) =
          (Vimp_Grammar_Tr.dest_id_position (SOME Markup.skolem) ctxt f,
           Vimp_Grammar_Tr.formals_of ctxt formals, SOME pbody, NONE)
          :: funcs_tr ctxt rest
      | funcs_tr _ t = raise TERM ("VIMP_Notation: funcs_tr", [t])

    (* A bare "return;" is a Const without argument, matched by neither the
```

```
      _stmt_return case nor the application/abstraction fallbacks. *)
    fun has_return (Const ("_stmt_return", _) $ _) = true
      | has_return (Const ("_stmt_return0", _)) = true
      | has_return (t $ u) = has_return t orelse has_return u
      | has_return (Abs (_, _, b)) = has_return b
      | has_return _ = false

    fun check_distinct kind xs =
      (case duplicates (op =) xs of
         [] => ()
       | ds => error ("VIMP program: duplicate " ^ kind ^ ": " ^ commas_quote ds))

    (* A formal's per-call binding and a global's cross-call persistence are
       incompatible storage classes for one name. *)
    fun check_no_global_formal_collision decls funcs =
      List.app (fn (n, formals, _, _) =>
        case filter (member (op =) decls) formals of
          [] => ()
        | bad => error ("VIMP program: " ^ quote n ^ " declares global(s) as formal(s): "
                        ^ commas_quote bad)) funcs

    fun mk_names [] = K c_Nil
      | mk_names (n :: ns) = K c_Cons $ HOLogic.mk_literal n $ mk_names ns

    (* A trailing "return e" becomes an explicit Return appended to the body. *)
    fun mk_body_ret ctxt NONE NONE = K c_SKIP
      | mk_body_ret ctxt (SOME c) NONE = Vimp_Grammar_Tr.stmts_opt_tr ctxt c
      | mk_body_ret ctxt NONE (SOME e) = K c_Return $ (K c_Some $ Vimp_Grammar_Tr.exp_tr ctxt e)
      | mk_body_ret ctxt (SOME c) (SOME e) =
          K c_Seq $ Vimp_Grammar_Tr.stmts_opt_tr ctxt c
                  $ (K c_Return $ (K c_Some $ Vimp_Grammar_Tr.exp_tr ctxt e))

    fun mk_proc_rep ctxt [] = K c_Nil
      | mk_proc_rep ctxt ((p, formals, body, result) :: rest) =
          K c_Cons
            $ (K c_Pair $ HOLogic.mk_literal p
                $ (K c_proc_decl_ext $ mk_names formals $ mk_body_ret ctxt body result $ K c_Unity))
            $ mk_proc_rep ctxt rest

    (* dummyT constructors only; type inference runs after the translation. *)
    fun prog_tr ctxt decls funcs_t =
      let
        val funcs = funcs_tr ctxt funcs_t
        val _ = check_distinct "procedure" (map (fn (n, _, _, _) => n) funcs)
        val _ = check_distinct "declared global" decls
        val _ = List.app (fn (n, formals, _, _) =>
                  check_distinct ("formal parameter of " ^ quote n) formals) funcs
        val _ = check_no_global_formal_collision decls funcs
        val (mains, procs) = List.partition (fn (n, _, _, _) => n = "main") funcs
        val main_ast =
          (case mains of
             [("main", [], NONE, NONE)] => NONE
           | [("main", [], SOME body, NONE)] =>
               if has_return body then error "VIMP program: main may not return" else SOME body
           | [("main", _, _, SOME _)] => error "VIMP program: main may not return"
           | [("main", _, _, NONE)] => error "VIMP program: main must have no formals"
           | [] => error "VIMP program: missing 'fun main() { ... }'"
           | _  => error "VIMP program: more than one 'fun main()'")
        val proc_rep = mk_proc_rep ctxt procs
        val main = mk_body_ret ctxt main_ast NONE
        val decl_globals = mk_names decls
      in K c_imp_prog $ proc_rep $ main $ decl_globals end
  in
    [("_IMP2", fn ctxt => fn [t] => Vimp_Grammar_Tr.stmts_opt_tr ctxt t | _ => raise Match),
     ("_PROGKW0", fn ctxt => fn [fs] => prog_tr ctxt [] fs | _ => raise Match),
     ("_PROGKW",
      fn ctxt => fn [g, fs] => prog_tr ctxt (Vimp_Grammar_Tr.names_of ctxt g) fs
                 | _ => raise Match)]
  end
›
```

Smoke tests

Four translations, each stated as the term it elaborates to: a branch and a check through the generated statement translation, and a program with a global and one with a procedure taking arguments through the whole-program

translation hand-written here. The theory stops loading if a translation stops elaborating or elaborates to something else, which no theorem elsewhere would notice – nothing in the proved pipeline reads concrete syntax.

```
lemma "imp ⟦ if (x < 10) { x = 0; } else { x = 1; } ⟧
  = gcom.If () (Less (V STR ''x'') (N 10)) (Assign () STR ''x'' (N 0))
    (Assign () STR ''x'' (N 1))"
  by eval
lemma "(imp ⟦ __voblint_check(!(x == 0)); ⟧ :: src_com)
  = Check () (0, 0) (src_exp.Not (src_exp.Eq (V STR ''x'') (N 0)))"
  by eval
lemma "(program { global Gx; fun ping() { Gx = Gx + 1; } fun main() { ping(); } } :: imp_prog)
  = ⦇proc_rep =
    [(STR ''main'', ⦇formals = [], body = Call () None STR ''ping'' []⦈),
    (STR ''ping'',
    ⦇formals = [],
    body = Assign () STR ''Gx'' (Plus (V STR ''Gx'') (N 1))⦈)],
    declared_global_vars = [STR ''Gx'']⦈"
  by eval
lemma "(program { fun add(a, b) { skip; return a + b; } fun main() { r = add(1, 2); } } :: imp_prog)
  = ⦇proc_rep =
    [(STR ''main'',
    ⦇formals = [], body = Call () (Some STR ''r'') STR ''add'' [N 1, N 2]⦈),
    (STR ''add'',
    ⦇formals = [STR ''a'', STR ''b''],
    body =
    Seq (SKIP ()) (Return () (Some (Plus (V STR ''a'') (V STR ''b'')))))⦈)],
    declared_global_vars = []⦈"
  by eval

end
```

Chapter 2

Voblint_Domain

2.1 Abstract_Domain

```
theory Abstract_Domain
  imports "TD.Warrowing"
begin

unbundle lattice_syntax
```

Abstract value domains

An executable domain supplies the required lattice operations, the widening and narrowing the solver applies (the vendored `warrowing` class), and an exact emptiness test. Sound domains add a concretization into integers. Abstract states and control-flow reachability are separate modules.

Exact and sound emptiness

An emptiness test is *exact* for a concretization when it holds of exactly the values that describe nothing, and *sound* when every value it holds of describes nothing, with no converse. Each layer states which of the two its test is: a single domain's and a single state's test are exact, the combined state's only sound, because two analyses can each describe some value while their intersection is empty.

```
definition sound_emptiness :: "('s ⇒ bool) ⇒ ('s ⇒ 'c set) ⇒ bool" where
  "sound_emptiness e γ' ⟷ (∀x. e x ⟶ γ' x = {})"

definition exact_emptiness :: "('s ⇒ bool) ⇒ ('s ⇒ 'c set) ⇒ bool" where
  "exact_emptiness e γ' ⟷ (∀x. e x ⟷ γ' x = {})"

lemma sound_emptinessI [intro]: "(⋀x. e x ⟹ γ' x = {}) ⟹ sound_emptiness e γ'"
  unfolding sound_emptiness_def by blast

lemma sound_emptinessD [dest]: "sound_emptiness e γ' ⟹ e x ⟹ γ' x = {}"
  unfolding sound_emptiness_def by blast

lemma exact_emptinessI [intro]: "(⋀x. e x ⟷ γ' x = {}) ⟹ exact_emptiness e γ'"
  unfolding exact_emptiness_def by blast

lemma exact_emptiness_sound: "exact_emptiness e γ' ⟹ sound_emptiness e γ'"
  unfolding exact_emptiness_def sound_emptiness_def by blast
```

Executable and sound domains

The vendored `widening` and `narrowing` classes constrain their type parameter (`'a::order`) instead of naming `order` as a superclass, so a class extending them starts from `order` and would reject $\bot$ and $\top$ in its axioms. The lattice classes therefore enter the same way, through the sort of the first parameter, and `numeric_domain` repeats that sort.

```
class executable_domain = warrowing +
  fixes is_empty :: "'a::{bounded_semilattice_sup_bot, order_top} ⇒ bool"
  fixes to_string :: "'a ⇒ String.literal"
```

A numeric domain adds a concretization `gamma` to an executable domain: bottom denotes no integer, top every integer, the order is sound for `gamma`, and `is_empty` decides emptiness exactly.

```
class numeric_domain = executable_domain +
  fixes gamma :: "'a::{bounded_semilattice_sup_bot, order_top} ⇒ int set" ("γ")
  assumes gamma_bot[simp]: "γ ⊥ = {}"
  assumes gamma_top[simp]: "γ ⊤ = UNIV"
  assumes gamma_mono: "a ≤ b ⟹ γ a ⊆ γ b"
  assumes is_empty_correct: "is_empty a ⟷ γ a = {}"

lemma exact_emptiness_is_empty: "exact_emptiness is_empty (γ :: 'a::numeric_domain ⇒ int set)"
  by (rule exact_emptinessI) (rule is_empty_correct)
```

`executable_domain` carries exactly the executable per-element operations a concrete domain's runtime representation needs: the lattice structure, the solver's `widen` and `narrow`, `is_empty` (a finite decision procedure on every real instance – Interval's bound comparison, Sign's constructor match, ...), and `to_string` for reporting a solved value back to a caller. `numeric_domain` extends it with `gamma`, which is not executable in general (an infinite `int set`) and exists purely to state and prove soundness. Splitting the class this way keeps `gamma` out of the type-class dictionary that code generation must materialize for any constant that only needs `is_empty` (the finite witness-bottom tests over an executable carrier, in particular): requesting `'a::executable_domain`

there never drags γ's code equation into the dependency closure, even though every `numeric_domain` instance is automatically a `executable_domain` instance too.

`is_empty` is a semantic classifier, not a structural equality test against `bot`: it mirrors Goblint's own `Lattice.Bot` signature (https://github.com/goblint/analyzer/blob/master/src/domain/lattice.ml): `val is_bot: t -> bool` is a per-domain operation there too, not a generic derived test. Goblint's own default implementation (`IntDomain0.Std` in https://github.com/goblint/analyzer/blob/master/src/cdomain/value/cdomains/intDomain0.ml) in fact defines it as structural equality against one canonical `bot_of` value, since most of its domains keep a single bottom representation (Interval's `bot () = None`, normalized on every operation). Voblint cannot take that shortcut: some Voblint domains admit representations with more than one empty-denoting value that are never normalized away (Interval's inverted bound pairs, e.g. `Ivl (Fin (5::'c)) (Fin (- 1))`, none of them favored over `bot` itself), so `a = bot` would silently miss some of them. Fixing `is_empty` as its own class operation, correct against γ rather than against $\bot$, makes every `numeric_domain` instance responsible for its own exact emptiness test, the same obligation every domain already carries for γ itself. The lattice constant `bot` stays the canonical representative; `is_empty` answers a different question (what a value denotes), and a proof that genuinely needs the canonical element still writes `a = bot` directly.

Concretization bounds

```
lemma gamma_sup_ub1[intro]: "γ a ⊆ γ (a ⊔ b)" for a b :: "'a::numeric_domain"
  by (rule gamma_mono[OF sup_ge1])

lemma gamma_sup_ub2[intro]: "γ b ⊆ γ (a ⊔ b)" for a b :: "'a::numeric_domain"
  by (rule gamma_mono[OF sup_ge2])
```

Emptiness is downward closed under the abstract order: it follows from `gamma_mono` alone, with no per-domain fact needed. This is what lets the generic transfer dispatcher's short-circuit stay monotone: a smaller input can only be witness-empty *more* often than a larger one, never less.

```
lemma is_empty_antimono:
  "a ≤ b ⟹ is_empty b ⟹ is_empty a" for a b :: "'a::numeric_domain"
  using gamma_mono unfolding is_empty_correct by blast
```

Domains with widening

`warrow` is deliberately not join-like: it takes the narrowing branch whenever `b` $\leq$ `a`, and narrowing moves the result down toward `b`, not up past `a`, so no `a` $\leq$ `a` $\nabla\Delta$ `b` counterpart to `warrowing_properties` holds in general. What does hold unconditionally in that regime is the matching upper bound on the `a` side, `warrow_le_when_le`, together with `warrowing_properties`' `b` $\leq$ `a` $\nabla\Delta$ `b` that gives the full sandwich `b` $\leq$ `a` $\nabla\Delta$ `b` $\leq$ `a`. `warrow_idem` is the fixpoint case of that sandwich.

Printing values

How a solved value is shown, following Goblint's notation (`1+3`$\mathbb{Z}$, $\top$, $\bot$). Nothing is proved about it: it is not part of any soundness claim.

A `String.literal` holds ASCII only, so a mathematical symbol is written as its Isabelle symbol name without the backslash — `<int>` for $\mathbb{Z}$ — and the CLI decodes these tokens into Unicode before rendering. No variable name, numeral or bracket a printer emits contains `<`, so the tokens are unambiguous.

```
abbreviation (input) sym_bottom :: String.literal where "sym_bottom ≡ STR ''<bottom>''"
abbreviation (input) sym_top :: String.literal where "sym_top ≡ STR ''<top>''"
abbreviation (input) sym_int :: String.literal where "sym_int ≡ STR ''<int>''"
abbreviation (input) sym_infinity :: String.literal where "sym_infinity ≡ STR ''<infinity>''"
abbreviation (input) sym_le :: String.literal where "sym_le ≡ STR ''<le>''"
abbreviation (input) sym_ge :: String.literal where "sym_ge ≡ STR ''<ge>''"
abbreviation (input) sym_and :: String.literal where "sym_and ≡ STR ''<and>''"

fun string_of_nat :: "nat ⇒ String.literal" where
  "string_of_nat n =
     (if n < 10 then String.implode [char_of (n + 48)]
      else string_of_nat (n div 10) + String.implode [char_of (n mod 10 + 48)])"

definition string_of_int :: "int ⇒ String.literal" where
  "string_of_int i =
     (if i < 0 then STR ''-'' + string_of_nat (nat (- i))
      else string_of_nat (nat i))"

end
```

2.2 Kind_Lifted

```
theory Kind_Lifted
  imports Abstract_Domain "Voblint_VIMP.Machine_Kind"
begin
```

Kind-tagged values

Goblint's `IntDomLifter` stores an integer domain element together with its `ikind`, so that operations without a kind argument, the lattice join and the solver's widening among them, can still act at the right kind. The value domain around it adds a bottom and a top, and turns a join of elements of different kinds into top. `'a kd` is that construction: `KD k a` is the element `a` at kind `k`, and it denotes only integers in the range of `k`.

The component domain supplies the kind-indexed operations of Goblint's `IntDomain.S` that the lifter needs: widening and narrowing at a kind, and an emptiness test against a kind's range.

```
class ik_domain = numeric_domain +
  fixes widen_ik :: "ikind ⇒ 'a::{bounded_semilattice_sup_bot, order_top} ⇒ 'a ⇒ 'a"
    and narrow_ik :: "ikind ⇒ 'a ⇒ 'a ⇒ 'a"
    and empty_ik :: "ikind ⇒ 'a ⇒ bool"
  assumes widen_ik_ge1: "a ≤ widen_ik k a b"
    and widen_ik_ge2: "b ≤ widen_ik k a b"
    and narrow_ik_ge: "b ≤ a ⟹ b ≤ narrow_ik k a b"
    and narrow_ik_le: "b ≤ a ⟹ narrow_ik k a b ≤ a"
    and empty_ik_correct: "empty_ik k a ⟷ γ a ∩ ik_range k = {}"

datatype 'a kd = KBot | KD ikind 'a | KTop
```

Order and join

Elements are comparable only at the same kind. A join across kinds is top, as in Goblint's value domain, which catches the lifter's `IncompatibleIKinds`.

```
instantiation kd :: (ik_domain) order_bot
begin

fun less_eq_kd :: "'a kd ⇒ 'a kd ⇒ bool" where
  "less_eq_kd KBot _ ⟷ True"
| "less_eq_kd _ KTop ⟷ True"
| "less_eq_kd (KD k a) (KD l b) ⟷ k = l ∧ a ≤ b"
| "less_eq_kd _ _ ⟷ False"

definition less_kd :: "'a kd ⇒ 'a kd ⇒ bool" where
  "less_kd a b ⟷ a ≤ b ∧ ¬ b ≤ a"

definition bot_kd :: "'a kd" where "bot_kd = KBot"

instance proof
  fix x y z :: "'a kd"
  show "x < y ⟷ x ≤ y ∧ ¬ y ≤ x" by (simp add: less_kd_def)
  show "x ≤ x" by (cases x) simp_all
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z"
    by (cases x; cases y; cases z) (auto intro: order_trans)
  show "x ≤ y ⟹ y ≤ x ⟹ x = y"
    by (cases x; cases y) (auto intro: antisym)
  show "bot ≤ x" by (simp add: bot_kd_def)
qed

end

instantiation kd :: (ik_domain) order_top
begin

definition top_kd :: "'a kd" where "top_kd = KTop"

instance proof
  fix x :: "'a kd"
  show "x ≤ top" by (cases x) (simp_all add: top_kd_def)
qed

end

instantiation kd :: (ik_domain) semilattice_sup
begin
```

```
fun sup_kd :: "'a kd ⇒ 'a kd ⇒ 'a kd" where
  "sup_kd KBot b = b"
| "sup_kd a KBot = a"
| "sup_kd (KD k a) (KD l b) = (if k = l then KD k (a ⊔ b) else KTop)"
| "sup_kd _ _ = KTop"

instance proof
  fix x y z :: "'a kd"
  show "x ≤ x ⊔ y" by (cases x; cases y) simp_all
  show "y ≤ x ⊔ y" by (cases x; cases y) simp_all
  show "y ≤ x ⟹ z ≤ x ⟹ y ⊔ z ≤ x"
    by (cases x; cases y; cases z) auto
qed

end

instance kd :: (ik_domain) bounded_semilattice_sup_bot ..
```

Widening and narrowing at the operands' kind

Two operands of one kind are widened or narrowed by the component domain at that kind. Widening operands of different kinds gives `KTop`.

```
instantiation kd :: (ik_domain) warrowing
begin

fun widen_kd :: "'a kd ⇒ 'a kd ⇒ 'a kd" where
  "widen_kd KBot b = b"
| "widen_kd a KBot = a"
| "widen_kd (KD k a) (KD l b) = (if k = l then KD k (widen_ik k a b) else KTop)"
| "widen_kd _ _ = KTop"

fun narrow_kd :: "'a kd ⇒ 'a kd ⇒ 'a kd" where
  "narrow_kd KTop b = b"
| "narrow_kd (KD k a) (KD l b) = (if k = l then KD k (narrow_ik k a b) else KD k a)"
| "narrow_kd a _ = KBot"

instance proof
  fix a b :: "'a kd"
  show "a ≤ a ∇ b" by (cases a; cases b) (simp_all add: widen_ik_ge1)
  show "b ≤ a ∇ b" by (cases a; cases b) (simp_all add: widen_ik_ge2)
  show "b ≤ a ⟹ b ≤ a △ b"
    by (cases a; cases b) (auto simp: narrow_ik_ge)
  show "b ≤ a ⟹ a △ b ≤ a"
    by (cases a; cases b) (auto simp: narrow_ik_le)
qed

end
```

Concretization

A kind-tagged element denotes the integers of its component that its kind can represent.

```
fun gamma_kd :: "'a::ik_domain kd ⇒ int set" where
  "gamma_kd KBot = {}"
| "gamma_kd (KD k a) = γ a ∩ ik_range k"
| "gamma_kd KTop = UNIV"

instantiation kd :: (ik_domain) numeric_domain
begin

fun is_empty_kd :: "'a kd ⇒ bool" where
  "is_empty_kd KBot ⟷ True"
| "is_empty_kd (KD k a) ⟷ empty_ik k a"
| "is_empty_kd KTop ⟷ False"

fun to_string_kd :: "'a kd ⇒ String.literal" where
  "to_string_kd KBot = to_string (bot :: 'a)"
| "to_string_kd (KD k a) = to_string a"
| "to_string_kd KTop = sym_top"

definition gamma_kd_def [simp]: "numeric_domain_class.gamma (a :: 'a kd) = gamma_kd a"

instance proof
  fix a b :: "'a kd"
  show "γ (bot :: 'a kd) = {}" by (simp add: bot_kd_def)
```

```
  show "γ (top :: 'a kd) = UNIV" by (simp add: top_kd_def)
  show "a ≤ b ⟹ γ a ⊆ γ b"
    by (cases a; cases b) (auto dest: gamma_mono)
  show "is_empty a ⟷ γ a = {}"
    by (cases a) (simp_all add: empty_ik_correct)
qed

end

end
```

2.3 Reachability_Lift

```
theory Reachability_Lift
  imports State_Concretization "HOL-Library.Monad_Syntax"
begin
```

Explicit reachability over an abstract carrier

The outer `Bot` represents an unreachable control-flow point; `Lifted a` represents a reachable point carrying `a`. The construction provides the order, joins, solver updates, concretization, and normalized transfer combinators independently of any particular abstract-state representation.

Mirrors Goblint's lifted-domain construction (https://github.com/goblint/analyzer/blob/master/src/framework/analyses.ml): `module Dom (LD: Lattice.S) = struct include Lattice.LiftConf (...) (LD) ... end`, whose outer `'Bot` means dead code and whose `unlift` only accepts `'Lifted x`. Reachability is carried as an explicit tag on top of the analysis-local state, not rediscovered from the state's own contents. `Bot` mirrors Goblint's lifted `'Bot`; `Lifted` mirrors `'Lifted`. Goblint's lift also carries an outer `'Top`, but not merely as a substitute for a missing payload `top`: `Lattice.LiftConf`'s `join`/`widen` fall back to `'Top` whenever the base domain's own `join`/`widen` raises `TopValue` or `Uncomparable`, i.e. whenever the base domain is partial enough that two of its values have no expressible join, and `unlift` rejects `'Top` exactly like `'Bot`. Every `numeric_domain` instance here is instead a total `bounded_semilattice_sup_bot` with its own `top`: $\sqcup$ and ∇ always return a value, so a lifted operation here can never fail the way Goblint's can, and `Lifted top` already says everything an outer `'Top` would. This lift therefore stays two-valued. At the raw lifted representation, `Bot` is strictly below every `Lifted a` regardless of `a` – including `Lifted bot`, which remains distinct from `Bot` even though `a` is itself locally bottom – since dead-by-construction and reachable-but-locally-bottom are the two concepts this lift exists to keep separate. The normalization discipline developed below (`normalize_lift`, `normalized_lift`) then layers a stronger, Voblint-specific invariant on top: every solver-facing payload an arbitrary caller-supplied predicate `empty_pred` classifies as bottom is additionally canonicalized to structural `Bot`, so `Lifted a` with an `empty_pred`-bottom `a` never actually escapes a value-producing step in practice, even though the raw datatype and order permit it. This theory does not know or assume that `empty_pred` means semantic emptiness; a carrier that instantiates it proves that coincidence itself.

The lifted order and lattice structure

```
datatype (plugins del: quickcheck_narrowing) 'a lifted =
  Bot | Lifted 'a
```

The vendored TD solver stores values at sort `bounded_semilattice_sup_bot` (`TD_side`'s `T :: ('x,'g,'d::bounded_semilattice_s eqsT`), so `lifted` instantiates that combined structure directly in one step – order, bottom, and join together – rather than building it up through the three separate parent classes (`order`, `bot`, `semilattice_sup`) and bridging them with intermediate `instance` declarations. This intentionally gives up the weaker standalone arities that instantiating each parent separately would have provided (`'a::type ⇒ 'a lifted :: bot` and `'a::order ⇒ 'a lifted :: order`), for an arbitrary or merely ordered payload; consequently the generic lifted lemmas below (`map_lift_mono`, `normalize_lift_mono`, `transfer_lift_mono`, `transfer_lift2_mono`) and the standalone widening/narrowing instances are stated at `semilattice_sup` as well. A caller that only needs the weak standalone `bot` arity does not automatically satisfy this, and has to restate at the sort its real call sites already require.

```
instantiation lifted :: (semilattice_sup) bounded_semilattice_sup_bot
begin

fun less_eq_lifted :: "'a lifted ⇒ 'a lifted ⇒ bool" where
    "less_eq_lifted Bot _ = True"
  | "less_eq_lifted (Lifted _) Bot = False"
  | "less_eq_lifted (Lifted a) (Lifted b) = (a ≤ b)"

definition less_lifted :: "'a lifted ⇒ 'a lifted ⇒ bool" where
  "less_lifted x y = (x ≤ y ∧ ¬ y ≤ x)"

definition bot_lifted :: "'a lifted" where "bot_lifted = Bot"

fun sup_lifted :: "'a lifted ⇒ 'a lifted ⇒ 'a lifted" where
    "sup_lifted Bot y = y"
  | "sup_lifted x Bot = x"
  | "sup_lifted (Lifted a) (Lifted b) = Lifted (a ⊔ b)"

instance
proof intro_classes
  fix x y z :: "'a lifted"
  show "(x < y) = (x ≤ y ∧ ¬ y ≤ x)" unfolding less_lifted_def by (rule refl)
```

```
  show "x ≤ x" by (cases x) simp_all
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z" by (cases x; cases y; cases z) simp_all
  show "x ≤ y ⟹ y ≤ x ⟹ x = y" by (cases x; cases y) simp_all
  show "x ≤ x ⊔ y" by (cases x; cases y) simp_all
  show "y ≤ x ⊔ y" by (cases x; cases y) simp_all
  show "y ≤ x ⟹ z ≤ x ⟹ y ⊔ z ≤ x" by (cases x; cases y; cases z) simp_all
  show "bot ≤ x" by (cases x) (simp_all add: bot_lifted_def)
qed

end

lemma bot_lifted_eq [simp]: "(bot :: 'a::semilattice_sup lifted) = Bot"
  unfolding bot_lifted_def by (rule refl)
```

The warrowing update rule needs `'a lifted` itself to carry `widen`/`narrow` once the executable trees it runs store a lifted payload. `Bot` keeps the same role here as in the `sup` instance above: a structural placeholder, neutral for `widen`/`narrow` against a `Lifted` operand, with only the `Lifted`/`Lifted` case deferring to the base domain's own operation. `quickcheck_narrowing` is disabled on the `lifted` datatype declaration above: Isabelle/HOL's own built-in `narrowing` class (for Quickcheck's narrowing-based counterexample search) collides with the vendored solver's `narrowing` class of the same base name, and the datatype package's default Quickcheck plugin registers an arity for `lifted` under that colliding name before this instantiation gets a chance to.

```
instantiation lifted :: ("{semilattice_sup,widening}") widening
begin

fun widen_lifted :: "'a lifted ⇒ 'a lifted ⇒ 'a lifted" where
  "widen_lifted Bot y = y"
| "widen_lifted x Bot = x"
| "widen_lifted (Lifted a) (Lifted b) = Lifted (a ▽ b)"

instance proof intro_classes
  fix x y :: "'a lifted"
  show "x ≤ x ▽ y" by (cases x; cases y) (simp_all add: widen_ge1)
  show "y ≤ x ▽ y" by (cases x; cases y) (simp_all add: widen_ge2)
qed

end

instantiation lifted :: ("{semilattice_sup,narrowing}") narrowing
begin

fun narrow_lifted :: "'a lifted ⇒ 'a lifted ⇒ 'a lifted" where
  "narrow_lifted Bot y = Bot"
| "narrow_lifted (Lifted a) Bot = Bot"
| "narrow_lifted (Lifted a) (Lifted b) = Lifted (a △ b)"

instance proof intro_classes
  fix a b :: "'a lifted"
  show "b ≤ a ⟹ b ≤ a △ b"
    by (cases a; cases b) (simp_all add: narrow_ge)
  show "b ≤ a ⟹ a △ b ≤ a"
    by (cases a; cases b) (simp_all add: narrow_le)
qed

end
```

Generic concretization

Reading a lifted value back: `Bot` denotes no state at all, so it concretizes to the empty set rather than to whatever the underlying domain's own bottom denotes. That difference is the whole point of the lift – an unreachable point and a point whose every coordinate is bottom are distinguishable here, and only the former is empty by construction.

```
definition gamma_lift :: "('a ⇒ 'b set) ⇒ 'a lifted ⇒ 'b set" where
  "gamma_lift gam x = (case x of Bot ⇒ {} | Lifted a ⇒ gam a)"

lemma gamma_lift_Bot [simp]: "gamma_lift gam Bot = {}"
  unfolding gamma_lift_def by simp

lemma gamma_lift_Lifted [simp]: "gamma_lift gam (Lifted a) = gam a"
  unfolding gamma_lift_def by simp
```

`gamma_liftE` recovers the `Lifted` payload a lifted membership witnesses: `Bot` concretizes to `{}`, so any `s ∈ gamma_lift gam x` already forces `x = Lifted a` for some `a` with `s ∈ gam a`. Named as an eliminator because

its witness is exactly this structural case split, replacing the repeated manual `obtain ... where "... = Lifted ..." "s ∈ ..." by (cases ...) auto` pattern with one rule application.

```
lemma gamma_liftE [elim]:
  assumes "s ∈ gamma_lift gam x"
  obtains a where "x = Lifted a" and "s ∈ gam a"
  using assms by (cases x) auto
```

Generic reachability combinators

`bind_lift` is the reachability monad's Kleisli bind, exactly `(≫=)`'s shape with `Bot` for `None` and `Lifted` for `Some`: `Bot` in, `Bot` out, without ever calling `f` – matching Goblint's `unlift` being called before dispatch rather than inside each analysis's `assign`/`branch`. `map_lift` is its functor counterpart (`Option.map`'s analogue), for pushing a total, never-re-collapsing operation through the lift. Both are representation-independent: neither one inspects whether a `Lifted` payload happens to be semantically empty. That is a separate concern, taken up below in *Canonical-bottom normalization*.

```
fun bind_lift :: "'a lifted ⇒ ('a ⇒ 'b lifted) ⇒ 'b lifted" where
  "bind_lift Bot f = Bot"
| "bind_lift (Lifted a) f = f a"

lemma bind_lift_right_identity [simp]: "bind_lift x Lifted = x"
  by (cases x) simp_all
```

`gamma_lift_bindI` follows `bind_lift`'s own structure: a witness for `xs` together with a step that carries any such witness through `f` gives a witness for the bind. Targets the sequential `And`/`Or` recursions that chain one filtering step's result into the next, replacing their manual `obtain d' where "... = Lifted d'" "s ∈ ..."` destructuring with one `cases`-driven rule.

```
lemma gamma_lift_bindI [intro]:
  assumes xs: "s ∈ gamma_lift gam xs"
      and step: "⋀a. s ∈ gam a ⟹ s ∈ gamma_lift gam' (f a)"
  shows "s ∈ gamma_lift gam' (bind_lift xs f)"
  using xs by (cases xs) (auto intro: step)
```

Registers `bind_lift` under `(≫=)`'s ad hoc overloading, so `x >>= f` and `do { a <- x; f a }` both parse to exactly `bind_lift x f` – a pure syntax translation resolved at elaboration time, so every lemma about `bind_lift` applies unchanged to code written with `do` notation. Registered here, immediately after `bind_lift`'s definition, so the reachability combinators below can use `do` notation too.

```
adhoc_overloading Monad_Syntax.bind == bind_lift

definition map_lift :: "('a ⇒ 'b) ⇒ 'a lifted ⇒ 'b lifted" where
  "map_lift f x = do { a <- x; Lifted (f a) }"

lemma map_lift_Bot [simp]: "map_lift f Bot = Bot"
  unfolding map_lift_def by simp

lemma map_lift_Lifted [simp]: "map_lift f (Lifted a) = Lifted (f a)"
  unfolding map_lift_def by simp
```

`map_lift` never invents or destroys the dead point, so the two carriers agree on the coarsest bottom test there is.

```
lemma map_lift_eq_Bot_iff [simp]: "map_lift f x = Bot ⟷ x = Bot"
  by (cases x) simp_all

lemma map_lift_mono:
  fixes x y :: "'a::semilattice_sup lifted"
  assumes f_mono: "⋀a b. a ≤ b ⟹ f a ≤ (f b :: 'b::semilattice_sup)"
    and le: "x ≤ y"
  shows "map_lift f x ≤ map_lift f y"
  using le by (cases x; cases y) (auto simp: f_mono)

lemma map_lift_id [simp]: "map_lift id x = x"
  by (cases x) simp_all

lemma map_lift_comp [simp]: "map_lift g (map_lift f x) = map_lift (g ∘ f) x"
  by (cases x) simp_all
```

Collapsing back to the plain, bot-carrying type

`collapse_lift` goes the opposite direction from `normalize_lift`: where `normalize_lift` turns a domain-produced witness-bottom value into structural `Bot`, `collapse_lift` turns `Bot` back into the plain type's own `bot`, so a caller

whose interface only has room for the plain type – not the `lifted` wrapper – can still receive a more precise lifted result. Unlike `normalize_lift`, no predicate is needed: `Bot` already says the value is empty, and `bot` is exactly the plain type's own representative of that.

```
fun collapse_lift :: "'a::bot lifted ⇒ 'a" where
    "collapse_lift Bot = bot"
  | "collapse_lift (Lifted a) = a"
```

Canonical-bottom normalization

`normalize_lift` is the one Voblint-specific piece on top of the generic combinators above: it turns a domain-produced witness-bottom raw result back into the canonical structural `Bot`, the counterpart of Goblint's transfer functions raising `Deadcode` when their own computation lands on bottom. `transfer_lift`/`transfer_lift2` compose `bind_lift` with `normalize_lift`: the shape every generic edge/combine dispatcher actually calls.

```
definition normalize_lift :: "('a ⇒ bool) ⇒ 'a ⇒ 'a lifted" where
  "normalize_lift empty_pred a = (if empty_pred a then Bot else Lifted a)"

lemma normalize_lift_bot [simp]: "empty_pred a ⟹ normalize_lift empty_pred a = Bot"
  unfolding normalize_lift_def by simp

lemma normalize_lift_not_bot [simp]:
  "¬ empty_pred a ⟹ normalize_lift empty_pred a = Lifted a"
  unfolding normalize_lift_def by simp

definition transfer_lift ::
  "('b ⇒ bool) ⇒ ('a ⇒ 'b) ⇒ 'a lifted ⇒ 'b lifted" where
  "transfer_lift empty_pred f x = do { a <- x; normalize_lift empty_pred (f a) }"

definition transfer_lift2 ::
  "('c ⇒ bool) ⇒ ('a ⇒ 'b ⇒ 'c) ⇒ 'a lifted ⇒ 'b lifted ⇒ 'c lifted" where
  "transfer_lift2 empty_pred f x y =
     do { a <- x; b <- y; normalize_lift empty_pred (f a b) }"

lemma transfer_lift_Bot [simp]: "transfer_lift empty_pred f Bot = Bot"
  unfolding transfer_lift_def by simp

lemma transfer_lift_Lifted [simp]:
  "transfer_lift empty_pred f (Lifted a) = normalize_lift empty_pred (f a)"
  unfolding transfer_lift_def by simp

lemma transfer_lift2_Bot_left [simp]: "transfer_lift2 empty_pred f Bot y = Bot"
  unfolding transfer_lift2_def by simp

lemma transfer_lift2_Bot_right [simp]: "transfer_lift2 empty_pred f (Lifted a) Bot = Bot"
  unfolding transfer_lift2_def by simp

lemma transfer_lift2_Lifted [simp]:
  "transfer_lift2 empty_pred f (Lifted a) (Lifted b) = normalize_lift empty_pred (f a b)"
  unfolding transfer_lift2_def by simp
```

`normalized_lift` names the discipline every solver-facing lifted value is meant to keep: `Bot` is the *sole* representation this construction lets a caller-supplied `empty_pred` classification collapse to – a `Lifted` payload `empty_pred` still calls bottom is a representation the framework never lets escape a value-producing step, not a second, competing way to say the same thing. `normalize_lift`'s own `if` makes every one of its outputs self-normalized: the `Lifted` branch only fires when `empty_pred` already said no. `transfer_lift`/`transfer_lift2` route every domain transfer step through exactly this `if`, so no transfer step can ever hand back a `Lifted` payload `empty_pred` still calls bottom, regardless of what the pre-lift inputs looked like.

The lemmas below extend this from a single transfer step to every other value-producing primitive a solver combines lifted values with: $\sqcup$ (joining several contributions at a program point) and $\nabla\Delta$ (the warrowing update rule). Both need only that `empty_pred` itself is downward closed under the payload's order (`mono` below) – exactly `is_empty_antimono`'s shape at `numeric_domain` – not any fact specific to how `empty_pred` itself is computed.

```
definition normalized_lift :: "('a ⇒ bool) ⇒ 'a lifted ⇒ bool" where
  "normalized_lift empty_pred x = (case x of Bot ⇒ True | Lifted a ⇒ ¬ empty_pred a)"

lemma normalized_lift_Bot [simp]: "normalized_lift empty_pred Bot"
  unfolding normalized_lift_def by simp

lemma normalized_lift_Lifted [simp]:
  "normalized_lift empty_pred (Lifted a) ⟷ ¬ empty_pred a"
  unfolding normalized_lift_def by simp
```

```
lemma normalize_lift_normalized [simp]:
  "normalized_lift empty_pred (normalize_lift empty_pred a)"
  unfolding normalize_lift_def by (cases "empty_pred a") simp_all

lemma transfer_lift_normalized [simp]:
  "normalized_lift empty_pred (transfer_lift empty_pred f x)"
  by (cases x) (simp_all add: transfer_lift_def)

lemma transfer_lift2_normalized [simp]:
  "normalized_lift empty_pred (transfer_lift2 empty_pred f x y)"
  by (cases x; cases y) (simp_all add: transfer_lift2_def)
```

`canonicalize_lift` re-normalizes an already-lifted value against `empty_pred`: a `Lifted` payload that has since become witness-bottom (e.g. read from a representation that does not itself carry the `transfer_lift` discipline) collapses to `Bot`, exactly as if it had been produced by a transfer step in the first place. It is `transfer_lift empty_pred id`, not a new bottom test: reusing `transfer_lift` at the identity function is what makes `canonicalize_lift_normalized` immediate from `transfer_lift_normalized` rather than a second proof of the same fact.

```
definition canonicalize_lift :: "('a ⇒ bool) ⇒ 'a lifted ⇒ 'a lifted" where
  "canonicalize_lift empty_pred = transfer_lift empty_pred id"

lemma canonicalize_lift_Bot [simp]: "canonicalize_lift empty_pred Bot = Bot"
  unfolding canonicalize_lift_def by simp

lemma canonicalize_lift_Lifted [simp]:
  "canonicalize_lift empty_pred (Lifted a) = normalize_lift empty_pred a"
  unfolding canonicalize_lift_def by simp

lemma canonicalize_lift_normalized [simp]:
  "normalized_lift empty_pred (canonicalize_lift empty_pred x)"
  unfolding canonicalize_lift_def by (rule transfer_lift_normalized)
```

Canonicalization is idempotent: a value already collapsed against `empty_pred` is left unchanged by collapsing it again – the defining property of a normalization operation.

```
lemma canonicalize_lift_idem [simp]:
  "canonicalize_lift empty_pred (canonicalize_lift empty_pred x)
     = canonicalize_lift empty_pred x"
  by (cases x) (auto simp: normalize_lift_def)
```

Generic monotonicity for the reachability dispatch, proved once here instead of at every instantiation (`gabs_state`, `gdefault_st`, ...): whenever the underlying transfer is monotone and the bottom predicate is downward closed (as `is_empty_state` always is, via `is_empty_state_antimono`), `transfer_lift`/`transfer_lift2` are monotone.

```
lemma normalize_lift_mono:
  fixes a b :: "'a::semilattice_sup"
  assumes f_le: "a ≤ b"
    and bot_mono: "empty_pred b ⟹ empty_pred a"
  shows "normalize_lift empty_pred a ≤ normalize_lift empty_pred b"
  unfolding normalize_lift_def using f_le bot_mono by auto

lemma transfer_lift_mono:
  fixes x1 x2 :: "'a::semilattice_sup lifted"
  assumes f_mono: "⋀a b. a ≤ b ⟹ f a ≤ f b"
    and bot_mono: "⋀a b. a ≤ b ⟹ empty_pred b ⟹ empty_pred a"
    and le: "x1 ≤ x2"
  shows "transfer_lift empty_pred f x1
         ≤ (transfer_lift empty_pred f x2 :: 'b::semilattice_sup lifted)"
  using assms by (cases x1; cases x2) (auto intro!:normalize_lift_mono)

lemma transfer_lift2_mono:
  fixes x1 x2 :: "'a::semilattice_sup lifted" and y1 y2 :: "'b::semilattice_sup lifted"
  assumes f_mono: "⋀a1 a2 b1 b2. a1 ≤ a2 ⟹ b1 ≤ b2 ⟹ f a1 b1 ≤ f a2 b2"
    and bot_mono: "⋀a b. a ≤ b ⟹ empty_pred b ⟹ empty_pred a"
    and lex: "x1 ≤ x2" and ley: "y1 ≤ y2"
  shows "transfer_lift2 empty_pred f x1 y1
         ≤ (transfer_lift2 empty_pred f x2 y2 :: 'c::semilattice_sup lifted)"
  using assms by (cases x1; cases x2; cases y1; cases y2)
     (auto intro: normalize_lift_mono f_mono bot_mono)
```

Solver update integration

The solver's update rules need `'d::{bounded_semilattice_sup_bot, warrowing}`. `'a lifted` already carries `widening` and `narrowing` separately; this registers their combination `warrowing`, which Isabelle does not derive from the

two arities by itself.

```
instance lifted :: ("{bounded_semilattice_sup_bot, warrowing}") warrowing ..
```

end

2.4 Three_Valued

```
theory Three_Valued
  imports Main
begin
```

Three-valued Boolean combinators

`bin_log f ann` lifts a Boolean connective `f` to `bool option` with the short-circuit reading `f` already has: one definite operand equal to the annihilator `ann` decides the result regardless of the other side, two definite operands are combined by `f`, and anything else is `None`. Goblint's `id_binary_log` (`base.ml`) has the same shape; `and_opt` and `or_opt` are its two instances.

Both expression evaluation and check classification combine three-valued answers this way, so the combinators sit below either of them.

```
definition bin_log ::
  "(bool ⇒ bool ⇒ bool) ⇒ bool ⇒ bool option ⇒ bool option ⇒ bool option" where
  "bin_log f ann x y =
     (if x = Some ann ∨ y = Some ann then Some ann
      else case (x, y) of (Some a, Some b) ⇒ Some (f a b) | _ ⇒ None)"

abbreviation and_opt :: "bool option ⇒ bool option ⇒ bool option" where
  "and_opt ≡ bin_log (∧) False"

abbreviation or_opt :: "bool option ⇒ bool option ⇒ bool option" where
  "or_opt ≡ bin_log (∨) True"

end
```

2.5 Query_Lift

```
theory Query_Lift
  imports Main
begin

unbundle lattice_syntax
```

Lifting a domain with a fresh bottom and top

Goblint answers `EvalInt` in `Lattice.Lift(IntDomTuple)` (`valueDomainQueries.ml` at `0dc12d355`): the integer domain with a fresh $\bot$ and $\top$ around it. The fresh top lets an analysis that does not understand a query decline it without knowing the value domain's own top, and the fresh bottom marks a query asked of an unreachable state. This is kept apart from the reachability lift of the solver's carrier: the two bottoms mean different things, and only this one has a fresh top.

```
datatype 'a query_lift = QBot | QLifted 'a | QTop

instantiation query_lift :: (ord) ord
begin

fun less_eq_query_lift :: "'a query_lift ⇒ 'a query_lift ⇒ bool" where
  "less_eq_query_lift QBot _ = True"
| "less_eq_query_lift _ QTop = True"
| "less_eq_query_lift (QLifted a) (QLifted b) = (a ≤ b)"
| "less_eq_query_lift _ _ = False"

definition less_query_lift :: "'a query_lift ⇒ 'a query_lift ⇒ bool" where
  "less_query_lift a b ⟷ a ≤ b ∧ ¬ b ≤ a"

instance ..

end

instance query_lift :: (order) order
proof
  fix x y z :: "'a query_lift"
  show "x < y ⟷ x ≤ y ∧ ¬ y ≤ x"
    by (simp add: less_query_lift_def)
  show "x ≤ x"
    by (cases x) simp_all
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z"
    by (cases x; cases y; cases z) auto
  show "x ≤ y ⟹ y ≤ x ⟹ x = y"
    by (cases x; cases y) auto
qed

instantiation query_lift :: (order) order_bot
begin
definition bot_query_lift :: "'a query_lift" where "bot_query_lift = QBot"
instance by standard (simp add: bot_query_lift_def)
end

instantiation query_lift :: (order) order_top
begin
definition top_query_lift :: "'a query_lift" where "top_query_lift = QTop"
instance
proof
  fix x :: "'a query_lift"
  show "x ≤ ⊤"
    by (cases x) (simp_all add: top_query_lift_def)
qed
end
```

`QTop` is the identity of the meet and `QBot` absorbs it; two lifted values meet in the underlying domain.

```
instantiation query_lift :: (inf) inf
begin

fun inf_query_lift :: "'a query_lift ⇒ 'a query_lift ⇒ 'a query_lift" where
  "inf_query_lift QBot _ = QBot"
| "inf_query_lift _ QBot = QBot"
| "inf_query_lift QTop b = b"
| "inf_query_lift a QTop = a"
| "inf_query_lift (QLifted a) (QLifted b) = QLifted (a ⊓ b)"
```

```
instance ..

end

instance query_lift :: (semilattice_inf) semilattice_inf
proof
  fix x y z :: "'a query_lift"
  show "x ⊓ y ≤ x"
    by (cases x; cases y) simp_all
  show "x ⊓ y ≤ y"
    by (cases x; cases y) simp_all
  show "x ≤ y ⟹ x ≤ z ⟹ x ≤ y ⊓ z"
    by (cases x; cases y; cases z) simp_all
qed
```

Concretization

A lifted value denotes what the underlying domain says; the fresh bottom denotes nothing and the fresh top everything. An exact underlying meet stays exact after lifting.

```
fun gamma_query_lift :: "('a ⇒ 'b set) ⇒ 'a query_lift ⇒ 'b set" where
  "gamma_query_lift γ QBot = {}"
| "gamma_query_lift γ (QLifted a) = γ a"
| "gamma_query_lift γ QTop = UNIV"

lemma gamma_query_lift_top [simp]: "gamma_query_lift γ ⊤ = UNIV"
  by (simp add: top_query_lift_def)

lemma gamma_query_lift_bot [simp]: "gamma_query_lift γ ⊥ = {}"
  by (simp add: bot_query_lift_def)

lemma gamma_query_lift_inf:
  assumes "⋀a b. γ (a ⊓ b) = γ a ∩ γ b"
  shows "gamma_query_lift γ (x ⊓ y) = gamma_query_lift γ x ∩ gamma_query_lift γ y"
  using assms by (cases x; cases y) simp_all

end
```

2.6 State_Concretization

```
theory State_Concretization
  imports "Voblint_VIMP.VIMP_Syntax" Abstract_Domain
begin
```

The stores an abstract state describes

Every kind of abstract state, pointwise or relational, denotes a set of stores. γ `d` is that set for every state type: each state theory registers its own concretization under this one constant with `adhoc_overloading`, and the type of `d` selects it.

The class parameter γ gives up its own notation here, so that γ has exactly one production in every theory that sees this one.

```
no_notation gamma ("γ")

consts gamma_S :: "'s ⇒ 'v set" ("γ")

adhoc_overloading gamma_S == gamma
```

An executable state is read back as the abstract state it represents, given the classifier that tells locals from globals. `readback` $\mathcal{G}$ `x` is that reading for every executable representation, registered the same way, so a concretization of an executable state is γ `(readback` $\mathcal{G}$ `x)` whatever its representation. Its notation $\varrho_{\mathcal{G}}$ `x` is opt-in, in the bundle `default_st_syntax`.

```
consts readback :: "('k ⇒ bool) ⇒ 'e ⇒ 'a"

end
```

2.7 Nonrelational_State

```
theory Nonrelational_State
  imports State_Concretization
begin
```

What a store of one abstract value per variable denotes, and when it denotes nothing

A non-relational abstract state, `('k, 'a) gabs_state`, pairs every program variable with one abstract value, independently of every other variable – the "non-relational" in the name. `gamma_state` says what that pairing denotes: exactly the concrete stores where each variable's value lies in its own component's concretization, i.e. the product of the per-component concretizations. Because that product is a conjunction over every variable, one witness-bottom component – one variable whose abstract value denotes no concrete value at all – already makes the whole state denote nothing, regardless of what every other component holds. `is_empty_state` names that witness-bottom condition directly, so a generic transfer dispatcher can detect and canonicalize it without testing `gamma_state` against the empty set at run time.

```
type_synonym ('k, 'a) gabs_state = "'k => 'a"


instance "fun" :: (type, bounded_semilattice_sup_bot) bounded_semilattice_sup_bot ..
```

Pointwise join on abstract states is idempotent because the value-domain semilattice structure lifts pointwise. Finite folds can therefore use the standard idempotent-join laws without a separate state-level assumption.

State concretization

```
definition gamma_state :: "('k, 'a::numeric_domain) gabs_state ⇒ ('k ⇒ int) set" where
  "gamma_state d = {s. ∀x. s x ∈ γ (d x)}"

adhoc_overloading gamma_S == gamma_state

lemma gamma_stateI [intro]:
  "(⋀x. s x ∈ γ (d x)) ⟹ s ∈ γ d"
  unfolding gamma_state_def by simp
```

State concretization laws

Every value-level concretization law, lifted to states. All of them fall out pointwise, which is exactly what makes a nonrelational state cheap: order, bottom and join are read one variable at a time, and no law here needs a relation between variables.

```
lemma gamma_state_mono:
  "sigma1 ≤ sigma2 ⟹ γ sigma1 ⊆ γ sigma2"
  for sigma1 sigma2 :: "('k, 'a::numeric_domain) gabs_state"
  unfolding gamma_state_def le_fun_def
  using gamma_mono by blast

lemma gamma_state_bot [simp]:
  "γ (bot :: ('k, 'a::numeric_domain) gabs_state) = {}"
  unfolding gamma_state_def bot_fun_def using gamma_bot by auto

lemma gamma_state_sup_ub1 [intro]:
  "γ sigma1 ⊆ γ (sigma1 ⊔ sigma2)"
  for sigma1 sigma2 :: "('k, 'a::numeric_domain) gabs_state"
  unfolding gamma_state_def sup_fun_def
  using gamma_sup_ub1 by blast

lemma gamma_state_sup_ub2 [intro]:
  "γ sigma2 ⊆ γ (sigma1 ⊔ sigma2)"
  for sigma1 sigma2 :: "('k, 'a::numeric_domain) gabs_state"
  unfolding gamma_state_def sup_fun_def
  using gamma_sup_ub2 by blast
```

Membership counterparts of `gamma_state_sup_ub1`/`gamma_state_sup_ub2`, for citing at a witness `s` directly (`using subsetD[OF gamma_state_sup_ub1 h]` otherwise repeats the same `subsetD` wrapper at every call site). `[intro]`: their conclusion has the distinctive `s ∈ γ (sigma1 ⊔ sigma2)` shape a join-injection goal actually has, and a witness already in one side's concretization is exactly the natural way to discharge it.

```
lemma gamma_state_supI1 [intro]:
  "s ∈ γ sigma1 ⟹ s ∈ γ (sigma1 ⊔ sigma2)"
```

```
  for sigma1 sigma2 :: "('k, 'a::numeric_domain) gabs_state"
  using gamma_state_sup_ub1 by blast

lemma gamma_state_supI2 [intro]:
  "s ∈ γ sigma2 ⟹ s ∈ γ (sigma1 ⊔ sigma2)"
  for sigma1 sigma2 :: "('k, 'a::numeric_domain) gabs_state"
  using gamma_state_sup_ub2 by blast
```

Left bare rather than `[dest]`: `x` is unconstrained by the premise, so a global destruction rule here would let `auto` pick a schematic variable unrelated to the goal at hand (as with the `Voblint_Solver` session's `env_indep_depsD`). Cite it explicitly with the variable the goal needs.

```
lemma gamma_stateD:
  "s ∈ γ d ⟹ s x ∈ γ (d x)"
  for d :: "('k, 'a::numeric_domain) gabs_state"
  unfolding gamma_state_def by simp
```

Witness-bottom abstract states

Witness-bottom is not "the state has exactly one empty component": any number of a state's components can independently denote the empty set of concrete integers at once, and `is_empty_state` does not distinguish those cases from each other or count them. It only asks whether at least one witness exists (`∃x. is_empty (d x)`), because one witness already suffices – `gamma_state`'s product structure means a single empty component collapses the whole state's concretization to the empty set of stores, so a second, third, or every remaining empty component would only reconfirm what the first already decided.

`is_empty_state` is a logical characterization of that condition, not an executable test: a state is a raw function `'k ⇒ 'a`, and for an infinite variable type `∃x. is_empty (d x)` has no code equation – there is no finite witness search for code generation to compile. An executable carrier supplies its own finite bottom test, which scans only the state's finitely many stored locations.

```
definition is_empty_state :: "('k, 'a::executable_domain) gabs_state ⇒ bool" where
  "is_empty_state d = (∃x. is_empty (d x))"

lemma is_empty_stateI [intro]:
  "is_empty (d x) ⟹ is_empty_state d"
  unfolding is_empty_state_def by (rule exI)

lemma is_empty_stateE [elim]:
  assumes "is_empty_state d"
  obtains x where "is_empty (d x)"
  using assms unfolding is_empty_state_def by blast

lemma is_empty_state_gamma_state_empty:
  assumes "is_empty_state d"
  shows "γ d = {}"
proof -
  obtain x where "is_empty (d x)" using assms by (rule is_empty_stateE)
  then show ?thesis
    using gamma_stateD[of _ d x] unfolding is_empty_correct by fastforce
qed

lemma gamma_state_empty_is_empty_state:
  assumes "γ d = {}"
  shows "is_empty_state d"
proof (rule ccontr)
  assume "¬ is_empty_state d"
  then have nonempty: "⋀x. ∃v. v ∈ γ (d x)"
    unfolding is_empty_state_def using is_empty_correct by blast
  define s where "s = (λx. SOME v. v ∈ γ (d x))"
  have s_prop: "s x ∈ γ (d x)" for x
    unfolding s_def using nonempty[of x] by (rule someI_ex)
  then have "s ∈ γ d"
    unfolding gamma_state_def by simp
  with assms show False by simp
qed

lemma is_empty_state_iff_gamma_state_empty:
  "is_empty_state d ⟷ γ d = {}"
  using is_empty_state_gamma_state_empty gamma_state_empty_is_empty_state by blast

lemma exact_emptiness_is_empty_state:
  "exact_emptiness is_empty_state (gamma_state :: ('k, 'a::numeric_domain) gabs_state ⇒ ('k ⇒ int) set)"
  by (rule exact_emptinessI) (rule is_empty_state_iff_gamma_state_empty)
```

```
lemma is_empty_state_bot [simp]:
  "is_empty_state (bot :: ('k, 'a::numeric_domain) gabs_state)"
  unfolding is_empty_state_def bot_fun_def
  using is_empty_correct gamma_bot by blast

lemma is_empty_state_antimono:
  "d1 ≤ d2 ⟹ is_empty_state d2 ⟹ is_empty_state d1"
  for d1 d2 :: "('k, 'a::numeric_domain) gabs_state"
  unfolding is_empty_state_def le_fun_def using is_empty_antimono by blast

end
```

2.8 Nonrelational_Reachability

theory `Nonrelational_Reachability`
  imports `Nonrelational_State Reachability_Lift`
begin

Reachability for pointwise abstract stores

Pointwise stores and structural reachability are independent domain constructions. This theory composes them: unreachable points denote no concrete store, and reachable payloads use γ. A lifted pointwise state is one more kind of state, so γ `d` covers it too.

adhoc_overloading `gamma_S == "gamma_lift gamma_state"`

end

2.9 Forward_Domain

```
theory Forward_Domain
  imports State_Concretization "Voblint_VIMP.VIMP_Expr"
begin
```

Truth tests on a numeric domain

A truth test answers `Some b` only when every integer the abstract value concretizes to has truthiness `b`; `None` means the value cannot decide.

```
locale sound_truth_test =
  fixes tobool :: "'a::numeric_domain ⇒ bool option"
  assumes tobool_sound:
    "tobool p = Some b ⟹ i ∈ γ p ⟹ truthy i = b"

end
```

2.10 Backward_Domain

```
theory Backward_Domain
  imports Nonrelational_Reachability Forward_Domain
begin
```

Inverse operators for refining against a guard

A forward transfer over-approximates what an assignment does; a backward one narrows values against a condition that is known to hold, which is what a branch on `b` learns about the stores that pass it. The interface a domain supplies for that is a set of inverse operators for arithmetic and comparison (`inv_less`, `inv_plus`, ...), each sound in the sense that every concrete pair the operator admits before the operation is still admitted after narrowing. This abstracts the backward-refinement operations Goblint's `BaseInvariant` implements concretely for its Base analysis; Goblint has no generic module signature this locale is a formalization of.

Backward-analysis locale

A semantic intersection preserves every concrete value shared by both operands and lies below both. It need not be the lattice infimum: a domain may normalize an empty result while its representation order still distinguishes several empty elements. The lower bounds make every refinement step reductive. Every domain's intersection has both, so they form one contract; `mono_intersection` adds monotonicity as a separate strengthening.

```
locale sound_intersection =
  fixes intersect :: "'a::numeric_domain => 'a => 'a"
  assumes intersect_sound[intro]:
    "n ∈ γ a ⟹ n ∈ γ b ⟹ n ∈ γ (intersect a b)"
    and intersect_reductive1[intro]: "intersect a b ≤ a"
    and intersect_reductive2[intro]: "intersect a b ≤ b"
```

Monotone intersection is a separate layer for the same reason as in the forward interfaces: soundness needs only `sound_intersection`, while the solver's least-solution theorem asks `intersect` to preserve the order in both arguments.

```
locale mono_intersection = sound_intersection +
  assumes intersect_mono[intro]:
    "a1 ≤ a2 ⟹ b1 ≤ b2 ⟹ intersect a1 b1 ≤ intersect a2 b2"
```

The inverse operators work on abstract values alone: given the result an operation must have produced, each narrows its operands to values that still admit every concrete pair producing it. `sound_inverse_ops` states them and their soundness with no state in sight, which is all the derived numeric queries (`Backward_Numeric_Queries`) read.

```
locale sound_inverse_ops = sound_intersection intersect
  for intersect :: "'a::numeric_domain => 'a => 'a" +
  fixes
    inv_less  :: "bool => 'a => 'a => 'a * 'a"
    and inv_eq    :: "bool => 'a => 'a => 'a * 'a"
    and inv_plus  :: "'a => 'a => 'a => 'a * 'a"
    and inv_minus :: "'a => 'a => 'a => 'a * 'a"
    and inv_times :: "'a => 'a => 'a => 'a * 'a"
  assumes
      inv_less_sound:
      "n1 ∈ γ a1 ⟹ n2 ∈ γ a2 ⟹ (n1 < n2) = res
       ⟹ n1 ∈ γ (fst (inv_less res a1 a2)) ∧ n2 ∈ γ (snd (inv_less res a1 a2))"
    and inv_eq_sound:
      "n1 ∈ γ a1 ⟹ n2 ∈ γ a2 ⟹ (n1 = n2) = res
       ⟹ n1 ∈ γ (fst (inv_eq res a1 a2)) ∧ n2 ∈ γ (snd (inv_eq res a1 a2))"
    and inv_plus_sound:
      "n1 ∈ γ a1 ⟹ n2 ∈ γ a2 ⟹ n1 + n2 ∈ γ r
       ⟹ n1 ∈ γ (fst (inv_plus r a1 a2)) ∧ n2 ∈ γ (snd (inv_plus r a1 a2))"
    and inv_minus_sound:
      "n1 ∈ γ a1 ⟹ n2 ∈ γ a2 ⟹ n1 - n2 ∈ γ r
       ⟹ n1 ∈ γ (fst (inv_minus r a1 a2)) ∧ n2 ∈ γ (snd (inv_minus r a1 a2))"
    and inv_times_sound:
      "n1 ∈ γ a1 ⟹ n2 ∈ γ a2 ⟹ n1 * n2 ∈ γ r
       ⟹ n1 ∈ γ (fst (inv_times r a1 a2)) ∧ n2 ∈ γ (snd (inv_times r a1 a2))"
begin
```

Projections of the five `inv_*_sound` assumptions above, one membership fact per operand instead of the pair conjunction the raw assumption gives. These, not the raw assumptions, are the `[intro]` rules: their conclusion has a distinctive `fst (inv_* ...)`/`snd (inv_* ...)` head, so a goal of exactly that shape picks the matching rule

directly, whereas the raw conjunction-valued assumption would leave automation to split a conjunction goal it did not ask for.

```
lemmas inv_less_sound_fst [intro] = inv_less_sound[THEN conjunct1]
lemmas inv_less_sound_snd [intro] = inv_less_sound[THEN conjunct2]

lemmas inv_eq_sound_fst [intro] = inv_eq_sound[THEN conjunct1]
lemmas inv_eq_sound_snd [intro] = inv_eq_sound[THEN conjunct2]

lemmas inv_plus_sound_fst [intro] = inv_plus_sound[THEN conjunct1]
lemmas inv_plus_sound_snd [intro] = inv_plus_sound[THEN conjunct2]

lemmas inv_minus_sound_fst [intro] = inv_minus_sound[THEN conjunct1]
lemmas inv_minus_sound_snd [intro] = inv_minus_sound[THEN conjunct2]

lemmas inv_times_sound_fst [intro] = inv_times_sound[THEN conjunct1]
lemmas inv_times_sound_snd [intro] = inv_times_sound[THEN conjunct2]

end
```

Conservative inverse operator

A domain too coarse to invert an operator (e.g. sign, which cannot narrow either operand of a plus/minus/times from its result) instantiates that inverse with this shared no-op: both operands pass through unchanged. The first argument is the known result or truth value, so the same no-op serves the arithmetic inverses and the comparison inverses. Its soundness obligation unfolds to the operands' own membership, so each instance discharges it with `inv_conservative_def`.

```
definition inv_conservative :: "'r => 'a => 'a => 'a * 'a" where
  "inv_conservative r a1 a2 = (a1, a2)"

end
```

2.11 Backward_Domain_Mono

theory Backward_Domain_Mono
  imports Backward_Domain
begin

Monotone inverse operators

A domain states its inverse operators' monotonicity with `le_pair`.

Pairwise order for reductive/monotone inverse operators

A minimal componentwise order on `'a * 'a`, used only to state each inverse operator's monotonicity and reductiveness as a single fact about its returned pair rather than as two separately-assumed projections. Deliberately not `'a * 'a`'s own $\leq$: some already-transitively-imported theory instantiates `('a,'b) prod :: order` here with a non-componentwise (order-with-a-<-disjunct) definition, so relying on the ambient $\leq$ would silently pick up the wrong relation. This local, self-contained $\leq$`_pair` avoids that entirely.

abbreviation `le_pair :: "'a::order × 'a ⇒ 'a × 'a ⇒ bool"` **where**
  `"le_pair p q ≡ fst p ≤ fst q ∧ snd p ≤ snd q"`

end

2.12 Numeric_Queries

```
theory Numeric_Queries
  imports State_Concretization
begin
```

Numeric relational query interface

Entailment/refutation of `<`/`=` over an abstract numeric value is not check-specific: it is the same question a backward/guard domain answers, phrased as a query instead of a narrowing. `less`/`eq` package that answer in Goblint's own shape – `ID.lt`/`ID.eq` – a definite `Some True`, a definite `Some False`, or `None` when neither direction is provable, the same three-way shape `tobool` already gives a single abstract value's truthiness.

`sound_numeric_queries` fixes that shape together with its soundness obligation, as `sound_truth_test` does for `tobool`. Kept in a session with no `exp`/`store` concept at all, so a domain that only has these two operations could still interpret it: an inverse operator refines an abstract value under an assumed truth value; `sound_numeric_queries` instead classifies an already-fixed pair of values as provably related, provably unrelated, or neither.

`numeric_query_judgments`, below, is how every domain here builds such an instance: it packages four yes/no judgments into the two queries and proves the obligation once, so no domain restates that step. Where the four judgments come from is the domain's business. Every `sound_inverse_ops` instance reads them off its own narrowing operators, in the `Backward_Numeric_Queries` theory, and gets an instance for free; a concrete domain may instead supply sharper, hand-tuned judgments when that derivation loses precision it cannot recover, or judgments unrelated to narrowing altogether.

The soundness obligation, stated so that a `Some` answer is exact rather than merely one-sided: every concrete pair drawn from the two values must relate the way the answer says. `None` carries no obligation at all, which is what lets a domain answer it whenever it cannot decide.

```
locale sound_numeric_queries =
  fixes less :: "'a::numeric_domain ⇒ 'a ⇒ bool option"
    and eq :: "'a ⇒ 'a ⇒ bool option"
  assumes less_sound[intro]:
      "less a b = Some r ⟹ i ∈ γ a ⟹ j ∈ γ b ⟹ (i < j) = r"
    and eq_sound[intro]:
      "eq a b = Some r ⟹ i ∈ γ a ⟹ j ∈ γ b ⟹ (i = j) = r"
```

Building an instance from four judgments

A domain rarely decides `less`/`eq` in one step. It decides four separate judgments – every witness pair satisfies `<`, every witness pair refutes it, and the same two for `=` – and the two queries are those four read in order: prefer the affirmative judgment, fall back to the refuting one, answer `None` when neither fires. `numeric_query_judgments` fixes the four and their soundness, and nothing else, so a domain interprets it whatever the judgments were read off: a narrowing operator, a comparison of bounds, a test that one side denotes nothing.

The two queries are named `query_less`/`query_eq` rather than `less`/`eq`, which would share a base name with the order's `(<)` and the vendored solver's `eq`. Interpreting the locale introduces them as ordinary top-level constants through a **defines** clause, and therefore with the code equation a constant living in a still-abstract locale context does not have.

`less_true` and `less_false` (and `eq_true` and `eq_false`) can both hold at once only when an operand denotes nothing: a live witness pair on both sides would have to satisfy `i < j` and ¬ `i < j` simultaneously. So on any pair of feasible operands the `if` choice between `Some True` and `Some False` below is never live.

```
locale numeric_query_judgments =
  fixes less_true :: "'a::numeric_domain ⇒ 'a ⇒ bool"
    and less_false :: "'a ⇒ 'a ⇒ bool"
    and eq_true :: "'a ⇒ 'a ⇒ bool"
    and eq_false :: "'a ⇒ 'a ⇒ bool"
  assumes less_true_sound:
      "less_true a b ⟹ i ∈ γ a ⟹ j ∈ γ b ⟹ i < j"
    and less_false_sound:
      "less_false a b ⟹ i ∈ γ a ⟹ j ∈ γ b ⟹ ¬ i < j"
    and eq_true_sound:
      "eq_true a b ⟹ i ∈ γ a ⟹ j ∈ γ b ⟹ i = j"
    and eq_false_sound:
      "eq_false a b ⟹ i ∈ γ a ⟹ j ∈ γ b ⟹ i ≠ j"
begin

definition query_less :: "'a ⇒ 'a ⇒ bool option" where
  "query_less a b = (if less_true a b then Some True else if less_false a b then Some False else None)"
```

```
definition query_eq :: "'a ⇒ 'a ⇒ bool option" where
  "query_eq a b = (if eq_true a b then Some True else if eq_false a b then Some False else None)"

lemma less_opt_sound:
  assumes "query_less a b = Some r" and "i ∈ γ a" and "j ∈ γ b"
  shows "(i < j) = r"
  using assms less_true_sound less_false_sound unfolding query_less_def by (auto split: if_splits)

lemma eq_opt_sound:
  assumes "query_eq a b = Some r" and "i ∈ γ a" and "j ∈ γ b"
  shows "(i = j) = r"
  using assms eq_true_sound eq_false_sound unfolding query_eq_def by (auto split: if_splits)

end

sublocale numeric_query_judgments ⊆ sound_numeric_queries query_less query_eq
proof
  fix a b r i j
  assume "query_less a b = Some r" and "i ∈ γ a" and "j ∈ γ b"
  then show "(i < j) = r" by (rule less_opt_sound)
next
  fix a b r i j
  assume "query_eq a b = Some r" and "i ∈ γ a" and "j ∈ γ b"
  then show "(i = j) = r" by (rule eq_opt_sound)
qed

end
```

2.13 Backward_Numeric_Queries

```
theory Backward_Numeric_Queries
  imports Backward_Domain Numeric_Queries
begin
```

Queries derived from backward inversion

Any `sound_inverse_ops` instance already proves `inv_less_sound`/ `intersect_sound`, so it answers entailment/refutation of `<`/`=` over two atomic values for free: narrowing under an assumed truth value yields an `is_empty` component only if no represented witness pair can satisfy that assumption. Thus emptiness of the inverse result gives a sound entailment/refutation test, though an imprecise inverse may still answer "unknown" even when the relation is already concrete-semantically fixed. The four judgments need the inverse operators and nothing about states, so they are defined in `sound_inverse_ops`'s own context. Turning the four into the two queries a caller asks for is not backward-specific and is not done here: this file ends by interpreting `numeric_query_judgments` at them, so every `sound_inverse_ops` instance is a numeric-query instance too.

Classification tests `is_empty`, not canonical-`bot` equality: a value can denote `{}` without being the representation's chosen `bot` element (`numeric_domain`'s whole reason for distinguishing the two), so a `= bot` test would underclassify any domain with non-canonical empty representations. `less_true`/`less_false`/`eq_true`/`eq_false` stay implementation judgments, not the public interface `Voblint_Domain.Numeric_Queries` gives: a caller after the generic default wants `less`/`eq`; these four remain named and proved so that a domain can state that its own sharper, hand-tuned query agrees with this generic default.

```
context sound_inverse_ops
begin
```

Comparison judgments

A comparison is decided by narrowing against its own negation: if refining the two values under ¬ `a < b` empties either side, no concrete pair can violate `a < b`, so the judgment holds. Nothing new is assumed of the domain – the answer comes from the narrowing operator it already supplies.

```
definition less_true :: "'a ⇒ 'a ⇒ bool" where
  "less_true a b ⟷ is_empty (fst (inv_less False a b)) ∨ is_empty (snd (inv_less False a b))"

definition less_false :: "'a ⇒ 'a ⇒ bool" where
  "less_false a b ⟷ is_empty (fst (inv_less True a b)) ∨ is_empty (snd (inv_less True a b))"

lemma less_true_sound:
  assumes "less_true a b" and "i ∈ γ a" and "j ∈ γ b"
  shows "i < j"
  using assms inv_less_sound[OF assms(2,3), of False]
  unfolding less_true_def
  by (auto simp: is_empty_correct)

lemma less_false_sound:
  assumes "less_false a b" and "i ∈ γ a" and "j ∈ γ b"
  shows "¬ i < j"
  using assms inv_less_sound[OF assms(2,3), of True]
  unfolding less_false_def
  by (auto simp: is_empty_correct)
```

Equality judgments

Two independent derivations, from different existing capabilities: `eq_true` from `less_false` in both directions (integer trichotomy – if every pair is provably ¬ `i < j` and every pair is provably ¬ `j < i`, every pair is `i = j`); `eq_false` from `intersect` (a common witness would belong to its concretization, so an `is_empty` intersection rules one out).

```
definition eq_true :: "'a ⇒ 'a ⇒ bool" where
  "eq_true a b ⟷ less_false a b ∧ less_false b a"

lemma eq_true_sound:
  assumes "eq_true a b" and "i ∈ γ a" and "j ∈ γ b"
  shows "i = j"
  using assms less_false_sound[of a b i j] less_false_sound[of b a j i]
  unfolding eq_true_def
  by fastforce

definition eq_false :: "'a ⇒ 'a ⇒ bool" where
  "eq_false a b ⟷ is_empty (intersect a b)"
```

```
lemma eq_false_sound:
  assumes "eq_false a b" and "i ∈ γ a" and "j ∈ γ b"
  shows "i ≠ j"
  using assms intersect_sound
  unfolding eq_false_def
  by (fastforce simp: is_empty_correct)

end
```

Packaging the derived judgments

The qualifier is mandatory rather than cosmetic: the packaging locale assumes the same four soundness statements this context proves, under the same four names, so an unqualified registration would try to declare each twice in every **global_interpretation** of `sound_inverse_ops`.

```
sublocale sound_inverse_ops ⊆ queries:
  numeric_query_judgments less_true less_false eq_true eq_false
  by unfold_locales
     (fact less_true_sound, fact less_false_sound, fact eq_true_sound, fact eq_false_sound)

end
```

2.14 Interval_Lattice

```
theory Interval_Lattice
  imports "Voblint_Domain.State_Concretization" "Voblint_VIMP.VIMP_Proc"
begin
```

Interval lattice

An interval `[l, u]` abstracts the integers `{n. l ≤ n ∧ n ≤ u}`. Bounds are extended integers (`MinInf` / `Fin n` / `PlusInf`). Empty (e.g. `[+inf, -inf]`) is `bot`; `[-inf, +inf]` is `top`.

Extended integer bounds

An interval needs endpoints that can say "no lower limit" and "no upper limit", so `eint` adds `MinInf` and `PlusInf` to the integers. Ordering puts `MinInf` below and `PlusInf` above everything, which makes `eint` a linear order; addition and subtraction extend the integer operations, with the mixed-infinity cases (`PlusInf + MinInf` and its mirror) pinned to a fixed choice rather than left undefined, since a total function is what

```
datatype eint = MinInf | Fin (eint_val: int) | PlusInf

fun eint_le :: "eint => eint => bool" where
    "eint_le MinInf  _       = True"
  | "eint_le _       PlusInf = True"
  | "eint_le (Fin n) (Fin m) = (n <= m)"
  | "eint_le _       MinInf  = False"
  | "eint_le PlusInf _       = False"

lemma eint_le_refl [simp]: "eint_le x x"
  by (cases x) simp_all

lemma eint_le_antisym: "eint_le x y ⟹ eint_le y x ⟹ x = y"
  by (cases x; cases y) simp_all

lemma eint_le_trans: "eint_le x y ⟹ eint_le y z ⟹ eint_le x z"
  by (cases x; cases y; cases z) simp_all

lemma eint_le_PlusInf [simp]: "eint_le x PlusInf"
  by (cases x) simp_all

lemma eint_le_MinInf_left [simp]: "eint_le MinInf x"
  by simp

lemma eint_le_linear: "eint_le x y ∨ eint_le y x"
  by (cases x; cases y) auto

instantiation eint :: ord begin
definition less_eq_eint :: "eint => eint => bool" where "less_eq_eint = eint_le"
definition less_eint    :: "eint => eint => bool" where
  "(a :: eint) < b = (eint_le a b ∧ ¬ eint_le b a)"
instance ..
end

declare less_eq_eint_def [simp]

instance eint :: linorder
proof intro_classes
  fix x y z :: eint
  show "(x < y) = (x ≤ y ∧ ¬ y ≤ x)" unfolding less_eint_def less_eq_eint_def by simp
  show "x ≤ x"                          unfolding less_eq_eint_def by (rule eint_le_refl)
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z"    unfolding less_eq_eint_def by (rule eint_le_trans)
  show "x ≤ y ⟹ y ≤ x ⟹ x = y"    unfolding less_eq_eint_def by (rule eint_le_antisym)
  show "x ≤ y ∨ y ≤ x"                  unfolding less_eq_eint_def by (rule eint_le_linear)
qed

instantiation eint :: plus begin
fun plus_eint :: "eint => eint => eint" where
    "plus_eint (Fin n)   (Fin m)   = Fin (n + m)"
  | "plus_eint (Fin _)   MinInf    = MinInf"
  | "plus_eint (Fin _)   PlusInf   = PlusInf"
  | "plus_eint MinInf    MinInf    = MinInf"
  | "plus_eint MinInf    (Fin _)   = MinInf"
  | "plus_eint MinInf    PlusInf   = MinInf"
  | "plus_eint PlusInf   MinInf    = PlusInf"
  | "plus_eint PlusInf   (Fin _)   = PlusInf"
```

```
  | "plus_eint PlusInf   PlusInf   = PlusInf"
instance ..
end

instantiation eint :: minus begin
fun minus_eint :: "eint => eint => eint" where
    "minus_eint (Fin n)   (Fin m)   = Fin (n - m)"
  | "minus_eint (Fin _)   MinInf    = PlusInf"
  | "minus_eint (Fin _)   PlusInf   = MinInf"
  | "minus_eint MinInf    MinInf    = MinInf"
  | "minus_eint MinInf    (Fin _)   = MinInf"
  | "minus_eint MinInf    PlusInf   = MinInf"
  | "minus_eint PlusInf   MinInf    = PlusInf"
  | "minus_eint PlusInf   (Fin _)   = PlusInf"
  | "minus_eint PlusInf   PlusInf   = PlusInf"
instance ..
end

lemma eint_le_PlusInf_iff: "eint_le PlusInf z ⟷ z = PlusInf"
  by (cases z) auto
lemma eint_le_MinInf_iff: "eint_le z MinInf ⟷ z = MinInf"
  by (cases z) auto
```

Carrier and concretization

An interval `Ivl l u` over extended integers denotes the integers between its bounds; a reversed pair denotes the empty set, so the raw carrier has many empty values.

```
datatype ivl = Ivl (ivl_lower: eint) (ivl_upper: eint)   — Ivl l u = [l, u]
```

Named exhaustion, cited instead of `(cases x) auto` at every destructuring site. It stays untagged on purpose: its conclusion matches any term, so the classical reasoner would apply it to every goal.

```
lemma ivl_exhaustE:
  obtains l u where "x = Ivl l u"
  by (cases x) auto

fun gamma_ivl :: "ivl => int set" where
    "gamma_ivl (Ivl l u) = {n. l ≤ Fin n ∧ Fin n ≤ u}"

definition ivl_of_int :: "int ⇒ ivl" where
  [simp]: "ivl_of_int n = Ivl (Fin n) (Fin n)"
```

Order

Interval inclusion on the bounds, with `Ivl PlusInf MinInf` as bottom and `ivl_top` as the unbounded interval.

```
instantiation ivl :: ord begin
definition less_eq_ivl :: "ivl => ivl => bool" where
  "(a::ivl) <= b = (case (a, b) of (Ivl l1 u1, Ivl l2 u2) => l2 ≤ l1 ∧ u1 ≤ u2)"
definition less_ivl :: "ivl => ivl => bool" where
  "(a::ivl) < b = (a <= b ∧ ¬ b <= a)"
instance ..
end

instantiation ivl :: bot begin
definition bot_ivl :: ivl where
  "bot_ivl = Ivl PlusInf MinInf"
instance ..
end

instantiation ivl :: order begin
instance proof intro_classes
  fix x y z :: ivl
  show "(x < y) = (x ≤ y ∧ ¬ y ≤ x)"
    unfolding less_ivl_def by simp
  show "x ≤ x"
    unfolding less_eq_ivl_def by (cases x) simp
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z"
    unfolding less_eq_ivl_def by (cases x; cases y; cases z) (auto intro: eint_le_trans)
  show "x ≤ y ⟹ y ≤ x ⟹ x = y"
    unfolding less_eq_ivl_def by (cases x; cases y) (auto intro: eint_le_antisym)
qed
end

instantiation ivl :: order_bot begin
```

```
instance proof intro_classes
  fix x :: ivl
  show "bot ≤ x"
    unfolding less_eq_ivl_def bot_ivl_def by (cases x) simp
qed
end

definition ivl_top :: ivl where
  "ivl_top = Ivl MinInf PlusInf"

instantiation ivl :: top begin
definition "top_ivl = ivl_top"
instance ..
end

instantiation ivl :: order_top begin
instance proof intro_classes
  fix x :: ivl
  show "x ≤ top"
    unfolding top_ivl_def by (cases x) (simp add: less_eq_ivl_def ivl_top_def)
qed
end

lemma gamma_ivl_bot: "gamma_ivl bot = {}"
  unfolding bot_ivl_def by auto

lemma gamma_ivl_top: "gamma_ivl ivl_top = UNIV"
  unfolding ivl_top_def by auto

lemma gamma_ivl_mono:
  "a ≤ b ⟹ gamma_ivl a ⊆ gamma_ivl b"
proof (cases a; cases b)
  fix l1 u1 l2 u2 :: eint
  assume "a = Ivl l1 u1" "b = Ivl l2 u2" and ab: "a ≤ b"
  from ab have le: "eint_le l2 l1" "eint_le u1 u2"
    unfolding less_eq_ivl_def ‹a = Ivl l1 u1› ‹b = Ivl l2 u2› by simp_all
  show "gamma_ivl a ⊆ gamma_ivl b"
    unfolding ‹a = Ivl l1 u1› ‹b = Ivl l2 u2›
    using le by (auto intro: eint_le_trans)
qed
```

Join

The join takes the smaller lower and the larger upper bound; the lemmas prove it is the least upper bound and instantiate `semilattice_sup`.

```
fun join_ivl :: "ivl => ivl => ivl" where
    "join_ivl (Ivl l1 u1) (Ivl l2 u2) =
       Ivl (if l1 ≤ l2 then l1 else l2)
           (if u2 ≤ u1 then u1 else u2)"

lemma join_ivl_le_ub1: "(a :: ivl) ≤ join_ivl a b"
proof (cases a; cases b)
  fix l1 u1 l2 u2 :: eint
  assume "a = Ivl l1 u1" "b = Ivl l2 u2"
  then show "a ≤ join_ivl a b"
    unfolding less_eq_ivl_def
    using eint_le_linear[of l1 l2] eint_le_linear[of u1 u2]
    by auto
qed

lemma join_ivl_le_ub2: "(b :: ivl) ≤ join_ivl a b"
proof (cases a; cases b)
  fix l1 u1 l2 u2 :: eint
  assume "a = Ivl l1 u1" "b = Ivl l2 u2"
  then show "b ≤ join_ivl a b"
    unfolding less_eq_ivl_def
    using eint_le_linear[of l1 l2] eint_le_linear[of u1 u2]
    by auto
qed

lemma join_ivl_le_least:
  "(a :: ivl) ≤ c ⟹ b ≤ c ⟹ join_ivl a b ≤ c"
proof (cases a; cases b; cases c)
  fix l1 u1 l2 u2 l3 u3 :: eint
  assume "a = Ivl l1 u1" "b = Ivl l2 u2" "c = Ivl l3 u3"
```

```
    and "a ≤ c" "b ≤ c"
  then show "join_ivl a b ≤ c"
    unfolding less_eq_ivl_def by auto
qed

instantiation ivl :: sup begin
definition sup_ivl :: "ivl => ivl => ivl" where
  "sup_ivl = join_ivl"
instance ..
end

instance ivl :: semilattice_sup
proof intro_classes
  fix x y z :: ivl
  show "x ≤ x ⊔ y" unfolding sup_ivl_def by (rule join_ivl_le_ub1)
  show "y ≤ x ⊔ y" unfolding sup_ivl_def by (rule join_ivl_le_ub2)
  show "y ≤ x ⟹ z ≤ x ⟹ y ⊔ z ≤ x"
    unfolding sup_ivl_def by (rule join_ivl_le_least)
qed

instance ivl :: bounded_semilattice_sup_bot ..
```

Meet

Interval intersection. Used by `bfilter_ivl` to refine a variable's interval on a guard (`x < n` narrows the upper bound).

```
instantiation ivl :: inf begin
fun inf_ivl :: "ivl => ivl => ivl" where
    "inf_ivl (Ivl l1 u1) (Ivl l2 u2) =
       Ivl (if l2 ≤ l1 then l1 else l2)
           (if u1 ≤ u2 then u1 else u2)"
instance ..
end

lemma meet_ivl_gamma:
  "n ∈ gamma_ivl a ⟹ n ∈ gamma_ivl b ⟹ n ∈ gamma_ivl (a ⊓ b)"
  by (cases a; cases b; auto split: if_splits intro: eint_le_trans)

lemma gamma_inf_ivl [simp]:
  "gamma_ivl (a ⊓ b) = gamma_ivl a ∩ gamma_ivl b"
  apply (cases a; cases b; simp split: if_splits )
  using eint_le_linear eint_le_trans by blast+

lemma meet_ivl_le_lb1: "(a :: ivl) ⊓ b ≤ a"
proof (cases a; cases b)
  fix l1 u1 l2 u2 :: eint
  assume h: "a = Ivl l1 u1" "b = Ivl l2 u2"
  show "a ⊓ b ≤ a"
    using h eint_le_linear[of l2 l1] eint_le_linear[of u1 u2]
    by (auto simp: less_eq_ivl_def split: if_splits)
qed

lemma meet_ivl_le_lb2: "(a :: ivl) ⊓ b ≤ b"
proof (cases a; cases b)
  fix l1 u1 l2 u2 :: eint
  assume h: "a = Ivl l1 u1" "b = Ivl l2 u2"
  show "a ⊓ b ≤ b"
    using h eint_le_linear[of l1 l2] eint_le_linear[of u1 u2]
    by (auto simp: less_eq_ivl_def split: if_splits)
qed

lemma meet_ivl_greatest: "(a :: ivl) ≤ c ⟹ a ≤ b ⟹ a ≤ c ⊓ b"
proof (cases a; cases b; cases c)
  fix l1 u1 l2 u2 l3 u3 :: eint
  assume h: "a = Ivl l1 u1" "b = Ivl l2 u2" "c = Ivl l3 u3" "a ≤ c" "a ≤ b"
  show "a ≤ c ⊓ b"
    using h by (auto simp: less_eq_ivl_def split: if_splits intro: eint_le_trans)
qed

instance ivl :: semilattice_inf
proof intro_classes
  fix x y z :: ivl
  show "x ⊓ y ≤ x" by (rule meet_ivl_le_lb1)
  show "x ⊓ y ≤ y" by (rule meet_ivl_le_lb2)
  show "x ≤ y ⟹ x ≤ z ⟹ x ≤ y ⊓ z" by (rule meet_ivl_greatest)
```

```
qed

instance ivl :: lattice ..
instance ivl :: bounded_lattice_bot ..
```

Executable interface

Unlike `bot`, `top` has no other representation: `gamma_ivl (Ivl l u) = UNIV` forces `l` $\leq$ `Fin n` and `Fin n` $\leq$ `u` for every `n`, which pins `l = MinInf` and `u = PlusInf` exactly — so a direct equality test against `ivl_top` is already exact, no bound comparison needed.

```
definition is_top_ivl :: "ivl ⇒ bool" where
  "is_top_ivl i = (i = ivl_top)"

lemma is_top_ivl_correct_gamma: "is_top_ivl i ⟷ gamma_ivl i = UNIV"
  by (cases i; case_tac x1; case_tac x2)
     (auto simp add: is_top_ivl_def ivl_top_def set_eq_iff; presburger)+
```

`bot` is one fixed empty interval (`Ivl PlusInf MinInf`), but the $\leq$ instance's bound comparison makes `x` $\leq$ `bot` true only for that exact representation: any other inverted bound pair (e.g. a guard refinement narrowing a variable against its own infeasible range, `Ivl (Fin 5) (Fin (- 1))`) is just as empty by `gamma_ivl`, yet not $\leq$ `bot` — the instance never normalizes. `is_bottom_ivl` tests emptiness directly against the bounds instead of against one particular representative.

Bound comparison alone is not quite enough, though: `Ivl MinInf MinInf` has `l` $\leq$ `u` (reflexively), yet no integer is "at" `MinInf` itself, so its `gamma_ivl` is still empty (symmetrically for `Ivl PlusInf PlusInf`). `l = PlusInf` and `u = MinInf` are exactly the two cases where `eint_le` can hold reflexively without any integer ever satisfying it, so both are called out explicitly alongside the general inverted-bounds case.

```
definition is_bottom_ivl :: "ivl ⇒ bool" where
  "is_bottom_ivl i =
     (case i of Ivl l u ⇒ l = PlusInf ∨ u = MinInf ∨ ¬ l ≤ u)"

lemma is_bottom_ivl_correct: "is_bottom_ivl i ⟷ gamma_ivl i = {}"
proof (cases i)
  case (Ivl l u)
  show ?thesis
    unfolding Ivl is_bottom_ivl_def
    by (cases l; cases u) auto
qed
```

Goblint shows an interval as `[l,u]`, an unconstrained end as the machine integer's limit in decimal; these bounds are unbounded, so it prints as ∞. Every empty representation prints as $\bot$, not only $\bot$. A standalone interval prints its top as $\top$, as Goblint's integer-domain lifter does, while a product component keeps the bounds.

```
fun string_of_eint :: "eint ⇒ String.literal" where
    "string_of_eint MinInf  = STR ''-'' + sym_infinity"
  | "string_of_eint PlusInf = STR ''+'' + sym_infinity"
  | "string_of_eint (Fin n) = string_of_int n"

fun string_of_ivl :: "ivl ⇒ String.literal" where
  "string_of_ivl (Ivl l u) =
     (if is_bottom_ivl (Ivl l u) then sym_bottom
      else STR ''['' + string_of_eint l + STR '','' + string_of_eint u + STR '']'')"
```

Canonical representatives

Canonical empty interval. The raw `ivl` lattice has infinitely many empty representations (any `Ivl l u` with `l > u`, or an infinite bound on the wrong side). `normalize_ivl` collapses every empty interval to the single $\bot$ representative `Ivl PlusInf MinInf`, leaving non-empty intervals untouched.

Arithmetic feeds its operands through `normalize_ivl` first. Since the pointwise sum/difference of $\bot$ with anything is again $\bot$, this keeps every empty result canonical. Without it, `[1,0] + [1,1] = [2,1]` would manufacture ever-new empty representations, which as context keys defeat fixpoint convergence in the recursive interval example.

```
definition normalize_ivl :: "ivl ⇒ ivl" where
  "normalize_ivl v =
     (case v of Ivl l u ⇒
        if l ≤ u ∧ l ≠ PlusInf ∧ u ≠ MinInf then v else bot)"

lemma normalize_ivl_gamma: "gamma_ivl (normalize_ivl v) = gamma_ivl v"
  by (cases v) (auto simp: normalize_ivl_def bot_ivl_def
        split: eint.splits if_splits intro: eint_le_trans)
```

```
lemma normalize_ivl_mono: "x ≤ y ⟹ normalize_ivl x ≤ normalize_ivl y"
  by (cases x; cases y)
     (auto simp: normalize_ivl_def less_eq_ivl_def bot_ivl_def
       eint_le_PlusInf_iff eint_le_MinInf_iff
       split: eint.splits if_splits intro: eint_le_trans)
```

Emptiness test on intervals. The raw `ivl` lattice contains many unnormalised empty intervals (any `Ivl l u` with `l > u`, including ⊥). `ivl_nonempty` captures exactly the intervals with non-empty concretization (proved below: implied by any witness, and monotone along ≤); multiplication uses it for proper bottom handling, which is what makes the precise corner product monotone.

```
fun ivl_nonempty :: "ivl => bool" where
  "ivl_nonempty (Ivl l u) = (l ≤ u ∧ l ≠ PlusInf ∧ u ≠ MinInf)"

lemma gamma_ivl_nonempty: "i ∈ gamma_ivl v ⟹ ivl_nonempty v"
  by (cases v; cases "case v of Ivl l _ => l"; cases "case v of Ivl _ u => u"; auto)
```

`normalize_ivl` picks one representative per abstract value: non-empty intervals are already determined by their bounds, and every empty one collapses to ⊥. This mirrors the component-local normalisation each Goblint integer domain performs for itself (`Interval.norm` sending `l > u` to `None`, `Congruence.normalize` pinning the residue and modulus), which is what makes a structural stability test on a representation meaningful.

`is_bottom_ivl` stays an exact test on arbitrary bounds rather than an equality against ⊥: `ivl` is an ordinary datatype, so `Ivl (Fin 5) (Fin 3)` remains writable by any caller and must still be recognised as empty.

```
lemma is_bottom_ivl_bot [simp]: "is_bottom_ivl bot"
  by (simp add: is_bottom_ivl_def bot_ivl_def)

lemma normalize_ivl_bot_case [simp]:
  "is_bottom_ivl v ⟹ normalize_ivl v = bot"
  by (cases v) (auto simp: normalize_ivl_def is_bottom_ivl_def)

lemma normalize_ivl_id [simp]:
  "¬ is_bottom_ivl v ⟹ normalize_ivl v = v"
  by (cases v) (simp add: normalize_ivl_def is_bottom_ivl_def)

lemma normalize_ivl_bot [simp]: "normalize_ivl bot = bot"
  by simp

lemma normalize_ivl_idem [simp]:
  "normalize_ivl (normalize_ivl v) = normalize_ivl v"
  by (cases "is_bottom_ivl v") simp_all

lemma normalize_ivl_le: "normalize_ivl v ≤ v"
  by (cases "is_bottom_ivl v") simp_all

lemma is_bottom_ivl_normalize [simp]:
  "is_bottom_ivl (normalize_ivl v) = is_bottom_ivl v"
  by (cases "is_bottom_ivl v") simp_all

definition mk_ivl :: "eint ⇒ eint ⇒ ivl" where
  [simp]: "mk_ivl l u = normalize_ivl (Ivl l u)"
```

`mk_ivl` is the canonical constructor: any operation that computes new bounds should build its result through it instead of applying `Ivl` to possibly inverted bounds and leaving the emptiness implicit. Its defining equation is a simp rule, like (⊓)'s above, so a caller that already rewrites with `normalize_ivl`'s equations sees through it rather than meeting an opaque constant.

```
lemma gamma_mk_ivl [simp]: "gamma_ivl (mk_ivl l u) = gamma_ivl (Ivl l u)"
  by (simp add: normalize_ivl_gamma)

lemma normalize_ivl_mk_ivl [simp]: "normalize_ivl (mk_ivl l u) = mk_ivl l u"
  by simp

lemma mk_ivl_mono:
  "Ivl l1 u1 ≤ Ivl l2 u2 ⟹ mk_ivl l1 u1 ≤ mk_ivl l2 u2"
  by (simp add: normalize_ivl_mono)
```

(⊓) is the one operation that is deliberately left un-normalised: it intersects the bounds and lets an infeasible intersection come out as an inverted pair.

```
lemma meet_ivl_not_canonical:
  "normalize_ivl (Ivl (Fin 0) (Fin 3) ⊓ Ivl (Fin 5) (Fin 9))
     ≠ Ivl (Fin 0) (Fin 3) ⊓ Ivl (Fin 5) (Fin 9)"
  by (simp add: normalize_ivl_def bot_ivl_def)
```

Normalising it instead would break `semilattice_inf`: the greatest-lower-bound law `meet_ivl_greatest` quantifies over *all* lower bounds, including the un-normalised empty ones a caller may write down, and those are not below $\bot$. The three facts below are exactly that counterexample – `Ivl (Fin 5) (Fin 3)` is a lower bound of both operands, yet it is not below the normalised meet. Canonicity therefore belongs to the operations that build new values (arithmetic, `mk_ivl`), not to the order-theoretic $(\sqcap)$.

```
lemma meet_ivl_normalized_breaks_greatest:
  "Ivl (Fin 5) (Fin 3) ≤ Ivl (Fin 0) (Fin 3)"
  "Ivl (Fin 5) (Fin 3) ≤ Ivl (Fin 5) (Fin 9)"
  "¬ Ivl (Fin 5) (Fin 3)
       ≤ normalize_ivl (Ivl (Fin 0) (Fin 3) ⊓ Ivl (Fin 5) (Fin 9))"
  by (simp_all add: less_eq_ivl_def normalize_ivl_def bot_ivl_def)
```

Semantic intersection

`intersect_ivl` is the intersection a backward filter uses, distinct from the order's `inf` on this raw carrier.

```
definition intersect_ivl :: "ivl ⇒ ivl ⇒ ivl" where
  [simp]: "intersect_ivl a b = normalize_ivl (a ⊓ b)"
```

$(\sqcap)$ is the greatest lower bound of the representation order; the operation a backward filter actually wants is the intersection of the concretizations, which is a weaker requirement — it need only preserve `gamma_ivl` and be reductive and monotone, and is free to pick any representative of the result. `intersect_ivl` is that operation: it normalizes the bound intersection so an infeasible result becomes $\bot$ rather than a reversed pair.

Goblint's interval `meet` is simultaneously both, because its carrier keeps only canonical inhabited values — `norm` maps `l > u` to `None`, so no element corresponding to `Ivl (Fin 5) (Fin 3)` exists there to be a common lower bound. On this larger raw carrier the two notions come apart, which is what `meet_ivl_normalized_breaks_greatest` records.

```
lemma gamma_intersect_ivl [simp]:
  "gamma_ivl (intersect_ivl a b) = gamma_ivl (a ⊓ b)"
  by (simp add: normalize_ivl_gamma)

lemma intersect_ivl_gamma:
  "n ∈ gamma_ivl a ⟹ n ∈ gamma_ivl b ⟹ n ∈ gamma_ivl (intersect_ivl a b)"
  using meet_ivl_gamma[of n a b] by (simp add: normalize_ivl_gamma)

lemma intersect_ivl_le1: "intersect_ivl a b ≤ a"
proof -
  have "normalize_ivl (a ⊓ b) ≤ a ⊓ b" by (rule normalize_ivl_le)
  also have "a ⊓ b ≤ a" using meet_ivl_le_lb1[of a b] by simp
  finally show ?thesis by simp
qed

lemma intersect_ivl_le2: "intersect_ivl a b ≤ b"
proof -
  have "normalize_ivl (a ⊓ b) ≤ a ⊓ b" by (rule normalize_ivl_le)
  also have "a ⊓ b ≤ b" using meet_ivl_le_lb2[of a b] by simp
  finally show ?thesis by simp
qed

lemma intersect_ivl_mono:
  assumes "a1 ≤ a2" and "b1 ≤ b2"
  shows "intersect_ivl a1 b1 ≤ intersect_ivl a2 b2"
proof -
  have "a1 ⊓ b1 ≤ a2 ⊓ b2" using inf_mono[OF assms] by simp
  then show ?thesis by (simp add: normalize_ivl_mono)
qed

lemma normalize_ivl_intersect_ivl [simp]:
  "normalize_ivl (intersect_ivl a b) = intersect_ivl a b"
  by simp

end
```

2.15 Sign_Lattice

```
theory Sign_Lattice
  imports "Voblint_Domain.State_Concretization"
begin
```

Sign lattice

sign abstracts integers by their sign: Bot – empty (unreachable / undefined) Neg – strictly negative n | n < 0 NonPos – non-positive {n | n ≤ 0} Zero – exactly zero 0 NonNeg – non-negative {n | n ≥ 0} Pos – strictly positive n | n > 0 Top – all integers UNIV

Seven-element lattice (Bot ≤ Neg,Zero,Pos ≤ NonPos,NonNeg ≤ Top). Finite; no widening needed.

Carrier and concretization

```
datatype sign = SBot | SNeg | SNonPos | SZero | SNonNeg | SPos | STop

fun gamma_sign :: "sign => int set" where
    "gamma_sign SBot    = {}"
  | "gamma_sign SNeg    = {n. n < 0}"
  | "gamma_sign SNonPos = {n. n ≤ 0}"
  | "gamma_sign SZero   = {0}"
  | "gamma_sign SNonNeg = {n. n ≥ 0}"
  | "gamma_sign SPos    = {n. n > 0}"
  | "gamma_sign STop    = UNIV"

fun sign_of_int :: "int => sign" where
  "sign_of_int n = (if n < 0 then SNeg else if n = 0 then SZero else SPos)"

lemma sign_of_int_gamma: "n : gamma_sign (sign_of_int n)"
  by (auto split: if_splits)
```

Order

sign_le puts SNeg and SZero below SNonPos, and SZero and SPos below SNonNeg; concretization is monotone in it.

```
fun sign_le :: "sign => sign => bool" where
    "sign_le SBot    _       = True"
  | "sign_le _       STop    = True"
  | "sign_le SNeg    SNeg    = True"
  | "sign_le SNeg    SNonPos = True"
  | "sign_le SNonPos SNonPos = True"
  | "sign_le SZero   SZero   = True"
  | "sign_le SZero   SNonPos = True"
  | "sign_le SZero   SNonNeg = True"
  | "sign_le SNonNeg SNonNeg = True"
  | "sign_le SPos    SPos    = True"
  | "sign_le SPos    SNonNeg = True"
  | "sign_le _       _       = False"

lemma sign_le_refl:    "sign_le s s"                              by (cases s) simp_all

lemma sign_le_antisym:
  assumes st: "sign_le s t" and ts: "sign_le t s"
  shows "s = t"
  using st ts by (cases s; cases t; simp)

lemma sign_le_trans:
  assumes st: "sign_le s t" and tu: "sign_le t u"
  shows "sign_le s u"
  using st tu by (cases s; cases t; cases u; simp)

lemma gamma_sign_mono:
  assumes st: "sign_le s t"
  shows "gamma_sign s <= gamma_sign t"
  using st by (cases s; cases t; auto)

instantiation sign :: ord begin
definition less_eq_sign :: "sign => sign => bool" where "(a::sign) <= b = sign_le a b"
definition less_sign    :: "sign => sign => bool" where
  "(a::sign) <  b = (sign_le a b ∧ ¬ sign_le b a)"
instance ..
end
```

```
instance sign :: preorder
proof intro_classes
  fix x y z :: sign
  show "(x < y) = (x ≤ y ∧ ¬ y ≤ x)"
    unfolding less_sign_def less_eq_sign_def by simp
  show "x ≤ x"
    by (simp add: less_eq_sign_def sign_le_refl)
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z"
    unfolding less_eq_sign_def by (rule sign_le_trans)
qed

instantiation sign :: order begin
instance proof intro_classes
  fix x y :: sign
  assume "x ≤ y" "y ≤ x"
  then show "x = y"
    unfolding less_eq_sign_def by (blast intro: sign_le_antisym)
qed
end
```

The bottom instance lifts pointwise to abstract states and enables monotone least-upper-bound iteration.

```
instantiation sign :: bot begin
definition "bot_sign = SBot"
instance ..
end

instantiation sign :: order_bot begin
instance proof intro_classes
  fix x :: sign
  show "bot ≤ x"
    unfolding less_eq_sign_def bot_sign_def by simp
qed
end

instantiation sign :: top begin
definition "top_sign = STop"
instance ..
end

instantiation sign :: order_top begin
instance proof intro_classes
  fix x :: sign
  show "x ≤ top"
    unfolding less_eq_sign_def top_sign_def by (cases x) simp_all
qed
end

lemma gamma_sign_top: "gamma_sign top = UNIV"
  unfolding top_sign_def by simp
```

Join

`join_sign` returns the least sign covering both arguments, for example `SNeg` and `SZero` join to `SNonPos`.

```
fun join_sign :: "sign => sign => sign" where
    "join_sign SBot    b       = b"
  | "join_sign a       SBot    = a"
  | "join_sign STop    _       = STop"
  | "join_sign _       STop    = STop"
  | "join_sign SNeg    SNeg    = SNeg"
  | "join_sign SNeg    SZero   = SNonPos"
  | "join_sign SNeg    SNonPos = SNonPos"
  | "join_sign SZero   SNeg    = SNonPos"
  | "join_sign SZero   SZero   = SZero"
  | "join_sign SZero   SPos    = SNonNeg"
  | "join_sign SZero   SNonPos = SNonPos"
  | "join_sign SZero   SNonNeg = SNonNeg"
  | "join_sign SNonPos SNeg    = SNonPos"
  | "join_sign SNonPos SZero   = SNonPos"
  | "join_sign SNonPos SNonPos = SNonPos"
  | "join_sign SNonNeg SZero   = SNonNeg"
  | "join_sign SNonNeg SPos    = SNonNeg"
  | "join_sign SNonNeg SNonNeg = SNonNeg"
  | "join_sign SPos    SZero   = SNonNeg"
  | "join_sign SPos    SNonNeg = SNonNeg"
```

```
  | "join_sign SPos    SPos    = SPos"
  | "join_sign _       _       = STop"

lemma join_sign_ub1: "sign_le a (join_sign a b)"
  by (cases a; cases b; simp)

lemma join_sign_ub2: "sign_le b (join_sign a b)"
  by (cases a; cases b; simp)

lemma join_sign_least: "sign_le a csg ⟹ sign_le b csg ⟹ sign_le (join_sign a b) csg"
  by (cases a; cases b; cases csg; simp)

instantiation sign :: sup begin
definition sup_sign :: "sign => sign => sign" where
  "sup_sign = join_sign"
instance ..
end

instance sign :: semilattice_sup
proof intro_classes
  fix x y z :: sign
  show "x ≤ x ⊔ y"
    unfolding sup_sign_def less_eq_sign_def by (rule join_sign_ub1)
  show "y ≤ x ⊔ y"
    unfolding sup_sign_def less_eq_sign_def by (rule join_sign_ub2)
  show "y ≤ x ⟹ z ≤ x ⟹ y ⊔ z ≤ x"
    unfolding sup_sign_def less_eq_sign_def by (rule join_sign_least)
qed
```

`sign` is already an `order_bot` and a `semilattice_sup`, so the bounded semilattice comes for free.

```
instance sign :: bounded_semilattice_sup_bot ..
```

Meet

The sign meet is exact: it concretizes to the intersection of its operands. As a `semilattice_inf` instance it gives `inf_mono` for free.

```
instantiation sign :: inf begin

fun inf_sign :: "sign => sign => sign" where
    "inf_sign SBot    _       = SBot"
  | "inf_sign _       SBot    = SBot"
  | "inf_sign STop    b       = b"
  | "inf_sign a       STop    = a"
  | "inf_sign SNeg    SNeg    = SNeg"
  | "inf_sign SNeg    SNonPos = SNeg"
  | "inf_sign SNonPos SNeg    = SNeg"
  | "inf_sign SNonPos SNonPos = SNonPos"
  | "inf_sign SNonPos SZero   = SZero"
  | "inf_sign SZero   SNonPos = SZero"
  | "inf_sign SNonPos SNonNeg = SZero"
  | "inf_sign SNonNeg SNonPos = SZero"
  | "inf_sign SZero   SZero   = SZero"
  | "inf_sign SZero   SNonNeg = SZero"
  | "inf_sign SNonNeg SZero   = SZero"
  | "inf_sign SNonNeg SNonNeg = SNonNeg"
  | "inf_sign SNonNeg SPos    = SPos"
  | "inf_sign SPos    SNonNeg = SPos"
  | "inf_sign SPos    SPos    = SPos"
  | "inf_sign _       _       = SBot"

instance ..

end

lemma gamma_inf_sign [simp]:
  "gamma_sign (a ⊓ b) = gamma_sign a ∩ gamma_sign b"
  by (cases a; cases b) auto

instance sign :: semilattice_inf
proof intro_classes
  fix x y z :: sign
  show "x ⊓ y ≤ x"
    by (cases x; cases y; auto simp: less_eq_sign_def)
  show "x ⊓ y ≤ y"
    by (cases x; cases y; auto simp: less_eq_sign_def)
```

```
  show "x ≤ y ⟹ x ≤ z ⟹ x ≤ y ⊓ z"
    by (cases x; cases y; cases z; auto simp: less_eq_sign_def)
qed

instance sign :: lattice ..
instance sign :: bounded_lattice_bot ..
```

Executable interface

`SBot` is the only empty value a finite enumerated domain can have (every other constructor denotes a nonempty set of integers), so a direct equality test is already exact — unlike Interval's analogous fact, which cannot use equality against one representative because Interval's bound-pair representation has many empty values besides its canonical `bot`. Exposed here, at the domain's own theory, rather than inlined where a caller happens to need it, so every consumer (not just one) gets the same domain-owned fact.

```
definition is_bottom_sign :: "sign ⇒ bool" where
  "is_bottom_sign s = (s = SBot)"

lemma is_bottom_sign_correct: "is_bottom_sign s ⟷ gamma_sign s = {}"
  unfolding is_bottom_sign_def
  by (cases s) (auto intro: exI[of _ "-1"] exI[of _ "0"] exI[of _ "1"])
```

`STop` is likewise the unique top of a finite enumeration.

```
definition is_top_sign :: "sign ⇒ bool" where
  "is_top_sign s = (s = STop)"

lemma is_top_sign_correct_gamma: "is_top_sign s ⟷ gamma_sign s = UNIV"
  by(cases s) (auto simp: is_top_sign_def set_eq_iff; presburger)+
```

Goblint has no sign value domain; its tutorial sign analysis prints `-`, `0` and `+`, which the two non-strict elements extend as $\leq$`0` and $\geq$`0`.

```
fun string_of_sign :: "sign ⇒ String.literal" where
    "string_of_sign SBot    = sym_bottom"
  | "string_of_sign SNeg    = STR ''-''"
  | "string_of_sign SNonPos = sym_le + STR ''0''"
  | "string_of_sign SZero   = STR ''0''"
  | "string_of_sign SNonNeg = sym_ge + STR ''0''"
  | "string_of_sign SPos    = STR ''+''"
  | "string_of_sign STop    = sym_top"

end
```

2.16 Parity_Lattice

```
theory Parity_Lattice
  imports "Voblint_Domain.State_Concretization"
begin
```

Parity lattice

parity abstracts integers by their parity: Bot – empty (unreachable) Even – n | even n Odd – n | odd n Top – all integers

Four-element lattice (`Bot` $\leq$ `Even,Odd` $\leq$ `Top`). Finite; widen = sup. This analysis does not implement backward guard refinement. Its branch transfer therefore preserves parity information from the incoming state.

Carrier and concretization

```
datatype parity = PBot | PEven | POdd | PTop

fun gamma_parity :: "parity => int set" where
    "gamma_parity PBot  = {}"
  | "gamma_parity PEven = {n. even n}"
  | "gamma_parity POdd  = {n. odd n}"
  | "gamma_parity PTop  = UNIV"

fun parity_of_int :: "int => parity" where
  "parity_of_int n = (if even n then PEven else POdd)"

lemma parity_of_int_gamma: "n ∈ gamma_parity (parity_of_int n)" by auto
```

Order

The flat order with `PBot` below and `PTop` above the two parities; `gamma_parity_mono` shows concretization respects it.

```
fun parity_le :: "parity => parity => bool" where
    "parity_le PBot  _     = True"
  | "parity_le _     PTop  = True"
  | "parity_le PEven PEven = True"
  | "parity_le POdd  POdd  = True"
  | "parity_le _     _     = False"

lemma parity_le_refl: "parity_le s s" by (cases s) simp_all

lemma parity_le_antisym: "parity_le s t ⟹ parity_le t s ⟹ s = t"
  by (cases s; cases t; simp)

lemma parity_le_trans: "parity_le s t ⟹ parity_le t u ⟹ parity_le s u"
  by (cases s; cases t; cases u; simp)

lemma gamma_parity_mono: "parity_le s t ⟹ gamma_parity s ⊆ gamma_parity t"
  by (cases s; cases t; auto)

instantiation parity :: ord begin
definition less_eq_parity :: "parity => parity => bool" where "(a::parity) ≤ b = parity_le a b"
definition less_parity    :: "parity => parity => bool" where
  "(a::parity) <  b = (parity_le a b ∧ ¬ parity_le b a)"
instance ..
end

instance parity :: preorder
proof
  fix x y z :: parity
  show "(x < y) = (x ≤ y ∧ ¬ y ≤ x)" unfolding less_parity_def less_eq_parity_def by simp
  show "x ≤ x" by (simp add: less_eq_parity_def parity_le_refl)
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z" unfolding less_eq_parity_def by (rule parity_le_trans)
qed

instantiation parity :: order begin
instance proof
  fix x y :: parity
  assume "x ≤ y" "y ≤ x"
  then show "x = y" unfolding less_eq_parity_def by (blast intro: parity_le_antisym)
qed
end
```

```
instantiation parity :: bot begin
definition "bot_parity = PBot"
instance ..
end

instantiation parity :: order_bot begin
instance proof
  fix x :: parity
  show "bot ≤ x" unfolding less_eq_parity_def bot_parity_def by simp
qed
end

instantiation parity :: top begin
definition "top_parity = PTop"
instance ..
end

instantiation parity :: order_top begin
instance proof intro_classes
  fix x :: parity
  show "x ≤ top"
    unfolding less_eq_parity_def top_parity_def by (cases x) simp_all
qed
end

lemma gamma_parity_top: "gamma_parity top = UNIV"
  unfolding top_parity_def by simp
```

Join

Joining two different parities loses all information and yields `PTop`. The lemmas prove `join_parity` is the least upper bound.

```
fun join_parity :: "parity => parity => parity" where
    "join_parity PBot  b     = b"
  | "join_parity a     PBot  = a"
  | "join_parity PTop  _     = PTop"
  | "join_parity _     PTop  = PTop"
  | "join_parity PEven PEven = PEven"
  | "join_parity POdd  POdd  = POdd"
  | "join_parity _     _     = PTop"

lemma join_parity_ub1: "parity_le a (join_parity a b)" by (cases a; cases b; simp)
lemma join_parity_ub2: "parity_le b (join_parity a b)" by (cases a; cases b; simp)
lemma join_parity_least: "parity_le a c ⟹ parity_le b c ⟹ parity_le (join_parity a b) c"
  by (cases a; cases b; cases c; simp)

instantiation parity :: sup begin
definition sup_parity :: "parity => parity => parity" where "sup_parity = join_parity"
instance ..
end

instance parity :: semilattice_sup
proof
  fix x y z :: parity
  show "x ≤ x ⊔ y" unfolding sup_parity_def less_eq_parity_def by (rule join_parity_ub1)
  show "y ≤ x ⊔ y" unfolding sup_parity_def less_eq_parity_def by (rule join_parity_ub2)
  show "y ≤ x ⟹ z ≤ x ⟹ y ⊔ z ≤ x"
    unfolding sup_parity_def less_eq_parity_def by (rule join_parity_least)
qed

instance parity :: bounded_semilattice_sup_bot ..
```

Meet

Parity is closed under intersection: the meet of two classes is their common class, or bottom when one is even and the other odd.

```
instantiation parity :: inf begin

fun inf_parity :: "parity => parity => parity" where
    "inf_parity PTop b    = b"
  | "inf_parity a    PTop = a"
  | "inf_parity a    b    = (if a = b then a else PBot)"

instance ..
```

```
end

lemma gamma_inf_parity [simp]:
  "gamma_parity (a ⊓ b) = gamma_parity a ∩ gamma_parity b"
  by (cases a; cases b; auto)

instance parity :: semilattice_inf
proof intro_classes
  fix x y z :: parity
  show "x ⊓ y ≤ x"
    by (cases x; cases y; simp add: less_eq_parity_def)
  show "x ⊓ y ≤ y"
    by (cases x; cases y; simp add: less_eq_parity_def)
  show "x ≤ y ⟹ x ≤ z ⟹ x ≤ y ⊓ z"
    by (cases x; cases y; cases z; simp add: less_eq_parity_def)
qed

instance parity :: lattice ..
instance parity :: bounded_lattice_bot ..
```

Executable interface

`PBot` is the only value with empty concretization, so a direct equality test decides emptiness exactly. Finiteness of the carrier is not what settles this — a finite domain may perfectly well have two empty values. What settles it is the four concretizations themselves: the other three are each inhabited.

```
definition is_bottom_parity :: "parity ⇒ bool" where
  "is_bottom_parity p = (p = PBot)"

lemma is_bottom_parity_correct: "is_bottom_parity p ⟷ gamma_parity p = {}"
  unfolding is_bottom_parity_def
  by (cases p) (auto intro: exI[of _ "0"] exI[of _ "1"])
```

`PTop` is the unique top of a finite enumeration, the same reasoning as Sign's `is_top_sign`.

```
definition is_top_parity :: "parity ⇒ bool" where
  "is_top_parity p = (p = PTop)"

lemma is_top_parity_correct_gamma: "is_top_parity p ⟷ gamma_parity p = UNIV"
  by(cases p) (auto simp: is_top_parity_def set_eq_iff; presburger)+
```

Goblint has no parity domain. A parity is the congruence class modulo 2, so it prints in Goblint's congruence notation: $2\mathbb{Z}$, $1+2\mathbb{Z}$, and $\mathbb{Z}$ for a product component's top. A standalone parity prints its top as $\top$.

```
fun string_of_parity :: "parity ⇒ String.literal" where
    "string_of_parity PBot  = sym_bottom"
  | "string_of_parity PEven = STR ''2'' + sym_int"
  | "string_of_parity POdd  = STR ''1+2'' + sym_int"
  | "string_of_parity PTop  = sym_int"

end
```

2.17 Congruence_Lattice

```
theory Congruence_Lattice
  imports "Voblint_Domain.State_Concretization" "HOL-Computational_Algebra.Euclidean_Algorithm"
begin
```

Congruence lattice

Goblint's congruence domain as a normalized subtype of `(int * int) option`, with concretization `gamma_congruence`, the lattice order, join and the executable tests the `numeric_domain` instance needs.

Carrier and concretization

The representation follows Goblint's congruence domain: bottom is `None` and `Some (c, m)` denotes the class of integers congruent to `c` modulo `m`. Modulus zero denotes the singleton `{c}`.

Isabelle's order laws quantify over every carrier value, so the public type is the subtype of normalized representations. Public construction applies the same normalization discipline as Goblint: a nonzero modulus becomes positive and its residue lies in the half-open range from zero to the modulus.

```
type_synonym congruence_rep = "(int * int) option"

fun normalized_congruence_rep :: "congruence_rep => bool" where
  "normalized_congruence_rep None = True"
| "normalized_congruence_rep (Some (c, m)) =
     (m = 0 ∨ (0 <= c ∧ c < m))"

fun normalize_congruence_rep :: "congruence_rep => congruence_rep" where
  "normalize_congruence_rep None = None"
| "normalize_congruence_rep (Some (c, m)) =
     (if m = 0 then Some (c, 0)
      else
        let p = abs m;
            r = c mod p
        in Some (r, p))"

lemma normalized_normalize_congruence_rep [simp]:
  "normalized_congruence_rep (normalize_congruence_rep x)"
proof (cases x)
  case None
  then show ?thesis by simp
next
  case (Some p)
  obtain c m where p: "p = (c, m)"
    by (cases p)
  show ?thesis
  proof (cases "m = 0")
    case True
    then show ?thesis unfolding Some p by simp
  next
    case False
    then have positive: "0 < abs m" by simp
    have nonnegative: "0 <= c mod abs m"
      using positive by simp
    have less: "c mod abs m < abs m"
      by (rule pos_mod_bound[OF positive])
    show ?thesis
      unfolding Some p
      using False nonnegative less
      by (simp add: Let_def)
  qed
qed

lemma normalize_congruence_rep_fixed:
  assumes "normalized_congruence_rep x"
  shows "normalize_congruence_rep x = x"
proof (cases x)
  case None
  then show ?thesis by simp
next
  case (Some p)
  obtain c m where p: "p = (c, m)"
    by (cases p)
  show ?thesis
  proof (cases "m = 0")
```

```
    case True
    then show ?thesis unfolding Some p by simp
  next
    case False
    with assms have bounds: "0 <= c" "c < m"
      unfolding Some p by simp_all
    then have positive: "0 < m" by linarith
    have "c mod m = c"
      by (rule mod_pos_pos_trivial[OF bounds])
    with positive False show ?thesis
      unfolding Some p by (simp add: Let_def abs_of_pos)
  qed
qed

lemma normalize_congruence_rep_idem [simp]:
  "normalize_congruence_rep (normalize_congruence_rep x) =
   normalize_congruence_rep x"
  by (rule normalize_congruence_rep_fixed) simp

lemma normalized_congruence_rep_modulus_nonnegative:
  assumes "normalized_congruence_rep (Some (c, m))"
  shows "0 <= m"
  using assms by auto

lemma normalized_congruence_rep_nonzero:
  assumes "normalized_congruence_rep (Some (c, m))"
      and "m ≠ 0"
  shows "0 <= c" and "c < m" and "0 < m"
  using assms by auto
typedef congruence =
  "{x :: congruence_rep. normalized_congruence_rep x}"
  morphisms Rep_congruence Abs_congruence
  by (rule exI[of _ None]) simp

setup_lifting type_definition_congruence

lemma normalized_Rep_congruence [simp]:
  "normalized_congruence_rep (Rep_congruence a)"
  using Rep_congruence[of a] by simp

lemma congruence_rep_cases [case_names Bot Class]:
  obtains (Bot) "Rep_congruence a = None"
    | (Class) c m where "Rep_congruence a = Some (c, m)"
proof (cases "Rep_congruence a")
  case (Some p)
  then show thesis by (cases p) (auto intro: Class)
qed (rule Bot)
instantiation congruence :: equal
begin

definition equal_congruence :: "congruence => congruence => bool" where
  "equal_congruence a b = (Rep_congruence a = Rep_congruence b)"

instance
proof
  fix a b :: congruence
  show "HOL.equal a b ⟷ a = b"
    unfolding equal_congruence_def
    by (simp only: Rep_congruence_inject)
qed

end

lift_definition mk_congruence :: "int => int => congruence"
  is "λc m. normalize_congruence_rep (Some (c, m))"
  by (rule normalized_normalize_congruence_rep)

lift_definition bottom_congruence :: congruence
  is None
  by simp

declare mk_congruence.rep_eq [simp] bottom_congruence.rep_eq [simp]

lemma congruence_cases [cases type: congruence]:
  obtains "a = bottom_congruence"
    | c m where "a = mk_congruence c m"
```

```
proof (cases "Rep_congruence a")
  case None
  have "Rep_congruence a = Rep_congruence bottom_congruence"
    using None by simp
  then have "a = bottom_congruence"
    by (simp only: Rep_congruence_inject)
  then show thesis by (rule that)
next
  case (Some p)
  obtain c m where p: "p = (c, m)"
    by (cases p)
  have normalized: "normalized_congruence_rep (Some (c, m))"
    using normalized_Rep_congruence[of a] unfolding Some p by simp
  then have fixed:
    "normalize_congruence_rep (Some (c, m)) = Some (c, m)"
    by (rule normalize_congruence_rep_fixed)
  have reps: "Rep_congruence a = Rep_congruence (mk_congruence c m)"
  proof -
    have "Rep_congruence a = Some (c, m)"
      using Some p by simp
    also have "... = Rep_congruence (mk_congruence c m)"
      using fixed by simp
    finally show ?thesis .
  qed
  then have "a = mk_congruence c m"
    by (simp only: Rep_congruence_inject)
  then show thesis by (rule that)
qed

fun gamma_congruence_rep :: "congruence_rep => int set" where
  "gamma_congruence_rep None = {}"
| "gamma_congruence_rep (Some (c, m)) = {n. m dvd n - c}"

definition gamma_congruence :: "congruence => int set" where
  "gamma_congruence a = gamma_congruence_rep (Rep_congruence a)"

lemma congruence_class_subset_iff:
  fixes m1 m2 c1 c2 :: int
  shows
    "{n. m1 dvd n - c1} ⊆ {n. m2 dvd n - c2} ⟷
     m2 dvd m1 ∧ m2 dvd c1 - c2"
proof
  assume subset:
    "{n. m1 dvd n - c1} ⊆ {n. m2 dvd n - c2}"
  have offset: "m2 dvd c1 - c2"
    using subsetD[OF subset, of c1] by simp
  have shifted: "m2 dvd (c1 + m1) - c2"
    using subsetD[OF subset, of "c1 + m1"] by simp
  have "m2 dvd ((c1 + m1) - c2) - (c1 - c2)"
    by (rule dvd_diff[OF shifted offset])
  then have "m2 dvd m1" by simp
  with offset show "m2 dvd m1 ∧ m2 dvd c1 - c2" by blast
next
  assume divisibility: "m2 dvd m1 ∧ m2 dvd c1 - c2"
  show "{n. m1 dvd n - c1} ⊆ {n. m2 dvd n - c2}"
  proof
    fix n
    assume "n ∈ {n. m1 dvd n - c1}"
    then have n: "m1 dvd n - c1" by simp
    have first: "m2 dvd n - c1"
      by (rule dvd_trans[OF divisibility[THEN conjunct1] n])
    have "m2 dvd (n - c1) + (c1 - c2)"
      by (rule dvd_add[OF first divisibility[THEN conjunct2]])
    then show "n ∈ {n. m2 dvd n - c2}" by simp
  qed
qed

lemma gamma_normalize_congruence_rep [simp]:
  "gamma_congruence_rep (normalize_congruence_rep x) =
   gamma_congruence_rep x"
proof (cases x)
  case None
  then show ?thesis by simp
next
  case (Some p)
  obtain c m where p: "p = (c, m)"
```

```
    by (cases p)
  show ?thesis
  proof (cases "m = 0")
    case True
    then show ?thesis unfolding Some p by simp
  next
    case False
    let ?modulus = "abs m"
    let ?residue = "c mod ?modulus"
    have modulus1: "m dvd ?modulus" by simp
    have modulus2: "?modulus dvd m" by simp
    have offset2: "?modulus dvd c - ?residue"
      by (rule dvd_minus_mod)
    have offset1: "m dvd c - ?residue"
      using offset2 by (simp only: abs_dvd_iff)
    have negated_offset: "m dvd -(c - ?residue)"
      using offset1 by (simp only: dvd_minus_iff)
    have offset1_reverse: "m dvd ?residue - c"
      using negated_offset by simp
    have classes:
      "{n. ?modulus dvd n - ?residue} = {n. m dvd n - c}"
    proof (rule Set.subset_antisym)
      show "{n. ?modulus dvd n - ?residue} ⊆
        {n. m dvd n - c}"
        unfolding congruence_class_subset_iff
        using modulus1 offset1_reverse by blast
      show "{n. m dvd n - c} ⊆
        {n. ?modulus dvd n - ?residue}"
        unfolding congruence_class_subset_iff
        using modulus2 offset2 by blast
    qed
    show ?thesis
      unfolding Some p
      using False classes by (simp add: Let_def)
  qed
qed

lemma gamma_congruence_rep_inject:
  assumes normalized_x: "normalized_congruence_rep x"
      and normalized_y: "normalized_congruence_rep y"
      and gamma_eq: "gamma_congruence_rep x = gamma_congruence_rep y"
  shows "x = y"
proof (cases x)
  case None
  show ?thesis
  proof (cases y)
    case None
    with ‘x = None‘ show ?thesis by simp
  next
    case (Some p)
    obtain c m where p: "p = (c, m)"
      by (cases p)
    have "c ∈ gamma_congruence_rep y"
      unfolding Some p by simp
    moreover have "gamma_congruence_rep y = {}"
      using gamma_eq ‘x = None‘ by simp
    ultimately show ?thesis by simp
  qed
next
  case (Some p1)
  obtain c1 m1 where p1: "p1 = (c1, m1)"
    by (cases p1)
  show ?thesis
  proof (cases y)
    case None
    have "c1 ∈ gamma_congruence_rep x"
      unfolding Some p1 by simp
    moreover have "gamma_congruence_rep x = {}"
      using gamma_eq None by simp
    ultimately show ?thesis by simp
  next
    case (Some p2)
    obtain c2 m2 where p2: "p2 = (c2, m2)"
      by (cases p2)
    have subset12:
      "{n. m1 dvd n - c1} ⊆ {n. m2 dvd n - c2}"
```

```
    using gamma_eq unfolding ‘x = Some p1‘ Some p1 p2 by auto
  have subset21:
    "{n. m2 dvd n - c2} ⊆ {n. m1 dvd n - c1}"
    using gamma_eq unfolding ‘x = Some p1‘ Some p1 p2 by auto
  have divisibility12: "m2 dvd m1 ∧ m2 dvd c1 - c2"
    using subset12 unfolding congruence_class_subset_iff .
  have divisibility21: "m1 dvd m2 ∧ m1 dvd c2 - c1"
    using subset21 unfolding congruence_class_subset_iff .
  have nonnegative1: "0 <= m1"
    using normalized_x unfolding ‘x = Some p1‘ p1
    by (rule normalized_congruence_rep_modulus_nonnegative)
  have nonnegative2: "0 <= m2"
    using normalized_y unfolding Some p2
    by (rule normalized_congruence_rep_modulus_nonnegative)
  have moduli: "m1 = m2"
    by (rule Int.zdvd_antisym_nonneg[OF nonnegative1 nonnegative2
          divisibility21[THEN conjunct1] divisibility12[THEN conjunct1]])
  have residues: "c1 = c2"
  proof (cases "m1 = 0")
    case True
    with moduli divisibility12 show ?thesis by simp
  next
    case False
    have bounds1: "0 <= c1" "c1 < m1" "0 < m1"
      using normalized_x False unfolding ‘x = Some p1‘ p1
      by (rule normalized_congruence_rep_nonzero)+
    have bounds2: "0 <= c2" "c2 < m1"
      using normalized_y False moduli unfolding Some p2
      by auto
    have mod_eq: "c1 mod m1 = c2 mod m1"
      using divisibility12 moduli
      by (simp only: Euclidean_Rings.euclidean_ring_cancel_class.mod_eq_dvd_iff)
    have left: "c1 mod m1 = c1"
      by (rule mod_pos_pos_trivial[OF bounds1(1) bounds1(2)])
    have right: "c2 mod m1 = c2"
      by (rule mod_pos_pos_trivial[OF bounds2])
    show ?thesis using mod_eq left right by simp
  qed
  show ?thesis
    unfolding ‘x = Some p1‘ Some p1 p2 moduli residues by simp
 qed
qed

lemma gamma_mk_congruence [simp]:
 "gamma_congruence (mk_congruence c m) = {n. m dvd n - c}"
 unfolding gamma_congruence_def
 by (simp only: mk_congruence.rep_eq gamma_normalize_congruence_rep
     gamma_congruence_rep.simps)

lemma mk_congruence_member [simp]:
 "c ∈ gamma_congruence (mk_congruence c m)"
 by simp

lemma gamma_bottom_congruence [simp]:
 "gamma_congruence bottom_congruence = {}"
 unfolding gamma_congruence_def by simp

lemma gamma_congruence_empty_iff [simp]:
 "gamma_congruence a = {} ⟷ a = bottom_congruence"
proof (cases "Rep_congruence a")
 case None
 have "Rep_congruence a = Rep_congruence bottom_congruence"
   using None by simp
 then have "a = bottom_congruence"
   by (simp only: Rep_congruence_inject)
 with None show ?thesis
   unfolding gamma_congruence_def by simp
next
 case (Some p)
 obtain c m where p: "p = (c, m)"
   by (cases p)
 have member: "c ∈ gamma_congruence a"
   unfolding gamma_congruence_def Some p by simp
 have "a ≠ bottom_congruence"
 proof
   assume "a = bottom_congruence"
```

```
    with Some show False by simp
  qed
  with member show ?thesis by blast
qed

lemma gamma_congruence_inject:
  assumes "gamma_congruence a = gamma_congruence b"
  shows "a = b"
proof -
  have reps: "Rep_congruence a = Rep_congruence b"
  proof (rule gamma_congruence_rep_inject)
    show "normalized_congruence_rep (Rep_congruence a)"
      by simp
    show "normalized_congruence_rep (Rep_congruence b)"
      by simp
    show "gamma_congruence_rep (Rep_congruence a) =
      gamma_congruence_rep (Rep_congruence b)"
      using assms unfolding gamma_congruence_def .
  qed
  from reps show ?thesis
    by (simp only: Rep_congruence_inject)
qed

lemma mk_congruence_negative_modulus [simp]:
  "mk_congruence 5 (-4) = mk_congruence 1 4"
  by (rule Rep_congruence_inject[THEN iffD1])
     (simp add: Let_def)


definition congruence_of_int :: "int => congruence" where
  "congruence_of_int n = mk_congruence n 0"

lemma congruence_of_int_gamma [simp]:
  "n ∈ gamma_congruence (congruence_of_int n)"
  unfolding congruence_of_int_def by simp
```

Order

A congruence value denotes a set of integers: `None` denotes none at all and `Some (c, m)` the numbers congruent to `c` modulo `m`, with `m = 0` meaning the single number `c`. Containment between two such sets is decidable arithmetic: `Some (c1, m1)` sits inside `Some (c2, m2)` exactly when the coarser modulus `m2` divides both `m1` and the offset `c1 - c2`. That test is `congruence_le_rep` just below, and it is what everything else here is built on.

From it come the order instance on the normalized type `congruence`, its bottom (the empty class) and top (every integer, `Some (0, 1)`), the join — the coarsest class containing both arguments, computed through the gcd of the moduli and the difference of the residues — and the executable emptiness and printing operations the abstract-domain interface asks for.

```
fun congruence_le_rep :: "congruence_rep => congruence_rep => bool" where
  "congruence_le_rep None _ = True"
| "congruence_le_rep (Some _) None = False"
| "congruence_le_rep (Some (c1, m1)) (Some (c2, m2)) =
     (m2 dvd m1 ∧ m2 dvd c1 - c2)"

lemma congruence_le_rep_iff:
  "congruence_le_rep x y ⟷
   gamma_congruence_rep x ⊆ gamma_congruence_rep y"
proof (cases x)
  case None
  then show ?thesis by simp
next
  case (Some p1)
  obtain c1 m1 where p1: "p1 = (c1, m1)"
    by (cases p1)
  have x_rep: "x = Some (c1, m1)"
    using Some p1 by simp
  show ?thesis
  proof (cases y)
    case None
    have member: "c1 ∈ gamma_congruence_rep x"
      unfolding x_rep by simp
    have not_subset:
      "¬ gamma_congruence_rep x ⊆ gamma_congruence_rep y"
    proof
      assume subset: "gamma_congruence_rep x ⊆ gamma_congruence_rep y"
      have "c1 ∈ gamma_congruence_rep y"
```

```
        by (rule subsetD[OF subset member])
      with None show False by simp
    qed
    with None show ?thesis unfolding x_rep by simp
  next
    case (Some p2)
    obtain c2 m2 where p2: "p2 = (c2, m2)"
      by (cases p2)
    have y_rep: "y = Some (c2, m2)"
      using Some p2 by simp
    show ?thesis
      unfolding x_rep y_rep
      by (simp only: congruence_le_rep.simps gamma_congruence_rep.simps
          congruence_class_subset_iff)
  qed
qed

definition congruence_le :: "congruence => congruence => bool" where
  "congruence_le a b =
     congruence_le_rep (Rep_congruence a) (Rep_congruence b)"

lemma congruence_le_iff_gamma:
  "congruence_le a b ⟷
   gamma_congruence a ⊆ gamma_congruence b"
  unfolding congruence_le_def gamma_congruence_def
  by (rule congruence_le_rep_iff)

instantiation congruence :: ord
begin

definition less_eq_congruence :: "congruence => congruence => bool" where
  "(a :: congruence) <= b = congruence_le a b"

definition less_congruence :: "congruence => congruence => bool" where
  "(a :: congruence) < b = (a <= b ∧ ¬ b <= a)"

instance ..

end

lemma less_eq_congruence_iff_gamma:
  "a ≤ b ⟷ gamma_congruence a ⊆ gamma_congruence b"
  for a b :: congruence
  unfolding less_eq_congruence_def by (rule congruence_le_iff_gamma)

instance congruence :: order
proof intro_classes
  fix a b c :: congruence
  show "(a < b) = (a <= b ∧ ¬ b <= a)"
    unfolding less_congruence_def ..
  show "a <= a"
    unfolding less_eq_congruence_iff_gamma by simp
  assume ab: "a <= b" and bc: "b <= c"
  show "a <= c"
    using ab bc
    unfolding less_eq_congruence_iff_gamma
    by blast
next
  fix a b :: congruence
  assume ab: "a <= b" and ba: "b <= a"
  have gamma_eq: "gamma_congruence a = gamma_congruence b"
    using ab ba
    unfolding less_eq_congruence_iff_gamma
    by blast
  show "a = b"
    by (rule gamma_congruence_inject[OF gamma_eq])
qed

instantiation congruence :: bot
begin

definition bot_congruence :: congruence where
  "bot_congruence = bottom_congruence"

instance ..
```

```
end

instance congruence :: order_bot
proof intro_classes
  fix a :: congruence
  show "bot <= a"
    unfolding less_eq_congruence_iff_gamma bot_congruence_def
    by simp
qed

instantiation congruence :: top
begin

definition top_congruence :: congruence where
  "top_congruence = mk_congruence 0 1"

instance ..

end

lemma gamma_top_congruence [simp]:
  "gamma_congruence (top :: congruence) = UNIV"
  unfolding top_congruence_def by auto

lemma mk_congruence_mod_one [simp]:
  "mk_congruence c 1 = (top :: congruence)"
proof (rule gamma_congruence_inject)
  show "gamma_congruence (mk_congruence c 1) =
    gamma_congruence (top :: congruence)"
    by auto
qed

instance congruence :: order_top
proof intro_classes
  fix a :: congruence
  show "a <= top"
    unfolding less_eq_congruence_iff_gamma
    by simp
qed
```

Join

Two classes join to the class modulo the gcd of both moduli and the residue difference, renormalized. The lemmas show the join is an upper bound in `gamma_congruence` and the least one in the order.

```
fun join_congruence_rep :: "congruence_rep => congruence_rep => congruence_rep" where
  "join_congruence_rep None y = y"
| "join_congruence_rep x None = x"
| "join_congruence_rep (Some (c1, m1)) (Some (c2, m2)) =
     Some (c1, gcd m1 (gcd m2 (c1 - c2)))"

lift_definition join_congruence :: "congruence => congruence => congruence"
  is "λx y. normalize_congruence_rep (join_congruence_rep x y)"
  by (rule normalized_normalize_congruence_rep)

declare join_congruence.rep_eq [simp]

lemma gamma_join_congruence [simp]:
  "gamma_congruence (join_congruence a b) =
   gamma_congruence_rep
     (join_congruence_rep (Rep_congruence a) (Rep_congruence b))"
  unfolding gamma_congruence_def by simp

lemma gamma_join_congruence_rep_ub1:
  "gamma_congruence_rep x ⊆
   gamma_congruence_rep (join_congruence_rep x y)"
proof (cases x)
  case None
  then show ?thesis by simp
next
  case (Some p1)
  obtain c1 m1 where p1: "p1 = (c1, m1)"
    by (cases p1)
  have x_rep: "x = Some (c1, m1)"
    using Some p1 by simp
  show ?thesis
```

```
  proof (cases y)
    case None
    with x_rep show ?thesis by simp
  next
    case (Some p2)
    obtain c2 m2 where p2: "p2 = (c2, m2)"
      by (cases p2)
    have y_rep: "y = Some (c2, m2)"
      using Some p2 by simp
    have modulus:
      "gcd m1 (gcd m2 (c1 - c2)) dvd m1"
      by (rule gcd_dvd1)
    show ?thesis
      unfolding x_rep y_rep
      apply (simp only: join_congruence_rep.simps gamma_congruence_rep.simps)
      apply (subst congruence_class_subset_iff)
      using modulus by simp
  qed
qed

lemma gamma_join_congruence_rep_ub2:
  "gamma_congruence_rep y ⊆
   gamma_congruence_rep (join_congruence_rep x y)"
proof (cases x)
  case None
  then show ?thesis by simp
next
  case (Some p1)
  obtain c1 m1 where p1: "p1 = (c1, m1)"
    by (cases p1)
  have x_rep: "x = Some (c1, m1)"
    using Some p1 by simp
  show ?thesis
  proof (cases y)
    case None
    with x_rep show ?thesis by simp
  next
    case (Some p2)
    obtain c2 m2 where p2: "p2 = (c2, m2)"
      by (cases p2)
    have y_rep: "y = Some (c2, m2)"
      using Some p2 by simp
    let ?g = "gcd m1 (gcd m2 (c1 - c2))"
    have modulus: "?g dvd m2"
      by (rule dvd_trans[OF gcd_dvd2 gcd_dvd1])
    have offset_forward: "?g dvd c1 - c2"
      by (rule dvd_trans[OF gcd_dvd2 gcd_dvd2])
    have offset: "?g dvd c2 - c1"
      by (subst dvd_diff_commute) (rule offset_forward)
    show ?thesis
      unfolding x_rep y_rep
      apply (simp only: join_congruence_rep.simps gamma_congruence_rep.simps)
      apply (subst congruence_class_subset_iff)
      using modulus offset by simp
  qed
qed

lemma gamma_join_congruence_rep_least:
  assumes xz:
    "gamma_congruence_rep x ⊆ gamma_congruence_rep z"
      and yz:
    "gamma_congruence_rep y ⊆ gamma_congruence_rep z"
  shows
    "gamma_congruence_rep (join_congruence_rep x y) ⊆
     gamma_congruence_rep z"
proof (cases x)
  case None
  with yz show ?thesis by simp
next
  case (Some p1)
  obtain c1 m1 where p1: "p1 = (c1, m1)"
    by (cases p1)
  have x_rep: "x = Some (c1, m1)"
    using Some p1 by simp
  show ?thesis
  proof (cases y)
```

```
    case None
    with x_rep xz show ?thesis by simp
  next
    case (Some p2)
    obtain c2 m2 where p2: "p2 = (c2, m2)"
      by (cases p2)
    have y_rep: "y = Some (c2, m2)"
      using Some p2 by simp
    show ?thesis
    proof (cases z)
      case None
      have member: "c1 ∈ gamma_congruence_rep x"
        unfolding x_rep by simp
      have "c1 ∈ gamma_congruence_rep z"
        by (rule subsetD[OF xz member])
      with None show ?thesis by simp
    next
      case (Some p3)
      obtain c3 m3 where p3: "p3 = (c3, m3)"
        by (cases p3)
      have z_rep: "z = Some (c3, m3)"
        using Some p3 by simp
      have x_divisibility:
        "m3 dvd m1 ∧ m3 dvd c1 - c3"
        using xz
        unfolding x_rep z_rep
        by (simp only: gamma_congruence_rep.simps
            congruence_class_subset_iff)
      have y_divisibility:
        "m3 dvd m2 ∧ m3 dvd c2 - c3"
        using yz
        unfolding y_rep z_rep
        by (simp only: gamma_congruence_rep.simps
            congruence_class_subset_iff)
      have offset: "m3 dvd c1 - c2"
      proof -
        have "m3 dvd (c1 - c3) - (c2 - c3)"
          by (rule dvd_diff[OF x_divisibility[THEN conjunct2]
                y_divisibility[THEN conjunct2]])
        then show ?thesis by simp
      qed
      have inner: "m3 dvd gcd m2 (c1 - c2)"
        by (rule gcd_greatest[OF y_divisibility[THEN conjunct1] offset])
      have modulus:
        "m3 dvd gcd m1 (gcd m2 (c1 - c2))"
        by (rule gcd_greatest[OF x_divisibility[THEN conjunct1] inner])
      show ?thesis
        unfolding x_rep y_rep z_rep
        apply (simp only: join_congruence_rep.simps gamma_congruence_rep.simps)
        apply (subst congruence_class_subset_iff)
        using modulus x_divisibility by simp
    qed
  qed
qed

lemma join_congruence_ub1:
  "a <= join_congruence a b"
  unfolding less_eq_congruence_iff_gamma
    gamma_congruence_def
  apply (simp only: join_congruence.rep_eq gamma_normalize_congruence_rep)
  by (rule gamma_join_congruence_rep_ub1)

lemma join_congruence_ub2:
  "b <= join_congruence a b"
  unfolding less_eq_congruence_iff_gamma
    gamma_congruence_def
  apply (simp only: join_congruence.rep_eq gamma_normalize_congruence_rep)
  by (rule gamma_join_congruence_rep_ub2)

lemma join_congruence_least:
  assumes "a <= c" and "b <= c"
  shows "join_congruence a b <= c"
  using assms
  unfolding less_eq_congruence_iff_gamma
    gamma_congruence_def
  apply (simp only: join_congruence.rep_eq gamma_normalize_congruence_rep)
```

```
  by (rule gamma_join_congruence_rep_least)

instantiation congruence :: sup
begin

definition sup_congruence :: "congruence => congruence => congruence" where
  "sup_congruence = join_congruence"

instance ..

end

instance congruence :: semilattice_sup
proof intro_classes
  fix a b c :: congruence
  show "a <= a ⊔ b"
    unfolding sup_congruence_def by (rule join_congruence_ub1)
  show "b <= a ⊔ b"
    unfolding sup_congruence_def by (rule join_congruence_ub2)
  show "b <= a ⟹ c <= a ⟹ b ⊔ c <= a"
    unfolding sup_congruence_def by (rule join_congruence_least)
qed

instance congruence :: bounded_semilattice_sup_bot ..
```

Meet

Intersecting two non-singleton congruence classes is the executable Chinese remainder construction. The Bezout coefficient computes one shared residue; the least common multiple is the period of every shared solution.

```
fun intersect_congruence_rep ::
    "congruence_rep => congruence_rep => congruence_rep"
where
  "intersect_congruence_rep None y = None"
| "intersect_congruence_rep x None = None"
| "intersect_congruence_rep (Some (c1, m1)) (Some (c2, m2)) =
     normalize_congruence_rep
       (if m1 = 0 then
          if m2 dvd c1 - c2 then Some (c1, 0) else None
        else if m2 = 0 then
          if m1 dvd c2 - c1 then Some (c2, 0) else None
        else
          let g = gcd m1 m2
          in if g dvd c2 - c1 then
               let s = fst (bezout_coefficients m1 m2);
                   q = (c2 - c1) div g
               in Some (c1 + m1 * (q * s), lcm m1 m2)
             else None)"

lemma normalized_intersect_congruence_rep [simp]:
  "normalized_congruence_rep (intersect_congruence_rep x y)"
  by (cases x; cases y)
     (auto simp: split_def split: if_splits prod.splits)

instantiation congruence :: inf
begin

lift_definition inf_congruence ::
    "congruence => congruence => congruence"
  is intersect_congruence_rep
  by simp

instance ..

end

lemma bezout_shared_residue:
  assumes compatible: "gcd m1 m2 dvd c2 - c1"
  defines
    "q == (c2 - c1) div gcd m1 m2"
  shows
    "m1 dvd
       (c1 + m1 * (q * fst (bezout_coefficients m1 m2))) - c1 ∧
     m2 dvd
       (c1 + m1 * (q * fst (bezout_coefficients m1 m2))) - c2"
proof (intro conjI)
```

```
  show
    "m1 dvd
      c1 + m1 * (q * fst (bezout_coefficients m1 m2)) - c1"
    by simp
  have bezout:
    "fst (bezout_coefficients m1 m2) * m1 +
     snd (bezout_coefficients m1 m2) * m2 =
     gcd m1 m2"
    by (rule bezout_coefficients_fst_snd)
  have quotient:
    "q * gcd m1 m2 = c2 - c1"
    unfolding q_def
    using dvd_mult_div_cancel[OF compatible]
    by (simp add: mult.commute)
  have scaled0:
    "q * (fst (bezout_coefficients m1 m2) * m1 +
       snd (bezout_coefficients m1 m2) * m2) =
     q * gcd m1 m2"
    by (rule arg_cong[OF bezout])
  have scaled:
    "m1 * (q * fst (bezout_coefficients m1 m2)) +
     m2 * (q * snd (bezout_coefficients m1 m2)) =
     q * gcd m1 m2"
    using scaled0 by (simp add: algebra_simps)
  show
    "m2 dvd
      c1 + m1 * (q * fst (bezout_coefficients m1 m2)) - c2"
    unfolding dvd_def
  proof (rule exI[of _ "- q * snd (bezout_coefficients m1 m2)"])
    show
      "c1 + m1 * (q * fst (bezout_coefficients m1 m2)) - c2 =
       m2 * (- q * snd (bezout_coefficients m1 m2))"
      using scaled quotient by (simp add: algebra_simps)
  qed
qed


lemma gamma_intersect_congruence_rep:
  assumes normalized_x: "normalized_congruence_rep x"
      and normalized_y: "normalized_congruence_rep y"
  shows
    "gamma_congruence_rep (intersect_congruence_rep x y) =
     gamma_congruence_rep x ∩ gamma_congruence_rep y"
proof (cases x)
  case None
  then show ?thesis by simp
next
  case (Some cm1)
  note x_some = Some
  obtain c1 m1 where cm1: "cm1 = (c1, m1)"
    by (cases cm1)
  show ?thesis
  proof (cases y)
    case None
    with x_some show ?thesis by simp
  next
    case (Some cm2)
    note y_some = Some
    obtain c2 m2 where cm2: "cm2 = (c2, m2)"
      by (cases cm2)
    show ?thesis
    proof (cases "m1 = 0")
      case True
      with x_some y_some cm1 cm2 show ?thesis
        by auto
    next
      case m1_nonzero: False
      show ?thesis
      proof (cases "m2 = 0")
        case True
        with x_some y_some cm1 cm2 show ?thesis
          by auto
      next
        case m2_nonzero: False
        let ?g = "gcd m1 m2"
        let ?q = "(c2 - c1) div ?g"
```

```
        let ?s = "fst (bezout_coefficients m1 m2)"
        let ?z = "c1 + m1 * (?q * ?s)"
        show ?thesis
        proof (cases "?g dvd c2 - c1")
          case incompatible: False
          have empty:
            "{n. m1 dvd n - c1} ∩
             {n. m2 dvd n - c2} = {}"
          proof (rule ccontr)
            assume
              "{n. m1 dvd n - c1} ∩
               {n. m2 dvd n - c2} ~= {}"
            then obtain n where
              n1: "m1 dvd n - c1" and
              n2: "m2 dvd n - c2"
              by blast
            have g1: "?g dvd n - c1"
              using gcd_dvd1 n1 by (rule dvd_trans)
            have g2: "?g dvd n - c2"
              using gcd_dvd2 n2 by (rule dvd_trans)
            have "?g dvd (n - c1) - (n - c2)"
              by (rule dvd_diff[OF g1 g2])
            then have "?g dvd c2 - c1"
              by simp
            with incompatible show False by contradiction
          qed
          with x_some y_some cm1 cm2 m1_nonzero m2_nonzero
            incompatible
          show ?thesis by simp
        next
          case compatible: True
          have shared:
            "m1 dvd ?z - c1 ∧ m2 dvd ?z - c2"
            by (rule bezout_shared_residue[OF compatible])
          have classes:
            "{n. lcm m1 m2 dvd n - ?z} =
             {n. m1 dvd n - c1} ∩
             {n. m2 dvd n - c2}"
          proof -
            have "m1 dvd n - ?z ⟷ m1 dvd n - c1" "m2 dvd n - ?z ⟷ m2 dvd n - c2" for n
              using shared dvd_add_left_iff[of m1 "?z - c1" "n - ?z"]
                dvd_add_left_iff[of m2 "?z - c2" "n - ?z"]
              by simp_all
            then show ?thesis by auto
          qed
          have normalized_gamma:
            "gamma_congruence_rep
               (normalize_congruence_rep
                 (Some (?z, lcm m1 m2))) =
             {n. lcm m1 m2 dvd n - ?z}"
            by (simp only: gamma_normalize_congruence_rep
                gamma_congruence_rep.simps)
          from x_some y_some cm1 cm2 m1_nonzero m2_nonzero
            compatible normalized_gamma classes
          show ?thesis
            by (simp only: intersect_congruence_rep.simps if_False
                if_True Let_def gamma_congruence_rep.simps)
        qed
      qed
    qed
  qed
qed

lemma gamma_inf_congruence [simp]:
  "gamma_congruence (a ⊓ b) =
   gamma_congruence a ∩ gamma_congruence b"
proof -
  have norm_a: "normalized_congruence_rep (Rep_congruence a)"
    by simp
  have norm_b: "normalized_congruence_rep (Rep_congruence b)"
    by simp
  show ?thesis
    using gamma_intersect_congruence_rep[OF norm_a norm_b]
    by (simp add: gamma_congruence_def inf_congruence.rep_eq)
qed
```

```
instance congruence :: semilattice_inf
  by standard (auto simp: less_eq_congruence_iff_gamma)

instance congruence :: lattice ..
instance congruence :: bounded_lattice_bot ..
```

Executable interface

Executable bottom and top tests, each proved against `gamma_congruence`.

```
definition is_bottom_congruence :: "congruence => bool" where
  "is_bottom_congruence a = (a = bot)"

lemma is_bottom_congruence_correct:
  "is_bottom_congruence a ⟷ gamma_congruence a = {}"
  unfolding is_bottom_congruence_def bot_congruence_def
  by simp

definition is_top_congruence :: "congruence => bool" where
  "is_top_congruence a = (a = top)"

lemma is_top_congruence_correct_gamma:
  "is_top_congruence a ⟷ gamma_congruence a = UNIV"
  unfolding is_top_congruence_def
  by (metis gamma_congruence_inject gamma_top_congruence)
```

Goblint's congruence notation: a constant prints as itself, the class of `r` modulo `m` as `r+mℤ`, with a zero residue and a unit modulus left out, so `3ℤ`, `1+3ℤ` and `ℤ`. A standalone congruence prints its top as ⊤.

```
definition string_of_congruence :: "congruence ⇒ String.literal" where
  "string_of_congruence c =
     (case Rep_congruence c of
        None ⇒ sym_bottom
      | Some (r, m) ⇒
          if m = 0 then string_of_int r
          else (if r = 0 then STR '''' else string_of_int r + STR ''+'')
             + (if m = 1 then STR '''' else string_of_int m) + sym_int)"

end
```

2.18 Int_Lattice

```
theory Int_Lattice
  imports Sign_Lattice Interval_Lattice Parity_Lattice Congruence_Lattice
begin
```

Integer product lattice

The carrier keeps the four scalar abstractions independently. Its concretization is their intersection; componentwise order and join preserve the existing domain structures. Cross-domain reduction is separate from this carrier definition.

Carrier

```
record int_dom =
  int_sign :: sign
  int_ivl :: ivl
  int_parity :: parity
  int_congruence :: congruence
```

Order, join and meet

The record package represents a closed record through an extensible scheme. Giving the scheme a componentwise instance makes the closed `int_dom` abbreviation usable at every existing lattice-polymorphic boundary. The scheme's `more` field is Isabelle machinery, not a semantic extension point: new integer components belong in `int_dom` and must participate explicitly in its lattice, concretization, and exact bottom decision.

```
instantiation int_dom_ext :: (bounded_lattice) bounded_lattice
begin

definition less_eq_int_dom_ext ::
  "'a int_dom_scheme => 'a int_dom_scheme => bool" where
  "less_eq_int_dom_ext a b ⟷
     int_sign a <= int_sign b ∧
     int_ivl a <= int_ivl b ∧
     int_parity a <= int_parity b ∧
     int_congruence a <= int_congruence b ∧
     more a <= more b"

definition less_int_dom_ext ::
  "'a int_dom_scheme => 'a int_dom_scheme => bool" where
  "less_int_dom_ext a b ⟷ a ≤ b ∧ ¬ b ≤ a"

definition sup_int_dom_ext ::
  "'a int_dom_scheme => 'a int_dom_scheme => 'a int_dom_scheme" where
  "sup_int_dom_ext a b =
     int_dom_ext
       (sup (int_sign a) (int_sign b))
       (sup (int_ivl a) (int_ivl b))
       (sup (int_parity a) (int_parity b))
       (sup (int_congruence a) (int_congruence b))
       (sup (more a) (more b))"

definition inf_int_dom_ext ::
  "'a int_dom_scheme => 'a int_dom_scheme => 'a int_dom_scheme" where
  "inf_int_dom_ext a b =
     int_dom_ext
       (inf (int_sign a) (int_sign b))
       (inf (int_ivl a) (int_ivl b))
       (inf (int_parity a) (int_parity b))
       (inf (int_congruence a) (int_congruence b))
       (inf (more a) (more b))"

definition bot_int_dom_ext :: "'a int_dom_scheme" where
  "bot_int_dom_ext = int_dom_ext bot bot bot bot bot"

definition top_int_dom_ext :: "'a int_dom_scheme" where
  "top_int_dom_ext = int_dom_ext top top top top top"

instance
proof intro_classes
  fix x y z :: "'a int_dom_scheme"
  show "x < y ⟷ x ≤ y ∧ ¬ y ≤ x"
    by (simp add: less_int_dom_ext_def)
```

```
  show "x ≤ x"
    by (simp add: less_eq_int_dom_ext_def)
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z"
    by (auto simp: less_eq_int_dom_ext_def intro: order_trans)
  show "x ≤ y ⟹ y ≤ x ⟹ x = y"
    by (cases x; cases y)
       (auto simp: less_eq_int_dom_ext_def intro: antisym)
  show "x ≤ sup x y"
    by (simp add: less_eq_int_dom_ext_def sup_int_dom_ext_def)
  show "y ≤ sup x y"
    by (simp add: less_eq_int_dom_ext_def sup_int_dom_ext_def)
  show "y ≤ x ⟹ z ≤ x ⟹ sup y z ≤ x"
    by (auto simp: less_eq_int_dom_ext_def sup_int_dom_ext_def)
  show "inf x y ≤ x"
    by (simp add: less_eq_int_dom_ext_def inf_int_dom_ext_def)
  show "inf x y ≤ y"
    by (simp add: less_eq_int_dom_ext_def inf_int_dom_ext_def)
  show "x ≤ y ⟹ x ≤ z ⟹ x ≤ inf y z"
    by (auto simp: less_eq_int_dom_ext_def inf_int_dom_ext_def)
  show "bot ≤ x"
    by (simp add: less_eq_int_dom_ext_def bot_int_dom_ext_def)
  show "x ≤ top"
    by (simp add: less_eq_int_dom_ext_def top_int_dom_ext_def)
qed

end

definition int_dom_sip :: "sign => ivl => parity => int_dom" where
  "int_dom_sip s i p =
     (top :: int_dom)(|int_sign := s, int_ivl := i, int_parity := p|)"

definition int_dom_sipc ::
  "sign => ivl => parity => congruence => int_dom" where
  "int_dom_sipc s i p c =
     (top :: int_dom)(|
       int_sign := s,
       int_ivl := i,
       int_parity := p,
       int_congruence := c
     |)"

lemma int_congruence_int_dom_sip [simp]:
  "int_congruence (int_dom_sip s i p) = top"
  by (simp add: int_dom_sip_def top_int_dom_ext_def)
```

Concretization

A product value denotes the integers every component admits: `gamma_int_dom` intersects the sign, interval, parity and congruence concretizations.

```
definition gamma_int_dom :: "'a int_dom_scheme => int set" where
  "gamma_int_dom d =
     gamma_sign (int_sign d) ∩
     gamma_ivl (int_ivl d) ∩
     gamma_parity (int_parity d) ∩
     gamma_congruence (int_congruence d)"
```

Every component meet is exact, so the product meet is too: it concretizes to the intersection of its operands. Query answers from several analyses are combined by ⊓, which therefore neither loses nor invents values.

```
lemma gamma_inf_int_dom [simp]:
  "gamma_int_dom (a ⊓ b) = gamma_int_dom a ∩ gamma_int_dom b"
  by (auto simp: gamma_int_dom_def inf_int_dom_ext_def)

lemma gamma_int_dom_update_sign [simp]:
  "gamma_int_dom (d(|int_sign := s|)) =
   gamma_sign s ∩
   gamma_ivl (int_ivl d) ∩
   gamma_parity (int_parity d) ∩
   gamma_congruence (int_congruence d)"
  by (simp add: gamma_int_dom_def)

lemma gamma_int_dom_update_ivl [simp]:
  "gamma_int_dom (d(|int_ivl := i|)) =
   gamma_sign (int_sign d) ∩
   gamma_ivl i ∩
```

```
   gamma_parity (int_parity d) ∩
   gamma_congruence (int_congruence d)"
  by (simp add: gamma_int_dom_def)

lemma gamma_int_dom_update_parity [simp]:
  "gamma_int_dom (d(|int_parity := p|)) =
   gamma_sign (int_sign d) ∩
   gamma_ivl (int_ivl d) ∩
   gamma_parity p ∩
   gamma_congruence (int_congruence d)"
  by (simp add: gamma_int_dom_def)

lemma gamma_int_dom_update_congruence [simp]:
  "gamma_int_dom (d(|int_congruence := c|)) =
   gamma_sign (int_sign d) ∩
   gamma_ivl (int_ivl d) ∩
   gamma_parity (int_parity d) ∩
   gamma_congruence c"
  by (simp add: gamma_int_dom_def)

fun interval_fact_of_sign :: "sign ⇒ ivl" where
  "interval_fact_of_sign SBot = ⊥"
| "interval_fact_of_sign SNeg = Ivl MinInf (Fin (-1))"
| "interval_fact_of_sign SNonPos = Ivl MinInf (Fin 0)"
| "interval_fact_of_sign SZero = Ivl (Fin 0) (Fin 0)"
| "interval_fact_of_sign SNonNeg = Ivl (Fin 0) PlusInf"
| "interval_fact_of_sign SPos = Ivl (Fin 1) PlusInf"
| "interval_fact_of_sign STop = ⊤"

lemma gamma_interval_fact_of_sign [simp]:
  "gamma_ivl (interval_fact_of_sign s) = gamma_sign s"
  by (cases s) (auto simp: bot_ivl_def top_ivl_def ivl_top_def)

fun congruence_fact_of_parity :: "parity => congruence" where
  "congruence_fact_of_parity PBot = bot"
| "congruence_fact_of_parity PEven = mk_congruence 0 2"
| "congruence_fact_of_parity POdd = mk_congruence 1 2"
| "congruence_fact_of_parity PTop = top"

lemma gamma_congruence_fact_of_parity [simp]:
  "gamma_congruence (congruence_fact_of_parity p) = gamma_parity p"
  by (cases p)
     (auto simp: bot_congruence_def)

lemma gamma_intersect_ivl_exact:
  "gamma_ivl (intersect_ivl a b) = gamma_ivl a ∩ gamma_ivl b"
  using gamma_ivl_mono[OF intersect_ivl_le1]
        gamma_ivl_mono[OF intersect_ivl_le2]
        intersect_ivl_gamma
  by blast
```

Executable interface

Deciding emptiness of the product needs more than the components' own tests, since each can be inhabited while their intersection is empty. Interval and congruence are checked jointly, after restricting the congruence by parity; `is_bottom_int_dom_correct` and `is_top_int_dom_correct_gamma` tie the tests to `gamma_int_dom`.

```
lemma inter_nonempty_iff:
  "A ∩ B ≠ {} ⟷ (∃x. x ∈ A ∧ x ∈ B)"
  by blast

lemma congruence_between_iff:
  assumes normalized: "normalized_congruence_rep (Some (c, m))"
  shows
    "(∃n::int. l ≤ n ∧ n ≤ u ∧ m dvd n - c) ⟷
     (if m = 0 then l ≤ c ∧ c ≤ u
      else l ≤ u ∧ l + ((c - l) mod m) ≤ u)"
proof (cases "m = 0")
  case True
  then show ?thesis by simp
next
  case False
  from normalized_congruence_rep_nonzero[OF normalized False]
  have modulus: "0 < m" by simp
  show ?thesis
  proof
```

```
    assume "∃n::int. l ≤ n ∧ n ≤ u ∧ m dvd n - c"
    then obtain n where n: "l ≤ n" "n ≤ u" "m dvd n - c"
      by blast
    have offset: "m dvd c - n"
      using n(3) by (metis dvd_minus_iff minus_diff_eq)
    have mod_eq: "(c - l) mod m = (n - l) mod m"
      using offset
      by (simp add:
          Euclidean_Rings.euclidean_ring_cancel_class.mod_eq_dvd_iff)
    have remainder_le: "(n - l) mod m ≤ n - l"
      by (rule Euclidean_Rings.zmod_le_nonneg_dividend) (use n(1) in linarith)
    show
      "if m = 0 then l ≤ c ∧ c ≤ u
       else l ≤ u ∧ l + ((c - l) mod m) ≤ u"
      using False mod_eq n(1,2) remainder_le
      by simp
  next
    assume rhs:
      "if m = 0 then l ≤ c ∧ c ≤ u
       else l ≤ u ∧ l + ((c - l) mod m) ≤ u"
    let ?n = "l + ((c - l) mod m)"
    have lower: "l ≤ ?n"
      using modulus by simp
    have congruent: "m dvd ?n - c"
    proof -
      have base: "m dvd (c - l) - (c - l) mod m"
        by (rule dvd_minus_mod)
      have "m dvd - ((c - l) - (c - l) mod m)"
        using base by (simp only: dvd_minus_iff)
      also have
        "- ((c - l) - (c - l) mod m) = ?n - c"
        by simp
      finally show ?thesis .
    qed
    have upper: "?n ≤ u"
      using rhs False by simp
    show "∃n::int. l ≤ n ∧ n ≤ u ∧ m dvd n - c"
      by (rule exI[of _ ?n]) (use lower upper congruent in blast)
  qed
qed

lemma congruence_class_nonempty:
  "∃n::int. m dvd n - c"
  by (rule exI[of _ c]) simp

lemma exists_congruence_le:
  assumes normalized: "normalized_congruence_rep (Some (c, m))"
      and nonzero: "m ≠ 0"
  shows "∃n::int. n ≤ u ∧ m dvd n - c"
proof -
  from normalized_congruence_rep_nonzero[OF normalized nonzero]
  have one_le: "1 ≤ m" by simp
  let ?q = "abs (c - u) + 1"
  have q_nonnegative: "0 ≤ ?q" by simp
  have difference_le_q: "c - u ≤ ?q"
    using abs_ge_self[of "c - u"] by linarith
  have scale: "?q ≤ m * ?q"
    using mult_right_mono[OF one_le q_nonnegative] by simp
  let ?n = "c - m * ?q"
  have upper: "?n ≤ u"
    using difference_le_q scale by linarith
  have congruent: "m dvd ?n - c"
    by simp
  show ?thesis
    by (rule exI[of _ ?n]) (use upper congruent in blast)
qed

lemma exists_congruence_ge:
  assumes normalized: "normalized_congruence_rep (Some (c, m))"
      and nonzero: "m ≠ 0"
  shows "∃n::int. l ≤ n ∧ m dvd n - c"
proof -
  from normalized_congruence_rep_nonzero[OF normalized nonzero]
  have one_le: "1 ≤ m" by simp
  let ?q = "abs (l - c) + 1"
  have q_nonnegative: "0 ≤ ?q" by simp
```

```
  have difference_le_q: "l - c ≤ ?q"
    using abs_ge_self[of "l - c"] by linarith
  have scale: "?q ≤ m * ?q"
    using mult_right_mono[OF one_le q_nonnegative] by simp
  let ?n = "c + m * ?q"
  have lower: "l ≤ ?n"
    using difference_le_q scale by linarith
  have congruent: "m dvd ?n - c"
    by simp
  show ?thesis
    by (rule exI[of _ ?n]) (use lower congruent in blast)
qed

fun ivl_congruence_rep_nonempty ::
  "ivl => congruence_rep => bool" where
  "ivl_congruence_rep_nonempty i None = False"
| "ivl_congruence_rep_nonempty (Ivl MinInf MinInf) (Some cm) = False"
| "ivl_congruence_rep_nonempty (Ivl MinInf (Fin u)) (Some (c, m)) =
     (if m = 0 then c ≤ u else True)"
| "ivl_congruence_rep_nonempty (Ivl MinInf PlusInf) (Some cm) = True"
| "ivl_congruence_rep_nonempty (Ivl (Fin l) MinInf) (Some cm) = False"
| "ivl_congruence_rep_nonempty (Ivl (Fin l) (Fin u)) (Some (c, m)) =
     (if m = 0 then l ≤ c ∧ c ≤ u
      else l ≤ u ∧ l + ((c - l) mod m) ≤ u)"
| "ivl_congruence_rep_nonempty (Ivl (Fin l) PlusInf) (Some (c, m)) =
     (if m = 0 then l ≤ c else True)"
| "ivl_congruence_rep_nonempty (Ivl PlusInf u) (Some cm) = False"

lemma ivl_congruence_rep_nonempty_correct:
  assumes normalized: "normalized_congruence_rep r"
  shows
    "ivl_congruence_rep_nonempty i r ⟷
     gamma_ivl i ∩ gamma_congruence_rep r ≠ {}"
proof (cases r)
  case None
  then show ?thesis by simp
next
  case (Some cm)
  obtain c m where cm: "cm = (c, m)"
    by (cases cm)
  have rep: "r = Some (c, m)"
    using Some cm by simp
  have normalized': "normalized_congruence_rep (Some (c, m))"
    using normalized unfolding rep .
  show ?thesis
  proof (cases i)
    case (Ivl l u)
    show ?thesis
      unfolding rep Ivl
      by (cases l; cases u; cases "m = 0")
         (auto simp: inter_nonempty_iff
            congruence_between_iff[OF normalized']
            congruence_class_nonempty
            exists_congruence_le[OF normalized']
            exists_congruence_ge[OF normalized'])
  qed
qed

fun parity_accepts :: "parity => int => bool" where
  "parity_accepts PBot n = False"
| "parity_accepts PEven n = even n"
| "parity_accepts POdd n = odd n"
| "parity_accepts PTop n = True"

lemma parity_accepts_gamma [simp]:
  "parity_accepts p n ⟷ n ∈ gamma_parity p"
  by (cases p) simp_all

lemma parity_accepts_difference:
  assumes concrete: "p = PEven ∨ p = POdd"
      and accepts: "parity_accepts p r"
  shows "2 dvd n - r ⟷ parity_accepts p n"
  using concrete accepts
  by (cases p) auto

lemma dvd_shift_modulus:
```

```
  fixes m n c :: int
  shows "m dvd n - (c + m) ⟷ m dvd n - c"
proof
  assume shifted: "m dvd n - (c + m)"
  have "m dvd (n - (c + m)) + m"
    by (rule dvd_add[OF shifted dvd_refl[of m]])
  then show "m dvd n - c" by simp
next
  assume original: "m dvd n - c"
  have "m dvd (n - c) - m"
    by (rule dvd_diff[OF original dvd_refl[of m]])
  also have "(n - c) - m = n - (c + m)"
    by simp
  finally show "m dvd n - (c + m)" .
qed

lemma gamma_normalized_lcm_two:
  "gamma_congruence_rep
     (normalize_congruence_rep (Some (r, lcm m 2))) =
   {n. m dvd n - r ∧ 2 dvd n - r}"
  by (simp only: gamma_normalize_congruence_rep
      gamma_congruence_rep.simps
      GCD.semiring_gcd_class.lcm_least_iff)

lemma congruent_even_modulus:
  fixes m n c :: int
  assumes modulus: "even m"
      and congruent: "m dvd n - c"
  shows "even n ⟷ even c"
proof -
  obtain k where difference: "n - c = m * k"
    using congruent unfolding dvd_def by blast
  have difference_even: "even (n - c)"
    using modulus difference by simp
  have parity_eq: "even (n - c) = even (n + c)"
    by (rule even_diff_iff)
  have sum_even: "even (n + c)"
    using parity_eq difference_even by blast
  show ?thesis
    using sum_even
    by (simp only: Parity.semiring_parity_class.even_add)
qed

lemma normalized_lcm_two_inter_parity:
  assumes concrete: "p = PEven ∨ p = POdd"
      and accepts: "parity_accepts p r"
  shows
    "gamma_congruence_rep
       (normalize_congruence_rep (Some (r, lcm m 2))) =
     gamma_parity p ∩ {n. m dvd n - r}"
proof -
  have difference:
    "⋀n. 2 dvd n - r ⟷ parity_accepts p n"
    by (rule parity_accepts_difference[OF concrete accepts])
  show ?thesis
    unfolding gamma_normalized_lcm_two
    using difference by auto
qed

lemma even_modulus_disjoint_parity:
  assumes concrete: "p = PEven ∨ p = POdd"
      and modulus: "even m"
      and rejects: "¬ parity_accepts p c"
  shows "gamma_parity p ∩ {n. m dvd n - c} = {}"
proof (rule equals0I)
  fix n
  assume member:
    "n ∈ gamma_parity p ∩ {n. m dvd n - c}"
  from member have accepts: "parity_accepts p n"
    by simp
  from member have congruent: "m dvd n - c"
    by simp
  have same: "even n ⟷ even c"
    by (rule congruent_even_modulus[OF modulus congruent])
  from concrete rejects accepts same show False
    by (cases p) auto
```

```
qed

lemma parity_accepts_add_odd:
  assumes concrete: "p = PEven ∨ p = POdd"
      and rejects: "¬ parity_accepts p c"
      and modulus: "odd m"
  shows "parity_accepts p (c + m)"
  using concrete rejects modulus by (cases p) auto

fun restrict_congruence_rep_by_parity ::
  "parity => congruence_rep => congruence_rep" where
  "restrict_congruence_rep_by_parity PBot r = None"
| "restrict_congruence_rep_by_parity PTop r = r"
| "restrict_congruence_rep_by_parity p None = None"
| "restrict_congruence_rep_by_parity p (Some (c, m)) =
    (if m = 0 then
       if parity_accepts p c then Some (c, 0) else None
     else if even m ∧ ¬ parity_accepts p c then None
     else
       normalize_congruence_rep
         (Some
           (c + (if parity_accepts p c then 0 else m),
            lcm m 2)))"

lemma normalized_restrict_congruence_rep_by_parity:
  assumes "normalized_congruence_rep r"
  shows
    "normalized_congruence_rep
       (restrict_congruence_rep_by_parity p r)"
  using assms
  by (cases p; cases r)
     (auto simp only: restrict_congruence_rep_by_parity.simps
       normalized_congruence_rep.simps
       normalized_normalize_congruence_rep
       option.simps
       split: prod.splits if_splits)

lemma restrict_concrete_parity_correct:
  assumes concrete: "p = PEven ∨ p = POdd"
  shows
    "gamma_congruence_rep
       (restrict_congruence_rep_by_parity p (Some (c, m))) =
     gamma_parity p ∩ gamma_congruence_rep (Some (c, m))"
proof (cases "m = 0")
  case True
  with concrete show ?thesis
    by (cases p) auto
next
  case nonzero: False
  show ?thesis
  proof (cases "even m")
    case modulus: True
    show ?thesis
    proof (cases "parity_accepts p c")
      case accepts: True
      have reduced:
        "restrict_congruence_rep_by_parity p (Some (c, m)) =
         normalize_congruence_rep (Some (c, lcm m 2))"
        using concrete nonzero modulus accepts
        by (cases p) simp_all
      have exact:
        "gamma_congruence_rep
           (normalize_congruence_rep (Some (c, lcm m 2))) =
         gamma_parity p ∩ {n. m dvd n - c}"
        by (rule normalized_lcm_two_inter_parity[OF concrete accepts])
      show ?thesis
        unfolding reduced
        using exact by simp
    next
      case rejects: False
      have reduced:
        "restrict_congruence_rep_by_parity p (Some (c, m)) = None"
        using concrete nonzero modulus rejects
        by (cases p) simp_all
      have disjoint:
        "gamma_parity p ∩ {n. m dvd n - c} = {}"
```

```
      by (rule even_modulus_disjoint_parity[OF concrete modulus rejects])
    show ?thesis
      unfolding reduced
      using disjoint by simp
  qed
 next
  case modulus: False
  show ?thesis
  proof (cases "parity_accepts p c")
    case accepts: True
    have reduced:
      "restrict_congruence_rep_by_parity p (Some (c, m)) =
       normalize_congruence_rep (Some (c, lcm m 2))"
      using concrete nonzero modulus accepts
      by (cases p) simp_all
    have exact:
      "gamma_congruence_rep
         (normalize_congruence_rep (Some (c, lcm m 2))) =
       gamma_parity p ∩ {n. m dvd n - c}"
      by (rule normalized_lcm_two_inter_parity[OF concrete accepts])
    show ?thesis
      unfolding reduced
      using exact by simp
  next
    case rejects: False
    have odd_modulus: "odd m"
      using modulus by simp
    have shifted_accepts: "parity_accepts p (c + m)"
      by (rule parity_accepts_add_odd[OF concrete rejects odd_modulus])
    have reduced:
      "restrict_congruence_rep_by_parity p (Some (c, m)) =
       normalize_congruence_rep (Some (c + m, lcm m 2))"
      using concrete nonzero modulus rejects
      by (cases p) simp_all
    have exact:
      "gamma_congruence_rep
         (normalize_congruence_rep (Some (c + m, lcm m 2))) =
       gamma_parity p ∩ {n. m dvd n - (c + m)}"
      by (rule normalized_lcm_two_inter_parity[
            OF concrete shifted_accepts])
    show ?thesis
      unfolding reduced
      using exact by (simp only: gamma_congruence_rep.simps
          dvd_shift_modulus)
  qed
 qed
qed

lemma restrict_congruence_rep_by_parity_correct:
 assumes normalized: "normalized_congruence_rep r"
 shows
  "gamma_congruence_rep
     (restrict_congruence_rep_by_parity p r) =
   gamma_parity p ∩ gamma_congruence_rep r"
proof (cases p)
 case PBot
 then show ?thesis by simp
next
 case PTop
 then show ?thesis by simp
next
 case destination: PEven
 show ?thesis
 proof (cases r)
  case None
  with destination show ?thesis by simp
 next
  case (Some cm)
  obtain c m where cm: "cm = (c, m)"
    by (cases cm)
  have rep: "r = Some (c, m)"
    using Some cm by simp
  show ?thesis
    unfolding destination rep
    by (rule restrict_concrete_parity_correct) simp
 qed
```

```
next
  case destination: POdd
  show ?thesis
  proof (cases r)
    case None
    with destination show ?thesis by simp
  next
    case (Some cm)
    obtain c m where cm: "cm = (c, m)"
      by (cases cm)
    have rep: "r = Some (c, m)"
      using Some cm by simp
    show ?thesis
      unfolding destination rep
      by (rule restrict_concrete_parity_correct) simp
  qed
qed


lift_definition restrict_congruence_by_parity ::
  "parity => congruence => congruence"
  is restrict_congruence_rep_by_parity
  by (rule normalized_restrict_congruence_rep_by_parity)

declare restrict_congruence_by_parity.rep_eq [simp]

lemma gamma_restrict_congruence_by_parity [simp]:
  "gamma_congruence (restrict_congruence_by_parity p c) =
   gamma_parity p ∩ gamma_congruence c"
proof -
  have normalized:
    "normalized_congruence_rep (Rep_congruence c)"
    by simp
  have raw:
    "gamma_congruence_rep
       (restrict_congruence_rep_by_parity p (Rep_congruence c)) =
     gamma_parity p ∩
     gamma_congruence_rep (Rep_congruence c)"
    by (rule restrict_congruence_rep_by_parity_correct[OF normalized])
  show ?thesis
    unfolding gamma_congruence_def
    using raw by simp
qed

definition ivl_congruence_nonempty ::
  "ivl => congruence => bool" where
  "ivl_congruence_nonempty i c =
   ivl_congruence_rep_nonempty i (Rep_congruence c)"

lemma ivl_congruence_nonempty_correct:
  "ivl_congruence_nonempty i c ⟷
   gamma_ivl i ∩ gamma_congruence c ≠ {}"
proof -
  have normalized:
    "normalized_congruence_rep (Rep_congruence c)"
    by simp
  show ?thesis
    unfolding ivl_congruence_nonempty_def gamma_congruence_def
    by (rule ivl_congruence_rep_nonempty_correct[OF normalized])
qed

definition is_bottom_int_dom :: "'a int_dom_scheme => bool" where
  "is_bottom_int_dom d =
     (¬ ivl_congruence_nonempty
       (intersect_ivl
         (interval_fact_of_sign (int_sign d))
         (int_ivl d))
       (restrict_congruence_by_parity
         (int_parity d)
         (int_congruence d)))"

lemma is_bottom_int_dom_correct:
  "is_bottom_int_dom d ⟷ gamma_int_dom d = {}"
proof -
  let ?i =
    "intersect_ivl
```

```
      (interval_fact_of_sign (int_sign d))
      (int_ivl d)"
  let ?c =
    "restrict_congruence_by_parity
      (int_parity d)
      (int_congruence d)"
  have nonempty:
    "ivl_congruence_nonempty ?i ?c ⟷
     gamma_ivl ?i ∩ gamma_congruence ?c ≠ {}"
    by (rule ivl_congruence_nonempty_correct)
  have restricted:
    "gamma_congruence ?c =
     gamma_parity (int_parity d) ∩
     gamma_congruence (int_congruence d)"
    by simp
  have interval:
    "gamma_ivl ?i =
     gamma_sign (int_sign d) ∩ gamma_ivl (int_ivl d)"
    by (simp only: gamma_intersect_ivl_exact
          gamma_interval_fact_of_sign)
  show ?thesis
    unfolding is_bottom_int_dom_def gamma_int_dom_def
    using nonempty restricted interval by blast
qed
```

Semantic fullness, symmetric to `is_bottom_int_dom` above: `d = top` additionally pins the record's own `more` extension field to its top, which `gamma_int_dom` never inspects, so it is strictly stronger than `gamma_int_dom d = UNIV` at this instance's actual, extension-polymorphic `'a int_dom_scheme` type. Testing each of the four concrete components' own exact fullness instead reads off `gamma_int_dom`'s own definition directly, the same move `is_bottom_int_dom` already makes for emptiness.

```
definition is_top_int_dom :: "'a int_dom_scheme => bool" where
  "is_top_int_dom d ⟷
     is_top_sign (int_sign d) ∧ is_top_ivl (int_ivl d) ∧
     is_top_parity (int_parity d) ∧ is_top_congruence (int_congruence d)"

lemma is_top_int_dom_correct_gamma: "is_top_int_dom d ⟷ gamma_int_dom d = UNIV"
  unfolding is_top_int_dom_def gamma_int_dom_def
  using is_top_sign_correct_gamma is_top_ivl_correct_gamma
        is_top_parity_correct_gamma is_top_congruence_correct_gamma
  by auto

lemma gamma_int_dom_mono:
  assumes ab: "a ≤ b"
  shows "gamma_int_dom a ⊆ gamma_int_dom b"
proof -
  from ab have le:
    "int_sign a ≤ int_sign b"
    "int_ivl a ≤ int_ivl b"
    "int_parity a ≤ int_parity b"
    "int_congruence a ≤ int_congruence b"
    by (simp_all add: less_eq_int_dom_ext_def)
  have S: "gamma_sign (int_sign a) ⊆ gamma_sign (int_sign b)"
    using le(1) unfolding less_eq_sign_def by (rule gamma_sign_mono)
  have I: "gamma_ivl (int_ivl a) ⊆ gamma_ivl (int_ivl b)"
    using le(2) by (rule gamma_ivl_mono)
  have P: "gamma_parity (int_parity a) ⊆ gamma_parity (int_parity b)"
    using le(3) unfolding less_eq_parity_def by (rule gamma_parity_mono)
  have C:
    "gamma_congruence (int_congruence a) ⊆
     gamma_congruence (int_congruence b)"
    using le(4)
    unfolding less_eq_congruence_iff_gamma .
  show ?thesis
    unfolding gamma_int_dom_def using S I P C by auto
qed
```

Constants and printing

`int_dom_of_int` pins every component to one integer, so it concretizes to exactly that singleton; the printer follows Goblint's tuple format.

```
definition int_dom_of_int :: "int => int_dom" where
  "int_dom_of_int n =
     (top :: int_dom)⦇
       int_sign := sign_of_int n,
```

```
      int_ivl := Ivl (Fin n) (Fin n),
      int_parity := parity_of_int n,
      int_congruence := congruence_of_int n
    ⦈"

lemma gamma_int_dom_of_int [simp]:
  "gamma_int_dom (int_dom_of_int n) = {n}"
  by (auto simp: int_dom_of_int_def gamma_int_dom_def
       sign_of_int_gamma parity_of_int_gamma)
```

Goblint's integer-domain tuple prints a value one of its components pins to a single integer as that integer, a value no component constrains as $\top$, and anything else as its components, each named and separated by `;`. The bottom test runs first, so a component that concretizes to one integer pins a live product to exactly that integer.

```
fun ivl_constant :: "ivl ⇒ int option" where
    "ivl_constant (Ivl (Fin l) (Fin u)) = (if l = u then Some l else None)"
  | "ivl_constant _ = None"

definition congruence_constant :: "congruence ⇒ int option" where
  "congruence_constant c =
     (case Rep_congruence c of
        Some (r, m) ⇒ if m = 0 then Some r else None
      | None ⇒ None)"

definition int_dom_constant :: "'a int_dom_scheme ⇒ int option" where
  "int_dom_constant d =
     (if int_sign d = SZero then Some 0
      else case ivl_constant (int_ivl d) of
        Some n ⇒ Some n
      | None ⇒ congruence_constant (int_congruence d))"

lemma gamma_ivl_constant:
  "ivl_constant i = Some n ⟹ gamma_ivl i = {n}"
  by (induction i rule: ivl_constant.induct) (auto split: if_splits)

lemma gamma_congruence_constant:
  "congruence_constant c = Some n ⟹ gamma_congruence c = {n}"
  unfolding congruence_constant_def gamma_congruence_def
  by (auto split: option.splits if_splits)
```

Printing a live product as the integer `int_dom_constant` names loses nothing: that integer is all the product denotes.

```
lemma gamma_int_dom_constant_subset:
  "int_dom_constant d = Some n ⟹ gamma_int_dom d ⊆ {n}"
  using gamma_ivl_constant gamma_congruence_constant
  unfolding int_dom_constant_def gamma_int_dom_def
  by (auto split: if_splits option.splits)

definition string_of_int_dom :: "'a int_dom_scheme ⇒ String.literal" where
  "string_of_int_dom d =
     (if is_bottom_int_dom d then sym_bottom
      else if is_top_int_dom d then sym_top
      else case int_dom_constant d of
        Some n ⇒ string_of_int n
      | None ⇒
          STR ''signs:'' + string_of_sign (int_sign d)
        + STR ''; intervals:'' + string_of_ivl (int_ivl d)
        + STR ''; parities:'' + string_of_parity (int_parity d)
        + STR ''; congruences:'' + string_of_congruence (int_congruence d))"

end
```

Chapter 3

Voblint_Solver

3.1 Strategy_Program_Fold

```
theory Strategy_Program_Fold
  imports Strategy_Tree_Program
begin
```

Folding a right-hand side from contribution programs

An equation's right-hand side is assembled from a list of contributions — one per incoming contribution, such as an intra predecessor, a call return, or a routed activation hook — joined into one answer. Each contribution is a *program*, not a finished tree: the fold sequences them with $\gg=$, joining each answer into a running accumulator, and the result is again a program. Nothing is compiled here. A generator compiles once, at the end, where the solver takes over.

That is why the characterizations below can speak one vocabulary on both sides. `traverse_program`, `sides_of_program` and `dep_program` ask a contribution the same questions they ask the fold, and each hypothesis `∀p ∈ set ps. sp_wf p` is what makes asking them of a program well posed at all: an arbitrary program may ignore or duplicate its continuation, and then what it publishes would depend on what follows it.

The fold never emits a `Side` itself: a fold of Side-free contributions stays Side-free, which is what lets a generator publish once, after the whole fold, instead of once per contribution — several updates to one key during a single right-hand-side evaluation otherwise change what the vendored solver's per-origin update rule widens against.

What a caller varies is *how much of each answer accumulates*. The fold therefore takes a projection `prj` into the accumulator and an embedding `emb` back out. At the identity pair the whole answer accumulates. A D/G instance instead projects the local half into the accumulator and embeds the result as `DG d bot`; the global half of a contribution simply does not reach the node's answer, and a contribution that has one to share must publish it itself through `Side`. Nothing here makes that happen.

The two parameters are used as a projection and a final embedding, but the fold assumes no inverse or homomorphism law relating them. Order preservation is required only where it is actually used, by the environment-monotonicity results; the publication and dependency characterizations hold for arbitrary `prj` and `emb`.

Join-folds over a list

`⨆x←xs. f x` joins `f x` over the list `xs`; the empty join is ⊥. It is built like HOL's `sum_list`: `join_list` is the list fold of the commutative monoid `(⊔, ⊥)`, and the binder maps before it folds.

```
context bounded_semilattice_sup_bot
begin

sublocale join_list: comm_monoid_list sup bot
  defines join_list = join_list.F ..

end

open_bundle join_list_syntax
begin

syntax
  "_join_list" :: "pttrn ⇒ 'a list ⇒ 'b ⇒ 'b"
    (‹(‹indent=3 notation=‹binder ⨆››⨆_←_. _)› [0, 51, 10] 10)
syntax_consts
  "_join_list" == join_list
translations
  "⨆x←xs. b" == "CONST join_list (CONST map (λx. b) xs)"

end
```

Every characterization below states the fold as a list join over already-computed values, so its consumers keep needing the same handful of facts about that: the seed of a right fold can be moved out, a bound against the join is a bound against every element, one element is below the join, only the element set matters, a pointwise bound lifts, two lists that agree elementwise agree, and an all-⊥ join is ⊥.

They are collected here, ahead of the fold they serve, rather than restated per generator. `foldr_sup_seed_swap` is the one with content — it needs only `semilattice_sup` — and `foldr_join_list` and `foldr_join_seed_out` read a right fold with a ⊥ or an arbitrary seed as a list join.

```
lemma foldr_sup_seed_swap:
  fixes h :: "'a ⇒ 'b::semilattice_sup"
  shows "foldr (λt acc'. h t ⊔ acc') ts (acc ⊔ x) = x ⊔ foldr (λt acc'. h t ⊔ acc') ts acc"
  by (induction ts arbitrary: acc) (simp_all add: sup_assoc sup_commute sup_left_commute)
```

```
lemma foldr_join_list:
  fixes h :: "'a ⇒ 'b::bounded_semilattice_sup_bot"
  shows "foldr (λt acc'. h t ⊔ acc') ts bot = (⨆t←ts. h t)"
  by (induction ts) simp_all

lemma foldr_join_seed_out:
  fixes h :: "'a ⇒ 'b::bounded_semilattice_sup_bot"
  shows "foldr (λt acc'. h t ⊔ acc') ts a = a ⊔ (⨆t←ts. h t)"
  using foldr_sup_seed_swap[of h ts bot a] by (simp add: foldr_join_list)

lemma foldr_sup_le_iff [simp]:
  fixes h :: "'a ⇒ 'b::bounded_semilattice_sup_bot"
  shows "(⨆t←xs. h t) ≤ y ⟷ (∀x ∈ set xs. h x ≤ y)"
  by (induction xs) auto

lemma foldr_sup_member_le [intro]:
  fixes h :: "'a ⇒ 'b::bounded_semilattice_sup_bot"
  assumes "x ∈ set xs"
  shows "h x ≤ (⨆t←xs. h t)"
  using assms by (induction xs) (auto intro: le_supI2)

lemma foldr_sup_mono:
  fixes f g :: "'a ⇒ 'b::bounded_semilattice_sup_bot"
  assumes "⋀x. x ∈ set xs ⟹ f x ≤ g x"
  shows "(⨆t←xs. f t) ≤ (⨆t←xs. g t)"
  using assms by (auto intro: order_trans[OF _ foldr_sup_member_le])

lemma foldr_sup_set_cong:
  fixes h :: "'a ⇒ 'b::bounded_semilattice_sup_bot"
  assumes eq: "set xs = set ys"
  shows "(⨆t←xs. h t) = (⨆t←ys. h t)"
  by (rule order_antisym) (auto simp: eq)
```

Two lists whose elements agree under their respective observations have the same join. This is what lets a correspondence proof relate two generators' contribution lists elementwise instead of reasoning about the folds themselves.

```
lemma foldr_sup_list_all2_cong:
  fixes f :: "'a ⇒ 'c::bounded_semilattice_sup_bot"
  assumes "list_all2 (λx y. f x = g y) xs ys"
  shows "(⨆x←xs. f x) = (⨆y←ys. g y)"
  using assms by (induction rule: list_all2_induct) simp_all
```

The two ways such a correspondence is built and consumed: establishing it between two maps over one list, and reading it as an equality of unions. Neither mentions a fold, a tree, or a graph.

```
lemma list_all2_map_diag:
  "(⋀x. x ∈ set xs ⟹ P (f x) (g x)) ⟹ list_all2 P (map f xs) (map g xs)"
  by (induction xs) simp_all

lemma list_all2_Union_eq:
  assumes "list_all2 (λa b. f a = g b) xs ys"
  shows "(⋃x∈set xs. f x) = (⋃y∈set ys. g y)"
  using assms by (induction rule: list_all2_induct) auto

lemma foldr_sup_bot_of_all_bot:
  fixes L :: "'a list" and h :: "'a ⇒ 'b::bounded_semilattice_sup_bot"
  assumes "⋀x. x ∈ set L ⟹ h x = bot"
  shows "(⨆x←L. h x) = bot"
  using assms by (induction L) simp_all
```

Folding contributions into one right-hand side

Several contribution programs, one right-hand side: run each in list order, join its answer into the accumulator, and answer with the accumulator. `prj` and `emb` keep the fold generic in which part of an answer accumulates – that is what later lets the D/G layer join the local half while the global half travels as a side effect.

```
fun fold_rhs_program_projected ::
  "('a ⇒ 'b) ⇒ ('b ⇒ 'a) ⇒ 'b::semilattice_sup
   ⇒ ('k, 'g, 'a, 'a) strategy_program list
   ⇒ ('k, 'g, 'a, 'a) strategy_program"
where
  "fold_rhs_program_projected prj emb acc [] = sp_return (emb acc)"
| "fold_rhs_program_projected prj emb acc (p # ps) =
     do {
       res ← p;
```

```
      fold_rhs_program_projected prj emb (acc ⊔ prj res) ps
    }"
```

The accumulator read declaratively off a valuation, rather than run. Every answer characterization below is stated through it.

```
fun fold_acc_projected ::
  "('a::bounded_semilattice_sup_bot ⇒ 'b::semilattice_sup) ⇒ 'b
   ⇒ ('k + 'g ⇒ 'a) ⇒ ('k, 'g, 'a, 'a) strategy_program list ⇒ 'b"
where
  "fold_acc_projected prj acc σ [] = acc"
| "fold_acc_projected prj acc σ (p # ps) =
    fold_acc_projected prj (acc ⊔ prj (traverse_program p σ)) σ ps"
```

Declarative characterizations

Contributions are traversed in list order: each answer joins the running accumulator before the next program runs. The publication and dependency characterizations forget that order, the underlying operation being commutative — a set union for `dep_aux`, a join seeded at `bot` for `sides_of_rhs` — and neither sees `prj` or `emb` at all: what a fold publishes and reads is its contributions' business, not its accumulator's. That is also why both are independent of the accumulator it starts from.

```
theorem traverse_fold_rhs_program_projected_char:
  assumes "∀p ∈ set ps. sp_wf p"
  shows "traverse_program (fold_rhs_program_projected prj emb acc ps) σ
     = emb (fold_acc_projected prj acc σ ps)"
  using assms
  by (induction ps arbitrary: acc)
     (simp_all add: sp_compile_def sp_compile_with_bind traverse_rhs_sp_wf)

theorem sides_of_program_fold_rhs_program_projected_char:
  fixes ps :: "('x, 'g, 'd::bounded_semilattice_sup_bot, 'd) strategy_program list"
  assumes "∀p ∈ set ps. sp_wf p"
  shows "sides_of_program (fold_rhs_program_projected prj emb acc ps) σ z
     = (⨆p←ps. sides_of_program p σ z)"
  using assms
  by (induction ps arbitrary: acc)
     (auto simp add: sp_compile_def sp_compile_with_bind sides_of_rhs_sp_wf traverse_rhs_sp_wf)

theorem dep_program_fold_rhs_program_projected_char:
  assumes "∀p ∈ set ps. sp_wf p"
  shows "dep_program σ (fold_rhs_program_projected prj emb acc ps)
     = (⋃p∈set ps. dep_program σ p)"
  using assms
  by (induction ps arbitrary: acc)
     (auto simp add: sp_compile_def sp_compile_with_bind dep_aux_sp_wf traverse_rhs_sp_wf)

lemma fold_acc_projected_as_foldr:
  fixes prj :: "'a::bounded_semilattice_sup_bot ⇒ 'b::bounded_semilattice_sup_bot"
  shows "fold_acc_projected prj acc σ ps
           = acc ⊔ (⨆p←ps. prj (traverse_program p σ))"
  by (induction ps arbitrary: acc) (simp_all add: sup_assoc)

lemma fold_acc_projected_acc_mono:
  fixes prj :: "'a::bounded_semilattice_sup_bot ⇒ 'b::bounded_semilattice_sup_bot"
  assumes "acc1 ≤ acc2"
  shows "fold_acc_projected prj acc1 σ ts ≤ fold_acc_projected prj acc2 σ ts"
  using assms by (simp add: fold_acc_projected_as_foldr le_supI1)
```

Monotonicity in the *environment* rather than the accumulator. This is where `prj` and `emb` stop being arbitrary: a fold inherits its contributions' monotonicity only if the two also preserve order. Publications and dependencies need no such hypothesis, since neither passes through them.

```
lemma fold_acc_projected_env_mono:
  fixes prj :: "'a::bounded_semilattice_sup_bot ⇒ 'b::bounded_semilattice_sup_bot"
  assumes prj_mono: "mono prj"
    and progs_mono: "⋀p. p ∈ set ps ⟹ traverse_program p σ1 ≤ traverse_program p σ2"
  shows "fold_acc_projected prj acc σ1 ps ≤ fold_acc_projected prj acc σ2 ps"
  unfolding fold_acc_projected_as_foldr
  by (rule sup_mono[OF order_refl])
     (rule foldr_sup_mono, rule monoD[OF prj_mono, OF progs_mono])

lemma traverse_fold_rhs_program_projected_mono:
  fixes prj :: "'a::bounded_semilattice_sup_bot ⇒ 'b::bounded_semilattice_sup_bot"
  assumes wf: "∀p ∈ set ps. sp_wf p"
```

```
    and prj_mono: "mono prj"
    and emb_mono: "mono emb"
    and progs_mono: "⋀p. p ∈ set ps ⟹ traverse_program p σ1 ≤ traverse_program p σ2"
  shows "traverse_program (fold_rhs_program_projected prj emb acc ps) σ1
       ≤ traverse_program (fold_rhs_program_projected prj emb acc ps) σ2"
  unfolding traverse_fold_rhs_program_projected_char[OF wf]
  by (rule monoD[OF emb_mono])
     (rule fold_acc_projected_env_mono[OF prj_mono progs_mono])

lemma sides_of_program_fold_rhs_program_projected_acc_indep:
  fixes ps :: "('x, 'g, 'd::bounded_semilattice_sup_bot, 'd) strategy_program list"
  assumes wf: "∀p ∈ set ps. sp_wf p"
  shows "sides_of_program (fold_rhs_program_projected prj emb acc1 ps) σ
       = sides_of_program (fold_rhs_program_projected prj emb acc2 ps) σ"
  by (simp add: fun_eq_iff sides_of_program_fold_rhs_program_projected_char[OF wf])

lemma dep_program_fold_rhs_program_projected_acc_indep:
  assumes wf: "∀p ∈ set ps. sp_wf p"
  shows "dep_program σ (fold_rhs_program_projected prj emb acc1 ps)
     = dep_program σ (fold_rhs_program_projected prj emb acc2 ps)"
  by (simp add: dep_program_fold_rhs_program_projected_char[OF wf])
```

Accumulating the whole answer

The identity instance: nothing is projected away, so the accumulator has the answer type and every contribution joins into the node's answer whole.

```
lemma sp_wf_fold_rhs_program_projected [intro]:
  "∀p ∈ set ps. sp_wf p ⟹ sp_wf (fold_rhs_program_projected prj emb acc ps)"
  by (induction ps arbitrary: acc) auto

definition fold_rhs_program ::
  "'a::semilattice_sup
   ⇒ ('k, 'g, 'a, 'a) strategy_program list
   ⇒ ('k, 'g, 'a, 'a) strategy_program"
where
  "fold_rhs_program acc ps = fold_rhs_program_projected id id acc ps"

lemma sp_wf_fold_rhs_program [intro]:
  "∀p ∈ set ps. sp_wf p ⟹ sp_wf (fold_rhs_program acc ps)"
  unfolding fold_rhs_program_def by (rule sp_wf_fold_rhs_program_projected)

lemma fold_rhs_program_simps [simp, code]:
  "fold_rhs_program acc [] = sp_return acc"
  "fold_rhs_program acc (p # ps) =
     do { res ← p; fold_rhs_program (acc ⊔ res) ps }"
  by (simp_all add: fold_rhs_program_def)
```

The answer as a right fold. `fold_rhs_program` threads the accumulator in evaluation order, which is a left fold; proofs about a node's contributions are usually already phrased as a right fold, and `traverse_rhs` only ever joins, so the two agree.

```
theorem traverse_fold_rhs_program_char_foldr:
  assumes "∀p ∈ set ps. sp_wf p"
  shows "traverse_program (fold_rhs_program acc ps) σ
     = foldr (λp acc'. traverse_program p σ ⊔ acc') ps acc"
  using assms
  by (induction ps arbitrary: acc)
     (simp_all add: sp_compile_def sp_compile_with_bind foldr_sup_seed_swap
        traverse_rhs_sp_wf)

theorem sides_of_program_fold_rhs_program_char:
  fixes ps :: "('x,'g,'d::bounded_semilattice_sup_bot,'d) strategy_program list"
  assumes wf: "∀p ∈ set ps. sp_wf p"
  shows "sides_of_program (fold_rhs_program acc ps) σ z
     = (⨆p←ps. sides_of_program p σ z)"
  unfolding fold_rhs_program_def
  by (rule sides_of_program_fold_rhs_program_projected_char[OF wf])

theorem dep_program_fold_rhs_program_char:
  assumes wf: "∀p ∈ set ps. sp_wf p"
  shows "dep_program σ (fold_rhs_program acc ps) = (⋃p∈set ps. dep_program σ p)"
  unfolding fold_rhs_program_def
  by (rule dep_program_fold_rhs_program_projected_char[OF wf])
```

Dependency-property preservation

A fold of programs whose query set is (respectively environment-independent, monotone in the environment) is itself environment-independent (monotone) – the list-level analogue of `env_indep_deps`/`mono_tree_deps`'s own single-tree closure facts, and what lets a generator's whole-node dependency obligation reduce to a per-hook one.

Both read straight off `dep_program_fold_rhs_program_projected_char`: a fold's dependencies are the union of its contributions', so whatever closure property each contribution has, the union inherits.

```
lemma env_indep_deps_fold_rhs_program_projected:
  assumes wf: "∀p ∈ set ps. sp_wf p"
    and "⋀p. p ∈ set ps ⟹ env_indep_deps (sp_compile p)"
  shows "env_indep_deps (sp_compile (fold_rhs_program_projected prj emb acc ps))"
  using assms
  unfolding env_indep_deps_def
  by (auto simp add: dep_program_fold_rhs_program_projected_char[OF wf]; blast)

end
```

3.2 Strategy_Tree_Program

```
theory Strategy_Tree_Program
  imports "TD.Basics_side" "HOL-Library.Monad_Syntax"
begin
```

A typed frontend over the homogeneous vendor tree

`strategy_tree` fixes query results and terminal answers to the same carrier `'d` and therefore does not support a polymorphic bind. `strategy_program` supplies a continuation-based typed interface: a program may produce any result type `'a` while its generated vendor tree continues to use `'d` for solver values and contributions. `sp_compile_with` encodes the final result into `'d` and produces the vendor tree consumed by the solver.

```
type_synonym ('x,'g,'d,'a) strategy_program =
  "('a ⇒ ('x,'g,'d) strategy_tree) ⇒ ('x,'g,'d) strategy_tree"

definition sp_return :: "'a ⇒ ('x,'g,'d,'a) strategy_program" where
  "sp_return a k = k a"

definition sp_bind ::
  "('x,'g,'d,'a) strategy_program
   ⇒ ('a ⇒ ('x,'g,'d,'b) strategy_program)
   ⇒ ('x,'g,'d,'b) strategy_program" where
  "sp_bind m f k = m (λv. f v k)"

definition sp_read_local :: "'x ⇒ ('x,'g,'d,'d) strategy_program" where
  "sp_read_local x k = QueryL x k"

definition sp_read_global :: "'g ⇒ ('x,'g,'d,'d) strategy_program" where
  "sp_read_global g k = QueryG g k"

definition sp_publish :: "'g ⇒ 'd ⇒ ('x,'g,'d,unit) strategy_program" where
  "sp_publish g d k = Side g d (k ())"
```

`sp_read_at` reads an unknown in the solver's combined address space `'x + 'g`, dispatching `Inl` to `sp_read_local` and `Inr` to `sp_read_global` instead of fixing the constructor at the call site.

```
fun sp_read_at :: "'x + 'g ⇒ ('x,'g,'d,'d) strategy_program" where
  "sp_read_at (Inl x) = sp_read_local x"
| "sp_read_at (Inr g) = sp_read_global g"

adhoc_overloading Monad_Syntax.bind == sp_bind
```

`sp_read_local`/`sp_read_global`/`sp_publish` build a program one effect at a time. `sp_lift_tree` instead embeds an already-built vendor tree – the shape a backend combinator produces by folding several contribution trees into one – as the program that runs it and continues, by recursing over the tree's own constructors directly: an `Answer` becomes a pure result, a `QueryL`/`QueryG` becomes a read continuing into the lifted subtree, a `Side` becomes a published contribution continuing into the lifted subtree. Defined directly against the constructors, its primitive-recursion equations are already the normal form needed by proofs about vendor trees.

```
primrec sp_lift_tree :: "('x,'g,'d) strategy_tree ⇒ ('x,'g,'d,'d) strategy_program" where
  "sp_lift_tree (Answer d)   k = k d"
| "sp_lift_tree (QueryL x f) k = QueryL x (λd. sp_lift_tree (f d) k)"
| "sp_lift_tree (QueryG g f) k = QueryG g (λd. sp_lift_tree (f d) k)"
| "sp_lift_tree (Side g d t) k = Side g d (sp_lift_tree t k)"

lemma traverse_rhs_sp_lift_tree [simp]:
  "traverse_rhs (sp_lift_tree t k) σ = traverse_rhs (k (traverse_rhs t σ)) σ"
  by (induction t arbitrary: k) (auto intro: rangeI)

lemma dep_aux_sp_lift_tree [simp]:
  "dep_aux σ (sp_lift_tree t k) = dep_aux σ t ∪ dep_aux σ (k (traverse_rhs t σ))"
  by (induction t arbitrary: k) (auto intro: rangeI)

lemma sides_of_rhs_sp_lift_tree [simp]:
  fixes t :: "('x, 'g, 'd::bounded_semilattice_sup_bot) strategy_tree"
  shows "sides_of_rhs (sp_lift_tree t k) σ
         = sides_of_rhs t σ ⊔ sides_of_rhs (k (traverse_rhs t σ)) σ"
  by (induction t arbitrary: k) (auto simp: Let_def sup_fun_def fun_upd_def ac_simps intro: rangeI)
```

`traverse_rhs_sp_lift_tree_mono`: environment-monotonicity of the answer `sp_lift_tree` produces, not of its dependency set – if `t` is monotone in the environment, every continuation `k v` is monotone in the environment, and `k` is monotone in the value it receives, then `sp_lift_tree t k` is monotone in the environment.

```
lemma traverse_rhs_sp_lift_tree_mono:
```

```
  fixes t :: "('x, 'g, 'd::{order,bot}) strategy_tree"
  assumes t_mono:
    "⋀σ1 σ2. σ1 ≤ σ2 ⟹ traverse_rhs t σ1 ≤ traverse_rhs t σ2"
  assumes k_mono_env:
    "⋀v σ1 σ2. σ1 ≤ σ2 ⟹ traverse_rhs (k v) σ1 ≤ traverse_rhs (k v) σ2"
  assumes k_mono_val:
    "⋀σ v1 v2. v1 ≤ v2 ⟹ traverse_rhs (k v1) σ ≤ traverse_rhs (k v2) σ"
  assumes le: "σ1 ≤ σ2"
  shows "traverse_rhs (sp_lift_tree t k) σ1 ≤ traverse_rhs (sp_lift_tree t k) σ2"
proof -
  have "traverse_rhs (k (traverse_rhs t σ1)) σ1
      ≤ traverse_rhs (k (traverse_rhs t σ1)) σ2"
    by (rule k_mono_env[OF le])
  also have "...
      ≤ traverse_rhs (k (traverse_rhs t σ2)) σ2"
    by (rule k_mono_val[OF t_mono[OF le]])
  finally show ?thesis
    by simp
qed
```

`mono_tree_deps` gives the generally applicable composition rule: the continuation may query more unknowns as its received value grows. `env_indep_deps` is also preserved, under the stronger condition that the continuation's dependency set is independent of both its argument and the environment.

`sp_compile` only accepts a program whose answer is already `'d`. `sp_compile_with encode` generalizes that to any `'a`, packing the final answer through `encode`; `sp_compile` is its `id` specialization.

```
definition sp_compile_with ::
  "('a ⇒ 'd) ⇒ ('x,'g,'d,'a) strategy_program ⇒ ('x,'g,'d) strategy_tree" where
  "sp_compile_with encode p = p (Answer o encode)"

definition sp_compile :: "('x,'g,'d,'d) strategy_program ⇒ ('x,'g,'d) strategy_tree" where
  "sp_compile p = sp_compile_with id p"
```

Compiling programs to vendor trees

One rewrite per program constructor, each `simp`, so compiling a concrete program is normalization rather than proof: the simplifier builds the tree from the program's own syntax, and a transfer written against the program layer never has to name a tree constructor.

```
lemma sp_compile_with_bind_read_local [simp]:
  "sp_compile_with encode ((sp_read_local x) ≫= f) = QueryL x (λd. sp_compile_with encode (f d))"
  by (simp add: sp_compile_with_def sp_bind_def sp_read_local_def)

lemma sp_compile_with_bind_read_global [simp]:
  "sp_compile_with encode ((sp_read_global g) ≫= f) = QueryG g (λd. sp_compile_with encode (f d))"
  by (simp add: sp_compile_with_def sp_bind_def sp_read_global_def)

lemma sp_compile_with_bind_publish [simp]:
  "sp_compile_with encode ((sp_publish g d) ≫= f) = Side g d (sp_compile_with encode (f ()))"
  by (simp add: sp_compile_with_def sp_bind_def sp_publish_def)

lemma sp_compile_with_return [simp]:
  "sp_compile_with encode (sp_return a) = Answer (encode a)"
  by (simp add: sp_compile_with_def sp_return_def)

lemma sp_compile_with_read_local [simp]:
  "sp_compile_with encode (sp_read_local x) = QueryL x (Answer o encode)"
  by (simp add: sp_compile_with_def sp_read_local_def)

lemma sp_compile_with_read_global [simp]:
  "sp_compile_with encode (sp_read_global g) = QueryG g (Answer o encode)"
  by (simp add: sp_compile_with_def sp_read_global_def)

lemma sp_compile_with_publish [simp]:
  "sp_compile_with encode (sp_publish g d) = Side g d (Answer (encode ()))"
  by (simp add: sp_compile_with_def sp_publish_def)

lemma sp_compile_with_bind_read_at [simp]:
  "sp_compile_with encode ((sp_read_at src) ≫= f) =
     (case src of Inl x ⇒ QueryL x (λd. sp_compile_with encode (f d))
                | Inr g ⇒ QueryG g (λd. sp_compile_with encode (f d)))"
  by (cases src) simp_all

lemma sp_compile_with_read_at [simp]:
  "sp_compile_with encode (sp_read_at src) =
```

```
    (case src of Inl x ⇒ QueryL x (Answer o encode) | Inr g ⇒ QueryG g (Answer o encode))"
  by (cases src) simp_all

lemma sp_compile_sp_lift_tree [simp]: "sp_compile (sp_lift_tree t) = t"
  by (induction t) (simp_all add: sp_compile_def sp_compile_with_def)
```

Programs that run their continuation once

A `('x, 'g, 'd, 'a) strategy_program` is any function from a continuation to a tree, so nothing stops one from ignoring its continuation, or from using it twice. Such a program has no semantics the tree vocabulary can describe: what it publishes and what it reads would depend on the continuation it is handed.

`sp_wf p` names the programs that do not do this. It says `p` commutes with grafting: continuing into `k` and then grafting `h` onto every leaf of the result is the same as continuing into `k` already grafted. A program that dropped its continuation would lose `h` on the left and keep it on the right; one that used it twice would duplicate the grafting. Everything the generator builds runs its continuation once, and this is the condition under which asking a program what it publishes is a well-posed question at all.

Stating it this way rather than as `p = sp_lift_tree (sp_compile p)` is what lets it apply to a transfer, whose answer type is the analysis' own `'dl` rather than the solver's `'d`: there is no tree whose leaves carry a `'dl`, so a transfer has no compilation to be recoverable from, but it commutes with grafting all the same. For a program that does answer `'d` the two agree, which is `sp_wfD`.

```
lemma sp_lift_tree_sp_lift_tree:
  "sp_lift_tree (sp_lift_tree t g) k = sp_lift_tree t (λv. sp_lift_tree (g v) k)"
  by (induction t) simp_all

definition sp_wf :: "('x,'g,'d,'a) strategy_program ⇒ bool" where
  "sp_wf p ⟷ (∀k h. p (λv. sp_lift_tree (k v) h) = sp_lift_tree (p k) h)"

lemma sp_wfI:
  "(⋀k h. p (λv. sp_lift_tree (k v) h) = sp_lift_tree (p k) h) ⟹ sp_wf p"
  by (simp add: sp_wf_def)

lemma sp_wf_graft:
  "sp_wf p ⟹ p (λv. sp_lift_tree (k v) h) = sp_lift_tree (p k) h"
  by (simp add: sp_wf_def)
```

The homogeneous reading: a program answering the solver's own value type is recoverable from the tree it compiles to. This is the form every consumer below uses, and `sp_compile` only accepts such a program in the first place.

```
lemma sp_wfD:
  assumes "sp_wf p"
  shows "p k = sp_lift_tree (sp_compile p) k"
  using sp_wf_graft[OF assms, where k = Answer and h = k]
  by (simp add: sp_compile_def sp_compile_with_def)
```

Closure under the constructors a generator actually builds with, so a well-formedness side condition discharges by `simp` at the use site instead of being threaded through as a hypothesis.

```
lemma sp_wf_lift_tree [intro, simp]: "sp_wf (sp_lift_tree t)"
  by (simp add: sp_wf_def sp_lift_tree_sp_lift_tree)

lemma sp_wf_return [intro, simp]: "sp_wf (sp_return a)"
  by (simp add: sp_wf_def sp_return_def)

lemma sp_wf_read_local [intro, simp]: "sp_wf (sp_read_local x)"
  by (simp add: sp_wf_def sp_read_local_def)

lemma sp_wf_read_global [intro, simp]: "sp_wf (sp_read_global g)"
  by (simp add: sp_wf_def sp_read_global_def)

lemma sp_wf_publish [intro, simp]: "sp_wf (sp_publish g d)"
  by (simp add: sp_wf_def sp_publish_def)

lemma sp_wf_read_at [intro, simp]: "sp_wf (sp_read_at src)"
  by (cases src) simp_all

lemma sp_wf_bind [intro]:
  assumes "sp_wf p" and "⋀v. sp_wf (f v)"
  shows "sp_wf (p ⤜ f)"
proof (rule sp_wfI)
  fix k h
  have "(p ⤜ f) (λv. sp_lift_tree (k v) h) = p (λv. sp_lift_tree (f v k) h)"
```

```
    by (simp add: sp_bind_def sp_wf_graft[OF assms(2)])
  also have "... = sp_lift_tree (p (λv. f v k)) h"
    by (rule sp_wf_graft[OF assms(1)])
  finally show "(p ⤜ f) (λv. sp_lift_tree (k v) h) = sp_lift_tree ((p ⤜ f) k) h"
    by (simp add: sp_bind_def)
qed
```

Post-composing a program's answer with an encoding, so a transfer answering `'dl` becomes a contribution answering the packed `('dl,'dg) dg_state` the generator folds. It is `sp_compile_with`'s program-level counterpart: compiling it is compiling through the same encoding.

```
definition sp_map ::
  "('a ⇒ 'b) ⇒ ('x,'g,'d,'a) strategy_program ⇒ ('x,'g,'d,'b) strategy_program"
where
  "sp_map e p = (λk. p (λv. k (e v)))"

lemma sp_map_apply [simp]: "sp_map e p k = p (λv. k (e v))"
  by (simp add: sp_map_def)

lemma sp_compile_with_sp_map [simp]:
  "sp_compile_with e' (sp_map e p) = sp_compile_with (e' ∘ e) p"
  by (simp add: sp_map_def sp_compile_with_def comp_def)

lemma sp_compile_sp_map [simp]: "sp_compile (sp_map e p) = sp_compile_with e p"
  by (simp add: sp_compile_def sp_compile_with_def comp_def)

lemma sp_wf_map [intro, simp]: "sp_wf p ⟹ sp_wf (sp_map e p)"
  by (simp add: sp_wf_def)
```

What a well-formed program answers, publishes and reads, asked of the program rather than of a tree. Each is the tree vocabulary applied to its compilation, so the three carry over unchanged — and the point of stating them is that a characterization of a combinator can now speak one vocabulary on both sides, instead of a program on the left and its contributions' trees on the right.

```
abbreviation traverse_program ::
  "('x,'g,'d::bot,'d) strategy_program ⇒ ('x + 'g ⇒ 'd) ⇒ 'd"
  where "traverse_program p σ ≡ traverse_rhs (sp_compile p) σ"

abbreviation sides_of_program ::
  "('x,'g,'d::bounded_semilattice_sup_bot,'d) strategy_program
   ⇒ ('x + 'g ⇒ 'd) ⇒ 'x + 'g ⇒ 'd"
  where "sides_of_program p σ ≡ sides_of_rhs (sp_compile p) σ"

abbreviation dep_program ::
  "('x + 'g ⇒ 'd::bot) ⇒ ('x,'g,'d,'d) strategy_program ⇒ ('x + 'g) set"
  where "dep_program σ p ≡ dep_aux σ (sp_compile p)"
```

Running a well-formed program under any continuation: its own effects happen, then the continuation runs on the value it answered.

Left bare. Their left-hand sides are `f (p k) σ` with both `p` and `k` schematic, which is not a higher-order pattern: as `simp` rules they match any application under one of the three observers and spawn an `sp_wf` subgoal for it — including their own right-hand sides, where `sides_of_program p σ` unfolds to `sides_of_rhs (sp_compile p) σ` and matches at `?p := sp_compile`. Nothing loops while `sp_wf sp_compile` is unprovable by the simpset, but that is an accident to rely on, so each use cites the rule.

```
lemma traverse_rhs_sp_wf:
  assumes "sp_wf p"
  shows "traverse_rhs (p k) σ = traverse_rhs (k (traverse_program p σ)) σ"
  by (subst sp_wfD[OF assms]) simp

lemma dep_aux_sp_wf:
  assumes "sp_wf p"
  shows "dep_aux σ (p k) = dep_program σ p ∪ dep_aux σ (k (traverse_program p σ))"
  by (subst sp_wfD[OF assms]) simp

lemma sides_of_rhs_sp_wf:
  fixes p :: "('x,'g,'d::bounded_semilattice_sup_bot,'d) strategy_program"
  assumes "sp_wf p"
  shows "sides_of_rhs (p k) σ
           = sides_of_program p σ ⊔ sides_of_rhs (k (traverse_program p σ)) σ"
  by (subst sp_wfD[OF assms]) simp
```

The three together: what a proof that has just exposed `p k` under an observer needs, cited as one name.

```
lemmas sp_wf_observes = traverse_rhs_sp_wf sides_of_rhs_sp_wf dep_aux_sp_wf
```

The id-specialized counterparts of the sp_compile_with_* equations above, stated directly against sp_compile so a proof reasoning about compiled programs does not have to unfold sp_compile_def at every step.

```
lemma sp_compile_bind_read_local [simp]:
  "sp_compile ((sp_read_local x) ≫= f) = QueryL x (λd. sp_compile (f d))"
  by (simp add: sp_compile_def)

lemma sp_compile_bind_read_global [simp]:
  "sp_compile ((sp_read_global g) ≫= f) = QueryG g (λd. sp_compile (f d))"
  by (simp add: sp_compile_def)

lemma sp_compile_bind_publish [simp]:
  "sp_compile ((sp_publish g d) ≫= f) = Side g d (sp_compile (f ()))"
  by (simp add: sp_compile_def)

lemma sp_compile_return [simp]: "sp_compile (sp_return a) = Answer a"
  by (simp add: sp_compile_def)

lemma sp_compile_read_local [simp]: "sp_compile (sp_read_local x) = QueryL x Answer"
  by (simp add: sp_compile_def)

lemma sp_compile_read_global [simp]: "sp_compile (sp_read_global g) = QueryG g Answer"
  by (simp add: sp_compile_def)

lemma sp_compile_bind_read_at [simp]:
  "sp_compile ((sp_read_at src) ≫= f) =
     (case src of Inl x ⇒ QueryL x (λd. sp_compile (f d))
                | Inr g ⇒ QueryG g (λd. sp_compile (f d)))"
  by (cases src) simp_all

lemma sp_compile_read_at [simp]:
  "sp_compile (sp_read_at src) =
     (case src of Inl x ⇒ QueryL x Answer | Inr g ⇒ QueryG g Answer)"
  by (cases src) simp_all
```

Left bare rather than [simp]: using this equation on arbitrary binds exposes m as a raw CPS function application. The specialized equations above preserve the program abstraction and give the simplifier the vendor-tree constructors it needs.

```
lemma sp_compile_bind: "sp_compile (m ≫= f) = m (λx. sp_compile (f x))"
  by (simp add: sp_compile_def sp_compile_with_def sp_bind_def)

lemma sp_compile_with_bind: "sp_compile_with encode (m ≫= f) = m (λx. sp_compile_with encode (f x))"
  by (simp add: sp_compile_with_def sp_bind_def)
```

Monad laws

The three laws, so a caller may rewrite a program before compiling it and know the tree is unchanged. All three hold by function extensionality alone – a program is a continuation transformer, and ≫= is composition – so no tree-level reasoning is involved.

```
lemma sp_bind_sp_return_left [simp]: "(sp_return a) ≫= f = f a"
  by (rule ext) (simp add: sp_bind_def sp_return_def)

lemma sp_bind_sp_return_right [simp]: "m ≫= sp_return = m"
  by (rule ext) (simp add: sp_bind_def sp_return_def)

lemma sp_bind_assoc:
  fixes m :: "('x, 'g, 'd, 'a) strategy_program"
    and f :: "'a ⇒ ('x, 'g, 'd, 'b) strategy_program"
    and g :: "'b ⇒ ('x, 'g, 'd, 'c) strategy_program"
  shows "(m ≫= f) ≫= g = m ≫= (λx. f x ≫= g)"
  by (rule ext) (simp add: sp_bind_def)

end
```

3.3 Strategy_Tree_Side_Buffering

```
theory Strategy_Tree_Side_Buffering
  imports "TD.Basics_side"
begin
```

Per-key buffering of one right-hand side's side effects

One right-hand side may emit several `Side` writes to the same key. Declaratively that is already a join — `sides_of_rhs`'s own `Side` equation joins into the key it writes — but a solver applies its update rule once per `Side`, on the accumulate reached so far. An update rule that keeps one slot per write origin therefore records a partial accumulate, and the global it feeds can fall below a value that same origin has already established; the drop destabilizes that global's readers, whose re-evaluation restores it, and the two alternate without converging.

`buffer_sides` removes the cause rather than the symptom: it accumulates every write per key and flushes each key exactly once, so an update rule only ever sees a completed contribution. Nothing here is specific to procedure calls, contexts, or the D/G layer — the property being repaired is that a right-hand side may name one key twice, which any equation generator can do.

The per-key accumulator

An association list rather than a map: the flush below emits one `Side` per entry in list order, and `acc_add` appends an unseen key at the end, so that order is first occurrence in the traversal. A map would leave the flush order to the key type's own arrangement, which is exactly the scheduling detail a narrowing update rule is sensitive to.

```
fun acc_add ::
  "'g ⇒ 'd::bounded_semilattice_sup_bot ⇒ ('g × 'd) list ⇒ ('g × 'd) list"
where
  "acc_add k d [] = [(k, d)]"
| "acc_add k d ((k', d') # kvs) =
     (if k' = k then (k', d' ⊔ d) # kvs else (k', d') # acc_add k d kvs)"

fun acc_at :: "'g ⇒ ('g × 'd::bounded_semilattice_sup_bot) list ⇒ 'd" where
  "acc_at k [] = bot"
| "acc_at k ((k', d') # kvs) = (if k' = k then d' else bot) ⊔ acc_at k kvs"

definition acc_val ::
  "('g × 'd::bounded_semilattice_sup_bot) list ⇒ 'x + 'g ⇒ 'd" where
  "acc_val acc z = (case z of Inl _ ⇒ bot | Inr k ⇒ acc_at k acc)"

lemma acc_val_Inl [simp]: "acc_val acc (Inl u) = bot"
  by (simp add: acc_val_def)

lemma acc_val_Inr [simp]: "acc_val acc (Inr k) = acc_at k acc"
  by (simp add: acc_val_def)

lemma acc_val_Nil [simp]: "acc_val [] z = bot"
  by (cases z) simp_all

lemma acc_at_add:
  "acc_at k' (acc_add k d acc) = acc_at k' acc ⊔ (if k' = k then d else bot)"
  by (induction acc) (auto simp: ac_simps)

lemma acc_val_add:
  "acc_val (acc_add k d acc) z = acc_val acc z ⊔ (if z = Inr k then d else bot)"
  by (cases z) (simp_all add: acc_at_add)

lemma acc_val_Cons:
  "acc_val ((k, d) # kvs) z = acc_val kvs z ⊔ (if z = Inr k then d else bot)"
  by (cases z) (auto simp: ac_simps)
```

Key order and distinctness What the flush order rests on: an unseen key is appended, a seen one is joined in place, so the key list only ever grows at the end and stays distinct.

```
lemma acc_keys_add:
  "map fst (acc_add k d acc)
     = (if k ∈ set (map fst acc) then map fst acc else map fst acc @ [k])"
  by (induction acc) (auto simp add: rev_image_eqI)

lemma acc_add_append:
  "k ∉ set (map fst acc) ⟹ acc_add k d acc = acc @ [(k, d)]"
  by (induction acc) (auto simp add: rev_image_eqI)
```

```
lemma distinct_acc_add:
  "distinct (map fst acc) ⟹ distinct (map fst (acc_add k d acc))"
  by (cases "k ∈ set (map fst acc)") (simp_all add: acc_keys_add)
```

Buffering a right-hand side

The traversal that does the buffering: writes are collected into the accumulator on the way down and emitted at the next flush point, so a query branch cannot lose the writes made before it.

A flush point is every answer, and every local read `QueryL y` with `publish_before y`. Flushing only at answers gives the strongest guarantee: no key is written twice on any path, whatever the tree. But it also holds back a write past the local reads that follow it, and a local read is what makes the solver evaluate the unknown read. A call site publishes the callee's entry state and then reads the callee's exit; held back to the answer, the publication reaches the solver only after the callee has been solved from an entry it has not yet received. Goblint's normal-call transfer issues `sidel` before `getl`, and its solver applies a side effect when it is issued, so there the callee starts from the published entry. A flush before such a read costs the at-most-once guarantee for keys written again after it, so `distinct_side_path_buffer_sides` below states when it still holds, and the caller chooses `publish_before` where it can tell that it does.

A key whose completed contribution is $\bot$ is not flushed. Publishing it would be a no-op for every consumer — `sides_of_rhs` joins it in, and a join with $\bot$ changes nothing — while still costing the solver an update-rule application and a destabilization of that global's readers. The generator appends the publication to every answer path without asking whether the transfer wrote anything, so on a node that touches no global this is the only place the emptiness is known.

```
primrec flush_sides ::
  "('g × 'd::bounded_semilattice_sup_bot) list ⇒ ('x, 'g, 'd) strategy_tree
   ⇒ ('x, 'g, 'd) strategy_tree"
where
  "flush_sides [] t = t"
| "flush_sides (kv # kvs) t =
     (if snd kv ≤ bot then flush_sides kvs t
      else Side (fst kv) (snd kv) (flush_sides kvs t))"

abbreviation flushed :: "('g × 'd::bounded_semilattice_sup_bot) list ⇒ ('g × 'd) list"
  where "flushed acc ≡ filter (λkv. ¬ snd kv ≤ bot) acc"

primrec buffer_aux ::
  "('x ⇒ bool) ⇒ ('g × 'd) list
   ⇒ ('x, 'g, 'd::bounded_semilattice_sup_bot) strategy_tree
   ⇒ ('x, 'g, 'd) strategy_tree"
where
  "buffer_aux publish_before acc (Answer d) = flush_sides acc (Answer d)"
| "buffer_aux publish_before acc (QueryL y g) =
     (if publish_before y
      then flush_sides acc (QueryL y (λv. buffer_aux publish_before [] (g v)))
      else QueryL y (λv. buffer_aux publish_before acc (g v)))"
| "buffer_aux publish_before acc (QueryG y g) =
     QueryG y (λv. buffer_aux publish_before acc (g v))"
| "buffer_aux publish_before acc (Side y d t) = buffer_aux publish_before (acc_add y d acc) t"

definition buffer_sides ::
  "('x ⇒ bool) ⇒ ('x, 'g, 'd::bounded_semilattice_sup_bot) strategy_tree
   ⇒ ('x, 'g, 'd) strategy_tree"
where
  "buffer_sides publish_before t = buffer_aux publish_before [] t"

definition buffer_eqs ::
  "('x ⇒ 'x ⇒ bool) ⇒ ('x, 'g, 'd::bounded_semilattice_sup_bot) eqsT
   ⇒ ('x, 'g, 'd) eqsT" where
  "buffer_eqs publish_before T = (λx. buffer_sides (publish_before x) (T x))"

lemma buffer_eqs_apply [simp]:
  "buffer_eqs publish_before T x = buffer_sides (publish_before x) (T x)"
  by (simp add: buffer_eqs_def)
```

The declarative reading is unchanged

Buffering is invisible to the declarative reading: the local value is untouched, and the contribution at each key is the same join as before, only emitted at a flush point. That is what makes this a scheduling change and not a semantic one, wherever the flush points are.

```
lemma traverse_rhs_flush_sides [simp]:
  "traverse_rhs (flush_sides acc t) σ = traverse_rhs t σ"
  by (induction acc) simp_all

lemma traverse_rhs_buffer_aux [simp]:
  "traverse_rhs (buffer_aux publish_before acc t) σ = traverse_rhs t σ"
  by (induction t arbitrary: acc) simp_all

lemma traverse_rhs_buffer_sides [simp]:
  "traverse_rhs (buffer_sides publish_before t) σ = traverse_rhs t σ"
  by (simp add: buffer_sides_def)

lemma sides_of_rhs_flush_sides [simp]:
  "sides_of_rhs (flush_sides acc t) σ z = sides_of_rhs t σ z ⊔ acc_val acc z"
  by (induction acc arbitrary: z)
     (auto simp add: Let_def acc_val_Cons ac_simps dest!: le_bot)

lemma sides_of_rhs_buffer_aux [simp]:
  "sides_of_rhs (buffer_aux publish_before acc t) σ z = sides_of_rhs t σ z ⊔ acc_val acc z"
  by (induction t arbitrary: acc z) (auto simp add: acc_val_add Let_def ac_simps)

lemma sides_of_rhs_buffer_sides [simp]:
  "sides_of_rhs (buffer_sides publish_before t) σ = sides_of_rhs t σ"
  by (rule ext) (simp add: buffer_sides_def)

lemma dep_aux_flush_sides [simp]:
  "dep_aux σ (flush_sides acc t) = dep_aux σ t"
  by (induction acc) simp_all

lemma dep_aux_buffer_aux [simp]:
  "dep_aux σ (buffer_aux publish_before acc t) = dep_aux σ t"
  by (induction t arbitrary: acc) simp_all

lemma dep_aux_buffer_sides [simp]:
  "dep_aux σ (buffer_sides publish_before t) = dep_aux σ t"
  by (simp add: buffer_sides_def)
```

The operational property the update rules rely on

Semantic preservation alone does not say a solver sees fewer writes; it says the writes sum to the same thing. `side_path` names the keys one evaluation actually writes, in the order it writes them — the continuations are functions, so which `Side` nodes an evaluation meets depends on σ, and the statement has to be relative to it.

```
primrec side_path ::
  "('x + 'g ⇒ 'd) ⇒ ('x, 'g, 'd) strategy_tree ⇒ 'g list" where
  "side_path σ (Answer d) = []"
| "side_path σ (QueryL y g) = side_path σ (g (σ (Inl y)))"
| "side_path σ (QueryG y g) = side_path σ (g (σ (Inr y)))"
| "side_path σ (Side y d t) = y # side_path σ t"

lemma side_path_flush_sides [simp]:
  "side_path σ (flush_sides acc t) = map fst (flushed acc) @ side_path σ t"
  by (induction acc) simp_all
```

The flush points split one evaluation of the original tree into segments, and the buffer writes a key once for each segment in which the key receives a value other than $\bot$. The buffered evaluation therefore writes each key at most once exactly when no key receives such a value in two segments. `settled_writes C P` walks the evaluation and checks this: `C` holds the keys an earlier flush point has already written, and `P` the keys the current segment will write at the next one.

```
primrec settled_writes ::
  "('x ⇒ bool) ⇒ ('x + 'g ⇒ 'd) ⇒ 'g set ⇒ 'g set
   ⇒ ('x, 'g, 'd::bounded_semilattice_sup_bot) strategy_tree ⇒ bool"
where
  "settled_writes publish_before σ C P (Answer d) = True"
| "settled_writes publish_before σ C P (QueryL y g) =
     (if publish_before y
      then settled_writes publish_before σ (C ∪ P) {} (g (σ (Inl y)))
      else settled_writes publish_before σ C P (g (σ (Inl y))))"
| "settled_writes publish_before σ C P (QueryG y g) =
     settled_writes publish_before σ C P (g (σ (Inr y)))"
| "settled_writes publish_before σ C P (Side y d t) =
     ((d ≤ bot ∨ y ∉ C)
      ∧ settled_writes publish_before σ C (if d ≤ bot then P else insert y P) t)"
```

```
lemma flushed_keys_acc_add:
  "fst ‘ set (flushed (acc_add k d acc))
     = fst ‘ set (flushed acc) ∪ (if d ≤ bot then {} else {k})"
proof (induction acc)
  case Nil
  then show ?case by simp
next
  case (Cons kv acc)
  obtain k' d' where kv: "kv = (k', d')" by (cases kv)
  show ?case
  proof (cases "k' = k")
    case True
    then show ?thesis
      by (cases "d' ≤ bot"; cases "d ≤ bot") (auto simp add: kv)
  next
    case False
    then show ?thesis
      using Cons.IH by (cases "d' ≤ bot") (auto simp add: kv)
  qed
qed

lemma distinct_side_path_buffer_aux:
  assumes "distinct (map fst acc)"
    and "fst ‘ set (flushed acc) ∩ C = {}"
    and "settled_writes publish_before σ C (fst ‘ set (flushed acc)) t"
  shows "distinct (side_path σ (buffer_aux publish_before acc t))
         ∧ set (side_path σ (buffer_aux publish_before acc t)) ∩ C = {}"
  using assms
proof (induction t arbitrary: acc C)
  case (Answer d)
  then show ?case by (auto simp add: distinct_map_filter)
next
  case (QueryL y g)
  show ?case
  proof (cases "publish_before y")
    case True
    have "distinct (side_path σ (buffer_aux publish_before [] (g (σ (Inl y)))))
        ∧ set (side_path σ (buffer_aux publish_before [] (g (σ (Inl y)))))
            ∩ (C ∪ fst ‘ set (flushed acc)) = {}"
      by (rule QueryL.IH) (use QueryL.prems(3) True in simp_all)
    then show ?thesis
      using QueryL.prems(1,2) True by (auto simp add: distinct_map_filter)
  next
    case False
    then show ?thesis using QueryL.prems by (simp add: QueryL.IH)
  qed
next
  case (QueryG y g)
  then show ?case by simp
next
  case (Side y d t)
  have "distinct (map fst (acc_add y d acc))"
    using Side.prems(1) by (rule distinct_acc_add)
  moreover have "fst ‘ set (flushed (acc_add y d acc)) ∩ C = {}"
    unfolding flushed_keys_acc_add using Side.prems(2,3) by auto
  moreover have "settled_writes publish_before σ C (fst ‘ set (flushed (acc_add y d acc))) t"
    unfolding flushed_keys_acc_add using Side.prems(3) by (cases "d ≤ bot") simp_all
  ultimately have "distinct (side_path σ (buffer_aux publish_before (acc_add y d acc) t))
      ∧ set (side_path σ (buffer_aux publish_before (acc_add y d acc) t)) ∩ C = {}"
    by (rule Side.IH)
  then show ?case by simp
qed

theorem distinct_side_path_buffer_sides:
  assumes "settled_writes publish_before σ {} {} t"
  shows "distinct (side_path σ (buffer_sides publish_before t))"
  using distinct_side_path_buffer_aux[where acc = "[]" and C = "{}"] assms
  by (simp add: buffer_sides_def)
```

With no flush point before a local read nothing is ever closed, so the condition holds for every tree and every evaluation: flushing at answers only writes each key at most once unconditionally.

```
lemma settled_writes_answers_only:
  "settled_writes (λ_. False) σ {} P t"
  by (induction t arbitrary: P) simp_all

corollary distinct_side_path_buffer_sides_answers_only:
```

```
  "distinct (side_path σ (buffer_sides (λ_. False) t))"
  by (rule distinct_side_path_buffer_sides) (rule settled_writes_answers_only)
```

The elision, pinned at both polarities: a completed contribution of $\bot$ costs the evaluation no write, and one carrying a value still costs exactly one. Stated over `side_path` because the writes an evaluation performs are what the elision is for.

```
lemma side_path_flush_sides_bot [simp]:
  "side_path σ (flush_sides [(k, bot)] t) = side_path σ t"
  by simp

lemma side_path_flush_sides_value:
  assumes "¬ d ≤ bot"
  shows "side_path σ (flush_sides [(k, d)] t) = k # side_path σ t"
  using assms by simp
```

So a buffered system writes each key at most once per evaluation whenever its writes settle: every key an evaluation contributes to is flushed exactly once, with its completed contribution, and a key the evaluation never touches is not written at all. Flushing at answers only replaces the generator-level assumption that no two contributions of one right-hand side can name the same key — an assumption about how equations happen to be built, which a shared resume node refutes — with a property of the equations handed to the solver. A caller that also flushes before local reads takes that assumption back for the keys written on both sides of such a read, and must know there are none.

Buffering is idempotent

The entries a flush drops are exactly the ones it would have skipped, so flushing the filtered accumulator is the same tree.

```
lemma flush_sides_flushed [simp]: "flush_sides (flushed acc) t = flush_sides acc t"
  by (induction acc) simp_all

lemma acc_add_fold_append:
  "distinct (map fst (acc0 @ acc))
     ⟹ foldl (λa kv. acc_add (fst kv) (snd kv) a) acc0 acc = acc0 @ acc"
  by (induction acc arbitrary: acc0) (auto simp add: acc_add_append)

lemma buffer_aux_flush_sides:
  "buffer_aux publish_before acc0 (flush_sides acc t)
     = buffer_aux publish_before
         (foldl (λa kv. acc_add (fst kv) (snd kv) a) acc0 (flushed acc)) t"
  by (induction acc arbitrary: acc0) simp_all

lemma buffer_sides_idem [simp]:
  "buffer_sides publish_before (buffer_sides publish_before t) = buffer_sides publish_before t"
proof -
  have "⋀acc. distinct (map fst acc)
      ⟹ buffer_aux publish_before [] (buffer_aux publish_before acc t)
          = buffer_aux publish_before acc t"
    by (induction t)
       (auto simp add: buffer_aux_flush_sides acc_add_fold_append distinct_acc_add
             distinct_map_filter)
  then show ?thesis by (simp add: buffer_sides_def)
qed
```

The one transfer every consumer needs

`part_post_solution` — the solver interface the analyzer soundness spine consumes — is stated over $\texttt{dep}_L$, `traverse_rhs` and `sides_of_rhs` alone. All three are invariant under buffering, wherever the flush points are, so a solution of the buffered system is a solution of the original one. Buffering preserves this declarative right-hand-side semantics and the post-solution obligation the soundness proof consumes; it does not claim the buffered and unbuffered systems drive the solver through the same operational iteration, or, under widening, to the same computed post-solution.

```
lemma dep_buffer_eqs [simp]: "dep (buffer_eqs publish_before T) σ x = dep T σ x"
  by (simp add: dep_def)

lemma dep_L_buffer_eqs [simp]: "dep_L (buffer_eqs publish_before T) σ x = dep_L T σ x"
  by (simp add: dep_L_def)

theorem part_post_solution_buffer_eqs [simp]:
  "part_post_solution (buffer_eqs publish_before T) x σ vars
```

```
    = part_post_solution T x σ vars"
  by simp

end
```

3.4 Strategy_Tree_Pair

```
theory Strategy_Tree_Pair
  imports Strategy_Tree_Side_Buffering
begin
```

A right-hand side as a pair

A side-effecting right-hand side is often presented as a function from a valuation to a pair: the contributions it publishes, a partial map from unknowns to values, and its own value. A strategy tree evaluates to the same pair. The second component is `traverse_rhs`; the first is `sides_of_rhs` restricted to the keys `side_path` visits, so a key that receives only $\bot$ stays in the domain and a key that receives nothing does not. Both components come from the existing evaluators.

```
definition rhs_sides ::
  "('x,'g,'d::bounded_semilattice_sup_bot) strategy_tree ⇒ ('x + 'g ⇒ 'd) ⇒ 'x + 'g ⇀ 'd"
where
  "rhs_sides t σ = (Some ∘ sides_of_rhs t σ) |` (Inr ` set (side_path σ t))"

definition rhs_pair ::
  "('x,'g,'d::bounded_semilattice_sup_bot) strategy_tree ⇒ ('x + 'g ⇒ 'd)
     ⇒ ('x + 'g ⇀ 'd) × 'd"
where
  "rhs_pair t σ = (rhs_sides t σ, traverse_rhs t σ)"
```

The partial map loses nothing

Off the visited keys `sides_of_rhs` is $\bot$, so reading the map back with `mlup` recovers it.

```
lemma sides_of_rhs_Inl [simp]: "sides_of_rhs t σ (Inl u) = ⊥"
  by (induction t) (simp_all add: Let_def)

lemma sides_of_rhs_off_path:
  "g ∉ set (side_path σ t) ⟹ sides_of_rhs t σ (Inr g) = ⊥"
  by (induction t) (auto simp: Let_def)

lemma rhs_sides_apply:
  "rhs_sides t σ z =
     (if z ∈ Inr ` set (side_path σ t) then Some (sides_of_rhs t σ z) else None)"
  by (simp add: rhs_sides_def restrict_map_def)

lemma dom_rhs_sides: "dom (rhs_sides t σ) = Inr ` set (side_path σ t)"
  by (auto simp: dom_def rhs_sides_apply)
```

Side effects only reach global unknowns; a local unknown is never in the domain.

```
lemma rhs_sides_Inl [simp]: "rhs_sides t σ (Inl u) = None"
  by (auto simp: rhs_sides_apply)

lemma mlup_rhs_sides: "mlup (rhs_sides t σ) = sides_of_rhs t σ"
proof
  fix z
  show "mlup (rhs_sides t σ) z = sides_of_rhs t σ z"
    by (cases z) (auto simp: mlup_def rhs_sides_apply sides_of_rhs_off_path image_iff)
qed
```

Several side effects to one unknown appear once in the domain, with the join of their values, in the order of the traversal.

```
lemma rhs_sides_Side:
  "rhs_sides (Side g d t) σ = (rhs_sides t σ)(Inr g ↦ mlup (rhs_sides t σ) (Inr g) ⊔ d)"
  by (rule ext) (auto simp: mlup_rhs_sides rhs_sides_apply Let_def)

lemma rhs_pair_two_sides:
  "rhs_pair (Side g a (Side g b (Answer d))) σ = ([Inr g ↦ b ⊔ a], d)"
  unfolding rhs_pair_def by (auto simp: rhs_sides_apply Let_def intro!: ext)
```

The certificate in pair form

A side map is below a valuation exactly when every contribution it publishes is below its target.

```
lemma sides_of_rhs_le_iff_rhs_sides:
  "sides_of_rhs t σ ≤ σ ⟷ (∀z d. rhs_sides t σ z = Some d ⟶ d ≤ σ z)"
proof
  assume "sides_of_rhs t σ ≤ σ"
```

```
  then show "∀z d. rhs_sides t σ z = Some d ⟶ d ≤ σ z"
    by (auto simp: rhs_sides_apply le_fun_def)
next
  assume bound: "∀z d. rhs_sides t σ z = Some d ⟶ d ≤ σ z"
  show "sides_of_rhs t σ ≤ σ"
  proof (rule le_funI)
    fix z
    show "sides_of_rhs t σ z ≤ σ z"
    proof (cases "z ∈ Inr ‘ set (side_path σ t)")
      case True
      then show ?thesis
        using bound by (simp add: rhs_sides_apply)
    next
      case False
      then have "sides_of_rhs t σ z = ⊥"
        by (cases z) (auto simp: sides_of_rhs_off_path image_iff)
      then show ?thesis
        by simp
    qed
  qed
qed
```

`part_post_solution` with its local and side bounds stated on the pair: the value is below the unknown, and every published contribution is below its target. The bounds constrain the side-effect targets although they are global unknowns outside `vars`, and say nothing about unknowns outside `vars` that the solve never reached.

```
theorem part_post_solution_iff_rhs_pair:
  "part_post_solution T x σ vars ⟷
     x ∈ vars ∧ (∀u ∈ vars. dep_L T σ u ⊆ vars ∧
       (case rhs_pair (T u) σ of (ϱ, d) ⇒
          d ≤ σ (Inl u) ∧ (∀z d'. ϱ z = Some d' ⟶ d' ≤ σ z)))"
  unfolding rhs_pair_def by (simp add: sides_of_rhs_le_iff_rhs_sides)

end
```

3.5 TD_Solver_Bridge

```
theory TD_Solver_Bridge
  imports "TD.TD_side_Interface"
begin
```

The semantic boundary between Voblint and the vendored TD solver

Voblint relies on the vendored TD solver only through the locale below, which packages the solver facts the analysis pipeline uses.

What a pipeline asks of its solver

Four facts are everything the analysis pipeline uses about a solver: a solve in the solver's domain answers a partial post-solution over finitely many keys, and an answer of the executable solver `solve_c` places the query in that domain and is the solve itself. TD proves all four inside `TD_side_upd_rule` (`partial_post_solution`, `finite_stabl_solve`, `solve_dom_of_solve_c`, `solve_code_equation`). `certified_solver` names that contract once, over any equation system, so a pipeline assumes one locale instead of four facts and a solver discharges it once for all of its instances.

```
locale certified_solver =
  fixes solve :: "('x, 'g, 'd::bounded_semilattice_sup_bot) eqsT
                  ⇒ 'x ⇒ 'x set × ('x + 'g ⇒ 'd)"
    and solve_dom :: "('x, 'g, 'd) eqsT ⇒ 'x ⇒ bool"
    and solve_c :: "('x, 'g, 'd) eqsT ⇒ 'x ⇒ ('x set × ('x + 'g ⇒ 'd)) option"
  assumes solve_pp:
      "⋀eqs x. solve_dom eqs x
         ⟹ part_post_solution eqs x (snd (solve eqs x)) (fst (solve eqs x))"
    and solve_fin: "⋀eqs x. solve_dom eqs x ⟹ finite (fst (solve eqs x))"
    and dom_of_solve_c: "⋀eqs x. solve_c eqs x ≠ None ⟹ solve_dom eqs x"
    and solve_of_solve_c: "⋀eqs x sol. solve_c eqs x = Some sol ⟹ solve eqs x = sol"

end
```

3.6 Globals_Rule

```
theory Globals_Rule
  imports TD_Solver_Bridge
begin
```

Which rule updates a side-effected global

The vendored solver warrows every local unknown at a widening point, whichever instance runs. What its instances differ in is how a contribution side-effected into a global unknown is merged: joined into the value, joined per origin, warrowed into the value, warrowed per origin, or widened per origin with narrowing cut short once an origin has switched from widening to narrowing a given number of times. That merge is the one choice a caller makes, so it is a value here rather than a choice between five solvers: a single interpretation takes the rule as a parameter, and every solver fact holds for all five at once.

```
datatype globals_rule =
    Globals_Join
  | Globals_Per_Origin
  | Globals_Warrow
  | Globals_Warrow_Per_Origin
  | Globals_Bounded_Narrowing nat
```

One update state for every rule

The bounded-narrowing rule keeps a counter per global and origin beside the recorded contributions; the other four rules keep only the contributions. So the single interpretation runs on the state with the counter, and each of the four runs on its contribution part and leaves the counter unchanged. Every `update_rule` obligation speaks about the contributions alone, so the lifted rule inherits them.

The lifted rule also reports no change when the merged value equals the old one. The two warrowing rules answer with a new value whenever an origin's contribution changes, even when the merge over all origins stays the same, and the solver destabilizes every reader of a global it is handed a value for. Recording the contribution and reporting nothing keeps those readers stable.

```
definition changed :: "'d ⇒ 'd option ⇒ 'd option" where
  "changed d res = (case res of Some d'' ⇒ if d'' = d then None else Some d'' | None ⇒ None)"

lemma changed_SomeD: "changed d res = Some d'' ⟹ res = Some d''"
  by (cases res) (auto simp: changed_def split: if_splits)

lemma changed_NoneD: "changed d res = None ⟹ res = None ∨ res = Some d"
  by (cases res) (auto simp: changed_def split: if_splits)

definition lift_basic_rule ::
    "('d ⇒ 'x ⇒ 'g ⇒ 'd ⇒ ('x, 'g, 'd) ug_state ⇒ 'd option × ('x, 'g, 'd) ug_state)
     ⇒ 'd ⇒ 'x ⇒ 'g ⇒ 'd ⇒ ('x, 'g, 'd) ug_state_with_gas
     ⇒ 'd option × ('x, 'g, 'd) ug_state_with_gas" where
  "lift_basic_rule f d orig g d' state =
     (let (res, st) = f d orig g d' (ug_state.truncate state)
      in (changed d res, ug_state.ϱ_update (λ_. ug_state.ϱ st) state))"

lemma lift_basic_ruleE:
  assumes "(res, state') = lift_basic_rule f d orig g d' state"
  obtains res0 st where "(res0, st) = f d orig g d' (ug_state.truncate state)"
    and "res = changed d res0" and "ug_state.ϱ state' = ug_state.ϱ st"
  using assms by (auto simp: lift_basic_rule_def split: prod.splits)

lemma update_rule_lift_basic_rule:
  assumes f: "update_rule init_basic_ug_state f"
  shows "update_rule init_ug_state_with_gas (lift_basic_rule f)"
proof
  show "∀g orig. rho_lookup (ug_state.ϱ init_ug_state_with_gas) g orig ≤ ⊥"
    by (simp add: init_ug_state_with_gas_def fmlookup_default_def)
next
  fix res state' d orig g d' state g' orig'
  assume step: "(res, state') = lift_basic_rule f d orig g d' state" and ne: "g' ≠ g"
  from step obtain res0 st where e: "(res0, st) = f d orig g d' (ug_state.truncate state)"
    and r: "ug_state.ϱ state' = ug_state.ϱ st" by (rule lift_basic_ruleE)
  show "rho_lookup (ug_state.ϱ state') g' orig' = rho_lookup (ug_state.ϱ state) g' orig'"
    using update_rule.update_global_untouched(1)[OF f e ne] by (simp add: r)
next
  fix res state' d orig g d' state g' orig'
  assume step: "(res, state') = lift_basic_rule f d orig g d' state" and ne: "orig' ≠ orig"
  from step obtain res0 st where e: "(res0, st) = f d orig g d' (ug_state.truncate state)"
```

```
    and r: "ug_state.ϱ state' = ug_state.ϱ st" by (rule lift_basic_ruleE)
  show "rho_lookup (ug_state.ϱ state') g' orig' = rho_lookup (ug_state.ϱ state) g' orig'"
    using update_rule.update_global_untouched(2)[OF f e ne] by (simp add: r)
next
  fix res state' d orig g d' state
  assume step: "(res, state') = lift_basic_rule f d orig g d' state"
  from step obtain res0 st where e: "(res0, st) = f d orig g d' (ug_state.truncate state)"
    and r: "ug_state.ϱ state' = ug_state.ϱ st" by (rule lift_basic_ruleE)
  show "d' ≤ rho_lookup (ug_state.ϱ state') g orig"
    using update_rule.update_global_recorded_in_rho[OF f e] by (simp add: r)
next
  fix state' d orig g d' state
  assume step: "(None, state') = lift_basic_rule f d orig g d' state"
    and inv: "∀orig. rho_lookup (ug_state.ϱ state) g orig ≤ d"
  from step obtain res0 st where e: "(res0, st) = f d orig g d' (ug_state.truncate state)"
    and c: "None = changed d res0" and r: "ug_state.ϱ state' = ug_state.ϱ st"
    by (rule lift_basic_ruleE)
  from changed_NoneD[OF c[symmetric]] show "rho_lookup (ug_state.ϱ state') g orig ≤ d"
  proof
    assume "res0 = None"
    then show ?thesis
      using update_rule.update_global_preserves_rho_invariant(1)[OF f] e inv by (simp add: r)
  next
    assume "res0 = Some d"
    then show ?thesis
      using update_rule.update_global_preserves_rho_invariant(2)[OF f] e inv by (simp add: r)
  qed
next
  fix d'' state' d orig g d' state orig'
  assume step: "(Some d'', state') = lift_basic_rule f d orig g d' state"
    and inv: "∀orig. rho_lookup (ug_state.ϱ state) g orig ≤ d"
  from step obtain res0 st where e: "(res0, st) = f d orig g d' (ug_state.truncate state)"
    and c: "Some d'' = changed d res0" and r: "ug_state.ϱ state' = ug_state.ϱ st"
    by (rule lift_basic_ruleE)
  have "res0 = Some d''" by (rule changed_SomeD) (use c in simp)
  then show "rho_lookup (ug_state.ϱ state') g orig' ≤ d''"
    using update_rule.update_global_preserves_rho_invariant(2)[OF f] e inv by (simp add: r)
qed
```

The rule as a value

`update_global_of` maps each `globals_rule` to its update rule, so one interpretation of `TD_side_upd_rule` covers every rule a configuration can select.

```
definition update_global_of ::
    "globals_rule ⇒ 'd ⇒ 'x ⇒ 'g ⇒ 'd::{bounded_semilattice_sup_bot,warrowing}
       ⇒ ('x, 'g, 'd) ug_state_with_gas ⇒ 'd option × ('x, 'g, 'd) ug_state_with_gas" where
  "update_global_of r =
     (case r of
        Globals_Join ⇒ lift_basic_rule update_global_always_join
      | Globals_Per_Origin ⇒ lift_basic_rule update_global_per_origin
      | Globals_Warrow ⇒ lift_basic_rule update_global_warrowing_apinis
      | Globals_Warrow_Per_Origin ⇒ lift_basic_rule update_global_warrowing_per_origin
      | Globals_Bounded_Narrowing n ⇒ update_global_bounded_narrowing n)"

lemma update_rule_update_global_of:
  "update_rule init_ug_state_with_gas (update_global_of r)"
  by (cases r)
     (simp_all add: update_global_of_def update_rule_lift_basic_rule
        always_join.update_rule_axioms per_origin.update_rule_axioms
        warrowing_apinis.update_rule_axioms warrowing_per_origin.update_rule_axioms
        bounded_narrowing.update_rule_axioms)

global_interpretation TD_side_rule_Interp:
  TD_side_upd_rule init_ug_state_with_gas "update_global_of r" T for r T
  defines TD_side_rule_Interp_solve = TD_side_rule_Interp.solve
    and TD_side_rule_Interp_solve_c = TD_side_rule_Interp.solve_c
    and TD_side_rule_Interp_solve_rec_c = TD_side_rule_Interp.solve_rec_c
  by (simp add: TD_side_upd_rule.intro update_rule_update_global_of)
```

The solver contract of the pipeline, once for every rule.

```
lemma td_certified_solver:
  "certified_solver (TD_side_rule_Interp_solve r)
     (TD_side_rule_Interp.solve_dom TYPE('g) TYPE('d::{bounded_semilattice_sup_bot,warrowing}) r)
     (TD_side_rule_Interp_solve_c r)"
  by unfold_locales
```

```
     (erule TD_side_rule_Interp.partial_post_solution[OF _ surjective_pairing],
      erule TD_side_rule_Interp.finite_stabl_solve,
      erule TD_side_rule_Interp.solve_dom_of_solve_c,
      simp add: TD_side_rule_Interp.solve_code_equation)
```

end

3.7 Solver_Trace

```
theory Solver_Trace
  imports Globals_Rule
begin
```

A solver trace inside the exported code

The vendored interface hides the solver's state fields (`TD_side_Interface`). The traced equations below restate the solver's own terms, so this theory names those fields again, as input abbreviations of the hidden constants.

```
abbreviation (input) point where "point ≡ TD_side_upd_rule.state.point"
abbreviation (input) point_update where "point_update ≡ TD_side_upd_rule.state.point_update"
abbreviation (input) σ where "σ ≡ TD_side.state.σ"
abbreviation (input) σ_update where "σ_update ≡ TD_side.state.σ_update"
```

The executable solver can report its steps. `trace_event` is the identity on the logic's side: it takes a channel name and a suspended event and returns `()`, so every equation below is proved equal to the untraced one by unfolding it. Code export maps the constant to a target-language hook (`Trace_Run`); the hook forces the suspension only when tracing is on, so an untraced run never builds an event. The Eval target keeps the equation, so proofs by evaluation run the same code without any hook.

The equations are alternative code equations for the vendored solver's executable functions. They leave the vendored definitions and proofs untouched and replace only the equations code export uses, in theories that import this one.

```
definition trace_event :: "String.literal ⇒ (unit ⇒ 'e) ⇒ unit" where
  "trace_event channel e = ()"
```

One event per solver step that Goblint's tracing of its top-down solvers reports (https://github.com/goblint/analyzer/blob/master/src/solver/td_simplified.ml, https://github.com/goblint/analyzer/blob/master/src/solver/td3.ml), plus two with no counterpart there: `Ev_Rhs`, the value a right-hand side returns, and the start and end of one solve, and `Ev_Wpoint_Clear`, an iteration of an already stable unknown dropping it from the widening points. A local unknown is `'x`, a global one `'g`, a value `'d`. Booleans are the solver's own sets read at the step: `called`, `stabl`, and `point`, the widening points, and whether an old value is ⊥.

```
datatype ('x, 'g, 'd) solver_event =
    Ev_Start 'x
  | Ev_Stop
  | Ev_Query 'x 'x bool bool
  | Ev_Query_Wpoint 'x bool
  | Ev_Iterate_From_Query 'x
  | Ev_Add_Infl "'x + 'g" 'x
  | Ev_Answer 'x 'x 'd
  | Ev_Query_Global 'x 'g
  | Ev_Answer_Global 'x 'g 'd
  | Ev_Iterate 'x bool bool bool
  | Ev_Eq 'x
  | Ev_Rhs 'x 'd
  | Ev_Still_Unstable 'x
  | Ev_Widen 'x bool
  | Ev_Sol 'x bool 'd 'd 'd
  | Ev_Wpoint_Remove 'x bool
  | Ev_Wpoint_Clear 'x bool
  | Ev_Update 'x bool bool 'd 'd
  | Ev_Iterate_Changed 'x
  | Ev_Side 'x 'g 'd
  | Ev_Update_Global 'x 'g bool 'd 'd
  | Ev_Destabilize "'x + 'g"
  | Ev_Stable_Remove 'x
```

Destabilization. `destab_opt x` clears the unknowns that read `x` from the stable set and continues through each one not currently being solved.

```
declare destab_iter_opt.simps [code del] destab_opt.simps [code del]

lemma destab_iter_opt_traced [code]:
  fixes i :: "('x + 'g, 'x list) fmap"
  shows
    "destab_iter_opt [] i s c = (i, s)"
    "destab_iter_opt (y # ys) i s c =
      (let _ = trace_event STR ''solver''
                (λ_. (Ev_Stable_Remove y :: ('x, 'g, unit) solver_event)) in
       let (i, s) = (if y ∈ c then (i, s - {y}) else destab_opt (Inl y) i (s - {y}) c) in
```

```
     destab_iter_opt ys i s c)"
  by (simp_all only: destab_iter_opt.simps trace_event_def Let_def)

lemma destab_opt_traced [code]:
  fixes x :: "'x + 'g"
  shows
    "destab_opt x i s c =
      (let _ = trace_event STR ''solver''
                (λ_. (Ev_Destabilize x :: ('x, 'g, unit) solver_event)) in
       destab_iter_opt (fmlookup_default i [] x) (fmdrop x i) s c)"
  by (simp only: destab_opt.simps trace_event_def Let_def)
```

The solver. The equation below is the vendored `solve_rec_c` equation with events added at the steps Goblint traces: `Q` is a query, `I` an iteration, `R` one evaluation of the right-hand side, `E` one instruction of it.

```
unbundle lattice_syntax

declare TD_side_rule_Interp.solve_rec_c.simps [code del]

lemma solve_rec_c_traced [code]:
  fixes T :: "('x, 'g, 'd::{bounded_semilattice_sup_bot,warrowing}) eqsT"
  shows "TD_side_rule_Interp_solve_rec_c r T s = (case s of
    TD_side_upd_rule.func_state.Q (y, x, state, ug_state) ⇒
      (let _ = trace_event STR ''solver'' (λ_. (Ev_Query y x (x ∈ stabl state) (x ∈ called state)
                :: ('x, 'g, 'd) solver_event)) in
       Option.bind (
        if x ∈ called state then
          (let _ = trace_event STR ''solver''
                    (λ_. (Ev_Query_Wpoint x (x ∈ point state) :: ('x, 'g, 'd) solver_event)) in
           Some ((σ state) (Inl x), state(| point := insert x (point state) |), ug_state))
        else
          (let _ = trace_event STR ''solver''
                    (λ_. (Ev_Iterate_From_Query x :: ('x, 'g, 'd) solver_event)) in
           TD_side_rule_Interp_solve_rec_c r T
             (TD_side_upd_rule.func_state.I (x, state(| called := insert x (called state) |), ug_state))))
        (λ(xd, state, ug_state).
          let _ = trace_event STR ''solver'' (λ_. (Ev_Add_Infl (Inl x) y :: ('x, 'g, 'd) solver_event)) in
          let _ = trace_event STR ''solver'' (λ_. (Ev_Answer y x xd :: ('x, 'g, 'd) solver_event)) in
          Some (xd, state(|infl := fminsert (infl state) (Inl x) y|), ug_state)))
  | TD_side_upd_rule.func_state.I (x, state, ug_state) ⇒
      (let _ = trace_event STR ''solver''
                (λ_. (Ev_Iterate x (x ∈ called state) (x ∈ stabl state) (x ∈ point state)
                  :: ('x, 'g, 'd) solver_event)) in
       if x ∉ stabl state then Option.bind (
        TD_side_rule_Interp_solve_rec_c r T (TD_side_upd_rule.func_state.R (x, state, ug_state)))
        (λ(d_new, state1, ug_state1).
          let _ = trace_event STR ''solver''
                    (λ_. (Ev_Widen x (x ∈ point state) :: ('x, 'g, 'd) solver_event)) in
          let d_new' = (if x ∈ point state then (σ state1) (Inl x) ∇Δ d_new else d_new) in
          let _ = trace_event STR ''solver''
                    (λ_. (Ev_Sol x (x ∈ point state) ((σ state1) (Inl x)) d_new d_new'
                      :: ('x, 'g, 'd) solver_event)) in
          if (σ state1) (Inl x) = d_new' then
            (let _ = trace_event STR ''solver''
                      (λ_. (Ev_Wpoint_Remove x (x ∈ point state1) :: ('x, 'g, 'd) solver_event)) in
             Some (d_new', state1(| called := called state1 - {x}, point := (point state1) - {x}|),
               ug_state1))
          else
            (let _ = trace_event STR ''solver''
                      (λ_. (Ev_Update x (x ∈ point state1) ((σ state1) (Inl x) = ⊥)
                              ((σ state1) (Inl x)) d_new' :: ('x, 'g, 'd) solver_event)) in
             let (infl1, stabl1) = destab_opt (Inl x) (infl state1) (stabl state1) (called state1) in
             let _ = trace_event STR ''solver''
                      (λ_. (Ev_Iterate_Changed x :: ('x, 'g, 'd) solver_event)) in
             TD_side_rule_Interp_solve_rec_c r T (TD_side_upd_rule.func_state.I (x, state1(| infl := infl1, stabl :=
stabl1,
               σ := (σ state1)(Inl x := d_new') |), ug_state1))))
      else
        (let _ = trace_event STR ''solver''
                  (λ_. (Ev_Wpoint_Clear x (x ∈ point state) :: ('x, 'g, 'd) solver_event)) in
         Some ((σ state) (Inl x), state(| called := called state - {x}, point := (point state) - {x}|),
          ug_state)))
  | TD_side_upd_rule.func_state.R (x, state, ug_state) ⇒
      (let _ = trace_event STR ''solver'' (λ_. (Ev_Eq x :: ('x, 'g, 'd) solver_event)) in
       Option.bind (
        TD_side_rule_Interp_solve_rec_c r T
          (TD_side_upd_rule.func_state.E (x, T x, (λ_. ⊥), state (|stabl := insert x (stabl state)|), ug_state)))
```

```
       (λ(xd, state, ug_state). if x ∈ stabl state then Some (xd, state, ug_state)
         else (let _ = trace_event STR ''solver''
                         (λ_. (Ev_Still_Unstable x :: ('x, 'g, 'd) solver_event)) in
               TD_side_rule_Interp_solve_rec_c r T (TD_side_upd_rule.func_state.R (x, state, ug_state)))))
  | TD_side_upd_rule.func_state.E (x, t, sides, state, ug_state) ⇒ (case t of
       Answer d ⇒
         (let _ = trace_event STR ''solver'' (λ_. (Ev_Rhs x d :: ('x, 'g, 'd) solver_event)) in
          Some (d, state, ug_state))
     | QueryL y g ⇒ (
         Option.bind (TD_side_rule_Interp_solve_rec_c r T (TD_side_upd_rule.func_state.Q (x, y, state, ug_state)))
           (λ(yd, state, ug_state).
             TD_side_rule_Interp_solve_rec_c r T (TD_side_upd_rule.func_state.E (x, g yd, sides, state, ug_state))))
     | QueryG y g ⇒
         (let _ = trace_event STR ''solver'' (λ_. (Ev_Query_Global x y :: ('x, 'g, 'd) solver_event)) in
          let _ = trace_event STR ''solver'' (λ_. (Ev_Add_Infl (Inr y) x :: ('x, 'g, 'd) solver_event)) in
          let _ = trace_event STR ''solver''
                    (λ_. (Ev_Answer_Global x y ((σ state) (Inr y)) :: ('x, 'g, 'd) solver_event)) in
          TD_side_rule_Interp_solve_rec_c r T (TD_side_upd_rule.func_state.E (x, g ((σ state) (Inr y)), sides,
            state(| infl := fminsert (infl state) (Inr y) x|), ug_state)))
     | Side y d t ⇒
         (let _ = trace_event STR ''solver'' (λ_. (Ev_Side x y d :: ('x, 'g, 'd) solver_event)) in
          let d = sides y ⊔ d in
          let sides = sides(y := d) in
          let (upd, ug_state) = update_global_of r (σ state (Inr y)) x y d ug_state in
          case upd of None ⇒ TD_side_rule_Interp_solve_rec_c r T (TD_side_upd_rule.func_state.E (x, t, sides, state,
ug_state))
           | Some d ⇒
             let _ = trace_event STR ''solver''
                       (λ_. (Ev_Update_Global x y ((σ state) (Inr y) = ⊥) ((σ state) (Inr y)) d
                         :: ('x, 'g, 'd) solver_event)) in
             let (infl', stabl') = destab_opt (Inr y) (infl state) (stabl state) (called state) in
             TD_side_rule_Interp_solve_rec_c r T (TD_side_upd_rule.func_state.E (x, t, sides, state(| infl := infl',
stabl := stabl',
               σ := (σ state)(Inr y := d)|), ug_state)))))"
  by (rule trans[OF TD_side_rule_Interp.solve_rec_c.simps])
     (simp only: trace_event_def Let_def)
```

One solve, from its root unknown to its result, between a start and a stop event.

```
declare TD_side_rule_Interp.solve_code_equation [code del]

lemma solve_traced [code]:
  fixes T :: "('x, 'g, 'd::{bounded_semilattice_sup_bot,warrowing}) eqsT"
  shows "TD_side_rule_Interp_solve r T x =
    (let _ = trace_event STR ''solver'' (λ_. (Ev_Start x :: ('x, 'g, 'd) solver_event)) in
     let res = TD_side_rule_Interp_solve_c r T x in
     let _ = trace_event STR ''solver'' (λ_. (Ev_Stop :: ('x, 'g, 'd) solver_event)) in
     case res of Some v ⇒ v
     | None ⇒ Code.abort (STR ''Input not in domain'') (λ_. TD_side_rule_Interp_solve r T x))"
  by (rule trans[OF TD_side_rule_Interp.solve_code_equation])
     (simp only: trace_event_def Let_def)
```

The same events around the executable solver, for a caller that reads its answer directly. It is `solve_c` in the logic; a traced code equation of that caller names it in place of `solve_c`.

```
definition solve_c_traced ::
    "globals_rule ⇒ ('x, 'g, 'd::{bounded_semilattice_sup_bot,warrowing}) eqsT ⇒ 'x
       ⇒ ('x set × ('x + 'g ⇒ 'd)) option" where
  "solve_c_traced r T x =
    (let _ = trace_event STR ''solver'' (λ_. (Ev_Start x :: ('x, 'g, 'd) solver_event)) in
     let res = TD_side_rule_Interp_solve_c r T x in
     let _ = trace_event STR ''solver'' (λ_. (Ev_Stop :: ('x, 'g, 'd) solver_event)) in
     res)"

lemma solve_c_traced_eq: "solve_c_traced r T x = TD_side_rule_Interp_solve_c r T x"
  by (simp add: solve_c_traced_def trace_event_def)

end
```

Chapter 4

Voblint_CFG

4.1 CFG_Def

```
theory CFG_Def
  imports "Voblint_VIMP.VIMP_Semantics" "HOL-Library.Countable"
begin
```

Procedure-aware control-flow graph

A `cfg` is an interprocedural control-flow graph over two disjoint transition relations, one per phenomenon:

- `intra` — ordinary context-preserving flow, labelled by an `edge_action` that is a total within-context store transformer. Executed by `edge_step`.
- `calls` — context-crossing calls, labelled by a `call_action` and carrying the call-site, the callee entry node, and the continuation node.

Nodes are procedure-aware: `FunctionEntry p` and `FunctionResult p` are first-class nodes, not keyed side slots. There is no global exit node and no call/return matching side relation: a call edge already names its callee entry and its continuation, and a return is an ordinary intra edge into `FunctionResult p`. Calls carry incompatible typing from intra flow, so `edge_step` has no call case and `intra` can never carry a call — by typing, not by a side condition.

Translation from VIMP to CFG and the collecting/equation layers ride on this structure.

Nodes

```
datatype cfg_node =
    Statement (node_stmt: nat)
  | FunctionEntry (entry_proc: pname)
  | FunctionResult (result_proc: pname)

instance cfg_node :: countable
  by countable_datatype
```

The solver unknown for a program point is a CFG node. Analysis-facing code keeps the short name `pp` for it; a return node is `FunctionResult p`, a callee entry `FunctionEntry p`, an ordinary location `Statement n`.

```
type_synonym pp = cfg_node
```

Edge actions and call actions

`edge_action` labels intra flow. It has no call constructor: every action is a total store transformer within one activation, so an intra edge can never denote a call. A `EA_Ret e p` writes the return value into `ret_var` in the callee's own context; its graph target is `FunctionResult p` (enforced by `wf_cfg`), which is why return summarisation is ordinary predecessor folding over `FunctionResult p`.

`EA_Body p` labels the one edge leaving `FunctionEntry p`, so an activation's first step inside the callee is distinguishable from an ordinary `EA_Nop`. Concretely it is the identity: a VIMP procedure declares no locals beyond its formals, which the call already bound. It exists so that an analysis has a transition to attach procedure-entry work to — Goblint runs `body` on exactly this edge — and it carries the procedure name because that work is per-procedure.

`call_action` labels call edges. `CallEdge dst formals args` records the caller destination variable, the callee's formal parameter names, and the actual arguments; the callee identity and the continuation are the two target nodes of the `calls` tuple, not payload of the action. Carrying `formals` on the edge lets the caller-side entry transfer `call_enter` bind actuals to formals without consulting a procedure table, keeping the trace kernel independent of the source declaration environment.

Both action types are generic in the variable type `'v` and the expression type `'e`, as the commands they are compiled from are. `edge_action` and `call_action` are the instances at source names and surface expressions.

```
datatype ('v, 'e) gedge_action =
    EA_Nop
  | EA_Assign   (ea_var: 'v) (ea_rhs: 'e)
  | EA_Special  (ea_special_op: "'e gspecial_call") (ea_special_dst: 'v)
  | EA_Assume    (ea_cond: 'e)
  | EA_AssumeNot (ea_cond: 'e)
  | EA_Body     (ea_body_proc: pname)
  | EA_Ret      (ea_ret_val: "'e option") (ea_ret_proc: pname)
  | EA_Check    (ea_check_label: check_label) (ea_check_cond: 'e)
```

```
datatype ('v, 'e) gcall_action =
    CallEdge (ce_dst: "'v option") (ce_formals: "'v list") (ce_args: "'e list")

type_synonym edge_action = "(vname, src_exp) gedge_action"
type_synonym call_action = "(vname, src_exp) gcall_action"

instance gedge_action :: (countable, countable) countable
  by countable_datatype

instance gcall_action :: (countable, countable) countable
  by countable_datatype
```

`call_info` is the call-boundary metadata an interprocedural transfer may consult: the caller destination, the callee, its formals, and the actual arguments. Goblint's `Analyses.Spec` hands the same four to both halves of its return protocol – `combine_env man lval fexp f args fc au ask` and `combine_assign` with the identical argument shape – so a transfer that only ever sees the caller and callee-exit states is strictly weaker than that interface. The call site and the callee-exit point are the combine tree's own two program-point arguments, so they are not duplicated here.

Goblint's `fexp` has no separate counterpart: VIMP calls a statically named procedure, so `ci_callee` already carries what the call expression identifies. The callee context `fc` belongs to the routed-analysis layer rather than to a transfer-record field, and `Queries.ask` over the callee exit has none: there is no query bus.

```
record call_info =
  ci_dst     :: "var_id option"
  ci_callee  :: pname
  ci_formals :: "var_id list"
  ci_args    :: "exp list"

definition call_info_of :: "(var_id, exp) gcall_action ⇒ pname ⇒ call_info" where
  "call_info_of ca p =
     (| ci_dst = ce_dst ca, ci_callee = p,
       ci_formals = ce_formals ca, ci_args = ce_args ca |)"

lemma call_info_of_simps [simp]:
  "ci_dst (call_info_of ca p) = ce_dst ca"
  "ci_callee (call_info_of ca p) = p"
  "ci_formals (call_info_of ca p) = ce_formals ca"
  "ci_args (call_info_of ca p) = ce_args ca"
  by (simp_all add: call_info_of_def)
```

CFG record: two relations

`calls` is a four-place relation `(call_site, act, callee_entry, continuation)`: the callee entry and the continuation are ordinary nodes, kept explicit so no matching side relation is needed. The flat (call-free) CFG is exactly the `calls = {}` fragment, so every fact quantified over `intra` is verbatim a flat-CFG fact.

```
record ('v, 'e) gcfg =
  intra     :: "(cfg_node × ('v, 'e) gedge_action × cfg_node) set"
  calls     :: "(cfg_node × ('v, 'e) gcall_action × cfg_node × cfg_node) set"
  cfg_entry :: cfg_node

type_synonym cfg = "(vname, src_exp) gcfg"
```

Intra edge execution

`special_step` is `special_call`'s own step function, factored out of `edge_step` so that extending the special-call vocabulary only touches this function (via `special_result`, shared with the source small-step semantics) and `special_step_nonempty` below, not every lemma stated over `edge_action`. `Nondet_Int` havocs the destination; `Min`/`Max` write exactly one, deterministic value, so no single equation characterizes every `sc` uniformly any more – callers reasoning about an as-yet-unclassified `sc` case-split on it instead. A written value is converted to its destination's kind, as in `pstep`.

```
fun special_step :: "kenv ⇒ exp gspecial_call ⇒ var_id ⇒ store ⇒ store set" where
  "special_step κ sc x s = {s(x := kind_conv κ (Some x) v) |v. special_result eval sc s v}"

lemma special_step_nonempty [simp]: "special_step κ sc x s ≠ {}"
  by (cases sc) auto
```

`edge_step` is the single primitive semantics of an intra action. It is defined for every constructor and has no call case; guards are the only source of an empty result.

```
fun edge_step :: "kenv ⇒ (var_id, exp) gedge_action ⇒ store ⇒ store set" where
  "edge_step κ EA_Nop s = {s}"
```

```
| "edge_step κ (EA_Assign x a) s = {s(x := kind_conv κ (Some x) (eval a s))}"
| "edge_step κ (EA_Special sc x) s = special_step κ sc x s"
| "edge_step κ (EA_Assume b) s = (if truthy (eval b s) then {s} else {})"
| "edge_step κ (EA_AssumeNot b) s = (if truthy (eval b s) then {} else {s})"
| "edge_step κ (EA_Body p) s = {s}"
| "edge_step κ (EA_Ret e p) s =
    {s(ReturnVar := (case e of None ⇒ s ReturnVar | Some a ⇒ eval a s))}"
| "edge_step κ (EA_Check l c) s = {s}"
```

One intra transition on a `(node, store)` pair. An intra path is its `star`, so reflexivity, transitivity and the induction rule come from `HOL-IMP.Star` rather than from a bespoke closure. The name avoids `intra_step`, which is the *source*-level `pstep` fragment.

```
definition cfg_intra_step ::
    "kenv ⇒ (var_id, exp) gcfg ⇒ cfg_node × store ⇒ cfg_node × store ⇒ bool" where
  "cfg_intra_step κ g p q ⟷
     (∃a. (fst p, a, fst q) ∈ intra g ∧ snd q ∈ edge_step κ a (snd p))"

lemma cfg_intra_stepI [intro]:
  "(u, a, v) ∈ intra g ⟹ s' ∈ edge_step κ a s ⟹ cfg_intra_step κ g (u, s) (v, s')"
  by (auto simp: cfg_intra_step_def)

lemma cfg_intra_stepE [elim]:
  assumes "cfg_intra_step κ g (u, s) (v, s')"
  obtains a where "(u, a, v) ∈ intra g" "s' ∈ edge_step κ a s"
  using assms by (auto simp: cfg_intra_step_def)

lemma intra_path_single:
  "(u, a, v) ∈ intra g ⟹ s' ∈ edge_step κ a s ⟹ star (cfg_intra_step κ g) (u, s) (v, s')"
  by (intro star_step1 cfg_intra_stepI)

lemma intra_path_nop:
  "(u, EA_Nop, v) ∈ intra g ⟹ star (cfg_intra_step κ g) (u, s) (v, s)"
  by (rule intra_path_single[where a = EA_Nop]) simp_all
```

An intra path, `star (cfg_intra_step κ g)`, is the reflexive-transitive closure of `edge_step` along `intra` edges, on `(node, store)` pairs. It deliberately excludes `calls`: an intra path never pushes or pops an activation, so it lifts to the collecting semantics without any stack-representation side condition, and to a stack-preserving CFG run at any stack.

Node set (derived, not stored)

`cfg_nodes` is reconstructed from the endpoints of `intra`, every node occurring in `calls`, and `cfg_entry`. It is not a record field.

```
definition cfg_nodes :: "('v, 'e) gcfg ⇒ cfg_node set" where
  "cfg_nodes g =
     {u. ∃a v. (u, a, v) ∈ intra g}
     ∪ {v. ∃u a. (u, a, v) ∈ intra g}
     ∪ {u. ∃act ce after. (u, act, ce, after) ∈ calls g}
     ∪ {ce. ∃u act after. (u, act, ce, after) ∈ calls g}
     ∪ {after. ∃u act ce. (u, act, ce, after) ∈ calls g}
     ∪ {cfg_entry g}"
```

Whole-program completion is the entry procedure's `FunctionResult`. A graph whose entry is not a procedure entry is never compiled; the fallback keeps the function total and aborts in generated code.

```
definition cfg_exit :: "('v, 'e) gcfg ⇒ cfg_node" where
  "cfg_exit g =
    (case cfg_entry g of
       FunctionEntry p ⇒ FunctionResult p
     | n ⇒ Code.abort (STR ''cfg_exit: entry is not a procedure entry'') (λ_. n))"
```

Well-formedness

`wf_cfg` is structural only: call edges enter procedure-entry nodes; no intra edge enters a procedure-entry node (so a callee is reached only across a call); and a return action lands on the matching procedure result. No compiler-correctness or source-semantic property is included.

```
definition wf_cfg :: "('v, 'e) gcfg ⇒ bool" where
  "wf_cfg g ⟷
     (∀u act ce after. (u, act, ce, after) ∈ calls g ⟶ (∃p. ce = FunctionEntry p))
   ∧ (∀u a v. (u, a, v) ∈ intra g ⟶ (∀p. v ≠ FunctionEntry p))
   ∧ (∀u e p v. (u, EA_Ret e p, v) ∈ intra g ⟶ v = FunctionResult p)"
```

`calls_source_unique` is a separate, optional side condition: one call edge per call site. `wf_cfg` does not include it, and the collecting semantics never needs it — both the return clause and the context semantics name an edge by its effect. A solved routed table does need it, to match the edge a return reads against the one its callee was entered through; compiled programs satisfy it by construction.

```
definition calls_source_unique :: "('v, 'e) gcfg ⇒ bool" where
  "calls_source_unique g ⟷
     (∀u ca1 ce1 af1 ca2 ce2 af2.
        (u, ca1, ce1, af1) ∈ calls g ⟶ (u, ca2, ce2, af2) ∈ calls g
        ⟶ ca1 = ca2 ∧ ce1 = ce2 ∧ af1 = af2)"

lemma calls_source_unique_edgesD:
  assumes "calls_source_unique g"
    and "(u, ca1, ce1, af1) ∈ calls g" and "(u, ca2, ce2, af2) ∈ calls g"
  shows "ca1 = ca2" and "ce1 = ce2" and "af1 = af2"
  using assms unfolding calls_source_unique_def by blast+
```

Structural invariants

What holds of a graph's node set no matter where the graph came from: an edge's endpoints are nodes, and finitely many edges give finitely many nodes. Both are read off the definition of `cfg_nodes`, which is why they need no well-formedness assumption.

```
lemma intra_endpoints_in_nodes:
  assumes "(u, a, v) ∈ intra g"
  shows "u ∈ cfg_nodes g" and "v ∈ cfg_nodes g"
  using assms by (auto simp: cfg_nodes_def)

lemma call_endpoints_in_nodes:
  assumes "(u, act, ce, after) ∈ calls g"
  shows "u ∈ cfg_nodes g" and "ce ∈ cfg_nodes g" and "after ∈ cfg_nodes g"
  using assms by (auto simp: cfg_nodes_def)
```

Each of the five `cfg_nodes` disjuncts is a projection of `intra g` or `calls g`, so it inherits their finiteness as a finite image; the sixth is a singleton.

```
lemma cfg_nodes_finite:
  assumes "finite (intra g)" and "finite (calls g)"
  shows "finite (cfg_nodes g)"
proof -
  have "{u. ∃a v. (u, a, v) ∈ intra g} = (λ(u, a, v). u) ‘ intra g"
    and "{v. ∃u a. (u, a, v) ∈ intra g} = (λ(u, a, v). v) ‘ intra g"
    by force+
  moreover
  have "{u. ∃act ce after. (u, act, ce, after) ∈ calls g} = (λ(u, act, ce, after). u) ‘ calls g"
    and "{ce. ∃u act after. (u, act, ce, after) ∈ calls g} = (λ(u, act, ce, after). ce) ‘ calls g"
    and "{after. ∃u act ce. (u, act, ce, after) ∈ calls g}
           = (λ(u, act, ce, after). after) ‘ calls g"
    by force+
  ultimately show ?thesis
    unfolding cfg_nodes_def using assms by simp
qed
```

Executable orders

Structural orders make `sorted_list_of_set` a deterministic executable enumeration of intra and call relations. They affect only solver representation, not CFG semantics. `src_exp` already derives `linorder` in `Voblint_VIMP.VIMP_Syntax`.

```
derive linorder gspecial_call
derive linorder gedge_action
derive linorder gcall_action
derive linorder cfg_node
```

Executable enumeration

Stable list views for the TD bridge: the intra and call edge sets sorted by their structural order. Both guard on `finite (intra g)`/`finite (calls g)`: every compiled graph satisfies it, so the `Code.abort` branch never fires in practice, and names the violation instead of failing on an uninformative `sorted_list_of_set` pattern-match error. Placed here rather than in the framework session's own equation-generation enumerations, so any consumer of this session alone — a compiled graph's `finite_cfg` interpretation among them — gets them for free.

```
definition cfg_intra_list ::
    "('v::linorder, 'e::linorder) gcfg ⇒ (cfg_node × ('v, 'e) gedge_action × cfg_node) list" where
  "cfg_intra_list g =
```

```
     (if finite (intra g) then sorted_list_of_set (intra g)
      else Code.abort (STR ''cfg_intra_list: infinite intra edge set'') (λ_. []))"

lemma cfg_intra_list_code [code]:
  "cfg_intra_list g = sorted_list_of_set (intra g)"
  unfolding cfg_intra_list_def by (cases "finite (intra g)") auto

definition cfg_calls_list ::
    "('v::linorder, 'e::linorder) gcfg
       ⇒ (cfg_node × ('v, 'e) gcall_action × cfg_node × cfg_node) list" where
  "cfg_calls_list g =
     (if finite (calls g) then sorted_list_of_set (calls g)
      else Code.abort (STR ''cfg_calls_list: infinite calls edge set'') (λ_. []))"

lemma cfg_calls_list_code [code]:
  "cfg_calls_list g = sorted_list_of_set (calls g)"
  unfolding cfg_calls_list_def by (cases "finite (calls g)") auto

lemma set_cfg_intra_list [simp]:
  "finite (intra g) ⟹ set (cfg_intra_list g) = intra g"
  unfolding cfg_intra_list_def by simp

lemma set_cfg_calls_list [simp]:
  "finite (calls g) ⟹ set (cfg_calls_list g) = calls g"
  unfolding cfg_calls_list_def by simp
```

Every `cfg_nodes` element, listed once: the intra endpoints, the call-site/ callee-entry/continuation triples, and the distinguished entry, deduplicated. This is the canonical finite node enumeration a public per-node result table draws its key domain from, distinct from `cfg_intra_list`'s edge-indexed view.

```
definition cfg_node_list :: "('v::linorder, 'e::linorder) gcfg ⇒ cfg_node list" where
  "cfg_node_list g =
     remdups
       (concat (map (λ(u, a, v). [u, v]) (cfg_intra_list g)) @
        concat (map (λ(u, act, ce, after). [u, ce, after]) (cfg_calls_list g)) @
        [cfg_entry g])"

lemma set_cfg_node_list [simp]:
  assumes "finite (intra g)" and "finite (calls g)"
  shows "set (cfg_node_list g) = cfg_nodes g"
  unfolding cfg_node_list_def cfg_nodes_def
  using set_cfg_intra_list[OF assms(1)] set_cfg_calls_list[OF assms(2)]
  by force
```

A `cfg` whose two transition relations are finite. Bundles the two finiteness facts every enumeration and every compiled-graph interpretation needs, so a caller states one assumption instead of two, and `finite_nodes` becomes derivable rather than re-proved at each instance.

```
locale finite_cfg =
  fixes g :: "('v, 'e) gcfg"
  assumes finite_intra [intro, simp]: "finite (intra g)"
    and finite_calls [intro, simp]: "finite (calls g)"
begin

lemma finite_nodes [simp]: "finite (cfg_nodes g)"
  by (rule cfg_nodes_finite) simp_all

end
```

Expressions observed by an edge, shared by diagnostics and transfer footprints.

```
fun edge_expressions :: "('v, 'e) gedge_action ⇒ 'e list" where
  "edge_expressions (EA_Assign x e) = [e]"
| "edge_expressions (EA_Assume e) = [e]"
| "edge_expressions (EA_AssumeNot e) = [e]"
| "edge_expressions (EA_Check l e) = [e]"
| "edge_expressions (EA_Ret (Some e) p) = [e]"
| "edge_expressions (EA_Special (Min a b) x) = [a, b]"
| "edge_expressions (EA_Special (Max a b) x) = [a, b]"
| "edge_expressions _ = []"

end
```

4.2 CFG_Transfer

```
theory CFG_Transfer
  imports CFG_Def
begin
```

What each kind of edge does to the store

Three transfers, one per thing an edge can do. `edge_collect` runs an ordinary edge on a whole set of stores at once, by lifting the single-store `edge_step` pointwise — derived from the concrete step rather than restated, so the two cannot drift apart. `call_enter` builds the callee's opening store: evaluate the actuals in the caller, reset the locals, bind the formals. `combine_collect` does the reverse at a return: keep the callee's globals, restore the caller's locals, and write the callee's return value into the destination.

All the payload comes from the edge itself, so none of these needs the procedure table.

```
definition edge_collect :: "kenv ⇒ (var_id, exp) gedge_action ⇒ store set ⇒ store set" where
  "edge_collect κ a S = {t. ∃s∈S. t ∈ edge_step κ a s}"

lemma edge_collect_simps [simp]:
  "edge_collect κ EA_Nop S = S"
  "edge_collect κ (EA_Assign x a) S = {s(x := kind_conv κ (Some x) (eval a s)) | s. s ∈ S}"
  "edge_collect κ (EA_Special sc x) S = {t. ∃s∈S. t ∈ special_step κ sc x s}"
  "edge_collect κ (EA_Assume b) S = {s. s ∈ S ∧ truthy (eval b s)}"
  "edge_collect κ (EA_AssumeNot b) S = {s. s ∈ S ∧ ¬ truthy (eval b s)}"
  "edge_collect κ (EA_Body p) S = S"
  "edge_collect κ (EA_Ret e p) S =
     {s(ReturnVar := (case e of None ⇒ s ReturnVar | Some a ⇒ eval a s)) | s. s ∈ S}"
  "edge_collect κ (EA_Check l c) S = S"
  unfolding edge_collect_def by (auto split: if_splits)

lemma edge_collect_single:
  "edge_collect κ a {s} = edge_step κ a s"
  by (cases a) (auto split: if_splits)

lemma edge_collect_empty_set [simp]: "edge_collect κ a {} = {}"
  by (cases a) auto

lemma edge_collect_mono:
  assumes "S ⊆ T"
  shows "edge_collect κ a S ⊆ edge_collect κ a T"
  using assms by (cases a) auto
```

Return-value transfer

Return-value rehydration at the caller: write the callee's return slot into the destination over the combined store (callee globals, caller locals), converted to the destination. It is fixed by the call's destination `dst`, which the `CallEdge` already records — no side lookup.

```
definition combine_collect ::
    "kenv ⇒ var_id option ⇒ store ⇒ store ⇒ store" where
  "combine_collect κ dst s t = combine_assign dst (kind_conv κ dst (t ReturnVar)) (combine_env is_global s t)"
```

Call-entry transfer

Caller-side entry transfer at a call. The actuals are evaluated in the caller store, the callee locals are reset (`enter_state`, globals preserved), and the resulting values are bound to the callee formals, each converted to its formal. All payload comes from the `CallEdge`, so the transfer needs no procedure table. This is exactly the entered store produced by the source `pstep Call` rule.

```
definition call_enter :: "kenv ⇒ (var_id, exp) gcall_action ⇒ store ⇒ store" where
  "call_enter κ ca s =
     (case ca of CallEdge dst pars actuals ⇒
        bind_formals pars (map2 (λx. kind_conv κ (Some x)) pars (map (λe. eval e s) actuals))
          (enter_state is_global s))"

lemma call_enter_CallEdge:
  "call_enter κ (CallEdge dst pars actuals) s
     = bind_formals pars (map2 (λx. kind_conv κ (Some x)) pars (map (λe. eval e s) actuals))
         (enter_state is_global s)"
  by (simp add: call_enter_def)
```

A parameterless call is exactly `enter_state`: no actuals to evaluate and no formals to bind.

```
lemma call_enter_Nil [simp]:
```

```
  "call_enter κ (CallEdge dst [] []) s = enter_state is_global s"
  by (simp add: call_enter_CallEdge)
```

What an edge may change

An edge may change only the variable it assigns, and a return only its result slot.

```
fun edge_writes :: "(var_id, exp) gedge_action ⇒ var_id list" where
  "edge_writes (EA_Assign x a) = [x]"
| "edge_writes (EA_Special sc x) = [x]"
| "edge_writes (EA_Ret e p) = [ReturnVar]"
| "edge_writes _ = []"
```

The frame each concrete step respects: a name outside its write set keeps its value. A call entry may only bind its formals, and a return takes every global from the callee except the destination it assigns.

```
lemma edge_step_frame:
  "t ∈ edge_step κ a s ⟹ x ∉ set (edge_writes a) ⟹ t x = s x"
  by (cases a) (auto split: if_splits)

lemma call_enter_frame:
  "is_global x ⟹ x ∉ set pars ⟹ call_enter κ (CallEdge dst pars args) s x = s x"
  by (simp add: call_enter_CallEdge bind_formals_other enter_state_def)

lemma combine_collect_frame:
  "is_global x ⟹ dst ≠ Some x ⟹ combine_collect κ dst s t x = t x"
  by (cases dst) (auto simp: combine_collect_def)
```

The write set runs in generated code.

```
lemmas [code] = edge_writes.simps
```

What an edge reads

An edge reads the variables of its expressions.

```
definition edge_reads :: "(var_id, exp) gedge_action ⇒ var_id list" where
  "edge_reads a = sorted_list_of_set (⋃ (exp_vars ‘ set (edge_expressions a)))"
```

Finite global footprints

The declared globals among those names, as duplicate-free lists a transfer folds over to read or publish them.

```
definition global_names_in :: "('v ⇒ bool) ⇒ 'v list ⇒ 'v list" where
  "global_names_in G xs = remdups (filter G xs)"

lemma set_global_names_in [simp]:
  "set (global_names_in G xs) = {x ∈ set xs. G x}"
  by (simp add: global_names_in_def)

lemma distinct_global_names_in [simp]: "distinct (global_names_in G xs)"
  by (simp add: global_names_in_def)

definition edge_global_reads :: "(var_id ⇒ bool) ⇒ (var_id, exp) gedge_action ⇒ var_id list" where
  "edge_global_reads G a = global_names_in G (edge_reads a)"
```

The writes are those of the concrete step, so they follow the return slot.

```
definition edge_global_writes :: "(var_id ⇒ bool) ⇒ (var_id, exp) gedge_action ⇒ var_id list" where
  "edge_global_writes G a = global_names_in G (edge_writes a)"

definition call_global_reads :: "(var_id ⇒ bool) ⇒ exp list ⇒ var_id list" where
  "call_global_reads G es = global_names_in G (sorted_list_of_set (⋃ (exp_vars ‘ set es)))"

lemma set_edge_global_writes [simp]:
  "set (edge_global_writes G a) = {x ∈ set (edge_writes a). G x}"
  by (simp add: edge_global_writes_def)

lemma distinct_global_footprints [simp]:
  "distinct (edge_global_reads G a)"
  "distinct (edge_global_writes G a)"
  "distinct (call_global_reads G es)"
  by (simp_all add: edge_global_reads_def edge_global_writes_def call_global_reads_def)

end
```

4.3 CFG_Prune

```
theory CFG_Prune
  imports CFG_Transfer
begin
```

Procedure-aware structural reachability

`cfg_succ_rel` is the derived structural dependency graph the analysis pruning and cone proofs run on — not the concrete execution relation. It has four sources, induced by `intra g` and `calls g`:

- INTRA: an ordinary edge `(u, a, v) ∈ intra g` gives `u → v` (ordinary flow).
- ENTRY: a call `(c, ca, FunctionEntry p, k) ∈ calls g` gives `c → FunctionEntry p` — the callee entry's abstract state depends on the caller state routed through the analysis's own enter operation.
- `COMB_CALLER`: the same call gives `c → k` — the continuation depends on the saved caller state via `combine_collect`. This is not a concrete execution edge; execution does not skip the callee.
- `COMB_RESULT`: the same call gives `FunctionResult p → k` — the continuation depends on the callee's result, `combine_collect`'s callee-exit argument.

  `c → k` and `FunctionResult p → k` are combine dependencies of the analysis, kept visibly separate from `intra g`. They are never added to `intra g`.

The structural successor relation

```
definition cfg_succ_rel :: "('v, 'e) gcfg ⇒ (cfg_node × cfg_node) set" where
  "cfg_succ_rel g =
     {(u, v) | u a v. (u, a, v) ∈ intra g}
   ∪ {(c, ce) | c ca ce k. (c, ca, ce, k) ∈ calls g}
   ∪ {(c, k) | c ca ce k. (c, ca, ce, k) ∈ calls g}
   ∪ {(FunctionResult p, k) | c ca p k. (c, ca, FunctionEntry p, k) ∈ calls g}"

lemma cfg_succ_rel_intra:
  "(u, a, v) ∈ intra g ⟹ (u, v) ∈ cfg_succ_rel g"
  unfolding cfg_succ_rel_def by blast

lemma cfg_succ_rel_entry:
  "(c, ca, ce, k) ∈ calls g ⟹ (c, ce) ∈ cfg_succ_rel g"
  unfolding cfg_succ_rel_def by blast

lemma cfg_succ_rel_comb_caller:
  "(c, ca, ce, k) ∈ calls g ⟹ (c, k) ∈ cfg_succ_rel g"
  unfolding cfg_succ_rel_def by blast

lemma cfg_succ_rel_comb_result:
  "(c, ca, FunctionEntry p, k) ∈ calls g ⟹ (FunctionResult p, k) ∈ cfg_succ_rel g"
  unfolding cfg_succ_rel_def by blast

lemma cfg_succ_rel_cases:
  assumes "(y, z) ∈ cfg_succ_rel g"
  obtains (INTRA) a where "(y, a, z) ∈ intra g"
    | (ENTRY) ca k where "(y, ca, z, k) ∈ calls g"
    | (COMB_CALLER) ca ce where "(y, ca, ce, z) ∈ calls g"
    | (COMB_RESULT) c ca p k where "(c, ca, FunctionEntry p, k) ∈ calls g"
                                   "y = FunctionResult p" "z = k"
  using assms unfolding cfg_succ_rel_def by blast
```

Reachability: reflexive-transitive closure of the successor relation

Reachability over that successor relation. Because the relation already relates a caller to its continuation and a callee result to the same continuation, this closure crosses procedure boundaries without any separate interprocedural notion.

```
definition cfg_reaches :: "('v, 'e) gcfg ⇒ cfg_node ⇒ cfg_node ⇒ bool" where
  "cfg_reaches g v v0 ⟷ (v, v0) ∈ (cfg_succ_rel g)*"
```

Reachability inside one activation

The same closure without entering callees: a call site steps straight to its continuation. These are the steps a routed equation reads backwards, so this is the reachability the solver's coverage argument needs.

```
definition local_succ_rel :: "('v, 'e) gcfg ⇒ (cfg_node × cfg_node) set" where
  "local_succ_rel g =
     {(u, v). ∃a. (u, a, v) ∈ intra g}
     ∪ {(u, k). ∃ca q. (u, ca, FunctionEntry q, k) ∈ calls g}"

definition local_reaches :: "('v, 'e) gcfg ⇒ cfg_node ⇒ cfg_node ⇒ bool" where
  "local_reaches g u v ⟷ (u, v) ∈ (local_succ_rel g)*"

lemma local_reaches_refl [simp]: "local_reaches g v v"
  by (simp add: local_reaches_def)

lemma local_reaches_trans:
  "local_reaches g u v ⟹ local_reaches g v w ⟹ local_reaches g u w"
  unfolding local_reaches_def by (rule rtrancl_trans)

lemma local_reaches_intra_step:
  "(u, a, v) ∈ intra g ⟹ local_reaches g v w ⟹ local_reaches g u w"
  unfolding local_reaches_def local_succ_rel_def
  by (rule converse_rtrancl_into_rtrancl) blast+

lemma local_reaches_comb_step:
  "(u, ca, FunctionEntry q, k) ∈ calls g ⟹ local_reaches g k w ⟹ local_reaches g u w"
  unfolding local_reaches_def local_succ_rel_def
  by (rule converse_rtrancl_into_rtrancl) blast+

lemma local_reaches_cfg_reaches:
  "local_reaches g u v ⟹ cfg_reaches g u v"
  unfolding local_reaches_def cfg_reaches_def
proof (induction rule: rtrancl_induct)
  case (step y z)
  then have "(y, z) ∈ cfg_succ_rel g"
    unfolding local_succ_rel_def by (auto simp: cfg_succ_rel_intra cfg_succ_rel_comb_caller)
  with step.IH show ?case by (rule rtrancl_into_rtrancl)
qed simp

end
```

4.4 CFG_Exec

```
theory CFG_Exec
  imports CFG_Transfer
begin
```

How a control-flow graph runs

`cstep` executes an arbitrary CFG over a node, a store, and a stack of suspended activations. Each `cframe` records where to resume, which variable receives the result, and the caller's store. There are three rules, one per graph phenomenon: follow an `intra` edge and apply its transfer; follow a `calls` edge, enter the callee and push a frame; at a `FunctionResult`, pop the top frame and combine the callee's store into the caller's.

A return follows no edge. The stack supplies the continuation, so one `FunctionResult` node serves every caller of a procedure and recursion needs no duplicated nodes. Nothing here mentions the compiler: this is the execution of any graph, just as `valid_activation_trace` is the activation-trace semantics of any graph.

```
type_synonym cframe = "cfg_node × var_id option × store"
type_synonym cconf = "cfg_node × store × cframe list"

inductive cstep :: "kenv ⇒ (var_id, exp) gcfg ⇒ cconf ⇒ cconf ⇒ bool"
    ("(_,_ ⊢/ _ →c/ _)" [51, 51, 51, 51] 50) for κ g where
  Intra [intro]:
    "(u, a, v) ∈ intra g ⟹ s' ∈ edge_step κ a s ⟹ κ,g ⊢ (u, s, stk) →c (v, s', stk)"
| Call [intro]:
    "(u, CallEdge dst pars actuals, FunctionEntry q, cont) ∈ calls g ⟹
     κ,g ⊢ (u, s, stk)
       →c (FunctionEntry q, call_enter κ (CallEdge dst pars actuals) s, (cont, dst, s) # stk)"
| Return [intro]:
    "κ,g ⊢ (FunctionResult q, t, (cont, dst, caller) # stk)
       →c (cont, combine_collect κ dst caller t, stk)"
```

One inversion rule, because the three clauses are told apart by the edge taken rather than by the shape of the configuration. It stays plain `[elim]` so the classical reasoner does not split every `cstep` hypothesis three ways before trying anything else.

```
inductive_cases cstep_E [elim]: "κ,g ⊢ (u, s, stk) →c y"

abbreviation csteps :: "kenv ⇒ (var_id, exp) gcfg ⇒ cconf ⇒ cconf ⇒ bool"
    ("(_,_ ⊢/ _ →c*/ _)" [51, 51, 51, 51] 50) where
  "csteps κ g x y ≡ star (cstep κ g) x y"
```

Single-step and small-step lemmas

Introduction lemmas for the store-preserving edge actions the simulation steps through: body, assume and assume-not.

```
lemma cstep_body:
  assumes "(u, EA_Body p, v) ∈ intra g"
  shows "κ,g ⊢ (u, s, stk) →c (v, s, stk)"
  by (rule cstep.Intra[OF assms]) simp

lemma cstep_assume:
  assumes "(u, EA_Assume b, v) ∈ intra g" and "truthy (eval b s)"
  shows "κ,g ⊢ (u, s, stk) →c (v, s, stk)"
  by (rule cstep.Intra[OF assms(1)]) (use assms(2) in simp)

lemma cstep_assume_not:
  assumes "(u, EA_AssumeNot b, v) ∈ intra g" and "¬ truthy (eval b s)"
  shows "κ,g ⊢ (u, s, stk) →c (v, s, stk)"
  by (rule cstep.Intra[OF assms(1)]) (use assms(2) in simp)
```

An intra path is a `cstep` run at any stack: every `cfg_intra_step` is a `cstep.Intra`, which passes the stack through untouched. Callers that only need to move along local edges can therefore reason on `(node, store)` pairs and lift the result here.

```
lemma intra_path_imp_cstep_star:
  "star (cfg_intra_step κ g) x y ⟹ (κ,g ⊢ (fst x, snd x, stk) →c* (fst y, snd y, stk))"
proof (induction rule: star.induct)
  case (refl a) show ?case by simp
next
  case (step a b c)
  obtain ua sa where a: "a = (ua, sa)" by (cases a)
  obtain ub sb where b: "b = (ub, sb)" by (cases b)
  from step.hyps(1) a b obtain e where "(ua, e, ub) ∈ intra g" "sb ∈ edge_step κ e sa" by auto
```

```
  hence "κ,g ⊢ (ua, sa, stk) →c (ub, sb, stk)" by (rule cstep.Intra)
  with step.IH a b show ?case by (auto intro: star.step)
qed
```

Activation-stack matching

Both stacks record procedure activations. `act_frames` removes the CFG-only continuation component and retains the caller store and destination used by the source.

```
fun act_frames :: "frame list ⇒ (store × var_id option) list" where
  "act_frames [] = []"
| "act_frames (Frame s d # frs) = (s, d) # act_frames frs"

definition cframe_act :: "cframe ⇒ store × var_id option" where
  "cframe_act cf = (case cf of (cont, d, s) ⇒ (s, d))"
```

`frames_match` ties a source frame stack to a CFG activation stack: the activation frames agree, in order, on caller store and destination (the continuation node is the CFG's own bookkeeping and is not visible to the source).

```
definition frames_match :: "frame list ⇒ cframe list ⇒ bool" where
  "frames_match frs stk = (act_frames frs = map cframe_act stk)"

lemma frames_match_Nil [simp]: "frames_match [] []"
  by (simp add: frames_match_def)
```

An activation frame matches the top CFG activation on caller store and destination.

```
lemma frames_match_activation:
  "frames_match (Frame s d # frs) ((cont, d, s) # stk)
     = frames_match frs stk"
  by (simp add: frames_match_def cframe_act_def)

end
```

4.5 Activation_Trace_Def

```
theory Activation_Trace_Def
  imports CFG_Transfer
begin
```

Activation traces

An `activation_trace` represents one procedure activation of a sequential run and its concrete ancestry. Its own path contains `(cfg_node, store)` pairs; structural constructors record the suspended caller and, after resumption, the completed callee. Activation contexts are projections of this structure rather than fields stored in the trace.

Local flow follows `intra`, including `EA_Ret` into the matching `FunctionResult`. A `calls` tuple supplies the call site, callee entry, and continuation. The call rule builds the parameter-bound entered store with `call_enter`. The resume rule combines caller locals, callee globals, and the return value with `combine_collect`.

The shape is adapted from the local traces of Schwarz et al., *Improving Thread-Modular Abstract Interpretation* (SAS 2021), and Schwarz and Erhard, *Data Race Detection by Digest-Driven Abstract Interpretation* (arXiv:2511.11055, 2025). Theirs is a multithreaded semantics: a local trace is one thread's view, in which operations of different threads are only partially ordered. An activation trace is one activation of a sequential run, and a return composes the completed callee into its suspended caller.

The datatype

- `Root p` — the main activation, with local path `p`.
- `Call caller p` — a callee whose local path `p` starts at the entered store; `caller` is the exact suspended caller, frozen at the call node.
- `Resume current callee p` — the activation continued past a completed call. `current` is that activation frozen at its call node (the value that spawned `callee`); `callee` is the retained completed callee subtree; `p` is the continued path.

An `activation_path` lists the nodes one activation passed and the store it held at each. An activation trace carries the path of its own activation, with the surrounding activations beside it rather than on it.

```
type_synonym activation_path = "(cfg_node × store) list"

datatype activation_trace =
    Root "activation_path"
  | Call (activation_trace_caller: "activation_trace") "activation_path"
  | Resume (activation_trace_current: "activation_trace")
    (activation_trace_callee: "activation_trace") "activation_path"
```

Observers

`path_of` is the activation path. `sink_node` and `sink_store` return its final program point and final store.

```
fun path_of :: "activation_trace ⇒ activation_path" where
  "path_of (Root p)       = p"
| "path_of (Call _ p)     = p"
| "path_of (Resume _ _ p) = p"

definition entry_store :: "activation_trace ⇒ store" where
  "entry_store t = snd (hd (path_of t))"

definition sink_node :: "activation_trace ⇒ cfg_node" where
  "sink_node t = fst (last (path_of t))"

definition sink_store :: "activation_trace ⇒ store" where
  "sink_store t = snd (last (path_of t))"
```

`caller_of` recovers the creating caller of an activation. It descends the frozen `caller` field of a `Resume`, so it works uniformly for a returned callee of any constructor — this is what makes nested and recursive returns compose.

```
fun caller_of :: "activation_trace ⇒ activation_trace option" where
  "caller_of (Root _)             = None"
| "caller_of (Call caller _)      = Some caller"
| "caller_of (Resume current _ _) = caller_of current"
```

Extension and context projection

`extend` appends one step to the innermost local path; it never touches an outer constructor's caller/callee fields.

```
fun extend :: "activation_trace ⇒ (cfg_node * store) ⇒ activation_trace" where
  "extend (Root p) x       = Root (p @ [x])"
| "extend (Call c p) x     = Call c (p @ [x])"
| "extend (Resume c d p) x = Resume c d (p @ [x])"
```

The closure relation

`valid_activation_trace` is the least set closed under four concrete operations: an initial main activation at `cfg_entry`; an `intra` step; a call; and a return. Each rule reads exactly the relation for its phenomenon. `intra` carries no side condition — calls are not `intra` members, so they are untraversable by typing. `call` enters the callee named by the `calls` edge at the entered store `call_enter`. `ret` matches the callee's `FunctionResult p` against the `FunctionEntry p` of a concrete `calls` edge leaving the caller's node, and resumes at the continuation stored in that same edge; the resumed state is `combine_collect`. There is no `combines` lookup and no scan for a compatible context.

```
inductive_set valid_activation_trace ::
    "kenv ⇒ (var_id, exp) gcfg ⇒ store set ⇒ activation_trace set"
    ("𝒯_,_,_")
  for κ g and S where
  root:
    "s ∈ S
     ⟹ Root [(cfg_entry g, s)] ∈ 𝒯κ,g,S"
| intra:
    "t ∈ 𝒯κ,g,S
     ⟹ (sink_node t, a, v) ∈ intra g
     ⟹ s' ∈ edge_step κ a (sink_store t)
     ⟹ extend t (v, s') ∈ 𝒯κ,g,S"
| call:
    "caller ∈ 𝒯κ,g,S
     ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont)
          ∈ calls g
     ⟹ Call caller
          [(FunctionEntry p,
            call_enter κ (CallEdge dst pars args) (sink_store caller))]
        ∈ 𝒯κ,g,S"
| ret:
    "callee ∈ 𝒯κ,g,S
     ⟹ caller_of callee = Some caller
     ⟹ sink_node callee = FunctionResult p
     ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont)
          ∈ calls g
     ⟹ Resume caller callee
          (path_of caller
             @ [(cont, combine_collect κ dst
                         (sink_store caller) (sink_store callee))])
        ∈ 𝒯κ,g,S"

inductive_cases valid_activation_trace_RootE [elim]:
  "Root p ∈ 𝒯κ,g,S"

inductive_cases valid_activation_trace_CallE [elim]:
  "Call caller p ∈ 𝒯κ,g,S"

inductive_cases valid_activation_trace_ResumeE [elim]:
  "Resume caller callee p ∈ 𝒯κ,g,S"
```

Structural lemmas

How the three trace shapes answer the projections every later proof reads them through – path, sink, caller, entry store. Kept as `simp` rules so an induction over `valid_activation_trace` never has to case on the constructor merely to look up a sink.

```
lemma extend_simps [simp]:
  "path_of (extend t x) = path_of t @ [x]"
  "caller_of (extend t x) = caller_of t"
  "sink_node (extend t x) = fst x"
  "sink_store (extend t x) = snd x"
  by (cases t; auto simp: sink_node_def sink_store_def)+

lemma sink_single_simps [simp]:
  "sink_node (Root [(n, s)]) = n"
```

```
  "sink_store (Root [(n, s)]) = s"
  "sink_node (Call c [(n, s)]) = n"
  "sink_store (Call c [(n, s)]) = s"
  by (simp_all add: sink_node_def sink_store_def)
```

The lemmas up to the caller-chain closure all speak about the traces of one graph from one set of initial stores, so the block fixes $\mathcal{G}$, g and S and writes $\mathcal{T}$ for $\mathcal{T}_{\mathcal{G},\mathtt{g},\mathtt{S}}$. Outside the block every lemma holds for arbitrary $\mathcal{G}$, g and S.

```
lemma valid_activation_trace_path_nonempty:
  "t ∈ (𝒯_κ,g,S) ⟹ path_of t ≠ []"
  by (induction t rule: valid_activation_trace.induct) auto

lemma entry_store_extend [simp]:
  assumes "path_of t ≠ []"
  shows "entry_store (extend t x) = entry_store t"
  using assms by (simp add: entry_store_def)
```

Design invariants

A valid `activation_trace.Call` activation has a valid caller, even after intra steps have extended its local path.

```
lemma valid_activation_trace_Call_caller_valid:
  "u ∈ (𝒯_κ,g,S) ⟹ u = Call cc q ⟹ cc ∈ (𝒯_κ,g,S)"
proof (induction arbitrary: cc q rule: valid_activation_trace.induct)
  case (intra t a v s')
  from intra.prems obtain q' where "t = Call cc q'"
    by (cases t) auto
  then show ?case using intra.IH by simp
qed auto
```

A valid `Resume` retains its callee as a valid activation trace, and its frozen caller is forced to be exactly `caller_of callee` — a return cannot invent a caller.

```
lemma valid_activation_trace_Resume_fields:
  "u ∈ (𝒯_κ,g,S) ⟹ u = Resume cc dd q
   ⟹ dd ∈ (𝒯_κ,g,S) ∧ caller_of dd = Some cc"
proof (induction arbitrary: cc dd q rule: valid_activation_trace.induct)
  case (intra t a v s')
  from intra.prems obtain q' where "t = Resume cc dd q'"
    by (cases t) auto
  then show ?case using intra.IH by simp
qed auto
```

Every caller recovered from a valid activation trace by `caller_of` is itself valid.

```
lemma valid_activation_trace_caller_valid:
  "t ∈ (𝒯_κ,g,S) ⟹ caller_of t = Some c ⟹ c ∈ (𝒯_κ,g,S)"
proof (induction t arbitrary: c)
  case (Root x)
  then show ?case by simp
next
  case (Call caller p)
  have "caller ∈ (𝒯_κ,g,S)"
    using valid_activation_trace_Call_caller_valid[OF Call.prems(1) refl] .
  with Call.prems(2) show ?case by simp
next
  case (Resume caller callee p)
  from valid_activation_trace_Resume_fields[OF Resume.prems(1) refl]
  have cd: "callee ∈ (𝒯_κ,g,S)" "caller_of callee = Some caller" by auto
  have cv: "caller ∈ (𝒯_κ,g,S)" using Resume.IH(2)[OF cd(1)] cd(2) by simp
  from Resume.prems(2) have "caller_of caller = Some c" by simp
  then show ?case using Resume.IH(1)[OF cv] by simp
qed
```

A `Root` activation starts at `cfg_entry`: `root` creates it there and `intra` only appends.

```
lemma valid_activation_trace_Root_entry:
  "u ∈ (𝒯_κ,g,S) ⟹ u = Root p ⟹ fst (hd p) = cfg_entry g"
proof (induction arbitrary: p rule: valid_activation_trace.induct)
  case (intra t a v s')
  from intra.prems obtain p' where t: "t = Root p'" and p: "p = p' @ [(v, s')]"
    by (cases t) auto
  have "fst (hd p') = cfg_entry g" using intra.IH[OF t] .
  moreover have "p' ≠ []" using valid_activation_trace_path_nonempty[OF intra.hyps(1)] t by simp
  ultimately show ?case using p by simp
qed auto
```

A `Resume` extends its caller's own local path, so both share a head node.

```
lemma valid_activation_trace_Resume_path:
  "u ∈ (𝒯_κ,g,S) ⟹ u = Resume cc dd q ⟹ ∃xs. q = path_of cc @ xs ∧ xs ≠ []"
proof (induction arbitrary: cc dd q rule: valid_activation_trace.induct)
  case (intra t a v s')
  from intra.prems obtain q' where t: "t = Resume cc dd q'" and q: "q = q' @ [(v, s')]"
    by (cases t) auto
  from intra.IH[OF t] obtain xs where "q' = path_of cc @ xs" by blast
  then show ?case using q by auto
qed auto
```

A callerless activation is the root one, and its local `path_of` starts at `cfg_entry`. The `Resume` case needs the caller's own entry, which term induction supplies (the caller is a subterm) — rule induction would only offer the callee.

```
lemma valid_activation_trace_caller_None_entry:
  "t ∈ (𝒯_κ,g,S) ⟹ caller_of t = None ⟹ fst (hd (path_of t)) = cfg_entry g"
proof (induction t)
  case (Root p)
  then show ?case using valid_activation_trace_Root_entry[OF Root.prems(1) refl] by simp
next
  case (Call caller p)
  then show ?case by simp
next
  case (Resume caller callee q)
  from valid_activation_trace_Resume_fields[OF Resume.prems(1) refl]
  have cd: "callee ∈ (𝒯_κ,g,S)" "caller_of callee = Some caller" by auto
  have cv: "caller ∈ (𝒯_κ,g,S)" using Resume.IH(2)[OF cd(1)] cd(2)
    using valid_activation_trace_caller_valid[OF cd(1) cd(2)] by simp
  from Resume.prems(2) have "caller_of caller = None" by simp
  with Resume.IH(1)[OF cv] have hcaller: "fst (hd (path_of caller)) = cfg_entry g" by simp
  from valid_activation_trace_Resume_path[OF Resume.prems(1) refl] obtain xs where
    q: "q = path_of caller @ xs" by blast
  show ?case using hcaller q valid_activation_trace_path_nonempty[OF cv] by simp
qed
```

Caller ancestry

`ancestors t` is the `caller_of` chain above `t`; `callers t` adds `t` itself and is the set a caller-chain invariant ranges over.

```
fun ancestors :: "activation_trace ⇒ activation_trace set" where
  "ancestors (Root _) = {}"
| "ancestors (Call caller _) = insert caller (ancestors caller)"
| "ancestors (Resume current _ _) = ancestors current"

abbreviation callers :: "activation_trace ⇒ activation_trace set" where
  "callers t ≡ insert t (ancestors t)"

lemma ancestors_extend [simp]: "ancestors (extend t x) = ancestors t"
  by (cases t) simp_all

lemma callers_refl: "t ∈ callers t"
  by simp

lemma ancestors_caller: "caller_of t = Some c ⟹ callers c ⊆ ancestors t"
  by (induction t) auto

lemma callers_caller_subset: "caller_of t = Some c ⟹ callers c ⊆ callers t"
  using ancestors_caller by blast
```

Generic caller-chain closure

A caller-chain-quantified predicate `P` holds along the whole `callers` chain of every valid activation trace once its four `valid_activation_trace` obligations are discharged. Each obligation reads its own generating trace's induction hypothesis as the whole-chain fact `∀u ∈ callers _. P u`, not a single-node fact — this is exactly what `ret` needs to recover its caller's own chain fact from the callee's.

```
lemma caller_chain_closure:
  fixes P :: "activation_trace ⇒ bool"
  assumes Root: "⋀s. s ∈ S ⟹ P (Root [(cfg_entry g, s)])"
    and Intra: "⋀t a v s'. t ∈ (𝒯_κ,g,S) ⟹ (∀u ∈ callers t. P u)
        ⟹ (sink_node t, a, v) ∈ intra g ⟹ s' ∈ edge_step κ a (sink_store t)
        ⟹ P (extend t (v, s'))"
    and Call: "⋀caller dst pars args p cont. caller ∈ (𝒯_κ,g,S)
```

```
        ⟹ (∀u ∈ callers caller. P u)
        ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont)
           ∈ calls g
        ⟹ P (Call caller
                [(FunctionEntry p, call_enter κ (CallEdge dst pars args) (sink_store caller))])"
    and Ret: "⋀callee caller p dst pars args cont. callee ∈ (𝒯κ,g,s)
        ⟹ (∀u ∈ callers callee. P u)
        ⟹ caller_of callee = Some caller ⟹ sink_node callee = FunctionResult p
        ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont)
           ∈ calls g
        ⟹ P (Resume caller callee (path_of caller
                @ [(cont, combine_collect κ dst (sink_store caller) (sink_store callee))]))"
  shows "t ∈ (𝒯κ,g,s) ⟹ ∀u ∈ callers t. P u"
proof (induction rule: valid_activation_trace.induct)
  case (root s)
  then show ?case using Root[OF root] by simp
next
  case (intra t a v s')
  then show ?case using Intra[OF intra.hyps(1) intra.IH intra.hyps(2,3)] by auto
next
  case (call caller dst pars args p cont)
  then show ?case using Call[OF call.hyps(1) call.IH call.hyps(2)] by auto
next
  case (ret callee caller p dst pars args cont)
  then show ?case
    using Ret[OF ret.hyps(1) ret.IH ret.hyps(2,3,4)] ancestors_caller[OF ret.hyps(2)] by auto
qed
```

Stable context entry invariant

An activation's entry store is fixed once it starts: a call names the store the callee begins with, and resuming a caller keeps the caller's own first store rather than adopting the callee's. Context policies that read the entry store rely on this – otherwise the context a trace carries could change under it as the trace grows.

```
definition call_enter_store ::
    "kenv ⇒ (var_id, exp) gcfg ⇒ cfg_node ⇒ store ⇒ store ⇒ bool" where
  "call_enter_store κ g c s t ⟷
     (∃dst pars args p cont. (c, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
        ∧ t = call_enter κ (CallEdge dst pars args) s)"


lemma entry_store_Resume_caller:
  "path_of caller ≠ [] ⟹
     entry_store (Resume caller callee (path_of caller @ [x])) = entry_store caller"
  by (simp add: entry_store_def hd_append)

end
```

4.6 Activation_Trace_Context

```
theory Activation_Trace_Context
  imports Activation_Trace_Def
begin
```

Activation context selection and context-indexed collecting

The activation context is computed from the concrete trace after the fact and is activation-stable: fixed when the activation is created and unchanged by the calls it later makes and returns from. `Root` carries the seed; a `activation_trace.Call` routes the caller context on the call site and the entered store — `sink_node parent` is exactly the call site, since `Activation_Trace_Def.extend` only ever appends to the callee path and leaves `parent` frozen there; a `Resume` keeps the resumed caller's own context, so a completed call does not repartition the caller. Exposing the call site to `enterc` is what makes a call-site-keyed context (a k-call-string) expressible as an instance rather than only as a generator-side hook.

```
fun activation_context_of ::
    "(cfg_node ⇒ 'c ⇒ store ⇒ 'c) ⇒ 'c ⇒ activation_trace ⇒ 'c" where
  "activation_context_of enterc initial_ctx (Root _)                  = initial_ctx"
| "activation_context_of enterc initial_ctx (Call parent p)           =
     enterc (sink_node parent) (activation_context_of enterc initial_ctx parent) (snd (hd p))"
| "activation_context_of enterc initial_ctx (Resume current callee _) = activation_context_of enterc initial_ctx current"
```

`activation_context_of` fixes one context per trace by decoding the entered store exactly: a `activation_trace.Call` activation's context is derived from its caller's OWN selected context, not re-derived from the concrete store independently at each node. This is what keeps a chosen context stable across a CALL/RETURN pair: the callee's context in `Resume` resumes through `current`, so a completed call cannot repartition the caller.

`activation_context_of` is the deterministic special case: a total function from the entered store to a context, never a relation admitting several contexts for one concrete activation. The unit and call-site/k-call-string routing instances are exactly this shape, and the executable routed solver's own `route` is typed to return a single context, not a set. The entry-state instance is not: a call may answer several overlapping alternatives, each covering the same concrete activation and routing it to its own context, so its concrete semantics is stated directly over `activation_context_rel`/`call_context_policy` rather than over `activation_context_of`. `activation_context_of` is therefore not the primary semantics; it is the deterministic compatibility witness for the functional case, connected to `activation_context_rel` via `context_policy_of_fun` and `activation_context_rel_of_fun_iff`.

```
lemma activation_context_extend_nonempty:
  "path_of t ≠ [] ⟹ activation_context_of enterc initial_ctx (extend t x) = activation_context_of enterc initial_ctx
t"
  by (cases t) (auto simp: hd_append)
```

Admissible callee contexts, as a set per call

An analysis's entry operation answers a *list* of alternatives and its context selector is applied to each, so one concrete call may be admitted in several contexts. `call_context_policy` is the semantic shape of that whole-call protocol: given the call site, the caller's own context, the call's information, the caller store and the entered store, it returns the set of callee contexts that may describe this transition. The set may be empty or have several elements; `call_context_total_on` below requires it to be non-empty for every covered call. Both stores are arguments so an instance can tie the chosen context to one alternative whose continuation half covers the caller and whose entry half covers the entered store, never mixing two.

A functional policy — the unit context, a k-call-string — returns a singleton, `context_policy_of_fun`. The abstract selector an analysis actually runs stays a function of the abstract entry state; only the activation-trace semantics is set-valued.

```
type_synonym 'c call_context_policy =
  "cfg_node ⇒ 'c ⇒ call_info ⇒ store ⇒ store ⇒ 'c set"

definition context_policy_of_fun ::
  "(cfg_node ⇒ 'c ⇒ store ⇒ 'c) ⇒ 'c call_context_policy"
where
  "context_policy_of_fun f u ctx ci caller entered = {f u ctx entered}"

lemma context_policy_of_fun_iff [simp]:
  "ctx' ∈ context_policy_of_fun f u ctx ci caller entered ⟷ ctx' = f u ctx entered"
  by (simp add: context_policy_of_fun_def)

lemma context_policy_of_fun_nonempty [simp]:
  "context_policy_of_fun f u ctx ci caller entered ≠ {}"
  by (simp add: context_policy_of_fun_def)
```

The context-indexed collecting semantics over the activation-trace semantics `valid_activation_trace`: which activation a context policy admits, and what it collects.

`admits_call_context κ g adm u ctx p s es ctx'`: from node `u` in context `ctx` with store `s`, some `calls` edge of `g` enters `p` with exactly the store `es`, and `adm` admits `ctx'` for that edge. It is `call_enter_store` with the callee and the admitted context made explicit. Naming the edge by its effect rather than assuming it unique keeps the semantics honest on a graph with two call edges leaving one node, exactly as `valid_activation_trace.ret` does.

```
definition admits_call_context ::
  "kenv ⇒ (var_id, exp) gcfg ⇒ 'c call_context_policy
     ⇒ cfg_node ⇒ 'c ⇒ pname ⇒ store ⇒ store ⇒ 'c ⇒ bool"
where
  "admits_call_context κ g adm u ctx p s es ctx' ⟷
     (∃dst pars args cont.
        (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
        ∧ es = call_enter κ (CallEdge dst pars args) s
        ∧ ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) p) s es)"

lemma admits_call_contextI:
  assumes "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and "ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) p) s
           (call_enter κ (CallEdge dst pars args) s)"
  shows "admits_call_context κ g adm u ctx p s (call_enter κ (CallEdge dst pars args) s) ctx'"
  using assms unfolding admits_call_context_def by blast

lemma admits_call_contextE:
  assumes "admits_call_context κ g adm u ctx p s es ctx'"
  obtains dst pars args cont
    where "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
      and "es = call_enter κ (CallEdge dst pars args) s"
      and "ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) p) s es"
  using assms unfolding admits_call_context_def by blast

lemma admits_call_context_call_enter_storeD:
  "admits_call_context κ g adm u ctx p s es ctx' ⟹ call_enter_store κ g u s es"
  unfolding admits_call_context_def call_enter_store_def by blast

lemma admits_call_context_of_fun:
  "admits_call_context κ g (context_policy_of_fun f) u ctx p s es ctx'
     ⟷ (∃dst pars args cont.
           (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
           ∧ es = call_enter κ (CallEdge dst pars args) s)
       ∧ ctx' = f u ctx es"
  unfolding admits_call_context_def by auto
```

`activation_context_rel κ adm c₀ g t ctx`: the trace `t` may carry context `ctx`. A root carries c_0; a call carries any context admitted for its creating transition, computed from the caller's own carried context; a resumed caller keeps what it carried.

```
inductive activation_context_rel ::
  "kenv ⇒ 'c call_context_policy ⇒ 'c ⇒ (var_id, exp) gcfg ⇒ activation_trace ⇒ 'c ⇒ bool"
  for κ :: kenv and adm :: "'c call_context_policy" and c₀ :: 'c and g :: "(var_id, exp) gcfg"
where
  Root [intro]: "activation_context_rel κ adm c₀ g (Root xs) c₀"
| Call [intro]:
    "activation_context_rel κ adm c₀ g caller c
     ⟹ admits_call_context κ g adm (sink_node caller) c p (sink_store caller) es c'
     ⟹ activation_context_rel κ adm c₀ g (Call caller ((FunctionEntry p, es) # xs)) c'"
| Resume [intro]:
    "activation_context_rel κ adm c₀ g current c
     ⟹ activation_context_rel κ adm c₀ g (Resume current callee xs) c"


inductive_cases activation_context_rel_RootE [elim!]:
  "activation_context_rel κ adm c₀ g (Root xs) ctx"

inductive_cases activation_context_rel_CallE [elim]:
  "activation_context_rel κ adm c₀ g (Call parent xs) ctx"

inductive_cases activation_context_rel_ResumeE [elim]:
  "activation_context_rel κ adm c₀ g (Resume current callee xs) ctx"

lemma activation_context_rel_Root_iff [simp]:
  "activation_context_rel κ adm c₀ g (Root xs) ctx ⟷ ctx = c₀"
  by blast
```

```
lemma activation_context_rel_Resume_iff [simp]:
  "activation_context_rel κ adm c₀ g (Resume current callee xs) ctx
     ⟷ activation_context_rel κ adm c₀ g current ctx"
  by blast

lemma activation_context_rel_Call_Nil [simp]:
  "¬ activation_context_rel κ adm c₀ g (Call parent []) ctx"
  by blast
```

The carried context of a called activation depends on its caller and its entry record only: the rest of the local path is irrelevant, which is what makes an intra step context-preserving.

```
lemma activation_context_rel_Call_iff:
  "activation_context_rel κ adm c₀ g (Call parent ((n, es) # xs)) ctx'
     ⟷ (∃ctx p. n = FunctionEntry p
           ∧ activation_context_rel κ adm c₀ g parent ctx
           ∧ admits_call_context κ g adm (sink_node parent) ctx p (sink_store parent) es ctx')"
  by blast

lemma activation_context_rel_extend:
  assumes "path_of t ≠ []"
  shows "activation_context_rel κ adm c₀ g (extend t x) ctx
          ⟷ activation_context_rel κ adm c₀ g t ctx"
proof (cases t)
  case (Call parent xs)
  then obtain n es ys where "xs = (n, es) # ys" using assms by (cases xs) auto
  then show ?thesis using Call by (simp add: activation_context_rel_Call_iff)
qed simp_all
```

Context entry invariant

The invariants below concern the traces of one graph from one set of initial stores, so the block fixes `g` and `S`.

```
context
  fixes g :: "(var_id, exp) gcfg" and S :: "store set"
begin
```

A called activation carries exactly the contexts admitted for the transition from its immediate caller, each computed from a context the caller itself carries. Stated as an equivalence: forwards, a RETURN argument recovers the caller's slot from the callee's; backwards, it re-enters the callee at a context derived from whichever slot the resumed caller is being placed in.

```
lemma activation_context_rel_caller_entry:
  assumes "callee ∈ (𝒯κ,g,S)"
  shows "∀u ∈ callers callee. ∀c. caller_of u = Some c
       ⟶ (∃p es. hd (path_of u) = (FunctionEntry p, es)
            ∧ (∀ctx'. activation_context_rel κ adm c₀ g u ctx'
                 ⟷ (∃ctx. activation_context_rel κ adm c₀ g c ctx
                      ∧ admits_call_context κ g adm (sink_node c) ctx p (sink_store c) es ctx')))"
proof -
  define P where "P u ⟷ (∀c. caller_of u = Some c
       ⟶ (∃p es. hd (path_of u) = (FunctionEntry p, es)
            ∧ (∀ctx'. activation_context_rel κ adm c₀ g u ctx'
                 ⟷ (∃ctx. activation_context_rel κ adm c₀ g c ctx
                      ∧ admits_call_context κ g adm (sink_node c) ctx p (sink_store c) es ctx'))))"
    for u
  have "∀u ∈ callers callee. P u"
  proof (rule caller_chain_closure[OF _ _ _ _ assms])
    fix s show "P (Root [(cfg_entry g, s)])" by (simp add: P_def)
  next
    fix t a v s' assume t: "t ∈ (𝒯κ,g,S)" and IH: "∀u ∈ callers t. P u"
    have "path_of t ≠ []" using valid_activation_trace_path_nonempty[OF t] .
    with IH[THEN bspec, OF callers_refl] show "P (extend t (v, s'))"
      by (simp add: P_def activation_context_rel_extend hd_append)
  next
    fix caller dst pars args p cont
    show "P (Call caller
        [(FunctionEntry p, call_enter κ (CallEdge dst pars args) (sink_store caller))])"
      unfolding P_def by (auto simp: activation_context_rel_Call_iff)
  next
    fix callee caller p dst pars args cont
    assume cv: "callee ∈ (𝒯κ,g,S)" and IH: "∀u ∈ callers callee. P u"
      and cof: "caller_of callee = Some caller"
    have "path_of caller ≠ []"
      using valid_activation_trace_path_nonempty[OF valid_activation_trace_caller_valid[OF cv cof]]
```

```
    .
  moreover have "P caller" using IH ancestors_caller[OF cof] by blast
  ultimately show "P (Resume caller callee
      (path_of caller @ [(cont, combine_collect κ dst (sink_store caller) (sink_store callee))]))"
    by (simp add: P_def hd_append)
 qed
 then show ?thesis unfolding P_def .
qed

lemma activation_context_rel_caller_entryE:
 assumes "callee ∈ (𝒯κ,g,S)" and "caller_of callee = Some caller"
   and "activation_context_rel κ adm c0 g callee ctx'"
 obtains ctx p where "hd (path_of callee) = (FunctionEntry p, entry_store callee)"
   and "activation_context_rel κ adm c0 g caller ctx"
   and "admits_call_context κ g adm (sink_node caller) ctx p (sink_store caller)
        (entry_store callee) ctx'"
proof -
 from activation_context_rel_caller_entry[OF assms(1), THEN bspec, OF callers_refl,
     rule_format, OF assms(2)]
 obtain p es where hd: "hd (path_of callee) = (FunctionEntry p, es)"
   and iff: "∀ctx'. activation_context_rel κ adm c0 g callee ctx'
       ⟷ (∃ctx. activation_context_rel κ adm c0 g caller ctx
            ∧ admits_call_context κ g adm (sink_node caller) ctx p (sink_store caller) es ctx')"
   by force
 have "es = entry_store callee" using hd by (simp add: entry_store_def)
 with hd iff assms(3) show thesis using that by blast
qed

lemma activation_context_rel_caller_entryI:
 assumes "callee ∈ (𝒯κ,g,S)" and "caller_of callee = Some caller"
   and "hd (path_of callee) = (FunctionEntry p, entry_store callee)"
   and "activation_context_rel κ adm c0 g caller ctx"
   and "admits_call_context κ g adm (sink_node caller) ctx p (sink_store caller)
        (entry_store callee) ctx'"
 shows "activation_context_rel κ adm c0 g callee ctx'"
proof -
 from activation_context_rel_caller_entry[OF assms(1), THEN bspec, OF callers_refl,
     rule_format, OF assms(2)]
 obtain p' es where hd: "hd (path_of callee) = (FunctionEntry p', es)"
   and iff: "∀ctx'. activation_context_rel κ adm c0 g callee ctx'
       ⟷ (∃ctx. activation_context_rel κ adm c0 g caller ctx
            ∧ admits_call_context κ g adm (sink_node caller) ctx p' (sink_store caller) es ctx')"
   by force
 have "p' = p" and "es = entry_store callee" using hd assms(3) by simp_all
 with iff assms(4,5) show ?thesis by blast
qed
```

A functional policy carries exactly `activation_context_of`'s context on every valid activation trace. The restriction to valid activation traces is what supplies the call edge the relational clause names.

```
lemma activation_context_rel_of_fun_iff:
 assumes "t ∈ (𝒯κ,g,S)"
 shows "activation_context_rel κ (context_policy_of_fun f) c0 g t ctx
        ⟷ activation_context_of f c0 t = ctx"
proof -
 define P where "P u ⟷ (∀ctx. activation_context_rel κ (context_policy_of_fun f) c0 g u ctx
                          ⟷ activation_context_of f c0 u = ctx)" for u
 have "∀u ∈ callers t. P u"
 proof (rule caller_chain_closure[OF _ _ _ _ assms])
   fix s show "P (Root [(cfg_entry g, s)])" by (simp add: P_def eq_commute)
 next
   fix t a v s' assume t: "t ∈ (𝒯κ,g,S)" and IH: "∀u ∈ callers t. P u"
   have "path_of t ≠ []" using valid_activation_trace_path_nonempty[OF t] .
   with IH[THEN bspec, OF callers_refl] show "P (extend t (v, s'))"
     by (simp add: P_def activation_context_extend_nonempty activation_context_rel_extend)
 next
   fix caller dst pars args p cont
   assume IH: "∀u ∈ callers caller. P u"
     and e: "(sink_node caller, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
   have Pc: "P caller" using IH callers_refl by blast
   show "P (Call caller
       [(FunctionEntry p, call_enter κ (CallEdge dst pars args) (sink_store caller))])"
     unfolding P_def activation_context_rel_Call_iff admits_call_context_of_fun
     using Pc[unfolded P_def] e by auto
 next
   fix callee caller p dst pars args cont
   assume IH: "∀u ∈ callers callee. P u" and cof: "caller_of callee = Some caller"
```

```
    have "P caller" using IH ancestors_caller[OF cof] by blast
    then show "P (Resume caller callee
        (path_of caller @ [(cont, combine_collect κ dst (sink_store caller) (sink_store callee))]))"
      by (simp add: P_def)
  qed
  then show ?thesis unfolding P_def using callers_refl by blast
qed
```

Functional context entry invariant

Every ancestor of a valid activation trace is itself valid: `caller_chain_closure`'s own four cases are exactly `valid_activation_trace`'s four introduction rules, so the closure fact and validity coincide.

```
lemma callers_valid:
  assumes "callee ∈ (𝒯κ,g,S)" and "u ∈ callers callee"
  shows "u ∈ (𝒯κ,g,S)"
proof -
  have "∀u ∈ callers callee. u ∈ (𝒯κ,g,S)"
  proof (rule caller_chain_closure[OF _ _ _ _ assms(1)], goal_cases Root Intra Call Ret)
    case (Root s) then show ?case by (rule valid_activation_trace.root)
  next
    case (Intra t a v s') then show ?case by (blast intro: valid_activation_trace.intra)
  next
    case (Call caller dst pars args p cont) then show ?case
      by (blast intro: valid_activation_trace.call)
  next
    case (Ret callee caller p dst pars args cont) then show ?case
      by (blast intro: valid_activation_trace.ret)
  qed
  then show ?thesis using assms(2) by blast
qed
```

The context of an activation is exactly what `enterc` computes from its immediate caller's own context and the entered store. This is what lets a RETURN argument construct the callee's context directly from the caller's chosen one, rather than rediscovering it after the fact. A special case of `activation_context_rel_caller_entry` at the functional policy `context_policy_of_fun`, where `activation_context_rel_of_fun_iff` pins both sides' witness context to `activation_context_of` and `admits_call_context_of_fun` reads off the `enterc`-equation and the call-edge witness in one step.

```
lemma activation_context_entry_invariant:
  assumes "callee ∈ (𝒯κ,g,S)"
  shows "∀u ∈ callers callee. ∀c. caller_of u = Some c ⟶
      activation_context_of enterc initial_ctx u = enterc (sink_node c) (activation_context_of enterc initial_ctx
c) (entry_store u)
      ∧ call_enter_store κ g (sink_node c) (sink_store c) (entry_store u)"
proof (intro ballI allI impI)
  fix u c assume mem: "u ∈ callers callee" and cof: "caller_of u = Some c"
  have uv: "u ∈ (𝒯κ,g,S)" using callers_valid[OF assms mem] .
  have cv: "c ∈ (𝒯κ,g,S)" using valid_activation_trace_caller_valid[OF uv cof]
    .
  have tc_u: "activation_context_rel κ (context_policy_of_fun enterc) initial_ctx g u
              (activation_context_of enterc initial_ctx u)"
    by (simp only: activation_context_rel_of_fun_iff[OF uv])
  obtain p ctx where hd: "hd (path_of u) = (FunctionEntry p, entry_store u)"
    and tc_c: "activation_context_rel κ (context_policy_of_fun enterc) initial_ctx g c ctx"
    and adm: "admits_call_context κ g (context_policy_of_fun enterc) (sink_node c) ctx p
              (sink_store c) (entry_store u) (activation_context_of enterc initial_ctx u)"
    by (rule activation_context_rel_caller_entryE[OF uv cof tc_u])
  have activation_context_c: "activation_context_of enterc initial_ctx c = ctx"
    using tc_c by (simp only: activation_context_rel_of_fun_iff[OF cv])
  have adm': "activation_context_of enterc initial_ctx u = enterc (sink_node c) ctx (entry_store u)"
    using adm unfolding admits_call_context_of_fun by simp
  have activation_context_eq: "activation_context_of enterc initial_ctx u
                = enterc (sink_node c) (activation_context_of enterc initial_ctx c) (entry_store u)"
    unfolding activation_context_c using adm' .
  show "activation_context_of enterc initial_ctx u
        = enterc (sink_node c) (activation_context_of enterc initial_ctx c) (entry_store u)
      ∧ call_enter_store κ g (sink_node c) (sink_store c) (entry_store u)"
    using activation_context_eq admits_call_context_call_enter_storeD[OF adm] by (intro conjI)
qed

lemma key_entry_invariant_call_enterD [dest]:
  assumes "callee ∈ (𝒯κ,g,S)" and "caller_of callee = Some caller"
  shows "call_enter_store κ g (sink_node caller) (sink_store caller) (entry_store callee)"
  using activation_context_entry_invariant[OF assms(1), THEN bspec, OF callers_refl, rule_format, OF
```

```
    assms(2)]
  by (rule conjunct2)

end
```

Conditional totality

A policy may admit no context for a call. The callee activation then carries no context and drops out of every bucket, but the resumed caller keeps its own, so the continuation is still collected. CALL and RETURN hold vacuously at such a call, and a claim may leave the continuation uncovered although a valid activation trace reaches it there (`total_dropped_unsound` in `Example_Non_Vacuity`). `call_context_total_on` κ `cover adm g` rules that out exactly where it matters: at every call edge, every store the claimed `cover` admits at the call site has some admissible callee context. A dead call site owes nothing. The graph of a function is total outright.

```
definition call_context_total_on ::
  "kenv ⇒ (cfg_node ⇒ 'c ⇒ store set) ⇒ 'c call_context_policy ⇒ (var_id, exp) gcfg ⇒ bool"
where
  "call_context_total_on κ cover adm g ⟷
     (∀u dst pars args p cont ctx s.
        (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
        ⟶ s ∈ cover u ctx
        ⟶ adm u ctx (call_info_of (CallEdge dst pars args) p) s
              (call_enter κ (CallEdge dst pars args) s) ≠ {})"

lemma call_context_total_onE:
  assumes "call_context_total_on κ cover adm g"
    and "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and "s ∈ cover u ctx"
  obtains ctx' where "admits_call_context κ g adm u ctx p s
                        (call_enter κ (CallEdge dst pars args) s) ctx'"
  using assms unfolding call_context_total_on_def
  by (blast intro: admits_call_contextI)

lemma call_context_total_on_of_fun [simp]:
  "call_context_total_on κ cover (context_policy_of_fun f) g"
  unfolding call_context_total_on_def context_policy_of_fun_def by blast
```

The activation-indexed context collecting

The activation-sensitive collecting is the sink stores of valid activation traces reaching `v` that may carry the queried context `c`. One trace may carry several contexts, so the buckets of one node cover its stores without partitioning them.

```
definition activation_collect ::
  "kenv ⇒ 'c call_context_policy ⇒ 'c ⇒ (var_id, exp) gcfg
     ⇒ store set ⇒ cfg_node ⇒ 'c ⇒ store set"
    ("𝒜_,_,_,_,_") where
  "activation_collect κ adm c₀ g S v c =
     {sink_store t | t. t ∈ 𝒯κ,g,S ∧ sink_node t = v
                        ∧ activation_context_rel κ adm c₀ g t c}"

lemma activation_collect_I [intro]:
  "t ∈ 𝒯κ,g,S ⟹ sink_node t = v ⟹ activation_context_rel κ adm c₀ g t c
   ⟹ sink_store t ∈ 𝒜κ,adm,c₀,g,S v c"
  unfolding activation_collect_def by blast
```

Every collected state has a valid activation trace witness carrying the queried `c`.

```
lemma activation_collect_E [elim]:
  assumes "s ∈ 𝒜κ,adm,c₀,g,S v c"
  obtains t where "t ∈ 𝒯κ,g,S" "sink_node t = v"
    "activation_context_rel κ adm c₀ g t c" "sink_store t = s"
  using assms unfolding activation_collect_def by blast
```

Under a functional policy the buckets are `activation_context_of`'s fibres.

```
lemma activation_collect_of_fun:
  "𝒜κ,context_policy_of_fun f,c₀,g,S v c =
     {sink_store t | t. t ∈ 𝒯κ,g,S ∧ sink_node t = v ∧ activation_context_of f c₀ t = c}"
  unfolding activation_collect_def
  by (auto simp: activation_context_rel_of_fun_iff)


end
```

4.7 Activation_Trace_Collect

```
theory Activation_Trace_Collect
  imports Activation_Trace_Context
begin
```

Which stores can occur at each node

$\mathcal{C}_{\mathcal{G},g,S}$ `v` is the set of stores a run can actually have at node `v`: take every valid activation trace that ends there and keep its final store. This is the concrete set an analysis has to over-approximate, so it is the target of every soundness statement downstream.

`valid_activation_trace` has four clauses — root, intra step, call, return — each reading exactly the relation for its phenomenon: `intra` for local extension, `calls` for entering a callee and for recovering a continuation. The return clause recovers the caller from the completed callee's own ancestry through `caller_of`, never by an independent choice.

There is no global exit node: whole-program completion is collection at `FunctionResult main`, and a procedure result is an ordinary collected node rather than a separate summary mechanism.

Forgetful projections

`node_collect` is the concrete collecting view: the sink stores of valid activation traces reaching node `v`. `activation_collect` is the context-indexed collector, keeping those whose trace may carry the queried context; the context type is not required finite, and a bucket is not claimed to be an exact activation identity.

```
definition node_collect ::
    "kenv ⇒ (var_id, exp) gcfg ⇒ store set ⇒ cfg_node ⇒ store set"
    ("C_,_,_") where
  "node_collect κ g S v =
     {sink_store t | t. t ∈ T_κ,g,S ∧ sink_node t = v}"
```

Up to the inhabitation witness, every lemma concerns one graph and one set of initial stores, so the block fixes $\mathcal{G}$, `g` and `S` and writes `(valid_activation_trace κ g S)` and `(node_collect κ g S) v` for `valid_activation_trace κ g S` and $\mathcal{C}_{\mathcal{G},g,S}$ `v`.

```
context
  fixes g :: "(var_id, exp) gcfg" and S :: "store set"
begin

lemma node_collect_I [intro]:
  "t ∈ (T_κ,g,S) ⟹ sink_store t ∈ (C_κ,g,S) (sink_node t)"
  unfolding node_collect_def by blast
```

Every collected state has a valid activation trace witness.

```
lemma node_collect_E [elim]:
  assumes "s ∈ (C_κ,g,S) v"
  obtains t where "t ∈ (T_κ,g,S)" "sink_node t = v" "sink_store t = s"
  using assms unfolding node_collect_def by blast
```

`node_collect` is closed under intra flow: extending a witness trace by one `intra` edge is `valid_activation_trace.intra`. Closure under *call* and *return* steps is deliberately absent here — `valid_activation_trace.ret` requires the popped activation to be the witness trace's own caller, which a bare CFG step does not supply.

```
lemma node_collect_intra_step:
  assumes s: "s ∈ (C_κ,g,S) u"
    and e: "(u, a, v) ∈ intra g"
    and st: "s' ∈ edge_step κ a s"
  shows "s' ∈ (C_κ,g,S) v"
proof -
  from s
  obtain t where t: "t ∈ (T_κ,g,S)" and n: "sink_node t = u" and w:
    "sink_store t = s"
    by (rule node_collect_E)
  have "extend t (v, s') ∈ (T_κ,g,S)"
    by (rule valid_activation_trace.intra[OF t]) (use e st n w in simp_all)
  from node_collect_I[OF this] show ?thesis by simp
qed
```

The activation-preserving form. Following an intra path only `extend`s the trace, so the resulting witness is the *same* activation: same caller and same entry node. Callers that must know *which* activation reached a node need this: the collection at a node merges every activation that reaches it.

```
lemma valid_activation_trace_intra_path_extend_pair:
  "star (cfg_intra_step κ g) x y ⟹ t ∈ (𝒯κ,g,S) ⟹ sink_node t = fst x ⟹ sink_store t = snd x
   ⟹ ∃t'. t' ∈ (𝒯κ,g,S) ∧ sink_node t' = fst y ∧ sink_store t' = snd y
           ∧ caller_of t' = caller_of t ∧ fst (hd (path_of t')) = fst (hd (path_of t))"
proof (induction arbitrary: t rule: star.induct)
  case (refl x)
  then show ?case by blast
next
  case (step x y z)
  obtain u s where x: "x = (u, s)" by (cases x)
  obtain w s1 where y: "y = (w, s1)" by (cases y)
  from step.hyps(1) x y obtain a where
    e: "(u, a, w) ∈ intra g" and st: "s1 ∈ edge_step κ a s"
    by auto
  have ext: "extend t (w, s1) ∈ (𝒯κ,g,S)"
    by (rule valid_activation_trace.intra[OF step.prems(1)])
       (use e st step.prems(2,3) x in simp_all)
  have hd_ext: "fst (hd (path_of (extend t (w, s1)))) = fst (hd (path_of t))"
    using valid_activation_trace_path_nonempty[OF step.prems(1)] by simp
  from step.IH[OF ext] y obtain t' where
    "t' ∈ (𝒯κ,g,S)" "sink_node t' = fst z" "sink_store t' = snd z"
    "caller_of t' = caller_of (extend t (w, s1))"
    "fst (hd (path_of t')) = fst (hd (path_of (extend t (w, s1))))" by auto
  then show ?case using hd_ext by auto
qed

lemma valid_activation_trace_intra_path_extend:
  assumes p: "star (cfg_intra_step κ g) (sink_node t, sink_store t) (v, s')"
    and t: "t ∈ (𝒯κ,g,S)"
  obtains t' where "t' ∈ (𝒯κ,g,S)" "sink_node t' = v" "sink_store t' = s'"
    "caller_of t' = caller_of t" "fst (hd (path_of t')) = fst (hd (path_of t))"
  using valid_activation_trace_intra_path_extend_pair[OF p t] by auto
```

A call and its matching return *do* compose, which is what makes the missing single-step closure above harmless in practice. The callee trace is built from the caller's by `valid_activation_trace.call`, so `valid_activation_trace.ret`'s side condition — that the activation being popped is this trace's own caller — holds by construction, and running the callee to its `FunctionResult` along an intra path preserves it. What a caller supplies is the call edge and an intra path through the callee; what it gets back is the continuation node with the combined store.

```
lemma node_collect_call_return_step:
  assumes s: "s ∈ (𝒞κ,g,S) u"
      and ce: "(u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls g"
      and body: "star (cfg_intra_step κ g) (FunctionEntry q, call_enter κ (CallEdge dst pars args) s)
                  (FunctionResult q, t)"
  shows "combine_collect κ dst s t ∈ (𝒞κ,g,S) cont"
proof -
  from s obtain caller where cv: "caller ∈ (𝒯κ,g,S)"
    and cn: "sink_node caller = u" and cs: "sink_store caller = s"
    by (rule node_collect_E)
  let ?entered = "Call caller [(FunctionEntry q, call_enter κ (CallEdge dst pars args) s)]"
  have eval: "?entered ∈ (𝒯κ,g,S)"
    using valid_activation_trace.call [OF cv, where dst = dst and pars = pars and args = args
                            and p = q and cont = cont]
      ce cn cs by simp
  obtain callee where dv: "callee ∈ (𝒯κ,g,S)"
    and dn: "sink_node callee = FunctionResult q" and ds: "sink_store callee = t"
    and dc: "caller_of callee = caller_of ?entered"
    by (rule valid_activation_trace_intra_path_extend [OF _ eval]) (use body in simp_all)
  have "Resume caller callee
        (path_of caller @ [(cont, combine_collect κ dst (sink_store caller) (sink_store callee))])
        ∈ (𝒯κ,g,S)"
    by (rule valid_activation_trace.ret [OF dv _ dn]) (use dc ce cn in simp_all)
  from node_collect_I [OF this] show ?thesis
    using cs ds by (simp add: sink_node_def sink_store_def)
qed
```

Collecting equations

(1) Initial stores are collected at `cfg_entry`.

```
lemma node_collect_init:
  "s ∈ S ⟹ s ∈ (𝒞κ,g,S) (cfg_entry g)"
  using node_collect_I[OF valid_activation_trace.root] by simp
```

Context-sensitive / context-insensitive bridge

Bridge (1): every context bucket, and so their union, is included in the context-insensitive collection.

```
theorem activation_collect_le_node_collect:
  "A_κ,adm,c0,g,S v c ⊆ (C_κ,g,S) v"
  unfolding activation_collect_def node_collect_def by blast

theorem Union_activation_collect_le_node_collect:
  "(⋃c. A_κ,adm,c0,g,S v c) ⊆ (C_κ,g,S) v"
  using activation_collect_le_node_collect by blast
```

Bridge (2): the converse needs every valid activation trace to carry some context. That premise is the whole content of the direction – a policy that leaves some trace uncontexted loses the stores on it, and the union then falls short. A functional policy has it outright, since `activation_context_of` is total; a relational one earns it from conditional totality, which is `Activation_Trace_Abstract`'s business. No finiteness assumption either way.

```
theorem node_collect_eq_Union_activation_of_has_context:
  assumes has_ctx:
    "⋀t. t ∈ (T_κ,g,S) ⟹ ∃c. activation_context_rel κ adm c0 g t c"
  shows "(C_κ,g,S) v = (⋃c. A_κ,adm,c0,g,S v c)"
proof
  show "(C_κ,g,S) v ⊆ (⋃c. A_κ,adm,c0,g,S v c)"
  proof
    fix x assume "x ∈ (C_κ,g,S) v"
    then obtain t where t: "t ∈ (T_κ,g,S)" "sink_node t = v" "sink_store t = x"
      by (rule node_collect_E)
    from has_ctx [OF t(1)] obtain c where "activation_context_rel κ adm c0 g t c" ..
    with t show "x ∈ (⋃c. A_κ,adm,c0,g,S v c)"
      by blast
  qed
next
  show "(⋃c. A_κ,adm,c0,g,S v c) ⊆ (C_κ,g,S) v"
    by (rule Union_activation_collect_le_node_collect)
qed

theorem node_collect_eq_Union_activation_of_fun:
  "(C_κ,g,S) v
     = (⋃c. A_κ,context_policy_of_fun f,c0,g,S v c)"
  by (rule node_collect_eq_Union_activation_of_has_context)
     (simp add: activation_context_rel_of_fun_iff)

end
```

The collecting semantics is inhabited

`valid_activation_trace` and `node_collect` are defined over an arbitrary graph, and every soundness statement downstream is an over-approximation claim about them; if they were empty, all of it would hold vacuously. A two-node graph with one assignment edge settles that inside this session, without borrowing a graph from the compiler: the initial store is collected at the entry, and the assigned store is collected one edge later.

```
abbreviation activation_trace_witness_var :: var_id where
  "activation_trace_witness_var ≡ GlobalId (STR ''x'')"

definition activation_trace_witness_cfg :: "(var_id, exp) gcfg" where
  "activation_trace_witness_cfg =
     (| intra = {(Statement 0, EA_Assign activation_trace_witness_var (TN I32 1), Statement 1)},
       calls = {},
       cfg_entry = Statement 0 |)"

lemma activation_trace_witness_entry [simp]: "cfg_entry activation_trace_witness_cfg = Statement 0"
  by (simp add: activation_trace_witness_cfg_def)

lemma activation_trace_witness_edge [simp]:
  "(Statement 0, EA_Assign activation_trace_witness_var (TN I32 1), Statement 1)
     ∈ intra activation_trace_witness_cfg"
  by (simp add: activation_trace_witness_cfg_def)
```

The witness runs for every kind environment.

```
lemma activation_trace_witness_root:
  "Root [(Statement 0, s)] ∈ T_κ,activation_trace_witness_cfg,{s}"
proof -
  have "Root [(cfg_entry activation_trace_witness_cfg, s)]
          ∈ T_κ,activation_trace_witness_cfg,{s}"
```

```
    by (rule valid_activation_trace.root) simp
  then show ?thesis by simp
qed

theorem node_collect_witness:
  "s ∈ C_κ,activation_trace_witness_cfg,{s} (Statement 0)"
  "s(activation_trace_witness_var := kind_conv κ (Some activation_trace_witness_var) (ik_norm I32 1))
     ∈ C_κ,activation_trace_witness_cfg,{s} (Statement 1)"
proof -
  have root: "Root [(Statement 0, s)] ∈ T_κ,activation_trace_witness_cfg,{s}"
    by (rule activation_trace_witness_root)
  show "s ∈ C_κ,activation_trace_witness_cfg,{s} (Statement 0)"
    using node_collect_I[OF root] by (simp add: sink_node_def sink_store_def)
  have "extend (Root [(Statement 0, s)])
          (Statement 1, s(activation_trace_witness_var
             := kind_conv κ (Some activation_trace_witness_var) (ik_norm I32 1)))
          ∈ T_κ,activation_trace_witness_cfg,{s}"
    by (rule valid_activation_trace.intra[OF root,
          where a = "EA_Assign activation_trace_witness_var (TN I32 1)" and v = "Statement 1"])
       (simp_all add: activation_trace_witness_cfg_def sink_node_def sink_store_def)
  from node_collect_I[OF this]
  show "s(activation_trace_witness_var := kind_conv κ (Some activation_trace_witness_var) (ik_norm I32 1))
          ∈ C_κ,activation_trace_witness_cfg,{s} (Statement 1)"
    by (simp add: sink_node_def sink_store_def)
qed

end
```

4.8 Activation_Trace_Abstract

```
theory Activation_Trace_Abstract
  imports Activation_Trace_Collect
begin
```

What an analysis must satisfy to be sound

An analysis claims: "at node v, in context c, only these stores can occur". This theory says what such a claim must satisfy to be believed, and proves that satisfying it is enough — every valid activation trace's final store really does lie in the set claimed for its own node and every context it carries. No domain, solver or dependency graph appears here; they instantiate this.

The claim is a function `cover` from a node and a context to a set of stores, and there are five obligations on it: the seed stores are covered at the entry (`INIT`), each intra edge keeps them covered (`INTRA`), a call covers the entered store at the callee in every context the policy admits (`CALL`), a return covers the combined store back at the caller (`RETURN`), and every covered call admits at least one context (`TOTAL`).

`RETURN` is the load-bearing one. The callee is read at a context admitted for the transition from the caller's own carried context, so a return may only compose a callee whose context came from the caller it is resuming — not any context that happens to cover that callee's entry. This correlation is not an extra parameter to assume: `activation_context_rel_caller_entry` makes it a theorem about `valid_activation_trace`.

`TOTAL` is what makes the buckets meaningful rather than merely safe. A resumed caller may carry a context under which its callee was never assigned one; the callee's exit store would then be bounded by no bucket, and the combined store by nothing. With it, every valid activation trace carries some context, so the context-insensitive collection is exactly the union of the buckets.

A context is not claimed to identify an activation: two activations may carry the same context, and one activation may carry several. The theorem is that each trace lands in every slot it carries.

The abstract interface

`cover` is the claim itself, `adm` the context policy it is indexed by, and c_0 the context the seed stores arrive under. An analysis interprets this once, with its own solved table in place of `cover`; nothing below asks how that table was computed.

```
locale activation_coverage =
  fixes κ :: kenv
    and g :: "(var_id, exp) gcfg" and S :: "store set"
    and cover :: "cfg_node ⇒ 'c ⇒ store set"
    and adm :: "'c call_context_policy"
    and c₀ :: 'c
  assumes INIT[intro]: "⋀s. s ∈ S ⟹ s ∈ cover (cfg_entry g) c₀"
    and INTRA[intro]: "⋀u a v c s s'. (u, a, v) ∈ intra g
        ⟹ s ∈ cover u c ⟹ s' ∈ edge_step κ a s ⟹ s' ∈ cover v c"
    and CALL[intro]: "⋀u dst pars args p cont c c' s.
        (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
        ⟹ s ∈ cover u c
        ⟹ c' ∈ adm u c (call_info_of (CallEdge dst pars args) p) s
              (call_enter κ (CallEdge dst pars args) s)
        ⟹ call_enter κ (CallEdge dst pars args) s ∈ cover (FunctionEntry p) c'"
    and RETURN[intro]: "⋀u dst pars args p cont c c' p' s t es.
        (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
        ⟹ s ∈ cover u c
        ⟹ admits_call_context κ g adm u c p' s es c'
        ⟹ t ∈ cover (FunctionResult p) c'
        ⟹ combine_collect κ dst s t ∈ cover cont c"
    and TOTAL: "call_context_total_on κ cover adm g"
begin
```

The trace-level objects at the locale's fixed arguments: the collection $\mathcal{C}$, the valid activation traces $\mathcal{T}$, the context relation `carries`, the buckets $\mathcal{A}$, and the admitted call contexts `admits`.

```
abbreviation collect :: "cfg_node ⇒ store set" ("𝒞")
  where "𝒞 ≡ 𝒞κ,g,S"
abbreviation traces :: "activation_trace set" ("𝒯")
  where "𝒯 ≡ 𝒯κ,g,S"
abbreviation carries :: "activation_trace ⇒ 'c ⇒ bool"
  where "carries ≡ activation_context_rel κ adm c₀ g"
abbreviation buckets :: "cfg_node ⇒ 'c ⇒ store set" ("𝒜")
  where "𝒜 ≡ 𝒜κ,adm,c₀,g,S"
abbreviation admits :: "cfg_node ⇒ 'c ⇒ pname ⇒ store ⇒ store ⇒ 'c ⇒ bool"
```

```
  where "admits ≡ admits_call_context κ g adm"
```

`trace_covered u`: `u` carries some context, and its sink store is admitted at its own node under every context it carries.

```
definition trace_covered :: "activation_trace ⇒ bool" where
  "trace_covered u ⟷
     (∃c. carries u c)
     ∧ (∀c. carries u c ⟶ sink_store u ∈ cover (sink_node u) c)"

lemma trace_coveredI:
  assumes "carries u c0"
    and "⋀c. carries u c ⟹ sink_store u ∈ cover (sink_node u) c"
  shows "trace_covered u"
  using assms unfolding trace_covered_def by blast

lemma trace_covered_someE:
  assumes "trace_covered u"
  obtains c where "carries u c" "sink_store u ∈ cover (sink_node u) c"
  using assms unfolding trace_covered_def by blast

lemma trace_coveredD:
  "trace_covered u ⟹ carries u c
   ⟹ sink_store u ∈ cover (sink_node u) c"
  unfolding trace_covered_def by blast
```

Closure under the four constructor clauses

`init_covered`: the main activation's seed store is admitted at c_0, the one context every `Root` carries.

```
lemma init_covered: "s ∈ S ⟹ trace_covered (Root [(cfg_entry g, s)])"
  by (auto simp: trace_covered_def)
```

`intra_covered`: an intra edge preserves the carried contexts (`activation_context_rel_extend`) and covers the concrete step through `INTRA` at each of them.

```
lemma intra_covered:
  assumes e: "(sink_node t, a, v) ∈ intra g"
    and st: "s' ∈ edge_step κ a (sink_store t)" and pne: "path_of t ≠ []" and iht: "trace_covered t"
  shows "trace_covered (extend t (v, s'))"
  using iht unfolding trace_covered_def
  by (auto simp: activation_context_rel_extend[OF pne] intro: INTRA[OF e _ st])
```

`call_covered`: `TOTAL` gives the new activation some context, from a context its caller carries; `CALL` covers the entered store at every context it carries, each of which names its own edge out of the call site.

```
lemma call_covered:
  assumes e: "(sink_node caller, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and ihc: "trace_covered caller"
  shows "trace_covered (Call caller
          [(FunctionEntry p, call_enter κ (CallEdge dst pars args) (sink_store caller))])"
proof -
  let ?es = "call_enter κ (CallEdge dst pars args) (sink_store caller)"
  from ihc obtain c where c: "carries caller c"
    and cov: "sink_store caller ∈ cover (sink_node caller) c"
    by (rule trace_covered_someE)
  obtain c' where
    adm: "admits (sink_node caller) c p (sink_store caller) ?es c'"
    by (rule call_context_total_onE[OF TOTAL e cov])
  have some: "carries (Call caller [(FunctionEntry p, ?es)]) c'"
    using c adm by (rule activation_context_rel.Call)
  show ?thesis
  proof (rule trace_coveredI[OF some])
    fix c' assume "carries (Call caller [(FunctionEntry p, ?es)]) c'"
    then obtain c0 where c0: "carries caller c0"
      and adm0: "admits (sink_node caller) c0 p (sink_store caller) ?es c'"
      by (auto simp: activation_context_rel_Call_iff)
    from adm0 obtain dst' pars' args' cont'
      where e': "(sink_node caller, CallEdge dst' pars' args', FunctionEntry p, cont') ∈ calls g"
        and es: "?es = call_enter κ (CallEdge dst' pars' args') (sink_store caller)"
        and Rc: "c' ∈ adm (sink_node caller) c0 (call_info_of (CallEdge dst' pars' args') p)
                   (sink_store caller) ?es"
      by (rule admits_call_contextE)
    have "sink_store caller ∈ cover (sink_node caller) c0"
      using ihc c0 by (rule trace_coveredD)
    from CALL[OF e' this Rc[unfolded es]] es
    show "sink_store (Call caller [(FunctionEntry p, ?es)])
```

```
          ∈ cover (sink_node (Call caller [(FunctionEntry p, ?es)])) c'" by simp
  qed
qed
```

`return_covered`: the matched return. For each context `c` the resumed caller carries, `TOTAL` re-enters the callee at a context `c'` admitted from `c` along the callee's own entry edge (`activation_context_rel_caller_entry`, both directions), the callee's hypothesis bounds its exit at `c'`, and `RETURN` composes the two at `c` — never at a context rediscovered independently.

```
lemma return_covered:
  assumes callee_val: "callee ∈ 𝒯"
    and cof: "caller_of callee = Some caller"
    and res: "sink_node callee = FunctionResult p"
    and comb: "(sink_node caller, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and ih_caller: "trace_covered caller"
    and ih_callee: "trace_covered callee"
  shows "trace_covered (Resume caller callee (path_of caller
              @ [(cont, combine_collect κ dst (sink_store caller) (sink_store callee))]))"
proof -
  let ?u = "Resume caller callee (path_of caller
    @ [(cont, combine_collect κ dst (sink_store caller) (sink_store callee))])"
  have sn: "sink_node ?u = cont"
    and ss: "sink_store ?u = combine_collect κ dst (sink_store caller) (sink_store callee)"
    by (simp_all add: sink_node_def sink_store_def)
  from ih_caller obtain c1 where c1: "carries caller c1"
    by (rule trace_covered_someE)
  have some: "carries ?u c1" using c1 by simp
  from ih_callee obtain ce where ce: "carries callee ce"
    by (rule trace_covered_someE)
  obtain c0 p' where hd: "hd (path_of callee) = (FunctionEntry p', entry_store callee)"
    and adm: "admits (sink_node caller) c0 p' (sink_store caller)
                (entry_store callee) ce"
    by (rule activation_context_rel_caller_entryE[OF callee_val cof ce])
  from adm obtain dst' pars' args' cont'
    where e': "(sink_node caller, CallEdge dst' pars' args', FunctionEntry p', cont') ∈ calls g"
      and es: "entry_store callee = call_enter κ (CallEdge dst' pars' args') (sink_store caller)"
    by (rule admits_call_contextE)
  show ?thesis
  proof (rule trace_coveredI[OF some])
    fix c assume "carries ?u c"
    then have cc: "carries caller c" by simp
    have s_cov: "sink_store caller ∈ cover (sink_node caller) c"
      using ih_caller cc by (rule trace_coveredD)
    obtain c' where adm': "admits (sink_node caller) c p' (sink_store caller)
                             (call_enter κ (CallEdge dst' pars' args') (sink_store caller)) c'"
      by (rule call_context_total_onE[OF TOTAL e' s_cov])
    have tc': "carries callee c'"
      by (rule activation_context_rel_caller_entryI[OF callee_val cof hd cc adm'[folded es]])
    have t_cov: "sink_store callee ∈ cover (FunctionResult p) c'"
      using trace_coveredD[OF ih_callee tc'] res by simp
    have "combine_collect κ dst (sink_store caller) (sink_store callee) ∈ cover cont c"
      by (rule RETURN[OF comb s_cov adm'[folded es] t_cov])
    then show "sink_store ?u ∈ cover (sink_node ?u) c" by (simp add: sn ss)
  qed
qed
```

The generic soundness theorem

The bound holds along the whole caller chain, by `valid_activation_trace` rule induction feeding the four closure lemmas. The chain, not a per-node statement, is what the `RETURN` case needs: it recovers its caller structurally, and the caller lies in the callee's chain.

```
lemma caller_chain_covered:
  "t ∈ 𝒯 ⟹ ∀u ∈ callers t. trace_covered u"
proof (rule caller_chain_closure)
  fix s assume "s ∈ S"
  then show "trace_covered (Root [(cfg_entry g, s)])" by (rule init_covered)
next
  fix t a v s' assume ht: "t ∈ 𝒯" and ch: "∀u ∈ callers t. trace_covered u"
    and e: "(sink_node t, a, v) ∈ intra g" and st: "s' ∈ edge_step κ a (sink_store t)"
  have pt: "path_of t ≠ []" using ht valid_activation_trace_path_nonempty by blast
  have iht: "trace_covered t" using ch callers_refl by blast
  show "trace_covered (extend t (v, s'))" by (rule intra_covered[OF e st pt iht])
next
  fix caller dst pars args p cont
  assume ch: "∀u ∈ callers caller. trace_covered u"
```

```
    and e: "(sink_node caller, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
  have ihc: "trace_covered caller" using ch callers_refl by blast
  show "trace_covered (Call caller
          [(FunctionEntry p, call_enter κ (CallEdge dst pars args) (sink_store caller))])"
    by (rule call_covered[OF e ihc])
next
  fix callee caller p dst pars args cont
  assume cv: "callee ∈ 𝒯" and ch: "∀u ∈ callers callee. trace_covered u"
    and cof: "caller_of callee = Some caller" and res: "sink_node callee = FunctionResult p"
    and e: "(sink_node caller, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
  have ih_caller: "trace_covered caller" using ch cof callers_caller_subset callers_refl by blast
  have ih_callee: "trace_covered callee" using ch callers_refl by blast
  show "trace_covered (Resume caller callee (path_of caller
            @ [(cont, combine_collect κ dst (sink_store caller) (sink_store callee))]))"
    by (rule return_covered[OF cv cof res e ih_caller ih_callee])
qed
```

Every valid activation trace carries a context, and its sink lies in every slot it carries: `valid_activation_trace` is soundly over-approximated by `trace_covered`.

```
theorem valid_activation_trace_covered: "t ∈ 𝒯 ⟹ trace_covered t"
proof -
  assume "t ∈ 𝒯"
  then have "∀u ∈ callers t. trace_covered u" by (rule caller_chain_covered)
  then show "trace_covered t" using callers_refl by blast
qed

theorem valid_activation_trace_has_context:
  assumes "t ∈ 𝒯"
  obtains c where "carries t c"
  using valid_activation_trace_covered[OF assms] by (rule trace_covered_someE)

theorem valid_activation_trace_covered_at:
  "t ∈ 𝒯 ⟹ carries t c
   ⟹ sink_store t ∈ cover (sink_node t) c"
  using valid_activation_trace_covered by (rule trace_coveredD)
```

Bridge (2) of `Voblint_CFG.Activation_Trace_Collect`, earned: under `TOTAL` the context-insensitive collection is the union of the buckets.

```
theorem node_collect_eq_Union_activation_collect:
  "𝒞 v = (⋃c. 𝒜 v c)"
proof
  show "𝒞 v ⊆ (⋃c. 𝒜 v c)"
  proof
    fix x assume "x ∈ 𝒞 v"
    then obtain t where t: "t ∈ 𝒯" "sink_node t = v" "sink_store t = x"
      by (rule node_collect_E)
    from valid_activation_trace_has_context[OF t(1)] obtain c where "carries t c" .
    with t show "x ∈ (⋃c. 𝒜 v c)" by blast
  qed
next
  show "(⋃c. 𝒜 v c) ⊆ 𝒞 v"
    by (rule Union_activation_collect_le_node_collect)
qed
```

The two results together: the claims of all contexts contain the context-insensitive collection.

```
corollary node_collect_covered: "𝒞 v ⊆ (⋃c. cover v c)"
proof
  fix x assume "x ∈ 𝒞 v"
  then obtain t where t: "t ∈ 𝒯" "sink_node t = v" "sink_store t = x"
    by (rule node_collect_E)
  from valid_activation_trace_has_context[OF t(1)] obtain c where "carries t c" .
  then have "x ∈ cover v c"
    using valid_activation_trace_covered_at[OF t(1)] t(2,3) by blast
  then show "x ∈ (⋃c. cover v c)" by blast
qed

end
```

Non-vacuity

The interface is satisfiable for every graph, seed set, and initial context: choosing the top cover `cover = (λ_ _. UNIV)` together with any total functional context policy discharges all five obligations, `TOTAL` included, since a functional policy is total (`call_context_total_on_of_fun`). It is not satisfiable at an arbitrary `adm`: taking

`adm = (λ_ _ _ _ _. {})` fails `TOTAL` for any graph with a reachable call, however wide `cover` is chosen — `TOTAL` genuinely constrains the context policy, not merely the cover.

```
lemma activation_coverage_exists:
  "activation_coverage κ g S (λ_ _. UNIV) (context_policy_of_fun (λ_ _ _. ())) ()"
  by unfold_locales auto

end
```

Chapter 5

Voblint_Compile

5.1 VIMP_Proc_to_CFG

```
theory VIMP_Proc_to_CFG
  imports "Voblint_CFG.CFG_Def"
begin
```

Procedure-aware CFG compilation for `src_com` programs

Compile a `src_com` program directly into the two-relation procedure-aware CFG. Nodes are `Statement n` (allocated from a counter), `FunctionEntry p`, and `FunctionResult p`. The compiler threads the enclosing procedure name `p`, so a source `Return` and the normal fall-through both reach `FunctionResult p` through an `EA_Ret` intra edge.

Compilation is continuation passing: the node reached when a fragment falls through is an input, not a result. A node therefore denotes a program point between transfer functions, and no fragment invents a node to hold a control flow that its own source construct does not have.

Layout of one procedure `p` with body `b`:

- `FunctionEntry p --EA_Body p--> entry(b)`;
- the compiled body edges, with the epilogue node as the body's continuation;
- `epilogue --EA_Ret None p--> FunctionResult p` (normal fall-through returns without a value);
- each source `Return e` inside `b` compiles to `--EA_Ret e p--> FunctionResult p` and ignores its continuation, so an early return leaves no node behind.

Control effects and their source counterparts:

- source `Return` becomes an explicit `EA_Ret` edge into `FunctionResult`, which also does the work the runtime `Unwind` does at source level;
- source activation handling (`Restore` and the frame pop) becomes `FunctionResult` + the `calls`-edge continuation.

Calls are emitted only into `calls`; a call never appears as an executable intra edge. `Restore` / `Unwind` are runtime-only (excluded by `source_com`); their compile clauses are trivial one-node stubs that never occur for a source program.

Static allocation size

`csize c` is the number of `Statement` indices the fragment of `c` allocates, fixed by the source syntax alone. `compile` needs it before it recurses: `Seq` names the entry node of `c2` — the continuation it hands to `c1` — and that node is `Statement (n + csize c1)`. `compile_next_id` below ties the two together, so the two definitions cannot drift apart silently.

```
fun csize :: "('a, 'v, 'e) gcom ⇒ nat" where
  "csize (SKIP i) = 1"
| "csize (Assign i x a) = 1"
| "csize (Check i l c) = 1"
| "csize (Seq c1 c2) = csize c1 + csize c2"
| "csize (If i b c1 c2) = 1 + csize c1 + csize c2"
| "csize (While i b c) = 1 + csize c"
| "csize (Call i dst q actuals) = 1"
| "csize (Return i e) = 1"
| "csize (Restore i) = 1"
| "csize (Unwind i) = 1"

lemma csize_pos: "0 < csize c"
  by (induction c) auto
```

Command compilation

`compile Π p c k n` returns `(n', entry, intra_edges, call_edges)`, allocating `Statement` indices in the counter range `[n, n')`. The continuation `k` is an *input*: it is the node normal execution reaches when the fragment falls through. A fragment therefore never reports an exit, and a command that cannot fall through simply ignores `k`.

Every fragment enters at `Statement n`, so the entry is determined by the counter and is reported only for convenience. The procedure table Π is threaded so a `Call` can label its `CallEdge` with the callee's declared formals; every other clause ignores it.

Consequences of handing the continuation down rather than reporting an exit:

- `Seq` needs no connecting `EA_Nop`: `c1`'s continuation *is* `c2`'s entry;
- `gcom.If` allocates no merge node: both branches receive the same `k`, and the join happens at `k`, which is the program point after the conditional;
- `While` needs no back-edge node: the body's continuation is the loop head;
- `Return` ignores `k` and emits only its `EA_Ret` edge, so no unreachable continuation node is invented;
- `Call` resumes at `k`, so the resume point is the following program point. `Restore` and `Unwind` are runtime-only (excluded by `source_com`), so their clauses are never exercised for a source program. They stay transparent: one node and a `EA_Nop` to the continuation. Making them dead ends instead would cost the unconditional form of `compile_prog_entry_cfg_reaches_exit` for no gain, since no source program reaches these clauses.

A call edge into an undeclared procedure carries no formals: `Call` compiles against whatever Π `q` says at compile time, declared or not, so its formal list is this classifier rather than a lookup that could fail. Naming it lets lemma statements about compiled call edges cite `call_formals` instead of restating the case split; the `[simp]` tag keeps it transparent to existing automation.

```
definition call_formals :: "('a, 'v, 'e) gptable ⇒ pname ⇒ 'v list" where
  [simp]: "call_formals Π q =
    (case Π q of
       Some decl ⇒ formals decl
     | None ⇒ Code.abort (STR ''call_formals: call to an undeclared procedure'') (λ_. []))"

fun compile ::
  "('a, 'v, 'e) gptable ⇒ pname ⇒ ('a, 'v, 'e) gcom ⇒ cfg_node ⇒ nat
   ⇒ nat × cfg_node
        × (cfg_node × ('v, 'e) gedge_action × cfg_node) set
        × (cfg_node × ('v, 'e) gcall_action × cfg_node × cfg_node) set"
where
  "compile Π p (SKIP i) k n =
     (Suc n, Statement n, {(Statement n, EA_Nop, k)}, {})"
| "compile Π p (Assign i x a) k n =
     (Suc n, Statement n, {(Statement n, EA_Assign x a, k)}, {})"
| "compile Π p (Check i l c) k n =
     (Suc n, Statement n, {(Statement n, EA_Check l c, k)}, {})"
| "compile Π p (Seq c1 c2) k n =
     (let (n1, en1, E1, K1) = compile Π p c1 (Statement (n + csize c1)) n;
          (n2, en2, E2, K2) = compile Π p c2 k (n + csize c1)
      in (n2, en1, E1 ∪ E2, K1 ∪ K2))"
| "compile Π p (If i b c1 c2) k n =
     (let (n1, en1, E1, K1) = compile Π p c1 k (Suc n);
          (n2, en2, E2, K2) = compile Π p c2 k n1
      in (n2, Statement n,
          {(Statement n, EA_Assume b, if is_SKIP c1 then k else en1),
           (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else en2)}
            ∪ (if is_SKIP c1 then {} else E1) ∪ (if is_SKIP c2 then {} else E2),
          K1 ∪ K2))"
| "compile Π p (While i b c) k n =
     (let (n1, en1, E1, K1) = compile Π p c (Statement n) (Suc n)
      in (n1, Statement n,
          {(Statement n, EA_Assume b, en1), (Statement n, EA_AssumeNot b, k)} ∪ E1,
          K1))"
| "compile Π p (Call i dst q actuals) k n =
     (case special_table q of
        Some desc ⇒
          (case classify_special desc actuals of
             None ⇒ (Suc n, Statement n, {(Statement n, EA_Nop, k)}, {})
           | Some sc ⇒
               (case dst of
                  Some x ⇒
                    (Suc n, Statement n, {(Statement n, EA_Special sc x, k)}, {})
                | None ⇒
                    (Suc n, Statement n, {(Statement n, EA_Nop, k)}, {})))
      | None ⇒
          (Suc n, Statement n, {},
           {(Statement n,
             CallEdge dst (call_formals Π q) actuals,
             FunctionEntry q, k)}))"
| "compile Π p (Return i e) k n =
```

```
     (Suc n, Statement n, {(Statement n, EA_Ret e p, FunctionResult p)}, {})"
| "compile Π p (Restore i) k n =
     (Suc n, Statement n, {(Statement n, EA_Nop, k)}, {})"
| "compile Π p (Unwind i) k n =
     (Suc n, Statement n, {(Statement n, EA_Nop, k)}, {})"
```

Procedure and program compilation

A procedure wraps its body between `FunctionEntry p` and `FunctionResult p`. The body is compiled with the *epilogue* node `Statement r` as its continuation, and the epilogue carries the implicit `EA_Ret None p`: normal fall-through returns without a value. A value is published only by an explicit source `Return e`, which compiles to its own `EA_Ret (Some e) p` edge inside the body.

The epilogue exists so that every edge into `FunctionResult p` is a return edge. Passing `FunctionResult p` directly as the body continuation would let a trailing assignment target the result node. It is allocated *after* the body, so statement indices stay in source order and `r` lies inside the procedure's own counter range — which is what keeps edge locality a statement about one fragment.

The epilogue edge is emitted for every body, also for one all of whose paths end in an explicit `Return`; there the epilogue is dead code. Emitting it unconditionally makes every node of a procedure reach its result, which the solver's coverage argument relies on (`prog_node_reaches`).

```
definition compile_proc ::
  "('a, 'v, 'e) gptable ⇒ pname ⇒ ('a, 'v, 'e) gproc_decl ⇒ nat
   ⇒ nat × (cfg_node × ('v, 'e) gedge_action × cfg_node) set
       × (cfg_node × ('v, 'e) gcall_action × cfg_node × cfg_node) set"
where
  "compile_proc Π p decl n =
     (let r = n + csize (body decl);
          (n', ben, E, K) = compile Π p (body decl) (Statement r) n
      in (Suc r,
          insert (FunctionEntry p, EA_Body p, ben)
            (insert (Statement r, EA_Ret None p, FunctionResult p) E),
          K))"

fun compile_procs ::
  "('a, 'v, 'e) gptable ⇒ pname list ⇒ nat
   ⇒ nat × (cfg_node × ('v, 'e) gedge_action × cfg_node) set
       × (cfg_node × ('v, 'e) gcall_action × cfg_node × cfg_node) set"
where
  "compile_procs Π [] n = (n, {}, {})"
| "compile_procs Π (p # ps) n =
     (case Π p of
        None ⇒ compile_procs Π ps n
      | Some decl ⇒
          (let (n1, E, K) = compile_proc Π p decl n;
               (n2, E', K') = compile_procs Π ps n1
           in (n2, E ∪ E', K ∪ K')))"
```

The whole program: every declared procedure, plus `main_body Π` compiled under `prog_main_name` (which the well-formedness predicate keeps disjoint from `ps`). There is no global exit: whole-program completion is `FunctionResult prog_main_name`, and the entry is `FunctionEntry prog_main_name`.

```
definition compile_prog ::
  "('a, 'v, 'e) gptable ⇒ pname list ⇒ ('v, 'e) gcfg"
where
  "compile_prog Π ps =
     (let (n1, Eprocs, Kprocs) = compile_procs Π ps 0;
          (n2, Emain, Kmain) =
            compile_proc Π prog_main_name (|formals = [], body = main_body Π|) n1
      in (| intra = Eprocs ∪ Emain, calls = Kprocs ∪ Kmain,
            cfg_entry = FunctionEntry prog_main_name |))"
```

Decomposition of a whole compiled program into the callee pass and the `main` fragment.

```
lemma compile_prog_intra_split:
  obtains n1 Eprocs Kprocs n2 Emain Kmain where
    "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    "compile_proc Π prog_main_name (|formals = [], body = (main_body Π)|) n1 = (n2, Emain, Kmain)"
    "intra (compile_prog Π ps) = Eprocs ∪ Emain"
    "calls (compile_prog Π ps) = Kprocs ∪ Kmain"
proof -
  obtain n1 Eprocs Kprocs where procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    by (rule prod_cases3)
  obtain n2 Emain Kmain
    where mainc: "compile_proc Π prog_main_name (|formals = [], body = (main_body Π)|) n1
```

```
                  = (n2, Emain, Kmain)"
    by (rule prod_cases3)
  show ?thesis
    by (rule that[OF procs mainc]) (simp_all add: compile_prog_def procs mainc Let_def)
qed

lemma cfg_entry_compile_prog [simp]:
  "cfg_entry (compile_prog Π ps) = FunctionEntry prog_main_name"
  by (simp add: compile_prog_def Let_def split: prod.splits)

lemma cfg_exit_compile_prog [simp]:
  "cfg_exit (compile_prog Π ps) = FunctionResult prog_main_name"
  by (simp add: cfg_exit_def)
```

Allocation arithmetic and statement ranges

A fragment allocates exactly `csize c` indices. This is the obligation the forward numbering rests on: `Seq` names `c2`'s entry before compiling `c1`, so a clause allocating more or fewer nodes than its `csize` claims would leave that continuation dangling. The equation subsumes the counter monotonicity used throughout the ownership lemmas.

```
lemma compile_next_id:
  "compile Π p c k n = (n', en, E, K) ⟹ n' = n + csize c"
proof (induction c arbitrary: k n n' en E K rule: gcom.induct)
  case (Seq c1 c2)
  then obtain n1 en1 E1 K1 n2 en2 E2 K2 where
    c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, en1, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, en2, E2, K2)"
    and n': "n' = n2"
    by (auto simp: Let_def split: prod.splits)
  from Seq.IH(2)[OF c2] n' show ?case by simp
next
  case (If i b c1 c2)
  then obtain n1 en1 E1 K1 n2 en2 E2 K2 where
    c1: "compile Π p c1 k (Suc n) = (n1, en1, E1, K1)"
    and c2: "compile Π p c2 k n1 = (n2, en2, E2, K2)" and n': "n' = n2"
    by (auto split: prod.splits)
  from If.IH(1)[OF c1] If.IH(2)[OF c2] n' show ?case by simp
next
  case (While i b c)
  then obtain n1 en1 E1 K1 where
    c1: "compile Π p c (Statement n) (Suc n) = (n1, en1, E1, K1)" and n': "n' = n1"
    by (auto split: prod.splits)
  from While.IH[OF c1] n' show ?case by simp
qed (auto split: prod.splits option.splits)
```

Every fragment enters at its base counter. Handing the continuation down does not change this, which is why `SKIP` keeps a node of its own instead of reporting the continuation as its entry.

```
lemma compile_entry:
  "compile Π p c k n = (n', en, E, K) ⟹ en = Statement n"
  by (induction c arbitrary: k n n' en E K rule: gcom.induct)
     (auto simp: Let_def split: prod.splits option.splits)
```

Fragment decomposition

The composite clauses, packaged as elimination rules. Every consumer of `compile` needs the same destructuring — sub-fragments, their base counters, and the edge split — and `csize` makes the sub-counters explicit, so no auxiliary equation has to be carried along.

```
lemma compile_SeqE [elim]:
  assumes "compile Π p (Seq c1 c2) k n = (n', en, E, K)"
  obtains n1 E1 K1 n2 E2 K2 where
    "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    "en = Statement n" "E = E1 ∪ E2" "K = K1 ∪ K2"
proof -
  obtain n1 en1 E1 K1 where
    c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, en1, E1, K1)"
    by (cases "compile Π p c1 (Statement (n + csize c1)) n") auto
  obtain n2 en2 E2 K2 where
    c2: "compile Π p c2 k (n + csize c1) = (n2, en2, E2, K2)"
    by (cases "compile Π p c2 k (n + csize c1)") auto
  have e1: "en1 = Statement n" using compile_entry[OF c1] .
  have e2: "en2 = Statement (n + csize c1)" using compile_entry[OF c2] .
```

```
  from assms c1 c2 have "en = Statement n" "E = E1 ∪ E2" "K = K1 ∪ K2"
    using e1 by (auto simp: Let_def)
  with c1 c2 e1 e2 show ?thesis by (auto intro: that)
qed

lemma compile_IfE [elim]:
  assumes "compile Π p (If i b c1 c2) k n = (n', en, E, K)"
  obtains n1 E1 K1 n2 E2 K2 where
    "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    "compile Π p c2 k (Suc n + csize c1) = (n2, Statement (Suc n + csize c1), E2, K2)"
    "en = Statement n"
    "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)),
          (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1))} ∪ (if is_SKIP c1 then
{} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    "K = K1 ∪ K2"
proof -
  obtain n1 en1 E1 K1 where c1: "compile Π p c1 k (Suc n) = (n1, en1, E1, K1)"
    by (cases "compile Π p c1 k (Suc n)") auto
  have e1: "en1 = Statement (Suc n)" using compile_entry[OF c1] .
  have i1: "n1 = Suc n + csize c1" using compile_next_id[OF c1] by simp
  obtain n2 en2 E2 K2 where c2: "compile Π p c2 k n1 = (n2, en2, E2, K2)"
    by (cases "compile Π p c2 k n1") auto
  have e2: "en2 = Statement (Suc n + csize c1)" using compile_entry[OF c2] i1 by simp
  from assms c1 c2 e1 e2
  have "en = Statement n"
    "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)),
          (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1))} ∪ (if is_SKIP c1 then
{} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    "K = K1 ∪ K2"
    by (auto simp: Let_def)
  with c1 c2 e1 e2 i1 show ?thesis by (auto intro: that)
qed

lemma compile_WhileE [elim]:
  assumes "compile Π p (While i b c) k n = (n', en, E, K)"
  obtains n1 E1 K1 where
    "compile Π p c (Statement n) (Suc n) = (n1, Statement (Suc n), E1, K1)"
    "en = Statement n"
    "E = {(Statement n, EA_Assume b, Statement (Suc n)),
          (Statement n, EA_AssumeNot b, k)} ∪ E1"
    "K = K1"
proof -
  obtain n1 en1 E1 K1 where c1: "compile Π p c (Statement n) (Suc n) = (n1, en1, E1, K1)"
    by (cases "compile Π p c (Statement n) (Suc n)") auto
  have e1: "en1 = Statement (Suc n)" using compile_entry[OF c1] .
  from assms c1 e1
  have "en = Statement n"
    "E = {(Statement n, EA_Assume b, Statement (Suc n)),
          (Statement n, EA_AssumeNot b, k)} ∪ E1"
    "K = K1"
    by (auto simp: Let_def)
  with c1 e1 show ?thesis by (auto intro: that)
qed
```

The sub-fragment projections. Descending into one branch needs only that branch's compile tuple and the fact that its edges are among the enclosing fragment's; the full rules above additionally hand back the sibling's counter and edge sets, which such a proof binds and never uses — leaving their types unconstrained — and the exact `E = ...` equation, which it then has to weaken to a subset by hand. These state the weaker fact directly.

```
lemma compile_Seq_leftE [elim]:
  assumes "compile Π p (Seq c1 c2) k n = (n', en, E, K)"
  obtains m F L where
    "compile Π p c1 (Statement (n + csize c1)) n = (m, Statement n, F, L)"
    "F ⊆ E" "L ⊆ K"
proof -
  from assms obtain n1 E1 K1 n2 E2 K2 where
    "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    "en = Statement n" "E = E1 ∪ E2" "K = K1 ∪ K2"
    by (rule compile_SeqE)
  then show ?thesis by (intro that) auto
qed

lemma compile_Seq_rightE [elim]:
  assumes "compile Π p (Seq c1 c2) k n = (n', en, E, K)"
  obtains m F L where
```

```
    "compile Π p c2 k (n + csize c1) = (m, Statement (n + csize c1), F, L)"
    "F ⊆ E" "L ⊆ K"
proof -
  from assms obtain n1 E1 K1 n2 E2 K2 where
    "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    "en = Statement n" "E = E1 ∪ E2" "K = K1 ∪ K2"
    by (rule compile_SeqE)
  then show ?thesis by (intro that) auto
qed

lemma compile_If_leftE [elim]:
  assumes "compile Π p (If i b c1 c2) k n = (n', en, E, K)"
      and "¬ is_SKIP c1"
  obtains m F L where
    "compile Π p c1 k (Suc n) = (m, Statement (Suc n), F, L)"
    "F ⊆ E" "L ⊆ K"
proof -
  from assms(1) obtain n1 E1 K1 n2 E2 K2 where
    "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    "compile Π p c2 k (Suc n + csize c1) = (n2, Statement (Suc n + csize c1), E2, K2)"
    "en = Statement n"
    "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)),
          (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1))} ∪ (if is_SKIP c1 then
{} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    "K = K1 ∪ K2"
    by (rule compile_IfE)
  with assms(2) show ?thesis by (intro that) auto
qed

lemma compile_If_rightE [elim]:
  assumes "compile Π p (If i b c1 c2) k n = (n', en, E, K)"
      and "¬ is_SKIP c2"
  obtains m F L where
    "compile Π p c2 k (Suc n + csize c1) = (m, Statement (Suc n + csize c1), F, L)"
    "F ⊆ E" "L ⊆ K"
proof -
  from assms(1) obtain n1 E1 K1 n2 E2 K2 where
    "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    "compile Π p c2 k (Suc n + csize c1) = (n2, Statement (Suc n + csize c1), E2, K2)"
    "en = Statement n"
    "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)),
          (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1))} ∪ (if is_SKIP c1 then
{} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    "K = K1 ∪ K2"
    by (rule compile_IfE)
  with assms(2) show ?thesis by (intro that) auto
qed

lemma compile_While_bodyE [elim]:
  assumes "compile Π p (While i b c) k n = (n', en, E, K)"
  obtains m F L where
    "compile Π p c (Statement n) (Suc n) = (m, Statement (Suc n), F, L)"
    "F ⊆ E" "L ⊆ K"
proof -
  from assms obtain n1 E1 K1 where
    "compile Π p c (Statement n) (Suc n) = (n1, Statement (Suc n), E1, K1)"
    "en = Statement n"
    "E = {(Statement n, EA_Assume b, Statement (Suc n)),
          (Statement n, EA_AssumeNot b, k)} ∪ E1"
    "K = K1"
    by (rule compile_WhileE)
  then show ?thesis by (intro that) auto
qed
```

A branching fragment's own two edges, for the callers that want them rather than a sub-fragment: the guard edges out of `Statement n`, which are all the fragment adds beyond its branches.

```
lemma compile_If_assume_edges:
  assumes "compile Π p (If i b c1 c2) k n = (n', en, E, K)"
  shows "(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)) ∈ E"
        "(Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1)) ∈ E"
proof -
  from assms obtain n1 E1 K1 n2 E2 K2 where
    "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    "compile Π p c2 k (Suc n + csize c1) = (n2, Statement (Suc n + csize c1), E2, K2)"
    "en = Statement n"
```

```
    and E: "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)),
                (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1))} ∪ (if is_SKIP
c1 then {} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    "K = K1 ∪ K2"
    by (rule compile_IfE)
  show "(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)) ∈ E" using E
    by blast
  show "(Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1)) ∈ E"
    using E by blast
qed

lemma compile_While_assume_edges:
  assumes "compile Π p (While i b c) k n = (n', en, E, K)"
  shows "(Statement n, EA_Assume b, Statement (Suc n)) ∈ E"
        "(Statement n, EA_AssumeNot b, k) ∈ E"
proof -
  from assms obtain n1 E1 K1 where
    "compile Π p c (Statement n) (Suc n) = (n1, Statement (Suc n), E1, K1)"
    "en = Statement n"
    and E: "E = {(Statement n, EA_Assume b, Statement (Suc n)),
                (Statement n, EA_AssumeNot b, k)} ∪ E1"
    "K = K1"
    by (rule compile_WhileE)
  show "(Statement n, EA_Assume b, Statement (Suc n)) ∈ E" using E by blast
  show "(Statement n, EA_AssumeNot b, k) ∈ E" using E by blast
qed
```

The procedure layout, packaged the same way: the body is compiled at `n` with the epilogue as its continuation, the entry bracket points at the body entry `Statement n`, and the epilogue `Statement (n + csize (body decl))` carries the implicit return.

```
lemma compile_procE [elim]:
  assumes "compile_proc Π p decl n = (n', E, K)"
  obtains Eb where
    "compile Π p (body decl) (Statement (n + csize (body decl))) n
       = (n + csize (body decl), Statement n, Eb, K)"
    "E = insert (FunctionEntry p, EA_Body p, Statement n)
          (insert (Statement (n + csize (body decl)), EA_Ret None p, FunctionResult p) Eb)"
    "n' = Suc (n + csize (body decl))"
proof -
  define r where "r = n + csize (body decl)"
  obtain m ben Eb Kb where
    cb: "compile Π p (body decl) (Statement r) n = (m, ben, Eb, Kb)"
    by (rule prod_cases4)
  have e: "ben = Statement n" using compile_entry[OF cb] .
  have i: "m = r" using compile_next_id[OF cb] unfolding r_def by simp
  from assms cb e i
  have "E = insert (FunctionEntry p, EA_Body p, Statement n)
             (insert (Statement r, EA_Ret None p, FunctionResult p) Eb)"
    "K = Kb" "n' = Suc r"
    unfolding compile_proc_def r_def by (auto simp: Let_def)
  with cb e i show ?thesis unfolding r_def by (auto intro: that)
qed
```

The elaborated graph

The graph the analyses run on is compiled from the elaborated procedures.

```
definition elab_cfg :: "imp_prog ⇒ kind_decls ⇒ (var_id, exp) gcfg" where
  "elab_cfg p ds = compile_prog (elab_ptable p ds) (prog_procs p)"

end
```

5.2 Compile_Wellformed

```
theory Compile_Wellformed
  imports VIMP_Proc_to_CFG
begin
```

What is true of the graph the compiler produces

The compiler theory says how a program becomes nodes and edges; this one says what holds of the result. Three facts leave the session. A compiled `calls` set never has two edges out of one node, because each fragment claims fresh `Statement` indices for its call leaves (`compile_prog_calls_source_unique`) — the context-sensitive analyses need it to read a call site's callee off the graph. A compiled graph is finite (`compile_prog_finite`), which every solver instantiation requires. And a compiled graph satisfies `wf_cfg` (`compile_prog_wf`), the compiler-agnostic contract the D/G layer is stated against.

`frag_stmts` is the local vocabulary: the set of `Statement` indices an edge set mentions, which is how a fragment's claim on the counter is expressed and how two procedures are shown not to overlap.

Call-source uniqueness

Each compiled fragment allocates its `Call` leaves at distinct, freshly claimed `Statement` indices, so a compiled `calls` set never has two different edges out of the same source node. This is a structural fact about the compiler, not about `wf_cfg` (which is deliberately compiler-agnostic) or about any particular program: a hand-built CFG whose source node has two distinct outgoing call edges can still violate it, but no `compile_prog` output can, and it is what lets a context-routing hook resolve the one call at a node from the source node alone.

```
lemma compile_calls_source_range:
  "compile Π p c k n = (n', en, E, K) ⟹ (u, act, ce, af) ∈ K
   ⟹ ∃j. u = Statement j ∧ n ≤ j ∧ j < n'"
proof (induction c arbitrary: k n n' en E K rule: gcom.induct)
  case (Seq c1 c2)
  from Seq.prems(1) obtain n1 en1 E1 K1 n2 en2 E2 K2 where
      c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, en1, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, en2, E2, K2)"
    and n': "n' = n2" and K: "K = K1 ∪ K2"
    by (auto simp: Let_def split: prod.splits)
  have i1: "n1 = n + csize c1" using compile_next_id[OF c1] .
  have i2: "n2 = n + csize c1 + csize c2" using compile_next_id[OF c2] by simp
  from Seq.prems(2) K consider (L) "(u, act, ce, af) ∈ K1" | (R) "(u, act, ce, af) ∈ K2" by auto
  then show ?case
  proof cases
    case L with Seq.IH(1)[OF c1 L] i1 i2 show ?thesis unfolding n' by auto
  next
    case R with Seq.IH(2)[OF c2 R] i1 i2 show ?thesis unfolding n' by auto
  qed
next
  case (If i b c1 c2)
  from If.prems(1) obtain n1 en1 E1 K1 n2 en2 E2 K2 where
      c1: "compile Π p c1 k (Suc n) = (n1, en1, E1, K1)"
    and c2: "compile Π p c2 k n1 = (n2, en2, E2, K2)"
    and n': "n' = n2" and K: "K = K1 ∪ K2"
    by (auto split: prod.splits)
  have i1: "n1 = Suc n + csize c1" using compile_next_id[OF c1] by simp
  have i2: "n2 = n1 + csize c2" using compile_next_id[OF c2] .
  from If.prems(2) K consider (L) "(u, act, ce, af) ∈ K1" | (R) "(u, act, ce, af) ∈ K2" by auto
  then show ?case
  proof cases
    case L with If.IH(1)[OF c1 L] i1 i2 show ?thesis unfolding n' by auto
  next
    case R with If.IH(2)[OF c2 R] i1 i2 show ?thesis unfolding n' by auto
  qed
next
  case (While i b c)
  from While.prems(1) obtain n1 en1 E1 K1 where
      c1: "compile Π p c (Statement n) (Suc n) = (n1, en1, E1, K1)"
    and n': "n' = n1" and K: "K = K1"
    by (auto split: prod.splits)
  from While.prems(2) K have "(u, act, ce, af) ∈ K1" by simp
  with While.IH[OF c1] show ?case unfolding n' by auto
qed (auto split: prod.splits option.splits)
```

Two call sets whose sources lie on either side of one counter value cannot share a source, so each being source-unique makes their union source-unique. Both compound commands that split the counter, `Seq` and `gcom.If`,

close their case with it.

```
lemma calls_source_unique_Un:
  assumes u1: "(u, ca1, ce1, af1) ∈ K1 ⟹ (u, ca2, ce2, af2) ∈ K1
               ⟹ ca1 = ca2 ∧ ce1 = ce2 ∧ af1 = af2"
    and u2: "(u, ca1, ce1, af1) ∈ K2 ⟹ (u, ca2, ce2, af2) ∈ K2
               ⟹ ca1 = ca2 ∧ ce1 = ce2 ∧ af1 = af2"
    and lo: "⋀u ca ce af. (u, ca, ce, af) ∈ K1 ⟹ ∃j. u = Statement j ∧ j < m"
    and hi: "⋀u ca ce af. (u, ca, ce, af) ∈ K2 ⟹ ∃j. u = Statement j ∧ m ≤ j"
    and e1: "(u, ca1, ce1, af1) ∈ K1 ∪ K2" and e2: "(u, ca2, ce2, af2) ∈ K1 ∪ K2"
  shows "ca1 = ca2 ∧ ce1 = ce2 ∧ af1 = af2"
  using u1 u2 lo hi e1
  by (metis Un_iff cfg_node.inject(1) e2 leD)

lemma compile_calls_source_unique:
  "compile Π p c k n = (n', en, E, K) ⟹ (u, ca1, ce1, af1) ∈ K ⟹ (u, ca2, ce2, af2) ∈ K
   ⟹ ca1 = ca2 ∧ ce1 = ce2 ∧ af1 = af2"
proof (induction c arbitrary: k n n' en E K rule: gcom.induct)
  case (Seq c1 c2)
  from Seq.prems(1) obtain n1 en1 E1 K1 n2 en2 E2 K2 where
      cc1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, en1, E1, K1)"
    and cc2: "compile Π p c2 k (n + csize c1) = (n2, en2, E2, K2)"
    and K: "K = K1 ∪ K2"
    by (auto simp: Let_def split: prod.splits)
  have i1: "n1 = n + csize c1" using compile_next_id[OF cc1] .
  show ?case
    by (rule calls_source_unique_Un[OF Seq.IH(1)[OF cc1] Seq.IH(2)[OF cc2], where m = n1])
       (use compile_calls_source_range[OF cc1] compile_calls_source_range[OF cc2] i1
            Seq.prems(2,3) K in ‹fastforce+›)
next
  case (If i b c1 c2)
  from If.prems(1) obtain n1 en1 E1 K1 n2 en2 E2 K2 where
      cc1: "compile Π p c1 k (Suc n) = (n1, en1, E1, K1)"
    and cc2: "compile Π p c2 k n1 = (n2, en2, E2, K2)"
    and K: "K = K1 ∪ K2"
    by (auto split: prod.splits)
  show ?case
    by (rule calls_source_unique_Un[OF If.IH(1)[OF cc1] If.IH(2)[OF cc2], where m = n1])
       (use compile_calls_source_range[OF cc1] compile_calls_source_range[OF cc2]
            If.prems(2,3) K in ‹fastforce+›)
next
  case (While i b c)
  from While.prems(1) obtain n1 en1 E1 K1 where
      cc1: "compile Π p c (Statement n) (Suc n) = (n1, en1, E1, K1)"
    and K: "K = K1"
    by (auto split: prod.splits)
  from While.prems(2) K have "(u, ca1, ce1, af1) ∈ K1" by simp
  moreover from While.prems(3) K have "(u, ca2, ce2, af2) ∈ K1" by simp
  ultimately show ?case using While.IH[OF cc1] by blast
qed (auto split: prod.splits option.splits)

lemma compile_proc_counter_mono:
  "compile_proc Π p decl n = (n', E, K) ⟹ n ≤ n'"
  by auto

lemma compile_proc_calls_source_range:
  assumes "compile_proc Π p decl n = (n', E, K)" and "(u, act, ce, af) ∈ K"
  shows "∃j. u = Statement j ∧ n ≤ j ∧ j < n'"
proof -
  from assms(1) obtain Eb where
      body: "compile Π p (body decl) (Statement (n + csize (body decl))) n
               = (n + csize (body decl), Statement n, Eb, K)"
    and n': "n' = Suc (n + csize (body decl))"
    by (rule compile_procE)
  from compile_calls_source_range[OF body assms(2)] show ?thesis unfolding n' by auto
qed

lemma compile_proc_calls_source_unique:
  assumes "compile_proc Π p decl n = (n', E, K)"
    and "(u, ca1, ce1, af1) ∈ K" and "(u, ca2, ce2, af2) ∈ K"
  shows "ca1 = ca2 ∧ ce1 = ce2 ∧ af1 = af2"
proof -
  from assms(1) obtain Eb where
    body: "compile Π p (body decl) (Statement (n + csize (body decl))) n
             = (n + csize (body decl), Statement n, Eb, K)"
    by (rule compile_procE)
  from compile_calls_source_unique[OF body assms(2,3)] show ?thesis .
```

```
qed

lemma compile_procs_counter_mono:
  "compile_procs Π ps n = (n', E, K) ⟹ n ≤ n'"
proof (induction ps arbitrary: n n' E K)
  case Nil then show ?case by simp
next
  case (Cons p ps)
  show ?case
  proof (cases "Π p")
    case None with Cons show ?thesis by simp
  next
    case (Some decl)
    with Cons.prems obtain n1 E0 K0 n2 E' K' where
        cp: "compile_proc Π p decl n = (n1, E0, K0)"
      and rest: "compile_procs Π ps n1 = (n2, E', K')" and n': "n' = n2"
      by (auto split: prod.splits)
    from compile_proc_counter_mono[OF cp] Cons.IH[OF rest] n' show ?thesis by simp
  qed
qed
```

Every edge of a `compile_procs` pass comes from one declared member's `compile_proc` fragment, compiled at some offset inside the pass's counter range; conversely every declared member contributes such a fragment. The three facts below are what every structural property of a pass reduces to.

```
lemma compile_procs_intra_origin:
  assumes "compile_procs Π ps n = (n', E, K)" and "e ∈ E"
  shows "∃p decl m m' Ep Kp. p ∈ set ps ∧ Π p = Some decl
          ∧ compile_proc Π p decl m = (m', Ep, Kp) ∧ n ≤ m ∧ m' ≤ n' ∧ e ∈ Ep"
  using assms
proof (induction ps arbitrary: n n' E K)
  case Nil then show ?case by simp
next
  case (Cons q qs)
  show ?case
  proof (cases "Π q")
    case None
    with Cons.prems have "compile_procs Π qs n = (n', E, K)" by simp
    from Cons.IH[OF this Cons.prems(2)] show ?thesis by (blast intro: list.set_intros(2))
  next
    case (Some declq)
    from Cons.prems(1) Some obtain n1 Eq Kq n2 E' K' where
        cq: "compile_proc Π q declq n = (n1, Eq, Kq)"
      and rest: "compile_procs Π qs n1 = (n2, E', K')"
      and eq: "n' = n2" "E = Eq ∪ E'"
      by (auto split: prod.splits)
    have mono: "n ≤ n1" "n1 ≤ n2"
      using compile_proc_counter_mono[OF cq] compile_procs_counter_mono[OF rest] .
    show ?thesis
    proof (cases "e ∈ Eq")
      case True
      have "q ∈ set (q # qs) ∧ Π q = Some declq ∧ compile_proc Π q declq n = (n1, Eq, Kq)
              ∧ n ≤ n ∧ n1 ≤ n' ∧ e ∈ Eq"
        using Some cq mono(2) eq(1) True by simp
      then show ?thesis by blast
    next
      case False
      with Cons.prems(2) eq(2) have "e ∈ E'" by simp
      from Cons.IH[OF rest this] obtain p decl m m' Ep Kp where
        "p ∈ set qs" "Π p = Some decl" "compile_proc Π p decl m = (m', Ep, Kp)"
        "n1 ≤ m" "m' ≤ n2" "e ∈ Ep" by blast
      with mono(1) eq(1)
      have "p ∈ set (q # qs) ∧ Π p = Some decl ∧ compile_proc Π p decl m = (m', Ep, Kp)
              ∧ n ≤ m ∧ m' ≤ n' ∧ e ∈ Ep"
        by (auto intro: le_trans)
      then show ?thesis by blast
    qed
  qed
qed

lemma compile_procs_calls_origin:
  assumes "compile_procs Π ps n = (n', E, K)" and "e ∈ K"
  shows "∃p decl m m' Ep Kp. p ∈ set ps ∧ Π p = Some decl
          ∧ compile_proc Π p decl m = (m', Ep, Kp) ∧ n ≤ m ∧ m' ≤ n' ∧ e ∈ Kp"
  using assms
proof (induction ps arbitrary: n n' E K)
  case Nil then show ?case by simp
```

```
next
  case (Cons q qs)
  show ?case
  proof (cases "Π q")
    case None
    with Cons.prems have "compile_procs Π qs n = (n', E, K)" by simp
    from Cons.IH[OF this Cons.prems(2)] show ?thesis by (blast intro: list.set_intros(2))
  next
    case (Some declq)
    from Cons.prems(1) Some obtain n1 Eq Kq n2 E' K' where
        cq: "compile_proc Π q declq n = (n1, Eq, Kq)"
      and rest: "compile_procs Π qs n1 = (n2, E', K')"
      and eq: "n' = n2" "K = Kq ∪ K'"
      by (auto split: prod.splits)
    have mono: "n ≤ n1" "n1 ≤ n2"
      using compile_proc_counter_mono[OF cq] compile_procs_counter_mono[OF rest] .
    show ?thesis
    proof (cases "e ∈ Kq")
      case True
      have "q ∈ set (q # qs) ∧ Π q = Some declq ∧ compile_proc Π q declq n = (n1, Eq, Kq)
            ∧ n ≤ n ∧ n1 ≤ n' ∧ e ∈ Kq"
        using Some cq mono(2) eq(1) True by simp
      then show ?thesis by blast
    next
      case False
      with Cons.prems(2) eq(2) have "e ∈ K'" by simp
      from Cons.IH[OF rest this] obtain p decl m m' Ep Kp where
        "p ∈ set qs" "Π p = Some decl" "compile_proc Π p decl m = (m', Ep, Kp)"
        "n1 ≤ m" "m' ≤ n2" "e ∈ Kp" by blast
      with mono(1) eq(1)
      have "p ∈ set (q # qs) ∧ Π p = Some decl ∧ compile_proc Π p decl m = (m', Ep, Kp)
            ∧ n ≤ m ∧ m' ≤ n' ∧ e ∈ Kp"
        by (auto intro: le_trans)
      then show ?thesis by blast
    qed
  qed
qed

lemma compile_procs_member:
  assumes "compile_procs Π ps n = (n', E, K)" and "p ∈ set ps" and "Π p = Some decl"
  shows "∃m m' Ep Kp. compile_proc Π p decl m = (m', Ep, Kp)
         ∧ n ≤ m ∧ m' ≤ n' ∧ Ep ⊆ E ∧ Kp ⊆ K"
  using assms
proof (induction ps arbitrary: n n' E K)
  case Nil then show ?case by simp
next
  case (Cons q qs)
  show ?case
  proof (cases "Π q")
    case None
    with Cons.prems have "compile_procs Π qs n = (n', E, K)" "p ∈ set qs" by auto
    then show ?thesis using Cons.IH Cons.prems(3) by blast
  next
    case (Some declq)
    from Cons.prems(1) Some obtain n1 Eq Kq n2 E' K' where
        cq: "compile_proc Π q declq n = (n1, Eq, Kq)"
      and rest: "compile_procs Π qs n1 = (n2, E', K')"
      and eq: "n' = n2" "E = Eq ∪ E'" "K = Kq ∪ K'"
      by (auto split: prod.splits)
    have mono: "n ≤ n1" "n1 ≤ n2"
      using compile_proc_counter_mono[OF cq] compile_procs_counter_mono[OF rest] .
    show ?thesis
    proof (cases "p = q")
      case True
      with Cons.prems(3) Some have "decl = declq" by simp
      with True cq mono(2) eq
      have "compile_proc Π p decl n = (n1, Eq, Kq) ∧ n ≤ n ∧ n1 ≤ n' ∧ Eq ⊆ E ∧ Kq ⊆ K"
        by simp
      then show ?thesis by blast
    next
      case False
      with Cons.prems(2) have "p ∈ set qs" by simp
      from Cons.IH[OF rest this Cons.prems(3)] obtain m m' Ep Kp where
        "compile_proc Π p decl m = (m', Ep, Kp)" "n1 ≤ m" "m' ≤ n2" "Ep ⊆ E'" "Kp ⊆ K'"
        by blast
      with mono(1) eq
```

```
      have "compile_proc Π p decl m = (m', Ep, Kp) ∧ n ≤ m ∧ m' ≤ n' ∧ Ep ⊆ E ∧ Kp ⊆ K"
        by (auto intro: le_trans)
      then show ?thesis by blast
    qed
  qed
qed

lemma compile_procs_calls_source_range:
  assumes "compile_procs Π ps n = (n', E, K)" and "(u, act, ce, af) ∈ K"
  shows "∃j. u = Statement j ∧ n ≤ j ∧ j < n'"
  using compile_procs_calls_origin[OF assms] compile_proc_calls_source_range by fastforce

lemma compile_procs_calls_source_unique:
  "compile_procs Π ps n = (n', E, K) ⟹ (u, ca1, ce1, af1) ∈ K ⟹ (u, ca2, ce2, af2) ∈ K
   ⟹ ca1 = ca2 ∧ ce1 = ce2 ∧ af1 = af2"
proof (induction ps arbitrary: n n' E K)
  case Nil then show ?case by simp
next
  case (Cons p ps)
  show ?case
  proof (cases "Π p")
    case None with Cons show ?thesis by simp
  next
    case (Some decl)
    with Cons.prems(1) obtain n1 E0 K0 n2 E' K' where
        cp: "compile_proc Π p decl n = (n1, E0, K0)"
      and rest: "compile_procs Π ps n1 = (n2, E', K')" and K: "K = K0 ∪ K'"
      by (auto split: prod.splits)
    have not_both: "⋀u' act1' ce1' af1' act2' ce2' af2'.
        (u', act1', ce1', af1') ∈ K0 ⟹ (u', act2', ce2', af2') ∈ K' ⟹ False"
      using compile_proc_calls_source_range[OF cp]
            compile_procs_calls_source_range[OF rest]
      by fastforce
    from Cons.prems(2) K have m1: "(u, ca1, ce1, af1) ∈ K0 ∨ (u, ca1, ce1, af1) ∈ K'" by auto
    from Cons.prems(3) K have m2: "(u, ca2, ce2, af2) ∈ K0 ∨ (u, ca2, ce2, af2) ∈ K'" by auto
    consider (LL) "(u, ca1, ce1, af1) ∈ K0" "(u, ca2, ce2, af2) ∈ K0"
           | (RR) "(u, ca1, ce1, af1) ∈ K'" "(u, ca2, ce2, af2) ∈ K'"
           | (LR) "(u, ca1, ce1, af1) ∈ K0" "(u, ca2, ce2, af2) ∈ K'"
           | (RL) "(u, ca1, ce1, af1) ∈ K'" "(u, ca2, ce2, af2) ∈ K0"
      using m1 m2 by blast
    then show ?thesis
    proof cases
      case LL then show ?thesis using compile_proc_calls_source_unique[OF cp] by blast
    next
      case RR then show ?thesis using Cons.IH[OF rest] by blast
    next
      case LR then show ?thesis using not_both by blast
    next
      case RL then show ?thesis using not_both by blast
    qed
  qed
qed
```

Whole-program call-source uniqueness: the fact a routed-context analysis needs to resolve the formals of the one call at a node without ambiguity, for any `compile_prog` output. Combines the declared-procedures pass and the `main` pass the same way the well-formedness theorem combines its own two passes.

```
theorem compile_prog_calls_source_unique:
  assumes "(u, ca1, ce1, af1) ∈ calls (compile_prog Π ps)"
    and "(u, ca2, ce2, af2) ∈ calls (compile_prog Π ps)"
  shows "ca1 = ca2 ∧ ce1 = ce2 ∧ af1 = af2"
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (|formals = [], body = (main_body Π)|) n1
                  = (n2, Emain, Kmain)"
    and g: "calls (compile_prog Π ps) = Kprocs ∪ Kmain"
    by (rule compile_prog_intra_split)
  have not_both: "⋀u' act1' ce1' af1' act2' ce2' af2'.
      (u', act1', ce1', af1') ∈ Kprocs ⟹ (u', act2', ce2', af2') ∈ Kmain ⟹ False"
    using compile_procs_calls_source_range[OF procs] compile_proc_calls_source_range[OF mainc]
    by fastforce
  from assms show ?thesis unfolding g
    using compile_procs_calls_source_unique[OF procs] compile_proc_calls_source_unique[OF mainc]
      not_both by (elim UnE) (blast | (drule not_both; blast))+
qed
```

The `Statement` indices a compiled fragment touches. Sources are always freshly allocated, so they lie in the fragment's own counter range; a target may in addition be the continuation, which belongs to the enclosing construct. `kstmt` names that one extra index, and is empty when the continuation is not a `Statement` node.

```
definition frag_stmts ::
  "(cfg_node × ('v, 'e) gedge_action × cfg_node) set
   ⇒ (cfg_node × ('v, 'e) gcall_action × cfg_node × cfg_node) set ⇒ nat set"
where
  "frag_stmts E K =
     {j. ∃a v. (Statement j, a, v) ∈ E} ∪ {j. ∃u a. (u, a, Statement j) ∈ E}
   ∪ {j. ∃act ce af. (Statement j, act, ce, af) ∈ K}
   ∪ {j. ∃u act ce. (u, act, ce, Statement j) ∈ K}"

fun kstmt :: "cfg_node ⇒ nat set" where
  "kstmt (Statement j) = {j}"
| "kstmt (FunctionEntry q) = {}"
| "kstmt (FunctionResult q) = {}"

lemma kstmt_iff [simp]: "j ∈ kstmt v ⟷ v = Statement j"
  by (cases v) auto

lemma frag_stmts_empty [simp]: "frag_stmts {} {} = {}"
  by (simp add: frag_stmts_def)

lemma frag_stmts_Un:
  "frag_stmts (E1 ∪ E2) (K1 ∪ K2) = frag_stmts E1 K1 ∪ frag_stmts E2 K2"
  by (auto simp: frag_stmts_def)

lemma frag_stmts_E_srcI [intro]:
  "(Statement j, a, v) ∈ E ⟹ j ∈ frag_stmts E K"
  unfolding frag_stmts_def by blast

lemma frag_stmts_E_tgtI [intro]:
  "(u, a, Statement j) ∈ E ⟹ j ∈ frag_stmts E K"
  unfolding frag_stmts_def by blast

lemma frag_stmts_K_srcI [intro]:
  "(Statement j, act, ce, af) ∈ K ⟹ j ∈ frag_stmts E K"
  unfolding frag_stmts_def by blast

lemma frag_stmts_K_tgtI [intro]:
  "(u, act, ce, Statement j) ∈ K ⟹ j ∈ frag_stmts E K"
  unfolding frag_stmts_def by blast

lemma compile_frag_stmts_range:
  "compile Π p c k n = (n', en, E, K) ⟹ frag_stmts E K ⊆ {n..<n'} ∪ kstmt k"
proof (induction c arbitrary: k n n' en E K rule: gcom.induct)
  case (Seq c1 c2)
  from Seq.prems obtain n1 en1 E1 K1 n2 en2 E2 K2 where
    c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, en1, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, en2, E2, K2)"
    and n': "n' = n2" and E: "E = E1 ∪ E2" and K: "K = K1 ∪ K2"
    by (auto simp: Let_def split: prod.splits)
  have i1: "n1 = n + csize c1" using compile_next_id[OF c1] .
  have i2: "n2 = n + csize c1 + csize c2" using compile_next_id[OF c2] by simp
  have r1: "frag_stmts E1 K1 ⊆ {n..<n1} ∪ {n + csize c1}" using Seq.IH(1)[OF c1] by simp
  have r2: "frag_stmts E2 K2 ⊆ {n + csize c1..<n2} ∪ kstmt k" using Seq.IH(2)[OF c2] .
  show ?case
    using r1 r2 i1 i2 csize_pos[of c2]
    unfolding E K n' by (simp only: frag_stmts_Un) auto
next
  case (If i b c1 c2)
  from If.prems obtain n1 E1 K1 n2 E2 K2 where
    c1: "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and c2: "compile Π p c2 k (Suc n + csize c1) =
      (n2, Statement (Suc n + csize c1), E2, K2)"
    and E: "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)),
      (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1))}
     ∪ (if is_SKIP c1 then {} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    and K: "K = K1 ∪ K2"
    by (rule compile_IfE)
  have ids: "n1 = Suc n + csize c1" "n2 = Suc n + csize c1 + csize c2"
    "n' = n + csize (If i b c1 c2)"
    using compile_next_id[OF c1] compile_next_id[OF c2] compile_next_id[OF If.prems]
    by simp_all
  have sub: "frag_stmts E K ⊆ frag_stmts E1 K1 ∪ frag_stmts E2 K2
```

```
      ∪ {n, Suc n, Suc n + csize c1} ∪ kstmt k"
    unfolding E K frag_stmts_def by auto
  show ?case
    using sub If.IH(1)[OF c1] If.IH(2)[OF c2] ids csize_pos[of c1] csize_pos[of c2]
    by fastforce
next
  case (While i b c)
  from While.prems obtain n1 en1 E1 K1 where
    c1: "compile Π p c (Statement n) (Suc n) = (n1, en1, E1, K1)"
    and n': "n' = n1"
    and E: "E = {(Statement n, EA_Assume b, en1), (Statement n, EA_AssumeNot b, k)} ∪ E1"
    and K: "K = {} ∪ K1"
    by (auto split: prod.splits)
  have e1: "en1 = Statement (Suc n)" using compile_entry[OF c1] .
  have i1: "n1 = Suc n + csize c" using compile_next_id[OF c1] .
  have r1: "frag_stmts E1 K1 ⊆ {Suc n..<n1} ∪ {n}" using While.IH[OF c1] by simp
  have sub: "frag_stmts E K ⊆ frag_stmts E1 K1 ∪ {n} ∪ kstmt en1 ∪ kstmt k"
    unfolding E K frag_stmts_def by auto
  show ?case
    using sub r1 e1 i1 csize_pos[of c] unfolding n' by fastforce
qed (auto simp: frag_stmts_def split: option.splits)
```

Finiteness

Each pass emits finitely many edges. Everything downstream that folds over the graph – the join at a node, the equation enumeration, the solver's variable set – needs this and nothing else about the compiler.

```
lemma compile_finite:
  "compile Π p c k n = (n', en, E, K) ⟹ finite E ∧ finite K"
  by (induction c arbitrary: k n n' en E K rule: gcom.induct)
     (auto simp: Let_def split: prod.splits option.splits)

lemma compile_proc_finite:
  "compile_proc Π p decl n = (n', E, K) ⟹ finite E ∧ finite K"
  by (auto simp: compile_proc_def Let_def split: prod.splits dest: compile_finite)

lemma compile_procs_finite:
  "compile_procs Π ps n = (n', E, K) ⟹ finite E ∧ finite K"
proof (induction ps arbitrary: n n' E K)
  case Nil then show ?case by simp
next
  case (Cons p ps)
  show ?case
  proof (cases "Π p")
    case None with Cons show ?thesis by simp
  next
    case (Some decl)
    with Cons.prems obtain n1 E0 K0 n2 E' K' where
      cp: "compile_proc Π p decl n = (n1, E0, K0)"
      and rest: "compile_procs Π ps n1 = (n2, E', K')"
      and E: "E = E0 ∪ E'" and K: "K = K0 ∪ K'"
      by (auto split: prod.splits)
    from compile_proc_finite[OF cp] Cons.IH[OF rest] show ?thesis
      unfolding E K by simp
  qed
qed

lemma compile_prog_finite:
  "finite (intra (compile_prog Π ps))
   ∧ finite (calls (compile_prog Π ps))"
  unfolding compile_prog_def
  by (auto simp: Let_def split: prod.splits
       dest: compile_procs_finite compile_proc_finite)
```

Edge shapes for well-formedness

Every emitted call edge targets a `FunctionEntry`; a call is never an intra edge.

```
lemma compile_call_ce_entry:
  "compile Π p c k n = (n', en, E, K) ⟹ (u, act, ce, af) ∈ K
   ⟹ ∃q. ce = FunctionEntry q"
  by (induction c arbitrary: k n n' en E K)
     (auto simp: Let_def split: prod.splits if_splits option.splits)
```

Edge endpoints: a source is always an allocated `Statement`; a target is an allocated `Statement`, the own `FunctionResult p`, or the continuation. The continuation disjunct is what continuation passing adds, and it is what lets the two

`wf_cfg` conditions below be discharged without carrying a side condition through the induction.

```
lemma compile_E_shape:
  "compile Π p c k n = (n', en, E, K) ⟹ (u, a, v) ∈ E
   ⟹ (∃j. u = Statement j)
       ∧ (v = k ∨ v = FunctionResult p ∨ (∃j. v = Statement j))"
proof (induction c arbitrary: k n n' en E K)
  case (Seq c1 c2)
  then obtain n1 en1 E1 K1 n2 en2 E2 K2 where
    c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, en1, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, en2, E2, K2)"
    and E: "E = E1 ∪ E2"
    by (auto simp: Let_def split: prod.splits)
  consider (L) "(u, a, v) ∈ E1" | (R) "(u, a, v) ∈ E2"
    using Seq.prems(2) E by auto
  then show ?case
  proof cases
    case L from Seq.IH(1)[OF c1 L] show ?thesis by auto
  next
    case R from Seq.IH(2)[OF c2 R] show ?thesis by auto
  qed
next
  case (If i b c1 c2)
  then obtain n1 en1 E1 K1 n2 en2 E2 K2 where
    c1: "compile Π p c1 k (Suc n) = (n1, en1, E1, K1)"
    and c2: "compile Π p c2 k n1 = (n2, en2, E2, K2)"
    and E: "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else en1), (Statement n, EA_AssumeNot b, if is_SKIP
c2 then k else en2)}
                ∪ (if is_SKIP c1 then {} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    by (auto split: prod.splits)
  have e1: "en1 = Statement (Suc n)" using compile_entry[OF c1] .
  have e2: "en2 = Statement n1" using compile_entry[OF c2] .
  consider (Guard) "(u, a, v) ∈ {(Statement n, EA_Assume b, if is_SKIP c1 then k else en1),
                                 (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else en2)}"
    | (L) "(u, a, v) ∈ E1" | (R) "(u, a, v) ∈ E2"
    using If.prems(2) E by (auto split: if_splits)
  then show ?case
  proof cases
    case Guard then show ?thesis using e1 e2 by (auto split: if_splits)
  next
    case L from If.IH(1)[OF c1 L] show ?thesis by auto
  next
    case R from If.IH(2)[OF c2 R] show ?thesis by auto
  qed
next
  case (While i b c)
  then obtain n1 en1 E1 K1 where
    c1: "compile Π p c (Statement n) (Suc n) = (n1, en1, E1, K1)"
    and E: "E = {(Statement n, EA_Assume b, en1), (Statement n, EA_AssumeNot b, k)} ∪ E1"
    by (auto split: prod.splits)
  have e1: "en1 = Statement (Suc n)" using compile_entry[OF c1] .
  consider (Guard) "(u, a, v) ∈ {(Statement n, EA_Assume b, en1),
                                 (Statement n, EA_AssumeNot b, k)}"
    | (Body) "(u, a, v) ∈ E1"
    using While.prems(2) E by auto
  then show ?case
  proof cases
    case Guard then show ?thesis using e1 by auto
  next
    case Body from While.IH[OF c1 Body] show ?thesis by auto
  qed
qed (auto split: prod.splits option.splits)
```

Every `EA_Ret` edge lands on the matching `FunctionResult` — the enclosing procedure name in the action equals the one in the target node.

```
lemma compile_ret_wf:
  "compile Π p c k n = (n', en, E, K) ⟹ (u, EA_Ret e q, v) ∈ E
   ⟹ ∀r. k ≠ FunctionResult r ⟹ v = FunctionResult q"
  by (induction c arbitrary: k n n' en E K)
     (auto simp: Let_def split: prod.splits if_splits option.splits)

lemma compile_proc_call_ce_entry:
  "compile_proc Π p decl n = (n', E, K) ⟹ (u, act, ce, af) ∈ K
   ⟹ ∃q. ce = FunctionEntry q"
  by (auto simp: compile_proc_def Let_def split: prod.splits
       dest: compile_call_ce_entry)
```

```
lemma compile_proc_intra_tgt_not_entry:
  "compile_proc Π p decl n = (n', E, K) ⟹ (u, a, v) ∈ E ⟹ v ≠ FunctionEntry q"
  by (auto simp: compile_proc_def Let_def split: prod.splits if_splits
       dest: compile_E_shape compile_entry)

lemma compile_proc_ret_wf:
  "compile_proc Π p decl n = (n', E, K) ⟹ (u, EA_Ret e q, v) ∈ E
   ⟹ v = FunctionResult q"
  by (auto simp: compile_proc_def Let_def split: prod.splits if_splits
       dest: compile_ret_wf)

lemma compile_procs_call_ce_entry:
  "compile_procs Π ps n = (n', E, K) ⟹ (u, act, ce, af) ∈ K
   ⟹ ∃q. ce = FunctionEntry q"
  using compile_procs_calls_origin compile_proc_call_ce_entry by metis

lemma compile_procs_intra_tgt_not_entry:
  "compile_procs Π ps n = (n', E, K) ⟹ (u, a, v) ∈ E ⟹ v ≠ FunctionEntry q"
  using compile_procs_intra_origin compile_proc_intra_tgt_not_entry by metis

lemma compile_procs_ret_wf:
  "compile_procs Π ps n = (n', E, K) ⟹ (u, EA_Ret e q, v) ∈ E ⟹ v = FunctionResult q"
  using compile_procs_intra_origin compile_proc_ret_wf by metis
```

The compiled program is well-formed

The per-pass shape facts above, collected into the single predicate the CFG layer states its own theorems over. Nothing after this point has to know that a graph was compiled rather than written by hand.

```
theorem compile_prog_wf: "wf_cfg (compile_prog Π ps)"
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (|formals = [], body = (main_body Π)|) n1
                  = (n2, Emain, Kmain)"
    and g: "intra (compile_prog Π ps) = Eprocs ∪ Emain"
           "calls (compile_prog Π ps) = Kprocs ∪ Kmain"
    by (rule compile_prog_intra_split)
  show ?thesis
    unfolding wf_cfg_def g
    using compile_procs_call_ce_entry[OF procs] compile_proc_call_ce_entry[OF mainc]
      compile_procs_intra_tgt_not_entry[OF procs] compile_proc_intra_tgt_not_entry[OF mainc]
      compile_procs_ret_wf[OF procs] compile_proc_ret_wf[OF mainc]
    by (intro conjI allI impI; elim UnE) blast+
qed
```

Naming one compiled graph

A worked example names its graph with a definition and then wants the five facts above about that name rather than about `compile_prog`. Restating them per example costs a proof each and re-derives nothing: the facts hold of every compiled graph, with no well-formedness hypothesis. Interpreting this locale at the defining equation supplies all five at once, already phrased in the example's own name.

```
locale compiled_cfg = finite_cfg g
  for Π :: "'a ptable" and ps :: "pname list" and g :: cfg +
  assumes cfg_eq: "g = compile_prog Π ps"
begin
```

`entry` and `exit` are deliberately not `[simp]`: an interpretation would make them global rewrites that unfold `cfg_entry g` under the example's own graph name, which the placement examples reason about symbolically. The finiteness facts (`finite_intra`, `finite_calls`) come from the `finite_cfg` sublocale and carry no such risk.

```
lemma entry: "cfg_entry g = FunctionEntry prog_main_name"
  unfolding cfg_eq by (rule cfg_entry_compile_prog)

lemma exit: "cfg_exit g = FunctionResult prog_main_name"
  unfolding cfg_eq by (rule cfg_exit_compile_prog)

lemma wf: "wf_cfg g"
  unfolding cfg_eq by (rule compile_prog_wf)

end

end
```

5.3 Compile_Invariants

```
theory Compile_Invariants
  imports Compile_Wellformed "Voblint_CFG.CFG_Prune"
begin
```

What a program must satisfy, and what its graph then satisfies

Two things live here. `wf_compile_input` is the contract a source program has to meet before it is worth compiling: the procedure names are distinct, the entry procedure is declared and takes no arguments, and every body is a well-formed source command. `wf_program_compile_input_exec` is the executable form of that contract, and the check the command-line tool actually runs — the contract itself quantifies over the whole procedure table, which no generated code can enumerate, so the executable form walks the declaration list instead and is proved sufficient.

The remaining results are about the graph that comes out: a source `Return` compiles to an edge into its own `FunctionResult`, every statement index is listed in source post-order (`prog_stmt_post_order`) so a report can name its source command, and the entry node reaches the exit.

Compiler input well-formedness

`prog_main_name` names the distinguished entry procedure, declared in Π with an empty formal list; `main_body Π` reads its body back out. The callee list `ps` enumerates the other declared procedures (`dom Π` minus `prog_main_name`), so the entry's nodes never collide with a callee's, yet `FunctionEntry prog_main_name` is an ordinary activation of a declared procedure like any other.

```
definition gwf_compile_input ::
  "('e ⇒ 'v set) ⇒ 'v ⇒ ('v ⇒ bool) ⇒ ('a, 'v, 'e) gptable ⇒ pname list ⇒ bool" where
  "gwf_compile_input vars rv 𝒢 Π ps ⟷
     distinct ps ∧
     set ps = {p. Π p ~= None} - {prog_main_name} ∧
     prog_main_name ∉ set ps ∧
     gwf_source_program vars rv 𝒢 Π"

abbreviation wf_compile_input :: "(vname => bool) => 'a ptable => pname list => bool" where
  "wf_compile_input ≡ gwf_compile_input exp_vnames ret_var"
```

The leaf definitions an executable instance unfolds to decide the contract. No abstract proof unfolds them, so they only ever fire under `unfolding wf_compile_input_simps`.

```
declare
  greserved_ret_var_def [wf_compile_input_simps]
  gsource_exp_def [wf_compile_input_simps]
  gvalid_formal_def [wf_compile_input_simps]
  value_providing_def [wf_compile_input_simps]
  special_table_def [wf_compile_input_simps]
  special_pname_nondet_int_def [wf_compile_input_simps]
  special_pname_min_def [wf_compile_input_simps]
  special_pname_max_def [wf_compile_input_simps]
  ret_var_def [wf_compile_input_simps]
  prog_main_name_def [wf_compile_input_simps]
  main_body_def [wf_compile_input_simps]

lemma wf_compile_inputD:
  assumes "gwf_compile_input vars rv 𝒢 Π ps"
  shows "greserved_ret_var rv 𝒢" and "Π prog_main_name = Some (|formals = [], body = (main_body Π)|)"
    and "gwf_source_com vars rv Π (main_body Π)" and "no_return (main_body Π)"
    and "Π p = Some decl ⟹ gwf_proc_decl vars rv 𝒢 Π decl"
    and "Π p = Some decl ⟹ special_table p = None"
    and "source_pi Π" and "source_com (main_body Π)"
    and "distinct ps" and "set ps = {p. Π p ≠ None} - {prog_main_name}"
    and "prog_main_name ∉ set ps"
  using wf_source_programD [of vars rv 𝒢 Π]
    assms[unfolded gwf_compile_input_def] by auto

lemmas wf_compile_input_reserved_ret_var [dest] = wf_compile_inputD(1)

abbreviation wf_program_compile_input :: "imp_prog ⇒ bool" where
  "wf_program_compile_input p ≡
    wf_compile_input (declared_global p) (prog_table p) (prog_procs p)"
```

An executable, sufficient reformulation

`wf_source_program`'s two $\forall$`p. Π p = ... ⟶ ...` conjuncts range over `Π :: pname => proc_decl option`'s entire (unbounded) function domain, so `wf_program_compile_input` cannot be exported/evaluated as literally stated – Isabelle's code generator has nothing to enumerate a function's domain against. `proc_rep` is always finite in practice (`prog_table p = map_of (proc_rep p)`), so both conjuncts are provably implied by the corresponding `list_all` check over `proc_rep p` directly: every entry `proc_rep p` actually lists is checked, which is at least as strong as checking only the ones `map_of` would resolve a lookup to.

`wf_program_compile_input_exec` only needs to be *sufficient* for the CLI well-formedness gate to be sound (never silently accepting a source program `wf_program_compile_input` would reject) – not a decision procedure complete in the other direction, which `map_of`'s first-occurrence-wins semantics on a `proc_rep` with a shadowed duplicate key would make materially harder to establish for no operational benefit here.

```
definition wf_program_compile_input_exec :: "imp_prog => bool" where
  "wf_program_compile_input_exec p ⟷
     (let procs = proc_rep p; 𝒢 = declared_global p; pi = map_of procs
      in reserved_ret_var 𝒢 ∧
         distinct (prog_procs p) ∧
         set (prog_procs p) = set (map fst procs) - {prog_main_name} ∧
         prog_main_name ∉ set (prog_procs p) ∧
         pi prog_main_name = Some (⦇formals = [], body = prog_main p⦈) ∧
         wf_source_com pi (prog_main p) ∧ no_return (prog_main p) ∧
         list_all (λ(_, decl). wf_proc_decl 𝒢 pi decl) procs ∧
         list_all (λ(q, _). special_table q = None) procs ∧
         list_all lits_fit (com_exps (prog_main p)) ∧
         list_all (λ(_, decl). list_all lits_fit (com_exps (body decl))) procs)"

lemma wf_program_compile_input_exec_sound:
  assumes "wf_program_compile_input_exec p"
  shows "wf_program_compile_input p"
proof -
  define procs where "procs = proc_rep p"
  define 𝒢 where "𝒢 = declared_global p"
  define pi where "pi = map_of procs"
  have pi_eq: "pi = prog_table p"
    unfolding pi_def procs_def prog_table_def ..
  from assms have h:
    "reserved_ret_var 𝒢"
    "distinct (prog_procs p)"
    "set (prog_procs p) = set (map fst procs) - {prog_main_name}"
    "prog_main_name ∉ set (prog_procs p)"
    "pi prog_main_name = Some (⦇formals = [], body = prog_main p⦈)"
    "wf_source_com pi (prog_main p)"
    "no_return (prog_main p)"
    "list_all (λ(_, decl). wf_proc_decl 𝒢 pi decl) procs"
    "list_all (λ(q, _). special_table q = None) procs"
    unfolding wf_program_compile_input_exec_def procs_def 𝒢_def pi_def Let_def
    by simp_all
  have domain: "set (map fst procs) = {q. pi q ≠ None}"
    unfolding pi_def by (induction procs) auto
  have every_decl: "⋀q decl. pi q = Some decl ⟹ wf_proc_decl 𝒢 pi decl"
  proof -
    fix q decl assume "pi q = Some decl"
    then have "(q, decl) ∈ set procs" unfolding pi_def by (rule map_of_SomeD)
    with h(8) show "wf_proc_decl 𝒢 pi decl" by (auto simp: list_all_iff)
  qed
  have every_special: "⋀q. pi q ≠ None ⟹ special_table q = None"
  proof -
    fix q assume "pi q ≠ None"
    then    obtain decl where "(q, decl) ∈ set procs"
      using domain by force
    with h(9) show "special_table q = None" by (auto simp: list_all_iff)
  qed
  have main_eq: "main_body pi = prog_main p" using pi_eq by simp
  have wf_source_program: "wf_source_program 𝒢 pi"
    unfolding gwf_source_program_def main_eq
    using h(1) h(5) h(6) h(7) every_decl every_special by blast
  have wf_compile_input: "wf_compile_input 𝒢 pi (prog_procs p)"
    unfolding gwf_compile_input_def
    using h(2) h(3) h(4) wf_source_program
    using domain by blast
  show ?thesis
    unfolding 𝒢_def[symmetric] pi_eq[symmetric] using wf_compile_input .
qed
```

`compile_prog` takes a `proc_table` and a `pname list`, so a domain holding only an `imp_prog` needs the two-projection wrapper below. The entry name is `prog_main_name` and its body is looked up, so neither is threaded through as a parameter. This is the graph of the surface program; the analyses and the OCaml export use the graph of the elaborated program, `elab_cfg`.

```
definition prog_cfg :: "imp_prog => cfg" where
  "prog_cfg p = compile_prog (prog_table p) (prog_procs p)"
```

Syntactic occurrence predicates

`returns_in` asks a purely syntactic question – does this command contain a `Return` of exactly this expression – which is the shape the compiler lemmas below need. They induct over the source command, so their hypothesis has to be readable off the syntax rather than off the graph the compiler produced from it.

```
fun returns_in :: "'e option ⇒ ('a, 'v, 'e) gcom ⇒ bool" where
  "returns_in e (Return i e') = (e = e')"
| "returns_in e (Seq c1 c2) = (returns_in e c1 ∨ returns_in e c2)"
| "returns_in e (If i _ c1 c2) = (returns_in e c1 ∨ returns_in e c2)"
| "returns_in e (While i _ c) = returns_in e c"
| "returns_in e _ = False"
```

Return edges and call-freeness

Every source `Return e` in procedure `p` compiles to an intra edge that reaches `FunctionResult p` through `EA_Ret e p`.

```
lemma compile_return_edge:
  "compile Π p c k n = (n', en, E, K) ⟹ returns_in e c
   ⟹ ∃j. (Statement j, EA_Ret e p, FunctionResult p) ∈ E"
proof (induction c arbitrary: k n n' en E K rule: gcom.induct)
  case (Seq c1 c2)
  from Seq.prems(1) obtain n1 en1 E1 K1 n2 en2 E2 K2 where
    c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, en1, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, en2, E2, K2)"
    and E: "E = E1 ∪ E2"
    by (auto simp: Let_def split: prod.splits)
  from Seq.prems(2) have "returns_in e c1 ∨ returns_in e c2" by simp
  then show ?case using Seq.IH(1)[OF c1] Seq.IH(2)[OF c2] E by auto
next
  case (If i b c1 c2)
  from If.prems(1) obtain n1 en1 E1 K1 n2 en2 E2 K2 where
    c1: "compile Π p c1 k (Suc n) = (n1, en1, E1, K1)"
    and c2: "compile Π p c2 k n1 = (n2, en2, E2, K2)"
    and E: "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else en1),
                 (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else en2)}
                ∪ (if is_SKIP c1 then {} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    by (auto split: prod.splits)
  from If.prems(2) have "returns_in e c1 ∨ returns_in e c2" by simp
  then show ?case using If.IH(1)[OF c1] If.IH(2)[OF c2] E
    by (auto split: if_splits)
next
  case (While i b c)
  from While.prems(1) obtain n1 en1 E1 K1 where
    c1: "compile Π p c (Statement n) (Suc n) = (n1, en1, E1, K1)"
    and E: "E = {(Statement n, EA_Assume b, en1), (Statement n, EA_AssumeNot b, k)} ∪ E1"
    by (auto split: prod.splits)
  from While.prems(2) have "returns_in e c" by simp
  then show ?case using While.IH[OF c1] E by auto
next
  case (Return i e') then show ?case by (auto split: if_splits)
qed auto
```

Public compiler invariants

A `Return` fragment ignores its continuation: it emits its result edge and nothing else, so no node exists to carry a control flow the source does not have.

```
theorem compile_Return_ignores_continuation:
  "compile Π p (Return i e) k n = compile Π p (Return i e) k' n"
  by simp
```

Where each statement index came from in the source

A source position is nowhere in the compiled graph, yet a report has to name one. What follows supplies the missing link in two steps: an enumeration of the statement indices a fragment allocates, ordered the way a front

end completes its reductions, and the lemmas pinning that enumeration to the compiler's own allocation.

Statement order in the source

Which `Statement` index `compile` gives each command of a fragment, listed in the order a bottom-up parser completes the commands: a reduction finishes only once its whole right-hand side has been read, so an `if` is listed after both its branches and a loop after its body. Every clause allocates the same index its `compile` counterpart does: a leaf takes `n`; `Seq` hands `c2` the counter `c1` ends at; `gcom.If` and `While` take `n` for the guard and start the body at `Suc n`. A front end that records one position per reduction can pair its `k`th position with this list's `k`th index without reproducing `compile`'s counter arithmetic.

```
fun com_stmt_post_order :: "nat ⇒ ('a, 'v, 'e) gcom ⇒ cfg_node list" where
  "com_stmt_post_order n (SKIP i) = [Statement n]"
| "com_stmt_post_order n (Assign i x a) = [Statement n]"
| "com_stmt_post_order n (Check i l c) = [Statement n]"
| "com_stmt_post_order n (Seq c1 c2) =
     com_stmt_post_order n c1 @ com_stmt_post_order (n + csize c1) c2"
| "com_stmt_post_order n (If i b c1 c2) =
     com_stmt_post_order (Suc n) c1 @ com_stmt_post_order (Suc n + csize c1) c2
       @ [Statement n]"
| "com_stmt_post_order n (While i b c) = com_stmt_post_order (Suc n) c @ [Statement n]"
| "com_stmt_post_order n (Call i dst q actuals) = [Statement n]"
| "com_stmt_post_order n (Return i e) = [Statement n]"
| "com_stmt_post_order n (Restore i) = [Statement n]"
| "com_stmt_post_order n (Unwind i) = [Statement n]"
```

The enumeration has one entry per index the fragment allocates: a shorter or longer list would silently misalign every position after it.

```
lemma length_com_stmt_post_order [simp]: "length (com_stmt_post_order n c) = csize c"
  by (induction c arbitrary: n) auto
```

`com_stmt_post_order` answers this for one fragment given the counter it starts at. A whole program needs those starting counters, and they are not a fragment-local fact: `compile_procs` lays the procedures out one after another and `compile_prog` compiles `main` after all of them, whatever order the source wrote them in.

`procs_stmt_next` below is that layout and nothing else — each declared procedure advances the counter past its body and past the one index `compile_proc` reserves for the epilogue. Undeclared names advance it by nothing, matching the clause `compile_procs` skips.

```
fun procs_stmt_next :: "('a, 'v, 'e) gptable ⇒ pname list ⇒ nat ⇒ nat" where
  "procs_stmt_next Π [] n = n"
| "procs_stmt_next Π (p # ps) n =
     (case Π p of
        None ⇒ procs_stmt_next Π ps n
      | Some decl ⇒ procs_stmt_next Π ps (Suc (n + csize (body decl))))"
```

Each definition paired with the statement indices its body owns, in the order a bottom-up parser finishes them. `main` comes last because that is where `compile_prog` compiles it, not because of where it appears in the source — which is exactly why a front end cannot pair a flat position list against the program and must group by definition first.

```
fun defs_stmt_post_order ::
  "'a ptable ⇒ pname list ⇒ nat ⇒ (pname × cfg_node list) list"
where
  "defs_stmt_post_order Π [] n = []"
| "defs_stmt_post_order Π (p # ps) n =
     (case Π p of
        None ⇒ defs_stmt_post_order Π ps n
      | Some decl ⇒
          (p, com_stmt_post_order n (body decl))
            # defs_stmt_post_order Π ps (Suc (n + csize (body decl))))"

definition prog_stmt_post_order :: "imp_prog ⇒ (pname × cfg_node list) list" where
  "prog_stmt_post_order p =
     defs_stmt_post_order (prog_table p) (prog_procs p) 0
       @ [(prog_main_name,
           com_stmt_post_order (procs_stmt_next (prog_table p) (prog_procs p) 0)
             (prog_main p))]"
```

Connectivity of a compiled graph

A compiled fragment's entry reaches its continuation or its procedure result along the structural successor relation. Nothing downstream consumes this: a routed D/G system's own coverage witness is `vars_cover_exec`,

decided per node from whether a caller actually published a seed, not from graph reachability. This connectivity fact is instead what pins the transparent `Restore`/`Unwind` encoding — emitting nothing for those clauses would make them dead ends and force a `source_com` hypothesis onto the whole-program statement.

The fragment-relative forms the compiler inductions use: an edge of a fragment included in the graph.

```
lemma local_reaches_intra_sub:
  "(u, a, v) ∈ E ⟹ E ⊆ intra g ⟹ local_reaches g u v"
  by (auto intro: local_reaches_intra_step)

lemma local_reaches_comb_caller_sub:
  "(c, ca, FunctionEntry q, k) ∈ K ⟹ K ⊆ calls g ⟹ local_reaches g c k"
  by (auto intro: local_reaches_comb_step)
```

Compiled entry reaches its exit or its procedure result

A fragment's entry leads to its continuation or, along an explicit `Return` edge, to the enclosing procedure's `FunctionResult`. Both stay inside the activation, so the statement is about `local_reaches`.

```
lemma compile_reaches:
  assumes "compile Π p c k n = (n', en, E, K)" and "E ⊆ intra g" and "K ⊆ calls g"
  shows "local_reaches g en k ∨ local_reaches g en (FunctionResult p)"
  using assms
proof (induction c arbitrary: k n n' en E K)
  case (Seq c1 c2)
  from Seq.prems(1) obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    and res: "en = Statement n" "E = E1 ∪ E2" "K = K1 ∪ K2"
    by (rule compile_SeqE)
  have h1: "local_reaches g (Statement n) (Statement (n + csize c1))
          ∨ local_reaches g (Statement n) (FunctionResult p)"
    using Seq.IH(1)[OF c1] Seq.prems res by auto
  have h2: "local_reaches g (Statement (n + csize c1)) k
          ∨ local_reaches g (Statement (n + csize c1)) (FunctionResult p)"
    using Seq.IH(2)[OF c2] Seq.prems res by auto
  from h1 h2 show ?case unfolding res(1) by (blast intro: local_reaches_trans)
next
  case (If i b c1 c2)
  from If.prems(1) obtain n1 E1 K1 E2 K2 where
      c1: "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and res: "en = Statement n"
    "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)),
          (Statement n, EA_AssumeNot b,
            if is_SKIP c2 then k else Statement (Suc n + csize c1))}
         ∪ (if is_SKIP c1 then {} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    "K = K1 ∪ K2"
    by (rule compile_IfE)
  have b1: "local_reaches g (Statement n) (if is_SKIP c1 then k else Statement (Suc n))"
    using res(2) If.prems(2) by (blast intro: local_reaches_intra_sub)
  show ?case
  proof (cases "is_SKIP c1")
    case True with b1 res(1) show ?thesis by simp
  next
    case False
    have "local_reaches g (Statement (Suc n)) k
        ∨ local_reaches g (Statement (Suc n)) (FunctionResult p)"
      using If.IH(1)[OF c1] If.prems res False by auto
    with b1 False show ?thesis unfolding res(1) by (auto intro: local_reaches_trans)
  qed
qed (auto simp: Let_def split: prod.splits option.splits
     intro: local_reaches_intra_sub local_reaches_comb_caller_sub)

lemma compile_proc_reaches_result:
  assumes "compile_proc Π p decl n = (n', E, K)"
      and "E ⊆ intra g" "K ⊆ calls g"
  shows "local_reaches g (FunctionEntry p) (FunctionResult p)"
proof -
  let ?r = "n + csize (body decl)"
  obtain Eb where
      body: "compile Π p (body decl) (Statement ?r) n = (?r, Statement n, Eb, K)"
    and E_eq: "E = insert (FunctionEntry p, EA_Body p, Statement n)
                    (insert (Statement ?r, EA_Ret None p, FunctionResult p) Eb)"
    by (rule compile_procE[OF assms(1)])
  have Ebg: "Eb ⊆ intra g" using E_eq assms(2) by auto
  have entry: "local_reaches g (FunctionEntry p) (Statement n)"
```

```
    using E_eq assms(2) by (blast intro: local_reaches_intra_sub)
  have epi: "local_reaches g (Statement ?r) (FunctionResult p)"
    using E_eq assms(2) by (blast intro: local_reaches_intra_sub)
  from compile_reaches[OF body Ebg assms(3)] entry epi show ?thesis
    by (blast intro: local_reaches_trans)
qed

theorem compile_prog_entry_cfg_reaches_exit:
  "cfg_reaches (compile_prog Π ps)
     (cfg_entry (compile_prog Π ps)) (cfg_exit (compile_prog Π ps))"
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      cmain: "compile_proc Π prog_main_name ⦇formals = [], body = (main_body Π)⦈ n1
                = (n2, Emain, Kmain)"
    and g: "intra (compile_prog Π ps) = Eprocs ∪ Emain"
           "calls (compile_prog Π ps) = Kprocs ∪ Kmain"
    by (rule compile_prog_intra_split)
  have "cfg_reaches (compile_prog Π ps)
         (FunctionEntry prog_main_name) (FunctionResult prog_main_name)"
    using g by (intro local_reaches_cfg_reaches compile_proc_reaches_result[OF cmain]) auto
  then show ?thesis by simp
qed
```

The typed program is a compiler input

The typed program is a well-formed compiler input whenever the source program is: elaboration resolves every name to an identity distinct from the return slot, and keeps the control structure the contract inspects.

```
theorem elab_wf_compile_input:
  assumes wf: "wf_compile_input (declared_global p) (prog_table p) ps"
  shows "gwf_compile_input exp_vars ReturnVar is_global (elab_ptable p ds) ps"
proof -
  from wf have src: "wf_source_program (declared_global p) (prog_table p)"
    by (simp add: gwf_compile_input_def)
  note main = elab_ptable_main [OF src, of ds]
  have "gwf_source_com exp_vars ReturnVar (elab_ptable p ds) (main_body (elab_ptable p ds))"
    using src by (simp add: main(2) gwf_source_com_elab_com gwf_source_program_def)
  moreover have "no_return (main_body (elab_ptable p ds))"
    using src by (simp add: main(2) gwf_source_program_def)
  moreover have "gwf_proc_decl exp_vars ReturnVar is_global (elab_ptable p ds) decl"
    if "elab_ptable p ds q = Some decl" for q decl
    using that gwf_proc_decl_elab [OF src] by (auto simp: elab_ptable_def)
  moreover have "special_table q = None" if "elab_ptable p ds q ≠ None" for q
    using that src by (simp add: gwf_source_program_def)
  ultimately show ?thesis
    using wf main(1)
    by (simp add: gwf_compile_input_def gwf_source_program_def greserved_ret_var_def)
qed

end
```

5.4 Residual_Location

```
theory Residual_Location
  imports VIMP_Proc_to_CFG
begin
```

Where a partly executed command sits in the graph

Running a command part way leaves a *residual*: the piece still to execute. `control_at Π p c0 k n r v` states that residual `r` of the fragment `c0` — compiled at offset `n` with continuation `k` — corresponds to CFG node `v`. It tracks control only; store agreement is the simulation relation's business.

A command that *reduces to* `SKIP` is located at the continuation `k`, because `compile` takes the continuation as an input rather than allocating an exit node of its own. A literal source `SKIP` is the exception: it keeps its own statement node until its `EA_Nop` edge is taken.

The relation also locates positions no execution reaches, such as the second half of a `Seq` whose first half always returns. That is harmless: the compiled graph has the matching edges there too, since every procedure body ends at an epilogue whose return edge `compile_proc` always emits.

Located residuals

`Restore` and `Unwind` have no clause: they are runtime-only activation markers rather than compiled commands. Once a `Return` fires the CFG is already at `FunctionResult p` through the `EA_Ret` edge, and the activation return that remains is performed by `cstep` popping a frame, not by any fragment node.

```
inductive control_at ::
  "('a, var_id, exp) gptable ⇒ pname ⇒ ('a, var_id, exp) gcom ⇒ cfg_node ⇒ nat ⇒ ('a, var_id, exp) gcom ⇒ cfg_node
⇒ bool"
  for Π :: "('a, var_id, exp) gptable" and p :: pname
where
  Skip [intro]:
    "control_at Π p (SKIP i) k n (SKIP i) (Statement n)"
| Assign [intro]:
    "control_at Π p (Assign i x a) k n (Assign i x a) (Statement n)"
| AssignDone [intro]:
    "control_at Π p (Assign i x a) k n (SKIP i) k"
| Check [intro]:
    "control_at Π p (VIMP_Proc.gcom.Check i l c) k n (VIMP_Proc.gcom.Check i l c) (Statement n)"
| CheckDone [intro]:
    "control_at Π p (VIMP_Proc.gcom.Check i l c) k n (SKIP i) k"
| SeqLeft [intro]:
    "control_at Π p c1 (Statement (n + csize c1)) n r v ⟹
     control_at Π p (Seq c1 c2) k n (Seq r c2) v"
| SeqRight [intro]:
    "control_at Π p c2 k (n + csize c1) r v ⟹
     control_at Π p (Seq c1 c2) k n r v"
| IfHead [intro]:
    "control_at Π p (If i b c1 c2) k n (If i b c1 c2) (Statement n)"
| IfLeft [intro]:
    "control_at Π p c1 k (Suc n) r v ⟹ ¬ is_SKIP c1 ⟹
     control_at Π p (If i b c1 c2) k n r v"
| IfRight [intro]:
    "control_at Π p c2 k (Suc n + csize c1) r v ⟹ ¬ is_SKIP c2 ⟹
     control_at Π p (If i b c1 c2) k n r v"
| IfDone [intro]:
    "c1 = SKIP j ∨ c2 = SKIP j ⟹ control_at Π p (If i b c1 c2) k n (SKIP j) k"
| WhileHead [intro]:
    "control_at Π p (While i b c) k n (While i b c) (Statement n)"
| WhileUnfolded [intro]:
    "control_at Π p (While i b c) k n
       (If i b (Seq c (While i b c)) (SKIP i)) (Statement n)"
| WhileBody [intro]:
    "control_at Π p c (Statement n) (Suc n) r v ⟹
     control_at Π p (While i b c) k n (Seq r (While i b c)) v"
| WhileDone [intro]:
    "control_at Π p (While i b c) k n (SKIP i) k"
| CallHead [intro]:
    "control_at Π p (Call i dst q actuals) k n (Call i dst q actuals) (Statement n)"
| CallDone [intro]:
    "control_at Π p (Call i dst q actuals) k n (SKIP i) k"
| ReturnHead [intro]:
    "control_at Π p (Return i e) k n (Return i e) (Statement n)"
```

Inversion by the shape of the fragment `c0`, which is what every consumer knows and cases on. `Seq`, `gcom.If` and `While` stay plain `[elim]` because their clauses recurse into a sub-fragment, and an eager rule would chase the nesting; the rest are `[elim!]`. `Restore` and `Unwind` have no clause at all, so inverting them is outright refutation — that is what makes `control_at_not_unwind` a one-liner.

```
inductive_cases control_at_SkipE [elim!]:    "control_at Π p (SKIP i) k n r v"
inductive_cases control_at_AssignE [elim!]:  "control_at Π p (Assign i x a) k n r v"
inductive_cases control_at_CheckE [elim!]:
  "control_at Π p (VIMP_Proc.gcom.Check i l b) k n r v"
inductive_cases control_at_SeqE [elim]:      "control_at Π p (Seq c1 c2) k n r v"
inductive_cases control_at_IfE [elim]:       "control_at Π p (If i b c1 c2) k n r v"
inductive_cases control_at_WhileE [elim]:    "control_at Π p (While i b c) k n r v"
inductive_cases control_at_CallE [elim!]:
  "control_at Π p (Call i dst q actuals) k n r v"
inductive_cases control_at_ReturnE [elim!]:  "control_at Π p (Return i e) k n r v"
inductive_cases control_at_RestoreE [elim!]: "control_at Π p (Restore i) k n r v"
inductive_cases control_at_UnwindE [elim!]:  "control_at Π p (Unwind i) k n r v"
```

Initial location

A source command is initially located at its entry node `Statement n`.

```
lemma control_at_initial:
  "source_com c ⟹ control_at Π p c k n c (Statement n)"
  by (induction c arbitrary: k n) auto
```

Normal completion is located at the continuation

A completed residual is at the continuation already, except for a source `SKIP`, whose own node still carries one store-preserving `EA_Nop` edge to it: both branches of a conditional are compiled with the same continuation, so a completed branch is at `k` directly.

```
lemma compile_control_at_SKIP_exit_path:
  "control_at Π p c0 k n (SKIP i) v ⟹ compile Π p c0 k n = (n', en, E, K)
   ⟹ E ⊆ intra g ⟹ star (cfg_intra_step κ g) (v, s) (k, s)"
proof (induction c0 k n "(SKIP i) :: ('a, var_id, exp) gcom" v arbitrary: n' en E K rule:
  control_at.induct)
  case (Skip k n)
  then have "(Statement n, EA_Nop, k) ∈ intra g" by auto
  then show ?case by (rule intra_path_nop)
next
  case (SeqRight c2 k n c1 v)
  from SeqRight.prems(1) obtain n2 E2 K2 where
    c2: "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    and "E2 ⊆ E"
    by (rule compile_SeqE) blast
  with SeqRight.prems(2) show ?case by (intro SeqRight.hyps(2)[OF c2]) blast
next
  case (IfLeft c1 k n v i b c2)
  from IfLeft.prems(1) obtain n1 E1 K1 where
    c1: "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)" and "E1 ⊆ E"
    by (rule compile_If_leftE[OF _ IfLeft.hyps(3)]) blast
  with IfLeft.prems(2) show ?case by (intro IfLeft.hyps(2)[OF c1]) blast
next
  case (IfRight c2 k n c1 v i b)
  from IfRight.prems(1) obtain n2 E2 K2 where
    c2: "compile Π p c2 k (Suc n + csize c1) = (n2, Statement (Suc n + csize c1), E2, K2)"
    and "E2 ⊆ E"
    by (rule compile_If_rightE[OF _ IfRight.hyps(3)]) blast
  with IfRight.prems(2) show ?case by (intro IfRight.hyps(2)[OF c2]) blast
qed simp_all
```

Located residuals remain source commands

Execution never introduces the runtime-only commands into a residual: whatever remains of a source procedure is itself source syntax. The simulation needs this to rule out the frame-popping cases when it is reasoning inside a procedure body.

```
lemma control_at_source_com:
  assumes "control_at Π p c0 k n r v" and "source_com c0"
  shows "source_com r"  using assms
  by (induction rule: control_at.induct) auto

lemma source_com_no_Restore:
  "source_com c ⟹ ¬ is_Restore c"
```

```
  by auto
end
```

5.5 Residual_Edges

```
theory Residual_Edges
  imports Residual_Location "Voblint_CFG.CFG_Exec"
begin
```

The next CFG edge of a located residual

If a residual is located at a node and the command about to run is a base one — an assignment, a check, a call, a return — then the edge the compiler emitted for it really is in the graph, and taking the source step re-locates the successor residual at that edge's target. Every simulation step reads its edge off one of these facts.

`intra_step` collects the source steps that stay inside one activation, and `intra_step_simulation` assembles the facts below into the statement that any such step is matched by a run of `cstep`.

Six proofs here induct on `control_at` and repeat the same descent: project the sub-fragment's compile tuple, apply the induction hypothesis, lift the result back through `control_at.SeqRight` / `IfLeft` / `IfRight`. It looks factorable and is not. A shared *descend to a base clause* lemma would have to carry the residual down unchanged, and `SeqLeft` and `WhileBody` rebuild it — they relate the outer residual `Seq r c2` to the inner `r`. So the descent survives only in the three clauses that pass the residual through, which is half the cases in two of the six proofs and none of the rest.

Located base residuals emit their compiled edge

A *base* residual is one the compiler turns into a single intra edge, rather than a branch, a call, or a relocation. `emitted_action` is the action on that edge. An ordinary `Call` has none, because it is compiled to a call edge instead; a `Call` that `special_table` classifies is resolved in place, and `wf_source_com` fixes its destination to `Some x`, so only that shape emits an action.

The annotation of the statement a residual runs next. A step leaves the annotation of the command it reduces on the `SKIP` it produces, so this is what the completed residual carries.

```
fun stmt_ann :: "('a, var_id, exp) gcom ⇒ 'a" where
  "stmt_ann (SKIP i) = i"
| "stmt_ann (Assign i x a) = i"
| "stmt_ann (VIMP_Proc.gcom.Check i l c) = i"
| "stmt_ann (Seq c1 c2) = stmt_ann c1"
| "stmt_ann (If i b c1 c2) = i"
| "stmt_ann (While i b c) = i"
| "stmt_ann (Call i dst q actuals) = i"
| "stmt_ann (Return i e) = i"
| "stmt_ann (Restore i) = i"
| "stmt_ann (Unwind i) = i"

fun emitted_action :: "('a, var_id, exp) gcom ⇒ (var_id, exp) gedge_action option" where
  "emitted_action (Assign i x a) = Some (EA_Assign x a)"
| "emitted_action (VIMP_Proc.gcom.Check i l c) = Some (EA_Check l c)"
| "emitted_action (Call i (Some x) q actuals) =
     (case special_table q of
        None ⇒ None
      | Some desc ⇒ map_option (λsc. EA_Special sc x) (classify_special desc actuals))"
| "emitted_action _ = None"
```

Which action it is does not matter to the recursive `control_at` cases: they only carry an edge up through the enclosing fragment and re-locate the completed residual at the same node. So one induction serves every base shape.

```
lemma control_at_emitted_edge:
  "control_at Π p c0 k n r v ⟹ emitted_action r = Some a ⟹
   compile Π p c0 k n = (n', en, E, K) ⟹
   ∃j w. v = Statement j ∧ (Statement j, a, w) ∈ E
       ∧ control_at Π p c0 k n (SKIP (stmt_ann r)) w"
proof (induction arbitrary: n' en E K rule: control_at.induct)
  case (Assign i x' a' k n0)
  then show ?case by auto
next
  case (Check i l b' k n0)
  then show ?case by auto
next
  case (CallHead i dst' q' actuals' k n0)
  show ?case
  proof (cases dst')
    case None with CallHead.prems(1) show ?thesis by simp
```

```
  next
    case (Some x)
    with CallHead.prems(1) obtain desc sc where
      sp: "special_table q' = Some desc" and cl: "classify_special desc actuals' = Some sc"
        and act: "a = EA_Special sc x"
      by (auto split: option.splits)
    have "(Statement n0, EA_Special sc x, k) ∈ E"
      using CallHead.prems(2) sp cl Some by auto
    with act show ?thesis by auto
  qed
next
  case (SeqRight c2 k n0 c1 r v)
  from SeqRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2 k (n0 + csize c1) = (n2, Statement (n0 + csize c1), E2, K2)"
    and sub: "E2 ⊆ E"
    by (rule compile_Seq_rightE)
  from SeqRight.IH[OF SeqRight.prems(1) c2c] obtain j w where
    jw: "v = Statement j" "(Statement j, a, w) ∈ E2"
        "control_at Π p c2 k (n0 + csize c1) (SKIP (stmt_ann r)) w" by blast
  have "control_at Π p (Seq c1 c2) k n0 (SKIP (stmt_ann r)) w"
    using control_at.SeqRight[OF jw(3)] .
  then show ?case using jw sub by blast
next
  case (IfLeft c1 k n0 r v i b c2)
  from IfLeft.prems(2) obtain n1 E1 K1 where
    c1c: "compile Π p c1 k (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_If_leftE[OF _ IfLeft.hyps(2)])
  from IfLeft.IH[OF IfLeft.prems(1) c1c] obtain j w where
    jw: "v = Statement j" "(Statement j, a, w) ∈ E1"
        "control_at Π p c1 k (Suc n0) (SKIP (stmt_ann r)) w" by blast
  have "control_at Π p (If i b c1 c2) k n0 (SKIP (stmt_ann r)) w"
    using control_at.IfLeft[OF jw(3) IfLeft.hyps(2)] .
  then show ?case using jw sub by blast
next
  case (IfRight c2 k n0 c1 r v i b)
  from IfRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2 k (Suc n0 + csize c1)
            = (n2, Statement (Suc n0 + csize c1), E2, K2)"
    and sub: "E2 ⊆ E"
    by (rule compile_If_rightE[OF _ IfRight.hyps(2)])
  from IfRight.IH[OF IfRight.prems(1) c2c] obtain j w where
    jw: "v = Statement j" "(Statement j, a, w) ∈ E2"
        "control_at Π p c2 k (Suc n0 + csize c1) (SKIP (stmt_ann r)) w" by blast
  have "control_at Π p (If i b c1 c2) k n0 (SKIP (stmt_ann r)) w"
    using control_at.IfRight[OF jw(3) IfRight.hyps(2)] .
  then show ?case using jw sub by blast
qed simp_all
```

The check a residual runs next, when its leftmost command is one. A located residual about to run a check sits at that check's `EA_Check` edge, however much sequencing wraps it.

```
fun next_check :: "('a, var_id, exp) gcom ⇒ (check_label × exp) option" where
  "next_check (VIMP_Proc.gcom.Check i l c) = Some (l, c)"
| "next_check (Seq c1 c2) = next_check c1"
| "next_check _ = None"

lemma control_at_next_check_edge:
  "control_at Π p c0 k n r v ⟹ next_check r = Some (l, c) ⟹
   compile Π p c0 k n = (n', en, E, K) ⟹ ∃w. (v, EA_Check l c, w) ∈ E"
proof (induction arbitrary: n' en E K rule: control_at.induct)
  case (SeqLeft c1 n0 r v c2 k)
  from SeqLeft.prems(2) obtain m F L
    where "compile Π p c1 (Statement (n0 + csize c1)) n0 = (m, Statement n0, F, L)"
      and "F ⊆ E"
    by (rule compile_Seq_leftE)
  with SeqLeft.IH SeqLeft.prems(1) show ?case by auto
next
  case (SeqRight c2 k n0 c1 r v)
  from SeqRight.prems(2) obtain m F L
    where "compile Π p c2 k (n0 + csize c1) = (m, Statement (n0 + csize c1), F, L)"
      and "F ⊆ E"
    by (rule compile_Seq_rightE)
  with SeqRight.IH SeqRight.prems(1) show ?case by blast
next
  case (IfLeft c1 k n0 r v i b c2)
  from IfLeft.prems(2) obtain m F L
```

```
    where "compile Π p c1 k (Suc n0) = (m, Statement (Suc n0), F, L)" and "F ⊆ E"
    by (rule compile_If_leftE[OF _ IfLeft.hyps(2)])
  with IfLeft.IH IfLeft.prems(1) show ?case by blast
next
  case (IfRight c2 k n0 c1 r v i b)
  from IfRight.prems(2) obtain m F L
    where "compile Π p c2 k (Suc n0 + csize c1) = (m, Statement (Suc n0 + csize c1), F, L)"
      and "F ⊆ E"
    by (rule compile_If_rightE[OF _ IfRight.hyps(2)])
  with IfRight.IH IfRight.prems(1) show ?case by blast
next
  case (WhileBody c n0 r v i b k)
  from WhileBody.prems(2) obtain m F L
    where "compile Π p c (Statement n0) (Suc n0) = (m, Statement (Suc n0), F, L)"
      and "F ⊆ E"
    by (rule compile_While_bodyE)
  with WhileBody.IH WhileBody.prems(1) show ?case by auto
qed auto
```

A located conditional also arises from a loop unfolding (`While` steps to `If b (Seq c (While b c)) SKIP`), so the `WhileUnfolded` case is real: the true branch re-locates the loop body followed by the loop, the false branch re-locates `SKIP` at the loop's continuation.

```
lemma control_at_if_edges:
  "control_at Π p c0 k n r v ⟹ r = If i b c1 c2 ⟹ source_com r ⟹
   compile Π p c0 k n = (n', en, E, K) ⟹
   ∃j en1 en2. v = Statement j
       ∧ (Statement j, EA_Assume b, en1) ∈ E ∧ (Statement j, EA_AssumeNot b, en2) ∈ E
       ∧ control_at Π p c0 k n c1 en1 ∧ control_at Π p c0 k n c2 en2"
proof (induction arbitrary: n' en E K rule: control_at.induct)
  case (IfHead i b' c1' c2' k n0)
  from IfHead.prems(1) have r: "b' = b" "c1' = c1" "c2' = c2" by auto
  note E = compile_If_assume_edges[OF IfHead.prems(3)]
  from IfHead.prems(2) r have src: "source_com c1'" "source_com c2'" by auto
  have ca1: "control_at Π p (If i b' c1' c2') k n0 c1'
      (if is_SKIP c1' then k else Statement (Suc n0))"
    using src(1) by (cases "is_SKIP c1'") (auto intro: control_at_initial)
  have ca2: "control_at Π p (If i b' c1' c2') k n0 c2'
      (if is_SKIP c2' then k else Statement (Suc n0 + csize c1'))"
  proof (cases "is_SKIP c2'")
    case True then show ?thesis by auto
  next
    case False
    have "control_at Π p (If i b' c1' c2') k n0 c2'
        (Statement (Suc n0 + csize c1'))"
      by (rule control_at.IfRight[OF control_at_initial[OF src(2)] False])
    with False show ?thesis by simp
  qed
  from E ca1 ca2 r show ?case by auto
next
  case (SeqRight c2' k n0 c1' r v)
  from SeqRight.prems(3) obtain n2 E2 K2 where
    c2c: "compile Π p c2' k (n0 + csize c1') = (n2, Statement (n0 + csize c1'), E2, K2)"
    and sub: "E2 ⊆ E"
    by (rule compile_Seq_rightE)
  from SeqRight.IH[OF SeqRight.prems(1) SeqRight.prems(2) c2c] obtain j e1 e2 where
    jw: "v = Statement j" "(Statement j, EA_Assume b, e1) ∈ E2"
        "(Statement j, EA_AssumeNot b, e2) ∈ E2"
        "control_at Π p c2' k (n0 + csize c1') c1 e1"
        "control_at Π p c2' k (n0 + csize c1') c2 e2" by blast
  have "control_at Π p (Seq c1' c2') k n0 c1 e1" "control_at Π p (Seq c1' c2') k n0 c2 e2"
    using control_at.SeqRight[OF jw(4)]
          control_at.SeqRight[OF jw(5)] .
  then show ?case using jw sub by blast
next
  case (IfLeft c1' k n0 r v i b' c2')
  from IfLeft.prems(3) obtain n1 E1 K1 where
    c1c: "compile Π p c1' k (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_If_leftE[OF _ IfLeft.hyps(2)])
  from IfLeft.IH[OF IfLeft.prems(1) IfLeft.prems(2) c1c] obtain j e1 e2 where
    jw: "v = Statement j" "(Statement j, EA_Assume b, e1) ∈ E1"
        "(Statement j, EA_AssumeNot b, e2) ∈ E1"
        "control_at Π p c1' k (Suc n0) c1 e1" "control_at Π p c1' k (Suc n0) c2 e2" by blast
  have "control_at Π p (If i b' c1' c2') k n0 c1 e1" "control_at Π p (If i b' c1' c2') k n0 c2 e2"
    using control_at.IfLeft[OF jw(4) IfLeft.hyps(2)] control_at.IfLeft[OF jw(5) IfLeft.hyps(2)] .
  then show ?case using jw sub by blast
```

```
next
  case (IfRight c2' k n0 c1' r v i b')
  from IfRight.prems(3) obtain n2 E2 K2 where
    c2c: "compile Π p c2' k (Suc n0 + csize c1')
            = (n2, Statement (Suc n0 + csize c1'), E2, K2)"
    and sub: "E2 ⊆ E"
    by (rule compile_If_rightE[OF _ IfRight.hyps(2)])
  from IfRight.IH[OF IfRight.prems(1) IfRight.prems(2) c2c] obtain j e1 e2 where
    jw: "v = Statement j" "(Statement j, EA_Assume b, e1) ∈ E2"
        "(Statement j, EA_AssumeNot b, e2) ∈ E2"
        "control_at Π p c2' k (Suc n0 + csize c1') c1 e1"
        "control_at Π p c2' k (Suc n0 + csize c1') c2 e2" by blast
  have "control_at Π p (If i b' c1' c2') k n0 c1 e1" "control_at Π p (If i b' c1' c2') k n0 c2 e2"
    using control_at.IfRight[OF jw(4) IfRight.hyps(2)] control_at.IfRight[OF jw(5) IfRight.hyps(2)]
    .
  then show ?case using jw sub by blast
next
  case (WhileUnfolded i b'' c'' k n0)
  from WhileUnfolded.prems(1)
  have r: "b'' = b" "Seq c'' (While i b'' c'') = c1" "(SKIP i) = c2" by auto
  note E = compile_While_assume_edges[OF WhileUnfolded.prems(3)]
  from WhileUnfolded.prems(2) have src: "source_com c''" by auto
  have ca1: "control_at Π p (While i b'' c'') k n0 (Seq c'' (While i b'' c'')) (Statement (Suc n0))"
    by (rule control_at.WhileBody[OF control_at_initial[OF src]])
  have ca2: "control_at Π p (While i b'' c'') k n0 (SKIP i) k"
    by (rule control_at.WhileDone)
  from E ca1 ca2 r show ?case by auto
qed simp_all
```

SKIP relocation to the continuation

A completed sub-command reaches its fragment's continuation by CFG steps that keep the store and the activation stack.

A located `SKIP` — a completed sub-command — is at its fragment's continuation already, except for a source `SKIP`, which still has its own node and one `EA_Nop` edge. `compile_control_at_SKIP_exit_path` already establishes that route as a store-preserving intra path; an intra path never touches the activation stack, so `intra_path_imp_cstep_star` reads it back as a `cstep` run at whatever stack is current.

```
lemma control_at_skip_to_exit:
  assumes "control_at Π p c0 k n r v" and "is_SKIP r"
      and "compile Π p c0 k n = (n', en, E, K)" and "E ⊆ intra g"
  shows "κ,g ⊢ (v, s, stk) →c* (k, s, stk)"
proof -
  from assms(2) obtain j where r: "r = SKIP j" by auto
  show ?thesis
    using intra_path_imp_cstep_star[
            OF compile_control_at_SKIP_exit_path[OF assms(1)[unfolded r] assms(3,4)],
            where stk = stk]
    by simp
qed
```

Completed-head sequence relocation

Under continuation passing the head's own last edge already targets the entry of `c2` — or, when the continuation is the enclosing `While`, the loop head — so relocation is exactly `control_at_skip_to_exit`.

```
lemma control_at_seq_skip_reloc:
  "control_at Π p c0 k n r v ⟹ r = Seq (SKIP i) c2 ⟹
   compile Π p c0 k n = (n', en, E, K) ⟹ E ⊆ intra g ⟹ source_com c0 ⟹
   ∃v'. control_at Π p c0 k n c2 v' ∧ (κ,g ⊢ (v, s, stk) →c* (v', s, stk))"
proof (induction arbitrary: n' en E K rule: control_at.induct)
  case (SeqLeft c1 n0 r_in v c2r k)
  from SeqLeft.prems(1) have ri: "is_SKIP r_in" and c2eq: "c2r = c2" by auto
  from SeqLeft.prems(2) obtain n1 E1 K1 where
    c1c: "compile Π p c1 (Statement (n0 + csize c1)) n0 = (n1, Statement n0, E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_Seq_leftE)
  have src2: "source_com c2r" using SeqLeft.prems(4) by simp
  from control_at_skip_to_exit[OF SeqLeft.hyps(1) ri c1c subset_trans[OF sub SeqLeft.prems(3)]]
  have sk: "κ,g ⊢ (v, s, stk) →c* (Statement (n0 + csize c1), s, stk)" .
  have "control_at Π p (Seq c1 c2r) k n0 c2r (Statement (n0 + csize c1))"
    by (rule control_at.SeqRight[OF control_at_initial[OF src2]])
  then have "control_at Π p (Seq c1 c2r) k n0 c2 (Statement (n0 + csize c1))" using c2eq by simp
  with sk show ?case by blast
```

```
next
  case (WhileBody c n0 r_in v i b k)
  from WhileBody.prems(1) have ri: "is_SKIP r_in" and c2eq: "c2 = While i b c" by auto
  from WhileBody.prems(2) obtain n1 E1 K1 where
    cc: "compile Π p c (Statement n0) (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_While_bodyE)
  from control_at_skip_to_exit[OF WhileBody.hyps(1) ri cc subset_trans[OF sub WhileBody.prems(3)]]
  have sk: "κ,g ⊢ (v, s, stk) →c* (Statement n0, s, stk)" .
  have "control_at Π p (While i b c) k n0 (While i b c) (Statement n0)"
    by (rule control_at.WhileHead)
  then have "control_at Π p (While i b c) k n0 c2 (Statement n0)" using c2eq by simp
  with sk show ?case by blast
next
  case (SeqRight c2' k n0 c1 r v)
  from SeqRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2' k (n0 + csize c1) = (n2, Statement (n0 + csize c1), E2, K2)"
    and sub: "E2 ⊆ E"
    by (rule compile_Seq_rightE)
  have src2: "source_com c2'" using SeqRight.prems(4) by simp
  from SeqRight.IH[OF SeqRight.prems(1) c2c subset_trans[OF sub SeqRight.prems(3)] src2]
  obtain v' where v': "control_at Π p c2' k (n0 + csize c1) c2 v'"
    "κ,g ⊢ (v, s, stk) →c* (v', s, stk)" by blast
  have "control_at Π p (Seq c1 c2') k n0 c2 v'"
    using control_at.SeqRight[OF v'(1)] .
  with v'(2) show ?case by blast
next
  case (IfLeft c1 k n0 r v i b c2')
  from IfLeft.prems(2) obtain n1 E1 K1 where
    c1c: "compile Π p c1 k (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_If_leftE[OF _ IfLeft.hyps(2)])
  have src1: "source_com c1" using IfLeft.prems(4) by simp
  from IfLeft.IH[OF IfLeft.prems(1) c1c subset_trans[OF sub IfLeft.prems(3)] src1]
  obtain v' where v': "control_at Π p c1 k (Suc n0) c2 v'"
    "κ,g ⊢ (v, s, stk) →c* (v', s, stk)" by blast
  have "control_at Π p (If i b c1 c2') k n0 c2 v'" using control_at.IfLeft[OF v'(1) IfLeft.hyps(2)]
    .
  with v'(2) show ?case by blast
next
  case (IfRight c2' k n0 c1 r v i b)
  from IfRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2' k (Suc n0 + csize c1)
            = (n2, Statement (Suc n0 + csize c1), E2, K2)"
    and sub: "E2 ⊆ E"
    by (rule compile_If_rightE[OF _ IfRight.hyps(2)])
  have src2: "source_com c2'" using IfRight.prems(4) by simp
  from IfRight.IH[OF IfRight.prems(1) c2c subset_trans[OF sub IfRight.prems(3)] src2]
  obtain v' where v': "control_at Π p c2' k (Suc n0 + csize c1) c2 v'"
    "κ,g ⊢ (v, s, stk) →c* (v', s, stk)" by blast
  have "control_at Π p (If i b c1 c2') k n0 c2 v'"
    using control_at.IfRight[OF v'(1) IfRight.hyps(2)]
    .
  with v'(2) show ?case by blast
qed simp_all
```

The caller continuation after a call

A call site's caller has, after the call, a finite left-spine of residual continuations `afters`: the source commands sequenced after the call, in execution order, collected while descending the activation's left-associated `Seq` spine. `seq_after c afters` wraps them around the active head `c`: `seq_after c [s3, s4] = Seq (Seq c s3) s4`. A tail-position call has `afters = []` and the caller resumes at bare `c`. The list holds pending commands *inside one activation*; suspended *caller activations* are the recursive nesting of `csim` instead.

```
fun seq_after :: "('a, var_id, exp) gcom ⇒ ('a, var_id, exp) gcom list ⇒ ('a, var_id, exp) gcom"
  where
  "seq_after c [] = c"
| "seq_after c (a # as) = seq_after (Seq c a) as"

lemma seq_after_append: "seq_after (seq_after c xs) ys = seq_after c (xs @ ys)"
  by (induction xs arbitrary: c) auto

lemma seq_after_snoc: "seq_after c (xs @ [a]) = Seq (seq_after c xs) a"
  by (simp add: seq_after_append[symmetric])

lemma next_check_seq_after [simp]: "next_check (seq_after c afters) = next_check c"
```

```
  by (induction afters arbitrary: c) auto
```

A `seq_after` spine equals a non-`Seq` atom only when its head does and the continuation list is empty. These discriminators close the leaf cases of the spine inductions.

```
lemma seq_after_eq_iff [simp]:
  "(seq_after c afters = SKIP i) = (c = SKIP i ∧ afters = [])"
  "(SKIP i = seq_after c afters) = (c = SKIP i ∧ afters = [])"
  "(seq_after c afters = Restore i) = (c = Restore i ∧ afters = [])"
  "(Restore i = seq_after c afters) = (c = Restore i ∧ afters = [])"
  "(seq_after c afters = Unwind i) = (c = Unwind i ∧ afters = [])"
  "(Unwind i = seq_after c afters) = (c = Unwind i ∧ afters = [])"
  "(seq_after c afters = Assign i x a) = (c = Assign i x a ∧ afters = [])"
  "(Assign i x a = seq_after c afters) = (c = Assign i x a ∧ afters = [])"
  "(seq_after c afters = VIMP_Proc.gcom.Check i l b) = (c = VIMP_Proc.gcom.Check i l b ∧ afters = [])"
  "(VIMP_Proc.gcom.Check i l b = seq_after c afters) = (c = VIMP_Proc.gcom.Check i l b ∧ afters = [])"
  "(seq_after c afters = If i b c1 c2) = (c = If i b c1 c2 ∧ afters = [])"
  "(If i b c1 c2 = seq_after c afters) = (c = If i b c1 c2 ∧ afters = [])"
  "(seq_after c afters = While i b cw) = (c = While i b cw ∧ afters = [])"
  "(While i b cw = seq_after c afters) = (c = While i b cw ∧ afters = [])"
  "(seq_after c afters = Return i e) = (c = Return i e ∧ afters = [])"
  "(Return i e = seq_after c afters) = (c = Return i e ∧ afters = [])"
  "(seq_after c afters = Call i dst q actuals) = (c = Call i dst q actuals ∧ afters = [])"
  "(Call i dst q actuals = seq_after c afters) = (c = Call i dst q actuals ∧ afters = [])"
  by (induction afters arbitrary: c; auto)+

lemma source_com_seq_afterD [dest]:
  "source_com (seq_after c afters) ⟹
   source_com c ∧ (∀a ∈ set afters. source_com a)"
proof (induction afters arbitrary: c)
  case Nil
  then show ?case by simp
next
  case (Cons a afters)
  have h: "source_com (seq_after (Seq c a) afters)"
    using Cons.prems by simp
  from Cons.IH[OF h] have h1: "source_com (Seq c a)"
    and h2: "∀b ∈ set afters. source_com b" by blast+
  from h1 h2 show ?case by auto
qed

lemma control_at_head_return_afters_no_Restore:
  assumes "control_at Π p c0 k n (seq_after (Return i e) afters) v"
      and "source_com c0"
  shows "∀a ∈ set afters. ¬ is_Restore a"
proof (rule ballI)
  fix a
  assume ain: "a ∈ set afters"
  from control_at_source_com[OF assms] source_com_seq_afterD
  have "source_com a" using ain by blast
  then show "¬ is_Restore a" by (rule source_com_no_Restore)
qed
```

A `Call` at the bottom-left of a `seq_after` spine still sits on the call edge the compiler emitted at the call node, and the source step re-locates `seq_after (SKIP i) afters` at the call's continuation node. The `SeqLeft` / `WhileBody` cases peel the outermost continuation, the passthrough cases keep it.

```
lemma control_at_seq_after_call_edge:
  "control_at Π p c0 k n r v ⟹ r = seq_after (Call i dst q actuals) afters ⟹
   compile Π p c0 k n = (n', en, E, K) ⟹ special_table q = None ⟹
   ∃j w. v = Statement j
       ∧ (Statement j, CallEdge dst (call_formals Π q) actuals,
          FunctionEntry q, w) ∈ K
       ∧ control_at Π p c0 k n (seq_after (SKIP i) afters) w"
proof (induction arbitrary: afters n' en E K rule: control_at.induct)
  case (CallHead ia dst' q' actuals' k n0 afters)
  then show ?case by (auto simp: Let_def split: prod.splits)
next
  case (SeqLeft c1 n0 r v c2 k afters)
  obtain xs where afx: "afters = xs @ [c2]" and req: "r = seq_after (Call i dst q actuals) xs"
    using SeqLeft.prems(1) by (cases afters rule: rev_cases) (auto simp: seq_after_snoc)
  from SeqLeft.prems(2) obtain n1 E1 K1 where
    c1c: "compile Π p c1 (Statement (n0 + csize c1)) n0 = (n1, Statement n0, E1, K1)"
    and sub: "K1 ⊆ K"
    by (rule compile_Seq_leftE)
  from SeqLeft.IH[OF req c1c SeqLeft.prems(3)] obtain j w where
    jw: "v = Statement j"
```

```
      "(Statement j, CallEdge dst (call_formals Π q) actuals,
        FunctionEntry q, w) ∈ K1"
      "control_at Π p c1 (Statement (n0 + csize c1)) n0 (seq_after (SKIP i) xs) w" by blast
  have "control_at Π p (Seq c1 c2) k n0 (seq_after (SKIP i) afters) w"
    using control_at.SeqLeft[OF jw(3)] by (simp add: afx seq_after_snoc)
  then show ?case using jw sub by blast
next
  case (SeqRight c2 k n0 c1 r v afters)
  from SeqRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2 k (n0 + csize c1) = (n2, Statement (n0 + csize c1), E2, K2)"
    and sub: "K2 ⊆ K"
    by (rule compile_Seq_rightE)
  from SeqRight.IH[OF SeqRight.prems(1) c2c SeqRight.prems(3)] obtain j w where
    jw: "v = Statement j"
       "(Statement j, CallEdge dst (call_formals Π q) actuals,
         FunctionEntry q, w) ∈ K2"
       "control_at Π p c2 k (n0 + csize c1) (seq_after (SKIP i) afters) w" by blast
  have "control_at Π p (Seq c1 c2) k n0 (seq_after (SKIP i) afters) w"
    using control_at.SeqRight[OF jw(3)] .
  then show ?case using jw sub by blast
next
  case (IfLeft c1 k n0 r v ia b c2 afters)
  from IfLeft.prems(2) obtain n1 E1 K1 where
    c1c: "compile Π p c1 k (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub: "K1 ⊆ K"
    by (rule compile_If_leftE[OF _ IfLeft.hyps(2)])
  from IfLeft.IH[OF IfLeft.prems(1) c1c IfLeft.prems(3)] obtain j w where
    jw: "v = Statement j"
       "(Statement j, CallEdge dst (call_formals Π q) actuals,
         FunctionEntry q, w) ∈ K1"
       "control_at Π p c1 k (Suc n0) (seq_after (SKIP i) afters) w" by blast
  have "control_at Π p (If ia b c1 c2) k n0 (seq_after (SKIP i) afters) w"
    using control_at.IfLeft[OF jw(3) IfLeft.hyps(2)] .
  then show ?case using jw sub by blast
next
  case (IfRight c2 k n0 c1 r v ia b afters)
  from IfRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2 k (Suc n0 + csize c1)
            = (n2, Statement (Suc n0 + csize c1), E2, K2)"
    and sub: "K2 ⊆ K"
    by (rule compile_If_rightE[OF _ IfRight.hyps(2)])
  from IfRight.IH[OF IfRight.prems(1) c2c IfRight.prems(3)] obtain j w where
    jw: "v = Statement j"
       "(Statement j, CallEdge dst (call_formals Π q) actuals,
         FunctionEntry q, w) ∈ K2"
       "control_at Π p c2 k (Suc n0 + csize c1) (seq_after (SKIP i) afters) w" by blast
  have "control_at Π p (If ia b c1 c2) k n0 (seq_after (SKIP i) afters) w"
    using control_at.IfRight[OF jw(3) IfRight.hyps(2)] .
  then show ?case using jw sub by blast
next
  case (WhileBody c n0 r v ia b k afters)
  obtain xs where afx: "afters = xs @ [While ia b c]"
      and req: "r = seq_after (Call i dst q actuals) xs"
    using WhileBody.prems(1) by (cases afters rule: rev_cases) (auto simp: seq_after_snoc)
  from WhileBody.prems(2) obtain n1 E1 K1 where
    cc: "compile Π p c (Statement n0) (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub: "K1 ⊆ K"
    by (rule compile_While_bodyE)
  from WhileBody.IH[OF req cc WhileBody.prems(3)] obtain j w where
    jw: "v = Statement j"
       "(Statement j, CallEdge dst (call_formals Π q) actuals,
         FunctionEntry q, w) ∈ K1"
       "control_at Π p c (Statement n0) (Suc n0) (seq_after (SKIP i) xs) w" by blast
  have "control_at Π p (While ia b c) k n0 (seq_after (SKIP i) afters) w"
    using control_at.WhileBody[OF jw(3)] by (simp add: afx seq_after_snoc)
  then show ?case using jw sub by blast
qed simp_all

lemma control_at_seq_after_return_edge:
  "control_at Π p c0 k n r v ⟹ r = seq_after (Return i e) afters ⟹
   compile Π p c0 k n = (n', en, E, K) ⟹
   ∃j. v = Statement j ∧ (Statement j, EA_Ret e p, FunctionResult p) ∈ E"
proof (induction arbitrary: afters n' en E K rule: control_at.induct)
  case (ReturnHead i e' k n0 afters)
  then show ?case by auto
next
```

```
  case (SeqLeft c1 n0 r v c2 k afters)
  obtain xs where afx: "afters = xs @ [c2]" and req: "r = seq_after (Return i e) xs"
    using SeqLeft.prems(1) by (cases afters rule: rev_cases) (auto simp: seq_after_snoc)
  from SeqLeft.prems(2) obtain n1 E1 K1 where
    c1c: "compile Π p c1 (Statement (n0 + csize c1)) n0 = (n1, Statement n0, E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_Seq_leftE)
  from SeqLeft.IH[OF req c1c] show ?case using sub by blast
next
  case (SeqRight c2 k n0 c1 r v afters)
  from SeqRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2 k (n0 + csize c1) = (n2, Statement (n0 + csize c1), E2, K2)"
    and sub: "E2 ⊆ E"
    by (rule compile_Seq_rightE)
  from SeqRight.IH[OF SeqRight.prems(1) c2c] show ?case using sub by blast
next
  case (IfLeft c1 k n0 r v i b c2 afters)
  from IfLeft.prems(2) obtain n1 E1 K1 where
    c1c: "compile Π p c1 k (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_If_leftE[OF _ IfLeft.hyps(2)])
  from IfLeft.IH[OF IfLeft.prems(1) c1c] show ?case using sub by blast
next
  case (IfRight c2 k n0 c1 r v i b afters)
  from IfRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2 k (Suc n0 + csize c1)
           = (n2, Statement (Suc n0 + csize c1), E2, K2)"
    and sub: "E2 ⊆ E"
    by (rule compile_If_rightE[OF _ IfRight.hyps(2)])
  from IfRight.IH[OF IfRight.prems(1) c2c] show ?case using sub by blast
next
  case (WhileBody c n0 r v ia b k afters)
  obtain xs where afx: "afters = xs @ [While ia b c]" and req: "r = seq_after (Return i e) xs"
    using WhileBody.prems(1) by (cases afters rule: rev_cases) (auto simp: seq_after_snoc)
  from WhileBody.prems(2) obtain n1 E1 K1 where
    cc: "compile Π p c (Statement n0) (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_While_bodyE)
  from WhileBody.IH[OF req cc] show ?case using sub by blast
qed simp_all

lemma control_at_return_edge:
  "control_at Π p c0 k n r v ⟹ r = Return i e ⟹
   compile Π p c0 k n = (n', en, E, K) ⟹
   ∃j. v = Statement j ∧ (Statement j, EA_Ret e p, FunctionResult p) ∈ E"
  by (rule control_at_seq_after_return_edge[where afters = "[]"]) simp_all
```

The intra-procedural source steps

`intra_step` is the fragment of `pstep` that stays inside one activation without initiating a return: assignment, sequencing (head execution and head completion), both conditionals, and the loop unfolding. It excludes `Call` (pushes a frame), `Return` (produces `Unwind`), and the runtime-only `Restore` / `Unwind` steps. Every intra step preserves the frame stack and keeps the residual source-shaped.

```
inductive intra_step ::
  "kenv ⇒ ('a, var_id, exp) gptable ⇒ ('a, var_id, exp) gcom × store × frame list
    ⇒ ('a, var_id, exp) gcom × store × frame list ⇒ bool" for κ Π where
  IAssign [intro]:
    "intra_step κ Π (Assign i x a, s, frs) ((SKIP i), s(x := kind_conv κ (Some x) (eval a s)), frs)"
| ISpecial [intro]: "special_table q = Some desc ⟹ classify_special desc actuals = Some sc ⟹
             special_result eval sc s v ⟹
             intra_step κ Π (Call i (Some x) q actuals, s, frs) ((SKIP i), s(x := kind_conv κ (Some x) v), frs)"
| ICheck [intro]:  "intra_step κ Π (VIMP_Proc.gcom.Check i l b, s, frs) ((SKIP i), s, frs)"
| ISeq1 [intro]:   "intra_step κ Π (Seq (SKIP i) c2, s, frs) (c2, s, frs)"
| ISeq2 [intro]:   "intra_step κ Π (c1, s, frs) (c1', s', frs) ⟹
           intra_step κ Π (Seq c1 c2, s, frs) (Seq c1' c2, s', frs)"
| IIfTrue [intro]: "truthy (eval b s) ⟹ intra_step κ Π (If i b c1 c2, s, frs) (c1, s, frs)"
| IIfFalse [intro]:"¬ truthy (eval b s) ⟹ intra_step κ Π (If i b c1 c2, s, frs) (c2, s, frs)"
| IWhile [intro]:
  "intra_step κ Π (While i b c, s, frs) (If i b (Seq c (While i b c)) (SKIP i), s, frs)"


inductive_cases intra_SkipE [elim!]:   "intra_step κ Π ((SKIP i), s, frs) y"
inductive_cases intra_AssignE [elim!]: "intra_step κ Π (Assign i x a, s, frs) y"
inductive_cases intra_CheckE [elim!]:  "intra_step κ Π (VIMP_Proc.gcom.Check i l b, s, frs) y"
inductive_cases intra_SeqE [elim]:    "intra_step κ Π (Seq c1 c2, s, frs) y"
```

```
inductive_cases intra_IfE [elim!]:     "intra_step κ Π (If i b c1 c2, s, frs) y"
inductive_cases intra_WhileE [elim]:  "intra_step κ Π (While i b c, s, frs) y"
inductive_cases intra_CallE [elim]:   "intra_step κ Π (Call i dst q actuals, s, frs) y"
inductive_cases intra_ReturnE [elim!]: "intra_step κ Π (Return i e, s, frs) y"

lemma intra_Seq_cases:
  "intra_step κ Π (Seq c1 c2, s, frs) (c', s', frs') ⟹
   (is_SKIP c1 ∧ c' = c2 ∧ s' = s ∧ frs' = frs) ∨
   (∃c1'. c' = Seq c1' c2 ∧ frs' = frs ∧ intra_step κ Π (c1, s, frs) (c1', s', frs))"
  by auto

lemma intra_If_cases:
  "intra_step κ Π (If i b c1 c2, s, frs) (c', s', frs') ⟹
   (truthy (eval b s) ∧ c' = c1 ∧ s' = s ∧ frs' = frs) ∨
   (¬ truthy (eval b s) ∧ c' = c2 ∧ s' = s ∧ frs' = frs)"
  by auto

lemma intra_step_any_frame:
  "intra_step κ Π (c, s, frs) (c', s', frs') ⟹ intra_step κ Π (c, s, fr) (c', s', fr)"
  by (induction "(c, s, frs)" "(c', s', frs')" arbitrary: c s frs c' s' frs'
      rule: intra_step.induct) auto

lemma intra_step_frame_eq:
  "intra_step κ Π (c, s, frs) (c', s', frs') ⟹ frs' = frs"
  by (induction "(c, s, frs)" "(c', s', frs')" arbitrary: c s frs c' s' frs'
      rule: intra_step.induct) auto
```

Intra-step simulation

The base cases of that simulation, once: a located residual with an emitted action moves along its own edge in one `cstep` and re-locates as a completed sub-command. Which action it is only decides the store, which the caller supplies as `s' ∈ edge_step κ a s`.

```
lemma control_at_base_step:
  assumes loc: "control_at Π p c0 k n r v"
      and act: "emitted_action r = Some a"
      and cmp: "compile Π p c0 k n = (n', en, E, K)"
      and sub: "E ⊆ intra g"
      and stp: "s' ∈ edge_step κ a s"
  obtains w where
    "control_at Π p c0 k n (SKIP (stmt_ann r)) w"
    "κ,g ⊢ (v, s, stk) →c* (w, s', stk)"
proof -
  from control_at_emitted_edge[OF loc act cmp] obtain j w where
    jw: "v = Statement j" "(Statement j, a, w) ∈ E"
        "control_at Π p c0 k n (SKIP (stmt_ann r)) w" by blast
  have "(Statement j, a, w) ∈ intra g" using jw(2) sub by blast
  from cstep.Intra[OF this stp] have "κ,g ⊢ (Statement j, s, stk) →c (w, s', stk)" .
  then show ?thesis using jw(1) jw(3) that by auto
qed
```

A located conditional takes whichever assume edge the store selects and lands on the branch located at that edge's target. `gcom.If` heads and loop unfoldings both reach it.

```
lemma control_at_if_step:
  assumes loc: "control_at Π p c0 k n (If i b c1 c2) v"
      and stp: "intra_step κ Π (If i b c1 c2, s, frs) (c', s', frs')"
      and cmp: "compile Π p c0 k n = (n', en, E, K)"
      and sub: "E ⊆ intra g"
      and src: "source_com (If i b c1 c2)"
  shows "frs' = frs
         ∧ (∃v'. control_at Π p c0 k n c' v' ∧ (κ,g ⊢ (v, s, stk) →c* (v', s', stk)))"
proof -
  from control_at_if_edges[OF loc refl src cmp] obtain j en1 en2 where
    jj: "v = Statement j"
       "(Statement j, EA_Assume b, en1) ∈ E" "(Statement j, EA_AssumeNot b, en2) ∈ E"
       "control_at Π p c0 k n c1 en1" "control_at Π p c0 k n c2 en2" by blast
  from intra_If_cases[OF stp] show ?thesis
  proof (elim disjE conjE)
    assume t: "truthy (eval b s)" "c' = c1" "s' = s" "frs' = frs"
    have "(Statement j, EA_Assume b, en1) ∈ intra g" using jj(2) sub by blast
    from star_step1[of "cstep κ g", OF cstep_assume[OF this t(1)]]
    show ?thesis using t jj(1,4) by blast
  next
    assume f: "¬ truthy (eval b s)" "c' = c2" "s' = s" "frs' = frs"
    have "(Statement j, EA_AssumeNot b, en2) ∈ intra g" using jj(3) sub by blast
```

```
    from star_step1[of "cstep κ g", OF cstep_assume_not[OF this f(1)]]
    show ?thesis using f jj(1,5) by blast
  qed
qed
```

A located `Seq r c2` either has finished its head, and relocates to `c2`, or steps inside `r`. Only the caller knows which sub-fragment `r` is located in, so it supplies that second case.

```
lemma control_at_seq_step:
  assumes loc: "control_at Π p c0 k n (Seq r c2) v"
      and stp: "intra_step κ Π (Seq r c2, s, frs) (c', s', frs')"
      and cmp: "compile Π p c0 k n = (n', en, E, K)"
      and sub: "E ⊆ intra g"
      and src: "source_com c0"
      and head: "⋀r'. intra_step κ Π (r, s, frs) (r', s', frs) ⟹
                  ∃v'. control_at Π p c0 k n (Seq r' c2) v'
                       ∧ (κ,g ⊢ (v, s, stk) →c* (v', s', stk))"
  shows "frs' = frs
         ∧ (∃v'. control_at Π p c0 k n c' v' ∧ (κ,g ⊢ (v, s, stk) →c* (v', s', stk)))"
  using intra_Seq_cases[OF stp]
proof (elim disjE exE conjE)
  assume r: "is_SKIP r" and out: "c' = c2" "s' = s" "frs' = frs"
  then obtain j where rj: "r = SKIP j" by auto
  from control_at_seq_skip_reloc[OF loc[unfolded rj] refl cmp sub src] show ?thesis
    using out by blast
next
  fix r' assume "c' = Seq r' c2" "frs' = frs" "intra_step κ Π (r, s, frs) (r', s', frs)"
  with head show ?thesis by blast
qed
```

The theory's conclusion: an `intra_step` of a located residual leaves the frame stack alone, and the graph follows it to a node at which the successor residual is located again. This is the fact the simulation relation is preserved by.

```
theorem intra_step_simulation:
  "control_at Π p c0 k n c v ⟹
   intra_step κ Π (c, s, frs) (c', s', frs') ⟹
   compile Π p c0 k n = (n', en, E, K) ⟹ E ⊆ intra g ⟹ source_com c0 ⟹
   frs' = frs
   ∧ (∃v'. control_at Π p c0 k n c' v'
        ∧ (κ,g ⊢ (v, s, stk) →c* (v', s', stk)))"
proof (induction arbitrary: c' s' frs' n' en E K rule: control_at.induct)
  case (Skip i k n0) then show ?case by blast
next
  case (Assign i x a k n0)
  from Assign.prems(1)
  have out: "c' = (SKIP i)" "s' = s(x := kind_conv κ (Some x) (eval a s))" "frs' = frs"
    by auto
  have act: "emitted_action (Assign i x a) = Some (EA_Assign x a)" by simp
  have mem: "s' ∈ edge_step κ (EA_Assign x a) s" using out(2) by simp
  obtain w where "control_at Π p (Assign i x a) k n0 (SKIP i) w"
      and "κ,g ⊢ (Statement n0, s, stk) →c* (w, s', stk)"
    by (rule control_at_base_step[OF control_at.Assign act Assign.prems(2)
                                     Assign.prems(3) mem]) auto
  then show ?case using out(1,3) by auto
next
  case (AssignDone i x a k n0) then show ?case by blast
next
  case (Check i l b k n0)
  from Check.prems(1) have out: "c' = (SKIP i)" "s' = s" "frs' = frs"
    by auto
  have act: "emitted_action (VIMP_Proc.gcom.Check i l b) = Some (EA_Check l b)" by simp
  have mem: "s' ∈ edge_step κ (EA_Check l b) s" using out(2) by simp
  obtain w where "control_at Π p (VIMP_Proc.gcom.Check i l b) k n0 (SKIP i) w"
      and "κ,g ⊢ (Statement n0, s, stk) →c* (w, s', stk)"
    by (rule control_at_base_step[OF control_at.Check act Check.prems(2)
                                     Check.prems(3) mem]) auto
  then show ?case using out(1,3) by auto
next
  case (CheckDone i l b k n0) then show ?case by blast
next
  case (SeqLeft c1 n0 r v c2 k)
  from SeqLeft.prems(2) obtain n1 E1 K1 where
    c1c: "compile Π p c1 (Statement (n0 + csize c1)) n0 = (n1, Statement n0, E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_Seq_leftE)
  have src1: "source_com c1" using SeqLeft.prems(4) by simp
```

```
  show ?case
    by (rule control_at_seq_step[OF control_at.SeqLeft[OF SeqLeft.hyps] SeqLeft.prems])
       (use SeqLeft.IH[OF _ c1c subset_trans[OF sub SeqLeft.prems(3)] src1] in blast)
next
  case (SeqRight c2 k n0 c1 r v)
  from SeqRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2 k (n0 + csize c1) = (n2, Statement (n0 + csize c1), E2, K2)"
    and sub: "E2 ⊆ E"
    by (rule compile_Seq_rightE)
  have src2: "source_com c2" using SeqRight.prems(4) by simp
  from SeqRight.IH[OF SeqRight.prems(1) c2c subset_trans[OF sub SeqRight.prems(3)] src2]
  have fr: "frs' = frs" and
    "∃v'. control_at Π p c2 k (n0 + csize c1) c' v'
          ∧ (κ,g ⊢ (v, s, stk) →c* (v', s', stk))" by auto
  then obtain v' where v': "control_at Π p c2 k (n0 + csize c1) c' v'"
    "κ,g ⊢ (v, s, stk) →c* (v', s', stk)" by blast
  have "control_at Π p (Seq c1 c2) k n0 c' v'"
    using control_at.SeqRight[OF v'(1)] .
  with fr v'(2) show ?case by blast
next
  case (IfHead i b c1 c2 k n0)
  show ?case by (rule control_at_if_step[OF control_at.IfHead IfHead.prems])
next
  case (IfLeft c1 k n0 r v i b c2)
  from IfLeft.prems(2) obtain n1 E1 K1 where
    c1c: "compile Π p c1 k (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub1: "E1 ⊆ E"
    by (rule compile_If_leftE[OF _ IfLeft.hyps(2)])
  have src1: "source_com c1" using IfLeft.prems(4) by simp
  from IfLeft.IH[OF IfLeft.prems(1) c1c subset_trans[OF sub1 IfLeft.prems(3)] src1]
  have fr: "frs' = frs" and
    "∃v'. control_at Π p c1 k (Suc n0) c' v'
          ∧ (κ,g ⊢ (v, s, stk) →c* (v', s', stk))" by auto
  then obtain v' where v': "control_at Π p c1 k (Suc n0) c' v'"
    "κ,g ⊢ (v, s, stk) →c* (v', s', stk)" by blast
  have "control_at Π p (If i b c1 c2) k n0 c' v'" using control_at.IfLeft[OF v'(1) IfLeft.hyps(2)] .
  with fr v'(2) show ?case by blast
next
  case (IfRight c2 k n0 c1 r v i b)
  from IfRight.prems(2) obtain n2 E2 K2 where
    c2c: "compile Π p c2 k (Suc n0 + csize c1)
            = (n2, Statement (Suc n0 + csize c1), E2, K2)"
    and sub2: "E2 ⊆ E"
    by (rule compile_If_rightE[OF _ IfRight.hyps(2)])
  have src2: "source_com c2" using IfRight.prems(4) by simp
  from IfRight.IH[OF IfRight.prems(1) c2c subset_trans[OF sub2 IfRight.prems(3)] src2]
  have fr: "frs' = frs" and
    "∃v'. control_at Π p c2 k (Suc n0 + csize c1) c' v'
          ∧ (κ,g ⊢ (v, s, stk) →c* (v', s', stk))" by auto
  then obtain v' where v': "control_at Π p c2 k (Suc n0 + csize c1) c' v'"
    "κ,g ⊢ (v, s, stk) →c* (v', s', stk)" by blast
  have "control_at Π p (If i b c1 c2) k n0 c' v'" using control_at.IfRight[OF v'(1) IfRight.hyps(2)]
    .
  with fr v'(2) show ?case by blast
next
  case (IfDone i b c1 j c2 k n0) then show ?case by blast
next
  case (WhileHead i b cW k n0)
  from WhileHead.prems(1)
  have out: "c' = If i b (Seq cW (While i b cW)) (SKIP i)" "s' = s" "frs' = frs"
    by auto
  have "control_at Π p (While i b cW) k n0 (If i b (Seq cW (While i b cW)) (SKIP i)) (Statement n0)"
    by (rule control_at.WhileUnfolded)
  then show ?case using out by auto
next
  case (WhileUnfolded i b cW k n0)
  have srcif: "source_com (If i b (Seq cW (While i b cW)) (SKIP i))" using WhileUnfolded.prems(4)
    by simp
  show ?case
    by (rule control_at_if_step[OF control_at.WhileUnfolded WhileUnfolded.prems(1-3) srcif])
next
  case (WhileBody cW n0 r v i b k)
  from WhileBody.prems(2) obtain n1 E1 K1 where
    cc: "compile Π p cW (Statement n0) (Suc n0) = (n1, Statement (Suc n0), E1, K1)"
    and sub: "E1 ⊆ E"
    by (rule compile_While_bodyE)
```

```
  have srcW: "source_com cW" using WhileBody.prems(4) by simp
  show ?case
    by (rule control_at_seq_step[OF control_at.WhileBody[OF WhileBody.hyps] WhileBody.prems])
       (use WhileBody.IH[OF _ cc subset_trans[OF sub WhileBody.prems(3)] srcW] in blast)
next
  case (WhileDone i b cW k n0) then show ?case by blast
next
  case (CallHead i dst q actuals k n0)
  from CallHead.prems(1) obtain x desc sc v where
    out: "dst = Some x" "special_table q = Some desc" "classify_special desc actuals = Some sc"
         "special_result eval sc s v" "c' = (SKIP i)" "s' = s(x := kind_conv κ (Some x) v)"
          "frs' = frs"
    by auto
  have act: "emitted_action (Call i (Some x) q actuals) = Some (EA_Special sc x)"
    using out(2,3) by simp
  have mem: "s' ∈ edge_step κ (EA_Special sc x) s" using out(4,6) by auto
  obtain w where "control_at Π p (Call i (Some x) q actuals) k n0 (SKIP i) w"
     and "κ,g ⊢ (Statement n0, s, stk) →c* (w, s', stk)"
    by (rule control_at_base_step[OF control_at.CallHead act
            CallHead.prems(2)[unfolded out(1)] CallHead.prems(3) mem]) auto
  then show ?case using out(1,5,7) by auto
next
  case (CallDone i dst q actuals k n0) then show ?case by blast
next
  case (ReturnHead i e k n0) then show ?case by blast
qed


end
```

5.6 Simulation_Relation

```
theory Simulation_Relation
  imports Residual_Edges
begin
```

What it means for a source state and a graph state to agree

`csim` relates a source configuration `(c, s, frs)` to a CFG configuration `(v, s, stk)` holding the very same store. The two represent suspended callers differently: the source nests each caller's continuation inside the single command `c`, as `Seq (Seq inner (Restore i)) after` wrappings, while the CFG keeps them as continuation nodes on its frame stack. `csim` pairs the two off, one layer per suspended caller.

Both nestings run outermost-first, so the top command layer pairs with the last frame. After `main` calls `f` calls `g` the source command is `Seq (Seq (Seq (Seq C Restore) B) Restore) A` — `A` is main's continuation and `C` the running residual of `g` — while the CFG sits at `g`'s node with continuation frames `[B's node, A's node]`.

Three constructors, one per phase an activation can be in. `Base` relates a single activation with nothing beneath it. `Nested` peels one suspended caller. `Returning` covers the skew just after a callee finishes: the CFG has already reached `FunctionResult p`, while the source is still propagating `Restore` or `Unwind` towards its frame-pop rule. Proving that execution preserves `csim` is the next theory's business; this one defines the relation and the vocabulary it is stated in.

Return-store update

The store published by a return, common to the source `Return` step and the compiled `EA_Ret` edge.

```
definition ret_store :: "exp option ⇒ store ⇒ store" where
  "ret_store e s = s(ReturnVar := (case e of None ⇒ s ReturnVar | Some a ⇒ eval a s))"

lemma ret_store_None [simp]: "ret_store None s = s"
  by (simp add: ret_store_def)

lemma ret_store_Some [simp]: "ret_store (Some e) s = s(ReturnVar := eval e s)"
  by (simp add: ret_store_def)

lemma edge_step_EA_Ret_ret_store_mem [simp]: "ret_store e s ∈ edge_step κ (EA_Ret e p) s"
  by (simp add: ret_store_def)
```

Bare Unwind is stuck

A lone `Unwind` has no source step: only the `Seq` rules propagate it, and the frame pop it is heading for fires on `Seq (Unwind j) (Restore i)`.

```
lemma pstep_Unwind_stuck: "¬ κ,Π ⊢ ((Unwind i), s, frs) →p x"
  by auto
```

Why a top-level `Return` must not reach the source: an empty frame stack has nowhere to unwind to, so a run that starts at `Unwind` or `Return` never completes.

```
lemma Unwind_not_pcompletes: "¬ (κ,Π ⊢ (Unwind (), s, []) →p* (SKIP (), t, []))"
proof (rule notI)
  assume "κ,Π ⊢ (Unwind (), s, []) →p* (SKIP (), t, [])"
  then show False
    by (cases rule: star.cases) (auto simp: pstep_Unwind_stuck)
qed

lemma Return_empty_not_pcompletes: "¬ (κ,Π ⊢ (Return () e, s, []) →p* (SKIP (), t, []))"
proof (rule notI)
  assume "κ,Π ⊢ (Return () e, s, []) →p* (SKIP (), t, [])"
  then obtain y where step: "κ,Π ⊢ (Return () e, s, []) →p y"
      and rest: "κ,Π ⊢ y →p* (SKIP (), t, [])"
    by (cases rule: star.cases) auto
  from step have "y = (Unwind (), ret_store e s, [])"
    by (cases e) (auto simp: ret_store_def)
  with rest have "κ,Π ⊢ (Unwind (), ret_store e s, []) →p* (SKIP (), t, [])" by simp
  then show False
    by (cases rule: star.cases) (auto simp: pstep_Unwind_stuck)
qed
```

The returning (frame-pop) source shapes

Once a callee finishes, the CFG is already at `FunctionResult p` while the source propagates toward the frame-pop rule. `unwinding` describes an `Unwind` travelling up through the callee's residual, dropping the dead code

it skips (never the activation's own `Restore`); `pop_ready` describes the whole activation body in that phase — either the normal fall-through `Restore` or an `Unwind` spine `Seq u Restore` from an explicit `Return`. Both pop the single caller frame with one `cstep` return; `unwinding` propagation is matched by *no* CFG step, because the CFG waits at `FunctionResult p`.

```
fun unwinding :: "('a, var_id, exp) gcom ⇒ bool" where
  "unwinding (Unwind i) = True"
| "unwinding (Seq u c2) = (unwinding u ∧ ¬ is_Restore c2)"
| "unwinding _ = False"

fun pop_ready :: "('a, var_id, exp) gcom ⇒ bool" where
  "pop_ready (Restore i) = True"
| "pop_ready (Seq u (Restore i)) = unwinding u"
| "pop_ready _ = False"
```

The annotation of the `Restore` a returning activation pops: the call it came from, which is also the annotation of the `SKIP` the pop leaves behind.

```
fun pop_ann :: "('a, var_id, exp) gcom ⇒ 'a option" where
  "pop_ann (Restore i) = Some i"
| "pop_ann (Seq u (Restore i)) = Some i"
| "pop_ann _ = None"

lemma unwinding_not_SKIP: "unwinding u ⟹ ¬ is_SKIP u"
  by (cases u) auto

lemma pop_ready_not_SKIP: "pop_ready w ⟹ ¬ is_SKIP w"
  by (cases w rule: pop_ready.cases) auto

lemma pop_ready_not_Unwind: "pop_ready w ⟹ w ≠ (Unwind i)"
  by (cases w rule: pop_ready.cases) auto
```

Compile-into-g evidence for an active fragment

`compiled_at Π g p body_p k n` is an activation's certificate. It says two things at once, because every activation needs both: `body_p` is genuinely procedure `p`'s body, and that body, compiled at the exact offset `n` with continuation `k`, is embedded in the target graph `g` — its intra edges lie in `intra g` and its call edges in `calls g`.

The offset matters because compilation is shift-parametric: neither `n` nor `k` is fixed by the syntax, so graph membership *at the chosen offset* is the missing fact and this predicate supplies it, at the same `k` and `n` that `control_at` uses. The procedure identity matters because `control_at`'s fragment parameter is otherwise free, and fall-through preservation is false for a fragment that is not a whole body.

A procedure body's continuation is its epilogue node, which is why the fall-through return edge is stated from `k`. `compile_proc` emits that edge for every body.

The offset itself is anchored to the graph: the body enters at the target of `p`'s `EA_Body` edge, and `k` is the epilogue right after the body. Without the anchor the body's edges could be matched at a shifted offset, and the relation would locate a residual at a node no execution of `p` uses.

```
definition compiled_at ::
  "('a, var_id, exp) gptable ⇒ (var_id, exp) gcfg ⇒ pname ⇒ ('a, var_id, exp) gcom ⇒ cfg_node ⇒ nat ⇒ bool"
    where
  "compiled_at Π g p body_p k n ⟷
     (∃decl n' en E K. Π p = Some decl ∧ body_p = body decl
        ∧ compile Π p body_p k n = (n', en, E, K)
        ∧ E ⊆ intra g ∧ K ⊆ calls g
        ∧ (k, EA_Ret None p, FunctionResult p) ∈ intra g
        ∧ (FunctionEntry p, EA_Body p, Statement n) ∈ intra g
        ∧ k = Statement (n + csize body_p))"

lemma compiled_atI [intro]:
  "Π p = Some decl ⟹ body_p = body decl ⟹
   compile Π p body_p k n = (n', en, E, K) ⟹ E ⊆ intra g ⟹ K ⊆ calls g ⟹
   (k, EA_Ret None p, FunctionResult p) ∈ intra g ⟹
   (FunctionEntry p, EA_Body p, Statement n) ∈ intra g ⟹ k = Statement (n + csize body_p) ⟹
   compiled_at Π g p body_p k n"
  unfolding compiled_at_def by blast

lemma compiled_atE [elim]:
  assumes "compiled_at Π g p body_p k n"
  obtains decl n' en E K where
    "Π p = Some decl" "body_p = body decl"
    "compile Π p body_p k n = (n', en, E, K)" "E ⊆ intra g" "K ⊆ calls g"
    "(k, EA_Ret None p, FunctionResult p) ∈ intra g"
  using assms unfolding compiled_at_def by blast
```

The procedure-identity half on its own, for the many proofs that only need the declaration behind an activation.

```
lemma compiled_at_decl:
  assumes "compiled_at Π g p body_p k n"
  obtains decl where "Π p = Some decl" "body_p = body decl"
  using assms unfolding compiled_at_def by blast

lemma compiled_at_exit:
  "compiled_at Π g p body_p k n ⟹ (k, EA_Ret None p, FunctionResult p) ∈ intra g"
  unfolding compiled_at_def by auto
```

The simulation relation

What it means for a source configuration and a graph configuration to be at the same place. Three shapes, one per stack situation: a single activation, a call in progress with the caller recorded on both stacks, and the moment between a return and the frame pop. All three carry the same store, so the relation constrains position and stack only.

The index `p` in `Π, g, p ⊢ cfg ≈ dg` is the procedure of the outermost activation of `cfg`, the one with no caller below it in the relation. `Nested` and `Returning` record the call that suspended that activation, `Call i dst q args`: the activation inside `Seq inner (Restore i)` runs `q`, and a returning activation is at `FunctionResult q` of that same `q`. Recursion is harmless because the index belongs to an activation, not to a body.

```
inductive csim :: "('a, var_id, exp) gptable ⇒ (var_id, exp) gcfg ⇒ pname
                   ⇒ ('a, var_id, exp) gcom × store × frame list
                   ⇒ cfg_node × store × cframe list ⇒ bool"
    ("(_,_,_ ⊢/ _ ≈/ _)" [51, 51, 51, 51, 51] 50) for Π g where
  Base:
    "control_at Π p body_p k n c v ⟹ compiled_at Π g p body_p k n ⟹
     Π, g, p ⊢ (c, s, []) ≈ (v, s, [])"
| Nested:
    "Π, g, q ⊢ (inner, s, frs) ≈ (v, s, stk) ⟹
     control_at Π pc body_pc kc nc (seq_after (SKIP i) afters) cont ⟹
     control_at Π pc body_pc kc nc (seq_after (Call i dst q args) afters) site ⟹
     compiled_at Π g pc body_pc kc nc ⟹
     Π, g, pc ⊢ (seq_after (Seq inner (Restore i)) afters, s, frs @ [Frame caller dst])
       ≈ (v, s, stk @ [(cont, dst, caller)])"
| Returning:
    "pop_ready w ⟹ pop_ann w = Some i ⟹
     control_at Π pc body_pc kc nc (seq_after (SKIP i) afters) cont ⟹
     control_at Π pc body_pc kc nc (seq_after (Call i dst q args) afters) site ⟹
     compiled_at Π g pc body_pc kc nc ⟹
     Π, g, pc ⊢ (seq_after w afters, callee, [Frame caller dst])
       ≈ (FunctionResult q, callee, [(cont, dst, caller)])"
```

On its own `csim` is a *structural* correspondence, not a reachability one. For an unannotated program it is not functional either: a residual carries no record of which occurrence it came from, so two textually equal statements share every location. A caller that needs the semantic node pairs it with membership in `node_collect` at the same node. Annotated programs whose annotations are the compiler's own locations are the case where the node is determined.

Inversion at an empty source frame stack. Only `Base` can produce one — `Nested` and `Returning` both append a caller frame — so the CFG stack is empty too, the stores agree, and the whole activation certificate comes back. `csim`'s introduction rules stay undeclared: choosing between `Base`, `Nested` and `Returning` is the step each preservation proof exists to make, and it belongs in the proof text rather than in a search.

```
inductive_cases csim_empty_stackE [elim]: "Π, g, p ⊢ (c, s, []) ≈ (v, t, stk)"
```

`Nested` and `Returning` both locate the *resumed caller* — the ordinary residual `seq_after (SKIP i) afters` that the return will land in — at the continuation node `cont`. `Returning` carries exactly the one frame the return pops; outer callers are consumed by the `Nested` layers wrapping it, and after the pop the caller resumes as a `Base` activation with that wrapping untouched.

```
lemma csim_store_eq:
  "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk) ⟹ s = t"
  by (induction "(c, s, frs)" "(v, t, stk)" arbitrary: c frs v stk rule: csim.induct) auto
```

Returning commands and the frame-pop phase

`is_returning c` holds when the leftmost-innermost of `c` is `Restore` or `Unwind`: the shape of every returning-phase command, ordinary or `Nested`-wrapped.

```
fun is_returning :: "('a, var_id, exp) gcom ⇒ bool" where
```

```
  "is_returning (Restore i) = True"
| "is_returning (Unwind i) = True"
| "is_returning (Seq c1 c2) = is_returning c1"
| "is_returning _ = False"

lemma is_returning_seq_after [simp]:
  "is_returning (seq_after c afters) = is_returning c"
  by (induction afters arbitrary: c) auto
```

The call-redex source shape

`head_call c` holds when the leftmost-innermost of `c` is a `Call` to a procedure `special_table` does not classify: the shape of an active residual whose next `pstep` pushes a frame. A call `special_table` classifies resolves in place instead (`intra_step`'s `ISpecial`), so it is deliberately excluded here, mirroring `intra_step`'s own exclusion of `Call`.

```
fun head_call :: "('a, var_id, exp) gcom ⇒ bool" where
  "head_call (Call i dst q actuals) = (special_table q = None)"
| "head_call (Seq c1 c2) = head_call c1"
| "head_call _ = False"

lemma head_call_seq_after [simp]:
  "head_call (seq_after c afters) = head_call c"
  by (induction afters arbitrary: c) auto

lemma head_call_not_SKIP: "head_call c ⟹ ¬ is_SKIP c"
  by (cases c) auto

lemma head_call_not_Unwind: "head_call c ⟹ c ≠ (Unwind i)"
  by (cases c) auto

lemma head_call_seq_after_form:
  "head_call c ⟹ ∃i dst q actuals afters.
     c = seq_after (Call i dst q actuals) afters ∧ special_table q = None"
proof (induction c)
  case (Seq c1 c2)
  from Seq.prems have "head_call c1" by simp
  from Seq.IH(1)[OF this] obtain i dst q actuals afters where
    "c1 = seq_after (Call i dst q actuals) afters" "special_table q = None" by blast
  then have "Seq c1 c2 = seq_after (Call i dst q actuals) (afters @ [c2])"
            "special_table q = None"
    by (simp_all add: seq_after_snoc)
  then show ?case by blast
qed auto

lemma unwinding_not_head_call: "unwinding u ⟹ ¬ head_call u"
  by (induction u rule: unwinding.induct) auto

lemma pop_ready_not_head_call: "pop_ready w ⟹ ¬ head_call w"
  by (cases w rule: pop_ready.cases) (auto simp: unwinding_not_head_call)
```

The return-initiation source shape

`head_return c` holds when the leftmost-innermost of `c` is a source `Return`: the shape whose next `pstep` initiates a return.

```
fun head_return :: "('a, var_id, exp) gcom ⇒ bool" where
  "head_return (Return i e) = True"
| "head_return (Seq c1 c2) = head_return c1"
| "head_return _ = False"

lemma head_return_seq_after [simp]:
  "head_return (seq_after c afters) = head_return c"
  by (induction afters arbitrary: c) auto

lemma head_return_not_SKIP: "head_return c ⟹ ¬ is_SKIP c"
  by (cases c) auto

lemma head_return_seq_after_form:
  "head_return c ⟹ ∃i e afters. c = seq_after (Return i e) afters"
proof (induction c)
  case (Seq c1 c2)
  from Seq.prems have "head_return c1" by simp
  from Seq.IH(1)[OF this] obtain i e afters where "c1 = seq_after (Return i e) afters" by blast
  then have "Seq c1 c2 = seq_after (Return i e) (afters @ [c2])" by (simp add: seq_after_snoc)
```

```
  then show ?case by blast
qed auto

lemma unwinding_not_head_return: "unwinding u ⟹ ¬ head_return u"
  by (induction u rule: unwinding.induct) auto

lemma pop_ready_not_head_return: "pop_ready w ⟹ ¬ head_return w"
  by (cases w rule: pop_ready.cases) (auto simp: unwinding_not_head_return)
```

Source well-formedness: the base-activation return guard

`ret_guarded allow_return c` is the deep source-only well-formedness check: an explicit `Return` (or an in-flight `Unwind`) is admissible only where `allow_return` holds — inside a called activation. Permission is *granted by the activation tail*: the right child `Restore` of a `Seq` marks a callee body, so its left child is checked with `allow_return = True`, whereas ordinary continuations inherit the ambient permission. A bare `Restore` (the transient tail after the callee body reduces to `SKIP`) is always admissible. Crucially this looks *behind* `Restore` boundaries: a `Return` hidden in a base-activation continuation is rejected, which a head-only check misses — exactly the configuration a frame pop would later surface at the base.

```
fun ret_guarded :: "bool ⇒ ('a, var_id, exp) gcom ⇒ bool" where
  "ret_guarded allow_return (SKIP i) = True"
| "ret_guarded allow_return (Assign i x a) = True"
| "ret_guarded allow_return (VIMP_Proc.gcom.Check i l b) = True"
| "ret_guarded allow_return (Seq c1 c2) =
     (if is_Restore c2 then ret_guarded True c1
      else ret_guarded allow_return c1 ∧ ret_guarded allow_return c2)"
| "ret_guarded allow_return (If i b c1 c2) =
     (ret_guarded allow_return c1 ∧ ret_guarded allow_return c2)"
| "ret_guarded allow_return (While i b c) = ret_guarded allow_return c"
| "ret_guarded allow_return (Call i dst p actuals) = True"
| "ret_guarded allow_return (Return i e) = allow_return"
| "ret_guarded allow_return (Restore i) = True"
| "ret_guarded allow_return (Unwind i) = allow_return"

lemma ret_guarded_True_source: "source_com c ⟹ ret_guarded True c"
  by (induction c) (auto split: if_splits)

lemma ret_guarded_False_source_not_head_return:
  "source_com c ⟹ ret_guarded False c ⟹ ¬ head_return c"
  by (induction c) (auto split: if_splits)


lemma ret_guarded_pstep:
  assumes bodies: "⋀p decl. Π p = Some decl ⟹ source_com (body decl)"
      and step: "κ,Π ⊢ (c, s, frs) →p (c', s', frs')"
      and "ret_guarded allow_return c"
  shows "ret_guarded allow_return c'"
  using step assms(3)
proof (induction "(c, s, frs)" "(c', s', frs')"
       arbitrary: c s frs c' s' frs' allow_return rule: pstep.induct)
  case Call
  then show ?case using bodies ret_guarded_True_source by auto
next
  case (Seq2 c1 s1 f1 c1' s1' f1' c2)
  then show ?case by (auto split: if_splits)
qed (auto split: if_splits)
```

The runtime invariant `csim_step` asks for: the active command may not return until a call has opened an activation for it. Only the command matters — neither the store nor the frame stack — and unlike a head-only guard this survives a `pstep`, because it already forbids the base-activation returns that a frame pop would expose.

```
definition return_safe :: "('a, var_id, exp) gcom ⇒ bool" where
  "return_safe c ⟷ ret_guarded False c"

lemma return_safe_if_no_return:
  assumes "source_com c" and "no_return c"
  shows "return_safe c"
  using assms
  unfolding return_safe_def
  by (induction c) (auto split: if_splits)

lemma return_safe_not_head_return:
  "return_safe c ⟹ source_com c ⟹ ¬ head_return c"
  by (auto simp: return_safe_def ret_guarded_False_source_not_head_return)
```

```
lemma return_safe_pstep:
  assumes bodies: "⋀p decl. Π p = Some decl ⟹ source_com (body decl)"
      and step: "κ,Π ⊢ (c, s, frs) →p (c', s', frs')"
      and wf: "return_safe c"
  shows "return_safe c'"
  using ret_guarded_pstep[OF bodies step] wf by (simp add: return_safe_def)


lemma unwinding_seq_after_Unwind:
  "(∀a ∈ set afters. ¬ is_Restore a) ⟹ unwinding (seq_after (Unwind i) afters)"
  by (induction afters rule: rev_induct) (auto simp: seq_after_snoc)
```

Source-step classification

Sorting a source step into the three cases the simulation treats separately: an ordinary intra step, a call, or a return in progress. The classification is what lets each preservation proof assume it is in exactly one case rather than re-deriving the exclusions from `pstep` every time.

```
lemma intra_step_not_returning:
  "intra_step κ Π (c, s, frs) (c', s', frs') ⟹ ¬ is_returning c"
  by (induction "(c, s, frs)" "(c', s', frs')" arbitrary: c s frs c' s' frs'
      rule: intra_step.induct) auto
```

A declared procedure's name is never one `special_table` also classifies: the same disjointness `procs_embedded` bundles in the next theory, supplied directly since this lemma sits upstream of that definition. Without it the `pstep.Call` case cannot rule out `c` also matching `pstep.Special`, which `head_call` excludes.

```
lemma pstep_intra_classify:
  assumes disj: "⋀p decl. Π p = Some decl ⟹ special_table p = None"
  shows "κ,Π ⊢ (c, s, frs) →p (c', s', frs') ⟹ ¬ head_call c ⟹ ¬ head_return c ⟹
   ¬ is_returning c ⟹ intra_step κ Π (c, s, frs) (c', s', frs')"
proof (induction "(c, s, frs)" "(c', s', frs')"
       arbitrary: c s frs c' s' frs' rule: pstep.induct)
  case (Seq2 c1 s1 f1 c1' s1' f1' c2)
  from Seq2.prems have "¬ head_call c1" "¬ head_return c1" "¬ is_returning c1" by auto
  from Seq2.hyps(2)[OF this] have ih: "intra_step κ Π (c1, s1, f1) (c1', s1', f1')" .
  from intra_step_frame_eq[OF ih] ih show ?case by auto
next
  case (Call i p decl actuals dst vals callee s1 frs0)
  then show ?case using disj by simp
qed auto


lemma control_at_not_returning:
  "control_at Π p c0 k n r v ⟹ ¬ is_returning r"
  by (induction rule: control_at.induct) auto

lemma control_at_not_unwind:
  "control_at Π p c0 k n r v ⟹ r ≠ (Unwind i)"
  using control_at_not_returning by fastforce
```

`is_returning` classifies the active head. The `unwinding` and `pop_ready` predicates refine that class with the nested residual shape needed by frame-pop proofs; they deliberately overlap instead of forming a phase partition.

```
lemma unwinding_is_returning: "unwinding u ⟹ is_returning u"
  by (induction u rule: unwinding.induct) auto

lemma pop_ready_is_returning: "pop_ready w ⟹ is_returning w"
  by (cases w rule: pop_ready.cases) (auto simp: unwinding_is_returning)

lemma is_returning_next_check: "is_returning c ⟹ next_check c = None"
  by (induction c) auto
```

A source configuration about to run a check is, under `csim`, at a node whose `EA_Check` edge carries that check. Suspended callers only wrap the running residual, and the frame-pop phase runs no check.

```
lemma csim_next_check_edge:
  assumes "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk)" and "next_check c = Some (l, e)"
  shows "∃w. (v, EA_Check l e, w) ∈ intra g"
  using assms
proof (induction "(c, s, frs)" "(v, t, stk)" arbitrary: c s frs v t stk rule: csim.induct)
  case (Base p body_p k n c v s)
  from Base.hyps(2) obtain n' en E K
    where "compile Π p body_p k n = (n', en, E, K)" and "E ⊆ intra g"
```

```
    by blast
  with control_at_next_check_edge [OF Base.hyps(1) Base.prems] show ?case by blast
next
  case (Returning w i pc body_pc kc nc afters cont callee caller dst p)
  then show ?case by (simp add: is_returning_next_check pop_ready_is_returning)
qed simp
```

Stepping the frame-pop phase

How the phase between a return and its frame pop behaves: it neither changes the store nor touches the frames, and it stays in the phase until the pop. That is what lets the simulation hold the graph side still while the source side unwinds.

```
lemma pstep_unwinding:
  assumes "unwinding u" and "κ,Π ⊢ (u, s, frs) →p (u', s', frs')"
  shows "s' = s ∧ frs' = frs ∧ unwinding u'"
  using assms
proof (induction u arbitrary: u' s' frs')
  case (Seq u1 c2)
  from Seq.prems(1) have u1w: "unwinding u1" and c2R: "¬ is_Restore c2" by auto
  from Seq.prems(2) consider
      (dead) j where "u' = Unwind j" "s' = s" "frs' = frs"
    | (step) c1' where "u' = Seq c1' c2" "κ,Π ⊢ (u1, s, frs) →p (c1', s', frs')"
    using c2R unwinding_not_SKIP[OF u1w] by auto
  then show ?case
  proof cases
    case dead then show ?thesis by simp
  next
    case (step c1')
    from Seq.IH(1)[OF u1w step(2)] have "s' = s" "frs' = frs" "unwinding c1'" by simp_all
    then show ?thesis using step(1) c2R by simp
  qed
next
  case SKIP    from SKIP.prems(1)    show ?case by simp
next
  case Assign  from Assign.prems(1)  show ?case by simp
next
  case Check   from Check.prems(1)   show ?case by simp
next
  case If      from If.prems(1)      show ?case by simp
next
  case While   from While.prems(1)   show ?case by simp
next
  case Call    from Call.prems(1)    show ?case by simp
next
  case Return  from Return.prems(1)  show ?case by simp
next
  case Restore from Restore.prems(1) show ?case by simp
next
  case (Unwind i)
  from Unwind.prems(2) show ?case using pstep_Unwind_stuck by blast
qed

lemma pstep_pop_ready_head:
  assumes "pop_ready w" and "pop_ann w = Some i"
      and "κ,Π ⊢ (w, s, Frame fr dst # frs) →p x"
  shows "x = (SKIP i, combine_assign dst (kind_conv κ dst (s ReturnVar)) (combine_env is_global fr s), frs)
       ∨ (∃w'. x = (w', s, Frame fr dst # frs) ∧ pop_ready w' ∧ pop_ann w' = Some i)"
  using assms
proof (cases w rule: pop_ready.cases)
  case 1
  with assms(2,3) show ?thesis by auto
next
  case (2 u)
  hence uw: "unwinding u" using assms(1) by simp
  from assms(3) show ?thesis
    unfolding 2 using assms(2) unfolding 2
    by (elim SeqSE) (auto dest: pstep_unwinding[OF uw] simp: unwinding_not_SKIP[OF uw])
qed (use assms(1) in auto)
```

Stepping through a `seq_after` spine

A step of a `seq_after` spine is a step of its head, with the pending continuations carried along untouched. `SKIP` and `Unwind` heads are excluded because those relocate into the spine instead of stepping in place.

```
lemma pstep_seq_after_headE [elim]:
```

```
  assumes step: "κ,Π ⊢ (seq_after w afters, s, frs) →p src'"
      and wsk: "¬ is_SKIP w" and wunw: "⋀i. w ≠ Unwind i"
  obtains h' s' fz where
    "src' = (seq_after h' afters, s', fz)"
    "κ,Π ⊢ (w, s, frs) →p (h', s', fz)"
proof -
  have "∃h' s' fz. src' = (seq_after h' afters, s', fz)
         ∧ κ,Π ⊢ (w, s, frs) →p (h', s', fz)"
    using step
  proof (induction afters arbitrary: src' rule: rev_induct)
    case Nil
    then obtain h' s' fz where "src' = (h', s', fz)"
      "κ,Π ⊢ (w, s, frs) →p (h', s', fz)"
      by (cases src') auto
    then show ?case by auto
  next
    case (snoc a xs)
    from snoc.prems have "κ,Π ⊢ (Seq (seq_after w xs) a, s, frs) →p src'"
      by (simp add: seq_after_snoc)
    then obtain B' s' fz where
      A: "src' = (Seq B' a, s', fz)"
        and B: "κ,Π ⊢ (seq_after w xs, s, frs) →p (B', s', fz)"
      using wsk wunw by auto
    from snoc.IH[OF B] obtain h' where
      "B' = seq_after h' xs" "κ,Π ⊢ (w, s, frs) →p (h', s', fz)" by auto
    then show ?case using A by (auto simp: seq_after_snoc)
  qed
  then show ?thesis using that by blast
qed


end
```

5.7 Simulation_Preservation

```
theory Simulation_Preservation
  imports Simulation_Relation Compile_Invariants
begin
```

Every source step is matched by the graph

The result the session exists for: if `csim` holds and the source takes one step, the graph can follow and `csim` holds again (`csim_step`); hence the same for a whole run (`csim_star`). A source redex is one of four things — a call, a return initiation, a return already in progress, or an ordinary step inside the activation — and there is one completion theorem per case, which `csim_step` dispatches to.

`procs_embedded` is the static side condition every case but the returning phase needs: every declared procedure's body has been compiled into this graph, with its entry and exit wiring present. It is deliberately not part of `csim`, which says only how two configurations correspond, so that the returning phase can proceed without it. `procs_embedded_compile_prog` discharges it for any graph `compile_prog` actually produced.

```
definition procs_embedded :: "('a, var_id, exp) gptable ⇒ (var_id, exp) gcfg ⇒ bool" where
  "procs_embedded Π g ⟷
     (∀p decl. Π p = Some decl ⟶
        (∃k n n' en E K.
           compile Π p (body decl) k n = (n', en, E, K)
         ∧ E ⊆ intra g ∧ K ⊆ calls g
         ∧ (FunctionEntry p, EA_Body p, en) ∈ intra g
         ∧ (k, EA_Ret None p, FunctionResult p) ∈ intra g
         ∧ k = Statement (n + csize (body decl))
         ∧ source_com (body decl) ∧ special_table p = None))"

lemma procs_embedded_proc:
  assumes "procs_embedded Π g" and "Π p = Some decl"
  obtains k n n' en E K where
    "compile Π p (body decl) k n = (n', en, E, K)"
    "E ⊆ intra g" "K ⊆ calls g"
    "(FunctionEntry p, EA_Body p, en) ∈ intra g"
    "(k, EA_Ret None p, FunctionResult p) ∈ intra g"
    "k = Statement (n + csize (body decl))"
    "source_com (body decl)" "special_table p = None"
  using assms unfolding procs_embedded_def by blast

lemma procs_embedded_source_com:
  assumes "procs_embedded Π g" and "Π p = Some decl"
  shows "source_com (body decl)"
  using assms by (blast elim: procs_embedded_proc)
```

A declared procedure's name is never one `special_table` also classifies: `procs_embedded` bundles this disjointness alongside compiler-correctness so a declared call and a special call are always distinguishable without a separate well-formedness premise.

```
lemma procs_embedded_special_table_none:
  assumes "procs_embedded Π g" and "Π p = Some decl"
  shows "special_table p = None"
  using assms by (blast elim: procs_embedded_proc)
```

Everything needed to start a fresh activation of `p`, in one step: the certificate its `csim` layer will carry, the initial location of its body, and the `EA_Body` edge from `FunctionEntry p` that reaches that location. Call preservation is otherwise the same destructuring written out by hand.

```
lemma procs_embedded_activation:
  assumes pc: "procs_embedded Π g" and decl: "Π p = Some decl"
  obtains k n en where
    "compiled_at Π g p (body decl) k n"
    "control_at Π p (body decl) k n (body decl) en"
    "(FunctionEntry p, EA_Body p, en) ∈ intra g"
    "source_com (body decl)"
proof -
  obtain k n n' en E K where
    cb: "compile Π p (body decl) k n = (n', en, E, K)"
      and Esub: "E ⊆ intra g" and Ksub: "K ⊆ calls g"
      and entry: "(FunctionEntry p, EA_Body p, en) ∈ intra g"
      and ex: "(k, EA_Ret None p, FunctionResult p) ∈ intra g"
      and kk: "k = Statement (n + csize (body decl))"
      and src: "source_com (body decl)"
      and sp: "special_table p = None"
    by (rule procs_embedded_proc[OF pc decl])
```

```
  have entry': "(FunctionEntry p, EA_Body p, Statement n) ∈ intra g"
    using entry compile_entry[OF cb] by simp
  have cacc: "compiled_at Π g p (body decl) k n"
    by (rule compiled_atI[OF decl refl cb Esub Ksub ex entry' kk])
  have ctrl: "control_at Π p (body decl) k n (body decl) en"
    using control_at_initial[OF src, of Π p k n, folded compile_entry[OF cb]] .
  from cacc ctrl entry src show ?thesis by (rule that)
qed
```

Frame-stack plumbing

A single `pstep` touches only the head region of the frame stack, so a bottom segment `extra` rides along unchanged when the active part `frs` is non-empty. This bridges the `Nested` snoc (the inner step threads all frames) to the induction hypothesis (stated on the inner frames alone).

```
lemma pstep_frame_restrict:
  "κ,Π ⊢ (c, s, fr) →p (c', s', frs') ⟹ fr = frs @ extra ⟹ frs ≠ [] ⟹
   ∃frs''. frs' = frs'' @ extra ∧ κ,Π ⊢ (c, s, frs) →p (c', s', frs'')"
proof (induction "(c, s, fr)" "(c', s', frs')"
       arbitrary: c s fr frs c' s' frs' rule: pstep.induct)
  case (Seq2 c1 s1 f1 c1' s1' f1' c2 frs)
  from Seq2.hyps(2)[OF Seq2.prems] obtain frs'' where
    ih: "f1' = frs'' @ extra" "κ,Π ⊢ (c1, s1, frs) →p (c1', s1', frs'')" by blast
  show ?case
    by (rule exI[of _ frs'']) (use ih in auto)
next
  case (Call i p decl actuals dst vals callee s1 frs0 frs)
  then show ?case by auto
next
  case (RestoreStep i s1 fr0 dst frs0 frs)
  from RestoreStep.prems obtain frs1 where "frs = Frame fr0 dst # frs1" "frs0 = frs1 @ extra"
    by (auto simp: Cons_eq_append_conv)
  then show ?case by auto
next
  case (UnwindAct i j s1 fr0 dst frs0 frs)
  from UnwindAct.prems obtain frs1 where "frs = Frame fr0 dst # frs1" "frs0 = frs1 @ extra"
    by (auto simp: Cons_eq_append_conv)
  then show ?case by auto
qed auto
```

Unlike `pstep_frame_restrict` this needs no non-empty active stack: a call-headed command never heads with `SKIP` / `Restore` / `Unwind`, so no pop can fire.

```
lemma pstep_call_frame_restrict:
  "κ,Π ⊢ (c, s, fr) →p (c', s', frs') ⟹ head_call c ⟹ fr = frs @ extra ⟹
   ∃frs''. frs' = frs'' @ extra ∧ κ,Π ⊢ (c, s, frs) →p (c', s', frs'')"
proof (induction "(c, s, fr)" "(c', s', frs')"
       arbitrary: c s fr frs c' s' frs' rule: pstep.induct)
  case (Seq2 c1 s1 f1 c1' s1' f1' c2 frs)
  from Seq2.prems(1) have "head_call c1" by simp
  from Seq2.hyps(2)[OF this Seq2.prems(2)] obtain frs'' where
    ih: "f1' = frs'' @ extra" "κ,Π ⊢ (c1, s1, frs) →p (c1', s1', frs'')" by blast
  show ?case by (rule exI[of _ frs'']) (use ih in auto)
next
  case (Call i p decl actuals dst vals callee s1 frs0 frs)
  then show ?case by auto
qed auto
```

The CFG dual: a `cstep` also touches only the head of the stack, so an extra bottom segment rides along — with no non-emptiness needed, since the return step already requires a non-empty stack.

```
lemma cstep_frame_extend:
  "κ,g ⊢ (u, s, stk) →c (u', s', stk') ⟹
   κ,g ⊢ (u, s, stk @ E) →c (u', s', stk' @ E)"
  by (erule cstep.cases) auto

lemma cstep_star_frame_extend:
  assumes "κ,g ⊢ c →c* c'"
  shows
    "κ,g ⊢ (fst c, fst (snd c), snd (snd c) @ E) →c* (fst c', fst (snd c'), snd (snd c') @ E)"
  using assms
proof (induction rule: star.induct)
  case (refl a) show ?case by simp
next
  case (step a b c)
  obtain ua sa stka where a: "a = (ua, sa, stka)" by (cases a)
```

```
  obtain ub sb stkb where b: "b = (ub, sb, stkb)" by (cases b)
  from step.hyps(1) a b have "κ,g ⊢ (ua, sa, stka) →c (ub, sb, stkb)" by simp
  hence "κ,g ⊢ (ua, sa, stka @ E) →c (ub, sb, stkb @ E)" by (rule cstep_frame_extend)
  with step.IH a b show ?case by (auto intro: star.step)
qed
```

The `Nested` step-inversion rule: stepping a wrapped activation `Seq inner (Restore i)` is stepping `inner`, provided `inner` has neither completed nor begun to unwind (either of which would relocate rather than step).

```
lemma pstep_seq_after_seq_restore:
  assumes step: "κ,Π ⊢ (seq_after (Seq inner (Restore i)) afters, s, frs) →p src'"
      and nsk: "¬ is_SKIP inner" and nunw: "⋀j. inner ≠ Unwind j"
  obtains inner' s' fz where
    "src' = (seq_after (Seq inner' (Restore i)) afters, s', fz)"
    "κ,Π ⊢ (inner, s, frs) →p (inner', s', fz)"
proof -
  obtain h' s' fz where
    src': "src' = (seq_after h' afters, s', fz)"
      and hstep: "κ,Π ⊢ (Seq inner (Restore i), s, frs) →p (h', s', fz)"
    by (rule pstep_seq_after_headE[OF step]) auto
  from hstep nsk obtain inner' where
    "h' = Seq inner' (Restore i)" "κ,Π ⊢ (inner, s, frs) →p (inner', s', fz)"
    by (auto simp: nunw)
  with src' show ?thesis using that by blast
qed
```

The `Nested` step shared by every completion theorem: the inner activation's run lifts under the caller frame and its relation re-wraps as `Nested`. `csim` pins the graph store to the source store, so the lifted run ends at the source store.

```
lemma csim_Nested_lift:
  assumes run: "∃cfg'. (κ,g ⊢ (v0, s0, stk0) →c* cfg') ∧ Π, g, q ⊢ (inner', s', fz') ≈ cfg'"
      and loc: "control_at Π pc body_pc kc nc (seq_after (SKIP i) afters) cont"
      and cloc: "control_at Π pc body_pc kc nc (seq_after (Call i dst q args) afters) site"
      and cacc: "compiled_at Π g pc body_pc kc nc"
  shows "∃cfg'. (κ,g ⊢ (v0, s0, stk0 @ [(cont, dst, caller)]) →c* cfg')
             ∧ Π, g, pc ⊢ (seq_after (Seq inner' (Restore i)) afters, s', fz' @ [Frame caller dst]) ≈ cfg'"
proof -
  from run obtain v' t' stk' where
    inner: "κ,g ⊢ (v0, s0, stk0) →c* (v', t', stk')"
      and sim: "Π, g, q ⊢ (inner', s', fz') ≈ (v', t', stk')" by auto
  have teq: "t' = s'" using csim_store_eq[OF sim] by simp
  have "κ,g ⊢ (v0, s0, stk0 @ [(cont, dst, caller)]) →c* (v', s', stk' @ [(cont, dst, caller)])"
    using cstep_star_frame_extend[OF inner, of "[(cont, dst, caller)]"] teq by simp
  moreover have "Π, g, pc ⊢ (seq_after (Seq inner' (Restore i)) afters, s', fz' @ [Frame caller dst]) ≈ (v', s',
stk' @ [(cont, dst, caller)])"
    by (rule csim.Nested[OF sim[unfolded teq] loc cloc cacc])
  ultimately show ?thesis by blast
qed
```

Call preservation

A source `Call` at the head of the active residual is matched by two `cstep`s — the call edge to `FunctionEntry q` and the `EA_Body` edge to the callee body — and adds exactly one `Nested` layer: the callee body becomes a fresh `Base` activation of `q`, and the caller resumes at `seq_after (SKIP i) afters`. Everything the callee needs comes from the single `procs_embedded` certificate.

```
lemma csim_call_base:
  assumes pc: "procs_embedded Π g"
      and loc: "control_at Π p body_p kk n (seq_after (Call i dst q actuals) afters) v"
      and cacc: "compiled_at Π g p body_p kk n"
      and decl: "Π q = Some decl"
  shows "∃cfg'. (κ,g ⊢ (v, s, []) →c* cfg')
             ∧ Π, g, p ⊢ (seq_after (Seq (body decl) (Restore i)) afters,
                          bind_formals (formals decl) (map2 (λx. kind_conv κ (Some x)) (formals decl) (map (λe. eval
e s) actuals))
                            (enter_state is_global s),
                          [Frame s dst]) ≈ cfg'"
proof -
  let ?callee =
    "bind_formals (formals decl) (map2 (λx. kind_conv κ (Some x)) (formals decl) (map (λe. eval e s) actuals)) (enter_state
is_global s)"
  have spNone: "special_table q = None" by (rule procs_embedded_special_table_none[OF pc decl])
  from cacc obtain n' en E K where comp: "compile Π p body_p kk n = (n', en, E, K)"
      and Ksub: "K ⊆ calls g" by blast
```

```
  from control_at_seq_after_call_edge[OF loc refl comp spNone] obtain j w where
    vk: "v = Statement j"
    and edgeK: "(Statement j, CallEdge dst (call_formals Π q) actuals,
                FunctionEntry q, w) ∈ K"
    and callerSKIP: "control_at Π p body_p kk n (seq_after (SKIP i) afters) w" by blast
  have edge: "(Statement j, CallEdge dst (formals decl) actuals, FunctionEntry q, w)
               ∈ calls g" using edgeK Ksub by (auto simp: decl)
  have cstep1:
    "κ,g ⊢ (Statement j, s, []) →c (FunctionEntry q, call_enter κ (CallEdge dst (formals decl) actuals) s, [(w, dst,
s)])"
    by (rule cstep.Call[OF edge])
  have ce: "call_enter κ (CallEdge dst (formals decl) actuals) s = ?callee"
    by (rule call_enter_CallEdge)
  obtain kq m en_q where
    caccq: "compiled_at Π g q (body decl) kq m"
      and ctrlq: "control_at Π q (body decl) kq m (body decl) en_q"
      and entry: "(FunctionEntry q, EA_Body q, en_q) ∈ intra g"
    by (rule procs_embedded_activation[OF pc decl])
  have cstep2: "κ,g ⊢ (FunctionEntry q, ?callee, [(w, dst, s)]) →c (en_q, ?callee, [(w, dst, s)])"
    using cstep_body[OF entry] .
  have star: "κ,g ⊢ (v, s, []) →c* (en_q, ?callee, [(w, dst, s)])"
    using cstep1[unfolded ce] cstep2 vk by (simp add: star.step)
  have baseCallee: "Π, g, q ⊢ (body decl, ?callee, []) ≈ (en_q, ?callee, [])"
    by (rule csim.Base[OF ctrlq caccq])
  have "Π, g, p ⊢ (seq_after (Seq (body decl) (Restore i)) afters, ?callee, [] @ [Frame s dst]) ≈ (en_q, ?callee,
[] @ [(w, dst, s)])"
    by (rule csim.Nested[OF baseCallee callerSKIP loc cacc])
  then have "Π, g, p ⊢ (seq_after (Seq (body decl) (Restore i)) afters, ?callee, [Frame s dst]) ≈ (en_q, ?callee,
[(w, dst, s)])"
    by simp
  with star show ?thesis by blast
qed

theorem csim_call_completion:
  "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk) ⟹ procs_embedded Π g ⟹ head_call c ⟹
   κ,Π ⊢ (c, s, frs) →p src' ⟹
   ∃cfg'. (κ,g ⊢ (v, t, stk) →c* cfg') ∧ Π, g, ϱ ⊢ src' ≈ cfg'"
proof (induction "(c, s, frs)" "(v, t, stk)" arbitrary: c s frs v t stk src' rule: csim.induct)
  case (Base p body_p kk n cc vv ss)
  from head_call_seq_after_form[OF Base.prems(2)] obtain i dst q actuals afters where
    ceq: "cc = seq_after (Call i dst q actuals) afters" and spNone: "special_table q = None"
    by blast
  have step: "κ,Π ⊢ (seq_after (Call i dst q actuals) afters, ss, []) →p src'"
    using Base.prems(3) ceq by simp
  have w1: "¬ is_SKIP (Call i dst q actuals)" by simp
  have w2: "⋀j. Call i dst q actuals ≠ Unwind j" by simp
  obtain h' s' fz where
    src': "src' = (seq_after h' afters, s', fz)"
      and pcall: "κ,Π ⊢ (Call i dst q actuals, ss, []) →p (h', s', fz)"
    by (rule pstep_seq_after_headE[OF step w1 w2])
  from pcall spNone obtain decl where
    qdecl: "Π q = Some decl"
      and heq: "h' = Seq (body decl) (Restore i)"
      and seq: "s' = bind_formals (formals decl) (map2 (λx. kind_conv κ (Some x)) (formals decl) (map (λe. eval e
ss) actuals))
                       (enter_state is_global ss)"
      and fzeq: "fz = [Frame ss dst]"
    by auto
  have loc': "control_at Π p body_p kk n (seq_after (Call i dst q actuals) afters) vv"
    using Base.hyps(1) ceq by simp
  from csim_call_base[OF Base.prems(1) loc' Base.hyps(2) qdecl, where s = ss]
  obtain cfg' where
    cstar: "κ,g ⊢ (vv, ss, []) →c* cfg'"
      and csimr: "Π, g, p ⊢ (seq_after (Seq (body decl) (Restore i)) afters,
                bind_formals (formals decl) (map2 (λx. kind_conv κ (Some x)) (formals decl) (map (λe. eval e ss) actuals))
                  (enter_state is_global ss),
                [Frame ss dst]) ≈ cfg'" by blast
  from csimr have "Π, g, p ⊢ src' ≈ cfg'" by (simp add: src' heq seq fzeq)
  with cstar show ?case by (intro exI conjI)
next
  case (Returning w i pc body_pc kc nc afters cont dst p args site callee caller)
  from Returning.prems(2) have "head_call w" by simp
  with pop_ready_not_head_call[OF Returning.hyps(1)] show ?case by simp
next
  case (Nested q inner s0 frs0 v0 stk0 pc body_pc kc nc i afters cont dst args site caller)
  have headinner: "head_call inner" using Nested.prems(2) by simp
```

```
  have nsk: "¬ is_SKIP inner" using headinner by (rule head_call_not_SKIP)
  have nunw: "⋀j. inner ≠ Unwind j" using headinner by (rule head_call_not_Unwind)
  obtain inner' s' fz where
    src': "src' = (seq_after (Seq inner' (Restore i)) afters, s', fz)"
      and stepin: "κ,Π ⊢ (inner, s0, frs0 @ [Frame caller dst]) →p (inner', s', fz)"
    by (rule pstep_seq_after_seq_restore[OF Nested.prems(3) nsk nunw])
  from pstep_call_frame_restrict[OF stepin headinner refl] obtain fz' where
    fz: "fz = fz' @ [Frame caller dst]"
      and stepin': "κ,Π ⊢ (inner, s0, frs0) →p (inner', s', fz')" by blast
  from csim_Nested_lift[OF Nested.hyps(2)[OF Nested.prems(1) headinner stepin'] Nested.hyps(3,4,5)]
  show ?case by (simp add: src' fz)
qed
```

Intra-procedural preservation and callee fall-through

The bulk of the simulation: a step that stays inside one activation. Either the callee body still has work, in which case the step happens under the enclosing `Restore` and the stack is untouched, or the body is finished and control falls through to that `Restore` – the only place a callee reaches its own exit without a return statement.

```
lemma intra_step_seq_after_seq_restore:
  "intra_step κ Π (seq_after (Seq inner (Restore i)) afters, s, frs) src' ⟹
   (is_SKIP inner ∧ src' = (seq_after (Restore i) afters, s, frs))
   ∨ (∃inner' s'. src' = (seq_after (Seq inner' (Restore i)) afters, s', frs)
        ∧ intra_step κ Π (inner, s, frs) (inner', s', frs))"
proof (induction afters arbitrary: src' rule: rev_induct)
  case Nil
  obtain c' s' frs' where sc: "src' = (c', s', frs')" by (cases src')
  from Nil.prems sc have "intra_step κ Π (Seq inner (Restore i), s, frs) (c', s', frs')" by simp
  from intra_Seq_cases[OF this] show ?case by (auto simp: sc)
next
  case (snoc a xs)
  obtain c' s' frs' where sc: "src' = (c', s', frs')" by (cases src')
  from snoc.prems sc
  have "intra_step κ Π (Seq (seq_after (Seq inner (Restore i)) xs) a, s, frs) (c', s', frs')"
    by (simp add: seq_after_snoc)
  from intra_Seq_cases[OF this] obtain B' where
    A: "c' = Seq B' a" "frs' = frs"
      and B: "intra_step κ Π (seq_after (Seq inner (Restore i)) xs, s, frs) (B', s', frs)"
    by auto
  from snoc.IH[OF B] show ?case
  proof (rule disjE)
    assume "is_SKIP inner ∧ (B', s', frs) = (seq_after (Restore i) xs, s, frs)"
    then show ?case using A sc by (auto simp: seq_after_snoc)
  next
    assume "∃inner' s''. (B', s', frs) = (seq_after (Seq inner' (Restore i)) xs, s'', frs)
              ∧ intra_step κ Π (inner, s, frs) (inner', s'', frs)"
    then show ?case using A sc by (auto simp: seq_after_snoc)
  qed
qed
```

A `Nested` callee that *completes* (`inner = SKIP`) exposes `Restore` through the `ISeq1` fall-through; the CFG runs the completed callee to its exit (`control_at_skip_to_exit`) and takes the `EA_Ret None p` edge into `FunctionResult p` (`compiled_at_exit`), landing in a `Returning` activation.

```
theorem csim_intra_completion:
  "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk) ⟹ procs_embedded Π g ⟹
   intra_step κ Π (c, s, frs) src' ⟹
   ∃cfg'. (κ,g ⊢ (v, t, stk) →c* cfg') ∧ Π, g, ϱ ⊢ src' ≈ cfg'"
proof (induction "(c, s, frs)" "(v, t, stk)" arbitrary: c s frs v t stk src' rule: csim.induct)
  case (Base p body_p kk n cc vv ss)
  obtain c' s' frs' where sc: "src' = (c', s', frs')" by (cases src')
  from Base.prems(2) sc have istep: "intra_step κ Π (cc, ss, []) (c', s', frs')" by simp
  from Base.hyps(2) obtain decl where pdecl: "Π p = Some decl" "body_p = body decl"
    by (rule compiled_at_decl)
  have srcbody: "source_com body_p"
    using procs_embedded_source_com[OF Base.prems(1) ‹Π p = Some decl›] ‹body_p = body decl›
    by simp
  from Base.hyps(2) obtain n' en E K where comp: "compile Π p body_p kk n = (n', en, E, K)"
      and Esub: "E ⊆ intra g" by blast
  from intra_step_simulation[OF Base.hyps(1) istep comp Esub srcbody, where stk = "[]"]
  obtain v' where feq: "frs' = []" and loc': "control_at Π p body_p kk n c' v'"
      and cstar: "κ,g ⊢ (vv, ss, []) →c* (v', s', [])" by blast
  have "Π, g, p ⊢ (c', s', []) ≈ (v', s', [])" by (rule csim.Base[OF loc' Base.hyps(2)])
  with cstar show ?case using sc feq by auto
next
  case (Returning w i pc body_pc kc nc afters cont dst p args site callee caller)
```

```
  from Returning.prems(2) have "¬ is_returning (seq_after w afters)"
    by (cases src') (auto dest: intra_step_not_returning)
  with Returning.hyps(1) show ?case by (simp add: pop_ready_is_returning)
next
  case (Nested q inner s0 frs0 v0 stk0 pc body_pc kc nc i afters cont dst args site caller)
  from Nested.prems(2)
  have "intra_step κ Π (seq_after (Seq inner (Restore i)) afters, s0, frs0 @ [Frame caller dst]) src'"
    .
  from intra_step_seq_after_seq_restore[OF this] show ?case
  proof (rule disjE)
    assume fall:
      "is_SKIP inner ∧ src' = (seq_after (Restore i) afters, s0, frs0 @ [Frame caller dst])"
    then have innerSKIP: "is_SKIP inner"
      and src': "src' = (seq_after (Restore i) afters, s0, frs0 @ [Frame caller dst])" by auto
    from innerSKIP obtain j where ij: "inner = SKIP j" by auto
    have baseInner: "Π, g, q ⊢ (SKIP j, s0, frs0) ≈ (v0, s0, stk0)"
      using Nested.hyps(1) ij by simp
    from baseInner obtain c0in kin nin where
      ctrl: "control_at Π q c0in kin nin (SKIP j) v0" and cacc: "compiled_at Π g q c0in kin nin"
        and frs0nil: "frs0 = []" and stk0nil: "stk0 = []"
    proof (cases rule: csim.cases)
      case (Base body_p2 k2 n2) with that show ?thesis by auto
    qed simp_all
    from cacc obtain n' en E K where comp: "compile Π q c0in kin nin = (n', en, E, K)"
      and Esub: "E ⊆ intra g"
      and exitedge: "(kin, EA_Ret None q, FunctionResult q) ∈ intra g"
      by blast
    have star1: "κ,g ⊢ (v0, s0, [(cont, dst, caller)]) →c* (kin, s0, [(cont, dst, caller)])"
      by (rule control_at_skip_to_exit[OF ctrl _ comp Esub]) simp
    have "κ,g ⊢ (kin, s0, [(cont, dst, caller)]) →c (FunctionResult q, s0, [(cont, dst, caller)])"
      using cstep.Intra[OF exitedge edge_step_EA_Ret_ret_store_mem] by simp
    with star1
    have star:
      "κ,g ⊢ (v0, s0, [(cont, dst, caller)]) →c* (FunctionResult q, s0, [(cont, dst, caller)])"
      by (rule star_trans[OF _ star_step1])
    have "Π, g, pc ⊢ (seq_after (Restore i) afters, s0, [Frame caller dst]) ≈ (FunctionResult q, s0, [(cont, dst,
caller)])"
      by (rule csim.Returning[OF _ _ Nested.hyps(3) Nested.hyps(4) Nested.hyps(5)]) simp_all
    then have "Π, g, pc ⊢ src' ≈ (FunctionResult q, s0, [(cont, dst, caller)])"
      using src' frs0nil by simp
    with star show ?case using stk0nil by auto
  next
    assume "∃inner' s'.
              src' = (seq_after (Seq inner' (Restore i)) afters, s', frs0 @ [Frame caller dst])
              ∧ intra_step κ Π (inner, s0, frs0 @ [Frame caller dst])
                   (inner', s', frs0 @ [Frame caller dst])"
    then obtain inner' s' where
      src': "src' = (seq_after (Seq inner' (Restore i)) afters, s', frs0 @ [Frame caller dst])"
        and stepin: "intra_step κ Π (inner, s0, frs0 @ [Frame caller dst])
                       (inner', s', frs0 @ [Frame caller dst])"
      by blast
    from intra_step_any_frame[OF stepin]
    have stepin': "intra_step κ Π (inner, s0, frs0) (inner', s', frs0)" .
    from csim_Nested_lift[OF Nested.hyps(2)[OF Nested.prems(1) stepin'] Nested.hyps(3,4,5)]
    show ?case by (simp add: src')
  qed
qed
```

Return initiation

A source `Return` in head position fires `Return e --> Unwind`, turning the innermost callee activation into its returning phase, while the CFG takes the `EA_Ret e p` edge to `FunctionResult p`. The enclosing `Nested` wrapper becomes `Returning`: its body `Seq (seq_after (Unwind j) cafters) (Restore i)` is `pop_ready` because the callee's pending continuations `cafters` — source residuals of a `control_at` location — carry no `Restore`. The `frs ≠ []` premise excludes the ill-formed base `Return` (a stuck top-level return); `csim_step` discharges it from `return_safe`.

```
theorem csim_return_init_completion:
  "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk) ⟹ procs_embedded Π g ⟹ frs ≠ [] ⟹
   head_return c ⟹ κ,Π ⊢ (c, s, frs) →p src' ⟹
   ∃cfg'. (κ,g ⊢ (v, t, stk) →c* cfg') ∧ Π, g, ϱ ⊢ src' ≈ cfg'"
proof (induction "(c, s, frs)" "(v, t, stk)" arbitrary: c s frs v t stk src' rule: csim.induct)
  case (Base p body_p kk n cc vv ss)
  from Base.prems(2) show ?case by simp
next
  case (Returning w i pc body_pc kc nc afters cont dst p args site callee caller)
```

```
  from Returning.prems(3) pop_ready_not_head_return[OF Returning.hyps(1)]
  show ?case by simp
next
  case (Nested q inner s0 frs0 v0 stk0 pc body_pc kc nc i afters cont dst args site caller)
  have hr_inner: "head_return inner" using Nested.prems(3) by simp
  have nsk: "¬ is_SKIP inner" using hr_inner by (rule head_return_not_SKIP)
  have nunw: "inner ≠ Unwind j" for j using hr_inner by auto
  obtain inner' s' fz where
    src': "src' = (seq_after (Seq inner' (Restore i)) afters, s', fz)"
      and stepin: "κ,Π ⊢ (inner, s0, frs0 @ [Frame caller dst]) →p (inner', s', fz)"
    by (rule pstep_seq_after_seq_restore[OF Nested.prems(4) nsk nunw])
  show ?case
  proof (cases "frs0 = []")
    case True
    have baseInner: "Π, g, q ⊢ (inner, s0, []) ≈ (v0, s0, stk0)" using Nested.hyps(1) True by simp
    obtain c0in kin nin where
      stk0nil: "stk0 = []"
        and ctrl: "control_at Π q c0in kin nin inner v0"
        and cacc: "compiled_at Π g q c0in kin nin"
      using baseInner by blast
    from cacc obtain decl where pdecl: "Π q = Some decl" "c0in = body decl"
      by (rule compiled_at_decl)
    have srcc0: "source_com c0in"
      using procs_embedded_source_com[OF Nested.prems(1) ‹Π q = Some decl›]
            ‹c0in = body decl›
      by simp
    obtain ir e cafters where innerform: "inner = seq_after (Return ir e) cafters"
      using head_return_seq_after_form[OF hr_inner] by blast
    — the head step is Return e --> Unwind, leaving the continuations cafters
    have stepin1:
      "κ,Π ⊢ (seq_after (Return ir e) cafters, s0, [Frame caller dst]) →p (inner', s', fz)"
      using stepin True innerform by simp
    obtain h' where inner'form: "inner' = seq_after h' cafters"
       and hstep: "κ,Π ⊢ (Return ir e, s0, [Frame caller dst]) →p (h', s', fz)"
      by (rule pstep_seq_after_headE[OF stepin1]) auto
    have hUnw: "h' = Unwind ir" and s'eq: "s' = ret_store e s0" and fzeq: "fz = [Frame caller dst]"
      using hstep by (cases e; auto simp: ret_store_def)+
    — the callee continuations carry no Restore, so the Unwind spine is pop_ready
    have noRest: "∀a ∈ set cafters. ¬ is_Restore a"
      by (rule control_at_head_return_afters_no_Restore[OF ctrl[unfolded innerform] srcc0])
    have popready: "pop_ready (Seq (seq_after (Unwind ir) cafters) (Restore i))"
      using unwinding_seq_after_Unwind[OF noRest] by simp
    — CFG: one Intra step along the return edge to FunctionResult q
    from cacc obtain n' en E K where comp: "compile Π q c0in kin nin = (n', en, E, K)"
       and Esub: "E ⊆ intra g" by blast
    obtain j where vk: "v0 = Statement j"
       and edge: "(Statement j, EA_Ret e q, FunctionResult q) ∈ E"
      using control_at_seq_after_return_edge[OF ctrl[unfolded innerform] refl comp] by blast
    have edgeg: "(Statement j, EA_Ret e q, FunctionResult q) ∈ intra g"
      using edge Esub by blast
    have cstep1:
      "κ,g ⊢ (Statement j, s0, [(cont, dst, caller)]) →c (FunctionResult q, ret_store e s0, [(cont, dst, caller)])"
      using cstep.Intra[OF edgeg edge_step_EA_Ret_ret_store_mem] .
    — the enclosing wrapper becomes Returning
    have rel: "Π, g, pc ⊢ (seq_after (Seq (seq_after (Unwind ir) cafters) (Restore i)) afters, ret_store e s0,
        [Frame caller dst]) ≈ (FunctionResult q, ret_store e s0, [(cont, dst, caller)])"
      by (rule csim.Returning[OF popready _ Nested.hyps(3) Nested.hyps(4) Nested.hyps(5)]) simp
    have srcshape: "src' = (seq_after (Seq (seq_after (Unwind ir) cafters) (Restore i)) afters,
                            ret_store e s0, [Frame caller dst])"
      using src' inner'form hUnw s'eq fzeq by simp
    have "κ,g ⊢ (v0, s0, stk0 @ [(cont, dst, caller)]) →c* (FunctionResult q, ret_store e s0, [(cont, dst, caller)])"
      using cstep1 vk stk0nil by auto
    with rel srcshape show ?thesis by auto
  next
    case False
    from pstep_frame_restrict[OF stepin refl False] obtain fz' where
      fz: "fz = fz' @ [Frame caller dst]"
        and stepin': "κ,Π ⊢ (inner, s0, frs0) →p (inner', s', fz')" by blast
    from csim_Nested_lift[OF Nested.hyps(2)[OF Nested.prems(1) False hr_inner stepin']
                             Nested.hyps(3,4,5)]
    show ?thesis by (simp add: src' fz)
  qed
qed
```

Return propagation and the frame pop

The half of the simulation where the two sides advance at different rates. A source return walks out of the nested residual one dead layer at a time while the graph stays at the callee result node, and only the final pop takes a graph step – so the correspondence is a star, not a step-for-step match.

```
lemma csim_returning_frames_nonempty:
  "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk) ⟹ is_returning c ⟹ frs ≠ [] ∧ stk ≠ []"
  by (erule csim.cases) (auto dest: control_at_not_returning)

lemma csim_not_unwind:
  "Π, g, ϱ ⊢ ((Unwind i), s, frs) ≈ cfg' ⟹ False"
  by (erule csim.cases) (auto dest: control_at_not_unwind pop_ready_not_Unwind)
```

One `pstep` of a base returning activation `seq_after w afters` (`pop_ready w`, framed by the single caller frame, CFG at `FunctionResult p`) either *pops*: the source resumes at `seq_after (SKIP i) afters` with the combined store and the CFG performs exactly one return `cstep` to `cont`, rebuilt as a `Base` activation; or *propagates*: an `Unwind` advances one dead-code layer and the CFG stays at `FunctionResult p` with *zero* `cstep`, rebuilt as a `Returning` activation.

```
lemma csim_returning_base_completion:
  assumes pr: "pop_ready w" and ann: "pop_ann w = Some i"
      and loc: "control_at Π pc body_pc kc nc (seq_after (SKIP i) afters) cont"
      and cloc: "control_at Π pc body_pc kc nc (seq_after (Call i dst q args) afters) site"
      and cacc: "compiled_at Π g pc body_pc kc nc"
      and step: "κ,Π ⊢ (seq_after w afters, callee, [Frame caller dst]) →p src'"
  shows "∃cfg'. (κ,g ⊢ (FunctionResult q, callee, [(cont, dst, caller)]) →c* cfg')
             ∧ Π, g, pc ⊢ src' ≈ cfg'"
proof -
  have wsk: "¬ is_SKIP w" using pr by (rule pop_ready_not_SKIP)
  have wunw: "w ≠ Unwind j" for j using pr by (rule pop_ready_not_Unwind)
  obtain h' s' frs' where
    src': "src' = (seq_after h' afters, s', frs')"
      and hstep: "κ,Π ⊢ (w, callee, [Frame caller dst]) →p (h', s', frs')"
    by (rule pstep_seq_after_headE[OF step wsk wunw])
  let ?rs = "combine_collect κ dst caller callee"
  from pstep_pop_ready_head[OF pr ann hstep] show ?thesis
  proof (rule disjE)
    assume "(h', s', frs') =
            (SKIP i, combine_assign dst (kind_conv κ dst (callee ReturnVar)) (combine_env is_global caller callee),
[])"
    hence h': "h' = SKIP i" "s' = ?rs" "frs' = []" by (auto simp: combine_collect_def)
    have "κ,g ⊢ (FunctionResult q, callee, [(cont, dst, caller)]) →c (cont, ?rs, [])"
      by (rule cstep.Return)
    moreover have "Π, g, pc ⊢ (seq_after (SKIP i) afters, ?rs, []) ≈ (cont, ?rs, [])"
      by (rule csim.Base[OF loc cacc])
    ultimately show ?thesis using src' h' by auto
  next
    assume "∃w'. (h', s', frs') = (w', callee, [Frame caller dst])
                ∧ pop_ready w' ∧ pop_ann w' = Some i"
    then obtain w' where h': "h' = w'" "s' = callee" "frs' = [Frame caller dst]"
        and pr': "pop_ready w'" and ann': "pop_ann w' = Some i" by auto
    have "Π, g, pc ⊢ (seq_after w' afters, callee, [Frame caller dst]) ≈ (FunctionResult q, callee, [(cont, dst,
caller)])"
      by (rule csim.Returning[OF pr' ann' loc cloc cacc])
    with src' h' show ?thesis by auto
  qed
qed
```

Deep return completion: the base case above, threaded out through any number of `Nested` wrappers. No `procs_embedded` is needed — the returning phase runs on the certificates `csim` already carries.

```
theorem csim_returning_completion:
  "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk) ⟹ is_returning c ⟹
   κ,Π ⊢ (c, s, frs) →p src' ⟹
   ∃cfg'. (κ,g ⊢ (v, t, stk) →c* cfg') ∧ Π, g, ϱ ⊢ src' ≈ cfg'"
proof (induction "(c, s, frs)" "(v, t, stk)" arbitrary: c s frs v t stk src' rule: csim.induct)
  case (Base p body_p kk n cc vv ss)
  from control_at_not_returning[OF Base.hyps(1)] Base.prems(1) show ?case by simp
next
  case (Returning w i pc body_pc kc nc ao cont dst p args site callee caller)
  from csim_returning_base_completion[OF Returning.hyps(1) Returning.hyps(2) Returning.hyps(3)
                                         Returning.hyps(4) Returning.hyps(5) Returning.prems(2)]
  show ?case .
next
  case (Nested q inner s0 frs0 v0 stk0 pc body_pc kc nc i afters cont dst args site caller)
```

```
  have retinner: "is_returning inner" using Nested.prems(1) by simp
  have nsk: "¬ is_SKIP inner" using retinner by auto
  have nunw: "inner ≠ Unwind j" for j
  proof
    assume "inner = Unwind j"
    with Nested.hyps(1) show False by (blast dest: csim_not_unwind)
  qed
  have frsne: "frs0 ≠ []"
    using csim_returning_frames_nonempty[OF Nested.hyps(1) retinner] by simp
  obtain inner' s' fz where
    src': "src' = (seq_after (Seq inner' (Restore i)) afters, s', fz)"
      and stepin: "κ,Π ⊢ (inner, s0, frs0 @ [Frame caller dst]) →p (inner', s', fz)"
    by (rule pstep_seq_after_seq_restore[OF Nested.prems(2) nsk nunw])
  from pstep_frame_restrict[OF stepin refl frsne] obtain fz' where
    fz: "fz = fz' @ [Frame caller dst]"
      and stepin': "κ,Π ⊢ (inner, s0, frs0) →p (inner', s', fz')" by blast
  from csim_Nested_lift[OF Nested.hyps(2)[OF retinner stepin'] Nested.hyps(3,4,5)]
  show ?case by (simp add: src' fz)
qed
```

Single-step and finite-execution forward simulation

Only a `Base` activation has an empty stack, and its located residual is a source command, so `return_safe` forbids it from heading with `Return`. This is where `return_safe` discharges the nonempty-frame premise of `csim_return_init_completion`.

```
lemma csim_head_return_frames:
  assumes SIM: "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk)"
      and PC: "procs_embedded Π g"
      and WF: "return_safe c"
      and RET: "head_return c"
  shows "frs ≠ []"
proof (rule ccontr)
  assume "¬ frs ≠ []"
  hence base: "Π, g, ϱ ⊢ (c, s, []) ≈ (v, t, stk)" using SIM by simp
  obtain p body_p kk n where
    ctrl: "control_at Π p body_p kk n c v" and cacc: "compiled_at Π g p body_p kk n"
    using base by blast
  from cacc obtain decl where pdecl: "Π p = Some decl" "body_p = body decl"
    by (rule compiled_at_decl)
  have "source_com body_p"
    using procs_embedded_source_com[OF PC ‹Π p = Some decl›] ‹body_p = body decl› by simp
  hence "source_com c" using control_at_source_com[OF ctrl] by simp
  from return_safe_not_head_return[OF WF this] RET show False by simp
qed
```

One-step forward simulation: the four-way redex classification (`head_call` / `head_return` / `is_returning` / otherwise-intra) dispatched to the four completion theorems above. `return_safe` is used only to supply the nonempty frame for the return-initiation branch; the statement exposes none of the internal classifiers, compile tuples, offsets, or frame conditions.

```
theorem csim_step:
  assumes SIM: "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk)"
      and PC: "procs_embedded Π g"
      and WF: "return_safe c"
      and STEP: "κ,Π ⊢ (c, s, frs) →p src'"
  shows "∃cfg'. (κ,g ⊢ (v, t, stk) →c* cfg') ∧ Π, g, ϱ ⊢ src' ≈ cfg'"
proof -
  consider (call) "head_call c" | (ret) "head_return c" | (returning) "is_returning c"
    | (intra) "¬ head_call c" "¬ head_return c" "¬ is_returning c" by blast
  then show ?thesis
  proof cases
    case call
    from csim_call_completion[OF SIM PC call STEP] show ?thesis .
  next
    case ret
    from csim_head_return_frames[OF SIM PC WF ret]
    have "frs ≠ []" .
    from csim_return_init_completion[OF SIM PC this ret STEP] show ?thesis .
  next
    case returning
    from csim_returning_completion[OF SIM returning STEP] show ?thesis .
  next
    case intra
    obtain c' s' frs' where sc: "src' = (c', s', frs')" by (cases src')
```

```
    have "intra_step κ Π (c, s, frs) (c', s', frs')"
      by (rule pstep_intra_classify[OF procs_embedded_special_table_none[OF PC]
            STEP[unfolded sc] intra(1) intra(2) intra(3)])
    from csim_intra_completion[OF SIM PC this] show ?thesis by (simp add: sc)
  qed
qed

lemma csim_run:
  assumes PC: "procs_embedded Π g"
      and RUN: "κ,Π ⊢ sc →_p* sc'"
      and SIM: "Π, g, ϱ ⊢ sc ≈ dg"
      and WF: "return_safe (fst sc)"
  shows "∃dg'. (κ,g ⊢ dg →_c* dg') ∧ Π, g, ϱ ⊢ sc' ≈ dg'"
  using RUN SIM WF
proof (induction arbitrary: dg rule: star.induct)
  case (refl a)
  obtain v0 t0 k0 where "dg = (v0, t0, k0)" by (cases dg)
  with refl show ?case by auto
next
  case (step a b cc)
  obtain c0 s0 f0 where a: "a = (c0, s0, f0)" by (cases a)
  obtain c1 s1 f1 where b: "b = (c1, s1, f1)" by (cases b)
  obtain v0 t0 k0 where dgd: "dg = (v0, t0, k0)" by (cases dg)
  have SIMa: "Π, g, ϱ ⊢ (c0, s0, f0) ≈ (v0, t0, k0)" using step.prems(1) a dgd by simp
  have WFa: "return_safe c0" using step.prems(2) a by simp
  have STEP: "κ,Π ⊢ (c0, s0, f0) →_p (c1, s1, f1)" using step.hyps(1) a b by simp
  from csim_step[OF SIMa PC WFa STEP] obtain dg1 where
    both: "(κ,g ⊢ (v0, t0, k0) →_c* dg1) ∧ Π, g, ϱ ⊢ (c1, s1, f1) ≈ dg1" ..
  from both have run1: "κ,g ⊢ (v0, t0, k0) →_c* dg1" by (rule conjunct1)
  from both have sim1: "Π, g, ϱ ⊢ (c1, s1, f1) ≈ dg1" by (rule conjunct2)
  have bodies: "⋀p decl. Π p = Some decl ⟹ source_com (body decl)"
    using procs_embedded_source_com[OF PC] .
  have WFb: "return_safe (fst b)" using return_safe_pstep[OF bodies STEP WFa] b by simp
  have sim1b: "Π, g, ϱ ⊢ b ≈ dg1" using sim1 b by simp
  from step.IH[OF sim1b WFb] obtain dg' where
    run2: "κ,g ⊢ dg1 →_c* dg'" and sim2: "Π, g, ϱ ⊢ cc ≈ dg'" by blast
  from run1 run2 have "κ,g ⊢ (v0, t0, k0) →_c* dg'" by (rule star_trans)
  with sim2 show ?case using dgd by blast
qed

theorem csim_star:
  assumes SIM: "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk)"
      and PC: "procs_embedded Π g"
      and WF: "return_safe c"
      and RUN: "κ,Π ⊢ (c, s, frs) →_p* src'"
  shows "∃cfg'. (κ,g ⊢ (v, t, stk) →_c* cfg') ∧ Π, g, ϱ ⊢ src' ≈ cfg'"
  using csim_run[OF PC RUN SIM] WF by simp
```

The static certificate for a whole compiled program

The static source contract establishes the runtime return guard for the root activation.

```
lemma wf_compile_input_return_safe:
  assumes "gwf_compile_input vars ReturnVar is_global Π ps"
  shows "return_safe (main_body Π)"
  using wf_compile_inputD(8)[OF assms] wf_compile_inputD(4)[OF assms]
  by (rule return_safe_if_no_return)
```

`procs_embedded` is the static certificate `csim` reads: every procedure declared in Π has its body fragment, entry `EA_Body` wiring and `EA_Ret None` exit wiring in the target graph. It covers the entry procedure too, since Π `prog_main_name = Some (⦇formals = [], body = main_body Π⦈)` makes `FunctionEntry prog_main_name` an ordinary activation. Discharging it from actual `compile_prog` output, rather than assuming it forever, is what closes the static side of the simulation.

```
theorem procs_embedded_compile_prog:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
  shows "procs_embedded Π (compile_prog Π ps)"
  unfolding procs_embedded_def
proof (intro allI impI)
  fix p decl assume pd: "Π p = Some decl"
  let ?g = "compile_prog Π ps"
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (⦇formals = [], body = main_body Π⦈) n1
                = (n2, Emain, Kmain)"
    and intra_g: "intra ?g = Eprocs ∪ Emain" and calls_g: "calls ?g = Kprocs ∪ Kmain"
```

```
    by (rule compile_prog_intra_split)
  have srccom: "source_com (body decl)"
    using wf_compile_inputD(7)[OF wf] pd unfolding source_pi_def by blast
  have "∃m m' Ef Kf. compile_proc Π p decl m = (m', Ef, Kf)
        ∧ Ef ⊆ intra ?g ∧ Kf ⊆ calls ?g"
  proof (cases "p = prog_main_name")
    case True
    with pd wf_compile_inputD(2)[OF wf]
    have "decl = (|formals = [], body = main_body Π|)" by simp
    then show ?thesis using mainc True intra_g calls_g by blast
  next
    case False
    with pd wf_compile_inputD(10)[OF wf] have "p ∈ set ps" by auto
    from compile_procs_member[OF procs this pd] obtain m m' Ef Kf where
      cp: "compile_proc Π p decl m = (m', Ef, Kf)" and "Ef ⊆ Eprocs" "Kf ⊆ Kprocs"
      by blast
    then have "Ef ⊆ intra ?g" "Kf ⊆ calls ?g" using intra_g calls_g by auto
    with cp show ?thesis by blast
  qed
  then obtain m m' Ef Kf where cp: "compile_proc Π p decl m = (m', Ef, Kf)"
    and Esub: "Ef ⊆ intra ?g" and Ksub: "Kf ⊆ calls ?g" by blast
  from cp obtain Eb where
    cb: "compile Π p (body decl) (Statement (m + csize (body decl))) m
         = (m + csize (body decl), Statement m, Eb, Kf)"
    and Edef: "Ef = insert (FunctionEntry p, EA_Body p, Statement m)
                (insert (Statement (m + csize (body decl)), EA_Ret None p, FunctionResult p) Eb)"
    by (rule compile_procE)
  have Ebsub: "Eb ⊆ intra ?g" using Edef Esub by auto
  have ent: "(FunctionEntry p, EA_Body p, Statement m) ∈ intra ?g" using Edef Esub by auto
  have ext: "(Statement (m + csize (body decl)), EA_Ret None p, FunctionResult p) ∈ intra ?g"
    using Edef Esub by auto
  show "∃k n n' en E K. compile Π p (body decl) k n = (n', en, E, K)
        ∧ E ⊆ intra ?g ∧ K ⊆ calls ?g
        ∧ (FunctionEntry p, EA_Body p, en) ∈ intra ?g
        ∧ (k, EA_Ret None p, FunctionResult p) ∈ intra ?g
        ∧ k = Statement (n + csize (body decl))
        ∧ source_com (body decl) ∧ special_table p = None"
    using cb Ebsub Ksub ent ext srccom wf_compile_inputD(6)[OF wf pd] by blast
qed
```

The certificate is inhabited

Everything above is stated under `procs_embedded`, and `csim` relates configurations only where that certificate holds; a reader is owed one graph that satisfies it, or the theorems could all be vacuous. The smallest program — `main` with an empty body and no callees — is that witness, and it carries through to a `csim` relating a real source configuration to a real node of a real compiled graph.

```
definition witness_pi :: "(unit, var_id, exp) gptable" where
  "witness_pi p = (if p = STR ''main'' then Some (|formals = [], body = SKIP ()|) else None)"

abbreviation witness_cfg :: "(var_id, exp) gcfg" where
  "witness_cfg ≡ compile_prog witness_pi []"

lemma main_body_witness [simp]: "main_body witness_pi = SKIP ()"
  by (simp add: main_body_def witness_pi_def prog_main_name_def)

lemma witness_wf: "gwf_compile_input exp_vars ReturnVar is_global witness_pi []"
  by (auto simp: gwf_compile_input_def gwf_source_program_def witness_pi_def
        main_body_def prog_main_name_def
        gwf_proc_decl_def greserved_ret_var_def
        special_table_def special_pname_nondet_int_def
        special_pname_min_def special_pname_max_def)

theorem procs_embedded_witness: "procs_embedded witness_pi witness_cfg"
  by (rule procs_embedded_compile_prog[OF witness_wf])

theorem csim_witness:
  obtains v where "witness_pi, witness_cfg, STR ''main'' ⊢ (SKIP (), s, []) ≈ (v, s, [])"
proof -
  have decl: "witness_pi (STR ''main'') = Some (|formals = [], body = SKIP ()|)"
    by (simp add: witness_pi_def)
  obtain k n en where
    cacc: "compiled_at witness_pi witness_cfg (STR ''main'')
           (body (|formals = [], body = SKIP ()|)) k n"
      and ctrl: "control_at witness_pi (STR ''main'')
           (body (|formals = [], body = SKIP ()|)) k n (body (|formals = [], body = SKIP ()|)) en"
```

```
    by (rule procs_embedded_activation[OF procs_embedded_witness decl])
  have "witness_pi, witness_cfg, STR ''main'' ⊢ (SKIP (), s, []) ≈ (en, s, [])"
    by (rule csim.Base[OF ctrl[simplified] cacc[simplified]])
  then show ?thesis ..
qed

end
```

5.8 Source_Progress

```
theory Source_Progress
  imports Simulation_Preservation
begin
```

Reachable source configurations are never stuck

A stuck source configuration would silently cut executions short: every later program point would be unreachable under `pstep`, so an unreachability verdict could hold for the wrong reason. Progress rules this out for accepted programs: a reachable configuration either has finished — `SKIP` with no caller frame left — or can take a step.

The shapes that could get stuck are the runtime markers: a bare `Unwind`, and a `Restore` or `Seq (Unwind j) (Restore i)` with no frame to pop. `csim` already pairs every `Restore` with its frame and confines `Unwind` to the returning phase, so it is reused here as the invariant; the only additional fact is that call sites are well formed.

Source commands

```
lemma pstep_seq_after_lift:
  "κ,Π ⊢ (w, s, frs) →p (w', s', frs')
   ⟹ κ,Π ⊢ (seq_after w afters, s, frs) →p (seq_after w' afters, s', frs')"
  by (induction afters arbitrary: w w') auto
```

A well-formed source command is `SKIP` or steps, at any frame stack. A `Return` steps too; whether its `Unwind` can later pop a frame is the simulation's concern, not this lemma's.

```
lemma wf_source_com_progress:
  assumes distinct_formals: "⋀p decl. Π p = Some decl ⟹ distinct (formals decl)"
      and "gwf_source_com vars ReturnVar Π c"
  shows "is_SKIP c ∨ (∃cfg'. κ,Π ⊢ (c, s, frs) →p cfg')"
  using assms(2)
proof (induction c)
  case (Seq c1 c2)
  from Seq.prems have "gwf_source_com vars ReturnVar Π c1" by simp
  from Seq.IH(1)[OF this] show ?case
  proof
    assume "is_SKIP c1"
    then obtain j where "c1 = SKIP j" by auto
    then show ?thesis by blast
  next
    assume "∃cfg'. κ,Π ⊢ (c1, s, frs) →p cfg'"
    then obtain c1' s' frs' where "κ,Π ⊢ (c1, s, frs) →p (c1', s', frs')" by auto
    then show ?thesis by blast
  qed
next
  case (If i b c1 c2)
  show ?case by (cases "truthy (eval b s)") blast+
next
  case (Return i e)
  show ?case by (cases e) blast+
next
  case (Call i dst p actuals)
  show ?case
  proof (cases "special_table p")
    case None
    with Call.prems obtain decl where
      decl: "Π p = Some decl" and arity: "length actuals = length (formals decl)"
      by (auto split: option.splits)
    show ?thesis
      using pstep_Call[where Π = Π and p = p, OF decl arity distinct_formals[OF decl]]
      by blast
  next
    case (Some desc)
    with Call.prems obtain sc x where
      sc: "classify_special desc actuals = Some sc" and dst: "dst = Some x"
      by (auto split: option.splits)
    obtain v where "special_result eval sc s v" using special_result_ex [of eval sc s] by blast
    with Some sc show ?thesis unfolding dst by blast
  qed
qed auto
```

```
lemma control_at_wf_source_com:
  "control_at Π p c0 k n r v ⟹ gwf_source_com vars ReturnVar Π c0 ⟹ gwf_source_com vars ReturnVar Π r"
  by (induction rule: control_at.induct) auto
```

The frame-pop phase

A configuration unwinding a finished frame keeps stepping until it is `Unwind` (`unwinding_progress`); a `Returning` one then pops its frame.

```
lemma unwinding_progress:
  "unwinding u ⟹ (⋀j. u ≠ Unwind j) ⟹ ∃u'. κ,Π ⊢ (u, s, frs) →p (u', s, frs)"
proof (induction u)
  case (Seq u1 c2)
  then have u1: "unwinding u1" and c2: "¬ is_Restore c2" by simp_all
  show ?case
  proof (cases "∃j. u1 = Unwind j")
    case True
    then obtain j where "u1 = Unwind j" by blast
    then show ?thesis using c2 by blast
  next
    case False
    then have "u1 ≠ Unwind j" for j by blast
    from Seq.IH(1)[OF u1 this] obtain u1' where "κ,Π ⊢ (u1, s, frs) →p (u1', s, frs)" ..
    then show ?thesis by blast
  qed
next
  case (Unwind x)
  then show ?case by blast
qed simp_all
```

The frame a `Returning` configuration carries is what lets it pop.

```
lemma pop_ready_progress:
  assumes "pop_ready w"
  shows "∃cfg'. κ,Π ⊢ (w, s, Frame fr dst # frs) →p cfg'"
  using assms
proof (cases w rule: pop_ready.cases)
  case 1
  then show ?thesis by blast
next
  case (2 u)
  with assms have uw: "unwinding u" by simp
  show ?thesis
  proof (cases "∃j. u = Unwind j")
    case True
    then obtain j where "u = Unwind j" by blast
    then show ?thesis using 2 by blast
  next
    case False
    then have "u ≠ Unwind j" for j by blast
    from unwinding_progress[OF uw this] obtain u' where
      "κ,Π ⊢ (u, s, Frame fr dst # frs) →p (u', s, Frame fr dst # frs)" ..
    then show ?thesis using 2 by blast
  qed
qed (use assms in simp_all)
```

Progress under the simulation relation

Only a `Base` activation can have finished: `Nested` and `Returning` both carry a caller frame and a pending `Restore` that will pop it. No graph fact is used, so the relation serves here purely as a shape invariant.

```
lemma csim_progress_cfg:
  assumes "Π, g, ϱ ⊢ cfg ≈ dg"
      and wf_decls: "⋀p decl. Π p = Some decl ⟹ gwf_proc_decl vars ReturnVar is_global Π decl"
  shows "(is_SKIP (fst cfg) ∧ snd (snd cfg) = []) ∨ (∃cfg'. κ,Π ⊢ cfg →p cfg')"
  using assms(1)
proof (induction rule: csim.induct)
  case (Base p body_p k n c v s)
  from Base.hyps(2) obtain decl where decl: "Π p = Some decl" and body_p: "body_p = body decl"
    by (rule compiled_at_decl)
  have "gwf_source_com vars ReturnVar Π body_p"
    using wf_decls[OF decl] body_p by (simp add: gwf_proc_decl_def)
  from control_at_wf_source_com[OF Base.hyps(1) this]
  have "gwf_source_com vars ReturnVar Π c" .
  from wf_source_com_progress[OF _ this, where s = s and frs = "[]"] wf_decls
  show ?case by (auto simp: gwf_proc_decl_def)
next
  case (Nested q inner s frs v stk pc body_pc kc nc i afters cont dst args site caller)
  let ?F = "[Frame caller dst]"
  have "∃w'. κ,Π ⊢ (Seq inner (Restore i), s, frs @ ?F) →p w'"
    using Nested.IH
```

```
  proof
    assume "is_SKIP (fst (inner, s, frs)) ∧ snd (snd (inner, s, frs)) = []"
    then obtain j where "inner = SKIP j" by auto
    then show ?thesis by blast
  next
    assume "∃cfg'. κ,Π ⊢ (inner, s, frs) →p cfg'"
    then obtain i' s' f' where "κ,Π ⊢ (inner, s, frs) →p (i', s', f')" by auto
    from pstep_frame_extend[OF this, of ?F]
    show ?thesis by blast
  qed
  then obtain w' s' f' where "κ,Π ⊢ (Seq inner (Restore i), s, frs @ ?F) →p (w', s', f')"
    by auto
  from pstep_seq_after_lift[OF this, of afters] show ?case by auto
next
  case (Returning w i pc body_pc kc nc afters cont dst p args site callee caller)
  from pop_ready_progress[OF Returning.hyps(1), of κ Π callee caller dst "[]"]
  obtain w' s' f' where "κ,Π ⊢ (w, callee, [Frame caller dst]) →p (w', s', f')"
    by auto
  from pstep_seq_after_lift[OF this, of afters] show ?case by auto
qed

theorem csim_progress:
  assumes "Π, g, ϱ ⊢ (c, s, frs) ≈ dg"
      and "⋀p decl. Π p = Some decl ⟹ gwf_proc_decl vars ReturnVar is_global Π decl"
  shows "(is_SKIP c ∧ frs = []) ∨ (∃cfg'. κ,Π ⊢ (c, s, frs) →p cfg')"
  using csim_progress_cfg[OF assms] by simp
```

Progress for accepted programs

The premise is source well-formedness alone; no compiled graph is involved. `csim` still needs some graph with every body embedded, and the complete graph — every edge present — embeds anything, so it stands in for the compiler's output. The finished shape is exactly `(SKIP, s, [])`: `main` contains no `Return` (`no_return`), so a top-level `Unwind` never arises.

```
theorem source_progress:
  assumes wf: "gwf_source_program vars ReturnVar is_global Π"
      and run: "κ,Π ⊢ (main_body Π, s0, []) →p* (c, s, frs)"
  shows "(is_SKIP c ∧ frs = []) ∨ (∃cfg'. κ,Π ⊢ (c, s, frs) →p cfg')"
proof -
  define g :: "(var_id, exp) gcfg" where
    "g = (|intra = UNIV, calls = UNIV, cfg_entry = FunctionEntry prog_main_name|)"
  have pc: "procs_embedded Π g"
    unfolding procs_embedded_def
  proof (intro allI impI)
    fix p decl assume pd: "Π p = Some decl"
    obtain n' en E K where
      cb: "compile Π p (body decl) (Statement (0 + csize (body decl))) 0 = (n', en, E, K)"
      by (metis prod_cases4)
    have "source_com (body decl)" "special_table p = None"
      using wf_source_programD(6,7)[OF wf] pd by (auto simp: source_pi_def)
    with cb show "∃k n n' en E K. compile Π p (body decl) k n = (n', en, E, K)
          ∧ E ⊆ intra g ∧ K ⊆ calls g
          ∧ (FunctionEntry p, EA_Body p, en) ∈ intra g
          ∧ (k, EA_Ret None p, FunctionResult p) ∈ intra g
          ∧ k = Statement (n + csize (body decl))
          ∧ source_com (body decl) ∧ special_table p = None"
      by (intro exI[of _ "Statement (0 + csize (body decl))"] exI[of _ "0 :: nat"] exI[of _ n']
            exI[of _ en] exI[of _ E] exI[of _ K])
         (simp add: g_def)
  qed
  have main_decl: "Π prog_main_name = Some (|formals = [], body = main_body Π|)"
    by (rule wf_source_programD(2)[OF wf])
  obtain k n en where
    cacc: "compiled_at Π g prog_main_name (body (|formals = [], body = main_body Π|)) k n"
    and ctrl: "control_at Π prog_main_name (body (|formals = [], body = main_body Π|)) k n
                (body (|formals = [], body = main_body Π|)) en"
    by (rule procs_embedded_activation[OF pc main_decl])
  have base: "Π, g, prog_main_name ⊢ (main_body Π, s0, []) ≈ (en, s0, [])"
    using csim.Base[OF ctrl cacc] by simp
  have safe: "return_safe (main_body Π)"
    using wf_source_programD(8,4)[OF wf] by (rule return_safe_if_no_return)
  from csim_star[OF base pc safe run] obtain dg where "Π, g, prog_main_name ⊢ (c, s, frs) ≈ dg"
    by blast
  then show ?thesis
    by (rule csim_progress) (rule wf_source_programD(5)[OF wf])
qed
```

```
end
```

5.9 Procedure_Ownership

```
theory Procedure_Ownership
  imports Compile_Invariants "Voblint_CFG.Activation_Trace_Def"
begin
```

Each procedure owns a disjoint block of nodes

An activation's own local `path_of` stays inside one compiled procedure fragment. A *fragment* of procedure `r` compiled over `[m, m')` is `FunctionEntry r`, `FunctionResult r`, and the `Statement` nodes in that counter range. Every intra edge keeps both endpoints in one fragment, every call edge keeps its source and continuation there (the callee `FunctionEntry` is the only cross-fragment endpoint), and distinct fragments are node-disjoint, so a single activation entered at `FunctionEntry p` reaches `FunctionResult q` only for `p = q`.

```
definition pfn :: "pname ⇒ nat ⇒ nat ⇒ cfg_node set" where
  "pfn r m m' = insert (FunctionEntry r) (insert (FunctionResult r) (Statement ` {m..<m'}))"

lemma pfn_disjoint:
  "p ≠ r ⟹ mp' ≤ m ∨ m' ≤ mp ⟹ x ∈ pfn p mp mp' ⟹ x ∉ pfn r m m'"
  by (auto simp: pfn_def)
```

Edges keep endpoints in one fragment

At command level both endpoints lie in the fragment's own node set or are its continuation (`compile_E_shape`). At procedure level the continuation is the epilogue node, which is inside the fragment's range, so the extra alternative collapses.

```
lemma compile_K_shape:
  "compile Π p c k n = (n', en, E, K) ⟹ (u, ce, tgt, af) ∈ K
   ⟹ (∃j. u = Statement j) ∧ (∃q. tgt = FunctionEntry q)
       ∧ (af = k ∨ (∃j. af = Statement j))"
proof (induction c arbitrary: k n n' en E K)
  case (Seq c1 c2)
  from Seq.prems(1) obtain n1 E1 K1 n2 E2 K2 where
    c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    and K: "K = K1 ∪ K2"
    by (rule compile_SeqE)
  from Seq.prems(2) K show ?case using Seq.IH(1)[OF c1] Seq.IH(2)[OF c2] by auto
next
  case (If i b c1 c2)
  from If.prems(1) obtain n1 E1 K1 n2 E2 K2 where
    c1: "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and c2: "compile Π p c2 k (Suc n + csize c1)
               = (n2, Statement (Suc n + csize c1), E2, K2)"
    and K: "K = K1 ∪ K2"
    by (rule compile_IfE)
  from If.prems(2) K show ?case using If.IH(1)[OF c1] If.IH(2)[OF c2] by auto
next
  case (While i b c)
  from While.prems(1) obtain n1 E1 K1 where
    c1: "compile Π p c (Statement n) (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and K: "K = K1"
    by (rule compile_WhileE)
  from While.prems(2) K show ?case using While.IH[OF c1] by auto
qed (auto split: if_splits option.splits)

lemma compile_intra_pfn:
  assumes cp: "compile Π p c k n = (n', en, E, K)" and e: "(u, a, v) ∈ E"
  shows "u ∈ insert k (pfn p n n') ∧ v ∈ insert k (pfn p n n')"
proof -
  have rng: "frag_stmts E K ⊆ {n..<n'} ∪ kstmt k" using compile_frag_stmts_range[OF cp] .
  from compile_E_shape[OF cp e] obtain ku where u: "u = Statement ku"
    and vshape: "v = k ∨ v = FunctionResult p ∨ (∃kv. v = Statement kv)" by blast
  have "ku ∈ frag_stmts E K" using frag_stmts_E_srcI[of ku a v E] e u by simp
  hence uin: "u ∈ insert k (pfn p n n')" using rng u by (auto simp: pfn_def)
  have vin: "v ∈ insert k (pfn p n n')"
  proof (cases "v = k ∨ v = FunctionResult p")
    case True then show ?thesis by (auto simp: pfn_def)
  next
    case False
    then obtain kv where v: "v = Statement kv" using vshape by blast
    have "kv ∈ frag_stmts E K" using frag_stmts_E_tgtI[of u a kv E] e v by simp
    then show ?thesis using rng v False by (auto simp: pfn_def)
  qed
```

```
  from uin vin show ?thesis ..
qed

lemma compile_calls_pfn:
  assumes cp: "compile Π p c k n = (n', en, E, K)" and e: "(u, ce, tgt, af) ∈ K"
  shows "u ∈ insert k (pfn p n n') ∧ af ∈ insert k (pfn p n n')"
proof -
  have rng: "frag_stmts E K ⊆ {n..<n'} ∪ kstmt k" using compile_frag_stmts_range[OF cp] .
  from compile_K_shape[OF cp e] obtain ku where u: "u = Statement ku"
    and afshape: "af = k ∨ (∃kaf. af = Statement kaf)" by blast
  have "ku ∈ frag_stmts E K" using frag_stmts_K_srcI[of ku ce tgt af K E] e u by simp
  hence uin: "u ∈ insert k (pfn p n n')" using rng u by (auto simp: pfn_def)
  have "af ∈ insert k (pfn p n n')"
  proof (cases "af = k")
    case True then show ?thesis by simp
  next
    case False
    then obtain kaf where af: "af = Statement kaf" using afshape by blast
    have "kaf ∈ frag_stmts E K" using frag_stmts_K_tgtI[of u ce tgt kaf K E] e af by simp
    then show ?thesis using rng af False by (auto simp: pfn_def)
  qed
  with uin show ?thesis ..
qed
```

The entry wiring edge targets the first statement index of the fragment, which is what identifies a fragment by its start counter alone.

```
lemma compile_proc_entry_edge:
  assumes cp: "compile_proc Π p decl n = (n', E, K)"
  shows "(FunctionEntry p, EA_Body p, Statement n) ∈ E"
  using cp by auto

lemma compile_proc_intra_pfn:
  assumes cp: "compile_proc Π r decl n = (n', E, K)" and e: "(u, a, v) ∈ E"
  shows "u ∈ pfn r n n' ∧ v ∈ pfn r n n'"
proof -
  from cp obtain Eb where
    cb: "compile Π r (body decl) (Statement (n + csize (body decl))) n
          = (n + csize (body decl), Statement n, Eb, K)"
    and E: "E = insert (FunctionEntry r, EA_Body r, Statement n)
              (insert (Statement (n + csize (body decl)), EA_Ret None r, FunctionResult r) Eb)"
    and n': "n' = Suc (n + csize (body decl))"
    by (rule compile_procE)
  have kin: "Statement (n + csize (body decl)) ∈ pfn r n n'"
    using n' by (auto simp: pfn_def)
  have entryin: "Statement n ∈ pfn r n n'"
    using n' csize_pos[of "body decl"] by (auto simp: pfn_def)
  have inner: "insert (Statement (n + csize (body decl)))
                 (pfn r n (n + csize (body decl))) ⊆ pfn r n n'"
    using n' by (auto simp: pfn_def)
  consider "(u, a, v) = (FunctionEntry r, EA_Body r, Statement n)"
    | "(u, a, v) = (Statement (n + csize (body decl)), EA_Ret None r, FunctionResult r)"
    | "(u, a, v) ∈ Eb"
    using e E by (auto split: if_splits)
  then show ?thesis
  proof cases
    case 1 then show ?thesis using entryin by (auto simp: pfn_def)
  next
    case 2 then show ?thesis using kin by (auto simp: pfn_def)
  next
    case 3 then show ?thesis using compile_intra_pfn[OF cb] inner by blast
  qed
qed

lemma compile_proc_calls_pfn:
  assumes cp: "compile_proc Π r decl n = (n', E, K)" and e: "(u, ce, tgt, af) ∈ K"
  shows "u ∈ pfn r n n' ∧ af ∈ pfn r n n'"
proof -
  from cp obtain Eb where
    cb: "compile Π r (body decl) (Statement (n + csize (body decl))) n
          = (n + csize (body decl), Statement n, Eb, K)"
    and n': "n' = Suc (n + csize (body decl))"
    by (rule compile_procE)
  have inner: "insert (Statement (n + csize (body decl)))
                 (pfn r n (n + csize (body decl))) ⊆ pfn r n n'"
    using n' by (auto simp: pfn_def)
  show ?thesis using compile_calls_pfn[OF cb e] inner by blast
```

qed

A fragment of another procedure over a disjoint counter range sources none of its edges in `pfn r m m'`.

```
lemma compile_proc_src_notin_pfn:
  assumes cp: "compile_proc Π p decl mp = (mp', Ep, Kp)"
    and neq: "p ≠ r" and disjoint: "mp' ≤ m ∨ m' ≤ mp"
  shows "(u, a, v) ∈ Ep ⟹ u ∉ pfn r m m'"
    and "(u, ce, tgt, af) ∈ Kp ⟹ u ∉ pfn r m m'"
  using compile_proc_intra_pfn[OF cp] compile_proc_calls_pfn[OF cp] pfn_disjoint[OF neq disjoint]
  by blast+

lemma compile_procs_intra_src_notin_pfn:
  assumes cps: "compile_procs Π ps n = (n', E, K)" and e: "(u, a, v) ∈ E"
    and absent: "r ∉ set ps" and disjoint: "n' ≤ m ∨ m' ≤ n"
  shows "u ∉ pfn r m m'"
proof -
  from compile_procs_intra_origin[OF cps e] obtain p decl mp mp' Ep Kp
    where "p ∈ set ps" and cp: "compile_proc Π p decl mp = (mp', Ep, Kp)"
      and "n ≤ mp" "mp' ≤ n'" and ep: "(u, a, v) ∈ Ep" by blast
  have "p ≠ r" using ‹p ∈ set ps› absent by blast
  moreover have "mp' ≤ m ∨ m' ≤ mp" using ‹n ≤ mp› ‹mp' ≤ n'› disjoint by auto
  ultimately show ?thesis using compile_proc_src_notin_pfn(1)[OF cp _ _ ep] by blast
qed

lemma compile_procs_calls_src_notin_pfn:
  assumes cps: "compile_procs Π ps n = (n', E, K)" and e: "(u, ce, tgt, af) ∈ K"
    and absent: "r ∉ set ps" and disjoint: "n' ≤ m ∨ m' ≤ n"
  shows "u ∉ pfn r m m'"
proof -
  from compile_procs_calls_origin[OF cps e] obtain p decl mp mp' Ep Kp
    where "p ∈ set ps" and cp: "compile_proc Π p decl mp = (mp', Ep, Kp)"
      and "n ≤ mp" "mp' ≤ n'" and ep: "(u, ce, tgt, af) ∈ Kp" by blast
  have "p ≠ r" using ‹p ∈ set ps› absent by blast
  moreover have "mp' ≤ m ∨ m' ≤ mp" using ‹n ≤ mp› ‹mp' ≤ n'› disjoint by auto
  ultimately show ?thesis using compile_proc_src_notin_pfn(2)[OF cp _ _ ep] by blast
qed
```

A procedure entry has a unique out-edge

Within a compiled procedure only the entry wiring edge sources from `FunctionEntry` (bodies source from `Statement` nodes by `compile_E_shape`, the exit wiring from the exit `Statement`), so the source name is the procedure and the target is unique.

```
lemma compile_proc_entry_mem:
  assumes cp: "compile_proc Π q decl n = (n', E, K)" and e: "(FunctionEntry r, a, v) ∈ E"
  shows "r = q"
proof -
  from cp obtain Eb where
    cb: "compile Π q (body decl) (Statement (n + csize (body decl))) n
          = (n + csize (body decl), Statement n, Eb, K)"
    and E: "E = insert (FunctionEntry q, EA_Body q, Statement n)
              (insert (Statement (n + csize (body decl)), EA_Ret None q, FunctionResult q) Eb)"
    by (rule compile_procE)
  have "⋀aa vv. (FunctionEntry r, aa, vv) ∉ Eb" using compile_E_shape[OF cb] by blast
  then show ?thesis using e E by (auto split: if_splits)
qed

lemma compile_proc_entry_unique:
  assumes cp: "compile_proc Π q decl n = (n', E, K)"
    and e1: "(FunctionEntry r, a1, v1) ∈ E" and e2: "(FunctionEntry r, a2, v2) ∈ E"
  shows "v1 = v2"
proof -
  from cp obtain Eb where
    cb: "compile Π q (body decl) (Statement (n + csize (body decl))) n
          = (n + csize (body decl), Statement n, Eb, K)"
    and E: "E = insert (FunctionEntry q, EA_Body q, Statement n)
              (insert (Statement (n + csize (body decl)), EA_Ret None q, FunctionResult q) Eb)"
    by (rule compile_procE)
  have nb: "⋀aa vv. (FunctionEntry r, aa, vv) ∉ Eb" using compile_E_shape[OF cb] by blast
  from e1 E nb have "v1 = Statement n" by (auto split: if_splits)
  moreover from e2 E nb have "v2 = Statement n" by (auto split: if_splits)
  ultimately show ?thesis by simp
qed

lemma compile_procs_entry_mem:
```

```
  "compile_procs Π ps n = (n', E, K) ⟹ (FunctionEntry r, a, v) ∈ E ⟹ r ∈ set ps"
  using compile_procs_intra_origin compile_proc_entry_mem by metis
```

Under `distinct ps` a member's fragment owns every edge of the pass that sources in the member's nodes: the other members are compiled over disjoint counter ranges, and their edges stay in their own fragments.

```
lemma compile_procs_member_frag:
  assumes "compile_procs Π ps n = (n', E, K)" and "distinct ps"
    and "r ∈ set ps" and "Π r = Some decl"
  shows "∃m m' Er Kr. compile_proc Π r decl m = (m', Er, Kr) ∧ n ≤ m ∧ m' ≤ n'
           ∧ Er ⊆ E ∧ Kr ⊆ K
           ∧ (∀u a v. (u, a, v) ∈ E ⟶ u ∈ pfn r m m' ⟶ (u, a, v) ∈ Er)
           ∧ (∀u ce tgt af. (u, ce, tgt, af) ∈ K ⟶ u ∈ pfn r m m'
                ⟶ (u, ce, tgt, af) ∈ Kr)"
  using assms
proof (induction ps arbitrary: n n' E K)
  case Nil then show ?case by simp
next
  case (Cons q qs)
  show ?case
  proof (cases "Π q")
    case None
    with Cons.prems have "compile_procs Π qs n = (n', E, K)" "distinct qs" "r ∈ set qs"
      by auto
    from Cons.IH[OF this Cons.prems(4)] show ?thesis .
  next
    case (Some dq)
    with Cons.prems(1) obtain n1 E0 K0 n2 E' K' where
       cp0: "compile_proc Π q dq n = (n1, E0, K0)"
      and rest: "compile_procs Π qs n1 = (n2, E', K')"
      and n': "n' = n2" and E: "E = E0 ∪ E'" and K: "K = K0 ∪ K'"
      by (auto split: prod.splits)
    have qnotin: "q ∉ set qs" and dqs: "distinct qs" using Cons.prems(2) by simp_all
    show ?thesis
    proof (cases "r = q")
      case True
      with Cons.prems(4) Some cp0 have cpr: "compile_proc Π r decl n = (n1, E0, K0)" by simp
      have rnotin: "r ∉ set qs" using qnotin True by simp
      have tailE: "⋀u a v. (u, a, v) ∈ E' ⟹ u ∉ pfn r n n1"
        using compile_procs_intra_src_notin_pfn[OF rest _ rnotin] by blast
      have tailK: "⋀u ce tgt af. (u, ce, tgt, af) ∈ K' ⟹ u ∉ pfn r n n1"
        using compile_procs_calls_src_notin_pfn[OF rest _ rnotin] by blast
      have "∀u a v. (u, a, v) ∈ E ⟶ u ∈ pfn r n n1 ⟶ (u, a, v) ∈ E0"
        using E tailE by blast
      moreover have "∀u ce tgt af. (u, ce, tgt, af) ∈ K ⟶ u ∈ pfn r n n1
                       ⟶ (u, ce, tgt, af) ∈ K0"
        using K tailK by blast
      moreover have "n1 ≤ n'" using compile_procs_counter_mono[OF rest] n' by simp
      moreover have "E0 ⊆ E" "K0 ⊆ K" using E K by auto
      ultimately show ?thesis using cpr by blast
    next
      case False
      have rqs: "r ∈ set qs" using Cons.prems(3) False by simp
      from Cons.IH[OF rest dqs rqs Cons.prems(4)] obtain m m' Er Kr where
         cpr: "compile_proc Π r decl m = (m', Er, Kr)" and bnds: "n1 ≤ m" "m' ≤ n2"
        and sub: "Er ⊆ E'" "Kr ⊆ K'"
        and ownE: "∀u a v. (u, a, v) ∈ E' ⟶ u ∈ pfn r m m' ⟶ (u, a, v) ∈ Er"
        and ownK: "∀u ce tgt af. (u, ce, tgt, af) ∈ K' ⟶ u ∈ pfn r m m'
                     ⟶ (u, ce, tgt, af) ∈ Kr"
        by blast
      have qr: "q ≠ r" using False by simp
      have headE: "⋀u a v. (u, a, v) ∈ E0 ⟹ u ∉ pfn r m m'"
        and headK: "⋀u ce tgt af. (u, ce, tgt, af) ∈ K0 ⟹ u ∉ pfn r m m'"
        using compile_proc_src_notin_pfn[OF cp0 qr] bnds(1) by blast+
      have "∀u a v. (u, a, v) ∈ E ⟶ u ∈ pfn r m m' ⟶ (u, a, v) ∈ Er"
        using E ownE headE by blast
      moreover have "∀u ce tgt af. (u, ce, tgt, af) ∈ K ⟶ u ∈ pfn r m m'
                       ⟶ (u, ce, tgt, af) ∈ Kr"
        using K ownK headK by blast
      moreover have "n ≤ m" using compile_proc_counter_mono[OF cp0] bnds(1) by simp
      moreover have "m' ≤ n'" using bnds(2) n' by simp
      moreover have "Er ⊆ E" "Kr ⊆ K" using sub E K by auto
      ultimately show ?thesis using cpr by blast
    qed
  qed
qed
```

```
lemma compile_procs_entry_unique:
  assumes cps: "compile_procs Π ps n = (n', E, K)" and distinct: "distinct ps"
    and e1: "(FunctionEntry r, a1, v1) ∈ E" and e2: "(FunctionEntry r, a2, v2) ∈ E"
  shows "v1 = v2"
proof -
  from compile_procs_intra_origin[OF cps e1] obtain decl
    where rin: "r ∈ set ps" and decl: "Π r = Some decl"
    using compile_proc_entry_mem by metis
  from compile_procs_member_frag[OF cps distinct rin decl] obtain m m' Er Kr where
      cp: "compile_proc Π r decl m = (m', Er, Kr)"
    and own: "∀u a v. (u, a, v) ∈ E ⟶ u ∈ pfn r m m' ⟶ (u, a, v) ∈ Er" by blast
  have "(FunctionEntry r, a1, v1) ∈ Er" "(FunctionEntry r, a2, v2) ∈ Er"
    using own e1 e2 by (auto simp: pfn_def)
  then show ?thesis by (rule compile_proc_entry_unique[OF cp])
qed
```

In a whole compiled program `FunctionEntry r` has a unique out-target: `main` and the callees are node-disjoint (`prog_main_name ∉ set ps`), and each contributes a single entry edge.

```
lemma compile_prog_entry_out_unique:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and e1: "(FunctionEntry r, a1, v1) ∈ intra (compile_prog Π ps)"
    and e2: "(FunctionEntry r, a2, v2) ∈ intra (compile_prog Π ps)"
  shows "v1 = v2"
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (|formals = [], body = (main_body Π)|) n1
                = (n2, Emain, Kmain)"
    and EI: "intra (compile_prog Π ps) = Eprocs ∪ Emain"
    by (rule compile_prog_intra_split)
  have main_mem: "⋀b w. (FunctionEntry r, b, w) ∈ Emain ⟹ r = prog_main_name"
    using compile_proc_entry_mem[OF mainc] by simp
  have procs_mem: "⋀b w. (FunctionEntry r, b, w) ∈ Eprocs ⟹ r ∈ set ps"
    using compile_procs_entry_mem[OF procs] by simp
  show ?thesis
  proof (cases "r = prog_main_name")
    case True
    have "(FunctionEntry r, a1, v1) ∈ Emain" "(FunctionEntry r, a2, v2) ∈ Emain"
      using e1 e2 EI procs_mem True wf_compile_inputD(11)[OF wf] by auto
    then show ?thesis using compile_proc_entry_unique[OF mainc] by blast
  next
    case False
    have "(FunctionEntry r, a1, v1) ∈ Eprocs" "(FunctionEntry r, a2, v2) ∈ Eprocs"
      using e1 e2 EI main_mem False by auto
    then show ?thesis using compile_procs_entry_unique[OF procs wf_compile_inputD(9)[OF wf]]
      by blast
  qed
qed
```

Whole-program edge shapes

Two shape facts a consumer of a compiled graph may use without redoing the callee/`main` split: a call edge always leaves a `Statement` node, and an entry edge names a procedure the table declares. The first is a range property each pass has on its own; the second needs well-formed input, which is what ties the compiled procedure list back to the table.

```
lemma compile_prog_calls_source_stmt:
  assumes "(u, ce, tgt, af) ∈ calls (compile_prog Π ps)"
  shows "∃k. u = Statement k"
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (|formals = [], body = (main_body Π)|) n1
                = (n2, Emain, Kmain)"
    and KC: "calls (compile_prog Π ps) = Kprocs ∪ Kmain"
    by (rule compile_prog_intra_split)
  from assms KC show ?thesis
    using compile_procs_calls_source_range[OF procs] compile_proc_calls_source_range[OF mainc]
    by blast
qed

lemma compile_prog_entry_declared:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and e: "(FunctionEntry p, a, v) ∈ intra (compile_prog Π ps)"
  shows "∃d. Π p = Some d"
```

```
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (|formals = [], body = (main_body Π)|) n1
                = (n2, Emain, Kmain)"
    and EI: "intra (compile_prog Π ps) = Eprocs ∪ Emain"
    by (rule compile_prog_intra_split)
  from e EI consider "(FunctionEntry p, a, v) ∈ Eprocs" | "(FunctionEntry p, a, v) ∈ Emain"
    by auto
  then show ?thesis
  proof cases
    case 1
    then have "p ∈ set ps" using compile_procs_entry_mem[OF procs] by blast
    with wf_compile_inputD(10)[OF wf] show ?thesis by auto
  next
    case 2
    then have "p = prog_main_name" by (rule compile_proc_entry_mem[OF mainc])
    with wf_compile_inputD(2)[OF wf] show ?thesis by blast
  qed
qed
```

Edges stay in the activation fragment

The fragment a procedure's entry wiring identifies owns every edge of the program that sources in its nodes. The callee pass and `main` occupy disjoint counter ranges, and inside the pass `compile_procs_member_frag` settles ownership; the entry edge pins the start counter through `compile_prog_entry_out_unique`.

```
lemma compile_prog_entry_frag:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and decl: "Π r = Some d"
    and cb: "compile_proc Π r d m = (m', Ep, Kp)"
    and ent: "(FunctionEntry r, EA_Body r, Statement m) ∈ intra (compile_prog Π ps)"
  shows "(∀u a v. (u, a, v) ∈ intra (compile_prog Π ps) ⟶ u ∈ pfn r m m'
            ⟶ (u, a, v) ∈ Ep)
       ∧ (∀u ce tgt af. (u, ce, tgt, af) ∈ calls (compile_prog Π ps)
            ⟶ u ∈ pfn r m m' ⟶ (u, ce, tgt, af) ∈ Kp)"
proof -
  let ?g = "compile_prog Π ps"
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (|formals = [], body = (main_body Π)|) n1
                = (n2, Emain, Kmain)"
    and EI: "intra ?g = Eprocs ∪ Emain" and KC: "calls ?g = Kprocs ∪ Kmain"
    by (rule compile_prog_intra_split)
  have distinct: "distinct ps" and setps: "set ps = {p. Π p ≠ None} - {prog_main_name}"
    and main_notin: "prog_main_name ∉ set ps"
    using wf_compile_inputD(9,10,11)[OF wf] by blast+
  show ?thesis
  proof (cases "r = prog_main_name")
    case True
    have dd: "d = (|formals = [], body = (main_body Π)|)"
      using decl True wf_compile_inputD(2)[OF wf] by simp
    have entmain: "(FunctionEntry prog_main_name, EA_Body prog_main_name, Statement n1) ∈ intra ?g"
      using EI compile_proc_entry_edge[OF mainc] by simp
    have entr: "(FunctionEntry prog_main_name, EA_Body prog_main_name, Statement m) ∈ intra ?g"
      using ent True by simp
    have mn: "m = n1" using compile_prog_entry_out_unique[OF wf entr entmain] by simp
    have mout: "m' = n2" "Ep = Emain" "Kp = Kmain" using cb mainc True dd mn by simp_all
    show ?thesis unfolding True mn mout
      using EI KC compile_procs_intra_src_notin_pfn[OF procs _ main_notin]
        compile_procs_calls_src_notin_pfn[OF procs _ main_notin] by auto
  next
    case False
    have rin: "r ∈ set ps" using decl setps False by auto
    from compile_procs_member_frag[OF procs distinct rin decl] obtain m0 m0' Er Kr where
        cpr: "compile_proc Π r d m0 = (m0', Er, Kr)" and upper: "m0' ≤ n1"
      and Ersub: "Er ⊆ Eprocs"
      and ownE: "∀u a v. (u, a, v) ∈ Eprocs ⟶ u ∈ pfn r m0 m0' ⟶ (u, a, v) ∈ Er"
      and ownK: "∀u ce tgt af. (u, ce, tgt, af) ∈ Kprocs ⟶ u ∈ pfn r m0 m0'
                  ⟶ (u, ce, tgt, af) ∈ Kr"
      by blast
    have ent0: "(FunctionEntry r, EA_Body r, Statement m0) ∈ intra ?g"
      using EI Ersub compile_proc_entry_edge[OF cpr] by auto
    have mm: "m = m0" using compile_prog_entry_out_unique[OF wf ent ent0] by auto
    have mout: "m' = m0'" "Ep = Er" "Kp = Kr" using cb cpr mm by simp_all
    have neq: "prog_main_name ≠ r" using False by simp
```

```
    show ?thesis unfolding mm mout
      using EI KC ownE ownK compile_proc_src_notin_pfn[OF mainc neq] upper by blast
  qed
qed
```

Given a genuine procedure fragment (declared body, entry wiring) and a node in it, every intra edge leaving that node, and every call leaving it, lands its same-activation successor back in the fragment.

```
lemma frag_edge_intra:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and decl: "Π r = Some d"
    and cb: "compile_proc Π r d m = (m', Ep, Kp)"
    and ent: "(FunctionEntry r, EA_Body r, Statement m) ∈ intra (compile_prog Π ps)"
    and uin: "u ∈ pfn r m m'"
    and e: "(u, a, v) ∈ intra (compile_prog Π ps)"
  shows "v ∈ pfn r m m'"
  using compile_prog_entry_frag[OF wf decl cb ent] compile_proc_intra_pfn[OF cb] uin e by blast

lemma frag_edge_calls:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and decl: "Π r = Some d"
    and cb: "compile_proc Π r d m = (m', Ep, Kp)"
    and ent: "(FunctionEntry r, EA_Body r, Statement m) ∈ intra (compile_prog Π ps)"
    and uin: "u ∈ pfn r m m'"
    and e: "(u, ce, tgt, af) ∈ calls (compile_prog Π ps)"
  shows "af ∈ pfn r m m'"
  using compile_prog_entry_frag[OF wf decl cb ent] compile_proc_calls_pfn[OF cb] uin e by blast
```

Procedure locality of a valid activation

Every declared procedure of a compiled program owns a fragment, identified by the start counter its entry wiring edge points at.

```
lemma compile_prog_proc_frag:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and decl: "Π q = Some d"
  obtains m m' Ep Kp where
    "compile_proc Π q d m = (m', Ep, Kp)"
    "(FunctionEntry q, EA_Body q, Statement m) ∈ intra (compile_prog Π ps)"
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (⦇formals = [], body = (main_body Π)⦈) n1
                = (n2, Emain, Kmain)"
    and EI: "intra (compile_prog Π ps) = Eprocs ∪ Emain"
    by (rule compile_prog_intra_split)
  show thesis
  proof (cases "q = prog_main_name")
    case True
    have dd: "d = (⦇formals = [], body = (main_body Π)⦈)"
      using decl True wf_compile_inputD(2)[OF wf] by simp
    have "(FunctionEntry q, EA_Body q, Statement n1) ∈ intra (compile_prog Π ps)"
      using EI compile_proc_entry_edge[OF mainc] True by auto
    then show ?thesis using that[of n1 n2 Emain Kmain] mainc True dd by simp
  next
    case False
    have rin: "q ∈ set ps" using decl wf_compile_inputD(10)[OF wf] False by auto
    obtain m m' Ep Kp where cp0: "compile_proc Π q d m = (m', Ep, Kp)" and sub: "Ep ⊆ Eprocs"
      using compile_procs_member[OF procs rin decl] by blast
    have "(FunctionEntry q, EA_Body q, Statement m) ∈ intra (compile_prog Π ps)"
      using EI sub compile_proc_entry_edge[OF cp0] by auto
    then show ?thesis using that cp0 by blast
  qed
qed
```

`frag_ok u`: the activation `u` either lies wholly inside a genuine procedure fragment (declared body, entry wiring, all path nodes in the fragment) or is a single-node activation stuck at an undeclared `FunctionEntry` — which can never advance and never reaches a `FunctionResult`.

```
definition frag_ok :: "('a, 'v, 'e) gptable ⇒ pname list ⇒ activation_trace ⇒ bool" where
  "frag_ok Π ps u ⟷
     (∃r d m m' Ep Kp. Π r = Some d
        ∧ compile_proc Π r d m = (m', Ep, Kp)
        ∧ (FunctionEntry r, EA_Body r, Statement m) ∈ intra (compile_prog Π ps)
        ∧ fst (hd (path_of u)) = FunctionEntry r
        ∧ (∀nd ∈ fst ‘ set (path_of u). nd ∈ pfn r m m'))
```

```
   ∨ (∃p s. path_of u = [(FunctionEntry p, s)] ∧ Π p = None)"

lemma frag_okI_frag:
  assumes "Π r = Some d" "compile_proc Π r d m = (m', Ep, Kp)"
    "(FunctionEntry r, EA_Body r, Statement m) ∈ intra (compile_prog Π ps)"
    "fst (hd (path_of u)) = FunctionEntry r"
    "⋀nd. nd ∈ fst ‘ set (path_of u) ⟹ nd ∈ pfn r m m'"
  shows "frag_ok Π ps u"
  unfolding frag_ok_def using assms by blast

lemma frag_okE [elim]:
  assumes "frag_ok Π ps u"
  obtains (frag) r d m m' Ep Kp where "Π r = Some d" "compile_proc Π r d m = (m', Ep, Kp)"
    "(FunctionEntry r, EA_Body r, Statement m) ∈ intra (compile_prog Π ps)"
    "fst (hd (path_of u)) = FunctionEntry r" "∀nd ∈ fst ‘ set (path_of u). nd ∈ pfn r m m'"
  | (stub) p s where "path_of u = [(FunctionEntry p, s)]" "Π p = None"
  using assms unfolding frag_ok_def by meson

lemma sink_in_path_nodes:
  "t ∈ 𝒯κ,g,S ⟹ sink_node t ∈ fst ‘ set (path_of t)"
  using valid_activation_trace_path_nonempty by (auto simp: sink_node_def)
```

Extending a fragment-local activation by one node keeps it fragment-local when the step to that node stays inside every genuine fragment holding its sink and no step leaves a stub. An intra step and the resumption after a call are the two such extensions.

```
lemma frag_ok_snoc:
  assumes t: "t ∈ 𝒯κ,compile_prog Π ps,S" and ok: "frag_ok Π ps t"
    and pu: "path_of u = path_of t @ [(v, s)]"
    and stay: "⋀r d m m' Ep Kp. Π r = Some d ⟹ compile_proc Π r d m = (m', Ep, Kp) ⟹
      (FunctionEntry r, EA_Body r, Statement m) ∈ intra (compile_prog Π ps) ⟹
      sink_node t ∈ pfn r m m' ⟹ v ∈ pfn r m m'"
    and nostub: "⋀q. sink_node t = FunctionEntry q ⟹ Π q = None ⟹ False"
  shows "frag_ok Π ps u"
  using ok
proof (cases rule: frag_okE)
  case (frag r d m m' Ep Kp)
  have "sink_node t ∈ pfn r m m'" using sink_in_path_nodes[OF t] frag(5) by blast
  from stay[OF frag(1,2,3) this] have v_in: "v ∈ pfn r m m'" .
  show ?thesis
    by (rule frag_okI_frag[OF frag(1,2,3)])
       (use frag(4,5) valid_activation_trace_path_nonempty[OF t] v_in in ‹auto simp: pu hd_append›)
next
  case (stub q s0)
  have "sink_node t = FunctionEntry q" using stub(1) by (simp add: sink_node_def)
  from nostub[OF this stub(2)] show ?thesis ..
qed
```

Every activation in the caller chain of a valid activation trace is fragment-local. The property is carried over the whole `callers` chain so that the return case, which resumes the caller, can read the caller's own fragment (`caller ∈ callers callee`).

```
lemma valid_activation_trace_frag_callers:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and t: "t ∈ 𝒯κ,compile_prog Π ps,S"
  shows "∀u ∈ callers t. frag_ok Π ps u"
proof -
  let ?g = "compile_prog Π ps"
  show ?thesis
  proof (rule caller_chain_closure)
    fix s assume "s ∈ S"
    have main_decl: "Π prog_main_name = Some (|formals = [], body = (main_body Π)|)"
      by (rule wf_compile_inputD(2)[OF wf])
    obtain m m' Ep Kp where
      cb: "compile_proc Π prog_main_name (|formals = [], body = (main_body Π)|) m = (m', Ep, Kp)"
      and ent: "(FunctionEntry prog_main_name, EA_Body prog_main_name, Statement m) ∈ intra ?g"
      by (rule compile_prog_proc_frag[OF wf main_decl])
    show "frag_ok Π ps (Root [(cfg_entry ?g, s)])"
      by (rule frag_okI_frag[OF main_decl cb ent])
         (simp_all add: pfn_def)
  next
    fix t :: "activation_trace" and a v s'
    assume ht: "t ∈ 𝒯κ,?g,S" and ch: "∀u ∈ callers t. frag_ok Π ps u"
      and e: "(sink_node t, a, v) ∈ intra ?g"
    have "frag_ok Π ps t" using ch callers_refl by blast
    then show "frag_ok Π ps (extend t (v, s'))"
```

```
      by (rule frag_ok_snoc[OF ht _ _ frag_edge_intra[OF wf _ _ _ _ e]])
         (use e in ‹auto dest: compile_prog_entry_declared[OF wf]›)
  next
    fix caller :: "activation_trace" and dst pars args p cont
    assume e: "(sink_node caller, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls ?g"
    show "frag_ok Π ps (Call caller
            [(FunctionEntry p, call_enter κ (CallEdge dst pars args) (sink_store caller))])"
    proof (cases "Π p")
      case None
      then show ?thesis unfolding frag_ok_def by fastforce
    next
      case (Some d)
      obtain m m' Ep Kp where
        cb: "compile_proc Π p d m = (m', Ep, Kp)"
        and ent: "(FunctionEntry p, EA_Body p, Statement m) ∈ intra ?g"
        by (rule compile_prog_proc_frag[OF wf Some])
      show ?thesis
        by (rule frag_okI_frag[OF Some cb ent]) (simp_all add: pfn_def)
    qed
  next
    fix callee caller :: "activation_trace" and p dst pars args cont
    assume cvcallee: "callee ∈ 𝒯_κ,?g,S"
      and ch: "∀u ∈ callers callee. frag_ok Π ps u"
      and cof: "caller_of callee = Some caller" and res: "sink_node callee = FunctionResult p"
      and e: "(sink_node caller, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls ?g"
    have cin: "caller ∈ callers callee" using cof callers_caller_subset callers_refl by blast
    have cv: "caller ∈ 𝒯_κ,?g,S"
      using valid_activation_trace_caller_valid[OF cvcallee cof] .
    have "frag_ok Π ps caller" using ch cin by blast
    then show "frag_ok Π ps (Resume caller callee (path_of caller
          @ [(cont, combine_collect κ dst (sink_store caller) (sink_store callee))]))"
      by (rule frag_ok_snoc[OF cv _ _ frag_edge_calls[OF wf _ _ _ _ e]])
         (use e in ‹auto dest: compile_prog_calls_source_stmt›)
  next
    show "t ∈ 𝒯_κ,?g,S" by (rule t)
  qed
qed
```

An activation entered at `FunctionEntry p` reaches `FunctionResult q` only for `p = q`.

```
lemma valid_activation_trace_entry_result_eq:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and t: "t ∈ 𝒯_κ,compile_prog Π ps,S"
    and en: "fst (hd (path_of t)) = FunctionEntry p"
    and sk: "sink_node t = FunctionResult q"
  shows "p = q"
proof -
  have "frag_ok Π ps t"
    using valid_activation_trace_frag_callers[OF wf t] callers_refl by blast
  then show ?thesis
  proof (cases rule: frag_okE)
    case (frag r d m m' Ep Kp)
    have "sink_node t ∈ pfn r m m'" using sink_in_path_nodes[OF t] frag(5) by blast
    with frag(4) en sk show ?thesis by (auto simp: pfn_def)
  next
    case (stub p' s)
    then have "sink_node t = FunctionEntry p'" by (simp add: sink_node_def)
    with sk show ?thesis by simp
  qed
qed

end
```

5.10 Live_Nodes

```
theory Live_Nodes
  imports Procedure_Ownership
begin
```

Every node of a procedure reaches its result

A procedure's nodes are the endpoints of its own intra edges and call continuations. Every edge out of one of them lands on another (`prog_node_intra`, `prog_node_calls`), and every one reaches `FunctionResult q` along intra edges and call-site-to-continuation steps (`prog_node_reaches`) — exactly the steps a routed equation reads backwards. Code after a `Return` is no exception: it still ends at the epilogue, whose return edge `compile_proc` always emits.

The whole program

Each declared procedure's fragment sits inside the compiled program, so its edges and their reachability carry over to the whole graph.

```
lemma compile_prog_proc_frag_sub:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps" and decl: "Π q = Some d"
  obtains m m' Ep Kp where
    "compile_proc Π q d m = (m', Ep, Kp)"
    "(FunctionEntry q, EA_Body q, Statement m) ∈ intra (compile_prog Π ps)"
    "Ep ⊆ intra (compile_prog Π ps)" "Kp ⊆ calls (compile_prog Π ps)"
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (⦇formals = [], body = (main_body Π)⦈) n1
                = (n2, Emain, Kmain)"
    and EI: "intra (compile_prog Π ps) = Eprocs ∪ Emain"
    and KC: "calls (compile_prog Π ps) = Kprocs ∪ Kmain"
    by (rule compile_prog_intra_split)
  show thesis
  proof (cases "q = prog_main_name")
    case True
    have dd: "d = (⦇formals = [], body = (main_body Π)⦈)"
      using decl True wf_compile_inputD(2)[OF wf] by simp
    have "(FunctionEntry q, EA_Body q, Statement n1) ∈ intra (compile_prog Π ps)"
      using EI compile_proc_entry_edge[OF mainc] True by auto
    then show ?thesis using that[of n1 n2 Emain Kmain] mainc True dd EI KC by simp
  next
    case False
    have rin: "q ∈ set ps" using decl wf_compile_inputD(10)[OF wf] False by auto
    from compile_procs_member_frag[OF procs wf_compile_inputD(9)[OF wf] rin decl]
    obtain m m' Ep Kp where cp0: "compile_proc Π q d m = (m', Ep, Kp)"
        and sub: "Ep ⊆ Eprocs" "Kp ⊆ Kprocs"
      by blast
    have "(FunctionEntry q, EA_Body q, Statement m) ∈ intra (compile_prog Π ps)"
      using EI sub compile_proc_entry_edge[OF cp0] by auto
    then show ?thesis using that cp0 sub EI KC by blast
  qed
qed
```

Call targets are declared

A call edge of a well-formed program enters only a declared procedure, so every callee has a fragment.

```
lemma compile_calls_target_declared:
  "compile Π p c k n = (n', en, E, K) ⟹ gwf_source_com vars rv Π c
   ⟹ (u, ca, FunctionEntry q, k') ∈ K ⟹ ∃d. Π q = Some d"
proof (induction c arbitrary: k n n' en E K)
  case (Seq c1 c2)
  from Seq.prems(1) obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    and K: "K = K1 ∪ K2"
    by (rule compile_SeqE)
  show ?case using Seq.IH(1)[OF c1] Seq.IH(2)[OF c2] Seq.prems(2,3) K by auto
next
  case (If i b c1 c2)
  from If.prems(1) obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and c2: "compile Π p c2 k (Suc n + csize c1) = (n2, Statement (Suc n + csize c1), E2, K2)"
```

```
    and K: "K = K1 ∪ K2"
    by (rule compile_IfE)
  show ?case using If.IH(1)[OF c1] If.IH(2)[OF c2] If.prems(2,3) K by auto
next
  case (While i b c)
  from While.prems(1) obtain n1 E1 K1 where
      c1: "compile Π p c (Statement n) (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and K: "K = K1"
    by (rule compile_WhileE)
  show ?case using While.IH[OF c1] While.prems(2,3) K by auto
qed (auto split: option.splits)

lemma compile_prog_calls_target_declared:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and e: "(u, ca, FunctionEntry q, k) ∈ calls (compile_prog Π ps)"
  shows "∃d. Π q = Some d"
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (⦇formals = [], body = (main_body Π)⦈) n1
                = (n2, Emain, Kmain)"
    and KC: "calls (compile_prog Π ps) = Kprocs ∪ Kmain"
    by (rule compile_prog_intra_split)
  from e KC consider (P) "(u, ca, FunctionEntry q, k) ∈ Kprocs"
    | (M) "(u, ca, FunctionEntry q, k) ∈ Kmain"
    by auto
  then show ?thesis
  proof cases
    case P
    from compile_procs_calls_origin[OF procs P] obtain r decl m m' Ep Kp where
        decl: "Π r = Some decl" and cp: "compile_proc Π r decl m = (m', Ep, Kp)"
      and eK: "(u, ca, FunctionEntry q, k) ∈ Kp"
      by blast
    from cp obtain Eb where
        cb: "compile Π r (body decl) (Statement (m + csize (body decl))) m
               = (m + csize (body decl), Statement m, Eb, Kp)"
      by (rule compile_procE)
    have "gwf_source_com vars rv Π (body decl)"
      using wf_compile_inputD(5)[OF wf decl] by (simp add: gwf_proc_decl_def)
    from compile_calls_target_declared[OF cb this eK] show ?thesis .
  next
    case M
    from mainc obtain Eb where
        cb: "compile Π prog_main_name (main_body Π) (Statement (n1 + csize (main_body Π))) n1
               = (n1 + csize (main_body Π), Statement n1, Eb, Kmain)"
      by (rule compile_procE) simp
    from compile_calls_target_declared[OF cb wf_compile_inputD(3)[OF wf] M] show ?thesis .
  qed
qed
```

Every compiled node reaches the continuation or the result

A fragment's nodes all lead out of it: to its continuation or, along an explicit `Return`, to the procedure result. `compile_reaches` states this for the entry; the lemmas below extend it to every edge source and target.

```
lemma compile_src_reaches:
  assumes "compile Π p c k n = (n', en, E, K)" and "E ⊆ intra g" and "K ⊆ calls g"
    and "(u, a, v) ∈ E ∨ (u, ca, ce, v) ∈ K"
  shows "local_reaches g u k ∨ local_reaches g u (FunctionResult p)"
  using assms
proof (induction c arbitrary: k n n' en E K u a v ca ce)
  case (Seq c1 c2)
  from Seq.prems(1) obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    and E: "E = E1 ∪ E2" and K: "K = K1 ∪ K2"
    by (rule compile_SeqE)
  have sub: "E1 ⊆ intra g" "K1 ⊆ calls g" "E2 ⊆ intra g" "K2 ⊆ calls g"
    using Seq.prems(2,3) E K by auto
  have mid: "local_reaches g (Statement (n + csize c1)) k
             ∨ local_reaches g (Statement (n + csize c1)) (FunctionResult p)"
    by (rule compile_reaches[OF c2 sub(3,4)])
  from Seq.prems(4) E K consider "(u, a, v) ∈ E1 ∨ (u, ca, ce, v) ∈ K1"
    | "(u, a, v) ∈ E2 ∨ (u, ca, ce, v) ∈ K2"
    by blast
  then show ?case
```

```
  proof cases
    case 1
    from Seq.IH(1)[OF c1 sub(1,2) 1] mid show ?thesis
      by (blast intro: local_reaches_trans)
  next
    case 2
    from Seq.IH(2)[OF c2 sub(3,4) 2] show ?thesis .
  qed
next
  case (If i b c1 c2)
  from If.prems(1) obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and c2: "compile Π p c2 k (Suc n + csize c1) = (n2, Statement (Suc n + csize c1), E2, K2)"
    and E: "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)),
                (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1))} ∪ (if is_SKIP
c1 then {} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    and K: "K = K1 ∪ K2"
    by (rule compile_IfE)
  have subK: "K1 ⊆ calls g" "K2 ⊆ calls g" using If.prems(3) K by auto
  have subE1: "¬ is_SKIP c1 ⟹ E1 ⊆ intra g" and subE2: "¬ is_SKIP c2 ⟹ E2 ⊆ intra g"
    using If.prems(2) E by auto
  have K1_skip: "is_SKIP c1 ⟹ K1 = {}" using c1 by auto
  have K2_skip: "is_SKIP c2 ⟹ K2 = {}" using c2 by auto
  have e1: "¬ is_SKIP c1 ⟹ local_reaches g (Statement (Suc n)) k
            ∨ local_reaches g (Statement (Suc n)) (FunctionResult p)"
    using compile_reaches[OF c1 subE1 subK(1)] by blast
  have e2: "¬ is_SKIP c2 ⟹ local_reaches g (Statement (Suc n + csize c1)) k
            ∨ local_reaches g (Statement (Suc n + csize c1)) (FunctionResult p)"
    using compile_reaches[OF c2 subE2 subK(2)] by blast
  have head: "local_reaches g (Statement n) k ∨ local_reaches g (Statement n) (FunctionResult p)"
  proof (cases "is_SKIP c1")
    case True
    then show ?thesis using If.prems(2) E by (auto intro: local_reaches_intra_step)
  next
    case False
    have "(Statement n, EA_Assume b, Statement (Suc n)) ∈ intra g"
      using If.prems(2) E False by auto
    with e1[OF False] show ?thesis by (blast intro: local_reaches_intra_step)
  qed
  from If.prems(4) E K consider "u = Statement n"
    | "¬ is_SKIP c1" "(u, a, v) ∈ E1 ∨ (u, ca, ce, v) ∈ K1"
    | "¬ is_SKIP c2" "(u, a, v) ∈ E2 ∨ (u, ca, ce, v) ∈ K2"
    using K1_skip K2_skip by (auto split: if_splits)
  then show ?case
  proof cases
    case 1 then show ?thesis using head by simp
  next
    case 2 from If.IH(1)[OF c1 subE1[OF 2(1)] subK(1) 2(2)] show ?thesis .
  next
    case 3 from If.IH(2)[OF c2 subE2[OF 3(1)] subK(2) 3(2)] show ?thesis .
  qed
next
  case (While i b c)
  from While.prems(1) obtain n1 E1 K1 where
      c1: "compile Π p c (Statement n) (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and E: "E = {(Statement n, EA_Assume b, Statement (Suc n)),
                (Statement n, EA_AssumeNot b, k)} ∪ E1"
    and K: "K = K1"
    by (rule compile_WhileE)
  have sub: "E1 ⊆ intra g" "K1 ⊆ calls g" using While.prems(2,3) E K by auto
  have exit: "local_reaches g (Statement n) k"
    using While.prems(2) E by (auto intro: local_reaches_intra_step)
  from While.prems(4) E K consider "u = Statement n" | "(u, a, v) ∈ E1 ∨ (u, ca, ce, v) ∈ K1"
    by blast
  then show ?case
  proof cases
    case 1 then show ?thesis using exit by simp
  next
    case 2
    from While.IH[OF c1 sub 2] exit show ?thesis by (blast intro: local_reaches_trans)
  qed
qed (auto split: option.splits intro: local_reaches_intra_step local_reaches_comb_step)

lemma compile_entry_src:
  assumes "compile Π p c k n = (n', en, E, K)"
  shows "(∃a w. (Statement n, a, w) ∈ E) ∨ (∃ca ce w. (Statement n, ca, ce, w) ∈ K)"
```

```
  using assms
proof (induction c arbitrary: k n n' en E K)
  case (Seq c1 c2)
  from Seq.prems obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    and "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    and E: "E = E1 ∪ E2" and K: "K = K1 ∪ K2"
    by (rule compile_SeqE)
  from Seq.IH(1)[OF c1] E K show ?case by blast
qed (auto simp: Let_def split: prod.splits option.splits)

lemma compile_tgt_src:
  assumes "compile Π p c k n = (n', en, E, K)"
    and "(u, a, v) ∈ E ∨ (u, ca, ce, v) ∈ K"
  shows "v = k ∨ v = FunctionResult p
        ∨ (∃a' w. (v, a', w) ∈ E) ∨ (∃ca' ce' w. (v, ca', ce', w) ∈ K)"
  using assms
proof (induction c arbitrary: k n n' en E K u a v ca ce)
  case (Seq c1 c2)
  from Seq.prems(1) obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    and E: "E = E1 ∪ E2" and K: "K = K1 ∪ K2"
    by (rule compile_SeqE)
  from Seq.prems(2) E K consider "(u, a, v) ∈ E1 ∨ (u, ca, ce, v) ∈ K1"
    | "(u, a, v) ∈ E2 ∨ (u, ca, ce, v) ∈ K2"
    by blast
  then show ?case
  proof cases
    case 1
    from Seq.IH(1)[OF c1 1] compile_entry_src[OF c2] E K show ?thesis by blast
  next
    case 2
    from Seq.IH(2)[OF c2 2] E K show ?thesis by blast
  qed
next
  case (If i b c1 c2)
  from If.prems(1) obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and c2: "compile Π p c2 k (Suc n + csize c1) = (n2, Statement (Suc n + csize c1), E2, K2)"
    and E: "E = {(Statement n, EA_Assume b, if is_SKIP c1 then k else Statement (Suc n)),
                (Statement n, EA_AssumeNot b, if is_SKIP c2 then k else Statement (Suc n + csize c1))} ∪ (if is_SKIP
c1 then {} else E1) ∪ (if is_SKIP c2 then {} else E2)"
    and K: "K = K1 ∪ K2"
    by (rule compile_IfE)
  have K1_skip: "is_SKIP c1 ⟹ K1 = {}" using c1 by auto
  have K2_skip: "is_SKIP c2 ⟹ K2 = {}" using c2 by auto
  have s1:
    "¬ is_SKIP c1 ⟹ (∃a w. (Statement (Suc n), a, w) ∈ E) ∨ (∃ca ce w. (Statement (Suc n), ca, ce, w) ∈ K)"
    using compile_entry_src[OF c1] E K by auto
  have s2: "¬ is_SKIP c2 ⟹ (∃a w. (Statement (Suc n + csize c1), a, w) ∈ E)
            ∨ (∃ca ce w. (Statement (Suc n + csize c1), ca, ce, w) ∈ K)"
    using compile_entry_src[OF c2] E K by auto
  from If.prems(2) E K consider
      "u = Statement n" "v = (if is_SKIP c1 then k else Statement (Suc n))"
    | "u = Statement n" "v = (if is_SKIP c2 then k else Statement (Suc n + csize c1))"
    | "¬ is_SKIP c1" "(u, a, v) ∈ E1 ∨ (u, ca, ce, v) ∈ K1"
    | "¬ is_SKIP c2" "(u, a, v) ∈ E2 ∨ (u, ca, ce, v) ∈ K2"
    using K1_skip K2_skip by (auto split: if_splits)
  then show ?case
  proof cases
    case 1 then show ?thesis using s1 by (auto split: if_splits)
  next
    case 2 then show ?thesis using s2 by (auto split: if_splits)
  next
    case 3 from If.IH(1)[OF c1 3(2)] E K 3(1) show ?thesis by auto
  next
    case 4 from If.IH(2)[OF c2 4(2)] E K 4(1) show ?thesis by auto
  qed
next
  case (While i b c)
  from While.prems(1) obtain n1 E1 K1 where
      c1: "compile Π p c (Statement n) (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and E: "E = {(Statement n, EA_Assume b, Statement (Suc n)),
                (Statement n, EA_AssumeNot b, k)} ∪ E1"
    and K: "K = K1"
```

```
    by (rule compile_WhileE)
  have s1: "(∃a w. (Statement (Suc n), a, w) ∈ E) ∨ (∃ca ce w. (Statement (Suc n), ca, ce, w) ∈ K)"
    using compile_entry_src[OF c1] E K by auto
  from While.prems(2) E K consider "u = Statement n" "v = Statement (Suc n)"
    | "u = Statement n" "v = k" | "(u, a, v) ∈ E1 ∨ (u, ca, ce, v) ∈ K1"
    by blast
  then show ?case
  proof cases
    case 1 then show ?thesis using s1 by simp
  next
    case 2 then show ?thesis by simp
  next
    case 3 from While.IH[OF c1 3] E K show ?thesis by blast
  qed
qed (auto split: option.splits)
```

A procedure's nodes

The endpoints of a procedure's own intra edges and call continuations. Sources reach the continuation or the result by `compile_src_reaches`; a target is the continuation, the result, or itself a source (`compile_tgt_src`); and the continuation of a procedure body is the epilogue, whose return edge is unconditional.

```
definition proc_nodes ::
  "(cfg_node × ('v, 'e) gedge_action × cfg_node) set
   ⇒ (cfg_node × ('v, 'e) gcall_action × cfg_node × cfg_node) set ⇒ cfg_node set"
where
  "proc_nodes E K =
     {u. ∃a v. (u, a, v) ∈ E} ∪ {v. ∃u a. (u, a, v) ∈ E}
   ∪ {u. ∃ca ce k. (u, ca, ce, k) ∈ K} ∪ {k. ∃u ca ce. (u, ca, ce, k) ∈ K}"

lemma compile_proc_node_reaches:
  assumes cp: "compile_proc Π q d m = (m', Ep, Kp)"
    and Ep: "Ep ⊆ intra g" and Kp: "Kp ⊆ calls g"
    and x: "x ∈ proc_nodes Ep Kp"
  shows "local_reaches g x (FunctionResult q)"
proof -
  let ?r = "Statement (m + csize (body d))"
  from cp obtain Eb where
      cb: "compile Π q (body d) ?r m = (m + csize (body d), Statement m, Eb, Kp)"
    and E: "Ep = insert (FunctionEntry q, EA_Body q, Statement m)
                 (insert (?r, EA_Ret None q, FunctionResult q) Eb)"
    by (rule compile_procE)
  have Eb: "Eb ⊆ intra g" using E Ep by auto
  have epi: "local_reaches g ?r (FunctionResult q)"
    using E Ep by (auto intro: local_reaches_intra_step)
  have srcE: "local_reaches g u (FunctionResult q)" if "(u, a, v) ∈ Eb" for u a v
    using compile_src_reaches[OF cb Eb Kp, of u a v] that epi local_reaches_trans by blast
  have srcK: "local_reaches g u (FunctionResult q)" if "(u, ca, ce, v) ∈ Kp" for u ca ce v
    using compile_src_reaches[OF cb Eb Kp, of u _ v ca ce] that epi local_reaches_trans by blast
  have src_of: "local_reaches g v (FunctionResult q)"
    if
      "v = ?r ∨ v = FunctionResult q ∨ (∃a' w. (v, a', w) ∈ Eb) ∨ (∃ca' ce' w. (v, ca', ce', w) ∈ Kp)"
    for v
    using that epi srcE srcK by auto
  have tgtE: "local_reaches g v (FunctionResult q)" if "(u, a, v) ∈ Eb" for u a v
    using compile_tgt_src[OF cb, of u a v] that src_of by blast
  have tgtK: "local_reaches g v (FunctionResult q)" if "(u, ca, ce, v) ∈ Kp" for u ca ce v
    using compile_tgt_src[OF cb, of u _ v ca ce] that src_of by blast
  have body: "local_reaches g (Statement m) (FunctionResult q)"
    using compile_entry_src[OF cb] srcE srcK by blast
  have ent: "local_reaches g (FunctionEntry q) (FunctionResult q)"
    by (rule compile_proc_reaches_result[OF cp Ep Kp])
  from x show ?thesis
    unfolding proc_nodes_def E using srcE srcK tgtE tgtK epi body ent by auto
qed

lemma proc_nodes_pfn:
  assumes cp: "compile_proc Π q d m = (m', Ep, Kp)" and x: "x ∈ proc_nodes Ep Kp"
  shows "x ∈ pfn q m m'"
  using x compile_proc_intra_pfn[OF cp] compile_proc_calls_pfn[OF cp]
  unfolding proc_nodes_def by blast

definition prog_node :: "('a, 'v, 'e) gptable ⇒ pname list ⇒ pname ⇒ cfg_node ⇒ bool" where
  "prog_node Π ps q x ⟷
     (∃d m m' Ep Kp. Π q = Some d ∧ compile_proc Π q d m = (m', Ep, Kp)
        ∧ (FunctionEntry q, EA_Body q, Statement m) ∈ intra (compile_prog Π ps)
```

```
        ∧ Ep ⊆ intra (compile_prog Π ps) ∧ Kp ⊆ calls (compile_prog Π ps)
        ∧ x ∈ proc_nodes Ep Kp)"

lemma prog_node_entry:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps" and decl: "Π q = Some d"
  shows "prog_node Π ps q (FunctionEntry q)"
proof -
  from compile_prog_proc_frag_sub[OF wf decl] obtain m m' Ep Kp where
      cp: "compile_proc Π q d m = (m', Ep, Kp)"
    and ent: "(FunctionEntry q, EA_Body q, Statement m) ∈ intra (compile_prog Π ps)"
    and sub: "Ep ⊆ intra (compile_prog Π ps)" "Kp ⊆ calls (compile_prog Π ps)"
    by blast
  have "FunctionEntry q ∈ proc_nodes Ep Kp"
    using compile_proc_entry_edge[OF cp] unfolding proc_nodes_def by blast
  then show ?thesis unfolding prog_node_def using decl cp ent sub by blast
qed

lemma prog_node_result:
  "prog_node Π ps q x ⟹ prog_node Π ps q (FunctionResult q)"
proof -
  assume "prog_node Π ps q x"
  then obtain d m m' Ep Kp where
      decl: "Π q = Some d" and cp: "compile_proc Π q d m = (m', Ep, Kp)"
    and ent: "(FunctionEntry q, EA_Body q, Statement m) ∈ intra (compile_prog Π ps)"
    and sub: "Ep ⊆ intra (compile_prog Π ps)" "Kp ⊆ calls (compile_prog Π ps)"
    unfolding prog_node_def by auto
  from cp have "(Statement (m + csize (body d)), EA_Ret None q, FunctionResult q) ∈ Ep"
    by (rule compile_procE) simp
  then have "FunctionResult q ∈ proc_nodes Ep Kp" unfolding proc_nodes_def by blast
  then show ?thesis unfolding prog_node_def using decl cp ent sub by blast
qed

lemma prog_node_reaches:
  "prog_node Π ps q x ⟹ local_reaches (compile_prog Π ps) x (FunctionResult q)"
  unfolding prog_node_def using compile_proc_node_reaches by auto

lemma prog_node_intra:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and u: "prog_node Π ps q u" and e: "(u, a, v) ∈ intra (compile_prog Π ps)"
  shows "prog_node Π ps q v"
proof -
  from u obtain d m m' Ep Kp where
      decl: "Π q = Some d" and cp: "compile_proc Π q d m = (m', Ep, Kp)"
    and ent: "(FunctionEntry q, EA_Body q, Statement m) ∈ intra (compile_prog Π ps)"
    and sub: "Ep ⊆ intra (compile_prog Π ps)" "Kp ⊆ calls (compile_prog Π ps)"
    and un: "u ∈ proc_nodes Ep Kp"
    unfolding prog_node_def by auto
  have "(u, a, v) ∈ Ep"
    using compile_prog_entry_frag[OF wf decl cp ent] proc_nodes_pfn[OF cp un] e by blast
  then have "v ∈ proc_nodes Ep Kp" unfolding proc_nodes_def by blast
  then show ?thesis unfolding prog_node_def using decl cp ent sub by blast
qed

lemma prog_node_calls:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and u: "prog_node Π ps q u" and e: "(u, ca, ce, k) ∈ calls (compile_prog Π ps)"
  shows "prog_node Π ps q k"
proof -
  from u obtain d m m' Ep Kp where
      decl: "Π q = Some d" and cp: "compile_proc Π q d m = (m', Ep, Kp)"
    and ent: "(FunctionEntry q, EA_Body q, Statement m) ∈ intra (compile_prog Π ps)"
    and sub: "Ep ⊆ intra (compile_prog Π ps)" "Kp ⊆ calls (compile_prog Π ps)"
    and un: "u ∈ proc_nodes Ep Kp"
    unfolding prog_node_def by auto
  have "(u, ca, ce, k) ∈ Kp"
    using compile_prog_entry_frag[OF wf decl cp ent] proc_nodes_pfn[OF cp un] e by blast
  then have "k ∈ proc_nodes Ep Kp" unfolding proc_nodes_def by blast
  then show ?thesis unfolding prog_node_def using decl cp ent sub by blast
qed

lemma prog_node_main_entry:
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
  shows "prog_node Π ps prog_main_name (cfg_entry (compile_prog Π ps))"
  using prog_node_entry[OF wf wf_compile_inputD(2)[OF wf]] by simp

lemma prog_node_callee_entry:
```

```
  assumes wf: "gwf_compile_input vars rv 𝒢 Π ps"
    and e: "(u, ca, FunctionEntry q, k) ∈ calls (compile_prog Π ps)"
  shows "prog_node Π ps q (FunctionEntry q)"
  using compile_prog_calls_target_declared[OF wf e] prog_node_entry[OF wf] by blast

end
```

5.11 Source_To_Trace

```
theory Source_To_Trace
  imports Procedure_Ownership Simulation_Preservation "Voblint_CFG.Activation_Trace_Collect"
begin
```

A source run is an activation trace

What the session is for. A graph configuration is tied to a valid activation trace: the activation currently running *is* the trace, and the runtime frame stack is exactly its chain of callers. Each `cstep` rule lines up with one `valid_activation_trace` constructor, so advancing the graph by an edge extends the trace by a step.

Composed with the simulation, this turns a run of the source program into a trace, and so turns any store the source can reach into a member of the collecting semantics at that node. Those are the statements the soundness session consumes; everything else in this session exists to make them true.

The representation invariant

`stack_repr` walks `caller_of` in lockstep with the runtime frame list: each `cframe (cont, dst, s)` pins the caller activation `caller` — its frozen store `sink_store caller = s` — and records the concrete `calls` edge that spawned the child, whose callee `FunctionEntry p` equals the child's (path-invariant) entry node.

```
inductive stack_repr :: "(var_id, exp) gcfg ⇒ cframe list ⇒ activation_trace ⇒ bool" for g where
  empty [intro]: "caller_of t = None ⟹ stack_repr g [] t"
| frame [intro]: "caller_of t = Some caller ⟹ sink_store caller = s
          ⟹ fst (hd (path_of t)) = FunctionEntry p
          ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
          ⟹ stack_repr g stk caller
          ⟹ stack_repr g ((cont, dst, s) # stk) t"
```

Inversion by the shape of the runtime stack. A frame's clause recurses on the shorter stack, so it stays plain `[elim]`; an empty stack has only one clause and is `[elim!]`.

```
inductive_cases stack_repr_NilE [elim!]: "stack_repr g [] t"
inductive_cases stack_repr_ConsE [elim]: "stack_repr g (cf # stk) t"
```

`activation_trace_repr` pins a valid activation trace to a located configuration: the trace's sink is the current node/store, and its caller chain is the runtime stack.

```
definition activation_trace_repr ::
  "kenv ⇒ (var_id, exp) gcfg ⇒ store set ⇒ cconf ⇒ activation_trace ⇒ bool" where
  "activation_trace_repr κ g S cf t = (case cf of (v, s, stk) ⇒
     t ∈ 𝒯_{κ,g,S} ∧ sink_node t = v ∧ sink_store t = s ∧ stack_repr g stk t)"

definition located_activation_trace :: "kenv ⇒ (var_id, exp) gcfg ⇒ store set ⇒ cconf ⇒ bool" where
  "located_activation_trace κ g S cf = (∃t. activation_trace_repr κ g S cf t)"
```

`stack_repr` reads only the top `caller_of` and the path-invariant entry node, so it transfers across activations sharing both (an `Activation_Trace_Def.extend` appends — entry unchanged — and a `Resume` prepends the same activation's path — entry unchanged).

```
lemma stack_repr_cong:
  "stack_repr g stk t1 ⟹ caller_of t2 = caller_of t1
   ⟹ fst (hd (path_of t2)) = fst (hd (path_of t1)) ⟹ stack_repr g stk t2"
  by (cases rule: stack_repr.cases) auto
```

The return step

The load-bearing case. A `cstep` return pops the top frame; `stack_repr` identifies the caller `c` (`caller_of` recovers it) and its spawning `calls` edge, whose callee `FunctionEntry q` matches the returning `FunctionResult q` by `valid_activation_trace_entry_result_eq` — so the trace composes by `valid_activation_trace.ret`, and the combined store equals the `cstep` store.

```
lemma activation_trace_repr_Return:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and rep: "activation_trace_repr κ (compile_prog Π ps) S
                (FunctionResult q, tst, (cont, dst, caller) # stk) t0"
  shows "activation_trace_repr κ (compile_prog Π ps) S
          (cont, combine_collect κ dst caller tst, stk)
          (Resume (the (caller_of t0)) t0
             (path_of (the (caller_of t0)) @ [(cont, combine_collect κ dst caller tst)]))"
proof -
  let ?g = "compile_prog Π ps"
```

```
  from rep
  have t0v: "t0 ∈ 𝒯_κ,?g,S" and sn0: "sink_node t0 = FunctionResult q"
    and ss0: "sink_store t0 = tst"
    and stk0: "stack_repr ?g ((cont, dst, caller) # stk) t0"
    by (auto simp: activation_trace_repr_def)
  from stk0 obtain c p pars args where cof: "caller_of t0 = Some c"
    and ssc: "sink_store c = caller" and entryp: "fst (hd (path_of t0)) = FunctionEntry p"
    and edge: "(sink_node c, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls ?g"
    and stkc: "stack_repr ?g stk c"
    by (cases rule: stack_repr.cases) auto
  have cvalid: "c ∈ 𝒯_κ,?g,S"
    using valid_activation_trace_caller_valid[OF t0v cof] .
  have pq: "p = q" using valid_activation_trace_entry_result_eq[OF wf t0v entryp sn0] .
  have cthe: "the (caller_of t0) = c" using cof by simp
  let ?r = "combine_collect κ dst caller tst"
  let ?t' = "Resume c t0 (path_of c @ [(cont, ?r)])"
  have edge_q: "(sink_node c, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls ?g"
    using edge pq by simp
  have valid': "?t' ∈ 𝒯_κ,?g,S"
    using valid_activation_trace.ret[OF t0v cof sn0 edge_q] ssc ss0 by simp
  have sn': "sink_node ?t' = cont" by (simp add: sink_node_def)
  have ss': "sink_store ?t' = ?r" by (simp add: sink_store_def)
  have entry': "fst (hd (path_of ?t')) = fst (hd (path_of c))"
    using valid_activation_trace_path_nonempty[OF cvalid] by simp
  have stk': "stack_repr ?g stk ?t'"
    using stack_repr_cong[OF stkc, of ?t'] entry' by simp
  show ?thesis
    unfolding cthe using valid' sn' ss' stk' by (simp add: activation_trace_repr_def)
qed
```

The invariant is preserved by located CFG steps

Each `cstep` rule maps to one `valid_activation_trace` constructor: intra ↦ `Activation_Trace_Def.extend`, call ↦ `activation_trace.Call`, return ↦ `Resume` (`activation_trace_repr_Return`).

```
lemma cstep_preserves_activation_trace_repr:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and step: "κ,(compile_prog Π ps) ⊢ cf →_c cf'"
    and rep: "activation_trace_repr κ (compile_prog Π ps) S cf t"
  shows "∃t'. activation_trace_repr κ (compile_prog Π ps) S cf' t'"
  using step rep
proof (cases rule: cstep.cases)
  case (Intra u a v s' s stk')
  let ?g = "compile_prog Π ps"
  from rep have tv: "t ∈ 𝒯_κ,?g,S" and sn: "sink_node t = u"
    and ss: "sink_store t = s"
    and str: "stack_repr ?g stk' t"
    using Intra by (auto simp: activation_trace_repr_def)
  have e1: "(sink_node t, a, v) ∈ intra ?g" using Intra sn by simp
  have e2: "s' ∈ edge_step κ a (sink_store t)" using Intra ss by simp
  have "extend t (v, s') ∈ 𝒯_κ,?g,S"
    using valid_activation_trace.intra[OF tv e1 e2] .
  moreover have "stack_repr ?g stk' (extend t (v, s'))"
    using stack_repr_cong[OF str] valid_activation_trace_path_nonempty[OF tv] by simp
  ultimately have "activation_trace_repr κ ?g S (v, s', stk') (extend t (v, s'))"
    by (simp add: activation_trace_repr_def)
  then show ?thesis using Intra by auto
next
  case (Call u dst pars actuals q cont s stk')
  let ?g = "compile_prog Π ps"
  from rep have tv: "t ∈ 𝒯_κ,?g,S" and sn: "sink_node t = u"
    and ss: "sink_store t = s"
    and str: "stack_repr ?g stk' t"
    using Call by (auto simp: activation_trace_repr_def)
  have edge: "(sink_node t, CallEdge dst pars actuals, FunctionEntry q, cont) ∈ calls ?g"
    using Call sn by simp
  let ?child = "Call t [(FunctionEntry q, call_enter κ (CallEdge dst pars actuals) s)]"
  have child_valid: "?child ∈ 𝒯_κ,?g,S"
    using valid_activation_trace.call[OF tv edge] ss by simp
  have "stack_repr ?g ((cont, dst, s) # stk') ?child"
  proof (rule stack_repr.frame)
    show "caller_of ?child = Some t" by simp
    show "sink_store t = s" using ss .
    show "fst (hd (path_of ?child)) = FunctionEntry q" by simp
    show "(sink_node t, CallEdge dst pars actuals, FunctionEntry q, cont) ∈ calls ?g"
      using edge .
```

```
    show "stack_repr ?g stk' t" using str .
  qed
  with child_valid have "activation_trace_repr κ ?g S
      (FunctionEntry q, call_enter κ (CallEdge dst pars actuals) s,
       (cont, dst, s) # stk') ?child"
    by (simp add: activation_trace_repr_def sink_node_def sink_store_def)
  then show ?thesis using Call by auto
next
  case (Return q tst cont dst caller stk')
  from rep Return(1)
  have rep': "activation_trace_repr κ (compile_prog Π ps) S
                (FunctionResult q, tst, (cont, dst, caller) # stk') t" by simp
  from activation_trace_repr_Return[OF wf rep'] show ?thesis using Return(2) by auto
qed

lemma located_activation_trace_entry:
  assumes "s ∈ S"
  shows "located_activation_trace κ g S (cfg_entry g, s, [])"
proof -
  have "activation_trace_repr κ g S (cfg_entry g, s, []) (Root [(cfg_entry g, s)])"
    using assms
    by (auto simp: activation_trace_repr_def sink_node_def sink_store_def
      valid_activation_trace.root)
  then show ?thesis by (auto simp: located_activation_trace_def)
qed

lemma cstep_preserves_located_activation_trace:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and "located_activation_trace κ (compile_prog Π ps) S cf"
    and "κ,(compile_prog Π ps) ⊢ cf →c cf'"
  shows "located_activation_trace κ (compile_prog Π ps) S cf'"
proof -
  from assms(2) obtain t where "activation_trace_repr κ (compile_prog Π ps) S cf t"
    by (auto simp: located_activation_trace_def)
  then obtain t' where "activation_trace_repr κ (compile_prog Π ps) S cf' t'"
    using cstep_preserves_activation_trace_repr[OF wf assms(3)] by blast
  then show ?thesis by (auto simp: located_activation_trace_def)
qed

lemma csteps_preserve_located_activation_trace:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and "located_activation_trace κ (compile_prog Π ps) S cf"
    and "κ,(compile_prog Π ps) ⊢ cf →c* cf'"
  shows "located_activation_trace κ (compile_prog Π ps) S cf'"
  using assms(3) assms(2)
proof (induction rule: star.induct)
  case (refl a)
  then show ?case .
next
  case (step a b c)
  have "located_activation_trace κ (compile_prog Π ps) S b"
    by (rule cstep_preserves_located_activation_trace[OF wf step.prems step.hyps(1)])
  then show ?case by (rule step.IH)
qed
```

The initial main activation

The program entry `FunctionEntry prog_main_name` is an ordinary `csim.Base` activation: the distinguished main procedure is declared in Π (`wf_compile_input`), so its body fragment is certified by `procs_embedded_compile_prog`. One `EA_Body` edge crosses from `FunctionEntry prog_main_name` to the body entry `en`, where the `Base` activation simulates the source `main_body Π`.

```
lemma compile_prog_main_base:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
  obtains en where
    "(FunctionEntry prog_main_name, EA_Body prog_main_name, en) ∈ intra (compile_prog Π ps)"
    "Π, compile_prog Π ps, prog_main_name ⊢ (main_body Π, s, []) ≈ (en, s, [])"
proof -
  let ?g = "compile_prog Π ps"
  have pc: "procs_embedded Π ?g" by (rule procs_embedded_compile_prog[OF wf])
  have main_decl: "Π prog_main_name = Some (⦇formals = [], body = (main_body Π)⦈)"
    by (rule wf_compile_inputD(2)[OF wf])
  obtain k m en where
    cacc: "compiled_at Π ?g prog_main_name (body (⦇formals = [], body = (main_body Π)⦈)) k m"
      and ctrl: "control_at Π prog_main_name (body (⦇formals = [], body = (main_body Π)⦈)) k m
                 (body (⦇formals = [], body = (main_body Π)⦈)) en"
```

```
    and entry: "(FunctionEntry prog_main_name, EA_Body prog_main_name, en) ∈ intra ?g"
    by (rule procs_embedded_activation[OF pc main_decl])
  have bodyeq: "body (⦇formals = [], body = (main_body Π)⦈) = (main_body Π)" by simp
  have base: "Π, ?g, prog_main_name ⊢ (main_body Π, s, []) ≈ (en, s, [])"
    by (rule csim.Base[OF ctrl[unfolded bodyeq] cacc[unfolded bodyeq]])
  from entry base show ?thesis ..
qed
```

Top-level structural facts of the invariant

A trace has no caller exactly when its activation is at the top level (empty runtime stack).

```
lemma stack_repr_Nil_iff:
  "stack_repr g stk t ⟹ (stk = []) = (caller_of t = None)"
  by (cases rule: stack_repr.cases) auto
```

The source bridge

Composing the initial `csim.Base` with `csim_star` (the source-to-`cstep` simulation) and the located invariant: every source run produces a matching valid activation trace at the simulated node.

```
theorem compile_prog_run_has_activation_trace:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and s0: "s0 ∈ S"
    and run: "κ,Π ⊢ (main_body Π, s0, []) →_p* (residual, s, frs)"
  shows "∃v stk t. Π, compile_prog Π ps, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)
                 ∧ activation_trace_repr κ (compile_prog Π ps) S (v, s, stk) t"
proof -
  let ?g = "compile_prog Π ps"
  have pc: "procs_embedded Π ?g" by (rule procs_embedded_compile_prog[OF wf])
  have swf: "return_safe (main_body Π)" by (rule wf_compile_input_return_safe[OF wf])
  obtain en where entry: "(FunctionEntry prog_main_name, EA_Body prog_main_name, en) ∈ intra ?g"
    and base: "Π, ?g, prog_main_name ⊢ (main_body Π, s0, []) ≈ (en, s0, [])"
    by (rule compile_prog_main_base[OF wf])
  have loc0:
    "located_activation_trace κ (compile_prog Π ps) S (FunctionEntry prog_main_name, s0, [])"
    using located_activation_trace_entry[where g = "compile_prog Π ps", OF s0] by simp
  have step0: "κ,?g ⊢ (FunctionEntry prog_main_name, s0, []) →_c (en, s0, [])"
    by (rule cstep_body[OF entry])
  have loc_en: "located_activation_trace κ ?g S (en, s0, [])"
    by (rule cstep_preserves_located_activation_trace[OF wf loc0 step0])
  from csim_star[OF base pc swf run] obtain cf'
    where run_c: "κ,?g ⊢ (en, s0, []) →_c* cf'"
      and sim': "Π, ?g, prog_main_name ⊢ (residual, s, frs) ≈ cf'"
    by blast
  obtain v s' stk where cf': "cf' = (v, s', stk)" by (cases cf')
  have seq: "s' = s" using csim_store_eq[OF sim'[unfolded cf']] by simp
  have loc_v: "located_activation_trace κ ?g S (v, s, stk)"
    using csteps_preserve_located_activation_trace[OF wf loc_en run_c] cf' seq by simp
  from loc_v obtain t where "activation_trace_repr κ ?g S (v, s, stk) t"
    by (auto simp: located_activation_trace_def)
  then show ?thesis using sim' cf' seq by blast
qed
```

The plain projected source bridge: a reachable source store lies in the activation-trace collecting `activation_collect` at the simulated node, in a context the witness trace carries. A relational policy may leave a trace without any context, so the bridge asks for one: `has_ctx` is what `activation_coverage`'s `TOTAL` buys downstream, and what a functional policy has outright (`source_store_in_activation_collect_of_fun`).

```
theorem source_store_in_activation_collect:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and s0: "s0 ∈ S"
    and run: "κ,Π ⊢ (main_body Π, s0, []) →_p* (residual, s, frs)"
    and has_ctx: "⋀t. t ∈ 𝒯_κ,compile_prog Π ps,S
                  ⟹ ∃c. activation_context_rel κ adm c₀ (compile_prog Π ps) t c"
  shows "∃v stk t c. Π, compile_prog Π ps, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)
                  ∧ activation_trace_repr κ (compile_prog Π ps) S (v, s, stk) t
                  ∧ activation_context_rel κ adm c₀ (compile_prog Π ps) t c
                  ∧ s ∈ 𝒜_κ,adm,c₀,compile_prog Π ps,S v c"
proof -
  let ?g = "compile_prog Π ps"
  from compile_prog_run_has_activation_trace[OF wf s0 run] obtain v stk t
    where sim: "Π, ?g, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)"
      and rep: "activation_trace_repr κ ?g S (v, s, stk) t" by blast
  from rep have tv: "t ∈ 𝒯_κ,?g,S" and sn: "sink_node t = v"
```

```
    and ss: "sink_store t = s"
    by (auto simp: activation_trace_repr_def)
  from has_ctx[OF tv] obtain c where tc: "activation_context_rel κ adm c₀ ?g t c" by blast
  have "s ∈ 𝒜_{κ,adm,c₀,?g,S} v c"
    using activation_collect_I[OF tv sn tc] ss by simp
  then show ?thesis using sim rep tc by blast
qed
```

A functional policy carries `activation_context_of`'s context on every valid activation trace, so it needs no context witness.

```
corollary source_store_in_activation_collect_of_fun:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and s0: "s0 ∈ S"
    and run: "κ,Π ⊢ (main_body Π, s0, []) →_p* (residual, s, frs)"
  shows "∃v stk t c. Π, compile_prog Π ps, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)
                   ∧ activation_trace_repr κ (compile_prog Π ps) S (v, s, stk) t
                   ∧ activation_context_of f c₀ t = c
                   ∧ s ∈ 𝒜_{κ,context_policy_of_fun f,c₀,compile_prog Π ps,S} v c"
proof -
  let ?g = "compile_prog Π ps"
  from compile_prog_run_has_activation_trace[OF wf s0 run] obtain v stk t
    where sim: "Π, ?g, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)"
      and rep: "activation_trace_repr κ ?g S (v, s, stk) t" by blast
  from rep have tv: "t ∈ 𝒯_{κ,?g,S}" and sn: "sink_node t = v"
    and ss: "sink_store t = s"
    by (auto simp: activation_trace_repr_def)
  have tc:
    "activation_context_rel κ (context_policy_of_fun f) c₀ ?g t (activation_context_of f c₀ t)"
    by (simp add: activation_context_rel_of_fun_iff[OF tv])
  have "s ∈ 𝒜_{κ,context_policy_of_fun f,c₀,?g,S} v
             (activation_context_of f c₀ t)"
    using activation_collect_I[OF tv sn tc] ss by simp
  then show ?thesis using sim rep by blast
qed
```

Typed runs reach the typed graph

Every run of the typed program is simulated on the typed graph, and its store is the sink of a valid activation trace there. Both are the bridge above at the elaborated program; nothing about kinds enters the simulation proof.

```
theorem source_run_has_activation_trace:
  assumes wf: "wf_program_compile_input p"
    and s0: "s0 ∈ S"
    and run:
      "κ,(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →_p* (residual, s, frs)"
  shows "∃v stk t. elab_ptable p ds, elab_cfg p ds, prog_main_name
                     ⊢ (residual, s, frs) ≈ (v, s, stk)
                   ∧ activation_trace_repr κ (elab_cfg p ds) S (v, s, stk) t"
  unfolding elab_cfg_def
  by (rule compile_prog_run_has_activation_trace [OF elab_wf_compile_input [OF wf] s0 run])
```

Hence every store a typed run reaches lies in the typed collecting semantics at the node simulating it. This is the inclusion a sound typed analysis result bounds.

```
theorem source_reaches_node_collect:
  assumes wf: "wf_program_compile_input p"
    and s0: "s0 ∈ S"
    and run:
      "κ,(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →_p* (residual, s, frs)"
  shows "∃v stk. elab_ptable p ds, elab_cfg p ds, prog_main_name
                   ⊢ (residual, s, frs) ≈ (v, s, stk)
                 ∧ s ∈ 𝒞_{κ,elab_cfg p ds,S} v"
proof -
  from source_run_has_activation_trace [OF wf s0 run] obtain v stk t
    where sim: "elab_ptable p ds, elab_cfg p ds, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)"
      and rep: "activation_trace_repr κ (elab_cfg p ds) S (v, s, stk) t"
    by blast
  from rep have "s ∈ 𝒞_{κ,elab_cfg p ds,S} v"
    by (auto simp: activation_trace_repr_def node_collect_def)
  with sim show ?thesis by blast
qed

end
```

5.12 Kind_Invariants

```
theory Kind_Invariants
  imports Source_To_Trace
begin
```

Typed stores stay typed

Every identity other than the return slot holds a value of its kind in every store a valid activation trace of the typed graph reaches. Writes to a variable convert to its kind (`kind_conv`), and a call entry resets locals to `0` and binds each formal to its actual converted to the formal's kind. The return slot is shared by procedures returning different kinds, so it is excluded: it holds a value of the returning procedure's kind, which the destination conversion re-establishes at the caller.

```
definition in_kinds :: "kenv ⇒ store ⇒ bool" where
  "in_kinds κ t ⟷ (∀v. v ≠ ReturnVar ⟶ t v ∈ ik_range (κ v))"

lemma in_kinds_update [simp]:
  "in_kinds κ t ⟹ in_kinds κ (t(v := ik_norm (κ v) n))"
  "in_kinds κ t ⟹ in_kinds κ (t(ReturnVar := n))"
  by (auto simp: in_kinds_def simp del: ik_range_iff)

lemma in_kinds_bind_formals:
  "in_kinds κ t ⟹ (⋀x n. (x, n) ∈ set (zip xs ns) ⟹ n ∈ ik_range (κ x))
   ⟹ in_kinds κ (bind_formals xs ns t)"
proof (induction xs arbitrary: ns t)
  case (Cons x xs)
  then show ?case
    by (cases ns) (auto simp: in_kinds_def simp del: ik_range_iff)
qed simp

lemma in_kinds_enter_state: "in_kinds κ t ⟹ in_kinds κ (enter_state is_global t)"
  by (auto simp: in_kinds_def enter_state_def enter_frame_def simp del: ik_range_iff)

lemma in_kinds_combine_env:
  "in_kinds κ s ⟹ in_kinds κ t ⟹ in_kinds κ (combine_env g s t)"
  by (auto simp: in_kinds_def simp del: ik_range_iff)
```

Graphs with typed calls

A call edge passes typed actuals when each actual is well-typed, does not read the return slot, and has the kind of the formal it binds.

```
definition calls_in_kinds :: "kenv ⇒ (var_id, exp) gcfg ⇒ bool" where
  "calls_in_kinds κ g ⟷
     (∀u dst pars es v w. (u, CallEdge dst pars es, v, w) ∈ calls g ⟶
        list_all (λe. wt_exp κ e ∧ ReturnVar ∉ exp_vars e) es ∧ args_ok κ pars es)"

lemma zip_map2_self: "zip xs (map2 f xs ys) = map (λ(x, y). (x, f x y)) (zip xs ys)"
proof (induction xs arbitrary: ys)
  case (Cons x xs) then show ?case by (cases ys) simp_all
qed simp

lemma in_kinds_call_enter:
  assumes t: "in_kinds κ t"
  shows "in_kinds κ (call_enter κ (CallEdge dst pars es) t)"
proof -
  have "n ∈ ik_range (κ x)"
    if "(x, n) ∈ set (zip pars (map2 (λx. kind_conv κ (Some x)) pars (map (λe. eval e t) es)))"
    for x n
    using that by (auto simp: zip_map2_self kind_conv_simps)
  then show ?thesis
    unfolding call_enter_CallEdge
    by (rule in_kinds_bind_formals[OF in_kinds_enter_state[OF t]])
qed
```

A return resumes one of the suspended activations below a trace (`ancestors`), so the invariant is carried for all of them at once.

```
lemma valid_activation_trace_callers_in_kinds:
  assumes vimp: "calls_in_kinds κ g" and S: "⋀s. s ∈ S ⟹ in_kinds κ s"
    and t: "t ∈ 𝒯⟨κ,g,S⟩"
  shows "in_kinds κ (sink_store t) ∧ (∀c ∈ ancestors t. in_kinds κ (sink_store c))"
  using t
proof (induction t rule: valid_activation_trace.induct)
```

```
  case (root s)
  then show ?case by (simp add: S)
next
  case (intra t a v s')
  then show ?case
    by (cases a)
      (auto simp: in_kinds_def kind_conv_simps split: option.splits if_splits simp del:
        ik_range_iff)
next
  case (call caller dst pars args p cont)
  then show ?case by (simp add: in_kinds_call_enter)
next
  case (ret callee caller p dst pars args cont)
  from ret.IH ancestors_caller [OF ret.hyps(2)]
  have "in_kinds κ (sink_store caller)" "∀c ∈ ancestors caller. in_kinds κ (sink_store c)"
    and callee: "in_kinds κ (sink_store callee)"
    by auto
  moreover have "in_kinds κ (combine_collect κ dst (sink_store caller) (sink_store callee))"
    using calculation
    by (cases dst) (auto simp: combine_collect_def in_kinds_combine_env kind_conv_simps)
  ultimately show ?case by (simp add: sink_store_def)
qed

theorem valid_activation_trace_in_kinds:
  assumes "calls_in_kinds κ g" and "⋀s. s ∈ S ⟹ in_kinds κ s"
    and "t ∈ 𝒯κ,g,S"
  shows "in_kinds κ (sink_store t)"
  using valid_activation_trace_callers_in_kinds [OF assms] by blast

theorem node_collect_in_kinds:
  assumes "calls_in_kinds κ g" and "⋀s. s ∈ S ⟹ in_kinds κ s"
    and "s ∈ 𝒞κ,g,S v"
  shows "in_kinds κ s"
  using assms(3) valid_activation_trace_in_kinds [OF assms(1,2)]
  by (auto simp: node_collect_def)
```

The typed graph has typed calls

Every call edge the compiler emits comes from a `activation_trace.Call` in a compiled body, with that call's destination and actuals and the callee's formals.

```
fun com_calls :: "('a, 'v, 'e) gcom ⇒ ('v option × pname × 'e list) set" where
  "com_calls (gcom.Call _ dst q es) = {(dst, q, es)}"
| "com_calls (Seq c1 c2) = com_calls c1 ∪ com_calls c2"
| "com_calls (If _ _ c1 c2) = com_calls c1 ∪ com_calls c2"
| "com_calls (While _ _ c) = com_calls c"
| "com_calls _ = {}"

lemma compile_calls_from:
  "compile Π p c k n = (n', en, E, K) ⟹ (u, CallEdge dst pars es, v, w) ∈ K
   ⟹ ∃q. (dst, q, es) ∈ com_calls c ∧ pars = call_formals Π q"
proof (induction c arbitrary: k n n' en E K)
  case (Seq c1 c2)
  from Seq.prems(1) obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 (Statement (n + csize c1)) n = (n1, Statement n, E1, K1)"
    and c2: "compile Π p c2 k (n + csize c1) = (n2, Statement (n + csize c1), E2, K2)"
    and K: "K = K1 ∪ K2"
    by (rule compile_SeqE)
  show ?case using Seq.IH(1)[OF c1] Seq.IH(2)[OF c2] Seq.prems(2) K by auto
next
  case (If i b c1 c2)
  from If.prems(1) obtain n1 E1 K1 n2 E2 K2 where
      c1: "compile Π p c1 k (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and c2: "compile Π p c2 k (Suc n + csize c1) = (n2, Statement (Suc n + csize c1), E2, K2)"
    and K: "K = K1 ∪ K2"
    by (rule compile_IfE)
  show ?case using If.IH(1)[OF c1] If.IH(2)[OF c2] If.prems(2) K by auto
next
  case (While i b c)
  from While.prems(1) obtain n1 E1 K1 where
      c1: "compile Π p c (Statement n) (Suc n) = (n1, Statement (Suc n), E1, K1)"
    and K: "K = K1"
    by (rule compile_WhileE)
  show ?case using While.IH[OF c1] While.prems(2) K by auto
qed (auto split: option.splits simp del: call_formals_def)
```

```
lemma compile_prog_calls_from:
  assumes main: "Π prog_main_name = Some (|formals = [], body = main_body Π|)"
    and e: "(u, CallEdge dst pars es, v, w) ∈ calls (compile_prog Π ps)"
  shows "∃r d q. Π r = Some d ∧ (dst, q, es) ∈ com_calls (body d) ∧ pars = call_formals Π q"
proof -
  obtain n1 Eprocs Kprocs n2 Emain Kmain where
      procs: "compile_procs Π ps 0 = (n1, Eprocs, Kprocs)"
    and mainc: "compile_proc Π prog_main_name (|formals = [], body = main_body Π|) n1
                = (n2, Emain, Kmain)"
    and KC: "calls (compile_prog Π ps) = Kprocs ∪ Kmain"
    by (rule compile_prog_intra_split)
  from e KC consider (P) "(u, CallEdge dst pars es, v, w) ∈ Kprocs"
    | (M) "(u, CallEdge dst pars es, v, w) ∈ Kmain"
    by auto
  then show ?thesis
  proof cases
    case P
    from compile_procs_calls_origin [OF procs P] obtain r d m m' Ep Kp where
        d: "Π r = Some d" and cp: "compile_proc Π r d m = (m', Ep, Kp)"
      and eK: "(u, CallEdge dst pars es, v, w) ∈ Kp"
      by blast
    from cp obtain Eb where
        cb: "compile Π r (body d) (Statement (m + csize (body d))) m
               = (m + csize (body d), Statement m, Eb, Kp)"
      by (rule compile_procE)
    from compile_calls_from [OF cb eK] d show ?thesis by blast
  next
    case M
    from mainc obtain Eb where
        cb: "compile Π prog_main_name (main_body Π) (Statement (n1 + csize (main_body Π))) n1
               = (n1 + csize (main_body Π), Statement n1, Eb, Kmain)"
      by (rule compile_procE) simp
    from compile_calls_from [OF cb M] main show ?thesis
      by (metis gproc_decl.select_convs(2))
  qed
qed

lemma init_stores_in_kinds: "t ∈ init_stores κ ⟹ in_kinds κ t"
  by (auto simp: init_stores_def in_kinds_def all_in_kinds_def simp del: ik_range_iff)
```

Runs of the elaborated program stay within their kinds

Elaboration puts a conversion on every argument, so the call edges of the elaborated graph pass well-typed arguments to formals of the right kinds.

```
lemma com_calls_elab_com:
  "com_calls (elab_com G fm κ r c)
     = (λ(dst, q, es). (map_option (resolve_var G r) dst, q,
                        elab_args (resolve_var G r) κ (map (resolve_var G q) (fm q)) es))
       ` com_calls c"
  by (induction c) (auto simp: image_Un)

lemma call_formals_elab_ptable:
  "call_formals (elab_ptable p ds) q = map (resolve_var (declared_global p) q) (prog_formals p q)"
  by (cases "prog_table p q") (auto simp: elab_ptable_def prog_formals_def)

theorem calls_in_kinds_elab_cfg:
  assumes wf: "wf_program_compile_input p"
  shows "calls_in_kinds (kenv_of ds) (elab_cfg p ds)"
  unfolding calls_in_kinds_def
proof (intro allI impI)
  fix u dst pars es v w
  assume e: "(u, CallEdge dst pars es, v, w) ∈ calls (elab_cfg p ds)"
  from wf have src: "wf_source_program (declared_global p) (prog_table p)"
    by (simp add: gwf_compile_input_def)
  from compile_prog_calls_from [OF elab_ptable_main(1) [OF src] e [unfolded elab_cfg_def]]
  obtain r d q where d: "elab_ptable p ds r = Some d" and c: "(dst, q, es) ∈ com_calls (body d)"
    and pars: "pars = call_formals (elab_ptable p ds) q"
    by blast
  let ?ϱ = "resolve_var (declared_global p) r"
  let ?fs = "map (resolve_var (declared_global p) q) (prog_formals p q)"
  from d c obtain es0 where es: "es = elab_args ?ϱ (kenv_of ds) ?fs es0"
    by (auto simp: elab_ptable_def com_calls_elab_com)
  have "list_all (λe. wt_exp (kenv_of ds) e ∧ ReturnVar ∉ exp_vars e) es"
    using elab_args_wt [of "kenv_of ds" ?ϱ ?fs es0] exp_vars_elab_args
      [of _ ?ϱ "kenv_of ds" ?fs es0]
```

```
      ReturnVar_notin_range_resolve_var [of "declared_global p" r]
    unfolding es list_all_iff by blast
  moreover have "args_ok (kenv_of ds) pars es"
    using elab_args_wt [of "kenv_of ds" ?ϱ ?fs es0]
    by (simp add: es pars call_formals_elab_ptable del: call_formals_def)
  ultimately show "list_all (λe. wt_exp (kenv_of ds) e ∧ ReturnVar ∉ exp_vars e) es
                   ∧ args_ok (kenv_of ds) pars es" ..
qed
```

Hence a run of the elaborated program that starts from a store within its kinds keeps every identity other than the return slot at a value of its kind, through assignments, declarations, calls, returns and recursion. The premise on the initial store is explicit; the initial stores of the soundness theorems satisfy it (`?t ∈ init_stores ?κ ⟹ in_kinds ?κ ?t`).

```
theorem run_in_kinds:
  assumes wf: "wf_program_compile_input p"
    and s0: "in_kinds (kenv_of ds) s0"
    and run:
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →p* (residual, s, frs)"
  shows "in_kinds (kenv_of ds) s"
proof -
  from source_reaches_node_collect [OF wf _ run, of "{s0}"]
  obtain v where "s ∈ C(kenv_of ds,elab_cfg p ds,{s0}) v" by blast
  then show ?thesis
    using node_collect_in_kinds [OF calls_in_kinds_elab_cfg [OF wf]] s0 by blast
qed

corollary run_from_init_in_kinds:
  assumes "wf_program_compile_input p"
    and "s0 ∈ init_stores (kenv_of ds)"
    and
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →p* (residual, s, frs)"
  shows "in_kinds (kenv_of ds) s"
  using run_in_kinds [OF assms(1) init_stores_in_kinds [OF assms(2)] assms(3)] .

end
```

Chapter 6

Voblint_Framework

6.1 CFG_Enumeration

```
theory CFG_Enumeration
  imports "Voblint_CFG.CFG_Def"
begin
```

Solver-facing CFG enumeration

Backward predecessor relations and deterministic edge enumeration over a compiled two-relation CFG. These are the equation-generation and code-generation views the TD solver reads to build and evaluate the constraint system; they are not part of the concrete CFG semantics — `valid_activation_trace` never refers to them. Ordinary control flow is enumerated over `intra`; procedure calls — entry routing and return combining — over `calls`. There is no unified edge set and no separate combine relation: a return is recovered from the same `calls` tuple that created the activation.

Every enumeration appears twice: as a set, which the proofs reason over, and as a list, which code generation and the solver's folds consume, identified by a `set_..._list` lemma whenever the edge set is finite. On the return side there is one of each — the set `call_targets` and the list `call_target_list` — and every other return view is an image or a `map` of those, so the views cannot come to disagree about which edges they enumerate.

Intra predecessors

The ordinary incoming transitions of a node: an intra edge is a total store transformer within one activation, so this is the transfer fold the equation system runs at every node — including the `EA_Ret e p` edges into `FunctionResult p`, by which return values are summarised as ordinary predecessor folding.

```
definition intra_predecessors ::
  "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gedge_action) set"
    where
  "intra_predecessors g v = {(u, a). (u, a, v) ∈ intra g}"

lemma intra_predecessors_iff [simp]:
  "(u, a) ∈ intra_predecessors g v ⟷ (u, a, v) ∈ intra g"
  by (simp add: intra_predecessors_def)

lemma finite_intra_predecessors [intro]:
  assumes "finite (intra g)"
  shows "finite (intra_predecessors g v)"
proof -
  have "intra_predecessors g v
        ⊆ (λe :: cfg_node × _ × cfg_node. (fst e, fst (snd e))) ` intra g"
    unfolding intra_predecessors_def by force
  then show ?thesis using assms finite_subset finite_imageI by blast
qed

definition intra_predecessor_list ::
    "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gedge_action) list"
      where
  "intra_predecessor_list g v =
     map (λ(u, a, w). (u, a)) (filter (λ(u, a, w). w = v) (cfg_intra_list g))"

lemma set_intra_predecessor_list [simp]:
  assumes "finite (intra g)"
  shows "set (intra_predecessor_list g v) = intra_predecessors g v"
  unfolding intra_predecessor_list_def intra_predecessors_def
  using set_cfg_intra_list[OF assms] by (force simp: image_iff)
```

Call-entry enumeration

The call tuples whose callee entry is the queried node `v` (a `FunctionEntry p`). The continuation does not affect the callee-entry state, so it is projected away; the entry contribution is the caller state routed through the call action's parameter binding, which is what an analysis's entry operation computes.

```
definition entry_calls ::
  "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action) set"
    where
  "entry_calls g v = {(c, ca). ∃k. (c, ca, v, k) ∈ calls g}"

lemma entry_calls_iff [simp]:
  "(c, ca) ∈ entry_calls g v ⟷ (∃k. (c, ca, v, k) ∈ calls g)"
  by (simp add: entry_calls_def)

lemma finite_entry_calls [intro]:
```

```
  assumes "finite (calls g)"
  shows "finite (entry_calls g v)"
proof -
  have "entry_calls g v
        ⊆ (λe :: cfg_node × _ × cfg_node × cfg_node. (fst e, fst (snd e))) ‘ calls g"
    unfolding entry_calls_def by force
  then show ?thesis using assms finite_subset finite_imageI by blast
qed

definition entry_call_list ::
  "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action) list"
    where
  "entry_call_list g v =
     map (λ(c, ca, ce, k). (c, ca)) (filter (λ(c, ca, ce, k). ce = v) (cfg_calls_list g))"

lemma set_entry_call_list [simp]:
  assumes "finite (calls g)"
  shows "set (entry_call_list g v) = entry_calls g v"
  unfolding entry_call_list_def entry_calls_def
  using set_cfg_calls_list[OF assms] by (force simp: image_iff)
```

Outgoing call enumeration (caller perspective)

The call tuples whose call site is the queried node `v`: the entry, action, and continuation of every call `v` makes. This is the caller-side counterpart of `entry_calls` (callee-indexed) — routing a call-entry seed publication at its call site needs the callee entry and the whole `call_action` together, which the continuation-indexed enumerations below do not expose.

```
definition call_successors ::
    "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action × cfg_node) set"
      where
  "call_successors g v = {(ce, ca, k). (v, ca, ce, k) ∈ calls g}"

lemma call_successors_iff [simp]:
  "(ce, ca, k) ∈ call_successors g v ⟷ (v, ca, ce, k) ∈ calls g"
  by (simp add: call_successors_def)

lemma finite_call_successors [intro]:
  assumes "finite (calls g)"
  shows "finite (call_successors g v)"
proof -
  have "call_successors g v
        ⊆ (λ(c, ca, ce, k). (ce, ca, k)) ‘ calls g"
    unfolding call_successors_def by force
  then show ?thesis using assms finite_subset finite_imageI by blast
qed

definition call_successor_list ::
    "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action × cfg_node) list"
      where
  "call_successor_list g v =
     map (λ(c, ca, ce, k). (ce, ca, k)) (filter (λ(c, ca, ce, k). c = v) (cfg_calls_list g))"

lemma set_call_successor_list [simp]:
  assumes "finite (calls g)"
  shows "set (call_successor_list g v) = call_successors g v"
  unfolding call_successor_list_def call_successors_def
  using set_cfg_calls_list[OF assms] by (force simp: image_iff)
```

Call targets at a continuation

The call edges whose continuation is `v`, each paired with the procedure its callee entry names. This is the one continuation-indexed enumeration that reads `calls` directly; every return-side view below is an image or a map of it.

`call_target_at` decodes a single call edge, selecting on the continuation and reading the callee's procedure off `FunctionEntry p` in one total function, so there is no unreachable-but-present branch and no later definition has to re-establish that the node it is looking at really is a callee entry.

Naming the callee by its procedure rather than by the `FunctionResult p` node its result is read at keeps the call site's own data — where the call is, what it passes, where it returns to — separate from the callee, which is what lets a resolver choose among candidates rather than read a callee the enumeration already fixed. The result node is then `FunctionResult p` at the point of use.

```
definition call_targets ::
```

```
    "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action × pname) set"
      where
  "call_targets g v = {(c, ca, p). (c, ca, FunctionEntry p, v) ∈ calls g}"

lemma call_targets_iff [simp]:
  "(c, ca, p) ∈ call_targets g v ⟷ (c, ca, FunctionEntry p, v) ∈ calls g"
  by (simp add: call_targets_def)

fun call_target_at ::
    "cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action × cfg_node × cfg_node)
       ⇒ (cfg_node × (var_id, exp) gcall_action × pname) option" where
  "call_target_at v (c, ca, FunctionEntry p, k) =
     (if k = v then Some (c, ca, p) else None)"
| "call_target_at v _ = None"

definition call_target_list ::
    "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action × pname) list"
      where
  "call_target_list g v = List.map_filter (call_target_at v) (cfg_calls_list g)"

lemma set_map_filter_call_target_at:
  "set (List.map_filter (call_target_at v) es)
     = {(c, ca, p). (c, ca, FunctionEntry p, v) ∈ set es}"
proof (induction es)
  case Nil
  show ?case by simp
next
  case (Cons e es)
  obtain c ca ce k where e: "e = (c, ca, ce, k)" by (cases e) auto
  show ?case by (cases ce) (auto simp: e Cons.IH)
qed
```

Membership, in the form a call-site obligation states it: the enumeration lists exactly the call edges of `g` whose continuation is `v`, each paired with the procedure its callee entry names.

```
lemma set_call_target_list [simp]:
  assumes "finite (calls g)"
  shows "set (call_target_list g v) = call_targets g v"
  unfolding call_target_list_def call_targets_def
  by (simp add: set_map_filter_call_target_at set_cfg_calls_list[OF assms])

lemma finite_call_targets [intro]:
  assumes "finite (calls g)"
  shows "finite (call_targets g v)"
proof -
  have "call_targets g v = set (call_target_list g v)"
    using set_call_target_list[OF assms] by simp
  then show ?thesis by simp
qed
```

The enumeration lists each call edge once: `cfg_calls_list` is duplicate-free and `call_target_at` discards exactly the components a decoded entry does not determine. This is what makes the site and target lists below duplicate-free too, so the generator builds one contribution per site-target pair rather than one per pair of co-listed callees.

```
lemma distinct_map_filter_call_target_at:
  assumes "distinct es"
  shows "distinct (List.map_filter (call_target_at v) es)"
  using assms
proof (induction es)
  case Nil
  show ?case by simp
next
  case (Cons e es)
  obtain c ca ce k where e: "e = (c, ca, ce, k)" by (cases e) auto
  show ?case
    using Cons by (cases ce) (auto simp: e set_map_filter_call_target_at)
qed

lemma distinct_call_target_list [simp]:
  assumes "finite (calls g)"
  shows "distinct (call_target_list g v)"
  unfolding call_target_list_def
  by (rule distinct_map_filter_call_target_at) (simp add: cfg_calls_list_def assms)

lemma call_target_list_iff:
```

```
  assumes "finite (calls g)"
  shows "(c, ca, p) ∈ set (call_target_list g v)
         ⟷ (c, ca, FunctionEntry p, v) ∈ calls g"
  by (simp add: set_call_target_list[OF assms])
```

Return enumeration

The return edges, as an image of `call_targets` that keeps the triggering `call_action` whole. The return contribution is the caller state and the callee-exit state assembled by `combine`$^\sharp$. The callee's `FunctionResult p` node comes back directly — recovered from the callee entry `FunctionEntry p` — so no separate combine relation is needed and no client has to rebuild the result node.

The action is kept whole rather than packaged into the call's `call_info`, because the generator must route the callee-entry seed publication and the return-side context read from the *same* action, exactly as `entry_call_list` keeps `ca` whole on the entry side. This layer is the only one holding the `calls` relation, so whatever a consumer needs about a call site is projected here: a combine tree never sees the CFG, and `wf_cfg` does not force a call site's outgoing call edge to be unique, so there is no well-defined downstream lookup that could recover it instead.

```
definition return_call_actions ::
    "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action × cfg_node) set"
      where
  "return_call_actions g v =
     (λ(c, ca, p). (c, ca, FunctionResult p)) ‘ call_targets g v"

lemma return_call_actions_iff [simp]:
  "(c, ca, r) ∈ return_call_actions g v
     ⟷ (∃p. r = FunctionResult p ∧ (c, ca, FunctionEntry p, v) ∈ calls g)"
  by (auto simp: return_call_actions_def image_iff Bex_def)

lemma finite_return_call_actions [intro]:
  assumes "finite (calls g)"
  shows "finite (return_call_actions g v)"
  using finite_call_targets[OF assms] by (simp add: return_call_actions_def)

definition return_call_action_list ::
    "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action × cfg_node) list"
      where
  "return_call_action_list g v =
     map (λ(c, ca, p). (c, ca, FunctionResult p)) (call_target_list g v)"

lemma set_return_call_action_list [simp]:
  assumes "finite (calls g)"
  shows "set (return_call_action_list g v) = return_call_actions g v"
  unfolding return_call_action_list_def return_call_actions_def
  by (simp add: set_call_target_list[OF assms])
```

Call sites and the resolver adapter

The call site alone, with no callee: what a call-site-owned resolver is given. The list is duplicate-free, so a site with several statically known callees is listed once; with `distinct_static_targets` below, resolving every listed site therefore visits each site-target pair exactly once.

`static_targets` below is the resolver that answers from the CFG and ignores the caller's abstract state — the behaviour a statically enumerated generator already has. A resolver reading that state instead is what makes an indirect call expressible, and the shape of the equation does not change when it does. Two limits are worth stating, because the resolver type `cfg ⇒ cfg_node ⇒ cfg_node ⇒ call_action ⇒ 'd ⇒ pname list` does not: a resolver may answer with any procedures at all, so a sound instance needs an explicit coverage assumption relating its answer to the concrete callees reachable at the site; and the sites themselves come from `call_target_list`, so a call with no statically enumerated target is not listed and no resolver is consulted for it. What this supports is therefore state-dependent choice among enumerated candidates, not a fully late-bound call whose site exists independently of any known target.

```
definition call_site_list ::
    "(var_id, exp) gcfg ⇒ cfg_node ⇒ (cfg_node × (var_id, exp) gcall_action) list"
      where
  "call_site_list g v = remdups (map (λ(c, ca, p). (c, ca)) (call_target_list g v))"

lemma distinct_call_site_list [simp]:
  "distinct (call_site_list g v)"
  by (simp add: call_site_list_def)

lemma set_call_site_list [simp]:
```

```
  assumes "finite (calls g)"
  shows "set (call_site_list g v)
         = {(c, ca) | c ca. ∃p. (c, ca, FunctionEntry p, v) ∈ calls g}"
  unfolding call_site_list_def
  by (force simp: set_call_target_list[OF assms] call_targets_def image_iff)

definition static_targets ::
    "(var_id, exp) gcfg ⇒ cfg_node ⇒ cfg_node ⇒ (var_id, exp) gcall_action ⇒ pname list"
      where
  "static_targets g v cc ca =
     map (λ(c, a, p). p)
       (filter (λ(c, a, p). c = cc ∧ a = ca) (call_target_list g v))"

lemma static_targets_iff [simp]:
  assumes "finite (calls g)"
  shows "p ∈ set (static_targets g v cc ca)
         ⟷ (cc, ca, FunctionEntry p, v) ∈ calls g"
  unfolding static_targets_def
  by (force simp: call_target_list_iff[OF assms, symmetric] image_iff)

lemma distinct_static_targets [simp]:
  assumes "finite (calls g)"
  shows "distinct (static_targets g v cc ca)"
proof -
  let ?es = "filter (λ(c, a, p). c = cc ∧ a = ca) (call_target_list g v)"
  have "inj_on (λ(c, a, p). p) (set ?es)" by (auto simp: inj_on_def)
  moreover have "distinct ?es" using distinct_call_target_list[OF assms] by simp
  ultimately show ?thesis unfolding static_targets_def by (simp add: distinct_map)
qed
```

Exactly when a site has a statically known callee, which is the form a coverage obligation stated over the enumeration needs.

```
lemma static_targets_neq_Nil_iff [simp]:
  assumes "finite (calls g)"
  shows "static_targets g v cc ca ≠ []
         ⟷ (∃p. (cc, ca, FunctionEntry p, v) ∈ calls g)"
proof -
  have "static_targets g v cc ca ≠ [] ⟷ (∃p. p ∈ set (static_targets g v cc ca))"
    by (cases "static_targets g v cc ca") auto
  thus ?thesis using static_targets_iff[OF assms] by blast
qed

definition static_resolve ::
  "(var_id, exp) gcfg ⇒ cfg_node ⇒ cfg_node ⇒ (var_id, exp) gcall_action ⇒ 'd ⇒ pname list"
    where
  "static_resolve g v cc ca d = static_targets g v cc ca"

lemma static_resolve_iff [simp]:
  assumes "finite (calls g)"
  shows "p ∈ set (static_resolve g v cc ca d)
         ⟷ (cc, ca, FunctionEntry p, v) ∈ calls g"
  unfolding static_resolve_def by (rule static_targets_iff[OF assms])
```

Which unknowns a run has to have solved

`vars_cover g vars` is the one recurring premise every post-solution soundness theorem here needs: the solved key set `vars` contains the CFG entry, the target of every `intra` edge, and both halves of every call — the callee's entry and the caller's continuation.

Read it as a statement about a solver run, not about the graph. It quantifies over every edge `g` has, so a graph carrying a call the run never activated does not satisfy it: in a routed system a callee's entry unknown is solved only once some caller publishes its seed, and an unreachable call site publishes nothing. That is why a run establishes it by deciding `vars_cover_exec` against the keys it actually solved, on a graph *pruned* to what it can reach, rather than by an argument about connectivity.

The four components travel together, so callers state and discharge one premise instead of four positional ones. Global, not locale-local: every analysis instance and the executable pipeline cite it under this name.

```
definition vars_cover :: "(var_id, exp) gcfg ⇒ (cfg_node × unit) set ⇒ bool" where
  "vars_cover g vars ⟷
     (cfg_entry g, ()) ∈ vars
   ∧ (∀u a v. (u, a, v) ∈ intra g ⟶ (v, ()) ∈ vars)
   ∧ (∀c dst fs as q k. (c, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
        ⟶ (FunctionEntry q, ()) ∈ vars ∧ (k, ()) ∈ vars)"
```

```
lemma vars_coverI [intro]:
  assumes "(cfg_entry g, ()) ∈ vars"
    and "⋀u a v. (u, a, v) ∈ intra g ⟹ (v, ()) ∈ vars"
    and "⋀c dst fs as q k. (c, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
           ⟹ (FunctionEntry q, ()) ∈ vars"
    and "⋀c dst fs as q k. (c, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
           ⟹ (k, ()) ∈ vars"
  shows "vars_cover g vars"
  unfolding vars_cover_def using assms by blast

lemma vars_cover_entryD [dest]: "vars_cover g vars ⟹ (cfg_entry g, ()) ∈ vars"
  unfolding vars_cover_def by blast

lemma vars_cover_edgeD [dest]:
  "vars_cover g vars ⟹ (u, a, v) ∈ intra g ⟹ (v, ()) ∈ vars"
  unfolding vars_cover_def by blast

lemma vars_cover_enterD [dest]:
  assumes cover: "vars_cover g vars"
  shows "⋀c dst fs as q k. (c, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
     ⟹ (FunctionEntry q, ()) ∈ vars"
  using cover unfolding vars_cover_def by blast

lemma vars_cover_combineD [dest]:
  assumes cover: "vars_cover g vars"
  shows "⋀c dst fs as q k. (c, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
     ⟹ (k, ()) ∈ vars"
  using cover unfolding vars_cover_def by blast
```

Coverage of a finite graph is testable: walk the two edge enumerations and test membership. The test is sufficient, not equivalent, and deliberately so. `vars_cover` asks only about calls of the shape `(c, CallEdge dst fs as, FunctionEntry q, k)`, while the test asks about every tuple the enumeration yields, so it is the stronger of the two: it also covers a `calls` entry whose action or target has some other shape. Turning it into an equivalence would mean assuming that `g` has no such entry, which is a well-formedness fact about the graph and not something a coverage check should carry.

```
definition vars_cover_exec :: "(var_id, exp) gcfg ⇒ (cfg_node × unit) set ⇒ bool"
  where
  "vars_cover_exec g vars ⟷
     (cfg_entry g, ()) ∈ vars
   ∧ list_all (λ(u, a, v). (v, ()) ∈ vars) (cfg_intra_list g)
   ∧ list_all (λ(c, ca, ce, k). (ce, ()) ∈ vars ∧ (k, ()) ∈ vars) (cfg_calls_list g)"
```

Coverage when the unknowns carry a context

`vars_cover` is fixed at the unit context: its unknowns are nodes, so it can demand that every successor of every edge was solved. A context-sensitive system cannot demand that. Its unknowns are node-context pairs, and which contexts a node was solved at is decided by the run, so coverage becomes *closure*: whenever a pair was solved, the pairs its edges lead to were solved too. Hence the conditional shape here against the unconditional shape above.

Where a call leads is not a fact about the graph. A context policy computes the callee's context from the call site, the caller's context, the callee, and in general the state published at the call — which is why `succ_ctx` is a parameter rather than something read off `g`. An instance supplies its own route already applied to its own solved environment.

The return side takes no such argument: a combine resumes the caller's own activation, so the continuation is covered at the context the call was made in.

```
definition ctx_vars_cover ::
    "(var_id, exp) gcfg ⇒ (cfg_node ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pname ⇒ 'c) ⇒ 'c
       ⇒ (cfg_node × 'c) set ⇒ bool" where
  "ctx_vars_cover g succ_ctx rc vars ⟷
     (cfg_entry g, rc) ∈ vars
   ∧ (∀u a v ctx. (u, ctx) ∈ vars ⟶ (u, a, v) ∈ intra g ⟶ (v, ctx) ∈ vars)
   ∧ (∀u ctx dst fs as q k. (u, ctx) ∈ vars
        ⟶ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
        ⟶ (FunctionEntry q, succ_ctx u ctx (CallEdge dst fs as) q) ∈ vars
          ∧ (k, ctx) ∈ vars)"

lemma ctx_vars_coverI [intro]:
  assumes "(cfg_entry g, rc) ∈ vars"
    and "⋀u a v ctx. (u, ctx) ∈ vars ⟹ (u, a, v) ∈ intra g ⟹ (v, ctx) ∈ vars"
    and "⋀u ctx dst fs as q k. (u, ctx) ∈ vars
           ⟹ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
```

```
        ⟹ (FunctionEntry q, succ_ctx u ctx (CallEdge dst fs as) q) ∈ vars"
    and "⋀u ctx dst fs as q k. (u, ctx) ∈ vars
        ⟹ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g ⟹ (k, ctx) ∈ vars"
  shows "ctx_vars_cover g succ_ctx rc vars"
  unfolding ctx_vars_cover_def using assms by blast

lemma ctx_vars_cover_entryD [dest]:
  "ctx_vars_cover g succ_ctx rc vars ⟹ (cfg_entry g, rc) ∈ vars"
  unfolding ctx_vars_cover_def by blast

lemma ctx_vars_cover_edgeD [dest]:
  "ctx_vars_cover g succ_ctx rc vars ⟹ (u, ctx) ∈ vars ⟹ (u, a, v) ∈ intra g
    ⟹ (v, ctx) ∈ vars"
  unfolding ctx_vars_cover_def by blast

lemma ctx_vars_cover_enterD [dest]:
  assumes cover: "ctx_vars_cover g succ_ctx rc vars"
  shows "⋀u ctx dst fs as q k. (u, ctx) ∈ vars
    ⟹ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
    ⟹ (FunctionEntry q, succ_ctx u ctx (CallEdge dst fs as) q) ∈ vars"
  using cover unfolding ctx_vars_cover_def by blast

lemma ctx_vars_cover_combineD [dest]:
  assumes cover: "ctx_vars_cover g succ_ctx rc vars"
  shows "⋀u ctx dst fs as q k. (u, ctx) ∈ vars
    ⟹ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g ⟹ (k, ctx) ∈ vars"
  using cover unfolding ctx_vars_cover_def by blast
```

Contextual coverage is testable the way the unit one is, with one difference that decides the shape: closure quantifies over the pairs the run solved, and those are not enumerable from the graph. So the test walks the solved keys against the two edge enumerations rather than walking the graph alone, which is why it takes them as the set the solver returns and iterates over it.

Sufficient rather than equivalent, and deliberately so. At a call the test asks for the continuation at every action shape, while `ctx_vars_cover` asks only about `CallEdge`; demanding the callee entry off a non-call action would need a context the policy never computes, so that one stays guarded.

```
definition ctx_vars_cover_exec ::
    "(var_id, exp) gcfg ⇒ (cfg_node ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pname ⇒ 'c) ⇒ 'c
      ⇒ (cfg_node × 'c) set ⇒ bool" where
  "ctx_vars_cover_exec g succ_ctx rc vs ⟷
     (cfg_entry g, rc) ∈ vs
   ∧ (∀(u, ctx) ∈ vs.
        list_all (λ(u', a, v). u' ≠ u ∨ (v, ctx) ∈ vs) (cfg_intra_list g)
     ∧ list_all (λ(c, ca, ce, k). c ≠ u ∨ ((k, ctx) ∈ vs
           ∧ (case ce of FunctionEntry q ⇒ (ce, succ_ctx u ctx ca q) ∈ vs
                | _ ⇒ True)))
         (cfg_calls_list g))"

lemma ctx_vars_cover_of_exec:
  assumes finE: "finite (intra g)" and finC: "finite (calls g)"
    and cover: "ctx_vars_cover_exec g succ_ctx rc (vs :: (cfg_node × 'c) set)"
  shows "ctx_vars_cover g succ_ctx rc vs"
proof (intro ctx_vars_coverI)
  show "(cfg_entry g, rc) ∈ vs"
    using cover unfolding ctx_vars_cover_exec_def by blast
next
  fix u a v ctx
  assume "(u, ctx) ∈ vs" and "(u, a, v) ∈ intra g"
  then show "(v, ctx) ∈ vs"
    using cover unfolding ctx_vars_cover_exec_def
    by (fastforce simp: list_all_iff set_cfg_intra_list [OF finE])
next
  fix u ctx dst fs as q k
  assume "(u, ctx) ∈ vs"
    and "(u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g"
  then show "(FunctionEntry q, succ_ctx u ctx (CallEdge dst fs as) q) ∈ vs"
    using cover unfolding ctx_vars_cover_exec_def
    by (fastforce simp: list_all_iff set_cfg_calls_list [OF finC])
next
  fix u ctx dst fs as q k
  assume "(u, ctx) ∈ vs"
    and "(u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g"
  then show "(k, ctx) ∈ vs"
    using cover unfolding ctx_vars_cover_exec_def
    by (fastforce simp: list_all_iff set_cfg_calls_list [OF finC])
```

qed

Coverage along live dependencies

`ctx_vars_cover` demands closure from every solved pair, including pairs whose solved state is bottom and calls whose entered state is bottom. A demand-driven solver never solves what such a pair would lead to, so the unconditional closure fails on dead call sites and on procedures nothing calls, while soundness only ever follows a dependency from a state that holds a concrete store. The live form asks for exactly that much.

`live u ctx` says the state solved at `(u, ctx)` may hold a store; an intra successor is demanded only from a live pair. `succ u ctx ca q` is the callee context a call routes to, or `None` when the entered state is bottom and the call contributes nothing; the callee entry is demanded only when it is `Some`. The continuation stays unconditional: a call site is queried only by its own continuation's call tree, so a solver never solves the one without the other.

```
definition ctx_vars_cover_live ::
    "(var_id, exp) gcfg ⇒ (cfg_node ⇒ 'c ⇒ bool) ⇒ (cfg_node ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pname ⇒ 'c
option)
       ⇒ 'c ⇒ (cfg_node × 'c) set ⇒ bool" where
  "ctx_vars_cover_live g live succ rc vars ⟷
     (cfg_entry g, rc) ∈ vars
   ∧ (∀u a v ctx. (u, ctx) ∈ vars ⟶ live u ctx ⟶ (u, a, v) ∈ intra g
        ⟶ (v, ctx) ∈ vars)
   ∧ (∀u ctx dst fs as q k. (u, ctx) ∈ vars
        ⟶ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
        ⟶ (k, ctx) ∈ vars
          ∧ (∀ctx'. succ u ctx (CallEdge dst fs as) q = Some ctx'
               ⟶ (FunctionEntry q, ctx') ∈ vars))"

lemma ctx_vars_cover_liveI [intro]:
  assumes "(cfg_entry g, rc) ∈ vars"
    and "⋀u a v ctx. (u, ctx) ∈ vars ⟹ live u ctx ⟹ (u, a, v) ∈ intra g
           ⟹ (v, ctx) ∈ vars"
    and "⋀u ctx dst fs as q k ctx'. (u, ctx) ∈ vars
           ⟹ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
           ⟹ succ u ctx (CallEdge dst fs as) q = Some ctx'
           ⟹ (FunctionEntry q, ctx') ∈ vars"
    and "⋀u ctx dst fs as q k. (u, ctx) ∈ vars
           ⟹ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g ⟹ (k, ctx) ∈ vars"
  shows "ctx_vars_cover_live g live succ rc vars"
  unfolding ctx_vars_cover_live_def using assms by blast

lemma ctx_vars_cover_live_entryD [dest]:
  "ctx_vars_cover_live g live succ rc vars ⟹ (cfg_entry g, rc) ∈ vars"
  unfolding ctx_vars_cover_live_def by blast

lemma ctx_vars_cover_live_edgeD [dest]:
  "ctx_vars_cover_live g live succ rc vars ⟹ (u, ctx) ∈ vars ⟹ live u ctx
     ⟹ (u, a, v) ∈ intra g ⟹ (v, ctx) ∈ vars"
  unfolding ctx_vars_cover_live_def by blast

lemma ctx_vars_cover_live_enterD [dest]:
  "ctx_vars_cover_live g live succ rc vars ⟹ (u, ctx) ∈ vars
     ⟹ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g
     ⟹ succ u ctx (CallEdge dst fs as) q = Some ctx' ⟹ (FunctionEntry q, ctx') ∈ vars"
  unfolding ctx_vars_cover_live_def by blast

lemma ctx_vars_cover_live_combineD [dest]:
  "ctx_vars_cover_live g live succ rc vars ⟹ (u, ctx) ∈ vars
     ⟹ (u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls g ⟹ (k, ctx) ∈ vars"
  unfolding ctx_vars_cover_live_def by blast

end
```

6.2 DG_State

```
theory DG_State
  imports "Voblint_Domain.State_Concretization"
begin
```

The value a D/G unknown carries

An analysis chooses a flow-sensitive answer domain `D` and a flow-insensitive side-effect domain `G`. The framework keeps them opaque and stores them in separate components of `dg_state`; it never copies a global component into a local answer.

This theory is that carrier and its order, and nothing else: no address, no solver tree, no transfer. Everything above it only projects and repacks the two components, which is what makes those constructions independent of the concrete domains.

A lattice copy type for D-times-G unknown values

The solver's single value type must order local and global halves componentwise. Raw pairs cannot: this repository loads `HOL-Library.Product_Lexorder`, so every `'l * 'g` in scope already carries the lexicographic order, and a componentwise instance for the same type would clash with it. `dg_state` is a distinct carrier so that both orders can coexist.

```
datatype ('l, 'g) dg_state = DG (dg_local: 'l) (dg_global: 'g)

instantiation dg_state :: (ord, ord) ord
begin

definition less_eq_dg_state :: "('a, 'b) dg_state ⇒ ('a, 'b) dg_state ⇒ bool" where
  "less_eq_dg_state d1 d2 = (dg_local d1 ≤ dg_local d2 ∧ dg_global d1 ≤ dg_global d2)"

definition less_dg_state :: "('a, 'b) dg_state ⇒ ('a, 'b) dg_state ⇒ bool" where
  "less_dg_state d1 d2 = (d1 ≤ d2 ∧ ¬ d2 ≤ d1)"

instance ..

end

instance dg_state :: (order, order) order
proof intro_classes
  fix x y z :: "('a, 'b) dg_state"
  show "x < y ⟷ x ≤ y ∧ ¬ y ≤ x" by (simp add: less_dg_state_def)
  show "x ≤ x" by (simp add: less_eq_dg_state_def)
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z"
    by (auto simp: less_eq_dg_state_def intro: order_trans)
  show "x ≤ y ⟹ y ≤ x ⟹ x = y"
    by (auto simp: less_eq_dg_state_def intro: dg_state.expand antisym)
qed

instantiation dg_state :: (semilattice_sup, semilattice_sup) semilattice_sup
begin

definition sup_dg_state :: "('a, 'b) dg_state ⇒ ('a, 'b) dg_state ⇒ ('a, 'b) dg_state" where
  "sup_dg_state d1 d2 = DG (dg_local d1 ⊔ dg_local d2) (dg_global d1 ⊔ dg_global d2)"

instance
  by standard (auto simp: less_eq_dg_state_def sup_dg_state_def)

end

instantiation dg_state :: (order_bot, order_bot) order_bot
begin

definition bot_dg_state :: "('a, 'b) dg_state" where
  "bot_dg_state = DG bot bot"

instance
  by standard (simp add: less_eq_dg_state_def bot_dg_state_def)

end

instance dg_state ::
  (bounded_semilattice_sup_bot, bounded_semilattice_sup_bot) bounded_semilattice_sup_bot ..
```

Reading the two components back

Every construction above this theory projects and repacks, so the four projection equations and the constructor comparison are what its proofs should meet instead of the instance definitions. The order itself stays unfolded: `less_eq_dg_state_def` is not a simp rule, because a bound between two values that are not both constructor terms reads better as one $\leq$ than as a conjunction of two.

```
lemma dg_local_sup [simp]:
  fixes x y :: "('a::semilattice_sup, 'b::semilattice_sup) dg_state"
  shows "dg_local (x ⊔ y) = dg_local x ⊔ dg_local y"
  by (simp add: sup_dg_state_def)

lemma dg_global_sup [simp]:
  fixes x y :: "('a::semilattice_sup, 'b::semilattice_sup) dg_state"
  shows "dg_global (x ⊔ y) = dg_global x ⊔ dg_global y"
  by (simp add: sup_dg_state_def)

lemma dg_local_bot [simp]:
  "dg_local (bot :: ('a::order_bot, 'b::order_bot) dg_state) = bot"
  by (simp add: bot_dg_state_def)

lemma dg_global_bot [simp]:
  "dg_global (bot :: ('a::order_bot, 'b::order_bot) dg_state) = bot"
  by (simp add: bot_dg_state_def)

lemma DG_le_DG [simp]:
  fixes d d' :: "'a::ord" and g g' :: "'b::ord"
  shows "DG d g ≤ DG d' g' ⟷ d ≤ d' ∧ g ≤ g'"
  by (simp add: less_eq_dg_state_def)

instantiation dg_state ::
  ("{bounded_semilattice_sup_bot, warrowing}", "{bounded_semilattice_sup_bot, warrowing}") warrowing
begin

definition widen_dg_state ::
  "('a, 'b) dg_state ⇒ ('a, 'b) dg_state ⇒ ('a, 'b) dg_state"
where
  "widen_dg_state a b = DG ((dg_local a) ∇ (dg_local b)) ((dg_global a) ∇ (dg_global b))"

definition narrow_dg_state ::
  "('a, 'b) dg_state ⇒ ('a, 'b) dg_state ⇒ ('a, 'b) dg_state"
where
  "narrow_dg_state a b = DG ((dg_local a) Δ (dg_local b)) ((dg_global a) Δ (dg_global b))"

instance
  by standard
    (simp_all add: less_eq_dg_state_def widen_dg_state_def narrow_dg_state_def
      narrowing_class.narrow_ge narrowing_class.narrow_le
      widening_class.widen_ge1 widening_class.widen_ge2)

end

end
```

6.3 DG_Constraint_Programs

```
theory DG_Constraint_Programs
  imports DG_State "Voblint_Solver.Strategy_Program_Fold"
begin

context
begin
```

This theory states its observations at the program level — `traverse_program`, `sides_of_program`, `dep_program` — reaching for `sp_compile` only where a statement must meet the vendored solver's own tree vocabulary, and does not unfold it globally: the named observation equations in `Voblint_Solver.Strategy_Tree_Program` are stated against `sp_compile` itself, and proofs meet those rather than the compiler's representation.

What a right-hand side over the packed D/G carrier is

What a generator hands the solver, and how several of those combine. An edge former takes its source as an address in the solver's own valuation space and its published slot as an explicit key, and produces a strategy program the solver can run: the program reads what it needs, publishes what it must, and answers with a `dg_state`. A generator with several such programs for one unknown — several predecessors, several call sites — folds them here, and the fold's own answer, publications and dependencies are each the join, respectively union, over the members.

`Voblint_Framework.DG_State` supplies the value type and its lattice. Nothing here inspects a domain, a variable, a global classifier, or a CFG: an address and a key are all a former sees, which is why the same former reads a contribution-only unknown and an equation-driven one.

Low-level edge formers over a solution address

The two formers in this subsection take a step in the direct shape `'dl ⇒ 'dg ⇒ 'dg × 'dl`, which no analysis writes: a specification supplies manager-native transfers, and `DG_Spec` compiles those. They survive because the keyed-solver update-rule regression needs a step it can vary directly, without a `dg_spec` in the way, and that regression is their only consumer. Read them as the minimal thing a right-hand side can be, not as the shape production equations take.

An edge or combine former takes its source as an *address* in the solver's own valuation space `'x + 'k` and its published slot as an explicit key, rather than fixing the source to a local unknown and the slot to the `unit` key – what lets a keyed generator target an arbitrary activation-indexed unknown directly, with no separate address-rewriting pass.

A program point whose value is carried by a contribution-only unknown – one with no equation of its own, so that its value is exactly the join of what was published to it – is then read by exactly the same former as one carried by an equation-driven unknown, since `QueryL` and `QueryG` project the same valuation.

`sp_read_at` itself (`Voblint_Solver.Strategy_Tree_Program`) is a generic solver-address dispatcher with no notion of `D`/`G`; here it is instantiated at `'d = ('dl, 'dg) dg_state`, so the value it returns is already one of this file's packed `DG _ bot` / `DG bot _` slots, not a raw `D`. Reading through an address is therefore an operation on already-packed DG state, not a general "query something of unknown kind" primitive.

```
definition dg_edge_program_at ::
  "('dl::bounded_semilattice_sup_bot ⇒ 'dg::bounded_semilattice_sup_bot ⇒ 'dg × 'dl)
   ⇒ 'x + 'k ⇒ 'k
   ⇒ ('x, 'k, ('dl, 'dg) dg_state, ('dl, 'dg) dg_state) strategy_program"
where
  "dg_edge_program_at step src gk =
     (do {
       d ← sp_read_at src;
       g ← sp_read_global gk;
       _ ← sp_publish gk (DG bot (fst (step (dg_local d) (dg_global g))));
       sp_return (DG (snd (step (dg_local d) (dg_global g))) bot)
     })"

definition dg_edge_contribution_program_at ::
  "('dl::bounded_semilattice_sup_bot ⇒ 'dg::bounded_semilattice_sup_bot ⇒ 'dg × 'dl)
   ⇒ 'x + 'k ⇒ 'k
   ⇒ ('x, 'k, ('dl, 'dg) dg_state, ('dl, 'dg) dg_state) strategy_program"
where
  "dg_edge_contribution_program_at step src gk =
     (do {
       d ← sp_read_at src;
       g ← sp_read_global gk;
```

```
      sp_return (DG (snd (step (dg_local d) (dg_global g))) (fst (step (dg_local d) (dg_global g))))
    })"

lemma sp_wf_dg_edge_program_at [intro, simp]: "sp_wf (dg_edge_program_at step src gk)"
  unfolding dg_edge_program_at_def by (intro sp_wf_bind) simp_all

lemma sp_wf_dg_edge_contribution_program_at [intro, simp]:
  "sp_wf (dg_edge_contribution_program_at step src gk)"
  unfolding dg_edge_contribution_program_at_def by (intro sp_wf_bind) simp_all
```

The framework boundary, as theorems

Two independent opaque domains. An analysis provides `step : D ⇒ G ⇒ G × D` — transfer plus publication; the framework reads the source address's `D` and the published slot's `G`, and transports the step's `Side : G` and `Answer : D`. Slot packing (`DG _ bot` / `DG bot _`) encodes the two-typed unknown space into the solver's single value type; the framework never looks inside `D` or `G`. The two theorems say so for *every* step and every address: an answer carries no `G`, a publication no `D`.

Why a Side-free analogue exists

`dg_edge_program_at` publishes its `G` contribution as soon as it is evaluated (`Side`). When a generator folds several such programs into one equation's right-hand side — several intra predecessors, or several return sites — each one's `Side` is published at a different moment within the same evaluation, and the vendored solver's warrowing/APINIS update rule gates convergence per *origin*: repeated writes to one key from one origin can destabilize the equation's own dependency and never converge.

`dg_edge_contribution_program_at` is the Side-free analogue. It answers the *unsplit* `(G, D)` result as one `dg_state` and publishes nothing, so a caller folds several of them with `fold_rhs_program` — whose own `Side` is empty — and splits the aggregate once, after every contribution has been read. The two lemmas below are what makes that substitution legitimate: each half of the contribution answer is exactly the observation the Side form makes.

Folding a list of programs into one D/G value

One unknown can have several right-hand sides — one per intra predecessor, one per call site — and the solver takes one tree per equation. `side_rhs_fold_dg` is that reduction: it runs the programs in order, joins their answers into a running local accumulator, and answers `DG acc bot`, letting each program's own publications pass through untouched. `side_acc_dg` is the same accumulator read declaratively off a valuation, and it is what the characterization lemmas below state the fold's three observations — answer, publications, dependencies — in terms of.

Both this and `fold_rhs_program` are specializations of one fold, `fold_rhs_program_projected`, and the choice of specialization is the whole direct/buffered split. The identity specialization accumulates complete answers, so a contribution's global half joins into the node's answer; this one accumulates only `dg_local` and embeds the result as `DG d bot`, so a global half never reaches the answer and each program publishes its own `Side` as it runs. The buffered generator wants the first — it gathers every global contribution into the answer precisely so it can publish once per node — and the direct generator wants the second. Replacing either fold by the other would exchange the two generators, not simplify them.

The narrow definitions here exist to keep the D/G vocabulary the generators and the routed soundness development actually reason in, rather than spelling out `fold_rhs_program_projected dg_local (λd. DG d bot)` in every statement. Their algebra is inherited: each observation and monotonicity fact below is a specialization of the generic one.

```
definition side_rhs_fold_dg ::
  "'d::bounded_semilattice_sup_bot
   ⇒ ('x, 'g, ('d, 'h::bounded_semilattice_sup_bot) dg_state,
        ('d, 'h) dg_state) strategy_program list
   ⇒ ('x, 'g, ('d, 'h) dg_state, ('d, 'h) dg_state) strategy_program"
where
  "side_rhs_fold_dg acc ps = fold_rhs_program_projected dg_local (λd. DG d bot) acc ps"

lemma side_rhs_fold_dg_simps [simp, code]:
  "side_rhs_fold_dg acc [] = sp_return (DG acc bot)"
  "side_rhs_fold_dg acc (p # ps) =
     do {
       res ← p;
       side_rhs_fold_dg (acc ⊔ dg_local res) ps
     }"
  by (simp_all add: side_rhs_fold_dg_def)
```

```
lemma sp_wf_side_rhs_fold_dg [intro]:
  "∀p ∈ set ps. sp_wf p ⟹ sp_wf (side_rhs_fold_dg acc ps)"
  unfolding side_rhs_fold_dg_def by (rule sp_wf_fold_rhs_program_projected)

definition side_acc_dg ::
  "'d::bounded_semilattice_sup_bot
   ⇒ ('x + 'g ⇒ ('d, 'h::bounded_semilattice_sup_bot) dg_state)
   ⇒ ('x, 'g, ('d, 'h) dg_state, ('d, 'h) dg_state) strategy_program list ⇒ 'd"
where
  "side_acc_dg acc τ ps = fold_acc_projected dg_local acc τ ps"

lemma side_acc_dg_simps [simp, code]:
  "side_acc_dg acc τ [] = acc"
  "side_acc_dg acc τ (p # ps) =
     side_acc_dg (acc ⊔ dg_local (traverse_program p τ)) τ ps"
  by (simp_all add: side_acc_dg_def)

lemma traverse_side_rhs_fold_dg:
  assumes wf: "∀p ∈ set ps. sp_wf p"
  shows "traverse_program (side_rhs_fold_dg acc ps) τ = DG (side_acc_dg acc τ ps) bot"
  unfolding side_rhs_fold_dg_def side_acc_dg_def
  by (rule traverse_fold_rhs_program_projected_char[OF wf])
```

`side_rhs_fold_dg`'s accumulator only ever reaches the terminal `Answer`, so its dependency set is independent of the accumulator: the recursion skeleton over `ps` never branches on `acc`.

```
lemma dep_program_side_rhs_fold_dg_char:
  assumes wf: "∀p ∈ set ps. sp_wf p"
  shows "dep_program sigma (side_rhs_fold_dg acc ps) = (⋃p∈set ps. dep_program sigma p)"
  unfolding side_rhs_fold_dg_def
  by (rule dep_program_fold_rhs_program_projected_char[OF wf])

lemma dep_program_side_rhs_fold_dg_acc_indep:
  assumes wf: "∀p ∈ set ps. sp_wf p"
  shows "dep_program sigma (side_rhs_fold_dg acc1 ps)
     = dep_program sigma (side_rhs_fold_dg acc2 ps)"
  by (simp add: dep_program_side_rhs_fold_dg_char[OF wf])
```

The two order-preservation facts the generic fold asks for before it will carry its contributions' environment-monotonicity: the D/G instance's projection and its embedding.

```
lemma mono_dg_local: "mono (dg_local :: ('d::order, 'h::order) dg_state ⇒ 'd)"
  by (rule monoI) (simp add: less_eq_dg_state_def)

lemma mono_DG_bot:
  "mono (λd. DG d bot :: ('d::order, 'h::order_bot) dg_state)"
  by (rule monoI) simp
```

Environment-monotonicity of the fold, given every folded program is itself environment-monotone. The content is `traverse_fold_rhs_program_projected_mono`'s; this instance only supplies the two monotonicity facts above.

```
lemma side_rhs_fold_dg_mono:
  assumes wf: "∀p ∈ set ps. sp_wf p"
    and prog_mono: "∀p ∈ set ps. ∀s1 s2. s1 ≤ s2 ⟶ traverse_program p s1 ≤ traverse_program p s2"
  shows "⋀acc s1 s2. s1 ≤ s2 ⟹
           traverse_program (side_rhs_fold_dg acc ps) s1
             ≤ traverse_program (side_rhs_fold_dg acc ps) s2"
  using prog_mono
  unfolding side_rhs_fold_dg_def
  by (auto intro!: traverse_fold_rhs_program_projected_mono[OF wf mono_dg_local mono_DG_bot])
```

The fold's contributions are carried only by the per-contribution Side nodes; the accumulator flows into the final `Answer` (whose own sides are `bot`), so the side map is acc-independent — the same fact `∀p∈set ?ps. sp_wf p ⟹ dep_program ?sigma (side_rhs_fold_dg ?acc1.0 ?ps) = dep_program ?sigma (side_rhs_fold_dg ?acc2.0 ?ps)` established for dependencies, mirrored here for sides. This is what lets `⟦∀p∈set ?ps. sp_wf p; ∀p∈set ?ps. ∀s1 s2. s1 ≤ s2 ⟶ traverse_program p s1 ≤ traverse_program p s2; ?s1.0 ≤ ?s2.0⟧ ⟹ traverse_program (side_rhs_fold ?acc ?ps) ?s1.0 ≤ traverse_program (side_rhs_fold_dg ?acc ?ps) ?s2.0`'s proof strategy repeat for `sides_of_rhs` without also needing `traverse_rhs`-monotonicity as a hypothesis.

```
lemma sides_of_program_side_rhs_fold_dg_acc_indep:
  assumes wf: "∀p ∈ set ps. sp_wf p"
  shows "sides_of_program (side_rhs_fold_dg acc1 ps) sigma
     = sides_of_program (side_rhs_fold_dg acc2 ps) sigma"
  unfolding side_rhs_fold_dg_def
  by (rule sides_of_program_fold_rhs_program_projected_acc_indep[OF wf])
```

The declarative twin of `side_acc_dg`: `side_rhs_fold_dg`'s contribution at any one key is a plain fold over each element's own `sides_of_rhs`, seeded at `bot` rather than the running local accumulator – `∀p∈set ?ps. sp_wf p ⟹ sides_of_program (side_rhs_fold_dg ?acc1.0 ?ps) ?sigma = sides_of_program (side_rhs_fold_dg ?acc2.0 ?ps) ?sigma` already shows the accumulator never affects a side read, so unfolding `sp_lift_tree`'s side equation and re-seeding at `bot` after each step is exact, not merely a bound.

```
lemma sides_of_program_side_rhs_fold_dg_char:
  fixes ps :: "('x, 'k, ('d::bounded_semilattice_sup_bot, 'h::bounded_semilattice_sup_bot) dg_state,
                ('d, 'h) dg_state) strategy_program list"
  assumes wf: "∀p ∈ set ps. sp_wf p"
  shows "sides_of_program (side_rhs_fold_dg acc ps) τ z
         = (⨆p←ps. sides_of_program p τ z)"
  unfolding side_rhs_fold_dg_def
  by (rule sides_of_program_fold_rhs_program_projected_char[OF wf])

lemma side_acc_dg_as_foldr:
  "side_acc_dg acc τ ps = acc ⊔ (⨆p←ps. dg_local (traverse_program p τ))"
  by (simp add: side_acc_dg_def fold_acc_projected_as_foldr)
```

The same equation with the accumulator left as the fold's seed, which is how a proof that already has a right fold in hand meets it.

```
lemma side_acc_dg_as_foldr_seeded:
  "side_acc_dg acc τ ps = foldr (λp a. dg_local (traverse_program p τ) ⊔ a) ps acc"
  by (simp only: side_acc_dg_as_foldr foldr_join_seed_out)
```

Pushing a projection through a join-fold

A `dg_state` join is componentwise, so either projection commutes with a join-fold. These are what let a proof that has folded whole states read off one component afterwards, rather than carrying two folds along.

```
lemma dg_local_foldr_generic:
  "dg_local (foldr (λt acc'. h t ⊔ acc') L
      (acc::('d::bounded_semilattice_sup_bot, 'h::bounded_semilattice_sup_bot) dg_state))
    = foldr (λt acc'. dg_local (h t) ⊔ acc') L (dg_local acc)"
  by (induction L) simp_all

lemma dg_global_foldr_generic:
  "dg_global (foldr (λt acc'. h t ⊔ acc') L
      (acc::('d::bounded_semilattice_sup_bot, 'h::bounded_semilattice_sup_bot) dg_state))
    = foldr (λt acc'. dg_global (h t) ⊔ acc') L (dg_global acc)"
  by (induction L) simp_all

lemma dg_local_join_list:
  "dg_local (⨆t←L. h t :: ('d::bounded_semilattice_sup_bot, 'h::bounded_semilattice_sup_bot) dg_state)
    = (⨆t←L. dg_local (h t))"
  by (induction L) simp_all

lemma dg_global_join_list:
  "dg_global (⨆t←L. h t :: ('d::bounded_semilattice_sup_bot, 'h::bounded_semilattice_sup_bot) dg_state)
    = (⨆t←L. dg_global (h t))"
  by (induction L) simp_all

lemma foldr_dg_global_sides_char:
  fixes L :: "('x, 'k, ('d::bounded_semilattice_sup_bot, 'h::bounded_semilattice_sup_bot) dg_state,
               ('d, 'h) dg_state) strategy_program list"
  shows "(⨆t←L. dg_global (sides_of_program t τ z))
      = dg_global (⨆t←L. sides_of_program t τ z)"
  by (simp add: dg_global_foldr_generic foldr_join_list[symmetric])

lemma foldr_sides_dg_local_bot:
  fixes L :: "('x, 'k, ('d::bounded_semilattice_sup_bot, 'h::bounded_semilattice_sup_bot) dg_state,
               ('d, 'h) dg_state) strategy_program list"
  assumes "⋀t. t ∈ set L ⟹ dg_local (sides_of_program t τ z) = bot"
  shows "dg_local (⨆t←L. sides_of_program t τ z) = bot"
  using assms
  by (induction L) simp_all

lemma DG_sup_bot_left:
  "DG (bot::'d::bounded_semilattice_sup_bot) a ⊔ DG bot b = DG bot (a ⊔ b)"
  by (simp add: sup_dg_state_def bot_dg_state_def)
```

What one folded program contributes

Lower bounds, the counterpart of the upper bounds above: every folded program's answer and contribution is below the fold's, and the accumulator only grows. A soundness proof reaches one selected contribution through

these, having reached the fold itself through the generator.

All three are the same fact about a join-fold — `foldr_sup_member_le`, proved once with the rest of that algebra where the fold itself is defined — read off the two characterizations above.

```
lemma side_acc_dg_ge_acc:
  "acc ≤ side_acc_dg acc σ ts"
  by (simp add: side_acc_dg_as_foldr)

lemma dg_local_traverse_le_side_acc_dg:
  assumes "p ∈ set ps"
  shows "dg_local (traverse_program p σ) ≤ side_acc_dg acc σ ps"
  unfolding side_acc_dg_as_foldr
  using foldr_sup_member_le[where h = "λp. dg_local (traverse_program p σ)", OF assms]
  by (rule le_supI2)

lemma sides_le_side_rhs_fold_dg:
  assumes wf: "∀p ∈ set ps. sp_wf p"
    and mem: "p ∈ set ps"
  shows "sides_of_program p σ k ≤ sides_of_program (side_rhs_fold_dg acc ps) σ k"
  unfolding sides_of_program_side_rhs_fold_dg_char[OF wf]
  by (rule foldr_sup_member_le[OF mem])

end

end
```

6.4 DG_Manager

```
theory DG_Manager
  imports DG_State "Voblint_Solver.Strategy_Tree_Program" Analysis_Query
begin
```

A manager capability interface for the D/G packed carrier

`man` is a capability fragment of Goblint's own manager record: `man_local` is the current local value, `man_global`/`man_sideg` are *capabilities* a transfer runs to read or publish shared state, without naming which solver key that state lives at, and `man_ask` is the query channel, Goblint's `man.ask`. Goblint's manager carries more – an event emitter, the current node and edge, spawn and split – and none of that is modelled here, because no transfer in this development asks for it. Every `Spec` transfer takes a manager, including ones whose global component is trivial – Goblint's default `Spec` sets `G = Lattice.Unit`, `V = EmptyV`, and its transfers still take the manager and mostly return `man.local`.

`man_ask` answers a query about the stores the transfer's predecessor state describes. Like `man_global` it is a program, so a query handler may read shared state and the dependency shows up in the compiled equations. A handler may also ask in turn; the specification layer (`DG_Spec`) bounds the recursion and answers $\top$ on a cycle, as Goblint's `MCP` does.

Five type parameters run through this theory and every one built on it:

- `'x` — a local solver unknown, the name of a program point under a context.
- `'k` — a solver global key, the whole side-effect address space.
- `'v` — an analysis-visible global name, Goblint's `V`. It embeds into `'k`, and is exactly the part of that space an analysis may address.
- `'dl` — the local domain, Goblint's `D`.
- `'dg` — the analysis-global domain, Goblint's `G`.

Both capabilities take a global name rather than a key: an analysis says `man_global m v`, naming which of its globals it means, and never builds a solver key. Which key that name lives at is the manager's business — `mk_dg_man` takes the embedding `'v ⇒ 'k` and closes it into both fields. An analysis with a single global instantiates `'v = unit` and writes `man_global m ()`. The separation matters because `'k` also carries the routed activation slots, which belong to the generator and to no analysis.

A Base-style specification ignores the global channel entirely: it calls neither capability, so its compiled equations carry no `QueryG` and no `Side`. An effectful specification reaches the current routed global slot through `man_global`/`man_sideg`. Which of the two a given analysis is therefore shows up in its compiled trees, not in this interface.

`dg_read_global` and `dg_sideg` are what interpret those capabilities against the packed `('dl,'dg) dg_state` carrier; `mk_dg_man` is where the embedding `unknown_of` is closed into them, so it is the only place a name becomes a key. Either way a transfer written against `man`'s fields sees neither the packed carrier nor a key.

The manager record

```
record ('x,'k,'v,'dl,'dg) man =
  man_local :: 'dl
  man_global :: "'v ⇒ ('x,'k,('dl,'dg) dg_state,'dg) strategy_program"
  man_sideg :: "'v ⇒ 'dg ⇒ ('x,'k,('dl,'dg) dg_state,unit) strategy_program"
  man_ask :: "query ⇒ ('x,'k,('dl,'dg) dg_state,answer) strategy_program"
```

Packed-carrier primitives

A solver unknown holds a whole `dg_state`, so reading one yields both halves and only one of them is ever wanted. `dg_read_at` reads any unknown and keeps `dg_local`; `dg_read_global` reads a global key and keeps `dg_global`; `dg_sideg` publishes a global contribution, padding the local half it does not write with `bot`.

These three are where the packed carrier is taken apart and put back together. The framework's own transfer constructors use only these and `man`'s fields, and an analysis author is expected to do the same; the transfer type does not enforce it, since a transfer may build any `strategy_program` it likes. The tree formers that assemble a right-hand side do construct `DG` values directly.

```
definition dg_read_at :: "'x + 'k ⇒ ('x,'k,('dl,'dg) dg_state,'dl) strategy_program" where
  "dg_read_at src = sp_read_at src ⤜ (sp_return o dg_local)"
```

```
definition dg_read_global :: "'k ⇒ ('x,'k,('dl,'dg) dg_state,'dg) strategy_program" where
  "dg_read_global gk = sp_read_global gk ≫= (sp_return o dg_global)"

definition dg_sideg :: "'k ⇒ 'dg ⇒ ('x,'k,('dl::bot,'dg) dg_state,unit) strategy_program" where
  "dg_sideg gk gd = sp_publish gk (DG bot gd)"
```

What the primitives do, observed

Each primitive contributes one step to a compiled tree, and the three observations a solver makes of that tree — its answer, its publications, its dependencies — see exactly that step and then continue into the continuation. Proving it once here is what keeps later proofs off the encoding: without these, every fact about a compiled transfer has to case on the address and unfold `(≫=)`, `sp_return` and the read primitives to reach its own subject. They are `simp` rules in the direction that eliminates a primitive and exposes the value it read, and they terminate because the program strictly shrinks at each step.

```
lemma traverse_dg_read_at [simp]:
  "traverse_rhs (dg_read_at src K) τ = traverse_rhs (K (dg_local (τ src))) τ"
  by (cases src) (simp_all add: dg_read_at_def sp_bind_def sp_read_local_def
      sp_read_global_def sp_return_def)

lemma sides_dg_read_at [simp]:
  "sides_of_rhs (dg_read_at src K) τ = sides_of_rhs (K (dg_local (τ src))) τ"
  by (cases src) (simp_all add: dg_read_at_def sp_bind_def sp_read_local_def
      sp_read_global_def sp_return_def)

lemma dep_aux_dg_read_at [simp]:
  "dep_aux τ (dg_read_at src K) = insert src (dep_aux τ (K (dg_local (τ src))))"
  by (cases src) (simp_all add: dg_read_at_def sp_bind_def sp_read_local_def
      sp_read_global_def sp_return_def)

lemma traverse_dg_read_global [simp]:
  "traverse_rhs (dg_read_global gk K) τ = traverse_rhs (K (dg_global (τ (Inr gk)))) τ"
  by (simp add: dg_read_global_def sp_bind_def sp_read_global_def sp_return_def)

lemma sides_dg_read_global [simp]:
  "sides_of_rhs (dg_read_global gk K) τ = sides_of_rhs (K (dg_global (τ (Inr gk)))) τ"
  by (simp add: dg_read_global_def sp_bind_def sp_read_global_def sp_return_def)

lemma dep_aux_dg_read_global [simp]:
  "dep_aux τ (dg_read_global gk K)
     = insert (Inr gk) (dep_aux τ (K (dg_global (τ (Inr gk)))))"
  by (simp add: dg_read_global_def sp_bind_def sp_read_global_def sp_return_def)

lemma traverse_dg_sideg [simp]:
  "traverse_rhs (dg_sideg gk gd K) τ = traverse_rhs (K ()) τ"
  by (simp add: dg_sideg_def sp_publish_def)

lemma dep_aux_dg_sideg [simp]:
  "dep_aux τ (dg_sideg gk gd K) = dep_aux τ (K ())"
  by (simp add: dg_sideg_def sp_publish_def)
```

Publication is the one primitive an observation does not simply pass through: it raises the contribution at its own key and leaves every other key to the continuation.

```
lemma sides_dg_sideg [simp]:
  "sides_of_rhs (dg_sideg gk gd K) τ = sides_of_rhs (K ()) τ ⊔ bot(Inr gk := DG bot gd)"
  by (simp add: dg_sideg_def sp_publish_def Let_def fun_eq_iff sup_fun_def)
```

Each primitive runs its continuation once, so a transfer assembled from them does too and the fold's well-formedness side conditions discharge by `simp` wherever a concrete transfer is in hand.

```
lemma sp_wf_dg_read_at [intro, simp]: "sp_wf (dg_read_at src)"
  unfolding dg_read_at_def by (intro sp_wf_bind) (simp_all add: comp_def)

lemma sp_wf_dg_read_global [intro, simp]: "sp_wf (dg_read_global gk)"
  unfolding dg_read_global_def by (intro sp_wf_bind) (simp_all add: comp_def)

lemma sp_wf_dg_sideg [intro, simp]: "sp_wf (dg_sideg gk gd)"
  by (simp add: dg_sideg_def)
```

Constructing a manager

`mk_dg_man` closes the name-to-key embedding into both effectful fields, so a transfer built against `man` never sees a key itself. A manager-native transfer should reach shared state through the record fields rather than call

dg_read_global/dg_sideg or name a key directly, so that a future manager (instrumented, differently routed, or backed by distinct global unknowns) is a second interpretation of the same fields rather than a change to any transfer.

A built manager answers every query with ⊤, the claim that holds of every store. Whoever runs a specification installs a real query channel on top: the equation generator installs the specification's own query (dg_spec_edge_program), and a product of specifications installs the meet of its components' answers.

```
definition mk_dg_man :: "'dl::bot ⇒ ('v ⇒ 'k) ⇒ ('x, 'k, 'v, 'dl, 'dg) man" where
  "mk_dg_man d unknown_of = (|
     man_local = d,
     man_global = (λv. dg_read_global (unknown_of v)),
     man_sideg = (λv. dg_sideg (unknown_of v)),
     man_ask = (λ_. sp_return ⊤) |)"
```

What a built manager answers, and the one way it is rebuilt: a second transfer stage runs from the point the first reached, and since only the local value changes, that update is again a manager built at the new value. These are the rules the proofs need; mk_dg_man itself stays folded, so a manager travels through a goal as one term instead of a three-field record literal. Keeping mk_dg_man folded also preserves matching: an assumption about a built manager would stop matching if the goal decayed into a record literal.

The two capability rules are stated unapplied. Applying one is the same rule plus a beta step, so the unapplied form subsumes the applied one and also rewrites the occurrences that carry no argument — a manager passed whole to an entry transfer, for instance.

```
lemma mk_dg_man_simps [simp]:
  "man_local (mk_dg_man d unknown_of) = d"
  "man_global (mk_dg_man d unknown_of) = (λv. dg_read_global (unknown_of v))"
  "man_sideg (mk_dg_man d unknown_of) = (λv. dg_sideg (unknown_of v))"
  "man_ask (mk_dg_man d unknown_of) = (λ_. sp_return ⊤)"
  by (simp_all add: mk_dg_man_def)

lemma mk_dg_man_local_update [simp]:
  "mk_dg_man d unknown_of(|man_local := e|) = mk_dg_man e unknown_of"
  "mk_dg_man d unknown_of(|man_ask := A, man_local := e|) = mk_dg_man e unknown_of(|man_ask := A|)"
  by (simp_all add: mk_dg_man_def)

type_synonym ('x,'k,'v,'dl,'dg) man_transfer =
  "('x, 'k, 'v, 'dl, 'dg) man ⇒ ('x,'k,('dl,'dg) dg_state,'dl) strategy_program"
```

A query handler takes the same manager and answers the query in the answer lattice, Goblint's Spec.query man q.

```
type_synonym ('x,'k,'v,'dl,'dg) man_query =
  "('x, 'k, 'v, 'dl, 'dg) man ⇒ query ⇒ ('x,'k,('dl,'dg) dg_state,answer) strategy_program"
```

A combine-shaped transfer takes the same manager plus the callee-exit value as an ordinary trailing argument, the way Goblint's combine_env/combine_assign take the same man plus a D.t.

```
type_synonym ('x,'k,'v,'dl,'dg) man_combine_transfer =
  "('x, 'k, 'v, 'dl, 'dg) man ⇒ 'dl ⇒ ('x,'k,('dl,'dg) dg_state,'dl) strategy_program"
```

A call answers a list of alternatives, and each alternative is a pair: the value the caller resumes with once the callee returns, and the value the callee starts from. This is Goblint's Spec.enter, whose result is a list of such pairs. The two halves must be produced by one run of the transfer, because an alternative's continuation is only meaningful against the callee entry it was computed with; a program answering the pairs keeps them together and leaves the caller free to consume them however the equation shape needs.

The result is not a 'dl, so an entry transfer is not a man_transfer: it cannot be the answer of an equation on its own, and whatever consumes it must reduce the list to one local value first.

```
type_synonym 'dl enter_result = "'dl × 'dl"

type_synonym ('x,'k,'v,'dl,'dg) man_enter_transfer =
  "('x, 'k, 'v, 'dl, 'dg) man
   ⇒ ('x,'k,('dl,'dg) dg_state,'dl enter_result list) strategy_program"
```

A second transfer stage runs from the point the first reached by updating man_local in place — m(|man_local := d|) — leaving both capabilities as they were. Nothing is extracted out of the program and re-wrapped, and the record's own update equations are all the later proofs need about it.

```
end
```

6.5 DG_Spec

```
theory DG_Spec
  imports DG_Manager "Voblint_CFG.CFG_Def"
begin
```

What an analysis supplies: manager-native transfers

An analysis supplies one manager-native transfer per edge action and the pieces of the call/return protocol. A transfer takes a `man` and returns a `strategy_program`: global reads are `man_global` calls, global publications are `man_sideg` calls, and a transfer that makes neither compiles to a tree with no `QueryG` and no `Side` at all. This replaces the state-threading shape `'dl ⇒ 'dg ⇒ 'dg × 'dl`, which forced every transfer to republish the global it read even where the global channel is provably inert.

What a program answers with depends on the field. An edge transfer and a combine stage answer the successor local value, so their compiled tree is an equation's right-hand side outright. Entry answers a list of alternatives instead, which no equation carries: the routed call program consumes that list, running one activation and return combine per alternative and joining what they contribute.

There is one interface here, not a hierarchy of them. A specification is *local-only* when every field is a pure function of the values it is handed — `local_transfer` for the edge fields, `local_enter_transfer` for entry — and *effectful* when some field reads or publishes through `man_global`/`man_sideg`. Those words describe what a given specification's compiled trees do, and are read off those trees – they are not a classification the framework branches on.

A `analysis_event` is an analyzer-visible occurrence distinct from an ordinary control-flow transfer: a domain may observe it, but it must not by itself refine execution. The channel mirrors Goblint's separation of its ordinary `Spec` transfer methods (`assign`/`branch`/`skip`/...) from `Spec.event`. The check itself does not: Goblint handles `__goblint_check` in `special`, as the library description `Assert {check = true; refine = false}`, and its `Events.Assert` is emitted only by the `abortUnless` analysis and refines the state in Base (goblint/analyzer `5320a6b7`: `libraryFunctions.ml`, `assert.ml`, `base.ml`). Voblint's sole current event is a check's condition, which never refines; the vocabulary is deliberately left open rather than pre-populated, so that a future VIMP source construct with no current counterpart (e.g. a diagnostic-only annotation) adds a constructor here instead of a new specification field. A construct that narrows feasible execution, such as `assume`, is not a candidate: it belongs on `dgs_branch` or on a new refining field, not here.

It is declared here, beside the `dgs_event` field it types, because the specification record and the edge-action dispatcher that feeds it are the only things that ever case on it.

```
datatype analysis_event =
  Check_Event check_label exp

record ('x,'k,'v,'dl,'dg) dg_spec =
  dgs_skip          :: "('x, 'k, 'v, 'dl, 'dg) man_transfer" ("skip♯")
  dgs_assign        :: "var_id ⇒ exp ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("assign♯")
  dgs_special       :: "exp gspecial_call ⇒ var_id ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer"
    ("special♯")
  dgs_branch        :: "exp ⇒ bool ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("branch♯")
  dgs_body          :: "pname ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("body♯")
  dgs_return        :: "exp option ⇒ pname ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("return♯")
  dgs_enter         :: "call_info ⇒ ('x, 'k, 'v, 'dl, 'dg) man_enter_transfer"
    ("enter♯")
  dgs_event         :: "analysis_event ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("event♯")
  dgs_combine_env   :: "call_info ⇒ ('x, 'k, 'v, 'dl, 'dg) man_combine_transfer"
    ("combine'_env♯")
  dgs_combine_assign :: "call_info ⇒ ('x, 'k, 'v, 'dl, 'dg) man_combine_transfer"
    ("combine'_assign♯")
  dgs_query         :: "('x, 'k, 'v, 'dl, 'dg) man_query"
```

`dgs_query` is Goblint's `Spec.query`: it answers a query from the manager it is given, and may read globals or ask further queries through it. What an answer claims is fixed by `eval_holds`; answering ⊤ claims nothing.

Every field carries the Goblint `Spec` method name it answers to, marked ♯. A specification therefore reads in the analyzer's vocabulary, and the marker keeps the abstract operation apart from the concrete one it over-approximates: `combine_env` merges two stores, whereas `combine_env`♯ is the field a domain fills to describe that merge. The whole-state combine wears the same marker as `combine`♯, so it means one thing on both sides of the transfer boundary.

Two of the names contain an underscore, which a mixfix template would otherwise read as an argument slot, so they escape it as `'_`. The selector names stay available for record construction and update, neither of which takes notation.

Procedure-return combine is split the same way Goblint's `Spec` splits it: an environment merge followed by a return-value assign, both taking the same manager plus the callee exit. The composed form sequences them monadically – `combine_assign` runs from the point `combine_env` reached, against the same manager with `man_local` updated to that point, so any effects `combine_env` emitted stay in the program and nothing is extracted into a pure pair in between.

```
definition dg_spec_combine_transfer ::
  "('x, 'k, 'v, 'dl, 'dg) dg_spec ⇒ call_info ⇒ ('x, 'k, 'v, 'dl, 'dg) man_combine_transfer"
where
  "dg_spec_combine_transfer S ci m exit =
     do {
       d_env ← combine_env♯ S ci m exit;
       combine_assign♯ S ci (m(|man_local := d_env|)) exit
     }"
```

`EA_Check` routes through `event`♯ rather than `skip`♯: a check is an analysis event (on the channel that mirrors Goblint's `Spec.event`; Goblint's own check goes through `special`, see `analysis_event` above), and conflating it with skip would make a future domain's non-identity skip silently change what a check edge does. A concrete `dg_spec` therefore supplies its own notion of a check event directly, the same way it already supplies `body`♯/`return`♯.

```
fun dg_spec_step ::
  "('x, 'k, 'v, 'dl, 'dg, 'z) dg_spec_scheme ⇒ (var_id,exp) gedge_action ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer"
where
  "dg_spec_step S EA_Nop            = skip♯ S"
| "dg_spec_step S (EA_Assign x e)   = assign♯ S x e"
| "dg_spec_step S (EA_Special sc x) = special♯ S sc x"
| "dg_spec_step S (EA_Assume b)     = branch♯ S b True"
| "dg_spec_step S (EA_AssumeNot b)  = branch♯ S b False"
| "dg_spec_step S (EA_Body p)       = body♯ S p"
| "dg_spec_step S (EA_Ret e p)      = return♯ S e p"
| "dg_spec_step S (EA_Check l cnd)  = event♯ S (Check_Event l cnd)"
```

Compiling a specification to right-hand sides

An edge equation reads the source unknown, builds the manager around it with the routed key closed in, and runs the specification's transfer. Every generator's edge right-hand side is a `dg_spec_edge_program`.

The driver underneath: read the source unknown, close the key into a fresh manager around what came back, and run the transfer once. The transfer itself never sees the key, only whatever the manager's fields already close over.

```
definition transfer_program_at ::
  "('x, 'k, 'v, 'dl, 'dg) man_transfer ⇒ 'x + 'k ⇒ ('v ⇒ 'k)
   ⇒ ('x,'k,('dl::bot,'dg) dg_state,'dl) strategy_program"
where
  "transfer_program_at transfer src unknown_of =
     do {
       d ← dg_read_at src;
       transfer (mk_dg_man d unknown_of)
     }"

definition transfer_program ::
  "('x, 'k, 'v, 'dl::bot, 'dg::bot) man_transfer ⇒ 'x + 'k ⇒ ('v ⇒ 'k)
   ⇒ ('x,'k,('dl,'dg) dg_state,('dl,'dg) dg_state) strategy_program"
where
  "transfer_program T src unknown_of = sp_map (λd. DG d bot) (transfer_program_at T src unknown_of)"
```

Before the specification's edge transfer runs, its query channel is set to the specification's own answers about the state the edge starts from, as Goblint's `MCP` builds `outer_man` around the predecessor. Every transfer on the edge sees the same channel, and nothing it computes changes it.

A handler may ask in turn, through a manager whose channel is the same construction one level down. As in Goblint's `MCP.query'`, a query already being asked answers $\top$ instead of recursing, the result lattice's top on a cycle. The depth bound has no counterpart in Goblint: queries range over an infinite type, so cycle detection alone does not make the recursion total. Exhausting it aborts the generated program, so a bound that is too small shows up as a failed run and never as a silent loss of precision. Logically `Code.abort` is its fallback, here $\top$, which holds of every store, so the bound plays no part in soundness.

The channel depends on the specification only through its handler, so a specification derived by overriding other fields keeps the same channel.

```
definition query_depth :: nat where
  "query_depth = 1024"
```

```
fun ask_with ::
  "('x, 'k, 'v, 'dl, 'dg) man_query ⇒ nat ⇒ query set ⇒ ('x, 'k, 'v, 'dl, 'dg) man
   ⇒ query ⇒ ('x,'k,('dl,'dg) dg_state,answer) strategy_program"
where
  "ask_with Q 0 asked m q =
     Code.abort (STR ''query recursion exceeded query_depth'') (λ_. sp_return ⊤)"
| "ask_with Q (Suc n) asked m q =
     (if q ∈ asked then sp_return ⊤
      else Q (m⦇man_ask := ask_with Q n (insert q asked) m⦈) q)"

definition outer_man ::
  "('x, 'k, 'v, 'dl, 'dg) man_query ⇒ ('x, 'k, 'v, 'dl, 'dg) man ⇒ ('x, 'k, 'v, 'dl, 'dg) man"
where
  "outer_man Q m = m⦇man_ask := ask_with Q query_depth {} m⦈"

lemma outer_man_simps [simp]:
  "man_local (outer_man Q m) = man_local m"
  "man_global (outer_man Q m) = man_global m"
  "man_sideg (outer_man Q m) = man_sideg m"
  "man_ask (outer_man Q m) = ask_with Q query_depth {} m"
  by (simp_all add: outer_man_def)

definition dg_spec_edge_program ::
  "('x, 'k, 'v, 'dl::bot, 'dg::bot) dg_spec ⇒ (var_id,exp) gedge_action ⇒ 'x + 'k ⇒ ('v ⇒ 'k)
   ⇒ ('x,'k,('dl,'dg) dg_state,('dl,'dg) dg_state) strategy_program"
where
  "dg_spec_edge_program S a src unknown_of =
     transfer_program (λm. dg_spec_step S a (outer_man (dgs_query S) m)) src unknown_of"
```

What compilation costs an observation: exactly the source read, and then the transfer run at the value that read produced. Everything a proof about a compiled edge needs from the encoding is here, so nothing downstream has to case on the address or unfold the program monad again.

Left untagged. Rewriting a compiled tree back into a manager application grows the term, and the direction is only wanted where a proof is about to reason about the transfer itself; the primitive laws in `Voblint_Framework.DG_Manager` are the ones that should fire everywhere.

```
lemma traverse_transfer_program:
  "traverse_program (transfer_program T src unknown_of) τ
     = traverse_rhs (sp_compile_with (λd. DG d bot) (T (mk_dg_man (dg_local (τ src)) unknown_of))) τ"
  by (simp add: transfer_program_def transfer_program_at_def sp_compile_with_def sp_bind_def)

lemma sides_transfer_program:
  "sides_of_program (transfer_program T src unknown_of) τ
     = sides_of_rhs (sp_compile_with (λd. DG d bot) (T (mk_dg_man (dg_local (τ src)) unknown_of))) τ"
  by (simp add: transfer_program_def transfer_program_at_def sp_compile_with_def sp_bind_def)

lemma dep_transfer_program:
  "dep_program τ (transfer_program T src unknown_of)
     = insert src
         (dep_aux τ (sp_compile_with (λd. DG d bot) (T (mk_dg_man (dg_local (τ src)) unknown_of))))"
  by (simp add: transfer_program_def transfer_program_at_def sp_compile_with_def sp_bind_def)
```

Specifications whose transfers run their continuation once

A `man_transfer` is a function from a continuation to a tree, so a specification could in principle write one that drops or duplicates it. `sp_wf` rules that out, and the generator's fold needs it: what a contribution publishes is only well posed for a transfer that runs its continuation once.

`dg_spec_wf` asks it of the four ways a specification is run — an edge step, a query, the entry alternatives, and the two-stage combine — and only at the managers that actually occur, the ones `mk_dg_man` builds. An edge step and a query handler run under a query channel installed around that manager, so they are asked for every channel that is itself well formed; a wrapper that reruns a step at a rebuilt manager needs exactly that generality. That restriction is not a weakening: a transfer never meets any other manager, and an arbitrary one has opaque capability fields about which nothing could be said. A specification written with the manager's own primitives satisfies all four by `simp`, since every primitive and every combinator carries a closure lemma.

```
definition dg_spec_wf :: "('x, 'k, 'v, 'dl::bot, 'dg) dg_spec ⇒ bool" where
  "dg_spec_wf S ⟷
     (∀a d unknown_of ch. (∀q. sp_wf (ch q))
          ⟶ sp_wf (dg_spec_step S a ((mk_dg_man d unknown_of)⦇man_ask := ch⦈)))
     ∧ (∀d unknown_of ch q. (∀q'. sp_wf (ch q'))
          ⟶ sp_wf (dgs_query S ((mk_dg_man d unknown_of)⦇man_ask := ch⦈) q))
     ∧ (∀ci d unknown_of. sp_wf (enter♯ S ci (mk_dg_man d unknown_of)))
     ∧ (∀ci d unknown_of ex. sp_wf (dg_spec_combine_transfer S ci (mk_dg_man d unknown_of) ex))"
```

```
lemma dg_spec_wf_step_ask:
  "dg_spec_wf S ⟹ (⋀q. sp_wf (ch q))
   ⟹ sp_wf (dg_spec_step S a ((mk_dg_man d unknown_of)(|man_ask := ch|)))"
  by (simp add: dg_spec_wf_def)

lemma dg_spec_wf_query:
  "dg_spec_wf S ⟹ (⋀q. sp_wf (ch q))
   ⟹ sp_wf (dgs_query S ((mk_dg_man d unknown_of)(|man_ask := ch|)) q)"
  by (simp add: dg_spec_wf_def)

lemma sp_wf_ask_with:
  assumes "dg_spec_wf S"
  shows "sp_wf (ask_with (dgs_query S) n asked (mk_dg_man d unknown_of) q)"
proof (induction n arbitrary: asked q)
  case (Suc n)
  then show ?case
    using assms by (simp add: dg_spec_wf_def)
qed simp

lemma dg_spec_wf_step:
  "dg_spec_wf S ⟹ sp_wf (dg_spec_step S a (outer_man (dgs_query S) (mk_dg_man d unknown_of)))"
  unfolding outer_man_def by (intro dg_spec_wf_step_ask sp_wf_ask_with)

lemma dg_spec_wf_enter:
  "dg_spec_wf S ⟹ sp_wf (enter♯ S ci (mk_dg_man d unknown_of))"
  by (simp add: dg_spec_wf_def)

lemma dg_spec_wf_combine:
  "dg_spec_wf S ⟹ sp_wf (dg_spec_combine_transfer S ci (mk_dg_man d unknown_of) ex)"
  by (simp add: dg_spec_wf_def)

lemma sp_wf_transfer_program [intro]:
  assumes "⋀d. sp_wf (T (mk_dg_man d unknown_of))"
  shows "sp_wf (transfer_program T src unknown_of)"
  unfolding transfer_program_def transfer_program_at_def
  by (intro sp_wf_map sp_wf_bind sp_wf_dg_read_at assms)

lemma sp_wf_dg_spec_edge_program [intro]:
  "dg_spec_wf S ⟹ sp_wf (dg_spec_edge_program S a src unknown_of)"
  unfolding dg_spec_edge_program_def
  by (intro sp_wf_transfer_program) (rule dg_spec_wf_step)
```

Writing a transfer

An analysis author writes manager-native transfers and nothing else. The framework turns them into solver right-hand sides; `QueryL`, `QueryG` and `Side` belong to that step, and a specification field that names one has reached past the interface into the solver's. That is a discipline, not something the types rule out: a transfer returns a `strategy_program` and could build any of them by hand. Three recipes cover every field of a `dg_spec` without doing so:

- A transfer that only transforms the local value is `local_transfer f`, and its compiled tree is one read and an answer: no `QueryG`, no `Side`.
- A transfer that reads or publishes shared state runs the capabilities in sequence and answers the new local value — `do {g <- man_global m v; _ <- man_sideg m v g'; sp_return d'}` — which is the same shape as the Goblint method that reads `man.global`, calls `man.sideg`, and returns a `D.t`.
- An entry transfer answers the list of alternatives instead of one value: `sp_return [(continuation, entry), ...]`, possibly after the same capability calls. It never folds that list; the routed call program does.

Local-only transfers

The local-only shape: a pure transformation of the local value, touching no global at all. Its compiled edge tree is one source read followed directly by the answer – no `QueryG`, no `Side` – so its side contribution is `bot` at every key and its dependency set is exactly the source unknown. The global channel simply does not appear in these compile-down facts, which is what makes a local-only specification's soundness argument collapse to plain inclusions on pure functions.

```
definition local_transfer :: "('dl ⇒ 'dl) ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" where
  "local_transfer f m = sp_return (f (man_local m))"
```

```
lemma local_transfer_outer_man [simp]:
  "local_transfer f (outer_man Q m) = local_transfer f m"
  by (simp add: local_transfer_def)
```

Answers and the local query handler

A transfer that consults the query channel receives the answers as an ordinary function argument, so a proof about the transfer quantifies over them. A question nobody answers is answered $\top$, which claims nothing.

```
type_synonym channel = "query ⇒ answer"
```

The local query handler: answers are a pure function of the local value. Such a handler never asks in turn, so the recursive channel the generator installs around it reduces to the handler itself.

```
definition local_query :: "('dl ⇒ channel) ⇒ ('x, 'k, 'v, 'dl, 'dg) man_query" where
  "local_query h m q = sp_return (h (man_local m) q)"

lemma local_query_update [simp]:
  "local_query h (m⦇man_ask := ch⦈) = local_query h m"
  by (simp add: local_query_def fun_eq_iff)

lemma local_transfer_update [simp]:
  "local_transfer f (m⦇man_ask := ch⦈) = local_transfer f m"
  by (simp add: local_transfer_def)

lemma ask_with_local_query:
  "ask_with (local_query h) n asked m q
     = sp_return (if n = 0 ∨ q ∈ asked then ⊤ else h (man_local m) q)"
  by (cases n) (simp_all add: local_query_def)

lemma outer_man_local_query [simp]:
  "outer_man (local_query h) m = m⦇man_ask := local_query h m⦈"
  unfolding outer_man_def
  by (simp add: ask_with_local_query query_depth_def local_query_def fun_eq_iff)
```

The entry counterpart: the alternatives are a pure function of the caller value, so the program reads no unknown and publishes nothing. `f` answers the whole list, which is what lets a local-only specification still offer several alternatives.

```
definition local_enter_transfer ::
  "('dl ⇒ 'dl enter_result list) ⇒ ('x, 'k, 'v, 'dl, 'dg) man_enter_transfer"
where
  "local_enter_transfer f m = sp_return (f (man_local m))"

lemma sp_compile_transfer_program_local_transfer:
  "sp_compile (transfer_program (local_transfer f) (Inl x) unknown_of)
     = QueryL x (λa. Answer (DG (f (dg_local a)) bot))"
  "sp_compile (transfer_program (local_transfer f) (Inr g) unknown_of)
     = QueryG g (λa. Answer (DG (f (dg_local a)) bot))"
  by (simp_all add: transfer_program_def transfer_program_at_def local_transfer_def
      dg_read_at_def sp_bind_assoc sp_compile_def sp_compile_with_def
      sp_bind_def sp_return_def sp_read_local_def sp_read_global_def)

lemma traverse_local_transfer_program [simp]:
  "traverse_program (transfer_program (local_transfer f) src unknown_of) τ = DG (f (dg_local (τ src))) bot"
  by (cases src) (simp_all add: sp_compile_transfer_program_local_transfer)

lemma sides_local_transfer_program [simp]:
  "sides_of_program (transfer_program (local_transfer f) src unknown_of) τ k = bot"
  by (cases src) (simp_all add: sp_compile_transfer_program_local_transfer)

lemma dep_local_transfer_program [simp]:
  "dep_program τ (transfer_program (local_transfer f) src unknown_of) = {src}"
  by (cases src) (simp_all add: sp_compile_transfer_program_local_transfer)
```

Local-only combine

The combine counterpart: a pure function of the caller-continuation and callee exit states, no global contact. When both stages — `combine_env`$^\sharp$ and `combine_assign`$^\sharp$ — are local, the whole return pipeline collapses monadically to one pure composition – the sequencing updates `man_local` and extracts nothing – and the compiled combine tree is two reads and an answer, with no contribution and dependencies exactly the two sources.

```
definition local_combine_transfer ::
  "('dl ⇒ 'dl ⇒ 'dl) ⇒ ('x, 'k, 'v, 'dl, 'dg) man_combine_transfer"
```

```
where
  "local_combine_transfer f m exit = sp_return (f (man_local m) exit)"

lemma dg_spec_combine_transfer_local:
  assumes "combine_env♯ S ci = local_combine_transfer ce"
    and "combine_assign♯ S ci = local_combine_transfer ca"
  shows "dg_spec_combine_transfer S ci m exit
         = sp_return (ca (ce (man_local m) exit) exit)"
  by (simp add: dg_spec_combine_transfer_def assms local_combine_transfer_def)
```

The reads every compiled transfer makes

A compiled edge tree begins by reading its source unknown, and a compiled combine tree by reading the call site and the callee exit, before the transfer runs at all. Those reads are therefore dependencies of *any* specification's tree, effectful or not – unlike the global key, which appears only when the transfer actually queries it. Coverage arguments need exactly these, so they are stated once here rather than as an exact dependency set that would hold only for one kind of specification.

```
lemma dep_transfer_program_source:
  "src ∈ dep_program τ (transfer_program T src gk)"
  by (simp add: dep_transfer_program)

lemma dep_dg_spec_edge_program_source:
  "src ∈ dep_program τ (dg_spec_edge_program S a src gk)"
  unfolding dg_spec_edge_program_def by (rule dep_transfer_program_source)
```

A default specification, and overriding its fields

`local_dg_spec_template` is the identity local-only specification: every transfer hands back the local value it was given, and a return combine keeps the caller's. An analysis starts from it and overrides the fields it implements – `local_dg_spec_template(|dgs_assign := ..., dgs_branch := ...|)` – the way a Goblint `Spec` includes `DefaultSpec` and defines only the transfers it needs.

It is named a template, not a default, because it is not Goblint's default behavior and nothing here treats it as one: it is the record every field of which is still to be chosen.

This is a construction scaffold, not a soundness result. Leaving a field at the identity is a claim about that analysis: that ignoring the edge is sound for it. It is false in general – an assignment analysis that leaves `dgs_assign` at the identity does not track assignments – and nothing here checks it. Whether a specification, defaulted fields included, means anything is settled by `analysis_contract` together with the entry obligation that locale deliberately leaves open — never by how the record was assembled.

It is also not a transcription of Goblint's `IdentitySpec`, and the return combine is where the two differ: both stages here keep the value they are handed, so the composed default keeps the *caller* continuation, whereas Goblint's `combine_env` answers the callee exit and its `combine_assign` answers `man.local`, so the composed default there ends at the callee's value. Both are neutral for their own purpose; this one is neutral for *record update*. A wrapper that overrides only the final stage relies on that: were the default to substitute the callee exit first, the wrapper would run against a continuation the caller never had. Changing the default is therefore an audit of every partial override, not a one-line edit.

```
definition local_dg_spec_template :: "('x, 'k, 'v, 'D, 'G) dg_spec" where
  "local_dg_spec_template = (|
     dgs_skip = local_transfer id,
     dgs_assign = (λx e. local_transfer id),
     dgs_special = (λsc x. local_transfer id),
     dgs_branch = (λb pol. local_transfer id),
     dgs_body = (λp. local_transfer id),
     dgs_return = (λe p. local_transfer id),
     dgs_enter = (λci. local_enter_transfer (λd. [(d, d)])),
     dgs_event = (λev. local_transfer id),
     dgs_combine_env = (λci. local_combine_transfer (λd de. d)),
     dgs_combine_assign = (λci. local_combine_transfer (λd de. d)),
     dgs_query = (λm q. sp_return ⊤) |)"
```

The argument-free template must generalize in generated ML. Expanding the local transfer makes its skip field a function value, avoiding the value restriction on the application `local_transfer id` while keeping the template itself named.

```
lemmas [code] = local_dg_spec_template_def[unfolded local_transfer_def]

lemma local_dg_spec_template_simps [simp]:
  "skip♯ local_dg_spec_template = local_transfer id"
  "assign♯ local_dg_spec_template x e = local_transfer id"
```

```
  "special♯ local_dg_spec_template sc x = local_transfer id"
  "branch♯ local_dg_spec_template b pol = local_transfer id"
  "body♯ local_dg_spec_template p = local_transfer id"
  "return♯ local_dg_spec_template eo p = local_transfer id"
  "enter♯ local_dg_spec_template ci = local_enter_transfer (λd. [(d, d)])"
  "event♯ local_dg_spec_template ev = local_transfer id"
  "combine_env♯ local_dg_spec_template ci = local_combine_transfer (λd de. d)"
  "combine_assign♯ local_dg_spec_template ci = local_combine_transfer (λd de. d)"
  "dgs_query local_dg_spec_template m q = sp_return ⊤"
  by (simp_all add: local_dg_spec_template_def)
```

The pure edge dispatch

`local_spec_step` is the pure counterpart of `dg_spec_step`'s dispatch: it selects the operation the edge names. A component's step on an arbitrary edge is this dispatch over its per-edge fields.

```
fun local_spec_step ::
  "('D ⇒ 'D) ⇒ (var_id ⇒ exp ⇒ 'D ⇒ 'D) ⇒ (exp gspecial_call ⇒ var_id ⇒ 'D ⇒ 'D)
   ⇒ (exp ⇒ bool ⇒ 'D ⇒ 'D) ⇒ (pname ⇒ 'D ⇒ 'D)
   ⇒ (exp option ⇒ pname ⇒ 'D ⇒ 'D)
   ⇒ (analysis_event ⇒ 'D ⇒ 'D) ⇒ (var_id,exp) gedge_action ⇒ 'D ⇒ 'D"
where
  "local_spec_step sk asn sp br bd rt ev EA_Nop = sk"
| "local_spec_step sk asn sp br bd rt ev (EA_Assign x e) = asn x e"
| "local_spec_step sk asn sp br bd rt ev (EA_Special sc x) = sp sc x"
| "local_spec_step sk asn sp br bd rt ev (EA_Assume b) = br b True"
| "local_spec_step sk asn sp br bd rt ev (EA_AssumeNot b) = br b False"
| "local_spec_step sk asn sp br bd rt ev (EA_Body p) = bd p"
| "local_spec_step sk asn sp br bd rt ev (EA_Ret e p) = rt e p"
| "local_spec_step sk asn sp br bd rt ev (EA_Check l cnd) = ev (Check_Event l cnd)"

fun event_action :: "analysis_event ⇒ (var_id,exp) gedge_action" where
  "event_action (Check_Event l cnd) = EA_Check l cnd"

end
```

6.6 DG_Spec_Sound

```
theory DG_Spec_Sound
  imports DG_Spec "Voblint_CFG.CFG_Transfer"
begin
```

What a manager-native specification owes a concretization

Soundness of a manager-native `dg_spec` relates its returned local value to an environment of analysis-visible globals. The manager maps each global name to a solver key; `genv` reads the global component at those keys.

A transfer starts in the input environment and establishes its result in that environment joined pointwise with its publications. Unchanged globals therefore retain their meaning without being republished. A specification may read and publish several names, all interpreted by the same environment. Concretizations that ignore globals discharge the global obligations vacuously. The unit-name compatibility lemma retains the older sufficient condition that publications alone describe the resulting global state.

What a call's alternatives must establish. The coverage contract is the part that does not mention a strategy program: `enter_runs` and `enter_deps` below do describe an entry program and the continuations it is handed, but what an alternative *means* is settled here, before any of that. `pairs` covers a concrete caller when *some* alternative accounts for it: the caller store lies in the continuation half and the store the callee starts from lies in the entry half, in the same pair. The quantifier is existential because alternatives are a disjunction — a call that splits into two of them has each covering the concrete runs it was computed for, and demanding that every alternative cover every caller would defeat the splitting the list exists to express.

Keeping it separate from any particular program is what lets an entry obligation be discharged once and reused: a proof about how alternatives are folded never has to restate what an alternative means.

```
definition entry_pairs_cover ::
  "('D ⇒ 's set) ⇒ 's ⇒ 's ⇒ 'D enter_result list ⇒ bool"
where
  "entry_pairs_cover gammaD caller entered pairs ⟷
     (∃cont entry. (cont, entry) ∈ set pairs
        ∧ caller ∈ gammaD cont ∧ entered ∈ gammaD entry)"

lemma entry_pairs_coverI [intro]:
  assumes "(cont, entry) ∈ set pairs"
    and "caller ∈ gammaD cont" and "entered ∈ gammaD entry"
  shows "entry_pairs_cover gammaD caller entered pairs"
  using assms unfolding entry_pairs_cover_def by blast

lemma entry_pairs_coverE [elim]:
  assumes "entry_pairs_cover gammaD caller entered pairs"
  obtains cont entry
    where "(cont, entry) ∈ set pairs"
      and "caller ∈ gammaD cont" and "entered ∈ gammaD entry"
  using assms unfolding entry_pairs_cover_def by blast

lemma entry_pairs_cover_Nil [simp]:
  "¬ entry_pairs_cover gammaD caller entered []"
  by (simp add: entry_pairs_cover_def)
```

An entry transfer is a program, so its alternatives cannot be read back out of it: the tree it builds answers into the solver's carrier, and a list of pairs does not live there. What *can* be said is how it behaves under a fixed solution. `enter_runs T m sigma pairs pub` says that under `sigma` the program hands its continuation exactly `pairs` and publishes exactly `pub` on the way, whatever the continuation then does — the reads resolve against `sigma`, so both are determined before any continuation is chosen.

This is what lets a proof name an alternative. Rather than extracting the list after the fact, a caller fixes the `pairs` the continuation receives and reasons under that, which is the ordinary way to reason about a continuation-passing program. `pub` is carried separately because the prefix's own publications are part of what the call establishes: an entry that reads a global and republishes it has those contributions in `pub`, not in anything the continuation returns.

```
definition enter_runs ::
  "('x, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) man_enter_transfer
   ⇒ ('x, 'k, 'v, 'D, 'G) man ⇒ ('x + 'k ⇒ ('D,'G) dg_state)
   ⇒ 'D enter_result list ⇒ ('x + 'k ⇒ ('D,'G) dg_state) ⇒ bool"
where
  "enter_runs T m sigma pairs pub ⟷
     (∀K. traverse_rhs (T m K) sigma = traverse_rhs (K pairs) sigma
          ∧ sides_of_rhs (T m K) sigma = pub ⊔ sides_of_rhs (K pairs) sigma)"
```

```
lemma enter_runsD_traverse:
  assumes "enter_runs T m sigma pairs pub"
  shows "traverse_rhs (T m K) sigma = traverse_rhs (K pairs) sigma"
  using assms unfolding enter_runs_def by blast

lemma enter_runsD_sides:
  assumes "enter_runs T m sigma pairs pub"
  shows "sides_of_rhs (T m K) sigma = pub ⊔ sides_of_rhs (K pairs) sigma"
  using assms unfolding enter_runs_def by blast
```

A pure entry hands over its alternatives directly and publishes nothing.

```
lemma enter_runs_local_enter_transfer [intro]:
  "enter_runs (local_enter_transfer f) m sigma (f (man_local m)) bot"
  by (simp add: enter_runs_def local_enter_transfer_def sp_return_def
      sup_fun_def bot_fun_def)
```

The same at a built manager, where `man_local` has already been read off. A caller whose alternatives are written out as a list — which is how every routed instance states its entry obligation — matches this one by ordinary unification, where the rule above would first have to reduce `man_local (mk_dg_man d key)` under a higher-order variable.

```
lemma enter_runs_local_enter_transfer_mk_dg_man [intro]:
  "enter_runs (local_enter_transfer f) (mk_dg_man d unknown_of) sigma (f d) bot"
  using enter_runs_local_enter_transfer[of f "mk_dg_man d unknown_of" sigma] by simp
```

The converse direction, which a consumer needs when it has an arbitrary run in hand rather than the one above: a pure entry's publication is pinned, so no case analysis on the run is possible.

```
lemma enter_runs_local_pub_bot:
  assumes "enter_runs (local_enter_transfer f) m sigma pairs pub"
  shows "pub = bot"
proof -
  have "sides_of_rhs (local_enter_transfer f m (λ_. Answer bot)) sigma
          = pub ⊔ sides_of_rhs (Answer bot) sigma"
    using assms by (rule enter_runsD_sides)
  then show ?thesis
    by (simp add: local_enter_transfer_def sp_return_def fun_eq_iff sup_fun_def bot_fun_def)
qed
```

Values and publications are not everything the generator needs to know about an entry program: what it *reads* matters too, and reading is invisible in both. An entry that consults a global depends on that unknown even when it publishes `bot` there, and the solver's admissibility arguments are stated over dependencies, not over answers.

`enter_deps` is deliberately a separate relation over the *same* `pairs`: a proof that obtained its alternatives from `enter_runs` and its dependencies from an independently quantified list would be talking about two different executions. Packaging the two together, with `entry_pairs_cover`, is what keeps one call's story single.

```
definition enter_deps ::
  "('x, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) man_enter_transfer
   ⇒ ('x, 'k, 'v, 'D, 'G) man ⇒ ('x + 'k ⇒ ('D,'G) dg_state)
   ⇒ 'D enter_result list ⇒ ('x + 'k) set ⇒ bool"
where
  "enter_deps T m sigma pairs deps ⟷
     (∀K. dep_aux sigma (T m K) = deps ∪ dep_aux sigma (K pairs))"

lemma enter_depsD:
  assumes "enter_deps T m sigma pairs deps"
  shows "dep_aux sigma (T m K) = deps ∪ dep_aux sigma (K pairs)"
  using assms unfolding enter_deps_def by blast
```

A pure entry reads nothing of its own.

```
lemma enter_deps_local_enter_transfer [intro]:
  "enter_deps (local_enter_transfer f) m sigma (f (man_local m)) {}"
  by (simp add: enter_deps_def local_enter_transfer_def sp_return_def)

lemma enter_deps_local_enter_transfer_mk_dg_man [intro]:
  "enter_deps (local_enter_transfer f) (mk_dg_man d unknown_of) sigma (f d) {}"
  using enter_deps_local_enter_transfer[of f "mk_dg_man d unknown_of" sigma] by simp
```

The analysis contract carries everything that is independent of how a call is compiled: the ordinary edge transfers and the return combine. Entry is not here. A $\texttt{enter}^\sharp$ answers a list, which is not an equation's answer, so what makes it sound is a property of the program the routed generator builds from it, stated through

`entry_pairs_cover` against the routed alternative the solved system actually selects — a property this locale alone cannot state, since it never mentions how a call compiles.

So `analysis_contract` is the entry-independent part and not a complete soundness statement: interpreting it alone leaves a call's entry entirely unconstrained. `routed_context` (`Routed_Context`, downstream in this session) is the complete statement, extending this contract with that entry obligation for the routed equation shape, and is what an analysis should be asked to establish.

Proof-level compiled combine form

A return combine compiled the way an edge transfer is: read the caller continuation and the callee exit from two unknowns, and run the composed combine against them. No generator builds this. The routed call program already holds the alternative's own continuation, so it runs `dg_spec_combine_transfer` against that value directly and never reads a caller unknown a second time.

It lives here, ahead of the locale that consumes it, because *stating* the return obligation at a pair of addresses is all it is for: a proof that quantifies over the pair needs a program to quantify over. Reading it in `Voblint_Framework.DG_Spec`, among the formers a generator really does build, invited exactly the wrong conclusion.

Both levels earn their place, which is why there are two of them. `combine_transfer_program` is used at an arbitrary `man_combine_transfer`, while `dg_spec_combine_program` is used at a selected specification, by `combine_sound_program` below, which restates the return obligation at two *solver addresses*. That is what the value form cannot express: a caller bounding the program from a post-solution reads it at two unknowns, so the obligation has to name the addresses, not the values they hold. `combine_program_at` is the one internal step, kept for symmetry with the edge driver rather than for a second consumer.

```
definition combine_program_at ::
  "('x, 'k, 'v, 'dl, 'dg) man_combine_transfer
   ⇒ 'x + 'k ⇒ 'x + 'k ⇒ ('v ⇒ 'k)
   ⇒ ('x,'k,('dl::bot,'dg) dg_state,'dl) strategy_program"
where
  "combine_program_at transfer src_cc src_ex unknown_of =
     do {
       dc ← dg_read_at src_cc;
       de ← dg_read_at src_ex;
       transfer (mk_dg_man dc unknown_of) de
     }"

definition combine_transfer_program ::
  "('x, 'k, 'v, 'dl::bot, 'dg::bot) man_combine_transfer ⇒ 'x + 'k ⇒ 'x + 'k ⇒ ('v ⇒ 'k)
   ⇒ ('x,'k,('dl,'dg) dg_state,('dl,'dg) dg_state) strategy_program"
where
  "combine_transfer_program T src_cc src_ex unknown_of =
     sp_map (λd. DG d ⊥) (combine_program_at T src_cc src_ex unknown_of)"

definition dg_spec_combine_program ::
  "('x, 'k, 'v, 'dl::bot, 'dg::bot) dg_spec ⇒ call_info ⇒ 'x + 'k ⇒ 'x + 'k ⇒ ('v ⇒ 'k)
   ⇒ ('x,'k,('dl,'dg) dg_state,('dl,'dg) dg_state) strategy_program"
where
  "dg_spec_combine_program S ci src_cc src_ex unknown_of =
     combine_transfer_program (dg_spec_combine_transfer S ci) src_cc src_ex unknown_of"
```

What compilation costs an observation here: the two source reads, and then the composed combine run at the values they produced. The counterpart of `Voblint_Framework.DG_Spec`'s edge bridges, and untagged for the same reason.

```
lemma traverse_combine_transfer_program:
  "traverse_program (combine_transfer_program T src_cc src_ex unknown_of) τ
     = traverse_rhs (sp_compile_with (λd. DG d ⊥)
         (T (mk_dg_man (dg_local (τ src_cc)) unknown_of) (dg_local (τ src_ex)))) τ"
  by (simp add: combine_transfer_program_def combine_program_at_def sp_compile_with_def
      sp_bind_def)

lemma sides_combine_transfer_program:
  "sides_of_program (combine_transfer_program T src_cc src_ex unknown_of) τ
     = sides_of_rhs (sp_compile_with (λd. DG d ⊥)
         (T (mk_dg_man (dg_local (τ src_cc)) unknown_of) (dg_local (τ src_ex)))) τ"
  by (simp add: combine_transfer_program_def combine_program_at_def sp_compile_with_def
      sp_bind_def)

lemma dep_combine_transfer_program:
  "dep_program τ (combine_transfer_program T src_cc src_ex unknown_of)
```

```
    = insert src_cc (insert src_ex
        (dep_aux τ (sp_compile_with (λd. DG d ⊥)
          (T (mk_dg_man (dg_local (τ src_cc)) unknown_of) (dg_local (τ src_ex))))))"
  by (simp add: combine_transfer_program_def combine_program_at_def sp_compile_with_def
      sp_bind_def)
```

What the compiled form observes when both stages are local — the combine counterparts of the edge-tree facts in `Voblint_Framework.DG_Spec`.

```
lemma traverse_local_combine_program [simp]:
  "traverse_program (combine_transfer_program (local_combine_transfer h) src_cc src_ex gk) τ
     = DG (h (dg_local (τ src_cc)) (dg_local (τ src_ex))) bot"
  by (simp add: traverse_combine_transfer_program local_combine_transfer_def
      sp_compile_with_def sp_return_def)

lemma sides_local_combine_program [simp]:
  "sides_of_program (combine_transfer_program (local_combine_transfer h) src_cc src_ex gk) τ k
     = bot"
  by (simp add: sides_combine_transfer_program local_combine_transfer_def
      sp_compile_with_def sp_return_def)

lemma dep_local_combine_program [simp]:
  "dep_program τ (combine_transfer_program (local_combine_transfer h) src_cc src_ex gk)
     = {src_cc, src_ex}"
  by (simp add: dep_combine_transfer_program local_combine_transfer_def
      sp_compile_with_def sp_return_def)
```

What a manager-native specification owes

A specification names its global facts by `'v`, and a key map places each name at a solver unknown. The global environment a valuation holds under a key map is what it holds at those unknowns; the same reading of a program's side-effects is the environment the program publishes.

```
definition genv :: "('v ⇒ 'k) ⇒ ('x + 'k ⇒ ('dl, 'dg) dg_state) ⇒ 'v ⇒ 'dg" where
  "genv key τ v = dg_global (τ (Inr (key v)))"
```

Applied to a name, the environment is the lookup it stands for.

```
declare genv_def [simp]
```

Every name at one key reads one value: the environment of a single slot.

```
lemma genv_const [simp]: "genv (λ_. k) τ = (λ_. dg_global (τ (Inr k)))"
  by (simp add: fun_eq_iff)
```

What an edge's compiled program answers under τ: its local result and the global environment its side-effects publish. The combine counterparts run the composed combine at a continuation `dc` and a callee exit `de`.

```
abbreviation edge_out where
  "edge_out S a src key τ ≡ dg_local (traverse_program (dg_spec_edge_program S a src key) τ)"

abbreviation edge_pub where
  "edge_pub S a src key τ ≡ genv key (sides_of_program (dg_spec_edge_program S a src key) τ)"

abbreviation combine_out where
  "combine_out S ci key dc de τ ≡
     dg_local (traverse_rhs (sp_compile_with (λd. DG d ⊥)
       (dg_spec_combine_transfer S ci (mk_dg_man dc key) de)) τ)"

abbreviation combine_pub where
  "combine_pub S ci key dc de τ ≡
     genv key (sides_of_rhs (sp_compile_with (λd. DG d ⊥)
       (dg_spec_combine_transfer S ci (mk_dg_man dc key) de)) τ)"
```

The contract a specification signs: its concretization is monotone, an edge's program over-approximates the concrete edge step, and its combine over-approximates the concrete return. Both obligations read the globals from the environment the valuation holds, and describe the result against that environment joined with what the program publishes: a global the step does not write is still what was read, so a step owes no publication for it.

```
locale analysis_contract =
  fixes κ :: kenv
    and S ::
      "('x, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec"
    and γ_DG :: "'D ⇒ ('v ⇒ 'G) ⇒ store set"
  assumes spec_wf: "dg_spec_wf S"
```

```
  and gammaDG_mono:
    "⟦d ≤ d'; e ≤ e'⟧ ⟹ γ_DG d e ⊆ γ_DG d' e'"
  and step_sound:
    "edge_collect κ a (γ_DG (dg_local (τ src)) (genv key τ))
       ⊆ γ_DG (edge_out S a src key τ) (genv key τ ⊔ edge_pub S a src key τ)"
  and combine_sound:
    "⟦s ∈ γ_DG q (genv key τ); t ∈ γ_DG r (genv key τ)⟧ ⟹
      combine_collect κ (ci_dst ci) s t
        ∈ γ_DG (combine_out S ci key q r τ) (genv key τ ⊔ combine_pub S ci key q r τ)"
```

The obligation is stated at *values* rather than at two unknown reads, because the routed call boundary applies combine at the continuation the entry produced, which is not in general what any unknown holds. Where the continuation *is* the caller's own solved value — the monovariant call shape — the tree form below is the instance at that value, so nothing is lost for a generator that reads both sides from unknowns.

```
lemma (in analysis_contract) combine_sound_program:
  assumes sc: "s ∈ γ_DG (dg_local (τ src_cc)) (genv key τ)"
    and se: "t ∈ γ_DG (dg_local (τ src_ex)) (genv key τ)"
  shows "combine_collect κ (ci_dst ci) s t
         ∈ γ_DG (dg_local (traverse_program (dg_spec_combine_program S ci src_cc src_ex key) τ))
             (genv key τ ⊔ genv key (sides_of_program (dg_spec_combine_program S ci src_cc src_ex key) τ))"
  using combine_sound[where q = "dg_local (τ src_cc)" and r = "dg_local (τ src_ex)"
      and τ = τ and key = key and ci = ci, OF sc se]
  by (simp add: dg_spec_combine_program_def traverse_combine_transfer_program
      sides_combine_transfer_program)

end
```

6.7 Transfer_Algebra

```
theory Transfer_Algebra
  imports "Voblint_CFG.CFG_Transfer" "Voblint_Domain.Nonrelational_State"
begin
```

The abstract-state algebra a whole-state transfer computes in

A Base-style analysis carries one pointwise abstract state per program point: a map from variable names to abstract values. This theory fixes the operations its transfer functions are assembled from – how a call resets the callee frame and binds its formals, how a return merges caller against callee and publishes the result value – together with their soundness and monotonicity against `gamma_state`.

It defines no interface. What an analysis supplies to the framework is a `dg_spec`: one manager-native transfer per edge action, taking the manager and returning a program. This theory is the algebra those transfers compute in, stated once so that no whole-state domain re-proves it.

Most of the operations themselves live lower still, in the language session, because they are generic in the state's codomain and the concrete semantics runs the very same constants: `combine_env`, `enter_frame`, `bind_formals`, `combine_assign` and `enter_binding` are all shared. What is genuinely new here is their meaning against a concretization, which is the first point at which the domain session and the CFG session are both in scope.

The structural environment combine

Soundness of the abstract combine: combining a caller store (sound for sc) with a callee-exit store (sound for se) yields a store sound for `combine_env 𝒢 sc se`. A pure `numeric_domain` fact – independent of any transfer function – reused by both the interprocedural constraint-system soundness and the effectful pipeline. `combine_env` is the fixed structural merge the Base call boundary uses; an analysis that wants a different one overrides its specification's own environment stage instead.

```
lemma combine_env_sound [intro]:
  fixes dc de :: "('k, 'a::numeric_domain) gabs_state"
  assumes sc: "s ∈ γ dc" and se: "t ∈ γ de"
  shows "combine_env 𝒢 s t ∈ γ (combine_env 𝒢 dc de)"
  using assms by (auto simp: gamma_state_def le_fun_def)
```

Generic soundness and monotonicity helpers

The two facts a call boundary keeps re-deriving: writing one variable, and binding a list of formals, each preserve soundness and monotonicity. They are stated for an arbitrary domain and no transfer function, so every analysis picks them up rather than proving its own copy.

```
lemma gamma_state_upd [intro]:
  fixes d :: "('k, 'a::numeric_domain) gabs_state"
  assumes s: "s ∈ γ d" and v: "v ∈ γ a"
  shows "s(x := v) ∈ γ (d(x := a))"
  using s v unfolding gamma_state_def by auto
```

Binding formals preserves soundness: pointwise-sound actual values bound to the same formals yield a sound entry state.

```
lemma bind_formals_sound [intro]:
  fixes d :: "('k, 'a::numeric_domain) gabs_state"
  assumes s: "s ∈ γ d"
    and vals: "list_all2 (λv a. v ∈ γ a) vs avs"
  shows "bind_formals xs vs s ∈ γ (bind_formals xs avs d)"
  using assms apply (induction xs arbitrary: vs avs s d)
  subgoal
    by simp
  subgoal for _ _ vs avs
    by (cases vs; cases avs) (auto simp: gamma_state_upd)
  done
```

Procedure entry: binding formals and resetting the frame

Procedure entry itself is not defined here. `enter_frame` and `enter_binding` (`Voblint_VIMP.VIMP_Proc`) are already generic in the state's codomain, and the concrete semantics is `enter_binding` at the reset value `0` and the language's evaluator (`call_enter`). An abstract domain runs the same constant at its own `top` and its own evaluator, so the two can never drift apart: there is one definition and two instances of it.

What this theory adds is the pair of facts that relate those two instances – the concrete entered store lands in the concretization of the abstract one, and the abstract one is monotone. Neither is expressible below here, because γ lives in the domain session and `enter_binding` in the language session.

```
lemma enter_frame_sound [intro]:
  fixes reset_val :: "'a::numeric_domain"
  assumes sv: "s ∈ γ d" and reset_full: "γ reset_val = UNIV"
  shows "enter_state 𝒢 s ∈ γ (enter_frame 𝒢 reset_val d)"
  unfolding gamma_state_def enter_state_def
  using gamma_stateD[OF sv] reset_full by auto
```

`bind_formals`'s own soundness and monotonicity are relational in the actual values, but that is not the shape a domain owns: an evaluator's soundness and monotonicity are pointwise in one actual. This turns the one into the other, so the entry lemmas below can take the fact a caller already has and nobody builds a `list_all2` by hand.

```
lemma list_all2_map_mapI [intro]:
  assumes "⋀x. x ∈ set xs ⟹ R (f x) (g x)"
  shows "list_all2 R (map f xs) (map g xs)"
  using assms by (induction xs) auto

lemma enter_binding_sound_list_all2 [intro]:
  fixes reset_val :: "'a::numeric_domain"
  assumes sv: "s ∈ γ d" and reset_full: "γ reset_val = UNIV"
    and vals: "list_all2 (λv a. v ∈ γ a)
                (map (λe. ev e s) es) (map (λe. aval_abs e d) es)"
  shows "enter_binding 𝒢 0 ev xs es s
           ∈ γ (enter_binding 𝒢 reset_val aval_abs xs es d)"
  unfolding enter_binding_def enter_state_def[symmetric]
  by (rule bind_formals_sound[OF enter_frame_sound[OF sv reset_full] vals])

lemma enter_binding_sound [intro]:
  fixes reset_val :: "'a::numeric_domain"
  assumes sv: "s ∈ γ d" and reset_full: "γ reset_val = UNIV"
    and eval: "⋀e. e ∈ set es ⟹ ev e s ∈ γ (aval_abs e d)"
  shows "enter_binding 𝒢 0 ev xs es s
           ∈ γ (enter_binding 𝒢 reset_val aval_abs xs es d)"
proof (rule enter_binding_sound_list_all2[OF sv reset_full])
  show "list_all2 (λv a. v ∈ γ a)
          (map (λe. ev e s) es) (map (λe. aval_abs e d) es)"
    by (rule list_all2_map_mapI) (rule eval)
qed
```

A call site reads the formals and actuals off `call_info` rather than passing them separately, but that projection stays at the call site: a domain writes `enter_binding 𝒢 top ev (ci_formals ci) (ci_args ci)`, exactly as the return side writes `combine_assign (ci_dst ci)`. No constant here wraps it.

The caller's own continuation is not computed here either. Goblint's `Spec.enter` answers caller/callee pairs and `constraints.ml` hands the caller half on to `combine_env`; a specification's entry field answers the same pairs, so a domain that wants to drop caller facts a callee could invalidate — Goblint's `varEq` filters its caller state by taint — returns the filtered state as the continuation half of its own alternatives. Nothing in this algebra chooses that half: the operations below are what a whole-state domain computes each half *with*.

end

6.8 State_Restriction

```
theory State_Restriction
  imports "Voblint_VIMP.VIMP_Globals" "Voblint_Domain.Nonrelational_State"
    "Voblint_Domain.Reachability_Lift"
begin
```

Reading one name of a combined state

The two pointwise equations, so a proof about a single variable never has to unfold `combine_env`. The nesting law is what lets a combine step read a returned value straight out of the callee exit: every call site owns `ret_var` as its own compiler-internal name and never a user-declared global (`reserved_ret_var`), so routing it through `combine_env` a second time would change nothing.

```
lemma combine_env_local_eq [simp]:
  "¬ 𝒢 x ⟹ combine_env 𝒢 sc se x = sc x"
  by (simp add: combine_env_def)

lemma combine_env_global_eq [simp]:
  "𝒢 x ⟹ combine_env 𝒢 sc se x = se x"
  by (simp add: combine_env_def)

lemma combine_env_combine_env_left [simp]:
  "combine_env 𝒢 (combine_env 𝒢 dc g) (combine_env 𝒢 de g) = combine_env 𝒢 dc g"
  by (auto simp: combine_env_def)

end
```

6.9 DG_Ownership_Split_Spec

```
theory DG_Ownership_Split_Spec
  imports "Voblint_Framework.MCP_Spec" Transfer_Algebra "Voblint_Domain.Nonrelational_Reachability"
    State_Restriction "Voblint_Solver.Strategy_Program_Fold"
begin
```

Publishing the global halves of an entry's alternatives

An entry transfer answers a list of alternatives, each a pair of a caller continuation and a callee entry state, rather than one successor value. What an analysis that keeps its globals on a shared unknown publishes for one call is the join, over every alternative, of both halves' global projections `rg`: a continuation carries global information as much as its callee entry does.

```
definition ownership_split_enter_sides ::
  "('d ⇒ 'd) ⇒ ('d::bounded_semilattice_sup_bot) enter_result list ⇒ 'd"
where
  "ownership_split_enter_sides rg pairs =
     (⨆(cont, entry)←pairs. rg cont ⊔ rg entry)"

lemma ownership_split_enter_sides_Nil [simp]:
  "ownership_split_enter_sides rg [] = bot"
  by (simp add: ownership_split_enter_sides_def)

lemma ownership_split_enter_sides_Cons [simp]:
  "ownership_split_enter_sides rg ((cont, entry) # pairs)
     = rg cont ⊔ rg entry ⊔ ownership_split_enter_sides rg pairs"
  by (simp add: ownership_split_enter_sides_def)
```

What the joint publication bounds, and what it is bounded by. The two member rules are what a soundness proof reaches for once it has selected a covering alternative: that pair's two halves are each below what the call published, so a concretization taken against the published global still contains them. They stay untagged — their conclusions are bare inequalities in `rg` and would broaden proof search wherever a $\leq$ goal appears.

```
lemma ownership_split_enter_sides_le_iff:
  "ownership_split_enter_sides rg pairs ≤ g
     ⟷ (∀(cont, entry) ∈ set pairs. rg cont ≤ g ∧ rg entry ≤ g)"
  by (induction pairs) auto

end
```

6.10 DG_Indexed_Generator

```
theory DG_Indexed_Generator
  imports DG_Constraint_Programs CFG_Enumeration "TD.TD_side_Interface"
    "Voblint_Solver.Strategy_Tree_Side_Buffering"
begin

context
begin
```

`sp_compile` stays folded here. Every fact this theory needs about a compiled program — what it answers, publishes and reads — is available as a named observation equation stated against `sp_compile` itself, so a proof meets those rather than the compiler's representation.

Keyed equation generators

Builds a strategy-tree equation system over a CFG from supplied edge/combine/extra program hooks, where every unknown carries an extra `'k` key alongside its node – context, activation, or any other routing tag a caller wants the generator to thread through. Specifications, routing conventions, and analysis-specific representations are instantiated above this layer; the generator itself never sees one. The ten parameters, once:

- `pred_sel` — which intra predecessors a node folds, each at its address.
- `site_sel` — which call sites return at a node.
- `route` — the callee context a call site selects, threaded to `cmb` and `extra` and never inspected here.
- `it` — the program for one intra edge.
- `cmb` — the program for one call returning at this node.
- `extra` — the routing protocol's own per-node programs, such as an activation-seed read.
- `buffer_key_at` — the node-owned global buffer address for a context.
- `bot0` — the local value a node starts its fold from.
- d_0 — the local value injected at the CFG entry.
- g_0 — the global value published at the CFG entry.

  Two encodings, and they are not peers. `routed_node_rhs` is the direct, reference one: each publishing hook emits its own `Side`, keyed and folded independently. `routed_node_rhs_buffered` is what production runs — every executable analysis in this development reaches the solver through it. It differs deliberately: contributions return their global part in the answer, so the node's own `buffer_key_at c` is published once, after the whole fold has been evaluated, and the enclosing `buffer_sides` then consolidates *every* key it names — activation seeds included — into one flush apiece. The flush comes before the next local read at a node where at most one call returns, and at the answer elsewhere.

  `routed_node_rhs_buffered_correspondence` relates the two declaratively: equal answer, publications and dependencies at a fixed valuation, under explicit per-hook hypotheses. That is all it says. Nothing here claims the two run the same: equal observations at each valuation leave the iteration order free, and where the first occurrence of a key falls decides which update the solver widens against. Deferring the node-global publication past the contributions is therefore a scheduling decision, not an encoding detail, and swapping the encodings is a question for the executable regression corpus rather than for these theorems.

The heterogeneous seeded keyed generator

The one context-generic generator. Every program it folds comes from a hook, so a monovariant and a context-sensitive analysis are both instances, and the generator itself never sees a specification:

- `pred_sel g v ctx` selects the intra predecessors folded as Answers into a node, each paired with the *address* in the solver's valuation space that carries its value. `intra_predecessor_addr_list` over `intra` addresses every predecessor at its own `(pp, 'c)` local unknown. Callee entries are not predecessors: every routed instance reaches them through published seeds.

- `site_sel g v` selects the call sites whose calls return at a node. `call_site_list` reads them off `calls`; an executable instance may answer from an index built once per graph, since the generator only folds the list.
- `it c src a` is the intra edge program itself – an instantiator supplies the compiled right-hand side for one edge action at one source address, e.g. a spec's compiled `dg_spec_edge_program` at the routed key. Which unknowns it reads and which keys it publishes are its own business; the generator makes no assumption about either.
- `cmb` is the procedure-return combine program (already fully abstract: the context-sensitive instance reads the callee exit under the routed context).
- `extra c v` supplies additional per-node programs folded after the intra and combine programs. Their Answers add to the node's local accumulator and their `Side` effects are collected — this is where a frame-entry seed *read* (a `QueryG` of the incoming seed slot) and the caller-side call-entry *publication* (a routed `Side` to the callee seed slot) live. Every routed instance supplies one, the context-insensitive one included: seeding is what activates a callee at all, so no analysis reaching a call leaves this hook empty.

The standard predecessor selection: every intra predecessor is carried by its own `(pp, 'c)` local unknown, so its address is `Inl` of that pair.

```
definition intra_predecessor_addr_list ::
  "(var_id, exp) gcfg ⇒ pp ⇒ 'c ⇒ ((pp × 'c + 'k) × (var_id, exp) gedge_action) list"
where
  "intra_predecessor_addr_list g v ctx =
     map (λ(u, a). (Inl (u, ctx), a)) (intra_predecessor_list g v)"
```

Everything that flows into one unknown, in one list: the intra edges entering `v`, the calls returning at it, and whatever the routing protocol adds there. Both generators fold exactly this list — the buffered one at its own hooks — and every bound, correspondence and monotonicity statement below is about it, so it is named once rather than spelled out at each.

The three membership rules are how a per-hook hypothesis reaches a selected contribution. They stay untagged: the list is a concatenation of two maps and a hook's own list, so a membership goal is decided by `auto` from the definition, and tagging them would put three rules in front of every `set` membership goal in scope.

```
definition routed_contribution_programs ::
  "((var_id, exp) gcfg ⇒ pp ⇒ 'c ⇒ ((pp × 'c + 'k) × (var_id, exp) gedge_action) list)
   ⇒ ((var_id, exp) gcfg ⇒ pp ⇒ (pp × (var_id, exp) gcall_action) list)
   ⇒ (pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c)
   ⇒ ('c ⇒ pp × 'c + 'k ⇒ (var_id, exp) gedge_action
        ⇒ (pp × 'c, 'k, ('d::bounded_semilattice_sup_bot,
                         'h::bounded_semilattice_sup_bot) dg_state,
                        ('d, 'h) dg_state) strategy_program)
   ⇒ ((pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pp
        ⇒ (pp × 'c, 'k, ('d, 'h) dg_state, ('d, 'h) dg_state) strategy_program)
   ⇒ ((pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ pp
        ⇒ (pp × 'c, 'k, ('d, 'h) dg_state, ('d, 'h) dg_state) strategy_program list)
   ⇒ (var_id, exp) gcfg ⇒ 'c ⇒ pp
   ⇒ (pp × 'c, 'k, ('d, 'h) dg_state, ('d, 'h) dg_state) strategy_program list"
where
  "routed_contribution_programs pred_sel site_sel route it cmb extra g ctx v =
     map (λ(src, a). it ctx src a) (pred_sel g v ctx)
     @ map (λ(cc, ca). cmb route ctx ca cc v) (site_sel g v)
     @ extra route ctx v"
```

Every contribution runs its continuation once, given that each hook does. The fold's observation lemmas all carry that side condition, and this is where a generator instance discharges it: the hooks a context policy supplies are built from `sp_return`, the reads and `sp_publish`, whose closure lemmas fire by `simp`.

```
lemma routed_contribution_programs_wf:
  assumes "⋀c' src a. sp_wf (it c' src a)"
    and "⋀c' ca cc ex. sp_wf (cmb route c' ca cc ex)"
    and "⋀c' w p. p ∈ set (extra route c' w) ⟹ sp_wf p"
  shows "∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb extra g ctx v).
           sp_wf p"
  using assms by (auto simp: routed_contribution_programs_def)

lemma routed_contribution_programs_combineI:
  "(cc, ca) ∈ set (site_sel g v)
   ⟹ cmb route ctx ca cc v
        ∈ set (routed_contribution_programs pred_sel site_sel route it cmb extra g ctx v)"
```

```
  by (force simp: routed_contribution_programs_def)

lemma routed_contribution_programs_extraI:
  "t ∈ set (extra route ctx v)
   ⟹ t ∈ set (routed_contribution_programs pred_sel site_sel route it cmb extra g ctx v)"
  by (simp add: routed_contribution_programs_def)

lemma routed_contribution_programsE:
  assumes "t ∈ set (routed_contribution_programs pred_sel site_sel route it cmb extra g ctx v)"
  obtains src a where "(src, a) ∈ set (pred_sel g v ctx)" "t = it ctx src a"
    | cc ca where "(cc, ca) ∈ set (site_sel g v)" "t = cmb route ctx ca cc v"
    | "t ∈ set (extra route ctx v)"
  using assms by (auto simp: routed_contribution_programs_def)
```

The local value a node's fold starts from: `bot0`, joined with the injected $\mathtt{d}_0$ at the CFG entry. An abbreviation, so every statement that spells the conditional out prints as `rhs_init` and needs no unfolding. `rhs_init_global` is the global counterpart: the injected $\mathtt{g}_0$ at the CFG entry, $\bot$ elsewhere.

```
abbreviation rhs_init :: "(var_id, exp) gcfg ⇒ 'd::sup ⇒ 'd ⇒ pp ⇒ 'd" where
  "rhs_init g bot0 d₀ v ≡ (if v = cfg_entry g then bot0 ⊔ d₀ else bot0)"

abbreviation rhs_init_global :: "(var_id, exp) gcfg ⇒ 'h::bot ⇒ pp ⇒ 'h" where
  "rhs_init_global g g₀ v ≡ (if v = cfg_entry g then g₀ else bot)"

definition routed_node_rhs ::
  "((var_id, exp) gcfg ⇒ pp ⇒ 'c ⇒ ((pp × 'c + 'k) × (var_id, exp) gedge_action) list)
   ⇒ ((var_id, exp) gcfg ⇒ pp ⇒ (pp × (var_id, exp) gcall_action) list)
   ⇒ ('c ⇒ 'k)
   ⇒ (pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c)
   ⇒ ('c ⇒ pp × 'c + 'k ⇒ (var_id, exp) gedge_action
        ⇒ (pp × 'c, 'k, ('d::bounded_semilattice_sup_bot,
                         'h::bounded_semilattice_sup_bot) dg_state,
                        ('d, 'h) dg_state) strategy_program)
   ⇒ ((pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pp
        ⇒ (pp × 'c, 'k, ('d, 'h) dg_state, ('d, 'h) dg_state) strategy_program)
   ⇒ ((pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ pp
        ⇒ (pp × 'c, 'k, ('d, 'h) dg_state, ('d, 'h) dg_state) strategy_program list)
   ⇒ (var_id, exp) gcfg ⇒ 'd ⇒ 'd ⇒ 'h
   ⇒ (pp × 'c, 'k, ('d, 'h) dg_state) eqsT"
where
  "routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ =
     (λ(v, c).
        let t = sp_compile (side_rhs_fold_dg (rhs_init g bot0 d₀ v)
                  (routed_contribution_programs pred_sel site_sel route it cmb extra g c v))
        in if v = cfg_entry g then Side (buffer_key_at c) (DG bot g₀) t else t)"

lemma eq_routed_node_rhs:
  assumes wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                     extra g ctx w). sp_wf p"
  shows
  "eq (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀)
      (v, ctx) τ =
   DG (side_acc_dg (rhs_init g bot0 d₀ v) τ
     (routed_contribution_programs pred_sel site_sel route it cmb extra g ctx v)) bot"
  by (simp add: routed_node_rhs_def Let_def
        traverse_side_rhs_fold_dg[OF wf])

lemma sides_routed_node_rhs:
  assumes wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                     extra g ctx w). sp_wf p"
  shows
  "sides_of_rhs (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀
      (v, ctx)) τ (Inr (buffer_key_at ctx)) =
   (if v = cfg_entry g then DG bot g₀ else bot)
   ⊔ (⨆t←routed_contribution_programs pred_sel site_sel route it cmb extra g ctx v.
        sides_of_program t τ (Inr (buffer_key_at ctx)))"
  by (simp add: routed_node_rhs_def Let_def
        sides_of_program_side_rhs_fold_dg_char[OF wf]
        bot_dg_state_def[symmetric] ac_simps)
```

The right-hand side as a join of its parts

At a fixed valuation the answer of `routed_node_rhs` is a join: the node's initial value, one value per intra edge entering it, one per call returning at it, and the values of the routing protocol's own programs, the seed read among them. `rhs_init` names the first part in the generator itself; the definitions below name the other three,

and `routed_node_rhs_parts` states the answer as the join of all four. Every part is the local half of a contribution's answer at the valuation; publications do not enter the answer and are characterized by `sides_routed_node_rhs`.

```
definition rhs_edge where
  "rhs_edge it τ ctx u a = dg_local (traverse_program (it ctx u a) τ)"

definition rhs_call where
  "rhs_call cmb route τ ctx u ca v = dg_local (traverse_program (cmb route ctx ca u v) τ)"

definition rhs_seed where
  "rhs_seed extra route τ ctx v = (⨆p←extra route ctx v. dg_local (traverse_program p τ))"

lemma routed_node_rhs_parts:
  assumes wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                    extra g c w). sp_wf p"
  shows
  "eq (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀)
      (v, c) τ =
   DG (rhs_init g bot0 d₀ v
       ⊔ (⨆(u, a)←pred_sel g v c. rhs_edge it τ c u a)
       ⊔ (⨆(u, ca)←site_sel g v. rhs_call cmb route τ c u ca v)
       ⊔ rhs_seed extra route τ c v) bot"
  by (simp add: eq_routed_node_rhs[OF wf] side_acc_dg_as_foldr routed_contribution_programs_def
       rhs_edge_def rhs_call_def rhs_seed_def split_def comp_def sup_assoc)
```

Buffered generator: fold Side-free contributions, publish once

`routed_node_rhs` writes a node's own key once per contribution that publishes there: several intra predecessors or several return call actions can each independently emit a `Side (buffer_key_at c) ...`, so several writes to the same `buffer_key_at c` land in one RHS evaluation. `routed_node_rhs_buffered` folds Side-free contribution programs (the caller's own `it_c` for `intra` and `cmb_c` for `comb`) with `fold_rhs_program` and publishes `buffer_key_at c` once, after every contribution has been read.

That fold shapes the node's *own* key, but each call returning at the node still publishes its callee's seed from inside its `cmb_c` program. Two call sites resuming at the same node, as the two branches of a conditional that both end in a call do, can therefore name one seed key twice. `buffer_sides` accumulates the writes per key and preserves the declarative `traverse_rhs`/`sides_of_rhs` value wherever it flushes, so the correspondence with the unbuffered generator below is unaffected by the choice of flush points.

The choice does decide when a seed reaches the solver. A call program publishes the seed and then reads the callee's exit, and that read is what makes the solver evaluate the callee. Held back to the answer, the seed arrives after the callee has been solved from ⊥, and the seed's change then sends the caller round again. So a node at which at most one call returns flushes before every local read, and the seed is published before the exit is read, in the order of Goblint's `sidel` and `getl`. A node at which several calls return flushes at the answer only: a flush before the first call's exit read would write a seed the second call may write again, and an update rule that keeps one slot per origin would see a partial contribution first in every evaluation. There `distinct_side_path_buffer_sides_answers_only` bounds the writes unconditionally; at the other nodes the bound `distinct_side_path_buffer_sides` holds as long as one call program does not write a seed again after reading an exit, which the shipped specifications' single-alternative entry transfers ensure.

```
definition routed_node_rhs_buffered ::
  "((var_id, exp) gcfg ⇒ pp ⇒ 'c ⇒ ((pp × 'c + 'k) × (var_id, exp) gedge_action) list)
   ⇒ ((var_id, exp) gcfg ⇒ pp ⇒ (pp × (var_id, exp) gcall_action) list)
   ⇒ ('c ⇒ 'k)
   ⇒ (pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c)
   ⇒ ('c ⇒ pp × 'c + 'k ⇒ (var_id, exp) gedge_action
        ⇒ (pp × 'c, 'k, ('d::bounded_semilattice_sup_bot,
                         'h::bounded_semilattice_sup_bot) dg_state,
                        ('d, 'h) dg_state) strategy_program)
   ⇒ ((pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pp
        ⇒ (pp × 'c, 'k, ('d, 'h) dg_state, ('d, 'h) dg_state) strategy_program)
   ⇒ ((pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ pp
        ⇒ (pp × 'c, 'k, ('d, 'h) dg_state, ('d, 'h) dg_state) strategy_program list)
   ⇒ (var_id, exp) gcfg ⇒ 'd ⇒ 'd ⇒ 'h
   ⇒ (pp × 'c, 'k, ('d, 'h) dg_state) eqsT"
where
  "routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀ =
     (λ(v, c).
        let acc0 = DG (rhs_init g bot0 d₀ v) (rhs_init_global g g₀ v);
            t = sp_compile (fold_rhs_program acc0
                  (routed_contribution_programs pred_sel site_sel route it_c cmb_c extra g c v))
        in buffer_sides (λ_. length (site_sel g v) ≤ 1) (sp_lift_tree t (λres.
          Side (buffer_key_at c) (DG bot (dg_global res)) (Answer (DG (dg_local res) bot)))))"
```

`fold_rhs_program` threads its accumulator in evaluation order (a left fold); the lemmas below need the equivalent right fold, since that is how each of their own goals is already phrased. `traverse_rhs` only ever joins, so the two forms agree by `foldr_sup_seed_swap`.

```
lemma eq_routed_node_rhs_buffered:
  assumes wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it_c cmb_c
                    extra g ctx w). sp_wf p"
  shows
  "eq (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀)
      (v, ctx) τ =
   DG (dg_local (foldr (λt acc'. traverse_program t τ ⊔ acc')
     (routed_contribution_programs pred_sel site_sel route it_c cmb_c extra g ctx v)
     (DG (rhs_init g bot0 d₀ v) (rhs_init_global g g₀ v)))) bot"
  by (simp add: routed_node_rhs_buffered_def Let_def
       traverse_fold_rhs_program_char_foldr[OF wf])
```

The buffered generator answers the same join of parts as the reference generator, over its own contribution programs. Its answer carries no global half; the global part of the initial accumulator leaves through the publication at `buffer_key_at c`.

```
lemma routed_node_rhs_buffered_parts:
  assumes wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                    extra g c w). sp_wf p"
  shows
  "eq (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀)
      (v, c) τ =
   DG (rhs_init g bot0 d₀ v
       ⊔ (⨆(u, a)←pred_sel g v c. rhs_edge it τ c u a)
       ⊔ (⨆(u, ca)←site_sel g v. rhs_call cmb route τ c u ca v)
       ⊔ rhs_seed extra route τ c v) bot"
proof -
  have "eq (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀)
      (v, c) τ
    = eq (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀) (v, c) τ"
    by (simp add: eq_routed_node_rhs_buffered[OF wf] eq_routed_node_rhs[OF wf]
        dg_local_foldr_generic side_acc_dg_as_foldr_seeded)
  then show ?thesis
    by (simp only: routed_node_rhs_parts[OF wf])
qed

lemma sides_routed_node_rhs_buffered:
  assumes wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it_c cmb_c
                    extra g ctx w). sp_wf p"
    and intra_free_at_key:
      "⋀c' src a σ. sides_of_program (it_c c' src a) σ (Inr (buffer_key_at c')) = bot"
    and comb_free_at_key: "⋀c' ca cc ex σ.
                    sides_of_program (cmb_c route c' ca cc ex) σ (Inr (buffer_key_at c')) = bot"
    and extra_free: "⋀c' w σ z x. x ∈ set (extra route c' w) ⟹ sides_of_program x σ z = bot"
  shows "sides_of_rhs (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g
      bot0 d₀ g₀ (v, ctx)) τ (Inr (buffer_key_at ctx)) =
   DG bot (dg_global (foldr (λt acc'. traverse_program t τ ⊔ acc')
     (routed_contribution_programs pred_sel site_sel route it_c cmb_c extra g ctx v)
     (DG (rhs_init g bot0 d₀ v) (rhs_init_global g g₀ v))))"
proof -
  have free: "⋀w σ x.
      x ∈ set (routed_contribution_programs pred_sel site_sel route it_c cmb_c extra g ctx w)
      ⟹ sides_of_program x σ (Inr (buffer_key_at ctx)) = bot"
    using intra_free_at_key comb_free_at_key extra_free
    by (blast elim: routed_contribution_programsE)
  show ?thesis
  proof (cases "v = cfg_entry g")
    case True
    have z0: "(sides_of_program (fold_rhs_program (DG (bot0 ⊔ d₀) g₀)
        (routed_contribution_programs pred_sel site_sel route it_c cmb_c extra g ctx
           (cfg_entry g))) τ) (Inr (buffer_key_at ctx)) = bot"
      by (simp only: sides_of_program_fold_rhs_program_char[OF wf]
          foldr_sup_bot_of_all_bot[OF free[where w = "cfg_entry g"]])
    from True show ?thesis
      by (simp add: routed_node_rhs_buffered_def Let_def
          traverse_fold_rhs_program_char_foldr[OF wf]
          bot_dg_state_def sup_dg_state_def z0)
  next
    case False
    have z0: "(sides_of_program (fold_rhs_program (DG bot0 bot)
       (routed_contribution_programs pred_sel site_sel route it_c cmb_c extra g ctx v)) τ) (Inr (buffer_key_at ctx))
= bot"
```

```
    by (simp only: sides_of_program_fold_rhs_program_char[OF wf]
        foldr_sup_bot_of_all_bot[OF free[where w = v]])
  from False show ?thesis
    by (simp add: routed_node_rhs_buffered_def Let_def
        traverse_fold_rhs_program_char_foldr[OF wf]
        bot_dg_state_def sup_dg_state_def z0)
 qed
qed
```

Correspondence: the buffered generator against the direct one

A pointwise correspondence between two parallel contribution lists – one Side-free (feeding `fold_rhs_program`), one Side-emitting (feeding `side_rhs_fold_dg`) – lifts to their whole-list fold, in both the local-answer and global-side components. This is the list-level engine behind `routed_node_rhs_buffered_correspondence`: each of `intra`, `comb`, and `extra` instantiates it once, and `list_all2_appendI` combines the three.

`routed_node_rhs_buffered`'s declarative value matches `routed_node_rhs`'s, given that each buffered hook is the Side-free contribution analogue of its unbuffered counterpart at the SAME `buffer_key_at ctx` slot. `intra` and `comb` carry the same six hypotheses – pairwise local/global/dep agreement, buffered-side freeness at the key, side purity, and off-key agreement – and `extra` is Side-free and answers only its local slot (`extra_free`/`extra_local_only` – `routed_entry_seed_programs` discharges both by direct inspection, since it answers on its `dg_local` half and issues no `Side`). A local-only intra hook discharges its six trivially: its program publishes nothing, so the contribution analogue is the program itself and every hypothesis is reflexive or `bot`; an intra hook that genuinely publishes at `buffer_key_at ctx` supplies its reshaped contribution program, exactly as a reshaped `routed_call_program` does for `cmb`.

```
lemma routed_node_rhs_buffered_correspondence:
  fixes cmb cmb_c ::
    "(pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pp
       ⇒ (pp × 'c, 'k, ('d::bounded_semilattice_sup_bot,
                        'h::bounded_semilattice_sup_bot) dg_state,
           ('d, 'h) dg_state) strategy_program"
  assumes wf_c: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it_c cmb_c
                     extra g ctx w). sp_wf p"
    and wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                   extra g ctx w). sp_wf p"
    and intra_t: "⋀c' src a τ. dg_local (traverse_program (it_c c' src a) τ)
                    = dg_local (traverse_program (it c' src a) τ)"
    and intra_side_pure: "⋀c' src a τ.
                           dg_local (sides_of_program (it c' src a) τ (Inr (buffer_key_at c'))) = bot"
    and intra_s: "⋀c' src a τ. dg_global (traverse_program (it_c c' src a) τ)
                    = dg_global (sides_of_program (it c' src a) τ (Inr (buffer_key_at c')))"
    and intra_free_at_key:
      "⋀c' src a τ. sides_of_program (it_c c' src a) τ (Inr (buffer_key_at c')) = bot"
    and intra_sides_off_key: "⋀c' src a τ z. z ≠ Inr (buffer_key_at c')
                    ⟹ sides_of_program (it_c c' src a) τ z = sides_of_program (it c' src a) τ z"
    and intra_dep: "⋀c' src a τ. dep_program τ (it_c c' src a) = dep_program τ (it c' src a)"
    and comb_t: "⋀c' ca cc ex τ. dg_local (traverse_program (cmb_c route c' ca cc ex) τ)
                    = dg_local (traverse_program (cmb route c' ca cc ex) τ)"
    and comb_side_pure: "⋀c' ca cc ex τ.
                    dg_local (sides_of_program (cmb route c' ca cc ex) τ (Inr (buffer_key_at c'))) = bot"
    and comb_s: "⋀c' ca cc ex τ. dg_global (traverse_program (cmb_c route c' ca cc ex) τ)
                    = dg_global (sides_of_program (cmb route c' ca cc ex) τ (Inr (buffer_key_at c')))"
    and comb_free_at_key: "⋀c' ca cc ex τ.
                    sides_of_program (cmb_c route c' ca cc ex) τ (Inr (buffer_key_at c')) = bot"
    and comb_sides_off_key: "⋀c' ca cc ex τ z. z ≠ Inr (buffer_key_at c')
                    ⟹ sides_of_program (cmb_c route c' ca cc ex) τ z
                       = sides_of_program (cmb route c' ca cc ex) τ z"
    and comb_dep: "⋀c' ca cc ex τ. dep_program τ (cmb_c route c' ca cc ex)
                    = dep_program τ (cmb route c' ca cc ex)"
    and extra_free: "⋀c' w τ z x. x ∈ set (extra route c' w) ⟹ sides_of_program x τ z = bot"
    and extra_local_only: "⋀c' w τ x.
                           x ∈ set (extra route c' w) ⟹ dg_global (traverse_program x τ) = bot"
  shows "traverse_rhs
           (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
             (v, ctx)) τ
         = traverse_rhs
           (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ"
    (is ?T)
    and "dep_aux τ
           (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
             (v, ctx))
         = dep_aux τ
           (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx))"
```

```
    (is ?D)
    and "sides_of_rhs
           (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
             (v, ctx)) τ
         = sides_of_rhs
           (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ"
    (is ?S)
proof -
  let ?intra_new = "map (λ(src, a). it_c ctx src a) (pred_sel g v ctx)"
  let ?intra_old = "map (λ(src, a). it ctx src a) (pred_sel g v ctx)"
  let ?comb_new = "map (λ(cc, ca). cmb_c route ctx ca cc v) (site_sel g v)"
  let ?comb_old = "map (λ(cc, ca). cmb route ctx ca cc v) (site_sel g v)"
  let ?extra = "extra route ctx v"
  let ?acc0 = "DG (rhs_init g bot0 d₀ v) (rhs_init_global g g₀ v)"
  let ?new = "routed_contribution_programs pred_sel site_sel route it_c cmb_c extra g ctx v"
  let ?old = "routed_contribution_programs pred_sel site_sel route it cmb extra g ctx v"
  have new_split: "?new = ?intra_new @ ?comb_new @ ?extra"
    by (simp add: routed_contribution_programs_def)
  have old_split: "?old = ?intra_old @ ?comb_old @ ?extra"
    by (simp add: routed_contribution_programs_def)
  have intra_local: "list_all2
      (λt_new t_old. dg_local (traverse_program t_new τ) = dg_local (traverse_program t_old τ))
      ?intra_new ?intra_old"
    by (rule list_all2_map_diag) (auto simp: intra_t)
  have intra_global: "list_all2
      (λt_new t_old. dg_global (traverse_program t_new τ)
                       = dg_global (sides_of_program t_old τ (Inr (buffer_key_at ctx))))
      ?intra_new ?intra_old"
    by (rule list_all2_map_diag) (auto simp: intra_s)
  have comb_local: "list_all2
      (λt_new t_old. dg_local (traverse_program t_new τ) = dg_local (traverse_program t_old τ))
      ?comb_new ?comb_old"
    by (rule list_all2_map_diag) (auto simp: comb_t)
  have comb_global: "list_all2
      (λt_new t_old. dg_global (traverse_program t_new τ)
                       = dg_global (sides_of_program t_old τ (Inr (buffer_key_at ctx))))
      ?comb_new ?comb_old"
    by (rule list_all2_map_diag) (auto simp: comb_s)
  have extra_local: "list_all2
      (λt_new t_old. dg_local (traverse_program t_new τ) = dg_local (traverse_program t_old τ))
      ?extra ?extra"
    by (rule list.rel_refl_strong) simp
  have extra_global: "list_all2
      (λt_new t_old. dg_global (traverse_program t_new τ)
                       = dg_global (sides_of_program t_old τ (Inr (buffer_key_at ctx))))
      ?extra ?extra"
    by (rule list.rel_refl_strong) (simp add: extra_free extra_local_only bot_dg_state_def)
  have local_list: "list_all2
      (λt_new t_old. dg_local (traverse_program t_new τ) = dg_local (traverse_program t_old τ))
      ?new ?old"
    unfolding new_split old_split
    using intra_local comb_local extra_local by (intro list_all2_appendI)
  have global_list: "list_all2 (λt_new t_old. dg_global (traverse_program t_new τ)
                                  = dg_global (sides_of_program t_old τ (Inr (buffer_key_at ctx))))
      ?new ?old"
    unfolding new_split old_split
    using intra_global comb_global extra_global by (intro list_all2_appendI)
  show ?T
  proof -
    have "traverse_rhs
        (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
          (v, ctx)) τ
        = DG (dg_local (foldr (λt acc'. traverse_program t τ ⊔ acc') ?new ?acc0)) bot"
      by (simp add: eq_routed_node_rhs_buffered[OF wf_c] routed_contribution_programs_def)
    also have "... = DG (dg_local ?acc0 ⊔ (⨆t←?new. dg_local (traverse_program t τ))) bot"
      by (simp add: dg_local_foldr_generic foldr_join_seed_out dg_local_join_list)
    also have "... = DG (dg_local ?acc0 ⊔ (⨆t←?old. dg_local (traverse_program t τ))) bot"
      by (simp only: foldr_sup_list_all2_cong[OF local_list])
    also have "... = DG (side_acc_dg (dg_local ?acc0) τ ?old) bot"
      by (simp add: side_acc_dg_as_foldr)
    also have "... = traverse_rhs
        (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ"
      by (simp add: eq_routed_node_rhs[OF wf] routed_contribution_programs_def)
    finally show ?thesis .
  qed
  have global_new: "sides_of_rhs
```

```
    (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
      (v, ctx)) τ (Inr (buffer_key_at ctx))
    = DG bot (dg_global (foldr (λt acc'. traverse_program t τ ⊔ acc') ?new ?acc0))"
  unfolding routed_contribution_programs_def[symmetric]
  by (intro sides_routed_node_rhs_buffered[OF wf_c])
     (use intra_free_at_key comb_free_at_key extra_free in blast)+
have dg_global_new_char: "dg_global (foldr (λt acc'. traverse_program t τ ⊔ acc') ?new ?acc0)
    = dg_global ?acc0
      ⊔ (⨆t←?old. dg_global (sides_of_program t τ (Inr (buffer_key_at ctx))))"
proof -
  have "dg_global (foldr (λt acc'. traverse_program t τ ⊔ acc') ?new ?acc0)
      = dg_global ?acc0 ⊔ (⨆t←?new. dg_global (traverse_program t τ))"
    by (simp add: dg_global_foldr_generic foldr_join_seed_out dg_global_join_list)
  also have "... = dg_global ?acc0
      ⊔ (⨆t←?old. dg_global (sides_of_program t τ (Inr (buffer_key_at ctx))))"
    by (simp only: foldr_sup_list_all2_cong[OF global_list])
  finally show ?thesis .
qed
have old_elem_side_pure: "⋀t. t ∈ set ?old
    ⟹ dg_local (sides_of_program t τ (Inr (buffer_key_at ctx))) = bot"
  using comb_side_pure extra_free
  by (auto simp: intra_side_pure bot_dg_state_def routed_contribution_programs_def
       split: prod.splits)
have old_fold_side_pure:
  "dg_local (⨆t←?old. sides_of_program t τ (Inr (buffer_key_at ctx))) = bot"
  by (rule foldr_sides_dg_local_bot) (rule old_elem_side_pure)
have old_fold_collapse: "(⨆t←?old. sides_of_program t τ (Inr (buffer_key_at ctx)))
    = DG bot (dg_global (⨆t←?old. sides_of_program t τ (Inr (buffer_key_at ctx))))"
  using old_fold_side_pure
  by (cases "⨆t←?old. sides_of_program t τ (Inr (buffer_key_at ctx))") simp
have old_sides: "sides_of_rhs
    (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ
    (Inr (buffer_key_at ctx))
    = DG bot (dg_global ?acc0
              ⊔ dg_global (⨆t←?old. sides_of_program t τ (Inr (buffer_key_at ctx))))"
proof -
  have "sides_of_rhs
      (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ
      (Inr (buffer_key_at ctx))
      = (if v = cfg_entry g then DG bot g₀ else bot)
        ⊔ (⨆t←?old. sides_of_program t τ (Inr (buffer_key_at ctx)))"
    by (rule sides_routed_node_rhs[OF wf])
  also have "... = DG bot (dg_global ?acc0
       ⊔ dg_global (⨆t←?old. sides_of_program t τ (Inr (buffer_key_at ctx))))"
    by (subst old_fold_collapse)
       (cases "v = cfg_entry g", simp_all add: DG_sup_bot_left bot_dg_state_def)
  finally show ?thesis .
qed
have S_at_key: "sides_of_rhs
    (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
      (v, ctx)) τ (Inr (buffer_key_at ctx))
    = sides_of_rhs
      (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ
      (Inr (buffer_key_at ctx))"
proof -
  have "sides_of_rhs
      (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
        (v, ctx)) τ (Inr (buffer_key_at ctx))
      = DG bot (dg_global (foldr (λt acc'. traverse_program t τ ⊔ acc') ?new ?acc0))"
    by (rule global_new)
  also have "... = DG bot (dg_global ?acc0
      ⊔ dg_global (⨆t←?old. sides_of_program t τ (Inr (buffer_key_at ctx))))"
    by (simp only: dg_global_new_char foldr_dg_global_sides_char)
  also have "... = sides_of_rhs
      (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ
      (Inr (buffer_key_at ctx))"
    by (rule old_sides[symmetric])
  finally show ?thesis .
qed
have intra_dep_list: "list_all2 (λt_new t_old. dep_program τ t_new = dep_program τ t_old)
    ?intra_new ?intra_old"
  by (rule list_all2_map_diag) (auto simp: intra_dep)
have comb_dep_list: "list_all2 (λt_new t_old. dep_program τ t_new = dep_program τ t_old)
    ?comb_new ?comb_old"
  by (rule list_all2_map_diag) (auto simp: comb_dep)
have extra_dep: "list_all2 (λt_new t_old. dep_program τ t_new = dep_program τ t_old)
```

```
      ?extra ?extra"
    by (rule list.rel_refl_strong) simp
  have dep_list: "list_all2 (λt_new t_old. dep_program τ t_new = dep_program τ t_old)
      ?new ?old"
    unfolding new_split old_split
    using intra_dep_list comb_dep_list extra_dep by (intro list_all2_appendI)
  show ?D
  proof -
    have "dep_aux τ
        (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
          (v, ctx))
        = (⋃t∈set ?new. dep_program τ t)"
      by (simp add: routed_node_rhs_buffered_def Let_def
          dep_program_fold_rhs_program_char[OF wf_c])
    also have "... = (⋃t∈set ?old. dep_program τ t)"
      by (rule list_all2_Union_eq[OF dep_list])
    also have "... = dep_aux τ
        (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx))"
      by (simp add: routed_node_rhs_def Let_def
          dep_program_side_rhs_fold_dg_char[OF wf])
    finally show ?thesis .
  qed
  have intra_sides_off: "⋀z. z ≠ Inr (buffer_key_at ctx)
      ⟹ list_all2 (λt_new t_old. sides_of_program t_new τ z = sides_of_program t_old τ z)
            ?intra_new ?intra_old"
    by (rule list_all2_map_diag) (auto simp: intra_sides_off_key)
  have comb_sides_off: "⋀z. z ≠ Inr (buffer_key_at ctx)
      ⟹ list_all2 (λt_new t_old. sides_of_program t_new τ z = sides_of_program t_old τ z)
            ?comb_new ?comb_old"
    by (rule list_all2_map_diag) (auto simp: comb_sides_off_key)
  have extra_sides_off: "⋀z.
      list_all2 (λt_new t_old. sides_of_program t_new τ z = sides_of_program t_old τ z)
        ?extra ?extra"
    by (rule list.rel_refl_strong) simp
  have sides_list_off: "⋀z. z ≠ Inr (buffer_key_at ctx)
      ⟹ list_all2 (λt_new t_old. sides_of_program t_new τ z = sides_of_program t_old τ z)
          ?new ?old"
    unfolding new_split old_split
    using intra_sides_off comb_sides_off extra_sides_off by (intro list_all2_appendI)
  have fold_sides_off: "⋀z. z ≠ Inr (buffer_key_at ctx) ⟹
      (⨆t←?new. sides_of_program t τ z) = (⨆t←?old. sides_of_program t τ z)"
    by (rule foldr_sup_list_all2_cong) (rule sides_list_off)
  show ?S
  proof (rule ext)
    fix z
    show "sides_of_rhs
        (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
          (v, ctx)) τ z
      = sides_of_rhs
        (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ z"
    proof (cases "z = Inr (buffer_key_at ctx)")
      case True
      with S_at_key show ?thesis by simp
    next
      case False
      have new_off: "sides_of_rhs
          (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀
            (v, ctx)) τ z
        = (⨆t←?new. sides_of_program t τ z)"
        using False
        by (simp add: routed_node_rhs_buffered_def Let_def
            sides_of_program_fold_rhs_program_char[OF wf_c] bot_dg_state_def[symmetric])
      have old_off: "sides_of_rhs
          (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ z
        = (⨆t←?old. sides_of_program t τ z)"
        using False
        by (simp add: routed_node_rhs_def Let_def
            sides_of_program_side_rhs_fold_dg_char[OF wf])
      from new_off old_off fold_sides_off[OF False] show ?thesis by simp
    qed
  qed
qed
```

The observational interface `part_post_solution` actually consumes: given the correspondence, a buffered post-solution is also an unbuffered one. Callers only need this fact, never the tree-shape argument behind `routed_node_rhs_buffered` itself – `part_post_solution`'s three conjuncts ($\mathtt{dep}_L$, `eq`, `sides_of_rhs`) are exactly `?D`/`?T`/`?S`, read pointwise at

each u ∈ vars.

```
lemma part_post_solution_routed_node_rhs_buffered:
  fixes cmb cmb_c ::
    "(pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pp
       ⇒ (pp × 'c, 'k, ('d::bounded_semilattice_sup_bot,
                        'h::bounded_semilattice_sup_bot) dg_state,
          ('d, 'h) dg_state) strategy_program"
  assumes wf_c: "⋀c' w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it_c
                      cmb_c extra g c' w). sp_wf p"
    and wf: "⋀c' w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                    extra g c' w). sp_wf p"
    and intra_t: "⋀c' src a τ. dg_local (traverse_program (it_c c' src a) τ)
                    = dg_local (traverse_program (it c' src a) τ)"
    and intra_side_pure: "⋀c' src a τ.
                           dg_local (sides_of_program (it c' src a) τ (Inr (buffer_key_at c'))) = bot"
    and intra_s: "⋀c' src a τ. dg_global (traverse_program (it_c c' src a) τ)
                    = dg_global (sides_of_program (it c' src a) τ (Inr (buffer_key_at c')))"
    and intra_free_at_key:
      "⋀c' src a τ. sides_of_program (it_c c' src a) τ (Inr (buffer_key_at c')) = bot"
    and intra_sides_off_key: "⋀c' src a τ z. z ≠ Inr (buffer_key_at c')
                    ⟹ sides_of_program (it_c c' src a) τ z = sides_of_program (it c' src a) τ z"
    and intra_dep: "⋀c' src a τ. dep_program τ (it_c c' src a) = dep_program τ (it c' src a)"
    and comb_t: "⋀c' ca cc ex τ. dg_local (traverse_program (cmb_c route c' ca cc ex) τ)
                    = dg_local (traverse_program (cmb route c' ca cc ex) τ)"
    and comb_side_pure: "⋀c' ca cc ex τ.
                    dg_local (sides_of_program (cmb route c' ca cc ex) τ (Inr (buffer_key_at c'))) = bot"
    and comb_s: "⋀c' ca cc ex τ. dg_global (traverse_program (cmb_c route c' ca cc ex) τ)
                    = dg_global (sides_of_program (cmb route c' ca cc ex) τ (Inr (buffer_key_at c')))"
    and comb_free_at_key: "⋀c' ca cc ex τ.
                    sides_of_program (cmb_c route c' ca cc ex) τ (Inr (buffer_key_at c')) = bot"
    and comb_sides_off_key: "⋀c' ca cc ex τ z. z ≠ Inr (buffer_key_at c')
                    ⟹ sides_of_program (cmb_c route c' ca cc ex) τ z
                       = sides_of_program (cmb route c' ca cc ex) τ z"
    and comb_dep: "⋀c' ca cc ex τ. dep_program τ (cmb_c route c' ca cc ex)
                    = dep_program τ (cmb route c' ca cc ex)"
    and extra_free: "⋀c' w τ z x. x ∈ set (extra route c' w) ⟹ sides_of_program x τ z = bot"
    and extra_local_only: "⋀c' w τ x.
                             x ∈ set (extra route c' w) ⟹ dg_global (traverse_program x τ) = bot"
    and pp_buf: "part_post_solution
        (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀)
        x sigma vars"
  shows "part_post_solution
      (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀) x sigma vars"
proof -
  let ?Tbuf =
    "routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it_c cmb_c extra g bot0 d₀ g₀"
  let ?Told =
    "routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀"
  have corr: "⋀u. traverse_rhs (?Tbuf u) sigma = traverse_rhs (?Told u) sigma
      ∧ dep_aux sigma (?Tbuf u) = dep_aux sigma (?Told u)
      ∧ sides_of_rhs (?Tbuf u) sigma = sides_of_rhs (?Told u) sigma"
  proof -
    fix u :: "pp × 'c"
    obtain v ctx where u: "u = (v, ctx)" by (cases u) auto
    show "traverse_rhs (?Tbuf u) sigma = traverse_rhs (?Told u) sigma
        ∧ dep_aux sigma (?Tbuf u) = dep_aux sigma (?Told u)
        ∧ sides_of_rhs (?Tbuf u) sigma = sides_of_rhs (?Told u) sigma"
      unfolding u
      using routed_node_rhs_buffered_correspondence
              [where cmb = cmb and cmb_c = cmb_c and it = it and it_c = it_c
                 and route = route and buffer_key_at = buffer_key_at,
               OF wf_c wf
                  intra_t intra_side_pure intra_s intra_free_at_key intra_sides_off_key intra_dep
                  comb_t comb_side_pure comb_s comb_free_at_key comb_sides_off_key comb_dep
                  extra_free extra_local_only]
      by simp
  qed
  have "part_post_solution ?Tbuf x sigma vars = part_post_solution ?Told x sigma vars"
    by (rule part_post_solution_cong) (use corr in blast)+
  with pp_buf show ?thesis by simp
qed
```

The same correspondence without reshaped hooks, for contribution programs that answer nothing on their global half. Such a program may still publish at `buffer_key_at ctx`: the buffered generator keeps that publication as it is and adds only the seed it hoists, which is the unbuffered generator's entry publication. A specification that

splits its state and publishes the global half through its manager is of this kind.

```
lemma routed_node_rhs_buffered_correspondence_answers_local:
  fixes cmb ::
    "(pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pp
       ⇒ (pp × 'c, 'k, ('d::bounded_semilattice_sup_bot,
                        'h::bounded_semilattice_sup_bot) dg_state,
           ('d, 'h) dg_state) strategy_program"
  assumes wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                  extra g ctx w). sp_wf p"
    and answers_local: "⋀w p τ. p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                  extra g ctx w) ⟹ dg_global (traverse_program p τ) = bot"
  shows "traverse_rhs
           (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀
             (v, ctx)) τ
         = traverse_rhs
           (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ"
    (is ?T)
    and "dep_aux τ
           (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀
             (v, ctx))
         = dep_aux τ
           (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx))"
    (is ?D)
    and "sides_of_rhs
           (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀
             (v, ctx)) τ
         = sides_of_rhs
           (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ"
    (is ?S)
proof -
  let ?ps = "routed_contribution_programs pred_sel site_sel route it cmb extra g ctx v"
  let ?acc0 = "DG (rhs_init g bot0 d₀ v) (rhs_init_global g g₀ v)"
  have glob: "dg_global (foldr (λt acc'. traverse_program t τ ⊔ acc') ?ps ?acc0)
      = dg_global ?acc0"
  proof -
    have "(⨆t←?ps. dg_global (traverse_program t τ)) = bot"
      by (rule foldr_sup_bot_of_all_bot) (rule answers_local)
    then show ?thesis
      by (simp only: dg_global_foldr_generic foldr_join_seed_out[where a = "dg_global ?acc0"]
          sup_bot_right)
  qed
  show ?T
  proof -
    have "traverse_rhs
        (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀
          (v, ctx)) τ
        = DG (foldr (λt acc'. dg_local (traverse_program t τ) ⊔ acc') ?ps (dg_local ?acc0)) bot"
      by (simp add: eq_routed_node_rhs_buffered[OF wf] dg_local_foldr_generic)
    also have "... = DG (side_acc_dg (dg_local ?acc0) τ ?ps) bot"
      by (simp add: side_acc_dg_as_foldr_seeded)
    also have "... = traverse_rhs
        (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ"
      by (simp add: eq_routed_node_rhs[OF wf])
    finally show ?thesis .
  qed
  show ?D
    by (simp add: routed_node_rhs_buffered_def routed_node_rhs_def Let_def
        dep_program_fold_rhs_program_char[OF wf] dep_program_side_rhs_fold_dg_char[OF wf])
  show ?S
  proof (rule ext)
    fix z
    show "sides_of_rhs
        (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀
          (v, ctx)) τ z
      = sides_of_rhs
        (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀ (v, ctx)) τ z"
      using glob
      by (cases "v = cfg_entry g")
         (auto simp: routed_node_rhs_buffered_def routed_node_rhs_def Let_def
            sides_of_program_fold_rhs_program_char[OF wf]
            sides_of_program_side_rhs_fold_dg_char[OF wf]
            traverse_fold_rhs_program_char_foldr[OF wf] sup_commute bot_dg_state_def[symmetric])
  qed
qed

lemma part_post_solution_routed_node_rhs_buffered_answers_local:
```

```
  fixes cmb ::
    "(pp ⇒ 'c ⇒ 'd ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pp
       ⇒ (pp × 'c, 'k, ('d::bounded_semilattice_sup_bot,
                        'h::bounded_semilattice_sup_bot) dg_state,
           ('d, 'h) dg_state) strategy_program"
  assumes wf: "⋀c' w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                    extra g c' w). sp_wf p"
    and answers_local: "⋀c' w p τ. p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                  extra g c' w) ⟹ dg_global (traverse_program p τ) = bot"
    and pp_buf: "part_post_solution
        (routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀)
        x sigma vars"
  shows "part_post_solution
      (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀) x sigma vars"
proof -
  let ?Tbuf =
    "routed_node_rhs_buffered pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀"
  let ?Told =
    "routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀"
  have corr: "⋀u. traverse_rhs (?Tbuf u) sigma = traverse_rhs (?Told u) sigma
      ∧ dep_aux sigma (?Tbuf u) = dep_aux sigma (?Told u)
      ∧ sides_of_rhs (?Tbuf u) sigma = sides_of_rhs (?Told u) sigma"
    proof -
      fix u :: "pp × 'c"
      obtain v ctx where u: "u = (v, ctx)" by (cases u) auto
      show "traverse_rhs (?Tbuf u) sigma = traverse_rhs (?Told u) sigma
          ∧ dep_aux sigma (?Tbuf u) = dep_aux sigma (?Told u)
          ∧ sides_of_rhs (?Tbuf u) sigma = sides_of_rhs (?Told u) sigma"
        unfolding u
        by (intro conjI routed_node_rhs_buffered_correspondence_answers_local[OF wf])
           (erule answers_local)+
    qed
  have "part_post_solution ?Tbuf x sigma vars = part_post_solution ?Told x sigma vars"
    by (rule part_post_solution_cong) (use corr in blast)+
  with pp_buf show ?thesis by simp
qed
```

Threefold monotonicity for an arbitrary generator instance

A generic fact about pointwise-ordered functions, function update, and join — nothing here mentions a strategy tree. It has exactly one use, in the publication case below, so it stays beside that call site rather than in generic infrastructure.

```
lemma fun_upd_sup_mono:
  fixes m1 m2 :: "'b ⇒ 'a::bounded_semilattice_sup_bot"
  assumes "m1 ≤ m2"
  shows "m1(y := m1 y ⊔ cd) ≤ m2(y := m2 y ⊔ cd)"
proof -
  have eq: "⋀m. m(y := m y ⊔ cd) = m ⊔ ((λ_. bot)(y := cd))"
    unfolding fun_upd_def sup_fun_def by (rule ext) simp
  show ?thesis unfolding eq by (rule sup_mono[OF assms order_refl])
qed
```

The generator never sees a specification: the intra hook supplies each edge's program, exactly as `cmb` and `extra` already supply theirs, so which unknowns an equation reads and which keys it publishes are properties of the supplied programs, not of the generator. The one key the generator names itself is `buffer_key_at c`, at the CFG entry, where it publishes g_0; a routed activation seed is not its own but arrives through `extra`.

Initial global values as side effects

We represent an initial global value as an actual contribution to its global unknown, so the solved unknown itself contains that value. Reads do not need to join initialization into the stored value implicitly. `with_init init T` provides this for any equation system: the right-hand side of an unknown `x` first publishes every pair of `init x`, then continues as `T x`. An instance that publishes no initial values passes `λ_. []`, and the system is `T`.

```
definition init_sides :: "('g × 'd) list ⇒ ('x, 'g, 'd) strategy_tree ⇒ ('x, 'g, 'd) strategy_tree"
where
  "init_sides xs t = foldr (λ(k, d) t. Side k d t) xs t"

definition with_init :: "('x ⇒ ('g × 'd) list) ⇒ ('x, 'g, 'd) eqsT ⇒ ('x, 'g, 'd) eqsT"
where
  "with_init init T = (λx. init_sides (init x) (T x))"

lemma with_init_Nil [simp]: "with_init (λ_. []) T = T"
```

```
  by (simp add: with_init_def init_sides_def)

lemma init_sides_Cons [simp]: "init_sides ((k, d) # xs) t = Side k d (init_sides xs t)"
  by (simp add: init_sides_def)

lemma init_sides_Nil [simp]: "init_sides [] t = t"
  by (simp add: init_sides_def)

lemma traverse_rhs_init_sides [simp]: "traverse_rhs (init_sides xs t) σ = traverse_rhs t σ"
proof (induction xs)
  case (Cons p xs) then show ?case by (cases p) simp
qed simp

lemma dep_aux_init_sides [simp]: "dep_aux σ (init_sides xs t) = dep_aux σ t"
proof (induction xs)
  case (Cons p xs) then show ?case by (cases p) simp
qed simp

lemma sides_of_rhs_init_sides_ge:
  "sides_of_rhs t σ ≤ sides_of_rhs (init_sides xs (t :: ('x, 'g, 'd::bounded_semilattice_sup_bot) strategy_tree)) σ"
proof (induction xs)
  case (Cons p xs)
  obtain k d where p: "p = (k, d)" by (cases p)
  have "sides_of_rhs (init_sides xs t) σ
        ≤ sides_of_rhs (init_sides (p # xs) t) σ"
    unfolding p le_fun_def by (simp add: Let_def)
  with Cons.IH show ?case by (rule order_trans)
qed simp

lemma sides_of_rhs_init_sides_at:
  "(k, d) ∈ set xs
   ⟹ d ≤ sides_of_rhs (init_sides xs (t :: ('x, 'g, 'd::bounded_semilattice_sup_bot) strategy_tree)) σ (Inr k)"
proof (induction xs)
  case (Cons p xs)
  obtain k' d' where p: "p = (k', d')" by (cases p)
  show ?case
  proof (cases "(k, d) = p")
    case True
    with p show ?thesis by (simp add: Let_def)
  next
    case False
    with Cons.prems have "d ≤ sides_of_rhs (init_sides xs t) σ (Inr k)"
      by (intro Cons.IH) simp
    also have "... ≤ sides_of_rhs (init_sides (p # xs) t) σ (Inr k)"
      unfolding p by (simp add: Let_def)
    finally show ?thesis .
  qed
qed simp
```

A post-solution of the wrapped system bounds the wrapped system's own equations, since the wrapper only adds side effects, and covers every initial value at its key.

```
lemma tree_covered_at_init_sides:
  "tree_covered_at (init_sides xs (t :: ('x, 'g, 'd::bounded_semilattice_sup_bot) strategy_tree)) σ u
   ⟹ tree_covered_at t σ u"
  unfolding tree_covered_at_def
  by simp (meson order_trans sides_of_rhs_init_sides_ge)

lemma dep_with_init [simp]: "dep (with_init init T) σ x = dep T σ x"
  by (simp add: dep_def with_init_def)

lemma post_bounded_with_init_at:
  assumes
    "post_bounded (with_init init (T :: ('x, 'g, 'd::bounded_semilattice_sup_bot) eqsT)) x σ vars"
    and "u ∈ vars" and "(k, d) ∈ set (init u)"
  shows "d ≤ σ (Inr k)"
proof -
  have "sides_of_rhs (init_sides (init u) (T u)) σ ≤ σ"
    using post_boundedD[OF assms(1,2)] by (simp add: with_init_def tree_covered_at_def)
  then have "sides_of_rhs (init_sides (init u) (T u)) σ (Inr k) ≤ σ (Inr k)"
    by (rule le_funD)
  with sides_of_rhs_init_sides_at[OF assms(3)] show ?thesis by (rule order_trans)
qed

lemma part_post_solution_with_init:
  "part_post_solution (with_init init (T :: ('x, 'g, 'd::bounded_semilattice_sup_bot) eqsT)) x σ vars
   ⟹ part_post_solution T x σ vars"
```

```
  unfolding part_post_solution_iff_tree_covered_at
proof (intro conjI ballI; elim conjE)
  assume all: "∀u∈vars. dep_L (with_init init T) σ u ⊆ vars
                ∧ tree_covered_at (with_init init T u) σ u"
  fix u assume u: "u ∈ vars"
  show "dep_L T σ u ⊆ vars" using all u by (simp add: dep_L_def)
  show "tree_covered_at (T u) σ u"
    using all u by (simp add: with_init_def) (blast intro: tree_covered_at_init_sides)
qed
end

end
```

6.11 DG_Reader_Transport

```
theory DG_Reader_Transport
  imports Routed_Context
begin
```

Reading a whole equation system through a pair of readers

A `reader` here maps one carrier to another — typically an implementation value to the mathematical value it stands for. This theory proves, once, that reading commutes with everything an equation is built from: a node's value, the side effects it emits, and the unknowns it queries. The only properties any proof in the chain uses are that a reader preserves `bot` and `(⊔)`, so they are the locale's two assumptions and the development is generic in both carriers. A concrete reader becomes a one-line interpretation.

Dependency commutation for the generator

The generator trees are non-branching (`QueryL` then `QueryG` then `Answer`/`Side`), so the queried-unknown set is structural: independent of the analysis step values and the valuation. Hence dependencies transport verbatim.

Carrier-generic whole-CFG commute

The commute facts below only ever use that a reader preserves `bot` and `(⊔)`; no proof in the chain inspects `default_st_to_fun` or `gabs_state` itself. `dg_reader_commute_gen` factors that out: a pair of local/global readers `Floc`/`Fglob` satisfying those two laws, from which every whole-tree and whole-equation-system commute fact in this chain is proved once.

Reading a pair componentwise is the datatype's own map, `map_dg_state`; its selector equations join the simplifier.

```
declare dg_state.map_sel [simp]
```

The locale assumes only that both readers preserve `bot` and joins; monotonicity and the commutation with `map_dg_state` follow.

```
locale dg_reader_commute_gen =
  fixes Floc :: "'a::bounded_semilattice_sup_bot ⇒ 'a2::bounded_semilattice_sup_bot"
    and Fglob :: "'b::bounded_semilattice_sup_bot ⇒ 'b2::bounded_semilattice_sup_bot"
  assumes Floc_bot: "Floc bot = bot"
      and Floc_sup: "⋀x y. Floc (x ⊔ y) = Floc x ⊔ Floc y"
      and Fglob_bot: "Fglob bot = bot"
      and Fglob_sup: "⋀x y. Fglob (x ⊔ y) = Fglob x ⊔ Fglob y"
begin

lemma Floc_mono: "x ≤ y ⟹ Floc x ≤ Floc y"
  by (metis Floc_sup le_iff_sup)

lemma Fglob_mono: "x ≤ y ⟹ Fglob x ≤ Fglob y"
  by (metis Fglob_sup le_iff_sup)

lemma map_dg_state_bot [simp]:
  "map_dg_state Floc Fglob (bot :: ('a,'b) dg_state) = bot"
  by (simp add: bot_dg_state_def Floc_bot Fglob_bot)

lemma map_dg_state_sup:
  "map_dg_state Floc Fglob (a ⊔ b :: ('a,'b) dg_state)
     = map_dg_state Floc Fglob a ⊔ map_dg_state Floc Fglob b"
  by (simp add: sup_dg_state_def Floc_sup Fglob_sup)
```

Bundled per-tree transport relation `dg_tree_st_commute` bundles the three facts one strategy tree must transport: its value, its side effects per key, and its dependencies.

```
definition dg_tree_st_commute ::
  "('u + 'k ⇒ ('a,'b) dg_state) ⇒ ('u, 'k, ('a,'b) dg_state) strategy_tree
    ⇒ ('u, 'k, ('a2,'b2) dg_state) strategy_tree ⇒ bool"
where
  "dg_tree_st_commute σ_st t_st t_abs ⟷
     map_dg_state Floc Fglob (traverse_rhs t_st σ_st) =
       traverse_rhs t_abs (map_dg_state Floc Fglob ∘ σ_st) ∧
     (∀k. map_dg_state Floc Fglob (sides_of_rhs t_st σ_st k) =
       sides_of_rhs t_abs (map_dg_state Floc Fglob ∘ σ_st) k) ∧
     dep_aux σ_st t_st =
       dep_aux (map_dg_state Floc Fglob ∘ σ_st) t_abs"
```

The same relation between two *programs*, bundled with the well-formedness each side needs for the fold's observations to be well posed. Carrying `sp_wf` inside the relation is what lets a list induction over two contribution lists keep it: a `list_all2` of this relation says every pair transports and every element runs its continuation once.

```
definition dg_prog_st_commute ::
  "('u + 'k ⇒ ('a,'b) dg_state)
   ⇒ ('u, 'k, ('a,'b) dg_state, ('a,'b) dg_state) strategy_program
   ⇒ ('u, 'k, ('a2,'b2) dg_state, ('a2,'b2) dg_state) strategy_program ⇒ bool"
where
  "dg_prog_st_commute σ_st p_st p_abs ⟷
     sp_wf p_st ∧ sp_wf p_abs
     ∧ dg_tree_st_commute σ_st (sp_compile p_st) (sp_compile p_abs)"
```

Routed heterogeneous CALL/COMB transport `routed_call_program`/`routed_entry_seed_programs` (`Voblint_Framewo` are the canonical heterogeneous routing shape: parametric only in a routing function `route` and a seed-key injection `seed`, with the seed payload carried on the `dg_local` half so `'D`/`'G` stay independent. The lemmas below transport `routed_call_program`, so any context-sensitive analysis instantiating `cmb` at that constant discharges CALL/COMB transport once here rather than re-deriving its own tree-commute reasoning.

The specification's own enter and combine appear inside the routed tree as compiled sub-trees, so their transport hypotheses are themselves tree commutes. Sequencing them is what `dg_tree_st_commute_seqcomp` does: a bind commutes when its head commutes and its continuation commutes at the head's answer.

```
lemma dg_tree_st_commute_seqcomp:
  assumes head: "dg_tree_st_commute σ_st t_st t_abs"
    and tail: "dg_tree_st_commute σ_st (k_st (traverse_rhs t_st σ_st))
                 (k_abs (traverse_rhs t_abs (map_dg_state Floc Fglob ∘ σ_st)))"
  shows "dg_tree_st_commute σ_st (sp_lift_tree t_st k_st) (sp_lift_tree t_abs k_abs)"
  using head tail
  unfolding dg_tree_st_commute_def
  by (simp add: map_dg_state_sup)

lemma dg_tree_st_commute_answer:
  "dg_tree_st_commute σ_st (Answer d) (Answer (map_dg_state Floc Fglob d))"
  by (simp add: dg_tree_st_commute_def)
```

A local-only transfer compiles to a single answer, so its commute is the reader equation on the pure function alone – no tree reasoning, and no hypothesis about the global half beyond the reader's own `bot` law.

```
lemma dg_tree_st_commute_local_transfer:
  assumes "Floc (f d) = F (Floc d)"
  shows "dg_tree_st_commute σ_st
           (sp_compile_with (λx. DG x bot) (local_transfer f (mk_dg_man d gk)))
           (sp_compile_with (λx. DG x bot) (local_transfer F (mk_dg_man (Floc d) gk)))"
  by (simp add: dg_tree_st_commute_def local_transfer_def mk_dg_man_def
      dg_state.map_sel bot_dg_state_def assms Floc_bot Fglob_bot)
```

What a compiled entry must satisfy to transport. Entry answers a list of caller-continuation/callee-entry pairs, which is not the solver's carrier, so the reader equation cannot be stated on the answer of a compiled tree the way the edge and combine transports are. It is stated on the program instead: run both entries against continuations that already agree on pairs mapped componentwise by the readers, and the two trees agree. This observes the entry program exactly as `enter_runs` does, and for the same reason.

```
definition dg_enter_st_commute ::
  "('u + 'k ⇒ ('a,'b) dg_state)
   ⇒ ('u,'k,('a,'b) dg_state,'a enter_result list) strategy_program
   ⇒ ('u,'k,('a2,'b2) dg_state,'a2 enter_result list) strategy_program ⇒ bool"
where
  "dg_enter_st_commute σ_st T_st T_abs ⟷
     (∀K_st K_abs.
        (∀ps. dg_tree_st_commute σ_st (K_st ps) (K_abs (map (map_prod Floc Floc) ps)))
          ⟶ dg_tree_st_commute σ_st (T_st K_st) (T_abs K_abs))"

end
end
```

6.12 Activation_Backbone

```
theory Activation_Backbone
  imports "Voblint_Domain.Nonrelational_State" "Voblint_CFG.Activation_Trace_Abstract"
begin
```

What one activation of a procedure may observe

Every concrete call creates one callee activation. Each step of that activation carries the contexts its creating call admits, a nested call derives its callee's contexts from those, and a return resumes the caller at the caller's own. `activation_context_rel` replays those choices along a trace, so a table indexed by `(node, context)` can be checked against concrete runs without carrying an auxiliary digest. That indexing is many-to-one and one-to-many at once: activations reaching the same context are collected together, and one activation may be collected under several contexts. The buckets of a node cover its stores; they need not partition them.

`cover v c` is the set of stores that table admits at node `v` in context `c`. Given the five local obligations of `activation_coverage` on it — `INIT` for the seed stores, `INTRA` per intra edge, `CALL` per call and admitted context, `RETURN` per return, `TOTAL` for at least one admitted context per covered call — `activation_collect_sound` bounds `activation_collect`, the set of stores some valid activation trace can leave at one `(node, context)`. It is the context-sensitive twin of a monovariant post-fixpoint argument: interpret `activation_coverage` at the supplied `cover`, then read off `valid_activation_trace_covered_at`.

`adm` says which contexts may describe a concrete call transition, and that is what indexes the collecting semantics. It is handed both stores, so a policy may inspect them, but need not: a call-string policy ignores them, and an entry-state policy is induced by the analyzer's own routing decisions on the abstract entry alternatives that cover them. Proving that a particular policy's `route` induces an `adm` total on the solved table is the routed context locale's job downstream, not this theorem's.

```
theorem activation_collect_sound:
  assumes "activation_coverage κ g S cover adm c₀"
  shows "𝒜κ,adm,c₀,g,S v c ⊆ cover v c"
proof -
  interpret G: activation_coverage κ g S cover adm c₀ by (fact assms)
  show ?thesis
  proof (rule subsetI)
    fix st assume "st ∈ 𝒜κ,adm,c₀,g,S v c"
    then obtain t where t: "t ∈ 𝒯κ,g,S"
      and sn: "sink_node t = v" and kc: "activation_context_rel κ adm c₀ g t c"
      and st: "sink_store t = st"
      by (rule activation_collect_E)
    have "sink_store t ∈ cover (sink_node t) c" using G.valid_activation_trace_covered_at[OF t kc]
      .
    then show "st ∈ cover v c" using sn st by simp
  qed
qed

end
```

6.13 DG_Ctx_Activation

```
theory DG_Ctx_Activation
  imports DG_Spec_Sound DG_Indexed_Generator
    "Voblint_Domain.Nonrelational_State"
begin
```

What a solved routed system says about one program point

A post-solution bounds a right-hand side by the value at its own unknown and every published side effect by the value at its target key. `dg_context_activation` fixes such a solution over the routed generator, a set `vars` of keys whose own equations it bounds, and a reader `sg` that answers the empty set off `vars`. The set may contain every key the solver visited or any subset. These facts derive the EDGE obligation the activation backbone asks for; `Routed_Context` derives COMB for the routed call shape. Entry and callee-seed coverage remain with the concrete analysis.

Both carriers are parameters: γ_{DG} interprets a D/G pair, while γ_M interprets whatever `sg` returns. An analysis whose reader is not a `gabs_state` therefore instantiates this locale directly. The generator buffers its writes at `Inr buffer_key`; `global_of` names the key each global name is read from, and `sg_cov` ties the reader at a covered key to γ_{DG} of the local slot against the environment `genv global_of sigma`.

```
locale dg_context_base =
  fixes S ::
    "(pp × 'c, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec"
    and γ_DG :: "'D ⇒ ('v ⇒ 'G) ⇒ store set"
    and g :: "(var_id, exp) gcfg" and buffer_key :: 'k and global_of :: "'v ⇒ 'k"
    and route :: "pp ⇒ 'c ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'c"
    and cmb :: "(pp ⇒ 'c ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c
                ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pp
                ⇒ (pp × 'c, 'k, ('D, 'G) dg_state, ('D, 'G) dg_state) strategy_program"
      — one program per call site: the site's own resolver decides which callees it folds
    and extra :: "(pp ⇒ 'c ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c ⇒ pp
                ⇒ (pp × 'c, 'k, ('D, 'G) dg_state, ('D, 'G) dg_state) strategy_program list"
    and bot0 d₀ :: 'D and g₀ :: 'G
    and sigma :: "pp × 'c + 'k ⇒ ('D, 'G) dg_state"
    and vars :: "(pp × 'c) set" and x0 :: "pp × 'c"
    and sg :: "pp × 'c + 'k ⇒ 'M"
    and γ_M :: "'M ⇒ store set"
  assumes cmb_wf: "⋀c ca cc v. sp_wf (cmb route c ca cc v)"
    and extra_wf: "⋀c v q. q ∈ set (extra route c v) ⟹ sp_wf q"
    and finE: "finite (intra g)"
    and pp: "post_bounded
               (routed_node_rhs intra_predecessor_addr_list call_site_list (λ_. buffer_key)
                  route (λc src a. dg_spec_edge_program S a src global_of) cmb extra g bot0 d₀ g₀)
               x0 sigma vars"
    and sg_cov[simp]: "⋀v c. (v, c) ∈ vars
        ⟹ γ_M (sg (Inl (v, c))) =
          γ_DG (dg_local (sigma (Inl (v, c)))) (genv global_of sigma)"
    and sg_uncov[simp]: "⋀v c. (v, c) ∉ vars
        ⟹ γ_M (sg (Inl (v, c))) = {}"
    and fwd: "⋀u a v c. (u, c) ∈ vars
        ⟹ γ_DG (dg_local (sigma (Inl (u, c)))) (genv global_of sigma) ≠ {}
        ⟹ (u, a, v) ∈ intra g
        ⟹ (v, c) ∈ vars"
```

The obligations above do not mention the concrete semantics, so every language shares them.

```
locale dg_context_activation =
  analysis_contract κ S γ_DG +
  dg_context_base S γ_DG g buffer_key global_of route cmb extra bot0 d₀ g₀
    sigma vars x0 sg γ_M
  for κ :: kenv
    and S ::
      "(pp × 'c, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec"
    and γ_DG g buffer_key global_of route cmb extra bot0 d₀ g₀ sigma vars x0 sg γ_M
begin

abbreviation Gen :: "(pp × 'c, 'k, ('D, 'G) dg_state) eqsT" where
  "Gen ≡ routed_node_rhs intra_predecessor_addr_list call_site_list (λ_. buffer_key)
           route (λc src a. dg_spec_edge_program S a src global_of) cmb extra g bot0 d₀ g₀"

abbreviation acc0 :: "pp ⇒ 'D" where
  "acc0 v ≡ (if v = cfg_entry g then bot0 ⊔ d₀ else bot0)"

abbreviation contribs :: "pp ⇒ 'c
```

```
    ⇒ (pp × 'c, 'k, ('D, 'G) dg_state, ('D, 'G) dg_state) strategy_program list" where
  "contribs v ctx ≡
     routed_contribution_programs intra_predecessor_addr_list call_site_list route
       (λc src a. dg_spec_edge_program S a src global_of) cmb extra g ctx v"
```

The reader's claim at a covered point (`sg_cov`) and the manager its call programs run under. Input-only, so facts of theory-level interpretations keep printing the explicit terms.

```
abbreviation (input) gamma_at :: "pp ⇒ 'c ⇒ store set" where
  "gamma_at v ctx ≡ γ_DG (dg_local (sigma (Inl (v, ctx)))) (genv global_of sigma)"

abbreviation (input) man_at where
  "man_at v ctx ≡ mk_dg_man (dg_local (sigma (Inl (v, ctx)))) global_of"
```

Every contribution the routed generator folds here runs its continuation once: the edge hook is a compiled specification step, and the call and extra hooks carry their own well-formedness as locale assumptions.

```
lemma contribs_wf [simp]: "∀q ∈ set (contribs v ctx). sp_wf q"
  by (rule routed_contribution_programs_wf)
     (auto intro: sp_wf_dg_spec_edge_program[OF spec_wf] cmb_wf extra_wf)
```

Post-solution elimination

The solver hypothesis, unpacked once into the two bounds every later proof uses: at a covered unknown the generated right-hand side is below the stored local, and its side writes are below the stored globals. Everything downstream cites these instead of `part_post_solution`.

```
lemma pp_eq_bound:
  "(v, ctx) ∈ vars ⟹ eq Gen (v, ctx) sigma ≤ sigma (Inl (v, ctx))"
  by (rule tree_covered_at_local[OF post_boundedD[OF pp]])

lemma pp_sides_bound:
  "(v, ctx) ∈ vars ⟹ sides_of_rhs (Gen (v, ctx)) sigma ≤ sigma"
  by (rule tree_covered_at_sides[OF post_boundedD[OF pp]])

lemma pp_entry_init_global_bound:
  assumes cov: "(cfg_entry g, ctx) ∈ vars"
  shows "g₀ ≤ dg_global (sigma (Inr buffer_key))"
proof -
  have "g₀ ≤ dg_global (sides_of_rhs (Gen (cfg_entry g, ctx)) sigma (Inr buffer_key))"
    unfolding routed_node_rhs_def Let_def
    by (simp add: Let_def sup_dg_state_def)
  also have "... ≤ dg_global (sigma (Inr buffer_key))"
    using pp_sides_bound[OF cov, THEN le_funD, of "Inr buffer_key"]
    by (simp add: less_eq_dg_state_def)
  finally show ?thesis .
qed
```

The local-carrier twin, proved the same way: `Gen`'s entry accumulator starts at `bot0` ⊔ d_0, `side_acc_dg_ge_acc` only grows it, and `pp_eq_bound` transports the bound across a covered point's own equation.

```
lemma pp_entry_init_local_bound:
  assumes cov: "(cfg_entry g, ctx) ∈ vars"
  shows "d₀ ≤ dg_local (sigma (Inl (cfg_entry g, ctx)))"
proof -
  have "d₀ ≤ dg_local (eq Gen (cfg_entry g, ctx) sigma)"
    by (simp add: eq_routed_node_rhs[OF contribs_wf])
       (rule order_trans[OF _ side_acc_dg_ge_acc], simp add: le_supI2)
  also have "... ≤ dg_local (sigma (Inl (cfg_entry g, ctx)))"
    using pp_eq_bound[OF cov] by (simp add: less_eq_dg_state_def)
  finally show ?thesis .
qed
```

The entry Side wrapper only grows the sides

At the entry the generator wraps its fold in one extra `Side` carrying the seed global, so a side read of the wrapped tree dominates the same read of the fold. Bounds proved against the fold therefore transport to `routed_node_rhs` without a separate entry case.

```
lemma sides_fold_le_Gen:
  "(sides_of_program (side_rhs_fold_dg (acc0 v) (contribs v ctx)) sigma) k
   ≤ sides_of_rhs (Gen (v, ctx)) sigma k"
  unfolding routed_node_rhs_def Let_def
  by (cases "v = cfg_entry g") (auto simp: Let_def intro: sup.cobounded1)
```

EDGE: the routed intra bounds and the guarded transport

Every intra predecessor of v contributes its edge program to v's own fold, so both bounds below are the post-solution read off that one membership.

```
lemma edge_program_mem_contribs:
  assumes e: "(u, a, v) ∈ intra g"
  shows "dg_spec_edge_program S a (Inl (u, ctx)) global_of ∈ set (contribs v ctx)"
proof -
  have "(Inl (u, ctx), a) ∈ set (intra_predecessor_addr_list g v ctx)"
    using e by (force simp: intra_predecessor_addr_list_def
        set_intra_predecessor_list[OF finE] intra_predecessors_def)
  thus ?thesis by (force simp: routed_contribution_programs_def)
qed

lemma edge_bound_local:
  assumes cov_v: "(v, ctx) ∈ vars"
    and e: "(u, a, v) ∈ intra g"
  shows "edge_out S a (Inl (u, ctx)) global_of sigma ≤ dg_local (sigma (Inl (v, ctx)))"
proof -
  have "edge_out S a (Inl (u, ctx)) global_of sigma
      ≤ side_acc_dg (acc0 v) sigma (contribs v ctx)"
    using dg_local_traverse_le_side_acc_dg[OF edge_program_mem_contribs[OF e]] .
  also have "... = dg_local (eq Gen (v, ctx) sigma)"
    by (simp add: eq_routed_node_rhs[OF contribs_wf])
  also have "... ≤ dg_local (sigma (Inl (v, ctx)))"
    using pp_eq_bound[OF cov_v] by (simp add: less_eq_dg_state_def)
  finally show ?thesis .
qed

lemma edge_bound_global:
  assumes cov_v: "(v, ctx) ∈ vars"
    and e: "(u, a, v) ∈ intra g"
  shows "dg_global (sides_of_program (dg_spec_edge_program S a (Inl (u, ctx)) global_of) sigma (Inr k))
          ≤ dg_global (sigma (Inr k))"
proof -
  have "dg_global (sides_of_program (dg_spec_edge_program S a (Inl (u, ctx)) global_of) sigma (Inr k))
      ≤ dg_global ((sides_of_program (side_rhs_fold_dg (acc0 v) (contribs v ctx)) sigma) (Inr k))"
    using sides_le_side_rhs_fold_dg[OF contribs_wf edge_program_mem_contribs[OF e], where k =
      "Inr k"]
    by (simp add: less_eq_dg_state_def)
  also have "... ≤ dg_global (sides_of_rhs (Gen (v, ctx)) sigma (Inr k))"
    using sides_fold_le_Gen[where k = "Inr k"]
    by (simp add: less_eq_dg_state_def)
  also have "... ≤ dg_global (sigma (Inr k))"
    using pp_sides_bound[OF cov_v, THEN le_funD, of "Inr k"]
    by (simp add: less_eq_dg_state_def)
  finally show ?thesis .
qed

lemma edge_bound_genv:
  assumes cov_v: "(v, ctx) ∈ vars"
    and e: "(u, a, v) ∈ intra g"
  shows "edge_pub S a (Inl (u, ctx)) global_of sigma ≤ genv global_of sigma"
  unfolding le_fun_def genv_def using edge_bound_global[OF cov_v e] by blast

theorem dg_ctx_act_edge:
  assumes e: "(u, a, v) ∈ intra g"
    and sin: "s ∈ γ_M (sg (Inl (u, ctx)))" and st: "s' ∈ edge_step κ a s"
  shows "s' ∈ γ_M (sg (Inl (v, ctx)))"
proof (cases "(u, ctx) ∈ vars")
  case False
  hence "γ_M (sg (Inl (u, ctx))) = {}" by (rule sg_uncov)
  thus ?thesis using sin by simp
next
  case True
  let ?d = "dg_local (sigma (Inl (u, ctx)))"
  let ?g = "genv global_of sigma"
  have sin': "s ∈ γ_DG ?d ?g"
    using sin True by simp
  have cov_v: "(v, ctx) ∈ vars"
    using True _ e by (rule fwd) (use sin' in blast)
  have "{s} ⊆ γ_DG ?d ?g" using sin' by simp
  hence "edge_collect κ a {s} ⊆ edge_collect κ a (γ_DG ?d ?g)" by (rule edge_collect_mono)
  moreover have "s' ∈ edge_collect κ a {s}" using st by (simp add: edge_collect_single)
  ultimately have "s' ∈ edge_collect κ a (γ_DG ?d ?g)" by blast
  hence "s' ∈ γ_DG
```

```
      (edge_out S a (Inl (u, ctx)) global_of sigma)
      (?g ⊔ edge_pub S a (Inl (u, ctx)) global_of sigma)"
    using step_sound[of a sigma "Inl (u, ctx)" global_of] by blast
  also have "... ⊆ gamma_at v ctx"
    by (rule gammaDG_mono[OF edge_bound_local[OF cov_v e]
          sup_least[OF order_refl edge_bound_genv[OF cov_v e]]])
  also have "... = γ_M (sg (Inl (v, ctx)))"
    using cov_v by simp
  finally show ?thesis .
qed

end

end
```

6.14 Routed_Call_Programs

```
theory Routed_Call_Programs
  imports DG_Spec_Sound DG_Indexed_Generator
    "Voblint_Domain.Nonrelational_State"
begin
```

The equations one call action generates

What a routed call is, as solver input, before anything is claimed about its soundness. Each callee a call action resolves to becomes one strategy program: it runs the specification's entry transfer on the caller's local value, chooses the callee's context from the entry half of the alternative that produced, publishes that entered state at a global proxy key — an *activation seed*, the only channel by which the entered state reaches the callee's entry unknown — reads the callee's exit back at that same context, and runs the specification's combine against the alternative's continuation half. The publication only joins into the seed and destabilizes its readers; the exit read is the query that makes the solver evaluate the callee at that context. The other half of the protocol is the callee's own entry equation reading its seed slot back: that is `routed_entry_seed_programs`.

This theory is construction and observation only: which unknowns a program reads, which keys it publishes to, and lower bounds on what it contributes at each. Whether those contributions are sound is `Routed_Context`'s question, which fixes the shape built here as `cmb` and `extra` and discharges CALL and COMB against it once.

The routed key space

```
datatype ('v,'c) global_unknown =
  Analysis_Buffer
| Analysis_Global 'v
| Activation_Seed (seed_pp: pp) (seed_ctx: 'c)
```

The solver's global unknowns have three owners. The generator uses `Analysis_Buffer` for a node's buffered global answer. `Analysis_Global` wraps a name of the analysis's own type `'v` — Goblint's `V` — and is the only kind an analysis can reach, through `man_global`/`man_sideg`. `Activation_Seed` holds the entry state published at a callee's activation, keyed by the callee entry point and the routed context.

The remaining constructors separate analysis effects from activation routing. `Analysis_Global` embeds this development's analogue of Goblint's `Spec.V` into the solver's global-key space — the namespace itself is `'v`, and this constructor is the address around it. Goblint's own Base analysis supplies a non-trivial one (privatization globals, thread-return globals) rather than a single global cell. `Activation_Seed` is not Goblint's — their framework records reachable contexts on a separate global whose value is a set of contexts, keyed by function. It is a solver adapter, not the translation of any Goblint constraint constructor: the vendored solver publishes to global keys only, so the one way to hand a callee's entry unknown a value is to publish at a global proxy that the entry equation reads back. Publication does not demand the callee; the caller's read of the callee's exit does. It is named to say so, so that no later reader takes it for a Goblint correspondence.

The types keep the two apart. A transfer names a global as a `'v` and never builds a key, because `mk_dg_man` holds the embedding; only the routing and generator layers apply `Activation_Seed`. Every routed context instantiates `'c` (call strings at `cfg_node list`, entry states at a value list, context-insensitive at `unit` — which is exactly Goblint's own `C = Printable.Unit`), and an analysis with one global takes `'v = unit`.

```
lemma activation_seed_ne_analysis_global [simp]:
  "Activation_Seed u c ≠ Analysis_Global v"
  "Analysis_Global v ≠ Activation_Seed u c"
  by simp_all
```

The canonical routed entry-seed publication and return combine

The routing combine reads the caller under its own context, the callee exit under the context `route` selects from the callee's entered value — the alternative's own second component, after `enter`$^\sharp$ has run — and the exact matched `call_action` (from `return_call_action_list`, never re-derived from the call site's outgoing edges). The spec's own enter and combine transfers run as compiled manager programs over the caller value, so whether the global slot `buffer_key` is read or published at all is the specification's business: a Base-style spec's transfers touch no global and the routed program then carries no `buffer_key` effect either, while an effectful spec's own `man_global`/`man_sideg` calls ride inside the compiled subtrees.

Parameter order matches `dg_context_activation`'s `cmb` calling convention: the generator supplies `route` as `cmb`'s own first argument (`cmb route c ca cc ex` in `routed_node_rhs`), so `routed_call_program S global_of seed`, closing over the spec, the shared slot, and the seed-key injection, is the value that instantiates `cmb`.

This program owns the whole call lifecycle for one call action: it reads the caller once, routes the callee context once (`ctx'`), and shares that one `ctx'` between the entry-seed publication and the combine's callee-exit read.

The publication hands the callee's entry its value, and the exit read, a `QueryL` of the callee's `FunctionResult` node at `ctx'`, is what makes the solver evaluate the callee; both happen in the one equation that combines the result, so `enter#` runs exactly once per call action rather than once per hook. The caller continuation is not reconstructed here: it is handed to `dg_spec_combine_transfer` as the manager's own local value, and that pipeline runs `combine_env`$^\sharp$ and `combine_assign`$^\sharp$ over it, so this layer only chooses addresses and keys. Only the half that cannot move stays with `routed_entry_seed_programs`: a callee's own entry equation is the one place that can read its own seed slot back. `route` is kept as a parameter of that hook purely to match `dg_context_activation`'s `extra` calling convention, which always supplies it.

The seed channel is `'D`-typed, so `'D` and `'G` stay independent. The publication writes the callee's freshly entered local state — a `'D`-typed value — into the `dg_local` half of the `dg_state` published at the seed key. A `dg_state` already carries both fields at every key regardless of whether the key is reached through `Inl` (a `(pp, 'c)` local unknown) or `Inr` (a `'k` global unknown): `dg_local`/`dg_global` are call-site conventions, not something `QueryL`/`QueryG` themselves enforce. Since `seed p ctx` is always distinct from `buffer_key` (`seed_ne_buffer_key`) and no other equation ever publishes to a seed key, nothing else reads or writes that key's `dg_local` half.

One resolved callee: enter the frame through the spec's compiled enter transfer, key the entered state, publish it at the routed seed, read that activation's exit back and run the spec's composed combine. The caller's local state is passed in rather than read here, because the call site reads it once and every target it resolves to is entered from that same value.

What one call alternative contributes. The continuation and the callee-entry value arrive together, from a single run of the specification's entry, and this is where they are used together: the entry half chooses the context and is published at that context's seed, and the continuation half is the local value the return combine runs against. Neither is recoverable from the other, which is why the pair travels this far intact.

Naming it separately is what lets the bounds below speak about *one* alternative: the program the generator builds is a fold over these, so every obligation about a selected alternative is a statement about one element of that fold rather than about the fold itself.

```
definition routed_call_alternative_program ::
  "(pp × 'c, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec ⇒ ('v ⇒ 'k)
   ⇒ (pp ⇒ 'c ⇒ 'k)
   ⇒ (pp ⇒ 'c ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'c)
   ⇒ ('D ⇒ bool)
   ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pname ⇒ 'D enter_result
   ⇒ (pp × 'c, 'k, ('D, 'G) dg_state, ('D, 'G) dg_state) strategy_program"
where
  "routed_call_alternative_program S global_of seed route is_bot ctx ca cc p alt =
     (case alt of (cont, entry) ⇒
        (if is_bot entry
         then sp_map (λd. DG d bot)
                (dg_spec_combine_transfer S (call_info_of ca p)
                  (mk_dg_man cont global_of) bot)
         else
           (let ctx' = route cc ctx entry ca
            in do {
                 _ ← sp_publish (seed (FunctionEntry p) ctx') (DG entry bot);
                 callee_state ← sp_read_local (FunctionResult p, ctx');
                 sp_map (λd. DG d bot)
                   (dg_spec_combine_transfer S (call_info_of ca p)
                     (mk_dg_man cont global_of)
                     (dg_local callee_state))
               }))"
```

Deliberately not a `simp` rule: it expands into a `Side` over a `QueryL` over a compiled combine, and letting that fire everywhere would put a whole call boundary into every goal mentioning the constant. The observation lemmas below cite it where they need it.

```
lemma routed_call_alternative_program_bot [simp]:
  assumes "is_bot entry"
  shows "routed_call_alternative_program S global_of seed route is_bot ctx ca cc p (cont, entry)
           = sp_map (λd. DG d bot)
               (dg_spec_combine_transfer S (call_info_of ca p)
                 (mk_dg_man cont global_of) bot)"
  using assms by (simp add: routed_call_alternative_program_def)

lemma routed_call_alternative_program_nonbot [simp]:
  assumes "¬ is_bot entry"
  shows "routed_call_alternative_program S global_of seed route is_bot ctx ca cc p (cont, entry)
     = do {
         _ ← sp_publish (seed (FunctionEntry p) (route cc ctx entry ca)) (DG entry bot);
         callee_state ← sp_read_local (FunctionResult p, route cc ctx entry ca);
```

```
        sp_map (λd. DG d bot)
          (dg_spec_combine_transfer S (call_info_of ca p)
            (mk_dg_man cont global_of)
            (dg_local callee_state))
      }"
  unfolding routed_call_alternative_program_def using assms by (simp add: Let_def)

definition routed_callee_call_program ::
  "(pp × 'c, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec ⇒ ('v ⇒ 'k)
   ⇒ (pp ⇒ 'c ⇒ 'k)
   ⇒ (pp ⇒ 'c ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'c)
   ⇒ ('D ⇒ bool)
   ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ 'D ⇒ pname
   ⇒ (pp × 'c, 'k, ('D, 'G) dg_state, ('D, 'G) dg_state) strategy_program"
where
  "routed_callee_call_program S global_of seed route is_bot ctx ca cc caller p =
     do {
       pairs ← enter♯ S (call_info_of ca p) (mk_dg_man caller global_of);
       side_rhs_fold_dg bot
         (map (routed_call_alternative_program S global_of seed route is_bot ctx ca cc p) pairs)
     }"
```

The call site's own contribution: read the caller state, ask `resolve` which procedures this site can enter given that state, and join what each resolved activation returns. The callee is the resolver's answer, not a component of the enumeration the equation system was built from, so a resolver reading the caller state selects the target at solve time. `static_targets` is the resolver that ignores the state and answers from the CFG, which reproduces the statically enumerated behaviour exactly.

An empty answer contributes ⊥. Under `static_targets` that happens only where the CFG has no call edge, i.e. where the enumeration produced no combine tree either.

```
definition routed_call_program ::
  "(pp × 'c, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec ⇒ ('v ⇒ 'k)
   ⇒ (pp ⇒ 'c ⇒ 'k)
   ⇒ (pp ⇒ pp ⇒ (var_id, exp) gcall_action ⇒ 'D ⇒ pname list)
   ⇒ ('D ⇒ bool)
   ⇒ (pp ⇒ 'c ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'c)
   ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pp ⇒ pp
   ⇒ (pp × 'c, 'k, ('D, 'G) dg_state, ('D, 'G) dg_state) strategy_program"
where
  "routed_call_program S global_of seed resolve is_bot route ctx ca cc v =
     do {
       caller_state ← sp_read_local (cc, ctx);
       side_rhs_fold_dg bot
         (map (routed_callee_call_program S global_of seed route is_bot ctx ca cc
                 (dg_local caller_state))
              (resolve v cc ca (dg_local caller_state)))
     }"
```

The seed read-back hook: at a callee entry it reads the seed out of the `dg_local` half, matching `routed_call_program`'s write.

```
definition routed_entry_seed_programs ::
  "(pp ⇒ 'c ⇒ 'k)
   ⇒ (pp ⇒ 'c ⇒ 'D::bounded_semilattice_sup_bot ⇒ (var_id, exp) gcall_action ⇒ 'c)
   ⇒ 'c ⇒ pp
   ⇒ (pp × 'c, 'k, ('D, 'G::bounded_semilattice_sup_bot) dg_state,
       ('D, 'G) dg_state) strategy_program list"
where
  "routed_entry_seed_programs seed route ctx v =
     (case v of FunctionEntry _ ⇒
        [do {
           seed_state ← sp_read_global (seed v ctx);
           sp_return (DG (dg_local seed_state) bot)
         }]
       | _ ⇒ [])"

lemma routed_entry_seed_programs_free:
  "x ∈ set (routed_entry_seed_programs seed route ctx v)
     ⟹ sides_of_program x tau z = bot"
  unfolding routed_entry_seed_programs_def
  by (cases v) (auto simp: sp_compile_def bot_fun_def)

lemma routed_entry_seed_programs_local_only:
  "x ∈ set (routed_entry_seed_programs seed route ctx v)
     ⟹ dg_global (traverse_program x tau) = bot"
```

```
  unfolding routed_entry_seed_programs_def by (cases v) (auto simp: sp_compile_def)
```

Every routed hook runs its continuation once, given that the specification's transfers do. The generator's fold asks for exactly that, and a context policy discharges it by naming its specification's `dg_spec_wf`.

```
lemma sp_wf_routed_entry_seed_programs [intro]:
  "x ∈ set (routed_entry_seed_programs seed route ctx v) ⟹ sp_wf x"
  unfolding routed_entry_seed_programs_def by (cases v) auto

lemma sp_wf_routed_call_alternative_program [intro]:
  assumes "dg_spec_wf S"
  shows
    "sp_wf (routed_call_alternative_program S global_of seed route is_bot ctx ca cc p alt)"
  unfolding routed_call_alternative_program_def Let_def
  by (auto intro!: sp_wf_map sp_wf_bind dg_spec_wf_combine[OF assms] split: prod.splits)

lemma sp_wf_routed_callee_call_program [intro]:
  assumes "dg_spec_wf S"
  shows
    "sp_wf (routed_callee_call_program S global_of seed route is_bot ctx ca cc caller p)"
  unfolding routed_callee_call_program_def
  by (intro sp_wf_bind dg_spec_wf_enter[OF assms] sp_wf_side_rhs_fold_dg)
     (auto intro: sp_wf_routed_call_alternative_program[OF assms])

lemma sp_wf_routed_call_program [intro]:
  assumes "dg_spec_wf S"
  shows
    "sp_wf (routed_call_program S global_of seed resolve is_bot route ctx ca cc v)"
  unfolding routed_call_program_def
  by (intro sp_wf_bind sp_wf_read_local sp_wf_side_rhs_fold_dg)
     (auto intro: sp_wf_routed_callee_call_program[OF assms])

lemma sp_wf_routed_call_alternative_programs [intro]:
  "dg_spec_wf S
     ⟹ ∀q ∈ set (map (routed_call_alternative_program S global_of seed route
             is_bot ctx ca cc p) pairs). sp_wf q"
  by (auto intro: sp_wf_routed_call_alternative_program)

lemma sp_wf_routed_callee_call_programs [intro]:
  "dg_spec_wf S
     ⟹ ∀q ∈ set (map (routed_callee_call_program S global_of seed route
             is_bot ctx ca cc caller) ps). sp_wf q"
  by (auto intro: sp_wf_routed_callee_call_program)
```

How a routed per-target program relates to its parts under a solution. Each bound is stated for *one* alternative of the list a single run of the entry produced, because the program is a fold over exactly those. These are the only facts CALL and COMB below need about its body.

```
lemma routed_callee_call_program_sides_ge_seed:
  assumes wfS: "dg_spec_wf S"
    and R: "enter_runs (enter♯ S (call_info_of (CallEdge dst pars args) p))
              (mk_dg_man caller global_of) σ pairs pub"
    and mem: "(cont, entry) ∈ set pairs"
    and nb: "¬ is_bot entry"
  shows "DG entry bot
         ≤ sides_of_program (routed_callee_call_program S global_of seed route is_bot ctx
              (CallEdge dst pars args) cc caller p) σ
              (Inr (seed (FunctionEntry p)
                 (route cc ctx entry (CallEdge dst pars args))))"
proof -
  let ?F = "routed_call_alternative_program S global_of seed route is_bot ctx
             (CallEdge dst pars args) cc p"
  let ?k = "Inr (seed (FunctionEntry p) (route cc ctx entry (CallEdge dst pars args)))"
  have "DG entry bot ≤ sides_of_program (?F (cont, entry)) σ ?k"
    unfolding routed_call_alternative_program_def using nb
    by (simp add: Let_def sp_compile_def)
  also have "… ≤ sides_of_program (side_rhs_fold_dg bot (map ?F pairs)) σ ?k"
    by (rule sides_le_side_rhs_fold_dg[OF sp_wf_routed_call_alternative_programs[OF wfS]])
       (use mem in force)
  also have "… ≤ sides_of_program (routed_callee_call_program S global_of seed route is_bot ctx
                   (CallEdge dst pars args) cc caller p) σ ?k"
    unfolding routed_callee_call_program_def
    by (simp add: sp_compile_bind sp_wf_observes enter_runsD_sides[OF R] sup_fun_def)
  finally show ?thesis .
qed
```

One rung further out: one resolved target's contribution survives the fold over the site's targets. Stated without

a locale so a proof that must place a publication inside the solution — one whose concretization reads the global channel — can walk outwards from a call site it has not yet routed.

```
lemma routed_call_program_sides_ge_at:
  assumes wfS: "dg_spec_wf S"
    and mem: "p ∈ set (resolve v cc ca (dg_local (σ (Inl (cc, ctx)))))"
  shows "sides_of_program (routed_callee_call_program S global_of seed route is_bot ctx ca cc
            (dg_local (σ (Inl (cc, ctx)))) p) σ z
          ≤ sides_of_program
              (routed_call_program S global_of seed resolve is_bot route ctx ca cc v) σ z"
proof -
  have "routed_callee_call_program S global_of seed route is_bot ctx ca cc
          (dg_local (σ (Inl (cc, ctx)))) p
          ∈ set (map (routed_callee_call_program S global_of seed route is_bot ctx ca cc
                       (dg_local (σ (Inl (cc, ctx)))))
                 (resolve v cc ca (dg_local (σ (Inl (cc, ctx))))))"
    using mem by simp
  from sides_le_side_rhs_fold_dg[OF sp_wf_routed_callee_call_programs[OF wfS] this,
       where acc = bot and k = z]
  show ?thesis
    by (simp add: routed_call_program_def sp_compile_bind sp_wf_observes)
qed

lemma routed_callee_call_program_sides_ge_combine:
  assumes wfS: "dg_spec_wf S"
    and R: "enter_runs (enter♯ S (call_info_of ca p))
              (mk_dg_man caller global_of) σ pairs pub"
    and mem: "(cont, entry) ∈ set pairs"
  shows "sides_of_program
           (routed_call_alternative_program S global_of seed route is_bot ctx ca cc p (cont, entry))
             σ z
           ≤ sides_of_program
              (routed_callee_call_program S global_of seed route is_bot ctx ca cc caller p) σ z"
proof -
  let ?F = "routed_call_alternative_program S global_of seed route is_bot ctx ca cc p"
  have "sides_of_program (?F (cont, entry)) σ z
          ≤ sides_of_program (side_rhs_fold_dg bot (map ?F pairs)) σ z"
    by (rule sides_le_side_rhs_fold_dg[OF sp_wf_routed_call_alternative_programs[OF wfS]])
       (use mem in force)
  also have "... ≤ sides_of_program
                     (routed_callee_call_program S global_of seed route is_bot ctx ca cc caller p) σ z"
    unfolding routed_callee_call_program_def
    by (simp add: sp_compile_bind sp_wf_observes enter_runsD_sides[OF R] sup_fun_def)
  finally show ?thesis .
qed
```

And the answer: the value a selected alternative's return combine produces is below what the call answers, because the call answers the join over every alternative. This is the bound COMB needs — one alternative's combine is covered, not all of them jointly. It holds for a bottom alternative too, whose combine runs against $\bot$.

```
lemma routed_callee_call_program_traverse_ge:
  assumes wfS: "dg_spec_wf S"
    and R: "enter_runs (enter♯ S (call_info_of ca p))
              (mk_dg_man caller global_of) σ pairs pub"
    and mem: "(cont, entry) ∈ set pairs"
  shows "dg_local (traverse_program
           (routed_call_alternative_program S global_of seed route is_bot ctx ca cc p (cont, entry))
             σ)
          ≤ dg_local (traverse_program
              (routed_callee_call_program S global_of seed route is_bot ctx ca cc caller p) σ)"
proof -
  let ?F = "routed_call_alternative_program S global_of seed route is_bot ctx ca cc p"
  have "dg_local (traverse_program (?F (cont, entry)) σ)
          ≤ side_acc_dg bot σ (map ?F pairs)"
    by (rule dg_local_traverse_le_side_acc_dg) (use mem in force)
  also have "... = dg_local (traverse_program
                     (routed_callee_call_program S global_of seed route is_bot ctx ca cc caller p) σ)"
    unfolding routed_callee_call_program_def
    by (simp add: sp_compile_bind sp_wf_observes enter_runsD_traverse[OF R]
        traverse_side_rhs_fold_dg[OF sp_wf_routed_call_alternative_programs[OF wfS]])
  finally show ?thesis .
qed
```

When a routed combine publishes nothing at the analysis global

Whether a program needs reshaping before the buffered generator can consume it is a property of that program, not of the analysis that produced it. The two facts below say when a routed combine needs none.

- Its answer always carries `bot` on the globals half: the per-call-site fold accumulates only `dg_local`.
- It publishes nothing at `buffer_key` as soon as the spec's own enter and combine sub-programs publish nothing there — the only side the routed program adds is the seed, and a seed key is never `buffer_key`.

  A program with both properties is already its own contribution analogue, so the buffered generator's `cmb_c` hook can be the program itself. A spec whose transfers do publish at `buffer_key` fails the second and must supply a reshaped hook instead.

```
lemma sides_side_rhs_fold_dg_bot:
  assumes wf: "∀t ∈ set ts. sp_wf t"
    and "⋀t. t ∈ set ts ⟹ sides_of_program t σ z = bot"
  shows "sides_of_program (side_rhs_fold_dg acc ts) σ z = bot"
  using assms
  by (simp add: sides_of_program_side_rhs_fold_dg_char[OF wf] foldr_sup_bot_of_all_bot)

lemma routed_call_program_global_free:
  assumes wfS: "dg_spec_wf S"
  shows "dg_global (traverse_program
          (routed_call_program S global_of seed resolve is_bot route ctx ca cc v) σ) = bot"
  by (simp add: routed_call_program_def sp_compile_bind sp_wf_observes
      traverse_side_rhs_fold_dg[OF sp_wf_routed_callee_call_programs[OF wfS]])
```

Every hook the routed generator builds from a well-formed specification answers nothing on its global half: an edge answers its transfer's local value, a call answers the folded return values, and a seed answers its local slot. So the buffered generator a solver runs is the unbuffered one the framework is stated over, at every specification, whether or not it publishes at the analysis global.

```
lemma dg_global_traverse_dg_spec_edge_program:
  assumes "dg_spec_wf S"
  shows "dg_global (traverse_program (dg_spec_edge_program S a src unknown_of) σ) = bot"
proof -
  let ?P = "transfer_program_at (λm. dg_spec_step S a (outer_man (dgs_query S) m)) src unknown_of"
  have wf: "sp_wf ?P"
    unfolding transfer_program_at_def
    by (intro sp_wf_bind sp_wf_dg_read_at) (rule dg_spec_wf_step[OF assms])
  have "?P (λv. Answer (DG v bot))
     = sp_lift_tree (?P (λv. Answer (DG v bot))) (λd. Answer (DG (dg_local d) bot))"
    using sp_wf_graft[OF wf, where k = "λv. Answer (DG v bot)"
        and h = "λd. Answer (DG (dg_local d) bot)"]
    by simp
  then have "traverse_rhs (?P (λv. Answer (DG v bot))) σ
     = DG (dg_local (traverse_rhs (?P (λv. Answer (DG v bot))) σ)) bot"
    by (metis (no_types, lifting) traverse_rhs.simps(1) traverse_rhs_sp_lift_tree)
  then show ?thesis
    by (simp add: dg_spec_edge_program_def transfer_program_def sp_compile_def
        sp_compile_with_def) (metis dg_state.sel(2))
qed

theorem pp_routed_of_buffered:
  assumes wfS: "dg_spec_wf S"
    and ne: "⋀p ctx. seed p ctx ≠ buffer_key"
    and pp: "part_post_solution
     (routed_node_rhs_buffered intra_predecessor_addr_list call_site_list (λ_. buffer_key) route
        (λctx' src a. dg_spec_edge_program S a src global_of)
        (routed_call_program S global_of seed resolve is_bot)
        (routed_entry_seed_programs seed)
        g bot0 d₀ g₀)
      x0 sigma vars"
  shows "part_post_solution
     (routed_node_rhs intra_predecessor_addr_list call_site_list (λ_. buffer_key) route
        (λctx' src a. dg_spec_edge_program S a src global_of)
        (routed_call_program S global_of seed resolve is_bot)
        (routed_entry_seed_programs seed)
        g bot0 d₀ g₀)
      x0 sigma vars"
proof (rule part_post_solution_routed_node_rhs_buffered_answers_local[OF _ _ pp])
  show "⋀c' w. ∀p ∈ set (routed_contribution_programs intra_predecessor_addr_list
          call_site_list route (λctx' src a. dg_spec_edge_program S a src global_of)
```

```
          (routed_call_program S global_of seed resolve is_bot)
          (routed_entry_seed_programs seed) g c' w). sp_wf p"
    by (rule routed_contribution_programs_wf)
       (auto intro: sp_wf_dg_spec_edge_program[OF wfS] sp_wf_routed_call_program[OF wfS])
  show "⋀c' w p τ. p ∈ set (routed_contribution_programs intra_predecessor_addr_list
          call_site_list route (λctx' src a. dg_spec_edge_program S a src global_of)
          (routed_call_program S global_of seed resolve is_bot)
          (routed_entry_seed_programs seed) g c' w)
        ⟹ dg_global (traverse_program p τ) = bot"
    by (auto elim!: routed_contribution_programsE
        simp: routed_call_program_global_free[OF wfS] routed_entry_seed_programs_local_only
          dg_global_traverse_dg_spec_edge_program[OF wfS])
qed
```

The entry half of the condition is now stated through `enter_runs`: an entry program publishes nothing at `buffer_key` exactly when the `pub` it runs to is $\bot$ there. It has to be stated this way: an entry program answers a list of alternatives, not the solver's value type, so it has no compiled tree of its own to observe.

```
lemma routed_callee_call_program_side_free_at_buffer_key:
  assumes wfS: "dg_spec_wf S"
    and enter_free: "⋀ci d pairs pub.
        enter_runs (enter♯ S ci) (mk_dg_man d global_of) σ pairs pub ⟹ pub (Inr buffer_key) = bot"
    and enter_runs_ex: "⋀ci d. ∃pairs pub.
        enter_runs (enter♯ S ci) (mk_dg_man d global_of) σ pairs pub"
    and comb_free: "⋀ci d de z. sides_of_rhs (sp_compile_with (λx. DG x bot)
        (dg_spec_combine_transfer S ci (mk_dg_man d global_of) de)) σ z = bot"
    and ne: "⋀p ctx'. seed (FunctionEntry p) ctx' ≠ buffer_key"
  shows "sides_of_program
          (routed_callee_call_program S global_of seed route is_bot ctx ca cc caller p) σ (Inr buffer_key)
         = bot"
proof -
  obtain pairs pub
    where R: "enter_runs (enter♯ S (call_info_of ca p))
                (mk_dg_man caller global_of) σ pairs pub"
    using enter_runs_ex by blast
  have alts: "⋀t. t ∈ set (map (routed_call_alternative_program S global_of seed route is_bot ctx
                                 ca cc p) pairs)
                ⟹ sides_of_program t σ (Inr buffer_key) = bot"
    by (auto simp: routed_call_alternative_program_def Let_def sp_compile_def
        comb_free ne[THEN not_sym] split: if_splits)
  have "sides_of_program
          (routed_callee_call_program S global_of seed route is_bot ctx ca cc caller p) σ (Inr buffer_key)
        = pub (Inr buffer_key)
          ⊔ sides_of_program (side_rhs_fold_dg bot
                (map (routed_call_alternative_program S global_of seed route is_bot ctx ca cc p)
                  pairs))
              σ (Inr buffer_key)"
    unfolding routed_callee_call_program_def
    by (simp add: sp_compile_bind sp_wf_observes enter_runsD_sides[OF R] sup_fun_def)
  also have "... = bot"
  proof -
    have "sides_of_program (side_rhs_fold_dg bot
            (map (routed_call_alternative_program S global_of seed route is_bot ctx ca cc p) pairs))
            σ (Inr buffer_key) = bot"
      by (rule sides_side_rhs_fold_dg_bot[OF sp_wf_routed_call_alternative_programs[OF wfS]])
         (rule alts)
    then show ?thesis using enter_free[OF R] by simp
  qed
  finally show ?thesis .
qed

lemma routed_call_program_side_free_at_buffer_key:
  assumes wfS: "dg_spec_wf S"
    and enter_free: "⋀ci d pairs pub.
        enter_runs (enter♯ S ci) (mk_dg_man d global_of) σ pairs pub ⟹ pub (Inr buffer_key) = bot"
    and enter_runs_ex: "⋀ci d. ∃pairs pub.
        enter_runs (enter♯ S ci) (mk_dg_man d global_of) σ pairs pub"
    and comb_free: "⋀ci d de z. sides_of_rhs (sp_compile_with (λx. DG x bot)
        (dg_spec_combine_transfer S ci (mk_dg_man d global_of) de)) σ z = bot"
    and ne: "⋀p ctx'. seed (FunctionEntry p) ctx' ≠ buffer_key"
  shows "sides_of_program (routed_call_program S global_of seed resolve is_bot route ctx ca cc v)
          σ (Inr buffer_key) = bot"
proof -
  have at_free: "⋀caller p.
      sides_of_program (routed_callee_call_program S global_of seed route is_bot ctx ca cc caller p)
        σ (Inr buffer_key) = bot"
    by (rule routed_callee_call_program_side_free_at_buffer_key[OF wfS])
```

```
      (rule enter_free enter_runs_ex comb_free ne, assumption?)+
  have "sides_of_program (side_rhs_fold_dg bot
         (map (routed_callee_call_program S global_of seed route is_bot ctx ca cc
                 (dg_local (σ (Inl (cc, ctx)))))
            (resolve v cc ca (dg_local (σ (Inl (cc, ctx)))))) σ (Inr buffer_key) = bot"
    by (rule sides_side_rhs_fold_dg_bot[OF sp_wf_routed_callee_call_programs[OF wfS]])
      (auto simp: at_free)
  then show ?thesis by (simp add: routed_call_program_def sp_compile_bind sp_wf_observes)
qed

end
```

6.15 Routed_Context

```
theory Routed_Context
  imports Routed_Call_Programs DG_Ctx_Activation "Voblint_Framework.MCP_Spec" Transfer_Algebra
    "Voblint_Domain.Nonrelational_Reachability"
    Activation_Backbone
begin
```

One route, CALL and COMB discharged once

`dg_context_activation` already discharges EDGE (`dg_ctx_act_edge`) generically off the post-solution, independent of `route`/`cmb`/`extra`: intra edges never route. COMB cannot be discharged that generically, because `cmb` is an unconstrained parameter: a context-sensitive analysis whose entry-seed publication and return combine are hand-written has to rederive the routing argument itself. This theory fixes `cmb` and `extra` to the one canonical shape `Routed_Call_Programs` builds — parametric only in a routing function `route` and a seed-key injection `seed` — and discharges CALL and COMB as theorems of that shape: a k-call-string or a partial-tabulation context becomes an interpretation of this locale, not a second proof development.

The routed-context locale: D and G independently typed

`routed_context` instantiates `dg_context_activation` at `routed_call_program`/`routed_entry_seed_programs`, so `S`'s own `'D`/`'G` stay as independent as that locale already keeps them: no `'D = 'G` constraint is threaded in by this locale's `for` clause. `dg_context_activation` itself carries no routing-specific content: every fact it supplies (`pp_eq_bound`, `pp_sides_bound`, `sides_fold_le_Gen`, `edge_bound_local`/`_global`, `dg_ctx_act_edge`) is already generic in `cmb`/`extra`, so instantiating it at `routed_call_program`/`routed_entry_seed_programs` reuses those proofs unchanged; only the seed-specific reasoning below, which `dg_context_activation` never has since seeding is `routed_call_program`'s own addition, is carried out here.

Beyond `dg_context_activation`'s parameters: `seed` injects a routed `(pp, 'c)` pair into the global-key space; `adm` is the trace-semantic context relation keying the activation-trace collecting semantics; and two obligations about it cannot be discharged generically. `routing_adequate` is adequacy: whenever `adm` admits a context for a real call edge and a covered caller store, some alternative of the specification's own entry run describes that call — its continuation half containing the caller store, its entry half containing the entered store, and its route being exactly the admitted context. The three come from one alternative rather than three, which is what keeps a multi-alternative entry from mixing one alternative's continuation with another's callee frame. `routing_total` is existence: every covered call admits some context. Restricting both to an edge of `g` (rather than quantifying over every value of the `call_action` type) matches how CALL and COMB below only ever invoke them at a matched edge, and keeps the obligations provable for an abstract domain that is exact on edges actually present in the program without being exact everywhere. These are per-instance proof obligations, not locale theorems.

`calls_unique` ties the edge a return reads to the edge its callee was entered through; the collecting semantics itself never needs it, and compiled programs have it for free.

```
locale routed_context_base =
  dg_context_base S γ_DG g buffer_key global_of route
    "routed_call_program S global_of seed resolve is_bot"
      "routed_entry_seed_programs seed"
    bot0 d₀ g₀ sigma vars x0 sg γ_M
  for S ::
    "(pp × 'c, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec"
    and γ_DG :: "'D ⇒ ('v ⇒ 'G) ⇒ store set"
    and g buffer_key and global_of :: "'v ⇒ 'k"
    and route ("context♯")
    and bot0 d₀ g₀ sigma vars x0 sg
    and seed :: "pp ⇒ 'c ⇒ 'k"
    and resolve :: "pp ⇒ pp ⇒ (var_id, exp) gcall_action ⇒ 'D ⇒ pname list"
    and is_bot :: "'D ⇒ bool"
    and γ_M :: "'M ⇒ store set" +
  fixes adm :: "'c call_context_policy"
  assumes finC: "finite (calls g)"
    and calls_unique: "calls_source_unique g"
    and seed_ne_buffer_key[simp]: "⋀p ctx. seed p ctx ≠ buffer_key"
    and seed_ne_global_of[simp]: "⋀p ctx v. seed p ctx ≠ global_of v"
    and is_bot_bot[simp]: "is_bot bot"
    and is_bot_sound: "⋀d g. is_bot d ⟹ γ_DG d g = {}"
    and resolve_sound:
    "⋀u ctx dst pars args p cont s.
       (u, ctx) ∈ vars
       ⟹ (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
       ⟹ s ∈ γ_DG (dg_local (sigma (Inl (u, ctx)))) (genv global_of sigma)
```

```
      ⟹ p ∈ set (resolve cont u (CallEdge dst pars args)
                    (dg_local (sigma (Inl (u, ctx)))))"
```

The routing obligations read the concrete call entry; everything else above is independent of the language.

```
locale routed_context =
  dg_context_activation κ S γ_DG g buffer_key global_of route
    "routed_call_program S global_of seed resolve is_bot"
      "routed_entry_seed_programs seed"
    bot0 d₀ g₀ sigma vars x0 sg γ_M +
  routed_context_base S γ_DG g buffer_key global_of route bot0 d₀ g₀ sigma vars x0 sg
    seed resolve is_bot γ_M adm
  for κ :: kenv
    and S ::
      "(pp × 'c, 'k, 'v, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec"
    and γ_DG g buffer_key global_of
    and route ("context♯")
    and bot0 d₀ g₀ sigma vars x0 sg seed resolve is_bot γ_M adm +
  assumes routing_adequate:
    "⋀u ctx dst pars args p cont s ctx'.
       (u, ctx) ∈ vars
       ⟹ (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
       ⟹ s ∈ γ_DG (dg_local (sigma (Inl (u, ctx)))) (genv global_of sigma)
       ⟹ ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) p) s
            (call_enter κ (CallEdge dst pars args) s)
       ⟹ ∃pairs pub deps cont' entry.
            enter_runs (enter♯ S (call_info_of (CallEdge dst pars args) p))
              (mk_dg_man (dg_local (sigma (Inl (u, ctx)))) global_of) sigma pairs pub
          ∧ enter_deps (enter♯ S (call_info_of (CallEdge dst pars args) p))
              (mk_dg_man (dg_local (sigma (Inl (u, ctx)))) global_of) sigma pairs deps
          ∧ (cont', entry) ∈ set pairs
          ∧ s ∈ γ_DG cont' (genv global_of sigma)
          ∧ call_enter κ (CallEdge dst pars args) s
              ∈ γ_DG entry (genv global_of sigma)
          ∧ context♯ u ctx entry (CallEdge dst pars args) = ctx'
          ∧ (FunctionEntry p, ctx') ∈ vars"
    and routing_total:
    "⋀u ctx dst pars args p cont s.
       (u, ctx) ∈ vars
       ⟹ (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
       ⟹ s ∈ γ_DG (dg_local (sigma (Inl (u, ctx)))) (genv global_of sigma)
       ⟹ adm u ctx (call_info_of (CallEdge dst pars args) p) s
                    (call_enter κ (CallEdge dst pars args) s) ≠ {}"
    and comb_fwd:
    "⋀cl c1 dst pars args p cont.
       (cl, c1) ∈ vars ⟹ (cl, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
       ⟹ (cont, c1) ∈ vars"
begin
```

The reader's claim at a key, and the call contexts `adm` admits. Input-only, like `gamma_at`, so interpreted facts keep printing the explicit terms.

```
abbreviation (input) cover :: "pp ⇒ 'c ⇒ store set" where
  "cover v ctx ≡ γ_M (sg (Inl (v, ctx)))"

abbreviation (input) admits where
  "admits ≡ admits_call_context κ g adm"
```

One place unpacks the call's entry protocol. `routing_adequate` is a nest of existentials over one entry run, and both CALL and COMB need the same witnesses: unpacking it separately in each would let them drift onto *different* alternatives of the same call, which is precisely the correlation the pair list exists to keep.

Invoke it with `obtain`; it is not an elimination rule for the classical reasoner.

```
lemma routed_enter_witness:
  assumes covV: "(u, ctx) ∈ vars"
    and ce: "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ gamma_at u ctx"
    and Rc: "ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) p) s
               (call_enter κ (CallEdge dst pars args) s)"
  obtains pairs pub deps cont' entry
    where "enter_runs (enter♯ S (call_info_of (CallEdge dst pars args) p))
             (man_at u ctx) sigma pairs pub"
      and "enter_deps (enter♯ S (call_info_of (CallEdge dst pars args) p))
             (man_at u ctx) sigma pairs deps"
      and "(cont', entry) ∈ set pairs"
      and "s ∈ γ_DG cont' (genv global_of sigma)"
```

```
    and "call_enter κ (CallEdge dst pars args) s
           ∈ γ_DG entry (genv global_of sigma)"
    and "context♯ u ctx entry (CallEdge dst pars args) = ctx'"
    and "(FunctionEntry p, ctx') ∈ vars"
  using routing_adequate[OF covV ce sin Rc] that by blast
```

The alternative the eliminator hands over cannot be bottom, and no case distinction is needed to see it: a bottom alternative concretizes to nothing, while this one demonstrably contains the store the callee actually starts from. A bottom alternative therefore never describes a real callee execution — it stays in the fold only because its combine stage may still have effects.

```
lemma entry_cover_not_bot:
  assumes "s' ∈ γ_DG entry gv"
  shows "¬ is_bot entry"
proof
  assume "is_bot entry"
  then have "γ_DG entry gv = {}" by (rule is_bot_sound)
  with assms show False by simp
qed

lemma le_dg_state_localD: "d ≤ d' ⟹ dg_local d ≤ dg_local d'"
  by (simp add: less_eq_dg_state_def)

lemma le_dg_state_globalD: "d ≤ d' ⟹ dg_global d ≤ dg_global d'"
  by (simp add: less_eq_dg_state_def)
```

Reaching a resolved callee's contribution

A call edge reaches the generated equation in two hops: the call site's own program is one of the node's contribs, and the resolved callee's program is one summand of that site's fold. The second hop needs a concrete store at the call site, since a resolver may drop a target the abstract caller state rules out; where a store exists, the edge's own callee survives resolution.

```
lemma resolved_site_mem:
  assumes ce: "(cc, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
  shows "routed_call_program S global_of seed resolve is_bot context♯ ctx
           (CallEdge dst pars args) cc cont
         ∈ set (contribs cont ctx)"
proof -
  have "(cc, CallEdge dst pars args) ∈ set (call_site_list g cont)"
    using ce by (auto simp: set_call_site_list[OF finC])
  then show ?thesis by (rule routed_contribution_programs_combineI)
qed

lemma resolved_target_mem:
  assumes covV: "(cc, ctx) ∈ vars"
    and ce: "(cc, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ gamma_at cc ctx"
  shows "routed_callee_call_program S global_of seed context♯ is_bot ctx (CallEdge dst pars args) cc
           (dg_local (sigma (Inl (cc, ctx)))) p
         ∈ set (map (routed_callee_call_program S global_of seed context♯ is_bot ctx
                       (CallEdge dst pars args) cc
                     (dg_local (sigma (Inl (cc, ctx)))))
                    (resolve cont cc (CallEdge dst pars args)
                       (dg_local (sigma (Inl (cc, ctx))))))"
  using resolve_sound[OF covV ce sin] by simp

lemma resolved_at_le_site_acc:
  assumes covV: "(cc, ctx) ∈ vars"
    and ce: "(cc, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ gamma_at cc ctx"
  shows "dg_local (traverse_program
           (routed_callee_call_program S global_of seed context♯ is_bot ctx (CallEdge dst pars args) cc
              (dg_local (sigma (Inl (cc, ctx)))) p) sigma)
       ≤ side_acc_dg (acc0 cont) sigma (contribs cont ctx)"
proof -
  have "dg_local (traverse_program
           (routed_callee_call_program S global_of seed context♯ is_bot ctx (CallEdge dst pars args) cc
              (dg_local (sigma (Inl (cc, ctx)))) p) sigma)
     ≤ dg_local (traverse_program
           (routed_call_program S global_of seed resolve is_bot context♯ ctx
              (CallEdge dst pars args) cc cont)
           sigma)"
    using dg_local_traverse_le_side_acc_dg[OF resolved_target_mem[OF covV ce sin], where acc = bot]
    by (simp add: routed_call_program_def sp_compile_bind sp_wf_observes
```

```
      traverse_side_rhs_fold_dg[OF sp_wf_routed_callee_call_programs[OF spec_wf]])
  also have "... ≤ side_acc_dg (acc0 cont) sigma (contribs cont ctx)"
    by (rule dg_local_traverse_le_side_acc_dg[OF resolved_site_mem[OF ce]])
  finally show ?thesis .
qed

lemma resolved_at_le_site_sides:
  assumes covV: "(cc, ctx) ∈ vars"
    and ce: "(cc, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ gamma_at cc ctx"
  shows "sides_of_program
           (routed_callee_call_program S global_of seed context♯ is_bot ctx (CallEdge dst pars args) cc
              (dg_local (sigma (Inl (cc, ctx)))) p) sigma z
         ≤ sides_of_program (side_rhs_fold_dg (acc0 cont) (contribs cont ctx)) sigma z"
proof -
  have "sides_of_program
           (routed_callee_call_program S global_of seed context♯ is_bot ctx (CallEdge dst pars args) cc
              (dg_local (sigma (Inl (cc, ctx)))) p) sigma z
      ≤ sides_of_program
           (routed_call_program S global_of seed resolve is_bot context♯ ctx
              (CallEdge dst pars args) cc cont)
           sigma z"
    by (rule routed_call_program_sides_ge_at
          [where resolve = resolve and v = cont and cc = cc and ctx = ctx and σ = sigma,
           OF spec_wf resolve_sound[OF covV ce sin]])
  also have "... ≤ sides_of_program (side_rhs_fold_dg (acc0 cont) (contribs cont ctx)) sigma z"
    by (rule sides_le_side_rhs_fold_dg[OF contribs_wf resolved_site_mem[OF ce]])
  finally show ?thesis .
qed
```

CALL: the routed callee entry

The CALL obligation, ending in `routed_context_call`: the store a call hands the callee is covered by the callee entry read at the routed context. The bound does not go there directly. It goes through the seed unknown, which is the only place the caller's own program and the callee's entry equation meet.

```
lemma routed_seed_read_bound:
  assumes covV: "(FunctionEntry p, ctx') ∈ vars"
  shows "dg_local (sigma (Inr (seed (FunctionEntry p) ctx')))
           ≤ dg_local (sigma (Inl (FunctionEntry p, ctx')))"
proof -
  let ?t = "do { s ← sp_read_global (seed (FunctionEntry p) ctx');
                 sp_return (DG (dg_local s) bot) }"
  have mem: "?t ∈ set (contribs (FunctionEntry p) ctx')"
    by (rule routed_contribution_programs_extraI) (simp add: routed_entry_seed_programs_def)
  have "dg_local (sigma (Inr (seed (FunctionEntry p) ctx')))
      = dg_local (traverse_program ?t sigma)"
    by (simp add: sp_compile_def)
  also have "... ≤ side_acc_dg (acc0 (FunctionEntry p)) sigma (contribs (FunctionEntry p) ctx')"
    using dg_local_traverse_le_side_acc_dg[OF mem] .
  also have "... = dg_local (eq Gen (FunctionEntry p, ctx') sigma)"
    by (simp add: eq_routed_node_rhs[OF contribs_wf])
  also have "... ≤ dg_local (sigma (Inl (FunctionEntry p, ctx')))"
    using pp_eq_bound[OF covV] by (simp add: less_eq_dg_state_def)
  finally show ?thesis .
qed
```

The published callee-entry state is bounded by the seed unknown itself, and that bound needs only the continuation's coverage: the seed is written by the combine program living at the continuation, so nothing about the callee entry's own unknown enters the argument. `routed_seed_publish_bound_local` adds the one further hop from the seed to the callee-entry local, which is where the callee's own entry equation reads it back, and that hop is what needs the callee entry covered.

```
lemma routed_seed_publish_bound_seed:
  assumes covV: "(u, ctx) ∈ vars"
    and ce: "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ gamma_at u ctx"
    and covV_cont: "(cont, ctx) ∈ vars"
    and R: "enter_runs (enter♯ S (call_info_of (CallEdge dst pars args) p))
              (man_at u ctx) sigma pairs pub"
    and mem: "(cont', entry) ∈ set pairs"
    and nb: "¬ is_bot entry"
  shows "entry
         ≤ dg_local (sigma (Inr (seed (FunctionEntry p)
               (context♯ u ctx entry (CallEdge dst pars args)))))"
```

```
proof -
  let ?ctx' = "context♯ u ctx entry (CallEdge dst pars args)"
  let ?k = "Inr (seed (FunctionEntry p) ?ctx')"
  let ?t = "routed_callee_call_program S global_of seed context♯ is_bot ctx (CallEdge dst pars args) u
             (dg_local (sigma (Inl (u, ctx)))) p"
  have seedb: "DG entry bot ≤ sides_of_program ?t sigma ?k"
    using spec_wf R mem nb by (rule routed_callee_call_program_sides_ge_seed)
  have "entry ≤ dg_local (sides_of_program ?t sigma ?k)"
    using le_dg_state_localD[OF seedb] by simp
  also have "... ≤ dg_local (sides_of_program
     (side_rhs_fold_dg (acc0 cont) (contribs cont ctx)) sigma ?k)"
    by (rule le_dg_state_localD[OF resolved_at_le_site_sides[OF covV ce sin]])
  also have "... ≤ dg_local (sides_of_rhs (Gen (cont, ctx)) sigma ?k)"
    by (rule le_dg_state_localD[OF sides_fold_le_Gen])
  also have "... ≤ dg_local (sigma ?k)"
    using pp_sides_bound[OF covV_cont, THEN le_funD, of ?k]
    by (rule le_dg_state_localD)
  finally show ?thesis .
qed

lemma routed_seed_publish_bound_local:
  assumes covV_call: "(u, ctx) ∈ vars"
    and ce: "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ gamma_at u ctx"
    and covV_cont: "(cont, ctx) ∈ vars"
    and R: "enter_runs (enter♯ S (call_info_of (CallEdge dst pars args) p))
             (man_at u ctx) sigma pairs pub"
    and mem: "(cont', entry) ∈ set pairs"
    and nb: "¬ is_bot entry"
    and covV: "(FunctionEntry p, context♯ u ctx entry (CallEdge dst pars args)) ∈ vars"
  shows "entry
       ≤ dg_local (sigma (Inl (FunctionEntry p,
            context♯ u ctx entry (CallEdge dst pars args))))"
  by (rule order_trans
       [OF routed_seed_publish_bound_seed[OF covV_call ce sin covV_cont R mem nb]
           routed_seed_read_bound[OF covV]])

theorem routed_context_call:
  assumes ce: "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ cover u ctx"
    and Rc: "ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) p) s
             (call_enter κ (CallEdge dst pars args) s)"
  shows "call_enter κ (CallEdge dst pars args) s ∈ cover (FunctionEntry p) ctx'"
proof (cases "(u, ctx) ∈ vars")
  case False
  hence "cover u ctx = {}" by (rule sg_uncov)
  thus ?thesis using sin by simp
next
  case True
  let ?g = "genv global_of sigma"
  have covV_cont: "(cont, ctx) ∈ vars"
    using comb_fwd[OF True ce] .
  have sin': "s ∈ γ_DG (dg_local (sigma (Inl (u, ctx)))) ?g"
    using sin True by simp
  obtain pairs pub deps cont' entry
    where Rr: "enter_runs (enter♯ S (call_info_of (CallEdge dst pars args) p))
                (man_at u ctx) sigma pairs pub"
      and D: "enter_deps (enter♯ S (call_info_of (CallEdge dst pars args) p))
                (man_at u ctx) sigma pairs deps"
      and mem: "(cont', entry) ∈ set pairs"
      and ecov: "call_enter κ (CallEdge dst pars args) s ∈ γ_DG entry ?g"
      and req: "context♯ u ctx entry (CallEdge dst pars args) = ctx'"
      and covV: "(FunctionEntry p, ctx') ∈ vars"
    by (rule routed_enter_witness[OF True ce sin' Rc])
  let ?ctx' = "context♯ u ctx entry (CallEdge dst pars args)"
  have nb: "¬ is_bot entry" using ecov by (rule entry_cover_not_bot)
  have "γ_DG entry ?g ⊆ γ_DG (dg_local (sigma (Inl (FunctionEntry p, ?ctx')))) ?g"
    by (rule gammaDG_mono
          [OF routed_seed_publish_bound_local[OF True ce sin' covV_cont Rr mem nb covV[folded req]]
              order_refl])
  also have "... = cover (FunctionEntry p) ?ctx'"
    using covV[folded req] by simp
  finally show ?thesis using ecov req by auto
qed
```

COMB: the routed return combine

The COMB obligation, ending in `routed_context_comb`: the state a return combines is covered by the continuation read at the caller's own context. The return takes one alternative among those the enter step produced, so the work is to bound that single alternative – once locally, once at the global key – against what the continuation's unknown already holds.

```
lemma routed_comb_bound_local:
  assumes covV: "(cl, c1) ∈ vars"
    and ce: "(cl, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ gamma_at cl c1"
    and covV_cont: "(cont, c1) ∈ vars"
    and R: "enter_runs (enter♯ S (call_info_of (CallEdge dst pars args) p))
              (man_at cl c1) sigma pairs pub"
    and mem: "(cont', entry) ∈ set pairs"
  shows "dg_local (traverse_program
           (routed_call_alternative_program S global_of seed context♯ is_bot c1
              (CallEdge dst pars args) cl p (cont', entry)) sigma)
         ≤ dg_local (sigma (Inl (cont, c1)))"
proof -
  have "dg_local (traverse_program
           (routed_call_alternative_program S global_of seed context♯ is_bot c1
              (CallEdge dst pars args) cl p (cont', entry)) sigma)
      ≤ dg_local (traverse_program
           (routed_callee_call_program S global_of seed context♯ is_bot c1 (CallEdge dst pars args) cl
              (dg_local (sigma (Inl (cl, c1)))) p) sigma)"
    by (rule routed_callee_call_program_traverse_ge[OF spec_wf R mem])
  also have "... ≤ side_acc_dg (acc0 cont) sigma (contribs cont c1)"
    by (rule resolved_at_le_site_acc[OF covV ce sin])
  also have "... = dg_local (eq Gen (cont, c1) sigma)"
    by (simp add: eq_routed_node_rhs[OF contribs_wf])
  also have "... ≤ dg_local (sigma (Inl (cont, c1)))"
    using pp_eq_bound[OF covV_cont] by (simp add: less_eq_dg_state_def)
  finally show ?thesis .
qed

lemma routed_comb_bound_global:
  assumes covV: "(cl, c1) ∈ vars"
    and ce: "(cl, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ gamma_at cl c1"
    and covV_cont: "(cont, c1) ∈ vars"
    and R: "enter_runs (enter♯ S (call_info_of (CallEdge dst pars args) p))
              (man_at cl c1) sigma pairs pub"
    and mem: "(cont', entry) ∈ set pairs"
  shows "dg_global (sides_of_program
           (routed_call_alternative_program S global_of seed context♯ is_bot c1
              (CallEdge dst pars args) cl p (cont', entry)) sigma (Inr k))
         ≤ dg_global (sigma (Inr k))"
proof -
  have "dg_global (sides_of_program
           (routed_call_alternative_program S global_of seed context♯ is_bot c1
              (CallEdge dst pars args) cl p (cont', entry)) sigma (Inr k))
      ≤ dg_global (sides_of_program
           (routed_callee_call_program S global_of seed context♯ is_bot c1 (CallEdge dst pars args) cl
              (dg_local (sigma (Inl (cl, c1)))) p) sigma (Inr k))"
    by (rule le_dg_state_globalD[OF routed_callee_call_program_sides_ge_combine[OF spec_wf R mem]])
  also have "... ≤ dg_global (sides_of_program
      (side_rhs_fold_dg (acc0 cont) (contribs cont c1)) sigma (Inr k))"
    by (rule le_dg_state_globalD[OF resolved_at_le_site_sides[OF covV ce sin]])
  also have "... ≤ dg_global (sides_of_rhs (Gen (cont, c1)) sigma (Inr k))"
    by (rule le_dg_state_globalD[OF sides_fold_le_Gen])
  also have "... ≤ dg_global (sigma (Inr k))"
    using pp_sides_bound[OF covV_cont, THEN le_funD, of "Inr k"]
    by (rule le_dg_state_globalD)
  finally show ?thesis .
qed

lemma routed_comb_bound_genv:
  assumes covV: "(cl, c1) ∈ vars"
    and ce: "(cl, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ gamma_at cl c1"
    and covV_cont: "(cont, c1) ∈ vars"
    and R: "enter_runs (enter♯ S (call_info_of (CallEdge dst pars args) p))
              (man_at cl c1) sigma pairs pub"
    and mem: "(cont', entry) ∈ set pairs"
  shows "genv global_of (sides_of_program
```

```
          (routed_call_alternative_program S global_of seed context♯ is_bot c1
             (CallEdge dst pars args) cl p (cont', entry)) sigma)
        ≤ genv global_of sigma"
  unfolding le_fun_def genv_def
  using routed_comb_bound_global[OF covV ce sin covV_cont R mem] by blast

theorem routed_context_comb:
  assumes ce: "(cl, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and s: "s ∈ cover cl c1"
    and adm: "admits cl c1 p' s es ctx'"
    and t: "t ∈ cover (FunctionResult p) ctx'"
  shows "combine_collect κ dst s t ∈ cover cont c1"
proof (cases "(cl, c1) ∈ vars")
  case False
  hence "cover cl c1 = {}" by (rule sg_uncov)
  thus ?thesis using s by simp
next
  case True
  let ?g = "genv global_of sigma"
  let ?ci = "call_info_of (CallEdge dst pars args) p"
  have sin: "s ∈ γ_DG (dg_local (sigma (Inl (cl, c1)))) ?g"
    using s True by simp
  — The callee was entered along some edge out of cl; call-source uniqueness makes it the edge the return reads.
  from adm obtain dst2 pars2 args2 cont2
    where e2: "(cl, CallEdge dst2 pars2 args2, FunctionEntry p', cont2) ∈ calls g"
      and es_eq: "es = call_enter κ (CallEdge dst2 pars2 args2) s"
      and Rc2: "ctx' ∈ adm cl c1 (call_info_of (CallEdge dst2 pars2 args2) p') s es"
    by (rule admits_call_contextE)
  have same: "CallEdge dst2 pars2 args2 = CallEdge dst pars args" and pp': "p' = p"
    using calls_source_unique_edgesD(1,2)[OF calls_unique e2 ce] by simp_all
  have Rc: "ctx' ∈ adm cl c1 ?ci s (call_enter κ (CallEdge dst pars args) s)"
    using Rc2 unfolding es_eq same pp' .
  obtain pairs pub deps cont' entry
    where Rr: "enter_runs (enter♯ S ?ci)
               (man_at cl c1) sigma pairs pub"
      and D: "enter_deps (enter♯ S ?ci)
               (man_at cl c1) sigma pairs deps"
      and mem: "(cont', entry) ∈ set pairs"
      and ccov: "s ∈ γ_DG cont' ?g"
      and ecov: "call_enter κ (CallEdge dst pars args) s ∈ γ_DG entry ?g"
      and req: "context♯ cl c1 entry (CallEdge dst pars args) = ctx'"
    by (rule routed_enter_witness[OF True ce sin Rc])
  let ?ex_ctx = "context♯ cl c1 entry (CallEdge dst pars args)"
  let ?alt = "routed_call_alternative_program S global_of seed context♯ is_bot c1
               (CallEdge dst pars args) cl p (cont', entry)"
  have nb: "¬ is_bot entry" using ecov by (rule entry_cover_not_bot)
  show ?thesis
  proof (cases "(FunctionResult p, ?ex_ctx) ∈ vars")
    case False
    hence "cover (FunctionResult p) ?ex_ctx = {}" by (rule sg_uncov)
    with req have "cover (FunctionResult p) ctx' = {}" by simp
    with t show ?thesis by simp
  next
    case True
    have covV_cont: "(cont, c1) ∈ vars"
      using comb_fwd[OF ‹(cl, c1) ∈ vars› ce] .
    have tin: "t ∈ γ_DG (dg_local (sigma (Inl (FunctionResult p, ?ex_ctx)))) ?g"
      using t req True by simp
    let ?sub = "sp_compile_with (λd. DG d bot)
                  (dg_spec_combine_transfer S ?ci (mk_dg_man cont' global_of)
                    (dg_local (sigma (Inl (FunctionResult p, ?ex_ctx)))))"
    have tr: "traverse_program ?alt sigma = traverse_rhs ?sub sigma"
      using nb by (simp add: sp_compile_def)
    have sd:
      "genv global_of (sides_of_program ?alt sigma) = genv global_of (sides_of_rhs ?sub sigma)"
    proof (rule ext)
      fix w
      have knk: "(Inr (global_of w) :: (pp × 'c) + 'k) ≠ Inr (seed (FunctionEntry p) ?ex_ctx)"
        by (rule not_sym) simp
      show "genv global_of (sides_of_program ?alt sigma) w = genv global_of (sides_of_rhs ?sub sigma) w"
        using nb
        by (simp add: genv_def Let_def sp_compile_def fun_upd_other[OF knk] del: fun_upd_apply)
    qed
    have "combine_collect κ dst s t
       ∈ γ_DG (dg_local (traverse_rhs ?sub sigma)) (?g ⊔ genv global_of (sides_of_rhs ?sub sigma))"
      using combine_sound[where q = cont'
```

```
        and r = "dg_local (sigma (Inl (FunctionResult p, ?ex_ctx)))"
        and τ = sigma and key = global_of and ci = ?ci, OF ccov tin]
    by simp
  also have "... = γ_DG (dg_local (traverse_program ?alt sigma))
                        (?g ⊔ genv global_of (sides_of_program ?alt sigma))"
    by (simp only: tr sd)
  also have "... ⊆ γ_DG (dg_local (sigma (Inl (cont, c1)))) ?g"
    by (rule gammaDG_mono[OF
          routed_comb_bound_local[OF ‹(cl, c1) ∈ vars› ce sin covV_cont Rr mem]
          sup_least[OF order_refl
            routed_comb_bound_genv[OF ‹(cl, c1) ∈ vars› ce sin covV_cont Rr mem]]])
  also have "... = cover cont c1"
    using covV_cont by simp
  finally show ?thesis .
 qed
qed
```

Activation-collect soundness against the routed local unknown

Every activation-collected store at any `(v, ctx)` pair is concretized by the routed local unknown's own γ_M reading. The five obligations of `activation_coverage` are this locale's own facts: the two entry bounds discharge `INIT` together, `dg_ctx_act_edge` is `INTRA`, the CALL and COMB theorems above are `CALL` and `RETURN`, and `routing_total` read through the reader is `TOTAL`. An instance therefore gets its activation-indexed soundness theorem by interpretation alone.

```
lemma routed_call_context_total:
  "call_context_total_on κ cover adm g"
  unfolding call_context_total_on_def
proof (intro allI impI)
  fix u dst pars args p cont ctx s
  assume ce: "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sin: "s ∈ cover u ctx"
  have covV: "(u, ctx) ∈ vars"
  proof (rule ccontr)
    assume "(u, ctx) ∉ vars"
    then have "cover u ctx = {}" by (rule sg_uncov)
    with sin show False by simp
  qed
  with sin have "s ∈ gamma_at u ctx" by simp
  from routing_total[OF covV ce this]
  show "adm u ctx (call_info_of (CallEdge dst pars args) p) s
                (call_enter κ (CallEdge dst pars args) s) ≠ {}" .
qed

theorem activation_collect_dg_sound:
  fixes S0 :: "store set" and c₀ :: 'c
  assumes entry_cov: "(cfg_entry g, c₀) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d₀ e₀"
    and init_env_le: "e₀ ≤ genv global_of sigma"
  shows "A_κ,adm,c₀,g,S0 v c ⊆ cover v c"
proof (rule activation_collect_sound[where cover = "cover"], unfold_locales)
  fix s0 assume s0mem: "s0 ∈ S0"
  have le_local: "d₀ ≤ dg_local (sigma (Inl (cfg_entry g, c₀)))"
    by (rule pp_entry_init_local_bound[OF entry_cov])
  have "γ_DG d₀ e₀
        ⊆ gamma_at (cfg_entry g) c₀"
    by (rule gammaDG_mono[OF le_local init_env_le])
  with s0mem init_sound have "s0 ∈ gamma_at (cfg_entry g) c₀" by blast
  thus "s0 ∈ cover (cfg_entry g) c₀"
    using entry_cov by simp
next
  fix u a v' c' s' s''
  assume "(u, a, v') ∈ intra g" "s' ∈ cover u c'" "s'' ∈ edge_step κ a s'"
  thus "s'' ∈ cover v' c'" by (rule dg_ctx_act_edge)
next
  fix u dst pars args p cont c' c'' s'
  assume ce: "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sm: "s' ∈ cover u c'"
    and Rc: "c'' ∈ adm u c' (call_info_of (CallEdge dst pars args) p) s'
               (call_enter κ (CallEdge dst pars args) s')"
  show "call_enter κ (CallEdge dst pars args) s' ∈ cover (FunctionEntry p) c''"
    using routed_context_call[OF ce sm Rc] .
next
  fix cl dst pars args p cont c1 c' p' s' t es
  assume ce: "(cl, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
    and sm: "s' ∈ cover cl c1"
```

```
    and adm: "admits cl c1 p' s' es c'"
    and tm: "t ∈ cover (FunctionResult p) c'"
  show "combine_collect κ dst s' t ∈ cover cont c1"
    using routed_context_comb[OF ce sm adm tm] .
next
  show "call_context_total_on κ cover adm g"
    by (rule routed_call_context_total)
qed
```

Every valid activation trace of a covered program carries some context under this instance's own `adm`: the same four EDGE/CALL/COMB/TOTAL facts that bound the buckets above also make `activation_coverage` total here, so a caller can discharge the `has_ctx` premise `source_store_in_activation_collect` asks for without restating the interpretation itself.

```
lemma routed_valid_activation_trace_has_context:
  fixes S0 :: "store set" and c₀ :: 'c
  assumes entry_cov: "(cfg_entry g, c₀) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d₀ e₀"
    and init_env_le: "e₀ ≤ genv global_of sigma"
    and tv: "t ∈ 𝒯_κ,g,S0"
  shows "∃c. activation_context_rel κ adm c₀ g t c"
proof -
  interpret G: activation_coverage κ g S0 cover adm c₀
  proof unfold_locales
    fix s0 assume s0mem: "s0 ∈ S0"
    have le_local: "d₀ ≤ dg_local (sigma (Inl (cfg_entry g, c₀)))"
      by (rule pp_entry_init_local_bound[OF entry_cov])
    have "γ_DG d₀ e₀
          ⊆ gamma_at (cfg_entry g) c₀"
      by (rule gammaDG_mono[OF le_local init_env_le])
    with s0mem init_sound have "s0 ∈ gamma_at (cfg_entry g) c₀" by blast
    thus "s0 ∈ cover (cfg_entry g) c₀"
      using entry_cov by simp
  next
    fix u a v' c' s' s''
    assume "(u, a, v') ∈ intra g" "s' ∈ cover u c'" "s'' ∈ edge_step κ a s'"
    thus "s'' ∈ cover v' c'" by (rule dg_ctx_act_edge)
  next
    fix u dst pars args p cont c' c'' s'
    assume ce: "(u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
      and sm: "s' ∈ cover u c'"
      and Rc: "c'' ∈ adm u c' (call_info_of (CallEdge dst pars args) p) s'
                 (call_enter κ (CallEdge dst pars args) s')"
    show "call_enter κ (CallEdge dst pars args) s' ∈ cover (FunctionEntry p) c''"
      using routed_context_call[OF ce sm Rc] .
  next
    fix cl dst pars args p cont c1 c' p' s' t es
    assume ce: "(cl, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g"
      and sm: "s' ∈ cover cl c1"
      and adm: "admits cl c1 p' s' es c'"
      and tm: "t ∈ cover (FunctionResult p) c'"
    show "combine_collect κ dst s' t ∈ cover cont c1"
      using routed_context_comb[OF ce sm adm tm] .
  next
    show "call_context_total_on κ cover adm g"
      by (rule routed_call_context_total)
  qed
  show ?thesis using G.valid_activation_trace_has_context[OF tv] by blast
qed

end
```

The context relation a solved entry-state table induces

A state-dependent policy picks a context from an abstract entry state, and a pure entry answers a list of them, so one concrete call may be routed to several contexts — one per alternative whose continuation covers the caller store and whose entry covers the entered store. `routed_entry_context_rel alts` is exactly that set, for an entry operation whose alternatives are the pure function `alts` of the call information and the caller's solved local value: it admits `ctx'` whenever some alternative has those two coverings and its `route` is `ctx'`. Stated as an equivalence, both routed obligations come for free: adequacy is the definition read forwards, and totality is the alternatives covering the concrete call at all, which is what `entry_pairs_cover` says. No decoder of the concrete store is involved: the context is induced by the analyzer's own decision, as Goblint's `context` applied to each `enter` alternative induces it.

The relation is stated over `alts` rather than over an arbitrary entry run because a run does not determine its alternative list in general — over a one-point carrier every list traverses alike — while a pure entry's list is the function's own answer.

```
definition routed_entry_context_rel ::
  "(call_info ⇒ 'D ⇒ 'D enter_result list)
     ⇒ ('D ⇒ ('v ⇒ 'G) ⇒ store set) ⇒ (pp × 'c + 'k ⇒ ('D, 'G) dg_state) ⇒ ('v ⇒ 'k)
     ⇒ (pp ⇒ 'c ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'c) ⇒ 'c call_context_policy"
where
  "routed_entry_context_rel alts γ_DG sigma global_of route u ctx ci caller entered =
     {ctx'. ∃cont entry.
        (cont, entry) ∈ set (alts ci (dg_local (sigma (Inl (u, ctx)))))
      ∧ caller ∈ γ_DG cont (genv global_of sigma)
      ∧ entered ∈ γ_DG entry (genv global_of sigma)
      ∧ ctx' = route u ctx entry (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci))}"
```

Invoke with `obtain`: the alternative is a witness the classical reasoner should not be left to guess.

```
lemma routed_entry_context_relE:
  assumes
    "ctx' ∈ routed_entry_context_rel alts γ_DG sigma global_of route u ctx ci caller entered"
  obtains cont entry
    where "(cont, entry) ∈ set (alts ci (dg_local (sigma (Inl (u, ctx)))))"
      and "caller ∈ γ_DG cont (genv global_of sigma)"
      and "entered ∈ γ_DG entry (genv global_of sigma)"
      and "ctx' = route u ctx entry (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci))"
  using assms unfolding routed_entry_context_rel_def by blast
```

Totality is the alternatives covering the concrete call: exactly the `entry_pairs_cover` obligation every routed instance already carries.

```
lemma routed_entry_context_rel_total:
  assumes "entry_pairs_cover (λd. γ_DG d (genv global_of sigma)) caller entered
             (alts ci (dg_local (sigma (Inl (u, ctx)))))"
  shows
    "routed_entry_context_rel alts γ_DG sigma global_of route u ctx ci caller entered ≠ {}"
  using assms unfolding routed_entry_context_rel_def
  by blast

end
```

6.16 DG_Keyed_Split_Spec

```
theory DG_Keyed_Split_Spec
  imports "Voblint_Framework.MCP_Spec" Transfer_Algebra "Voblint_Domain.Nonrelational_Reachability"
    "Voblint_Solver.Strategy_Tree_Side_Buffering"
    "Voblint_Solver.Strategy_Program_Fold"
begin
```

One solver unknown per global name

Keeping every program global in one shared value makes reading a global depend on all of them, and writing one wakes every reader of any other. Goblint keys its globals by name instead, and this theory does the same for a whole-state component.

A global name `x` lives at its own unknown. Its value there is a global half that describes `x` alone, cut out of a whole state by `rg x`. A transfer reads only the names its edge mentions and publishes only the names its edge can write. Every other global it hands the wrapped transfer as `free`, a global half that claims nothing about the unread names, so the wrapped transfer stays sound on it. Its result says nothing useful about the unread names either, and it is not published: the next point reads them from their own unknowns again.

That last step is sound because the edge leaves the unpublished globals of a concrete store unchanged. Which names an edge may change is a fact of the concrete semantics (`edge_writes`), established in `CFG_Transfer`. Which names it reads only affects precision: a name left out is read as `free`.

Reading and publishing named globals

Reading a list of names folds their global halves into an accumulator, each value cut to the name it was read at. Publishing a list of names sends each its own cut of one whole state. The value-level counterparts are `view_of` and `pub_sides`.

```
fun read_view ::
  "('x, 'k, 'v, 'd::semilattice_sup, 'd) man ⇒ ('v ⇒ 'd ⇒ 'd) ⇒ 'v list ⇒ 'd
   ⇒ ('x,'k,('d,'d) dg_state,'d) strategy_program"
where
  "read_view m rg [] acc = sp_return acc"
| "read_view m rg (x # xs) acc = man_global m x ≫= (λg. read_view m rg xs (rg x g ⊔ acc))"

fun publish_at ::
  "('x, 'k, 'v, 'd, 'd) man ⇒ ('v ⇒ 'd ⇒ 'd) ⇒ 'v list ⇒ 'd
   ⇒ ('x,'k,('d,'d) dg_state,unit) strategy_program"
where
  "publish_at m rg [] r = sp_return ()"
| "publish_at m rg (x # xs) r = man_sideg m x (rg x r) ≫= (λ_. publish_at m rg xs r)"

definition view_of :: "('v ⇒ 'd ⇒ 'd) ⇒ 'v list ⇒ ('v ⇒ 'd) ⇒ 'd
    ⇒ 'd::semilattice_sup" where
  "view_of rg xs e acc = fold (λx acc. rg x (e x) ⊔ acc) xs acc"
```

The global half a whole environment describes, over a list of names.

```
definition full_view :: "('v ⇒ 'd ⇒ 'd) ⇒ 'v list ⇒ ('v ⇒ 'd)
    ⇒ 'd::bounded_semilattice_sup_bot" where
  "full_view rg xs e = view_of rg xs e bot"

definition pub_sides :: "('v ⇒ 'k) ⇒ ('v ⇒ 'd ⇒ 'd) ⇒ 'v list ⇒ 'd
    ⇒ 'x + 'k ⇒ ('d::bounded_semilattice_sup_bot, 'd) dg_state" where
  "pub_sides key rg xs r = foldr (λx acc. acc ⊔ bot(Inr (key x) := DG bot (rg x r))) xs bot"

lemma view_of_ge_acc: "acc ≤ view_of rg xs e acc"
  unfolding view_of_def
  by (induction xs arbitrary: acc) (auto intro: order_trans[OF sup_ge2])

lemma view_of_ge_mem: "x ∈ set xs ⟹ rg x (e x) ≤ view_of rg xs e acc"
  unfolding view_of_def
proof (induction xs arbitrary: acc)
  case Nil then show ?case by simp
next
  case (Cons y ys)
  show ?case
  proof (cases "x = y")
    case True
    then show ?thesis
      using view_of_ge_acc[of "rg y (e y) ⊔ acc" rg ys e]
      by (simp add: view_of_def)
```

```
  next
    case False
    with Cons show ?thesis by simp
  qed
qed

lemma view_of_le:
  assumes "acc ≤ v" and "⋀x. x ∈ set xs ⟹ rg x (e x) ≤ v"
  shows "view_of rg xs e acc ≤ v"
  using assms unfolding view_of_def by (induction xs arbitrary: acc) auto

lemma full_view_mono:
  assumes rg_mono: "⋀x v v'. v ≤ v' ⟹ rg x v ≤ rg x v'"
    and le: "⋀x. e x ≤ e' x"
  shows "full_view rg xs e ≤ full_view rg xs e'"
  unfolding full_view_def
proof (rule view_of_le)
  fix x assume "x ∈ set xs"
  then show "rg x (e x) ≤ view_of rg xs e' bot"
    using rg_mono[OF le[of x], of x] view_of_ge_mem[of x xs rg e' bot] by (blast intro: order_trans)
qed simp

lemma pub_sides_ge:
  "x ∈ set xs ⟹ DG bot (rg x r) ≤ pub_sides key rg xs r (Inr (key x))"
  unfolding pub_sides_def
  by (induction xs) (auto simp: sup_fun_def intro: le_supI1 le_supI2)
```

What a solver observes of them Reading names is observed as a dependency on each of their unknowns and nothing else; publishing names is observed as one side effect per name.

```
lemma traverse_read_view [simp]:
  "traverse_rhs (read_view (mk_dg_man d key) rg xs acc K) τ
     = traverse_rhs (K (view_of rg xs (genv key τ) acc)) τ"
  by (induction xs arbitrary: acc) (simp_all add: view_of_def sp_bind_def sp_return_def)

lemma sides_read_view [simp]:
  "sides_of_rhs (read_view (mk_dg_man d key) rg xs acc K) τ
     = sides_of_rhs (K (view_of rg xs (genv key τ) acc)) τ"
  by (induction xs arbitrary: acc) (simp_all add: view_of_def sp_bind_def sp_return_def)

lemma dep_aux_read_view [simp]:
  "dep_aux τ (read_view (mk_dg_man d key) rg xs acc K)
     = (Inr ∘ key) ‘ set xs ∪ dep_aux τ (K (view_of rg xs (genv key τ) acc))"
  by (induction xs arbitrary: acc) (auto simp: view_of_def sp_bind_def sp_return_def)

lemma traverse_publish_at [simp]:
  "traverse_rhs (publish_at (mk_dg_man d key) rg xs r K) τ = traverse_rhs (K ()) τ"
  by (induction xs) (simp_all add: sp_bind_def sp_return_def)

lemma sides_publish_at [simp]:
  "sides_of_rhs (publish_at (mk_dg_man d key) rg xs r K) τ
     = sides_of_rhs (K ()) τ ⊔ pub_sides key rg xs r"
  by (induction xs) (simp_all add: pub_sides_def sp_bind_def sp_return_def ac_simps)

lemma dep_aux_publish_at [simp]:
  "dep_aux τ (publish_at (mk_dg_man d key) rg xs r K) = dep_aux τ (K ())"
  by (induction xs) (simp_all add: sp_bind_def sp_return_def)

lemma side_path_read_view [simp]:
  "side_path τ (read_view (mk_dg_man d key) rg xs acc K)
     = side_path τ (K (view_of rg xs (genv key τ) acc))"
  by (induction xs arbitrary: acc)
     (simp_all add: view_of_def dg_read_global_def sp_read_global_def sp_bind_def sp_return_def)

lemma side_path_publish_at [simp]:
  "side_path τ (publish_at (mk_dg_man d key) rg xs r K)
     = map key xs @ side_path τ (K ())"
  by (induction xs) (simp_all add: dg_sideg_def sp_publish_def sp_bind_def sp_return_def)

lemma sp_wf_read_view [intro]:
  "(⋀x. sp_wf (man_global m x)) ⟹ sp_wf (read_view m rg xs acc)"
  by (induction xs arbitrary: acc) (auto intro!: sp_wf_bind)

lemma sp_wf_publish_at [intro]:
  "(⋀x v. sp_wf (man_sideg m x v)) ⟹ sp_wf (publish_at m rg xs r)"
  by (induction xs) (auto intro!: sp_wf_bind)
```

The keyed transfers

Each transfer reads its names, recombines them with the point's local half, runs the wrapped whole-state operation, publishes the names it may change and answers with the local half of the result. `cmb` and `rl` are the binary recombination of a local and a global half and the local projection.

```
definition keyed_transfer ::
  "('d ⇒ 'd ⇒ 'd) ⇒ ('d ⇒ 'd) ⇒ ('v ⇒ 'd ⇒ 'd) ⇒ ('v list ⇒ 'd)
   ⇒ 'v list ⇒ 'v list ⇒ ('d ⇒ 'd)
   ⇒ ('x, 'k, 'v, 'd::bounded_semilattice_sup_bot, 'd) man_transfer"
where
  "keyed_transfer cmb rl rg free R W f m =
     read_view m rg R (free R) ≫= (λg.
     publish_at m rg W (f (cmb (man_local m) g)) ≫= (λ_.
     sp_return (rl (f (cmb (man_local m) g)))))"

definition keyed_combine_transfer ::
  "('d ⇒ 'd ⇒ 'd) ⇒ ('d ⇒ 'd) ⇒ ('v ⇒ 'd ⇒ 'd) ⇒ ('v list ⇒ 'd)
   ⇒ 'v list ⇒ ('d ⇒ 'd ⇒ 'd)
   ⇒ ('x, 'k, 'v, 'd::bounded_semilattice_sup_bot, 'd) man_combine_transfer"
where
  "keyed_combine_transfer cmb rl rg free W h m de =
     publish_at m rg W (h (cmb (man_local m) (free [])) (cmb de (free []))) ≫= (λ_.
     sp_return (rl (h (cmb (man_local m) (free [])) (cmb de (free [])))))"

definition keyed_enter_transfer ::
  "('d ⇒ 'd ⇒ 'd) ⇒ ('d ⇒ 'd) ⇒ ('v ⇒ 'd ⇒ 'd) ⇒ ('v list ⇒ 'd)
   ⇒ 'v list ⇒ 'v list ⇒ ('d ⇒ 'd enter_result list)
   ⇒ ('x, 'k, 'v, 'd::bounded_semilattice_sup_bot, 'd) man_enter_transfer"
where
  "keyed_enter_transfer cmb rl rg free R W en m =
     read_view m rg R (free R) ≫= (λg.
     publish_at m rg W (⨆(c, e)←en (cmb (man_local m) g). e) ≫= (λ_.
     sp_return (map (λ(c, e). (rl c, rl e)) (en (cmb (man_local m) g)))))"
```

A call answers the alternatives its entry computes from the recombined state and publishes each name in `W` cut from the join of the entered states.

```
lemma enter_runs_keyed_enter_transfer:
  "enter_runs (keyed_enter_transfer cmb rl rg free R W en) (mk_dg_man d key) σ
     (map (λ(c, e). (rl c, rl e)) (en (cmb d (view_of rg R (genv key σ) (free R)))))
     (pub_sides key rg W
        (⨆(c, e)←en (cmb d (view_of rg R (genv key σ) (free R))). e))"
  unfolding enter_runs_def keyed_enter_transfer_def
  by (simp add: sp_bind_def sp_return_def ac_simps)

lemma enter_deps_keyed_enter_transfer:
  "enter_deps (keyed_enter_transfer cmb rl rg free R W en) (mk_dg_man d key) σ
     (map (λ(c, e). (rl c, rl e)) (en (cmb d (view_of rg R (genv key σ) (free R)))))
     ((Inr ∘ key) ‘ set R)"
  unfolding enter_deps_def keyed_enter_transfer_def
  by (simp add: sp_bind_def sp_return_def)
```

Only the manager's local value and its two global capabilities are read, so the query channel installed around a transfer does not change it.

```
lemma read_view_outer_man [simp]:
  "read_view (outer_man Q m) rg xs acc = read_view m rg xs acc"
  by (induction xs arbitrary: acc) simp_all

lemma publish_at_outer_man [simp]:
  "publish_at (outer_man Q m) rg xs r = publish_at m rg xs r"
  by (induction xs) simp_all

lemma keyed_transfer_outer_man [simp]:
  "keyed_transfer cmb rl rg free R W f (outer_man Q m) = keyed_transfer cmb rl rg free R W f m"
  by (simp add: keyed_transfer_def)
```

The lifter

Every edge runs the component's closed step through `keyed_transfer`, reading the globals its expressions mention and publishing the globals it may assign. A return reads nothing and may assign its destination. The return pipeline is wrapped once rather than stage by stage.

A call reads the globals its actuals mention and passes every other global as `free`. The context a call enters is computed from that state, and the analyzer recomputes it outside the solver from the same view of the solved

environment.

```
definition keyed_split_spec ::
  "(var_id ⇒ bool) ⇒ ('d ⇒ 'd ⇒ 'd) ⇒ ('d ⇒ 'd)
   ⇒ (var_id ⇒ 'd ⇒ 'd) ⇒ (var_id list ⇒ 'd) ⇒ 'd local_spec
   ⇒ ('x, 'k, var_id, 'd::bounded_semilattice_sup_bot, 'd) dg_spec"
where
  "keyed_split_spec 𝒢 cmb rl rg free c =
     (let step = (λa. keyed_transfer cmb rl rg free (edge_global_reads 𝒢 a)
                         (edge_global_writes 𝒢 a) ⟦a⟧♯c)
      in local_dg_spec_template⦇
         dgs_skip := step EA_Nop,
         dgs_assign := (λx e. step (EA_Assign x e)),
         dgs_special := (λsc x. step (EA_Special sc x)),
         dgs_branch := (λb pol. step (if pol then EA_Assume b else EA_AssumeNot b)),
         dgs_body := (λp. step (EA_Body p)),
         dgs_return := (λe p. step (EA_Ret e p)),
         dgs_enter := (λci. keyed_enter_transfer cmb rl rg free
            (call_global_reads 𝒢 (ci_args ci)) (global_names_in 𝒢 (ci_formals ci))
            (λd. ls_enter c (ls_channel c d) ci (d, d))),
         dgs_event := (λev. step (event_action ev)),
         dgs_combine_assign := (λci. keyed_combine_transfer cmb rl rg free
            (global_names_in 𝒢 (case_option [] (λx. [x]) (ci_dst ci)))
            (λdc de. ls_combine c (ls_channel c dc) (ls_channel c de) ci dc de)) ⦈)"

lemma dg_spec_step_keyed_split_spec [simp]:
  "dg_spec_step (keyed_split_spec 𝒢 cmb rl rg free c) a
     = keyed_transfer cmb rl rg free (edge_global_reads 𝒢 a) (edge_global_writes 𝒢 a)
         ⟦a⟧♯c"
  unfolding keyed_split_spec_def Let_def by (cases a) simp_all

lemma dgs_enter_keyed_split_spec [simp]:
  "enter♯ (keyed_split_spec 𝒢 cmb rl rg free c) ci
     = keyed_enter_transfer cmb rl rg free (call_global_reads 𝒢 (ci_args ci))
         (global_names_in 𝒢 (ci_formals ci)) (λd. ls_enter c (ls_channel c d) ci (d, d))"
  unfolding keyed_split_spec_def Let_def by simp

lemma dgs_query_keyed_split_spec [simp]:
  "dgs_query (keyed_split_spec 𝒢 cmb rl rg free c) m q = sp_return ⊤"
  unfolding keyed_split_spec_def Let_def by simp

lemma dg_spec_combine_transfer_keyed_split_spec [simp]:
  "dg_spec_combine_transfer (keyed_split_spec 𝒢 cmb rl rg free c) ci
     = keyed_combine_transfer cmb rl rg free (global_names_in 𝒢 (case_option [] (λx. [x]) (ci_dst ci)))
         (λdc de. ls_combine c (ls_channel c dc) (ls_channel c de) ci dc de)"
  unfolding dg_spec_combine_transfer_def keyed_split_spec_def Let_def
  by (simp add: local_combine_transfer_def fun_eq_iff)

lemma dg_spec_wf_keyed_split_spec [intro, simp]:
  "dg_spec_wf (keyed_split_spec 𝒢 cmb rl rg free c)"
  unfolding dg_spec_wf_def
  by (auto simp: keyed_transfer_def keyed_enter_transfer_def keyed_combine_transfer_def
      intro!: sp_wf_bind)
```

Soundness of the keyed lifter

A point's state is its local half recombined with the global half its environment describes over the program's global names `xs`. The carrier owes monotone operations, and that a name outside a read set is cut to something below `free` of that set. What it owes about concretization is one frame law, `mix`: a store described by a result, whose globals outside `W` agree with a store described against an environment, is described by the result's local half recombined with the result's own cut at the names in `W` and the environment at every other name.

```
theorem keyed_split_contract:
  fixes cmb :: "'d::bounded_semilattice_sup_bot ⇒ 'd ⇒ 'd"
    and gm :: "'d ⇒ store set"
  assumes sound: "sound_local_spec κ gm c"
    and cmb_mono: "⋀d d' g g'. d ≤ d' ⟹ g ≤ g' ⟹ cmb d g ≤ cmb d' g'"
    and rg_mono: "⋀x v v'. v ≤ v' ⟹ rg x v ≤ rg x v'"
    and rg_free: "⋀x R v. x ∉ set R ⟹ rg x v ≤ free R"
    and mix: "⋀l0 e r s s' W. s ∈ gm (cmb l0 (full_view rg xs e)) ⟹ s' ∈ gm r
        ⟹ (∀x. is_global x ⟶ x ∉ set W ⟶ s' x = s x)
        ⟹ s' ∈ gm (cmb (rl r) (full_view rg xs (λx. if x ∈ set W then rg x r else e x)))"
  shows "analysis_contract κ (keyed_split_spec is_global cmb rl rg free c)
           (λd e. gm (cmb d (full_view rg xs e)))"
proof -
  have gm_mono_all: "∀x y. x ≤ y ⟶ gm x ⊆ gm y"
```

```
  using sound unfolding sound_local_spec_def by (rule conjunct1)
then have gm_mono: "⋀x y. x ≤ y ⟹ gm x ⊆ gm y" by blast
have env_mono: "⋀d d' e e'. d ≤ d' ⟹ (⋀x. e x ≤ e' x)
    ⟹ gm (cmb d (full_view rg xs e)) ⊆ gm (cmb d' (full_view rg xs e'))"
  by (rule gm_mono, rule cmb_mono, assumption, rule full_view_mono[OF rg_mono]) simp
have view_ge: "full_view rg xs e ≤ view_of rg R e (free R)" for e R
  unfolding full_view_def
proof (rule view_of_le)
  fix x assume "x ∈ set xs"
  show "rg x (e x) ≤ view_of rg R e (free R)"
  proof (cases "x ∈ set R")
    case True then show ?thesis by (rule view_of_ge_mem)
  next
    case False
    then show ?thesis using rg_free view_of_ge_acc by (blast intro: order_trans)
  qed
qed simp
have out: "(λx. if x ∈ set W then rg x r else e x) x ≤ (e ⊔ genv key P) x"
  if pub: "⋀x. x ∈ set W ⟹ rg x r ≤ dg_global (P (Inr (key x)))"
  for e W r x key and P :: "'z + 'w ⇒ ('d, 'd) dg_state"
  using pub[of x] by (auto intro: le_supI2)
have pub: "rg x r ≤ dg_global ((bot ⊔ pub_sides key rg W r) (Inr (key x)))"
  if "x ∈ set W" for x W r key
  using pub_sides_ge[OF that, of rg r key] by (simp add: less_eq_dg_state_def)
let ?S = "keyed_split_spec is_global cmb rl rg free c"
show ?thesis
proof (unfold_locales, goal_cases wf mono step comb)
  case wf show ?case by (rule dg_spec_wf_keyed_split_spec)
next
  case (mono d d' e e')
  then show ?case by (intro env_mono) (simp_all add: le_fun_def)
next
  case (step a τ src key)
  let ?d = "dg_local (τ src)" and ?e = "genv key τ"
  let ?R = "edge_global_reads is_global a" and ?W = "edge_global_writes is_global a"
  let ?g = "view_of rg ?R ?e (free ?R)"
  let ?r = "⟦a⟧♯c (cmb ?d ?g)"
  have loc: "edge_out ?S a src key τ = rl ?r"
    by (simp add: dg_spec_edge_program_def traverse_transfer_program keyed_transfer_def
        sp_compile_with_def sp_bind_def sp_return_def)
  have sides: "sides_of_program (dg_spec_edge_program ?S a src key) τ
      = bot ⊔ pub_sides key rg ?W ?r"
    by (simp add: dg_spec_edge_program_def sides_transfer_program keyed_transfer_def
        sp_compile_with_def sp_bind_def sp_return_def bot_fun_def[symmetric])
  show ?case
  proof
    fix s' assume "s' ∈ edge_collect κ a (gm (cmb ?d (full_view rg xs ?e)))"
    then obtain s where s: "s ∈ gm (cmb ?d (full_view rg xs ?e))" and st: "s' ∈ edge_step κ a s"
      by (auto simp: edge_collect_def)
    have "s ∈ gm (cmb ?d ?g)"
      using s gm_mono[OF cmb_mono[OF order_refl view_ge]] by blast
    then have "s' ∈ edge_collect κ a (gm (cmb ?d ?g))" using st
      by (auto simp: edge_collect_def)
    then have r: "s' ∈ gm ?r" using closed_step_sound[OF sound] by blast
    have fr: "∀x. is_global x ⟶ x ∉ set ?W ⟶ s' x = s x"
      using edge_step_frame[OF st] by auto
    have "s' ∈ gm (cmb (rl ?r) (full_view rg xs (λx. if x ∈ set ?W then rg x ?r else ?e x)))"
      by (rule mix[OF s r fr])
    also have "... ⊆ gm (cmb (rl ?r) (full_view rg xs (?e ⊔ genv key (bot ⊔ pub_sides key rg ?W ?r))))"
      by (rule env_mono[OF order_refl], rule out, erule pub)
    finally show "s' ∈ gm (cmb (edge_out ?S a src key τ)
        (full_view rg xs (genv key τ ⊔ edge_pub ?S a src key τ)))"
      unfolding loc sides .
  qed
next
  case (comb s dc key τ t de ci)
  let ?e = "genv key τ" and ?W =
    "global_names_in is_global (case_option [] (λx. [x]) (ci_dst ci))"
  let ?h = "λdc de. ls_combine c (ls_channel c dc) (ls_channel c de) ci dc de"
  let ?r = "?h (cmb dc (free [])) (cmb de (free []))"
  have free_ge: "full_view rg xs ?e ≤ free []"
    unfolding full_view_def by (rule view_of_le) (simp_all add: rg_free)
  have sc: "s ∈ gm (cmb dc (free []))"
    using comb(1) gm_mono[OF cmb_mono[OF order_refl free_ge]] by blast
  have tc: "t ∈ gm (cmb de (free []))"
    using comb(2) gm_mono[OF cmb_mono[OF order_refl free_ge]] by blast
```

```
    have r: "combine_collect κ (ci_dst ci) s t ∈ gm ?r"
      by (rule closed_combine_sound[OF sound sc tc])
    have fr: "∀x. is_global x ⟶ x ∉ set ?W ⟶ combine_collect κ (ci_dst ci) s t x = t x"
      by (auto intro: combine_collect_frame)
    have "combine_collect κ (ci_dst ci) s t
        ∈ gm (cmb (rl ?r) (full_view rg xs (λx. if x ∈ set ?W then rg x ?r else ?e x)))"
      by (rule mix[OF comb(2) r fr])
    also have "... ⊆ gm (cmb (rl ?r) (full_view rg xs (?e ⊔ genv key (bot ⊔ pub_sides key rg ?W ?r))))"
      by (rule env_mono[OF order_refl], rule out, erule pub)
    finally show ?case
      by (simp add: keyed_combine_transfer_def sp_compile_with_def sp_bind_def sp_return_def
          bot_fun_def[symmetric])
  qed
qed

end
```

6.17 Analysis_Query

```
theory Analysis_Query
  imports "Voblint_Domain.Query_Lift" "Voblint_Domain.Int_Lattice" "Voblint_VIMP.Core_Exp"
begin

unbundle lattice_syntax
```

Queries one analysis answers for another

An analysis in a combination may ask its partners a question about the stores the current state describes, as a Goblint transfer asks `man.ask`. What an answer claims is fixed by one relation, `answer_holds q a s`: answer `a` to query `q` is true of store `s`. Answers form a meet-semilattice with top, as Goblint's per-query result lattices do (`queries.ml` at `5320a6b7`): $\top$ claims nothing, and $\sqcap$ combines what several analyses answered. The two laws below are all a combination needs to trust a combined answer; associativity and commutativity of the combination come from the class.

```
locale query_algebra =
  fixes answer_holds :: "'q ⇒ 'r::{semilattice_inf, order_top} ⇒ 's ⇒ bool"
  assumes top_sound: "answer_holds q ⊤ s"
      and inf_sound: "⟦answer_holds q a s; answer_holds q b s⟧ ⟹ answer_holds q (a ⊓ b) s"
begin
```

A channel answers every query. It holds at a store when each of its answers is true there. Component transfers are proved against this predicate, never against a particular partner.

```
definition channel_holds :: "('q ⇒ 'r) ⇒ 's ⇒ bool" where
  "channel_holds ch s ⟷ (∀q. answer_holds q (ch q) s)"

lemma channel_holdsD: "channel_holds ch s ⟹ answer_holds q (ch q) s"
  by (simp add: channel_holds_def)

lemma channel_holds_top [simp, intro]: "channel_holds (λ_. ⊤) s"
  by (simp add: channel_holds_def top_sound)

lemma channel_holds_inf [intro]:
  "channel_holds ch₁ s ⟹ channel_holds ch₂ s ⟹ channel_holds (λq. ch₁ q ⊓ ch₂ q) s"
  by (simp add: channel_holds_def inf_sound)

end
```

The value of an expression

Version 1 has one query kind, Goblint's `EvalInt`: which integers an expression may evaluate to (`queries.ml` line 108 at `0dc12d355`). Goblint answers it in `Lattice.Lift(IntDomTuple)`, and so does Voblint: the answer is an `int_dom` under `'a query_lift`. A comparison or logical operator evaluates to `0` or `1`, so its truth is the exact answer `1` or `0`; Goblint derives its former `MustBeEqual` and `MayBeLess` queries from `EvalInt` the same way (lines 528 to 539). The query is a constructor rather than a bare expression so that a later kind extends the datatype.

```
datatype query = EvalInt exp

type_synonym answer = "int_dom query_lift"

abbreviation gamma_answer :: "answer ⇒ int set" where
  "gamma_answer ≡ gamma_query_lift gamma_int_dom"
```

An answer claims that the expression evaluates into its concretization. `QTop` claims nothing: an analysis that does not understand the question declines with it. `QBot` admits no value, so a sound handler returns it only for a state that represents no concrete store. The meet combines two answers about one expression, and since the `int_dom` meet is exact, two sound analyses whose answers do not overlap describe no store in common.

An answer about an expression holds when the expression's value lies in the answer's concretization.

```
fun eval_holds :: "query ⇒ answer ⇒ store ⇒ bool" where
  "eval_holds (EvalInt e) a s ⟷ eval e s ∈ gamma_answer a"

interpretation eval_query: query_algebra eval_holds
proof
  fix q and a b :: answer and s
  show "eval_holds q ⊤ s"
    by (cases q) simp
  show "eval_holds q a s ⟹ eval_holds q b s ⟹ eval_holds q (a ⊓ b) s"
    by (cases q) (simp add: gamma_query_lift_inf[OF gamma_inf_int_dom])
```

```
qed

declare eval_query.top_sound [simp]
```

The exact answer for a known integer, and a known integer read back from an answer, as Goblint's `ID.of_int` and `ID.to_int`. A consumer that reads `n` back may use it for the expression at every store the answer holds at.

```
definition answer_of_int :: "int ⇒ answer" where
  "answer_of_int n = QLifted (int_dom_of_int n)"

lemma gamma_answer_of_int [simp]:
  "gamma_answer (answer_of_int n) = {n}"
  by (simp add: answer_of_int_def)

fun answer_const :: "answer ⇒ int option" where
  "answer_const (QLifted d) = int_dom_constant d"
| "answer_const _ = None"

lemma answer_const_of_int [simp]: "answer_const (answer_of_int n) = Some n"
  by (simp add: answer_of_int_def int_dom_of_int_def int_dom_constant_def
      top_int_dom_ext_def)
```

An answer excludes `n` when it admits no value `n`. The meet with the exact `n` decides it, since the `int_dom` meet and emptiness test are exact; so a parity or sign answer excludes a constant no component names.

```
fun answer_excludes :: "int ⇒ answer ⇒ bool" where
  "answer_excludes n (QLifted d) = is_bottom_int_dom (d ⊓ int_dom_of_int n)"
| "answer_excludes n QBot = True"
| "answer_excludes n QTop = False"

lemma answer_excludes_iff: "answer_excludes n a ⟷ n ∉ gamma_answer a"
  by (cases a) (auto simp: is_bottom_int_dom_correct)
```

An interval analysis knows only the interval component; the others stay at their top, so the answer claims exactly what the interval does.

```
definition answer_of_ivl :: "ivl ⇒ answer" where
  "answer_of_ivl i = QLifted ((⊤ :: int_dom)(|int_ivl := i|))"

lemma gamma_answer_of_ivl [simp]:
  "gamma_answer (answer_of_ivl i) = gamma_ivl i"
  by (simp add: answer_of_ivl_def top_int_dom_ext_def top_sign_def top_parity_def
      gamma_int_dom_def)

end
```

6.18 Local_Spec_Product

```
theory Local_Spec_Product
  imports "Voblint_Framework.DG_Spec_Sound"
begin
```

The product carrier

Two components run side by side on a pair of states. The pair is a datatype and not `'a × 'b`, for the reason `('l, 'g) dg_state` is: this repository loads `HOL-Library.Product_Lexorder`, so raw pairs already carry the lexicographic order, and the product needs the componentwise one. Every instance below, including the solver's widening and narrowing, is componentwise.

```
datatype ('a, 'b) analysis_product = Product (pleft: 'a) (pright: 'b)

instantiation analysis_product :: (ord, ord) ord
begin

definition less_eq_analysis_product ::
  "('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product ⇒ bool" where
  "less_eq_analysis_product p q ⟷ pleft p ≤ pleft q ∧ pright p ≤ pright q"

definition less_analysis_product ::
  "('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product ⇒ bool" where
  "less_analysis_product p q ⟷ p ≤ q ∧ ¬ q ≤ p"

instance ..

end

instance analysis_product :: (order, order) order
proof intro_classes
  fix x y z :: "('a, 'b) analysis_product"
  show "x < y ⟷ x ≤ y ∧ ¬ y ≤ x" by (simp add: less_analysis_product_def)
  show "x ≤ x" by (simp add: less_eq_analysis_product_def)
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z"
    by (auto simp: less_eq_analysis_product_def intro: order_trans)
  show "x ≤ y ⟹ y ≤ x ⟹ x = y"
    by (auto simp: less_eq_analysis_product_def intro: analysis_product.expand antisym)
qed

instantiation analysis_product :: (semilattice_sup, semilattice_sup) semilattice_sup
begin

definition sup_analysis_product ::
  "('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product" where
  "sup_analysis_product p q = Product (pleft p ⊔ pleft q) (pright p ⊔ pright q)"

instance
  by standard (auto simp: less_eq_analysis_product_def sup_analysis_product_def)

end

instantiation analysis_product :: (order_bot, order_bot) order_bot
begin

definition bot_analysis_product :: "('a, 'b) analysis_product" where
  "bot_analysis_product = Product ⊥ ⊥"

instance
  by standard (simp add: less_eq_analysis_product_def bot_analysis_product_def)

end

instance analysis_product ::
  (bounded_semilattice_sup_bot, bounded_semilattice_sup_bot) bounded_semilattice_sup_bot ..

lemma Product_le_Product [simp]:
  "Product a b ≤ Product a' b' ⟷ a ≤ a' ∧ b ≤ b'"
  by (simp add: less_eq_analysis_product_def)

instantiation analysis_product ::
  ("{bounded_semilattice_sup_bot, warrowing}", "{bounded_semilattice_sup_bot, warrowing}") warrowing
begin

definition widen_analysis_product ::
```

```
  "('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product"
where
  "widen_analysis_product p q = Product ((pleft p) ∇ (pleft q)) ((pright p) ∇ (pright q))"

definition narrow_analysis_product ::
  "('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product"
where
  "narrow_analysis_product p q =
     Product ((pleft p) ∆ (pleft q)) ((pright p) ∆ (pright q))"

instance
  by standard
    (simp_all add: less_eq_analysis_product_def widen_analysis_product_def
      narrow_analysis_product_def narrowing_class.narrow_ge narrowing_class.narrow_le
      widening_class.widen_ge1 widening_class.widen_ge2)

end


end
```

6.19 MCP_Spec

```
theory MCP_Spec
  imports "Voblint_Framework.DG_Spec_Sound" "Voblint_Domain.Reachability_Lift"
begin
```

Many cooperating analyses over one state

Goblint's `MCP` runs every activated analysis on its own part of one combined state (`mCP.ml` at `0dc12d355`). Voblint keeps the combined state as one type `'s`, a record with a field per registered analysis, and a component as operations on the whole record that touch only its own field. The activation list chooses which components run; the fields of inactive analyses are never read.

Every operation of a component receives the query channel of the manager Goblint hands it, `man.ask`, as a function `channel` from queries to answers: its query handler (`S.query man q`), its edge transfers, its entry (`S.enter man`) and both stages of its return. The return stages also receive the callee's channel, which Goblint's normal-call transfer in `constraints.ml` passes as `f_ask`. A handler may therefore ask further queries while it answers, and the channel that answers them is the one `ask_rec` below closes over the combined state, as Goblint's `MCP.query'` does.

The channel is a pure function of the state it describes. A component therefore neither reads nor publishes globals: the combined state covers analyses whose operations are local, and effectful analyses run as specifications of their own (`dgs_query` is the effectful handler).

A component holds only what runs. What its states describe, a set of stores, is not executable and is supplied beside it wherever soundness is stated, as `numeric_domain` keeps `gamma` out of `executable_domain`.

```
record 's local_spec =
  ls_query :: "channel ⇒ 's ⇒ channel"
  ls_skip :: "channel ⇒ 's ⇒ 's"
  ls_assign :: "channel ⇒ var_id ⇒ exp ⇒ 's ⇒ 's"
  ls_special :: "channel ⇒ exp gspecial_call ⇒ var_id ⇒ 's ⇒ 's"
  ls_branch :: "channel ⇒ exp ⇒ bool ⇒ 's ⇒ 's"
  ls_body :: "channel ⇒ pname ⇒ 's ⇒ 's"
  ls_return :: "channel ⇒ exp option ⇒ pname ⇒ 's ⇒ 's"
  ls_event :: "channel ⇒ analysis_event ⇒ 's ⇒ 's"
  ls_enter :: "channel ⇒ call_info ⇒ 's × 's ⇒ ('s × 's) list"
  ls_combine_env :: "channel ⇒ channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's"
  ls_combine_assign :: "channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's"
```

The edge transfers are separate fields, one per kind of edge, as Goblint's `Spec` has `skip`, `assign`, `special`, `branch`, `body`, `return` and `event`. A component that overrides one of them updates that field alone. The step on an arbitrary edge is derived from them.

```
definition ls_step :: "'s local_spec ⇒ channel ⇒ (var_id,exp) gedge_action ⇒ 's ⇒ 's"
  where
  "ls_step c ch = local_spec_step (ls_skip c ch) (ls_assign c ch) (ls_special c ch)
     (ls_branch c ch) (ls_body c ch) (ls_return c ch) (ls_event c ch)"

lemma ls_step_simps [simp]:
  "ls_step c ch EA_Nop = ls_skip c ch"
  "ls_step c ch (EA_Assign x e) = ls_assign c ch x e"
  "ls_step c ch (EA_Special sc x) = ls_special c ch sc x"
  "ls_step c ch (EA_Assume b) = ls_branch c ch b True"
  "ls_step c ch (EA_AssumeNot b) = ls_branch c ch b False"
  "ls_step c ch (EA_Body p) = ls_body c ch p"
  "ls_step c ch (EA_Ret r p) = ls_return c ch r p"
  "ls_step c ch (EA_Check l cnd) = ls_event c ch (Check_Event l cnd)"
  by (simp_all add: ls_step_def)

lemma ls_step_event_action [simp]: "ls_step c ch (event_action e) = ls_event c ch e"
  by (cases e) simp

lemma ls_step_update_other [simp]:
  "ls_step (c(|ls_query := h|)) = ls_step c"
  "ls_step (c(|ls_enter := e|)) = ls_step c"
  "ls_step (c(|ls_combine_env := ce|)) = ls_step c"
  "ls_step (c(|ls_combine_assign := ca|)) = ls_step c"
  by (simp_all add: ls_step_def fun_eq_iff)
```

A component whose edge transfers are one function of the edge, such as a lens onto a field or a fold over several components, is built from that function: each field is the function at its kind of edge.

```
definition make_local_spec ::
  "(channel ⇒ 's ⇒ channel) ⇒ (channel ⇒ (var_id,exp) gedge_action ⇒ 's ⇒ 's)
```

```
   ⇒ (channel ⇒ call_info ⇒ 's × 's ⇒ ('s × 's) list)
   ⇒ (channel ⇒ channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's)
   ⇒ (channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's) ⇒ 's local_spec" where
  "make_local_spec qry st en ce ca = (|
     ls_query = qry,
     ls_skip = (λch. st ch EA_Nop),
     ls_assign = (λch x e. st ch (EA_Assign x e)),
     ls_special = (λch sc x. st ch (EA_Special sc x)),
     ls_branch = (λch b pol. st ch (if pol then EA_Assume b else EA_AssumeNot b)),
     ls_body = (λch p. st ch (EA_Body p)),
     ls_return = (λch r p. st ch (EA_Ret r p)),
     ls_event = (λch ev. st ch (event_action ev)),
     ls_enter = en, ls_combine_env = ce, ls_combine_assign = ca |)"

lemma make_local_spec_sel [simp]:
  "ls_query (make_local_spec qry st en ce ca) = qry"
  "ls_step (make_local_spec qry st en ce ca) = st"
  "ls_enter (make_local_spec qry st en ce ca) = en"
  "ls_combine_env (make_local_spec qry st en ce ca) = ce"
  "ls_combine_assign (make_local_spec qry st en ce ca) = ca"
proof -
  have "ls_step (make_local_spec qry st en ce ca) ch a = st ch a" for ch a
  proof (cases a)
    case (EA_Check l cnd)
    then show ?thesis by (simp add: make_local_spec_def ls_step_def)
  qed (simp_all add: make_local_spec_def ls_step_def)
  then show "ls_step (make_local_spec qry st en ce ca) = st" by (simp add: fun_eq_iff)
qed (simp_all add: make_local_spec_def)
```

The return runs in the two stages of Goblint's `combine_env` and `combine_assign`, which both receive the callee's channel. Only the first receives the caller's: Goblint's `combine_assign` asks about the state `combine_env` produced, which describes no concrete store on its own here, since soundness and independence speak only about the composite.

```
abbreviation ls_combine ::
  "'s local_spec ⇒ channel ⇒ channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's" where
  "ls_combine c ch ch' ci x de ≡ ls_combine_assign c ch' ci (ls_combine_env c ch ch' ci x de) de"
```

A local specification is sound for a concretization `gamma` when each operation covers the concrete behaviour, whatever the other fields of the record hold. Every operation is proved against every channel that holds at the stores it is asked about, so its proof cannot depend on who answers: the caller's store for a step, an entry, a query and the first stage of a return, and the callee's exit store for the callee's channel.

```
definition sound_local_spec ::
  "kenv ⇒ ('s::order ⇒ store set) ⇒ 's local_spec ⇒ bool" where
  "sound_local_spec κ gm c ⟷
     (∀x y. x ≤ y ⟶ gm x ⊆ gm y)
     ∧ (∀ch a x. edge_collect κ a (gm x ∩ Collect (eval_query.channel_holds ch))
                 ⊆ gm (ls_step c ch a x))
     ∧ (∀ch s ci p. s ∈ gm (fst p) ⟶ eval_query.channel_holds ch s ⟶
         (∃q ∈ set (ls_enter c ch ci p). s ∈ gm (fst q)
            ∧ call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s
                ∈ gm (snd q)))
     ∧ (∀ch ch' s t ci x de. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶
         t ∈ gm de ⟶ eval_query.channel_holds ch' t ⟶
         combine_collect κ (ci_dst ci) s t ∈ gm (ls_combine c ch ch' ci x de))
     ∧ (∀ch s x q. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶
         eval_holds q (ls_query c ch x q) s)"
```

The step obligation splits into one law per kind of edge, each about the field that handles it: the store the edge produces lies in what the field returns, for every store and channel that hold before. A component proves the laws of the fields it defines, and a field it replaces needs only its own law again (`sound_local_spec_update`).

```
definition sound_skip :: "('s ⇒ store set) ⇒ (channel ⇒ 's ⇒ 's) ⇒ bool" where
  "sound_skip gm f ⟷
     (∀ch x s. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶ s ∈ gm (f ch x))"

definition sound_assign ::
  "kenv ⇒ ('s ⇒ store set) ⇒ (channel ⇒ var_id ⇒ exp ⇒ 's ⇒ 's) ⇒ bool" where
  "sound_assign κ gm f ⟷
     (∀ch x s y e. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶ s(y := kind_conv κ (Some y) (eval e s)) ∈ gm (f
ch y e x))"

definition sound_special ::
  "kenv ⇒ ('s ⇒ store set) ⇒ (channel ⇒ exp gspecial_call ⇒ var_id ⇒ 's ⇒ 's) ⇒ bool" where
  "sound_special κ gm f ⟷
```

```
    (∀ch x s sc y t. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶ t ∈ special_step κ sc y s
       ⟶ t ∈ gm (f ch sc y x))"

definition sound_branch :: "('s ⇒ store set) ⇒ (channel ⇒ exp ⇒ bool ⇒ 's ⇒ 's) ⇒ bool"
  where
  "sound_branch gm f ⟷
     (∀ch x s b pol. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶ truthy (eval b s) = pol
       ⟶ s ∈ gm (f ch b pol x))"

definition sound_body :: "('s ⇒ store set) ⇒ (channel ⇒ pname ⇒ 's ⇒ 's) ⇒ bool" where
  "sound_body gm f ⟷
     (∀ch x s p. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶ s ∈ gm (f ch p x))"

definition sound_return ::
  "('s ⇒ store set) ⇒ (channel ⇒ exp option ⇒ pname ⇒ 's ⇒ 's) ⇒ bool" where
  "sound_return gm f ⟷
     (∀ch x s r p. s ∈ gm x ⟶ eval_query.channel_holds ch s
       ⟶ s(ReturnVar := (case r of None ⇒ s ReturnVar | Some a ⇒ eval a s)) ∈ gm (f ch r p x))"

definition sound_event ::
  "('s ⇒ store set) ⇒ (channel ⇒ analysis_event ⇒ 's ⇒ 's) ⇒ bool" where
  "sound_event gm f ⟷
     (∀ch x s evt. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶ s ∈ gm (f ch evt x))"
```

The laws of the other fields. The handler's answers hold at every store its state describes. An entry covers the caller's store and the store the call enters by one of its pairs. The two return stages carry one law together, because only their composition describes a concrete store.

```
definition sound_query :: "('s ⇒ store set) ⇒ (channel ⇒ 's ⇒ channel) ⇒ bool" where
  "sound_query gm h ⟷
     (∀ch s x q. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶ eval_holds q (h ch x q) s)"

definition sound_enter ::
  "kenv ⇒ ('s ⇒ store set)
   ⇒ (channel ⇒ call_info ⇒ 's × 's ⇒ ('s × 's) list) ⇒ bool" where
  "sound_enter κ gm en ⟷
     (∀ch s ci p. s ∈ gm (fst p) ⟶ eval_query.channel_holds ch s ⟶
        (∃q ∈ set (en ch ci p). s ∈ gm (fst q)
           ∧ call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s ∈ gm (snd q)))"

definition sound_combine ::
  "kenv ⇒ ('s ⇒ store set)
   ⇒ (channel ⇒ channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's)
   ⇒ (channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's) ⇒ bool" where
  "sound_combine κ gm ce ca ⟷
     (∀ch ch' s t ci x de. s ∈ gm x ⟶ eval_query.channel_holds ch s ⟶
        t ∈ gm de ⟶ eval_query.channel_holds ch' t ⟶
        combine_collect κ (ci_dst ci) s t ∈ gm (ca ch' ci (ce ch ch' ci x de) de))"

theorem ls_step_sound_iff:
  "(∀ch a x. edge_collect κ a (gm x ∩ Collect (eval_query.channel_holds ch)) ⊆ gm (ls_step c ch a x))
   ⟷ sound_skip gm (ls_skip c) ∧ sound_assign κ gm (ls_assign c) ∧ sound_special κ gm (ls_special c)
     ∧ sound_branch gm (ls_branch c) ∧ sound_body gm (ls_body c)
     ∧ sound_return gm (ls_return c) ∧ sound_event gm (ls_event c)"
  (is "?step ⟷ ?fields")
proof
  assume step: ?step
  have at: "t ∈ gm (ls_step c ch act x)"
    if "s ∈ gm x" "eval_query.channel_holds ch s" "t ∈ edge_step κ act s" for ch act x s t
    using step that unfolding edge_collect_def by blast
  have skip: "s ∈ gm (ls_skip c ch x)"
    if "s ∈ gm x" "eval_query.channel_holds ch s" for ch x s
  proof -
    have "s ∈ gm (ls_step c ch EA_Nop x)" by (rule at[OF that]) simp
    then show ?thesis by simp
  qed
  have assign: "s(y := kind_conv κ (Some y) (eval e s)) ∈ gm (ls_assign c ch y e x)"
    if "s ∈ gm x" "eval_query.channel_holds ch s" for ch x s y e
  proof -
    have "s(y := kind_conv κ (Some y) (eval e s)) ∈ gm (ls_step c ch (EA_Assign y e) x)"
      by (rule at[OF that]) simp
    then show ?thesis by simp
  qed
  have body: "s ∈ gm (ls_body c ch p x)"
    if "s ∈ gm x" "eval_query.channel_holds ch s" for ch x s p
  proof -
    have "s ∈ gm (ls_step c ch (EA_Body p) x)" by (rule at[OF that]) simp
```

```
    then show ?thesis by simp
  qed
  have ret:
    "s(ReturnVar := (case r of None ⇒ s ReturnVar | Some a ⇒ eval a s)) ∈ gm (ls_return c ch r p x)"
    if "s ∈ gm x" "eval_query.channel_holds ch s" for ch x s r p
  proof -
    have "s(ReturnVar := (case r of None ⇒ s ReturnVar | Some a ⇒ eval a s)) ∈ gm (ls_step c ch (EA_Ret r p) x)"
      by (rule at[OF that]) simp
    then show ?thesis by simp
  qed
  have pol: "s ∈ gm (ls_branch c ch b pol x)"
    if "s ∈ gm x" "eval_query.channel_holds ch s" "truthy (eval b s) = pol" for ch b pol x s
  proof (cases pol)
    case True
    have "s ∈ gm (ls_step c ch (EA_Assume b) x)"
      by (rule at[OF that(1,2)]) (use that(3) True in simp)
    then show ?thesis using True by simp
  next
    case False
    have "s ∈ gm (ls_step c ch (EA_AssumeNot b) x)"
      by (rule at[OF that(1,2)]) (use that(3) False in simp)
    then show ?thesis using False by simp
  qed
  have eval: "s ∈ gm (ls_event c ch evt x)"
    if "s ∈ gm x" "eval_query.channel_holds ch s" for ch evt x s
  proof (cases evt)
    case (Check_Event l e)
    have "s ∈ gm (ls_step c ch (EA_Check l e) x)" by (rule at[OF that]) simp
    then show ?thesis using Check_Event by simp
  qed
  have special: "t ∈ gm (ls_special c ch sc y x)"
    if "s ∈ gm x" "eval_query.channel_holds ch s" "t ∈ special_step κ sc y s" for ch x s sc y t
  proof -
    have "t ∈ gm (ls_step c ch (EA_Special sc y) x)"
      by (rule at[OF that(1,2)]) (simp only: edge_step.simps that(3))
    then show ?thesis by (simp only: ls_step_simps)
  qed
  show ?fields
    unfolding sound_skip_def sound_assign_def sound_special_def sound_branch_def sound_body_def
      sound_return_def sound_event_def
    by (intro conjI allI impI; (rule skip assign special pol body ret eval; assumption))
next
  assume F: ?fields
  show ?step
  proof (intro allI subsetI)
    fix ch a x t
    assume "t ∈ edge_collect κ a (gm x ∩ Collect (eval_query.channel_holds ch))"
    then obtain s where s: "s ∈ gm x" "eval_query.channel_holds ch s" "t ∈ edge_step κ a s"
      unfolding edge_collect_def by blast
    show "t ∈ gm (ls_step c ch a x)"
    proof (cases a)
      case (EA_Special sc y)
      have "t ∈ special_step κ sc y s" using s(3) EA_Special by (simp only: edge_step.simps)
      then show ?thesis
        using F s(1,2) EA_Special unfolding sound_special_def by (simp only: ls_step_simps)
    next
      case (EA_Assume b)
      then have "t = s" "truthy (eval b s) = True" using s(3) by (simp_all split: if_splits)
      then show ?thesis
        using F s(1,2) EA_Assume unfolding sound_branch_def by (simp only: ls_step_simps)
    next
      case (EA_AssumeNot b)
      then have "t = s" "truthy (eval b s) = False" using s(3) by (simp_all split: if_splits)
      then show ?thesis
        using F s(1,2) EA_AssumeNot unfolding sound_branch_def by (simp only: ls_step_simps)
    qed (use F s in ‹auto simp: sound_skip_def sound_assign_def sound_body_def sound_return_def
                     sound_event_def›)
  qed
qed
```

The certificate is exactly the conjunction of a monotone concretization and one law per field, the two return stages sharing one law. A local specification is therefore built, taken apart and changed field by field.

```
theorem sound_local_spec_iff:
  "sound_local_spec κ gm c ⟷
     (∀x y. x ≤ y ⟶ gm x ⊆ gm y)
     ∧ sound_query gm (ls_query c)
```

```
    ∧ sound_skip gm (ls_skip c) ∧ sound_assign κ gm (ls_assign c)
    ∧ sound_special κ gm (ls_special c) ∧ sound_branch gm (ls_branch c)
    ∧ sound_body gm (ls_body c) ∧ sound_return gm (ls_return c)
    ∧ sound_event gm (ls_event c)
    ∧ sound_enter κ gm (ls_enter c)
    ∧ sound_combine κ gm (ls_combine_env c) (ls_combine_assign c)"
  unfolding sound_local_spec_def ls_step_sound_iff sound_query_def sound_enter_def
    sound_combine_def
  by (intro iffI; elim conjE; intro conjI; assumption)

lemma sound_local_specI:
  assumes "⋀x y. x ≤ y ⟹ gm x ⊆ gm y"
    and "sound_query gm (ls_query c)"
    and "sound_skip gm (ls_skip c)" and "sound_assign κ gm (ls_assign c)"
    and "sound_special κ gm (ls_special c)" and "sound_branch gm (ls_branch c)"
    and "sound_body gm (ls_body c)" and "sound_return gm (ls_return c)"
    and "sound_event gm (ls_event c)"
    and "sound_enter κ gm (ls_enter c)"
    and "sound_combine κ gm (ls_combine_env c) (ls_combine_assign c)"
  shows "sound_local_spec κ gm c"
  using assms unfolding sound_local_spec_iff by blast

lemma sound_local_specD:
  assumes "sound_local_spec κ gm c"
  shows sound_local_spec_monoD: "x ≤ y ⟹ gm x ⊆ gm y"
    and sound_local_spec_queryD: "sound_query gm (ls_query c)"
    and sound_local_spec_skipD: "sound_skip gm (ls_skip c)"
    and sound_local_spec_assignD: "sound_assign κ gm (ls_assign c)"
    and sound_local_spec_specialD: "sound_special κ gm (ls_special c)"
    and sound_local_spec_branchD: "sound_branch gm (ls_branch c)"
    and sound_local_spec_bodyD: "sound_body gm (ls_body c)"
    and sound_local_spec_returnD: "sound_return gm (ls_return c)"
    and sound_local_spec_eventD: "sound_event gm (ls_event c)"
    and sound_local_spec_enterD: "sound_enter κ gm (ls_enter c)"
    and sound_local_spec_combineD:
      "sound_combine κ gm (ls_combine_env c) (ls_combine_assign c)"
  using assms unfolding sound_local_spec_iff by blast+
```

A sound local specification stays sound when one field is replaced by one satisfying that field's law. A return stage is replaced together with the law of the composed return it forms with the other stage.

```
lemma sound_local_spec_update:
  assumes "sound_local_spec κ gm c"
  shows sound_local_spec_update_query:
      "sound_query gm h ⟹ sound_local_spec κ gm (c(|ls_query := h|))"
    and sound_local_spec_update_skip:
      "sound_skip gm sk ⟹ sound_local_spec κ gm (c(|ls_skip := sk|))"
    and sound_local_spec_update_assign:
      "sound_assign κ gm asn ⟹ sound_local_spec κ gm (c(|ls_assign := asn|))"
    and sound_local_spec_update_special:
      "sound_special κ gm sp ⟹ sound_local_spec κ gm (c(|ls_special := sp|))"
    and sound_local_spec_update_branch:
      "sound_branch gm br ⟹ sound_local_spec κ gm (c(|ls_branch := br|))"
    and sound_local_spec_update_body:
      "sound_body gm bd ⟹ sound_local_spec κ gm (c(|ls_body := bd|))"
    and sound_local_spec_update_return:
      "sound_return gm rt ⟹ sound_local_spec κ gm (c(|ls_return := rt|))"
    and sound_local_spec_update_event:
      "sound_event gm evt ⟹ sound_local_spec κ gm (c(|ls_event := evt|))"
    and sound_local_spec_update_enter:
      "sound_enter κ gm en ⟹ sound_local_spec κ gm (c(|ls_enter := en|))"
    and sound_local_spec_update_combine_env:
      "sound_combine κ gm ce (ls_combine_assign c)
       ⟹ sound_local_spec κ gm (c(|ls_combine_env := ce|))"
    and sound_local_spec_update_combine_assign:
      "sound_combine κ gm (ls_combine_env c) ca
       ⟹ sound_local_spec κ gm (c(|ls_combine_assign := ca|))"
  using assms unfolding sound_local_spec_iff by simp_all
```

Conservative defaults

A field whose concrete counterpart leaves the store unchanged has a default that decides nothing: the handler answers $\top$, and skip, body, event and branch keep the state. Each is sound for every concretization. Assign, special, return, entry and the second return stage change the store or the frame, so no default is sound for every concretization and an analysis supplies them. The first return stage defaults to the caller's state; it has no law

of its own, only the composed return does.

```
definition query_unknown :: "channel ⇒ 's ⇒ channel" where
  "query_unknown ch x = ⊤"

definition skip_identity :: "channel ⇒ 's ⇒ 's" where
  "skip_identity ch x = x"

definition body_identity :: "channel ⇒ pname ⇒ 's ⇒ 's" where
  "body_identity ch p x = x"

definition event_identity :: "channel ⇒ analysis_event ⇒ 's ⇒ 's" where
  "event_identity ch ev x = x"

definition branch_identity :: "channel ⇒ exp ⇒ bool ⇒ 's ⇒ 's" where
  "branch_identity ch b pol x = x"

definition combine_env_identity :: "channel ⇒ channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's"
  where
  "combine_env_identity ch ch' ci x de = x"

lemma sound_query_unknown: "sound_query gm query_unknown"
  by (simp add: sound_query_def query_unknown_def)

lemma sound_skip_identity: "sound_skip gm skip_identity"
  by (simp add: sound_skip_def skip_identity_def)

lemma sound_body_identity: "sound_body gm body_identity"
  by (simp add: sound_body_def body_identity_def)

lemma sound_event_identity: "sound_event gm event_identity"
  by (simp add: sound_event_def event_identity_def)

lemma sound_branch_identity: "sound_branch gm branch_identity"
  by (simp add: sound_branch_def branch_identity_def)
```

A local specification that supplies only the five operations without a default. An analysis starts from it and overrides the fields it decides more precisely; each override needs only its own law again (`sound_local_spec_update`).

```
definition conservative_local_spec ::
  "(channel ⇒ var_id ⇒ exp ⇒ 's ⇒ 's)
   ⇒ (channel ⇒ exp gspecial_call ⇒ var_id ⇒ 's ⇒ 's)
   ⇒ (channel ⇒ exp option ⇒ pname ⇒ 's ⇒ 's)
   ⇒ (channel ⇒ call_info ⇒ 's × 's ⇒ ('s × 's) list)
   ⇒ (channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's) ⇒ 's local_spec" where
  "conservative_local_spec asn sp rt en ca = (|
     ls_query = query_unknown, ls_skip = skip_identity, ls_assign = asn,
     ls_special = sp, ls_branch = branch_identity, ls_body = body_identity,
     ls_return = rt, ls_event = event_identity, ls_enter = en,
     ls_combine_env = combine_env_identity, ls_combine_assign = ca |)"

theorem sound_conservative_local_spec:
  assumes "⋀x y. x ≤ y ⟹ gm x ⊆ gm y"
    and "sound_assign κ gm asn" and "sound_special κ gm sp" and "sound_return gm rt"
    and "sound_enter κ gm en" and "sound_combine κ gm combine_env_identity ca"
  shows "sound_local_spec κ gm (conservative_local_spec asn sp rt en ca)"
  by (rule sound_local_specI)
     (simp_all add: conservative_local_spec_def assms sound_query_unknown sound_skip_identity
        sound_body_identity sound_event_identity sound_branch_identity)
```

A component leaves a concretization alone when none of its operations changes what that concretization says. For components that each own one field of a record this is the lens law that updating one field leaves the others.

```
definition mcp_frame :: "'s local_spec ⇒ ('s ⇒ store set) ⇒ bool" where
  "mcp_frame c gm ⟷
     (∀ch a x. gm (ls_step c ch a x) = gm x)
     ∧ (∀ch ci p q. q ∈ set (ls_enter c ch ci p) ⟶
          gm (fst q) = gm (fst p) ∧ gm (snd q) = gm (snd p))
     ∧ (∀ch ch' ci x de. gm (ls_combine c ch ch' ci x de) = gm x)"
```

Cooperating analyses come as pairs of a concretization and a component. They are independent when each leaves the others' concretizations alone.

```
type_synonym 's certified_component = "('s ⇒ store set) × 's local_spec"

fun mcp_independent :: "'s certified_component list ⇒ bool" where
  "mcp_independent [] ⟷ True"
```

```
| "mcp_independent ((g, c) # gcs) ⟷
     (∀(g', c') ∈ set gcs. mcp_frame c g' ∧ mcp_frame c' g) ∧ mcp_independent gcs"
```

The query channel

A handler answers a query given the channel through which it may ask further ones. `ask_rec` closes that recursion over one state, as Goblint's `MCP.query'` does: a query already being asked answers $\top$ instead of recursing, and every other query is answered by the handler under a channel that remembers it. The depth bound plays the role it plays in `ask_with`: queries range over an infinite type, so cycle detection alone does not make the recursion total, and exhausting the bound aborts the generated program. Logically the fallback is $\top$, so the bound plays no part in soundness. Goblint also caches answers per transfer; that changes only how often a handler runs.

```
fun ask_rec :: "(channel ⇒ 's ⇒ channel) ⇒ nat ⇒ query set ⇒ 's ⇒ channel" where
  "ask_rec H 0 asked x q =
     Code.abort (STR ''query recursion exceeded query_depth'') (λ_. ⊤)"
| "ask_rec H (Suc n) asked x q =
     (if q ∈ asked then ⊤ else H (ask_rec H n (insert q asked) x) x q)"
```

A handler that answers soundly under every sound channel makes the closed channel sound: by induction on the depth, each level's channel holds, so the handler's answer under it holds.

```
lemma ask_rec_sound:
  assumes "⋀ch q. eval_query.channel_holds ch s ⟹ eval_holds q (H ch x q) s"
  shows "eval_query.channel_holds (ask_rec H n asked x) s"
proof (induction n arbitrary: asked)
  case 0
  then show ?case by (simp add: eval_query.channel_holds_def)
next
  case (Suc n)
  then show ?case
    using assms by (auto simp: eval_query.channel_holds_def)
qed

definition ls_channel :: "'s local_spec ⇒ 's ⇒ channel" where
  "ls_channel c x = ask_rec (ls_query c) query_depth {} x"

lemma ls_channel_sound:
  assumes "sound_local_spec κ γ_c c" and "s ∈ γ_c x"
  shows "eval_query.channel_holds (ls_channel c x) s"
  unfolding ls_channel_def
  by (rule ask_rec_sound) (use assms in ‹simp add: sound_local_spec_def›)

lemma ls_channel_const: "ls_query c = (λch x. h x) ⟹ ls_channel c = h"
  by (simp add: ls_channel_def query_depth_def fun_eq_iff eval_nat_numeral)
```

The combined operations

The combined operations run the active components in turn. Soundness needs only `mcp_frame`: a component keeps what the other components' states describe, not their representation, so the soundness proof says nothing about the order of the fold. A component that updates only its own field of a product does leave the other fields unchanged, and for those the order does not matter; Goblint runs every component on the predecessor's part of the state, which is the same computation. Every component gets the same channel, as every Goblint component gets the same `man.ask`.

```
definition mcp_step ::
  "'s local_spec list ⇒ channel ⇒ (var_id,exp) gedge_action ⇒ 's ⇒ 's" where
  "mcp_step cs ch a x = fold (λc y. ls_step c ch a y) cs x"

definition mcp_en_from ::
  "'s local_spec list ⇒ channel ⇒ call_info ⇒ 's × 's ⇒ 's enter_result list"
    where
  "mcp_en_from cs ch ci p = fold (λc ps. concat (map (ls_enter c ch ci) ps)) cs [p]"

definition mcp_comb ::
  "'s local_spec list ⇒ channel ⇒ channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's"
    where
  "mcp_comb cs ch ch' ci x de = fold (λc y. ls_combine c ch ch' ci y de) cs x"
```

The combined handler meets every component's answer, each given the same channel, as `MCP.query'` meets the results of all analyses.

```
definition mcp_qry :: "'s local_spec list ⇒ channel ⇒ 's ⇒ channel" where
  "mcp_qry cs ch x q = fold (λc r. r ⊓ ls_query c ch x q) cs ⊤"
```

```
definition mcp_gamma :: "('s ⇒ store set) list ⇒ 's ⇒ store set" where
  "mcp_gamma Γ x = (⋂γi ∈ set Γ. γi x)"
```

Soundness of the combination

The combined state is sound for the intersection `mcp_gamma` of the components' concretizations. Step, combine, entry and query folds each stay sound because a component leaves the others' concretizations unchanged.

```
lemma mcp_gamma_Cons [simp]: "mcp_gamma (g # gs) x = g x ∩ mcp_gamma gs x"
  by (simp add: mcp_gamma_def)

lemma mcp_gamma_Nil [simp]: "mcp_gamma [] x = UNIV"
  by (simp add: mcp_gamma_def)
```

Components a fold runs leave an independent concretization as it was.

```
lemma gamma_fold_step:
  "(∀c ∈ set cs. mcp_frame c g) ⟹ g (fold (λc y. ls_step c ch a y) cs x) = g x"
  by (induction cs arbitrary: x) (simp_all add: mcp_frame_def)

lemma gamma_fold_comb:
  "(∀c ∈ set cs. mcp_frame c g) ⟹ g (fold (λc y. ls_combine c ch ch' ci y de) cs x) = g x"
  by (induction cs arbitrary: x) (simp_all add: mcp_frame_def)

lemma mcp_gamma_frame:
  "(∀g ∈ set gs. mcp_frame c g) ⟹ mcp_gamma gs (ls_step c ch a x) = mcp_gamma gs x"
  "(∀g ∈ set gs. mcp_frame c g) ⟹ mcp_gamma gs (ls_combine c ch ch' ci x de) = mcp_gamma gs x"
  by (auto simp: mcp_gamma_def mcp_frame_def intro!: INF_cong)

lemma mcp_independent_Cons_frames:
  assumes "mcp_independent ((g, c) # gcs)"
  shows "∀g' ∈ fst ` set gcs. mcp_frame c g'" "∀c' ∈ snd ` set gcs. mcp_frame c' g"
    "mcp_independent gcs"
  using assms by auto

lemma mcp_step_sound:
  assumes "∀(g, c) ∈ set gcs. sound_local_spec κ g c" and "mcp_independent gcs"
  shows "edge_collect κ a (mcp_gamma (map fst gcs) x ∩ Collect (eval_query.channel_holds ch))
          ⊆ mcp_gamma (map fst gcs) (mcp_step (map snd gcs) ch a x)"
  using assms
proof (induction gcs arbitrary: x)
  case Nil
  then show ?case by (simp add: mcp_step_def)
next
  case (Cons gc gcs)
  obtain g c where gc: "gc = (g, c)" by fastforce
  let ?O = "Collect (eval_query.channel_holds ch)"
  let ?x1 = "ls_step c ch a x"
  have frames: "∀g' ∈ fst ` set gcs. mcp_frame c g'" "∀c' ∈ snd ` set gcs. mcp_frame c' g"
    "mcp_independent gcs"
    using mcp_independent_Cons_frames[OF Cons.prems(2)[unfolded gc]] by simp_all
  have own: "edge_collect κ a (g x ∩ ?O) ⊆ g ?x1"
    using Cons.prems(1) unfolding gc by (simp add: sound_local_spec_def)
  have own_kept: "g (mcp_step (map snd gcs) ch a ?x1) = g ?x1"
    unfolding mcp_step_def by (rule gamma_fold_step) (use frames in auto)
  have rest: "edge_collect κ a (mcp_gamma (map fst gcs) ?x1 ∩ ?O)
      ⊆ mcp_gamma (map fst gcs) (mcp_step (map snd gcs) ch a ?x1)"
    by (rule Cons.IH) (use Cons.prems(1) frames in auto)
  have rest_eq: "mcp_gamma (map fst gcs) ?x1 = mcp_gamma (map fst gcs) x"
    by (rule mcp_gamma_frame(1)) (use frames in auto)
  have "edge_collect κ a (mcp_gamma (map fst (gc # gcs)) x ∩ ?O)
        ⊆ edge_collect κ a (g x ∩ ?O) ∩ edge_collect κ a (mcp_gamma (map fst gcs) ?x1 ∩ ?O)"
    unfolding gc by (intro Int_greatest edge_collect_mono) (auto simp: rest_eq)
  also have "... ⊆ mcp_gamma (map fst (gc # gcs)) (mcp_step (map snd (gc # gcs)) ch a x)"
    using own own_kept rest unfolding gc by (auto simp: mcp_step_def)
  finally show ?case .
qed

lemma mcp_comb_sound:
  assumes "∀(g, c) ∈ set gcs. sound_local_spec κ g c" and "mcp_independent gcs"
    and "s ∈ mcp_gamma (map fst gcs) x" and "eval_query.channel_holds ch s"
    and "t ∈ mcp_gamma (map fst gcs) de" and "eval_query.channel_holds ch' t"
  shows "combine_collect κ (ci_dst ci) s t
          ∈ mcp_gamma (map fst gcs) (mcp_comb (map snd gcs) ch ch' ci x de)"
  using assms
proof (induction gcs arbitrary: x)
```

```
  case Nil
  then show ?case by simp
next
  case (Cons gc gcs)
  obtain g c where gc: "gc = (g, c)" by fastforce
  let ?x1 = "ls_combine c ch ch' ci x de"
  have frames: "∀g' ∈ fst ` set gcs. mcp_frame c g'" "∀c' ∈ snd ` set gcs. mcp_frame c' g"
    "mcp_independent gcs"
    using mcp_independent_Cons_frames[OF Cons.prems(2)[unfolded gc]] by simp_all
  have own: "combine_collect κ (ci_dst ci) s t ∈ g ?x1"
    using Cons.prems(1,3-6) unfolding gc by (simp add: sound_local_spec_def)
  have own_kept: "g (mcp_comb (map snd gcs) ch ch' ci ?x1 de) = g ?x1"
    unfolding mcp_comb_def by (rule gamma_fold_comb) (use frames in auto)
  have rest_eq: "mcp_gamma (map fst gcs) ?x1 = mcp_gamma (map fst gcs) x"
    by (rule mcp_gamma_frame(2)) (use frames in auto)
  have rest: "combine_collect κ (ci_dst ci) s t
      ∈ mcp_gamma (map fst gcs) (mcp_comb (map snd gcs) ch ch' ci ?x1 de)"
    by (rule Cons.IH) (use Cons.prems frames rest_eq gc in auto)
  show ?case
    using own own_kept rest unfolding gc by (simp add: mcp_comb_def)
qed
```

Entry threads a list of pairs through the components. The invariant carries the concretizations of the components already run (`ds`), whose fields are set in both halves, and the components still to run (`gcs`), whose fields are still the caller's.

```
lemma mcp_en_fold_sound:
  assumes "∀(g, c) ∈ set gcs. sound_local_spec κ g c" and "mcp_independent gcs"
    and "∀c ∈ snd ` set gcs. ∀d ∈ set ds. mcp_frame c d"
    and "∃p ∈ set ps. s ∈ mcp_gamma (map fst gcs) (fst p) ∧ s ∈ mcp_gamma ds (fst p)
                    ∧ E ∈ mcp_gamma ds (snd p)"
    and A: "eval_query.channel_holds ch s"
    and E: "E = call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s"
  shows "∃p ∈ set (fold (λc ps. concat (map (ls_enter c ch ci) ps)) (map snd gcs) ps).
          s ∈ mcp_gamma (map fst gcs) (fst p) ∧ s ∈ mcp_gamma ds (fst p)
          ∧ E ∈ mcp_gamma (map fst gcs) (snd p) ∧ E ∈ mcp_gamma ds (snd p)"
  using assms(1-4)
proof (induction gcs arbitrary: ps ds)
  case Nil
  then show ?case by simp
next
  case (Cons gc gcs)
  obtain g c where gc: "gc = (g, c)" by fastforce
  have frames: "∀g' ∈ fst ` set gcs. mcp_frame c g'" "∀c' ∈ snd ` set gcs. mcp_frame c' g"
    "mcp_independent gcs"
    using mcp_independent_Cons_frames[OF Cons.prems(2)[unfolded gc]] by simp_all
  obtain p where p: "p ∈ set ps" "s ∈ g (fst p)" "s ∈ mcp_gamma (map fst gcs) (fst p)"
      "s ∈ mcp_gamma ds (fst p)" "E ∈ mcp_gamma ds (snd p)"
    using Cons.prems(4) unfolding gc by auto
  have "sound_local_spec κ g c" using Cons.prems(1) unfolding gc by simp
  then obtain q where q: "q ∈ set (ls_enter c ch ci p)" "s ∈ g (fst q)" "E ∈ g (snd q)"
    using p(2) A unfolding E sound_local_spec_def by meson
  have keep: "d (fst q) = d (fst p) ∧ d (snd q) = d (snd p)"
    if "d ∈ fst ` set gcs ∪ set ds" for d
  proof -
    have "mcp_frame c d" using that frames(1) Cons.prems(3) unfolding gc by auto
    then show ?thesis using q(1) unfolding mcp_frame_def by blast
  qed
  have q_in: "q ∈ set (concat (map (ls_enter c ch ci) ps))"
    using p(1) q(1) by auto
  have "∃p ∈ set (fold (λc ps. concat (map (ls_enter c ch ci) ps)) (map snd gcs)
                 (concat (map (ls_enter c ch ci) ps))).
          s ∈ mcp_gamma (map fst gcs) (fst p) ∧ s ∈ mcp_gamma (g # ds) (fst p)
          ∧ E ∈ mcp_gamma (map fst gcs) (snd p) ∧ E ∈ mcp_gamma (g # ds) (snd p)"
  proof (rule Cons.IH)
    show "∀(g, c) ∈ set gcs. sound_local_spec κ g c" using Cons.prems(1) by simp
    show "mcp_independent gcs" by (fact frames(3))
    show "∀c' ∈ snd ` set gcs. ∀d ∈ set (g # ds). mcp_frame c' d"
      using frames(2) Cons.prems(3) unfolding gc by auto
    show "∃p ∈ set (concat (map (ls_enter c ch ci) ps)).
            s ∈ mcp_gamma (map fst gcs) (fst p) ∧ s ∈ mcp_gamma (g # ds) (fst p)
            ∧ E ∈ mcp_gamma (g # ds) (snd p)"
    proof (rule bexI[OF _ q_in])
      show "s ∈ mcp_gamma (map fst gcs) (fst q) ∧ s ∈ mcp_gamma (g # ds) (fst q)
              ∧ E ∈ mcp_gamma (g # ds) (snd q)"
      proof -
        have "mcp_gamma (map fst gcs) (fst q) = mcp_gamma (map fst gcs) (fst p)"
```

```
          "mcp_gamma ds (fst q) = mcp_gamma ds (fst p)"
          "mcp_gamma ds (snd q) = mcp_gamma ds (snd p)"
        using keep by (auto simp: mcp_gamma_def intro!: INF_cong)
      then show ?thesis using p(3-5) q(2,3) by simp
    qed
  qed
  qed
  then show ?case unfolding gc by (auto simp: mcp_gamma_def)
qed

lemma mcp_qry_fold_sound:
  "s ∈ mcp_gamma (map fst gcs) x ⟹ eval_query.channel_holds ch s
   ⟹ ∀(g, c) ∈ set gcs. sound_local_spec κ g c
   ⟹ eval_holds q r s
   ⟹ eval_holds q (fold (λc r. r ⊓ ls_query c ch x q) (map snd gcs) r) s"
proof (induction gcs arbitrary: r)
  case (Cons gc gcs)
  obtain g c where gc: "gc = (g, c)" by fastforce
  have "eval_holds q (ls_query c ch x q) s"
    using Cons.prems unfolding gc by (simp add: sound_local_spec_def)
  with Cons.prems(4) have "eval_holds q (r ⊓ ls_query c ch x q) s"
    by (rule eval_query.inf_sound)
  then show ?case
    using Cons.IH[of "r ⊓ ls_query c ch x q"] Cons.prems(1-3) unfolding gc by simp
qed simp
```

A component as a specification

A component runs as the specification whose fields are its own operations, each given the component's closed channel on the state it is asked about. The specification reads no global and publishes none. Its channel is a pure function of the state, so a transfer asks it whatever it needs while it runs, as a Goblint transfer asks `man.ask`. Entry starts both halves at the caller's record.

The step a component takes on an edge, with its own channel.

```
definition closed_step :: "'s local_spec ⇒ (var_id,exp) gedge_action ⇒ 's ⇒ 's" where
  "closed_step c a d = ls_step c (ls_channel c d) a d"
```

$[\![\mathtt{a}]\!]^\sharp_{\mathtt{c}}\ \mathtt{d}$ is the abstract transfer of action `a`, the counterpart of the abstract evaluation $[\![\mathtt{e}]\!]^\sharp\ \mathtt{d}$; the subscript names the component.

```
abbreviation abs_step :: "(var_id,exp) gedge_action ⇒ 's local_spec ⇒ 's ⇒ 's"
  ("[\<_\]\<sharp>_") where
  "[\<a\]\<sharp>c ≡ closed_step c a"

definition dg_spec_of :: "'s local_spec ⇒ ('x, 'k, 'v, 's::bot, 'G) dg_spec" where
  "dg_spec_of c = local_dg_spec_template⦇
     dgs_skip := local_transfer [\<EA_Nop\]\<sharp>c,
     dgs_assign := (λx e. local_transfer [\<EA_Assign x e\]\<sharp>c),
     dgs_special := (λsc x. local_transfer [\<EA_Special sc x\]\<sharp>c),
     dgs_branch := (λb pol. local_transfer
                     [\<if pol then EA_Assume b else EA_AssumeNot b\]\<sharp>c),
     dgs_body := (λp. local_transfer [\<EA_Body p\]\<sharp>c),
     dgs_return := (λe p. local_transfer [\<EA_Ret e p\]\<sharp>c),
     dgs_enter := (λci. local_enter_transfer (λd. ls_enter c (ls_channel c d) ci (d, d))),
     dgs_event := (λev. local_transfer [\<event_action ev\]\<sharp>c),
     dgs_combine_env := (λci. local_combine_transfer
        (λdc de. ls_combine_env c (ls_channel c dc) (ls_channel c de) ci dc de)),
     dgs_combine_assign := (λci. local_combine_transfer
        (λdc de. ls_combine_assign c (ls_channel c de) ci dc de)),
     dgs_query := local_query (ls_channel c) ⦈"

lemma dg_spec_of_simps [simp]:
  "skip\<sharp> (dg_spec_of c) = local_transfer [\<EA_Nop\]\<sharp>c"
  "assign\<sharp> (dg_spec_of c) x e = local_transfer [\<EA_Assign x e\]\<sharp>c"
  "special\<sharp> (dg_spec_of c) sc x = local_transfer [\<EA_Special sc x\]\<sharp>c"
  "branch\<sharp> (dg_spec_of c) b pol
     = local_transfer [\<if pol then EA_Assume b else EA_AssumeNot b\]\<sharp>c"
  "body\<sharp> (dg_spec_of c) p = local_transfer [\<EA_Body p\]\<sharp>c"
  "return\<sharp> (dg_spec_of c) eo p = local_transfer [\<EA_Ret eo p\]\<sharp>c"
  "enter\<sharp> (dg_spec_of c) ci
     = local_enter_transfer (λd. ls_enter c (ls_channel c d) ci (d, d))"
  "event\<sharp> (dg_spec_of c) ev = local_transfer [\<event_action ev\]\<sharp>c"
  "combine_env\<sharp> (dg_spec_of c) ci = local_combine_transfer
     (λdc de. ls_combine_env c (ls_channel c dc) (ls_channel c de) ci dc de)"
  "combine_assign\<sharp> (dg_spec_of c) ci = local_combine_transfer
```

```
    (λdc de. ls_combine_assign c (ls_channel c de) ci dc de)"
  "dgs_query (dg_spec_of c) = local_query (ls_channel c)"
  by (simp_all add: dg_spec_of_def)

lemma dg_spec_step_dg_spec_of [simp]:
  "dg_spec_step (dg_spec_of c) a = local_transfer ⟦a⟧♯c"
  by (cases a) simp_all

lemma dg_spec_combine_transfer_dg_spec_of [simp]:
  "dg_spec_combine_transfer (dg_spec_of c) ci
     = local_combine_transfer (λdc de. ls_combine c (ls_channel c dc) (ls_channel c de) ci dc de)"
  by (intro ext) (simp add: dg_spec_combine_transfer_local local_combine_transfer_def)

lemma dg_spec_wf_dg_spec_of [intro, simp]: "dg_spec_wf (dg_spec_of c)"
  by (auto simp: dg_spec_wf_def local_query_def local_enter_transfer_def
      local_combine_transfer_def local_transfer_def)
```

Run as a specification, a sound component's step and return are sound outright: its own channel holds at every store its state describes, so the laws it proves against every holding channel apply to that one.

```
lemma closed_step_sound:
  assumes sound: "sound_local_spec κ gm c"
  shows "edge_collect κ a (gm d) ⊆ gm (⟦a⟧♯c d)"
proof -
  have "gm d = gm d ∩ Collect (eval_query.channel_holds (ls_channel c d))"
    using ls_channel_sound[OF sound] by blast
  moreover have "edge_collect κ a (gm d ∩ Collect (eval_query.channel_holds (ls_channel c d)))
      ⊆ gm (ls_step c (ls_channel c d) a d)"
    using sound unfolding sound_local_spec_def by blast
  ultimately show ?thesis by (simp add: closed_step_def)
qed

lemma closed_combine_sound:
  assumes sound: "sound_local_spec κ gm c" and "s ∈ gm dc" and "t ∈ gm de"
  shows "combine_collect κ (ci_dst ci) s t
           ∈ gm (ls_combine c (ls_channel c dc) (ls_channel c de) ci dc de)"
  using assms ls_channel_sound[OF sound assms(2)] ls_channel_sound[OF sound assms(3)]
  unfolding sound_local_spec_def by blast

theorem dg_spec_of_contract:
  assumes sound: "sound_local_spec κ gm c"
  shows "analysis_contract κ (dg_spec_of c) (λd g. gm d)"
proof (unfold_locales, goal_cases wf mono step comb)
  case wf
  then show ?case by (rule dg_spec_wf_dg_spec_of)
next
  case mono
  then show ?case using sound unfolding sound_local_spec_def by metis
next
  case (step a τ src gk)
  then show ?case
    using closed_step_sound[OF sound] by (simp add: dg_spec_edge_program_def)
next
  case comb
  then show ?case
    by (simp add: local_combine_transfer_def closed_combine_sound[OF sound])
qed
```

Several components as one

The combination of several components is itself a component. One component is its own combination, so a single analysis and an `MCP` of one analysis are the same specification. Several components run their returns one after the other, each through both of its stages: the combination's env stage runs every component's composite, and its assign stage keeps the state it is handed. Running every env stage before every assign stage would need the stages of different components to commute, which independence of their concretizations does not give. The composite needs the caller's channel, which only the env stage receives.

```
fun mcp_combine :: "'s local_spec list ⇒ 's local_spec" where
  "mcp_combine [c] = c"
| "mcp_combine cs =
     make_local_spec (mcp_qry cs) (mcp_step cs) (mcp_en_from cs) (mcp_comb cs) (λch' ci dc de. dc)"

definition mcp_spec :: "'s local_spec list ⇒ ('x, 'k, 'v, 's::bot, 'G) dg_spec" where
  "mcp_spec cs = dg_spec_of (mcp_combine cs)"
```

```
theorem mcp_combine_sound:
  assumes sound: "∀(γ_i, c_i) ∈ set comps. sound_local_spec κ γ_i c_i"
    and indep: "mcp_independent comps" and ne: "comps ≠ []"
  shows "sound_local_spec κ (mcp_gamma (map fst comps)) (mcp_combine (map snd comps))"
proof (cases "∃x. comps = [x]")
  case True
  then obtain γ_i c_i where "comps = [(γ_i, c_i)]" by fastforce
  then show ?thesis using sound by (simp add: sound_local_spec_def mcp_gamma_def)
next
  case False
  have comb: "mcp_combine (map snd comps) =
     make_local_spec (mcp_qry (map snd comps)) (mcp_step (map snd comps))
       (mcp_en_from (map snd comps)) (mcp_comb (map snd comps)) (λch' ci dc de. dc)"
    using False ne by (cases "map snd comps" rule: mcp_combine.cases) (auto simp: Cons_eq_map_conv)
  have enter: "∃q ∈ set (mcp_en_from (map snd comps) ch ci p).
                 s ∈ mcp_gamma (map fst comps) (fst q)
                 ∧ call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s
                     ∈ mcp_gamma (map fst comps) (snd q)"
    if "s ∈ mcp_gamma (map fst comps) (fst p)" "eval_query.channel_holds ch s" for ch s ci p
    using mcp_en_fold_sound[OF sound indep, of "[]" "[p]" s _ ch ci] that
    by (auto simp: mcp_en_from_def)
  have mono: "∀x y. x ≤ y ⟶ mcp_gamma (map fst comps) x ⊆ mcp_gamma (map fst comps) y"
    using sound by (fastforce simp: mcp_gamma_def sound_local_spec_def)
  have qry: "∀ch s x q. s ∈ mcp_gamma (map fst comps) x ⟶ eval_query.channel_holds ch s
               ⟶ eval_holds q (mcp_qry (map snd comps) ch x q) s"
    unfolding mcp_qry_def using mcp_qry_fold_sound[OF _ _ sound] by simp
  show ?thesis
    unfolding comb sound_local_spec_def
    using mono mcp_step_sound[OF sound indep] mcp_comb_sound[OF sound indep] enter qry
    by simp
qed

theorem mcp_contract:
  assumes "∀(γ_i, c_i) ∈ set comps. sound_local_spec κ γ_i c_i" and "mcp_independent comps"
    and "comps ≠ []"
  shows
    "analysis_contract κ (mcp_spec (map snd comps)) (λd g. mcp_gamma (map fst comps) d)"
  unfolding mcp_spec_def by (rule dg_spec_of_contract[OF mcp_combine_sound[OF assms]])
```

A component on one field of a larger state

A component over its own carrier `'c` runs on one field of the combined state through a lens: every operation reads the field, and writes back only the field, as Goblint's `inner_man` hands a component its own part of the `MCP` state. The channel passes through unchanged: it describes the stores of the whole state, which are the stores the field describes as well.

```
definition lens_of ::
  "('s ⇒ 'c) ⇒ ('s ⇒ 'c ⇒ 's) ⇒ 'c local_spec ⇒ 's local_spec"
where
  "lens_of get put c = make_local_spec
     (λch x. ls_query c ch (get x))
     (λch a x. put x (ls_step c ch a (get x)))
     (λch ci p. map (λ(q, e). (put (fst p) q, put (snd p) e))
                (ls_enter c ch ci (get (fst p), get (snd p))))
     (λch ch' ci x de. put x (ls_combine_env c ch ch' ci (get x) (get de)))
     (λch' ci x de. put x (ls_combine_assign c ch' ci (get x) (get de)))"

lemma get_mc_comb_lens_of:
  assumes "⋀x v. get (put x v) = v"
  shows
    "get (ls_combine (lens_of get put c) ch ch' ci x de) = ls_combine c ch ch' ci (get x) (get de)"
  by (simp add: lens_of_def assms)

theorem lens_of_sound:
  assumes sound: "sound_local_spec κ g c"
    and get_put: "⋀x v. get (put x v) = v"
    and get_mono: "⋀x y. x ≤ y ⟹ get x ≤ get y"
  shows "sound_local_spec κ (λx. g (get x)) (lens_of get put c)"
proof -
  have enter: "∃q ∈ set (ls_enter (lens_of get put c) ch ci p).
                 s ∈ g (get (fst q))
                 ∧ call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s
                     ∈ g (get (snd q))"
    if s_in: "s ∈ g (get (fst p))" and A: "eval_query.channel_holds ch s" for ch s ci p
  proof -
    have "s ∈ g (fst (get (fst p), get (snd p)))" using s_in by simp
```

```
    then obtain q where "q ∈ set (ls_enter c ch ci (get (fst p), get (snd p)))" "s ∈ g (fst q)"
        "call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s ∈ g (snd q)"
      using sound A unfolding sound_local_spec_def by metis
    then show ?thesis
      by (intro bexI[of _ "(put (fst p) (fst q), put (snd p) (snd q))"])
         (auto simp: lens_of_def get_put)
  qed
  show ?thesis
    using sound get_mono enter
    unfolding sound_local_spec_def get_mc_comb_lens_of[of get put, OF get_put]
    by (simp add: lens_of_def get_put)
qed
```

Replacing a component's handler

The handler is the only operation that neither changes a state nor is framed, so any other handler that answers soundly for the same states may replace it. This is how an analysis written without a query handler becomes a provider once it runs in the combined state.

```
definition with_qry ::
  "(channel ⇒ 's ⇒ channel) ⇒ 's local_spec ⇒ 's local_spec" where
  "with_qry h c = c(|ls_query := h|)"

theorem with_qry_sound:
  assumes "sound_local_spec κ gm c"
    and "⋀ch s x q. s ∈ gm x ⟹ eval_query.channel_holds ch s ⟹ eval_holds q (h ch x q) s"
  shows "sound_local_spec κ gm (with_qry h c)"
  using assms unfolding sound_local_spec_def with_qry_def by simp

lemma with_qry_frame: "mcp_frame c g ⟹ mcp_frame (with_qry h c) g"
  by (simp add: mcp_frame_def with_qry_def)
```

Normalizing what a component produces

A normalization `k` that keeps a state's concretization may be applied to what a component produces: each step, the entered half of each entry, and the return after its second stage. The caller half of an entry is the caller's state and stays as it is. The first stage of a return is left alone, because the component's soundness speaks only about the composite. The combined state of several analyses uses this to become unreachable as soon as one active analysis is, as Goblint's `MCP` raises `Deadcode`.

```
definition map_local_spec :: "('s ⇒ 's) ⇒ 's local_spec ⇒ 's local_spec" where
  "map_local_spec k c = make_local_spec (ls_query c)
     (λch a x. k (ls_step c ch a x))
     (λch ci p. map (λ(q, e). (q, k e)) (ls_enter c ch ci p))
     (ls_combine_env c)
     (λch' ci x de. k (ls_combine_assign c ch' ci x de))"

theorem map_local_spec_sound:
  assumes sound: "sound_local_spec κ g c"
    and keep: "⋀x. g (k x) = g x"
  shows "sound_local_spec κ g (map_local_spec k c)"
proof -
  have enter: "∃q ∈ set (ls_enter (map_local_spec k c) ch ci p).
                 s ∈ g (fst q)
                 ∧ call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s ∈ g (snd q)"
    if s_in: "s ∈ g (fst p)" and A: "eval_query.channel_holds ch s" for ch s ci p
  proof -
    obtain q where "q ∈ set (ls_enter c ch ci p)" "s ∈ g (fst q)"
        "call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s ∈ g (snd q)"
      using sound s_in A unfolding sound_local_spec_def by metis
    then show ?thesis
      by (intro bexI[of _ "(fst q, k (snd q))"]) (auto simp: map_local_spec_def keep)
  qed
  show ?thesis
    using sound enter unfolding sound_local_spec_def
    by (simp add: map_local_spec_def keep)
qed
```

Components whose entry answers once

An entry that answers a single alternative, paired with the caller's own state, is what the routed pipeline consumes. Lenses, combination and normalization keep that shape.

```
definition single_entry :: "'s local_spec ⇒ bool" where
```

```
  "single_entry c ⟷ (∀ch ci p. ∃e. ls_enter c ch ci p = [(fst p, e)])"

lemma single_entryD:
  "single_entry c ⟹ ls_enter c ch ci p = [(fst p, snd (hd (ls_enter c ch ci p)))]"
  unfolding single_entry_def by (metis list.sel(1) snd_conv)

lemma single_entry_lens_of:
  assumes single: "single_entry c" and put_get: "⋀x. put x (get x) = x"
  shows "single_entry (lens_of get put c)"
  unfolding single_entry_def
proof (intro allI)
  fix ch ci p
  obtain e where "ls_enter c ch ci (get (fst p), get (snd p)) = [(get (fst p), e)]"
    using single unfolding single_entry_def by (metis fst_conv)
  then show "∃e. ls_enter (lens_of get put c) ch ci p = [(fst p, e)]"
    by (simp add: lens_of_def put_get)
qed

lemma single_entry_map_local_spec:
  assumes "single_entry c"
  shows "single_entry (map_local_spec k c)"
  unfolding single_entry_def
proof (intro allI)
  fix ch ci p
  obtain e where "ls_enter c ch ci p = [(fst p, e)]"
    using assms unfolding single_entry_def by blast
  then show "∃e. ls_enter (map_local_spec k c) ch ci p = [(fst p, e)]"
    by (simp add: map_local_spec_def)
qed

lemma single_entry_with_qry: "single_entry c ⟹ single_entry (with_qry h c)"
  by (simp add: single_entry_def with_qry_def)

lemma single_entry_mcp_en_fold:
  assumes "∀c ∈ set cs. single_entry c"
  shows "∃e. fold (λc ps. concat (map (ls_enter c ch ci) ps)) cs [(d, e0)] = [(d, e)]"
  using assms
proof (induction cs arbitrary: e0)
  case (Cons c cs)
  obtain e1 where "ls_enter c ch ci (d, e0) = [(d, e1)]"
    using Cons.prems unfolding single_entry_def by (metis fst_conv list.set_intros(1))
  then show ?case using Cons.IH[of e1] Cons.prems by simp
qed simp

lemma single_entry_mcp_combine:
  assumes single: "∀c ∈ set cs. single_entry c" and ne: "cs ≠ []"
  shows "single_entry (mcp_combine cs)"
proof (cases "∃c. cs = [c]")
  case True
  then show ?thesis using single by auto
next
  case False
  then have "ls_enter (mcp_combine cs) = mcp_en_from cs"
    using ne by (cases cs rule: mcp_combine.cases) auto
  then show ?thesis
    unfolding single_entry_def mcp_en_from_def
    using single_entry_mcp_en_fold[OF single] by (metis prod.collapse)
qed
```

One field of a reachability-lifted record

The combined state is a record of fields under one reachability lift, whose `Bot` is the state no analysis can reach. Each field has a bottom of its own: a reachability-lifted field's is `Bot`, and a carrier with an unreachable element of its own uses that. A field's lens reads the field's bottom from the unreachable state, and writing any other value into it starts from the record whose fields are all at bottom. Writing bottom into the unreachable state leaves it unreachable, so writing back what was read changes nothing.

```
definition lift_get :: "('r ⇒ 'c::bot) ⇒ 'r lifted ⇒ 'c" where
  "lift_get f x = (case x of Bot ⇒ ⊥ | Lifted r ⇒ f r)"

definition lift_put :: "('r ⇒ 'c::bot ⇒ 'r) ⇒ 'r::bot lifted ⇒ 'c ⇒ 'r lifted" where
  "lift_put u x v =
     (case x of
        Bot ⇒ (if v = ⊥ then Bot else Lifted (u ⊥ v))
      | Lifted r ⇒ Lifted (u r v))"
```

```
lemma lift_get_simps [simp]:
  "lift_get f Bot = ⊥" "lift_get f (Lifted r) = f r"
  by (simp_all add: lift_get_def)

lemma lift_get_put:
  assumes "⋀r v. f (u r v) = v" and "f ⊥ = ⊥"
  shows "lift_get f (lift_put u x v) = v"
  using assms by (cases x) (simp_all add: lift_put_def)

lemma lift_put_get:
  assumes "⋀r. u r (f r) = r"
  shows "lift_put u x (lift_get f x) = x"
  using assms by (cases x) (simp_all add: lift_put_def)

lemma lift_get_mono:
  fixes f :: "'r::semilattice_sup ⇒ 'c::order_bot"
  assumes "⋀r r'. r ≤ r' ⟹ f r ≤ f r'"
  shows "x ≤ y ⟹ lift_get f x ≤ lift_get f y"
  using assms by (cases x; cases y) simp_all
```

Components that each own a distinct field are independent: a list of distinct analyses, each run on its own field, satisfies `mcp_independent` as soon as every one leaves every other field alone.

```
lemma mcp_independent_map:
  assumes "distinct as"
    and "⋀a b. a ∈ set as ⟹ b ∈ set as ⟹ a ≠ b ⟹ mcp_frame (c a) (g b)"
  shows "mcp_independent (map (λa. (g a, c a)) as)"
  using assms by (induction as) auto

end
```

6.20 Call_String_Context

```
theory Call_String_Context
  imports "Voblint_CFG.CFG_Def"
begin
```

A reusable bounded call-string context

The call-string route/enter-context pair: this theory owns only the call-string *data* and the one closed-term equality that makes `routed_context`'s routing agreement trivial for any bound `k` — it deliberately does not import `Routed_Context` or `DG_Ctx_Activation`, and fixes no domain, no solver, and no CFG. `cs_route`/`cs_context` (defined below) plug into `routed_context`'s `route`/`enterc` parameters at whatever concrete instantiation a caller chooses; nothing here decides what that instantiation is.

The call-string type and its two projections

Most recent call site first, unbounded as a type — `k` only bounds the *values* `cs_route`/`cs_context` produce, not the type itself. This mirrors Goblint's `C.t` being a value-level choice under a type-level bound only because OCaml forces the bound to be a type; here `k` is ordinary `nat` data.

```
type_synonym call_string = "cfg_node list"
```

The equation-level route: push the call site, keep only the most recent `k`. Ignores the entered abstract/executable value `d` entirely, so it is representation-independent for *any* domain without a per-domain commute lemma (`cs_route_indep_of_data` below).

```
definition cs_route :: "nat ⇒ pp ⇒ call_string ⇒ 'd ⇒ ('v, exp) gcall_action ⇒ call_string" where
  "cs_route k u ctx d ca = take k (u # ctx)"
```

The trace-semantic context function whose graph, `context_policy_of_fun (cs_context k)`, instantiates `routed_context`'s `adm`: same closed term as `cs_route`, over the concrete store `'s` the activation-trace semantics supplies instead of an abstract/executable `'d`.

```
definition cs_context :: "nat ⇒ cfg_node ⇒ call_string ⇒ 's ⇒ call_string" where
  "cs_context k u ctx s = take k (u # ctx)"
```

The two facts every instance needs

Routing agreement, generically: since neither side reads `d`/`s`, this holds for *any* `k`, `u`, `ctx`, `d`, `ca`, `s` with zero case analysis.

```
lemma cs_route_context_agree: "cs_route k u ctx d ca = cs_context k u ctx s"
  by (simp add: cs_route_def cs_context_def)
```

Representation independence: `cs_route` agrees with itself under *any* substitution for its data argument, since `d` never appears on the right-hand side. A caller needing `cs_route k u ctx (d::'exec) ca = cs_route k u ctx (repr d) ca` for a specific representation map `repr` – the map from an executable state to its function, say – gets it as an instance of this lemma, or just as directly by unfolding `cs_route_def` at the call site — both are one line.

```
lemma cs_route_indep_of_data: "cs_route k u ctx d ca = cs_route k u ctx d' ca"
  by (simp add: cs_route_def)
```

Bounded length

The one fact that makes this context space finite: routing truncates, so no context a `k`-bounded run produces is longer than `k`. Over a graph with finitely many nodes that bounds the whole space before any solve is attempted, rather than per run.

```
lemma cs_route_length: "length (cs_route k u ctx d ca) ≤ k"
  by (simp add: cs_route_def)
```

Truncation behaviour

Below the bound, pushing a call site loses nothing — a plain cons, not a truncation. Needed for any future precision argument: contexts stay distinct as long as the call histories being separated fit under `k`.

```
lemma cs_route_below_bound:
  "length ctx < k ⟹ cs_route k u ctx d ca = u # ctx"
  by (simp add: cs_route_def)
```

```
lemma cs_context_below_bound:
  "length ctx < k ⟹ cs_context k u ctx s = u # ctx"
  by (simp add: cs_context_def)
```

The central algebraic property of a bounded call string: projecting a longer bound's context down to a shorter one agrees with routing at the shorter bound directly. This is the `take k1 ctx_k2 = ctx_k1` fact any future `k1 <= k2` refinement theorem needs to relate the two analyses' unknown spaces.

```
lemma cs_route_k_mono:
  assumes "k1 ≤ k2"
  shows "take k1 (cs_route k2 u ctx d ca) = cs_route k1 u ctx d ca"
  using assms by (simp add: cs_route_def)
```

The call-string-keyed global-key type

The global-key space a call-string-routed equation system publishes into: one `Global` slot for the real globals, plus one `Seed` slot per (procedure entry, call-string) pair for the routed callee entry states. The `Seed` payload carries the call string itself, so the key shape does not depend on the bound `k` — only the payload's length does. Still pure data: no domain, no solver, no CFG.

```
datatype call_string_gk = Global | Seed (seed_pp: pp) (seed_cs: "cfg_node list")
```

end

6.21 Solved_Table

```
theory Solved_Table
  imports "Voblint_Domain.Nonrelational_Reachability" "Voblint_CFG.CFG_Def"
begin
```

Solved analysis results

The canonical, domain-generic shape of a finished analysis: a set of covered `pp × 'ctx` keys together with a total lookup into a per-point abstract state. Checks, reports, and rendering are downstream consumers of this table, not siblings of it.

The datatype constrains neither the key set nor the payloads. What the rest of this text describes — finitely many keys, and canonical payloads — is `wf_solved_table`, stated at the end of this theory. Its two halves are not established alike: an adapter canonicalizes every payload it publishes, so canonicality is unconditional, while finiteness holds only for those context policies whose whole context space is bounded in advance, and is carried as a hypothesis everywhere else.

`Bot` means "the solver's canonical result at this key is an outer `Bot`", or the key is absent from `covered_keys` altogether; `Lifted a` means "the canonical result is `Lifted`", with the local unknown projected to the abstract state `a`. This is a structural reading, not a semantic-emptiness test: raw solver values are canonicalized – a witness-bottom `Lifted` payload collapsed to `Bot` – *before* they reach this boundary (every public result adapter routes through `canonicalize_lift`), so by the time a value reaches `lookup_table` here, `Bot` denotes no store. The converse does not hold: the emptiness test is sound and need not be complete, so a `Lifted` state may still denote no store (a product whose interval field says `x = 2` and whose parity field says `x` is odd stays `Lifted`).

Per-point reachability

Per-point reachability is `'a lifted` itself (`Voblint_Domain.Reachability_Lift`), not a nominal copy of it: the reachability reading is fixed below rather than left to the caller, and stated directly in terms of `Bot`/ `Lifted`. `map_lift` is the functorial map (payload rewriting that leaves reachability alone).

The result table

`covered_keys` is the authoritative coverage surface — the keys the solver actually reached — while `table_at` is merely a total lookup function, defined everywhere but meaningful only on the key set. `lookup_table` keeps that distinction explicit rather than trusting `table_at` to answer sensibly off the key set.

`'ctx` carries no ordering or enumeration constraint: real context types include interval-vector and call-string contexts, which have neither.

```
datatype (plugins del: quickcheck_narrowing) ('ctx, 'a) solved_table =
  Solved_Table
    (covered_keys: "(pp × 'ctx) set")
    (table_at: "pp ⇒ 'ctx ⇒ 'a lifted")

definition table_contexts :: "('ctx, 'a) solved_table ⇒ pp ⇒ 'ctx set" where
  "table_contexts r v = snd ‘ Set.filter (λ(v', ctx). v' = v) (covered_keys r)"

definition lookup_table ::
  "('ctx, 'a) solved_table ⇒ pp ⇒ 'ctx ⇒ 'a lifted" where
  "lookup_table r v ctx =
     (if (v, ctx) ∈ covered_keys r then table_at r v ctx else Bot)"

lemma table_contexts_iff [simp]: "ctx ∈ table_contexts r v ⟷ (v, ctx) ∈ covered_keys r"
  unfolding table_contexts_def by force

lemma lookup_table_absent [simp]:
  "(v, ctx) ∉ covered_keys r ⟹ lookup_table r v ctx = Bot"
  unfolding lookup_table_def by simp
```

Telling "analysed and dead" from "never analysed"

`lookup_table` answers `Bot` for two unrelated reasons: the solver covered the key and stored `Bot` there, or it never covered the key at all. The first is a proved statement about the program; the second is the absence of one. A reader of the answer alone cannot tell which happened, so a claim about concrete unreachability must not be stated over it.

```
datatype (plugins del: quickcheck_narrowing) 'a context_result =
    Uncovered
```

```
  | Covered "'a lifted"
```

`'a context_result` keeps the two apart at the lookup, rather than pushing the distinction into `'a lifted`: making non-coverage a lattice element would put solver bookkeeping into the abstract domain and raise questions – how it orders against `Bot`, what a join with it means – that have no semantic answer. Coverage is metadata about a solve; `Bot` is a statement about a program.

```
definition lookup_coverage ::
  "('ctx, 'a) solved_table ⇒ pp ⇒ 'ctx ⇒ 'a context_result" where
  "lookup_coverage r v ctx =
     (if (v, ctx) ∈ covered_keys r then Covered (table_at r v ctx) else Uncovered)"

lemma lookup_coverage_covered [simp]:
  "(v, ctx) ∈ covered_keys r ⟹
     lookup_coverage r v ctx = Covered (table_at r v ctx)"
  unfolding lookup_coverage_def by simp

lemma lookup_coverage_uncovered [simp]:
  "(v, ctx) ∉ covered_keys r ⟹ lookup_coverage r v ctx = Uncovered"
  unfolding lookup_coverage_def by simp

lemma lookup_coverage_eq_Uncovered_iff:
  "lookup_coverage r v ctx = Uncovered ⟷ (v, ctx) ∉ covered_keys r"
  unfolding lookup_coverage_def by simp

lemma lookup_coverage_eq_Covered_iff:
  "lookup_coverage r v ctx = Covered x ⟷
     (v, ctx) ∈ covered_keys r ∧ table_at r v ctx = x"
  unfolding lookup_coverage_def by auto
```

Where the information is lost, named so that losing it is a visible step. `lookup_table` is exactly this totalization, which is why it is safe to keep for callers that only ever read a covered key – and why a soundness claim may not be stated over it directly.

```
fun context_result_or_bot :: "'a context_result ⇒ 'a lifted" where
  "context_result_or_bot Uncovered = Bot"
| "context_result_or_bot (Covered x) = x"

lemma lookup_table_eq_or_bot:
  "lookup_table r v ctx =
     context_result_or_bot (lookup_coverage r v ctx)"
  unfolding lookup_table_def lookup_coverage_def by simp
```

The logical core of every "the reported point is unreachable" claim, stated once here rather than per domain. It starts from `Covered Bot`, so an uncovered node cannot satisfy it however the caller obtained its inclusion.

```
lemma reported_covered_unreachable_empty:
  assumes lookup: "lookup_coverage r v ctx = Covered Bot"
    and sound: "C ⊆ gamma_lift gam (lookup_table r v ctx)"
  shows "C = {}"
  using sound unfolding lookup_table_eq_or_bot lookup by simp
```

The node-level reading, quantifying over the contexts at `v`. It asks only that every *stored* result there is `Bot`; an uncovered context is permitted rather than required to be absent, since whatever an uncovered context contributes is a question for the solve that produced the table, not for this predicate. A single `Covered (Lifted s)` at any context defeats it, which is the whole point: that context may still carry executions.

Read the vacuous case before using this: a node with *no* stored context satisfies it, since there is nothing to be non-`Bot`. On its own that would be a hole — "never analysed" passing as "proved dead". It is not one only where a caller separately knows that an unstored context contributes nothing, which is a property of the solve, not of this table. Do not lift this predicate out of a setting that establishes it.

```
definition result_node_is_bottom ::
  "('ctx, 'a) solved_table ⇒ pp ⇒ bool" where
  "result_node_is_bottom r v ⟷
     (∀ctx a. lookup_coverage r v ctx = Covered a ⟶ a = Bot)"

lemma result_node_is_bottomI:
  assumes "⋀ctx a. lookup_coverage r v ctx = Covered a ⟹ a = Bot"
  shows "result_node_is_bottom r v"
  unfolding result_node_is_bottom_def using assms by blast

lemma result_node_is_bottomD:
  assumes bot: "result_node_is_bottom r v"
    and covered: "(v, ctx) ∈ covered_keys r"
```

```
  shows "lookup_coverage r v ctx = Covered Bot"
proof -
  have eq: "lookup_coverage r v ctx = Covered (table_at r v ctx)"
    using covered by simp
  have "table_at r v ctx = Bot"
    using bot eq unfolding result_node_is_bottom_def by blast
  with eq show ?thesis by simp
qed

lemma lookup_table_LiftedD [dest]:
  "lookup_table r v ctx = Lifted st ⟹ ctx ∈ table_contexts r v"
  using lookup_table_absent by fastforce
```

Well-formed results

`Solved_Table` is an ordinary constructor, so the two properties the opening text describes are not enforced by the type and have to be stated.

Finiteness is the load-bearing one, and its failure is silent rather than loud: a per-node aggregate over `table_contexts` folds with `Finite_Set.fold`, which answers with its unit on an infinite carrier. The contextual check report's verdict aggregate is one such fold: over an infinite key set it reports a check dead whatever its contexts say.

Canonicality is what licenses reading `Bot` as concrete emptiness rather than as the solver's own structural answer. `Lifted` gets no such reading: the supplied emptiness predicate is sound, not complete. It is stated against a supplied emptiness predicate, the same way the normalization operations upstream are, so this layer keeps its freedom from any class constraint on the payload.

```
definition finite_solved_table :: "('ctx, 'a) solved_table ⇒ bool" where
  "finite_solved_table r ⟷ finite (covered_keys r)"

definition wf_solved_table ::
  "('a ⇒ bool) ⇒ ('ctx, 'a) solved_table ⇒ bool" where
  "wf_solved_table empty_pred r ⟷
     finite_solved_table r
   ∧ (∀v ctx st. lookup_table r v ctx = Lifted st ⟶ ¬ empty_pred st)"

lemma wf_solved_table_finite [dest]:
  "wf_solved_table empty_pred r ⟹ finite_solved_table r"
  unfolding wf_solved_table_def by simp

lemma wf_solved_table_LiftedD [dest]:
  "⟦wf_solved_table empty_pred r; lookup_table r v ctx = Lifted st⟧
     ⟹ ¬ empty_pred st"
  unfolding wf_solved_table_def by blast

end
```

6.22 Check_Result

```
theory Check_Result
  imports Main
begin
```

A flat three-valued check verdict

`check_result` as a flat join-semilattice: `Check_Unknown` is the top element, `Check_Proved`/`Check_Refuted` are incomparable, and their join is `Check_Unknown`. The canonical use is aggregating a check's verdict across several independently-sound sources of evidence (e.g. one abstract state per reachable calling context) without joining the underlying abstract states first: two sources agreeing stay that verdict, any disagreement collapses to `Check_Unknown` rather than asserting a verdict neither source alone established.

```
datatype check_result = Check_Proved | Check_Refuted | Check_Unknown

instantiation check_result :: semilattice_sup
begin

definition less_eq_check_result :: "check_result ⇒ check_result ⇒ bool" where
  "less_eq_check_result x y ⟷ x = y ∨ y = Check_Unknown"

definition less_check_result :: "check_result ⇒ check_result ⇒ bool" where
  "less_check_result x y ⟷ x ≤ y ∧ ¬ y ≤ x"

definition sup_check_result :: "check_result ⇒ check_result ⇒ check_result" where
  "sup_check_result x y = (if x = y then x else Check_Unknown)"

instance
  by standard
     (auto simp: less_eq_check_result_def less_check_result_def sup_check_result_def)

end
```

There is no dual bottom: `Check_Proved` and `Check_Refuted` have no common lower verdict, and the absence of any evidence at all is expressed one level up, by `check_result lifted`, not inside this type.

```
instantiation check_result :: top
begin

definition top_check_result :: check_result where
  "top_check_result = Check_Unknown"

instance ..

end

instance check_result :: order_top
  by standard (simp add: less_eq_check_result_def top_check_result_def)

end
```

6.23 Check_Report

```
theory Check_Report
  imports Check_Result CFG_Enumeration
begin
```

Whole-program executable check report

A single, domain-generic report over every compiled `EA_Check` edge: one entry per check, at its own source node, classified by whatever `classify` function a domain supplies. Entries are read directly off `intra` through the same deterministic order `cfg_intra_list` already gives the TD bridge, rather than sorting `pp` or expression values by hand. The compiled edges are the only record of where a check sits and what it says, so this report is a view of that one representation, not a parallel table.

This layer is deliberately generic in `classify`: it depends on no specific domain, only on a plain `'e ⇒ 's ⇒ check_result` function and its own soundness obligations below.

```
definition classify_checks ::
    "('vr::linorder, 'e::linorder) gcfg ⇒ (pp ⇒ 's) ⇒ ('e ⇒ 's ⇒ check_result)
       ⇒ (pp × 'e × check_result) list" where
  "classify_checks g env classify =
     map (λ(u, a, v). (u, ea_check_cond a, classify (ea_check_cond a) (env u)))
       (filter (λ(u, a, v). is_EA_Check a) (cfg_intra_list g))"
```

Membership unfolds to an `EA_Check` edge at the entry's own source node, classified there — the shape every soundness and worked-example use of `classify_checks` goes through.

```
lemma classify_checks_mem_iff:
  assumes "finite (intra g)"
  shows "(v, c, r) ∈ set (classify_checks g env classify)
     ⟷ (∃l tgt. (v, EA_Check l c, tgt) ∈ intra g) ∧ r = classify c (env v)"
  unfolding classify_checks_def set_map set_filter
  using set_cfg_intra_list[OF assms]
  by (auto simp: image_iff split: gedge_action.splits)
     (metis gedge_action.collapse surjective_pairing)
```

Soundness bridge: a `Check_Proved` report entry's condition genuinely holds at every reaching store, and a `Check_Refuted` entry's condition genuinely fails — given the same two facts every per-domain instance already proves once: a proved/refuted soundness obligation for the domain's own `classify`, and node-local collecting soundness (`reach v ≤ γ_S (env v)`) for the checked node. Neither theorem invents new domain reasoning; both only relocate an existing per-node fact to every entry of the whole-program report. `Check_Unknown` gets no counterpart.

```
theorem classify_checks_proved_sound:
  fixes γ_S :: "'s ⇒ store set"
  assumes fin: "finite (intra g)"
    and mem: "(v, c, Check_Proved) ∈ set (classify_checks g env classify)"
    and classify_proved: "⋀d s. classify c d = Check_Proved ⟹ s ∈ γ_S d ⟹ truthy (eval c s)"
    and node_sound: "reach v ≤ γ_S (env v)"
  shows "∀s ∈ reach v. truthy (eval c s)"
proof
  fix s assume s: "s ∈ reach v"
  have "classify c (env v) = Check_Proved"
    using mem classify_checks_mem_iff[OF fin, of v c Check_Proved env classify] by auto
  moreover have "s ∈ γ_S (env v)" using node_sound s by blast
  ultimately show "truthy (eval c s)" using classify_proved by blast
qed

theorem classify_checks_refuted_sound:
  fixes γ_S :: "'s ⇒ store set"
  assumes fin: "finite (intra g)"
    and mem: "(v, c, Check_Refuted) ∈ set (classify_checks g env classify)"
    and classify_refuted: "⋀d s. classify c d = Check_Refuted
                          ⟹ s ∈ γ_S d ⟹ ¬ truthy (eval c s)"
    and node_sound: "reach v ≤ γ_S (env v)"
  shows "∀s ∈ reach v. ¬ truthy (eval c s)"
proof
  fix s assume s: "s ∈ reach v"
  have "classify c (env v) = Check_Refuted"
    using mem classify_checks_mem_iff[OF fin, of v c Check_Refuted env classify] by auto
  moreover have "s ∈ γ_S (env v)" using node_sound s by blast
  ultimately show "¬ truthy (eval c s)" using classify_refuted by blast
qed
```

State-carrying sibling of `classify_checks`: the same traversal and classification, with the node's own abstract state attached to each entry. `classify_checks` stays the soundness-critical primitive every domain cites directly; the projection lemma below shows this report is exactly that one with a fourth field added, so `classify_checks_proved_sound` and `classify_checks_refuted_sound` reach it through the projection without restatement.

```
definition classify_checks_with_state ::
    "('vr::linorder, 'e::linorder) gcfg ⇒ (pp ⇒ 's) ⇒ ('e ⇒ 's ⇒ check_result)
       ⇒ (pp × 'e × check_result × 's) list" where
  "classify_checks_with_state g env classify =
     map (λ(u, c, r). (u, c, r, env u)) (classify_checks g env classify)"

lemma classify_checks_with_state_proj [simp]:
  "map (λ(u, c, r, s). (u, c, r)) (classify_checks_with_state g env classify)
     = classify_checks g env classify"
  unfolding classify_checks_with_state_def by (simp add: comp_def case_prod_beta)

end
```

6.24 Contextual_Check_Report

```
theory Contextual_Check_Report
  imports Check_Report Solved_Table Analysis_Query
begin
```

Contextual check verdicts over a solved result table

`classify_checks` reads exactly one abstract state per checked node. A context-sensitive result has one state per `(node, context)` pair instead, and some of those pairs represent no concrete execution at all: a branch that is dead inside one activation, or an activation a node is never reached under. Classifying such a point against its stored state answers vacuously: a classifier's soundness obligation only constrains the stores its state represents, so on a state representing none, either definite answer satisfies that obligation vacuously — a verdict read there is evidence of nothing, and reporting it as `Check_Proved` would fabricate a proof about executions that do not exist.

`contextual_verdict` keeps that case outside `check_result` rather than folding it into one of its three values. `Dead` means "no concrete execution is represented here"; it is neither `Check_Unknown`, which says only that the abstraction decided the condition neither way on a state representing some stores — it asserts no reachability, since those stores may over-approximate a point no execution reaches — nor `Check_Proved`.

`contextual_verdict` is literally `check_result lifted` (`Voblint_Domain.Reachability_Lift`): `Dead` is `Bot`, `Decided` is `Lifted`, and its order/bottom/join are `'a lifted`'s own generic instance at `check_result` (itself `semilattice_sup`, from `Voblint_Framework.Check_Result`) constructor for constructor: `Dead` $\leq$ `_` always, a `Decided` never sits below `Dead`, and two `Decided` verdicts compare and join through `check_result`'s own flat order. Reusing it here keeps one lattice construction for "structurally absent vs. present with a payload" instead of two.

```
type_synonym contextual_verdict = "check_result lifted"

abbreviation Dead :: contextual_verdict where
  "Dead ≡ Bot"

abbreviation Decided :: "check_result ⇒ contextual_verdict" where
  "Decided r ≡ Lifted r"
```

Classifying one contextual point

The reachability decision is not remade here: it was already made when the raw solved local unknown crossed the result boundary, one layer before the lifted readback relabels it structurally. A witness-bottom `Lifted` payload is collapsed to `Bot` by `canonicalize_lift`, so by the time a value is read back — as every public result adapter's raw value is — `Bot` already denotes no store. A `Lifted` state may still denote none, since the emptiness test is sound and incomplete, and classifying it is then vacuous but harmless. `classify_point` only refuses to classify against a state that represents nothing: a covered-but-dead activation, whose raw stored state a solver run could otherwise leave as a witness-bottom `Lifted`, reaches `classify_point` as `Bot` rather than fabricating a vacuous `Check_Proved` off an empty state.

```
fun classify_point ::
  "('e ⇒ 'a ⇒ check_result) ⇒ 'e ⇒ 'a lifted ⇒ contextual_verdict"
where
  "classify_point classify c Bot = Dead"
| "classify_point classify c (Lifted st) = Decided (classify c st)"
```

Classifying by query

A check `c` is decided by one question, the value of its truth as an integer, which every analysis in a product answers and whose combined answer is the meet. Goblint's `assert` analysis asks `EvalInt (c != 0)` through `Queries.eval_bool`; a comparison or logical operator already evaluates to `0` or `1`, so for those Voblint asks about `c` itself and an analysis sees the comparison rather than its test against zero. An exact `1` proves the check and an exact `0` refutes it. `QBot` holds at no store, so a sound combined answer is `QBot` only where the state represents no store, and `classify_answer` reads it as `Dead`. Anything else decides nothing.

```
fun classify_answer :: "answer ⇒ contextual_verdict" where
  "classify_answer QBot = Dead"
| "classify_answer a =
     (if answer_excludes 0 a then Decided Check_Proved
      else if answer_excludes 1 a then Decided Check_Refuted
      else Decided Check_Unknown)"
```

Aggregating the contexts observed at one check

Folding (⊔) from `Dead` over a finite set of observations. The fold's unit is the empty-set answer, so a check with no covered context needs no separate guard. The `[code]` equation is the usual `comp_fun_idem` transfer to a list fold: over an unordered set the value is well defined only because (⊔) is commutative, associative, and idempotent.

```
definition aggregate_verdicts :: "contextual_verdict set ⇒ contextual_verdict" where
  "aggregate_verdicts vs = Finite_Set.fold sup Dead vs"

lemma aggregate_verdicts_code [code]:
  "aggregate_verdicts (set vs) = List.fold sup vs Dead"
proof -
  interpret ci: comp_fun_idem "sup :: contextual_verdict ⇒ _"
    by (rule comp_fun_idem_sup)
  show ?thesis unfolding aggregate_verdicts_def by (rule ci.fold_set_fold)
qed

lemma aggregate_verdicts_empty [simp]: "aggregate_verdicts {} = Dead"
  unfolding aggregate_verdicts_def by simp

lemma aggregate_verdicts_insert [simp]:
  "finite vs ⟹ aggregate_verdicts (insert v vs) = v ⊔ aggregate_verdicts vs"
proof -
  assume "finite vs"
  interpret ci: comp_fun_idem "sup :: contextual_verdict ⇒ _"
    by (rule comp_fun_idem_sup)
  show ?thesis
    unfolding aggregate_verdicts_def using ‹finite vs› by simp
qed

lemma sup_contextual_verdict_eq_Dead_iff [simp]:
  "(x ⊔ y = Dead) ⟷ x = Dead ∧ y = Dead"
  by (cases x; cases y) simp_all
```

The finiteness premise is not bookkeeping: `Finite_Set.fold` returns its identity on an infinite carrier, so an infinite set of observations aggregates to `Dead` whatever its members are, and only the left-to-right direction would survive without it.

```
lemma aggregate_verdicts_eq_Dead_iff:
  assumes "finite vs"
  shows "aggregate_verdicts vs = Dead ⟷ (∀v ∈ vs. v = Dead)"
  using assms by (induction vs rule: finite_induct) simp_all
```

Whole-program contextual check report

The context-sensitive sibling of `classify_checks`: the same `EA_Check` traversal in the same `cfg_intra_list` order, but reading a `('ctx, 'a) solved_table` instead of a single `pp ⇒ 's` environment, and retaining one verdict per context covered at the checked node rather than collapsing them at construction time. `classify_checks_ctx_positions` below is the load-bearing consequence: the node and condition columns coincide with `classify_checks`'s, so a positional consumer pairing this report with the source's own check list stays aligned.

The per-check observations are a set, not a list. `'ctx` carries no ordering constraint — `table_contexts` is a set for precisely that reason, and real context types such as interval vectors have no total order — so no canonical list exists to produce. Nothing downstream needs one: `aggregate_verdicts` is order-independent by construction.

```
definition classify_checks_ctx ::
    "('vr::linorder, 'e::linorder) gcfg ⇒ ('ctx, 'a) solved_table ⇒ ('e ⇒ 'a ⇒ check_result)
       ⇒ (pp × 'e × ('ctx × contextual_verdict) set) list" where
  "classify_checks_ctx g r classify =
     map (λ(u, a, v). (u, ea_check_cond a,
           (λctx. (ctx, classify_point classify (ea_check_cond a) (lookup_table r u ctx)))
             ‘ table_contexts r u))
       (filter (λ(u, a, v). is_EA_Check a) (cfg_intra_list g))"

definition classify_checks_verdicts ::
    "('vr::linorder, 'e::linorder) gcfg ⇒ ('ctx, 'a) solved_table ⇒ ('e ⇒ 'a ⇒ check_result)
       ⇒ (pp × 'e × contextual_verdict) list" where
  "classify_checks_verdicts g r classify =
     map (λ(u, c, vs). (u, c, aggregate_verdicts (snd ‘ vs))) (classify_checks_ctx g r classify)"

lemma classify_checks_verdicts_proj [simp]:
  "map (λ(u, c, vs). (u, c, aggregate_verdicts (snd ‘ vs))) (classify_checks_ctx g r classify)
     = classify_checks_verdicts g r classify"
  unfolding classify_checks_verdicts_def by (rule refl)
```

```
lemma classify_checks_ctx_positions:
  "map (λ(u, c, vs). (u, c)) (classify_checks_ctx g r classify)
     = map (λ(u, c, res). (u, c)) (classify_checks g env classify')"
  unfolding classify_checks_ctx_def classify_checks_def
  by (simp add: comp_def case_prod_beta)
```

Membership unfolds to an `EA_Check` edge at the entry's own source node, with that node's whole context fan-out attached — the same shape `classify_checks_mem_iff` gives the context-insensitive report.

```
lemma classify_checks_ctx_mem_iff:
  assumes "finite (intra g)"
  shows "(v, c, vs) ∈ set (classify_checks_ctx g r classify)
     ⟷ (∃l tgt. (v, EA_Check l c, tgt) ∈ intra g)
         ∧ vs = (λctx. (ctx, classify_point classify c (lookup_table r v ctx))) ‘ table_contexts r v"
  unfolding classify_checks_ctx_def set_map set_filter
  using set_cfg_intra_list[OF assms]
  by (auto simp: image_iff split: gedge_action.splits)
     (metis gedge_action.collapse surjective_pairing)
```

Contextual proved/refuted soundness

The `Decided Check_Proved`/`Decided Check_Refuted` analogues of `classify_checks_proved_sound`/`classify_checks_refuted_sou` an aggregated verdict of `Decided Check_Proved` forces every observation folded into it to be `Dead` or `Decided Check_Proved` — never a disagreeing `Decided Check_Refuted` or `Decided Check_Unknown`, since either would collapse the join to `Decided Check_Unknown` instead. `Dead` observations contribute nothing and are filtered out at the call site, not here.

The order-theoretic route: every member of a finite set sits below the set's own fold-join (`aggregate_verdicts_member_le`), a decided, non-`Check_Unknown` aggregate already forces the set finite (`aggregate_verdicts_decided_finite`, since an infinite carrier folds to the identity `Dead`), and sitting below such a verdict has only two shapes (`contextual_verdict_le_Deci` Composing the three gives `aggregate_verdicts_decided_dest` directly, with no case-bash over the fold step: disagreement, from either side, could only have collapsed the join to `Decided Check_Unknown` instead.

```
lemma aggregate_verdicts_member_le:
  assumes "finite vs" and "v ∈ vs"
  shows "v ≤ aggregate_verdicts vs"
  using assms
  by (induction vs rule: finite_induct) (auto intro: order_trans sup_ge2)

lemma contextual_verdict_le_Decided_iff [simp]:
  assumes "r ≠ Check_Unknown"
  shows "v ≤ Decided r ⟷ v = Dead ∨ v = Decided r"
  using assms by (cases v; cases r) (simp_all add: less_eq_check_result_def)

lemma aggregate_verdicts_decided_finite:
  "aggregate_verdicts vs = Decided r ⟹ finite vs"
  by (rule ccontr) (simp add: aggregate_verdicts_def)

lemma aggregate_verdicts_decided_dest:
  assumes agg: "aggregate_verdicts vs = Decided r" and known: "r ≠ Check_Unknown"
  shows "∀v ∈ vs. v = Dead ∨ v = Decided r"
proof
  fix v
  assume "v ∈ vs"
  have "v ≤ aggregate_verdicts vs"
    using aggregate_verdicts_member_le[OF aggregate_verdicts_decided_finite[OF agg] ‹v ∈ vs›] .
  with agg known show "v = Dead ∨ v = Decided r" by simp
qed
```

One condition at one point, over its contexts

The verdict a table gives a condition at a point: the classifier's verdict at every context the point was solved at, aggregated. A check and an arithmetic obligation are both read this way.

```
definition point_verdict ::
    "('ctx, 'a) solved_table ⇒ ('e ⇒ 'a ⇒ check_result) ⇒ pp ⇒ 'e
       ⇒ contextual_verdict" where
  "point_verdict r classify v e =
     aggregate_verdicts
       ((λctx. classify_point classify e (lookup_table r v ctx)) ‘ table_contexts r v)"
```

A decided, non-`Check_Unknown` point verdict is the classifier's own reading at every `Lifted` context of the point.

```
lemma point_verdict_decided:
```

```
  assumes agg: "point_verdict r classify v e = Decided res" and known: "res ≠ Check_Unknown"
    and look: "lookup_table r v ctx = Lifted st"
  shows "classify e st = res"
proof -
  have "classify_point classify e (lookup_table r v ctx)
        ∈ (λc. classify_point classify e (lookup_table r v c)) ‘ table_contexts r v"
    using lookup_table_LiftedD[OF look] by blast
  with aggregate_verdicts_decided_dest[OF agg[unfolded point_verdict_def] known] look
  show ?thesis by auto
qed
```

A point with a `Lifted` entry among finitely many contexts is not `Dead`.

```
lemma point_verdict_not_dead:
  assumes fin: "finite (table_contexts r v)" and look: "lookup_table r v ctx = Lifted st"
  shows "point_verdict r classify v e ≠ Dead"
proof
  assume "point_verdict r classify v e = Dead"
  with fin have "∀x ∈ (λc. classify_point classify e (lookup_table r v c)) ‘ table_contexts r v.
                   x = Dead"
    unfolding point_verdict_def by (simp add: aggregate_verdicts_eq_Dead_iff)
  then have "classify_point classify e (lookup_table r v ctx) = Dead"
    using lookup_table_LiftedD[OF look] by blast
  with look show False by simp
qed
```

The context-indexed report analogue of `classify_checks_proved_sound`/ `classify_checks_refuted_sound`, stated once for any decided, non-`Check_Unknown` aggregate: it forces the classifier's own reading at every `Lifted` context to agree. The image set being folded is shown finite from the aggregate equation itself (`Finite_Set.fold_infinite`: an infinite carrier folds to the identity `Dead`, which a decided, non-`Check_Unknown` result already rules out), so no separate finiteness assumption on the covered contexts is needed.

```
lemma classify_checks_verdicts_mem_iff:
  assumes "finite (intra g)"
  shows "(v, c, vr) ∈ set (classify_checks_verdicts g ar classify)
     ⟷ (∃l tgt. (v, EA_Check l c, tgt) ∈ intra g)
         ∧ vr = aggregate_verdicts
                  ((λctx. classify_point classify c (lookup_table ar v ctx)) ‘ table_contexts ar v)"
proof -
  have img: "snd ‘ (λctx. (ctx, classify_point classify c (lookup_table ar v ctx)))
                 ‘ table_contexts ar v
           = (λctx. classify_point classify c (lookup_table ar v ctx)) ‘ table_contexts ar v"
    by (simp add: image_comp comp_def)
  have "(v, c, vr) ∈ set (classify_checks_verdicts g ar classify)
      ⟷ (∃vs. (v, c, vs) ∈ set (classify_checks_ctx g ar classify)
                ∧ vr = aggregate_verdicts (snd ‘ vs))"
    unfolding classify_checks_verdicts_def set_map by (force simp: image_iff)
  also have "... ⟷ (∃l tgt. (v, EA_Check l c, tgt) ∈ intra g)
         ∧ vr = aggregate_verdicts
                  ((λctx. classify_point classify c (lookup_table ar v ctx)) ‘ table_contexts ar v)"
    unfolding classify_checks_ctx_mem_iff[OF assms] using img by auto
  finally show ?thesis .
qed

theorem classify_checks_ctx_decided_sound:
  assumes fin: "finite (intra g)"
    and mem: "(v, c, Decided r) ∈ set (classify_checks_verdicts g ar classify)"
    and reach: "lookup_table ar v ctx = Lifted st"
    and known: "r ≠ Check_Unknown"
  shows "classify c st = r"
proof -
  have "point_verdict ar classify v c = Decided r"
    using classify_checks_verdicts_mem_iff[OF fin, of v c "Decided r" ar classify] mem
    by (simp add: point_verdict_def)
  then show ?thesis by (rule point_verdict_decided[OF _ known reach])
qed

end
```

6.25 Check_Answer

```
theory Check_Answer
  imports Contextual_Check_Report
begin
```

A domain's answer to the value of an expression

A domain that decides a comparison or logical operator `e` three-valuedly answers `EvalInt e` with `bool_answer`: such an expression evaluates to `0` or `1`, so a decided truth value is the exact integer, and an undecided one is the interval `[0, 1]`.

```
definition bool_answer :: "bool option ⇒ answer" where
  "bool_answer r =
     (case r of
        Some b ⇒ answer_of_int (if b then 1 else 0)
      | None ⇒ answer_of_ivl (Ivl (Fin 0) (Fin 1)))"

lemma classify_answer_gamma:
  "a ≠ QBot ⟹ classify_answer a =
     Decided (if 0 ∉ gamma_answer a then Check_Proved
              else if 1 ∉ gamma_answer a then Check_Refuted else Check_Unknown)"
  using answer_excludes_iff[of 0 a] answer_excludes_iff[of 1 a]
  by (cases a) (auto simp del: answer_excludes.simps)
```

Deciding a check from an answer

The one check consumer: ask the state's handler the check's question and read the verdict off the answer, as Goblint's `assert` analysis reads `Queries.eval_bool`. `classify_answer` reads an answer that admits no value as `Dead`, but the consumer reports it as `Check_Unknown`: a check is reported dead only where the state itself is `Bot`, never from the answer to its query.

The question is the check's truth value, which a language spells with its own expressions: `tx c` evaluates to `1` where `c` holds and to `0` elsewhere.

```
definition answer_check ::
    "(exp ⇒ exp) ⇒ ('v ⇒ query ⇒ answer) ⇒ exp ⇒ 'v ⇒ check_result" where
  "answer_check tx q c v =
     (case classify_answer (q v (EvalInt (tx c))) of Decided r ⇒ r | _ ⇒ Check_Unknown)"

lemma answer_check_sound:
  assumes truth: "eval (tx c) s = (if truthy (eval c s) then 1 else 0)"
    and holds: "eval_holds (EvalInt (tx c)) (q v (EvalInt (tx c))) s"
  shows answer_check_proved: "answer_check tx q c v = Check_Proved ⟹ truthy (eval c s)"
    and answer_check_refuted: "answer_check tx q c v = Check_Refuted ⟹ ¬ truthy (eval c s)"
proof -
  let ?a = "q v (EvalInt (tx c))"
  have val: "(if truthy (eval c s) then 1 else 0) ∈ gamma_answer ?a"
    using holds truth by simp
  then have "?a ≠ QBot" by auto
  then show "answer_check tx q c v = Check_Proved ⟹ truthy (eval c s)"
    and "answer_check tx q c v = Check_Refuted ⟹ ¬ truthy (eval c s)"
    using val by (auto simp: answer_check_def classify_answer_gamma split: if_splits)
qed

end
```

6.26 MCP_Registry

```
theory MCP_Registry
  imports MCP_Spec Check_Answer
begin
```

Registered analyses, run as one

A registry assigns each analysis one field of a combined state: the component that runs on it, whether the field is live, what a solved field publishes, what a published value describes, whether it is empty, and how it answers a query. The registry runs any distinct, nonempty list of its analyses as one component (`mcp_combine`), decides checks from the meet of their answers, and is sound whenever each field is. Nothing here depends on the language: the language enters through its evaluator `ev`, its return slot `rv`, the conversion and the global predicate of each program, and `tx`, the expression that asks for a check's truth value.

```
locale mcp_registry =
  fixes κ :: kenv
    and tx :: "exp ⇒ exp"
    and field ::
      "(var_id ⇒ bool) ⇒ 'p ⇒ 'a ⇒ ('s::{semilattice_sup, order_bot} lifted) local_spec"
    and live :: "'a ⇒ 's ⇒ bool"
    and rd :: "(var_id ⇒ bool) ⇒ 's ⇒ 'v::bot"
    and val_gamma :: "'a ⇒ 'v ⇒ store set"
    and val_empty :: "'a ⇒ 'v ⇒ bool"
    and field_empty :: "'p ⇒ 'a ⇒ 's ⇒ bool"
    and answer :: "'a ⇒ 'v ⇒ channel ⇒ query ⇒ answer"
  assumes truth: "eval (tx c) s = (if truthy (eval c s) then 1 else 0)"
    and field_sound: "sound_local_spec κ
        (λx. gamma_lift (val_gamma a) (map_lift (rd is_global) x)) (field is_global p a)"
    and field_frame: "a ≠ b ⟹
        mcp_frame (field 𝒢 p a) (λx. gamma_lift (val_gamma b) (map_lift (rd 𝒢) x))"
    and single_entry_field: "single_entry (field 𝒢 p a)"
    and dead: "¬ live a r ⟹ val_gamma a (rd 𝒢 r) = {}"
    and field_empty_rd: "field_empty p a r = val_empty a (rd is_global r)"
    and val_empty_gamma: "val_empty a v ⟹ val_gamma a v = {}"
    and val_gamma_bot: "val_gamma a ⊥ = {}"
    and answer_sound: "s ∈ val_gamma a v ⟹ eval_query.channel_holds ch s
        ⟹ eval_holds q (answer a v ch q) s"
begin

abbreviation part_gamma :: "(var_id ⇒ bool) ⇒ 'a ⇒ 's lifted ⇒ store set" where
  "part_gamma 𝒢 a x ≡ gamma_lift (val_gamma a) (map_lift (rd 𝒢) x)"

definition act_gamma :: "'a list ⇒ 'v ⇒ store set" where
  "act_gamma as v = (⋂a ∈ set as. val_gamma a v)"
```

Each analysis answers what it knows

A field answers a query as its published value does, so an analysis written without a query handler becomes a provider on the combined state (`with_qry`), as every Goblint analysis answers `EvalInt` from its domain. An answer may itself ask the channel, as Goblint's `base` asks about the operands of the expression it is asked about.

```
definition part_answer ::
    "(var_id ⇒ bool) ⇒ 'a ⇒ channel ⇒ 's lifted ⇒ query ⇒ answer" where
  "part_answer 𝒢 a ch x q = (case x of Bot ⇒ ⊤ | Lifted r ⇒ answer a (rd 𝒢 r) ch q)"

lemma part_answer_sound:
  "s ∈ part_gamma 𝒢 a x ⟹ eval_query.channel_holds ch s
     ⟹ eval_holds q (part_answer 𝒢 a ch x q) s"
  by (cases x) (auto simp: part_answer_def intro: answer_sound)

definition mcp_field :: "(var_id ⇒ bool) ⇒ 'p ⇒ 'a ⇒ 's lifted local_spec" where
  "mcp_field 𝒢 p a = with_qry (part_answer 𝒢 a) (field 𝒢 p a)"

lemma mcp_field_sound:
  "sound_local_spec κ (part_gamma is_global a)
     (mcp_field is_global p a)"
  unfolding mcp_field_def
  by (rule with_qry_sound[OF field_sound]) (rule part_answer_sound)

lemma mcp_field_frame: "a ≠ b ⟹ mcp_frame (mcp_field 𝒢 p a) (part_gamma 𝒢 b)"
  unfolding mcp_field_def by (rule with_qry_frame[OF field_frame])
```

The active analyses, run as one

The combined state is unreachable as soon as one active analysis says its field is, as Goblint's `MCP` raises `Deadcode` when one component does. The normalization changes no concretization: a field that describes nothing empties the intersection already.

```
definition mcp_norm :: "'a list ⇒ 's lifted ⇒ 's lifted" where
  "mcp_norm as x =
     (case x of Bot ⇒ Bot | Lifted r ⇒ if list_all (λa. live a r) as then x else Bot)"

lemma mcp_gamma_norm:
  "mcp_gamma (map (part_gamma 𝒢) as) (mcp_norm as x) = mcp_gamma (map (part_gamma 𝒢) as) x"
  by (cases x) (auto simp: mcp_norm_def mcp_gamma_def list_all_iff dest: dead)

definition mcp_comp :: "'a list ⇒ (var_id ⇒ bool) ⇒ 'p ⇒ 's lifted local_spec" where
  "mcp_comp as 𝒢 p = map_local_spec (mcp_norm as) (mcp_combine (map (mcp_field 𝒢 p) as))"

theorem mcp_comp_sound:
  assumes "distinct as" and "as ≠ []"
  shows "sound_local_spec κ
           (mcp_gamma (map (part_gamma is_global) as)) (mcp_comp as is_global p)"
proof -
  let ?gcs = "map (λa. (part_gamma is_global a, mcp_field is_global p a)) as"
  have "sound_local_spec κ (mcp_gamma (map fst ?gcs)) (mcp_combine (map snd ?gcs))"
    by (rule mcp_combine_sound)
       (auto simp: mcp_field_sound assms intro!: mcp_independent_map mcp_field_frame)
  then show ?thesis
    unfolding mcp_comp_def
    by (intro map_local_spec_sound) (simp_all add: comp_def mcp_gamma_norm)
qed

lemma single_entry_mcp_comp: "as ≠ [] ⟹ single_entry (mcp_comp as 𝒢 p)"
  unfolding mcp_comp_def mcp_field_def
  by (intro single_entry_map_local_spec single_entry_mcp_combine)
     (auto simp: single_entry_field single_entry_with_qry)
```

What the combined state publishes

The combined state denotes the stores every active analysis admits, and it is empty as soon as one active analysis reports its own field empty.

```
lemma mcp_gamma_rd:
  "mcp_gamma (map (part_gamma 𝒢) as) x = gamma_lift (act_gamma as) (map_lift (rd 𝒢) x)"
  if "as ≠ []"
  using that by (cases x) (auto simp: mcp_gamma_def act_gamma_def)

lemma act_gamma_bot: "as ≠ [] ⟹ act_gamma as ⊥ = {}"
  by (auto simp: act_gamma_def val_gamma_bot)

definition mcp_emp :: "'a list ⇒ 'p ⇒ 's ⇒ bool" where
  "mcp_emp as p r = list_ex (λa. field_empty p a r) as"

definition mcp_empty_v :: "'a list ⇒ 'v ⇒ bool" where
  "mcp_empty_v as v = list_ex (λa. val_empty a v) as"

lemma mcp_emp_rd: "mcp_emp as p r = mcp_empty_v as (rd is_global r)"
  by (simp add: mcp_emp_def mcp_empty_v_def field_empty_rd)

lemma mcp_empty_v_gamma: "mcp_empty_v as v ⟹ act_gamma as v = {}"
  by (auto simp: mcp_empty_v_def act_gamma_def list_ex_iff dest: val_empty_gamma)

lemma sound_emptiness_mcp: "sound_emptiness (mcp_empty_v as) (act_gamma as)"
  by (rule sound_emptinessI) (rule mcp_empty_v_gamma)
```

Deciding a check from every active analysis

The combined answer meets the active analyses' answers, as `MCP.query'` does, and a check is decided from it as Goblint's `assert` reads `Queries.eval_bool`. The question goes through the channel the combined answer closes over itself (`ask_rec`), so an answer that asks about a subexpression reaches every active analysis, and a query cycle answers ⊤.

```
definition mcp_answer :: "'a list ⇒ channel ⇒ 'v ⇒ query ⇒ answer" where
  "mcp_answer as ch v q = fold (λa r. r ⊓ answer a v ch q) as ⊤"

lemma mcp_answer_fold_sound:
```

```
  "∀a ∈ set as. s ∈ val_gamma a v ⟹ eval_query.channel_holds ch s
   ⟹ eval_holds q r s ⟹ eval_holds q (fold (λa r. r ⊓ answer a v ch q) as r) s"
  by (induction as arbitrary: r) (auto intro: eval_query.inf_sound answer_sound)

lemma mcp_answer_sound:
  "s ∈ act_gamma as v ⟹ eval_query.channel_holds ch s
     ⟹ eval_holds q (mcp_answer as ch v q) s"
  unfolding mcp_answer_def act_gamma_def by (rule mcp_answer_fold_sound) auto

definition mcp_ask :: "'a list ⇒ 'v ⇒ channel" where
  "mcp_ask as v = ask_rec (mcp_answer as) query_depth {} v"

lemma mcp_ask_sound:
  assumes s: "s ∈ act_gamma as v"
  shows "eval_query.channel_holds (mcp_ask as v) s"
proof -
  show ?thesis
    unfolding mcp_ask_def by (rule ask_rec_sound) (rule mcp_answer_sound[OF s])
qed

definition mcp_classify :: "'a list ⇒ exp ⇒ 'v ⇒ check_result" where
  "mcp_classify as = answer_check tx (mcp_ask as)"

lemma mcp_classify_sound:
  assumes s: "s ∈ act_gamma as v"
  shows mcp_classify_proved: "mcp_classify as c v = Check_Proved ⟹ truthy (eval c s)"
    and mcp_classify_refuted: "mcp_classify as c v = Check_Refuted ⟹ ¬ truthy (eval c s)"
  using answer_check_sound[where tx = tx and q = "mcp_ask as" and c = c and v = v,
      OF truth eval_query.channel_holdsD[OF mcp_ask_sound[OF s]]]
  by (simp_all add: mcp_classify_def)

end

end
```

6.27 Bool_Refine

```
theory Bool_Refine
  imports Main
begin
```

Refining a state by a Boolean condition

A guard narrows the state by the truth value its condition must have. The Boolean structure of the condition is the same for every language: a negation flips the required truth value; a conjunction that must hold and a disjunction that must fail narrow by both operands in turn; a disjunction that must hold, or a conjunction that must fail, holds through one of its operands, so each operand that can still have the required value narrows the state on its own and the results are joined, as Goblint's `BaseInvariant` joins the alternatives of a disjunction (`baseInvariant.ml`, lines 724-731 at `0dc12d355`). An operand is dropped when its forward value rules the required truth value out. When neither operand can have it, no store satisfies the condition, and the language decides the result: bottom, as Goblint's `Deadcode`, or the state unchanged when its caller discards the branch by other means.

A language supplies a view of its conditions as negation, conjunction, disjunction or atom, a size that decreases on operands, how an atom narrows a state, the feasibility test, the join and the infeasible result.

```
datatype 'e bool_view = BV_Not 'e | BV_And 'e 'e | BV_Or 'e 'e | BV_Atom
```

The parameters of that refinement. The size bound on operands makes the recursion on the condition well-founded.

```
locale bool_refine =
  fixes view :: "'e ⇒ 'e bool_view" and sz :: "'e ⇒ nat"
    and atom :: "'e ⇒ bool ⇒ 's ⇒ 's" and feas :: "'e ⇒ bool ⇒ 's ⇒ bool"
    and join :: "'s ⇒ 's ⇒ 's" and dead :: "'s ⇒ 's"
  assumes sz_not: "view e = BV_Not a ⟹ sz a < sz e"
    and sz_and: "view e = BV_And a b ⟹ sz a < sz e ∧ sz b < sz e"
    and sz_or: "view e = BV_Or a b ⟹ sz a < sz e ∧ sz b < sz e"
begin

function brefine :: "'e ⇒ bool ⇒ 's ⇒ 's" where
  "brefine e pol σ =
     (case view e of
        BV_Not a ⇒ brefine a (¬ pol) σ
      | BV_And a b ⇒
          if pol then brefine b True (brefine a True σ)
          else (case (feas a False σ, feas b False σ) of
                  (True, True) ⇒ join (brefine a False σ) (brefine b False σ)
                | (True, False) ⇒ brefine a False σ
                | (False, True) ⇒ brefine b False σ
                | (False, False) ⇒ dead σ)
      | BV_Or a b ⇒
          if pol then (case (feas a True σ, feas b True σ) of
                  (True, True) ⇒ join (brefine a True σ) (brefine b True σ)
                | (True, False) ⇒ brefine a True σ
                | (False, True) ⇒ brefine b True σ
                | (False, False) ⇒ dead σ)
          else brefine b False (brefine a False σ)
      | BV_Atom ⇒ atom e pol σ)"
  by pat_completeness auto
termination
  by (relation "measure (λ(e, pol, σ). sz e)")
     (auto dest: sz_not sz_and sz_or)

declare brefine.simps [simp del]
```

A store that satisfies the condition with the required truth value stays described: the atom's narrowing, the feasibility test and the join are sound, and the view reflects how the language evaluates negation, conjunction and disjunction.

```
lemma brefine_sound:
  fixes ev :: "'e ⇒ 't ⇒ int" and γ :: "'s ⇒ 't set"
  assumes atom_sound: "⋀e pol σ t. view e = BV_Atom ⟹ t ∈ γ σ ⟹ (ev e t ≠ 0) = pol
      ⟹ t ∈ γ (atom e pol σ)"
    and ev_not: "⋀e a t. view e = BV_Not a ⟹ (ev e t ≠ 0) = (ev a t = 0)"
    and ev_and: "⋀e a b t. view e = BV_And a b ⟹ (ev e t ≠ 0) = (ev a t ≠ 0 ∧ ev b t ≠ 0)"
    and ev_or: "⋀e a b t. view e = BV_Or a b ⟹ (ev e t ≠ 0) = (ev a t ≠ 0 ∨ ev b t ≠ 0)"
    and feas_sound: "⋀e pol σ t. t ∈ γ σ ⟹ (ev e t ≠ 0) = pol ⟹ feas e pol σ"
    and join1: "⋀x y t. t ∈ γ x ⟹ t ∈ γ (join x y)"
```

```
    and join2: "⋀x y t. t ∈ γ y ⟹ t ∈ γ (join x y)"
  shows "t ∈ γ σ ⟹ (ev e t ≠ 0) = pol ⟹ t ∈ γ (brefine e pol σ)"
proof (induction e pol σ rule: brefine.induct)
  case (1 e pol σ)
  show ?case
  proof (cases "view e")
    case (BV_Not a)
    with 1 ev_not[OF BV_Not, of t] show ?thesis by (subst brefine.simps) simp
  next
    case (BV_And a b)
    show ?thesis
    proof (cases pol)
      case True
      with 1 ev_and[OF BV_And, of t] BV_And show ?thesis by (subst brefine.simps) simp
    next
      case False
      with 1 ev_and[OF BV_And, of t] BV_And feas_sound[of t σ a False] feas_sound[of t σ b False]
      show ?thesis
        by (subst brefine.simps) (auto split: bool.splits intro: join1 join2)
    qed
  next
    case (BV_Or a b)
    show ?thesis
    proof (cases pol)
      case True
      with 1 ev_or[OF BV_Or, of t] BV_Or feas_sound[of t σ a True] feas_sound[of t σ b True]
      show ?thesis
        by (subst brefine.simps) (auto split: bool.splits intro: join1 join2)
    next
      case False
      with 1 ev_or[OF BV_Or, of t] BV_Or show ?thesis by (subst brefine.simps) simp
    qed
  next
    case BV_Atom
    with 1 atom_sound show ?thesis by (subst brefine.simps) simp
  qed
qed

end

end
```

6.28 DG_Analysis_Adapter

```
theory DG_Analysis_Adapter
  imports Routed_Context Contextual_Check_Report
begin
```

Public result and check-report adapter for a routed DG analysis

Every concrete routed-context analysis needs the same public `solved_table`/check report/soundness triple, and each one follows the identical shape once a routed D/G equation system is solved: read the local unknown at every covered `(node, context)` pair, together with the analysis-wide global unknown, into an `solved_table`, classify every compiled check against it via `classify_checks_verdicts`, and discharge that report's own soundness from the activation-indexed collecting semantics (`Voblint_Framework.Activation_Backbone`) already available once EDGE/CALL/COMB are in hand from `routed_context`. `dg_analysis_adapter` derives that whole triple once, generic in a domain instance (a `classify` function with its own soundness obligations), a context instance (an interpretation of `routed_context`), and the published value (a publication map `rd` from a local and a global value of the solver's carrier into a value type `'v` with its own concretization γ_V and exact emptiness test $\texttt{empty}_V$; a non-relational analysis publishes its abstract store), leaving solver choice orthogonal: which concrete solver produced `sigma`/`sg` is an interpretation-site argument, never a locale parameter, matching how Interval's own Warrow/join/per-origin solver choice is already orthogonal to its context.

`gammaDG_rd` asks that publishing a local value together with a global one keeps exactly the stores the pair denotes, γ_{DG} `d g' = gamma_lift` γ_V `(rd d g')`. The global value the table pairs with every local one is the solved global unknown at `Inr buffer_key`, the pairing the coverage assumption `sg_cov` of `dg_context_activation` concretizes. A concretization that reads both components, an ownership split say, therefore interprets this locale directly. An analysis whose concretization ignores the global half instantiates `rd` at a function constant in its second argument.

```
locale dg_analysis_adapter =
  routed_context κ S γ_DG g buffer_key global_of route bot0 d₀ g₀ sigma
    vars x0 sg seed "static_resolve g" is_bot γ_M adm
  for κ :: kenv
    and S ::
      "(pp × 'c, 'k, 'n, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec"
    and γ_DG :: "'D ⇒ ('n ⇒ 'G) ⇒ store set"
    and g buffer_key and global_of :: "'n ⇒ 'k"
    and route :: "pp ⇒ 'c ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'c"
    and bot0 d₀ :: 'D and g₀ :: 'G
    and sigma :: "pp × 'c + 'k ⇒ ('D, 'G) dg_state"
    and vars :: "(pp × 'c) set"
    and x0 :: "pp × 'c"
    and sg :: "pp × 'c + 'k ⇒ 'M"
    and seed :: "pp ⇒ 'c ⇒ 'k"
    and is_bot :: "'D ⇒ bool"
    and γ_M :: "'M ⇒ store set"
    and adm :: "'c call_context_policy" +
  fixes rd :: "'D ⇒ ('n ⇒ 'G) ⇒ 'v lifted"
    and γ_V :: "'v ⇒ store set"
    and empty_V :: "'v ⇒ bool"
    and classify :: "exp ⇒ 'v ⇒ check_result"
  assumes gammaDG_rd: "⋀d g'. γ_DG d g' = gamma_lift γ_V (rd d g')"
    and empty_sound: "⋀v. empty_V v ⟹ γ_V v = {}"
    and classify_proved:
    "⋀c d s. classify c d = Check_Proved ⟹ s ∈ γ_V d ⟹ truthy (eval c s)"
    and classify_refuted:
    "⋀c d s. classify c d = Check_Refuted ⟹ s ∈ γ_V d ⟹ ¬ truthy (eval c s)"
    and vars_finite: "finite vars"
begin
```

The public result table

Built from the locale's own solved `vars`/`sigma` pair, mirroring how `dg_result_for` builds an `solved_table` from an already-solved key set and reader. The solved local unknown and the solved global at `Inr buffer_key` are published into `'v lifted` by `rd`, and the one collapse this table then needs is `canonicalize_lift` against $\texttt{empty}_V$, so a `Lifted` payload that denotes no store without the solver's own `Bot` tag reads as `Bot` here, matching `classify_point`'s documented discipline of never fabricating a vacuous `Check_Proved` off an empty state.

```
definition analyse_result :: "('c, 'v) solved_table" where
  "analyse_result = Solved_Table vars
     (λv ctx. canonicalize_lift empty_V
        (rd (dg_local (sigma (Inl (v, ctx)))) (genv global_of sigma)))"
```

```
lemma lookup_table_analyse_result:
  "lookup_table analyse_result v ctx =
     (if (v, ctx) ∈ vars
      then canonicalize_lift empty_V
             (rd (dg_local (sigma (Inl (v, ctx)))) (genv global_of sigma))
      else Bot)"
  unfolding lookup_table_def analyse_result_def by simp
```

Canonicality holds unconditionally here: `canonicalize_lift` is exactly what rules out a `Lifted` payload that is secretly empty, and it runs on every entry the table publishes.

Finiteness cannot be proved here, because `vars` is a fixed arbitrary set and this locale deliberately does not know which solver produced it — that is what keeps it independent of solver choice. It is therefore an assumption, `vars_finite`, and an interpretation discharges it from the solver it actually ran: `finite_stabl_solve` says a terminating solve stabilizes finitely many unknowns, and the interpretation site is the only place that knows `vars` is that stable set.

Nothing about the context type enters, which is why entry-state contexts are covered as readily as monovariant ones despite their context space being unbounded.

```
lemma analyse_result_canonical:
  assumes "lookup_table analyse_result v ctx = Lifted st"  shows "¬ empty_V st"
  using assms
  unfolding lookup_table_analyse_result
  by (cases "rd (dg_local (sigma (Inl (v, ctx)))) (genv global_of sigma)")
     (auto simp: normalize_lift_def split: if_splits)

theorem wf_analyse_result: "wf_solved_table empty_V analyse_result"
  unfolding wf_solved_table_def finite_solved_table_def
  using vars_finite analyse_result_canonical by (simp add: analyse_result_def)
```

The soundness bridge γ_M `(sg (Inl (v, ctx)))` steps need: a covered point's guarded reading agrees exactly with `analyse_result`'s own `lookup_table` answer at the same key, `Bot` concretizing to the empty set and `Lifted` to γ_V of its payload.

Collapsing a payload that denotes nothing to `Bot` does not change what the value denotes.

```
lemma gamma_lift_canonicalize [simp]:
  "gamma_lift γ_V (canonicalize_lift empty_V x) = gamma_lift γ_V x"
  by (cases x) (simp_all add: normalize_lift_def empty_sound)

lemma gammaM_sg_eq_lookup_table:
  assumes cov: "(v, ctx) ∈ vars"
  shows "γ_M (sg (Inl (v, ctx))) = gamma_lift γ_V (lookup_table analyse_result v ctx)"
proof -
  have "γ_M (sg (Inl (v, ctx))) = γ_DG (dg_local (sigma (Inl (v, ctx))))
          (genv global_of sigma)"
    using cov by simp
  also have "... = gamma_lift γ_V
                       (rd (dg_local (sigma (Inl (v, ctx)))) (genv global_of sigma))"
    by (rule gammaDG_rd)
  also have "... = gamma_lift γ_V (lookup_table analyse_result v ctx)"
    using cov unfolding lookup_table_analyse_result by simp
  finally show ?thesis .
qed
```

The node-soundness bridge a concrete instance's own report-soundness proof needs, phrased directly against `analyse_result` rather than against `analyse_report_ctx` (defined below): every activation-collected store at `v` is concretized by `analyse_result`'s own reading, $\bot$ when uncovered or witness-bottom, its `Lifted` payload otherwise — composing `activation_collect_dg_sound` with `gammaM_sg_eq_lookup_table`, case-split on coverage. A concrete instance whose own public result reads through this same `analyse_result` (up to a proved value equality) gets its own node-soundness bridge from this lemma directly, instead of re-deriving it from `routed_context`'s primitives by hand.

```
lemma analyse_result_node_sound:
  fixes S0 :: "store set" and c_0 :: 'c
  assumes entry_cov: "(cfg_entry g, c_0) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d_0 e_0"
    and init_env_le: "e_0 ≤ genv global_of sigma"
  shows "A_κ,adm,c_0,g,S0 v ctx
           ⊆ gamma_lift γ_V (lookup_table analyse_result v ctx)"
proof -
  have "A_κ,adm,c_0,g,S0 v ctx
          ⊆ γ_M (sg (Inl (v, ctx)))"
```

```
    by (rule activation_collect_dg_sound[OF entry_cov init_sound init_env_le])
  also have "... = gamma_lift γ_V (lookup_table analyse_result v ctx)"
  proof (cases "(v, ctx) ∈ vars")
    case True
    show ?thesis by (rule gammaM_sg_eq_lookup_table[OF True])
  next
    case False
    hence "γ_M (sg (Inl (v, ctx))) = {}" by simp
    moreover from False have "lookup_table analyse_result v ctx = Bot"
      unfolding lookup_table_analyse_result by simp
    ultimately show ?thesis by simp
  qed
  finally show ?thesis by simp
qed
```

`analyse_result_node_sound` is stated in the vocabulary `Voblint_Framework.Solved_Table` states its lookup facts in: every covered lookup safely abstracts the activation-collected stores at its key. It is the general statement, with no bottom case baked in — the unreachability reading is one instantiation of it, below.

Note what the coverage case split above establishes and this inherits: inside this locale an *uncovered* key is not merely uninformative, since γ_M `(sg (Inl (v, ctx)))` is empty off `vars` and the collect is contained in it either way. That is a property of a solve satisfying this locale's assumptions, not of `lookup_table`, which is why the unreachability corollary still demands `Covered Bot`: a caller reading a result table without these assumptions in hand has no such guarantee.

```
lemma analyse_result_covered_unreachable:
  fixes S0 :: "store set" and c₀ :: 'c
  assumes entry_cov: "(cfg_entry g, c₀) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d₀ e₀"
    and init_env_le: "e₀ ≤ genv global_of sigma"
    and dead: "lookup_coverage analyse_result v ctx = Covered Bot"
  shows "A_κ,adm,c₀,g,S0 v ctx = {}"
  by (rule reported_covered_unreachable_empty
      [OF dead analyse_result_node_sound[OF entry_cov init_sound init_env_le]])

lemma covered_keys_analyse_result [simp]: "covered_keys analyse_result = vars"
  unfolding analyse_result_def by simp
```

The node-level statement, one activation bucket at a time. A covered context is `Bot` by the node predicate; an uncovered one reads `Bot` from `lookup_table` anyway, and here that is not a shrug — this locale's own soundness already puts the collect inside an empty concretization there. So every bucket is empty and so is their union.

`node_collect` decomposes into those buckets only for a context relation that comes from a function, which this locale does not assume of `adm`; the decomposition is therefore a hypothesis, discharged by `node_collect_eq_Union_activation_of_` at every instance whose relation has that shape.

```
lemma analyse_result_node_unreachable:
  fixes S0 :: "store set" and c₀ :: 'c
  assumes entry_cov: "(cfg_entry g, c₀) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d₀ e₀"
    and init_env_le: "e₀ ≤ genv global_of sigma"
    and union: "C_κ,g,S0 v
                  = (⋃ctx. A_κ,adm,c₀,g,S0 v ctx)"
    and dead: "result_node_is_bottom analyse_result v"
  shows "C_κ,g,S0 v = {}"
proof -
  have bucket: "A_κ,adm,c₀,g,S0 v ctx = {}" for ctx
  proof -
    have bot: "lookup_table analyse_result v ctx = Bot"
    proof (cases "(v, ctx) ∈ vars")
      case True
      have "lookup_coverage analyse_result v ctx = Covered Bot"
        using True by (intro result_node_is_bottomD[OF dead]) simp
      then show ?thesis unfolding lookup_table_eq_or_bot by simp
    next
      case False
      then show ?thesis unfolding lookup_table_analyse_result by simp
    qed
    have "A_κ,adm,c₀,g,S0 v ctx
          ⊆ gamma_lift γ_V (lookup_table analyse_result v ctx)"
      by (rule analyse_result_node_sound[OF entry_cov init_sound init_env_le])
    then show ?thesis unfolding bot by simp
  qed
  show ?thesis unfolding union using bucket by simp
qed
```

The context-indexed check report

The report the CLI prints, defined at the context-indexed level rather than after the join: one verdict per check, aggregated over exactly the contexts the solved table covers at that node. Aggregating first would classify against a joined state and lose the precision each context carries.

```
definition analyse_report_ctx :: "(pp × exp × contextual_verdict) list" where
  "analyse_report_ctx = classify_checks_verdicts g analyse_result classify"
```

Report soundness

S0 and c_0 are the two facts genuinely external to this locale: which concrete stores the analysis actually starts from, and which context the solved system covers the entry point under. Every existing concrete instance (e.g. Interval's entry-state and call-string contexts) discharges both as per-instance facts about its own solved system, not as locale theorems, since they depend on which keys a terminated solve actually reached — `entry_cov` mirrors those instances' own `entry_cov` context assumption, and `init_sound` is the initial-store premise every activation-trace-level soundness theorem in this development already takes.

Both endpoints below reach their classifier obligation the same way, and that route is this lemma: a collected store at a decided check sits inside some reachable state the report classified, and the report's verdict is what the classifier said about that very state. Only the appeal to `classify_proved` or `classify_refuted` afterwards differs.

```
lemma analyse_report_ctx_decided:
  fixes S0 :: "store set" and c₀ :: 'c and v :: cfg_node and c :: exp
  assumes mem: "(v, c, Decided r) ∈ set analyse_report_ctx"
    and entry_cov: "(cfg_entry g, c₀) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d₀ e₀"
    and init_env_le: "e₀ ≤ genv global_of sigma"
    and smem: "s ∈ 𝒜_κ,adm,c₀,g,S0 v ctx"
    and known: "r ≠ Check_Unknown"
  obtains st where "s ∈ γ_V st" and "classify c st = r"
proof -
  have node_sound: "𝒜_κ,adm,c₀,g,S0 v ctx
     ⊆ gamma_lift γ_V (lookup_table analyse_result v ctx)"
    by (rule analyse_result_node_sound[OF entry_cov init_sound init_env_le])
  obtain st where reach: "lookup_table analyse_result v ctx = Lifted st"
    and sst: "s ∈ γ_V st"
    using smem node_sound by (cases "lookup_table analyse_result v ctx") auto
  have "classify c st = r"
    using classify_checks_ctx_decided_sound[OF finE
          mem[unfolded analyse_report_ctx_def] reach known] .
  with sst show ?thesis by (rule that)
qed

theorem analyse_report_ctx_proved_sound:
  fixes S0 :: "store set" and c₀ :: 'c and v :: cfg_node and c :: exp
  assumes mem: "(v, c, Decided Check_Proved) ∈ set analyse_report_ctx"
    and entry_cov: "(cfg_entry g, c₀) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d₀ e₀"
    and init_env_le: "e₀ ≤ genv global_of sigma"
  shows "⋀ctx s. s ∈ 𝒜_κ,adm,c₀,g,S0 v ctx
          ⟹ truthy (eval c s)"
proof -
  fix ctx s
  assume smem: "s ∈ 𝒜_κ,adm,c₀,g,S0 v ctx"
  obtain st where sst: "s ∈ γ_V st" and "classify c st = Check_Proved"
    by (rule analyse_report_ctx_decided[OF mem entry_cov init_sound init_env_le smem]) simp
  thus "truthy (eval c s)" using classify_proved[OF _ sst] by blast
qed

theorem analyse_report_ctx_refuted_sound:
  fixes S0 :: "store set" and c₀ :: 'c and v :: cfg_node and c :: exp
  assumes mem: "(v, c, Decided Check_Refuted) ∈ set analyse_report_ctx"
    and entry_cov: "(cfg_entry g, c₀) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d₀ e₀"
    and init_env_le: "e₀ ≤ genv global_of sigma"
  shows "⋀ctx s. s ∈ 𝒜_κ,adm,c₀,g,S0 v ctx
          ⟹ ¬ truthy (eval c s)"
proof -
  fix ctx s
  assume smem: "s ∈ 𝒜_κ,adm,c₀,g,S0 v ctx"
  obtain st where sst: "s ∈ γ_V st" and "classify c st = Check_Refuted"
    by (rule analyse_report_ctx_decided[OF mem entry_cov init_sound init_env_le smem]) simp
```

```
  thus "¬ truthy (eval c s)" using classify_refuted[OF _ sst] by blast
qed

end

end
```

6.29 Routed_Analysis_Sound

```
theory Routed_Analysis_Sound
  imports DG_Analysis_Adapter
begin
```

Reading a solved routed system as an analysis result

A routed analysis is assembled from two independent choices: which abstract domain it computes in, and which context policy it routes calls by. This theory is the composition step that does not depend on either. It takes a solved routed equation system and derives the analysis-level soundness statement, so a concrete analysis supplies its domain facts and its policy facts and interprets this once, rather than repeating the derivation per (domain, policy) pair.

`solved_local_reader` is the one reader every instance uses: at a covered unknown it hands back the solution's local half, and elsewhere `bot`. Reading a global key gives `bot` too – the routed seed keys carry entry contributions, not program-point values, and a result table never reads them.

```
definition solved_local_reader ::
  "(pp × 'c) set ⇒ (pp × 'c + 'k ⇒ ('D::bounded_semilattice_sup_bot, 'G) dg_state)
   ⇒ pp × 'c + 'k ⇒ 'D"
where
  "solved_local_reader vars sigma k =
     (case k of Inl vc ⇒ (if vc ∈ vars then dg_local (sigma (Inl vc)) else bot)
              | Inr _ ⇒ bot)"

lemma solved_local_reader_covered [simp]:
  "vc ∈ vars ⟹ solved_local_reader vars sigma (Inl vc) = dg_local (sigma (Inl vc))"
  by (simp add: solved_local_reader_def)

lemma solved_local_reader_uncovered [simp]:
  "vc ∉ vars ⟹ solved_local_reader vars sigma (Inl vc) = bot"
  by (simp add: solved_local_reader_def)

lemma solved_local_reader_global [simp]:
  "solved_local_reader vars sigma (Inr k) = bot"
  by (simp add: solved_local_reader_def)
```

The reader returns only the local half. The global half enters through the concretization the locale below pairs with it, which publishes each local value together with the solved global at `Inr buffer_key`. Under that pairing the two coverage obligations `dg_context_activation` asks for hold by construction, given `gammaDG_rd` and that the publication map takes `bot` to `Bot`. Neither depends on the domain or the context policy, so no instance need prove them again.

The composition locale

Everything a routed analysis needs above its solved system, in one place: the domain enters through `S` and γ_{DG}, the context policy through `route`, `adm` and `seed`, and the solved system through `sigma`/`vars`. The fixed reader is `solved_local_reader`, and its concretization reads the solved global at `Inr buffer_key` as the second argument of `rd`, so the two coverage obligations are the one-line lemmas above.

An instance is then a single **interpretation**, and the theorems below are what it gets: a published result table, its per-node soundness, and the check report's proved/refuted verdicts.

```
locale routed_analysis =
  dg_analysis_adapter κ S γ_DG g buffer_key global_of route bot0 d₀ g₀ sigma
    vars x0 "solved_local_reader vars sigma" seed is_bot
    "λd. gamma_lift γ_V (rd d (genv global_of sigma))"
    adm rd γ_V empty_V classify
  for κ :: kenv
    and S ::
      "(pp × 'c, 'k, 'n, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec"
    and γ_DG :: "'D ⇒ ('n ⇒ 'G) ⇒ store set"
    and g buffer_key and global_of :: "'n ⇒ 'k"
    and route :: "pp ⇒ 'c ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'c"
    and bot0 d₀ :: 'D and g₀ :: 'G
    and sigma :: "pp × 'c + 'k ⇒ ('D, 'G) dg_state"
    and vars :: "(pp × 'c) set"
    and x0 :: "pp × 'c"
    and seed :: "pp ⇒ 'c ⇒ 'k"
    and is_bot :: "'D ⇒ bool"
    and adm :: "'c call_context_policy"
    and rd :: "'D ⇒ ('n ⇒ 'G) ⇒ 'v lifted"
```

```
    and γ_V :: "'v ⇒ store set"
    and empty_V :: "'v ⇒ bool"
    and classify :: "exp ⇒ 'v ⇒ check_result"

end
```

Chapter 7

Voblint_Exec

7.1 Default_St_Base

```
theory Default_St_Base
  imports "Voblint_VIMP.VIMP_Syntax" "HOL-Library.AList"
begin
```

Finite default-map representation

An abstract state has to be a finite thing before a solver can run on it. A `gdefault_dict` is a finite dictionary that answers every name it does not list with a default. A `gdefault_st_rep` is two of them, one per partition: a local dictionary and a global one. Reading a location consults the dictionary of its partition. Nothing forces a stored value to differ from its default.

Two such pairs can describe the same reading, so the type is quotiented by agreement of all lookups. That identifies list order and redundant duplicate entries only where they do not affect the reading: `map_of` answers with the first match, so reordering entries that disagree on a key is a genuine change and the quotient keeps it.

This theory settles the representation, that quotient, and the order, equality and point update defined on it. It does not decide which variable names are local and which are global – a `location` records a classification this theory never makes. That classifier arrives only in `Default_St_Transfer`, which is built on this.

Why two defaults, rather than a sparse map over a fixed `top`. Nipkow's `Abs_State` stores only the variables it has heard of and reads every other one as `top`. That is the smaller representation, and it makes the order and the join shorter to state, so the extra pair of defaults here needs a reason. Three, in fact, and each is load-bearing on its own.

- **C zero-initialization.** Every domain's entry state gives globals a non-`top` value and locals `top` — `⦇(STop, []), (SZero, [])⦈` for Sign, and the parity, interval and product analyses match it. It over-approximates `cinit_stores` $\mathcal{G}$ `= {s. ∀x. `$\mathcal{G}$ `x ⟶ s x = 0}`, which quantifies over *all* names the classifier calls global, for an arbitrary classifier and with no finiteness hypothesis. A fixed-`top` map can only express that by materializing every global, which makes the entry state a function of the program's declaration list and pushes that dependency into every soundness statement that mentions it.
- **The ownership split needs `bot` on the discarded side.** Publishing a state's global half to a shared unknown sends locals to `bot`, and `bot` is what makes that half a unit for the join that combines publications from every call site. `top` there would not be a unit. These half-states are not transient: they are stored in solver unknowns and compared against the solution.
- **The solver needs a least element on this type.** The ownership-split specification types both its local and global unknowns at this carrier and demands `bounded_semilattice_sup_bot` of it, so unknowns can start at `bot`. That element is the everywhere-`bot` state, which a sparse map over a fixed `top` cannot represent at all.

  What the defaults cost is visible throughout this theory: the order has to compare them, extensional equality has to pin them, and an emptiness test has to reason about them. That cost is the price of the three points above, not an accident of the first representation tried.

Default dictionaries

A key type with infinitely many keys: a finite dictionary always leaves one unlisted.

```
class infinite_key =
  assumes infinite_key_UNIV: "infinite (UNIV :: 'a set)"

instance String.literal :: infinite_key
  by standard (rule infinite_literal)

type_synonym ('k, 'a) gdefault_dict = "'a × ('k × 'a) list"

fun default_dict_get :: "('k, 'a) gdefault_dict => 'k => 'a" where
  "default_dict_get (d, ps) x = (case map_of ps x of Some a => a | None => d)"

fun default_dict_set :: "('k, 'a) gdefault_dict => 'k => 'a => ('k, 'a) gdefault_dict" where
  "default_dict_set (d, ps) x a = (d, (x, a) # AList.delete x ps)"

lemma map_of_resolved_delete:
  "map_of (AList.delete k ps) k' =
     (if k = k' then None else map_of ps k')"
  by (simp add: AList.delete_conv')
```

```
lemma map_of_resolved_none_iff:
  "map_of ps k = None ⟷ k ∉ set (map fst ps)"
  by (induction ps) auto

lemma default_dict_get_set [simp]:
  "default_dict_get (default_dict_set m x a) y =
     (if x = y then a else default_dict_get m y)"
  by (cases m) (simp add: map_of_resolved_delete)
```

A name escaping a finite list, drawn from any infinite supply. It is not listed, so a dictionary answers it with its default. Consumers instantiate the supply with all vnames, or with the ones a classifier leaves local.

```
lemma obtain_fresh_vname:
  assumes "infinite A"
  obtains x where "x ∈ A" and "x ∉ set xs"
proof -
  have "infinite (A - set xs)"
    by (rule Diff_infinite_finite[OF _ assms]) simp
  then obtain x where "x ∈ A - set xs"
    using infinite_imp_nonempty by blast
  then have "x ∈ A" "x ∉ set xs" by simp_all
  then show thesis by (rule that)
qed
```

The order on one dictionary compares the defaults and the finitely many listed names. An unlisted name reads the default on both sides, so this decides the pointwise order over all names.

```
fun le_default_dict :: "('k, 'a::order) gdefault_dict => ('k, 'a) gdefault_dict => bool" where
  "le_default_dict (d, ps) (e, qs) ⟷
     d ≤ e ∧
     list_all (λx. default_dict_get (d, ps) x ≤ default_dict_get (e, qs) x)
       (map fst ps @ map fst qs)"

lemma le_default_dict_iff:
  fixes m n :: "('k::infinite_key, 'a::order) gdefault_dict"
  shows "le_default_dict m n ⟷ (∀x. default_dict_get m x ≤ default_dict_get n x)"
proof -
  obtain d ps where m: "m = (d, ps)" by (cases m)
  obtain e qs where n: "n = (e, qs)" by (cases n)
  show ?thesis
  proof
    assume le: "le_default_dict m n"
    show "∀x. default_dict_get m x ≤ default_dict_get n x"
    proof
      fix x
      show "default_dict_get m x ≤ default_dict_get n x"
      proof (cases "x ∈ set (map fst ps @ map fst qs)")
        case True
        with le show ?thesis unfolding m n
          by (auto simp: list_all_iff simp del: default_dict_get.simps)
      next
        case False
        then have "map_of ps x = None" "map_of qs x = None"
          by (simp_all add: map_of_resolved_none_iff)
        with le show ?thesis unfolding m n by simp
      qed
    qed
  next
    assume le: "∀x. default_dict_get m x ≤ default_dict_get n x"
    obtain y :: 'k where "y ∈ UNIV" and fresh: "y ∉ set (map fst ps @ map fst qs)"
      by (rule obtain_fresh_vname[OF infinite_key_UNIV])
    then have "map_of ps y = None" "map_of qs y = None"
      by (simp_all add: map_of_resolved_none_iff)
    with le[rule_format, of y] have "d ≤ e" unfolding m n by simp
    with le show "le_default_dict m n" unfolding m n by (simp add: list_all_iff)
  qed
qed
```

Locations and raw lookup

A `location` names a local or a global variable, and a raw state is a pair of dictionaries read by `default_st_rep_get`.

```
datatype 'k glocation =
  Local_Location (location_vname: 'k)
| Global_Location (location_vname: 'k)
```

```
lemma all_location_iff:
  "(∀loc. P loc) ⟷ (∀x. P (Local_Location x)) ∧ (∀x. P (Global_Location x))"
  by (metis glocation.exhaust)
```

The local dictionary comes first, the global one second.

```
type_synonym ('k, 'a) gdefault_st_rep = "('k, 'a) gdefault_dict × ('k, 'a) gdefault_dict"

fun default_st_rep_get ::
  "('k, 'a::bot) gdefault_st_rep => 'k glocation => 'a" where
  "default_st_rep_get (l, g) (Local_Location x) = default_dict_get l x"
| "default_st_rep_get (l, g) (Global_Location x) = default_dict_get g x"
```

Extensional equality

Two raw states are equal when they agree on every location's lookup, whatever their dictionaries look like; `eq_default_st_rep` is an equivalence.

```
definition eq_default_st_rep ::
  "('k, 'a::bot) gdefault_st_rep => ('k, 'a) gdefault_st_rep => bool"
where
  "eq_default_st_rep s t ⟷
     default_st_rep_get s = default_st_rep_get t"

lemma equivp_eq_default_st_rep: "equivp eq_default_st_rep"
  unfolding eq_default_st_rep_def
  by (rule equivpI) (auto intro: reflpI sympI transpI)
```

The two rules every congruence proof below goes through: a raw operation respects `eq_default_st_rep` exactly when its lookup equation says the result depends on the arguments only through their lookups. Stating that once keeps the individual congruence proofs down to their own lookup rules.

```
lemma eq_default_st_repI [intro]:
  assumes "⋀loc. default_st_rep_get s loc = default_st_rep_get t loc"
  shows "eq_default_st_rep s t"
  unfolding eq_default_st_rep_def fun_eq_iff using assms by blast

lemma eq_default_st_repD:
  assumes "eq_default_st_rep s t"
  shows "default_st_rep_get s loc = default_st_rep_get t loc"
  using assms unfolding eq_default_st_rep_def fun_eq_iff by blast
```

Executable pointwise order

`le_default_st_rep_code` compares both dictionaries with `le_default_dict` and is shown equal to the pointwise order on lookups.

```
fun le_default_st_rep_code ::
  "('k, 'a::order_bot) gdefault_st_rep => ('k, 'a) gdefault_st_rep => bool"
where
  "le_default_st_rep_code (l1, g1) (l2, g2) ⟷
     le_default_dict l1 l2 ∧ le_default_dict g1 g2"
```

The characterization avoids the pair pattern, so that quotient-level statements can be discharged by `transfer` alone instead of re-opening both representatives.

```
lemma le_default_st_rep_code_iff:
  fixes s t :: "('k::infinite_key, 'a::order_bot) gdefault_st_rep"
  shows "le_default_st_rep_code s t ⟷
    (∀loc. default_st_rep_get s loc ≤ default_st_rep_get t loc)"
  by (cases s; cases t) (simp add: all_location_iff le_default_dict_iff)
```

The extensional quotient, lookup and point update

`gdefault_st` is the quotient of raw states by extensional equality. Lookup and point update lift to it because both respect the equivalence.

```
quotient_type ('k, 'a) gdefault_st =
  "('k, 'a::bot) gdefault_st_rep" / "eq_default_st_rep"
  morphisms rep_default_st Abs_default_st
  by (rule equivp_eq_default_st_rep)


lift_definition default_st_get ::
  "('k, 'a::bot) gdefault_st => 'k glocation => 'a"
```

```
  is default_st_rep_get
  by (simp add: eq_default_st_rep_def)

fun default_st_rep_set ::
  "('k, 'a::bot) gdefault_st_rep => 'k glocation => 'a => ('k, 'a) gdefault_st_rep" where
  "default_st_rep_set (l, g) (Local_Location x) a = (default_dict_set l x a, g)"
| "default_st_rep_set (l, g) (Global_Location x) a = (l, default_dict_set g x a)"

lemma default_st_rep_get_set [simp]:
  "default_st_rep_get (default_st_rep_set s loc a) loc' =
     (if loc = loc' then a else default_st_rep_get s loc')"
  by (cases s; cases loc; cases loc') simp_all

lemma eq_default_st_rep_set:
  assumes "eq_default_st_rep s t"
  shows "eq_default_st_rep (default_st_rep_set s loc a)
      (default_st_rep_set t loc a)"
  by (rule eq_default_st_repI) (simp add: eq_default_st_repD[OF assms])

lift_definition default_st_set ::
  "('k, 'a::bot) gdefault_st => 'k glocation => 'a => ('k, 'a) gdefault_st"
  is default_st_rep_set
  by (rule eq_default_st_rep_set)
```

Lookup is written `s⟨l⟩` and point update `s⟨l := a⟩`. The notation is a bundle: every theory that uses it opens it after `begin` and closes it with `unbundle no` before `end`, so it never reaches an importing theory and never changes a term. Both forms bind tighter than application, so `f s⟨l⟩` reads `f (s⟨l⟩)` and updates chain as `s⟨l := a⟩⟨l'⟩`. The state with local dictionary `(dl, ls)` and global dictionary `(dg, gs)` is written `⦇(dl, ls), (dg, gs)⦈`. Theories past `default_st_to_fun` open `default_st_syntax`, which adds the notation for the function a state represents to this bundle.

```
abbreviation default_st_mk ::
  "('k, 'a::bot) gdefault_dict => ('k, 'a) gdefault_dict => ('k, 'a) gdefault_st" where
  "default_st_mk l g ≡ Abs_default_st (l, g)"

bundle default_st_carrier_syntax
begin
notation default_st_get ("_⟨_⟩" [1000, 0] 1000)
notation default_st_set ("_⟨_ :=/ _⟩" [1000, 0, 0] 1000)
notation default_st_mk ("⦇_,/ _⦈")
end

unbundle default_st_carrier_syntax

lemma default_st_get_Abs [simp]:
  "(Abs_default_st s)⟨loc⟩ = default_st_rep_get s loc"
  by transfer simp

lemma Abs_default_st_rep_default_st [simp]:
  "Abs_default_st (rep_default_st s) = s"
  by (fact Lifting.Quotient_abs_rep [OF Quotient_gdefault_st])

lemma default_st_get_rep:
  "s⟨loc⟩ = default_st_rep_get (rep_default_st s) loc"
  by (simp add: default_st_get.rep_eq)
```

One lookup equation for the update, rather than a same/different pair: the conditional is what a caller has to reason about anyway, and the two special cases fall out of it by `simp`.

```
lemma default_st_get_set [simp]:
  "s⟨loc := a⟩⟨loc'⟩ = (if loc = loc' then a else s⟨loc'⟩)"
  by transfer simp

lemma default_st_eq_iff:
  "s = t ⟷
     default_st_get s = default_st_get t"
  by transfer (simp add: eq_default_st_rep_def)
```

The quotient-level counterpart of `eq_default_st_repI`: two quotient states are equal as soon as they look the same everywhere. Left untagged – as an `[intro]` rule it would attack every equation at this type by extensionality, which is rarely what a goal about a concrete state wants.

```
lemma default_st_eqI:
  assumes "⋀loc. s⟨loc⟩ = t⟨loc⟩"
  shows "s = t"
  using assms by (simp add: default_st_eq_iff fun_eq_iff)
```

Order, bottom and executable equality

Bottom is the pair of empty dictionaries with bottom defaults; the order lifts `le_default_st_rep_code`, which also yields an executable equality.

```
instantiation gdefault_st :: (type, bot) bot
begin
definition bot_gdefault_st ::
  "('a, 'b::bot) gdefault_st"
where
  "bot_gdefault_st = ⦇(bot, []), (bot, [])⦈"
instance ..
end

instantiation gdefault_st :: (infinite_key, order_bot) ord
begin
lift_definition less_eq_gdefault_st ::
  "('a::infinite_key, 'b::order_bot) gdefault_st =>
   ('a, 'b) gdefault_st => bool"
  is le_default_st_rep_code
  by (auto simp: le_default_st_rep_code_iff eq_default_st_rep_def)

definition less_gdefault_st ::
  "('a::infinite_key, 'b::order_bot) gdefault_st =>
   ('a, 'b) gdefault_st => bool"
where
  "less_gdefault_st s t ⟷
     s ≤ t ∧ ¬ t ≤ s"

instance ..
end


lemma le_default_st_iff:
  fixes s t :: "('k::infinite_key, 'a::order_bot) gdefault_st"
  shows "s ≤ t ⟷ (∀loc. s⟨loc⟩ ≤ t⟨loc⟩)"
  by transfer (rule le_default_st_rep_code_iff)


instance gdefault_st :: (infinite_key, order_bot) order
proof intro_classes
  fix s t u :: "('a::infinite_key, 'b::order_bot) gdefault_st"
  show "(s < t) ⟷ (s ≤ t ∧ ¬ t ≤ s)"
    by (simp add: less_gdefault_st_def)

  show "s ≤ s"
    by (simp add: le_default_st_iff)
  show "s ≤ t ⟹ t ≤ u ⟹ s ≤ u"
    by (auto simp: le_default_st_iff intro: order_trans)
  show "s ≤ t ⟹ t ≤ s ⟹ s = t"
    by (auto simp: le_default_st_iff
      default_st_eq_iff fun_eq_iff intro: order_antisym)
qed

context
  includes lattice_syntax
begin

lemma default_st_get_bot [simp]:
  "(⊥ :: ('k, 'a::bot) gdefault_st)⟨loc⟩ = ⊥"
  unfolding bot_gdefault_st_def by (cases loc) simp_all

end


lemma bot_le_default_st:
  "(bot :: ('k::infinite_key, 'a::order_bot) gdefault_st) ≤ s"
  by (simp add: le_default_st_iff)

instance gdefault_st :: (infinite_key, order_bot) order_bot
  by standard (rule bot_le_default_st)
instantiation gdefault_st :: (infinite_key, "{order_bot,equal}") equal
begin
definition equal_gdefault_st ::
  "('a::infinite_key, 'b::{order_bot,equal}) gdefault_st =>
   ('a, 'b) gdefault_st => bool"
where
```

```
  "equal_gdefault_st s t = (s ≤ t ∧ t ≤ s)"
instance
  by standard (auto simp: equal_gdefault_st_def intro: order_antisym)
end

unbundle no default_st_carrier_syntax

end
```

7.2 Default_St_Algebra

```
theory Default_St_Algebra
  imports Default_St_Base "Voblint_Domain.State_Concretization"
begin

unbundle default_st_carrier_syntax
```

Pointwise abstract-domain operations

Join, widening and narrowing are the same construction three times: apply an element operation to both dictionaries of a state, at the default and at every name either argument lists. This theory gives that construction once, instantiates it three times, and installs the lattice and warrowing classes the solver needs.

The combinator's support is the two argument supports with duplicate keys removed. Keeping both copies would denote the same lookup function, but every join would concatenate, and a solver iteration would then grow the materialized list without bound while each lookup scans it.

The pointwise combinator

```
fun map2_default_dict ::
  "('a => 'a => 'a) => ('k, 'a) gdefault_dict => ('k, 'a) gdefault_dict => ('k, 'a) gdefault_dict"
where
  "map2_default_dict f (d1, ps1) (d2, ps2) =
     (f d1 d2,
      map (λx. (x, f (default_dict_get (d1, ps1) x) (default_dict_get (d2, ps2) x)))
        (remdups (map fst ps1 @ map fst ps2)))"

fun map2_default_st_rep ::
  "('a => 'a => 'a) => ('k, 'a::bot) gdefault_st_rep => ('k, 'a) gdefault_st_rep =>
   ('k, 'a) gdefault_st_rep"
where
  "map2_default_st_rep f (l1, g1) (l2, g2) =
     (map2_default_dict f l1 l2, map2_default_dict f g1 g2)"

lemma map_of_map_key_list:
  "map_of (map (λk. (k, g k)) ks) x =
     (if x ∈ set ks then Some (g x) else None)"
  by (induction ks) auto

lemma default_dict_get_map2 [simp]:
  "default_dict_get (map2_default_dict f m1 m2) x =
     f (default_dict_get m1 x) (default_dict_get m2 x)"
  by (cases m1; cases m2)
    (auto simp: map_of_map_key_list map_of_resolved_none_iff[THEN iffD2])
```

From here on the combinator is read through its lookup equation. Its defining equation would otherwise unfold first and hide it from `default_dict_get_map2`.

```
declare map2_default_dict.simps [simp del]

lemma default_st_rep_get_map2 [simp]:
  "default_st_rep_get (map2_default_st_rep f s t) loc =
     f (default_st_rep_get s loc) (default_st_rep_get t loc)"
  by (cases s; cases t; cases loc) simp_all

lemma eq_default_st_rep_map2:
  assumes "eq_default_st_rep s1 s2"
    and "eq_default_st_rep t1 t2"
  shows "eq_default_st_rep (map2_default_st_rep f s1 t1)
      (map2_default_st_rep f s2 t2)"
  by (rule eq_default_st_repI)
     (simp add: eq_default_st_repD[OF assms(1)] eq_default_st_repD[OF assms(2)])
```

Join and semilattice structure

Join is the pointwise join of both dictionaries, which respects extensional equality and so lifts to `gdefault_st` as a bounded join-semilattice.

```
definition merge_default_st_rep ::
  "('k, 'a::bounded_semilattice_sup_bot) gdefault_st_rep =>
   ('k, 'a) gdefault_st_rep => ('k, 'a) gdefault_st_rep"
where
  "merge_default_st_rep s t = map2_default_st_rep (⊔) s t"
```

```
lemma default_st_rep_get_merge [simp]:
  "default_st_rep_get (merge_default_st_rep s t) loc =
     default_st_rep_get s loc ⊔ default_st_rep_get t loc"
  by (simp add: merge_default_st_rep_def)

lemma eq_default_st_rep_merge:
  assumes "eq_default_st_rep s1 s2"
    and "eq_default_st_rep t1 t2"
  shows "eq_default_st_rep (merge_default_st_rep s1 t1)
      (merge_default_st_rep s2 t2)"
  using assms unfolding merge_default_st_rep_def
  by (rule eq_default_st_rep_map2)

instantiation gdefault_st ::
  (infinite_key, bounded_semilattice_sup_bot) sup
begin
lift_definition sup_gdefault_st ::
  "('a::infinite_key, 'b::bounded_semilattice_sup_bot) gdefault_st =>
   ('a, 'b) gdefault_st => ('a, 'b) gdefault_st"
  is merge_default_st_rep
  by (rule eq_default_st_rep_merge)
instance ..
end

lemma default_st_get_sup [simp]:
  "(s ⊔ t)⟨loc⟩ = s⟨loc⟩ ⊔ t⟨loc⟩"
  by transfer (rule default_st_rep_get_merge)

instance gdefault_st ::
  (infinite_key, bounded_semilattice_sup_bot) semilattice_sup
proof intro_classes
  fix s t u :: "('a::infinite_key, 'b::bounded_semilattice_sup_bot) gdefault_st"
  show "s ≤ s ⊔ t"
    by (simp add: le_default_st_iff)
  show "t ≤ s ⊔ t"
    by (simp add: le_default_st_iff)
  show "s ≤ u ⟹ t ≤ u ⟹ s ⊔ t ≤ u"
    by (simp add: le_default_st_iff)
qed

instance gdefault_st ::
  (infinite_key, bounded_semilattice_sup_bot) bounded_semilattice_sup_bot ..
```

Widening and narrowing

Widening and narrowing act pointwise as well, inheriting the element domain's `warrowing` laws location by location.

```
definition widen_default_st_rep ::
  "('k, 'a::{bounded_semilattice_sup_bot, warrowing}) gdefault_st_rep =>
   ('k, 'a) gdefault_st_rep => ('k, 'a) gdefault_st_rep"
where
  "widen_default_st_rep s t = map2_default_st_rep (∇) s t"

lemma default_st_rep_get_widen [simp]:
  "default_st_rep_get (widen_default_st_rep s t) loc =
     default_st_rep_get s loc ∇ default_st_rep_get t loc"
  by (simp add: widen_default_st_rep_def)

lemma eq_default_st_rep_widen:
  assumes "eq_default_st_rep s1 s2"
    and "eq_default_st_rep t1 t2"
  shows "eq_default_st_rep (widen_default_st_rep s1 t1)
      (widen_default_st_rep s2 t2)"
  using assms unfolding widen_default_st_rep_def
  by (rule eq_default_st_rep_map2)
```

Widening and narrowing are lifted to a named constant first and only then installed as the class operation, unlike (⊔) which is lifted inside its own `instantiation`. The detour is forced: the `widening` and `narrowing` classes carry axioms, so their `instance` proofs need the lookup equation, and inside an `instantiation` block the class operation is not yet the same term as a constant lifted there – a lookup lemma stated on the latter fails to refine a goal about the former, and one proved before `instance` abstracts over the operation instead of fixing it. (⊔) escapes this because the bare `sup` class has no axioms to discharge.

```
lift_definition widen_on_default_st ::
```

```
  "('k::infinite_key, 'a::{bounded_semilattice_sup_bot, warrowing}) gdefault_st =>
   ('k, 'a) gdefault_st => ('k, 'a) gdefault_st"
  is widen_default_st_rep
  by (rule eq_default_st_rep_widen)

lemma default_st_get_widen_on [simp]:
  "(widen_on_default_st s t)⟨loc⟩ =
     s⟨loc⟩ ∇ t⟨loc⟩"
  by transfer (rule default_st_rep_get_widen)

instantiation gdefault_st :: (infinite_key, "{bounded_semilattice_sup_bot, warrowing}") widening
begin
definition widen_gdefault_st ::
  "('a::infinite_key, 'b::{bounded_semilattice_sup_bot, warrowing}) gdefault_st =>
   ('a, 'b) gdefault_st => ('a, 'b) gdefault_st"
where
  "widen_gdefault_st s t = widen_on_default_st s t"
instance
proof
  fix a b :: "('a::infinite_key, 'b::{bounded_semilattice_sup_bot, warrowing}) gdefault_st"
  show "a ≤ (a ∇ b)"
    by (simp add: le_default_st_iff widen_gdefault_st_def widen_ge1)
  show "b ≤ (a ∇ b)"
    by (simp add: le_default_st_iff widen_gdefault_st_def widen_ge2)
qed
end

lemma default_st_get_widen [simp]:
  "(s ∇ t)⟨loc⟩ = s⟨loc⟩ ∇ t⟨loc⟩"
  by (simp add: widen_gdefault_st_def)

definition narrow_default_st_rep ::
  "('k, 'a::{bounded_semilattice_sup_bot, warrowing}) gdefault_st_rep =>
   ('k, 'a) gdefault_st_rep => ('k, 'a) gdefault_st_rep"
where
  "narrow_default_st_rep s t = map2_default_st_rep (∆) s t"

lemma default_st_rep_get_narrow [simp]:
  "default_st_rep_get (narrow_default_st_rep s t) loc =
     default_st_rep_get s loc ∆ default_st_rep_get t loc"
  by (simp add: narrow_default_st_rep_def)

lemma eq_default_st_rep_narrow:
  assumes "eq_default_st_rep s1 s2"
    and "eq_default_st_rep t1 t2"
  shows "eq_default_st_rep (narrow_default_st_rep s1 t1)
      (narrow_default_st_rep s2 t2)"
  using assms unfolding narrow_default_st_rep_def
  by (rule eq_default_st_rep_map2)

lift_definition narrow_on_default_st ::
  "('k::infinite_key, 'a::{bounded_semilattice_sup_bot, warrowing}) gdefault_st =>
   ('k, 'a) gdefault_st => ('k, 'a) gdefault_st"
  is narrow_default_st_rep
  by (rule eq_default_st_rep_narrow)

lemma default_st_get_narrow_on [simp]:
  "(narrow_on_default_st s t)⟨loc⟩ =
     s⟨loc⟩ ∆ t⟨loc⟩"
  by transfer (rule default_st_rep_get_narrow)

instantiation gdefault_st :: (infinite_key, "{bounded_semilattice_sup_bot, warrowing}") narrowing
begin
definition narrow_gdefault_st ::
  "('a::infinite_key, 'b::{bounded_semilattice_sup_bot, warrowing}) gdefault_st =>
   ('a, 'b) gdefault_st => ('a, 'b) gdefault_st"
where
  "narrow_gdefault_st s t = narrow_on_default_st s t"
instance
  by standard
     (auto simp add: le_default_st_iff narrow_gdefault_st_def
       intro: narrow_ge narrow_le)
end

lemma default_st_get_narrow [simp]:
  "(s ∆ t)⟨loc⟩ = s⟨loc⟩ ∆ t⟨loc⟩"
```

```
  by (simp add: narrow_gdefault_st_def)

instance gdefault_st :: (infinite_key, "{bounded_semilattice_sup_bot, warrowing}") warrowing ..

unbundle no default_st_carrier_syntax

end
```

7.3 Default_St_Transfer

```
theory Default_St_Transfer
  imports Default_St_Algebra "Voblint_Framework.Transfer_Algebra"
begin

unbundle default_st_carrier_syntax
```

Refinement to variable-indexed states

Everything so far is stated about locations. A program, though, talks about variable names, and which names are global is a property of the program, not of the carrier. `location_of` is that classifier applied: it turns a name into the location holding its value, and `default_st_rep_to_fun` maps a whole state to the function `('k, 'a) gabs_state` it represents, the form soundness is stated over.

The rest of the theory is the operations a call and a return need – assignment, formal binding, restriction to one side of the ownership split, frame entry, and the caller/callee combination – each paired with the equation saying that performing it on the carrier and then taking the represented function agrees with performing its counterpart on that function. Those equations are the interface every soundness proof downstream actually uses.

Location classification and projection

```
definition location_of ::
  "('k => bool) => 'k => 'k glocation" where
  "location_of 𝒢 x =
     (if 𝒢 x then Global_Location x else Local_Location x)"
```

Both tags carry the vname they classify, so a classified location determines its own name and two names collide only when they are equal. Proofs about `location_of` therefore need no case split on `𝒢 x` against `𝒢 y`.

```
lemma location_vname_location_of [simp]:
  "location_vname (location_of 𝒢 x) = x"
  by (simp add: location_of_def)

lemma location_of_eq_iff [simp]:
  "location_of 𝒢 x = location_of 𝒢 y ⟷ x = y"
  by (auto simp: location_of_def)
```

Deliberately no `[simp]` rule matching `location_of 𝒢 x = Local_Location y` against a constructor. Such a rule reads well in isolation, but proofs here derive facts of exactly that shape and then use them as rewrites for `location_of`; a simp rule on the same left-hand side collapses the equation to its classification side and takes the rewrite away.

```
definition default_st_rep_to_fun ::
  "('k => bool) => ('k, 'a::bot) gdefault_st_rep => 'k => 'a" where
  "default_st_rep_to_fun 𝒢 s x =
     default_st_rep_get s (location_of 𝒢 x)"

lemma default_st_rep_to_fun_set_location [simp]:
  "default_st_rep_to_fun 𝒢
      (default_st_rep_set s (location_of 𝒢 x) a) =
   (default_st_rep_to_fun 𝒢 s)(x := a)"
proof (rule ext)
  fix y
  show "default_st_rep_to_fun 𝒢
      (default_st_rep_set s (location_of 𝒢 x) a) y =
    ((default_st_rep_to_fun 𝒢 s)(x := a)) y"
    unfolding default_st_rep_to_fun_def    by (cases "x = y"; cases "𝒢 x"; cases "𝒢 y";
       simp_all add: location_of_def)
qed

definition default_st_to_fun ::
  "('k => bool) => ('k, 'a::bot) gdefault_st => 'k => 'a"
where
  "default_st_to_fun 𝒢 s x =
     s⟨location_of 𝒢 x⟩"
```

The function a state represents is its `readback`, written $\varrho_{\mathcal{G}}$ `s`; a lifted state reads back pointwise under the lift. The notation joins the carrier notation in the opt-in bundle `default_st_syntax`: vendored theories bind ϱ as a variable, so it must never be global.

```
adhoc_overloading readback == default_st_to_fun
```

```
bundle default_st_syntax
begin
unbundle default_st_carrier_syntax
notation readback ("ϱ_")
end

unbundle default_st_syntax

lemma default_st_to_fun_bot [simp]:
  "ϱ_𝒢 (bot :: ('k, 'a::order_bot) gdefault_st) = bot"
  by (rule ext) (simp add: default_st_to_fun_def)
```

The state a run starts in gives every local one value and every global another: C initializes a declared global to zero and leaves a local unconstrained. Each domain's C-initial state is this construction at its own abstraction of zero and its whole-value element, so the equation for its function is one lemma rather than one per domain. The two empty dictionaries are abstracted directly rather than lifted, because a domain whose value type is itself a typedef would otherwise descend through both quotients.

```
definition initial_default_st :: "'a::bot => 'a => ('k, 'a) gdefault_st" where
  "initial_default_st local_value global_value =
     ⟪(local_value, []), (global_value, [])⟫"

lemma default_st_get_initial [simp]:
  "ϱ_𝒢 (initial_default_st local_value global_value) x =
   (if 𝒢 x then global_value else local_value)"
  unfolding default_st_to_fun_def initial_default_st_def
  by (auto simp: location_of_def split: if_splits)
```

The quotient-level projection agrees with the raw one at `s`'s own chosen representative – so any raw-level fact about `default_st_rep_to_fun` transports directly to `readback` through `rep_default_st`, with no separate quotient-respectfulness argument needed: `rep_default_st` is a genuine function of `s`, so anything defined through it is automatically well-defined on the quotient.

```
lemma default_st_to_fun_rep:
  "ϱ_𝒢 s = default_st_rep_to_fun 𝒢 (rep_default_st s)"
  unfolding default_st_to_fun_def default_st_rep_to_fun_def
  by (rule ext) (simp add: default_st_get_rep)

lemma default_st_to_fun_mono:
  assumes "s ≤ t"
  shows "ϱ_𝒢 s ≤ ϱ_𝒢 t"
  using assms
  unfolding default_st_to_fun_def le_fun_def
  by (simp add: le_default_st_iff)
```

Each quotient-level refinement equation is its raw counterpart transported along the quotient: unfolding the projection leaves a goal in `default_st_get`, `transfer` lowers it to `default_st_rep_get`, and the raw lemma – read at the same unfolded shape – closes it. An operation returning the quotient has no `rep_eq` to unfold instead, since its representative is fixed only up to `eq_default_st_rep`.

Assignment and formal binding

```
definition combine_assign_default_st_rep ::
  "('k => bool) => 'k option => 'a => ('k, 'a::bot) gdefault_st_rep
   => ('k, 'a) gdefault_st_rep"
where
  "combine_assign_default_st_rep 𝒢 dst v s =
     (case dst of None => s
      | Some x => default_st_rep_set s (location_of 𝒢 x) v)"

lemma eq_default_st_rep_combine_assign:
  assumes "eq_default_st_rep s t"
  shows "eq_default_st_rep (combine_assign_default_st_rep 𝒢 dst v s)
      (combine_assign_default_st_rep 𝒢 dst v t)"
  by (unfold combine_assign_default_st_rep_def; cases dst;
      simp_all add: assms eq_default_st_rep_set)

lift_definition combine_assign_default_st ::
  "('k => bool) => 'k option => 'a => ('k, 'a::bot) gdefault_st
   => ('k, 'a) gdefault_st"
  is combine_assign_default_st_rep
  by (rule eq_default_st_rep_combine_assign)
```

```
lemma default_st_get_combine_assign [simp]:
  "(combine_assign_default_st 𝒢 dst v s)⟨loc⟩ =
     (case dst of
        None => s⟨loc⟩
      | Some x => if location_of 𝒢 x = loc then v else s⟨loc⟩)"
  by transfer (auto simp add:combine_assign_default_st_rep_def split:option.splits)

lemma default_st_rep_to_fun_combine_assign [simp]:
  "default_st_rep_to_fun 𝒢
      (combine_assign_default_st_rep 𝒢 dst v s) =
   combine_assign dst v (default_st_rep_to_fun 𝒢 s)"
  by (cases dst)
     (simp_all add: combine_assign_default_st_rep_def)

lemma default_st_to_fun_combine_assign [simp]:
  "ϱ𝒢
      (combine_assign_default_st 𝒢 dst v s) =
   combine_assign dst v (ϱ𝒢 s)"
  unfolding default_st_to_fun_def
  by transfer
     (rule default_st_rep_to_fun_combine_assign[unfolded default_st_rep_to_fun_def])

definition bind_formals_default_st_rep ::
  "('k => bool) => 'k list => 'a list => ('k, 'a::bot) gdefault_st_rep
   => ('k, 'a) gdefault_st_rep"
where
  "bind_formals_default_st_rep 𝒢 xs avs s =
     fold (λ(x, a) t. default_st_rep_set t (location_of 𝒢 x) a)
       (zip xs avs) s"

lemma eq_default_st_rep_fold_set:
  "eq_default_st_rep s t ⟹
     eq_default_st_rep
       (fold (λ(x, a) t. default_st_rep_set t (location_of 𝒢 x) a) ps s)
       (fold (λ(x, a) t. default_st_rep_set t (location_of 𝒢 x) a) ps t)"
proof (induction ps arbitrary: s t)
  case Nil
  then show ?case by simp
next
  case (Cons p ps)
  then show ?case
    by (cases p) (simp_all add: eq_default_st_rep_set)
qed

lemma eq_default_st_rep_bind_formals:
  assumes "eq_default_st_rep s t"
  shows "eq_default_st_rep (bind_formals_default_st_rep 𝒢 xs avs s)
      (bind_formals_default_st_rep 𝒢 xs avs t)"
  unfolding bind_formals_default_st_rep_def
  using assms by (rule eq_default_st_rep_fold_set)

lift_definition bind_formals_default_st ::
  "('k => bool) => 'k list => 'a list => ('k, 'a::bot) gdefault_st
   => ('k, 'a) gdefault_st"
  is bind_formals_default_st_rep
  by (rule eq_default_st_rep_bind_formals)

lemma default_st_rep_to_fun_fold_set:
  "default_st_rep_to_fun 𝒢
      (fold (λ(x, a) t. default_st_rep_set t (location_of 𝒢 x) a) ps s) =
   fold (λ(x, a) t. t(x := a)) ps
      (default_st_rep_to_fun 𝒢 s)"
  by (induction ps arbitrary: s)
     (simp_all split: prod.splits)

lemma default_st_rep_to_fun_bind_formals [simp]:
  "default_st_rep_to_fun 𝒢
      (bind_formals_default_st_rep 𝒢 xs avs s) =
   bind_formals xs avs (default_st_rep_to_fun 𝒢 s)"
  unfolding bind_formals_default_st_rep_def
  by (rule default_st_rep_to_fun_fold_set)

lemma default_st_to_fun_bind_formals [simp]:
  "ϱ𝒢
      (bind_formals_default_st 𝒢 xs avs s) =
   bind_formals xs avs (ϱ𝒢 s)"
```

```
  unfolding default_st_to_fun_def
  by transfer
     (rule default_st_rep_to_fun_bind_formals[unfolded default_st_rep_to_fun_def])

lemma default_st_to_fun_set [simp]:
  "ϱ_𝒢 s⟨location_of 𝒢 x := a⟩ =
   (ϱ_𝒢 s)(x := a)"
proof (rule ext)
  fix y
  show "ϱ_𝒢 s⟨location_of 𝒢 x := a⟩ y =
    ((ϱ_𝒢 s)(x := a)) y"
    unfolding default_st_to_fun_def
    by (cases "x = y"; cases "𝒢 x"; cases "𝒢 y";
        simp_all add: location_of_def)
qed
```

Ownership restriction and combination

Each dictionary holds one partition, so the ownership operations below move whole dictionaries: a restriction keeps one and empties the other to `bot`, and the combination takes the caller's local dictionary and the callee's global one.

```
definition restrict_local_default_st_rep ::
  "('k, 'a::bot) gdefault_st_rep => ('k, 'a) gdefault_st_rep" where
  "restrict_local_default_st_rep s = (fst s, (bot, []))"

definition restrict_global_default_st_rep ::
  "('k, 'a::bot) gdefault_st_rep => ('k, 'a) gdefault_st_rep" where
  "restrict_global_default_st_rep s = ((bot, []), snd s)"

lemma default_st_rep_get_restrict_local:
  "default_st_rep_get (restrict_local_default_st_rep s) loc =
     (case loc of
        Local_Location x => default_st_rep_get s loc
      | Global_Location x => bot)"
  by (cases s; cases loc) (simp_all add: restrict_local_default_st_rep_def)

lemma default_st_rep_get_restrict_global:
  "default_st_rep_get (restrict_global_default_st_rep s) loc =
     (case loc of
        Local_Location x => bot
      | Global_Location x => default_st_rep_get s loc)"
  by (cases s; cases loc) (simp_all add: restrict_global_default_st_rep_def)

lemma eq_default_st_rep_restrict_local:
  assumes "eq_default_st_rep s t"
  shows "eq_default_st_rep (restrict_local_default_st_rep s)
      (restrict_local_default_st_rep t)"
  by (rule eq_default_st_repI)
     (simp add: default_st_rep_get_restrict_local eq_default_st_repD[OF assms]
       split: glocation.split)

lemma eq_default_st_rep_restrict_global:
  assumes "eq_default_st_rep s t"
  shows "eq_default_st_rep (restrict_global_default_st_rep s)
      (restrict_global_default_st_rep t)"
  by (rule eq_default_st_repI)
     (simp add: default_st_rep_get_restrict_global eq_default_st_repD[OF assms]
       split: glocation.split)

lift_definition restrict_local_default_st ::
  "('k, 'a::bot) gdefault_st => ('k, 'a) gdefault_st"
  is restrict_local_default_st_rep
  by (rule eq_default_st_rep_restrict_local)

lift_definition restrict_global_default_st ::
  "('k, 'a::bot) gdefault_st => ('k, 'a) gdefault_st"
  is restrict_global_default_st_rep
  by (rule eq_default_st_rep_restrict_global)
```

`restrict_local_default_st`/`restrict_global_default_st` preserve the caller's semantic default over the (potentially infinite) location space: the kept side carries over its input's own dictionary verbatim, whose default need not be $\bot$, and only the dropped side is replaced by the empty dictionary over $\bot$. A scope-parametric projection over a bounded materialized support would disagree with this pair outside its bound, so the pair is stated over the full location space and preserves the default by construction.

```
lemma default_st_get_restrict_local [simp]:
  "(restrict_local_default_st s)⟨loc⟩ =
     (case loc of
        Local_Location x => s⟨loc⟩
      | Global_Location x => bot)"
  by transfer (rule default_st_rep_get_restrict_local)

lemma default_st_get_restrict_global [simp]:
  "(restrict_global_default_st s)⟨loc⟩ =
     (case loc of
        Local_Location x => bot
      | Global_Location x => s⟨loc⟩)"
  by transfer (rule default_st_rep_get_restrict_global)

lemma default_st_to_fun_restrict_local [simp]:
  "ϱ_𝒢 (restrict_local_default_st s) x =
     (if 𝒢 x then bot else ϱ_𝒢 s x)"
  unfolding default_st_to_fun_def location_of_def
  by (cases "𝒢 x") simp_all

lemma default_st_to_fun_restrict_global [simp]:
  "ϱ_𝒢 (restrict_global_default_st s) x =
     (if 𝒢 x then ϱ_𝒢 s x else bot)"
  unfolding default_st_to_fun_def location_of_def
  by (cases "𝒢 x") simp_all

definition combine_default_st_rep ::
  "('k, 'a::bot) gdefault_st_rep => ('k, 'a) gdefault_st_rep => ('k, 'a) gdefault_st_rep"
where
  "combine_default_st_rep sc se = (fst sc, snd se)"

lemma default_st_rep_get_combine [simp]:
  "default_st_rep_get (combine_default_st_rep sc se) loc =
   (case loc of
      Local_Location x => default_st_rep_get sc loc
    | Global_Location x => default_st_rep_get se loc)"
  by (cases sc; cases se; cases loc) (simp_all add: combine_default_st_rep_def)

lemma eq_default_st_rep_combine:
  assumes "eq_default_st_rep sc1 sc2"
    and "eq_default_st_rep se1 se2"
  shows "eq_default_st_rep (combine_default_st_rep sc1 se1)
      (combine_default_st_rep sc2 se2)"
  by (rule eq_default_st_repI)
     (simp add: eq_default_st_repD[OF assms(1)] eq_default_st_repD[OF assms(2)]
      split: glocation.split)

lift_definition combine_default_st ::
  "('k, 'a::bot) gdefault_st => ('k, 'a) gdefault_st => ('k, 'a) gdefault_st"
  is combine_default_st_rep
  by (rule eq_default_st_rep_combine)

lemma default_st_get_combine [simp]:
  "(combine_default_st sc se)⟨loc⟩ =
     (case loc of
        Local_Location x => sc⟨loc⟩
      | Global_Location x => se⟨loc⟩)"
  by transfer (rule default_st_rep_get_combine)

lemma default_st_rep_to_fun_combine [simp]:
  "default_st_rep_to_fun 𝒢 (combine_default_st_rep sc se) =
   combine_env 𝒢 (default_st_rep_to_fun 𝒢 sc)
     (default_st_rep_to_fun 𝒢 se)"
proof (rule ext)
  fix x
  show "default_st_rep_to_fun 𝒢 (combine_default_st_rep sc se) x =
      combine_env 𝒢 (default_st_rep_to_fun 𝒢 sc)
        (default_st_rep_to_fun 𝒢 se) x"
    unfolding default_st_rep_to_fun_def combine_env_def location_of_def
    by (cases "𝒢 x") simp_all
qed

lemma default_st_to_fun_combine [simp]:
  "ϱ_𝒢 (combine_default_st sc se) =
   combine_env 𝒢 (ϱ_𝒢 sc)
     (ϱ_𝒢 se)"
```

```
proof (rule ext)
  fix x
  show "ϱ_𝒢 (combine_default_st sc se) x =
      combine_env 𝒢 (ϱ_𝒢 sc)
        (ϱ_𝒢 se) x"
    unfolding default_st_to_fun_def combine_env_def location_of_def
    by (cases "𝒢 x") simp_all
qed

lemma default_st_to_fun_sup [simp]:
  "ϱ_𝒢 (s ⊔ t) =
   ϱ_𝒢 s ⊔ ϱ_𝒢 t"
  by (rule ext) (simp add: default_st_to_fun_def sup_fun_def)
```

Frame entry

Entering a frame sets every local to `top_val` and keeps the globals; the lemmas carry this through the quotient and the readback.

```
definition enter_frame_D_default_st_rep ::
  "'a => ('k, 'a::bot) gdefault_st_rep => ('k, 'a) gdefault_st_rep"
where
  "enter_frame_D_default_st_rep top_val s = ((top_val, []), snd s)"

lemma default_st_rep_get_enter_frame_D [simp]:
  "default_st_rep_get (enter_frame_D_default_st_rep top_val s) loc =
   (case loc of
      Local_Location x => top_val
    | Global_Location x => default_st_rep_get s loc)"
  by (cases s; cases loc) (simp_all add: enter_frame_D_default_st_rep_def)

lemma eq_default_st_rep_enter_frame_D:
  assumes "eq_default_st_rep s t"
  shows "eq_default_st_rep (enter_frame_D_default_st_rep top_val s)
      (enter_frame_D_default_st_rep top_val t)"
  by (rule eq_default_st_repI)
     (simp add: eq_default_st_repD[OF assms] split: glocation.split)

lift_definition enter_frame_D_default_st ::
  "'a => ('k, 'a::bot) gdefault_st => ('k, 'a) gdefault_st"
  is enter_frame_D_default_st_rep
  by (rule eq_default_st_rep_enter_frame_D)

lemma default_st_get_enter_frame_D [simp]:
  "(enter_frame_D_default_st top_val s)⟨loc⟩ =
     (case loc of
        Local_Location x => top_val
      | Global_Location x => s⟨loc⟩)"
  by transfer (rule default_st_rep_get_enter_frame_D)

lemma default_st_rep_to_fun_enter_frame [simp]:
  "default_st_rep_to_fun 𝒢 (enter_frame_D_default_st_rep top_val s) =
   enter_frame 𝒢 top_val (default_st_rep_to_fun 𝒢 s)"
proof (rule ext)
  fix x
  show "default_st_rep_to_fun 𝒢 (enter_frame_D_default_st_rep top_val s) x =
      enter_frame 𝒢 top_val (default_st_rep_to_fun 𝒢 s) x"
    unfolding enter_frame_def default_st_rep_to_fun_def location_of_def
    by (cases "𝒢 x") simp_all
qed

lemma default_st_to_fun_enter_frame [simp]:
  "ϱ_𝒢
      (enter_frame_D_default_st top_val s) =
   enter_frame 𝒢 top_val (ϱ_𝒢 s)"
  unfolding default_st_to_fun_def
  by transfer
     (rule default_st_rep_to_fun_enter_frame[unfolded default_st_rep_to_fun_def])
```

Executable restriction and combination laws

Algebraic identities of the carrier itself: no abstract state, no concretization, no classifier. They say that combining and then projecting recovers the projected side, and that the two projections recombine by join. Executable trees use them to split and rebuild a routed state.

```
lemma combine_default_st_eq_restrict_sup:
```

```
  "combine_default_st A B =
     restrict_local_default_st A ⊔ restrict_global_default_st B"
  by (rule default_st_eqI) (simp split: glocation.split)

lemma restrict_local_default_st_combine [simp]:
  "restrict_local_default_st (combine_default_st A B) =
     restrict_local_default_st A"
  by (rule default_st_eqI) (simp split: glocation.split)

lemma restrict_global_default_st_combine [simp]:
  "restrict_global_default_st (combine_default_st A B) =
     restrict_global_default_st B"
  by (rule default_st_eqI) (simp split: glocation.split)

lemma restrict_local_default_st_split [simp]:
  "restrict_local_default_st (restrict_local_default_st A ⊔
      restrict_global_default_st B) = restrict_local_default_st A"
  by (rule default_st_eqI) (simp split: glocation.split)

lemma restrict_global_default_st_split [simp]:
  "restrict_global_default_st (restrict_local_default_st A ⊔
      restrict_global_default_st B) = restrict_global_default_st B"
  by (rule default_st_eqI) (simp split: glocation.split)
```

Combining reads only the local half of its first argument and the global half of its second, so a state recombines from its own two projections.

```
lemma combine_default_st_restrict_split [simp]:
  "combine_default_st (restrict_local_default_st x) (restrict_global_default_st x) = x"
  by (rule default_st_eqI) (simp split: glocation.split)

lemma combine_default_st_restrict_local_left [simp]:
  "combine_default_st (restrict_local_default_st x) y = combine_default_st x y"
  by (rule default_st_eqI) (simp split: glocation.split)

lemma combine_default_st_self_restrict_global [simp]:
  "combine_default_st x (restrict_global_default_st x) = x"
  by (rule default_st_eqI) (simp split: glocation.split)
```

The stores a carrier state describes

A carrier state denotes the stores its function denotes. This is the one concretization the carrier offers to semantic statements; the represented function itself stays a refinement device, used to relate carrier operations to their counterparts on `('k, 'a) gabs_state`. Only the classifier $\mathcal{G}$ is needed besides the state, because it decides which location holds each name's value. A context that fixes $\mathcal{G}$ may register `default_st_gamma` $\mathcal{G}$ under γ `_` with `adhoc_overloading`.

```
definition default_st_gamma ::
  "('k => bool) => ('k, 'a::numeric_domain) gdefault_st => ('k => int) set" where
  "default_st_gamma 𝒢 s = γ (ϱ𝒢 s)"

lemma default_st_gamma_mono:
  "s ≤ t ⟹ default_st_gamma 𝒢 s ⊆ default_st_gamma 𝒢 t"
  unfolding default_st_gamma_def by (intro gamma_state_mono default_st_to_fun_mono)

lemma default_st_gamma_bot [simp]: "default_st_gamma 𝒢 bot = {}"
  by (simp add: default_st_gamma_def)

unbundle no default_st_syntax

end
```

7.4 Default_St_Reachability

theory Default_St_Reachability
  imports Default_St_Transfer "Voblint_Domain.Nonrelational_Reachability"
begin

unbundle default_st_syntax

A lifted state reads back pointwise under the lift: `Bot` stays `Bot`, and a `Lifted` state becomes `Lifted` of the function it represents. This is how a solved local unknown, stored as an `('k, 'a) gdefault_st lifted`, enters the result boundary. The readback performs no bottom test. `canonicalize_lift` collapses a witness-bottom payload to `Bot` before it, and coverage stays a property of the solve: a covered key whose value is `Bot` remains a key.

adhoc_overloading readback == "λ𝒢. map_lift (default_st_to_fun 𝒢)"

end

7.5 Routed_Exec_Refinement

```
theory Routed_Exec_Refinement
  imports
    Default_St_Reachability
    "Voblint_Framework.DG_Ownership_Split_Spec"
    "Voblint_Framework.DG_Reader_Transport"
begin
```

The buffered generator at any component

The buffered generator a domain actually solves, reconciled with the unbuffered one the framework is stated over, for every component run as a specification. Both reshaping hooks are the identity: the buffered generator only asks a hook to hoist what it publishes at the buffered key `buffer_key`, and a component publishes nothing there, since its edge transfers, its entry and its return all read and write the local unknown only. Its queries do not change that: its channel is a pure function of the local value, so an asking transfer is still a local one.

```
theorem pp_dg_spec_of:
  assumes S: "S = dg_spec_of c"
    and ne: "⋀p ctx. seed p ctx ≠ buffer_key"
    and pp: "part_post_solution
     (routed_node_rhs_buffered intra_predecessor_addr_list call_site_list (λ_. buffer_key) route_st
        (λctx' src a. dg_spec_edge_program S a src (λ_. buffer_key))
        (routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot))
        (routed_entry_seed_programs seed)
        g bot0 d₀ g₀)
     x0 sigma_st vars"
  shows "part_post_solution
     (routed_node_rhs intra_predecessor_addr_list call_site_list (λ_. buffer_key) route_st
        (λctx' src a. dg_spec_edge_program S a src (λ_. buffer_key))
        (routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot))
        (routed_entry_seed_programs seed)
        g bot0 d₀ g₀)
     x0 sigma_st vars"
proof -
  have wf: "dg_spec_wf S" by (simp add: S)
  have intra_free: "sides_of_program (dg_spec_edge_program S a src (λ_. buffer_key)) τ z = bot"
    for a src τ z
    by (simp add: S dg_spec_edge_program_def)
  have cmb_free: "sides_of_program
      (routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot) route' ctx' ca cc ex)
     τ (Inr buffer_key) = bot" for route' ctx' ca cc ex τ
    by (rule routed_call_program_side_free_at_buffer_key[OF wf])
       (auto simp: S local_transfer_def local_combine_transfer_def ne
         dest!: enter_runs_local_pub_bot)
  show ?thesis
  proof (rule part_post_solution_routed_node_rhs_buffered
      [where cmb_c =
        "routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot)"
         and it_c = "λctx' src a. dg_spec_edge_program S a src (λ_. buffer_key)"])
    show "⋀c' w. ∀p ∈ set (routed_contribution_programs intra_predecessor_addr_list
            call_site_list route_st (λctx' src a. dg_spec_edge_program S a src (λ_. buffer_key))
            (routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot))
            (routed_entry_seed_programs seed) g c' w). sp_wf p"
      by (rule routed_contribution_programs_wf)
         (auto intro: sp_wf_dg_spec_edge_program[OF wf] sp_wf_routed_call_program[OF wf])
    then show "⋀c' w. ∀p ∈ set (routed_contribution_programs intra_predecessor_addr_list
            call_site_list route_st (λctx' src a. dg_spec_edge_program S a src (λ_. buffer_key))
            (routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot))
            (routed_entry_seed_programs seed) g c' w). sp_wf p" .
    show "⋀c' src a τ. dg_local (sides_of_program (dg_spec_edge_program S a src (λ_. buffer_key)) τ
            (Inr ((λ_. buffer_key) c'))) = bot"
      by (simp add: intra_free bot_dg_state_def)
    show "⋀c' src a τ. dg_global (traverse_program (dg_spec_edge_program S a src (λ_. buffer_key)) τ)
          = dg_global (sides_of_program (dg_spec_edge_program S a src (λ_. buffer_key)) τ
              (Inr ((λ_. buffer_key) c')))"
      by (simp add: S dg_spec_edge_program_def bot_dg_state_def)
    show "⋀c' src a τ. sides_of_program (dg_spec_edge_program S a src (λ_. buffer_key)) τ
            (Inr ((λ_. buffer_key) c')) = bot"
      by (rule intra_free)
    show "⋀c' ca cc ex τ. dg_local (sides_of_program
            (routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot) route_st c' ca cc ex)
            τ (Inr ((λ_. buffer_key) c'))) = bot"
      by (simp add: cmb_free bot_dg_state_def)
    show "⋀c' ca cc ex τ. dg_global (traverse_program
```

```
          (routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot) route_st c' ca cc ex) τ)
        = dg_global (sides_of_program
            (routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot) route_st c' ca cc ex)
            τ (Inr ((λ_. buffer_key) c'))))"
    by (simp add: routed_call_program_global_free[OF wf] cmb_free bot_dg_state_def)
  show "⋀c' ca cc ex τ. sides_of_program
          (routed_call_program S (λ_. buffer_key) seed (resolve_st g) (λd. d = Bot) route_st c' ca cc ex)
          τ (Inr ((λ_. buffer_key) c')) = bot"
    by (rule cmb_free)
  show "⋀c' w τ z x. x ∈ set (routed_entry_seed_programs seed route_st c' w)
        ⟹ sides_of_program x τ z = bot"
    by (rule routed_entry_seed_programs_free)
  show "⋀c' w τ x. x ∈ set (routed_entry_seed_programs seed route_st c' w)
        ⟹ dg_global (traverse_program x τ) = bot"
    by (rule routed_entry_seed_programs_local_only)
qed (simp_all add: pp)
qed

end
```

Chapter 8

Voblint_Routing

8.1 Compiled_Routed_Equations

```
theory Compiled_Routed_Equations
  imports
    "Voblint_Framework.Routed_Context"
    "Voblint_Compile.Compile_Wellformed"
    "HOL-Library.RBT"
begin
```

Executable routed equations for a compiled program

A routed configuration chooses the key a node buffers its own global part at, the key each global name is read and published at, a seed-key constructor, a routing function, a D/G specification, an initial local state and an initial global state. The remaining equation-system wiring is fixed: predecessor enumeration, edge execution, static call resolution, seed publication, and bottom initialization of every other unknown.

A context-insensitive analysis is this constructor at the unit context: `Analysis_Global ()` as the only global key, `Activation_Seed` as the seed constructor and the constant routing function. There is no second unit-context generator.

Solver policy is deliberately outside this constructor. Always-join, per-origin, and warrowing instances can solve the same equations without hiding that choice here.

The enumerations the equations read are stated over the edge relations and answer every query by re-enumerating a whole relation. The code equation at the end answers them from indexes built once per graph instead, so a solver step costs a lookup rather than a scan and sort of every edge.

```
definition compiled_routed_eqs_for ::
    "'K
      ⇒ ('N ⇒ 'K)
      ⇒ (pp ⇒ 'C ⇒ 'K)
      ⇒ (pp ⇒ 'C ⇒ 'D ⇒ (var_id, exp) gcall_action ⇒ 'C)
      ⇒ (pp × 'C, 'K, 'N, 'D::bounded_semilattice_sup_bot, 'G::bounded_semilattice_sup_bot) dg_spec
      ⇒ (var_id, exp) gcfg
      ⇒ 'D
      ⇒ 'G
      ⇒ (pp × 'C, 'K, ('D, 'G) dg_state) eqsT"
where
  "compiled_routed_eqs_for global global_of seed route S g initial initial_global =
     routed_node_rhs_buffered intra_predecessor_addr_list call_site_list
       (λ_. global) route
       (λctx' src a. dg_spec_edge_program S a src global_of)
       (routed_call_program S global_of seed (static_resolve g) (λd. d = bot))
       (routed_entry_seed_programs seed)
       g bot initial initial_global"
```

Per-node edge indexes

`group_by_key key_of f xs` files each element `f` accepts under its key, keeping list order within a key. A lookup is therefore the order-preserving filter of `xs` at that key, which is what lets it replace an enumeration inside a fold that observes order.

```
definition group_lookup :: "('k::linorder, 'y list) rbt ⇒ 'k ⇒ 'y list" where
  "group_lookup m k = (case RBT.lookup m k of None ⇒ [] | Some ys ⇒ ys)"

definition group_by_key ::
    "('x ⇒ 'k::linorder) ⇒ ('x ⇒ 'y option) ⇒ 'x list ⇒ ('k, 'y list) rbt" where
  "group_by_key key_of f xs =
     foldr (λx m. case f x of
                None ⇒ m
              | Some y ⇒ RBT.insert (key_of x) (y # group_lookup m (key_of x)) m)
       xs RBT.empty"

lemma group_lookup_group_by_key:
  "group_lookup (group_by_key key_of f xs) k
     = List.map_filter (λx. if key_of x = k then f x else None) xs"
  by (induction xs) (auto simp: group_by_key_def group_lookup_def split: option.splits)

definition intra_predecessor_index ::
    "(var_id, exp) gcfg ⇒ (cfg_node, (cfg_node × (var_id, exp) gedge_action) list) rbt"
     where
  "intra_predecessor_index g =
     group_by_key (λ(u, a, w). w) (λ(u, a, w). Some (u, a)) (cfg_intra_list g)"

lemma map_filter_intra_target:
```

```
  "List.map_filter
     (λx. if (λ(u, a, w). w) x = v then (λ(u, a, w). Some (u, a)) x else None) es
   = map (λ(u, a, w). (u, a)) (filter (λ(u, a, w). w = v) es)"
  by (induction es) auto

lemma group_lookup_intra_predecessor_index:
  "group_lookup (intra_predecessor_index g) v = intra_predecessor_list g v"
  unfolding intra_predecessor_index_def intra_predecessor_list_def group_lookup_group_by_key
  by (simp add: map_filter_intra_target)

lemma map_group_lookup_intra_predecessor_index:
  "map (λ(u, a). (Inl (u, ctx), a)) (group_lookup (intra_predecessor_index g) v)
     = intra_predecessor_addr_list g v ctx"
  by (simp add: intra_predecessor_addr_list_def group_lookup_intra_predecessor_index)

definition call_target_index ::
    "(var_id, exp) gcfg ⇒ (cfg_node, (cfg_node × (var_id, exp) gcall_action × pname) list) rbt"
      where
  "call_target_index g =
     group_by_key (λ(c, ca, ce, k). k) (λ(c, ca, ce, k). call_target_at k (c, ca, ce, k))
       (cfg_calls_list g)"

lemma call_target_at_key:
  "(if (λ(c, ca, ce, k). k) e = v
     then (λ(c, ca, ce, k). call_target_at k (c, ca, ce, k)) e else None)
   = call_target_at v e"
proof -
  obtain c ca ce k where e: "e = (c, ca, ce, k)" by (cases e) auto
  show ?thesis by (cases ce) (auto simp: e)
qed

lemma group_lookup_call_target_index:
  "group_lookup (call_target_index g) v = call_target_list g v"
  unfolding call_target_index_def call_target_list_def group_lookup_group_by_key
  by (simp add: call_target_at_key)
```

The indexes are let-bound outside the equation system, so a solver holding the system evaluates them once. The three selectors restate `intra_predecessor_addr_list`, `call_site_list` and `static_resolve` over a lookup in place of the enumeration. They ignore their graph argument, which is sound only because the generator applies every selector at the one graph it is given.

```
lemma compiled_routed_eqs_for_code [code]:
  "compiled_routed_eqs_for global global_of seed route S g initial initial_global =
     (let preds = intra_predecessor_index g; targets = call_target_index g
      in routed_node_rhs_buffered
           (λ_ v ctx. map (λ(u, a). (Inl (u, ctx), a)) (group_lookup preds v))
           (λ_ v. remdups (map (λ(c, ca, p). (c, ca)) (group_lookup targets v)))
           (λ_. global) route
           (λctx' src a. dg_spec_edge_program S a src global_of)
           (routed_call_program S global_of seed
              (λv cc ca d. map (λ(c, a, p). p)
                 (filter (λ(c, a, p). c = cc ∧ a = ca) (group_lookup targets v)))
              (λd. d = bot))
           (routed_entry_seed_programs seed)
           g bot initial initial_global)"
proof -
  note preds = map_group_lookup_intra_predecessor_index
  have sites: "remdups (map (λ(c, ca, p). (c, ca)) (group_lookup (call_target_index g) v))
      = call_site_list g v" for v
    by (simp add: call_site_list_def group_lookup_call_target_index)
  have resolve: "(λv cc ca d. map (λ(c, a, p). p)
                   (filter (λ(c, a, p). c = cc ∧ a = ca) (group_lookup (call_target_index g) v)))
      = static_resolve g"
    by (intro ext) (simp add: static_resolve_def static_targets_def group_lookup_call_target_index)
  show ?thesis
    unfolding compiled_routed_eqs_for_def routed_node_rhs_buffered_def
      routed_contribution_programs_def Let_def
    by (simp only: preds sites resolve)
qed

end
```

8.2 Context_Space_Finite

```
theory Context_Space_Finite
  imports "Voblint_Framework.Call_String_Context" "Voblint_Compile.Compile_Wellformed"
begin
```

Which context policies can have a finite solved key set

A published result table is well formed only if its key set is finite (`wf_solved_table`, in the framework's result layer). A key is a `(node, context)` pair, so one way to get there is to exhibit a fixed finite set the keys are drawn from and appeal to `finite_subset`. This theory builds that set, one per context policy.

What it builds is a candidate space, not a reachability or a termination result. Nothing below says which keys a solver run actually produces, and nothing below bounds how often the value at one key is raised – an interval unknown can ascend forever inside a one-element key space. Both halves of the containment are therefore hypotheses of every result here, not conclusions:

- `fst ‘ vars ⊆ cfg_nodes (compile_prog Pi ps)` — the keys are nodes of the program the solver was given. There is no theorem here connecting the vendored solver's returned key set to the unknowns its equation system mentions.
- `snd ‘ vars` lies in the policy's context space — routing never left it. For call strings that is a length-and-alphabet condition, and truncation alone does not establish it: a starting context whose elements are not nodes of this program stays short under `cs_route` without ever entering the space.

What differs between policies is how cheaply the second hypothesis can be met. Call strings bound the length for free: `cs_route` truncates every context to length at most `k` (`cs_route_length`, `Voblint_Framework.Call_String_Context`), and a compiled program's CFG has finitely many nodes (`cfg_nodes_finite`, `Voblint_CFG.CFG_Def`), so with the standard library's own `finite_lists_length_le` the space is finite for every `k` and every domain alike. The monovariant policy makes the hypothesis vacuous: its context type is `unit`, so there is no context space to bound and the node half is the whole story.

An entry-state context is a domain value rather than a bounded-length list over a finite alphabet, so it needs a separate argument. A finite value domain together with a fixed formals count supplies one – `sign list` at the callee's arity is a finite set – but an infinite-height domain such as `ivl` does not, and an ordinary widening policy bounds values rather than the context identities keyed by them. Bounding those needs a policy that controls key creation or merges keys (a gas budget, a tabulation cap), with real precision consequences that nothing here specifies. This development stops at the two policies whose context space needs no such decision.

The concrete analyses reach a finite key set by an unrelated route: they discharge their own obligation from the vendored solver's `finite_stabl_solve`, which yields a finite stable set from a terminating solve. Neither argument implies the other – a bounded candidate space does not make a solve terminate, and a terminating solve says nothing about the space its keys were drawn from.

```
lemma call_strings_bounded_finite:
  assumes "finite A"
  shows "finite {cs::call_string. set cs ⊆ A ∧ length cs ≤ k}"
  using assms by (rule finite_lists_length_le)

theorem compiled_call_strings_finite:
  "finite {cs::call_string. set cs ⊆ cfg_nodes (compile_prog Pi ps) ∧ length cs ≤ k}"
  using cfg_nodes_finite[OF compile_prog_finite[THEN conjunct1] compile_prog_finite[THEN conjunct2]]
  by (rule call_strings_bounded_finite)
```

The practical corollary an instance routed by call strings can cite directly: every `(node, call-string)` pair a `k`-bounded call-string analysis over a compiled program could ever solve for lies in a fixed finite set, so `vars` — whatever a particular solver run actually populates it with — is finite because it lies in a finite superset, not merely because the run happened to terminate.

```
theorem compiled_call_string_vars_finite:
  assumes nodes: "fst ‘ vars ⊆ cfg_nodes (compile_prog Pi ps)"
    and ctxs: "snd ‘ vars ⊆ {cs::call_string. set cs ⊆ cfg_nodes (compile_prog Pi ps)
                                ∧ length cs ≤ k}"
  shows "finite vars"
proof -
  have "vars ⊆ cfg_nodes (compile_prog Pi ps)
          × {cs::call_string. set cs ⊆ cfg_nodes (compile_prog Pi ps) ∧ length cs ≤ k}"
    using nodes ctxs by(auto; force)
  moreover have "finite (cfg_nodes (compile_prog Pi ps)
          × {cs::call_string. set cs ⊆ cfg_nodes (compile_prog Pi ps) ∧ length cs ≤ k})"
    using cfg_nodes_finite compile_prog_finite compiled_call_strings_finite
```

```
    by (blast intro: finite_cartesian_product)
  ultimately show ?thesis by (rule finite_subset)
qed
```

The seed-key space follows the same shape: one `Global` slot plus one `Seed` slot per (callee-entry node, call-string) pair. `seed_pp` ranges over `pp`, which is `cfg_node` (`Voblint_CFG.CFG_Def`) – a call-string seed is keyed by the callee's `FunctionEntry` node, not the raw procedure name – so this reuses `cfg_nodes_finite` again rather than needing a separate finiteness fact about `ps`.

```
theorem compiled_call_string_gk_finite:
  "finite ({Global} ∪ (⋃p ∈ cfg_nodes (compile_prog Pi ps). Seed p ‘
        {cs::call_string. set cs ⊆ cfg_nodes (compile_prog Pi ps) ∧ length cs ≤ k}))"
  using compiled_call_strings_finite[of Pi ps k]
    cfg_nodes_finite[OF compile_prog_finite[of Pi ps, THEN conjunct1]
      compile_prog_finite[of Pi ps, THEN conjunct2]]
  by auto
```

The monovariant policy

The same result for the monovariant policy, where it needs less: a context is `unit`, so the context factor is a singleton and the node bound is the entire hypothesis. Stated over `fst ‘ vars` rather than a product, since with one context there is nothing for a product to say.

```
theorem compiled_unit_vars_finite:
  fixes vars :: "(pp × unit) set"
  assumes "fst ‘ vars ⊆ cfg_nodes (compile_prog Pi ps)"
  shows "finite vars"
proof -
  have "vars ⊆ cfg_nodes (compile_prog Pi ps) × (UNIV :: unit set)"
    using assms by auto
  moreover have "finite (cfg_nodes (compile_prog Pi ps) × (UNIV :: unit set))"
    using cfg_nodes_finite compile_prog_finite by force
  ultimately show ?thesis by (rule finite_subset)
qed

end
```

Chapter 9

Voblint_Result

9.1 Source_Activation_Sound

```
theory Source_Activation_Sound
  imports "Voblint_Framework.Activation_Backbone" "Voblint_Compile.Source_To_Trace"
begin
```

End-to-end source-level activation soundness

A completed run of a compiled source program is collected at the result of its root activation. That a reachable store is collected at the node the simulation locates it at is `source_reaches_node_collect` in `Voblint_Compile.Source_To_Trace`.

Completed runs reach a procedure result

A source run that has completed at the top level (residual `SKIP`, empty frame stack) is collected at the `FunctionResult` of the activation it completed. Two structural facts do the work, both from the CFG layer: the located node reaches its fragment exit along store-preserving intra flow (`compile_control_at_SKIP_exit_path`), and `compiled_at` supplies the fall-through `EA_Ret None p` edge from that exit into `FunctionResult p`.

The witness is kept: the completing activation is the *root* one (empty frame stack, so `caller_of` is `None`), which is what pins its procedure — see `source_completes_node_collect_exit`. Projecting into `node_collect` first would lose that: the collection at a `FunctionResult` deliberately merges every activation of that procedure.

```
theorem source_completes_valid_activation_trace_result:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Pi ps"
    and s0: "s0 ∈ S"
    and run: "κ,Pi ⊢ (main_body Pi, s0, []) →_p* (SKIP (), s, [])"
  shows "∃t p. t ∈ 𝒯_κ,compile_prog Pi ps,S
              ∧ caller_of t = None
              ∧ fst (hd (path_of t)) = cfg_entry (compile_prog Pi ps)
              ∧ sink_node t = FunctionResult p
              ∧ sink_store t = s"
proof -
  let ?g = "compile_prog Pi ps"
  from compile_prog_run_has_activation_trace[OF wf s0 run]
  obtain v stk t where sim: "Pi, ?g, prog_main_name ⊢ (SKIP (), s, []) ≈ (v, s, stk)"
    and rep: "activation_trace_repr κ ?g S (v, s, stk) t" by blast
  have stk0: "stk = []" using sim by blast
  from rep stk0 have tv: "t ∈ 𝒯_κ,?g,S" and sn: "sink_node t = v"
    and ss: "sink_store t = s" and sr: "stack_repr ?g [] t"
    by (auto simp: activation_trace_repr_def)
  have cof: "caller_of t = None" using stack_repr_Nil_iff[OF sr] by simp
  have hd_t: "fst (hd (path_of t)) = cfg_entry ?g"
    by (rule valid_activation_trace_caller_None_entry[OF tv cof])
  — the located node of a completed activation is its procedure's result
  from sim obtain p body_p k n where
    ca: "control_at Pi p body_p k n (SKIP ()) v" and cat: "compiled_at Pi ?g p body_p k n"
    by blast
  from cat obtain n' en E K where
    cc: "compile Pi p body_p k n = (n', en, E, K)" and Esub: "E ⊆ intra ?g"
    and ret: "(k, EA_Ret None p, FunctionResult p) ∈ intra ?g"
    by blast
  — the whole extension is intra flow, so it stays inside this same (root) activation
  have path_to_ret: "star (cfg_intra_step κ ?g) (sink_node t, sink_store t) (FunctionResult p, s)"
  proof -
    have a: "star (cfg_intra_step κ ?g) (sink_node t, sink_store t) (k, s)"
      using compile_control_at_SKIP_exit_path[OF ca cc Esub] sn ss by simp
    have b: "star (cfg_intra_step κ ?g) (k, s) (FunctionResult p, s)"
      by (rule intra_path_single[OF ret]) simp
    from a b show ?thesis by (rule star_trans)

  qed
  from valid_activation_trace_intra_path_extend[OF path_to_ret tv]
  obtain t' where tvar_id: "t' ∈ 𝒯_κ,?g,S" and t'n:
    "sink_node t' = FunctionResult p"
    and t's: "sink_store t' = s" and t'c: "caller_of t' = caller_of t"
    and t'h: "fst (hd (path_of t')) = fst (hd (path_of t))" by blast
  show ?thesis
  proof (intro exI conjI)
    show "t' ∈ 𝒯_κ,?g,S" by (rule tvar_id)
    show "caller_of t' = None" using t'c cof by simp
    show "fst (hd (path_of t')) = cfg_entry ?g" using t'h hd_t by simp
    show "sink_node t' = FunctionResult p" by (rule t'n)
    show "sink_store t' = s" by (rule t's)
  qed
```

qed

Whole-program completion. The completing activation is the root one, so it entered at `FunctionEntry prog_main_name`; activation procedure locality (`valid_activation_trace_entry_result_eq`) then forces its result node to be `FunctionResult prog_main_name = cfg_exit`. No assumption about the *number* of declared procedures is needed: a program may call as many as it likes.

```
corollary source_completes_node_collect_exit:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Pi ps"
    and s0: "s0 ∈ S"
    and run: "κ,Pi ⊢ (main_body Pi, s0, []) →_p* (SKIP (), s, [])"
  shows "s ∈ C_κ,compile_prog Pi ps,S
             (cfg_exit (compile_prog Pi ps))"
proof -
  let ?g = "compile_prog Pi ps"
  from source_completes_valid_activation_trace_result[OF wf s0 run]
  obtain t p where tv: "t ∈ T_κ,?g,S" and hd_t:
    "fst (hd (path_of t)) = cfg_entry ?g"
    and sn: "sink_node t = FunctionResult p" and ss: "sink_store t = s" by blast
  have "fst (hd (path_of t)) = FunctionEntry prog_main_name"
    using hd_t by (simp add: compile_prog_def Let_def split: prod.splits)
  from valid_activation_trace_entry_result_eq[OF wf tv this sn] have "prog_main_name = p" .
  with sn ss tv show ?thesis
    using node_collect_I by fastforce
qed

end
```

9.2 DG_Result_Construction

```
theory DG_Result_Construction
  imports
    "Voblint_Framework.DG_Analysis_Adapter"
    "Voblint_Exec.Default_St_Reachability"
begin
```

What a solved D/G system publishes

Every analysis, at every context policy and every solver discipline, turns the solver's answer – a covered key set and a map from unknowns to `dg_state`s over its carrier – into an `solved_table` table of the locals. This theory states that construction once, over an arbitrary solved pair, so an analysis's result table is one application rather than a rewritten body.

Publishing a local unknown means two steps in sequence. `canonicalize_lift` collapses a stored `Lifted` payload that describes no store to `Bot`, so a dead point reads as dead; the publication map `rd` then turns the carrier into the value the table publishes. Coverage is separate from deadness: a key the solver never visited is absent from the table, and `lookup_table` answers `Bot` for it without any claim about the program.

`sol` is the already-solved pair, not the solve function. Passing the pair keeps one solve per table in generated code – the argument is evaluated once and the per-point closure captures it – which is why no analysis needs a separate `[code]` equation with an explicit `let`.

What a point publishes is read off its local unknown together with the solved global environment, each name read at its key `Inr (gk n)`: `rc` recombines the local half with that environment into the state the point describes. An analysis that keeps every variable in the local half passes a recombination that ignores the global.

```
definition dg_result_for ::
    "('s ⇒ 'v) ⇒ ('s ⇒ bool) ⇒ ('s lifted ⇒ ('n ⇒ 'g) ⇒ 's lifted) ⇒ ('n ⇒ 'k)
     ⇒ (pp × 'c) set × (pp × 'c + 'k ⇒ ('s lifted, 'g) dg_state)
     ⇒ ('c, 'v) solved_table" where
  "dg_result_for rd emp rc gk sol =
     Solved_Table (fst sol)
       (λv ctx. map_lift rd (canonicalize_lift emp
          (rc (dg_local (snd sol (Inl (v, ctx)))) (genv gk (snd sol)))))"

lemma covered_keys_dg_result_for [simp]:
  "covered_keys (dg_result_for rd emp rc gk sol) = fst sol"
  unfolding dg_result_for_def by simp
```

The one fact every soundness bridge needs about the table: a covered key reads back the normalized recombined state, an uncovered one answers `Bot`.

```
lemma lookup_table_dg_result_for [simp]:
  "lookup_table (dg_result_for rd emp rc gk sol) v ctx
     = (if (v, ctx) ∈ fst sol
        then map_lift rd (canonicalize_lift emp
               (rc (dg_local (snd sol (Inl (v, ctx)))) (genv gk (snd sol))))
        else Bot)"
  unfolding dg_result_for_def lookup_table_def by simp
```

Normalizing before publication agrees with normalizing after it whenever the two emptiness tests agree, so a soundness bridge stated after publication applies to the table built before it.

```
lemma map_lift_canonicalize_lift:
  assumes "⋀s. emp s = empty_V (rd s)"
  shows "map_lift rd (canonicalize_lift emp d) = canonicalize_lift empty_V (map_lift rd d)"
  by (cases d) (simp_all add: assms normalize_lift_def)
```

Collapsing a payload that denotes nothing to `Bot` does not change what the value denotes.

```
lemma gamma_lift_canonicalize_lift:
  assumes "⋀v. empty_V v ⟹ gm v = {}"
  shows "gamma_lift gm (canonicalize_lift empty_V x) = gamma_lift gm x"
  by (cases x) (simp_all add: normalize_lift_def assms)

end
```

9.3 Analysis_Surface

```
theory Analysis_Surface
  imports
    "Voblint_Framework.Solved_Table"
    "Voblint_Framework.Check_Report"
    "Voblint_Compile.Compile_Invariants"
begin
```

The surface a solved table publishes

What every domain, at every solver discipline and context policy, ends up publishing about a whole program: the state its solved table (an `solved_table`) holds at a point under a context, and the check report read off those states. `analysis_surface` states both once, generically in the domain `'a` and the context type `'c`, so a domain's public API is one interpretation rather than a rewritten copy per domain, solver discipline and policy. A context-insensitive run reads it at the one context `()`.

An interpretation publishes the complete surface for a solved table. Solver disciplines therefore share one construction and cannot accidentally expose only a subset of the result and report operations.

$\bot$ at an `Bot` point is the reading every existing report function already makes: nothing reaches the point, so no store does, and $\bot$ is the abstract state whose concretisation is empty.

`bot_state` is a parameter rather than the `order_bot` class operation, and that is a code-generation requirement, not a stylistic choice. A sort constraint here would make the surface's code equation polymorphic in `'a::order_bot`, and generating code for it at an abstract state — a function type `String.literal ⇒ 'b` — would demand an executable $\bot$ arity for that function type, which needs the domain to be `enum`. `String.literal` is not. Taking the element as a parameter keeps the equation free of sorts; every interpretation still passes $\bot$, where the concrete instance is executable exactly as it was before.

```
locale analysis_surface =
  fixes table :: "'p ⇒ ('c, 'a) solved_table"
    and bot_state :: 'a
    and classify :: "exp ⇒ 'a ⇒ check_result"
    and pcfg :: "'p ⇒ (var_id, exp) gcfg"
begin

definition state_at :: "'p ⇒ 'c ⇒ pp ⇒ 'a" where
  "state_at p ctx v =
     (case lookup_table (table p) v ctx of Bot ⇒ bot_state | Lifted st ⇒ st)"

definition report :: "'p ⇒ 'c ⇒ (pp × exp × check_result) list" where
  "report p ctx = classify_checks (pcfg p) (state_at p ctx) classify"
```

The same table read with its `Bot` entries kept visible: `True` marks a point whose entry in `table` is `Bot`, which a report consumer suppresses instead of printing the bottom state's vacuous verdict. Exact for the same reason `state_at` is – it is the `lookup_table` case split itself, not a second test on the raw unknown (`fst_reach_state_at` states exactly that).

The flag is a fact about the table and nothing more. This locale assumes nothing whatever about `table`, so it cannot and does not say that such a point is one no solver covered, or one no concrete run reaches. A caller wanting either reading owes it from its own soundness theorem about the table it supplies.

```
definition reach_state_at :: "'p ⇒ 'c ⇒ pp ⇒ bool × 'a" where
  "reach_state_at p ctx v =
     (case lookup_table (table p) v ctx of
        Bot ⇒ (True, bot_state) | Lifted st ⇒ (False, st))"
```

The two readings agree on the state: `reach_state_at` adds a column and changes no verdict, so a report that shows reachability classifies exactly what `report` does. Without this the two definitions are only visibly similar, and every caller reproves it by unfolding both.

```
lemma state_at_Bot [simp]:
  "lookup_table (table p) v ctx = Bot ⟹ state_at p ctx v = bot_state"
  by (simp add: state_at_def)

lemma state_at_Lifted [simp]:
  "lookup_table (table p) v ctx = Lifted st ⟹ state_at p ctx v = st"
  by (simp add: state_at_def)

lemma reach_state_at_Bot [simp]:
  "lookup_table (table p) v ctx = Bot ⟹ reach_state_at p ctx v = (True, bot_state)"
  by (simp add: reach_state_at_def)
```

```
lemma reach_state_at_Lifted [simp]:
  "lookup_table (table p) v ctx = Lifted st ⟹ reach_state_at p ctx v = (False, st)"
  by (simp add: reach_state_at_def)

lemma snd_reach_state_at [simp]:
  "snd (reach_state_at p ctx v) = state_at p ctx v"
  by (simp add: reach_state_at_def state_at_def split: lifted.splits)

lemma fst_reach_state_at [simp]:
  "fst (reach_state_at p ctx v) = (lookup_table (table p) v ctx = Bot)"
  by (simp add: reach_state_at_def split: lifted.splits)

definition report_with_state ::
  "'p ⇒ 'c ⇒ (pp × exp × check_result × bool × 'a) list" where
  "report_with_state p ctx =
     classify_checks_with_state (pcfg p) (reach_state_at p ctx)
       (λc (_, s). classify c s)"

end
```

Register the generic surface equations outside the locale so code generation can call them directly with the table, bottom state and classifier as parameters.

```
declare
  analysis_surface.state_at_def [code]
  analysis_surface.report_def [code]
  analysis_surface.reach_state_at_def [code]
  analysis_surface.report_with_state_def [code]

end
```

9.4 DG_Analysis

```
theory DG_Analysis
  imports
    DG_Result_Construction
    Analysis_Surface
    "Voblint_Framework.Routed_Analysis_Sound"
    "Voblint_Exec.Routed_Exec_Refinement"
    Source_Activation_Sound
    "Voblint_Routing.Compiled_Routed_Equations"
    "Voblint_Solver.TD_Solver_Bridge"
    "Voblint_Framework.DG_Keyed_Split_Spec"
begin
```

One context-sensitive analysis, assembled from its choices

A concrete analysis picks a domain implementation, an initial state, a way of telling activations apart, a solver and a classifier; everything between those choices and the published table is the same work at every domain and at every context policy. This theory does that work once.

The construction half is `dg_pipeline`, which carries no correctness assumptions, so its defining equations are unconditional and can be declared to the code generator. The correctness half is `dg_analysis`, which adds the domain and solver contracts and derives, for one program, the routed soundness statement every policy shares.

Vocabulary. A *routed* equation system publishes a callee's entry state at a global seed key instead of reading the callee entry directly; a *context* is the value the routing function attaches to a program point so that one point can be solved at several unknowns; *covered* keys are the unknowns the solver actually visited, which is not the same as the unknowns holding a bottom state.

What this covers, and what it does not. It is the assembly for executable non-relational analyses whose local unknown carries a whole reachability-lifted store and whose entry operation answers with a pure list of alternatives. The context type, the global-key type, the seed constructor, the routing function and the root context are all parameters; the carrier and the entry shape are not.

The construction

The pipeline runs one component, `comp` $\mathcal{G}$ `p`: a single analysis, or the combination of several. `init_st` is the unlifted entry state; the pipeline lifts it, because a reachability-lifted carrier is what the routed generator solves over. `empty p` is the carrier's executable emptiness test and `rd` $\mathcal{G}$ reads a carrier state back as the value the result table publishes. Nothing here fixes what the carrier or the published value is. `bot_state` is a parameter rather than the `order_bot` operation for the same code-generation reason `analysis_surface` states: a sort constraint here would demand an executable $\bot$ at a function type.

Everything one solve publishes, read back as values: the result table, the solved value of each global name, the seed a call published at a procedure entry under a context, what one edge's local step makes of a point's solved state, and where a call leads when it contributes at all.

```
record ('c, 'n, 'v) solved_run =
  run_table :: "('c, 'v) solved_table"
  run_shared :: "'n ⇒ 'v lifted"
  run_seed :: "pname ⇒ 'c ⇒ 'v lifted"
  run_step :: "pp ⇒ 'c ⇒ (var_id, exp) gedge_action ⇒ 'v lifted"
  run_succ :: "cfg_node ⇒ 'c ⇒ (var_id, exp) gcall_action ⇒ pname ⇒ 'c option"
```

`dg_pipeline` only fixes parameters: the component `comp`, the carrier operations `emp`, `rd` and `init_st`, the key a node buffers its own global part at and the key `global_of n` each global name lives at, the context policy `seed`/`route`/ `root_ctx`, the solver `solve` with its domain, the check classifier, and the placement of program state between the two halves of a solver unknown. The soundness assumptions live in the locales built on it.

The placement is six operations. `place_spec` builds the D/G specification the equations run, from the component. `place_cmb` recombines a local value with the solved global environment into the state a point describes, and `place_rl` and `place_rg` project a state onto the part each half keeps. `place_inits` lists the initial value of each global name, which the program entry publishes at that name's key `global_of n`. `place_enter` is the caller state a call enters from: it may describe more than `place_cmb`, so a call reads only the globals it needs. An analysis that keeps all of its state in the local half builds the specification with `dg_spec_of`, recombines by ignoring the environment, keeps every state locally, publishes `Bot`, lists no initial values and enters from the caller's state as it is.

```
locale dg_pipeline =
  fixes comp ::
```

```
    "(var_id ⇒ bool) ⇒ 'p ⇒ 's::semilattice_sup lifted local_spec"
    and emp :: "'p ⇒ 's ⇒ bool"
    and rd :: "(var_id ⇒ bool) ⇒ 's ⇒ 'v"
    and init_st :: 's
    and buffer_key :: 'k
    and global_of :: "'n ⇒ 'k"
    and seed :: "pp ⇒ 'c ⇒ 'k"
    and route :: "(var_id ⇒ bool) ⇒ pp ⇒ 'c ⇒ 's lifted ⇒ (var_id, exp) gcall_action ⇒ 'c"
    and root_ctx :: 'c
    and solve :: "(pp × 'c, 'k, ('s lifted, 's lifted) dg_state) eqsT
                  ⇒ pp × 'c
                  ⇒ (pp × 'c) set × (pp × 'c + 'k ⇒ ('s lifted, 's lifted) dg_state)"
    and solve_dom :: "(pp × 'c, 'k, ('s lifted, 's lifted) dg_state) eqsT ⇒ pp × 'c ⇒ bool"
    and bot_state :: 'v
    and classify :: "exp ⇒ 'v ⇒ check_result"
    and place_spec :: "'p ⇒ 's lifted local_spec
                       ⇒ (pp × 'c, 'k, 'n, 's lifted, 's lifted) dg_spec"
    and place_cmb :: "'p ⇒ 's lifted ⇒ ('n ⇒ 's lifted) ⇒ 's lifted"
    and place_rl :: "(var_id ⇒ bool) ⇒ 's lifted ⇒ 's lifted"
    and place_rg :: "(var_id ⇒ bool) ⇒ 's lifted ⇒ 's lifted"
    and place_inits :: "(var_id ⇒ bool) ⇒ 'p ⇒ ('n × 's lifted) list"
    and place_enter :: "'p ⇒ call_info ⇒ 's lifted ⇒ ('n ⇒ 's lifted) ⇒ 's lifted"
    and pcfg :: "'p ⇒ (var_id, exp) gcfg"
    and pentry :: "'p ⇒ cfg_node"
begin

definition analysis_spec :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ (pp × 'c, 'k, 'n, 's lifted, 's lifted) dg_spec" where
  "analysis_spec 𝒢 p = place_spec p (comp 𝒢 p)"
```

The state the component enters a callee with, from a whole caller state. The pipeline supports components whose entry answers one alternative, the caller's state paired with this one.

```
definition comp_entry :: "(var_id ⇒ bool) ⇒ 'p ⇒ call_info ⇒ 's lifted
    ⇒ 's lifted" where
  "comp_entry 𝒢 p ci w = snd (hd (ls_enter (comp 𝒢 p) (ls_channel (comp 𝒢 p) w) ci (w, w)))"
```

The alternative a call enters its callee with, as the solver stores it: the component enters from the caller state `place_enter` recombines, and each half of the answer keeps its local part.

```
definition entry_alt :: "(var_id ⇒ bool) ⇒ 'p ⇒ call_info ⇒ 's lifted
    ⇒ ('n ⇒ 's lifted) ⇒ 's lifted × 's lifted" where
  "entry_alt 𝒢 p ci d g =
     (let w = place_enter p ci d g in (place_rl 𝒢 w, place_rl 𝒢 (comp_entry 𝒢 p ci w)))"
```

The state a call enters its callee with.

```
definition entry_of :: "(var_id ⇒ bool) ⇒ 'p ⇒ call_info ⇒ 's lifted
    ⇒ ('n ⇒ 's lifted) ⇒ 's lifted"
  where "entry_of 𝒢 p ci d g = snd (entry_alt 𝒢 p ci d g)"
```

The unknown the solver is asked for. It is the program exit at the root context: a demand-driven solver is started at the answer a caller wants and explores backwards from it, so the root query is the exit and the entry point is reached as one of its dependencies.

```
definition root_query :: "'p ⇒ pp × 'c" where
  "root_query p = (cfg_exit (pcfg p), root_ctx)"
```

`root_query`'s own type mentions no domain, so a code equation for it carries a sort hypothesis the code generator cannot see and silently rejects. The two constants that use it inline it instead: their types do mention the domain, so their equations are accepted, and `root_query` never has to be executable.

The initial value of each global name is a side effect like any other: the program entry publishes it, once, at the name's key, as a run's initialization happens before its first statement. A placement that keeps its initial global part at the node's own key passes no such values.

```
definition init_publications :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ pp × 'c ⇒ ('k × ('s lifted, 's lifted) dg_state) list" where
  "init_publications 𝒢 p x =
     (if x = (pentry p, root_ctx)
      then map (λ(n, d). (global_of n, DG Bot d)) (place_inits 𝒢 p) else [])"
```

The initial global environment a run starts from: what the entry node buffers at its own key, read at every name living there, together with the root's initial publications.

```
definition init_env :: "(var_id ⇒ bool) ⇒ 'p ⇒ 'n ⇒ 's lifted" where
```

```
  "init_env 𝒢 p n =
     (if global_of n = buffer_key then place_rg 𝒢 (Lifted init_st) else Bot)
       ⊔ (⨆(m, d)←filter (λ(m, d). m = n) (place_inits 𝒢 p). d)"

definition equations :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ (pp × 'c, 'k,
        ('s lifted, 's lifted) dg_state) eqsT" where
  "equations 𝒢 p =
     with_init (init_publications 𝒢 p)
       (compiled_routed_eqs_for buffer_key global_of seed (route 𝒢)
          (analysis_spec 𝒢 p) (pcfg p) (Lifted init_st) (place_rg 𝒢 (Lifted init_st)))"

definition solution :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ (pp × 'c) set
        × (pp × 'c + 'k
            ⇒ ('s lifted, 's lifted) dg_state)" where
  "solution 𝒢 p = solve (equations 𝒢 p) (root_query p)"

definition terminates :: "(var_id ⇒ bool) ⇒ 'p ⇒ bool" where
  "terminates 𝒢 p = solve_dom (equations 𝒢 p) (root_query p)"

lemma solution_code: "solution 𝒢 p = solve (equations 𝒢 p) (cfg_exit (pcfg p), root_ctx)"
  by (simp add: solution_def root_query_def)

lemma terminates_code:
  "terminates 𝒢 p = solve_dom (equations 𝒢 p) (cfg_exit (pcfg p), root_ctx)"
  by (simp add: terminates_def root_query_def)

definition sol_vars :: "(var_id ⇒ bool) ⇒ 'p ⇒ (pp × 'c) set" where
  "sol_vars 𝒢 p = fst (solution 𝒢 p)"

definition sol_env :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ pp × 'c + 'k ⇒ ('s lifted, 's lifted) dg_state" where
  "sol_env 𝒢 p = snd (solution 𝒢 p)"

definition reader :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ pp × 'c + 'k ⇒ 's lifted" where
  "reader 𝒢 p = solved_local_reader (sol_vars 𝒢 p) (sol_env 𝒢 p)"

definition result :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ ('c, 'v) solved_table" where
  "result 𝒢 p = dg_result_for (rd 𝒢) (emp p) (place_cmb p) global_of (solution 𝒢 p)"
```

The solved global environment every recombination reads.

```
definition sol_global :: "(var_id ⇒ bool) ⇒ 'p ⇒ 'n ⇒ 's lifted" where
  "sol_global 𝒢 p = genv global_of (sol_env 𝒢 p)"
```

Where a call leads, as a function of the call site and the caller's context alone: this route applied to the state this solve published at that site. It names no concretization, so it belongs here rather than beside the soundness endpoints – which is what lets a registration that cannot export a binder publish it by spelling this out, exactly as it publishes `result`.

```
definition ctx_succ :: "(var_id ⇒ bool) ⇒ 'p ⇒ cfg_node ⇒ 'c
    ⇒ (var_id, exp) gcall_action ⇒ pname ⇒ 'c" where
  "ctx_succ 𝒢 p u ctx ca q =
     route 𝒢 u ctx (entry_of 𝒢 p (call_info_of ca q) (dg_local (sol_env 𝒢 p (Inl (u, ctx))))
                   (sol_global 𝒢 p)) ca"
```

Where a call leads when it contributes at all: `None` when the entered state is bottom, which is exactly when the routed call program drops the alternative, and otherwise the context `ctx_succ` names.

```
definition live_succ :: "(var_id ⇒ bool) ⇒ 'p ⇒ cfg_node ⇒ 'c
    ⇒ (var_id, exp) gcall_action ⇒ pname ⇒ 'c option" where
  "live_succ 𝒢 p u ctx ca q =
     (if entry_of 𝒢 p (call_info_of ca q) (dg_local (sol_env 𝒢 p (Inl (u, ctx))))
           (sol_global 𝒢 p) = Bot
      then None else Some (ctx_succ 𝒢 p u ctx ca q))"
```

The values one solution publishes. The seeds are read exactly as a table entry is, so an unwritten key reads as `Bot`. The shared global is read without that canonicalization: its local half is bottom by construction, which would make every value it holds read as `Bot`.

A step is what one edge's local step makes of a point's solved state: the term that edge contributes to its target's equation, re-evaluated on the solution. A target with several incoming edges stores only their join, so this is the

one place the contribution of a single edge can be read. It is the component's own step, run with the answers its handler gives on the recombined state, as the edge transfer runs it, and published with the state it describes.

The successor is `live_succ` read off the same solution, so a caller listing where every call leads does not solve the system again for each call.

The solution is an argument, so a caller holding the executable solver's answer reads it without solving again; `run` below reads the specification's solve.

```
definition solved_run_of :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ (pp × 'c) set × (pp × 'c + 'k ⇒ ('s lifted, 's lifted) dg_state)
    ⇒ ('c, 'n, 'v) solved_run" where
  "solved_run_of 𝒢 p sol =
     (let c = comp 𝒢 p;
          g = genv global_of (snd sol);
          read = (λd. map_lift (rd 𝒢) (canonicalize_lift (emp p) d))
      in ⦇ run_table = dg_result_for (rd 𝒢) (emp p) (place_cmb p) global_of sol,
           run_shared = (λn. map_lift (rd 𝒢) (g n)),
           run_seed = (λf ctx. read (place_cmb p
                         (dg_local (snd sol (Inr (seed (FunctionEntry f) ctx)))) g)),
           run_step = (λv ctx a. read (let d = place_cmb p (dg_local (snd sol (Inl (v, ctx)))) g
                                       in ⟦a⟧♯c d)),
           run_succ = (λu ctx ca q.
             let d = entry_of 𝒢 p (call_info_of ca q) (dg_local (snd sol (Inl (u, ctx)))) g
             in if d = Bot then None else Some (route 𝒢 u ctx d ca)) ⦈)"

definition run :: "(var_id ⇒ bool) ⇒ 'p ⇒ ('c, 'n, 'v) solved_run" where
  "run 𝒢 p = solved_run_of 𝒢 p (solution 𝒢 p)"

lemma run_table_run [simp]: "run_table (run 𝒢 p) = result 𝒢 p"
  by (simp add: run_def solved_run_of_def result_def Let_def)

lemma run_succ_run: "run_succ (run 𝒢 p) = live_succ 𝒢 p"
  by (simp add: run_def solved_run_of_def live_succ_def ctx_succ_def sol_env_def sol_global_def
      Let_def fun_eq_iff)
```

The contextual publication surface: the verdict a caller prints at a check, joined over its contexts. It is `classify_checks_verdicts` unchanged – generic in the context type already – so nothing here is policy-specific beyond supplying this pipeline's own result table.

```
definition verdict_report :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ (pp × exp × contextual_verdict) list" where
  "verdict_report 𝒢 p = classify_checks_verdicts (pcfg p) (result 𝒢 p) classify"
```

The same table read at one context: the state published there, and the check report off those states. These are `analysis_surface`'s readings of this pipeline's result, so every policy publishes the same surface; a context-insensitive run reads it at `()`.

```
definition state_at :: "(var_id ⇒ bool) ⇒ 'p ⇒ 'c ⇒ pp ⇒ 'v" where
  "state_at 𝒢 = analysis_surface.state_at (result 𝒢) bot_state"

definition report :: "(var_id ⇒ bool) ⇒ 'p ⇒ 'c ⇒ (pp × exp × check_result) list" where
  "report 𝒢 = analysis_surface.report (result 𝒢) bot_state classify pcfg"

definition report_with_state :: "(var_id ⇒ bool) ⇒ 'p
    ⇒ 'c ⇒ (pp × exp × check_result × bool × 'v) list" where
  "report_with_state 𝒢 = analysis_surface.report_with_state (result 𝒢) bot_state classify pcfg"

lemma state_at_unfold:
  "state_at 𝒢 p ctx v
     = (case lookup_table (result 𝒢 p) v ctx of Bot ⇒ bot_state | Lifted st ⇒ st)"
  by (simp add: state_at_def analysis_surface.state_at_def)
end
```

Register the exported locale equations for direct calls to the generic pipeline. Attributes inside the locale apply through interpretation; code generation here also needs equations for the constants with their locale parameters left abstract.

```
declare
  dg_pipeline.analysis_spec_def [code]
  dg_pipeline.comp_entry_def [code]
  dg_pipeline.entry_alt_def [code]
  dg_pipeline.entry_of_def [code]
  dg_pipeline.sol_global_def [code]
  dg_pipeline.init_publications_def [code]
  dg_pipeline.equations_def [code]
  dg_pipeline.solution_code [code]
```

```
dg_pipeline.terminates_code [code]
dg_pipeline.sol_vars_def [code]
dg_pipeline.sol_env_def [code]
dg_pipeline.reader_def [code]
dg_pipeline.result_def [code]
dg_pipeline.ctx_succ_def [code]
dg_pipeline.live_succ_def [code]
dg_pipeline.solved_run_of_def [code]
dg_pipeline.run_def [code]
dg_pipeline.verdict_report_def [code]
dg_pipeline.state_at_def [code]
dg_pipeline.report_def [code]
dg_pipeline.report_with_state_def [code]
```

The contracts

What an instance owes, and nothing more: its component is sound for the concretization its publication map induces, its entry answers one alternative, its two emptiness tests are exact, its seed keys are distinct from the analysis-wide global, its solver is a `certified_solver` (a post-solution over finitely many keys once it terminates), its classifier is correct, and its entry state describes every initial store. The equation system, the solve, the covered keys, the reader, the result table and the report are all fixed by `dg_pipeline` above and appear here only in conclusions. Each obligation is stated at the program's own declared globals, the one set the soundness statement uses.

```
locale dg_analysis =
  dg_pipeline comp emp rd init_st buffer_key global_of seed route root_ctx solve solve_dom
    bot_state classify place_spec place_cmb place_rl place_rg place_inits place_enter pcfg pentry
  + certified_solver solve solve_dom solve_c
  for κ :: kenv
    and pinit :: "'p ⇒ store set"
    and comp :: "(var_id ⇒ bool) ⇒ 'p ⇒ 's::semilattice_sup lifted local_spec"
    and emp :: "'p ⇒ 's ⇒ bool"
    and rd :: "(var_id ⇒ bool) ⇒ 's ⇒ 'v"
    and init_st :: 's
    and buffer_key :: 'k
    and global_of :: "'n ⇒ 'k"
    and seed :: "pp ⇒ 'c ⇒ 'k"
    and route :: "(var_id ⇒ bool) ⇒ pp ⇒ 'c ⇒ 's lifted ⇒ (var_id, exp) gcall_action ⇒ 'c"
    and root_ctx :: 'c
    and solve solve_dom
    and bot_state :: 'v
    and classify :: "exp ⇒ 'v ⇒ check_result"
    and γ_V :: "'v ⇒ store set"
    and empty_V :: "'v ⇒ bool"
    and solve_c :: "(pp × 'c, 'k, ('s lifted, 's lifted) dg_state) eqsT
                    ⇒ pp × 'c
                    ⇒ ((pp × 'c) set
                         × (pp × 'c + 'k ⇒ ('s lifted, 's lifted) dg_state)) option"
    and place_spec :: "'p ⇒ 's lifted local_spec
                       ⇒ (pp × 'c, 'k, 'n, 's lifted, 's lifted) dg_spec"
    and place_cmb :: "'p ⇒ 's lifted ⇒ ('n ⇒ 's lifted) ⇒ 's lifted"
    and place_rl :: "(var_id ⇒ bool) ⇒ 's lifted ⇒ 's lifted"
    and place_rg :: "(var_id ⇒ bool) ⇒ 's lifted ⇒ 's lifted"
    and place_inits :: "(var_id ⇒ bool) ⇒ 'p ⇒ ('n × 's lifted) list"
    and place_enter :: "'p ⇒ call_info ⇒ 's lifted ⇒ ('n ⇒ 's lifted) ⇒ 's lifted"
    and pcfg :: "'p ⇒ (var_id, exp) gcfg"
    and pentry :: "'p ⇒ cfg_node" +
  assumes comp_sound:
      "⋀p. sound_local_spec κ
               (λd. gamma_lift γ_V (map_lift (rd is_global) d))
               (comp is_global p)"
    and enter_single:
      "⋀p ci d. ls_enter (comp is_global p) (ls_channel (comp is_global p) d)
                   ci (d, d)
                 = [(d, comp_entry is_global p ci d)]"
    and place_contract:
      "⋀p. analysis_contract κ (analysis_spec is_global p)
               (λd e. gamma_lift γ_V
                        (map_lift (rd is_global) (place_cmb p d e)))"
    and place_enter_runs:
      "⋀p ci d σ. ∃pub. enter_runs (enter♯ (analysis_spec is_global p) ci)
          (mk_dg_man d global_of) σ
          [entry_alt is_global p ci d (genv global_of σ)] pub"
    and place_enter_deps:
      "⋀p ci d σ. ∃deps. enter_deps (enter♯ (analysis_spec is_global p) ci)
```

```
        (mk_dg_man d global_of) σ
        [entry_alt is_global p ci d (genv global_of σ)] deps"
  and place_entry_sound:
    "⋀p ci d σ pub s.
       enter_runs (enter♯ (analysis_spec is_global p) ci)
         (mk_dg_man d global_of) σ
         [entry_alt is_global p ci d (genv global_of σ)] pub
       ⟹ genv global_of pub ≤ genv global_of σ
       ⟹ s ∈ gamma_lift γV (map_lift (rd is_global)
            (place_cmb p d (genv global_of σ)))
       ⟹ s ∈ gamma_lift γV (map_lift (rd is_global)
                 (place_cmb p
                    (fst (entry_alt is_global p ci d (genv global_of σ)))
                    (genv global_of σ)))
         ∧ call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s
             ∈ gamma_lift γV (map_lift (rd is_global)
                 (place_cmb p
                    (snd (entry_alt is_global p ci d (genv global_of σ)))
                    (genv global_of σ)))"
  and place_cmb_bot: "⋀p e. place_cmb p Bot e = Bot"
  and place_init:
    "⋀p. gamma_lift γV (map_lift (rd is_global) (Lifted init_st))
       ⊆ gamma_lift γV (map_lift (rd is_global)
           (place_cmb p (Lifted init_st) (init_env is_global p)))"
  and empty_rd: "⋀p s. emp p s ⟷ emptyV (rd is_global s)"
  and emptyV_sound: "sound_emptiness emptyV γV"
  and seed_ne_buffer_key: "⋀v ctx. seed v ctx ≠ buffer_key"
  and seed_ne_global_of: "⋀v ctx n. seed v ctx ≠ global_of n"
  and classify_proved:
    "⋀c d s. classify c d = Check_Proved ⟹ s ∈ γV d ⟹ truthy (eval c s)"
  and classify_refuted:
    "⋀c d s. classify c d = Check_Refuted ⟹ s ∈ γV d
       ⟹ ¬ truthy (eval c s)"
  and bot_state_empty: "γV bot_state = {}"
  and init_sound:
    "⋀p. pinit p ⊆ γV (rd is_global init_st)"
  and pentry: "⋀p. pentry p = cfg_entry (pcfg p)"
  and pcfg_fin_intra: "⋀p. finite (intra (pcfg p))"
  and pcfg_fin_calls: "⋀p. finite (calls (pcfg p))"
  and pcfg_calls_unique: "⋀p. calls_source_unique (pcfg p)"
begin
```

The generic contract asks the published emptiness test only to be sound: a product of analyses may describe nothing without any one of them saying so. A particular instance may still be exact.

```
lemmas emptyV_soundD = sound_emptinessD [OF emptyV_sound]
```

Termination is a per-program side condition, and this is how a caller decides it: run the solver's own executable termination check on this program's equations. The premise stays `solve_dom` everywhere else, so nothing in the soundness argument depends on the check having been run.

```
lemma terminates_of_solve_c:
  assumes "solve_c (equations 𝒢 p) (root_query p) ≠ None"
  shows "terminates 𝒢 p"
  unfolding terminates_def by (rule dom_of_solve_c[OF assms])
```

An answer of the executable solver is the solve itself, so a run built from that answer is `run`, and its program needs no separate termination premise.

```
lemma solve_c_run:
  assumes "solve_c (equations 𝒢 p) (root_query p) = Some sol"
  shows "terminates 𝒢 p" and "solved_run_of 𝒢 p sol = run 𝒢 p"
  using assms terminates_of_solve_c solve_of_solve_c[OF assms]
  by (simp_all add: run_def solution_def)

lemma vars_finite_of_terminates:
  assumes "terminates 𝒢 p"
  shows "finite (sol_vars 𝒢 p)"
  using solve_fin[OF assms[unfolded terminates_def]]
  by (simp add: sol_vars_def solution_def)
```

The contexts a concrete call is admitted at under an entry-state policy: every context the entered state routes to. This is a top-level constant rather than something read out of a per-program interpretation, because an analysis publishes it – a caller stating a context-indexed collecting fact has to name the relation those contexts are indexed by, and an example checking a routing decision has to name it too.

With one alternative per call it is one context per abstract caller state; it is still a relation and not a function because several concrete callers sharing one abstract state reach the same context.

```
definition admitted_contexts :: "(var_id ⇒ bool) ⇒ 'p ⇒ 'c call_context_policy" where
  "admitted_contexts 𝒢 p =
     routed_entry_context_rel
       (λci d. [entry_alt 𝒢 p ci d (sol_global 𝒢 p)])
       (λd e. gamma_lift γ_V (map_lift (rd 𝒢) (place_cmb p d e)))
       (sol_env 𝒢 p) global_of (route 𝒢)"
```

What the assembly derives, for one program

Fixing one program `p`, this part equates the published table with the solved reader, puts the solver's answer in the routed spine's shape, and states routed soundness for any relation of admitted contexts.

```
context
  fixes p :: 'p
begin

abbreviation (input) pgs :: "var_id ⇒ bool" where "pgs ≡ is_global"
  — input-only, so interpreted facts print declared_global p
```

What a carrier state describes, published and concretized, and what a point's two halves describe once recombined against the solved global.

```
abbreviation cgam :: "'s lifted ⇒ store set" where
  "cgam d ≡ gamma_lift γ_V (map_lift (rd pgs) d)"

abbreviation (input) pgam :: "'s lifted ⇒ ('n ⇒ 's lifted) ⇒ store set" where
  "pgam d e ≡ cgam (place_cmb p d e)"

abbreviation (input) gsol :: "'n ⇒ 's lifted" where
  "gsol ≡ sol_global pgs p"

abbreviation entered :: "call_info ⇒ 's lifted ⇒ 's lifted" where
  "entered ci d ≡ entry_of pgs p ci d gsol"

lemma empty_rd_exact: "emp p s = empty_V (rd pgs s)"
  by (simp add: empty_rd)

lemma dg_spec_wf_analysis_spec [intro, simp]: "dg_spec_wf (analysis_spec pgs p)"
  by (rule analysis_contract.spec_wf[OF place_contract])

lemma pgam_Bot [simp]: "pgam Bot g = {}"
  by (simp add: place_cmb_bot)
```

The published table and the solved reader describe the same stores at every key. A caller states soundness against `lookup_table` of the result table, while the routed endpoints are stated against the reader; this is the equation between them, and it needs no coverage premise. At a covered key it is publication commuting with the normalization; at an uncovered one it is `Bot` against `Bot`, and both describe nothing.

```
lemma gamma_reader_eq_lookup:
  "pgam (reader pgs p (Inl (v, ctx))) gsol
     = gamma_lift γ_V (lookup_table (result pgs p) v ctx)"
proof -
  have gc: "gamma_lift γ_V (canonicalize_lift empty_V x) = gamma_lift γ_V x" for x
    by (rule gamma_lift_canonicalize_lift) (rule empty_V_soundD)
  show ?thesis
  proof (cases "(v, ctx) ∈ sol_vars pgs p")
    case True
    then show ?thesis
      by (simp add: reader_def result_def sol_vars_def sol_env_def sol_global_def gc
          map_lift_canonicalize_lift[of "emp p" "empty_V" "rd pgs", OF empty_rd_exact])
  next
    case False
    then show ?thesis by (simp add: reader_def result_def sol_vars_def bot_lifted_eq)
  qed
qed
```

The solver's answer, in the shape the routed spine consumes An analysis solves the buffered generator – a node with several intra predecessors or several returning calls publishes its analysis-wide contribution once per evaluation rather than once per contribution – while the framework states its soundness over the unbuffered one. `pp_routed` is that reconciliation at this pipeline's own equations, which the placement owes.

```
lemma pp_buffered:
```

```
  assumes solves: "terminates pgs p"
  shows "part_post_solution (equations pgs p) (root_query p)
          (sol_env pgs p) (sol_vars pgs p)"
  using solve_pp[OF solves[unfolded terminates_def]]
  by (simp add: sol_env_def sol_vars_def solution_def)

theorem pp_routed:
  assumes solves: "terminates pgs p"
  shows "part_post_solution
     (routed_node_rhs intra_predecessor_addr_list call_site_list (λ_. buffer_key) (route pgs)
        (λctx' src a. dg_spec_edge_program (analysis_spec pgs p) a src global_of)
        (routed_call_program (analysis_spec pgs p) global_of seed
           (static_resolve (pcfg p)) (λd. d = Bot))
        (routed_entry_seed_programs seed)
        (pcfg p) Bot (Lifted init_st) (place_rg pgs (Lifted init_st)))
     (root_query p) (sol_env pgs p) (sol_vars pgs p)"
  using part_post_solution_with_init[OF pp_buffered[OF solves, unfolded equations_def]]
  unfolding compiled_routed_eqs_for_def bot_lifted_eq
  by (rule pp_routed_of_buffered[OF dg_spec_wf_analysis_spec seed_ne_buffer_key])
```

The run starts below the solved environment once the solve covers the program entry: the entry node publishes the placement's initial global part at its own key, and each listed initial value at its name's key.

```
lemma init_publication_le_sol:
  assumes solves: "terminates pgs p"
    and entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ sol_vars pgs p"
    and mem: "(n, d) ∈ set (place_inits pgs p)"
  shows "d ≤ gsol n"
proof -
  have pp: "part_post_solution (equations pgs p) (root_query p) (sol_env pgs p) (sol_vars pgs p)"
    by (rule pp_buffered[OF solves])
  have "DG Bot d ≤ sol_env pgs p (Inr (global_of n))"
    by (rule post_bounded_with_init_at[OF post_bounded_of_part_post_solution[OF pp,
          unfolded equations_def] entry_cov])
       (use mem in ‹auto simp: init_publications_def pentry›)
  then show ?thesis by (simp add: sol_global_def less_eq_dg_state_def)
qed

lemma foldr_init_le:
  "∀(m, d) ∈ set xs. d ≤ f m
     ⟹ (⨆(m, d)←filter (λ(m, d). m = n) xs. d) ≤ (f n :: 's lifted)"
  by (induction xs) auto

lemma init_env_le_sol:
  assumes solves: "terminates pgs p"
    and entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ sol_vars pgs p"
    and own: "place_rg pgs (Lifted init_st) ≤ dg_global (sol_env pgs p (Inr buffer_key))"
  shows "init_env pgs p ≤ gsol"
proof (rule le_funI)
  fix n
  have "(⨆(m, d)←filter (λ(m, d). m = n) (place_inits pgs p). d) ≤ gsol n"
    by (rule foldr_init_le) (use init_publication_le_sol[OF solves entry_cov] in blast)
  moreover have "(if global_of n = buffer_key then place_rg pgs (Lifted init_st) else Bot)
                   ≤ gsol n"
    using own by (auto simp: sol_global_def)
  ultimately show "init_env pgs p n ≤ gsol n"
    unfolding init_env_def by simp
qed
```

The one entry alternative this carrier answers with The specification's entry answers with a single alternative, the recombined caller state and the callee frame entered from it, each split back by the placement. `entry_cover` is the only fact about it any context policy needs, namely that a concrete call from a described caller lands in the described callee.

What a call publishes on entry is below the solved global, once the node the call returns to is solved: that node's equation runs the call, and a post-solution bounds what every solved equation publishes.

```
lemma enter_pub_le_sol:
  assumes solves: "terminates pgs p"
    and cont: "(k, ctx) ∈ sol_vars pgs p"
    and ce: "(u, ca, FunctionEntry q, k) ∈ calls (pcfg p)"
    and R: "enter_runs (enter♯ (analysis_spec pgs p) (call_info_of ca q))
              (mk_dg_man (dg_local (sol_env pgs p (Inl (u, ctx)))) global_of)
              (sol_env pgs p) pairs pub"
  shows "genv global_of pub ≤ gsol"
```

```
proof (rule le_funI)
  fix n
  let ?K = "Inr (global_of n)"
  let ?σ = "sol_env pgs p"
  let ?callee = "routed_callee_call_program (analysis_spec pgs p) global_of seed (route pgs)
                  (λd. d = Bot) ctx ca u (dg_local (?σ (Inl (u, ctx)))) q"
  let ?call = "routed_call_program (analysis_spec pgs p) global_of seed
                (static_resolve (pcfg p)) (λd. d = Bot) (route pgs) ctx ca u k"
  let ?contribs = "routed_contribution_programs intra_predecessor_addr_list call_site_list (route pgs)
        (λctx' src a. dg_spec_edge_program (analysis_spec pgs p) a src global_of)
        (routed_call_program (analysis_spec pgs p) global_of seed
           (static_resolve (pcfg p)) (λd. d = Bot))
        (routed_entry_seed_programs seed) (pcfg p) ctx k"
  let ?rhs = "routed_node_rhs intra_predecessor_addr_list call_site_list (λ_. buffer_key) (route pgs)
        (λctx' src a. dg_spec_edge_program (analysis_spec pgs p) a src global_of)
        (routed_call_program (analysis_spec pgs p) global_of seed
           (static_resolve (pcfg p)) (λd. d = Bot))
        (routed_entry_seed_programs seed)
        (pcfg p) Bot (Lifted init_st) (place_rg pgs (Lifted init_st))"
  have fin: "finite (calls (pcfg p))"
    by (simp add: pcfg_fin_intra pcfg_fin_calls)
  have wf: "∀t ∈ set ?contribs. sp_wf t"
    by (rule routed_contribution_programs_wf) auto
  have "pub ?K ≤ sides_of_program ?callee ?σ ?K"
    unfolding routed_callee_call_program_def
    by (simp add: sp_compile_bind sp_wf_observes enter_runsD_sides[OF R] sup_fun_def)
  also have "... ≤ sides_of_program ?call ?σ ?K"
    by (rule routed_call_program_sides_ge_at[OF dg_spec_wf_analysis_spec])
       (use ce fin in simp)
  also have "... ≤ sides_of_program (side_rhs_fold_dg
      (if k = cfg_entry (pcfg p) then Bot ⊔ Lifted init_st else Bot) ?contribs)
      ?σ ?K"
  proof (rule sides_le_side_rhs_fold_dg[OF wf])
    have "(u, ca) ∈ set (call_site_list (pcfg p) k)" using ce fin by auto
    then show "?call ∈ set ?contribs" by (rule routed_contribution_programs_combineI)
  qed
  also have "... ≤ sides_of_rhs (?rhs (k, ctx)) ?σ ?K"
    unfolding routed_node_rhs_def Let_def
    by (cases "k = cfg_entry (pcfg p)") (auto simp: Let_def)
  also have "... ≤ ?σ ?K"
    using pp_routed[OF solves] cont by (blast dest: le_funD)
  finally show "genv global_of pub n ≤ gsol n"
    unfolding sol_global_def by (simp add: less_eq_dg_state_def)
qed

lemma entry_cover:
  assumes solves: "terminates pgs p"
    and cont: "(k, ctx) ∈ sol_vars pgs p"
    and ce: "(u, CallEdge dst pars args, FunctionEntry q, k) ∈ calls (pcfg p)"
    and sin: "s ∈ pgam (dg_local (sol_env pgs p (Inl (u, ctx)))) gsol"
  shows "entry_pairs_cover (λd. pgam d gsol) s (call_enter κ (CallEdge dst pars args) s)
           [entry_alt pgs p (call_info_of (CallEdge dst pars args) q)
              (dg_local (sol_env pgs p (Inl (u, ctx)))) gsol]"
proof -
  let ?ci = "call_info_of (CallEdge dst pars args) q"
  let ?d = "dg_local (sol_env pgs p (Inl (u, ctx)))"
  let ?alt = "entry_alt pgs p ?ci ?d gsol"
  obtain pub where R: "enter_runs (enter♯ (analysis_spec pgs p) ?ci)
      (mk_dg_man ?d global_of) (sol_env pgs p)
      [entry_alt pgs p ?ci ?d (genv global_of (sol_env pgs p))] pub"
    using place_enter_runs by blast
  have le: "genv global_of pub ≤ genv global_of (sol_env pgs p)"
    using enter_pub_le_sol[OF solves cont ce R] by (simp add: sol_global_def)
  have "s ∈ pgam (fst ?alt) gsol ∧ call_enter κ (CallEdge dst pars args) s ∈ pgam (snd ?alt) gsol"
    using place_entry_sound[OF R le sin[unfolded sol_global_def]]
    unfolding sol_global_def by (simp)
  then show ?thesis
    by (intro entry_pairs_coverI[of "fst ?alt" "snd ?alt"]) simp_all
qed
```

The routed soundness statement, at any admitted-context relation Which contexts a concrete call is admitted at is the one thing a context policy still chooses, so `adm` is a parameter here rather than a derived object. What the policy owes about it is exactly two facts: a context `adm` admits is the one this pipeline's route computes on the entered state, at a covered unknown (`cover_R`); and `adm` admits at least one context for every concrete call at a covered call site (`total_R`). Everything else below is fixed by the pipeline.

The specification is sound for the recombined concretization; the placement owes this, so the routed statement below never re-derives it.

```
interpretation dg_base: analysis_contract κ "analysis_spec pgs p"
    "λd e. pgam d e"
  by (rule place_contract)

lemma routed_analysis_sound_of_live:
  fixes adm :: "'c call_context_policy"
  assumes solves: "terminates pgs p"
    and fwd_ok: "⋀u a v ctx. (u, ctx) ∈ sol_vars pgs p
        ⟹ dg_local (sol_env pgs p (Inl (u, ctx))) ≠ Bot
        ⟹ (u, a, v) ∈ intra (pcfg p) ⟹ (v, ctx) ∈ sol_vars pgs p"
    and comb_fwd_ok: "⋀cl c1 dst pars args q cont. (cl, c1) ∈ sol_vars pgs p
        ⟹ (cl, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ (cont, c1) ∈ sol_vars pgs p"
    and cover_R: "⋀u ctx dst pars args q cont s ctx'.
        (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) q) s
              (call_enter κ (CallEdge dst pars args) s)
        ⟹ entered (call_info_of (CallEdge dst pars args) q)
              (dg_local (sol_env pgs p (Inl (u, ctx)))) ≠ Bot
        ⟹ route pgs u ctx
                (entered (call_info_of (CallEdge dst pars args) q)
                   (dg_local (sol_env pgs p (Inl (u, ctx)))))
                (CallEdge dst pars args) = ctx'
           ∧ (FunctionEntry q, ctx') ∈ sol_vars pgs p"
    and total_R: "⋀u ctx dst pars args q cont s.
        (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ s ∈ pgam (dg_local (sol_env pgs p (Inl (u, ctx)))) gsol
        ⟹ adm u ctx (call_info_of (CallEdge dst pars args) q) s
                     (call_enter κ (CallEdge dst pars args) s) ≠ {}"
  shows "routed_analysis κ (analysis_spec pgs p) (λd e. pgam d e) (pcfg p)
     buffer_key global_of
     (route pgs) Bot (Lifted init_st) (place_rg pgs (Lifted init_st)) (sol_env pgs p)
     (sol_vars pgs p) (root_query p)
     seed (λd. d = Bot) adm (λd e. map_lift (rd pgs) (place_cmb p d e)) γ_V empty_V
     classify"
proof (unfold_locales, goal_cases CmbWf ExtraWf FinE PP SgCov SgUncov Fwd FinC CallsUnique
    SeedUnknown SeedNeGlobal IsBotBot IsBotSound ResolveSound EnterCover EnterTotal CombFwd GammaRd
    EmptyExact ClProved ClRefuted VarsFin)
  case CmbWf show ?case by (rule sp_wf_routed_call_program[OF dg_spec_wf_analysis_spec])
next
  case ExtraWf then show ?case by (rule sp_wf_routed_entry_seed_programs)
next
  case FinE show ?case by (simp add: pcfg_fin_intra pcfg_fin_calls)
next
  case PP show ?case by (rule post_bounded_of_part_post_solution[OF pp_routed[OF solves]])
next
  case (SgCov v c) then show ?case by simp
next
  case (SgUncov v c) then show ?case by (simp add: bot_lifted_eq)
next
  case (Fwd u a v c)
  have "dg_local (sol_env pgs p (Inl (u, c))) ≠ Bot"
    using Fwd(2) by auto
  with Fwd(1,3) show ?case by (blast intro: fwd_ok)
next
  case FinC show ?case by (simp add: pcfg_fin_intra pcfg_fin_calls)
next
  case CallsUnique show ?case by (rule pcfg_calls_unique)
next
  case (SeedUnknown q ctx) show ?case by (rule seed_ne_buffer_key)
next
  case (SeedNeGlobal q ctx v) show ?case by (rule seed_ne_global_of)
next
  case IsBotBot show ?case by simp
next
  case (IsBotSound d g') then show ?case by simp
next
  case (ResolveSound u ctx dst pars args q cont s)
  then show ?case by (simp add: pcfg_fin_intra pcfg_fin_calls)
next
  case (EnterCover u ctx dst pars args q cont s ctx')
  let ?ci = "call_info_of (CallEdge dst pars args) q"
```

```
  let ?caller = "dg_local (sol_env pgs p (Inl (u, ctx)))"
  let ?g = "genv global_of (sol_env pgs p)"
  have sin: "s ∈ pgam ?caller gsol"
    using EnterCover(3) by (simp add: sol_global_def)
  have cov: "entry_pairs_cover (λd. pgam d gsol) s
      (call_enter κ (CallEdge dst pars args) s) [entry_alt pgs p ?ci ?caller gsol]"
    by (rule entry_cover[OF solves comb_fwd_ok[OF EnterCover(1,2)] EnterCover(2) sin])
  obtain c e where ce: "entry_alt pgs p ?ci ?caller gsol = (c, e)"
    by (cases "entry_alt pgs p ?ci ?caller gsol")
  have nbE: "entered ?ci ?caller ≠ Bot"
  proof
    assume "entered ?ci ?caller = Bot"
    with cov ce show False by (simp add: entry_pairs_cover_def entry_of_def place_cmb_bot)
  qed
  have req: "route pgs u ctx (entered ?ci ?caller) (CallEdge dst pars args) = ctx'"
    and covE: "(FunctionEntry q, ctx') ∈ sol_vars pgs p"
    using cover_R[OF EnterCover(1,2,4) nbE] by blast+
  obtain pub where runs: "enter_runs (enter♯ (analysis_spec pgs p) ?ci)
      (mk_dg_man ?caller global_of) (sol_env pgs p)
      [entry_alt pgs p ?ci ?caller ?g] pub"
    using place_enter_runs by blast
  obtain deps where deps: "enter_deps (enter♯ (analysis_spec pgs p) ?ci)
      (mk_dg_man ?caller global_of) (sol_env pgs p)
      [entry_alt pgs p ?ci ?caller ?g] deps"
    using place_enter_deps by blast
  have ce': "entry_alt pgs p ?ci ?caller ?g = (c, e)"
    using ce by (simp add: sol_global_def)
  have sc: "s ∈ pgam c ?g" and ec:
    "call_enter κ (CallEdge dst pars args) s ∈ pgam e ?g"
    using cov ce by (simp_all add: entry_pairs_cover_def sol_global_def)
  have re: "route pgs u ctx e (CallEdge dst pars args) = ctx'"
    using req ce by (simp add: entry_of_def)
  show ?case
    using runs deps sc ec re covE unfolding ce' by fastforce
next
  case (EnterTotal u ctx dst pars args q cont s)
  then show ?case by (intro total_R) (simp_all add: sol_global_def)
next
  case (CombFwd cl c1 dst pars args q cont)
  then show ?case by (rule comb_fwd_ok)
next
  case (GammaRd d g') show ?case by simp
next
  case (EmptyExact v) then show ?case by (rule empty_V_soundD)
next
  case (ClProved c d s) then show ?case by (rule classify_proved)
next
  case (ClRefuted c d s) then show ?case by (rule classify_refuted)
next
  case VarsFin show ?case by (rule vars_finite_of_terminates[OF solves])
qed

lemma routed_analysis_sound_of:
  fixes adm :: "'c call_context_policy"
  assumes solves: "terminates pgs p"
    and fwd_ok: "⋀u a v ctx. (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, a, v) ∈ intra (pcfg p) ⟹ (v, ctx) ∈ sol_vars pgs p"
    and comb_fwd_ok: "⋀cl c1 dst pars args q cont. (cl, c1) ∈ sol_vars pgs p
        ⟹ (cl, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ (cont, c1) ∈ sol_vars pgs p"
    and cover_R: "⋀u ctx dst pars args q cont s ctx'.
        (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) q) s
              (call_enter κ (CallEdge dst pars args) s)
        ⟹ route pgs u ctx
                (entered (call_info_of (CallEdge dst pars args) q)
                   (dg_local (sol_env pgs p (Inl (u, ctx)))))
                (CallEdge dst pars args) = ctx'
           ∧ (FunctionEntry q, ctx') ∈ sol_vars pgs p"
    and total_R: "⋀u ctx dst pars args q cont s.
        (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ s ∈ pgam (dg_local (sol_env pgs p (Inl (u, ctx)))) gsol
        ⟹ adm u ctx (call_info_of (CallEdge dst pars args) q) s
                     (call_enter κ (CallEdge dst pars args) s) ≠ {}"
```

```
  shows "routed_analysis κ (analysis_spec pgs p) (λd e. pgam d e) (pcfg p)
     buffer_key global_of
     (route pgs) Bot (Lifted init_st) (place_rg pgs (Lifted init_st)) (sol_env pgs p)
     (sol_vars pgs p) (root_query p)
     seed (λd. d = Bot) adm (λd e. map_lift (rd pgs) (place_cmb p d e)) γ_V empty_V
     classify"
  by (rule routed_analysis_sound_of_live [where adm = adm, OF solves _ comb_fwd_ok _ total_R])
     (blast intro: fwd_ok dest: cover_R)+
```

The published endpoint, under termination and coverage Under termination and closure of the solved keys, `activation_collect_sound_of` bounds every admitted activation by the solved table's entry at its context.

```
lemma cinit_le_init: "pinit p ⊆ pgam (Lifted init_st) (init_env pgs p)"
  using init_sound[of p] place_init[of p] by auto
```

Every activation the activation-trace semantics admits at a context is described by the solved table's entry for that context. The coverage premises are the shape a solver's own reachable set supplies: an edge out of an unknown the solve visited lands on one it also visited.

```
lemma activation_collect_sound_of:
  fixes adm :: "'c call_context_policy"
  assumes solves: "terminates pgs p"
    and entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ sol_vars pgs p"
    and fwd_ok: "⋀u a v ctx. (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, a, v) ∈ intra (pcfg p) ⟹ (v, ctx) ∈ sol_vars pgs p"
    and comb_fwd_ok: "⋀cl c1 dst pars args q cont. (cl, c1) ∈ sol_vars pgs p
        ⟹ (cl, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ (cont, c1) ∈ sol_vars pgs p"
    and cover_R: "⋀u ctx dst pars args q cont s ctx'.
        (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) q) s
              (call_enter κ (CallEdge dst pars args) s)
        ⟹ route pgs u ctx
                (entered (call_info_of (CallEdge dst pars args) q)
                   (dg_local (sol_env pgs p (Inl (u, ctx)))))
                (CallEdge dst pars args) = ctx'
            ∧ (FunctionEntry q, ctx') ∈ sol_vars pgs p"
    and total_R: "⋀u ctx dst pars args q cont s.
        (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ s ∈ pgam (dg_local (sol_env pgs p (Inl (u, ctx)))) gsol
        ⟹ adm u ctx (call_info_of (CallEdge dst pars args) q) s
                     (call_enter κ (CallEdge dst pars args) s) ≠ {}"
  shows "A_κ,adm,root_ctx,pcfg p,pinit p v ctx
          ⊆ pgam (reader pgs p (Inl (v, ctx))) gsol"
proof -
  interpret adapter: routed_analysis κ "analysis_spec pgs p" "λd e. pgam d e"
      "pcfg p" buffer_key global_of "route pgs" Bot "Lifted init_st"
      "place_rg pgs (Lifted init_st)" "sol_env pgs p" "sol_vars pgs p" "root_query p" seed
      "λd. d = Bot" adm "λd e. map_lift (rd pgs) (place_cmb p d e)" γ_V empty_V classify
    by (rule routed_analysis_sound_of
          [where adm = adm, OF solves fwd_ok comb_fwd_ok cover_R total_R])
  have init_env_le: "init_env pgs p ≤ genv global_of (sol_env pgs p)"
    using init_env_le_sol[OF solves entry_cov adapter.pp_entry_init_global_bound[OF entry_cov]]
    by (simp add: sol_global_def)
  show ?thesis
    using adapter.activation_collect_dg_sound[OF entry_cov _ init_env_le, of "pinit p" v
      ctx]
      cinit_le_init
    by (simp add: reader_def sol_global_def)
qed
```

The two context policies this assembly supports `entry_context_rel` is the relation an entry-state policy induces: a concrete call is admitted at every context some covering alternative of the entry answer routes to. With one alternative per call that is a single context per abstract caller state, but several concrete callers sharing an abstract state may still reach one context, which is exactly why a relation and not a function.

```
lemma admitted_contexts_alt:
  "admitted_contexts pgs p =
     routed_entry_context_rel (λci d. [entry_alt pgs p ci d gsol]) (λd e. pgam d e)
       (sol_env pgs p) global_of (route pgs)"
  by (simp add: admitted_contexts_def)

abbreviation entry_context_rel :: "'c call_context_policy" where
```

```
  "entry_context_rel ≡ admitted_contexts pgs p"

lemma entry_state_routed_analysis_sound:
  assumes solves: "terminates pgs p"
    and fwd_ok: "⋀u a v ctx. (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, a, v) ∈ intra (pcfg p) ⟹ (v, ctx) ∈ sol_vars pgs p"
    and call_fwd_ok: "⋀u ctx dst pars args q cont. (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ (FunctionEntry q,
              route pgs u ctx
                (entered (call_info_of (CallEdge dst pars args) q)
                   (dg_local (sol_env pgs p (Inl (u, ctx)))))
                (CallEdge dst pars args))
             ∈ sol_vars pgs p"
    and comb_fwd_ok: "⋀cl c1 dst pars args q cont. (cl, c1) ∈ sol_vars pgs p
        ⟹ (cl, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ (cont, c1) ∈ sol_vars pgs p"
  shows "routed_analysis κ (analysis_spec pgs p) (λd e. pgam d e) (pcfg p)
     buffer_key global_of
     (route pgs) Bot (Lifted init_st) (place_rg pgs (Lifted init_st)) (sol_env pgs p)
     (sol_vars pgs p) (root_query p)
     seed (λd. d = Bot) entry_context_rel
     (λd e. map_lift (rd pgs) (place_cmb p d e)) γ_V empty_V classify"
proof (rule routed_analysis_sound_of
    [where adm = entry_context_rel, OF solves fwd_ok comb_fwd_ok])
  fix u ctx dst pars args q cont and s :: "var_id ⇒ int" and ctx'
  assume covV: "(u, ctx) ∈ sol_vars pgs p"
    and ce: "(u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)"
    and Rc: "ctx' ∈ entry_context_rel u ctx (call_info_of (CallEdge dst pars args) q) s
               (call_enter κ (CallEdge dst pars args) s)"
  let ?ci = "call_info_of (CallEdge dst pars args) q"
  let ?caller = "dg_local (sol_env pgs p (Inl (u, ctx)))"
  from Rc[unfolded admitted_contexts_alt] obtain cont' entry
    where mem: "(cont', entry) ∈ set [entry_alt pgs p ?ci ?caller gsol]"
      and req0: "ctx' = route pgs u ctx entry
                   (CallEdge (ci_dst ?ci) (ci_formals ?ci) (ci_args ?ci))"
    by (rule routed_entry_context_relE)
  have "entry = entered ?ci ?caller" using mem by (simp add: entry_of_def) (metis snd_conv)
  with req0 have req: "route pgs u ctx (entered ?ci ?caller) (CallEdge dst pars args) = ctx'"
    by simp
  show "route pgs u ctx (entered ?ci ?caller) (CallEdge dst pars args) = ctx'
        ∧ (FunctionEntry q, ctx') ∈ sol_vars pgs p"
    using req call_fwd_ok[OF covV ce] by simp
next
  fix u ctx dst pars args q cont and s :: "var_id ⇒ int"
  assume covV: "(u, ctx) ∈ sol_vars pgs p"
    and ce: "(u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)"
    and sin: "s ∈ pgam (dg_local (sol_env pgs p (Inl (u, ctx)))) gsol"
  show "entry_context_rel u ctx (call_info_of (CallEdge dst pars args) q) s
                (call_enter κ (CallEdge dst pars args) s) ≠ {}"
    unfolding admitted_contexts_alt
    by (rule routed_entry_context_rel_total)
       (use entry_cover[OF solves comb_fwd_ok[OF covV ce] ce sin]
        in ‹simp add: sol_global_def›)
qed
```

The two endpoints an entry-state caller consumes: the per-context collecting bound, and the existence of a context for every valid activation trace, which a source-level caller needs to name a context witness at all. Both are the routed spine's own theorems at the interpretation just established.

```
context
  assumes solves: "terminates pgs p"
    and fwd_ok: "⋀u a v ctx. (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, a, v) ∈ intra (pcfg p) ⟹ (v, ctx) ∈ sol_vars pgs p"
    and call_fwd_ok: "⋀u ctx dst pars args q cont. (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ (FunctionEntry q,
              route pgs u ctx
                (entered (call_info_of (CallEdge dst pars args) q)
                   (dg_local (sol_env pgs p (Inl (u, ctx)))))
                (CallEdge dst pars args))
             ∈ sol_vars pgs p"
    and comb_fwd_ok: "⋀cl c1 dst pars args q cont. (cl, c1) ∈ sol_vars pgs p
        ⟹ (cl, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ (cont, c1) ∈ sol_vars pgs p"
begin
```

```
interpretation entry: routed_analysis κ "analysis_spec pgs p" "λd e. pgam d e"
    "pcfg p" buffer_key global_of "route pgs" Bot "Lifted init_st"
    "place_rg pgs (Lifted init_st)" "sol_env pgs p" "sol_vars pgs p" "root_query p" seed
    "λd. d = Bot" entry_context_rel "λd e. map_lift (rd pgs) (place_cmb p d e)"
    γ_V empty_V classify
  by (rule entry_state_routed_analysis_sound [OF solves fwd_ok call_fwd_ok comb_fwd_ok])

lemma entry_init_env_le:
  assumes entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ sol_vars pgs p"
  shows "init_env pgs p ≤ genv global_of (sol_env pgs p)"
  using init_env_le_sol[OF solves entry_cov entry.pp_entry_init_global_bound[OF entry_cov]]
  by (simp add: sol_global_def)

corollary entry_state_activation_collect_sound:
  assumes entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ sol_vars pgs p"
  shows "A_κ,entry_context_rel,root_ctx,pcfg p,pinit p v ctx
         ⊆ pgam (reader pgs p (Inl (v, ctx))) gsol"
  using entry.activation_collect_dg_sound
      [OF entry_cov _ entry_init_env_le[OF entry_cov], of "pinit p" v ctx]
    cinit_le_init
  by (simp add: reader_def sol_global_def)

theorem entry_state_has_context:
  assumes entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ sol_vars pgs p"
    and trace: "t ∈ T_κ,pcfg p,pinit p"
  shows
    "∃c. activation_context_rel κ entry_context_rel root_ctx (pcfg p) t c"
  using entry.routed_valid_activation_trace_has_context
      [OF entry_cov _ entry_init_env_le[OF entry_cov] trace] cinit_le_init
  by simp
```

The two together: covering the entry is enough for the activation buckets to exhaust the context-insensitive collection, so a caller holding only a `node_collect` membership – which is what a source run delivers – can pass to the bucket its own call history produced. Without this the per-context bounds above say nothing about a run whose context is not known in advance.

```
theorem entry_state_node_collect_eq_Union:
  assumes entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ sol_vars pgs p"
  shows "C_κ,pcfg p,pinit p v
         = (⋃ctx. A_κ,entry_context_rel,root_ctx,pcfg p,pinit p v ctx)"
  by (rule node_collect_eq_Union_activation_of_has_context)
     (rule entry_state_has_context [OF entry_cov])

end
```

A route that never reads the state it is handed is a function of the call site and the caller's context alone, so the contexts it admits are the graph of a context function on concrete stores – the shape the call-string and context-insensitive policies both publish. `route_const` is the whole of what such a policy owes; `ctx_fun` is its trace-semantic counterpart.

```
corollary fun_route_activation_collect_sound:
  fixes ctx_fun :: "cfg_node ⇒ 'c ⇒ store ⇒ 'c"
  assumes route_const: "⋀u ctx d ca s. route pgs u ctx d ca = ctx_fun u ctx s"
    and solves: "terminates pgs p"
    and fwd_ok: "⋀u a v ctx. (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, a, v) ∈ intra (pcfg p) ⟹ (v, ctx) ∈ sol_vars pgs p"
    and call_fwd_ok: "⋀u ctx dst pars args q cont d. (u, ctx) ∈ sol_vars pgs p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ (FunctionEntry q, route pgs u ctx d (CallEdge dst pars args))
             ∈ sol_vars pgs p"
    and comb_fwd_ok: "⋀cl c1 dst pars args q cont. (cl, c1) ∈ sol_vars pgs p
        ⟹ (cl, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ (cont, c1) ∈ sol_vars pgs p"
    and entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ sol_vars pgs p"
  shows "A_κ,context_policy_of_fun ctx_fun,root_ctx,pcfg p,pinit p v ctx
         ⊆ pgam (reader pgs p (Inl (v, ctx))) gsol"
proof (rule activation_collect_sound_of[OF solves entry_cov fwd_ok comb_fwd_ok])
  fix u ctx dst pars args q cont and s :: "var_id ⇒ int" and ctx'
  assume covV: "(u, ctx) ∈ sol_vars pgs p"
    and ce: "(u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)"
    and Rc: "ctx' ∈ context_policy_of_fun ctx_fun u ctx
               (call_info_of (CallEdge dst pars args) q) s
               (call_enter κ (CallEdge dst pars args) s)"
  have req: "route pgs u ctx d (CallEdge dst pars args) = ctx'" for d
    using Rc
    by (simp add:
```

```
      route_const[of _ _ _ _
        "call_enter κ (CallEdge dst pars args) s"])
  show "route pgs u ctx
          (entered (call_info_of (CallEdge dst pars args) q)
             (dg_local (sol_env pgs p (Inl (u, ctx)))))
          (CallEdge dst pars args) = ctx'
        ∧ (FunctionEntry q, ctx') ∈ sol_vars pgs p"
    using req call_fwd_ok[OF covV ce] by simp
next
  fix u ctx dst pars args q cont and s :: "var_id ⇒ int"
  show "context_policy_of_fun ctx_fun u ctx
                (call_info_of (CallEdge dst pars args) q) s
                (call_enter κ (CallEdge dst pars args) s) ≠ {}"
    by simp
qed
```

The functional route's counterpart of the entry-state union. It needs nothing at all: `activation_context_of` is total, so every valid activation trace carries a context without any coverage having been established.

```
theorem fun_route_node_collect_eq_Union:
  "𝒞κ,pcfg p,pinit p ᵛ
     = (⋃ctx. 𝒜κ,context_policy_of_fun ctx_fun,root_ctx,pcfg p,pinit p ᵛ ctx)"
  by (rule node_collect_eq_Union_activation_of_fun)

end

end
```

Every variable in the local half

The placement every analysis uses unless it shares program globals: the specification is the component's own `dg_spec_of`, a point's state is its local unknown alone, and nothing is published at the analysis-wide global. Its placement obligations follow from the component's soundness and its single-alternative entry, so an instance owes exactly what it owed before the placement was a parameter.

```
theorem dg_analysis_whole_stateI:
  fixes comp ::
    "(var_id ⇒ bool) ⇒ 'p ⇒ 's::semilattice_sup lifted local_spec"
    and seed :: "pp ⇒ 'c ⇒ 'k"
  assumes solver: "certified_solver solve solve_dom solve_c"
    and comp_sound:
      "⋀p. sound_local_spec κ
               (λd. gamma_lift γV (map_lift (rd is_global) d))
               (comp is_global p)"
    and enter_single:
      "⋀p ci d. ls_enter (comp is_global p) (ls_channel (comp is_global p) d)
                    ci (d, d)
                  = [(d, dg_pipeline.comp_entry comp is_global p ci d)]"
    and empty_rd: "⋀p s. emp p s ⟷ emptyV (rd is_global s)"
    and emptyV_sound: "sound_emptiness emptyV γV"
    and seed_ne: "⋀v ctx. seed v ctx ≠ buffer_key"
    and classify_proved:
      "⋀c d s. classify c d = Check_Proved ⟹ s ∈ γV d ⟹ truthy (eval c s)"
    and classify_refuted:
      "⋀c d s. classify c d = Check_Refuted ⟹ s ∈ γV d ⟹ ¬ truthy (eval c s)"
    and bot_state_empty: "γV bot_state = {}"
    and init_sound: "⋀p. pinit p ⊆ γV (rd is_global init_st)"
    and seed_ne_global: "⋀v ctx n. seed v ctx ≠ global_of n"
    and pentry: "⋀p. pentry p = cfg_entry (pcfg p)"
    and fin_intra: "⋀p. finite (intra (pcfg p))"
    and fin_calls: "⋀p. finite (calls (pcfg p))"
    and calls_unique: "⋀p. calls_source_unique (pcfg p)"
  shows "dg_analysis κ pinit comp emp rd init_st buffer_key global_of seed
           solve solve_dom bot_state classify γV emptyV solve_c
           (λ𝒢 c. dg_spec_of c) (λp d e. d) (λ𝒢 d. d) (λ𝒢 d. Bot) (λ𝒢 p. [])
           (λp ci d e. d) pcfg pentry"
proof (rule dg_analysis.intro[OF solver dg_analysis_axioms.intro], goal_cases CompSound
    EnterSingle Contract Runs Deps EntrySound CmbBot Init EmptyRd EmptyV SeedNe SeedNeGlobal
    ClProved ClRefuted BotState InitSound PEntry FinIntra FinCalls CallsUnique)
  case (CompSound p) show ?case by (rule comp_sound)
next
  case (EnterSingle p ci d) show ?case by (rule enter_single)
next
  case (Contract p) show ?case
    unfolding dg_pipeline.analysis_spec_def by (rule dg_spec_of_contract[OF comp_sound])
next
```

```
  case (Runs p ci d σ) show ?case
    by (rule exI, simp add: dg_pipeline.analysis_spec_def dg_spec_of_def enter_single
        dg_pipeline.entry_alt_def, rule enter_runs_local_enter_transfer_mk_dg_man)
next
  case (Deps p ci d σ) show ?case
    by (rule exI, simp add: dg_pipeline.analysis_spec_def dg_spec_of_def enter_single
        dg_pipeline.entry_alt_def, rule enter_deps_local_enter_transfer_mk_dg_man)
next
  case (EntrySound p ci d σ pub s)
  let ?c = "comp is_global p"
  obtain q where "q ∈ set (ls_enter ?c (ls_channel ?c d) ci (d, d))"
      "s ∈ gamma_lift γV (map_lift (rd is_global) (fst q))"
      "call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci)) s
         ∈ gamma_lift γV (map_lift (rd is_global) (snd q))"
    using comp_sound[of p] EntrySound(3) ls_channel_sound[OF comp_sound[of p]]
    unfolding sound_local_spec_def by (metis fst_conv)
  then show ?case
    by (simp add: enter_single dg_pipeline.entry_alt_def)
next
  case (CmbBot p g) show ?case by simp
next
  case (Init p) show ?case by simp
next
  case (EmptyRd p s) show ?case by (rule empty_rd)
next
  case EmptyV show ?case by (rule emptyV_sound)
next
  case (SeedNe v ctx) show ?case by (rule seed_ne)
next
  case (SeedNeGlobal v ctx n) show ?case by (rule seed_ne_global)
next
  case (ClProved c d s) then show ?case by (rule classify_proved)
next
  case (ClRefuted c d s) then show ?case by (rule classify_refuted)
next
  case BotState show ?case by (rule bot_state_empty)
next
  case (InitSound p) show ?case by (rule init_sound)
next
  case (PEntry p) show ?case by (rule pentry)
next
  case (FinIntra p) show ?case by (rule fin_intra)
next
  case (FinCalls p) show ?case by (rule fin_calls)
next
  case (CallsUnique p) show ?case by (rule calls_unique)
qed
```

Program globals keyed by name

The placement that keeps each program global at its own unknown `global_of x`. The specification is the component wrapped by the keyed lifter, and a point's state is its local half recombined with the solved environment over the program's declared globals. The root publishes each declared global's initial value at its key, and the node-owned buffer stays empty.

Beyond the component's own soundness, the carrier owes monotone operations, that a name outside a read set is cut below `free` of that set, and two facts about concretization: the frame law `mix` of `keyed_split_contract`, and that a state is described by its local half recombined with its own cuts.

```
theorem dg_analysis_keyedI:
  fixes comp ::
    "(var_id ⇒ bool) ⇒ 'p ⇒ 's::semilattice_sup lifted local_spec"
    and seed :: "pp ⇒ 'c ⇒ 'k"
    and global_of :: "var_id ⇒ 'k"
    and cmb :: "'s lifted ⇒ 's lifted ⇒ 's lifted"
    and rg :: "var_id ⇒ 's lifted ⇒ 's lifted"
    and pglobs :: "'p ⇒ var_id list"
  assumes solver: "certified_solver solve solve_dom solve_c"
    and comp_sound:
      "⋀p. sound_local_spec κ
              (λd. gamma_lift γV (map_lift (rd is_global) d))
              (comp is_global p)"
    and enter_single:
      "⋀p ci d. ls_enter (comp is_global p) (ls_channel (comp is_global p) d)
                  ci (d, d)
                = [(d, dg_pipeline.comp_entry comp is_global p ci d)]"
```

```
    and cmb_mono: "⋀d d' g g'. d ≤ d' ⟹ g ≤ g' ⟹ cmb d g ≤ cmb d' g'"
    and cmb_rl: "⋀x g. cmb (rl x) g = cmb x g"
    and rl_cmb: "⋀d g. rl (cmb d g) = rl d"
    and cmb_bot: "⋀g. cmb Bot g = Bot"
    and rg_mono: "⋀x v v'. v ≤ v' ⟹ rg x v ≤ rg x v'"
    and rg_free: "⋀x R v. x ∉ set R ⟹ rg x v ≤ free R"
    and mix: "⋀p l0 e r s s' W.
        s ∈ gamma_lift γ_V (map_lift (rd is_global) (cmb l0 (full_view rg (pglobs p) e)))
        ⟹ s' ∈ gamma_lift γ_V (map_lift (rd is_global) r)
        ⟹ (∀x. is_global x ⟶ x ∉ set W ⟶ s' x = s x)
        ⟹ s' ∈ gamma_lift γ_V (map_lift (rd is_global)
              (cmb (rl r) (full_view rg (pglobs p) (λx. if x ∈ set W then rg x r else e x))))"
    and init_view: "⋀p. gamma_lift γ_V (map_lift (rd is_global) (Lifted init_st))
        ⊆ gamma_lift γ_V (map_lift (rd is_global)
            (cmb (Lifted init_st) (full_view rg (pglobs p) (λx. rg x (Lifted init_st)))))"
    and empty_rd: "⋀p s. emp p s ⟷ empty_V (rd is_global s)"
    and empty_V_sound: "sound_emptiness empty_V γ_V"
    and seed_ne: "⋀v ctx. seed v ctx ≠ buffer_key"
    and seed_ne_global: "⋀v ctx n. seed v ctx ≠ global_of n"
    and classify_proved:
      "⋀c d s. classify c d = Check_Proved ⟹ s ∈ γ_V d ⟹ truthy (eval c s)"
    and classify_refuted:
      "⋀c d s. classify c d = Check_Refuted ⟹ s ∈ γ_V d ⟹ ¬ truthy (eval c s)"
    and bot_state_empty: "γ_V bot_state = {}"
    and init_sound: "⋀p. pinit p ⊆ γ_V (rd is_global init_st)"
    and pentry: "⋀p. pentry p = cfg_entry (pcfg p)"
    and fin_intra: "⋀p. finite (intra (pcfg p))"
    and fin_calls: "⋀p. finite (calls (pcfg p))"
    and calls_unique: "⋀p. calls_source_unique (pcfg p)"
  shows "dg_analysis κ pinit comp emp rd init_st buffer_key global_of seed
           solve solve_dom bot_state classify γ_V empty_V solve_c
           (λp c. keyed_split_spec is_global cmb rl rg free c)
           (λp d e. cmb d (full_view rg (pglobs p) e)) (λG. rl) (λG d. Bot)
           (λG p. map (λx. (x, rg x (Lifted init_st))) (pglobs p))
           (λp ci d e. cmb d (view_of rg (call_global_reads is_global (ci_args ci)) e
              (free (call_global_reads is_global (ci_args ci)))))
           pcfg pentry"
proof -
  let ?gm = "λp d. gamma_lift γ_V (map_lift (rd is_global) d)"
  have gm_mono: "⋀p x y. x ≤ y ⟹ ?gm p x ⊆ ?gm p y"
    using comp_sound unfolding sound_local_spec_def by metis
  show ?thesis
  proof (rule dg_analysis.intro[OF solver dg_analysis_axioms.intro], goal_cases CompSound
      EnterSingle Contract Runs Deps EntrySound CmbBot Init EmptyRd EmptyV SeedNe SeedNeGlobal
      ClProved ClRefuted BotState InitSound PEntry FinIntra FinCalls CallsUnique)
    case (CompSound p) show ?case by (rule comp_sound)
  next
    case (EnterSingle p ci d) show ?case by (rule enter_single)
  next
    case (Contract p) show ?case
      unfolding dg_pipeline.analysis_spec_def
      using comp_sound[where p = p] cmb_mono rg_mono rg_free mix[where p = p]
      by (rule keyed_split_contract)
  next
    case (Runs p ci d σ) show ?case
      using enter_runs_keyed_enter_transfer[of cmb rl rg free
          "call_global_reads is_global (ci_args ci)"
          "global_names_in is_global (ci_formals ci)"
          "λd. ls_enter (comp is_global p) (ls_channel (comp is_global p) d)
             ci (d, d)" d global_of σ]
      by (auto simp: dg_pipeline.analysis_spec_def dg_pipeline.entry_alt_def enter_single Let_def)
  next
    case (Deps p ci d σ) show ?case
      using enter_deps_keyed_enter_transfer[of cmb rl rg free
          "call_global_reads is_global (ci_args ci)"
          "global_names_in is_global (ci_formals ci)"
          "λd. ls_enter (comp is_global p) (ls_channel (comp is_global p) d)
             ci (d, d)" d global_of σ]
      by (auto simp: dg_pipeline.analysis_spec_def dg_pipeline.entry_alt_def enter_single Let_def)
  next
    case (EntrySound p ci d σ pub s)
    let ?G = is_global and ?xs = "pglobs p"
    let ?c = "comp ?G p"
    let ?e = "genv global_of σ"
    let ?R = "call_global_reads ?G (ci_args ci)"
    let ?v = "view_of rg ?R ?e (free ?R)"
```

```
  let ?w = "cmb d ?v"
  let ?E = "dg_pipeline.comp_entry comp ?𝒢 p ci ?w"
  let ?W = "global_names_in ?𝒢 (ci_formals ci)"
  let ?ce =
    "call_enter κ (CallEdge (ci_dst ci) (ci_formals ci) (ci_args ci))"
  have alt: "dg_pipeline.entry_alt comp (λ𝒢. rl)
      (λp ci d e. cmb d (view_of rg (call_global_reads is_global (ci_args ci)) e
         (free (call_global_reads is_global (ci_args ci)))))
      ?𝒢 p ci d ?e = (rl ?w, rl ?E)"
    by (simp add: dg_pipeline.entry_alt_def Let_def)
  have R0: "enter_runs (enter♯ (dg_pipeline.analysis_spec comp
       (λp. keyed_split_spec is_global cmb rl rg free) ?𝒢 p) ci)
      (mk_dg_man d global_of) σ [(rl ?w, rl ?E)] (pub_sides global_of rg ?W (?E ⊔ bot))"
    using enter_runs_keyed_enter_transfer[of cmb rl rg free ?R ?W
        "λd. ls_enter ?c (ls_channel ?c d) ci (d, d)" d global_of σ]
    by (simp add: dg_pipeline.analysis_spec_def enter_single)
  have pub: "pub = pub_sides global_of rg ?W (?E ⊔ bot)"
    using enter_runsD_sides[OF EntrySound(1)[unfolded alt], of "λ_. Answer bot"]
      enter_runsD_sides[OF R0, of "λ_. Answer bot"]
    by (simp add: bot_fun_def[symmetric])
  have E_Bot: "?E ⊔ Bot = ?E" by (cases ?E) simp_all
  have le: "rg x ?E ≤ ?e x" if "x ∈ set ?W" for x
  proof -
    have "DG bot (rg x ?E) ≤ pub (Inr (global_of x))"
      using pub_sides_ge[OF that, of rg "?E ⊔ bot" global_of] by (simp add: pub E_Bot)
    then have "rg x ?E ≤ genv global_of pub x" by (simp add: less_eq_dg_state_def)
    also have "... ≤ ?e x" using EntrySound(2) by (simp add: le_fun_def)
    finally show ?thesis .
  qed
  have view_ge: "full_view rg ?xs ?e ≤ ?v"
    unfolding full_view_def
  proof (rule view_of_le)
    fix x assume "x ∈ set ?xs"
    show "rg x (?e x) ≤ ?v"
    proof (cases "x ∈ set ?R")
      case True then show ?thesis by (rule view_of_ge_mem)
    next
      case False
      then show ?thesis using rg_free view_of_ge_acc by (blast intro: order_trans)
    qed
  qed simp
  have s0: "s ∈ ?gm p (cmb d (full_view rg ?xs ?e))" using EntrySound(3) by simp
  have sw: "s ∈ ?gm p ?w"
    using s0 gm_mono[OF cmb_mono[OF order_refl view_ge]] by blast
  have sE: "?ce s ∈ ?gm p ?E"
  proof -
    obtain q where "q ∈ set (ls_enter ?c (ls_channel ?c ?w) ci (?w, ?w))" "?ce s ∈ ?gm p (snd q)"
      using comp_sound[where p = p] sw ls_channel_sound[OF comp_sound[where p = p] sw]
      unfolding sound_local_spec_def by (metis fst_conv)
    then show ?thesis by (simp add: enter_single)
  qed
  have fr: "∀x. ?𝒢 x ⟶ x ∉ set ?W ⟶ ?ce s x = s x"
    by (auto intro: call_enter_frame)
  have "?ce s ∈ ?gm p (cmb (rl ?E) (full_view rg ?xs (λx. if x ∈ set ?W then rg x ?E else ?e x)))"
    by (rule mix[OF s0 sE fr])
  moreover have "cmb (rl ?E) (full_view rg ?xs (λx. if x ∈ set ?W then rg x ?E else ?e x))
      ≤ cmb (rl ?E) (full_view rg ?xs ?e)"
  proof (intro cmb_mono order_refl full_view_mono[OF rg_mono])
    fix x
    show "(if x ∈ set ?W then rg x ?E else ?e x) ≤ ?e x"
      using le[of x] by (cases "x ∈ set ?W") simp_all
  qed
  ultimately have "?ce s ∈ ?gm p (cmb (rl ?E) (full_view rg ?xs ?e))"
    using gm_mono by blast
  moreover have "s ∈ ?gm p (cmb (rl ?w) (full_view rg ?xs ?e))"
    using s0 by (simp add: rl_cmb cmb_rl)
  ultimately show ?case by (simp add: alt)
next
  case (CmbBot p g) show ?case by (simp add: cmb_bot)
next
  case (Init p)
  let ?xs = "pglobs p"
  let ?ie = "dg_pipeline.init_env init_st buffer_key global_of (λ𝒢 d. Bot)
    (λ𝒢 p. map (λx. (x, rg x (Lifted init_st))) (pglobs p)) is_global p"
  have ie: "rg x (Lifted init_st) ≤ ?ie x" if x: "x ∈ set ?xs" for x
  proof -
```

```
      have "rg x (Lifted init_st)
          ≤ (⨆(m, d)←filter (λ(m, d). m = x) (map (λy. (y, rg y (Lifted init_st))) ys). d)"
        if "x ∈ set ys" for ys
        using that by (induction ys) (auto intro: le_supI1 le_supI2)
      then show ?thesis using x by (simp add: dg_pipeline.init_env_def le_supI2)
    qed
    have "full_view rg ?xs (λx. rg x (Lifted init_st)) ≤ full_view rg ?xs ?ie"
      unfolding full_view_def
    proof (rule view_of_le)
      fix x assume x: "x ∈ set ?xs"
      show "rg x (rg x (Lifted init_st)) ≤ view_of rg ?xs ?ie bot"
        using rg_mono[OF ie[OF x], of x] view_of_ge_mem[OF x, of rg ?ie bot] by (rule order_trans)
    qed simp
    then show ?case
      using init_view[where p = p] gm_mono[OF cmb_mono[OF order_refl]] by blast
  next
    case (EmptyRd p s) show ?case by (rule empty_rd)
  next
    case EmptyV show ?case by (rule empty_V_sound)
  next
    case (SeedNe v ctx) show ?case by (rule seed_ne)
  next
    case (SeedNeGlobal v ctx n) show ?case by (rule seed_ne_global)
  next
    case (ClProved c d s) then show ?case by (rule classify_proved)
  next
    case (ClRefuted c d s) then show ?case by (rule classify_refuted)
  next
    case BotState show ?case by (rule bot_state_empty)
  next
    case (InitSound p) show ?case by (rule init_sound)
  next
    case (PEntry p) show ?case by (rule pentry)
  next
    case (FinIntra p) show ?case by (rule fin_intra)
  next
    case (FinCalls p) show ?case by (rule fin_calls)
  next
    case (CallsUnique p) show ?case by (rule calls_unique)
  qed
qed

end
```

9.5 DG_Live_Unknowns

```
theory DG_Live_Unknowns
  imports DG_Analysis "Voblint_Compile.Live_Nodes"
begin
```

Which solved keys a terminating solve is closed on

The key set a terminating solve returns is not one the soundness argument can use as it stands: it may hold keys the solver queried under an earlier value and no longer reads, and keys of procedures whose result it never solved. `live_unknowns` keeps a solved key when its node belongs to a procedure (`Voblint_Compile.Live_Nodes`) whose result is solved at the same context. From what each generated equation reads under the final valuation, `live_unknowns_cover` shows that set closed under every intra edge, every call continuation and every call that enters a live state, with no premise beyond termination and a well-formed program. The endpoints below restate the routed soundness over these keys and carry it to the published table.

What a routed equation reads

```
lemma dep_aux_routed_node_rhs:
  assumes wf: "⋀w. ∀q ∈ set (routed_contribution_programs pred_sel site_sel route
                    it cmb extra g cx w). sp_wf q"
  shows "dep_aux τ (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀
          (v, cx))
     = (⋃q∈set (routed_contribution_programs pred_sel site_sel route it cmb extra g cx v).
          dep_program τ q)"
  by (cases "v = cfg_entry g")
     (simp_all add: routed_node_rhs_def Let_def dep_program_side_rhs_fold_dg_char[OF wf])

lemma dep_L_routed_node_rhs_intra:
  assumes fin: "finite (intra g)" and e: "(u, a, v) ∈ intra g"
    and wf: "⋀w. ∀q ∈ set (routed_contribution_programs
                    intra_predecessor_addr_list call_site_list route
                    (λcx src a. dg_spec_edge_program S a src slot) cmb extra g cx w). sp_wf q"
  shows "(u, cx) ∈ dep_L (routed_node_rhs intra_predecessor_addr_list call_site_list buffer_key_at route
           (λcx src a. dg_spec_edge_program S a src slot) cmb extra g bot0 d₀ g₀) τ (v, cx)"
proof -
  have "dg_spec_edge_program S a (Inl (u, cx)) slot
         ∈ set (routed_contribution_programs intra_predecessor_addr_list call_site_list route
                  (λcx src a. dg_spec_edge_program S a src slot) cmb extra g cx v)"
    using e fin
    by (force simp: routed_contribution_programs_def intra_predecessor_addr_list_def
        intra_predecessors_def)
  then show ?thesis
    unfolding dep_L_def dep_def
    using dep_dg_spec_edge_program_source[of "Inl (u, cx)" τ S a slot]
    by (auto simp: dep_aux_routed_node_rhs[OF wf])
qed

lemma dep_program_routed_call_program:
  assumes wfS: "dg_spec_wf S"
  shows "dep_program τ (routed_call_program S global_of seed resolve is_bot route ctx ca cc v)
     = insert (Inl (cc, ctx))
         (⋃q∈set (resolve v cc ca (dg_local (τ (Inl (cc, ctx))))).
            dep_program τ (routed_callee_call_program S global_of seed route is_bot ctx ca cc
                     (dg_local (τ (Inl (cc, ctx)))) q))"
  by (simp add: routed_call_program_def sp_compile_bind sp_wf_observes
      dep_program_side_rhs_fold_dg_char[OF sp_wf_routed_callee_call_programs[OF wfS]])

lemma routed_call_program_mem:
  assumes fin: "finite (calls g)" and e: "(u, ca, FunctionEntry q, k) ∈ calls g"
  shows "routed_call_program S global_of seed resolve is_bot route cx ca u k
          ∈ set (routed_contribution_programs pred_sel call_site_list route it
                   (routed_call_program S global_of seed resolve is_bot) extra g cx k)"
proof -
  have "(u, ca) ∈ set (call_site_list g k)" using e fin by auto
  then show ?thesis by (rule routed_contribution_programs_combineI)
qed

lemma dep_L_routed_node_rhs_call_site:
  assumes fin: "finite (calls g)" and e: "(u, ca, FunctionEntry q, k) ∈ calls g"
    and wfS: "dg_spec_wf S"
    and wf: "⋀w. ∀q ∈ set (routed_contribution_programs pred_sel call_site_list route it
                    (routed_call_program S global_of seed resolve is_bot) extra g cx w). sp_wf q"
  shows "(u, cx) ∈ dep_L (routed_node_rhs pred_sel call_site_list buffer_key_at route it
```

```
          (routed_call_program S global_of seed resolve is_bot) extra g bot0 d₀ g₀) τ (k, cx)"
proof -
  have "Inl (u, cx)
        ∈ dep_program τ (routed_call_program S global_of seed resolve is_bot route cx ca u k)"
    by (simp add: dep_program_routed_call_program[OF wfS])
  then show ?thesis
    unfolding dep_L_def dep_def
    using routed_call_program_mem
          [OF fin e, of S global_of seed resolve is_bot route cx pred_sel it extra]
    by (auto simp: dep_aux_routed_node_rhs[OF wf])
qed

lemma dep_L_routed_node_rhs_callee_result:
  assumes fin: "finite (calls g)" and e: "(u, ca, FunctionEntry q, k) ∈ calls g"
    and deps: "enter_deps (enter♯ S (call_info_of ca q))
                (mk_dg_man (dg_local (τ (Inl (u, cx)))) global_of) τ pairs D"
    and mem: "(c, ent) ∈ set pairs"
    and res: "q ∈ set (resolve k u ca (dg_local (τ (Inl (u, cx)))))"
    and nb: "¬ is_bot ent"
    and wfS: "dg_spec_wf S"
    and wf: "⋀w. ∀q ∈ set (routed_contribution_programs pred_sel call_site_list route it
                    (routed_call_program S global_of seed resolve is_bot) extra g cx w). sp_wf q"
  shows "(FunctionResult q, route u cx ent ca)
         ∈ dep_L (routed_node_rhs pred_sel call_site_list buffer_key_at route it
              (routed_call_program S global_of seed resolve is_bot) extra g bot0 d₀ g₀) τ (k, cx)"
proof -
  let ?d = "dg_local (τ (Inl (u, cx)))"
  have callee: "dep_program τ (routed_callee_call_program S global_of seed route is_bot cx ca u ?d q)
      = D ∪ dep_program τ (side_rhs_fold_dg bot
          (map (routed_call_alternative_program S global_of seed route is_bot cx ca u q) pairs))"
    unfolding routed_callee_call_program_def sp_compile_bind
    by (rule enter_depsD[OF deps, simplified])
  have "Inl (FunctionResult q, route u cx ent ca)
        ∈ dep_program τ (routed_callee_call_program S global_of seed route is_bot cx ca u ?d q)"
  proof -
    have "Inl (FunctionResult q, route u cx ent ca)
          ∈ dep_program τ (routed_call_alternative_program S global_of seed route is_bot cx ca u q
                       (c, ent))"
      using nb by (simp add: routed_call_alternative_program_def Let_def)
    then show ?thesis
      unfolding callee
        dep_program_side_rhs_fold_dg_char[OF sp_wf_routed_call_alternative_programs[OF wfS]]
      using mem by auto
  qed
  then have "Inl (FunctionResult q, route u cx ent ca)
             ∈ dep_program τ (routed_call_program S global_of seed resolve is_bot route cx ca u k)"
    using res by (auto simp: dep_program_routed_call_program[OF wfS])
  then show ?thesis
    unfolding dep_L_def dep_def
    using routed_call_program_mem
          [OF fin e, of S global_of seed resolve is_bot route cx pred_sel it extra]
    by (auto simp: dep_aux_routed_node_rhs[OF wf])
qed
```

Keys of a solve whose activation returned

The solved key set is closed under the equations' reads (`sol_vars_dep_closed`). Walking those reads backwards, `live_unknowns_cover` shows `live_unknowns` closed under intra edges, call continuations and the entries of callees a call actually enters.

```
context dg_analysis
begin

context
  fixes ptab :: "'p ⇒ ('ann, var_id, exp) gptable" and pprocs :: "'p ⇒ pname list"
    and vars :: "exp ⇒ var_id set"
  assumes pcfg_compile: "⋀p. pcfg p = compile_prog (ptab p) (pprocs p)"
begin

lemma analysis_contribs_wf [simp]:
  "∀q ∈ set (routed_contribution_programs intra_predecessor_addr_list call_site_list
       (route is_global)
       (λctx' src a. dg_spec_edge_program (analysis_spec is_global p) a src
          global_of)
       (routed_call_program (analysis_spec is_global p) global_of seed rsv
          (λd. d = Bot))
```

```
      (routed_entry_seed_programs seed) cfg cx w). sp_wf q"
  by (rule routed_contribution_programs_wf) auto

lemma sol_vars_dep_closed:
  assumes solves: "terminates is_global p"
    and x: "x ∈ sol_vars is_global p"
    and y: "y ∈ dep_L
       (routed_node_rhs intra_predecessor_addr_list call_site_list (λ_. buffer_key)
          (route is_global)
          (λctx' src a. dg_spec_edge_program (analysis_spec is_global p) a src global_of)
          (routed_call_program (analysis_spec is_global p) global_of seed
             (static_resolve (pcfg p)) (λd. d = Bot))
          (routed_entry_seed_programs seed)
          (pcfg p) Bot (Lifted init_st) (place_rg is_global (Lifted init_st)))
       (sol_env is_global p) x"
  shows "y ∈ sol_vars is_global p"
  using pp_routed[OF solves] x y by blast

lemma prog_cfg_finite: "finite (intra (pcfg p))" "finite (calls (pcfg p))"
  unfolding pcfg_compile by (simp_all add: compile_prog_finite)

lemma sol_vars_back_intra:
  assumes solves: "terminates is_global p"
    and v: "(v, cx) ∈ sol_vars is_global p" and e: "(u, a, v) ∈ intra (pcfg p)"
  shows "(u, cx) ∈ sol_vars is_global p"
  by (rule sol_vars_dep_closed
        [OF solves v dep_L_routed_node_rhs_intra[OF prog_cfg_finite(1) e analysis_contribs_wf]])

lemma sol_vars_back_call_site:
  assumes solves: "terminates is_global p"
    and k: "(k, cx) ∈ sol_vars is_global p"
    and e: "(u, ca, FunctionEntry q, k) ∈ calls (pcfg p)"
  shows "(u, cx) ∈ sol_vars is_global p"
  by (rule sol_vars_dep_closed
        [OF solves k dep_L_routed_node_rhs_call_site
           [OF prog_cfg_finite(2) e dg_spec_wf_analysis_spec analysis_contribs_wf]])

lemma sol_vars_callee_result:
  assumes solves: "terminates is_global p"
    and k: "(k, cx) ∈ sol_vars is_global p"
    and e: "(u, ca, FunctionEntry q, k) ∈ calls (pcfg p)"
    and nb: "entry_of is_global p (call_info_of ca q)
               (dg_local (sol_env is_global p (Inl (u, cx))))
               (sol_global is_global p) ≠ Bot"
  shows "(FunctionResult q, ctx_succ is_global p u cx ca q)
         ∈ sol_vars is_global p"
proof -
  let ?d = "dg_local (sol_env is_global p (Inl (u, cx)))"
  let ?alt =
    "entry_alt is_global p (call_info_of ca q) ?d (sol_global is_global p)"
  have res: "q ∈ set (static_resolve (pcfg p) k u ca ?d)"
    using e prog_cfg_finite(2) by simp
  obtain D where deps: "enter_deps (enter♯ (analysis_spec is_global p) (call_info_of ca q))
      (mk_dg_man ?d global_of) (sol_env is_global p) [?alt] D"
    using place_enter_deps unfolding sol_global_def by blast
  have mem: "(fst ?alt, snd ?alt) ∈ set [?alt]" by simp
  show ?thesis
    unfolding ctx_succ_def entry_of_def
    by (rule sol_vars_dep_closed[OF solves k],
        rule dep_L_routed_node_rhs_callee_result[where resolve = "static_resolve (pcfg p)"
            and τ = "sol_env is_global p" and cx = cx,
          OF prog_cfg_finite(2) e deps mem res])
       (use nb in ‹simp add: entry_of_def›, rule dg_spec_wf_analysis_spec, rule
        analysis_contribs_wf)
qed

lemma sol_vars_back_reach:
  assumes solves: "terminates is_global p"
    and w: "(w, cx) ∈ sol_vars is_global p"
    and r: "local_reaches (pcfg p) y w"
  shows "(y, cx) ∈ sol_vars is_global p"
  using r[unfolded local_reaches_def]
proof (induction rule: converse_rtrancl_induct)
  case base show ?case by (rule w)
next
  case (step y z)
```

```
  from step.hyps(1) show ?case
    unfolding local_succ_rel_def
    using sol_vars_back_intra[OF solves step.IH] sol_vars_back_call_site[OF solves step.IH]
    by blast
qed
definition live_unknowns :: "'p ⇒ (pp × 'c) set" where
  "live_unknowns p =
     {(u, cx) ∈ sol_vars is_global p.
        ∃q. prog_node (ptab p) (pprocs p) q u
          ∧ (FunctionResult q, cx) ∈ sol_vars is_global p}"

lemma live_unknowns_sub: "live_unknowns p ⊆ sol_vars is_global p"
  unfolding live_unknowns_def by blast

lemma live_unknowns_of_result:
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
    and live: "prog_node (ptab p) (pprocs p) q y"
    and r: "(FunctionResult q, cx) ∈ sol_vars is_global p"
  shows "(y, cx) ∈ live_unknowns p"
proof -
  have "local_reaches (pcfg p) y (FunctionResult q)"
    unfolding pcfg_compile by (rule prog_node_reaches[OF live])
  then have "(y, cx) ∈ sol_vars is_global p" by (rule sol_vars_back_reach[OF solves r])
  then show ?thesis unfolding live_unknowns_def using live r by blast
qed

theorem live_unknowns_cover:
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
  shows "ctx_vars_cover_live (pcfg p) (λ_ _. True) (live_succ is_global p)
         root_ctx (live_unknowns p)"
proof (rule ctx_vars_cover_liveI)
  have root: "(FunctionResult prog_main_name, root_ctx) ∈ sol_vars is_global p"
    using pp_routed[OF solves] by (simp add: root_query_def pcfg_compile)
  show "(cfg_entry (pcfg p), root_ctx) ∈ live_unknowns p"
    using live_unknowns_of_result[OF wf solves prog_node_main_entry[OF wf] root]
    by (simp add: pcfg_compile)
next
  fix u a v ctx
  assume u: "(u, ctx) ∈ live_unknowns p" and e: "(u, a, v) ∈ intra (pcfg p)"
  from u obtain q where lq: "prog_node (ptab p) (pprocs p) q u"
    and r: "(FunctionResult q, ctx) ∈ sol_vars is_global p"
    unfolding live_unknowns_def by blast
  have "prog_node (ptab p) (pprocs p) q v"
    using prog_node_intra[OF wf lq] e by (simp add: pcfg_compile)
  then show "(v, ctx) ∈ live_unknowns p" by (rule live_unknowns_of_result[OF wf solves _ r])
next
  fix u ctx dst fs as q k
  assume u: "(u, ctx) ∈ live_unknowns p"
    and e: "(u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls (pcfg p)"
  from u obtain r where lr: "prog_node (ptab p) (pprocs p) r u"
    and res: "(FunctionResult r, ctx) ∈ sol_vars is_global p"
    unfolding live_unknowns_def by blast
  have "prog_node (ptab p) (pprocs p) r k"
    using prog_node_calls[OF wf lr] e by (simp add: pcfg_compile)
  then show "(k, ctx) ∈ live_unknowns p" by (rule live_unknowns_of_result[OF wf solves _ res])
next
  fix u ctx dst fs as q k ctx'
  assume u: "(u, ctx) ∈ live_unknowns p"
    and e: "(u, CallEdge dst fs as, FunctionEntry q, k) ∈ calls (pcfg p)"
    and s: "live_succ is_global p u ctx (CallEdge dst fs as) q = Some ctx'"
  from u obtain r where lr: "prog_node (ptab p) (pprocs p) r u"
    and res: "(FunctionResult r, ctx) ∈ sol_vars is_global p"
    unfolding live_unknowns_def by blast
  have "prog_node (ptab p) (pprocs p) r k"
    using prog_node_calls[OF wf lr] e by (simp add: pcfg_compile)
  then have k: "(k, ctx) ∈ sol_vars is_global p"
    using live_unknowns_of_result[OF wf solves _ res] live_unknowns_sub by blast
  from s have nb: "entry_of is_global p (call_info_of (CallEdge dst fs as) q)
              (dg_local (sol_env is_global p (Inl (u, ctx))))
              (sol_global is_global p) ≠ Bot"
    and c': "ctx' = ctx_succ is_global p u ctx (CallEdge dst fs as) q"
    by (auto simp: live_succ_def split: if_splits)
  have rq: "(FunctionResult q, ctx') ∈ sol_vars is_global p"
    unfolding c' by (rule sol_vars_callee_result[OF solves k e nb])
```

```
  have "prog_node (ptab p) (pprocs p) q (FunctionEntry q)"
    using prog_node_callee_entry[OF wf] e by (simp add: pcfg_compile)
  then show "(FunctionEntry q, ctx') ∈ live_unknowns p"
    by (rule live_unknowns_of_result[OF wf solves _ rq])
qed

lemma root_live_key:
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
  shows "root_query p ∈ live_unknowns p"
proof -
  have r: "(FunctionResult prog_main_name, root_ctx) ∈ sol_vars is_global p"
    using pp_routed[OF solves] by (simp add: root_query_def pcfg_compile)
  show ?thesis
    using live_unknowns_of_result[OF wf solves prog_node_result[OF prog_node_main_entry[OF wf]] r]
    by (simp add: root_query_def pcfg_compile)
qed
```

The entry-state endpoint over the live keys

The routed soundness statement restated over `live_unknowns`, whose coverage needs no premise beyond termination, and carried to the published table and the source-level endpoints under the functional context policy.

```
context
  fixes p :: 'p
begin

interpretation dg_base: analysis_contract κ "analysis_spec is_global p"
    "λd e. pgam p d e"
  by (rule place_contract)
```

Establishes `routed_analysis` over the live keys, for any relation admitting call contexts. Beyond termination and well-formedness a policy owes only its own two facts: an admitted context is the one this pipeline routes the entered state to, and every concrete call at a live key is admitted somewhere. `live_unknowns_cover` supplies callee-entry membership.

```
lemma routed_analysis_from_live_unknowns:
  fixes adm :: "'c call_context_policy"
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
    and cover_R: "⋀u ctx dst pars args q cont s ctx'. (u, ctx) ∈ live_unknowns p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) q) s
              (call_enter κ (CallEdge dst pars args) s)
        ⟹ route is_global u ctx
              (entered p (call_info_of (CallEdge dst pars args) q)
                 (dg_local (sol_env is_global p (Inl (u, ctx)))))
              (CallEdge dst pars args) = ctx'"
    and total_R: "⋀u ctx dst pars args q cont s. (u, ctx) ∈ live_unknowns p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ s ∈ pgam p (dg_local (sol_env is_global p (Inl (u, ctx)))) (gsol p)
        ⟹ adm u ctx (call_info_of (CallEdge dst pars args) q) s
                    (call_enter κ (CallEdge dst pars args) s) ≠ {}"
  shows "routed_analysis κ (analysis_spec is_global p) (λd e. pgam p d e) (pcfg p) buffer_key global_of
     (route is_global) Bot (Lifted init_st)
     (place_rg is_global (Lifted init_st))
     (sol_env is_global p) (live_unknowns p) (root_query p) seed (λd. d = Bot) adm
     (λd e. map_lift (rd is_global) (place_cmb p d e))
     γ_V empty_V classify"
proof (unfold_locales, goal_cases CmbWf ExtraWf FinE PP SgCov SgUncov Fwd FinC CallsUnique
    SeedUnknown SeedNeGlobal IsBotBot IsBotSound ResolveSound EnterCover EnterTotal CombFwd GammaRd
    EmptyExact ClProved ClRefuted VarsFin)
  case CmbWf show ?case by (rule sp_wf_routed_call_program[OF dg_spec_wf_analysis_spec])
next
  case ExtraWf then show ?case by (rule sp_wf_routed_entry_seed_programs)
next
  case FinE show ?case unfolding pcfg_compile by (simp add: compile_prog_finite)
next
  case PP show ?case
    by (rule post_bounded_subset[OF post_bounded_of_part_post_solution[OF pp_routed[OF solves]]
          root_live_key[OF wf solves] live_unknowns_sub])
next
  case (SgCov v c) then show ?case by simp
next
  case (SgUncov v c) then show ?case by simp
next
```

```
  case (Fwd u a v c)
  show ?case
    by (rule ctx_vars_cover_live_edgeD[OF live_unknowns_cover[OF wf solves] Fwd(1) _ Fwd(3)]) simp
next
  case FinC show ?case unfolding pcfg_compile by (simp add: compile_prog_finite)
next
  case CallsUnique show ?case
    unfolding calls_source_unique_def pcfg_compile
    using compile_prog_calls_source_unique by blast
next
  case (SeedUnknown q ctx) show ?case by (rule seed_ne_buffer_key)
next
  case (SeedNeGlobal q ctx v) show ?case by (rule seed_ne_global_of)
next  case IsBotBot show ?case by simp
next
  case (IsBotSound d g') then show ?case by simp
next
  case (ResolveSound u ctx dst pars args q cont s)
  then show ?case unfolding pcfg_compile by (simp add: compile_prog_finite)
next
  case (EnterCover u ctx dst pars args q cont s ctx')
  let ?ci = "call_info_of (CallEdge dst pars args) q"
  let ?caller = "dg_local (sol_env is_global p (Inl (u, ctx)))"
  let ?g = "genv global_of (sol_env is_global p)"
  let ?ent = "entered p ?ci ?caller"
  have sin: "s ∈ pgam p ?caller (gsol p)"
    using EnterCover(3) by (simp add: sol_global_def)
  have cov: "entry_pairs_cover (λd. pgam p d (gsol p)) s
     (call_enter κ (CallEdge dst pars args) s)
     [entry_alt is_global p ?ci ?caller (gsol p)]"
  proof (rule entry_cover[OF solves _ EnterCover(2) sin])
    show "(cont, ctx) ∈ sol_vars is_global p"
      using ctx_vars_cover_live_combineD[OF live_unknowns_cover[OF wf solves] EnterCover(1,2)]
        live_unknowns_sub by blast
  qed
  obtain c e where ce: "entry_alt is_global p ?ci ?caller (gsol p) = (c, e)"
    by (cases "entry_alt is_global p ?ci ?caller (gsol p)")
  have nbE: "?ent ≠ Bot"
  proof
    assume "?ent = Bot"
    with cov ce show False by (simp add: entry_pairs_cover_def entry_of_def place_cmb_bot)
  qed
  have req: "route is_global u ctx ?ent (CallEdge dst pars args) = ctx'"
    by (rule cover_R[OF EnterCover(1,2,4)])
  have covE: "(FunctionEntry q, ctx') ∈ live_unknowns p"
    by (rule ctx_vars_cover_live_enterD[OF live_unknowns_cover[OF wf solves] EnterCover(1,2)])
      (use nbE req in ‹simp add: live_succ_def ctx_succ_def›)
  obtain pub where runs: "enter_runs (enter♯ (analysis_spec is_global p) ?ci)
     (mk_dg_man ?caller global_of) (sol_env is_global p)
     [entry_alt is_global p ?ci ?caller ?g] pub"
    using place_enter_runs by blast
  obtain deps where deps: "enter_deps (enter♯ (analysis_spec is_global p) ?ci)
     (mk_dg_man ?caller global_of) (sol_env is_global p)
     [entry_alt is_global p ?ci ?caller ?g] deps"
    using place_enter_deps by blast
  have ce': "entry_alt is_global p ?ci ?caller ?g = (c, e)"
    using ce by (simp add: sol_global_def)
  have sc: "s ∈ pgam p c ?g"
    and ec:
      "call_enter κ (CallEdge dst pars args) s ∈ pgam p e ?g"
    using cov ce by (simp_all add: entry_pairs_cover_def sol_global_def)
  have re: "route is_global u ctx e (CallEdge dst pars args) = ctx'"
    using req ce by (simp add: entry_of_def)
  show ?case
    using runs deps sc ec re covE unfolding ce' by fastforce
next
  case (EnterTotal u ctx dst pars args q cont s)
  then show ?case by (intro total_R) (simp_all add: sol_global_def)
next
  case (CombFwd cl c1 dst pars args q cont)
  show ?case
    by (rule ctx_vars_cover_live_combineD[OF live_unknowns_cover[OF wf solves] CombFwd(1,2)])
next
  case (GammaRd d g') show ?case by simp
next
  case (EmptyExact v) then show ?case by (rule empty_V_soundD)
```

```
next
  case (ClProved c d s) then show ?case by (rule classify_proved)
next
  case (ClRefuted c d s) then show ?case by (rule classify_refuted)
next
  case VarsFin show ?case
    by (rule finite_subset[OF live_unknowns_sub vars_finite_of_terminates[OF solves]])
qed
```

The reader over the live keys agrees with the published one on them and describes nothing elsewhere, so a bound proved for it is a bound on the published table.

```
lemma gamma_live_reader_le:
  "pgam p (solved_local_reader (live_unknowns p) (sol_env is_global p) (Inl (v, ctx)))
      (gsol p)
    ⊆ pgam p (reader is_global p (Inl (v, ctx))) (gsol p)"
proof (cases "(v, ctx) ∈ live_unknowns p")
  case True
  then have "(v, ctx) ∈ sol_vars is_global p" by (rule subsetD[OF live_unknowns_sub])
  with True show ?thesis by (simp add: reader_def)
next
  case False then show ?thesis by (simp add: place_cmb_bot)
qed

theorem activation_collect_sound_live_unknowns:
  fixes adm :: "'c call_context_policy"
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
    and cover_R: "⋀u ctx dst pars args q cont s ctx'. (u, ctx) ∈ live_unknowns p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ ctx' ∈ adm u ctx (call_info_of (CallEdge dst pars args) q) s
              (call_enter κ (CallEdge dst pars args) s)
        ⟹ route is_global u ctx
              (entered p (call_info_of (CallEdge dst pars args) q)
                 (dg_local (sol_env is_global p (Inl (u, ctx)))))
              (CallEdge dst pars args) = ctx'"
    and total_R: "⋀u ctx dst pars args q cont s. (u, ctx) ∈ live_unknowns p
        ⟹ (u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)
        ⟹ s ∈ pgam p (dg_local (sol_env is_global p (Inl (u, ctx)))) (gsol p)
        ⟹ adm u ctx (call_info_of (CallEdge dst pars args) q) s
                     (call_enter κ (CallEdge dst pars args) s) ≠ {}"
  shows "𝒜κ,adm,root_ctx,pcfg p,pinit p v ctx
           ⊆ pgam p (reader is_global p (Inl (v, ctx))) (gsol p)"
proof -
  interpret live: routed_analysis κ "analysis_spec is_global p"
    "λd e. pgam p d e" "pcfg p" buffer_key global_of
      "route is_global" Bot
        "Lifted init_st" "place_rg is_global (Lifted init_st)"
      "sol_env is_global p" "live_unknowns p" "root_query p" seed "λd. d = Bot" adm
      "λd e. map_lift (rd is_global) (place_cmb p d e)"
      γV emptyV classify
    by (rule routed_analysis_from_live_unknowns[where adm = adm, OF wf solves cover_R total_R])
  have entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ live_unknowns p"
    by (rule ctx_vars_cover_live_entryD[OF live_unknowns_cover[OF wf solves]])
  have init_env_le:
    "init_env is_global p ≤ genv global_of (sol_env is_global p)"
    using init_env_le_sol[OF solves subsetD[OF live_unknowns_sub entry_cov]
      live.pp_entry_init_global_bound[OF entry_cov]]
    by (simp add: sol_global_def)
  have "𝒜κ,adm,root_ctx,pcfg p,pinit p v ctx
        ⊆ pgam p (solved_local_reader (live_unknowns p) (sol_env is_global p) (Inl (v, ctx)))
            (gsol p)"
    using live.activation_collect_dg_sound
        [OF entry_cov _ init_env_le, of "pinit p" v ctx] cinit_le_init
    by (simp add: sol_global_def)
  then show ?thesis using gamma_live_reader_le by blast
qed
```

A policy whose route ignores the entered state — a call string, or the unit context — admits exactly the context it computes, so both of its facts are immediate.

```
theorem fun_route_activation_collect_sound_of_terminates:
  fixes ctx_fun :: "cfg_node ⇒ 'c ⇒ store ⇒ 'c"
  assumes route_const: "⋀u ctx d ca s. route is_global u ctx d ca = ctx_fun u ctx s"
    and wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
      "terminates is_global p"
  shows "𝒜κ,context_policy_of_fun ctx_fun,root_ctx,pcfg p,pinit p v ctx
```

```
          ⊆ pgam p (reader is_global p (Inl (v, ctx))) (gsol p)"
proof (rule activation_collect_sound_live_unknowns[OF wf solves])
  fix u ctx dst pars args q cont and s :: "var_id ⇒ int" and ctx'
  assume "(u, ctx) ∈ live_unknowns p"
    and "(u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)"
    and Rc: "ctx' ∈ context_policy_of_fun ctx_fun u ctx (call_info_of (CallEdge dst pars args) q) s
               (call_enter κ (CallEdge dst pars args) s)"
  show "route is_global u ctx
          (entered p (call_info_of (CallEdge dst pars args) q)
             (dg_local (sol_env is_global p (Inl (u, ctx)))))
          (CallEdge dst pars args) = ctx'"
    using Rc
    by (simp add:
      route_const[of _ _ _ _
        "call_enter κ (CallEdge dst pars args) s"])
next
  fix u ctx dst pars args q cont and s :: "var_id ⇒ int"
  show "context_policy_of_fun ctx_fun u ctx (call_info_of (CallEdge dst pars args) q) s
                (call_enter κ (CallEdge dst pars args) s) ≠ {}"
    by simp
qed
```

The same fact read off the published surface, and what it yields at a program point, at a check and at a source run. A store reaching a point lies in one of the point's contexts, so the point is bounded by the union of the states published there. At the unit policy the union has the one member `()`.

```
theorem fun_route_state_at_sound:
  fixes ctx_fun :: "cfg_node ⇒ 'c ⇒ store ⇒ 'c"
  assumes route_const: "⋀u ctx d ca s. route is_global u ctx d ca = ctx_fun u ctx s"
    and wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
      "terminates is_global p"
  shows "𝒜κ,context_policy_of_fun ctx_fun,root_ctx,pcfg p,pinit p v ctx
           ⊆ γV (state_at is_global p ctx v)"
proof -
  have "𝒜κ,context_policy_of_fun ctx_fun,root_ctx,pcfg p,pinit p v ctx
        ⊆ gamma_lift γV (lookup_table (result is_global p) v ctx)"
    using fun_route_activation_collect_sound_of_terminates[OF route_const wf solves]
    unfolding gamma_reader_eq_lookup .
  then show ?thesis
    by (auto simp: state_at_unfold bot_state_empty split: lifted.splits)
qed

theorem fun_route_result_node_sound:
  fixes ctx_fun :: "cfg_node ⇒ 'c ⇒ store ⇒ 'c"
  assumes route_const: "⋀u ctx d ca s. route is_global u ctx d ca = ctx_fun u ctx s"
    and wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
      "terminates is_global p"
  shows "𝒞κ,pcfg p,pinit p v
           ⊆ (⋃ctx. γV (state_at is_global p ctx v))"
  using fun_route_node_collect_eq_Union[where ctx_fun = ctx_fun and p = p and v = v]
    fun_route_state_at_sound[OF route_const wf solves]
  by (simp add: SUP_mono')

theorem fun_route_report_proved_sound:
  fixes ctx_fun :: "cfg_node ⇒ 'c ⇒ store ⇒ 'c"
  assumes route_const: "⋀u ctx d ca s. route is_global u ctx d ca = ctx_fun u ctx s"
    and wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
      "terminates is_global p"
    and mem: "(v, c, Check_Proved) ∈ set (report is_global p ctx)"
  shows
    "∀s ∈ 𝒜κ,context_policy_of_fun ctx_fun,root_ctx,pcfg p,pinit p v ctx. truthy (eval c s)"
proof -
  have fin: "finite (intra (pcfg p))"
    unfolding pcfg_compile by (simp add: compile_prog_finite)
  show ?thesis
    by (rule classify_checks_proved_sound
          [where g = "pcfg p" and env = "state_at is_global p ctx"
             and classify = classify and γS = γV
             and reach =
               "λv. 𝒜κ,context_policy_of_fun ctx_fun,root_ctx,pcfg p,pinit p v ctx",
           OF fin _ classify_proved fun_route_state_at_sound[OF route_const wf solves]])
       (use mem in ‹simp add: report_def state_at_def analysis_surface.report_def›)
qed

theorem fun_route_report_refuted_sound:
  fixes ctx_fun :: "cfg_node ⇒ 'c ⇒ store ⇒ 'c"
```

```
  assumes route_const: "⋀u ctx d ca s. route is_global u ctx d ca = ctx_fun u ctx s"
    and wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
      "terminates is_global p"
    and mem: "(v, c, Check_Refuted) ∈ set (report is_global p ctx)"
  shows
    "∀s ∈ 𝒜κ,context_policy_of_fun ctx_fun,root_ctx,pcfg p,pinit p v ctx. ¬ truthy (eval c s)"
proof -
  have fin: "finite (intra (pcfg p))"
    unfolding pcfg_compile by (simp add: compile_prog_finite)
  show ?thesis
    by (rule classify_checks_refuted_sound
          [where g = "pcfg p" and env = "state_at is_global p ctx"
             and classify = classify and γS = γV
             and reach =
               "λv. 𝒜κ,context_policy_of_fun ctx_fun,root_ctx,pcfg p,pinit p v ctx",
           OF fin _ classify_refuted fun_route_state_at_sound[OF route_const wf solves]])
       (use mem in ‹simp add: report_def state_at_def analysis_surface.report_def›)
qed
```

Entry-state routing

Instantiates the live-key endpoints at `admitted_contexts`, the policy that routes a call by its entered state. Its two policy facts hold by construction, so the results need only termination and a well-formed program.

```
lemma entry_state_routed_analysis_from_live_unknowns:
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
  shows "routed_analysis κ (analysis_spec is_global p) (λd e. pgam p d e) (pcfg p) buffer_key global_of
     (route is_global) Bot (Lifted init_st)
     (place_rg is_global (Lifted init_st))
     (sol_env is_global p) (live_unknowns p) (root_query p) seed (λd. d = Bot)
     (admitted_contexts is_global p)
     (λd e. map_lift (rd is_global) (place_cmb p d e))
     γV emptyV classify"
proof (rule routed_analysis_from_live_unknowns[OF wf solves])
  fix u ctx dst pars args q cont and s :: "var_id ⇒ int" and ctx'
  assume "(u, ctx) ∈ live_unknowns p"
    and "(u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)"
    and Rc: "ctx' ∈ admitted_contexts is_global p u ctx
               (call_info_of (CallEdge dst pars args) q) s
               (call_enter κ (CallEdge dst pars args) s)"
  let ?ci = "call_info_of (CallEdge dst pars args) q"
  let ?caller = "dg_local (sol_env is_global p (Inl (u, ctx)))"
  let ?ent = "entered p ?ci ?caller"
  from Rc[unfolded admitted_contexts_alt] obtain cont' entry
    where mem: "(cont', entry) ∈ set [entry_alt is_global p ?ci ?caller (gsol p)]"
      and req0: "ctx' = route is_global u ctx entry
                   (CallEdge (ci_dst ?ci) (ci_formals ?ci) (ci_args ?ci))"
    by (rule routed_entry_context_relE)
  have "entry = ?ent" using mem by (simp add: entry_of_def) (metis snd_conv)
  then show "route is_global u ctx ?ent (CallEdge dst pars args) = ctx'"
    using req0 by simp
next
  fix u ctx dst pars args q cont and s :: "var_id ⇒ int"
  assume uL: "(u, ctx) ∈ live_unknowns p"
    and ce: "(u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls (pcfg p)"
    and sin: "s ∈ pgam p (dg_local (sol_env is_global p (Inl (u, ctx)))) (gsol p)"
  have cont: "(cont, ctx) ∈ sol_vars is_global p"
    using ctx_vars_cover_live_combineD[OF live_unknowns_cover[OF wf solves] uL ce]
      live_unknowns_sub by blast
  show "admitted_contexts is_global p u ctx
              (call_info_of (CallEdge dst pars args) q) s
              (call_enter κ (CallEdge dst pars args) s) ≠ {}"
    unfolding admitted_contexts_alt
    by (rule routed_entry_context_rel_total)
       (use entry_cover[OF solves cont ce sin]
        in ‹simp add: sol_global_def›)
qed

theorem entry_state_activation_collect_sound_of_terminates:
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
  shows "𝒜κ,admitted_contexts is_global p,root_ctx,pcfg p,pinit p v ctx
           ⊆ pgam p (reader is_global p (Inl (v, ctx))) (gsol p)"
proof -
  interpret live: routed_analysis κ "analysis_spec is_global p"
```

```
    "λd e. pgam p d e" "pcfg p" buffer_key global_of
      "route is_global" Bot
        "Lifted init_st" "place_rg is_global (Lifted init_st)"
      "sol_env is_global p" "live_unknowns p" "root_query p" seed "λd. d = Bot"
      "admitted_contexts is_global p"
      "λd e. map_lift (rd is_global) (place_cmb p d e)"
      γV emptyV classify
    by (rule entry_state_routed_analysis_from_live_unknowns[OF wf solves])
  have entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ live_unknowns p"
    by (rule ctx_vars_cover_live_entryD[OF live_unknowns_cover[OF wf solves]])
  have init_env_le:
    "init_env is_global p ≤ genv global_of (sol_env is_global p)"
    using init_env_le_sol[OF solves subsetD[OF live_unknowns_sub entry_cov]
      live.pp_entry_init_global_bound[OF entry_cov]]
    by (simp add: sol_global_def)
  have "Aκ,admitted_contexts is_global p,root_ctx,pcfg p,pinit p v ctx
        ⊆ pgam p (solved_local_reader (live_unknowns p) (sol_env is_global p) (Inl (v, ctx)))
            (gsol p)"
    using live.activation_collect_dg_sound
        [OF entry_cov _ init_env_le, of "pinit p" v ctx] cinit_le_init
    by (simp add: sol_global_def)
  then show ?thesis using gamma_live_reader_le by blast
qed

corollary entry_state_lookup_sound_of_terminates:
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
  shows "Aκ,admitted_contexts is_global p,root_ctx,pcfg p,pinit p v ctx
          ⊆ gamma_lift γV (lookup_table (result is_global p) v ctx)"
  using entry_state_activation_collect_sound_of_terminates[OF wf solves]
  unfolding gamma_reader_eq_lookup .

lemma entry_state_has_context_of_terminates:
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
    and t: "t ∈ Tκ,pcfg p,pinit p"
  shows "∃c. activation_context_rel κ (admitted_contexts is_global p)
              root_ctx (pcfg p) t c"
proof -
  interpret live: routed_analysis κ "analysis_spec is_global p"
    "λd e. pgam p d e" "pcfg p" buffer_key global_of
      "route is_global" Bot
        "Lifted init_st" "place_rg is_global (Lifted init_st)"
      "sol_env is_global p" "live_unknowns p" "root_query p" seed "λd. d = Bot"
      "admitted_contexts is_global p"
      "λd e. map_lift (rd is_global) (place_cmb p d e)"
      γV emptyV classify
    by (rule entry_state_routed_analysis_from_live_unknowns[OF wf solves])
  have entry_cov: "(cfg_entry (pcfg p), root_ctx) ∈ live_unknowns p"
    by (rule ctx_vars_cover_live_entryD[OF live_unknowns_cover[OF wf solves]])
  have init_env_le:
    "init_env is_global p ≤ genv global_of (sol_env is_global p)"
    using init_env_le_sol[OF solves subsetD[OF live_unknowns_sub entry_cov]
      live.pp_entry_init_global_bound[OF entry_cov]]
    by (simp add: sol_global_def)
  show ?thesis
    using live.routed_valid_activation_trace_has_context[OF entry_cov _ init_env_le t] cinit_le_init
    by simp
qed

theorem entry_state_node_collect_eq_Union_of_terminates:
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
  shows "Cκ,pcfg p,pinit p v
          = (⋃ctx. Aκ,admitted_contexts is_global p,root_ctx,pcfg p,pinit p v ctx)"
  by (rule node_collect_eq_Union_activation_of_has_context)
     (rule entry_state_has_context_of_terminates[OF wf solves])

theorem entry_state_result_node_sound:
  assumes wf: "gwf_compile_input vars ReturnVar is_global (ptab p) (pprocs p)" and solves:
    "terminates is_global p"
  shows "Cκ,pcfg p,pinit p v
          ⊆ (⋃ctx. γV (state_at is_global p ctx v))"
proof -
  have "Aκ,admitted_contexts is_global p,root_ctx,pcfg p,pinit p v ctx ⊆ γV (state_at is_global p ctx v)"
    for ctx
```

```
    using entry_state_lookup_sound_of_terminates[OF wf solves, of v ctx]
    by (auto simp: state_at_unfold bot_state_empty split: lifted.splits)
  then show ?thesis
    unfolding entry_state_node_collect_eq_Union_of_terminates[OF wf solves] by blast
qed

end

end

end

end
```

Chapter 10

Voblint_Nonrelational

10.1 Nonrelational_Domain

```
theory Nonrelational_Domain
  imports "Voblint_Compile.Kind_Invariants" "Voblint_Domain.Kind_Lifted"
    "Voblint_Framework.Analysis_Query"
begin
```

The analyses reason about the conversion by its value: from here on it unfolds.

```
declare kind_conv_simps [simp]
```

Values here are per variable, so γ is the class concretization alone.

```
no_notation gamma_S ("γ")
```

Comparisons from two queries

A domain that decides `<` and `=` decides every comparison: the others swap or negate those two answers.

```
fun cop_of :: "('a ⇒ 'a ⇒ bool option) ⇒ ('a ⇒ 'a ⇒ bool option) ⇒ cop ⇒ 'a ⇒ 'a ⇒ bool option"
where
  "cop_of lt eq CLt a b = lt a b"
| "cop_of lt eq CGt a b = lt b a"
| "cop_of lt eq CLe a b = map_option HOL.Not (lt b a)"
| "cop_of lt eq CGe a b = map_option HOL.Not (lt a b)"
| "cop_of lt eq CEq a b = eq a b"
| "cop_of lt eq CNe a b = map_option HOL.Not (eq a b)"

lemma cop_of_sound:
  fixes conc :: "'a ⇒ int set"
  assumes lt: "⋀a b r i j. lt a b = Some r ⟹ i ∈ conc a ⟹ j ∈ conc b ⟹ (i < j) = r"
    and eq: "⋀a b r i j. eq a b = Some r ⟹ i ∈ conc a ⟹ j ∈ conc b ⟹ (i = j) = r"
  shows "cop_of lt eq c a b = Some r ⟹ m ∈ conc a ⟹ n ∈ conc b ⟹ cop_eval c m n = r"
  by (cases c) (auto dest: lt eq)
```

Kind-aware integer domains

The concrete semantics wraps every result, signed or not. How an abstract domain treats a possible signed overflow is a separate choice, Goblint's `sem.int.signed_overflow`: `Assume_Top`, its default, answers the kind's top, and `Assume_Wraparound` wraps as the concrete semantics does. Both are sound here. An unsigned kind always wraps (`should_wrap` in `intDomain0.ml`).

```
datatype signed_overflow_policy = Assume_Top | Assume_Wraparound

definition should_wrap :: "signed_overflow_policy ⇒ ikind ⇒ bool" where
  "should_wrap ovp k ⟷ ¬ ik_signed k ∨ ovp = Assume_Wraparound"
```

The options of Goblint's `sem.int` group that the domains read. Goblint reads them from its global configuration at every use; here one value is a parameter of the evaluator and the transfers, and every soundness theorem holds for each value. A further option is a further field.

```
record sem_conf =
  sem_int_signed_overflow :: signed_overflow_policy

definition default_sem_conf :: sem_conf where
  "default_sem_conf = (| sem_int_signed_overflow = Assume_Top |)"
```

A remainder of in-range operands, and a quotient unless the divisor is `-1`, stays in the kind's range: no policy applies to them.

```
lemma c_mod_between: "min 0 a ≤ c_mod a b ∧ c_mod a b ≤ max 0 a"
proof (cases "b = 0")
  case False
  have "c_mod a b = sgn a * (|a| mod |b|)"
    using False by (simp add: c_mod_def c_div_def algebra_simps abs_mult_sgn
        minus_div_mult_eq_mod[symmetric] sgn_mult_abs; metis abs_sgn)
  moreover have "0 ≤ |a| mod |b|" "|a| mod |b| ≤ |a|"
    using False by (simp_all add: zmod_le_nonneg_dividend)
  ultimately show ?thesis by (cases "0 ≤ a") (auto simp: sgn_if)
qed (simp add: c_mod_def)

lemma c_mod_in_range: "m ∈ ik_range k ⟹ c_mod m n ∈ ik_range k"
  using c_mod_between[of m n] ik_min_le_zero[of k] zero_le_ik_max[of k]
  by (cases "0 ≤ m") (auto simp: min_def max_def simp del: ik_min_le_zero zero_le_ik_max)

lemma c_div_in_range:
```

```
  assumes m: "m ∈ ik_range k" and n: "n ∈ ik_range k" and n1: "n ≠ -1"
  shows "c_div m n ∈ ik_range k"
proof (cases "n = 0")
  case False
  define q where "q = |m| div |n|"
  have q0: "0 ≤ q" using False by (simp add: q_def pos_imp_zdiv_nonneg_iff)
  have "q * |n| = |m| - |m| mod |n|" by (simp add: q_def minus_mod_eq_div_mult)
  moreover have "0 ≤ |m| mod |n|" using False by simp
  ultimately have qn: "q * |n| ≤ |m|" by linarith
  have cd: "c_div m n = sgn m * sgn n * q" using False by (simp add: c_div_def q_def)
  have lo: "ik_min k = (if ik_signed k then - ik_max k - 1 else 0)"
    by (simp add: ik_min_def ik_max_def)
  have one: "1 ≤ ik_max k" by (cases k) (simp_all add: ik_max_def ik_half_def ik_mod_def)
  have mk: "ik_min k ≤ m" "m ≤ ik_max k" "ik_min k ≤ n" "n ≤ ik_max k" using m n by simp_all
  show ?thesis
  proof (cases "n = 1")
    case True
    then show ?thesis using m by simp
  next
    case n2: False
    with False n1 have "2 ≤ |n|" by linarith
    then have "2 * q ≤ |m|" using qn mult_left_mono[of 2 "|n|" q] q0 by (simp add: mult.commute)
    then show ?thesis using cd lo one mk q0 by (auto simp: sgn_if split: if_splits)
  qed
qed simp
```

The operations a typed transfer needs from an integer domain, after Goblint's `IntDomain.S`: every operation that produces a value takes the kind it computes at. `top_ik k` is Goblint's `top_of ik`, `of_int_ik` its `of_int ik`, `cast_ik ovp sk k` its `cast_to ?from_ik:sk k`, where `sk` is the operand's kind when it is known, and `arith_ik ovp k op operands_fit` the binary operation at kind `k`, whose concrete result is normalized to `k`; `operands_fit` says that both operands lie in `k`'s range, as they do when each operand has kind `k`. It does not say that the result does. A comparison answers `Some b` only when every pair of operands agrees on `b`. `refine_ik c a b` narrows both operands of a comparison known to hold, as `inv_bin_int` in Goblint's `baseInvariant.ml` does for a guard. `raw_aop_ik` is the operation on mathematical integers, which a backward step inverts, and `fits_ik k a` says that every value of `a` lies in `k`'s range, so that normalizing to `k` changes none of them.

`answer_ik` and `of_answer_ik` translate between a value and an answer to Goblint's `EvalInt` query, as Goblint's `IntDomTuple` is both the base value and the query result. `of_answer_ik sk` may use the kind `sk` of the queried expression, when it is known, to bound an answer that claims nothing.

```
class kind_int_domain = ik_domain +
  fixes top_ik :: "ikind ⇒ 'a::{bounded_semilattice_sup_bot, order_top}"
    and of_int_ik :: "ikind ⇒ int ⇒ 'a"
    and cast_ik :: "signed_overflow_policy ⇒ ikind option ⇒ ikind ⇒ 'a ⇒ 'a"
    and arith_ik :: "signed_overflow_policy ⇒ ikind ⇒ aop ⇒ bool ⇒ 'a ⇒ 'a ⇒ 'a"
    and cmp_ik :: "cop ⇒ 'a ⇒ 'a ⇒ bool option"
    and meet_ik :: "'a ⇒ 'a ⇒ 'a"
    and refine_ik :: "cop ⇒ 'a ⇒ 'a ⇒ 'a × 'a"
    and raw_aop_ik :: "aop ⇒ 'a ⇒ 'a ⇒ 'a"
    and fits_ik :: "ikind ⇒ 'a ⇒ bool"
    and answer_ik :: "'a ⇒ answer"
    and of_answer_ik :: "ikind option ⇒ answer ⇒ 'a"
  assumes top_ik_sound: "n ∈ ik_range k ⟹ n ∈ γ (top_ik k)"
    and of_int_ik_sound: "ik_norm k n ∈ γ (of_int_ik k n)"
    and cast_ik_sound:
      "(⋀k'. sk = Some k' ⟹ n ∈ ik_range k') ⟹ n ∈ γ a
         ⟹ ik_norm k n ∈ γ (cast_ik ovp sk k a)"
    and arith_ik_sound:
      "(operands_fit ⟹ m ∈ ik_range k ∧ n ∈ ik_range k) ⟹ m ∈ γ a ⟹ n ∈ γ b
         ⟹ ik_norm k (aop_eval op m n) ∈ γ (arith_ik ovp k op operands_fit a b)"
    and cmp_ik_sound:
      "cmp_ik c a b = Some r ⟹ m ∈ γ a ⟹ n ∈ γ b ⟹ cop_eval c m n = r"
    and meet_ik_sound: "m ∈ γ a ⟹ m ∈ γ b ⟹ m ∈ γ (meet_ik a b)"
    and refine_ik_sound:
      "cop_eval c m n ⟹ m ∈ γ a ⟹ n ∈ γ b
         ⟹ m ∈ γ (fst (refine_ik c a b)) ∧ n ∈ γ (snd (refine_ik c a b))"
    and raw_aop_ik_sound: "m ∈ γ a ⟹ n ∈ γ b ⟹ aop_eval op m n ∈ γ (raw_aop_ik op a b)"
    and fits_ik_sound: "fits_ik k a ⟹ n ∈ γ a ⟹ n ∈ ik_range k"
    and answer_ik_sound: "n ∈ γ a ⟹ n ∈ gamma_answer (answer_ik a)"
    and of_answer_ik_sound:
      "(⋀k. sk = Some k ⟹ n ∈ ik_range k) ⟹ n ∈ gamma_answer q
         ⟹ n ∈ γ (of_answer_ik sk q)"
```

A single-component domain answers with its component of `int_dom` and leaves the others at top, so its answer claims exactly what its value does.

```
abbreviation answer_with :: "(int_dom ⇒ int_dom) ⇒ answer" where
  "answer_with f ≡ QLifted (f ⊤)"
```

Typed abstract states

A typed abstract state maps every variable identity to a kind-tagged value. It describes the typed stores whose values lie in the state's values: `in_kinds` is part of the description, so a value the state does not constrain still stays in its variable's range.

```
type_synonym 'a tabs = "var_id ⇒ 'a kd"

definition gamma_tabs :: "kenv ⇒ 'a::ik_domain tabs ⇒ store set" where
  "gamma_tabs κ σ = {t. in_kinds κ t ∧ (∀v. t v ∈ γ (σ v))}"

lemma gamma_tabsD:
  assumes "t ∈ gamma_tabs κ σ"
  shows "in_kinds κ t" and "t v ∈ γ (σ v)"
  using assms by (simp_all add: gamma_tabs_def)

lemma gamma_tabsI:
  "in_kinds κ t ⟹ (⋀v. t v ∈ γ (σ v)) ⟹ t ∈ gamma_tabs κ σ"
  by (simp add: gamma_tabs_def)

lemma gamma_tabs_mono:
  assumes "σ₁ ≤ σ₂"
  shows "gamma_tabs κ σ₁ ⊆ gamma_tabs κ σ₂"
proof -
  have "x v ∈ γ (σ₂ v)" if "x v ∈ γ (σ₁ v)" for x v
    using that gamma_mono[OF le_funD[OF assms, of v]] by blast
  then show ?thesis by (auto simp: gamma_tabs_def simp del: gamma_kd.simps)
qed

lemma gamma_tabs_bot [simp]: "gamma_tabs κ ⊥ = {}"
  by (simp add: gamma_tabs_def bot_fun_def bot_kd_def)
```

Reading and evaluating

Reading a variable gives its value without the tag. An untagged top is the variable's whole range, as Goblint reads an unconstrained local at `top_of ik`; the return slot has no declared kind and reads as top.

```
fun kd_val :: "'a::kind_int_domain kd ⇒ 'a" where
  "kd_val KBot = ⊥"
| "kd_val (KD k a) = a"
| "kd_val KTop = ⊤"

definition read_var :: "kenv ⇒ 'a::kind_int_domain tabs ⇒ var_id ⇒ 'a" where
  "read_var κ σ v =
     (case σ v of KTop ⇒ if v = ReturnVar then ⊤ else top_ik (κ v) | d ⇒ kd_val d)"

lemma kd_val_sound: "n ∈ γ d ⟹ n ∈ γ (kd_val d)"
  by (cases d) simp_all
```

A tagged value lies in its tag's range, so a conversion from it knows its source.

```
definition kd_kind :: "'a kd ⇒ ikind option" where
  "kd_kind d = (case d of KD k a ⇒ Some k | _ ⇒ None)"

lemma kd_kind_range: "n ∈ γ d ⟹ kd_kind d = Some k ⟹ n ∈ ik_range k"
  by (cases d) (auto simp: kd_kind_def)

lemma read_var_sound:
  assumes "t ∈ gamma_tabs κ σ"
  shows "t v ∈ γ (read_var κ σ v)"
proof -
  have ty: "in_kinds κ t" and tv: "t v ∈ γ (σ v)"
    using gamma_tabsD[OF assms] by blast+
  show ?thesis
  proof (cases "σ v")
    case KTop
    show ?thesis
    proof (cases "v = ReturnVar")
      case False
      then have "t v ∈ ik_range (κ v)" using ty by (simp add: in_kinds_def)
      with KTop False show ?thesis by (simp add: read_var_def top_ik_sound)
    qed (use KTop in ‹simp add: read_var_def›)
  qed (use tv in ‹simp_all add: read_var_def›)
```

```
qed
```

A truth value is `0` or `1`; an undecided one is both.

```
definition bool_val :: "bool option ⇒ 'a::kind_int_domain" where
  "bool_val r =
     (case r of Some True ⇒ of_int_ik I32 1 | Some False ⇒ of_int_ik I32 0
      | None ⇒ of_int_ik I32 0 ⊔ of_int_ik I32 1)"

definition truth_ik :: "'a::kind_int_domain ⇒ bool option" where
  "truth_ik a = cmp_ik CNe a (of_int_ik I32 0)"

lemma of_int_ik_I32_01:
  "0 ∈ γ (of_int_ik I32 0 :: 'a::kind_int_domain)"
  "1 ∈ γ (of_int_ik I32 1 :: 'a::kind_int_domain)"
  using of_int_ik_sound[of I32 0] of_int_ik_sound[of I32 1]
  by (simp_all add: ik_norm_def ik_min_def ik_max_def)

lemma bool_val_sound:
  assumes "⋀b. r = Some b ⟹ P = b"
  shows "(if P then 1 else 0) ∈ γ (bool_val r :: 'a::kind_int_domain)"
proof (cases r)
  case None
  have "0 ∈ γ (of_int_ik I32 0 ⊔ of_int_ik I32 1 :: 'a)"
    "1 ∈ γ (of_int_ik I32 0 ⊔ of_int_ik I32 1 :: 'a)"
    using of_int_ik_I32_01 gamma_mono[OF sup.cobounded1] gamma_mono[OF sup.cobounded2]
    by blast+
  with None show ?thesis by (simp add: bool_val_def)
next
  case (Some b)
  with assms show ?thesis by (cases b) (simp_all add: bool_val_def of_int_ik_I32_01)
qed

lemma truth_ik_sound:
  "truth_ik a = Some b ⟹ n ∈ γ a ⟹ truthy n = b"
  unfolding truth_ik_def
  using cmp_ik_sound[of CNe a "of_int_ik I32 0" b n 0] of_int_ik_I32_01(1)
  by (simp add: truthy_def)
```

A conversion knows its operand's kind when the operand produces values of that kind: an operation always does, and a variable does when it is read at its declared kind.

```
definition src_kind :: "kenv ⇒ exp ⇒ ikind option" where
  "src_kind κ e =
     (case e of TVar ik v ⇒ if v ≠ ReturnVar ∧ ik = κ v then Some ik else None
      | _ ⇒ Some (exp_kind e))"

lemma src_kind_sound:
  assumes "in_kinds κ t" and "src_kind κ e = Some k"
  shows "eval e t ∈ ik_range k"
  using assms by (cases e) (auto simp: src_kind_def in_kinds_def split: if_splits)

lemma cast_ik_None_sound: "n ∈ γ a ⟹ ik_norm k n ∈ γ (cast_ik ovp None k a)"
  by (rule cast_ik_sound) simp_all
```

A conversion may name the kind of the expression it converts, as Goblint's `cast_to ?from_ik` does: a typed store gives that expression a value in its kind's range.

```
lemma cast_ik_src_sound:
  assumes "in_kinds κ t" and "eval e t ∈ γ a"
  shows "ik_norm k (eval e t) ∈ γ (cast_ik ovp (src_kind κ e) k a)"
  using assms(2) src_kind_sound[OF assms(1)] by (intro cast_ik_sound) auto
```

Evaluation consults the other analyses, as Goblint's `base` does (`eval_rv_ask_evalint` in `src/analyses/base.ml` at `0dc12d355`): every operand that is not a constant is asked about through the channel `ch`, and its answer is met with the analysis's own evaluation. Goblint takes the answer alone: there the evaluated state is the one the channel answers about, since CIL splits a conjunction into one guard per conjunct. A VIMP guard refines a whole conjunction in one state, so the second conjunct is evaluated in a state the first has already narrowed, finer than the channel's. `aeval` asks about the operands but not about the expression itself, which is how the analysis answers a query without asking it back; `aeval_ask` also asks about the expression. An answer that claims nothing, such as the one a query cycle gets, falls back to the analysis's own evaluation, where Goblint takes the kind's top. Under the channel that answers every query with ⊤, both are the plain evaluation.

```
context
  fixes sem :: sem_conf
```

begin

```
fun aeval :: "kenv ⇒ (query ⇒ answer) ⇒ exp ⇒ 'a::kind_int_domain tabs ⇒ 'a"
  and aeval_ask :: "kenv ⇒ (query ⇒ answer) ⇒ exp ⇒ 'a::kind_int_domain tabs ⇒ 'a" where
  "aeval κ ch (TN ik n) σ = of_int_ik ik n"
| "aeval κ ch (TVar ik v) σ = read_var κ σ v"
| "aeval κ ch (TArith ik op a b) σ =
     arith_ik (sem_int_signed_overflow sem) ik op (src_kind κ a = Some ik ∧ src_kind κ b = Some ik)
       (aeval_ask κ ch a σ) (aeval_ask κ ch b σ)"
| "aeval κ ch (TCast ik a) σ =
     cast_ik (sem_int_signed_overflow sem) (src_kind κ a) ik (aeval_ask κ ch a σ)"
| "aeval κ ch (TCmp op a b) σ =
     bool_val (cmp_ik op (aeval_ask κ ch a σ) (aeval_ask κ ch b σ))"
| "aeval κ ch (TNot a) σ = bool_val (map_option HOL.Not (truth_ik (aeval_ask κ ch a σ)))"
| "aeval κ ch (TAnd a b) σ =
     bool_val (case (truth_ik (aeval_ask κ ch a σ), truth_ik (aeval_ask κ ch b σ)) of
                 (Some False, _) ⇒ Some False | (_, Some False) ⇒ Some False
               | (Some True, Some True) ⇒ Some True | _ ⇒ None)"
| "aeval κ ch (TOr a b) σ =
     bool_val (case (truth_ik (aeval_ask κ ch a σ), truth_ik (aeval_ask κ ch b σ)) of
                 (Some True, _) ⇒ Some True | (_, Some True) ⇒ Some True
               | (Some False, Some False) ⇒ Some False | _ ⇒ None)"
| "aeval_ask κ ch e σ =
     (case e of TN ik n ⇒ of_int_ik ik n
      | _ ⇒ (case ch (EvalInt e) of QTop ⇒ aeval κ ch e σ
              | q ⇒ meet_ik (aeval κ ch e σ) (of_answer_ik (src_kind κ e) q)))"

lemma aeval_ask_of_sound:
  assumes "t ∈ gamma_tabs κ σ" and "eval_query.channel_holds ch t"
    and "eval e t ∈ γ (aeval κ ch e σ)"
  shows "eval e t ∈ γ (aeval_ask κ ch e σ)"
proof -
  have ans: "eval e t ∈ gamma_answer (ch (EvalInt e))"
    using eval_query.channel_holdsD[OF assms(2), of "EvalInt e"] by simp
  have rng: "⋀k. src_kind κ e = Some k ⟹ eval e t ∈ ik_range k"
    using src_kind_sound[OF gamma_tabsD(1)[OF assms(1)]] .
  show ?thesis
    using assms(3) of_answer_ik_sound[OF rng ans] meet_ik_sound
    by (cases e; cases "ch (EvalInt e)") auto
qed

theorem aeval_sound:
  assumes "t ∈ gamma_tabs κ σ" and "eval_query.channel_holds ch t"
  shows "eval e t ∈ γ (aeval κ ch e σ)"
proof (induction e)
  case (TN ik n) show ?case by (simp add: of_int_ik_sound)
next
  case (TVar ik v) show ?case by (simp add: read_var_sound[OF assms(1)])
next
  case (TArith ik op a b)
  have "src_kind κ e = Some ik ⟹ eval e t ∈ ik_range ik" for e
    using src_kind_sound[OF gamma_tabsD(1)[OF assms(1)]] .
  with TArith show ?case
    using aeval_ask_of_sound[OF assms] by (simp add: arith_ik_sound)
next
  case (TCast ik a)
  have "eval a t ∈ ik_range k" if "src_kind κ a = Some k" for k
    using src_kind_sound[OF gamma_tabsD(1)[OF assms(1)] that] .
  with TCast aeval_ask_of_sound[OF assms] show ?case by (simp add: cast_ik_sound)
next
  case (TCmp op a b) then show ?case
    using aeval_ask_of_sound[OF assms] by (simp add: bool_val_sound cmp_ik_sound)
next
  case (TNot a)
  have a: "eval a t ∈ γ (aeval_ask κ ch a σ)"
    by (rule aeval_ask_of_sound[OF assms TNot])
  have "(if ¬ truthy (eval a t) then 1 else 0)
      ∈ γ (bool_val (map_option HOL.Not (truth_ik (aeval_ask κ ch a σ))) :: 'a)"
    by (rule bool_val_sound) (use a truth_ik_sound in auto)
  moreover have "eval (TNot a) t = (if ¬ truthy (eval a t) then 1 else 0)"
    by simp
  ultimately show ?case by (simp only: aeval.simps)
next
  case (TAnd a b)
  have a: "eval a t ∈ γ (aeval_ask κ ch a σ)" and b: "eval b t ∈ γ (aeval_ask κ ch b σ)"
    using aeval_ask_of_sound[OF assms] TAnd by blast+
```

```
  show ?case
    unfolding eval.simps aeval.simps
    by (rule bool_val_sound)
       (use a b truth_ik_sound in ‹auto split: option.splits bool.splits›)
next
  case (TOr a b)
  have a: "eval a t ∈ γ (aeval_ask κ ch a σ)" and b: "eval b t ∈ γ (aeval_ask κ ch b σ)"
    using aeval_ask_of_sound[OF assms] TOr by blast+
  show ?case
    unfolding eval.simps aeval.simps
    by (rule bool_val_sound)
       (use a b truth_ik_sound in ‹auto split: option.splits bool.splits›)
qed

corollary aeval_ask_sound:
  assumes "t ∈ gamma_tabs κ σ" and "eval_query.channel_holds ch t"
  shows "eval e t ∈ γ (aeval_ask κ ch e σ)"
  by (rule aeval_ask_of_sound[OF assms aeval_sound[OF assms]])

end

end
```

10.2 Nonrelational_Transfer

```
theory Nonrelational_Transfer
  imports Nonrelational_Domain "Voblint_Framework.MCP_Spec" "Voblint_Framework.Bool_Refine"
begin
```

The typed non-relational transfer

What one CFG edge does to a typed abstract state, after Goblint's `base` analysis. An assignment evaluates its right-hand side and converts the value to the destination's kind (`cast_to`); a library call writes its result at the destination's kind; a guard is dropped when its condition is decided against it; a return writes the returned value into the return slot; a call resets the callee's locals to their kinds' whole ranges and binds the formals; and the return combine takes the callee's globals and writes the converted return value.

```
fun neg_cop :: "cop ⇒ cop" where
  "neg_cop CLt = CGe" | "neg_cop CLe = CGt" | "neg_cop CGt = CLe" | "neg_cop CGe = CLt"
| "neg_cop CEq = CNe" | "neg_cop CNe = CEq"

lemma cop_eval_neg_cop [simp]: "cop_eval (neg_cop c) m n ⟷ ¬ cop_eval c m n"
  by (cases c) auto
```

The Boolean structure of a typed condition, for `bool_refine`.

```
fun nonrel_view :: "exp ⇒ exp bool_view" where
  "nonrel_view (TNot a) = BV_Not a"
| "nonrel_view (TAnd a b) = BV_And a b"
| "nonrel_view (TOr a b) = BV_Or a b"
| "nonrel_view _ = BV_Atom"

lemma eval_in_range_kind:
  assumes "¬ (∃ik v. e = TVar ik v)"
  shows "eval e t ∈ ik_range (exp_kind e)"
  using assms by (cases e) auto
```

Normalizing an intermediate result changes no later sum or difference at the same kind, so an operation that wrapped is inverted exactly by the wrapping inverse.

```
lemma ik_norm_mod_eq: "ik_norm k m mod ik_mod k = m mod ik_mod k"
  by (simp add: ik_norm_mod mod_add_left_eq)

lemma ik_norm_cong: "x mod ik_mod k = y mod ik_mod k ⟹ ik_norm k x = ik_norm k y"
  by (metis ik_norm_mod mod_diff_cong)

lemma ik_norm_norm_add: "ik_norm k (ik_norm k m + n) = ik_norm k (m + n)"
  by (intro ik_norm_cong mod_add_cong ik_norm_mod_eq refl)

lemma ik_norm_norm_diff: "ik_norm k (ik_norm k m - n) = ik_norm k (m - n)"
  by (intro ik_norm_cong mod_diff_cong ik_norm_mod_eq refl)

lemma ik_norm_diff_norm: "ik_norm k (n - ik_norm k m) = ik_norm k (n - m)"
  by (intro ik_norm_cong mod_diff_cong ik_norm_mod_eq refl)
```

The smaller of two values is the left one where the left is at most the right, and the right one elsewhere; each case refines its operand by the comparison that selects it, so constants or disjoint ranges decide the result.

```
definition min_ik :: "'a::kind_int_domain ⇒ 'a ⇒ 'a" where
  "min_ik a b = fst (refine_ik CLe a b) ⊔ snd (refine_ik CGt a b)"

definition max_ik :: "'a::kind_int_domain ⇒ 'a ⇒ 'a" where
  "max_ik a b = snd (refine_ik CLe a b) ⊔ fst (refine_ik CGt a b)"

lemma min_ik_sound:
  assumes "m ∈ numeric_domain_class.gamma a" and "n ∈ numeric_domain_class.gamma b"
  shows "min m n ∈ numeric_domain_class.gamma (min_ik a b :: 'a::kind_int_domain)"
proof (cases "m ≤ n")
  case True
  then have "m ∈ numeric_domain_class.gamma (fst (refine_ik CLe a b))"
    using refine_ik_sound[of CLe m n a b] assms by simp
  with True show ?thesis using gamma_mono[OF sup.cobounded1] by (fastforce simp: min_ik_def)
next
  case False
  then have "n ∈ numeric_domain_class.gamma (snd (refine_ik CGt a b))"
    using refine_ik_sound[of CGt m n a b] assms by simp
  with False show ?thesis using gamma_mono[OF sup.cobounded2] by (fastforce simp: min_ik_def)
qed
```

```
lemma max_ik_sound:
  assumes "m ∈ numeric_domain_class.gamma a" and "n ∈ numeric_domain_class.gamma b"
  shows "max m n ∈ numeric_domain_class.gamma (max_ik a b :: 'a::kind_int_domain)"
proof (cases "m ≤ n")
  case True
  then have "n ∈ numeric_domain_class.gamma (snd (refine_ik CLe a b))"
    using refine_ik_sound[of CLe m n a b] assms by simp
  with True show ?thesis using gamma_mono[OF sup.cobounded1] by (fastforce simp: max_ik_def)
next
  case False
  then have "m ∈ numeric_domain_class.gamma (fst (refine_ik CGt a b))"
    using refine_ik_sound[of CGt m n a b] assms by simp
  with False show ?thesis using gamma_mono[OF sup.cobounded2] by (fastforce simp: max_ik_def)
qed
```

The minimum or maximum of two operands of one kind lies in that kind's range.

```
definition pair_kind :: "kenv ⇒ exp ⇒ exp ⇒ ikind option" where
  "pair_kind κ a b = (if src_kind κ a = src_kind κ b then src_kind κ a else None)"

context
  fixes sem :: sem_conf and κ :: kenv and ch :: "channel"
begin

definition nonrel_assign :: "var_id ⇒ exp ⇒ 'a::kind_int_domain tabs ⇒ 'a tabs" where
  "nonrel_assign x e σ = σ(x := KD (κ x) (cast_ik (sem_int_signed_overflow sem) (src_kind κ e) (κ x) (aeval_ask sem
κ ch e σ)))"

definition special_val :: "exp gspecial_call ⇒ var_id ⇒ 'a::kind_int_domain tabs ⇒ 'a" where
  "special_val sc x σ =
     (case sc of Nondet_Int ⇒ top_ik (κ x)
      | Min a b ⇒ cast_ik (sem_int_signed_overflow sem) (pair_kind κ a b) (κ x) (min_ik (aeval_ask sem κ ch a σ) (aeval_ask
sem κ ch b σ))
      | Max a b ⇒ cast_ik (sem_int_signed_overflow sem) (pair_kind κ a b) (κ x) (max_ik (aeval_ask sem κ ch a σ) (aeval_ask
sem κ ch b σ)))"

definition nonrel_special :: "exp gspecial_call ⇒ var_id ⇒ 'a::kind_int_domain tabs ⇒ 'a tabs" where
  "nonrel_special sc x σ = σ(x := KD (κ x) (special_val sc x σ))"
```

A guard narrows the variables it compares, as Goblint's `baseInvariant` does. A comparison refines each operand by the value it must have and passes that value down: a conversion whose operand already fits the target kind keeps every value, and a sum or difference that cannot overflow is inverted on mathematical integers. One that may overflow is inverted at its kind, with wraparound, for each operand of that kind, as Goblint's `inv_bin_int` does: every operand value in the kind's range is the wrapped difference of the result and the other operand. Any other node stops the refinement. A negation flips the polarity, a conjunction that holds and a disjunction that fails narrow by both sides.

```
definition narrow_var :: "var_id ⇒ 'a::kind_int_domain ⇒ 'a tabs ⇒ 'a tabs" where
  "narrow_var v a σ =
     (if v = ReturnVar then σ else σ(v := KD (κ v) (meet_ik a (read_var κ σ v))))"

fun refine_var :: "exp ⇒ 'a::kind_int_domain ⇒ 'a tabs ⇒ 'a tabs" where
  "refine_var (TVar ik v) a σ = narrow_var v a σ"
| "refine_var (TCast ik e) a σ =
     (if fits_ik ik (aeval_ask sem κ ch e σ) then refine_var e a σ else σ)"
| "refine_var (TArith ik op e₁ e₂) a σ =
     (let a₁ = aeval_ask sem κ ch e₁ σ; a₂ = aeval_ask sem κ ch e₂ σ
      in if fits_ik ik (raw_aop_ik op a₁ a₂) then
           (case op of
             AAdd ⇒ refine_var e₂ (raw_aop_ik ASub a a₁) (refine_var e₁ (raw_aop_ik ASub a a₂) σ)
           | ASub ⇒ refine_var e₂ (raw_aop_ik ASub a₁ a) (refine_var e₁ (raw_aop_ik AAdd a a₂) σ)
           | _ ⇒ σ)
         else if op ∉ {AAdd, ASub} then σ
         else let σ' = (if src_kind κ e₁ = Some ik
                        then refine_var e₁ (arith_ik (sem_int_signed_overflow sem) ik (if op = AAdd then ASub else
AAdd) False a a₂) σ else σ)
              in if src_kind κ e₂ = Some ik
                 then refine_var e₂ (if op = AAdd then arith_ik (sem_int_signed_overflow sem) ik ASub False a a₁ else
arith_ik (sem_int_signed_overflow sem) ik ASub False a₁ a) σ'
                 else σ')"
| "refine_var e a σ = σ"
```

The Boolean structure of a guard is the generic one of `bool_refine`: a comparison is the atom, narrowed by `refine_ik`; an operand of a disjunction that must hold, or of a conjunction that must fail, is kept only while its forward value admits the required truth value, and the kept alternatives are joined. When none is kept no store

satisfies the guard, and the result is bottom, as Goblint's `Deadcode`. A branch keeps the refinement only at the variables its guard reads, where the executable state stores it; a variable refined to bottom marks the branch unreachable.

```
definition nonrel_narrow :: "exp ⇒ bool ⇒ 'a::kind_int_domain tabs ⇒ 'a tabs" where
  "nonrel_narrow e pol σ =
     (case e of
        TCmp c l r ⇒
          (let (al, ar) = refine_ik (if pol then c else neg_cop c)
                            (aeval_ask sem κ ch l σ) (aeval_ask sem κ ch r σ)
           in refine_var l al (refine_var r ar σ))
      | _ ⇒ refine_var e (fst (refine_ik (if pol then CNe else CEq)
                         (aeval_ask sem κ ch e σ) (of_int_ik (exp_kind e) 0))) σ)"
```

An atom that leaves one of its variables without a value describes no store, and is bottom as a whole: joined with the other alternative of a disjunction, it then adds nothing, where the variables it did not narrow would otherwise bring the incoming state back. Only the atom's variables change, so checking them suffices.

```
definition nonrel_atom :: "exp ⇒ bool ⇒ 'a::kind_int_domain tabs ⇒ 'a tabs" where
  "nonrel_atom e pol σ =
     (let σ' = nonrel_narrow e pol σ in if ∃v ∈ exp_vars e. is_empty (σ' v) then ⊥ else σ')"

definition nonrel_feas :: "exp ⇒ bool ⇒ 'a::kind_int_domain tabs ⇒ bool" where
  "nonrel_feas e pol σ = (truth_ik (aeval_ask sem κ ch e σ) ≠ Some (¬ pol))"

lemma nonrel_bool_locale: "bool_refine nonrel_view size"
  by unfold_locales (auto elim: nonrel_view.elims)

definition nonrel_refine :: "exp ⇒ bool ⇒ 'a::kind_int_domain tabs ⇒ 'a tabs" where
  "nonrel_refine = bool_refine.brefine nonrel_view nonrel_atom nonrel_feas sup (λ_. ⊥)"

definition nonrel_branch :: "exp ⇒ bool ⇒ 'a::kind_int_domain tabs ⇒ 'a tabs" where
  "nonrel_branch b pol σ =
     (let refined = (λv. if v ∈ exp_vars b then nonrel_refine b pol σ v else σ v)
      in case truth_ik (aeval_ask sem κ ch b σ) of
           Some r ⇒ if r = pol then refined else ⊥
         | None ⇒ refined)"
```

The returned value is tagged with a kind whose range it lies in: a variable's declared kind, or the kind an operation normalized it to.

```
definition ret_kd :: "exp ⇒ 'a::kind_int_domain tabs ⇒ 'a kd" where
  "ret_kd e σ =
     (case e of TVar ik v ⇒ if v = ReturnVar then KTop else KD (κ v) (read_var κ σ v)
      | _ ⇒ KD (exp_kind e) (aeval_ask sem κ ch e σ))"

definition nonrel_return :: "exp option ⇒ 'a::kind_int_domain tabs ⇒ 'a tabs" where
  "nonrel_return r σ = (case r of None ⇒ σ | Some e ⇒ σ(ReturnVar := ret_kd e σ))"

definition nonrel_enter_frame :: "'a::kind_int_domain tabs ⇒ 'a tabs" where
  "nonrel_enter_frame σ = enter_frame is_global KTop σ"

definition nonrel_enter_state ::
    "var_id list ⇒ exp list ⇒ 'a::kind_int_domain tabs ⇒ 'a tabs" where
  "nonrel_enter_state ps es σ =
     bind_formals ps (map2 (λp e. KD (κ p) (cast_ik (sem_int_signed_overflow sem) (src_kind κ e) (κ p) (aeval_ask sem
κ ch e σ))) ps es)
       (nonrel_enter_frame σ)"

definition nonrel_combine_assign :: "var_id option ⇒ 'a::kind_int_domain tabs ⇒ 'a tabs ⇒ 'a tabs"
  where
  "nonrel_combine_assign dst σ σ_e =
     (case dst of None ⇒ σ
      | Some x ⇒ σ(x := KD (κ x)
          (cast_ik (sem_int_signed_overflow sem) (kd_kind (σ_e ReturnVar)) (κ x) (kd_val (σ_e ReturnVar)))))"
```

Soundness of each operation

Each operation's result describes the store its edge produces from any typed store the input state describes.

```
lemma in_kinds_upd:
  "in_kinds κ t ⟹ (x ≠ ReturnVar ⟹ n ∈ ik_range (κ x)) ⟹ in_kinds κ (t(x := n))"
  by (auto simp: in_kinds_def)

lemma gamma_tabs_upd:
  fixes d :: "'a::kind_int_domain kd"
```

```
  assumes "t ∈ gamma_tabs κ σ"
    and "n ∈ numeric_domain_class.gamma d" and "x ≠ ReturnVar ⟹ n ∈ ik_range (κ x)"
  shows "t(x := n) ∈ gamma_tabs κ (σ(x := d))"
  using gamma_tabsD[OF assms(1)] assms(2,3)
  by (intro gamma_tabsI in_kinds_upd) auto

lemma nonrel_assign_sound:
  assumes "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t"
  shows "t(x := ik_norm (κ x) (eval e t)) ∈ gamma_tabs κ (nonrel_assign x e σ)"
proof -
  have "ik_norm (κ x) (eval e t) ∈ numeric_domain_class.gamma (cast_ik (sem_int_signed_overflow sem) (src_kind κ e)
(κ x) (aeval_ask sem κ ch e σ))"
    by (rule cast_ik_src_sound[OF gamma_tabsD(1)[OF assms(1)] aeval_ask_sound[OF assms]])
  then show ?thesis unfolding nonrel_assign_def by (simp add: gamma_tabs_upd[OF assms(1)])
qed

lemma special_val_sound:
  assumes "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t" and
    "special_result eval sc t v"
  shows "ik_norm (κ x) v ∈ numeric_domain_class.gamma (special_val sc x σ)"
proof (cases sc)
  case Nondet_Int then show ?thesis by (simp add: special_val_def top_ik_sound)
next
  case (Min a b)
  then have v:
    "v ∈ numeric_domain_class.gamma (min_ik (aeval_ask sem κ ch a σ) (aeval_ask sem κ ch b σ))"
    using assms(3) min_ik_sound[OF aeval_ask_sound[OF assms(1,2)] aeval_ask_sound[OF assms(1,2)]]
    by simp
  have rng: "v ∈ ik_range k" if "pair_kind κ a b = Some k" for k
    using that assms(3) Min src_kind_sound[OF gamma_tabsD(1)[OF assms(1)], of a k]
      src_kind_sound[OF gamma_tabsD(1)[OF assms(1)], of b k]
    by (auto simp: pair_kind_def min_def split: if_splits)
  from Min show ?thesis
    by (simp only: special_val_def gspecial_call.case) (rule cast_ik_sound[OF rng v])
next
  case (Max a b)
  then have v:
    "v ∈ numeric_domain_class.gamma (max_ik (aeval_ask sem κ ch a σ) (aeval_ask sem κ ch b σ))"
    using assms(3) max_ik_sound[OF aeval_ask_sound[OF assms(1,2)] aeval_ask_sound[OF assms(1,2)]]
    by simp
  have rng: "v ∈ ik_range k" if "pair_kind κ a b = Some k" for k
    using that assms(3) Max src_kind_sound[OF gamma_tabsD(1)[OF assms(1)], of a k]
      src_kind_sound[OF gamma_tabsD(1)[OF assms(1)], of b k]
    by (auto simp: pair_kind_def max_def split: if_splits)
  from Max show ?thesis
    by (simp only: special_val_def gspecial_call.case) (rule cast_ik_sound[OF rng v])
qed

lemma nonrel_special_sound:
  assumes "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t" and
    "special_result eval sc t v"
  shows "t(x := ik_norm (κ x) v) ∈ gamma_tabs κ (nonrel_special sc x σ)"
  unfolding nonrel_special_def
  using gamma_tabs_upd[OF assms(1)] special_val_sound[OF assms] by simp

lemma narrow_var_sound:
  assumes "t ∈ gamma_tabs κ σ" and "t v ∈ numeric_domain_class.gamma a"
  shows "t ∈ gamma_tabs κ (narrow_var v a σ)"
proof (cases "v = ReturnVar")
  case False
  have rng: "t v ∈ ik_range (κ v)"
    using gamma_tabsD(1)[OF assms(1)] False by (simp add: in_kinds_def)
  have "t v ∈ numeric_domain_class.gamma (meet_ik a (read_var κ σ v))"
    by (rule meet_ik_sound[OF assms(2) read_var_sound[OF assms(1)]])
  with rng have "t(v := t v) ∈ gamma_tabs κ (σ(v := KD (κ v) (meet_ik a (read_var κ σ v))))"
    by (intro gamma_tabs_upd[OF assms(1)]) simp_all
  with False show ?thesis by (simp add: narrow_var_def)
qed (use assms in ‹simp add: narrow_var_def›)

lemma refine_var_sound:
  assumes "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t" and
    "eval e t ∈ numeric_domain_class.gamma a"
  shows "t ∈ gamma_tabs κ (refine_var e a σ)"
  using assms
proof (induction e arbitrary: a σ)
  case (TVar ik v) then show ?case by (simp add: narrow_var_sound)
```

```
next
  case (TCast ik e)
  show ?case
  proof (cases "fits_ik ik (aeval_ask sem κ ch e σ)")
    case True
    have "eval e t ∈ ik_range ik"
      by (rule fits_ik_sound[OF True aeval_ask_sound[OF TCast.prems(1,2)]])
    then have "eval e t ∈ numeric_domain_class.gamma a"
      using TCast.prems(3) by (simp del: ik_range_iff)
    with True TCast.IH[OF TCast.prems(1,2)] show ?thesis by simp
  qed (use TCast.prems in simp)
next
  case (TArith ik op e₁ e₂)
  define a₁ where "a₁ = aeval_ask sem κ ch e₁ σ"
  define a₂ where "a₂ = aeval_ask sem κ ch e₂ σ"
  have m: "eval e₁ t ∈ numeric_domain_class.gamma a₁"
    and n: "eval e₂ t ∈ numeric_domain_class.gamma a₂"
    using aeval_ask_sound[OF TArith.prems(1,2)] by (simp_all add: a₁_def a₂_def)
  show ?case
  proof (cases "fits_ik ik (raw_aop_ik op a₁ a₂)")
    case fits: True
    have "aop_eval op (eval e₁ t) (eval e₂ t) ∈ ik_range ik"
      by (rule fits_ik_sound[OF fits raw_aop_ik_sound[OF m n]])
    then have r: "aop_eval op (eval e₁ t) (eval e₂ t) ∈ numeric_domain_class.gamma a"
      using TArith.prems(3) by (simp del: ik_range_iff)
    show ?thesis
    proof (cases op)
      case AAdd
      have m': "eval e₁ t ∈ numeric_domain_class.gamma (raw_aop_ik ASub a a₂)"
        using raw_aop_ik_sound[OF r n, of ASub] AAdd by simp
      have n': "eval e₂ t ∈ numeric_domain_class.gamma (raw_aop_ik ASub a a₁)"
        using raw_aop_ik_sound[OF r m, of ASub] AAdd by simp
      have "refine_var (TArith ik op e₁ e₂) a σ
          = refine_var e₂ (raw_aop_ik ASub a a₁) (refine_var e₁ (raw_aop_ik ASub a a₂) σ)"
        using fits AAdd by (simp add: a₁_def a₂_def Let_def)
      with TArith.IH(2)[OF TArith.IH(1)[OF TArith.prems(1,2) m'] TArith.prems(2) n'] show ?thesis
        by simp
    next
      case ASub
      have m': "eval e₁ t ∈ numeric_domain_class.gamma (raw_aop_ik AAdd a a₂)"
        using raw_aop_ik_sound[OF r n, of AAdd] ASub by simp
      have n': "eval e₂ t ∈ numeric_domain_class.gamma (raw_aop_ik ASub a₁ a)"
        using raw_aop_ik_sound[OF m r, of ASub] ASub by simp
      have "refine_var (TArith ik op e₁ e₂) a σ
          = refine_var e₂ (raw_aop_ik ASub a₁ a) (refine_var e₁ (raw_aop_ik AAdd a a₂) σ)"
        using fits ASub by (simp add: a₁_def a₂_def Let_def)
      with TArith.IH(2)[OF TArith.IH(1)[OF TArith.prems(1,2) m'] TArith.prems(2) n'] show ?thesis
        by simp
    qed (use fits TArith.prems in ‹simp_all add: a₁_def a₂_def Let_def›)
  next
    case nofit: False
    show ?thesis
    proof (cases "op ∈ {AAdd, ASub}")
      case True
      have vimp: "in_kinds κ t" by (rule gamma_tabsD(1)[OF TArith.prems(1)])
      have r: "ik_norm ik (aop_eval op (eval e₁ t) (eval e₂ t)) ∈ numeric_domain_class.gamma a"
        using TArith.prems(3) by simp
      define σ' where "σ' = (if src_kind κ e₁ = Some ik
          then refine_var e₁ (arith_ik (sem_int_signed_overflow sem) ik (if op = AAdd then ASub else AAdd) False a
a₂) σ else σ)"
      have s1: "t ∈ gamma_tabs κ σ'"
      proof (cases "src_kind κ e₁ = Some ik")
        case k: True
        have "eval e₁ t ∈ ik_range ik" by (rule src_kind_sound[OF vimp k])
        then have "eval e₁ t ∈ numeric_domain_class.gamma (arith_ik (sem_int_signed_overflow sem) ik (if op = AAdd
then ASub else AAdd) False a a₂)"
          using arith_ik_sound[where k = ik and op = "if op = AAdd then ASub else AAdd", OF FalseE r
            n]
            True
          by (auto simp: ik_norm_norm_add ik_norm_norm_diff simp del: ik_range_iff)
        with k show ?thesis
          unfolding σ'_def using TArith.IH(1)[OF TArith.prems(1,2)] by simp
      qed (simp add: σ'_def TArith.prems(1))
      have eq: "refine_var (TArith ik op e₁ e₂) a σ = (if src_kind κ e₂ = Some ik
          then refine_var e₂ (if op = AAdd then arith_ik (sem_int_signed_overflow sem) ik ASub False a a₁ else arith_ik
(sem_int_signed_overflow sem) ik ASub False a₁ a) σ' else σ')"
```

```
      using nofit True by (simp add: a₁_def a₂_def Let_def σ'_def)
    have "t ∈ gamma_tabs κ (if src_kind κ e₂ = Some ik
      then refine_var e₂ (if op = AAdd then arith_ik (sem_int_signed_overflow sem) ik ASub False a a₁ else arith_ik
(sem_int_signed_overflow sem) ik ASub False a₁ a) σ' else σ')"
    proof (cases "src_kind κ e₂ = Some ik")
      case k: True
      have rng: "eval e₂ t ∈ ik_range ik" by (rule src_kind_sound[OF vimp k])
      have "eval e₂ t ∈ numeric_domain_class.gamma (if op = AAdd then arith_ik (sem_int_signed_overflow sem) ik
ASub False a a₁ else arith_ik (sem_int_signed_overflow sem) ik ASub False a₁ a)"
      proof (cases "op = AAdd")
        case True
        then show ?thesis
          using arith_ik_sound[where k = ik and op = ASub, OF FalseE r m] rng
          by (simp add: ik_norm_norm_diff del: ik_range_iff)
      next
        case False
        with ‹op ∈ {AAdd, ASub}› have "op = ASub" by simp
        then show ?thesis
          using arith_ik_sound[where k = ik and op = ASub, OF FalseE m r] rng
          by (simp add: ik_norm_diff_norm del: ik_range_iff)
      qed
      with k show ?thesis using TArith.IH(2)[OF s1 TArith.prems(2)] by simp
    qed (simp add: s1)
    with eq show ?thesis by simp
  qed (use nofit TArith.prems in ‹auto simp: a₁_def a₂_def Let_def›)
 qed
qed simp_all

lemma nonrel_refine_sound:
 assumes "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t" and
   "truthy (eval b t) = pol"
 shows "t ∈ gamma_tabs κ (nonrel_refine b pol σ)"
proof -
 let ?γ = "λσ. gamma_tabs κ σ ∩ Collect (eval_query.channel_holds ch)"
 have bare: "t ∈ ?γ (refine_var e (fst (refine_ik (if pol then CNe else CEq)
              (aeval_ask sem κ ch e σ) (of_int_ik (exp_kind e) 0))) σ)"
   if "t ∈ ?γ σ" and "(eval e t ≠ 0) = pol" for e pol σ t
 proof -
   have g: "t ∈ gamma_tabs κ σ" and h: "eval_query.channel_holds ch t" using that(1) by auto
   have holds: "cop_eval (if pol then CNe else CEq) (eval e t) 0" using that(2) by auto
   have "eval e t ∈ numeric_domain_class.gamma
          (fst (refine_ik (if pol then CNe else CEq) (aeval_ask sem κ ch e σ) (of_int_ik (exp_kind e) 0)))"
     using refine_ik_sound[OF holds aeval_ask_sound[OF g h, of e]
            of_int_ik_sound[of "exp_kind e" 0, unfolded ik_norm_zero]] by simp
   then show ?thesis using h by (simp add: refine_var_sound[OF g h])
 qed
 have narrow: "t ∈ ?γ (nonrel_narrow e pol σ)"
   if "t ∈ ?γ σ" and "(eval e t ≠ 0) = pol" for e pol σ t
 proof (cases e)
   case (TCmp c l r)
   let ?c = "if pol then c else neg_cop c"
   have g: "t ∈ gamma_tabs κ σ" and h: "eval_query.channel_holds ch t" using that(1) by auto
   have holds: "cop_eval ?c (eval l t) (eval r t)"
     using that(2) TCmp by (auto split: if_splits)
   obtain al ar where ref:
     "refine_ik ?c (aeval_ask sem κ ch l σ) (aeval_ask sem κ ch r σ) = (al, ar)"
     by (cases "refine_ik ?c (aeval_ask sem κ ch l σ) (aeval_ask sem κ ch r σ)")
   have "eval l t ∈ numeric_domain_class.gamma al" "eval r t ∈ numeric_domain_class.gamma ar"
     using refine_ik_sound[OF holds aeval_ask_sound[OF g h, where sem = sem]
       aeval_ask_sound[OF g h, where sem = sem]]
       ref
     by simp_all
   then show ?thesis using ref g h TCmp by (simp add: nonrel_narrow_def refine_var_sound)
 qed (use bare[OF that] in ‹simp_all add: nonrel_narrow_def›)
 have atom: "t ∈ ?γ (nonrel_atom e pol σ)"
   if "t ∈ ?γ σ" and "(eval e t ≠ 0) = pol" for e pol σ t
 proof -
   have n: "t ∈ ?γ (nonrel_narrow e pol σ)" by (rule narrow[OF that])
   then have "¬ is_empty (nonrel_narrow e pol σ v)" for v
     using gamma_tabsD(2)[of t κ "nonrel_narrow e pol σ" v] is_empty_correct by blast
   with n show ?thesis by (simp add: nonrel_atom_def Let_def)
 qed
 have feas: "nonrel_feas e pol σ" if "t ∈ ?γ σ" and "(eval e t ≠ 0) = pol" for e pol σ t
 proof -
   have g: "t ∈ gamma_tabs κ σ" and h: "eval_query.channel_holds ch t" using that(1) by auto
   have "eval e t ∈ numeric_domain_class.gamma (aeval_ask sem κ ch e σ)"
```

```
      by (rule aeval_ask_sound[OF g h])
    then show ?thesis
      using that(2) truth_ik_sound[of "aeval_ask sem κ ch e σ" "¬ pol" "eval e t"]
      by (auto simp: nonrel_feas_def truthy_def)
  qed
  have join1: "t ∈ ?γ (x ⊔ y)" if "t ∈ ?γ x" for x y :: "'a tabs" and t
    using that gamma_tabs_mono[OF sup_ge1[of x y]] by auto
  have join2: "t ∈ ?γ (x ⊔ y)" if "t ∈ ?γ y" for x y :: "'a tabs" and t
    using that gamma_tabs_mono[OF sup_ge2[of y x]] by auto
  have "t ∈ ?γ (bool_refine.brefine nonrel_view nonrel_atom nonrel_feas sup (λ_. ⊥) b pol σ)"
    apply (rule bool_refine.brefine_sound[OF nonrel_bool_locale, where ev = eval and γ = ?γ])
    subgoal by (rule atom)
    subgoal for e a t by (cases e rule: nonrel_view.cases) simp_all
    subgoal for e a c t by (cases e rule: nonrel_view.cases) simp_all
    subgoal for e a c t by (cases e rule: nonrel_view.cases) simp_all
    subgoal by (rule feas)
    subgoal by (rule join1)
    subgoal by (rule join2)
    subgoal using assms(1,2) by simp
    subgoal using assms(3) by (simp add: truthy_def)
    done
  then show ?thesis by (simp add: nonrel_refine_def)
qed

lemma gamma_tabs_restrict:
  "t ∈ gamma_tabs κ σ' ⟹ t ∈ gamma_tabs κ σ
    ⟹ t ∈ gamma_tabs κ (λv. if v ∈ S then σ' v else σ v)"
  by (simp add: gamma_tabs_def)

lemma nonrel_branch_sound:
  assumes "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t" and
    "truthy (eval b t) = pol"
  shows "t ∈ gamma_tabs κ (nonrel_branch b pol σ)"
proof -
  have refined: "t ∈ gamma_tabs κ (λv. if v ∈ exp_vars b then nonrel_refine b pol σ v else σ v)"
    by (rule gamma_tabs_restrict[OF nonrel_refine_sound[OF assms] assms(1)])
  show ?thesis
  proof (cases "truth_ik (aeval_ask sem κ ch b σ)")
    case (Some r)
    then have "truthy (eval b t) = r"
      by (rule truth_ik_sound[OF _ aeval_ask_sound[OF assms(1,2)]])
    with assms(3) have "r = pol" by simp
    with Some refined show ?thesis by (simp add: nonrel_branch_def Let_def)
  qed (use refined in ‹simp add: nonrel_branch_def Let_def›)
qed

lemma ret_kd_sound:
  assumes "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t"
  shows "eval e t ∈ numeric_domain_class.gamma (ret_kd e σ)"
proof (cases "∃ik v. e = TVar ik v")
  case True
  then obtain ik v where e: "e = TVar ik v" by blast
  show ?thesis
  proof (cases "v = ReturnVar")
    case False
    have "t v ∈ ik_range (κ v)"
      using gamma_tabsD(1)[OF assms(1)] False by (simp add: in_kinds_def)
    with False e read_var_sound[OF assms(1)] show ?thesis by (simp add: ret_kd_def)
  qed (simp add: ret_kd_def e)
next
  case False
  then have "ret_kd e σ = KD (exp_kind e) (aeval_ask sem κ ch e σ)"
    by (cases e) (auto simp: ret_kd_def)
  with aeval_ask_sound[OF assms] eval_in_range_kind[OF False] show ?thesis by simp
qed

lemma nonrel_return_sound:
  assumes "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t"
  shows "t(ReturnVar := (case r of None ⇒ t ReturnVar | Some e ⇒ eval e t))
           ∈ gamma_tabs κ (nonrel_return r σ)"
proof (cases r)
  case None with assms show ?thesis by (simp add: nonrel_return_def)
next
  case (Some e)
  with gamma_tabs_upd[OF assms(1) ret_kd_sound[OF assms]] show ?thesis
    by (simp add: nonrel_return_def)
```

```
qed
```

Binding a formal is an assignment in the callee's frame of a value the caller evaluates: each converts to the formal's kind.

```
lemma bind_formals_sound:
  assumes t: "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t"
    and s: "s ∈ gamma_tabs κ σ'"
  shows "bind_formals ps (map2 (λx. ik_norm (κ x)) ps (map (λe. eval e t) es)) s
          ∈ gamma_tabs κ (bind_formals ps (map2 (λp e. KD (κ p) (cast_ik (sem_int_signed_overflow sem)
              (src_kind κ e) (κ p) (aeval_ask sem κ ch e σ))) ps es) σ')"
  using s
proof (induction ps arbitrary: es s σ')
  case Nil then show ?case by simp
next
  case (Cons p ps)
  show ?case
  proof (cases es)
    case Nil with Cons.prems show ?thesis by simp
  next
    case (Cons e es')
    have "ik_norm (κ p) (eval e t) ∈ numeric_domain_class.gamma
        (cast_ik (sem_int_signed_overflow sem) (src_kind κ e) (κ p) (aeval_ask sem κ ch e σ))"
      by (rule cast_ik_src_sound[OF gamma_tabsD(1)[OF t] aeval_ask_sound[OF t ch]])
    then have "s(p := ik_norm (κ p) (eval e t)) ∈ gamma_tabs κ (σ'(p := KD (κ p)
        (cast_ik (sem_int_signed_overflow sem) (src_kind κ e) (κ p) (aeval_ask sem κ ch e σ))))"
      by (simp add: gamma_tabs_upd[OF Cons.prems])
    from Cons.IH[OF this, of es'] show ?thesis by (simp add: ‹es = e # es'›)
  qed
qed

lemma nonrel_enter_state_sound:
  assumes "t ∈ gamma_tabs κ σ" and ch: "eval_query.channel_holds ch t"
  shows "bind_formals ps (map2 (λx. ik_norm (κ x)) ps (map (λe. eval e t) es))
          (enter_state is_global t)
          ∈ gamma_tabs κ (nonrel_enter_state ps es σ)"
proof -
  have "enter_state is_global t ∈ gamma_tabs κ (nonrel_enter_frame σ)"
    using gamma_tabsD[OF assms(1)] in_kinds_enter_state
    by (intro gamma_tabsI)
      (auto simp: enter_state_def nonrel_enter_frame_def enter_frame_def combine_env_def)
  then show ?thesis unfolding nonrel_enter_state_def by (rule bind_formals_sound[OF assms])
qed

lemma nonrel_combine_sound:
  assumes "s ∈ gamma_tabs κ σ" and "t ∈ gamma_tabs κ σ_e"
  shows "combine_assign dst (kind_conv κ dst (t ReturnVar)) (combine_env is_global s t)
          ∈ gamma_tabs κ (nonrel_combine_assign dst (combine_env is_global σ σ_e) σ_e)"
proof -
  have tys: "in_kinds κ s" "in_kinds κ t"
    using gamma_tabsD(1)[OF assms(1)] gamma_tabsD(1)[OF assms(2)] by blast+
  have vs: "⋀v. s v ∈ numeric_domain_class.gamma (σ v)"
    "⋀v. t v ∈ numeric_domain_class.gamma (σ_e v)"
    using gamma_tabsD(2)[OF assms(1)] gamma_tabsD(2)[OF assms(2)] by blast+
  have cty: "in_kinds κ (combine_env is_global s t)"
    using tys by (auto simp: in_kinds_def combine_env_def)
  have cvs:
    "⋀v. combine_env is_global s t v ∈ numeric_domain_class.gamma (combine_env is_global σ σ_e v)"
    using vs by (simp add: combine_env_def)
  show ?thesis
  proof (cases dst)
    case None
    with cty cvs show ?thesis by (simp add: nonrel_combine_assign_def gamma_tabsI)
  next
    case (Some x)
    have rv: "ik_norm (κ x) (t ReturnVar) ∈ numeric_domain_class.gamma
        (cast_ik (sem_int_signed_overflow sem) (kd_kind (σ_e ReturnVar)) (κ x) (kd_val (σ_e ReturnVar)))"
      by (rule cast_ik_sound[OF kd_kind_range[OF vs(2)] kd_val_sound[OF vs(2)]])
    have "(combine_env is_global s t)(x := ik_norm (κ x) (t ReturnVar))
        ∈ gamma_tabs κ ((combine_env is_global σ σ_e)
            (x := KD (κ x) (cast_ik (sem_int_signed_overflow sem) (kd_kind (σ_e ReturnVar)) (κ x) (kd_val (σ_e ReturnVar)))))"
      using rv cty cvs by (intro gamma_tabsI in_kinds_upd) auto
    with Some show ?thesis by (simp add: nonrel_combine_assign_def)
  qed
qed

end
```

The typed component

The component answers `EvalInt` by evaluating without asking the expression itself back, as Goblint's `base` answers it with `eval_rv_no_ask_evalint`. Every transfer evaluates through the channel it receives. The entry has one alternative, the caller's state paired with the entered one.

```
fun nonrel_query :: "sem_conf ⇒ kenv ⇒ channel ⇒ 'a::kind_int_domain tabs ⇒ channel" where
  "nonrel_query sem κ ch σ (EvalInt e) = answer_ik (aeval sem κ ch e σ)"

definition nonrel_spec ::
    "sem_conf ⇒ kenv ⇒ 'a::kind_int_domain tabs local_spec" where
  "nonrel_spec sem κ =
     (| ls_query = nonrel_query sem κ,
       ls_skip = (λch d. d),
       ls_assign = (λch x e. nonrel_assign sem κ ch x e),
       ls_special = (λch sc x. nonrel_special sem κ ch sc x),
       ls_branch = (λch b pol. nonrel_branch sem κ ch b pol),
       ls_body = (λch p d. d),
       ls_return = (λch r p. nonrel_return sem κ ch r),
       ls_event = (λch evt d. d),
       ls_enter = (λch ci p.
         [(fst p, nonrel_enter_state sem κ ch (ci_formals ci) (ci_args ci) (fst p))]),
       ls_combine_env = (λch ch' ci. combine_env is_global),
       ls_combine_assign = (λch ci. nonrel_combine_assign sem κ (ci_dst ci)) |)"

theorem nonrel_spec_sound:
  "sound_local_spec κ (gamma_tabs κ)
     (nonrel_spec sem κ :: 'a::kind_int_domain tabs local_spec)"
proof -
  show ?thesis
  proof (rule sound_local_specI)
    show "⋀x y. x ≤ y ⟹ gamma_tabs κ x ⊆ gamma_tabs κ y"
      by (rule gamma_tabs_mono)
    show "sound_query (gamma_tabs κ) (ls_query (nonrel_spec sem κ))"
      unfolding sound_query_def
      apply (intro allI impI)
      subgoal for ch s x q
        by (cases q) (simp_all add: nonrel_spec_def answer_ik_sound aeval_sound)
      done
    show "sound_skip (gamma_tabs κ) (ls_skip (nonrel_spec sem κ))"
      by (simp add: nonrel_spec_def sound_skip_def)
    show "sound_assign κ (gamma_tabs κ) (ls_assign (nonrel_spec sem κ))"
      by (auto simp: nonrel_spec_def sound_assign_def intro!: nonrel_assign_sound)
    show "sound_special κ (gamma_tabs κ) (ls_special (nonrel_spec sem κ))"
      by (auto simp: nonrel_spec_def sound_special_def intro!: nonrel_special_sound)
    show "sound_branch (gamma_tabs κ) (ls_branch (nonrel_spec sem κ))"
      by (auto simp: nonrel_spec_def sound_branch_def intro!: nonrel_branch_sound)
    show "sound_body (gamma_tabs κ) (ls_body (nonrel_spec sem κ))"
      by (simp add: nonrel_spec_def sound_body_def)
    show "sound_return (gamma_tabs κ) (ls_return (nonrel_spec sem κ))"
      by (auto simp: nonrel_spec_def sound_return_def intro!: nonrel_return_sound)
    show "sound_event (gamma_tabs κ) (ls_event (nonrel_spec sem κ))"
      by (simp add: nonrel_spec_def sound_event_def)
    show "sound_enter κ (gamma_tabs κ) (ls_enter (nonrel_spec sem κ))"
      unfolding sound_enter_def
      by (auto simp: nonrel_spec_def call_enter_CallEdge intro:
        nonrel_enter_state_sound)
    show "sound_combine κ (gamma_tabs κ) (ls_combine_env (nonrel_spec sem κ))
        (ls_combine_assign (nonrel_spec sem κ))"
      unfolding sound_combine_def
      by (auto simp: nonrel_spec_def combine_collect_def intro:
        nonrel_combine_sound)
  qed
qed
```

Kind consistency

Concretization alone does not fix the tags: a cell tagged with another kind still describes stores in its variable's range. A Goblint-style state also keeps every tag at the variable's declared kind, so that an operation reading the cell runs at the right kind. The return slot has no declared kind and is exempt. The operations above, the join, the widening and the narrowing all keep the invariant.

```
definition kd_at :: "ikind ⇒ 'a kd ⇒ bool" where
  "kd_at k d ⟷ (∀l a. d = KD l a ⟶ l = k)"

lemma kd_at_simps [simp]:
```

```
  "kd_at k KBot" "kd_at k KTop" "kd_at k (KD l a) ⟷ l = k"
  by (auto simp: kd_at_def)

definition kind_consistent :: "kenv ⇒ 'a tabs ⇒ bool" where
  "kind_consistent κ σ ⟷ (∀v. v ≠ ReturnVar ⟶ kd_at (κ v) (σ v))"

lemma kind_consistent_bot [simp]: "kind_consistent κ ⊥"
  by (simp add: kind_consistent_def bot_fun_def bot_kd_def)
```

The guard refinement runs as the generic recursion at the typed instance.

```
global_interpretation nonrel_bool: bool_refine nonrel_view size "nonrel_atom sem κ ch"
  "nonrel_feas sem κ ch" sup "λ_. ⊥"
  for sem κ ch
  defines nonrel_refine_exec = nonrel_bool.brefine
  by (rule nonrel_bool_locale)

lemma nonrel_refine_code [code]: "nonrel_refine sem κ ch = nonrel_refine_exec sem κ ch"
  by (simp add: nonrel_refine_def fun_eq_iff nonrel_refine_exec_def)

end
```

10.3 Nonrelational_Exec

```
theory Nonrelational_Exec
  imports Nonrelational_Transfer "Voblint_Exec.Default_St_Transfer"
begin

unbundle default_st_syntax
```

The executable typed state

A solver compares and joins states, so it cannot run on the functions of `'a tabs`. The executable carrier is a pair of finite default dictionaries over variable identities, one for the locals and one for the globals. Each operation updates finitely many entries and agrees with its counterpart in `Nonrelational_Transfer` on the function the carrier represents, so the soundness of the transfer carries over unchanged.

```
lemma infinite_range_GlobalId: "infinite (range GlobalId)"
proof
  assume "finite (range GlobalId)"
  then have "finite (UNIV :: vname set)"
    by (rule finite_imageD) (simp add: inj_def)
  then show False using infinite_literal by simp
qed

lemma infinite_range_ScopedId: "infinite (range (ScopedId q))"
proof
  assume "finite (range (ScopedId q))"
  then have "finite (UNIV :: vname set)"
    by (rule finite_imageD) (simp add: inj_def)
  then show False using infinite_literal by simp
qed

instance var_id :: infinite_key
  by standard (rule infinite_super[OF subset_UNIV infinite_range_GlobalId])

type_synonym 'a tst = "(var_id, 'a kd) gdefault_st"

abbreviation nonrel_rd :: "'a::kind_int_domain tst ⇒ 'a tabs" where
  "nonrel_rd ≡ ϱis_global"

abbreviation nonrel_loc :: "var_id ⇒ var_id glocation" where
  "nonrel_loc ≡ location_of is_global"
```

Writing back a refined state

A guard changes only the variables it reads. Its executable form computes the refined function once and stores it at those variables.

```
fun var_list :: "exp ⇒ var_id list" where
  "var_list (TN _ _) = []"
| "var_list (TVar _ v) = [v]"
| "var_list (TArith _ _ a b) = var_list a @ var_list b"
| "var_list (TCast _ a) = var_list a"
| "var_list (TCmp _ a b) = var_list a @ var_list b"
| "var_list (TNot a) = var_list a"
| "var_list (TAnd a b) = var_list a @ var_list b"
| "var_list (TOr a b) = var_list a @ var_list b"

lemma set_var_list [simp]: "set (var_list e) = exp_vars e"
  by (induction e) auto

definition write_back :: "var_id list ⇒ 'a::kind_int_domain tabs ⇒ 'a tst ⇒ 'a tst" where
  "write_back vs σ st = fold (λv st. st⟨nonrel_loc v := σ v⟩) vs st"

lemma nonrel_rd_write_back:
  "nonrel_rd (write_back vs σ st) = (λv. if v ∈ set vs then σ v else nonrel_rd st v)"
  by (induction vs arbitrary: st) (auto simp: write_back_def fun_eq_iff)

context
  fixes sem :: sem_conf and κ :: kenv and ch :: "channel"
begin
```

The operations on the carrier

Each writes the entries its counterpart changes and reads the represented function.

```
definition assign_st :: "var_id ⇒ exp ⇒ 'a::kind_int_domain tst ⇒ 'a tst" where
  "assign_st x e st = st⟨nonrel_loc x := KD (κ x) (cast_ik (sem_int_signed_overflow sem) (src_kind κ e) (κ x) (aeval_ask
sem κ ch e (nonrel_rd st)))⟩"

definition special_st :: "exp gspecial_call ⇒ var_id ⇒ 'a::kind_int_domain tst ⇒ 'a tst" where
  "special_st sc x st = st⟨nonrel_loc x := KD (κ x) (special_val sem κ ch sc x (nonrel_rd st))⟩"

definition branch_st :: "exp ⇒ bool ⇒ 'a::kind_int_domain tst ⇒ 'a tst" where
  "branch_st b pol st =
     (let refined = write_back (var_list b) (nonrel_refine sem κ ch b pol (nonrel_rd st)) st
      in case truth_ik (aeval_ask sem κ ch b (nonrel_rd st)) of
           Some r ⇒ if r = pol then refined else ⊥
         | None ⇒ refined)"

definition return_st :: "exp option ⇒ 'a::kind_int_domain tst ⇒ 'a tst" where
  "return_st r st = (case r of None ⇒ st | Some e ⇒ st⟨nonrel_loc ReturnVar := ret_kd sem κ ch e (nonrel_rd st)⟩)"

definition enter_st :: "var_id list ⇒ exp list ⇒ 'a::kind_int_domain tst ⇒ 'a tst" where
  "enter_st ps es st =
     bind_formals_default_st is_global ps
       (map2 (λp e. KD (κ p) (cast_ik (sem_int_signed_overflow sem) (src_kind κ e) (κ p) (aeval_ask sem κ ch e (nonrel_rd
st)))) ps es)
       (enter_frame_D_default_st KTop st)"

definition combine_assign_st :: "var_id option ⇒ 'a::kind_int_domain tst ⇒ 'a tst ⇒ 'a tst" where
  "combine_assign_st dst st ste =
     (case dst of None ⇒ st
      | Some x ⇒ st⟨nonrel_loc x := KD (κ x)
          (cast_ik (sem_int_signed_overflow sem) (kd_kind (nonrel_rd ste ReturnVar)) (κ x) (kd_val (nonrel_rd ste ReturnVar)))⟩)"
```

Each agrees with its counterpart on the represented function.

```
lemma nonrel_rd_assign_st [simp]:
  "nonrel_rd (assign_st x e st) = nonrel_assign sem κ ch x e (nonrel_rd st)"
  by (simp add: assign_st_def nonrel_assign_def)

lemma nonrel_rd_special_st [simp]:
  "nonrel_rd (special_st sc x st) = nonrel_special sem κ ch sc x (nonrel_rd st)"
  by (simp add: special_st_def nonrel_special_def)

lemma nonrel_rd_branch_st [simp]:
  "nonrel_rd (branch_st b pol st) = nonrel_branch sem κ ch b pol (nonrel_rd st)"
  by (simp add: branch_st_def nonrel_branch_def nonrel_rd_write_back Let_def split: option.splits)

lemma nonrel_rd_return_st [simp]:
  "nonrel_rd (return_st r st) = nonrel_return sem κ ch r (nonrel_rd st)"
  by (simp add: return_st_def nonrel_return_def split: option.splits)

lemma nonrel_rd_enter_st [simp]:
  "nonrel_rd (enter_st ps es st) = nonrel_enter_state sem κ ch ps es (nonrel_rd st)"
  by (simp add: enter_st_def nonrel_enter_state_def nonrel_enter_frame_def)

lemma nonrel_rd_combine_assign_st [simp]:
  "nonrel_rd (combine_assign_st dst st ste) = nonrel_combine_assign sem κ dst (nonrel_rd st) (nonrel_rd ste)"
  by (simp add: combine_assign_st_def nonrel_combine_assign_def split: option.splits)

end
```

Unreachable states

A state denotes nothing when one of its variables does. The test reads the two defaults, each of which some unlisted identity of its partition answers, and the entries listed in the dictionary their identity's partition selects.

```
fun st_rep_is_bot :: "(var_id, 'a::kind_int_domain kd) gdefault_st_rep ⇒ bool" where
  "st_rep_is_bot ((dl, ls), (dg, gs)) ⟷
     is_empty dl ∨ is_empty dg
     ∨ (∃x ∈ set (map fst ls). ¬ is_global x ∧ is_empty (default_dict_get (dl, ls) x))
     ∨ (∃x ∈ set (map fst gs). is_global x ∧ is_empty (default_dict_get (dg, gs) x))"

lemma st_rep_is_bot_iff:
  "st_rep_is_bot s ⟷ (∃v. is_empty (default_st_rep_to_fun is_global s v))"
proof -
  obtain dl ls dg gs where s: "s = ((dl, ls), (dg, gs))" by (cases s) auto
  have get: "default_st_rep_to_fun is_global s v
      = (if is_global v then default_dict_get (dg, gs) v else default_dict_get (dl, ls) v)" for v
    by (simp add: s default_st_rep_to_fun_def location_of_def)
  show ?thesis
```

```
  proof
    assume "st_rep_is_bot s"
    then consider (dl) "is_empty dl" | (dg) "is_empty dg"
      | (l) x where "¬ is_global x" "is_empty (default_dict_get (dl, ls) x)"
      | (g) x where "is_global x" "is_empty (default_dict_get (dg, gs) x)"
      by (auto simp: s)
    then show "∃v. is_empty (default_st_rep_to_fun is_global s v)"
    proof cases
      case dl
      obtain v where v: "v ∈ range (ScopedId undefined)" "v ∉ set (map fst ls)"
        by (rule obtain_fresh_vname[OF infinite_range_ScopedId])
      from v(2) have "map_of ls v = None" by (rule map_of_resolved_none_iff[THEN iffD2])
      with dl v(1) show ?thesis
        by (intro exI[of _ v]) (auto simp: get)
    next
      case dg
      obtain v where v: "v ∈ range GlobalId" "v ∉ set (map fst gs)"
        by (rule obtain_fresh_vname[OF infinite_range_GlobalId])
      from v(2) have "map_of gs v = None" by (rule map_of_resolved_none_iff[THEN iffD2])
      with dg v(1) show ?thesis
        by (intro exI[of _ v]) (auto simp: get)
    qed (auto simp: get)
  next
    assume "∃v. is_empty (default_st_rep_to_fun is_global s v)"
    then obtain v where v: "is_empty (default_st_rep_to_fun is_global s v)" ..
    then have v': "is_empty (if is_global v then default_dict_get (dg, gs) v
                             else default_dict_get (dl, ls) v)"
      by (simp only: get)
    show "st_rep_is_bot s"
    proof (cases "is_global v")
      case True
      show ?thesis
      proof (cases "map_of gs v")
        case None with True v' show ?thesis by (simp add: s)
      next
        case (Some a)
        then have "(v, a) ∈ set gs" by (rule map_of_SomeD)
        with Some True v' show ?thesis by (force simp: s)
      qed
    next
      case False
      show ?thesis
      proof (cases "map_of ls v")
        case None with False v' show ?thesis by (simp add: s)
      next
        case (Some a)
        then have "(v, a) ∈ set ls" by (rule map_of_SomeD)
        with Some False v' show ?thesis by (force simp: s)
      qed
    qed
  qed
qed

lift_definition st_is_bot :: "'a::kind_int_domain tst ⇒ bool" is st_rep_is_bot
  by (simp add: st_rep_is_bot_iff default_st_rep_to_fun_def eq_default_st_rep_def)

lemma st_is_bot_iff: "st_is_bot st ⟷ (∃v. is_empty (nonrel_rd st v))"
  by (simp add: st_is_bot.rep_eq st_rep_is_bot_iff default_st_to_fun_rep)

lemma nonrel_rd_combine [simp]:
  "nonrel_rd (combine_default_st d e) = combine_env is_global (nonrel_rd d) (nonrel_rd (e :: 'a::kind_int_domain tst))"
  by simp

definition t_empty :: "'a::kind_int_domain tabs ⇒ bool" where
  "t_empty σ ⟷ (∃v. is_empty (σ v))"

lemma t_empty_sound: "t_empty σ ⟹ gamma_tabs κ σ = {}"
  by (auto simp: t_empty_def gamma_tabs_def is_empty_correct simp del: gamma_kd_def)
```

The executable component

The component of `Nonrelational_Transfer` on the carrier, over the unreachable state: a step whose result denotes nothing answers `Bot`.

```
definition nonrel_exec_spec ::
    "sem_conf ⇒ kenv ⇒ 'a::kind_int_domain tst lifted local_spec" where
```

```
  "nonrel_exec_spec sem κ =
     (| ls_query = (λch d q. case d of Bot ⇒ ⊤ | Lifted x ⇒ nonrel_query sem κ ch (nonrel_rd x) q),
       ls_skip = (λch d. d),
       ls_assign = (λch x e. transfer_lift st_is_bot (assign_st sem κ ch x e)),
       ls_special = (λch sc x. transfer_lift st_is_bot (special_st sem κ ch sc x)),
       ls_branch = (λch b pol. transfer_lift st_is_bot (branch_st sem κ ch b pol)),
       ls_body = (λch p d. d),
       ls_return = (λch r p. transfer_lift st_is_bot (return_st sem κ ch r)),
       ls_event = (λch evt d. d),
       ls_enter = (λch ci p.
         [(fst p, transfer_lift st_is_bot (enter_st sem κ ch (ci_formals ci) (ci_args ci)) (fst p))]),
       ls_combine_env = (λch ch' ci x y. do { a ← x; b ← y; Lifted (combine_default_st a b) }),
       ls_combine_assign = (λch ci. transfer_lift2 st_is_bot (combine_assign_st sem κ (ci_dst ci))) |)"

definition gamma_exec :: "kenv ⇒ 'a::kind_int_domain tst lifted ⇒ store set" where
  "gamma_exec κ d = gamma_lift (gamma_tabs κ) (map_lift nonrel_rd d)"

lemma gamma_exec_Bot [simp]: "gamma_exec κ Bot = {}"
  by (simp add: gamma_exec_def)

lemma gamma_exec_Lifted [simp]: "gamma_exec κ (Lifted x) = gamma_tabs κ (nonrel_rd x)"
  by (simp add: gamma_exec_def)

lemma st_is_bot_sound: "st_is_bot st ⟹ gamma_tabs κ (nonrel_rd st) = {}"
  by (rule t_empty_sound) (simp add: t_empty_def st_is_bot_iff)

lemma gamma_exec_normalize_lift [simp]:
  "s ∈ gamma_exec κ (normalize_lift st_is_bot y) ⟷ s ∈ gamma_tabs κ (nonrel_rd y)"
  using st_is_bot_sound[of y κ] by (auto simp: normalize_lift_def)

lemma gamma_exec_transfer_lift [simp]:
  "s ∈ gamma_exec κ (transfer_lift st_is_bot f (Lifted x)) ⟷ s ∈ gamma_tabs κ (nonrel_rd (f x))"
  using st_is_bot_sound[of "f x" κ] by (auto simp: transfer_lift_def normalize_lift_def)

lemma gamma_exec_transfer_lift2 [simp]:
  "s ∈ gamma_exec κ (transfer_lift2 st_is_bot f (Lifted x) (Lifted y))
     ⟷ s ∈ gamma_tabs κ (nonrel_rd (f x y))"
  using st_is_bot_sound[of "f x y" κ] by (auto simp: transfer_lift2_def normalize_lift_def)

lemma gamma_exec_mono: "x ≤ y ⟹ gamma_exec κ x ⊆ gamma_exec κ y"
proof (cases x; cases y)
  fix a b assume "x ≤ y" "x = Lifted a" "y = Lifted b"
  then have "nonrel_rd a ≤ nonrel_rd b" by (simp add: default_st_to_fun_mono)
  with ‹x = Lifted a› ‹y = Lifted b› show ?thesis by (simp add: gamma_tabs_mono)
qed simp_all

theorem nonrel_exec_spec_sound:
  "sound_local_spec κ (gamma_exec κ)
     (nonrel_exec_spec sem κ :: 'a::kind_int_domain tst lifted local_spec)"
proof -
  note S = nonrel_spec_sound[of κ]
  show ?thesis
  proof (rule sound_local_specI)
    show "⋀x y. x ≤ y ⟹ gamma_exec κ x ⊆ gamma_exec κ y"
      by (rule gamma_exec_mono)
    show "sound_query (gamma_exec κ) (ls_query (nonrel_exec_spec sem κ))"
      unfolding sound_query_def
      apply (intro allI impI)
      subgoal premises prems for ch s d q
      proof (cases d)
        case (Lifted x)
        with prems(1) have "s ∈ gamma_tabs κ (nonrel_rd x)" by simp
        from sound_local_spec_queryD[OF S, unfolded sound_query_def, rule_format,
            OF this prems(2)]
        show ?thesis by (simp add: nonrel_exec_spec_def nonrel_spec_def Lifted)
      qed (simp add: nonrel_exec_spec_def)
      done
    show "sound_skip (gamma_exec κ) (ls_skip (nonrel_exec_spec sem κ))"
      by (simp add: nonrel_exec_spec_def sound_skip_def)
    show "sound_body (gamma_exec κ) (ls_body (nonrel_exec_spec sem κ))"
      by (simp add: nonrel_exec_spec_def sound_body_def)
    show "sound_event (gamma_exec κ) (ls_event (nonrel_exec_spec sem κ))"
      by (simp add: nonrel_exec_spec_def sound_event_def)
    show "sound_assign κ (gamma_exec κ) (ls_assign (nonrel_exec_spec sem κ))"
      unfolding sound_assign_def
      apply (intro allI impI)
```

```
    subgoal premises prems for ch d s y e
    proof (cases d)
      case (Lifted x)
      with prems(1) have "s ∈ gamma_tabs κ (nonrel_rd x)" by simp
      from sound_local_spec_assignD[OF S, unfolded sound_assign_def, rule_format,
          OF this prems(2)]
      show ?thesis by (simp add: nonrel_exec_spec_def nonrel_spec_def Lifted)
    qed (use prems in simp)
    done
  show "sound_special κ (gamma_exec κ) (ls_special (nonrel_exec_spec sem κ))"
    unfolding sound_special_def
    apply (intro allI impI)
    subgoal premises prems for ch d s sc y t
    proof (cases d)
      case (Lifted x)
      with prems(1) have "s ∈ gamma_tabs κ (nonrel_rd x)" by simp
      from sound_local_spec_specialD[OF S, unfolded sound_special_def, rule_format,
          OF this prems(2) prems(3)]
      show ?thesis by (simp add: nonrel_exec_spec_def nonrel_spec_def Lifted)
    qed (use prems in simp)
    done
  show "sound_branch (gamma_exec κ) (ls_branch (nonrel_exec_spec sem κ))"
    unfolding sound_branch_def
    apply (intro allI impI)
    subgoal premises prems for ch d s b pol
    proof (cases d)
      case (Lifted x)
      with prems(1) have "s ∈ gamma_tabs κ (nonrel_rd x)" by simp
      from sound_local_spec_branchD[OF S, unfolded sound_branch_def, rule_format,
          OF this prems(2) prems(3)]
      show ?thesis by (simp add: nonrel_exec_spec_def nonrel_spec_def Lifted)
    qed (use prems in simp)
    done
  show "sound_return (gamma_exec κ) (ls_return (nonrel_exec_spec sem κ))"
    unfolding sound_return_def
    apply (intro allI impI)
    subgoal premises prems for ch d s r p
    proof (cases d)
      case (Lifted x)
      with prems(1) have "s ∈ gamma_tabs κ (nonrel_rd x)" by simp
      from sound_local_spec_returnD[OF S, unfolded sound_return_def, rule_format,
          OF this prems(2)]
      show ?thesis by (simp add: nonrel_exec_spec_def nonrel_spec_def Lifted)
    qed (use prems in simp)
    done
  show "sound_enter κ (gamma_exec κ) (ls_enter (nonrel_exec_spec sem κ))"
    unfolding sound_enter_def
    apply (intro allI impI)
    subgoal premises prems for ch s ci p
    proof -
      from prems(1) obtain x where p: "fst p = Lifted x"
        and sx: "s ∈ gamma_tabs κ (nonrel_rd x)"
        by (cases "fst p") simp_all
      from sound_local_spec_enterD[OF S, unfolded sound_enter_def, rule_format,
          where p = "(nonrel_rd x, nonrel_rd x)" and ci = ci, OF _ prems(2)] sx
      show ?thesis by (simp add: nonrel_exec_spec_def nonrel_spec_def p)
    qed
    done
  show "sound_combine κ (gamma_exec κ) (ls_combine_env (nonrel_exec_spec sem κ))
      (ls_combine_assign (nonrel_exec_spec sem κ))"
    unfolding sound_combine_def
    apply (intro allI impI)
    subgoal premises prems for ch ch' s t ci x de
    proof -
      from prems(1) obtain a where x: "x = Lifted a"
        and sa: "s ∈ gamma_tabs κ (nonrel_rd a)"
        by (cases x) simp_all
      from prems(3) obtain b where de: "de = Lifted b"
        and tb: "t ∈ gamma_tabs κ (nonrel_rd b)"
        by (cases de) simp_all
      from sound_local_spec_combineD[OF S, unfolded sound_combine_def, rule_format,
          where ci = ci, OF sa prems(2) tb prems(4)]
      show ?thesis by (simp add: nonrel_exec_spec_def nonrel_spec_def x de)
    qed
    done
qed
```

qed

unbundle `no default_st_syntax`

end

10.4 Nonrelational_Analysis

```
theory Nonrelational_Analysis
  imports
    Nonrelational_Exec
    "Voblint_Result.DG_Live_Unknowns"
    "Voblint_Solver.Globals_Rule"
    "Voblint_Framework.Call_String_Context"
begin

unbundle default_st_syntax
```

The typed non-relational analysis

The executable typed component at any kind-aware integer domain, run by the shared D/G pipeline over the typed CFG. The kind declarations `ds` are a parameter of the instance: they fix the kind environment the concretization, the transfer and the initial state read, and the program varies under them. The domain is a type parameter, so every constant of the instance takes the domain's `TYPE` first.

Verdicts

```
definition nonrel_classify :: "sem_conf ⇒ kenv ⇒ exp ⇒ 'a::kind_int_domain tabs ⇒ check_result"
  where
  "nonrel_classify sem κ c d =
     (case truth_ik (aeval sem κ (λ_. ⊤) c d) of
        Some True ⇒ Check_Proved
      | Some False ⇒ Check_Refuted
      | None ⇒ Check_Unknown)"

lemma nonrel_classify_sound:
  assumes s: "s ∈ gamma_tabs κ d"
  shows nonrel_classify_proved: "nonrel_classify sem κ c d = Check_Proved ⟹ truthy (eval c s)"
    and nonrel_classify_refuted: "nonrel_classify sem κ c d = Check_Refuted ⟹ ¬ truthy (eval c s)"
proof -
  have truth: "truthy (eval c s) = b"
    if "truth_ik (aeval sem κ (λ_. ⊤) c d) = Some b" for b
    using s by (rule truth_ik_sound[OF that aeval_sound]) simp
  show "nonrel_classify sem κ c d = Check_Proved ⟹ truthy (eval c s)"
    using truth by (auto simp: nonrel_classify_def split: option.splits bool.splits)
  show "nonrel_classify sem κ c d = Check_Refuted ⟹ ¬ truthy (eval c s)"
    using truth by (auto simp: nonrel_classify_def split: option.splits bool.splits)
qed
```

The initial state

Every global starts at zero at its declared kind; every local is unconstrained.

```
definition nonrel_init_st :: "kind_decls ⇒ 'a::kind_int_domain tst" where
  "nonrel_init_st ds =
     ⦇(KTop, []), (KD I32 (of_int_ik I32 0), map (λ(v, k). (v, KD k (of_int_ik k 0))) ds)⦈"

lemma nonrel_rd_init_st:
  "nonrel_rd (nonrel_init_st ds :: 'a::kind_int_domain tst)
     = (λv. if is_global v then KD (kenv_of ds v) (of_int_ik (kenv_of ds v) 0) else KTop)"
  by (auto simp: nonrel_init_st_def default_st_to_fun_def location_of_def kenv_of_def map_of_map
      fun_eq_iff split: option.splits)

lemma nonrel_init_sound:
  "init_stores (kenv_of ds)
     ⊆ gamma_tabs (kenv_of ds) (nonrel_rd (nonrel_init_st ds :: 'a::kind_int_domain tst))"
proof
  fix t assume t: "t ∈ init_stores (kenv_of ds)"
  have rng: "t v ∈ ik_range (kenv_of ds v)" for v
    using t by (simp add: init_stores_def all_in_kindsD del: ik_range_iff)
  show "t ∈ gamma_tabs (kenv_of ds) (nonrel_rd (nonrel_init_st ds))"
    unfolding nonrel_rd_init_st
  proof (rule gamma_tabsI[OF init_stores_in_kinds[OF t]], goal_cases)
    case (1 v)
    show ?case
    proof (cases v)
      case (GlobalId x)
      then have "t v = 0" using t by (simp add: init_stores_def)
      then show ?thesis
        using of_int_ik_sound[of "kenv_of ds v" "t v"] rng[of v] GlobalId
```

```
      by (simp del: ik_range_iff)
    qed simp_all
  qed
qed
```

The pipeline instance

Call-string contexts of bound `k`, after Goblint's `callstring.ml`: a call pushes its call site and keeps the `k` most recent. At `k = 0` every activation shares the empty string, the context-insensitive analysis. The joined global unknown and the certified solver run at any update rule `r`. The component reads no global, so every variable lives in a point's own state.

```
lemma elab_cfg_calls_source_unique: "calls_source_unique (elab_cfg p ds)"
  unfolding calls_source_unique_def elab_cfg_def using compile_prog_calls_source_unique by blast

lemma nonrel_exec_spec_sound_rd:
  "sound_local_spec κ
     (λd. gamma_lift (gamma_tabs κ) (map_lift nonrel_rd d))
     (nonrel_exec_spec sem κ :: 'a::kind_int_domain tst lifted local_spec)"
  using nonrel_exec_spec_sound[of κ sem, where 'a = 'a] by (simp add: gamma_exec_def[abs_def])
```

What the component owes the pipeline does not depend on the context policy, so it is proved once, for any route and root context.

```
lemma nonrel_routed:
  "dg_analysis
    (kenv_of ds) (λ_. init_stores (kenv_of ds))
    (λ_ _. nonrel_exec_spec sem (kenv_of ds) :: 'a::kind_int_domain tst lifted local_spec)
    (λ_. st_is_bot) (λ𝒢. ϱ𝒢) (nonrel_init_st ds)
    (Analysis_Global ()) Analysis_Global Activation_Seed
    (TD_side_rule_Interp_solve r)
    (TD_side_rule_Interp.solve_dom TYPE((unit, 'c) global_unknown)
       TYPE(('a tst lifted, 'a tst lifted) dg_state) r)
    ⊥ (nonrel_classify sem (kenv_of ds)) (gamma_tabs (kenv_of ds)) t_empty
    (TD_side_rule_Interp_solve_c r)
    (λ_ c. dg_spec_of c) (λp d e. d) (λ_ d. d) (λ_ d. Bot) (λ_ p. []) (λp ci d e. d)
    (λp. elab_cfg p ds) (λ_. FunctionEntry prog_main_name)"
proof (rule dg_analysis_whole_stateI[OF td_certified_solver], goal_cases CompSound EnterSingle
    EmptyRd EmptyV SeedNe ClProved ClRefuted BotState InitSound SeedNeGlobal PEntry FinIntra
    FinCalls CallsUnique)
  case (CompSound p) show ?case by (rule nonrel_exec_spec_sound_rd)
next
  case (EnterSingle p ci d) show ?case
    by (simp add: nonrel_exec_spec_def dg_pipeline.comp_entry_def)
next
  case (EmptyRd p s) show ?case by (simp add: st_is_bot_iff t_empty_def)
next
  case EmptyV show ?case by (rule sound_emptinessI) (rule t_empty_sound)
next
  case (SeedNe v ctx) show ?case by simp
next
  case (ClProved c d s) then show ?case by (intro nonrel_classify_proved)
next
  case (ClRefuted c d s) then show ?case by (intro nonrel_classify_refuted)
next
  case BotState show ?case by simp
next
  case (InitSound p) show ?case by (rule nonrel_init_sound)
next
  case (SeedNeGlobal v ctx n) show ?case by simp
next
  case (PEntry p) show ?case by (simp add: elab_cfg_def)
next
  case (FinIntra p) show ?case by (simp add: elab_cfg_def compile_prog_finite)
next
  case (FinCalls p) show ?case by (simp add: elab_cfg_def compile_prog_finite)
next
  case (CallsUnique p) show ?case by (rule elab_cfg_calls_source_unique)
qed

global_interpretation nonrel: dg_analysis
    "kenv_of ds" "λ_. init_stores (kenv_of ds)"
    "λ_ _. nonrel_exec_spec sem (kenv_of ds) :: 'a::kind_int_domain tst lifted local_spec"
    "λ_. st_is_bot" "λ𝒢. ϱ𝒢" "nonrel_init_st ds"
    "Analysis_Global ()" Analysis_Global Activation_Seed "λ_. cs_route k" "[]"
    "TD_side_rule_Interp_solve r"
```

```
    "TD_side_rule_Interp.solve_dom TYPE((unit, cfg_node list) global_unknown)
       TYPE((’a tst lifted, ’a tst lifted) dg_state) r"
    ⊥ "nonrel_classify sem (kenv_of ds)" "gamma_tabs (kenv_of ds)" t_empty
    "TD_side_rule_Interp_solve_c r"
    "λ_ c. dg_spec_of c" "λp d e. d" "λ_ d. d" "λ_ d. Bot" "λ_ p. []" "λp ci d e. d"
    "λp. elab_cfg p ds" "λ_. FunctionEntry prog_main_name"
  for sem ds r k
  by (rule nonrel_routed)
```

Entry-state contexts

A call's context is the abstract values of the callee's formals in the state the call enters. Goblint's default context is the whole entry state (`DefaultSpec.context`); the formals are the part of it a call determines, since the callee's other locals start unconstrained.

```
definition nonrel_es_ctx ::
    "(var_id ⇒ bool) ⇒ ’a::kind_int_domain tst lifted ⇒ (var_id, exp) gcall_action
       ⇒ ’a kd list" where
  "nonrel_es_ctx 𝒢 d ca =
     (case ca of CallEdge dst pars args ⇒
        (case d of Lifted st ⇒ map (ϱ𝒢 st) pars
         | Bot ⇒ map (λ_. ⊥) pars))"

global_interpretation nonrel_es: dg_analysis
    "kenv_of ds" "λ_. init_stores (kenv_of ds)"
    "λ_ _. nonrel_exec_spec sem (kenv_of ds) :: ’a::kind_int_domain tst lifted local_spec"
    "λ_. st_is_bot" "λ𝒢. ϱ𝒢" "nonrel_init_st ds"
    "Analysis_Global ()" Analysis_Global Activation_Seed "λ𝒢 u ctx d ca. nonrel_es_ctx 𝒢 d ca" "[]"
    "TD_side_rule_Interp_solve r"
    "TD_side_rule_Interp.solve_dom TYPE((unit, ’a kd list) global_unknown)
       TYPE((’a tst lifted, ’a tst lifted) dg_state) r"
    ⊥ "nonrel_classify sem (kenv_of ds)" "gamma_tabs (kenv_of ds)" t_empty
    "TD_side_rule_Interp_solve_c r"
    "λ_ c. dg_spec_of c" "λp d e. d" "λ_ d. d" "λ_ d. Bot" "λ_ p. []" "λp ci d e. d"
    "λp. elab_cfg p ds" "λ_. FunctionEntry prog_main_name"
  for sem ds r
  by (rule nonrel_routed)

unbundle no default_st_syntax

end
```

Chapter 11

Voblint__Analysis__Sign

11.1 Sign_Warrowing

```
theory Sign_Warrowing
  imports "Voblint_Domain.Sign_Lattice"
begin
```

Sign warrowing and domain instance

Sign is finite, so widening is join and narrowing keeps the left argument.

```
definition narrow_sign_td :: "sign ⇒ sign ⇒ sign" where
  "narrow_sign_td a b = a"

instantiation sign :: warrowing begin
  definition "((a :: sign) ▽ b) = join_sign a b"
  definition "((a :: sign) △ b) = narrow_sign_td a b"
instance proof intro_classes
  fix a b :: sign
  show "a ≤ (a ▽ b)"
    unfolding less_eq_sign_def widen_sign_def by (rule join_sign_ub1)
  show "b ≤ (a ▽ b)"
    unfolding less_eq_sign_def widen_sign_def by (rule join_sign_ub2)
  show "b ≤ a ⟹ b ≤ (a △ b)"
    unfolding narrow_sign_def narrow_sign_td_def by simp
  show "b ≤ a ⟹ (a △ b) ≤ a"
    unfolding narrow_sign_def narrow_sign_td_def by simp
qed
end

instantiation sign :: numeric_domain begin
definition gamma_abs_sign [simp]: "numeric_domain_class.gamma (a :: sign) = gamma_sign a"
definition is_empty_sign [simp]: "is_empty (a :: sign) = is_bottom_sign a"
definition to_string_sign [simp]: "to_string (a :: sign) = string_of_sign a"
instance proof intro_classes
  show "numeric_domain_class.gamma (bot :: sign) = {}"
    unfolding bot_sign_def by simp
next
  show "numeric_domain_class.gamma (top :: sign) = UNIV"
    by (simp add: gamma_sign_top)
next
  fix a b :: sign
  assume H: "a ≤ b"
  show "numeric_domain_class.gamma a ⊆ numeric_domain_class.gamma b"
  proof -
    have "gamma_sign a ⊆ gamma_sign b"
      using H unfolding less_eq_sign_def by (rule gamma_sign_mono)
    then show ?thesis by simp
  qed
next
  fix a :: sign
  show "is_empty a ⟷ numeric_domain_class.gamma a = {}"
    by (simp add: is_bottom_sign_correct)
qed
end

end
```

11.2 Sign_Arithmetic

```
theory Sign_Arithmetic
  imports Sign_Warrowing
    "Voblint_Domain.Forward_Domain"
    "Voblint_Domain.Numeric_Queries" "Voblint_Domain.Nonrelational_State"
begin
```

What arithmetic does to a known sign

Every operation here works on the seven-element lattice a variable's sign can take, from `SBot` (no value at all) through `SNeg`, `SNonPos`, `SZero`, `SNonNeg` and `SPos` to `STop` (any value). Addition and multiplication are given as complete tables over those seven cases; a combination the lattice cannot pin down — a negative plus a positive, say — lands on `STop`.

On top of the tables sit three queries that answer about the numbers a sign denotes rather than about the sign itself: `sign_lt`, `sign_eqb` and `sign_tobool` return `Some b` when every pair of concrete values agrees on `b` and `None` when they do not. Each operation and query is proved sound and monotone.

```
instantiation sign :: plus begin
fun plus_sign :: "sign => sign => sign" where
    "plus_sign SBot    _       = SBot"
  | "plus_sign _       SBot    = SBot"
  | "plus_sign SNeg    SNeg    = SNeg"
  | "plus_sign SNeg    SNonPos = SNeg"
  | "plus_sign SNonPos SNeg    = SNeg"
  | "plus_sign SNonPos SNonPos = SNonPos"
  | "plus_sign SPos    SPos    = SPos"
  | "plus_sign SPos    SNonNeg = SPos"
  | "plus_sign SNonNeg SPos    = SPos"
  | "plus_sign SNonNeg SNonNeg = SNonNeg"
  | "plus_sign SZero   b       = b"
  | "plus_sign a       SZero   = a"
  | "plus_sign _       _       = STop"
instance ..
end

instantiation sign :: minus begin
fun minus_sign :: "sign => sign => sign" where
    "minus_sign SBot    _       = SBot"
  | "minus_sign _       SBot    = SBot"
  | "minus_sign SNeg    SPos    = SNeg"
  | "minus_sign SNeg    SNonNeg = SNeg"
  | "minus_sign SPos    SNeg    = SPos"
  | "minus_sign SPos    SNonPos = SPos"
  | "minus_sign SNeg    SZero   = SNeg"
  | "minus_sign SPos    SZero   = SPos"
  | "minus_sign SZero   SZero   = SZero"
  | "minus_sign SZero   SNeg    = SPos"
  | "minus_sign SZero   SPos    = SNeg"
  | "minus_sign SZero   SNonNeg = SNonPos"
  | "minus_sign SZero   SNonPos = SNonNeg"
  | "minus_sign SNonNeg SZero   = SNonNeg"
  | "minus_sign SNonNeg SNeg    = SPos"
  | "minus_sign SNonNeg SNonPos = SNonNeg"
  | "minus_sign SNonPos SZero   = SNonPos"
  | "minus_sign SNonPos SPos    = SNeg"
  | "minus_sign SNonPos SNonNeg = SNonPos"
  | "minus_sign _       _       = STop"
instance ..
end

instantiation sign :: times begin
fun times_sign :: "sign => sign => sign" where
    "times_sign SBot    _       = SBot"
  | "times_sign _       SBot    = SBot"
  | "times_sign SZero   _       = SZero"
  | "times_sign _       SZero   = SZero"
  | "times_sign SNeg    SNeg    = SPos"
  | "times_sign SPos    SPos    = SPos"
  | "times_sign SNeg    SPos    = SNeg"
  | "times_sign SPos    SNeg    = SNeg"
  | "times_sign SNeg    SNonPos = SNonNeg"
  | "times_sign SNonPos SNeg    = SNonNeg"
  | "times_sign SNeg    SNonNeg = SNonPos"
  | "times_sign SNonNeg SNeg    = SNonPos"
```

```
  | "times_sign SPos    SNonNeg = SNonNeg"
  | "times_sign SNonNeg SPos    = SNonNeg"
  | "times_sign SPos    SNonPos = SNonPos"
  | "times_sign SNonPos SPos    = SNonPos"
  | "times_sign SNonNeg SNonNeg = SNonNeg"
  | "times_sign SNonNeg SNonPos = SNonPos"
  | "times_sign SNonPos SNonNeg = SNonPos"
  | "times_sign SNonPos SNonPos = SNonNeg"
  | "times_sign _       _       = STop"
instance ..
end

definition sign_div :: "sign ⇒ sign ⇒ sign" where
  "sign_div a b = (if a = SBot ∨ b = SBot then SBot else (a * b) ⊔ SZero)"

definition sign_mod :: "sign ⇒ sign ⇒ sign" where
  "sign_mod a b =
    (if a = SBot ∨ b = SBot then SBot else if b = SZero then a else a ⊔ SZero)"

lemma sign_div_sound:
  assumes "i ∈ gamma_sign a" "j ∈ gamma_sign b"
  shows "c_div i j ∈ gamma_sign (sign_div a b)"
  using assms
  by (cases a; cases b; auto simp: sign_div_def sup_sign_def
      intro: c_div_nonneg c_div_nonpos)

lemma sign_mod_sound:
  assumes "i ∈ gamma_sign a" "j ∈ gamma_sign b"
  shows "c_mod i j ∈ gamma_sign (sign_mod a b)"
  using assms
  by (cases a; cases b; auto simp: sign_mod_def sup_sign_def
      intro: c_mod_nonneg c_mod_nonpos)
```

A conversion to a machine kind keeps zero and the empty value. At an unsigned kind every value is nonnegative; at a signed kind wraparound can change any sign.

```
definition sign_cast :: "ikind ⇒ sign ⇒ sign" where
  "sign_cast ik a =
     (if a = SBot then SBot else if a = SZero then SZero
      else if ik_signed ik then STop else SNonNeg)"

lemma sign_cast_sound: "i ∈ gamma_sign a ⟹ ik_norm ik i ∈ gamma_sign (sign_cast ik a)"
  by (cases a) (auto simp: sign_cast_def ik_norm_nonneg_unsigned)
```

Comparison and truthiness queries

`sign_lt`/`sign_eqb`/`sign_tobool` are Sign's three-valued `bool option` queries: `Some True`/`Some False` when the two operands' sign bounds alone decide it, `None` otherwise. Every guard here is a `sign_le` test against a fixed threshold, so downward-closedness under $\leq$ follows from `sign_le`'s own transitivity, not from any extra reasoning about `SBot`.

```
fun sign_lt :: "sign ⇒ sign ⇒ bool option" where
  "sign_lt a b =
     (if sign_le a SNeg ∧ sign_le b SNonNeg then Some True
      else if sign_le a SNonPos ∧ sign_le b SPos then Some True
      else if sign_le b SNonPos ∧ sign_le a SNonNeg then Some False
      else if sign_le b SNeg ∧ sign_le a SPos then Some False
      else None)"

fun sign_eqb :: "sign ⇒ sign ⇒ bool option" where
  "sign_eqb a b =
     (if a = SZero ∧ b = SZero then Some True
      else if (sign_le a SNeg ∧ sign_le b SNonNeg) ∨ (sign_le b SNeg ∧ sign_le a SNonNeg)
           ∨ (sign_le a SPos ∧ sign_le b SNonPos) ∨ (sign_le b SPos ∧ sign_le a SNonPos)
      then Some False
      else None)"

fun sign_tobool :: "sign ⇒ bool option" where
  "sign_tobool a =
     (if sign_le a SNeg ∨ sign_le a SPos then Some True
      else if sign_le a SZero then Some False
      else None)"

lemma sign_lt_sound:
  assumes "sign_lt a b = Some c" and "i ∈ gamma_sign a" and "j ∈ gamma_sign b"
  shows "(i < j) = c"
```

```
  using assms unfolding less_eq_sign_def[symmetric]
  by (cases a; cases b; auto split: if_splits)

lemma sign_eqb_sound:
  assumes "sign_eqb a b = Some c" and "i ∈ gamma_sign a" and "j ∈ gamma_sign b"
  shows "(i = j) = c"
  using assms unfolding less_eq_sign_def[symmetric]
  by (cases a; cases b; auto split: if_splits)
```

Soundness and monotonicity of the arithmetic

Each table is sound for the integer operation and monotone in both operands.

```
lemma sign_plus_sound:
  assumes "i ∈ gamma_sign a" "j ∈ gamma_sign b"
  shows "i + j ∈ gamma_sign (a + b)"
  using assms by (cases a; cases b; auto)

lemma sign_minus_sound:
  assumes "i ∈ gamma_sign a" "j ∈ gamma_sign b"
  shows "i - j ∈ gamma_sign (a - b)"
  using assms by (cases a; cases b; auto)

lemma sign_times_sound:
  assumes "i ∈ gamma_sign a" "j ∈ gamma_sign b"
  shows "i * j ∈ gamma_sign (a * b)"
  using assms by (cases a; cases b; auto simp: mult_neg_neg mult_neg_pos mult_pos_neg
                                               zero_le_mult_iff mult_le_0_iff)

end
```

11.3 Sign_Backward

```
theory Sign_Backward
  imports
    Sign_Arithmetic
    "Voblint_Domain.Backward_Numeric_Queries"
begin
```

Sign backward filtering

The sign meet lives with the lattice in `Voblint_Domain.Sign_Lattice`.

Backward-analysis: inverse operators

`inv_less_sign` provides sign-specific refinement when a guard `e1 < e2` is known true or false. Plus/minus/times are too coarse for useful arithmetic inversion in sign, so this theory gives no inverse for them.

```
fun inv_less_sign :: "bool => sign => sign => sign * sign" where
    "inv_less_sign True  a1 a2 =
       (let a1' = if sign_le a2 SNonPos then a1 ⊓ SNeg else a1 ;
                a2' = if sign_le a1 SNonNeg then a2 ⊓ SPos else a2
        in (a1', a2'))"
  | "inv_less_sign False a1 a2 =
       (let a1' = if sign_le a2 SPos then a1 ⊓ SPos
                  else if sign_le a2 SNonNeg then a1 ⊓ SNonNeg
                  else a1 ;
                a2' = if sign_le a1 SNeg then a2 ⊓ SNeg
                  else if sign_le a1 SNonPos then a2 ⊓ SNonPos
                  else a2
        in (a1', a2'))"

lemma inv_less_sign_sound:
  "n1 ∈ gamma_sign a1 ⟹ n2 ∈ gamma_sign a2 ⟹ (n1 < n2) = res
   ⟹ n1 ∈ gamma_sign (fst (inv_less_sign res a1 a2))
     ∧ n2 ∈ gamma_sign (snd (inv_less_sign res a1 a2))"
  by (cases res; cases a1; cases a2;
      auto simp: less_eq_sign_def; linarith)
```

`inv_eq_sign` narrows on a guard `e1 = e2` known true or false. The true branch narrows both operands to their intersection, (⊓). The false branch only narrows a boundary singleton away from a wider operand that has `SZero` as one of its bounds: excluding `0` from `SNonNeg` leaves `SPos`, from `SNonPos` leaves `SNeg`; both directions, symmetric in which operand is at most `SZero`. Two jointly-at-most-`SZero` operands are jointly unreachable. Every other pair — including any pair where the excluded value would not sit at a representable lattice boundary — passes through unchanged: the seven-element lattice cannot express ñonzeroör an interior exclusion as a single value.

The guard tests `sign_le a SZero` (the set `{SBot, SZero}`), not literal equality `a = SZero`: `SBot` is strictly below `SZero`, so an equality guard would let a wider input at `SZero` trigger narrowing that a strictly more precise input at `SBot` would not — a genuine monotonicity failure, not merely a missed precision opportunity. The order-based guard closes that gap the same way `inv_less_sign`'s own `sign_le a2 SNonPos`-style guards do; it also makes the `SBot` case sound for free (vacuously, since `gamma_sign SBot = {}`), without `inv_less_sign`'s separate reliance on `SBot ⊓ _ = SBot`.

```
fun inv_eq_sign :: "bool => sign => sign => sign * sign" where
    "inv_eq_sign True  a1 a2 = (a1 ⊓ a2, a1 ⊓ a2)"
  | "inv_eq_sign False a1 a2 =
       (let a1' = if sign_le a1 SZero ∧ sign_le a2 SZero then SBot
                  else if sign_le a2 SZero ∧ sign_le a1 SNonNeg then a1 ⊓ SPos
                  else if sign_le a2 SZero ∧ sign_le a1 SNonPos then a1 ⊓ SNeg
                  else a1 ;
                a2' = if sign_le a1 SZero ∧ sign_le a2 SZero then SBot
                  else if sign_le a1 SZero ∧ sign_le a2 SNonNeg then a2 ⊓ SPos
                  else if sign_le a1 SZero ∧ sign_le a2 SNonPos then a2 ⊓ SNeg
                  else a2
        in (a1', a2'))"

end
```

11.4 Sign_Kind

```
theory Sign_Kind
  imports "Voblint_Analysis_Sign.Sign_Backward" "Voblint_Nonrelational.Nonrelational_Analysis"
    "Voblint_VIMP.VIMP_Notation"
begin
```

Signs at a kind

A sign carries no magnitude, so it cannot rule out an overflow: normalizing a nonzero sign to a kind answers the kind's top, every sign at a signed kind and the nonnegative ones at an unsigned kind (`sign_cast`). Zero survives every normalization. A conversion from a kind whose range lies inside the target's changes no value and keeps the sign. Sign has no Goblint counterpart, and it is much weaker at machine kinds than a sign analysis over integers that do not wrap.

```
lemma minus_one_in_signed_range: "ik_signed k ⟹ - 1 ∈ ik_range k"
  by (cases k) (simp_all add: ik_min_def ik_max_def ik_half_def)

lemma unsigned_range_nonneg: "¬ ik_signed k ⟹ n ∈ ik_range k ⟹ 0 ≤ n"
  by (simp add: ik_min_def)

instantiation sign :: ik_domain
begin

definition widen_ik_sign :: "ikind ⇒ sign ⇒ sign ⇒ sign" where
  "widen_ik_sign k a b = a ⊔ b"

definition narrow_ik_sign :: "ikind ⇒ sign ⇒ sign ⇒ sign" where
  "narrow_ik_sign k a b = a ⊓ b"

definition empty_ik_sign :: "ikind ⇒ sign ⇒ bool" where
  "empty_ik_sign k a ⟷ a = SBot ∨ (¬ ik_signed k ∧ a = SNeg)"

instance proof
  fix k :: ikind and a b :: sign
  show "a ≤ widen_ik k a b" "b ≤ widen_ik k a b" by (simp_all add: widen_ik_sign_def)
  show "b ≤ a ⟹ b ≤ narrow_ik k a b" "b ≤ a ⟹ narrow_ik k a b ≤ a"
    by (simp_all add: narrow_ik_sign_def inf_absorb2)
  show "empty_ik k a ⟷ numeric_domain_class.gamma a ∩ ik_range k = {}"
  proof
    assume "empty_ik k a"
    then show "numeric_domain_class.gamma a ∩ ik_range k = {}"
      using unsigned_range_nonneg[of k]
      by (fastforce simp: empty_ik_sign_def simp del: ik_range_iff)
  next
    assume e: "numeric_domain_class.gamma a ∩ ik_range k = {}"
    have "0 ∉ gamma_sign a" "1 ∉ gamma_sign a" "ik_signed k ⟹ - 1 ∉ gamma_sign a"
      using e minus_one_in_signed_range[of k] zero_in_ik_range[of k] one_in_ik_range[of k]
      by (auto simp del: ik_range_iff zero_in_ik_range one_in_ik_range)
    then show "empty_ik k a" by (cases a) (auto simp: empty_ik_sign_def)
  qed
qed

end

fun sign_aop :: "aop ⇒ sign ⇒ sign ⇒ sign" where
  "sign_aop AAdd a b = a + b"
| "sign_aop ASub a b = a - b"
| "sign_aop AMul a b = a * b"
| "sign_aop ADiv a b = sign_div a b"
| "sign_aop AMod a b = sign_mod a b"

lemma sign_aop_sound:
  "m ∈ gamma_sign a ⟹ n ∈ gamma_sign b ⟹ aop_eval op m n ∈ gamma_sign (sign_aop op a b)"
  by (cases op) (simp_all add: sign_plus_sound sign_minus_sound sign_times_sound
      sign_div_sound sign_mod_sound)


instantiation sign :: kind_int_domain
begin

definition top_ik_sign :: "ikind ⇒ sign" where
  "top_ik_sign k = (if ik_signed k then STop else SNonNeg)"

definition of_int_ik_sign :: "ikind ⇒ int ⇒ sign" where
```

```
  "of_int_ik_sign k n = sign_of_int (ik_norm k n)"

definition cast_ik_sign :: "signed_overflow_policy ⇒ ikind option ⇒ ikind ⇒ sign ⇒ sign" where
  "cast_ik_sign ovp sk k a =
     (case sk of
        Some s ⇒ if ik_widens s k then a else sign_cast k a
      | None ⇒ sign_cast k a)"
```

Some operations on in-range operands cannot leave a signed kind's range, whatever their magnitudes: a sum of opposite signs, a difference of equal signs or with a negative minuend, a product with zero, a quotient unless both operands may be negative (only the minimum divided by `-1` overflows), and every remainder. Their sign needs no normalization.

```
fun sign_safe :: "aop ⇒ sign ⇒ sign ⇒ bool" where
  "sign_safe AAdd a b ⟷ a ≤ SZero ∨ b ≤ SZero
     ∨ (a ≤ SNonNeg ∧ b ≤ SNonPos) ∨ (a ≤ SNonPos ∧ b ≤ SNonNeg)"
| "sign_safe ASub a b ⟷ b ≤ SZero ∨ (a ≤ SNonNeg ∧ b ≤ SNonNeg) ∨ (a ≤ SNeg ∧ b ≤ SNonPos)"
| "sign_safe AMul a b ⟷ a ≤ SZero ∨ b ≤ SZero"
| "sign_safe ADiv a b ⟷ a ≤ SNonNeg ∨ b ≤ SNonNeg"
| "sign_safe AMod a b ⟷ True"


lemma c_div_between:
  assumes "0 ≤ a ∨ 0 ≤ b"
  shows "min 0 a - (if 0 ≤ a then a else 0) ≤ c_div a b ∧ c_div a b ≤ max 0 a"
proof (cases "b = 0")
  case False
  have q1: "0 ≤ |a| div |b|" using False by (simp add: pos_imp_zdiv_nonneg_iff)
  have q2: "|a| div |b| ≤ |a|"
    by (cases "a = 0") (simp_all add: int_div_le_self)
  note q = q1 q2
  then show ?thesis
    using False assms by (cases "0 ≤ a"; cases "0 < b") (auto simp: c_div_def sgn_if)
qed (simp add: c_div_def)

lemma sign_safe_range:
  assumes s: "ik_signed k" and m: "m ∈ ik_range k" and n: "n ∈ ik_range k"
    and ma: "m ∈ gamma_sign a" and nb: "n ∈ gamma_sign b" and safe: "sign_safe op a b"
  shows "aop_eval op m n ∈ ik_range k"
proof -
  have lo: "ik_min k = - ik_max k - 1"
    using s by (simp add: ik_min_def ik_max_def)
  have mk: "ik_min k ≤ m" "m ≤ ik_max k" and nk: "ik_min k ≤ n" "n ≤ ik_max k"
    using m n by simp_all
  have sg: "⋀x c. x ∈ gamma_sign c ⟹ c ≤ SZero ⟹ x = 0"
    "⋀x c. x ∈ gamma_sign c ⟹ c ≤ SNonNeg ⟹ 0 ≤ x"
    "⋀x c. x ∈ gamma_sign c ⟹ c ≤ SNonPos ⟹ x ≤ 0"
    "⋀x c. x ∈ gamma_sign c ⟹ c ≤ SNeg ⟹ x < 0"
    by (case_tac [!] c) (auto simp: less_eq_sign_def)
  show ?thesis
  proof (cases op)
    case ADiv
    have "0 ≤ m ∨ 0 ≤ n" using safe ADiv sg(2)[OF ma] sg(2)[OF nb] by auto
    with c_div_between[of m n] mk nk lo ADiv show ?thesis by (auto split: if_splits)
  next
    case AMod
    have z: "ik_min k ≤ 0" "0 ≤ ik_max k" by simp_all
    with c_mod_between[of m n] mk AMod show ?thesis
      by (cases "0 ≤ m") (auto simp: min_def max_def simp del: ik_min_le_zero zero_le_ik_max)
  qed (use safe mk nk lo sg ma nb in ‹auto dest: sg›)
qed

definition arith_ik_sign :: "signed_overflow_policy ⇒ ikind ⇒ aop ⇒ bool ⇒ sign ⇒ sign ⇒ sign" where
  "arith_ik_sign ovp k op fits a b =
     (if fits ∧ ik_signed k ∧ sign_safe op a b then sign_aop op a b
      else sign_cast k (sign_aop op a b))"

definition cmp_ik_sign :: "cop ⇒ sign ⇒ sign ⇒ bool option" where
  "cmp_ik_sign = cop_of sign_lt sign_eqb"

definition meet_ik_sign :: "sign ⇒ sign ⇒ sign" where
  "meet_ik_sign a b = a ⊓ b"
```

A disequality with zero excludes zero from the other side, the sign counterpart of the bound exclusion of `inv_bin_int` for `Ne`.

```
fun sign_excl :: "sign ⇒ sign ⇒ sign" where
```

```
  "sign_excl SNonNeg SZero = SPos"
| "sign_excl SNonPos SZero = SNeg"
| "sign_excl SZero SZero = SBot"
| "sign_excl a b = a"

lemma sign_excl_sound:
  "m ≠ n ⟹ m ∈ gamma_sign a ⟹ n ∈ gamma_sign b ⟹ m ∈ gamma_sign (sign_excl a b)"
  by (induction a b rule: sign_excl.induct) auto
```

An order between two values refines each by the other's sign through the value-level `inv_less_sign`: a comparison reads values the kind already holds, so no wraparound is involved.

```
definition refine_ik_sign :: "cop ⇒ sign ⇒ sign ⇒ sign × sign" where
  "refine_ik_sign c a b =
     (case c of
        CEq ⇒ (a ⊓ b, a ⊓ b)
      | CNe ⇒ (sign_excl a b, sign_excl b a)
      | CLt ⇒ inv_less_sign True a b
      | CGe ⇒ inv_less_sign False a b
      | CGt ⇒ prod.swap (inv_less_sign True b a)
      | CLe ⇒ prod.swap (inv_less_sign False b a))"

definition raw_aop_ik_sign :: "aop ⇒ sign ⇒ sign ⇒ sign" where
  "raw_aop_ik_sign = sign_aop"

definition fits_ik_sign :: "ikind ⇒ sign ⇒ bool" where
  "fits_ik_sign k a ⟷ a = SBot ∨ a = SZero"

definition answer_ik_sign :: "sign ⇒ answer" where
  "answer_ik_sign a = answer_with (int_sign_update (λ_. a))"

definition of_answer_ik_sign :: "ikind option ⇒ answer ⇒ sign" where
  "of_answer_ik_sign sk q = (case q of QBot ⇒ ⊥ | QTop ⇒ ⊤ | QLifted d ⇒ int_sign d)"

instance proof
  fix n m :: int and k :: ikind and a b :: sign and op :: aop and c :: cop and r :: bool
  show "n ∈ ik_range k ⟹ n ∈ numeric_domain_class.gamma (top_ik k :: sign)"
    using unsigned_range_nonneg[of k n] by (auto simp: top_ik_sign_def simp del: ik_range_iff)
  show "ik_norm k n ∈ numeric_domain_class.gamma (of_int_ik k n :: sign)"
    by (simp add: of_int_ik_sign_def sign_of_int_gamma)
  show "(⋀k'. sk = Some k' ⟹ n ∈ ik_range k') ⟹ n ∈ numeric_domain_class.gamma a
      ⟹ ik_norm k n ∈ numeric_domain_class.gamma (cast_ik ovp sk k a)" for ovp sk
    by (cases sk) (auto simp: cast_ik_sign_def ik_widens_def sign_cast_sound split: if_splits)
  show "(fits ⟹ m ∈ ik_range k ∧ n ∈ ik_range k) ⟹ m ∈ numeric_domain_class.gamma a
      ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ ik_norm k (aop_eval op m n) ∈ numeric_domain_class.gamma (arith_ik ovp k op fits a b)" for ovp fits
    using sign_safe_range[of k m n a b op] sign_aop_sound[of m a n b op]
    by (auto simp: arith_ik_sign_def sign_cast_sound simp del: ik_range_iff)
  show "cmp_ik c a b = Some r ⟹ m ∈ numeric_domain_class.gamma a
      ⟹ n ∈ numeric_domain_class.gamma b ⟹ cop_eval c m n = r"
    unfolding cmp_ik_sign_def by simp (rule cop_of_sound[OF sign_lt_sound sign_eqb_sound])
  show "m ∈ numeric_domain_class.gamma a ⟹ m ∈ numeric_domain_class.gamma b
      ⟹ m ∈ numeric_domain_class.gamma (meet_ik a b)"
    by (simp add: meet_ik_sign_def)
  show "cop_eval c m n ⟹ m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ m ∈ numeric_domain_class.gamma (fst (refine_ik c a b))
        ∧ n ∈ numeric_domain_class.gamma (snd (refine_ik c a b))"
    using inv_less_sign_sound[of m a n b True] inv_less_sign_sound[of m a n b False]
      inv_less_sign_sound[of n b m a True] inv_less_sign_sound[of n b m a False]
      sign_excl_sound[of m n a b] sign_excl_sound[of n m b a]
    by (cases c) (auto simp: refine_ik_sign_def not_less)
  show "m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ aop_eval op m n ∈ numeric_domain_class.gamma (raw_aop_ik op a b)"
    by (simp add: raw_aop_ik_sign_def sign_aop_sound)
  show "fits_ik k a ⟹ n ∈ numeric_domain_class.gamma a ⟹ n ∈ ik_range k"
    by (auto simp: fits_ik_sign_def)
  show "n ∈ numeric_domain_class.gamma a ⟹ n ∈ gamma_answer (answer_ik a)"
    by (simp add: answer_ik_sign_def gamma_int_dom_def top_int_dom_ext_def gamma_sign_top
        gamma_parity_top top_ivl_def gamma_ivl_top)
  show "(⋀k. sk = Some k ⟹ n ∈ ik_range k) ⟹ n ∈ gamma_answer q
      ⟹ n ∈ numeric_domain_class.gamma (of_answer_ik sk q :: sign)" for sk q
    by (cases q)
      (simp_all add: of_answer_ik_sign_def gamma_int_dom_def gamma_sign_top gamma_parity_top)
qed

end
```

```
declare [[code abort: gamma_sign]]
```

Executed

An unsigned `x` is nonnegative whatever its arithmetic did.

```
definition sign_prog :: imp_prog where
  "sign_prog = program { fun f() { x = 0; x = x - 5; } fun main() { f(); } }"

lemma sign_value:
  "nonrel.state_at default_sem_conf [(ScopedId (STR ''f'') (STR ''x''), U8)] 0 Globals_Join is_global sign_prog []
     (FunctionResult (STR ''f'')) (ScopedId (STR ''f'') (STR ''x''))
     = KD U8 SNonNeg"
  by eval

end
```

Chapter 12

Voblint__Analysis__Interval

12.1 Interval_Warrowing

```
theory Interval_Warrowing
  imports "Voblint_Domain.Interval_Lattice"
begin
```

Interval widening and narrowing

The interval lattice has infinite ascending chains, so the solver needs widening to terminate and narrowing to recover precision afterwards. This theory defines both and instantiates intervals as a numeric domain.

Widening

Standard interval widening: keep a bound if it did not move outward, otherwise push it to the infinite end. Every widen-ascending chain therefore stabilises after at most two steps (each bound takes at most one outward jump).

```
fun widen_ivl_core :: "ivl => ivl => ivl" where
    "widen_ivl_core (Ivl l1 u1) (Ivl l2 u2) =
       Ivl (if l1 ≤ l2 then l1 else MinInf)
           (if u2 ≤ u1 then u1 else PlusInf)"

lemma a_le_widen_ivl_core: "(a :: ivl) ≤ widen_ivl_core a b"
proof (cases a; cases b)
  fix l1 u1 l2 u2 :: eint
  assume "a = Ivl l1 u1" "b = Ivl l2 u2"
  then show "a ≤ widen_ivl_core a b"
    unfolding less_eq_ivl_def by auto
qed

lemma b_le_widen_ivl_core: "(b :: ivl) ≤ widen_ivl_core a b"
proof (cases a; cases b)
  fix l1 u1 l2 u2 :: eint
  assume "a = Ivl l1 u1" "b = Ivl l2 u2"
  then show "b ≤ widen_ivl_core a b"
    unfolding less_eq_ivl_def
    using eint_le_linear[of l1 l2] eint_le_linear[of u2 u1]
    by auto
qed
```

Widening termination: every widen-ascending chain stabilises. At each step each bound either stays or jumps to `MinInf` / `PlusInf`, and once a bound reaches the infinite end it remains there. After at most two transitions the chain is constant.

```
lemma widen_ivl_core_terminates:
  assumes "∀i. widen_ivl_core (f i) (f (Suc i)) = f (Suc i)"
  shows "∃n. ∀j. n ≤ j ⟶ f j = f n"
proof -
  define lb where "lb i = ivl_lower (f i)" for i
  define ub where "ub i = ivl_upper (f i)" for i
  have f_eq: "f i = Ivl (lb i) (ub i)" for i
    unfolding lb_def ub_def by (cases "f i") simp
  have step:
    "lb (Suc i) = lb i ∨ lb (Suc i) = MinInf"
    "ub (Suc i) = ub i ∨ ub (Suc i) = PlusInf"
    for i
  proof -
    have "widen_ivl_core (Ivl (lb i) (ub i)) (Ivl (lb (Suc i)) (ub (Suc i))) =
          Ivl (lb (Suc i)) (ub (Suc i))"
      using assms[rule_format, of i] f_eq[of i] f_eq[of "Suc i"] by simp
    then have eqs:
      "lb (Suc i) = (if eint_le (lb i) (lb (Suc i)) then lb i else MinInf)"
      "ub (Suc i) = (if eint_le (ub (Suc i)) (ub i) then ub i else PlusInf)"
      by simp_all
    from eqs(1) show "lb (Suc i) = lb i ∨ lb (Suc i) = MinInf"
      by (auto split: if_splits)
    from eqs(2) show "ub (Suc i) = ub i ∨ ub (Suc i) = PlusInf"
      by (auto split: if_splits)
  qed
  have lb_stay: "lb i = MinInf ⟹ lb (Suc i) = MinInf" for i
    using step(1) by metis
  have ub_stay: "ub i = PlusInf ⟹ ub (Suc i) = PlusInf" for i
    using step(2) by metis
  have lb_stay_le: "lb i = MinInf ⟹ i ≤ j ⟹ lb j = MinInf" for i j
```

```
  proof (induct j)
    case 0 thus ?case by simp
  next
    case (Suc j) thus ?case using lb_stay le_SucE by metis
  qed
  have ub_stay_le: "ub i = PlusInf ⟹ i ≤ j ⟹ ub j = PlusInf" for i j
  proof (induct j)
    case 0 thus ?case by simp
  next
    case (Suc j) thus ?case using ub_stay le_SucE by metis
  qed
  have lb_const_if_no_MinInf:
    "(∀k. lb k ≠ MinInf) ⟹ lb j = lb 0" for j
  proof (induct j)
    case 0 thus ?case by simp
  next
    case (Suc j) thus ?case using step(1)[of j] by auto
  qed
  have ub_const_if_no_PlusInf:
    "(∀k. ub k ≠ PlusInf) ⟹ ub j = ub 0" for j
  proof (induct j)
    case 0 thus ?case by simp
  next
    case (Suc j) thus ?case using step(2)[of j] by auto
  qed
  obtain nl where nl: "∀j ≥ nl. lb j = lb nl"
  proof (cases "∃k. lb k = MinInf")
    case True
    then obtain k where "lb k = MinInf" by blast
    then have "∀j ≥ k. lb j = MinInf" using lb_stay_le by blast
    then have "∀j ≥ k. lb j = lb k" using ‹lb k = MinInf› by simp
    then show ?thesis by (rule that)
  next
    case False
    then have "∀k. lb k ≠ MinInf" by blast
    then have "∀j. lb j = lb 0" using lb_const_if_no_MinInf by blast
    then have "∀j ≥ 0. lb j = lb 0" by blast
    then show ?thesis by (rule that)
  qed
  obtain nu where nu: "∀j ≥ nu. ub j = ub nu"
  proof (cases "∃k. ub k = PlusInf")
    case True
    then obtain k where "ub k = PlusInf" by blast
    then have "∀j ≥ k. ub j = PlusInf" using ub_stay_le by blast
    then have "∀j ≥ k. ub j = ub k" using ‹ub k = PlusInf› by simp
    then show ?thesis by (rule that)
  next
    case False
    then have "∀k. ub k ≠ PlusInf" by blast
    then have "∀j. ub j = ub 0" using ub_const_if_no_PlusInf by blast
    then have "∀j ≥ 0. ub j = ub 0" by blast
    then show ?thesis by (rule that)
  qed
  let ?n = "max nl nu"
  have "∀j. ?n ≤ j ⟶ f j = f ?n"
  proof (intro allI impI)
    fix j assume jn: "?n ≤ j"
    have lb_eq: "lb j = lb ?n"
    proof -
      from jn have "nl ≤ j" "nl ≤ ?n" by auto
      then have "lb j = lb nl" "lb ?n = lb nl"
        using nl by blast+
      then show ?thesis by simp
    qed
    have ub_eq: "ub j = ub ?n"
    proof -
      from jn have "nu ≤ j" "nu ≤ ?n" by auto
      then have "ub j = ub nu" "ub ?n = ub nu"
        using nu by blast+
      then show ?thesis by simp
    qed
    show "f j = f ?n"
      using f_eq[of j] f_eq[of ?n] lb_eq ub_eq by simp
  qed
  then show ?thesis by blast
qed
```

Type-class widening for TD warrowing solver

Standard interval narrowing: when the recomputed value `b` refines the widened value `a` (`b ≤ a`), keep `a`'s finite bounds and only fill an infinite bound of `a` from `b`. A bound moves off an infinite endpoint at most once, so the narrowing descent is finite. Combined with the backward guard filters this recovers loop bounds under *every* update rule (a bounded local counter reads `[0, 20]` under `join`, `per_origin` and `warrow` alike). It does not help a *flow-insensitive global*: there the guard never bounds the value written back to the slot, so `[0, +inf]` is a genuine fixpoint and narrowing has nothing smaller to descend to.

```
fun narrow_ivl_td :: "ivl ⇒ ivl ⇒ ivl" where
  "narrow_ivl_td (Ivl l1 u1) (Ivl l2 u2) =
     Ivl (if l1 = MinInf then l2 else l1) (if u1 = PlusInf then u2 else u1)"

instantiation ivl :: warrowing begin
```

Widening carries the standard bot-law `bot ∇ x = x`, `x ∇ bot = x`: since ⊥ is the empty interval `Ivl PlusInf MinInf`, an unguarded `widen_ivl_core` from bot would jump straight to the top interval, topping any unknown on its first stabilisation. The guard keeps the first contribution exact.

```
  definition "((a :: ivl) ∇ b) =
     (if a = bot then b else if b = bot then a else widen_ivl_core a b)"
  definition "((a :: ivl) Δ b) = narrow_ivl_td a b"
instance proof intro_classes
  fix a b :: ivl
  show "a ≤ (a ∇ b)"
  proof (cases "a = bot")
    case True thus ?thesis by (simp add: widen_ivl_def)
  next
    case False thus ?thesis
      by (cases "b = bot") (simp_all add: widen_ivl_def a_le_widen_ivl_core)
  qed
  show "b ≤ (a ∇ b)"
  proof (cases "a = bot")
    case True thus ?thesis by (simp add: widen_ivl_def)
  next
    case False thus ?thesis
      by (cases "b = bot") (simp_all add: widen_ivl_def b_le_widen_ivl_core)
  qed
  show "b ≤ a ⟹ b ≤ (a Δ b)"
    unfolding narrow_ivl_def
    by (cases a; cases b) (auto simp: less_eq_ivl_def split: if_splits)
  show "b ≤ a ⟹ (a Δ b) ≤ a"
    unfolding narrow_ivl_def
    by (cases a; cases b) (auto simp: less_eq_ivl_def split: if_splits)
qed
end
```

Numeric domain instance

`numeric_domain` extends `executable_domain`, which includes the solver's `warrowing`, so the instance follows the widening and narrowing above.

```
instantiation ivl :: numeric_domain begin
definition gamma_abs_ivl [simp]: "numeric_domain_class.gamma (a :: ivl) = gamma_ivl a"
definition is_empty_ivl [simp]: "is_empty (a :: ivl) = is_bottom_ivl a"
definition to_string_ivl [simp]:
  "to_string (a :: ivl) = (if is_top_ivl a then sym_top else string_of_ivl a)"
instance proof intro_classes
  show "numeric_domain_class.gamma (bot :: ivl) = {}"
    by (simp add: gamma_ivl_bot)
next
  show "numeric_domain_class.gamma (top :: ivl) = UNIV"
    by (simp add: gamma_ivl_top top_ivl_def)
next
  fix a b :: ivl
  assume H: "a ≤ b"
  show "numeric_domain_class.gamma a ⊆ numeric_domain_class.gamma b"
  proof -
    have "gamma_ivl a ⊆ gamma_ivl b" using H by (rule gamma_ivl_mono)
    then show ?thesis by simp
  qed
next
  fix a :: ivl
  show "is_empty a ⟷ numeric_domain_class.gamma a = {}"
    by (simp add: is_bottom_ivl_correct)
```

qed
end

end

12.2 Interval_Arithmetic

```
theory Interval_Arithmetic
  imports Interval_Warrowing
begin
```

Adding, subtracting and multiplying whole ranges

Addition and subtraction of intervals move the two endpoints independently: the sum of `[a, b]` and `[c, d]` runs from `a + c` to `b + d`. Multiplication cannot, because a negative factor swaps which endpoint is larger, so `ivl_times_core` multiplies all four corner pairs and keeps their minimum and maximum. Every operation first `normalize_ivl`s its arguments — an interval whose lower bound exceeds its upper one denotes no number at all and collapses to `bot` — which is what keeps an unreachable operand from producing a spurious range.

Each operation is proved sound (concrete values drawn from the operands land in the computed interval) and monotone (widening either operand widens the result).

```
instantiation ivl :: plus begin
fun plus_ivl :: "ivl => ivl => ivl" where
    "plus_ivl (Ivl l1 u1) (Ivl l2 u2) =
       (case (normalize_ivl (Ivl l1 u1), normalize_ivl (Ivl l2 u2)) of
          (Ivl a b, Ivl c d) ⇒ normalize_ivl (Ivl (a + c) (b + d)))"
instance ..
end

instantiation ivl :: minus begin
fun minus_ivl :: "ivl => ivl => ivl" where
    "minus_ivl (Ivl l1 u1) (Ivl l2 u2) =
       (case (normalize_ivl (Ivl l1 u1), normalize_ivl (Ivl l2 u2)) of
          (Ivl a b, Ivl c d) ⇒ normalize_ivl (Ivl (a - d) (b - c)))"
instance ..
end
```

`ivl_min`/`ivl_max` bound the result of a `min`/`max` library call, whose conversion to its destination's kind raises an obligation: componentwise `min`/`max` of the bounds, exact whenever both operands are non-empty, normalized like `plus_ivl`/`minus_ivl` so an empty operand still collapses to the canonical ⊥.

```
fun ivl_min :: "ivl => ivl => ivl" where
    "ivl_min (Ivl l1 u1) (Ivl l2 u2) = normalize_ivl (Ivl (min l1 l2) (min u1 u2))"

fun ivl_max :: "ivl => ivl => ivl" where
    "ivl_max (Ivl l1 u1) (Ivl l2 u2) = normalize_ivl (Ivl (max l1 l2) (max u1 u2))"

lemma ivl_min_sound:
  assumes "m ∈ gamma_ivl a" and "n ∈ gamma_ivl b"
  shows "min m n ∈ gamma_ivl (ivl_min a b)"
proof -
  obtain l1 u1 l2 u2 where ab: "a = Ivl l1 u1" "b = Ivl l2 u2" by (cases a; cases b) blast
  have "Fin (min m n) = min (Fin m) (Fin n)" by (simp add: min_def)
  with assms show ?thesis
    by (simp add: ab normalize_ivl_gamma del: less_eq_eint_def)
       (meson min.coboundedI1 min.coboundedI2 order.trans)
qed

lemma ivl_max_sound:
  assumes "m ∈ gamma_ivl a" and "n ∈ gamma_ivl b"
  shows "max m n ∈ gamma_ivl (ivl_max a b)"
proof -
  obtain l1 u1 l2 u2 where ab: "a = Ivl l1 u1" "b = Ivl l2 u2" by (cases a; cases b) blast
  have "Fin (max m n) = max (Fin m) (Fin n)" by (simp add: max_def)
  with assms show ?thesis
    by (simp add: ab normalize_ivl_gamma del: less_eq_eint_def)
       (meson max.coboundedI1 max.coboundedI2 order.trans)
qed

lemma int_mult_in_corners_lo:
  fixes l1 u1 l2 u2 i j :: int
  assumes "l1 ≤ i" "i ≤ u1" "l2 ≤ j" "j ≤ u2"
  shows "min (l1*l2) (min (l1*u2) (min (u1*l2) (u1*u2))) ≤ i*j"
proof -
  consider (A) "0 ≤ i" "0 ≤ l2"
    | (B) "0 ≤ i" "l2 < 0"
    | (C) "i < 0" "0 ≤ u2"
    | (D) "i < 0" "u2 < 0"
    by linarith
```

```
  then show ?thesis
  proof cases
    case A
    have "l1*l2 ≤ i*l2"
      using assms(1) A(2)
      by (simp add: mult_right_mono)
    also have "... ≤ i*j"
      by (simp add: A(1) assms(3) mult_left_mono)
    finally show ?thesis by simp
  next
    case B
    have "u1*l2 ≤ i*l2"
      using assms(2) B(2) by simp
    also have "... ≤ i*j"
      using assms(3) B(1) by (rule mult_left_mono)
    finally show ?thesis by simp
  next
    case C
    have "l1*u2 ≤ i*u2"
      using assms(1) C(2) by (rule mult_right_mono)
    also have "... ≤ i*j"
      using assms(4) C(1) by simp
    finally show ?thesis by simp
  next
    case D
    have "u1*u2 ≤ i*u2"
      using assms(2) D(2) by simp
    also have "... ≤ i*j"
      using assms(4) D(1) by simp
    finally show ?thesis by simp
  qed
qed

lemma int_mult_in_corners_hi:
  fixes l1 u1 l2 u2 i j :: int
  assumes "l1 ≤ i" "i ≤ u1" "l2 ≤ j" "j ≤ u2"
  shows "i * j ≤ max (l1*l2) (max (l1*u2) (max (u1*l2) (u1*u2)))"
proof -
  have negated:
    "min ((-u1)*l2) (min ((-u1)*u2) (min ((-l1)*l2) ((-l1)*u2)))
       ≤ (-i)*j"
    by (rule int_mult_in_corners_lo) (use assms in simp_all)
  from negated show ?thesis
    by (simp add: min_def max_def; linarith)
qed
```

Precise multiplication on non-empty intervals. For all-finite operands the product range is the min / max over the four corner products (the box product attains its extrema at the corners); any infinite bound falls back to `ivl_top`. An empty operand yields $\bot$ – the proper bottom handling that keeps the operator monotone (mapping empties to `ivl_top` would destroy the ordering).

```
fun ivl_times_core :: "ivl => ivl => ivl" where
    "ivl_times_core (Ivl (Fin l1) (Fin u1)) (Ivl (Fin l2) (Fin u2)) =
       Ivl (Fin (min (l1*l2) (min (l1*u2) (min (u1*l2) (u1*u2)))))
           (Fin (max (l1*l2) (max (l1*u2) (max (u1*l2) (u1*u2)))))"
  | "ivl_times_core _ _ = ivl_top"

instantiation ivl :: times begin
definition times_ivl :: "ivl => ivl => ivl" where
  "times_ivl a b = (if ivl_nonempty a ∧ ivl_nonempty b then ivl_times_core a b else bot)"
instance ..
end
```

`times_ivl_def` is deliberately not `[simp]`: its conditional body would make `simp` split on `ivl_nonempty` before `ivl_times_sound` can fire. A proof that must unfold `*` directly adds it explicitly.

```
lemma ivl_plus_sound:
  assumes "i ∈ gamma_ivl a" "j ∈ gamma_ivl b"
  shows "i + j ∈ gamma_ivl (a + b)"
proof (cases a; cases b)
  fix l1 u1 l2 u2 :: eint
  assume "a = Ivl l1 u1" "b = Ivl l2 u2"
  with assms have bnds:
    "eint_le l1 (Fin i)" "eint_le (Fin i) u1"
    "eint_le l2 (Fin j)" "eint_le (Fin j) u2"
    by auto
```

```
  show "i + j ∈ gamma_ivl (a + b)"
    unfolding ‹a = Ivl l1 u1› ‹b = Ivl l2 u2›
    using bnds normalize_ivl_def
    by (cases l1; cases l2; cases u1; cases u2) auto
qed

lemma ivl_minus_sound:
  assumes "i ∈ gamma_ivl a" "j ∈ gamma_ivl b"
  shows "i - j ∈ gamma_ivl (a - b)"
proof (cases a; cases b)
  fix l1 u1 l2 u2 :: eint
  assume "a = Ivl l1 u1" "b = Ivl l2 u2"
  with assms have bnds:
    "eint_le l1 (Fin i)" "eint_le (Fin i) u1"
    "eint_le l2 (Fin j)" "eint_le (Fin j) u2"
    by auto
  show "i - j ∈ gamma_ivl (a - b)"
    unfolding ‹a = Ivl l1 u1› ‹b = Ivl l2 u2›
    using bnds normalize_ivl_def
    by (cases l1; cases l2; cases u1; cases u2) auto
qed

lemma ivl_times_sound:
  assumes "i ∈ gamma_ivl a" "j ∈ gamma_ivl b"
  shows "i * j ∈ gamma_ivl (a * b)"
proof -
  from assms have ne: "ivl_nonempty a" "ivl_nonempty b"
    by (auto intro: gamma_ivl_nonempty)
  hence eq: "a * b = ivl_times_core a b" by (simp add: times_ivl_def)
  show ?thesis
  proof (cases a; cases b)
    fix l1 u1 l2 u2 assume ab: "a = Ivl l1 u1" "b = Ivl l2 u2"
    show ?thesis
    proof (cases l1; cases u1; cases l2; cases u2)
      fix n1 m1 n2 m2 assume fin: "l1 = Fin n1" "u1 = Fin m1" "l2 = Fin n2" "u2 = Fin m2"
      from assms ab fin have bnds: "n1 ≤ i" "i ≤ m1" "n2 ≤ j" "j ≤ m2" by auto
      show ?thesis using eq ab fin
        int_mult_in_corners_lo[OF bnds] int_mult_in_corners_hi[OF bnds] by simp
    qed (use eq ab ne(1) ne(2) in ‹simp_all add: ivl_top_def›)
  qed
qed
```

Both operators route their operands through `normalize_ivl`; these rewrites expose that shape without the constructor-pattern gate, so monotonicity factors through `normalize_ivl`'s monotonicity and the pointwise `eint` bounds.

```
lemma plus_ivl_norm:
  "a + b = (case (normalize_ivl a, normalize_ivl b) of
              (Ivl l1 u1, Ivl l2 u2) ⇒ normalize_ivl (Ivl (l1 + l2) (u1 + u2)))"
  by (cases a; cases b) simp

lemma minus_ivl_norm:
  "a - b = (case (normalize_ivl a, normalize_ivl b) of
              (Ivl l1 u1, Ivl l2 u2) ⇒ normalize_ivl (Ivl (l1 - u2) (u1 - l2)))"
  by (cases a; cases b) simp
```

Truncating division

C division truncates toward zero, so `c_div` and `c_mod` differ from HOL's flooring `div`/`mod` on negative operands. These lemmas bound `c_mod` by both operands and show `c_div` monotone in the dividend, the facts interval division rests on.

```
lemma c_mod_modulus_bound:
  assumes "b ≠ 0"
  shows "abs (c_mod a b) < abs b"
proof -
  have "0 ≤ abs a mod abs b" "abs a mod abs b < abs b"
    using assms by (auto intro: pos_mod_sign pos_mod_bound)
  then show ?thesis unfolding c_mod_abs
    by (auto simp: sgn_if abs_mult)
qed

lemma c_mod_dividend_bound:
  "abs (c_mod a b) ≤ abs a"
  unfolding c_mod_abs
  using zmod_le_nonneg_dividend[of "abs a" "abs b"]
```

```
  by (cases "b = 0") (auto simp: sgn_if abs_mult intro: pos_mod_sign)

lemma c_div_nonneg_divisor:
  assumes "0 < b"
  shows "c_div a b = (if a < 0 then - ((-a) div b) else a div b)"
  using assms by (auto simp: c_div_def sgn_if)

lemma c_div_mono_dividend:
  assumes "a1 ≤ a2" "0 < b"
  shows "c_div a1 b ≤ c_div a2 b"
proof (cases "a1 < 0 ∧ 0 ≤ a2")
  case True
  have "c_div a1 b ≤ 0" "0 ≤ c_div a2 b"
    using True assms c_div_nonpos(1) c_div_nonneg(1) by auto
  then show ?thesis by linarith
next
  case False
  show ?thesis unfolding c_div_nonneg_divisor[OF assms(2)]
    using False assms
    by (auto split: if_splits intro: zdiv_mono1)
qed

lemma c_div_antimono_divisor:
  assumes "0 ≤ a" "0 < b1" "b1 ≤ b2"
  shows "c_div a b2 ≤ c_div a b1"
  using assms
  by (simp add: c_div_nonneg_divisor zdiv_mono2)

lemma c_div_positive_divisor_bounds:
  assumes "0 < l" "l ≤ b" "b ≤ u"
  shows "min (c_div a l) (c_div a u) ≤ c_div a b ∧
    c_div a b ≤ max (c_div a l) (c_div a u)"
proof (cases "0 ≤ a")
  case True
  have "c_div a u ≤ c_div a b" "c_div a b ≤ c_div a l"
    using assms True by (auto intro: c_div_antimono_divisor)
  then show ?thesis by auto
next
  case False
  have "c_div (-a) u ≤ c_div (-a) b" "c_div (-a) b ≤ c_div (-a) l"
    using assms False by (auto intro: c_div_antimono_divisor)
  then show ?thesis
    by (simp_all add: c_div_neg_dividend min_le_iff_disj le_max_iff_disj)
qed

lemma c_div_positive_corners:
  assumes "l ≤ i" "i ≤ u" "0 < c" "c ≤ j" "j ≤ d"
  shows "min (c_div l c) (c_div l d) ≤ c_div i j"
    and "c_div i j ≤ max (c_div u c) (c_div u d)"
proof -
  have "min (c_div l c) (c_div l d) ≤ c_div l j"
    using c_div_positive_divisor_bounds[OF assms(3-5), THEN conjunct1] .
  also have "c_div l j ≤ c_div i j"
    using assms by (auto intro: c_div_mono_dividend)
  finally show "min (c_div l c) (c_div l d) ≤ c_div i j" .
  have "c_div i j ≤ c_div u j"
    using assms by (auto intro: c_div_mono_dividend)
  also have "c_div u j ≤ max (c_div u c) (c_div u d)"
    using c_div_positive_divisor_bounds[OF assms(3-5), THEN conjunct2] .
  finally show "c_div i j ≤ max (c_div u c) (c_div u d)" .
qed

fun ivl_div_positive_core :: "ivl ⇒ ivl ⇒ ivl" where
  "ivl_div_positive_core (Ivl (Fin l) (Fin u)) (Ivl (Fin c) (Fin d)) =
    (if 0 < c then
      Ivl (Fin (min (c_div l c) (c_div l d))) (Fin (max (c_div u c) (c_div u d)))
     else ivl_top)"
| "ivl_div_positive_core a b = (if b = Ivl (Fin 1) (Fin 1) then a else ivl_top)"

definition ivl_div_positive :: "ivl ⇒ ivl ⇒ ivl" where
  "ivl_div_positive a b =
    (if ivl_nonempty a ∧ ivl_nonempty b
     then normalize_ivl (ivl_div_positive_core a b) else bot)"

lemma ivl_div_positive_core_sound:
  assumes "i ∈ gamma_ivl a" "j ∈ gamma_ivl b"
```

```
  shows "c_div i j ∈ gamma_ivl (ivl_div_positive_core a b)"
  using assms
  by (cases "(a, b)" rule: ivl_div_positive_core.cases)
     (auto simp: gamma_ivl_top intro: c_div_positive_corners)

lemma ivl_div_positive_sound:
  "i ∈ gamma_ivl a ⟹ j ∈ gamma_ivl b ⟹
    c_div i j ∈ gamma_ivl (ivl_div_positive a b)"
  unfolding ivl_div_positive_def
  by (auto simp: normalize_ivl_gamma
      intro: ivl_div_positive_core_sound dest: gamma_ivl_nonempty)

definition ivl_positive_part :: "ivl ⇒ ivl" where
  "ivl_positive_part a = intersect_ivl a (Ivl (Fin 1) PlusInf)"

definition ivl_zero_part :: "ivl ⇒ ivl" where
  "ivl_zero_part a = intersect_ivl a (Ivl (Fin 0) (Fin 0))"

definition ivl_div :: "ivl ⇒ ivl ⇒ ivl" where
  "ivl_div a b =
    (if ivl_nonempty a ∧ ivl_nonempty b then
      ivl_div_positive a (ivl_positive_part b) ⊔
      ivl_div_positive (Ivl (Fin 0) (Fin 0) - a)
        (ivl_positive_part (Ivl (Fin 0) (Fin 0) - b)) ⊔ ivl_zero_part b
     else bot)"

lemma ivl_div_join_left:
  "x ∈ gamma_ivl a ⟹ x ∈ gamma_ivl (a ⊔ b)"
  using gamma_ivl_mono[OF le_supI1] by blast

lemma ivl_div_join_right:
  "x ∈ gamma_ivl b ⟹ x ∈ gamma_ivl (a ⊔ b)"
  using gamma_ivl_mono[OF le_supI2] by blast

lemma ivl_div_sound:
  assumes "i ∈ gamma_ivl a" "j ∈ gamma_ivl b"
  shows "c_div i j ∈ gamma_ivl (ivl_div a b)"
proof -
  have ne: "ivl_nonempty a" "ivl_nonempty b"
    using assms by (auto dest: gamma_ivl_nonempty)
  show ?thesis
  proof (cases "j = 0")
    case True
    then have "c_div i j ∈ gamma_ivl (ivl_zero_part b)"
      unfolding ivl_zero_part_def using assms True
      by (intro intersect_ivl_gamma) auto
    then show ?thesis unfolding ivl_div_def using ne
      by (auto intro: ivl_div_join_right)
  next
    case nz: False
    show ?thesis
    proof (cases "0 < j")
      case True
      have "j ∈ gamma_ivl (ivl_positive_part b)"
        unfolding ivl_positive_part_def using assms True
        by (intro intersect_ivl_gamma) auto
      then have "c_div i j ∈ gamma_ivl (ivl_div_positive a (ivl_positive_part b))"
        using assms(1) by (auto intro: ivl_div_positive_sound)
      then show ?thesis unfolding ivl_div_def using ne
        by (auto intro: ivl_div_join_left)
    next
      case False
      have ni: "-i ∈ gamma_ivl (Ivl (Fin 0) (Fin 0) - a)"
        using ivl_minus_sound[of 0 _ i a] assms(1) by simp
      have nj: "-j ∈ gamma_ivl (ivl_positive_part (Ivl (Fin 0) (Fin 0) - b))"
        unfolding ivl_positive_part_def
        apply (rule intersect_ivl_gamma)
         apply (use ivl_minus_sound[of 0 "Ivl (Fin 0) (Fin 0)" j b] assms(2) in simp)
        using False nz by auto
      have "c_div i j ∈ gamma_ivl
        (ivl_div_positive (Ivl (Fin 0) (Fin 0) - a)
          (ivl_positive_part (Ivl (Fin 0) (Fin 0) - b)))"
        using ivl_div_positive_sound[OF ni nj]
        by (simp add: c_div_neg_dividend c_div_neg_divisor)
      then show ?thesis unfolding ivl_div_def using ne
        by (auto intro: ivl_div_join_left
```

```
            ivl_div_join_right)
    qed
  qed
qed
```

Remainder bounds

`ivl_mod_bound` bounds a remainder by the divisor alone. When the divisor interval excludes zero, `abs (c_mod i j) < abs j` gives a symmetric interval one inside the largest divisor magnitude; otherwise the bound is `ivl_top`.

```
fun ivl_mod_bound :: "ivl ⇒ ivl" where
  "ivl_mod_bound (Ivl (Fin l) (Fin u)) =
    (if 0 < l ∨ u < 0 then
      Ivl (Fin (1 - max (abs l) (abs u))) (Fin (max (abs l) (abs u) - 1))
     else ivl_top)"
| "ivl_mod_bound _ = ivl_top"

lemma abs_between_bounds:
  fixes l i u :: int
  assumes "l ≤ i" "i ≤ u"
  shows "abs i ≤ max (abs l) (abs u)"
  using assms by (auto simp: abs_if)

lemma ivl_mod_bound_sound:
  assumes "j ∈ gamma_ivl b"
  shows "c_mod i j ∈ gamma_ivl (ivl_mod_bound b)"
proof (cases b rule: ivl_mod_bound.cases)
  case (1 l u)
  with assms have bounds: "l ≤ j" "j ≤ u" by simp_all
  have magnitude: "abs j ≤ max (abs l) (abs u)"
    by (rule abs_between_bounds[OF bounds])
  show ?thesis unfolding 1 ivl_mod_bound.simps
    using bounds magnitude c_mod_modulus_bound[of j i]
    by (auto simp: gamma_ivl_top; linarith)
qed (simp_all add: gamma_ivl_top)

lemma ivl_mod_dividend_sound:
  assumes "i ∈ gamma_ivl a"
  shows "c_mod i j ∈ gamma_ivl (a ⊔ Ivl (Fin 0) (Fin 0))"
proof -
  have range: "min 0 i ≤ c_mod i j ∧ c_mod i j ≤ max 0 i"
    using c_mod_dividend_bound[of i j] c_mod_nonneg[of i j] c_mod_nonpos[of i j]
    by (auto simp: abs_if min_def max_def split: if_splits; linarith)
  obtain l u where a: "a = Ivl l u" by (cases a)
  show ?thesis using assms range
    unfolding a sup_ivl_def
    by (cases l; cases u; auto simp: min_def max_def split: if_splits)
qed

definition ivl_mod :: "ivl ⇒ ivl ⇒ ivl" where
  "ivl_mod a b =
    (if ivl_nonempty a ∧ ivl_nonempty b then
      intersect_ivl (a - ivl_div a b * b)
        (intersect_ivl (a ⊔ Ivl (Fin 0) (Fin 0)) (ivl_mod_bound b))
     else bot)"

lemma ivl_mod_sound:
  assumes "i ∈ gamma_ivl a" "j ∈ gamma_ivl b"
  shows "c_mod i j ∈ gamma_ivl (ivl_mod a b)"
proof -
  have ne: "ivl_nonempty a" "ivl_nonempty b"
    using assms by (auto dest: gamma_ivl_nonempty)
  have residual: "i - c_div i j * j ∈ gamma_ivl (a - ivl_div a b * b)"
    by (intro ivl_minus_sound ivl_times_sound ivl_div_sound assms)
  have "c_mod i j ∈ gamma_ivl (a - ivl_div a b * b)"
    using residual by (cases "j = 0") (simp_all add: c_mod_def)
  then show ?thesis unfolding ivl_mod_def
    using assms(1,2) intersect_ivl_gamma ivl_mod_bound_sound ivl_mod_dividend_sound ne(1,2)
    by presburger
qed
```

Results are canonical

Every public interval operation returns a canonical representative: a non-empty interval, or $\bot$ itself. No operation manufactures a fresh inverted bound pair such as `Ivl (Fin 2) (Fin 1)`, so structural equality on results

distinguishes exactly the abstract values they denote.

```
lemma normalize_ivl_plus [simp]: "normalize_ivl (a + b) = a + b"
  by (simp add: plus_ivl_norm split: ivl.splits prod.splits)

lemma normalize_ivl_minus [simp]: "normalize_ivl (a - b) = a - b"
  by (simp add: minus_ivl_norm split: ivl.splits prod.splits)

lemma normalize_ivl_ivl_min [simp]: "normalize_ivl (ivl_min a b) = ivl_min a b"
  by (cases a; cases b) simp

lemma normalize_ivl_ivl_max [simp]: "normalize_ivl (ivl_max a b) = ivl_max a b"
  by (cases a; cases b) simp

lemma is_bottom_ivl_times_core [simp]:
  "¬ is_bottom_ivl (ivl_times_core x y)"
proof (induction x y rule: ivl_times_core.induct)
  case (1 l1 u1 l2 u2)
  show ?case
    by (auto simp: is_bottom_ivl_def min_le_iff_disj le_max_iff_disj)
qed (simp_all add: is_bottom_ivl_def ivl_top_def)

lemma normalize_ivl_times [simp]: "normalize_ivl (a * b) = a * b"
  by (cases "ivl_nonempty a ∧ ivl_nonempty b") (auto simp: times_ivl_def)

end
```

12.3 Interval_Numeric_Queries

```
theory Interval_Numeric_Queries
  imports Interval_Warrowing "Voblint_Domain.Backward_Numeric_Queries"
begin
```

Interval interpretation of the generic numeric-query interface

Second-domain validation for `sound_numeric_queries` (`Voblint_Domain.Numeric_Queries`): a real, minimal Interval instance, interpreted without touching that generic theory. Entailment/refutation of `<`/`=` reduces to comparing bounds: two intervals' `<` relation is provable exactly when one interval's upper bound is strictly below the other's lower bound; `=` exactly when both are the same singleton point; $\neq$ exactly when the intervals are disjoint. Empty intervals (`l > u`) make every judgment vacuously true, the same role `SBot` plays for Sign.

```
lemma gamma_ivlD:
  assumes "x ∈ gamma_ivl (Ivl l u)"
  shows "l ≤ Fin x" and "Fin x ≤ u"
  using assms
  by auto

lemma ivl_upper_lower_less:
  assumes "Fin i ≤ u" and "u < l" and "l ≤ Fin j"
  shows "i < j"
  using le_less_trans[OF assms(1) less_le_trans[OF assms(2,3)]]
  by (simp add: less_eint_def less_eq_eint_def)

lemma ivl_upper_lower_less_eq:
  assumes "Fin i ≤ u" and "u ≤ l" and "l ≤ Fin j"
  shows "i ≤ j"
  using order_trans[OF assms(1) order_trans[OF assms(2,3)]]
  by (simp add: less_eq_eint_def)

lemma ivl_upper_lower_not_less_eq:
  assumes "Fin j ≤ u" and "u ≤ l" and "l ≤ Fin i"
  shows "¬ i < j"
  using ivl_upper_lower_less_eq[OF assms]
  by auto


lemma ivl_separated_not_equal:
  assumes "Fin i ≤ u1"
      and "u1 < l2"
      and "l2 ≤ Fin j"
  shows "i ≠ j"
  using ivl_upper_lower_less[OF assms]
  by auto

fun interval_less_true :: "ivl ⇒ ivl ⇒ bool" where
  "interval_less_true (Ivl l1 u1) (Ivl l2 u2) =
     (¬ l1 ≤ u1 ∨ ¬ l2 ≤ u2 ∨ u1 < l2)"

lemma interval_less_true_sound:
  assumes query: "interval_less_true a b"
      and ai: "i ∈ gamma_ivl a"
      and bj: "j ∈ gamma_ivl b"
  shows "i < j"
proof -
  obtain l1 u1 where a_def: "a = Ivl l1 u1"
    by (cases a)
  obtain l2 u2 where b_def: "b = Ivl l2 u2"
    by (cases b)

  from ai have i_bounds:
    "l1 ≤ Fin i" "Fin i ≤ u1"
    unfolding a_def
    by (auto dest: gamma_ivlD)

  from bj have j_bounds:
    "l2 ≤ Fin j" "Fin j ≤ u2"
    unfolding b_def
    by (auto dest: gamma_ivlD)

  from i_bounds j_bounds have nonempty:
    "l1 ≤ u1" "l2 ≤ u2"
    by order+
```

```
  with query have separated: "u1 < l2"
    unfolding a_def b_def
    by simp

  show ?thesis
    using i_bounds(2) separated j_bounds(1)
    by (rule ivl_upper_lower_less)
qed


fun interval_less_false :: "ivl ⇒ ivl ⇒ bool" where
  "interval_less_false (Ivl l1 u1) (Ivl l2 u2) =
     (¬ l1 ≤ u1 ∨ ¬ l2 ≤ u2 ∨ u2 ≤ l1)"

lemma interval_less_false_sound:
  assumes query: "interval_less_false a b"
      and ai: "i ∈ gamma_ivl a"
      and bj: "j ∈ gamma_ivl b"
  shows "¬ i < j"
proof -
  obtain l1 u1 where a_def: "a = Ivl l1 u1"
    by (cases a)
  obtain l2 u2 where b_def: "b = Ivl l2 u2"
    by (cases b)

  from ai have i_bounds:
    "l1 ≤ Fin i" "Fin i ≤ u1"
    unfolding a_def
    by (auto dest: gamma_ivlD)

  from bj have j_bounds:
    "l2 ≤ Fin j" "Fin j ≤ u2"
    unfolding b_def
    by (auto dest: gamma_ivlD)

  from i_bounds j_bounds have nonempty:
    "l1 ≤ u1" "l2 ≤ u2"
    by order+

  with query have separated: "u2 ≤ l1"
    unfolding a_def b_def
    by simp

  show ?thesis
    using j_bounds(2) separated i_bounds(1)
    by (rule ivl_upper_lower_not_less_eq)
qed

fun interval_eq_true :: "ivl ⇒ ivl ⇒ bool" where
  "interval_eq_true (Ivl l1 u1) (Ivl l2 u2) =
     (¬ l1 ≤ u1 ∨ ¬ l2 ≤ u2 ∨ (l1 = u1 ∧ l2 = u2 ∧ l1 = l2))"

lemma interval_eq_true_sound:
  assumes query: "interval_eq_true a b"
      and ai: "i ∈ gamma_ivl a"
      and bj: "j ∈ gamma_ivl b"
  shows "i = j"
proof -
  obtain l1 u1 where a_def: "a = Ivl l1 u1"
    by (cases a)
  obtain l2 u2 where b_def: "b = Ivl l2 u2"
    by (cases b)

  from ai have i_bounds:
    "l1 ≤ Fin i" "Fin i ≤ u1"
    unfolding a_def
    by (auto dest: gamma_ivlD)

  from bj have j_bounds:
    "l2 ≤ Fin j" "Fin j ≤ u2"
    unfolding b_def
    by (auto dest: gamma_ivlD)

  from i_bounds j_bounds have nonempty:
    "l1 ≤ u1" "l2 ≤ u2"
```

```
    by (meson order_trans)+

  with query have singleton:
    "l1 = u1" "l2 = u2" "l1 = l2"
    unfolding a_def b_def
    by auto

  from ai singleton have "Fin i = l1"
    unfolding a_def
    using i_bounds(1,2) by order

  moreover from bj singleton have "Fin j = l1"
    unfolding b_def
    using j_bounds(1,2) by order

  ultimately show ?thesis
    by blast
qed

fun interval_eq_false :: "ivl ⇒ ivl ⇒ bool" where
  "interval_eq_false (Ivl l1 u1) (Ivl l2 u2) =
     (¬ l1 ≤ u1 ∨ ¬ l2 ≤ u2 ∨ u1 < l2 ∨ u2 < l1)"

lemma interval_eq_false_sound:
  assumes query: "interval_eq_false a b"
      and ai: "i ∈ gamma_ivl a"
      and bj: "j ∈ gamma_ivl b"
  shows "i ≠ j"
proof -
  obtain l1 u1 where a_def: "a = Ivl l1 u1"
    by (cases a)
  obtain l2 u2 where b_def: "b = Ivl l2 u2"
    by (cases b)

  from ai have i_bounds:
    "l1 ≤ Fin i" "Fin i ≤ u1"
    unfolding a_def
    by (auto dest: gamma_ivlD)

  from bj have j_bounds:
    "l2 ≤ Fin j" "Fin j ≤ u2"
    unfolding b_def
    by (auto dest: gamma_ivlD)

  from i_bounds j_bounds have nonempty:
    "l1 ≤ u1" "l2 ≤ u2"
    by (meson order_trans)+

  with query have separated:
    "u1 < l2 ∨ u2 < l1"
    unfolding a_def b_def
    by force

  then show ?thesis
    using i_bounds(1,2) ivl_separated_not_equal j_bounds(1,2) by blast
qed
```

Goblint-style optional-Boolean queries

`interval_less`/`interval_eq` are the generic packaging of the four bound-comparison judgments above, so one interpretation both introduces them and discharges the query interface. The **defines** clause is what keeps them ordinary top-level constants, with the code equation the check layer needs.

```
global_interpretation interval_numeric_queries:
  numeric_query_judgments
    interval_less_true interval_less_false interval_eq_true interval_eq_false
  defines interval_less = interval_numeric_queries.query_less
    and interval_eq = interval_numeric_queries.query_eq
  by unfold_locales
     (auto intro: interval_less_true_sound interval_eq_true_sound
           dest: interval_less_false_sound interval_eq_false_sound)
```

Semantic intersection versus raw lattice meet

Unlike Sign, whose `SBot` is the lattice's unique empty representative, the raw `ivl` type has infinitely many non-canonical empty representations (any `Ivl l u` with `l > u`, documented at `normalize_ivl`), and only `Ivl PlusInf MinInf` is literally `bot`. Testing the raw `(⊓)` against `bot` therefore under-approximates disjointness: two disjoint, non-empty finite intervals meet to an empty-but-non-canonical result. The witness below is disjoint by `interval_eq_false`'s sound table, yet its representation-level meet is not `bot`.

The generic `eq_false` derived in `sound_inverse_ops`'s own context (`Voblint_Domain.Backward_Numeric_Queries`) instead tests semantic intersection via `is_empty`. `intersect_ivl` returns a value `is_empty` classifies as empty on the same witness, so the generic derivation classifies it correctly regardless of whether that value is the canonical $\bot$. Interval also exposes its hand-tuned query table; both operations agree on this disjointness witness. `interval_eq_true` is independent of either operation because it derives equality from `interval_less_false` in both directions.

```
lemma interval_meet_of_witness_not_bot:
  "Ivl (Fin 1) (Fin 2) ⊓ Ivl (Fin 5) (Fin 6) ≠ bot"
  by (simp add: bot_ivl_def)

lemma interval_intersect_of_witness_bot:
  "intersect_ivl (Ivl (Fin 1) (Fin 2)) (Ivl (Fin 5) (Fin 6)) = bot"
  by eval

end
```

12.4 Interval_Kind

```
theory Interval_Kind
  imports
    "Voblint_Analysis_Interval.Interval_Arithmetic"
    "Voblint_Analysis_Interval.Interval_Numeric_Queries"
     "Voblint_Nonrelational.Nonrelational_Domain"
begin
```

Intervals at a kind

The kind-indexed operations of Goblint's `IntervalFunctor` without widening thresholds. Widening keeps a bound that did not grow and moves a growing bound to the kind's limit. Narrowing replaces only a bound that sits at the kind's limit.

Goblint widens operands of the kind, whose bounds already lie in its range. The class laws must hold for every operand, so a growing bound moves to the kind's limit or to the new bound, whichever is further out; on operands in range this is the kind's limit.

Bounds are compared in the linear order of `eint`, not by unfolding it.

```
declare less_eq_eint_def [simp del]

definition ik_lo :: "ikind ⇒ eint" where "ik_lo k = Fin (ik_min k)"
definition ik_hi :: "ikind ⇒ eint" where "ik_hi k = Fin (ik_max k)"

fun widen_ivl_ik :: "ikind ⇒ ivl ⇒ ivl ⇒ ivl" where
  "widen_ivl_ik k (Ivl l0 u0) (Ivl l1 u1) =
     Ivl (if l0 ≤ l1 then l0 else min (ik_lo k) l1)
         (if u1 ≤ u0 then u0 else max (ik_hi k) u1)"

fun narrow_ivl_ik :: "ikind ⇒ ivl ⇒ ivl ⇒ ivl" where
  "narrow_ivl_ik k (Ivl l0 u0) (Ivl l1 u1) =
     Ivl (if l0 = ik_lo k then l1 else l0) (if u0 = ik_hi k then u1 else u0)"
```

The canonical bottom stays neutral, as for the kind-free widening.

```
instantiation ivl :: ik_domain
begin

definition widen_ik_ivl :: "ikind ⇒ ivl ⇒ ivl ⇒ ivl" where
  "widen_ik_ivl k a b = (if a = bot then b else if b = bot then a else widen_ivl_ik k a b)"

definition narrow_ik_ivl :: "ikind ⇒ ivl ⇒ ivl ⇒ ivl" where
  "narrow_ik_ivl = narrow_ivl_ik"

definition empty_ik_ivl :: "ikind ⇒ ivl ⇒ bool" where
  "empty_ik_ivl k a = is_bottom_ivl (a ⊓ Ivl (ik_lo k) (ik_hi k))"

instance proof
  fix k :: ikind and a b :: ivl
  obtain l0 u0 l1 u1 where a: "a = Ivl l0 u0" and b: "b = Ivl l1 u1"
    by (meson ivl_exhaustE)
  have lo: "(if l0 ≤ l1 then l0 else min (ik_lo k) l1) ≤ l0"
       "(if l0 ≤ l1 then l0 else min (ik_lo k) l1) ≤ l1"
    using linear[of l0 l1] by (auto intro: order.trans[OF min.cobounded2])
  have hi: "u0 ≤ (if u1 ≤ u0 then u0 else max (ik_hi k) u1)"
       "u1 ≤ (if u1 ≤ u0 then u0 else max (ik_hi k) u1)"
    using linear[of u0 u1] by (auto intro: order.trans[OF _ max.cobounded2])
  have core1: "a ≤ widen_ivl_ik k a b" and core2: "b ≤ widen_ivl_ik k a b"
    using lo hi by (simp_all only: a b widen_ivl_ik.simps less_eq_ivl_def prod.case ivl.case)
  have bot_le: "bot ≤ c" for c :: ivl
    by simp
  show "a ≤ widen_ik k a b"
    using core1 bot_le by (auto simp: widen_ik_ivl_def)
  show "b ≤ widen_ik k a b"
    using core2 bot_le by (auto simp: widen_ik_ivl_def)
  show "b ≤ a ⟹ b ≤ narrow_ik k a b"
    by (cases a; cases b) (auto simp: narrow_ik_ivl_def less_eq_ivl_def)
  show "b ≤ a ⟹ narrow_ik k a b ≤ a"
    by (cases a; cases b) (auto simp: narrow_ik_ivl_def less_eq_ivl_def)
  have range: "gamma_ivl (Ivl (ik_lo k) (ik_hi k)) = ik_range k"
    by (auto simp: ik_lo_def ik_hi_def less_eq_eint_def)
  show "empty_ik k a ⟷ numeric_domain_class.gamma a ∩ ik_range k = {}"
    by (simp add: empty_ik_ivl_def is_bottom_ivl_correct flip: range)
qed
```

```
end
```

Intervals as a kind-aware integer domain

An operation at kind `k` computes the mathematical interval and keeps it when it lies in `k`'s range. Otherwise it normalizes the interval as Goblint does: an unsigned kind wraps it when the wrapped interval is exact, and every other overflow answers `top_of k`.

```
definition ik_ivl :: "ikind ⇒ ivl" where "ik_ivl k = Ivl (ik_lo k) (ik_hi k)"

lemma gamma_ik_ivl [simp]: "gamma_ivl (ik_ivl k) = ik_range k"
  by (auto simp: ik_ivl_def ik_lo_def ik_hi_def less_eq_eint_def)
```

A kind that wraps under the overflow policy wraps a finite interval that spans less than its modulus and whose wrapped ends stay ordered; any other overflow gives the kind's whole range. This is Goblint's `norm` in `intervalDomain.ml`: under the default `sem.int.signed_overflow = assume_top` only an unsigned kind wraps, under `assume_wraparound` every kind does. A cast and an arithmetic result are treated alike; they differ only under `assume_none`, which assumes that no signed overflow occurs.

```
definition wrap_ivl_ik :: "ikind ⇒ int ⇒ int ⇒ ivl" where
  "wrap_ivl_ik k x y =
     (if y - x < ik_mod k ∧ ik_norm k x ≤ ik_norm k y
      then Ivl (Fin (ik_norm k x)) (Fin (ik_norm k y)) else ik_ivl k)"

definition norm_ivl_ik :: "signed_overflow_policy ⇒ ikind ⇒ ivl ⇒ ivl" where
  "norm_ivl_ik ovp k a =
     (if a ≤ ik_ivl k then a
      else case a of
        Ivl (Fin x) (Fin y) ⇒ if should_wrap ovp k then wrap_ivl_ik k x y else ik_ivl k
      | _ ⇒ ik_ivl k)"

lemma ik_max_min_mod: "ik_max k - ik_min k + 1 = ik_mod k"
  by (cases k) (simp_all add: ik_max_def ik_min_def ik_half_def ik_mod_def)

lemma wrap_ivl_ik_sound:
  assumes "x ≤ n" and "n ≤ y"
  shows "ik_norm k n ∈ gamma_ivl (wrap_ivl_ik k x y)"
proof (cases "y - x < ik_mod k ∧ ik_norm k x ≤ ik_norm k y")
  case True
  then have w: "y - x ≤ ik_max k - ik_min k" using ik_max_min_mod[of k] by linarith
  have "ik_norm k n = ik_norm k x + (n - x)"
    using True assms(1,2) ik_norm_interval_shift w by blast
  moreover have "ik_norm k y = ik_norm k x + (y - x)"
    using True assms(1,2) ik_norm_interval_shift order_trans w by blast
  ultimately show ?thesis
    using True assms by (simp add: wrap_ivl_ik_def less_eq_eint_def)
qed (auto simp: wrap_ivl_ik_def)

lemma norm_ivl_ik_sound:
  assumes "n ∈ gamma_ivl a"
  shows "ik_norm k n ∈ gamma_ivl (norm_ivl_ik ovp k a)"
proof (cases "a ≤ ik_ivl k")
  case True
  then have "n ∈ ik_range k" using gamma_ivl_mono[OF True] assms by auto
  with True assms show ?thesis by (simp add: norm_ivl_ik_def)
next
  case False
  obtain l u where a: "a = Ivl l u" by (cases a)
  show ?thesis
  proof (cases "∃x y. l = Fin x ∧ u = Fin y")
    case True
    then obtain x y where l: "l = Fin x" and u: "u = Fin y" by blast
    have "x ≤ n" "n ≤ y" using assms by (simp_all add: a l u less_eq_eint_def)
    with False show ?thesis by (simp add: norm_ivl_ik_def a l u wrap_ivl_ik_sound)
  next
    case nw: False
    then have "norm_ivl_ik ovp k a = ik_ivl k"
      using False by (cases l; cases u) (auto simp: norm_ivl_ik_def a)
    then show ?thesis by simp
  qed
qed

fun ivl_aop :: "aop ⇒ ivl ⇒ ivl ⇒ ivl" where
  "ivl_aop AAdd a b = a + b"
```

```
| "ivl_aop ASub a b = a - b"
| "ivl_aop AMul a b = a * b"
| "ivl_aop ADiv a b = ivl_div a b"
| "ivl_aop AMod a b = ivl_mod a b"

lemma ivl_aop_sound:
  "m ∈ gamma_ivl a ⟹ n ∈ gamma_ivl b ⟹ aop_eval op m n ∈ gamma_ivl (ivl_aop op a b)"
  by (cases op) (simp_all add: ivl_plus_sound ivl_minus_sound ivl_times_sound ivl_div_sound
      ivl_mod_sound)

fun ivl_lo :: "ivl ⇒ eint" where "ivl_lo (Ivl l u) = l"
fun ivl_hi :: "ivl ⇒ eint" where "ivl_hi (Ivl l u) = u"

lemma gamma_ivl_bounds:
  assumes "n ∈ gamma_ivl a"
  shows "ivl_lo a ≤ Fin n" and "Fin n ≤ ivl_hi a"
  using assms by (cases a; simp)+
```

The bounds a comparison known to hold puts on its operands: `m < n` bounds `m` by `n`'s upper bound less one and `n` by `m`'s lower bound plus one.

```
definition ivl_le_bounds :: "ivl ⇒ ivl ⇒ ivl × ivl" where
  "ivl_le_bounds a b = (a ⊓ Ivl MinInf (ivl_hi b), b ⊓ Ivl (ivl_lo a) PlusInf)"

definition ivl_lt_bounds :: "ivl ⇒ ivl ⇒ ivl × ivl" where
  "ivl_lt_bounds a b =
     (a ⊓ Ivl MinInf (ivl_hi b - Fin 1), b ⊓ Ivl (ivl_lo a + Fin 1) PlusInf)"

lemma eint_le_minus_one: "Fin m < Fin n ⟹ Fin n ≤ u ⟹ Fin m ≤ u - Fin 1"
  by (cases u) (auto simp: less_eq_eint_def less_eint_def)

lemma eint_plus_one_le: "Fin m < Fin n ⟹ l ≤ Fin m ⟹ l + Fin 1 ≤ Fin n"
  by (cases l) (auto simp: less_eq_eint_def less_eint_def)

lemma ivl_le_bounds_sound:
  assumes "m ≤ n" "m ∈ gamma_ivl a" "n ∈ gamma_ivl b"
  shows "m ∈ gamma_ivl (fst (ivl_le_bounds a b)) ∧ n ∈ gamma_ivl (snd (ivl_le_bounds a b))"
proof -
  have mn: "eint_le (Fin m) (Fin n)" using assms(1) by simp
  have "Fin m ≤ ivl_hi b" "ivl_lo a ≤ Fin n"
    using eint_le_trans[OF mn gamma_ivl_bounds(2)[OF assms(3), unfolded less_eq_eint_def]]
      eint_le_trans[OF gamma_ivl_bounds(1)[OF assms(2), unfolded less_eq_eint_def] mn]
    by (simp_all add: less_eq_eint_def)
  then show ?thesis using assms
    by (auto simp: ivl_le_bounds_def less_eq_eint_def intro!: meet_ivl_gamma)
qed

lemma ivl_lt_bounds_sound:
  assumes "m < n" "m ∈ gamma_ivl a" "n ∈ gamma_ivl b"
  shows "m ∈ gamma_ivl (fst (ivl_lt_bounds a b)) ∧ n ∈ gamma_ivl (snd (ivl_lt_bounds a b))"
proof -
  have lt: "Fin m < Fin n" using assms(1) by (simp add: less_eint_def)
  have "Fin m ≤ ivl_hi b - Fin 1" "ivl_lo a + Fin 1 ≤ Fin n"
    using eint_le_minus_one[OF lt gamma_ivl_bounds(2)[OF assms(3)]]
      eint_plus_one_le[OF lt gamma_ivl_bounds(1)[OF assms(2)]] by blast+
  then show ?thesis using assms
    by (auto simp: ivl_lt_bounds_def less_eq_eint_def intro!: meet_ivl_gamma)
qed
```

A disequality with a known constant `x` moves a bound of the other side that equals `x` past it, as `inv_bin_int` excludes the interval bounds for `Ne` (`baseInvariant.ml`, lines 395–416 at `0dc12d355`); a singleton equal to `x` becomes empty.

```
fun ivl_excl :: "ivl ⇒ ivl ⇒ ivl" where
  "ivl_excl (Ivl l u) (Ivl (Fin x) (Fin y)) =
     (if x = y then Ivl (if l = Fin x then Fin (x + 1) else l) (if u = Fin x then Fin (x - 1) else u)
      else Ivl l u)"
| "ivl_excl a b = a"

lemma ivl_excl_sound:
  "m ≠ n ⟹ m ∈ gamma_ivl a ⟹ n ∈ gamma_ivl b ⟹ m ∈ gamma_ivl (ivl_excl a b)"
  by (induction a b rule: ivl_excl.induct) (auto simp: less_eq_eint_def)

fun ivl_refine :: "cop ⇒ ivl ⇒ ivl ⇒ ivl × ivl" where
  "ivl_refine CLt a b = ivl_lt_bounds a b"
| "ivl_refine CLe a b = ivl_le_bounds a b"
| "ivl_refine CGt a b = prod.swap (ivl_lt_bounds b a)"
```

```
| "ivl_refine CGe a b = prod.swap (ivl_le_bounds b a)"
| "ivl_refine CEq a b = (a ⊓ b, a ⊓ b)"
| "ivl_refine CNe a b = (ivl_excl a b, ivl_excl b a)"

lemma ivl_refine_sound:
  "cop_eval c m n ⟹ m ∈ gamma_ivl a ⟹ n ∈ gamma_ivl b
     ⟹ m ∈ gamma_ivl (fst (ivl_refine c a b)) ∧ n ∈ gamma_ivl (snd (ivl_refine c a b))"
  by (cases c)
     (auto dest: ivl_le_bounds_sound ivl_lt_bounds_sound intro: meet_ivl_gamma ivl_excl_sound)

instantiation ivl :: kind_int_domain
begin

definition top_ik_ivl :: "ikind ⇒ ivl" where "top_ik_ivl = ik_ivl"

definition of_int_ik_ivl :: "ikind ⇒ int ⇒ ivl" where
  "of_int_ik_ivl k n = Ivl (Fin (ik_norm k n)) (Fin (ik_norm k n))"

definition cast_ik_ivl :: "signed_overflow_policy ⇒ ikind option ⇒ ikind ⇒ ivl ⇒ ivl" where
  "cast_ik_ivl ovp sk = norm_ivl_ik ovp"

definition arith_ik_ivl :: "signed_overflow_policy ⇒ ikind ⇒ aop ⇒ bool ⇒ ivl ⇒ ivl ⇒ ivl" where
  "arith_ik_ivl ovp k op fits a b = norm_ivl_ik ovp k (ivl_aop op a b)"

definition cmp_ik_ivl :: "cop ⇒ ivl ⇒ ivl ⇒ bool option" where
  "cmp_ik_ivl = cop_of interval_less interval_eq"

definition meet_ik_ivl :: "ivl ⇒ ivl ⇒ ivl" where "meet_ik_ivl a b = a ⊓ b"

definition refine_ik_ivl :: "cop ⇒ ivl ⇒ ivl ⇒ ivl × ivl" where "refine_ik_ivl = ivl_refine"

definition raw_aop_ik_ivl :: "aop ⇒ ivl ⇒ ivl ⇒ ivl" where "raw_aop_ik_ivl = ivl_aop"

definition fits_ik_ivl :: "ikind ⇒ ivl ⇒ bool" where "fits_ik_ivl k a ⟷ a ≤ ik_ivl k"

definition answer_ik_ivl :: "ivl ⇒ answer" where "answer_ik_ivl = answer_of_ivl"

definition of_answer_ik_ivl :: "ikind option ⇒ answer ⇒ ivl" where
  "of_answer_ik_ivl sk q =
     (let r = (case sk of Some k ⇒ ik_ivl k | None ⇒ ⊤)
      in case q of QBot ⇒ ⊥ | QTop ⇒ r | QLifted d ⇒ int_ivl d ⊓ r)"

instance proof
  fix n m :: int and k :: ikind and a b :: ivl and op :: aop and c :: cop and r :: bool
  show "n ∈ ik_range k ⟹ n ∈ numeric_domain_class.gamma (top_ik k :: ivl)"
    by (simp add: top_ik_ivl_def)
  show "ik_norm k n ∈ numeric_domain_class.gamma (of_int_ik k n :: ivl)"
    by (simp add: of_int_ik_ivl_def less_eq_eint_def)
  show "(⋀k'. sk = Some k' ⟹ n ∈ ik_range k') ⟹ n ∈ numeric_domain_class.gamma a
     ⟹ ik_norm k n ∈ numeric_domain_class.gamma (cast_ik ovp sk k a)" for ovp sk
    by (simp add: cast_ik_ivl_def norm_ivl_ik_sound)
  show "(fits ⟹ m ∈ ik_range k ∧ n ∈ ik_range k) ⟹ m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma
b
     ⟹ ik_norm k (aop_eval op m n) ∈ numeric_domain_class.gamma (arith_ik ovp k op fits a b)" for ovp fits
    by (simp add: arith_ik_ivl_def norm_ivl_ik_sound ivl_aop_sound)
  show "cmp_ik c a b = Some r ⟹ m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b ⟹ cop_eval
c m n = r"
    unfolding cmp_ik_ivl_def
    by (rule cop_of_sound[OF interval_numeric_queries.less_sound interval_numeric_queries.eq_sound])
  show "m ∈ numeric_domain_class.gamma a ⟹ m ∈ numeric_domain_class.gamma b
     ⟹ m ∈ numeric_domain_class.gamma (meet_ik a b)"
    by (simp add: meet_ik_ivl_def meet_ivl_gamma)
  show "cop_eval c m n ⟹ m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
     ⟹ m ∈ numeric_domain_class.gamma (fst (refine_ik c a b))
       ∧ n ∈ numeric_domain_class.gamma (snd (refine_ik c a b))"
    by (simp add: refine_ik_ivl_def ivl_refine_sound)
  show "m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
     ⟹ aop_eval op m n ∈ numeric_domain_class.gamma (raw_aop_ik op a b)"
    by (simp add: raw_aop_ik_ivl_def ivl_aop_sound)
  show "fits_ik k a ⟹ n ∈ numeric_domain_class.gamma a ⟹ n ∈ ik_range k"
    using gamma_ivl_mono[of a "ik_ivl k"] by (auto simp: fits_ik_ivl_def simp del: ik_range_iff)
  show "n ∈ numeric_domain_class.gamma a ⟹ n ∈ gamma_answer (answer_ik a)"
    by (simp add: answer_ik_ivl_def)
  show "(⋀k. sk = Some k ⟹ n ∈ ik_range k) ⟹ n ∈ gamma_answer q
     ⟹ n ∈ numeric_domain_class.gamma (of_answer_ik sk q :: ivl)" for sk q
    by (cases sk; cases q)
```

```
      (auto simp: of_answer_ik_ivl_def gamma_int_dom_def meet_ivl_gamma top_ivl_def gamma_ivl_top
        simp del: ik_range_iff)
```
qed

end

`kind_int_domain` extends `numeric_domain`, so the code of a typed analysis carries the concretization in its class dictionary. It is never called, and its equation is not executable at `int`.

declare `[[code abort: gamma_ivl]]`

end

12.5 Interval_Regression

```
theory Interval_Regression
  imports Interval_Kind "Voblint_Nonrelational.Nonrelational_Analysis" "Voblint_VIMP.VIMP_Notation"
begin
```

Regression: the typed Interval analysis, executed

Each case runs the certified solver on a typed program inside Isabelle and pins the published value of one variable. Termination is decided by evaluation.

```
abbreviation ivl_equations :: "kind_decls ⇒ imp_prog ⇒ cfg_node × cfg_node list ⇒
    (cfg_node × cfg_node list, (unit, cfg_node list) global_unknown,
     (ivl tst lifted, ivl tst lifted) dg_state) strategy_tree" where
  "ivl_equations ds p ≡ nonrel.equations default_sem_conf ds 0 is_global p"

abbreviation ivl_state_at :: "kind_decls ⇒ imp_prog ⇒ cfg_node ⇒ ivl tabs" where
  "ivl_state_at ds p ≡ nonrel.state_at default_sem_conf ds 0 Globals_Join is_global p []"

abbreviation main_var :: "vname ⇒ var_id" where
  "main_var x ≡ ScopedId (STR ''main'') x"
```

A counting loop at `I32`

The guard bounds the body and the loop exit.

```
definition loop_prog :: imp_prog where
  "loop_prog = program { fun main() { x = 0; while (x < 20) { x = x + 1; } } }"

lemma loop_terminates_c:
  "TD_side_rule_Interp_solve_c Globals_Join (ivl_equations [] loop_prog)
     (cfg_exit (elab_cfg loop_prog []), []) ≠ None"
  by eval

lemma loop_terminates:
  "nonrel.terminates TYPE(ivl) default_sem_conf [] 0 Globals_Join is_global loop_prog"
  unfolding nonrel.terminates_code
  using TD_side_rule_Interp.solve_dom_of_solve_c[OF loop_terminates_c] by simp

abbreviation loop_at :: "pp ⇒ ivl tabs" where
  "loop_at ≡ ivl_state_at [] loop_prog"

lemma loop_values:
  "loop_at (Statement 1) (main_var (STR ''x'')) = KD I32 (Ivl (Fin 0) (Fin 20))"
  "loop_at (Statement 2) (main_var (STR ''x'')) = KD I32 (Ivl (Fin 0) (Fin 19))"
  "loop_at (Statement 3) (main_var (STR ''x'')) = KD I32 (Ivl (Fin 20) (Fin 20))"
  by eval+
```

An unsigned char that wraps

`x` is a `U8`; `x + 10` is computed at `I32` and converted back. The value `260` leaves the range of `U8` and wraps to `4`, as in Goblint.

```
definition wrap_prog :: imp_prog where
  "wrap_prog = program { fun main() { x = 250; x = x + 10; } }"

definition wrap_kinds :: kind_decls where
  "wrap_kinds = [(main_var (STR ''x''), U8)]"

lemma wrap_terminates_c:
  "TD_side_rule_Interp_solve_c Globals_Join (ivl_equations wrap_kinds wrap_prog)
     (cfg_exit (elab_cfg wrap_prog wrap_kinds), []) ≠ None"
  by eval

abbreviation wrap_at :: "pp ⇒ ivl tabs" where
  "wrap_at ≡ ivl_state_at wrap_kinds wrap_prog"

lemma wrap_values:
  "wrap_at (Statement 1) (main_var (STR ''x'')) = KD U8 (Ivl (Fin 250) (Fin 250))"
  "wrap_at (Statement 2) (main_var (STR ''x'')) = KD U8 (Ivl (Fin 4) (Fin 4))"
  by eval+
```

A signed char that overflows

A signed kind does not wrap under Goblint's default overflow policy: converting `130` to `I8` answers the whole range.

```
definition swrap_prog :: imp_prog where
  "swrap_prog = program { fun main() { x = 120; x = x + 10; } }"

definition swrap_kinds :: kind_decls where
  "swrap_kinds = [(main_var (STR ''x''), I8)]"

abbreviation swrap_at :: "pp ⇒ ivl tabs" where
  "swrap_at ≡ ivl_state_at swrap_kinds swrap_prog"

lemma swrap_values:
  "swrap_at (Statement 2) (main_var (STR ''x'')) = KD I8 (Ivl (Fin (-128)) (Fin 127))"
  by eval
```

Backward refinement through arithmetic and conversions

Two calls give `f`'s formal `x` the range `[0,99]`. Then `x + 1` cannot overflow, so `x + 1 < 10` bounds `x` by `8`; without the inversion `y` would be `[0,99]` when `f` returns.

```
abbreviation f_var :: "vname ⇒ var_id" where
  "f_var x ≡ ScopedId (STR ''f'') x"

definition arith_prog :: imp_prog where
  "arith_prog = program {
     fun f(x) { y = 0; if (x + 1 < 10) { y = x; } }
     fun main() { f(0); f(99); } }"

lemma arith_refined:
  "ivl_state_at [] arith_prog
           (FunctionResult (STR ''f'')) (f_var (STR ''y''))
     = KD I32 (Ivl (Fin 0) (Fin 8))"
  by eval
```

Comparing a signed `x` with an unsigned `y` converts `x` to unsigned. When `x` is known non-negative the conversion keeps its value, so `x < y` bounds `x`; when `x` may be negative the conversion is not injective on its values and refines nothing.

```
definition mixed_kinds :: kind_decls where
  "mixed_kinds = [(f_var (STR ''y''), U32)]"

definition mixed_prog :: imp_prog where
  "mixed_prog = program {
     fun f(x) { y = 5; z = 0; if (x < y) { z = x; } }
     fun main() { f(0); f(9); } }"

definition mixed_neg_prog :: imp_prog where
  "mixed_neg_prog = program {
     fun f(x) { y = 5; z = 0; if (x < y) { z = x; } }
     fun main() { f(0 - 10); f(9); } }"

lemma mixed_refined:
  "ivl_state_at mixed_kinds mixed_prog
           (FunctionResult (STR ''f'')) (f_var (STR ''z''))
     = KD I32 (Ivl (Fin 0) (Fin 4))"
  "ivl_state_at mixed_kinds mixed_neg_prog
           (FunctionResult (STR ''f'')) (f_var (STR ''z''))
     = KD I32 (Ivl (Fin (-10)) (Fin 9))"
  by eval+

end
```

Chapter 13

Voblint_Analysis_Parity

13.1 Parity_Warrowing

```
theory Parity_Warrowing
  imports "Voblint_Domain.Parity_Lattice"
begin
```

Parity warrowing and domain instance

Parity is finite, so widening is join and narrowing keeps the left argument.

```
instantiation parity :: warrowing begin
  definition "((a :: parity) ∇ b) = join_parity a b"
  definition "((a :: parity) △ b) = a"
instance proof
  fix a b :: parity
  show "a ≤ (a ∇ b)" unfolding less_eq_parity_def widen_parity_def by (rule join_parity_ub1)
  show "b ≤ (a ∇ b)" unfolding less_eq_parity_def widen_parity_def by (rule join_parity_ub2)
  show "b ≤ a ⟹ b ≤ (a △ b)" unfolding narrow_parity_def by simp
  show "b ≤ a ⟹ (a △ b) ≤ a"
    unfolding narrow_parity_def by (simp add: less_eq_parity_def)
qed
end

instantiation parity :: numeric_domain begin
definition gamma_abs_parity [simp]: "numeric_domain_class.gamma (a :: parity) = gamma_parity a"
definition is_empty_parity [simp]: "is_empty (a :: parity) = is_bottom_parity a"
definition to_string_parity [simp]:
  "to_string (a :: parity) = (if is_top_parity a then sym_top else string_of_parity a)"
instance proof
  show "numeric_domain_class.gamma (bot :: parity) = {}" unfolding bot_parity_def by simp
next
  show "numeric_domain_class.gamma (top :: parity) = UNIV" by (simp add: gamma_parity_top)
next
  fix a b :: parity
  assume H: "a ≤ b"
  have "gamma_parity a ⊆ gamma_parity b"
    using H unfolding less_eq_parity_def by (rule gamma_parity_mono)
  then show "numeric_domain_class.gamma a ⊆ numeric_domain_class.gamma b" by simp
next
  fix a :: parity
  show "is_empty a ⟷ numeric_domain_class.gamma a = {}"
    by (simp add: is_bottom_parity_correct)
qed
end

end
```

13.2 Parity_Domain

```
theory Parity_Domain
  imports Parity_Warrowing
    "Voblint_Domain.Forward_Domain"
    "Voblint_Domain.Numeric_Queries" "Voblint_Domain.Nonrelational_State"
begin
```

Parity arithmetic

Parity arithmetic and comparison queries, with their soundness and monotonicity facts.

Abstract arithmetic

Addition and subtraction follow the parity rules of `int`; a product is even as soon as one factor is. Any bottom operand yields `PBot`.

```
instantiation parity :: plus begin
fun plus_parity :: "parity => parity => parity" where
    "plus_parity PBot  _     = PBot"
  | "plus_parity _     PBot  = PBot"
  | "plus_parity PEven PEven = PEven"
  | "plus_parity POdd  POdd  = PEven"
  | "plus_parity PEven POdd  = POdd"
  | "plus_parity POdd  PEven = POdd"
  | "plus_parity _     _     = PTop"
instance ..
end

instantiation parity :: minus begin
fun minus_parity :: "parity => parity => parity" where
    "minus_parity PBot  _     = PBot"
  | "minus_parity _     PBot  = PBot"
  | "minus_parity PEven PEven = PEven"
  | "minus_parity POdd  POdd  = PEven"
  | "minus_parity PEven POdd  = POdd"
  | "minus_parity POdd  PEven = POdd"
  | "minus_parity _     _     = PTop"
instance ..
end

instantiation parity :: times begin
fun times_parity :: "parity => parity => parity" where
    "times_parity PBot  _     = PBot"
  | "times_parity _     PBot  = PBot"
  | "times_parity PEven _     = PEven"
  | "times_parity _     PEven = PEven"
  | "times_parity POdd  POdd  = POdd"
  | "times_parity _     _     = PTop"
instance ..
end

lemma parity_plus_sound:
  "i ∈ gamma_parity a ⟹ j ∈ gamma_parity b ⟹ i + j ∈ gamma_parity (a + b)"
  by (cases a; cases b; auto)

lemma parity_minus_sound:
  "i ∈ gamma_parity a ⟹ j ∈ gamma_parity b ⟹ i - j ∈ gamma_parity (a - b)"
  by (cases a; cases b; auto)

lemma parity_times_sound:
  "i ∈ gamma_parity a ⟹ j ∈ gamma_parity b ⟹ i * j ∈ gamma_parity (a * b)"
  by (cases a; cases b; auto)
```

Comparison and truthiness queries

Parity carries no ordering information, so `parity_lt` is always `None`. Equality is decidable exactly when the two operands have opposite parity – an even value can never equal an odd one – and truthiness is decidable exactly for `POdd`, since every odd integer is nonzero.

```
fun parity_lt :: "parity ⇒ parity ⇒ bool option" where
  "parity_lt _ _ = None"

fun parity_eqb :: "parity ⇒ parity ⇒ bool option" where
```

```
    "parity_eqb PEven POdd = Some False"
  | "parity_eqb POdd PEven = Some False"
  | "parity_eqb _ _ = None"

fun parity_tobool :: "parity ⇒ bool option" where
    "parity_tobool POdd = Some True"
  | "parity_tobool _ = None"

lemma parity_lt_sound:
  "parity_lt a b = Some c ⟹ i ∈ gamma_parity a ⟹ j ∈ gamma_parity b ⟹ (i < j) = c"
  by simp

lemma parity_eqb_sound:
  "parity_eqb a b = Some c ⟹ i ∈ gamma_parity a ⟹ j ∈ gamma_parity b ⟹ (i = j) = c"
  by (cases a; cases b; auto)

definition parity_div :: "parity ⇒ parity ⇒ parity" where
  "parity_div a b = (if a = PBot ∨ b = PBot then PBot else PTop)"

definition parity_mod :: "parity ⇒ parity ⇒ parity" where
  "parity_mod a b =
    (if a = PBot ∨ b = PBot then PBot else if b = PEven then a else PTop)"

lemma parity_div_sound:
  "i ∈ gamma_parity a ⟹ j ∈ gamma_parity b ⟹
    c_div i j ∈ gamma_parity (parity_div a b)"
  by (cases a; cases b; simp add: parity_div_def)

lemma parity_mod_sound:
  "i ∈ gamma_parity a ⟹ j ∈ gamma_parity b ⟹
    c_mod i j ∈ gamma_parity (parity_mod a b)"
  by (cases a; cases b; auto simp: parity_mod_def c_mod_def)
```

A conversion to a machine kind preserves parity (`even (ik_norm ?ik ?n) = even ?n`), so the parity cast is the identity.

```
lemma parity_cast_sound: "i ∈ gamma_parity a ⟹ ik_norm ik i ∈ gamma_parity a"
  by (cases a) simp_all

end
```

13.3 Parity_Kind

```
theory Parity_Kind
  imports "Voblint_Analysis_Parity.Parity_Domain" "Voblint_Nonrelational.Nonrelational_Analysis"
    "Voblint_VIMP.VIMP_Notation"
begin
```

Parity at a kind

Normalizing to a kind changes a value by a multiple of `2 ^ w`, which is even, so every operation keeps its parity across a wraparound (`parity_cast_sound`). The typed operations are therefore the mathematical ones. Every kind's range holds an even and an odd value, so a parity meets the range exactly when it is not bottom, and no parity other than bottom bounds a value to a kind. Parity has no Goblint counterpart.

```
instantiation parity :: ik_domain
begin

definition widen_ik_parity :: "ikind ⇒ parity ⇒ parity ⇒ parity" where
  "widen_ik_parity k a b = a ⊔ b"

definition narrow_ik_parity :: "ikind ⇒ parity ⇒ parity ⇒ parity" where
  "narrow_ik_parity k a b = a ⊓ b"

definition empty_ik_parity :: "ikind ⇒ parity ⇒ bool" where
  "empty_ik_parity k a ⟷ a = PBot"

instance proof
  fix k :: ikind and a b :: parity
  show "a ≤ widen_ik k a b" "b ≤ widen_ik k a b" by (simp_all add: widen_ik_parity_def)
  show "b ≤ a ⟹ b ≤ narrow_ik k a b" "b ≤ a ⟹ narrow_ik k a b ≤ a"
    by (simp_all add: narrow_ik_parity_def inf_absorb2)
  show "empty_ik k a ⟷ numeric_domain_class.gamma a ∩ ik_range k = {}"
  proof (cases a)
    case PEven then show ?thesis
      by (simp add: empty_ik_parity_def disjoint_iff) (rule exI[of _ 0], simp)
  next
    case POdd then show ?thesis
      using one_in_ik_range[of k]
      by (simp add: empty_ik_parity_def disjoint_iff del: one_in_ik_range) (rule exI[of _ 1], simp)
  next
    case PTop then show ?thesis
      by (simp add: empty_ik_parity_def) (metis zero_in_ik_range empty_iff)
  qed (simp add: empty_ik_parity_def)
qed

end

fun parity_aop :: "aop ⇒ parity ⇒ parity ⇒ parity" where
  "parity_aop AAdd a b = a + b"
| "parity_aop ASub a b = a - b"
| "parity_aop AMul a b = a * b"
| "parity_aop ADiv a b = parity_div a b"
| "parity_aop AMod a b = parity_mod a b"

lemma parity_aop_sound:
  "m ∈ gamma_parity a ⟹ n ∈ gamma_parity b ⟹ aop_eval op m n ∈ gamma_parity (parity_aop op a b)"
  by (cases op) (simp_all add: parity_plus_sound parity_minus_sound parity_times_sound
      parity_div_sound parity_mod_sound)


instantiation parity :: kind_int_domain
begin

definition top_ik_parity :: "ikind ⇒ parity" where "top_ik_parity k = PTop"

definition of_int_ik_parity :: "ikind ⇒ int ⇒ parity" where
  "of_int_ik_parity k n = parity_of_int (ik_norm k n)"

definition cast_ik_parity :: "signed_overflow_policy ⇒ ikind option ⇒ ikind ⇒ parity ⇒ parity" where
  "cast_ik_parity ovp sk k a = a"

definition arith_ik_parity ::
  "signed_overflow_policy ⇒ ikind ⇒ aop ⇒ bool ⇒ parity ⇒ parity ⇒ parity" where
  "arith_ik_parity ovp k op fits = parity_aop op"

definition cmp_ik_parity :: "cop ⇒ parity ⇒ parity ⇒ bool option" where
```

```
  "cmp_ik_parity = cop_of parity_lt parity_eqb"

definition meet_ik_parity :: "parity ⇒ parity ⇒ parity" where
  "meet_ik_parity a b = a ⊓ b"

definition refine_ik_parity :: "cop ⇒ parity ⇒ parity ⇒ parity × parity" where
  "refine_ik_parity c a b = (if c = CEq then (a ⊓ b, a ⊓ b) else (a, b))"

definition raw_aop_ik_parity :: "aop ⇒ parity ⇒ parity ⇒ parity" where
  "raw_aop_ik_parity = parity_aop"

definition fits_ik_parity :: "ikind ⇒ parity ⇒ bool" where
  "fits_ik_parity k a ⟷ a = PBot"

definition answer_ik_parity :: "parity ⇒ answer" where
  "answer_ik_parity a = answer_with (int_parity_update (λ_. a))"

definition of_answer_ik_parity :: "ikind option ⇒ answer ⇒ parity" where
  "of_answer_ik_parity sk q = (case q of QBot ⇒ ⊥ | QTop ⇒ ⊤ | QLifted d ⇒ int_parity d)"

instance proof
  fix n m :: int and k :: ikind and a b :: parity and op :: aop and c :: cop and r :: bool
  show "n ∈ ik_range k ⟹ n ∈ numeric_domain_class.gamma (top_ik k :: parity)"
    by (simp add: top_ik_parity_def)
  show "ik_norm k n ∈ numeric_domain_class.gamma (of_int_ik k n :: parity)"
    by (simp add: of_int_ik_parity_def parity_of_int_gamma)
  show "(⋀k'. sk = Some k' ⟹ n ∈ ik_range k') ⟹ n ∈ numeric_domain_class.gamma a
      ⟹ ik_norm k n ∈ numeric_domain_class.gamma (cast_ik ovp sk k a)" for ovp sk
    by (simp add: cast_ik_parity_def parity_cast_sound)
  show "(fits ⟹ m ∈ ik_range k ∧ n ∈ ik_range k) ⟹ m ∈ numeric_domain_class.gamma a
      ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ ik_norm k (aop_eval op m n) ∈ numeric_domain_class.gamma (arith_ik ovp k op fits a b)" for ovp fits
    by (simp add: arith_ik_parity_def parity_cast_sound parity_aop_sound)
  show "cmp_ik c a b = Some r ⟹ m ∈ numeric_domain_class.gamma a
      ⟹ n ∈ numeric_domain_class.gamma b ⟹ cop_eval c m n = r"
    unfolding cmp_ik_parity_def by simp (rule cop_of_sound[OF parity_lt_sound parity_eqb_sound])
  show "m ∈ numeric_domain_class.gamma a ⟹ m ∈ numeric_domain_class.gamma b
      ⟹ m ∈ numeric_domain_class.gamma (meet_ik a b)"
    by (simp add: meet_ik_parity_def)
  show "cop_eval c m n ⟹ m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ m ∈ numeric_domain_class.gamma (fst (refine_ik c a b))
        ∧ n ∈ numeric_domain_class.gamma (snd (refine_ik c a b))"
    by (cases c) (simp_all add: refine_ik_parity_def)
  show "m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ aop_eval op m n ∈ numeric_domain_class.gamma (raw_aop_ik op a b)"
    by (simp add: raw_aop_ik_parity_def parity_aop_sound)
  show "fits_ik k a ⟹ n ∈ numeric_domain_class.gamma a ⟹ n ∈ ik_range k"
    by (simp add: fits_ik_parity_def)
  show "n ∈ numeric_domain_class.gamma a ⟹ n ∈ gamma_answer (answer_ik a)"
    by (simp add: answer_ik_parity_def gamma_int_dom_def top_int_dom_ext_def gamma_sign_top
        gamma_parity_top top_ivl_def gamma_ivl_top)
  show "(⋀k. sk = Some k ⟹ n ∈ ik_range k) ⟹ n ∈ gamma_answer q
      ⟹ n ∈ numeric_domain_class.gamma (of_answer_ik sk q :: parity)" for sk q
    by (cases q)
      (simp_all add: of_answer_ik_parity_def gamma_int_dom_def gamma_sign_top gamma_parity_top)
qed

end

declare [[code abort: gamma_parity]]
```

Executed

A step of 2 from 0 keeps x even at U8, across every wraparound.

```
definition parity_prog :: imp_prog where
  "parity_prog = program {
     fun f() { x = 0; while (x != 7) { x = x + 2; } }
     fun main() { f(); } }"

lemma parity_value:
  "nonrel.state_at default_sem_conf [(ScopedId (STR ''f'') (STR ''x''), U8)] 0 Globals_Join is_global parity_prog []
     (Statement 1) (ScopedId (STR ''f'') (STR ''x''))
     = KD U8 PEven"
  by eval

end
```

Chapter 14

Voblint_Analysis_Congruence

14.1 Congruence_Warrowing

```
theory Congruence_Warrowing
  imports "Voblint_Domain.Congruence_Lattice" "Voblint_Exec.Default_St_Reachability"
begin
```

Type-class warrowing for the TD warrowing solver

Congruence follows Sign's and Parity's own choice (`Sign_Lattice`, `Parity_Domain`): the vendored `widening`/`narrowing` classes require only the four upper/bracket laws below, not a termination proof, so a domain whose lattice is already well behaved gets a conservative instance rather than an invented acceleration mechanism. Widening is plain join; narrowing is the trivial left projection, which discharges `narrow_ge`/`narrow_le` immediately from `b <= a` without needing anything about congruence's own structure.

```
definition narrow_congruence_td :: "congruence => congruence => congruence" where
  "narrow_congruence_td a b = a"

instantiation congruence :: warrowing
begin

definition widen_congruence :: "congruence => congruence => congruence" where
  "((a :: congruence) ▽ b) = join_congruence a b"

definition narrow_congruence :: "congruence => congruence => congruence" where
  "((a :: congruence) △ b) = narrow_congruence_td a b"

instance proof intro_classes
  fix a b :: congruence
  show "a <= (a ▽ b)"
    unfolding widen_congruence_def by (rule join_congruence_ub1)
  show "b <= (a ▽ b)"
    unfolding widen_congruence_def by (rule join_congruence_ub2)
  show "b <= a ⟹ b <= (a △ b)"
    unfolding narrow_congruence_def narrow_congruence_td_def by simp
  show "b <= a ⟹ (a △ b) <= a"
    unfolding narrow_congruence_def narrow_congruence_td_def by simp
qed

end
```

Numeric domain instance

`numeric_domain` extends `executable_domain`, which includes the solver's `warrowing`, so the instance follows the widening and narrowing above.

```
instantiation congruence :: numeric_domain
begin

definition gamma_abs_congruence [simp]:
  "numeric_domain_class.gamma (a :: congruence) = gamma_congruence a"

definition is_empty_congruence [simp]:
  "is_empty (a :: congruence) = is_bottom_congruence a"

definition to_string_congruence [simp]:
  "to_string (a :: congruence) =
     (if is_top_congruence a then sym_top else string_of_congruence a)"

instance
proof intro_classes
  show "numeric_domain_class.gamma (bot :: congruence) = {}"
    unfolding bot_congruence_def by simp
next
  show "numeric_domain_class.gamma (top :: congruence) = UNIV"
    by simp
next
  fix a b :: congruence
  assume "a <= b"
  then show "numeric_domain_class.gamma a ⊆ numeric_domain_class.gamma b"
    unfolding less_eq_congruence_iff_gamma
    by simp
next
  fix a :: congruence
  show "is_empty a ⟷ numeric_domain_class.gamma a = {}"
    by (simp add: is_bottom_congruence_correct)
```

qed

end

end

14.2 Congruence_Arithmetic

```
theory Congruence_Arithmetic
  imports Congruence_Warrowing
    "Voblint_Domain.Numeric_Queries"
begin
```

Congruence arithmetic

Arithmetic follows the unbounded-integer fragment of Goblint's congruence domain. Every public result passes through the normalized carrier. Goblint raises when exactly one operand is bottom; the total HOL operations instead return bottom because no concrete arithmetic result exists for an empty operand.

```
fun plus_congruence_rep ::
    "congruence_rep => congruence_rep => congruence_rep"
where
  "plus_congruence_rep None _ = None"
| "plus_congruence_rep _ None = None"
| "plus_congruence_rep (Some (c1, m1)) (Some (c2, m2)) =
     normalize_congruence_rep (Some (c1 + c2, gcd m1 m2))"

fun minus_congruence_rep ::
    "congruence_rep => congruence_rep => congruence_rep"
where
  "minus_congruence_rep None _ = None"
| "minus_congruence_rep _ None = None"
| "minus_congruence_rep (Some (c1, m1)) (Some (c2, m2)) =
     normalize_congruence_rep (Some (c1 - c2, gcd m1 m2))"
```

Multiplication expands two classes into a constant plus three variable terms. The gcd of their coefficients contains every concrete product while retaining all congruence information justified by that expansion. The resulting class is a sound over-approximation; no exactness claim is needed.

```
fun times_congruence_rep ::
    "congruence_rep => congruence_rep => congruence_rep"
where
  "times_congruence_rep None _ = None"
| "times_congruence_rep _ None = None"
| "times_congruence_rep (Some (c1, m1)) (Some (c2, m2)) =
     normalize_congruence_rep
       (Some
         (c1 * c2,
          gcd (c1 * m2)
            (gcd (m1 * c2) (m1 * m2))))"

lemma normalized_plus_congruence_rep [simp]:
  "normalized_congruence_rep (plus_congruence_rep a b)"
  by (cases a; cases b;
      auto simp only: plus_congruence_rep.simps
        normalized_congruence_rep.simps
        normalized_normalize_congruence_rep
        split: prod.splits)

lemma normalized_minus_congruence_rep [simp]:
  "normalized_congruence_rep (minus_congruence_rep a b)"
  by (cases a; cases b;
      auto simp only: minus_congruence_rep.simps
        normalized_congruence_rep.simps
        normalized_normalize_congruence_rep
        split: prod.splits)

lemma normalized_times_congruence_rep [simp]:
  "normalized_congruence_rep (times_congruence_rep a b)"
  by (cases a; cases b;
      auto simp only: times_congruence_rep.simps
        normalized_congruence_rep.simps
        normalized_normalize_congruence_rep
        split: prod.splits)

instantiation congruence :: plus
begin

lift_definition plus_congruence ::
    "congruence => congruence => congruence"
  is plus_congruence_rep
  by simp
```

```
instance ..

end

instantiation congruence :: minus
begin

lift_definition minus_congruence ::
    "congruence => congruence => congruence"
  is minus_congruence_rep
  by simp

instance ..

end

instantiation congruence :: times
begin

lift_definition times_congruence ::
    "congruence => congruence => congruence"
  is times_congruence_rep
  by simp

instance ..

end
```

Semantic soundness

Each operation's soundness reduces to one divisibility fact about the two operand classes; `congruence_binop_sound` discharges the bottom operands and the representation plumbing once.

```
lemma congruence_binop_sound:
  assumes "i ∈ gamma_congruence a" and "j ∈ gamma_congruence b"
    and "⋀c1 m1 c2 m2. Rep_congruence a = Some (c1, m1) ⟹
           Rep_congruence b = Some (c2, m2) ⟹ m1 dvd i - c1 ⟹ m2 dvd j - c2 ⟹
           k ∈ gamma_congruence r"
  shows "k ∈ gamma_congruence r"
  using assms unfolding gamma_congruence_def
  by (cases rule: congruence_rep_cases[of a]; cases rule: congruence_rep_cases[of b]) auto

lemma congruence_plus_sound:
  assumes "i ∈ gamma_congruence a" and "j ∈ gamma_congruence b"
  shows "i + j ∈ gamma_congruence (a + b)"
  using assms
proof (rule congruence_binop_sound)
  fix c1 m1 c2 m2
  assume a: "Rep_congruence a = Some (c1, m1)" and b: "Rep_congruence b = Some (c2, m2)"
    and mi: "m1 dvd i - c1" and mj: "m2 dvd j - c2"
  have "gcd m1 m2 dvd (i - c1) + (j - c2)"
    by (rule dvd_add[OF dvd_trans[OF gcd_dvd1 mi] dvd_trans[OF gcd_dvd2 mj]])
  then have "gcd m1 m2 dvd (i + j) - (c1 + c2)" by (simp add: algebra_simps)
  then show "i + j ∈ gamma_congruence (a + b)"
    unfolding gamma_congruence_def plus_congruence.rep_eq a b
    by (simp only: plus_congruence_rep.simps gamma_normalize_congruence_rep
        gamma_congruence_rep.simps mem_Collect_eq)
qed

lemma congruence_minus_sound:
  assumes "i ∈ gamma_congruence a" and "j ∈ gamma_congruence b"
  shows "i - j ∈ gamma_congruence (a - b)"
  using assms
proof (rule congruence_binop_sound)
  fix c1 m1 c2 m2
  assume a: "Rep_congruence a = Some (c1, m1)" and b: "Rep_congruence b = Some (c2, m2)"
    and mi: "m1 dvd i - c1" and mj: "m2 dvd j - c2"
  have "gcd m1 m2 dvd (i - c1) - (j - c2)"
    by (rule dvd_diff[OF dvd_trans[OF gcd_dvd1 mi] dvd_trans[OF gcd_dvd2 mj]])
  then have "gcd m1 m2 dvd (i - j) - (c1 - c2)" by (simp add: algebra_simps)
  then show "i - j ∈ gamma_congruence (a - b)"
    unfolding gamma_congruence_def minus_congruence.rep_eq a b
    by (simp only: minus_congruence_rep.simps gamma_normalize_congruence_rep
        gamma_congruence_rep.simps mem_Collect_eq)
qed
```

```
lemma congruence_product_divisible:
  fixes c1 c2 m1 m2 i j :: int
  assumes mi: "m1 dvd i - c1"
      and mj: "m2 dvd j - c2"
  shows
    "gcd (c1 * m2) (gcd (m1 * c2) (m1 * m2))
       dvd i * j - c1 * c2"
proof -
  obtain ki where ki: "i - c1 = m1 * ki"
    using mi unfolding dvd_def by blast
  obtain kj where kj: "j - c2 = m2 * kj"
    using mj unfolding dvd_def by blast
  let ?g =
    "gcd (c1 * m2) (gcd (m1 * c2) (m1 * m2))"
  have d1: "?g dvd c1 * m2"
    by (rule gcd_dvd1)
  have outer:
    "?g dvd gcd (m1 * c2) (m1 * m2)"
    by (rule gcd_dvd2)
  have d2: "?g dvd m1 * c2"
    by (rule dvd_trans[OF outer gcd_dvd1])
  have d3: "?g dvd m1 * m2"
    by (rule dvd_trans[OF outer gcd_dvd2])
  have t1: "?g dvd c1 * (j - c2)"
    using dvd_mult2[OF d1, of kj] unfolding kj by (simp add: algebra_simps)
  have t2: "?g dvd c2 * (i - c1)"
    using dvd_mult2[OF d2, of ki] unfolding ki by (simp add: algebra_simps)
  have t3: "?g dvd (i - c1) * (j - c2)"
    using dvd_mult2[OF d3, of "ki * kj"]
    unfolding ki kj by (simp add: algebra_simps)
  have combined:
    "?g dvd c1 * (j - c2) + c2 * (i - c1) +
      (i - c1) * (j - c2)"
    by (rule dvd_add[OF dvd_add[OF t1 t2] t3])
  have product:
    "c1 * (j - c2) + c2 * (i - c1) +
       (i - c1) * (j - c2) =
     i * j - c1 * c2"
    by (simp add: algebra_simps)
  show ?thesis using combined unfolding product .
qed

lemma congruence_times_sound:
  assumes "i ∈ gamma_congruence a" and "j ∈ gamma_congruence b"
  shows "i * j ∈ gamma_congruence (a * b)"
  using assms
proof (rule congruence_binop_sound)
  fix c1 m1 c2 m2
  assume a: "Rep_congruence a = Some (c1, m1)" and b: "Rep_congruence b = Some (c2, m2)"
    and mi: "m1 dvd i - c1" and mj: "m2 dvd j - c2"
  from congruence_product_divisible[OF mi mj]
  show "i * j ∈ gamma_congruence (a * b)"
    unfolding gamma_congruence_def times_congruence.rep_eq a b
    by (simp only: times_congruence_rep.simps gamma_normalize_congruence_rep
        gamma_congruence_rep.simps mem_Collect_eq)
qed
```

Comparison and truthiness queries

Congruence classes carry no ordering information, so `congruence_lt` is always `None`. Equality and truthiness are decidable exactly when both sides are singleton classes (modulus `0`): `congruence_singleton` reads that singleton constant off the normalized representation.

```
definition congruence_singleton :: "congruence => int option" where
  "congruence_singleton a =
     (case Rep_congruence a of
        Some (c, m) => (if m = 0 then Some c else None)
      | None => None)"

lemma congruence_singleton_sound:
  assumes "congruence_singleton a = Some c" and "n ∈ gamma_congruence a"
  shows "n = c"
  using assms
  unfolding congruence_singleton_def gamma_congruence_def
  by (cases "Rep_congruence a" rule: option.exhaust) (auto split: if_splits)
```

```
definition congruence_lt :: "congruence => congruence => bool option" where
  "congruence_lt a b =
     (case (congruence_singleton a, congruence_singleton b) of
        (Some c1, Some c2) => Some (c1 < c2)
      | _ => None)"

definition congruence_eqb :: "congruence => congruence => bool option" where
  "congruence_eqb a b =
     (case (congruence_singleton a, congruence_singleton b) of
        (Some c1, Some c2) => Some (c1 = c2)
      | _ => None)"

definition congruence_tobool :: "congruence => bool option" where
  "congruence_tobool a =
     (case congruence_singleton a of Some c => Some (c ≠ 0) | None => None)"

lemma congruence_lt_sound:
  assumes "congruence_lt a b = Some c" and "i ∈ gamma_congruence a" and "j ∈ gamma_congruence b"
  shows "(i < j) = c"
proof -
  obtain c1 c2 where s1: "congruence_singleton a = Some c1" and s2:
    "congruence_singleton b = Some c2"
      and c_def: "c = (c1 < c2)"
    using assms(1) unfolding congruence_lt_def by (auto split: option.splits)
  have "i = c1" using congruence_singleton_sound[OF s1 assms(2)] .
  moreover have "j = c2" using congruence_singleton_sound[OF s2 assms(3)] .
  ultimately show ?thesis using c_def by simp
qed

lemma congruence_eqb_sound:
  assumes "congruence_eqb a b = Some c" and "i ∈ gamma_congruence a" and "j ∈ gamma_congruence b"
  shows "(i = j) = c"
proof -
  obtain c1 c2 where s1: "congruence_singleton a = Some c1" and s2:
    "congruence_singleton b = Some c2"
      and c_def: "c = (c1 = c2)"
    using assms(1) unfolding congruence_eqb_def by (auto split: option.splits)
  have "i = c1" using congruence_singleton_sound[OF s1 assms(2)] .
  moreover have "j = c2" using congruence_singleton_sound[OF s2 assms(3)] .
  ultimately show ?thesis using c_def by simp
qed

definition congruence_exact_binop ::
    "(int ⇒ int ⇒ int) ⇒ (congruence ⇒ congruence ⇒ congruence) ⇒
      congruence ⇒ congruence ⇒ congruence" where
  "congruence_exact_binop f fallback a b =
    (if is_empty a ∨ is_empty b then bot
     else case (congruence_singleton a, congruence_singleton b) of
       (Some x, Some y) ⇒ congruence_of_int (f x y)
     | _ ⇒ fallback a b)"

lemma congruence_exact_binop_sound:
  assumes "i ∈ gamma_congruence a" "j ∈ gamma_congruence b"
    and sound: "⋀a b i j. i ∈ gamma_congruence a ⟹ j ∈ gamma_congruence b ⟹
      f i j ∈ gamma_congruence (fallback a b)"
  shows "f i j ∈ gamma_congruence (congruence_exact_binop f fallback a b)"
  using assms congruence_singleton_sound
  unfolding congruence_exact_binop_def
  by (auto simp: is_bottom_congruence_correct congruence_of_int_def
      split: option.splits dest: congruence_singleton_sound)

lemma gamma_congruence_exact_division:
  assumes nz: "d ≠ 0" and dc: "d dvd c" and dm: "d dvd m"
  shows "gamma_congruence (mk_congruence (c div d) (m div d)) =
    (λn. c_div n d) ‘ {n. m dvd n - c}"
proof -
  obtain c' m' where c: "c = d * c'" and m: "m = d * m'"
    using dc dm by (auto simp: dvd_def)
  have cancel: "⋀k. c_div (d * k) d = k"
    using nz by (simp add: c_div_exact)
  have quotients: "c div d = c'" "m div d = m'"
    using nz by (simp_all add: c m)
  show ?thesis
  proof (rule set_eqI, rule iffI)
    fix q
```

```
    assume "q ∈ gamma_congruence (mk_congruence (c div d) (m div d))"
    then obtain k where q: "q - c' = m' * k"
      unfolding quotients by (auto simp: dvd_def)
    have member: "d * q ∈ {n. m dvd n - c}"
      unfolding c m using q by (auto simp: dvd_def algebra_simps)
    show "q ∈ (λn. c_div n d) ‘ {n. m dvd n - c}"
      using imageI[OF member, of "λn. c_div n d"] by (simp add: cancel)
  next
    fix q
    assume "q ∈ (λn. c_div n d) ‘ {n. m dvd n - c}"
    then obtain n k where q: "q = c_div n d" and n: "n - c = m * k"
      by (auto simp: dvd_def)
    have n_eq: "n = d * (c' + m' * k)"
      using n unfolding c m by (simp add: algebra_simps)
    show "q ∈ gamma_congruence (mk_congruence (c div d) (m div d))"
      unfolding quotients q n_eq by (simp add: cancel)
  qed
qed

definition congruence_divides :: "int ⇒ congruence ⇒ bool" where
  "congruence_divides d a =
    (case Rep_congruence a of None ⇒ True | Some (c, m) ⇒ d dvd c ∧ d dvd m)"

definition congruence_div_const :: "congruence ⇒ int ⇒ congruence" where
  "congruence_div_const a d =
    (case Rep_congruence a of None ⇒ bot
     | Some (c, m) ⇒
       if d = 0 then congruence_of_int 0
       else if congruence_divides d a then mk_congruence (c div d) (m div d)
       else top)"

lemma gamma_congruence_div_const:
  assumes "d ≠ 0" "congruence_divides d a"
  shows "gamma_congruence (congruence_div_const a d) =
    (λn. c_div n d) ‘ gamma_congruence a"
proof (cases "Rep_congruence a")
  case None
  then have "gamma_congruence a = {}" by (simp add: gamma_congruence_def)
  then show ?thesis by (simp add: congruence_div_const_def None bot_congruence_def)
next
  case (Some p)
  obtain c m where p: "p = (c, m)" by (cases p)
  have divisibility: "d dvd c" "d dvd m"
    using assms(2) unfolding congruence_divides_def Some p by simp_all
  have gamma: "gamma_congruence a = {n. m dvd n - c}"
    by (simp add: gamma_congruence_def Some p)
  show ?thesis
    using gamma_congruence_exact_division[OF assms(1) divisibility]
    by (simp add: congruence_div_const_def Some p assms gamma)
qed

lemma gamma_congruence_div_const_cases:
  "gamma_congruence (congruence_div_const a d) =
    (if is_empty a then {}
     else if d = 0 then {0}
     else if congruence_divides d a then (λn. c_div n d) ‘ gamma_congruence a
     else UNIV)"
proof (cases "is_empty a")
  case True
  then have rep: "Rep_congruence a = None"
    by (simp add: is_bottom_congruence_def bot_congruence_def)
  show ?thesis using True
    by (simp add: congruence_div_const_def rep bot_congruence_def
        is_bottom_congruence_correct gamma_congruence_def)
next
  case False
  then obtain c m where rep: "Rep_congruence a = Some (c, m)"
    by (cases "Rep_congruence a") (auto simp: is_bottom_congruence_correct gamma_congruence_def)
  show ?thesis
    using gamma_congruence_div_const[of d a]
    by (cases "d = 0"; cases "congruence_divides d a")
       (use False in ‹simp_all add: congruence_div_const_def rep congruence_of_int_def›)
qed

lemma congruence_div_const_sound:
  assumes "i ∈ gamma_congruence a"
```

```
  shows "c_div i d ∈ gamma_congruence (congruence_div_const a d)"
  using assms
  by (auto simp: gamma_congruence_div_const_cases is_bottom_congruence_correct)

definition congruence_div_fallback :: "congruence ⇒ congruence ⇒ congruence" where
  "congruence_div_fallback a b =
    (if is_empty a ∨ is_empty b then bot
     else case congruence_singleton b of
       Some c ⇒ congruence_div_const a c
     | None ⇒ top)"

lemma congruence_div_fallback_sound:
  assumes "i ∈ gamma_congruence a" "j ∈ gamma_congruence b"
  shows "c_div i j ∈ gamma_congruence (congruence_div_fallback a b)"
  using assms congruence_singleton_sound[OF _ assms(2)]

  unfolding congruence_div_fallback_def
  by (auto simp: is_bottom_congruence_correct split: option.splits
      intro: congruence_div_const_sound)

definition congruence_div :: "congruence ⇒ congruence ⇒ congruence" where
  "congruence_div = congruence_exact_binop c_div congruence_div_fallback"

definition congruence_mod :: "congruence ⇒ congruence ⇒ congruence" where
  "congruence_mod = congruence_exact_binop c_mod (λa b. a - top * b)"

lemma congruence_mod_fallback_sound:
  assumes "i ∈ gamma_congruence a" "j ∈ gamma_congruence b"
  shows "c_mod i j ∈ gamma_congruence (a - top * b)"
proof -
  have "i - c_div i j * j ∈ gamma_congruence (a - top * b)"
    by (intro congruence_minus_sound assms congruence_times_sound) simp_all
  then show ?thesis by (cases "j = 0") (simp_all add: c_mod_def)
qed

lemma congruence_div_sound:
  "i ∈ gamma_congruence a ⟹ j ∈ gamma_congruence b ⟹
    c_div i j ∈ gamma_congruence (congruence_div a b)"
  unfolding congruence_div_def
  by (rule congruence_exact_binop_sound) (auto intro: congruence_div_fallback_sound)

lemma congruence_mod_sound:
  "i ∈ gamma_congruence a ⟹ j ∈ gamma_congruence b ⟹
    c_mod i j ∈ gamma_congruence (congruence_mod a b)"
  unfolding congruence_mod_def
  by (rule congruence_exact_binop_sound) (auto intro: congruence_mod_fallback_sound)

end
```

14.3 Congruence_Kind

```
theory Congruence_Kind
  imports
    "Voblint_Analysis_Congruence.Congruence_Arithmetic" "Voblint_Nonrelational.Nonrelational_Domain"
begin
```

Congruences at a kind

Normalizing to kind `k` changes a value by a multiple of `2 ^ w`, the kind's modulus. A class `c + m`$\mathbb{Z}$ therefore survives the normalization modulo `gcd m (2 ^ w)`, and when that divisor is `2 ^ w` itself the class holds one value of the kind. This is Goblint's `handle_overflow` in `congruenceDomain.ml`, which keeps the largest power of two dividing `m`; it applies to every kind that wraps under the overflow policy.

```
lemma ik_mod_dvd_ik_norm: "ik_mod k dvd ik_norm k n - n"
proof -
  have "ik_norm k n - n = - ((n - ik_min k) - (n - ik_min k) mod ik_mod k)"
    by (simp add: ik_norm_mod)
  then show ?thesis by (simp only: dvd_minus_iff dvd_minus_mod)
qed

lemma ik_norm_eq_if_dvd:
  assumes "ik_mod k dvd n - c"
  shows "ik_norm k n = ik_norm k c"
proof -
  have "(n - ik_min k) mod ik_mod k = (c - ik_min k) mod ik_mod k"
    using assms by (simp add: mod_eq_dvd_iff)
  then show ?thesis by (simp add: ik_norm_mod)
qed

definition norm_cong_ik :: "ikind ⇒ congruence ⇒ congruence" where
  "norm_cong_ik k a =
     (case Rep_congruence a of
        None ⇒ bot
      | Some (c, m) ⇒
          (let g = gcd m (ik_mod k)
           in if g = ik_mod k then congruence_of_int (ik_norm k c) else mk_congruence c g))"

lemma gamma_congruence_Some:
  "Rep_congruence a = Some (c, m) ⟹ gamma_congruence a = {n. m dvd n - c}"
  by (simp add: gamma_congruence_def)

lemma norm_cong_ik_sound:
  assumes n: "n ∈ gamma_congruence a"
  shows "ik_norm k n ∈ gamma_congruence (norm_cong_ik k a)"
proof (cases a rule: congruence_rep_cases)
  case Bot
  then show ?thesis using n by (simp add: gamma_congruence_def)
next
  case (Class c m)
  let ?g = "gcd m (ik_mod k)"
  have "m dvd n - c" using n Class by (simp add: gamma_congruence_Some)
  then have gn: "?g dvd n - c" by (rule dvd_trans[rotated]) simp
  have "?g dvd ik_norm k n - n" by (rule dvd_trans[OF _ ik_mod_dvd_ik_norm]) simp
  with gn have g: "?g dvd ik_norm k n - c"
    using dvd_add[of ?g "ik_norm k n - n" "n - c"] by simp
  show ?thesis
  proof (cases "?g = ik_mod k")
    case True
    with gn have "ik_norm k n = ik_norm k c" by (intro ik_norm_eq_if_dvd) simp
    with True Class show ?thesis by (simp add: norm_cong_ik_def)
  next
    case False
    with g Class show ?thesis by (simp add: norm_cong_ik_def Let_def)
  qed
qed
```

The least value of a class at or above a bound decides whether the class meets a kind's range.

```
lemma cong_least_above:
  fixes m c lo :: int
  assumes m: "0 < m"
  defines "r ≡ lo + (c - lo) mod m"
  shows "lo ≤ r" and "m dvd r - c"
    and "m dvd n - c ⟹ lo ≤ n ⟹ r ≤ n"
proof -
```

```
  show "lo ≤ r" using m by (simp add: r_def)
  have "r - c = - ((c - lo) - (c - lo) mod m)" by (simp add: r_def)
  then show "m dvd r - c" by (simp only: dvd_minus_iff dvd_minus_mod)
  assume d: "m dvd n - c" and lo: "lo ≤ n"
  have "(n - lo) mod m = (c - lo) mod m"
    using d by (simp add: mod_eq_dvd_iff)
  moreover have "(n - lo) mod m ≤ n - lo"
    using lo m by (simp add: zmod_le_nonneg_dividend)
  ultimately show "r ≤ n" by (simp add: r_def)
qed

instantiation congruence :: ik_domain
begin

definition widen_ik_congruence :: "ikind ⇒ congruence ⇒ congruence ⇒ congruence" where
  "widen_ik_congruence k a b = a ⊔ b"

definition narrow_ik_congruence :: "ikind ⇒ congruence ⇒ congruence ⇒ congruence" where
  "narrow_ik_congruence k a b = a ⊓ b"

definition empty_ik_congruence :: "ikind ⇒ congruence ⇒ bool" where
  "empty_ik_congruence k a =
     (case Rep_congruence a of
        None ⇒ True
      | Some (c, m) ⇒
          if m = 0 then ¬ (ik_min k ≤ c ∧ c ≤ ik_max k)
          else ik_max k < ik_min k + (c - ik_min k) mod m)"

instance proof
  fix k :: ikind and a b :: congruence
  show "a ≤ widen_ik k a b" "b ≤ widen_ik k a b"
    by (simp_all add: widen_ik_congruence_def)
  show "b ≤ a ⟹ b ≤ narrow_ik k a b" "b ≤ a ⟹ narrow_ik k a b ≤ a"
    by (simp_all add: narrow_ik_congruence_def)
  show "empty_ik k a ⟷ numeric_domain_class.gamma a ∩ ik_range k = {}"
  proof (cases a rule: congruence_rep_cases)
    case Bot
    then show ?thesis by (simp add: empty_ik_congruence_def gamma_congruence_def)
  next
    case (Class c m)
    show ?thesis
    proof (cases "m = 0")
      case True
      with Class show ?thesis
        by (auto simp: empty_ik_congruence_def gamma_congruence_Some)
    next
      case False
      then have m: "0 < m" using normalized_Rep_congruence[of a] Class by auto
      define r where "r = ik_min k + (c - ik_min k) mod m"
      have r: "ik_min k ≤ r" "m dvd r - c" "⋀n. m dvd n - c ⟹ ik_min k ≤ n ⟹ r ≤ n"
        using cong_least_above[OF m, where lo = "ik_min k" and c = c] by (simp_all add: r_def)
      have "empty_ik k a ⟷ ik_max k < r"
        using Class False by (simp add: empty_ik_congruence_def r_def)
      moreover have "ik_max k < r ⟷ gamma_congruence a ∩ ik_range k = {}"
        using r by (fastforce simp: Class gamma_congruence_Some)
      ultimately show ?thesis by simp
    qed
  qed
qed

end
```

Congruences as a kind-aware integer domain

An operation at kind `k` computes the class of the mathematical result and then treats a possible overflow as Goblint's `congruenceDomain.ml` does under the overflow policy; a conversion follows its `cast_to`.

```
fun cong_aop :: "aop ⇒ congruence ⇒ congruence ⇒ congruence" where
  "cong_aop AAdd a b = a + b"
| "cong_aop ASub a b = a - b"
| "cong_aop AMul a b = a * b"
| "cong_aop ADiv a b = congruence_div a b"
| "cong_aop AMod a b = congruence_mod a b"

lemma cong_aop_sound:
  "m ∈ gamma_congruence a ⟹ n ∈ gamma_congruence b
```

```
    ⟹ aop_eval op m n ∈ gamma_congruence (cong_aop op a b)"
  by (cases op) (simp_all add: congruence_plus_sound congruence_minus_sound
      congruence_times_sound congruence_div_sound congruence_mod_sound)
```

Two classes with no common value are unequal, as in Goblint's `eq`.

```
definition cong_eq :: "congruence ⇒ congruence ⇒ bool option" where
  "cong_eq a b =
     (case congruence_eqb a b of
        Some r ⇒ Some r
      | None ⇒ if is_bottom_congruence (a ⊓ b) then Some False else None)"

lemma cong_eq_sound:
  "cong_eq a b = Some r ⟹ m ∈ gamma_congruence a ⟹ n ∈ gamma_congruence b ⟹ (m = n) = r"
  by (auto simp: cong_eq_def is_bottom_congruence_correct split: option.splits if_splits
      dest: congruence_eqb_sound)


fun cong_refine :: "cop ⇒ congruence ⇒ congruence ⇒ congruence × congruence" where
  "cong_refine CEq a b = (a ⊓ b, a ⊓ b)"
| "cong_refine _ a b = (a, b)"

lemma cong_refine_sound:
  "cop_eval c m n ⟹ m ∈ gamma_congruence a ⟹ n ∈ gamma_congruence b
     ⟹ m ∈ gamma_congruence (fst (cong_refine c a b))
       ∧ n ∈ gamma_congruence (snd (cong_refine c a b))"
  by (cases c) simp_all

definition cong_fits :: "ikind ⇒ congruence ⇒ bool" where
  "cong_fits k a =
     (case Rep_congruence a of
        None ⇒ True
      | Some (c, m) ⇒ m = 0 ∧ ik_min k ≤ c ∧ c ≤ ik_max k)"

lemma cong_fits_sound: "cong_fits k a ⟹ n ∈ gamma_congruence a ⟹ n ∈ ik_range k"
  by (cases a rule: congruence_rep_cases)
     (auto simp: cong_fits_def gamma_congruence_Some gamma_congruence_def)

definition cong_excludes :: "int ⇒ congruence ⇒ bool" where
  "cong_excludes x a = (case Rep_congruence a of None ⇒ True | Some (c, m) ⇒ ¬ m dvd x - c)"

lemma cong_excludes_sound: "cong_excludes x a ⟹ n ∈ gamma_congruence a ⟹ n ≠ x"
  by (auto simp: cong_excludes_def gamma_congruence_def split: option.splits)
```

A conversion keeps the class when the source kind's range lies inside the target's, wraps a constant, and answers top otherwise, as Goblint's `cast_to` does under every overflow policy.

```
definition cast_cong_ik :: "ikind ⇒ congruence ⇒ congruence" where
  "cast_cong_ik k a =
     (case Rep_congruence a of
        None ⇒ bot
      | Some (c, m) ⇒ if m = 0 then congruence_of_int (ik_norm k c) else top)"

lemma cast_cong_ik_sound:
  "n ∈ gamma_congruence a ⟹ ik_norm k n ∈ gamma_congruence (cast_cong_ik k a)"
  by (cases a rule: congruence_rep_cases)
     (simp add: gamma_congruence_def, auto simp: cast_cong_ik_def gamma_congruence_Some)
```

An arithmetic result follows Goblint's `add`, `sub`, `mul`, `div` and `rem`. A remainder of in-range operands, and a quotient by a divisor other than `-1`, stays in range and keeps its class. Otherwise a kind that wraps under the overflow policy normalizes the class to the kind (`handle_overflow`). A signed kind under `assume_top` keeps only a difference of two constants that lies in range and answers top for everything else. The Int product passes Goblint's `no_ov` case itself, when its interval shows that the result fits.

```
definition cong_arith_ik ::
    "signed_overflow_policy ⇒ ikind ⇒ aop ⇒ bool ⇒ congruence ⇒ congruence ⇒ congruence" where
  "cong_arith_ik ovp k op operands_fit a b =
     (let r = cong_aop op a b
      in if operands_fit ∧ (op = AMod ∨ op = ADiv ∧ cong_excludes (-1) b) then r
         else if should_wrap ovp k then norm_cong_ik k r
         else if op = ASub ∧ cong_fits k r then r
         else top)"

lemma cong_arith_ik_sound:
  assumes fit: "operands_fit ⟹ m ∈ ik_range k ∧ n ∈ ik_range k"
    and ma: "m ∈ gamma_congruence a" and nb: "n ∈ gamma_congruence b"
```

```
  shows "ik_norm k (aop_eval op m n) ∈ gamma_congruence (cong_arith_ik ovp k op operands_fit a b)"
proof -
  let ?r = "cong_aop op a b"
  have r: "aop_eval op m n ∈ gamma_congruence ?r" using ma nb by (rule cong_aop_sound)
  show ?thesis
  proof (cases "operands_fit ∧ (op = AMod ∨ op = ADiv ∧ cong_excludes (-1) b)")
    case True
    then have "aop_eval op m n ∈ ik_range k"
      using fit cong_excludes_sound[of "-1" b n] nb c_mod_in_range c_div_in_range by auto
    with True r show ?thesis by (simp add: cong_arith_ik_def)
  next
    case nr: False
    show ?thesis
    proof (cases "should_wrap ovp k")
      case True
      with nr r show ?thesis by (auto simp: cong_arith_ik_def Let_def norm_cong_ik_sound)
    next
      case False
      have "op = ASub ∧ cong_fits k ?r ⟹ aop_eval op m n ∈ ik_range k"
        using r cong_fits_sound by blast
      with nr False r show ?thesis by (auto simp: cong_arith_ik_def Let_def)
    qed
  qed
qed

instantiation congruence :: kind_int_domain
begin

definition top_ik_congruence :: "ikind ⇒ congruence" where "top_ik_congruence k = top"

definition of_int_ik_congruence :: "ikind ⇒ int ⇒ congruence" where
  "of_int_ik_congruence k n = congruence_of_int (ik_norm k n)"

definition cast_ik_congruence ::
    "signed_overflow_policy ⇒ ikind option ⇒ ikind ⇒ congruence ⇒ congruence" where
  "cast_ik_congruence ovp sk k a =
     (case sk of Some s ⇒ if ik_widens s k then a else cast_cong_ik k a | None ⇒ cast_cong_ik k a)"

definition arith_ik_congruence ::
    "signed_overflow_policy ⇒ ikind ⇒ aop ⇒ bool ⇒ congruence ⇒ congruence ⇒ congruence" where
  "arith_ik_congruence = cong_arith_ik"

definition cmp_ik_congruence :: "cop ⇒ congruence ⇒ congruence ⇒ bool option" where
  "cmp_ik_congruence = cop_of congruence_lt cong_eq"

definition meet_ik_congruence :: "congruence ⇒ congruence ⇒ congruence" where
  "meet_ik_congruence a b = a ⊓ b"

definition refine_ik_congruence :: "cop ⇒ congruence ⇒ congruence ⇒ congruence × congruence" where
  "refine_ik_congruence = cong_refine"

definition raw_aop_ik_congruence :: "aop ⇒ congruence ⇒ congruence ⇒ congruence" where
  "raw_aop_ik_congruence = cong_aop"

definition fits_ik_congruence :: "ikind ⇒ congruence ⇒ bool" where
  "fits_ik_congruence = cong_fits"

definition answer_ik_congruence :: "congruence ⇒ answer" where
  "answer_ik_congruence a = answer_with (int_congruence_update (λ_. a))"

definition of_answer_ik_congruence :: "ikind option ⇒ answer ⇒ congruence" where
  "of_answer_ik_congruence sk q = (case q of QBot ⇒ ⊥ | QTop ⇒ ⊤ | QLifted d ⇒ int_congruence d)"

instance proof
  fix n m :: int and k :: ikind and a b :: congruence and op :: aop and c :: cop and r :: bool
  show "n ∈ ik_range k ⟹ n ∈ numeric_domain_class.gamma (top_ik k :: congruence)"
    by (simp add: top_ik_congruence_def)
  show "ik_norm k n ∈ numeric_domain_class.gamma (of_int_ik k n :: congruence)"
    by (simp add: of_int_ik_congruence_def)
  show "(⋀k'. sk = Some k' ⟹ n ∈ ik_range k') ⟹ n ∈ numeric_domain_class.gamma a
      ⟹ ik_norm k n ∈ numeric_domain_class.gamma (cast_ik ovp sk k a)" for ovp sk
    by (cases sk)
       (auto simp: cast_ik_congruence_def ik_widens_def cast_cong_ik_sound simp del: ik_range_iff,
        auto)
  show "(fits ⟹ m ∈ ik_range k ∧ n ∈ ik_range k) ⟹ m ∈ numeric_domain_class.gamma a
      ⟹ n ∈ numeric_domain_class.gamma b
```

```
      ⟹ ik_norm k (aop_eval op m n) ∈ numeric_domain_class.gamma (arith_ik ovp k op fits a b)" for ovp fits
    using cong_arith_ik_sound[where operands_fit = fits and ovp = ovp and op = op and k = k and m =
      m and n = n]
    by (simp add: arith_ik_congruence_def)
  show "cmp_ik c a b = Some r ⟹ m ∈ numeric_domain_class.gamma a
      ⟹ n ∈ numeric_domain_class.gamma b ⟹ cop_eval c m n = r"
    unfolding cmp_ik_congruence_def
    by simp (rule cop_of_sound[OF congruence_lt_sound cong_eq_sound])
  show "m ∈ numeric_domain_class.gamma a ⟹ m ∈ numeric_domain_class.gamma b
      ⟹ m ∈ numeric_domain_class.gamma (meet_ik a b)"
    by (simp add: meet_ik_congruence_def)
  show "cop_eval c m n ⟹ m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ m ∈ numeric_domain_class.gamma (fst (refine_ik c a b))
        ∧ n ∈ numeric_domain_class.gamma (snd (refine_ik c a b))"
    by (simp add: refine_ik_congruence_def cong_refine_sound)
  show "m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ aop_eval op m n ∈ numeric_domain_class.gamma (raw_aop_ik op a b)"
    by (simp add: raw_aop_ik_congruence_def cong_aop_sound)
  show "fits_ik k a ⟹ n ∈ numeric_domain_class.gamma a ⟹ n ∈ ik_range k"
    using cong_fits_sound[of k a n] by (simp add: fits_ik_congruence_def)
  show "n ∈ numeric_domain_class.gamma a ⟹ n ∈ gamma_answer (answer_ik a)"
    by (simp add: answer_ik_congruence_def gamma_int_dom_def top_int_dom_ext_def gamma_sign_top
        gamma_parity_top top_ivl_def gamma_ivl_top)
  show "(⋀k. sk = Some k ⟹ n ∈ ik_range k) ⟹ n ∈ gamma_answer q
      ⟹ n ∈ numeric_domain_class.gamma (of_answer_ik sk q :: congruence)" for sk q
    by (cases q)
      (simp_all add: of_answer_ik_congruence_def gamma_int_dom_def gamma_sign_top gamma_parity_top)
qed

end
```

As for intervals, the class dictionary of a typed analysis carries the concretization, which is never called.

```
declare [[code abort: gamma_congruence]]

end
```

14.4 Congruence_Regression

```
theory Congruence_Regression
  imports Congruence_Kind "Voblint_Nonrelational.Nonrelational_Analysis"
    "Voblint_VIMP.VIMP_Notation"
begin
```

Regression: the typed Congruence analysis, executed

Each case runs the certified solver on a typed program inside Isabelle and pins the published value of one variable. Termination is decided by evaluation.

```
abbreviation cong_state_at :: "kind_decls ⇒ imp_prog ⇒ cfg_node ⇒ congruence tabs" where
  "cong_state_at ds p ≡ nonrel.state_at default_sem_conf ds 0 Globals_Join is_global p []"

abbreviation f_var :: "vname ⇒ var_id" where
  "f_var x ≡ ScopedId (STR ''f'') x"

abbreviation f_result :: "kind_decls ⇒ imp_prog ⇒ vname ⇒ congruence kd" where
  "f_result ds p x ≡ cong_state_at ds p (FunctionResult (STR ''f'')) (f_var x)"

abbreviation cong_equations :: "kind_decls ⇒ imp_prog ⇒ cfg_node × cfg_node list ⇒
    (cfg_node × cfg_node list, (unit, cfg_node list) global_unknown,
     (congruence tst lifted, congruence tst lifted) dg_state) strategy_tree" where
  "cong_equations ds p ≡ nonrel.equations default_sem_conf ds 0 is_global p"
```

Constants

Under Goblint's default `assume_top`, standalone Congruence answers top for signed addition and multiplication, constants included; only a difference of constants that stays in range is kept. An unsigned kind computes constants modulo its range. A `U8` operand is promoted to the signed `I32` first, so the examples use `U32`.

```
definition const_prog :: imp_prog where
  "const_prog = program { fun f() { x = 3; x = x * 5; } fun main() { f(); } }"

definition sub_prog :: imp_prog where
  "sub_prog = program { fun f() { x = 3; x = x - 5; } fun main() { f(); } }"

definition u32_x :: kind_decls where
  "u32_x = [(f_var (STR ''x''), U32)]"

lemma const_value:
  "f_result [] const_prog (STR ''x'') = KD I32 top"
  "f_result u32_x const_prog (STR ''x'') = KD U32 (mk_congruence 15 0)"
  "f_result [] sub_prog (STR ''x'') = KD I32 (mk_congruence (-2) 0)"
  by eval+
```

Classes through a loop

A step of `4` keeps an unsigned `x` in $4\mathbb{Z}$: wrapping shifts a value by a multiple of `2 ^ 32`, which `4` divides (`handle_overflow`). A step of `3` loses the class, since `3` does not divide `2 ^ 32`. At the signed `I32` the step answers top under `assume_top`; the interval product's `no_ov` recovers the class there.

```
definition pow2_prog :: imp_prog where
  "pow2_prog = program {
     fun f() { x = 0; while (x < 100) { x = x + 4; } }
     fun main() { f(); } }"

definition odd_step_prog :: imp_prog where
  "odd_step_prog = program {
     fun f() { x = 0; while (x < 99) { x = x + 3; } }
     fun main() { f(); } }"

lemma pow2_terminates_c:
  "TD_side_rule_Interp_solve_c Globals_Join (cong_equations [] pow2_prog)
     (cfg_exit (elab_cfg pow2_prog []), []) ≠ None"
  by eval

lemma pow2_terminates:
  "nonrel.terminates TYPE(congruence) default_sem_conf [] 0 Globals_Join is_global pow2_prog"
  unfolding nonrel.terminates_code
  using TD_side_rule_Interp.solve_dom_of_solve_c[OF pow2_terminates_c] by simp

lemma loop_values:
  "f_result u32_x pow2_prog (STR ''x'') = KD U32 (mk_congruence 0 4)"
```

```
  "f_result u32_x odd_step_prog (STR ''x'') = KD U32 top"
  "f_result [] pow2_prog (STR ''x'') = KD I32 top"
  by eval+
```

Overflow

`-6 + 10` wraps to `4` at `U32`, as in Goblint. A signed difference of constants that leaves the range answers top, Goblint's signed overflow under `assume_top`.

```
definition wrap_prog :: imp_prog where
  "wrap_prog = program { fun f() { x = 0 - 6; x = x + 10; } fun main() { f(); } }"

definition swrap_prog :: imp_prog where
  "swrap_prog = program { fun f() { x = 0 - 2147483647; x = x - 10; } fun main() { f(); } }"

lemma wrap_values:
  "f_result u32_x wrap_prog (STR ''x'') = KD U32 (mk_congruence 4 0)"
  "f_result [] swrap_prog (STR ''x'') = KD I32 top"
  by eval+
```

Conversions

`x` is a `U32` stepping by `8`. Widening it to the `I64` `y` keeps its class, as Goblint's `cast_to` does for a contained `from_ik`. Narrowing the class $6\mathbb{Z}$ from `I32` to `U8` answers top, as `cast_to` does for a non-constant; narrowing the constant `300` wraps it to `44`.

```
definition conv_prog :: imp_prog where
  "conv_prog = program {
     fun f() { x = 0; while (x < 200) { x = x + 8; } y = x; }
     fun main() { f(); } }"

definition conv_kinds :: kind_decls where
  "conv_kinds = [(f_var (STR ''x''), U32), (f_var (STR ''y''), I64)]"

definition narrow_prog :: imp_prog where
  "narrow_prog = program {
     fun f(c) { y = 0; if (c < 1) { y = 6; } z = y; w = 300; v = w; }
     fun main() { f(0); f(5); } }"

definition narrow_kinds :: kind_decls where
  "narrow_kinds = [(f_var (STR ''z''), U8), (f_var (STR ''v''), U8)]"

lemma conv_values:
  "f_result conv_kinds conv_prog (STR ''x'') = KD U32 (mk_congruence 0 8)"
  "f_result conv_kinds conv_prog (STR ''y'') = KD I64 (mk_congruence 0 8)"
  "f_result narrow_kinds narrow_prog (STR ''y'') = KD I32 (mk_congruence 0 6)"
  "f_result narrow_kinds narrow_prog (STR ''z'') = KD U8 top"
  "f_result narrow_kinds narrow_prog (STR ''v'') = KD U8 (mk_congruence 44 0)"
  by eval+

end
```

Chapter 15

Voblint_Analysis_Order

15.1 Order_Analysis

```
theory Order_Analysis
  imports "Voblint_Nonrelational.Nonrelational_Analysis" "HOL-Library.Product_Lexorder"
begin
```

The order domain at typed variables

A relational carrier over any variable type: a finite set of pairs `(x, y)`, each meaning that `x`'s value is at most `y`'s. Stored values are already normalized to their kinds, so the facts compare the values a run holds; a guard adds a fact only when it compares two variable reads, whose values the comparison sees unconverted. The flag records that every variable other than the return slot holds a value of its kind (`in_kinds`): a copy between two variables of one kind then passes the conversion unchanged.

```
datatype 'v order_rel = OBot | OFacts bool "('v × 'v) set"

instantiation order_rel :: (type) "{bounded_semilattice_sup_bot, order_top}"
begin

fun less_eq_order_rel :: "'v order_rel ⇒ 'v order_rel ⇒ bool" where
  "less_eq_order_rel OBot _ = True"
| "less_eq_order_rel (OFacts _ _) OBot = False"
| "less_eq_order_rel (OFacts t a) (OFacts u b) = (b ⊆ a ∧ (u ⟶ t))"

definition less_order_rel :: "'v order_rel ⇒ 'v order_rel ⇒ bool" where
  "less_order_rel a b ⟷ a ≤ b ∧ ¬ b ≤ a"

fun sup_order_rel :: "'v order_rel ⇒ 'v order_rel ⇒ 'v order_rel" where
  "sup_order_rel OBot b = b"
| "sup_order_rel a OBot = a"
| "sup_order_rel (OFacts t a) (OFacts u b) = OFacts (t ∧ u) (a ∩ b)"

definition bot_order_rel :: "'v order_rel" where "bot_order_rel = OBot"

definition top_order_rel :: "'v order_rel" where "top_order_rel = OFacts False {}"

instance proof
  fix x y z :: "'v order_rel"
  show "x < y ⟷ x ≤ y ∧ ¬ y ≤ x" by (simp add: less_order_rel_def)
  show "x ≤ x" by (cases x) simp_all
  show "x ≤ y ⟹ y ≤ z ⟹ x ≤ z" by (cases x; cases y; cases z) auto
  show "x ≤ y ⟹ y ≤ x ⟹ x = y" by (cases x; cases y) auto
  show "x ≤ x ⊔ y" "y ≤ x ⊔ y" by (cases x; cases y; auto)+
  show "y ≤ x ⟹ z ≤ x ⟹ y ⊔ z ≤ x" by (cases x; cases y; cases z) auto
  show "bot ≤ x" by (simp add: bot_order_rel_def)
  show "x ≤ top" by (cases x) (simp_all add: top_order_rel_def)
qed

end

instantiation order_rel :: (type) warrowing
begin

definition widen_order_rel :: "'v order_rel ⇒ 'v order_rel ⇒ 'v order_rel" where
  "widen_order_rel a b = a ⊔ b"

definition narrow_order_rel :: "'v order_rel ⇒ 'v order_rel ⇒ 'v order_rel" where
  "narrow_order_rel a b = b"

instance proof
  fix x y :: "'v order_rel"
  show "x ≤ x ∇ y" "y ≤ x ∇ y" by (simp_all add: widen_order_rel_def)
  show "y ≤ x ⟹ y ≤ x Δ y" "y ≤ x ⟹ x Δ y ≤ x"
    by (simp_all add: narrow_order_rel_def)
qed

end
```

An order value prints in the notation of Goblint's two-variable equality domain, which prints a conjunction `{x=y ∧ z=y}` and its extremes as ⊥ and ⊤; here each conjunct is an ordering `x≤y`. Printing needs a name for each variable and a linear order on them, so that `sorted_list_of_set` enumerates the facts deterministically (pairs are ordered lexicographically). The flag is not printed.

```
class order_name = linorder +
  fixes order_name :: "'a ⇒ String.literal"
```

```
instantiation String.literal :: order_name
begin

definition order_name_literal :: "String.literal ⇒ String.literal" where
  "order_name_literal x = x"

instance ..

end

fun string_of_facts :: "('v::order_name × 'v) list ⇒ String.literal" where
  "string_of_facts [] = STR ''''"
| "string_of_facts [(x, y)] = order_name x + sym_le + order_name y"
| "string_of_facts ((x, y) # p # ps) =
     order_name x + sym_le + order_name y + STR '' '' + sym_and + STR '' ''
     + string_of_facts (p # ps)"

instantiation order_rel :: (order_name) executable_domain
begin

definition is_empty_order_rel :: "'v::order_name order_rel ⇒ bool" where
  "is_empty_order_rel d ⟷ d = OBot"

definition to_string_order_rel :: "'v::order_name order_rel ⇒ String.literal" where
  "to_string_order_rel d =
     (case d of
        OBot ⇒ sym_bottom
      | OFacts _ ps ⇒
          (if ps = {} then sym_top
           else STR ''{'' + string_of_facts (sorted_list_of_set ps) + STR ''}''))"

instance ..

end

fun gamma_ord :: "kenv ⇒ var_id order_rel ⇒ store set" where
  "gamma_ord κ OBot = {}"
| "gamma_ord κ (OFacts t ps) = {s. (t ⟶ in_kinds κ s) ∧ (∀(x, y) ∈ ps. s x ≤ s y)}"

lemma gamma_ord_mono: "a ≤ b ⟹ gamma_ord κ a ⊆ gamma_ord κ b"
  by (cases a; cases b) auto

lemma gamma_ord_top [simp]: "gamma_ord κ top = UNIV"
  by (simp add: top_order_rel_def)

fun ord_forget :: "'v ⇒ 'v order_rel ⇒ 'v order_rel" where
  "ord_forget x OBot = OBot"
| "ord_forget x (OFacts t ps) = OFacts t {(a, b) ∈ ps. a ≠ x ∧ b ≠ x}"
```

Every write converts to its destination's kind except the return slot's, which no kind constrains, so a forgotten destination keeps the stores typed.

```
lemma ord_forget_sound:
  "s ∈ gamma_ord κ d ⟹ v = ik_norm (κ x) n ∨ x = ReturnVar
     ⟹ s(x := v) ∈ gamma_ord κ (ord_forget x d)"
  by (cases d) auto
```

Guards

The orders a comparison between `x` and `y` fixes when it has the truth value `pol`. A disequality fixes none: the carrier holds only weak orders.

```
fun cmp_facts :: "cop ⇒ bool ⇒ 'v ⇒ 'v ⇒ ('v × 'v) set" where
  "cmp_facts CLt True x y = {(x, y)}"
| "cmp_facts CLe True x y = {(x, y)}"
| "cmp_facts CGt True x y = {(y, x)}"
| "cmp_facts CGe True x y = {(y, x)}"
| "cmp_facts CEq True x y = {(x, y), (y, x)}"
| "cmp_facts CNe True x y = {}"
| "cmp_facts CLt False x y = {(y, x)}"
| "cmp_facts CLe False x y = {(y, x)}"
| "cmp_facts CGt False x y = {(x, y)}"
| "cmp_facts CGe False x y = {(x, y)}"
| "cmp_facts CEq False x y = {}"
| "cmp_facts CNe False x y = {(x, y), (y, x)}"
```

```
lemma cmp_facts_sound:
  "cop_eval c (s x) (s y) = pol ⟹ (a, b) ∈ cmp_facts c pol x y ⟹ s a ≤ s b"
  by (cases c; cases pol) auto

fun ord_branch :: "exp ⇒ bool ⇒ var_id order_rel ⇒ var_id order_rel" where
  "ord_branch (TCmp c (TVar k x) (TVar k' y)) pol (OFacts t ps) = OFacts t (ps ∪ cmp_facts c pol x y)"
| "ord_branch (TNot b) pol d = ord_branch b (¬ pol) d"
| "ord_branch _ _ d = d"

lemma ord_branch_sound:
  "s ∈ gamma_ord κ d ⟹ truthy (eval b s) = pol ⟹ s ∈ gamma_ord κ (ord_branch b pol d)"
proof (induction b pol d rule: ord_branch.induct)
  case (1 c k x k' y pol t ps)
  then show ?case using cmp_facts_sound[of c s x y pol] by (auto split: if_splits)
next
  case (2 b pol d)
  then show ?case by (auto split: if_splits)
qed simp_all
```

Verdicts

A pair `(x, y)` decides `x ≤ y` as true and `y < x` as false; with the reverse pair it also decides equality.

```
definition ord_le :: "var_id order_rel ⇒ var_id ⇒ var_id ⇒ bool" where
  "ord_le d x y = (case d of OBot ⇒ False | OFacts _ ps ⇒ x = y ∨ (x, y) ∈ ps)"

lemma ord_le_sound: "ord_le d x y ⟹ s ∈ gamma_ord κ d ⟹ s x ≤ s y"
  by (cases d) (auto simp: ord_le_def)

fun ord_decide :: "cop ⇒ var_id order_rel ⇒ var_id ⇒ var_id ⇒ bool option" where
  "ord_decide CLe d x y = (if ord_le d x y then Some True else None)"
| "ord_decide CGe d x y = (if ord_le d y x then Some True else None)"
| "ord_decide CLt d x y = (if ord_le d y x then Some False else None)"
| "ord_decide CGt d x y = (if ord_le d x y then Some False else None)"
| "ord_decide CEq d x y = (if ord_le d x y ∧ ord_le d y x then Some True else None)"
| "ord_decide CNe d x y = (if ord_le d x y ∧ ord_le d y x then Some False else None)"

lemma ord_decide_sound:
  "ord_decide c d x y = Some r ⟹ s ∈ gamma_ord κ d ⟹ cop_eval c (s x) (s y) = r"
  using ord_le_sound[of d x y s κ] ord_le_sound[of d y x s κ]
  by (cases c) (auto split: if_splits)

fun ord_classify :: "exp ⇒ var_id order_rel ⇒ check_result" where
  "ord_classify (TCmp c (TVar k x) (TVar k' y)) d =
     (case ord_decide c d x y of
        Some True ⇒ Check_Proved
      | Some False ⇒ Check_Refuted
      | None ⇒ Check_Unknown)"
| "ord_classify _ d = Check_Unknown"

lemma ord_classify_sound:
  assumes "s ∈ gamma_ord κ d"
  shows ord_classify_proved: "ord_classify c d = Check_Proved ⟹ truthy (eval c s)"
    and ord_classify_refuted: "ord_classify c d = Check_Refuted ⟹ ¬ truthy (eval c s)"
  using assms ord_decide_sound
  by (induction c d rule: ord_classify.induct;
      fastforce split: option.splits bool.splits if_splits)+
```

The query

The component answers `EvalInt` for a comparison between two variables from its own facts, as Goblint's `varEq` and `expRelation` do (`varEq.ml` line 595, `expRelation.ml` lines 53 to 70 at `0dc12d355`): a decided comparison is the exact truth value, anything else is ⊤. It asks nobody, as those analyses do not.

```
fun ord_query :: "channel ⇒ var_id order_rel lifted ⇒ channel" where
  "ord_query ch (Lifted d) (EvalInt (TCmp c (TVar k x) (TVar k' y))) =
     (case ord_decide c d x y of Some b ⇒ answer_of_int (if b then 1 else 0) | None ⇒ ⊤)"
| "ord_query ch d q = ⊤"

lemma ord_query_sound:
  "s ∈ gamma_ord κ d ⟹ eval_holds q (ord_query ch (Lifted d) q) s"
  using ord_decide_sound
  by (cases "(ch, Lifted d, q)" rule: ord_query.cases) (auto split: option.splits)
```

Assignments

At `x = e` the component asks about the state before the assignment, where `(κ x) e` evaluates to the new value of `x` and every other variable keeps its value. The exact answer `1` to `(κ x) e ≤ y` makes `(x, y)` true afterwards, and `y ≤ (κ x) e` makes `(y, x)` true. The candidate partners are the variables in `ys`. A copy from a variable of the same kind records both orders when the stores are typed and gives the destination the source's facts, as Goblint's `varEq` records an equality for an assignment between variables of one type (`varEq.ml` at `0dc12d355`).

```
definition ord_copy ::
    "kenv ⇒ var_id ⇒ exp ⇒ (var_id × var_id) set ⇒ (var_id × var_id) set" where
  "ord_copy κ x e ps = (case e of
     TVar k y ⇒ if y ≠ x ∧ y ≠ ReturnVar ∧ κ y = κ x
       then {(x, y), (y, x)} ∪ (λ(a, b). (x, b)) ‘ Set.filter (λ(a, b). a = y) ps
            ∪ (λ(a, b). (a, x)) ‘ Set.filter (λ(a, b). b = y) ps
       else {}
   | _ ⇒ {})"

definition ord_learn ::
    "kenv ⇒ var_id list ⇒ channel ⇒ var_id ⇒ exp ⇒ var_id order_rel ⇒ var_id order_rel" where
  "ord_learn κ ys ch x e d =
     (case d of
        OBot ⇒ OBot
      | OFacts t ps ⇒ OFacts t (ps
          ∪ (if t then ord_copy κ x e ps else {})
          ∪ set (map (λy. (x, y)) (filter (λy. y ≠ x
                ∧ answer_const (ch (EvalInt (TCmp CLe (TCast (κ x) e) (TVar (κ y) y)))) = Some 1) ys))
          ∪ set (map (λy. (y, x)) (filter (λy. y ≠ x
                ∧ answer_const (ch (EvalInt (TCmp CLe (TVar (κ y) y) (TCast (κ x) e)))) = Some 1) ys))))"

lemma eval_holds_constD:
  "eval_holds (EvalInt e) a s ⟹ answer_const a = Some n ⟹ eval e s = n"
  using gamma_int_dom_constant_subset by (cases a rule: answer_const.cases) auto

lemma ord_learn_sound:
  assumes post: "s(x := ik_norm (κ x) (eval e s)) ∈ gamma_ord κ d"
    and ch: "eval_query.channel_holds ch s"
  shows "s(x := ik_norm (κ x) (eval e s)) ∈ gamma_ord κ (ord_learn κ ys ch x e d)"
proof (cases d)
  case OBot with post show ?thesis by simp
next
  case (OFacts t ps)
  define s’ where "s’ = s(x := ik_norm (κ x) (eval e s))"
  have up: "s’ x ≤ s’ y"
    if "y ≠ x" "answer_const (ch (EvalInt (TCmp CLe (TCast (κ x) e) (TVar (κ y) y)))) = Some 1" for
      y
    using that eval_holds_constD[OF eval_query.channel_holdsD[OF ch] that(2)]
    by (simp add: s’_def split: if_splits)
  have down: "s’ y ≤ s’ x"
    if "y ≠ x" "answer_const (ch (EvalInt (TCmp CLe (TVar (κ y) y) (TCast (κ x) e)))) = Some 1" for
      y
    using that eval_holds_constD[OF eval_query.channel_holdsD[OF ch] that(2)]
    by (simp add: s’_def split: if_splits)
  have mem: "s’ ∈ gamma_ord κ (OFacts t ps)" using post OFacts by (simp add: s’_def)
  have copy: "s’ a ≤ s’ b" if t: "t" and ab: "(a, b) ∈ ord_copy κ x e ps" for a b
  proof -
    from ab obtain k y where e: "e = TVar k y" and y: "y ≠ x" "y ≠ ReturnVar" "κ y = κ x"
      and ab’: "(a, b) = (x, y) ∨ (a, b) = (y, x) ∨ (a = x ∧ (y, b) ∈ ps) ∨ (b = x ∧ (a, y) ∈ ps)"
      by (auto simp: ord_copy_def split: exp.splits if_splits)
    from post OFacts t have "in_kinds κ s’" by (simp add: s’_def)
    then have "s’ y ∈ ik_range (κ x)" using y unfolding in_kinds_def by metis
    then have "s’ x = s’ y" using y by (simp add: s’_def e del: ik_range_iff)
    then show ?thesis using ab’ mem by auto
  qed
  show ?thesis
    using mem up down copy unfolding s’_def[symmetric] by (auto simp: ord_learn_def OFacts)
qed
```

Calls

The callee starts from no facts and the caller learns none back. The callee's stores are typed when the caller's are, since a call converts each actual to its formal's kind; the caller's after the call when both sides' are, as the result converts to its destination's kind.

```
definition ord_in_kinds :: "var_id order_rel lifted ⇒ bool" where
  "ord_in_kinds d = (case d of Lifted (OFacts t _) ⇒ t | _ ⇒ False)"
```

```
definition ord_enter :: "var_id list ⇒ exp list ⇒ var_id order_rel lifted ⇒ var_id order_rel" where
  "ord_enter ps es d = OFacts (ord_in_kinds d) {}"
```

The component

Assignments forget their destination and learn, library calls forget it, and a return forgets the return slot. The caller learns nothing back from a call: a callee may change globals, so no fact survives it.

```
definition ord_spec :: "kenv ⇒ var_id list ⇒ var_id order_rel lifted local_spec"
  where
  "ord_spec κ ys =
     (| ls_query = ord_query,
       ls_skip = (λch d. d),
       ls_assign = (λch x e. map_lift (ord_learn κ ys ch x e ∘ ord_forget x)),
       ls_special = (λch sc x. map_lift (ord_forget x)),
       ls_branch = (λch b pol. map_lift (ord_branch b pol)),
       ls_body = (λch p d. d),
       ls_return = (λch r p d. case r of None ⇒ d | Some a ⇒ map_lift (ord_forget ReturnVar) d),
       ls_event = (λch evt d. d),
       ls_enter = (λch ci p. [(fst p, Lifted (ord_enter (ci_formals ci) (ci_args ci) (fst p)))]),
       ls_combine_env = combine_env_identity,
       ls_combine_assign = (λch ci d de. Lifted (OFacts (ord_in_kinds d ∧ ord_in_kinds de) {})) |)"

abbreviation gamma_ord_lift :: "kenv ⇒ var_id order_rel lifted ⇒ store set" where
  "gamma_ord_lift κ d ≡ gamma_lift (gamma_ord κ) (map_lift (λd. d) d)"

lemma gamma_ord_lift_simps [simp]:
  "gamma_ord_lift κ Bot = {}"
  "gamma_ord_lift κ (Lifted x) = gamma_ord κ x"
  "gamma_ord_lift κ (map_lift f (Lifted x)) = gamma_ord κ (f x)"
  by (simp_all add: map_lift_def)

lemma gamma_ord_lift_mono: "x ≤ y ⟹ gamma_ord_lift κ x ⊆ gamma_ord_lift κ y"
  by (cases x; cases y) (auto dest: gamma_ord_mono)

lemma ord_enter_sound:
  assumes "s ∈ gamma_ord_lift κ d"
  shows "bind_formals ps (map2 (λx. ik_norm (κ x)) ps (map (λe. eval e s) es))
           (enter_state is_global s)
           ∈ gamma_ord κ (ord_enter ps es d)"
proof -
  have "in_kinds κ (call_enter κ (CallEdge dst ps es) s)" if "ord_in_kinds d" for dst
    using assms that by (intro in_kinds_call_enter)
      (auto simp: ord_in_kinds_def split: lifted.splits order_rel.splits)
  then show ?thesis
    by (auto simp: ord_enter_def call_enter_CallEdge)
qed

lemma ord_in_kinds_in_kinds: "s ∈ gamma_ord_lift κ d ⟹ ord_in_kinds d ⟹ in_kinds κ s"
  by (auto simp: ord_in_kinds_def split: lifted.splits order_rel.splits)

lemma in_kinds_gcombine:
  "in_kinds κ s ⟹ in_kinds κ t
     ⟹ in_kinds κ (combine_assign dst (kind_conv κ dst (t ReturnVar)) (combine_env is_global s t))"
  by (cases dst) (auto simp: in_kinds_combine_env)

theorem ord_spec_sound:
  "sound_local_spec κ (gamma_ord_lift κ) (ord_spec κ ys)"
proof -
  have L: "∃x. d = Lifted x ∧ s ∈ gamma_ord κ x" if "s ∈ gamma_ord_lift κ d" for s d
    using that by (cases d) simp_all
  show ?thesis
  proof (rule sound_local_specI)
    show "⋀x y. x ≤ y ⟹ gamma_ord_lift κ x ⊆ gamma_ord_lift κ y" by (rule gamma_ord_lift_mono)
    show "sound_query (gamma_ord_lift κ) (ls_query (ord_spec κ ys))"
      by (auto simp: ord_spec_def sound_query_def ord_query_sound dest!: L)
    show "sound_skip (gamma_ord_lift κ) (ls_skip (ord_spec κ ys))"
      by (simp add: ord_spec_def sound_skip_def)
    show "sound_body (gamma_ord_lift κ) (ls_body (ord_spec κ ys))"
      by (simp add: ord_spec_def sound_body_def)
    show "sound_event (gamma_ord_lift κ) (ls_event (ord_spec κ ys))"
      by (simp add: ord_spec_def sound_event_def)
    show "sound_assign κ (gamma_ord_lift κ) (ls_assign (ord_spec κ ys))"
      by (auto dest!: L simp: ord_spec_def sound_assign_def
          intro!: ord_learn_sound ord_forget_sound)
```

```
    show "sound_special κ (gamma_ord_lift κ) (ls_special (ord_spec κ ys))"
      by (auto simp: ord_spec_def sound_special_def dest!: L intro: ord_forget_sound)
    show "sound_branch (gamma_ord_lift κ) (ls_branch (ord_spec κ ys))"
      using ord_branch_sound by (fastforce simp: ord_spec_def sound_branch_def dest!: L)
    show "sound_return (gamma_ord_lift κ) (ls_return (ord_spec κ ys))"
      by (auto simp: ord_spec_def sound_return_def dest!: L intro: ord_forget_sound
          split: option.splits)
    show "sound_enter κ (gamma_ord_lift κ) (ls_enter (ord_spec κ ys))"
      by (auto simp: ord_spec_def sound_enter_def call_enter_CallEdge
          intro: ord_enter_sound)
    show "sound_combine κ (gamma_ord_lift κ) (ls_combine_env (ord_spec κ ys))
        (ls_combine_assign (ord_spec κ ys))"
      by (auto simp: ord_spec_def sound_combine_def combine_env_identity_def
          combine_collect_def intro!: in_kinds_gcombine dest:
            ord_in_kinds_in_kinds)
  qed
qed
```

The candidate partners: every variable some procedure of the program declares (`program_vars`).

```
definition ord_vars :: "imp_prog ⇒ var_id list" where
  "ord_vars p = remdups (concat (map (λq. map (resolve_var (declared_global p) q)
     (filter (λx. x ≠ ret_var) (scope_vnames_list p q))) (prog_main_name # prog_procs p)))"
```

The pipeline instance

The same unit-context instance as the non-relational typed analyses, with the order component in place of a per-variable state. The concretization does not depend on the kind declarations, but the program's runs and initial stores do.

```
global_interpretation order_analysis: dg_analysis
    "kenv_of ds" "λ_. init_stores (kenv_of ds)"
    "λ_ p. ord_spec (kenv_of ds) (ord_vars p)" "λ_ d. d = OBot" "λ_ d. d" "OFacts True {}"
    "Analysis_Global ()" Analysis_Global Activation_Seed "λ_ _ _ _ _. ()" "()"
    "TD_side_rule_Interp_solve r"
    "TD_side_rule_Interp.solve_dom TYPE((unit, unit) global_unknown)
       TYPE((var_id order_rel lifted, var_id order_rel lifted) dg_state) r"
    OBot ord_classify "gamma_ord (kenv_of ds)" "λd. d = OBot"
    "TD_side_rule_Interp_solve_c r"
    "λ_ c. dg_spec_of c" "λp d e. d" "λ_ d. d" "λ_ d. Bot" "λ_ p. []" "λp ci d e. d"
    "λp. elab_cfg p ds" "λ_. FunctionEntry prog_main_name"
  for ds r
proof (rule dg_analysis_whole_stateI[OF td_certified_solver], goal_cases CompSound EnterSingle
    EmptyRd EmptyV SeedNe ClProved ClRefuted BotState InitSound SeedNeGlobal PEntry FinIntra
    FinCalls CallsUnique)
  case (CompSound p) show ?case by (rule ord_spec_sound)
next
  case (EnterSingle p ci d) show ?case
    by (simp add: ord_spec_def dg_pipeline.comp_entry_def)
next
  case (EmptyRd p s) show ?case by simp
next
  case EmptyV show ?case by (rule sound_emptinessI) simp
next
  case (SeedNe v ctx) show ?case by simp
next
  case (ClProved c d s) then show ?case by (intro ord_classify_proved)
next
  case (ClRefuted c d s) then show ?case by (intro ord_classify_refuted)
next
  case BotState show ?case by simp
next
  case (InitSound p) show ?case by (auto dest: init_stores_in_kinds)
next
  case (SeedNeGlobal v ctx n) show ?case by simp
next
  case (PEntry p) show ?case by (simp add: elab_cfg_def)
next
  case (FinIntra p) show ?case by (simp add: elab_cfg_def compile_prog_finite)
next
  case (FinCalls p) show ?case by (simp add: elab_cfg_def compile_prog_finite)
next
  case (CallsUnique p) show ?case by (rule elab_cfg_calls_source_unique)
qed

end
```

15.2 Order_Regression

```
theory Order_Regression
  imports Order_Analysis "Voblint_VIMP.VIMP_Notation"
begin
```

Regression: the typed order analysis, executed

Each case runs the certified solver on a typed program inside Isabelle and pins the order facts published at the end of `f`. Termination is decided by evaluation.

```
abbreviation f_var :: "vname ⇒ var_id" where
  "f_var x ≡ ScopedId (STR ''f'') x"

abbreviation ord_result :: "kind_decls ⇒ imp_prog ⇒ var_id order_rel" where
  "ord_result ds p ≡ order_analysis.state_at ds Globals_Join is_global p () (FunctionResult (STR ''f''))"
```

The loop exits when `x < y` fails, so `y ≤ x` holds at the exit. The body assigns `x`, which forgets every fact about it, so no fact survives into the loop.

```
definition loop_prog :: imp_prog where
  "loop_prog = program {
     fun f(x, y) { while (x < y) { x = x + 1; } }
     fun main() { f(0, 5); } }"

lemma loop_terminates_c:
  "TD_side_rule_Interp_solve_c Globals_Join (order_analysis.equations [] is_global loop_prog)
     (cfg_exit (elab_cfg loop_prog []), ()) ≠ None"
  by eval

lemma loop_terminates: "order_analysis.terminates [] Globals_Join is_global loop_prog"
  unfolding order_analysis.terminates_code
  using TD_side_rule_Interp.solve_dom_of_solve_c[OF loop_terminates_c] by simp

lemma loop_facts: "ord_result [] loop_prog = OFacts True {(f_var (STR ''y''), f_var (STR ''x''))}"
  by eval
```

The facts are about stored values, which already lie in their kinds: a `U8 x` and an `I32 y` compared through a conversion yield no fact, since the comparison does not see the variables' values unconverted.

```
definition mixed_prog :: imp_prog where
  "mixed_prog = program {
     fun f(x, y) { while (x < y) { y = y - 1; } }
     fun main() { f(0, 5); } }"

lemma mixed_facts:
  "ord_result [(f_var (STR ''x''), U8)] mixed_prog = OFacts True {}"
  by eval
```

Rendering

`to_string` renders bottom and top by their symbols and a set of facts as a sorted conjunction of ≤ facts, whatever the flag.

```
lemma to_string_order_rel_regression:
  "to_string (OBot :: vname order_rel) = sym_bottom"
  "to_string (OFacts True {} :: vname order_rel) = sym_top"
  "to_string (OFacts False {(STR ''y'', STR ''z''), (STR ''x'', STR ''y'')}) =
     STR ''{x<le>y <and> y<le>z}''"
  by eval+

end
```

Chapter 16

Voblint__Analysis__Int

16.1 Int_Warrowing

```
theory Int_Warrowing
  imports
    "Voblint_Domain.Int_Lattice"
    "Voblint_Analysis_Sign.Sign_Warrowing"
    "Voblint_Analysis_Parity.Parity_Warrowing"
    "Voblint_Analysis_Interval.Interval_Warrowing"
    "Voblint_Analysis_Congruence.Congruence_Warrowing"
begin
```

Composite widening and narrowing

Purely componentwise, with no reduced-product refinement attached to either operation. Widening already matches Goblint's own `~norefine:true` choice for `join`/`widen`. Narrowing is a deliberate divergence from Goblint's `IntDomTuple`, forced by the vendored TD solver's interface:

`narrow_ge: b <= a ==> b <= a △ b narrow_le: b <= a ==> a △ b <= a`

Suppose componentwise narrowing gives `b <= narrow_raw a b <= a` and refinement is then applied on top, Goblint-style: `refine mode (narrow_raw a b)`. Refinement is reductive (`Int_Refinement_Control.refine_reductive`), so the result stays `<= narrow_raw a b <= a` and `narrow_le` survives – but there is no general reason for `b <= refine mode (narrow_raw a b)` to hold. Refinement may push the result strictly below `b`.

The tempting escape is a stability argument: if `b` were already refinement-stable (`refine mode b = b`) and refinement is monotone, then `b = refine mode b <= refine mode (narrow_raw a b)` would follow from `b <= narrow_raw a b`. But that needs every value reaching narrowing to already be refinement-stable, and this carrier has no such invariant: the composite value `progressive_refinement_input`, `STop × [-1,0] × PEven × top`, is one where `refine mode b ~= b`. Join and widening deliberately do not refine either (mirroring Goblint's `~norefine:true` for both), so a value reaching narrowing by way of a widened solver state is not guaranteed stable. Building narrowing on an invariant the type does not enforce would be unsound at the class instance, not merely imprecise.

Composite widening and narrowing therefore run no refinement at all, matching every other component's own choice (`Sign_Warrowing`, `Interval_Warrowing`, `Parity_Warrowing`, `Congruence_Warrowing`): each component widens/narrows on its own terms, and the composite record update runs no cross-component step afterward.

The lattice instance in `Int_Lattice` sits on the record scheme `'a int_dom_scheme`, so `widen`/`narrow` follow the same route and need `warrowing` on the scheme's `more` field too. `unit`, the `more` type of the closed `int_dom`, gets the trivial instance.

```
instantiation unit :: warrowing
begin
definition widen_unit :: "unit => unit => unit" where "(a ▽ b) = ()"
definition narrow_unit :: "unit => unit => unit" where "(a △ b) = ()"
instance by intro_classes simp_all
end

instantiation int_dom_ext :: ("{bounded_lattice, warrowing}") warrowing
begin

definition widen_int_dom_ext :: "'a int_dom_scheme => 'a int_dom_scheme => 'a int_dom_scheme" where
  "((a :: 'a int_dom_scheme) ▽ b) =
     int_dom.extend
       (int_dom.truncate
         (a(|
           int_sign := ((int_sign a) ▽ (int_sign b)),
           int_ivl := ((int_ivl a) ▽ (int_ivl b)),
           int_parity := ((int_parity a) ▽ (int_parity b)),
           int_congruence := ((int_congruence a) ▽ (int_congruence b))
         |)))
       ((int_dom.more a) ▽ (int_dom.more b))"

definition narrow_int_dom_ext :: "'a int_dom_scheme => 'a int_dom_scheme => 'a int_dom_scheme" where
  "((a :: 'a int_dom_scheme) △ b) =
     int_dom.extend
       (int_dom.truncate
         (a(|
           int_sign := ((int_sign a) △ (int_sign b)),
           int_ivl := ((int_ivl a) △ (int_ivl b)),
           int_parity := ((int_parity a) △ (int_parity b)),
           int_congruence := ((int_congruence a) △ (int_congruence b))
         |)))
       ((int_dom.more a) △ (int_dom.more b))"
```

```
lemma int_dom_extend_truncate_select [simp]:
  "int_sign (int_dom.extend (int_dom.truncate r) m) = int_sign r"
  "int_ivl (int_dom.extend (int_dom.truncate r) m) = int_ivl r"
  "int_parity (int_dom.extend (int_dom.truncate r) m) = int_parity r"
  "int_congruence (int_dom.extend (int_dom.truncate r) m) = int_congruence r"
  "int_dom.more (int_dom.extend (int_dom.truncate r) m) = m"
  by (simp_all add: int_dom.defs)

instance proof intro_classes
  fix a b :: "'a int_dom_scheme"
  assume ba: "b <= a"
  have hb: "int_sign b <= int_sign a" "int_ivl b <= int_ivl a"
           "int_parity b <= int_parity a" "int_congruence b <= int_congruence a"
           "int_dom.more b <= int_dom.more a"
    using ba by (simp_all add: less_eq_int_dom_ext_def)
  have s: "int_sign b <= ((int_sign a) Δ (int_sign b))"
    by (rule narrow_ge[OF hb(1)])
  have i: "int_ivl b <= ((int_ivl a) Δ (int_ivl b))"
    by (rule narrow_ge[OF hb(2)])
  have p: "int_parity b <= ((int_parity a) Δ (int_parity b))"
    by (rule narrow_ge[OF hb(3)])
  have c: "int_congruence b <= ((int_congruence a) Δ (int_congruence b))"
    by (rule narrow_ge[OF hb(4)])
  have m: "int_dom.more b <= ((int_dom.more a) Δ (int_dom.more b))"
    by (rule narrow_ge[OF hb(5)])
  show "b <= (a Δ b)"
    unfolding narrow_int_dom_ext_def less_eq_int_dom_ext_def
    using s i p c m by simp
next
  fix a b :: "'a int_dom_scheme"
  assume ba: "b <= a"
  have hb: "int_sign b <= int_sign a" "int_ivl b <= int_ivl a"
           "int_parity b <= int_parity a" "int_congruence b <= int_congruence a"
           "int_dom.more b <= int_dom.more a"
    using ba by (simp_all add: less_eq_int_dom_ext_def)
  have s: "((int_sign a) Δ (int_sign b)) <= int_sign a"
    by (rule narrow_le[OF hb(1)])
  have i: "((int_ivl a) Δ (int_ivl b)) <= int_ivl a"
    by (rule narrow_le[OF hb(2)])
  have p: "((int_parity a) Δ (int_parity b)) <= int_parity a"
    by (rule narrow_le[OF hb(3)])
  have c: "((int_congruence a) Δ (int_congruence b)) <= int_congruence a"
    by (rule narrow_le[OF hb(4)])
  have m: "((int_dom.more a) Δ (int_dom.more b)) <= int_dom.more a"
    by (rule narrow_le[OF hb(5)])
  show "(a Δ b) <= a"
    unfolding narrow_int_dom_ext_def less_eq_int_dom_ext_def
    using s i p c m by simp
next
  fix a b :: "'a int_dom_scheme"
  have s: "int_sign a <= ((int_sign a) ∇ (int_sign b))"
    by (rule widen_ge1)
  have i: "int_ivl a <= ((int_ivl a) ∇ (int_ivl b))"
    by (rule widen_ge1)
  have p: "int_parity a <= ((int_parity a) ∇ (int_parity b))"
    by (rule widen_ge1)
  have c: "int_congruence a <= ((int_congruence a) ∇ (int_congruence b))"
    by (rule widen_ge1)
  have m: "int_dom.more a <= ((int_dom.more a) ∇ (int_dom.more b))"
    by (rule widen_ge1)
  show "a <= (a ∇ b)"
    unfolding widen_int_dom_ext_def less_eq_int_dom_ext_def
    using s i p c m by simp
next
  fix a b :: "'a int_dom_scheme"
  have s: "int_sign b <= ((int_sign a) ∇ (int_sign b))"
    by (rule widen_ge2)
  have i: "int_ivl b <= ((int_ivl a) ∇ (int_ivl b))"
    by (rule widen_ge2)
  have p: "int_parity b <= ((int_parity a) ∇ (int_parity b))"
    by (rule widen_ge2)
  have c: "int_congruence b <= ((int_congruence a) ∇ (int_congruence b))"
    by (rule widen_ge2)
  have m: "int_dom.more b <= ((int_dom.more a) ∇ (int_dom.more b))"
    by (rule widen_ge2)
```

```
  show "b <= (a ▽ b)"
    unfolding widen_int_dom_ext_def less_eq_int_dom_ext_def
    using s i p c m by simp
qed

end
```

Numeric domain instance

`numeric_domain` extends `executable_domain`, which includes the solver's `warrowing`, so the composite instance asks the same of the `more` field as the widening and narrowing above.

```
instantiation int_dom_ext ::
  ("{bounded_lattice, warrowing}") numeric_domain
begin

definition gamma_abs_int_dom_ext [simp]:
  "numeric_domain_class.gamma (d :: 'a int_dom_scheme) = gamma_int_dom d"

definition is_empty_int_dom_ext [simp]:
  "is_empty (d :: 'a int_dom_scheme) = is_bottom_int_dom d"

definition to_string_int_dom_ext [simp]:
  "to_string (d :: 'a int_dom_scheme) = string_of_int_dom d"

instance
proof intro_classes
  show "numeric_domain_class.gamma (bot :: 'a int_dom_scheme) = {}"
    by (simp add: gamma_int_dom_def bot_int_dom_ext_def
          bot_sign_def bot_ivl_def bot_parity_def)
next
  show "numeric_domain_class.gamma (top :: 'a int_dom_scheme) = UNIV"
    by (simp add: gamma_int_dom_def top_int_dom_ext_def
          gamma_sign_top gamma_ivl_top top_ivl_def gamma_parity_top)
next
  fix a b :: "'a int_dom_scheme"
  show "a ≤ b ⟹ numeric_domain_class.gamma a ⊆ numeric_domain_class.gamma b"
    by (simp add: gamma_int_dom_mono)
next
  fix a :: "'a int_dom_scheme"
  show "is_empty a ⟷ numeric_domain_class.gamma a = {}"
    by (simp add: is_bottom_int_dom_correct)
qed

end

end
```

16.2 Int_Refinement

```
theory Int_Refinement
  imports
    Int_Warrowing
    "HOL-Library.While_Combinator"
begin
```

Closed-world integer-domain refinement

Refinement exchanges a fixed vocabulary of facts between the components of the concrete integer-domain record. Reduction-step laws isolate progressive composition from the record representation.

```
definition int_reduction_step :: "(int_dom => int_dom) => bool" where
  "int_reduction_step step ⟷
     (∀d. gamma_int_dom (step d) = gamma_int_dom d) ∧
     (∀d. step d ≤ d) ∧
     mono step"

lemma int_reduction_step_exact:
  assumes "int_reduction_step step"
  shows "gamma_int_dom (step d) = gamma_int_dom d"
  using assms unfolding int_reduction_step_def by blast

lemma int_reduction_step_reductive:
  assumes "int_reduction_step step"
  shows "step d ≤ d"
  using assms unfolding int_reduction_step_def by blast

lemma int_reduction_step_mono:
  assumes "int_reduction_step step"
  shows "mono step"
  using assms unfolding int_reduction_step_def by blast

lemma int_reduction_step_id [simp]:
  "int_reduction_step id"
  unfolding int_reduction_step_def mono_def by simp

lemma int_reduction_step_comp:
  assumes f: "int_reduction_step f"
      and g: "int_reduction_step g"
  shows "int_reduction_step (g o f)"
proof (unfold int_reduction_step_def, intro conjI allI)
  fix d
  show "gamma_int_dom ((g o f) d) = gamma_int_dom d"
    using int_reduction_step_exact[OF g, of "f d"]
          int_reduction_step_exact[OF f, of d]
    by simp
next
  fix d
  have "g (f d) ≤ f d"
    by (rule int_reduction_step_reductive[OF g])
  also have "f d ≤ d"
    by (rule int_reduction_step_reductive[OF f])
  finally show "(g o f) d ≤ d"
    by simp
next
  show "mono (g o f)"
    using int_reduction_step_mono[OF f]
          int_reduction_step_mono[OF g]
    unfolding mono_def by auto
qed

fun apply_reduction_steps ::
  "(int_dom => int_dom) list => int_dom => int_dom" where
  "apply_reduction_steps [] d = d"
| "apply_reduction_steps (step # steps) d =
     apply_reduction_steps steps (step d)"

lemma int_reduction_steps:
  assumes "∀step ∈ set steps. int_reduction_step step"
  shows "int_reduction_step (apply_reduction_steps steps)"
  using assms
proof (induction steps)
  case Nil
  then show ?case
```

```
    unfolding int_reduction_step_def mono_def by simp
next
  case (Cons step steps)
  have step: "int_reduction_step step"
    using Cons.prems by simp
  have rest: "int_reduction_step (apply_reduction_steps steps)"
    using Cons.IH Cons.prems by simp
  have valid: "int_reduction_step (apply_reduction_steps steps o step)"
    by (rule int_reduction_step_comp[OF step rest])
  from valid show ?case
    unfolding int_reduction_step_def mono_def by simp
qed
```

Refinement facts

Two kinds of fact travel between components: an interval and a congruence. `interval_sign_fact` and `interval_parity_fact` read off the sign and parity an interval implies; the lemmas below show each over-approximates the interval and is monotone.

```
type_synonym interval_fact = ivl
type_synonym congruence_fact = congruence

definition interval_fact_of_ivl :: "ivl => interval_fact" where
  "interval_fact_of_ivl i = i"

definition interval_sign_fact :: "interval_fact => sign" where
  "interval_sign_fact i =
     (if is_bottom_ivl i then SBot
      else case i of Ivl l u =>
        if u < Fin 0 then SNeg
        else if u = Fin 0 then
          (if l = Fin 0 then SZero else SNonPos)
        else if Fin 0 < l then SPos
        else if l = Fin 0 then SNonNeg
        else STop)"

definition interval_parity_fact :: "interval_fact => parity" where
  "interval_parity_fact i =
     (if is_bottom_ivl i then PBot
      else case i of
        Ivl (Fin l) (Fin u) =>
          if l = u then if even l then PEven else POdd else PTop
      | _ => PTop)"

lemma interval_sign_fact_sound:
  "gamma_ivl i ⊆ gamma_sign (interval_sign_fact i)"
proof
  fix n
  assume n: "n ∈ gamma_ivl i"
  obtain l u where i: "i = Ivl l u"
    by (rule ivl_exhaustE)
  from n have bounds: "l ≤ Fin n" "Fin n ≤ u"
    unfolding i by auto
  from n have nonempty: "¬ is_bottom_ivl i"
    using is_bottom_ivl_correct by auto
  show "n ∈ gamma_sign (interval_sign_fact i)"
    unfolding interval_sign_fact_def i
    using bounds nonempty
    by (cases l; cases u)
       (auto simp: is_bottom_ivl_def less_eint_def
          split: if_splits)
qed

lemma interval_parity_fact_sound:
  "gamma_ivl i ⊆ gamma_parity (interval_parity_fact i)"
proof
  fix n
  assume n: "n ∈ gamma_ivl i"
  obtain l u where i: "i = Ivl l u"
    by (rule ivl_exhaustE)
  from n have bounds: "l ≤ Fin n" "Fin n ≤ u"
    unfolding i by auto
  from n have nonempty: "¬ is_bottom_ivl i"
    using is_bottom_ivl_correct by auto
  show "n ∈ gamma_parity (interval_parity_fact i)"
    unfolding interval_parity_fact_def i
```

```
    using bounds nonempty
    by (cases l; cases u)
       (auto simp: is_bottom_ivl_def
          split: if_splits)

qed

lemma interval_fact_of_sign_mono:
  assumes "s ≤ t"
  shows "interval_fact_of_sign s ≤ interval_fact_of_sign t"
  using assms
  by (cases s; cases t)
     (simp_all add: less_eq_sign_def less_eq_ivl_def
        bot_ivl_def top_ivl_def ivl_top_def)

lemma interval_sign_fact_mono:
  assumes "i ≤ j"
  shows "interval_sign_fact i ≤ interval_sign_fact j"
proof -
  obtain l1 u1 where i: "i = Ivl l1 u1"
    by (rule ivl_exhaustE)
  obtain l2 u2 where j: "j = Ivl l2 u2"
    by (rule ivl_exhaustE)
  show ?thesis
    using assms
    unfolding i j interval_sign_fact_def
    by (cases l1; cases u1; cases l2; cases u2)
       (auto simp: less_eq_ivl_def is_bottom_ivl_def
          less_eint_def less_eq_sign_def
          split: if_splits)
qed

lemma interval_parity_fact_mono:
  assumes "i ≤ j"
  shows "interval_parity_fact i ≤ interval_parity_fact j"
proof -
  obtain l1 u1 where i: "i = Ivl l1 u1"
    by (rule ivl_exhaustE)
  obtain l2 u2 where j: "j = Ivl l2 u2"
    by (rule ivl_exhaustE)
  show ?thesis
    using assms
    unfolding i j interval_parity_fact_def
    by (cases l1; cases u1; cases l2; cases u2)
       (auto simp: less_eq_ivl_def is_bottom_ivl_def
          less_eq_parity_def
          split: if_splits)

qed
```

Interval-fact fan-out

`refine_interval` intersects the sign and interval components into one interval fact and meets every interval-readable component with it. The step preserves the concretization, only shrinks the record and is monotone, so it is an `int_reduction_step`.

```
definition refine_sign_with_interval ::
  "interval_fact => sign => sign" where
  "refine_sign_with_interval fct s =
     interval_sign_fact fct ⊓ s"

definition refine_ivl_with_interval ::
  "interval_fact => ivl => ivl" where
  "refine_ivl_with_interval fct i = intersect_ivl fct i"

definition refine_parity_with_interval ::
  "interval_fact => parity => parity" where
  "refine_parity_with_interval fct p =
     interval_parity_fact fct ⊓ p"

definition interval_fact_of_int_dom :: "int_dom => interval_fact" where
  "interval_fact_of_int_dom d =
     intersect_ivl
       (interval_fact_of_sign (int_sign d))
       (interval_fact_of_ivl (int_ivl d))"
```

```
definition refine_interval :: "int_dom => int_dom" where
  "refine_interval d =
     (let fct = interval_fact_of_int_dom d
      in d⦇
           int_sign := refine_sign_with_interval fct (int_sign d),
           int_ivl := refine_ivl_with_interval fct (int_ivl d),
           int_parity := refine_parity_with_interval fct (int_parity d)
         ⦈)"

lemma gamma_refine_sign_with_interval [simp]:
  "gamma_sign (refine_sign_with_interval fct s) =
   gamma_sign (interval_sign_fact fct) ∩ gamma_sign s"
  by (simp add: refine_sign_with_interval_def)

lemma gamma_refine_ivl_with_interval:
  "gamma_ivl (refine_ivl_with_interval fct i) =
   gamma_ivl fct ∩ gamma_ivl i"
  by (simp only: refine_ivl_with_interval_def gamma_intersect_ivl_exact)

lemma gamma_refine_parity_with_interval [simp]:
  "gamma_parity (refine_parity_with_interval fct p) =
   gamma_parity (interval_parity_fact fct) ∩ gamma_parity p"
  by (simp add: refine_parity_with_interval_def)

lemma refine_sign_with_interval_le:
  "refine_sign_with_interval fct s ≤ s"
  unfolding refine_sign_with_interval_def
  by (rule inf_le2)

lemma refine_ivl_with_interval_le:
  "refine_ivl_with_interval fct i ≤ i"
  unfolding refine_ivl_with_interval_def
  by (rule intersect_ivl_le2)

lemma refine_parity_with_interval_le:
  "refine_parity_with_interval fct p ≤ p"
  unfolding refine_parity_with_interval_def
  by (rule inf_le2)

lemma refine_sign_with_interval_mono:
  assumes "fct1 ≤ fct2" and "s1 ≤ s2"
  shows
    "refine_sign_with_interval fct1 s1 ≤
     refine_sign_with_interval fct2 s2"
  unfolding refine_sign_with_interval_def
  by (rule inf_mono;
      use assms interval_sign_fact_mono in blast)

lemma refine_ivl_with_interval_mono:
  assumes "fct1 ≤ fct2" and "i1 ≤ i2"
  shows
    "refine_ivl_with_interval fct1 i1 ≤
     refine_ivl_with_interval fct2 i2"
  unfolding refine_ivl_with_interval_def
  by (rule intersect_ivl_mono; fact)

lemma refine_parity_with_interval_mono:
  assumes "fct1 ≤ fct2" and "p1 ≤ p2"
  shows
    "refine_parity_with_interval fct1 p1 ≤
     refine_parity_with_interval fct2 p2"
  unfolding refine_parity_with_interval_def
  by (rule inf_mono;
      use assms interval_parity_fact_mono in blast)

lemma gamma_interval_fact_of_int_dom:
  "gamma_ivl (interval_fact_of_int_dom d) =
   gamma_sign (int_sign d) ∩ gamma_ivl (int_ivl d)"
  unfolding interval_fact_of_int_dom_def interval_fact_of_ivl_def
  by (simp only: gamma_intersect_ivl_exact gamma_interval_fact_of_sign)

lemma interval_fact_of_int_dom_mono:
  assumes "d ≤ e"
  shows "interval_fact_of_int_dom d ≤ interval_fact_of_int_dom e"
proof -
  from assms have components:
```

```
    "int_sign d ≤ int_sign e"
    "int_ivl d ≤ int_ivl e"
    by (simp_all add: less_eq_int_dom_ext_def)
  show ?thesis
    unfolding interval_fact_of_int_dom_def interval_fact_of_ivl_def
    by (rule intersect_ivl_mono)
       (use components interval_fact_of_sign_mono in auto)
qed

lemma refine_interval_exact:
  "gamma_int_dom (refine_interval d) = gamma_int_dom d"
proof -
  let ?fct = "interval_fact_of_int_dom d"
  have fct:
    "gamma_ivl ?fct =
     gamma_sign (int_sign d) ∩ gamma_ivl (int_ivl d)"
    by (rule gamma_interval_fact_of_int_dom)
  have sign_cover:
    "gamma_ivl ?fct ⊆
     gamma_sign (interval_sign_fact ?fct)"
    by (rule interval_sign_fact_sound)
  have parity_cover:
    "gamma_ivl ?fct ⊆
     gamma_parity (interval_parity_fact ?fct)"
    by (rule interval_parity_fact_sound)
  show ?thesis
    unfolding refine_interval_def gamma_int_dom_def Let_def
    using fct sign_cover parity_cover
    by (auto simp: gamma_refine_ivl_with_interval)
qed

lemma refine_interval_reductive:
  "refine_interval d ≤ d"
  unfolding refine_interval_def Let_def
  by (simp add: less_eq_int_dom_ext_def
        refine_sign_with_interval_le
        refine_ivl_with_interval_le
        refine_parity_with_interval_le)

lemma refine_interval_mono:
  "mono refine_interval"
proof (rule monoI)
  fix d e :: int_dom
  assume de: "d ≤ e"
  from de have components:
    "int_sign d ≤ int_sign e"
    "int_ivl d ≤ int_ivl e"
    "int_parity d ≤ int_parity e"
    "int_congruence d ≤ int_congruence e"
    by (simp_all add: less_eq_int_dom_ext_def)
  have fct_le:
    "interval_fact_of_int_dom d ≤ interval_fact_of_int_dom e"
    by (rule interval_fact_of_int_dom_mono[OF de])
  show "refine_interval d ≤ refine_interval e"
    unfolding refine_interval_def Let_def
    by (simp add: less_eq_int_dom_ext_def
          refine_sign_with_interval_mono[OF fct_le components(1)]
          refine_ivl_with_interval_mono[OF fct_le components(2)]
          refine_parity_with_interval_mono[OF fct_le components(3)]
          components(4))
qed

lemma int_reduction_step_refine_interval:
  "int_reduction_step refine_interval"
  unfolding int_reduction_step_def
  using refine_interval_exact refine_interval_reductive refine_interval_mono
  by blast
```

Congruence-fact fan-out

`refine_congruence` first sharpens the congruence by the parity, then uses the result to round the interval bounds to the nearest class members (`congruence_lower_bound`, `congruence_upper_bound`) and to sharpen the parity. Like the interval fan-out it is an `int_reduction_step`.

```
definition congruence_fact_of_congruence ::
  "congruence => congruence_fact" where
```

```
  "congruence_fact_of_congruence c = c"

definition congruence_fact_of_int_dom ::
  "int_dom => congruence_fact" where
  "congruence_fact_of_int_dom d =
     restrict_congruence_by_parity
       (int_parity d)
       (congruence_fact_of_congruence (int_congruence d))"

fun congruence_lower_bound :: "int => int => eint => eint" where
  "congruence_lower_bound c m MinInf = MinInf"
| "congruence_lower_bound c m (Fin l) =
     Fin (l + ((c - l) mod m))"
| "congruence_lower_bound c m PlusInf = PlusInf"

fun congruence_upper_bound :: "int => int => eint => eint" where
  "congruence_upper_bound c m MinInf = MinInf"
| "congruence_upper_bound c m (Fin u) =
     Fin (u - ((u - c) mod m))"
| "congruence_upper_bound c m PlusInf = PlusInf"

fun refine_ivl_with_congruence_rep ::
  "congruence_rep => ivl => ivl" where
  "refine_ivl_with_congruence_rep None i = bot"
| "refine_ivl_with_congruence_rep (Some (c, m)) (Ivl l u) =
     (if m = 0 then
        intersect_ivl (Ivl (Fin c) (Fin c)) (Ivl l u)
      else
        intersect_ivl
          (mk_ivl
            (congruence_lower_bound c m l)
            (congruence_upper_bound c m u))
          (Ivl l u))"

definition refine_ivl_with_congruence ::
  "congruence_fact => ivl => ivl" where
  "refine_ivl_with_congruence fct i =
     refine_ivl_with_congruence_rep (Rep_congruence fct) i"

fun parity_fact_of_congruence_rep ::
  "congruence_rep => parity" where
  "parity_fact_of_congruence_rep None = PBot"
| "parity_fact_of_congruence_rep (Some (c, m)) =
     (if m = 0 ∨ even m then
        if even c then PEven else POdd
      else PTop)"

definition parity_fact_of_congruence ::
  "congruence_fact => parity" where
  "parity_fact_of_congruence fct =
     parity_fact_of_congruence_rep (Rep_congruence fct)"

definition refine_parity_with_congruence ::
  "congruence_fact => parity => parity" where
  "refine_parity_with_congruence fct p =
     parity_fact_of_congruence fct ⊓ p"

definition refine_congruence_with_congruence ::
  "congruence_fact => congruence => congruence" where
  "refine_congruence_with_congruence fct c = fct"

lemma gamma_congruence_fact_of_int_dom [simp]:
  "gamma_congruence (congruence_fact_of_int_dom d) =
   gamma_parity (int_parity d) ∩
   gamma_congruence (int_congruence d)"
  unfolding congruence_fact_of_int_dom_def
    congruence_fact_of_congruence_def
  by simp

lemma congruence_fact_of_int_dom_mono:
  assumes de: "d <= e"
  shows
    "congruence_fact_of_int_dom d <=
     congruence_fact_of_int_dom e"
proof -
  from de have parity:
```

```
    "int_parity d <= int_parity e"
    by (simp add: less_eq_int_dom_ext_def)
  from de have congruence:
    "int_congruence d <= int_congruence e"
    by (simp add: less_eq_int_dom_ext_def)
  have parity_gamma:
    "gamma_parity (int_parity d) ⊆
     gamma_parity (int_parity e)"
    using parity
    unfolding less_eq_parity_def
    by (rule gamma_parity_mono)
  have congruence_gamma:
    "gamma_congruence (int_congruence d) ⊆
     gamma_congruence (int_congruence e)"
    using congruence
    unfolding less_eq_congruence_iff_gamma .
  show ?thesis
    unfolding less_eq_congruence_iff_gamma
    using parity_gamma congruence_gamma by auto
qed

lemma congruence_fact_of_int_dom_le_congruence:
  "congruence_fact_of_int_dom d <= int_congruence d"
  unfolding less_eq_congruence_iff_gamma
  by auto

lemma congruence_lower_bound_le:
  fixes c m l n :: int
  assumes modulus: "0 < m"
      and lower: "l <= n"
      and congruent: "m dvd n - c"
  shows "l + ((c - l) mod m) <= n"
proof -
  have residues:
    "(c - l) mod m = (n - l) mod m"
    using congruent by (simp add: mod_eq_dvd_iff dvd_diff_commute)
  have nonnegative: "0 <= n - l"
    using lower by simp
  have remainder: "(n - l) mod m <= n - l"
    by (rule zmod_le_nonneg_dividend[OF nonnegative])
  from residues remainder show ?thesis by linarith
qed

lemma le_congruence_upper_bound:
  fixes c m n u :: int
  assumes modulus: "0 < m"
      and upper: "n <= u"
      and congruent: "m dvd n - c"
  shows "n <= u - ((u - c) mod m)"
proof -
  have residues:
    "(u - c) mod m = (u - n) mod m"
    using congruent by (simp add: mod_eq_dvd_iff)
  have nonnegative: "0 <= u - n"
    using upper by simp
  have remainder: "(u - n) mod m <= u - n"
    by (rule zmod_le_nonneg_dividend[OF nonnegative])
  from residues remainder show ?thesis by linarith
qed

lemma refine_ivl_with_congruence_cover:
  "gamma_ivl i ∩ gamma_congruence fct ⊆
   gamma_ivl (refine_ivl_with_congruence fct i)"
proof
  fix n
  assume member:
    "n ∈ gamma_ivl i ∩ gamma_congruence fct"
  obtain l u where i: "i = Ivl l u"
    by (rule ivl_exhaustE)
  have normalized:
    "normalized_congruence_rep (Rep_congruence fct)"
    by simp
  show
    "n ∈ gamma_ivl (refine_ivl_with_congruence fct i)"
  proof (cases "Rep_congruence fct")
    case None
```

```
    with member show ?thesis
      unfolding gamma_congruence_def by simp
  next
    case (Some cm)
    obtain c m where cm: "cm = (c, m)"
      by (cases cm)
    have rep: "Rep_congruence fct = Some (c, m)"
      using Some cm by simp
    have normalized':
      "normalized_congruence_rep (Some (c, m))"
      using normalized unfolding rep .
    from member have in_i: "n ∈ gamma_ivl i"
      by simp
    from member have congruent: "m dvd n - c"
      unfolding gamma_congruence_def rep by simp
    show ?thesis
    proof (cases "m = 0")
      case True
      from congruent True have n: "n = c" by simp
      have in_point:
        "n ∈ gamma_ivl (Ivl (Fin c) (Fin c))"
        using n by simp
      have in_i': "n ∈ gamma_ivl (Ivl l u)"
        using in_i unfolding i .
      have refined:
        "refine_ivl_with_congruence fct i =
         intersect_ivl (Ivl (Fin c) (Fin c)) (Ivl l u)"
        unfolding refine_ivl_with_congruence_def rep i
        using True by simp
      show ?thesis
        unfolding refined
        by (rule intersect_ivl_gamma[OF in_point in_i'])
    next
      case False
      have modulus: "0 < m"
        by (rule normalized_congruence_rep_nonzero[
              OF normalized' False])
      have lower:
        "congruence_lower_bound c m l <= Fin n"
        using in_i congruent modulus
        unfolding i
        by (cases l)
           (auto intro: congruence_lower_bound_le)
      have upper:
        "Fin n <= congruence_upper_bound c m u"
        using in_i congruent modulus
        unfolding i
        by (cases u)
           (auto intro: le_congruence_upper_bound)
      have in_hull:
        "n ∈ gamma_ivl
          (mk_ivl
            (congruence_lower_bound c m l)
            (congruence_upper_bound c m u))"
        using lower upper
        by (simp only: gamma_mk_ivl gamma_ivl.simps
            mem_Collect_eq)
      have in_i': "n ∈ gamma_ivl (Ivl l u)"
        using in_i unfolding i .
      have refined:
        "refine_ivl_with_congruence fct i =
         intersect_ivl
           (mk_ivl
             (congruence_lower_bound c m l)
             (congruence_upper_bound c m u))
           (Ivl l u)"
        unfolding refine_ivl_with_congruence_def rep i
        using False by simp
      show ?thesis
        unfolding refined
        by (rule intersect_ivl_gamma[OF in_hull in_i'])
    qed
  qed
qed

lemma refine_ivl_with_congruence_le:
```

```
  "refine_ivl_with_congruence fct i <= i"
proof (cases "Rep_congruence fct")
  case None
  then show ?thesis
    unfolding refine_ivl_with_congruence_def by simp
next
  case (Some cm)
  obtain c m where cm: "cm = (c, m)"
    by (cases cm)
  have rep: "Rep_congruence fct = Some (c, m)"
    using Some cm by simp
  obtain l u where i: "i = Ivl l u"
    by (rule ivl_exhaustE)
  show ?thesis
  proof (cases "m = 0")
    case True
    have refined:
      "refine_ivl_with_congruence fct i =
       intersect_ivl (Ivl (Fin c) (Fin c)) i"
      unfolding refine_ivl_with_congruence_def rep i
      using True by simp
    show ?thesis
      unfolding refined by (rule intersect_ivl_le2)
  next
    case False
    have refined:
      "refine_ivl_with_congruence fct i =
       intersect_ivl
         (mk_ivl
           (congruence_lower_bound c m l)
           (congruence_upper_bound c m u))
         i"
      unfolding refine_ivl_with_congruence_def rep i
      using False by simp
    show ?thesis
      unfolding refined by (rule intersect_ivl_le2)
  qed
qed

lemma normalize_refine_ivl_with_congruence [simp]:
  "normalize_ivl (refine_ivl_with_congruence fct i) =
   refine_ivl_with_congruence fct i"
  unfolding refine_ivl_with_congruence_def
  by (cases "Rep_congruence fct"; cases i)
     (auto simp only:
        refine_ivl_with_congruence_rep.simps
        normalize_ivl_bot
        normalize_ivl_intersect_ivl
        split: prod.splits if_splits)

lemma congruence_lower_bound_congruent:
  fixes c m l :: int
  shows
    "m dvd
     (l + ((c - l) mod m)) - c"
proof -
  have base:
    "m dvd (c - l) - ((c - l) mod m)"
    by (rule dvd_minus_mod)
  have negated:
    "m dvd - ((c - l) - ((c - l) mod m))"
    by (subst dvd_minus_iff) (rule base)
  have equal:
    "- ((c - l) - ((c - l) mod m)) =
     (l + ((c - l) mod m)) - c"
    by linarith
  from negated show ?thesis
    unfolding equal .
qed

lemma congruence_upper_bound_congruent:
  fixes c m u :: int
  shows
    "m dvd
     (u - ((u - c) mod m)) - c"
proof -
```

```
  have base:
    "m dvd (u - c) - ((u - c) mod m)"
    by (rule dvd_minus_mod)
  have equal:
    "(u - c) - ((u - c) mod m) =
     (u - ((u - c) mod m)) - c"
    by linarith
  from base show ?thesis
    unfolding equal .
qed

lemma congruence_lower_bound_mono:
  fixes c1 m1 c2 m2 :: int
    and l1 l2 :: eint
  assumes m1: "0 < m1"
      and m2: "0 < m2"
      and moduli: "m2 dvd m1"
      and residues: "m2 dvd c1 - c2"
      and bounds: "l2 <= l1"
  shows
    "congruence_lower_bound c2 m2 l2 <=
     congruence_lower_bound c1 m1 l1"
proof (cases l1)
  case MinInf
  then show ?thesis
    using bounds by (cases l2) simp_all
next
  case (Fin a)
  have l1: "l1 = Fin a"
    by (rule Fin)
  show ?thesis
  proof (cases l2)
    case MinInf
    then show ?thesis by simp
  next
    case (Fin b)
    have ba: "b <= a"
      using bounds l1 Fin by simp
    let ?n = "a + ((c1 - a) mod m1)"
    have a_le_n: "a <= ?n"
      using pos_mod_sign[OF m1, of "c1 - a"]
      by linarith
    have congruent1: "m1 dvd ?n - c1"
      by (rule congruence_lower_bound_congruent)
    have congruent2a: "m2 dvd ?n - c1"
      by (rule dvd_trans[OF moduli congruent1])
    have congruent2: "m2 dvd ?n - c2"
    proof -
      have combined:
        "m2 dvd (?n - c1) + (c1 - c2)"
        by (rule dvd_add[OF congruent2a residues])
      have equal:
        "(?n - c1) + (c1 - c2) = ?n - c2"
        by linarith
      from combined show ?thesis
        unfolding equal .
    qed
    have b_le_n: "b <= ?n"
      using ba a_le_n by linarith
    have
      "b + ((c2 - b) mod m2) <= ?n"
      by (rule congruence_lower_bound_le[
            OF m2 b_le_n congruent2])
    with l1 Fin show ?thesis by simp
  next
    case PlusInf
    with bounds l1 show ?thesis by simp
  qed
next
  case PlusInf
  then show ?thesis by simp
qed

lemma congruence_upper_bound_mono:
  fixes c1 m1 c2 m2 :: int
    and u1 u2 :: eint
```

```
  assumes m1: "0 < m1"
      and m2: "0 < m2"
      and moduli: "m2 dvd m1"
      and residues: "m2 dvd c1 - c2"
      and bounds: "u1 <= u2"
  shows
    "congruence_upper_bound c1 m1 u1 <=
     congruence_upper_bound c2 m2 u2"
proof (cases u1)
  case MinInf
  then show ?thesis by simp
next
  case (Fin a)
  have u1: "u1 = Fin a"
    by (rule Fin)
  show ?thesis
  proof (cases u2)
    case MinInf
    with bounds u1 show ?thesis by simp
  next
    case (Fin b)
    have ab: "a <= b"
      using bounds u1 Fin by simp
    let ?n = "a - ((a - c1) mod m1)"
    have n_le_a: "?n <= a"
      using pos_mod_sign[OF m1, of "a - c1"]
      by linarith
    have congruent1: "m1 dvd ?n - c1"
      by (rule congruence_upper_bound_congruent)
    have congruent2a: "m2 dvd ?n - c1"
      by (rule dvd_trans[OF moduli congruent1])
    have congruent2: "m2 dvd ?n - c2"
    proof -
      have combined:
        "m2 dvd (?n - c1) + (c1 - c2)"
        by (rule dvd_add[OF congruent2a residues])
      have equal:
        "(?n - c1) + (c1 - c2) = ?n - c2"
        by linarith
      from combined show ?thesis
        unfolding equal .
    qed
    have n_le_b: "?n <= b"
      using n_le_a ab by linarith
    have
      "?n <= b - ((b - c2) mod m2)"
      by (rule le_congruence_upper_bound[
            OF m2 n_le_b congruent2])
    with u1 Fin show ?thesis by simp
  next
    case PlusInf
    then show ?thesis by simp
  qed
next
  case PlusInf
  then show ?thesis
    using bounds by (cases u2) simp_all
qed

lemma intersect_ivl_eq_bot_if_disjoint:
  assumes
    "gamma_ivl a ∩ gamma_ivl b = {}"
  shows "intersect_ivl a b = bot"
proof -
  have empty:
    "gamma_ivl (intersect_ivl a b) = {}"
    using assms gamma_intersect_ivl_exact by simp
  have bottom:
    "is_bottom_ivl (intersect_ivl a b)"
    using empty is_bottom_ivl_correct by blast
  have normalized:
    "normalize_ivl (intersect_ivl a b) =
     intersect_ivl a b"
    by simp
  show ?thesis
    using normalize_ivl_bot_case[OF bottom] normalized
```

```
    by simp
qed

lemma refine_ivl_with_congruence_mono:
  assumes "fct1 <= fct2" "i1 <= i2"
  shows
    "refine_ivl_with_congruence fct1 i1 <=
     refine_ivl_with_congruence fct2 i2"
proof -
  have rep_le:
    "congruence_le_rep
       (Rep_congruence fct1)
       (Rep_congruence fct2)"
    using assms(1)
    unfolding less_eq_congruence_def congruence_le_def .
  have normalized1:
    "normalized_congruence_rep (Rep_congruence fct1)"
    by simp
  have normalized2:
    "normalized_congruence_rep (Rep_congruence fct2)"
    by simp
  obtain l1 u1 where i1: "i1 = Ivl l1 u1"
    by (rule ivl_exhaustE)
  obtain l2 u2 where i2: "i2 = Ivl l2 u2"
    by (rule ivl_exhaustE)
  have bounds: "l2 <= l1" "u1 <= u2"
    using assms(2)
    unfolding i1 i2 less_eq_ivl_def
    by simp_all
  have singleton_le_iff:
    "Ivl (Fin n) (Fin n) <= i ⟷
     n ∈ gamma_ivl i"
    for n i
    by (cases i) (simp add: less_eq_ivl_def)
  show ?thesis
  proof (cases "Rep_congruence fct1")
    case None
    then show ?thesis
      unfolding refine_ivl_with_congruence_def by simp
  next
    case (Some cm1)
    obtain c1 m1 where cm1: "cm1 = (c1, m1)"
      by (cases cm1)
    have rep1: "Rep_congruence fct1 = Some (c1, m1)"
      using Some cm1 by simp
    show ?thesis
    proof (cases "Rep_congruence fct2")
      case None
      with rep_le rep1 show ?thesis by simp
    next
      case (Some cm2)
      obtain c2 m2 where cm2: "cm2 = (c2, m2)"
        by (cases cm2)
      have rep2: "Rep_congruence fct2 = Some (c2, m2)"
        using Some cm2 by simp
      have relation:
        "m2 dvd m1" "m2 dvd c1 - c2"
        using rep_le unfolding rep1 rep2
        by simp_all
      have normalized1':
        "normalized_congruence_rep (Some (c1, m1))"
        using normalized1 unfolding rep1 .
      have normalized2':
        "normalized_congruence_rep (Some (c2, m2))"
        using normalized2 unfolding rep2 .
      show ?thesis
      proof (cases "m1 = 0")
        case True
        show ?thesis
        proof (cases "m2 = 0")
          case True2: True
          have residues: "c1 = c2"
            using relation(2) True2 by simp
          have point_le:
            "Ivl (Fin c1) (Fin c1) <=
             Ivl (Fin c2) (Fin c2)"
```

```
      using residues by simp
    have refined1:
      "refine_ivl_with_congruence fct1 i1 =
       intersect_ivl (Ivl (Fin c1) (Fin c1)) i1"
      unfolding refine_ivl_with_congruence_def rep1 i1
      using True by simp
    have refined2:
      "refine_ivl_with_congruence fct2 i2 =
       intersect_ivl (Ivl (Fin c2) (Fin c2)) i2"
      unfolding refine_ivl_with_congruence_def rep2 i2
      using True2 by simp
    show ?thesis
      unfolding refined1 refined2
      by (rule intersect_ivl_mono[OF point_le assms(2)])
  next
    case False
    have modulus2: "0 < m2"
      by (rule normalized_congruence_rep_nonzero(3)
            [OF normalized2' False])
    show ?thesis
    proof (cases "c1 ∈ gamma_ivl i1")
      case False
      have empty:
        "gamma_ivl (Ivl (Fin c1) (Fin c1)) ∩
         gamma_ivl i1 = {}"
        using False by auto
      have source_refined:
        "refine_ivl_with_congruence fct1 i1 =
         intersect_ivl
           (Ivl (Fin c1) (Fin c1)) i1"
        unfolding refine_ivl_with_congruence_def rep1 i1
        using True by simp
      have source_bottom:
        "refine_ivl_with_congruence fct1 i1 = bot"
        unfolding source_refined
        by (rule intersect_ivl_eq_bot_if_disjoint[OF empty])
      then show ?thesis by simp
    next
      case TrueMember: True
      have c1_i2: "c1 ∈ gamma_ivl i2"
        by (rule subsetD[OF gamma_ivl_mono[OF assms(2)]
              TrueMember])
      have c1_fct2: "c1 ∈ gamma_congruence fct2"
        unfolding gamma_congruence_def rep2
        using relation(2) by simp
      have c1_target:
        "c1 ∈ gamma_ivl
          (refine_ivl_with_congruence fct2 i2)"
        by (rule subsetD[
              OF refine_ivl_with_congruence_cover])
           (use c1_i2 c1_fct2 in auto)
      have point_le:
        "Ivl (Fin c1) (Fin c1) <=
         refine_ivl_with_congruence fct2 i2"
        using c1_target singleton_le_iff by blast
      have source_refined:
        "refine_ivl_with_congruence fct1 i1 =
         intersect_ivl
           (Ivl (Fin c1) (Fin c1)) i1"
        unfolding refine_ivl_with_congruence_def rep1 i1
        using True by simp
      have source_le:
        "refine_ivl_with_congruence fct1 i1 <=
         Ivl (Fin c1) (Fin c1)"
        unfolding source_refined
        by (rule intersect_ivl_le1)
      show ?thesis
        by (rule order_trans[OF source_le point_le])
    qed
  qed
next
  case False
  have modulus1: "0 < m1"
    by (rule normalized_congruence_rep_nonzero(3)
          [OF normalized1' False])
  have m2_nonzero: "m2 ≠ 0"
```

```
        using relation(1) modulus1 by auto
      have modulus2: "0 < m2"
        by (rule normalized_congruence_rep_nonzero(3)
              [OF normalized2' m2_nonzero])
      have lower:
        "congruence_lower_bound c2 m2 l2 <=
         congruence_lower_bound c1 m1 l1"
        by (rule congruence_lower_bound_mono[
              OF modulus1 modulus2 relation(1)
                relation(2) bounds(1)])
      have upper:
        "congruence_upper_bound c1 m1 u1 <=
         congruence_upper_bound c2 m2 u2"
        by (rule congruence_upper_bound_mono[
              OF modulus1 modulus2 relation(1)
                relation(2) bounds(2)])
      have raw_hull:
        "Ivl
           (congruence_lower_bound c1 m1 l1)
           (congruence_upper_bound c1 m1 u1) <=
         Ivl
           (congruence_lower_bound c2 m2 l2)
           (congruence_upper_bound c2 m2 u2)"
        using lower upper
        by (simp add: less_eq_ivl_def)
      have hull:
        "mk_ivl
           (congruence_lower_bound c1 m1 l1)
           (congruence_upper_bound c1 m1 u1) <=
         mk_ivl
           (congruence_lower_bound c2 m2 l2)
           (congruence_upper_bound c2 m2 u2)"
        by (rule mk_ivl_mono[OF raw_hull])
      have refined1:
        "refine_ivl_with_congruence fct1 i1 =
         intersect_ivl
           (mk_ivl
             (congruence_lower_bound c1 m1 l1)
             (congruence_upper_bound c1 m1 u1))
           i1"
        unfolding refine_ivl_with_congruence_def rep1 i1
        using False by simp
      have refined2:
        "refine_ivl_with_congruence fct2 i2 =
         intersect_ivl
           (mk_ivl
             (congruence_lower_bound c2 m2 l2)
             (congruence_upper_bound c2 m2 u2))
           i2"
        unfolding refine_ivl_with_congruence_def rep2 i2
        using m2_nonzero by simp
      show ?thesis
        unfolding refined1 refined2
        by (rule intersect_ivl_mono[OF hull assms(2)])
    qed
  qed
 qed
qed

lemma parity_fact_of_congruence_sound:
  "gamma_congruence fct ⊆
   gamma_parity (parity_fact_of_congruence fct)"
proof
  fix n
  assume member: "n ∈ gamma_congruence fct"
  show
    "n ∈ gamma_parity (parity_fact_of_congruence fct)"
  proof (cases "Rep_congruence fct")
    case None
    with member show ?thesis
      unfolding gamma_congruence_def by simp
  next
    case (Some cm)
    obtain c m where cm: "cm = (c, m)"
      by (cases cm)
    have rep: "Rep_congruence fct = Some (c, m)"
```

```
      using Some cm by simp
    have congruent: "m dvd n - c"
      using member
      unfolding gamma_congruence_def rep by simp
    show ?thesis
    proof (cases "m = 0 ∨ even m")
      case True
      have same_parity: "even n = even c"
      proof (cases "m = 0")
        case True
        with congruent have "n = c" by simp
        then show ?thesis by simp
      next
        case False
        with True have even_m: "even m" by simp
        have even_difference: "even (n - c)"
          using even_m congruent
          by (meson dvd_trans)
        then show ?thesis
          by (simp add: even_diff_iff)
      qed
      show ?thesis
        unfolding parity_fact_of_congruence_def rep
        using True same_parity
        by auto
    next
      case False
      then show ?thesis
        unfolding parity_fact_of_congruence_def rep
        by simp
    qed
  qed
qed

lemma parity_fact_of_congruence_mono:
  assumes "fct1 <= fct2"
  shows
    "parity_fact_of_congruence fct1 <=
     parity_fact_of_congruence fct2"
proof -
  have rep_le:
    "congruence_le_rep
       (Rep_congruence fct1)
       (Rep_congruence fct2)"
    using assms
    unfolding less_eq_congruence_def congruence_le_def .
  show ?thesis
  proof (cases "Rep_congruence fct1")
    case None
    then show ?thesis
      unfolding parity_fact_of_congruence_def
        less_eq_parity_def
      by simp
  next
    case (Some cm1)
    obtain c1 m1 where cm1: "cm1 = (c1, m1)"
      by (cases cm1)
    have rep1: "Rep_congruence fct1 = Some (c1, m1)"
      using Some cm1 by simp
    show ?thesis
    proof (cases "Rep_congruence fct2")
      case None
      with rep_le rep1 show ?thesis by simp
    next
      case (Some cm2)
      obtain c2 m2 where cm2: "cm2 = (c2, m2)"
        by (cases cm2)
      have rep2: "Rep_congruence fct2 = Some (c2, m2)"
        using Some cm2 by simp
      have relation:
        "m2 dvd m1" "m2 dvd c1 - c2"
        using rep_le unfolding rep1 rep2
        by simp_all
      show ?thesis
      proof (cases "m2 = 0 ∨ even m2")
        case False
```

```
        have target_top:
          "parity_fact_of_congruence fct2 = (top :: parity)"
          unfolding parity_fact_of_congruence_def rep2
            top_parity_def
          using False by simp
        show ?thesis
          unfolding target_top by simp
      next
        case True
        have source_precise: "m1 = 0 ∨ even m1"
        proof (cases "m2 = 0")
          case True
          with relation(1) have "m1 = 0" by simp
          then show ?thesis by simp
        next
          case False
          with True have even_m2: "even m2" by simp
          have "even m1"
            using even_m2 relation(1)
            by (meson dvd_trans)
          then show ?thesis by simp
        qed
        have same_parity: "even c1 = even c2"
        proof (cases "m2 = 0")
          case True
          with relation(2) have "c1 = c2" by simp
          then show ?thesis by simp
        next
          case False
          with True have even_m2: "even m2" by simp
          have even_difference: "even (c1 - c2)"
            using even_m2 relation(2)
            by (meson dvd_trans)
          then show ?thesis
            by (simp add: even_diff_iff)
        qed
        have equal_facts:
          "parity_fact_of_congruence fct1 =
           parity_fact_of_congruence fct2"
          unfolding parity_fact_of_congruence_def rep1 rep2
          using True source_precise same_parity
          by auto
        then show ?thesis by simp
      qed
    qed
  qed
qed

lemma gamma_refine_parity_with_congruence [simp]:
  "gamma_parity (refine_parity_with_congruence fct p) =
   gamma_parity (parity_fact_of_congruence fct) ∩
   gamma_parity p"
  by (simp add: refine_parity_with_congruence_def)

lemma refine_parity_with_congruence_le:
  "refine_parity_with_congruence fct p <= p"
  unfolding refine_parity_with_congruence_def
  by (rule inf_le2)

lemma refine_parity_with_congruence_mono:
  assumes "fct1 <= fct2" "p1 <= p2"
  shows
    "refine_parity_with_congruence fct1 p1 <=
     refine_parity_with_congruence fct2 p2"
  unfolding refine_parity_with_congruence_def
  by (rule inf_mono)
     (use assms parity_fact_of_congruence_mono in auto)

lemma refine_congruence_with_congruence_le:
  assumes "fct <= c"
  shows "refine_congruence_with_congruence fct c <= c"
  using assms
  unfolding refine_congruence_with_congruence_def .

lemma refine_congruence_with_congruence_mono:
  assumes "fct1 <= fct2"
```

```
  shows
    "refine_congruence_with_congruence fct1 c1 <=
     refine_congruence_with_congruence fct2 c2"
  using assms
  unfolding refine_congruence_with_congruence_def .

definition refine_congruence :: "int_dom => int_dom" where
  "refine_congruence d =
     (let fct = congruence_fact_of_int_dom d
      in d⦇
           int_ivl :=
             refine_ivl_with_congruence fct (int_ivl d),
           int_parity :=
             refine_parity_with_congruence fct (int_parity d),
           int_congruence :=
             refine_congruence_with_congruence
               fct (int_congruence d)
         ⦈)"

lemma refine_congruence_exact:
  "gamma_int_dom (refine_congruence d) = gamma_int_dom d"
proof -
  let ?fct = "congruence_fact_of_int_dom d"
  have ivl_cover:
    "gamma_ivl (int_ivl d) ∩
     gamma_congruence ?fct ⊆
     gamma_ivl
       (refine_ivl_with_congruence ?fct (int_ivl d))"
    by (rule refine_ivl_with_congruence_cover)
  have ivl_subset:
    "gamma_ivl
       (refine_ivl_with_congruence ?fct (int_ivl d))
     ⊆ gamma_ivl (int_ivl d)"
    by (rule gamma_ivl_mono)
       (rule refine_ivl_with_congruence_le)
  have parity_cover:
    "gamma_congruence ?fct ⊆
     gamma_parity
       (refine_parity_with_congruence
         ?fct (int_parity d))"
  proof -
    have fact_sound:
      "gamma_congruence ?fct ⊆
       gamma_parity (parity_fact_of_congruence ?fct)"
      by (rule parity_fact_of_congruence_sound)
    show ?thesis
      using fact_sound
      by auto
  qed
  show ?thesis
    unfolding refine_congruence_def gamma_int_dom_def
      Let_def refine_congruence_with_congruence_def
    using ivl_cover ivl_subset parity_cover
    by auto
qed

lemma refine_congruence_reductive:
  "refine_congruence d <= d"
  unfolding refine_congruence_def Let_def
  by (simp add: less_eq_int_dom_ext_def
        refine_ivl_with_congruence_le
        refine_parity_with_congruence_le
        refine_congruence_with_congruence_le
        congruence_fact_of_int_dom_le_congruence)

lemma refine_congruence_mono:
  "mono refine_congruence"
proof (rule monoI)
  fix d e :: int_dom
  assume de: "d <= e"
  from de have components:
    "int_sign d <= int_sign e"
    "int_ivl d <= int_ivl e"
    "int_parity d <= int_parity e"
    "int_congruence d <= int_congruence e"
    by (simp_all add: less_eq_int_dom_ext_def)
```

```
  have fct_le:
    "congruence_fact_of_int_dom d <=
     congruence_fact_of_int_dom e"
    by (rule congruence_fact_of_int_dom_mono[OF de])
  show "refine_congruence d <= refine_congruence e"
    unfolding refine_congruence_def Let_def
    by (simp add: less_eq_int_dom_ext_def
          components(1)
          refine_ivl_with_congruence_mono[
            OF fct_le components(2)]
          refine_parity_with_congruence_mono[
            OF fct_le components(3)]
          refine_congruence_with_congruence_mono[
            OF fct_le])
qed

lemma refine_congruence_reduction_step:
  "int_reduction_step refine_congruence"
  unfolding int_reduction_step_def
  using refine_congruence_exact
    refine_congruence_reductive
    refine_congruence_mono
  by blast

definition refinement_steps :: "(int_dom => int_dom) list" where
  "refinement_steps = [refine_interval, refine_congruence]"

definition refine_round :: "int_dom => int_dom" where
  "refine_round = apply_reduction_steps refinement_steps"

lemma refine_round_reduction_step:
  "int_reduction_step refine_round"
  unfolding refine_round_def
  by (rule int_reduction_steps)
     (simp add: refinement_steps_def
        int_reduction_step_refine_interval
        refine_congruence_reduction_step)

end
```

16.3 Int_Refinement_Control

```
theory Int_Refinement_Control
  imports Int_Refinement
begin
```

How often to run the exchange, and what a run costs

The laws in `Voblint_Analysis_Int.Int_Refinement` say what one component may tell another; they say nothing about when to ask. This theory settles that. `refine_round` is one full pass over the exchange; a `refine_mode` picks how many passes an analysis run takes — `Refine_Never` none, `Refine_Once` a single pass, `Refine_Fixpoint` passes until the value stops changing.

The fixpoint mode is the awkward one. Iteration is a `while_option` loop, so it returns `None` when it does not stabilize; `refine_fix` keeps the input in that branch to stay a total function, while the generated code keeps running the structural-equality loop. Every mode is proved reductive and concretization-preserving, but only `Refine_Never` and `Refine_Once` are monotone (the first is the identity, the second `refine_once_mono`), which is why callers that need monotonicity carry the `mode` $\neq$ `Refine_Fixpoint` side condition.

```
datatype refine_mode =
    Refine_Never
  | Refine_Once
  | Refine_Fixpoint

definition canonical_refine_step :: "int_dom => int_dom" where
  "canonical_refine_step d =
     (let d' = refine_round d
      in if is_empty d' then bot else d')"

definition refine_fix_option :: "int_dom => int_dom option" where
  "refine_fix_option d =
     while_option
       (λx. canonical_refine_step x ≠ x)
       canonical_refine_step
       d"
```

The option result is `None` exactly when structural iteration does not stabilize. The total logical wrapper returns the input in that branch; generated execution remains in the structural-equality loop.

```
definition refine_fix :: "int_dom => int_dom" where
  "refine_fix d =
     (case refine_fix_option d of
        Some r => r
      | None => d)"

definition refine :: "refine_mode => int_dom => int_dom" where
  "refine mode d =
     (case mode of
        Refine_Never => d
      | Refine_Once => refine_round d
      | Refine_Fixpoint => refine_fix d)"


lemma canonical_refine_step_exact:
  "gamma_int_dom (canonical_refine_step d) = gamma_int_dom d"
proof (cases "is_empty (refine_round d)")
  case True
  have round_empty:
    "gamma_int_dom (refine_round d) = {}"
    using True
    by (simp add: is_bottom_int_dom_correct)
  have input_empty: "gamma_int_dom d = {}"
    using round_empty
      int_reduction_step_exact[OF refine_round_reduction_step]
    by simp
  have bottom_empty:
    "gamma_int_dom (bot :: int_dom) = {}"
  proof -
    have "γ (bot :: int_dom) = {}"
      by (rule gamma_bot)
    then show ?thesis by simp
  qed
  show ?thesis
    unfolding canonical_refine_step_def Let_def
    using True input_empty bottom_empty by simp
```

```
next
  case False
  show ?thesis
    unfolding canonical_refine_step_def Let_def
    using False
      int_reduction_step_exact[OF refine_round_reduction_step]
    by simp
qed


lemma refine_fix_option_exact:
  assumes result: "refine_fix_option d = Some r"
  shows "gamma_int_dom r = gamma_int_dom d"
proof -
  have loop:
    "while_option
       (λx. canonical_refine_step x ≠ x)
       canonical_refine_step d = Some r"
    using result unfolding refine_fix_option_def .
  have invariant:
    "⋀x.
       gamma_int_dom x = gamma_int_dom d ⟹
       canonical_refine_step x ≠ x ⟹
       gamma_int_dom (canonical_refine_step x) =
       gamma_int_dom d"
    using canonical_refine_step_exact by simp
  show ?thesis
  proof (rule while_option_rule[
      where P="λx.
        gamma_int_dom x = gamma_int_dom d"
        and b="λx.
          canonical_refine_step x ≠ x"
        and c=canonical_refine_step
        and s=d
        and t=r])
    fix x
    assume "gamma_int_dom x = gamma_int_dom d"
      "canonical_refine_step x ≠ x"
    then show
      "gamma_int_dom (canonical_refine_step x) =
       gamma_int_dom d"
      using canonical_refine_step_exact by simp
  next
    show
      "while_option
        (λx. canonical_refine_step x ≠ x)
        canonical_refine_step d = Some r"
      by (rule loop)
  next
    show "gamma_int_dom d = gamma_int_dom d"
      by simp
  qed
qed

lemma refine_fix_option_stable:
  assumes result: "refine_fix_option d = Some r"
  shows "canonical_refine_step r = r"
proof -
  have loop:
    "while_option
       (λx. canonical_refine_step x ≠ x)
       canonical_refine_step d = Some r"
    using result unfolding refine_fix_option_def .
  have "¬ canonical_refine_step r ≠ r"
    by (rule while_option_stop[OF loop])
  then show ?thesis by simp
qed

lemma refine_round_bot [simp]:
  "refine_round (bot :: int_dom) = bot"
proof (rule antisym)
  show "refine_round (bot :: int_dom) <= bot"
    by (rule int_reduction_step_reductive[
          OF refine_round_reduction_step])
  show "bot <= refine_round (bot :: int_dom)"
    by simp
```

```
qed

lemma refine_fix_option_round_stable:
  assumes result: "refine_fix_option d = Some r"
  shows "refine_round r = r"
proof -
  have stable: "canonical_refine_step r = r"
    by (rule refine_fix_option_stable[OF result])
  show ?thesis
  proof (cases "is_empty (refine_round r)")
    case True
    have "r = bot"
      using stable True
      unfolding canonical_refine_step_def Let_def
      by simp
    then show ?thesis by simp
  next
    case False
    with stable show ?thesis
      unfolding canonical_refine_step_def Let_def
      by simp
  qed
qed


lemma refine_fix_exact [simp]:
  "gamma_int_dom (refine_fix d) = gamma_int_dom d"
proof (cases "refine_fix_option d")
  case None
  then show ?thesis
    unfolding refine_fix_def by simp
next
  case (Some r)
  have "gamma_int_dom r = gamma_int_dom d"
    by (rule refine_fix_option_exact[OF Some])
  with Some show ?thesis
    unfolding refine_fix_def by simp
qed

lemma refine_never [simp]:
  "refine Refine_Never d = d"
  by (simp add: refine_def)

lemma refine_once [simp]:
  "refine Refine_Once d = refine_round d"
  by (simp add: refine_def)

lemma refine_fixpoint [simp]:
  "refine Refine_Fixpoint d = refine_fix d"
  by (simp add: refine_def)

lemma refine_once_reduction_step:
  "int_reduction_step (refine Refine_Once)"
proof -
  have "refine Refine_Once = refine_round"
    by (rule ext) simp
  then show ?thesis
    by (simp add: refine_round_reduction_step)
qed

lemma refine_once_reductive:
  "refine Refine_Once d <= d"
  by (rule int_reduction_step_reductive[
        OF refine_once_reduction_step])

lemma refine_once_mono:
  "mono (refine Refine_Once)"
  by (rule int_reduction_step_mono[
        OF refine_once_reduction_step])

lemma refine_exact:
  "gamma_int_dom (refine mode d) = gamma_int_dom d"
proof (cases mode)
  case Refine_Never
  then show ?thesis by simp
next
```

```
  case Refine_Once
  then show ?thesis
    using int_reduction_step_exact[OF refine_round_reduction_step]
    by simp
next
  case Refine_Fixpoint
  then show ?thesis by simp
qed

lemma canonical_refine_step_reductive:
  "canonical_refine_step d <= d"
proof (cases "is_empty (refine_round d)")
  case True
  then show ?thesis
    unfolding canonical_refine_step_def Let_def
    by simp
next
  case False
  then show ?thesis
    unfolding canonical_refine_step_def Let_def
    using int_reduction_step_reductive[OF refine_round_reduction_step]
    by simp
qed

lemma refine_fix_option_reductive:
  assumes result: "refine_fix_option d = Some r"
  shows "r <= d"
proof -
  have loop:
    "while_option
       (λx. canonical_refine_step x ≠ x)
       canonical_refine_step d = Some r"
    using result unfolding refine_fix_option_def .
  show ?thesis
  proof (rule while_option_rule[
      where P="λx. x <= d"
        and b="λx. canonical_refine_step x ≠ x"
        and c=canonical_refine_step
        and s=d
        and t=r])
    fix x
    assume "x <= d" "canonical_refine_step x ≠ x"
    then show "canonical_refine_step x <= d"
      using canonical_refine_step_reductive[of x] by simp
  next
    show
      "while_option
        (λx. canonical_refine_step x ≠ x)
        canonical_refine_step d = Some r"
      by (rule loop)
  next
    show "d <= d" by simp
  qed
qed

lemma refine_fix_reductive [simp]:
  "refine_fix d <= d"
proof (cases "refine_fix_option d")
  case None
  then show ?thesis
    unfolding refine_fix_def by simp
next
  case (Some r)
  have "r <= d"
    by (rule refine_fix_option_reductive[OF Some])
  with Some show ?thesis
    unfolding refine_fix_def by simp
qed

lemma refine_reductive:
  "refine mode d <= d"
proof (cases mode)
  case Refine_Never
  then show ?thesis by simp
next
  case Refine_Once
```

```
  then show ?thesis
    using refine_once_reductive by simp
next
  case Refine_Fixpoint
  then show ?thesis by simp
qed
```

Witnesses

Concrete values for the claims above. `progressive_refinement_input` is the raw sum of an odd value in `[-1,-1]` and one in `[0,1]`: its components together denote only `0`, but one round still leaves Sign at `SNonPos`, so a further round makes progress and `Refine_Once` differs from `Refine_Fixpoint`.

```
definition progressive_refinement_input :: int_dom where
  "progressive_refinement_input =
     int_dom_sipc STop (Ivl (Fin (-1)) (Fin 0)) PEven (top :: congruence)"

lemma refinement_round_is_progressive:
  "refine_round progressive_refinement_input =
     int_dom_sipc SNonPos (Ivl (Fin 0) (Fin 0)) PEven (mk_congruence 0 2)"
  by eval

lemma refinement_once_not_fixpoint:
  "refine Refine_Once progressive_refinement_input
     ≠ refine Refine_Fixpoint progressive_refinement_input"
  by eval
```

Why `Voblint_Analysis_Int.Int_Warrowing` runs no refinement after `narrow`. `a` is a widened state (top); `b` a newer, more precise result whose components together denote only `0` while Sign has not caught up. Componentwise narrowing satisfies `b ≤ narrow a b ≤ a`; refining that result lets Sign derive `SNonPos` from the exact Interval bound, strictly below `b`'s `STop`, which breaks the solver's `narrow_ge` bracket.

```
lemma post_narrow_refinement_would_violate_narrow_ge:
  "let a = (top :: int_dom);
       b = int_dom_sipc STop (Ivl (Fin (-1)) (Fin 0)) PEven (top :: congruence)
   in b ≤ a ∧ ¬ (b ≤ refine Refine_Once (a △ b))"
  by eval

end
```

16.4 Int_Tuple_Kind

```
theory Int_Tuple_Kind
  imports
    "Voblint_Analysis_Interval.Interval_Kind" "Voblint_Analysis_Congruence.Congruence_Kind"
    "Voblint_Analysis_Int.Int_Refinement_Control"
begin
```

The reduced product of Interval and Congruence

Goblint's `IntDomTuple` (`int/intDomTuple.ml`) restricted to its Interval and Congruence components. A value denotes the integers both components admit. Its type parameter selects how the components refine each other, as Goblint's `ana.int.refinement` does.

```
class int_refinement =
  fixes int_mode :: "'a itself ⇒ refine_mode"

datatype int_never = Int_Never
datatype int_once = Int_Once
datatype int_fixpoint = Int_Fixpoint

instantiation int_never :: int_refinement
begin
definition "int_mode (_ :: int_never itself) = Refine_Never"
instance ..
end

instantiation int_once :: int_refinement
begin
definition "int_mode (_ :: int_once itself) = Refine_Once"
instance ..
end

instantiation int_fixpoint :: int_refinement
begin
definition "int_mode (_ :: int_fixpoint itself) = Refine_Fixpoint"
instance ..
end

datatype 'm int_tuple = IT (it_ivl: ivl) (it_cong: congruence)

definition gamma_it :: "'a int_tuple ⇒ int set" where
  "gamma_it a = gamma_ivl (it_ivl a) ∩ gamma_congruence (it_cong a)"

lemma gamma_it_IT [simp]: "gamma_it (IT i c) = gamma_ivl i ∩ gamma_congruence c"
  by (simp add: gamma_it_def)
```

Lattice

The order, join, bottom and top are taken componentwise.

```
instantiation int_tuple :: (type) "{order_top, bounded_semilattice_sup_bot}"
begin

definition less_eq_int_tuple :: "'a int_tuple ⇒ 'a int_tuple ⇒ bool" where
  "a ≤ b ⟷ it_ivl a ≤ it_ivl b ∧ it_cong a ≤ it_cong b"

definition less_int_tuple :: "'a int_tuple ⇒ 'a int_tuple ⇒ bool" where
  "a < b ⟷ a ≤ b ∧ ¬ b ≤ (a :: 'a int_tuple)"

definition sup_int_tuple :: "'a int_tuple ⇒ 'a int_tuple ⇒ 'a int_tuple" where
  "a ⊔ b = IT (it_ivl a ⊔ it_ivl b) (it_cong a ⊔ it_cong b)"

definition bot_int_tuple :: "'a int_tuple" where "bot = IT bot bot"

definition top_int_tuple :: "'a int_tuple" where "top = IT top top"

instance proof
  fix a b c :: "'a int_tuple"
  show "a < b ⟷ a ≤ b ∧ ¬ b ≤ a" by (rule less_int_tuple_def)
  show "a ≤ a" by (simp add: less_eq_int_tuple_def)
  show "a ≤ b ⟹ b ≤ c ⟹ a ≤ c"
    by (auto simp: less_eq_int_tuple_def intro: order.trans)
  show "a ≤ b ⟹ b ≤ a ⟹ a = b"
    by (cases a; cases b) (auto simp: less_eq_int_tuple_def)
  show "a ≤ a ⊔ b" "b ≤ a ⊔ b" by (simp_all add: less_eq_int_tuple_def sup_int_tuple_def)
```

```
  show "b ≤ a ⟹ c ≤ a ⟹ b ⊔ c ≤ a"
    by (simp add: less_eq_int_tuple_def sup_int_tuple_def)
  show "bot ≤ a" by (simp add: less_eq_int_tuple_def bot_int_tuple_def)
  show "a ≤ top" by (simp add: less_eq_int_tuple_def top_int_tuple_def)
qed

end

lemma IT_le_IT [simp]: "IT i c ≤ IT j d ⟷ i ≤ j ∧ c ≤ d"
  by (simp add: less_eq_int_tuple_def)
```

Exact emptiness

A class meets an interval when its least value above the lower bound, or its greatest value below the upper bound, lies inside. The test is exact, so emptiness of the product is decided exactly.

```
lemma cong_greatest_below:
  fixes m c hi :: int
  assumes m: "0 < m"
  defines "r ≡ hi - (hi - c) mod m"
  shows "r ≤ hi" and "m dvd r - c"
    and "m dvd n - c ⟹ n ≤ hi ⟹ n ≤ r"
proof -
  show "r ≤ hi" using m by (simp add: r_def)
  have "r - c = (hi - c) - (hi - c) mod m" by (simp add: r_def)
  then show "m dvd r - c" by (simp only: dvd_minus_mod)
  assume d: "m dvd n - c" and hi: "n ≤ hi"
  have "(hi - n) mod m = (hi - c) mod m"
  proof -
    have "m dvd (hi - n) - (hi - c)" using d dvd_minus_iff[of m "n - c"] by simp
    then show ?thesis by (simp add: mod_eq_dvd_iff)
  qed
  moreover have "(hi - n) mod m ≤ hi - n"
    using hi m by (simp add: zmod_le_nonneg_dividend)
  ultimately show "n ≤ r" by (simp add: r_def)
qed

fun cong_meets :: "int × int ⇒ eint ⇒ eint ⇒ bool" where
  "cong_meets (c, m) l u =
     (if m = 0 then l ≤ Fin c ∧ Fin c ≤ u
      else case (l, u) of
        (Fin x, Fin y) ⇒ x + (c - x) mod m ≤ y
      | (Fin x, PlusInf) ⇒ True
      | (MinInf, Fin y) ⇒ True
      | (MinInf, PlusInf) ⇒ True
      | _ ⇒ False)"

lemma cong_meets_correct:
  assumes "0 ≤ m"
  shows "cong_meets (c, m) l u ⟷ (∃n. m dvd n - c ∧ l ≤ Fin n ∧ Fin n ≤ u)"
proof (cases "m = 0")
  case True
  then show ?thesis by auto
next
  case False
  with assms have m: "0 < m" by simp
  show ?thesis
  proof (cases l; cases u)
    fix x y assume l: "l = Fin x" and u: "u = Fin y"
    note least = cong_least_above[OF m, where lo = x and c = c]
    show ?thesis
      using least False by (auto simp: l u) force
  next
    fix x assume "l = Fin x" "u = PlusInf"
    then show ?thesis
      using cong_least_above[OF m, where lo = x and c = c] False
      by auto
  next
    fix y assume "l = MinInf" "u = Fin y"
    then show ?thesis
      using cong_greatest_below[OF m, where hi = y and c = c] False
      by auto
  next
    assume "l = MinInf" "u = PlusInf"
    then show ?thesis using False by (auto intro: exI[of _ c])
  qed (auto simp: False)
```

```
qed

definition it_empty :: "ivl ⇒ congruence ⇒ bool" where
  "it_empty i c =
     (case Rep_congruence c of
        None ⇒ True
      | Some cm ⇒ ¬ cong_meets cm (ivl_lower i) (ivl_upper i))"

lemma it_empty_correct: "it_empty i c ⟷ gamma_ivl i ∩ gamma_congruence c = {}"
proof (cases c rule: congruence_rep_cases)
  case Bot
  then show ?thesis by (simp add: it_empty_def gamma_congruence_def)
next
  case (Class r m)
  have "0 ≤ m" using normalized_Rep_congruence[of c] Class by (cases "m = 0") auto
  then show ?thesis
    using cong_meets_correct[of m r "ivl_lower i" "ivl_upper i"] Class
    by (cases i) (auto simp: it_empty_def gamma_congruence_Some)
qed
```

Mutual refinement

Goblint's `Congruence.refine_with_interval`: a class meeting a finite interval in one value becomes that value, and one meeting it in none becomes bottom. The interval is then rounded to the class (`refine_ivl_with_congruence`, Goblint's `Interval.refine_with_congruence`). This is one round of `IntDomTuple.refine` in that order.

```
definition cong_refine_with_ivl :: "ivl ⇒ congruence ⇒ congruence" where
  "cong_refine_with_ivl i c =
     (case (i, Rep_congruence c) of
        (Ivl (Fin x) (Fin y), Some (r, m)) ⇒
          if m = 0 then (if x ≤ r ∧ r ≤ y then c else bot)
          else (let lo = x + (r - x) mod m; hi = y - (y - r) mod m
                in if hi < lo then bot else if lo = hi then congruence_of_int lo else c)
      | _ ⇒ c)"

lemma cong_refine_with_ivl_sound:
  assumes n: "n ∈ gamma_ivl i" "n ∈ gamma_congruence c"
  shows "n ∈ gamma_congruence (cong_refine_with_ivl i c)"
proof (cases "∃x y r m. i = Ivl (Fin x) (Fin y) ∧ Rep_congruence c = Some (r, m)")
  case True
  then obtain x y r m where i: "i = Ivl (Fin x) (Fin y)" and c: "Rep_congruence c = Some (r, m)"
    by blast
  have xy: "x ≤ n" "n ≤ y" using n(1) by (simp_all add: i)
  have d: "m dvd n - r" using n(2) c by (simp add: gamma_congruence_Some)
  show ?thesis
  proof (cases "m = 0")
    case True
    with d xy n(2) show ?thesis by (simp add: cong_refine_with_ivl_def i c)
  next
    case False
    then have m: "0 < m" using normalized_Rep_congruence[of c] c by auto
    have "x + (r - x) mod m ≤ n"
      using cong_least_above(3)[OF m d xy(1)] .
    moreover have "n ≤ y - (y - r) mod m"
      using cong_greatest_below(3)[OF m d xy(2)] .
    ultimately show ?thesis
      using False n(2) by (auto simp: cong_refine_with_ivl_def i c Let_def)
  qed
qed (use n in ‹auto simp: cong_refine_with_ivl_def split: ivl.splits eint.splits option.splits›)

definition it_refine :: "'m int_tuple ⇒ 'm int_tuple" where
  "it_refine a =
     (let c = cong_refine_with_ivl (it_ivl a) (it_cong a)
      in IT (refine_ivl_with_congruence c (it_ivl a)) c)"

lemma it_refine_sound: "n ∈ gamma_it a ⟹ n ∈ gamma_it (it_refine a)"
  using cong_refine_with_ivl_sound refine_ivl_with_congruence_cover
  by (cases a) (fastforce simp: it_refine_def Let_def)
```

`Refine_Never` leaves the components apart, `Refine_Once` exchanges once, and `Refine_Fixpoint` exchanges until nothing changes. Like the value-level `refine_fix`, the fixpoint keeps its input when the loop does not stop.

```
definition it_refine_mode :: "refine_mode ⇒ 'm int_tuple ⇒ 'm int_tuple" where
  "it_refine_mode md a =
     (case md of
        Refine_Never ⇒ a
```

```
      | Refine_Once ⇒ it_refine a
      | Refine_Fixpoint ⇒
          (case while_option (λx. it_refine x ≠ x) it_refine a of Some r ⇒ r | None ⇒ a))"

lemma it_refine_mode_sound: "n ∈ gamma_it a ⟹ n ∈ gamma_it (it_refine_mode md a)"
proof (cases md)
  case Refine_Fixpoint
  assume n: "n ∈ gamma_it a"
  have "n ∈ gamma_it r" if "while_option (λx. it_refine x ≠ x) it_refine a = Some r" for r
    using while_option_rule[where P = "λx. n ∈ gamma_it x", OF _ that] n it_refine_sound by blast
  with n Refine_Fixpoint show ?thesis by (auto simp: it_refine_mode_def split: option.splits)
qed (simp_all add: it_refine_mode_def it_refine_sound)
```

Domain instances

Widening and narrowing also act componentwise; a value is empty when its congruence class has no member between its interval's bounds.

```
instantiation int_tuple :: (type) numeric_domain
begin

definition widen_int_tuple :: "'a int_tuple ⇒ 'a int_tuple ⇒ 'a int_tuple" where
  "widen_int_tuple a b = IT (it_ivl a ∇ it_ivl b) (it_cong a ∇ it_cong b)"

definition narrow_int_tuple :: "'a int_tuple ⇒ 'a int_tuple ⇒ 'a int_tuple" where
  "narrow_int_tuple a b = IT (it_ivl a Δ it_ivl b) (it_cong a Δ it_cong b)"

definition is_empty_int_tuple :: "'a int_tuple ⇒ bool" where
  "is_empty_int_tuple a = it_empty (it_ivl a) (it_cong a)"
```

A value prints as bottom, as a single integer when a component pins one, and otherwise as each component.

```
definition to_string_int_tuple :: "'a int_tuple ⇒ String.literal" where
  "to_string_int_tuple a =
     (if it_empty (it_ivl a) (it_cong a) then sym_bottom
      else case ivl_constant (it_ivl a) of
        Some n ⇒ string_of_int n
      | None ⇒ (case congruence_constant (it_cong a) of
          Some n ⇒ string_of_int n
        | None ⇒ STR ''intervals:'' + to_string (it_ivl a) + STR ''; congruences:''
                    + to_string (it_cong a)))"

definition gamma_int_tuple :: "'a int_tuple ⇒ int set" where
  "gamma_int_tuple = gamma_it"

instance proof
  fix a b :: "'a int_tuple"
  show "a ≤ a ∇ b" "b ≤ a ∇ b"
    by (simp_all add: widen_int_tuple_def less_eq_int_tuple_def widen_ge1 widen_ge2)
  show "b ≤ a ⟹ b ≤ a Δ b" "b ≤ a ⟹ a Δ b ≤ a"
    by (simp_all add: narrow_int_tuple_def less_eq_int_tuple_def narrow_ge narrow_le)
  show "numeric_domain_class.gamma (bot :: 'a int_tuple) = {}"
    by (simp add: gamma_int_tuple_def bot_int_tuple_def bot_congruence_def)
  show "numeric_domain_class.gamma (top :: 'a int_tuple) = UNIV"
    using gamma_top[where 'a = ivl]
    by (simp add: gamma_int_tuple_def top_int_tuple_def)
  show "a ≤ b ⟹ numeric_domain_class.gamma a ⊆ numeric_domain_class.gamma b"
    using gamma_ivl_mono[of "it_ivl a" "it_ivl b"]
      gamma_mono[of "it_cong a" "it_cong b"]
    by (auto simp: gamma_int_tuple_def gamma_it_def less_eq_int_tuple_def)
  show "is_empty a ⟷ numeric_domain_class.gamma a = {}"
    by (simp add: is_empty_int_tuple_def gamma_int_tuple_def gamma_it_def it_empty_correct)
qed

end

lemma gamma_int_tuple_IT [simp]:
  "numeric_domain_class.gamma (IT i c) = gamma_ivl i ∩ gamma_congruence c"
  by (simp add: gamma_int_tuple_def)

lemma it_refine_IT_sound:
  "n ∈ gamma_ivl i ⟹ n ∈ gamma_congruence c
     ⟹ n ∈ numeric_domain_class.gamma (it_refine_mode md (IT i c))"
  using it_refine_mode_sound[of n "IT i c"] by (simp add: gamma_int_tuple_def)

instantiation int_tuple :: (type) ik_domain
begin
```

```
definition widen_ik_int_tuple :: "ikind ⇒ 'a int_tuple ⇒ 'a int_tuple ⇒ 'a int_tuple" where
  "widen_ik_int_tuple k a b =
     IT (widen_ik k (it_ivl a) (it_ivl b)) (widen_ik k (it_cong a) (it_cong b))"

definition narrow_ik_int_tuple :: "ikind ⇒ 'a int_tuple ⇒ 'a int_tuple ⇒ 'a int_tuple" where
  "narrow_ik_int_tuple k a b =
     IT (narrow_ik k (it_ivl a) (it_ivl b)) (narrow_ik k (it_cong a) (it_cong b))"

definition empty_ik_int_tuple :: "ikind ⇒ 'a int_tuple ⇒ bool" where
  "empty_ik_int_tuple k a = it_empty (it_ivl a ⊓ ik_ivl k) (it_cong a)"

instance proof
  fix k :: ikind and a b :: "'a int_tuple"
  show "a ≤ widen_ik k a b" "b ≤ widen_ik k a b"
    by (simp_all add: widen_ik_int_tuple_def less_eq_int_tuple_def widen_ik_ge1 widen_ik_ge2)
  show "b ≤ a ⟹ b ≤ narrow_ik k a b" "b ≤ a ⟹ narrow_ik k a b ≤ a"
    by (simp_all add: narrow_ik_int_tuple_def less_eq_int_tuple_def narrow_ik_ge narrow_ik_le)
  show "empty_ik k a ⟷ numeric_domain_class.gamma a ∩ ik_range k = {}"
    by (cases a) (auto simp: empty_ik_int_tuple_def it_empty_correct)
qed

end
```

Arithmetic follows `IntDomTuple.map2ovc`: the interval computes first, and when its mathematical result fits the kind, no value overflows (`no_ov`), so the congruence keeps its exact class. A conversion keeps the class on the same condition. Comparisons take the first component that decides, as `map2bool` takes the components' common answer. Every result is refined as the mode says.

```
instantiation int_tuple :: (int_refinement) kind_int_domain
begin

definition top_ik_int_tuple :: "ikind ⇒ 'a int_tuple" where
  "top_ik_int_tuple k = IT (top_ik k) (top_ik k)"

definition of_int_ik_int_tuple :: "ikind ⇒ int ⇒ 'a int_tuple" where
  "of_int_ik_int_tuple k n = IT (of_int_ik k n) (of_int_ik k n)"

definition cast_ik_int_tuple ::
    "signed_overflow_policy ⇒ ikind option ⇒ ikind ⇒ 'a int_tuple ⇒ 'a int_tuple" where
  "cast_ik_int_tuple ovp sk k a =
     it_refine_mode (int_mode TYPE('a)) (IT (cast_ik ovp sk k (it_ivl a))
                  (if fits_ik k (it_ivl a) then it_cong a else cast_ik ovp sk k (it_cong a)))"
```

The interval decides Goblint's `no_ov`: when the mathematical result fits, the congruence keeps its unnormalized class.

```
definition arith_ik_int_tuple ::
    "signed_overflow_policy ⇒ ikind ⇒ aop ⇒ bool ⇒ 'a int_tuple ⇒ 'a int_tuple ⇒ 'a int_tuple" where
  "arith_ik_int_tuple ovp k op fits a b =
     (let r = raw_aop_ik op (it_ivl a) (it_ivl b)
      in it_refine_mode (int_mode TYPE('a)) (IT (cast_ik ovp None k r)
                     (if fits_ik k r then raw_aop_ik op (it_cong a) (it_cong b)
                      else arith_ik ovp k op fits (it_cong a) (it_cong b))))"

definition cmp_ik_int_tuple :: "cop ⇒ 'a int_tuple ⇒ 'a int_tuple ⇒ bool option" where
  "cmp_ik_int_tuple c a b =
     (case cmp_ik c (it_ivl a) (it_ivl b) of
        Some r ⇒ Some r
      | None ⇒ cmp_ik c (it_cong a) (it_cong b))"

definition meet_ik_int_tuple :: "'a int_tuple ⇒ 'a int_tuple ⇒ 'a int_tuple" where
  "meet_ik_int_tuple a b =
     it_refine_mode (int_mode TYPE('a)) (IT (meet_ik (it_ivl a) (it_ivl b)) (meet_ik (it_cong a) (it_cong b)))"

definition refine_ik_int_tuple :: "cop ⇒ 'a int_tuple ⇒ 'a int_tuple ⇒ 'a int_tuple × 'a int_tuple"
  where
  "refine_ik_int_tuple c a b =
     (let (i1, i2) = refine_ik c (it_ivl a) (it_ivl b);
          (c1, c2) = refine_ik c (it_cong a) (it_cong b)
      in (it_refine_mode (int_mode TYPE('a)) (IT i1 c1), it_refine_mode (int_mode TYPE('a)) (IT i2 c2)))"

definition raw_aop_ik_int_tuple :: "aop ⇒ 'a int_tuple ⇒ 'a int_tuple ⇒ 'a int_tuple" where
  "raw_aop_ik_int_tuple op a b =
     IT (raw_aop_ik op (it_ivl a) (it_ivl b)) (raw_aop_ik op (it_cong a) (it_cong b))"
```

```
definition fits_ik_int_tuple :: "ikind ⇒ 'a int_tuple ⇒ bool" where
  "fits_ik_int_tuple k a ⟷ fits_ik k (it_ivl a) ∨ fits_ik k (it_cong a)"

definition answer_ik_int_tuple :: "'a int_tuple ⇒ answer" where
  "answer_ik_int_tuple a = answer_ik (it_ivl a) ⊓ answer_ik (it_cong a)"

definition of_answer_ik_int_tuple :: "ikind option ⇒ answer ⇒ 'a int_tuple" where
  "of_answer_ik_int_tuple sk q = IT (of_answer_ik sk q) (of_answer_ik sk q)"

instance proof
  fix n m :: int and k :: ikind and a b :: "'a int_tuple" and op :: aop and c :: cop and r :: bool
  show "n ∈ ik_range k ⟹ n ∈ numeric_domain_class.gamma (top_ik k :: 'a int_tuple)"
    using top_ik_sound[of n k, where 'a = ivl] top_ik_sound[of n k, where 'a = congruence]
    by (simp add: top_ik_int_tuple_def)
  show "ik_norm k n ∈ numeric_domain_class.gamma (of_int_ik k n :: 'a int_tuple)"
    using of_int_ik_sound[of k n, where 'a = ivl] of_int_ik_sound[of k n, where 'a = congruence]
    by (simp add: of_int_ik_int_tuple_def)
  show "(⋀k'. sk = Some k' ⟹ n ∈ ik_range k') ⟹ n ∈ numeric_domain_class.gamma a
      ⟹ ik_norm k n ∈ numeric_domain_class.gamma (cast_ik ovp sk k a)" for ovp sk
  proof -
    assume src: "⋀k'. sk = Some k' ⟹ n ∈ ik_range k'" and n: "n ∈ numeric_domain_class.gamma a"
    obtain i cg where a: "a = IT i cg" by (cases a)
    have ni: "n ∈ gamma_ivl i" and nc: "n ∈ gamma_congruence cg" using n by (simp_all add: a)
    have "ik_norm k n ∈ gamma_ivl (cast_ik ovp sk k i)"
      using cast_ik_sound[where a = i and k = k and ovp = ovp, OF src] ni by simp
    moreover have "ik_norm k n ∈ gamma_congruence
        (if fits_ik k i then cg else cast_ik ovp sk k cg)"
      using fits_ik_sound[of k i n] cast_ik_sound[where a = cg and k = k and ovp = ovp, OF src] ni
        nc
      by (auto simp del: ik_range_iff)
    ultimately show ?thesis
      unfolding cast_ik_int_tuple_def a int_tuple.sel by (rule it_refine_IT_sound)
  qed
  show "(fits ⟹ m ∈ ik_range k ∧ n ∈ ik_range k) ⟹ m ∈ numeric_domain_class.gamma a
      ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ ik_norm k (aop_eval op m n) ∈ numeric_domain_class.gamma (arith_ik ovp k op fits a b)" for ovp fits
  proof -
    assume fit: "fits ⟹ m ∈ ik_range k ∧ n ∈ ik_range k"
      and ma: "m ∈ numeric_domain_class.gamma a" and nb: "n ∈ numeric_domain_class.gamma b"
    obtain i1 c1 i2 c2 where a: "a = IT i1 c1" and b: "b = IT i2 c2" by (cases a; cases b)
    let ?r = "raw_aop_ik op i1 i2"
    have rv: "aop_eval op m n ∈ gamma_ivl ?r"
      using raw_aop_ik_sound[of m i1 n i2] ma nb by (simp add: a b)
    have "ik_norm k (aop_eval op m n) ∈ gamma_ivl (cast_ik ovp None k ?r)"
      using cast_ik_None_sound[where a = ?r and k = k and ovp = ovp] rv by simp
    moreover have "ik_norm k (aop_eval op m n) ∈ gamma_congruence
        (if fits_ik k ?r then raw_aop_ik op c1 c2 else arith_ik ovp k op fits c1 c2)"
      using fits_ik_sound[of k ?r] rv raw_aop_ik_sound[of m c1 n c2]
        arith_ik_sound[where k = k and op = op and operands_fit = fits and ovp = ovp and a = c1 and
          b = c2,
          OF fit] ma nb
      by (auto simp: a b simp del: ik_range_iff)
    ultimately show ?thesis
      unfolding arith_ik_int_tuple_def a b int_tuple.sel Let_def by (rule it_refine_IT_sound)
  qed
  show "cmp_ik c a b = Some r ⟹ m ∈ numeric_domain_class.gamma a
      ⟹ n ∈ numeric_domain_class.gamma b ⟹ cop_eval c m n = r"
    by (cases a; cases b)
       (auto simp: cmp_ik_int_tuple_def split: option.splits dest: cmp_ik_sound)
  show "m ∈ numeric_domain_class.gamma a ⟹ m ∈ numeric_domain_class.gamma b
      ⟹ m ∈ numeric_domain_class.gamma (meet_ik a b)"
    using meet_ik_sound[of m "it_ivl a" "it_ivl b"] meet_ik_sound[of m "it_cong a" "it_cong b"]
    by (cases a; cases b) (simp add: meet_ik_int_tuple_def it_refine_IT_sound)
  show "cop_eval c m n ⟹ m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ m ∈ numeric_domain_class.gamma (fst (refine_ik c a b))
        ∧ n ∈ numeric_domain_class.gamma (snd (refine_ik c a b))"
    using refine_ik_sound[of c m n "it_ivl a" "it_ivl b"]
      refine_ik_sound[of c m n "it_cong a" "it_cong b"]
    by (cases a; cases b)
       (auto simp: refine_ik_int_tuple_def it_refine_IT_sound split: prod.splits)
  show "m ∈ numeric_domain_class.gamma a ⟹ n ∈ numeric_domain_class.gamma b
      ⟹ aop_eval op m n ∈ numeric_domain_class.gamma (raw_aop_ik op a b)"
    using raw_aop_ik_sound[of m "it_ivl a" n "it_ivl b" op]
      raw_aop_ik_sound[of m "it_cong a" n "it_cong b" op]
    by (cases a; cases b) (simp add: raw_aop_ik_int_tuple_def)
  show "fits_ik k a ⟹ n ∈ numeric_domain_class.gamma a ⟹ n ∈ ik_range k"
```

```
    using fits_ik_sound[of k "it_ivl a" n] fits_ik_sound[of k "it_cong a" n]
    by (cases a) (auto simp: fits_ik_int_tuple_def simp del: ik_range_iff)
  show "n ∈ numeric_domain_class.gamma a ⟹ n ∈ gamma_answer (answer_ik a)"
    using answer_ik_sound[of n "it_ivl a"] answer_ik_sound[of n "it_cong a"]
    by (cases a) (simp add: answer_ik_int_tuple_def gamma_query_lift_inf[OF gamma_inf_int_dom])
  show "(⋀k. sk = Some k ⟹ n ∈ ik_range k) ⟹ n ∈ gamma_answer q
      ⟹ n ∈ numeric_domain_class.gamma (of_answer_ik sk q :: 'a int_tuple)" for sk q
    using of_answer_ik_sound[of sk n q, where 'a = ivl]
      of_answer_ik_sound[of sk n q, where 'a = congruence]
    by (simp add: of_answer_ik_int_tuple_def)
qed

end

end
```

16.5 Int_Regression

```
theory Int_Regression
  imports Int_Tuple_Kind "Voblint_Nonrelational.Nonrelational_Analysis" "Voblint_VIMP.VIMP_Notation"
begin
```

Regression: the typed Interval-Congruence product, executed

Each case runs the certified solver on a typed program inside Isabelle and pins the published value of one variable. Termination is decided by evaluation.

```
abbreviation it_state_at :: "kind_decls ⇒ imp_prog ⇒ cfg_node ⇒ int_fixpoint int_tuple tabs" where
  "it_state_at ds p ≡ nonrel.state_at default_sem_conf ds 0 Globals_Join is_global p []"

abbreviation it_equations :: "kind_decls ⇒ imp_prog ⇒ cfg_node × cfg_node list ⇒
    (cfg_node × cfg_node list, (unit, cfg_node list) global_unknown,
     (int_fixpoint int_tuple tst lifted, int_fixpoint int_tuple tst lifted) dg_state) strategy_tree" where
  "it_equations ds p ≡ nonrel.equations default_sem_conf ds 0 is_global p"

abbreviation f_var :: "vname ⇒ var_id" where
  "f_var x ≡ ScopedId (STR ''f'') x"

abbreviation f_result :: "kind_decls ⇒ imp_prog ⇒ vname ⇒ int_fixpoint int_tuple kd" where
  "f_result ds p x ≡ it_state_at ds p (FunctionResult (STR ''f'')) (f_var x)"
```

No overflow keeps the class

The interval bounds `x + 3` by `101`, so the addition cannot overflow (Goblint's `no_ov`) and the congruence keeps $3\mathbb{Z}$. The exit guard `x ≥ 99` meets the interval `[99, 101]`, and refining with $3\mathbb{Z}$ leaves `99`. The Congruence analysis alone answers top here (`Congruence_Regression`).

```
definition odd_step_prog :: imp_prog where
  "odd_step_prog = program {
     fun f() { x = 0; while (x < 99) { x = x + 3; } }
     fun main() { f(); } }"

lemma odd_step_terminates_c:
  "TD_side_rule_Interp_solve_c Globals_Join (it_equations [] odd_step_prog)
     (cfg_exit (elab_cfg odd_step_prog []), []) ≠ None"
  by eval

lemma odd_step_terminates:
  "nonrel.terminates TYPE(int_fixpoint int_tuple) default_sem_conf [] 0 Globals_Join is_global odd_step_prog"
  unfolding nonrel.terminates_code
  using TD_side_rule_Interp.solve_dom_of_solve_c[OF odd_step_terminates_c] by simp

lemma odd_step_value:
  "f_result [] odd_step_prog (STR ''x'') = KD I32 (IT (Ivl (Fin 99) (Fin 99)) (mk_congruence 99 0))"
  by eval
```

A guard the components refute together

`x` is `0` or `10`, so `x` $\in$ `[0, 10]` $\cap$ $10\mathbb{Z}$. Neither component alone refutes `x == 5`; their meet is empty, so the branch is dead and `y` stays `0`.

```
definition guard_prog :: imp_prog where
  "guard_prog = program {
     fun f(x) { y = 0; if (x == 5) { y = 1; } }
     fun main() { f(0); f(10); } }"

lemma guard_value:
  "f_result [] guard_prog (STR ''y'') = KD I32 (IT (Ivl (Fin 0) (Fin 0)) (mk_congruence 0 0))"
  by eval
```

Unsigned kinds

`x` is a `U8` stepping by `10` below `250`. The class $10\mathbb{Z}$ rounds the guarded interval to `[0, 240]`, so `x + 10` stays within the kind and the class survives the conversion back; the exit is exactly `250`. A constant that leaves the kind wraps in both components.

```
definition wrap_loop_prog :: imp_prog where
  "wrap_loop_prog = program {
     fun f() { x = 0; while (x < 250) { x = x + 10; } }
     fun main() { f(); } }"
```

```
definition wrap_prog :: imp_prog where
  "wrap_prog = program { fun f() { x = 250; x = x + 10; } fun main() { f(); } }"

lemma unsigned_values:
  "f_result [(f_var (STR ''x''), U8)] wrap_loop_prog (STR ''x'')
     = KD U8 (IT (Ivl (Fin 250) (Fin 250)) (mk_congruence 250 0))"
  "f_result [(f_var (STR ''x''), U8)] wrap_prog (STR ''x'')
     = KD U8 (IT (Ivl (Fin 4) (Fin 4)) (mk_congruence 4 0))"
  by eval+

end
```

Chapter 17

Voblint_CLI

17.1 Analysis_Config

```
theory Analysis_Config
  imports "Voblint_Analysis_Int.Int_Tuple_Kind" "Voblint_Analysis_Order.Order_Analysis"
    "Voblint_Analysis_Sign.Sign_Kind" "Voblint_Analysis_Parity.Parity_Kind"
begin
```

Running a typed analysis

The vocabulary of a typed run: the typed analyses a caller activates and the context policy. `run_voblint` runs the activated analyses as one.

Each refinement mode of Int is an analysis of its own.

```
datatype analysis =
  Interval | Congruence | Ana_Int refine_mode | Order | Sign | Parity

lemma analysis_cases
  [case_names Interval Congruence Int_Never Int_Once Int_Fixpoint Order Sign Parity]:
  obtains "a = Interval" | "a = Congruence" | "a = Ana_Int Refine_Never"
    | "a = Ana_Int Refine_Once" | "a = Ana_Int Refine_Fixpoint" | "a = Order"
    | "a = Sign" | "a = Parity"
  by (metis refine_mode.exhaust analysis.exhaust)
```

Contexts are call strings of bound `k` (`0` is context-insensitive) or entry states.

```
datatype context_policy = Call_String nat | Entry_State
```

Where a program's globals live: in every point's own state, flow-sensitively, or each at its own global unknown, as a flow-insensitive value read only by the edges that mention it.

```
datatype program_globals = Program_Globals_Flow_Sensitive | Program_Globals_Flow_Insensitive
```

The options of a run as one value, in the role of Goblint's configuration tree: the activated analyses (`ana.activated`), the context policy, the update rule of the globals, where the program's globals live, and the options the domains read (`sem`). Goblint reads an option from its global tree where it is used; here the value is passed down, and a further option is a further field.

```
record voblint_conf =
  conf_analyses :: "analysis list"
  conf_context :: context_policy
  conf_globals :: globals_rule
  conf_program_globals :: program_globals
  conf_sem :: sem_conf
```

The typed stores that reach a node from a store whose globals are zero at their kinds: what every soundness theorem of a typed run is stated over.

```
abbreviation reaching :: "kind_decls ⇒ imp_prog ⇒ cfg_node ⇒ store set" where
  "reaching ds p v ≡ 𝒞⟨kenv_of ds,elab_cfg p ds,init_stores (kenv_of ds)⟩ v"

end
```

17.2 MCP_Carrier

```
theory MCP_Carrier
  imports Analysis_Config "Voblint_Framework.MCP_Registry" "Voblint_Framework.Local_Spec_Product"
begin
```

The combined state of the typed analyses

Every typed analysis owns one field of the combined state, as in Goblint's MCP, and the typed analyses form a `mcp_registry` over the typed language. A non-relational field runs the typed component and answers through its evaluator, which may ask the channel about operands; the order field runs the order component and answers comparisons from its facts.

Fields

Int has one field per refinement mode, each holding the product at that mode.

```
type_synonym mcp_st =
  "(ivl tst lifted, (congruence tst lifted,
    ((int_never int_tuple tst lifted, (int_once int_tuple tst lifted,
      int_fixpoint int_tuple tst lifted) analysis_product) analysis_product,
    (sign tst lifted, (parity tst lifted, var_id order_rel lifted)
    analysis_product) analysis_product) analysis_product) analysis_product) analysis_product"

type_synonym mcp_val =
  "(ivl tabs lifted, (congruence tabs lifted,
    ((int_never int_tuple tabs lifted, (int_once int_tuple tabs lifted,
      int_fixpoint int_tuple tabs lifted) analysis_product) analysis_product,
    (sign tabs lifted, (parity tabs lifted, var_id order_rel lifted)
    analysis_product) analysis_product) analysis_product) analysis_product) analysis_product"

definition slot1 where "slot1 r = pleft r"
definition slot2 where "slot2 r = pleft (pright r)"
definition slot3 where "slot3 r = pleft (pright (pright r))"
definition slot4 where "slot4 r = pleft (pright (pright (pright r)))"
definition slot5 where "slot5 r = pleft (pright (pright (pright (pright r))))"
definition slot6 where "slot6 r = pright (pright (pright (pright (pright r))))"
definition slot3n where "slot3n r = pleft (slot3 r)"
definition slot3o where "slot3o r = pleft (pright (slot3 r))"
definition slot3f where "slot3f r = pright (pright (slot3 r))"

definition set_slot1 where "set_slot1 r v = Product v (pright r)"
definition set_slot2 where "set_slot2 r v = Product (pleft r) (Product v (pright (pright r)))"
definition set_slot3 where
  "set_slot3 r v = Product (pleft r) (Product (pleft (pright r))
     (Product v (pright (pright (pright r)))))"
definition set_slot4 where
  "set_slot4 r v = Product (pleft r) (Product (pleft (pright r)) (Product (pleft (pright (pright r)))
     (Product v (pright (pright (pright (pright r)))))))"
definition set_slot5 where
  "set_slot5 r v = Product (pleft r) (Product (pleft (pright r)) (Product (pleft (pright (pright r)))
     (Product (pleft (pright (pright (pright r)))) (Product v (pright (pright (pright (pright (pright r))))))))))"
definition set_slot6 where
  "set_slot6 r v = Product (pleft r) (Product (pleft (pright r)) (Product (pleft (pright (pright r)))
     (Product (pleft (pright (pright (pright r)))) (Product (pleft (pright (pright (pright (pright r))))) v))))"
definition set_slot3n where "set_slot3n r v = set_slot3 r (Product v (pright (slot3 r)))"
definition set_slot3o where
  "set_slot3o r v = set_slot3 r (Product (slot3n r) (Product v (slot3f r)))"
definition set_slot3f where
  "set_slot3f r v = set_slot3 r (Product (slot3n r) (Product (slot3o r) v))"

lemmas slot_defs [simp] = slot1_def slot2_def slot3_def slot4_def slot5_def slot6_def
  slot3n_def slot3o_def slot3f_def
  set_slot1_def set_slot2_def set_slot3_def set_slot4_def set_slot5_def set_slot6_def
  set_slot3n_def set_slot3o_def set_slot3f_def

definition mcp_rd :: "(var_id ⇒ bool) ⇒ mcp_st ⇒ mcp_val" where
  "mcp_rd 𝒢 r =
     Product (map_lift (default_st_to_fun 𝒢) (slot1 r))
     (Product (map_lift (default_st_to_fun 𝒢) (slot2 r))
     (Product (Product (map_lift (default_st_to_fun 𝒢) (slot3n r))
        (Product (map_lift (default_st_to_fun 𝒢) (slot3o r))
                 (map_lift (default_st_to_fun 𝒢) (slot3f r))))
     (Product (map_lift (default_st_to_fun 𝒢) (slot4 r))
     (Product (map_lift (default_st_to_fun 𝒢) (slot5 r)) (slot6 r)))))"
```

Each analysis on its own field

Each selectable analysis runs its own specification on one slot of the carrier, through a lens that reads and writes that slot alone; `val_gamma` reads the stores the slot denotes.

```
definition field ::
  "sem_conf ⇒ kenv ⇒ imp_prog ⇒ analysis ⇒ mcp_st lifted local_spec" where
  "field sem κ p a = (case a of
      Interval ⇒ lens_of (lift_get slot1) (lift_put set_slot1) (nonrel_exec_spec sem κ)
    | Congruence ⇒ lens_of (lift_get slot2) (lift_put set_slot2) (nonrel_exec_spec sem κ)
    | Ana_Int Refine_Never ⇒ lens_of (lift_get slot3n) (lift_put set_slot3n) (nonrel_exec_spec sem κ)
    | Ana_Int Refine_Once ⇒ lens_of (lift_get slot3o) (lift_put set_slot3o) (nonrel_exec_spec sem κ)
    | Ana_Int Refine_Fixpoint ⇒ lens_of (lift_get slot3f) (lift_put set_slot3f) (nonrel_exec_spec sem κ)
    | Sign ⇒ lens_of (lift_get slot4) (lift_put set_slot4) (nonrel_exec_spec sem κ)
    | Parity ⇒ lens_of (lift_get slot5) (lift_put set_slot5) (nonrel_exec_spec sem κ)
    | Order ⇒ lens_of (lift_get slot6) (lift_put set_slot6) (ord_spec κ (ord_vars p)))"

fun val_gamma :: "kenv ⇒ analysis ⇒ mcp_val ⇒ store set" where
  "val_gamma κ Interval v = gamma_lift (gamma_tabs κ) (slot1 v)"
| "val_gamma κ Congruence v = gamma_lift (gamma_tabs κ) (slot2 v)"
| "val_gamma κ (Ana_Int Refine_Never) v = gamma_lift (gamma_tabs κ) (slot3n v)"
| "val_gamma κ (Ana_Int Refine_Once) v = gamma_lift (gamma_tabs κ) (slot3o v)"
| "val_gamma κ (Ana_Int Refine_Fixpoint) v = gamma_lift (gamma_tabs κ) (slot3f v)"
| "val_gamma κ Sign v = gamma_lift (gamma_tabs κ) (slot4 v)"
| "val_gamma κ Parity v = gamma_lift (gamma_tabs κ) (slot5 v)"
| "val_gamma κ Order v = gamma_ord_lift κ (slot6 v)"

fun live :: "analysis ⇒ mcp_st ⇒ bool" where
  "live Interval r = (slot1 r ≠ Bot)"
| "live Congruence r = (slot2 r ≠ Bot)"
| "live (Ana_Int Refine_Never) r = (slot3n r ≠ Bot)"
| "live (Ana_Int Refine_Once) r = (slot3o r ≠ Bot)"
| "live (Ana_Int Refine_Fixpoint) r = (slot3f r ≠ Bot)"
| "live Sign r = (slot4 r ≠ Bot)"
| "live Parity r = (slot5 r ≠ Bot)"
| "live Order r = (slot6 r ≠ Bot)"

definition t_lift_empty :: "'a::kind_int_domain tabs lifted ⇒ bool" where
  "t_lift_empty x = (case x of Bot ⇒ True | Lifted d ⇒ t_empty d)"

fun val_empty :: "analysis ⇒ mcp_val ⇒ bool" where
  "val_empty Interval v = t_lift_empty (slot1 v)"
| "val_empty Congruence v = t_lift_empty (slot2 v)"
| "val_empty (Ana_Int Refine_Never) v = t_lift_empty (slot3n v)"
| "val_empty (Ana_Int Refine_Once) v = t_lift_empty (slot3o v)"
| "val_empty (Ana_Int Refine_Fixpoint) v = t_lift_empty (slot3f v)"
| "val_empty Sign v = t_lift_empty (slot4 v)"
| "val_empty Parity v = t_lift_empty (slot5 v)"
| "val_empty Order v = (slot6 v = Bot)"

definition st_lift_empty :: "'a::kind_int_domain tst lifted ⇒ bool" where
  "st_lift_empty x = (case x of Bot ⇒ True | Lifted d ⇒ st_is_bot d)"

fun field_empty :: "analysis ⇒ mcp_st ⇒ bool" where
  "field_empty Interval r = st_lift_empty (slot1 r)"
| "field_empty Congruence r = st_lift_empty (slot2 r)"
| "field_empty (Ana_Int Refine_Never) r = st_lift_empty (slot3n r)"
| "field_empty (Ana_Int Refine_Once) r = st_lift_empty (slot3o r)"
| "field_empty (Ana_Int Refine_Fixpoint) r = st_lift_empty (slot3f r)"
| "field_empty Sign r = st_lift_empty (slot4 r)"
| "field_empty Parity r = st_lift_empty (slot5 r)"
| "field_empty Order r = (slot6 r = Bot)"

definition t_answer ::
  "sem_conf ⇒ kenv ⇒ 'a::kind_int_domain tabs lifted ⇒ channel ⇒ channel" where
  "t_answer sem κ x ch q = (case x of Bot ⇒ ⊤ | Lifted d ⇒ nonrel_query sem κ ch d q)"

fun answer :: "sem_conf ⇒ kenv ⇒ analysis ⇒ mcp_val ⇒ channel ⇒ channel" where
  "answer sem κ Interval v ch = t_answer sem κ (slot1 v) ch"
| "answer sem κ Congruence v ch = t_answer sem κ (slot2 v) ch"
| "answer sem κ (Ana_Int Refine_Never) v ch = t_answer sem κ (slot3n v) ch"
| "answer sem κ (Ana_Int Refine_Once) v ch = t_answer sem κ (slot3o v) ch"
| "answer sem κ (Ana_Int Refine_Fixpoint) v ch = t_answer sem κ (slot3f v) ch"
| "answer sem κ Sign v ch = t_answer sem κ (slot4 v) ch"
| "answer sem κ Parity v ch = t_answer sem κ (slot5 v) ch"
| "answer sem κ Order v ch = ord_query ch (slot6 v)"
```

A check asks for the truth value of its condition: a comparison or a logical operator is that value already, and any other condition is compared with zero.

```
fun truth_exp :: "exp ⇒ exp" where
  "truth_exp (TCmp op a b) = TCmp op a b"
| "truth_exp (TNot a) = TNot a"
| "truth_exp (TAnd a b) = TAnd a b"
| "truth_exp (TOr a b) = TOr a b"
| "truth_exp c = TCmp CNe c (TN I32 0)"

lemma eval_truth_exp: "eval (truth_exp c) s = (if truthy (eval c s) then 1 else 0)"
  by (cases c) (simp_all add: truthy_def ik_norm_def ik_min_def ik_max_def)
```

The registry

Each field is sound for its own slot and leaves every other slot's stores in place, so the fields together interpret the registry locale `mcp_registry`.

```
lemma t_answer_sound:
  assumes "s ∈ gamma_lift (gamma_tabs κ) x" and "eval_query.channel_holds ch s"
  shows "eval_holds q (t_answer sem κ (x :: 'a::kind_int_domain tabs lifted) ch q) s"
proof (cases x)
  case (Lifted d)
  from assms Lifted have "s ∈ gamma_tabs κ d" by simp
  from sound_local_spec_queryD[OF nonrel_spec_sound[of κ, where 'a = 'a],
      unfolded sound_query_def, rule_format, OF this assms(2)]
  show ?thesis by (simp add: t_answer_def Lifted nonrel_spec_def)
qed (simp add: t_answer_def)

lemma lens_lift_sound:
  fixes f :: "mcp_st ⇒ 'c::{semilattice_sup, order_bot}"
  assumes sound: "sound_local_spec κ g cmp"
    and get_put: "⋀r v. f (u r v) = v" and get_bot: "f ⊥ = ⊥"
    and mono: "⋀r r'. r ≤ r' ⟹ f r ≤ f r'"
  shows "sound_local_spec κ
          (λx. g (lift_get f x)) (lens_of (lift_get f) (lift_put u) cmp)"
  by (rule lens_of_sound[OF sound])
     (simp_all add: lift_get_put[OF get_put get_bot] lift_get_mono[OF mono])

lemma lens_lift_frame:
  assumes "⋀x v. g (lift_put u x v) = g x"
  shows "mcp_frame (lens_of get (lift_put u) cmp) g"
  unfolding mcp_frame_def lens_of_def by (auto simp: assms)

lemma lens_lift_sound_eq:
  fixes f :: "mcp_st ⇒ 'c::{semilattice_sup, order_bot}"
  assumes sound: "sound_local_spec κ gm cmp"
    and get_put: "⋀r v. f (u r v) = v" and get_bot: "f ⊥ = ⊥"
    and mono: "⋀r r'. r ≤ r' ⟹ f r ≤ f r'"
    and eq: "⋀x. g x = gm (lift_get f x)"
  shows "sound_local_spec κ g
          (lens_of (lift_get f) (lift_put u) cmp)"
proof -
  have "g = (λx. gm (lift_get f x))" by (rule ext) (rule eq)
  then show ?thesis using lens_lift_sound[OF sound get_put get_bot mono] by simp
qed

lemma mcp_bot_slots [simp]:
  "slot1 (⊥ :: mcp_st) = Bot" "slot2 (⊥ :: mcp_st) = Bot" "slot3n (⊥ :: mcp_st) = Bot"
  "slot3o (⊥ :: mcp_st) = Bot" "slot3f (⊥ :: mcp_st) = Bot"
  "slot4 (⊥ :: mcp_st) = Bot" "slot5 (⊥ :: mcp_st) = Bot" "slot6 (⊥ :: mcp_st) = Bot"
  "slot1 (⊥ :: mcp_val) = Bot" "slot2 (⊥ :: mcp_val) = Bot" "slot3n (⊥ :: mcp_val) = Bot"
  "slot3o (⊥ :: mcp_val) = Bot" "slot3f (⊥ :: mcp_val) = Bot"
  "slot4 (⊥ :: mcp_val) = Bot" "slot5 (⊥ :: mcp_val) = Bot" "slot6 (⊥ :: mcp_val) = Bot"
  by (simp_all add: bot_analysis_product_def bot_lifted_def)

lemma field_sound:
  "sound_local_spec κ
     (λx. gamma_lift (val_gamma κ a) (map_lift (mcp_rd is_global) x)) (field sem κ p a)"
proof -
  note nonrel = lens_lift_sound_eq[OF nonrel_exec_spec_sound]
  note simps = less_eq_analysis_product_def lift_get_def mcp_rd_def gamma_exec_def
    bot_analysis_product_def bot_lifted_def
  show ?thesis
  proof (cases a rule: analysis_cases)
    case Order then show ?thesis
      by (simp add: field_def)
```

```
        (rule lens_lift_sound_eq[OF ord_spec_sound]; auto simp: simps split: lifted.splits)
  qed (simp add: field_def; rule nonrel; auto simp: simps split: lifted.splits)+
qed

lemma field_frame:
  "a ≠ b ⟹ mcp_frame (field sem κ p a)
     (λx. gamma_lift (val_gamma κ b) (map_lift (mcp_rd 𝒢) x))"
  by (cases a rule: analysis_cases; cases b rule: analysis_cases; simp add: field_def;
      rule lens_lift_frame;
      auto simp: lift_put_def mcp_rd_def bot_analysis_product_def bot_lifted_def
        split: lifted.splits)

lemma ord_answer_sound:
  "s ∈ gamma_ord_lift κ x ⟹ eval_holds q (ord_query ch x q) s"
  by (cases x) (simp_all add: ord_query_sound)

lemma single_entry_nonrel_exec_spec: "single_entry (nonrel_exec_spec sem κ)"
  by (simp add: single_entry_def nonrel_exec_spec_def)

lemma single_entry_ord_spec: "single_entry (ord_spec κ ys)"
  by (simp add: single_entry_def ord_spec_def)

lemma single_entry_field: "single_entry (field sem κ p a)"
  by (cases a rule: analysis_cases) (auto simp: field_def intro!: single_entry_lens_of lift_put_get
      single_entry_nonrel_exec_spec single_entry_ord_spec)

global_interpretation mcp: mcp_registry "kenv_of ds"
    truth_exp "λ𝒢 p a. field sem (kenv_of ds) p a" live mcp_rd "val_gamma (kenv_of ds)" val_empty
    "λp. field_empty" "answer sem (kenv_of ds)"
  for sem ds
  defines part_answer = mcp.part_answer and mcp_field = mcp.mcp_field
    and mcp_norm = mcp.mcp_norm and mcp_comp = mcp.mcp_comp and mcp_emp = mcp.mcp_emp
    and mcp_empty_v = mcp.mcp_empty_v and mcp_answer = mcp.mcp_answer
    and mcp_ask = mcp.mcp_ask and mcp_classify = mcp.mcp_classify
proof (unfold_locales, goal_cases Truth FieldSound Frame Single Dead EmptyRd EmptyGamma Bot Answer)
  case (Truth c s) show ?case by (rule eval_truth_exp)
next
  case (FieldSound p a) show ?case by (rule field_sound)
next
  case (Frame a b 𝒢 p) then show ?case by (rule field_frame)
next
  case (Single 𝒢 p a) show ?case by (rule single_entry_field)
next
  case (Dead a r 𝒢) then show ?case by (cases a rule: analysis_cases) (simp_all add: mcp_rd_def)
next
  case (EmptyRd p a r) show ?case
    by (cases a rule: analysis_cases)
       (simp_all add: mcp_rd_def st_lift_empty_def t_lift_empty_def st_is_bot_iff t_empty_def split:
        lifted.splits)
next
  case (EmptyGamma a v) then show ?case
    by (cases a rule: analysis_cases)
      (auto simp: t_lift_empty_def dest: t_empty_sound split: lifted.splits)
next
  case (Bot a) show ?case
    by (cases a rule: analysis_cases) (simp_all add: bot_analysis_product_def bot_lifted_def)
next
  case (Answer s a v ch q) then show ?case
    by (cases a rule: analysis_cases) (auto intro: t_answer_sound ord_answer_sound)
qed

end
```

17.3 MCP

```
theory MCP
  imports MCP_Carrier
begin
```

The active typed analyses, run through the pipeline

A caller activates a list of typed analyses, as Goblint's `ana.activated`. The registry runs the distinct ones as one component; the pipeline solves it on the typed CFG at call-string contexts of bound `k`, exactly as it solves one analysis.

```
definition activation_of :: "analysis list ⇒ analysis list" where
  "activation_of as = (if as = [] then [Interval] else remdups as)"

lemma activation_of_ne [simp]: "activation_of as ≠ []"
  by (simp add: activation_of_def)

lemma distinct_activation_of [simp]: "distinct (activation_of as)"
  by (simp add: activation_of_def)
```

Every active field starts at its analysis's own entry state, every other at bottom.

```
definition mcp_init :: "kind_decls ⇒ analysis list ⇒ mcp_st" where
  "mcp_init ds as =
     Product (if Interval ∈ set as then Lifted (nonrel_init_st ds) else Bot)
     (Product (if Congruence ∈ set as then Lifted (nonrel_init_st ds) else Bot)
     (Product (Product (if Ana_Int Refine_Never ∈ set as then Lifted (nonrel_init_st ds) else Bot)
        (Product (if Ana_Int Refine_Once ∈ set as then Lifted (nonrel_init_st ds) else Bot)
                 (if Ana_Int Refine_Fixpoint ∈ set as then Lifted (nonrel_init_st ds) else Bot)))
     (Product (if Sign ∈ set as then Lifted (nonrel_init_st ds) else Bot)
     (Product (if Parity ∈ set as then Lifted (nonrel_init_st ds) else Bot)
              (if Order ∈ set as then Lifted (OFacts True {}) else Bot)))))"

lemma mcp_init_sound:
  "init_stores (kenv_of ds) ⊆ mcp.act_gamma ds as (mcp_rd is_global (mcp_init ds as))"
proof
  fix t assume t: "t ∈ init_stores (kenv_of ds)"
  have "t ∈ val_gamma (kenv_of ds) a (mcp_rd is_global (mcp_init ds as))" if m: "a ∈ set as" for a
  proof (cases a rule: analysis_cases)
    case Order with m show ?thesis
      by (simp add: mcp_init_def mcp_rd_def init_stores_in_kinds[OF t])
  qed (use m in ‹simp_all add: mcp_init_def mcp_rd_def subsetD[OF nonrel_init_sound t]›)
  then show "t ∈ mcp.act_gamma ds as (mcp_rd is_global (mcp_init ds as))"
    by (simp add: mcp.act_gamma_def)
qed
```

What the combination owes the pipeline does not depend on the context policy, so it is proved once, for any route and root context.

```
lemma mcp_cs_routed:
  "dg_analysis
    (kenv_of ds) (λ_. init_stores (kenv_of ds))
    (mcp_comp sem ds (activation_of as)) (mcp_emp (activation_of as)) mcp_rd
    (mcp_init ds (activation_of as))
    Analysis_Buffer Analysis_Global Activation_Seed
    (TD_side_rule_Interp_solve r)
    (TD_side_rule_Interp.solve_dom TYPE((var_id, 'c) global_unknown)
       TYPE((mcp_st lifted, mcp_st lifted) dg_state) r)
    ⊥ (mcp_classify sem ds (activation_of as)) (mcp.act_gamma ds (activation_of as))
    (mcp_empty_v (activation_of as))
    (TD_side_rule_Interp_solve_c r)
    (λ_ c. dg_spec_of c) (λp d e. d) (λ_ d. d) (λ_ d. Bot) (λ_ p. []) (λp ci d e. d)
    (λp. elab_cfg p ds) (λ_. FunctionEntry prog_main_name)"
proof (rule dg_analysis_whole_stateI[OF td_certified_solver], goal_cases CompSound EnterSingle
    EmptyRd EmptyV SeedNe ClProved ClRefuted BotState InitSound SeedNeGlobal PEntry FinIntra
    FinCalls CallsUnique)
  case (CompSound p)
  have eq: "mcp_gamma (map (mcp.part_gamma ds is_global) (activation_of as))
      = (λd. gamma_lift (mcp.act_gamma ds (activation_of as)) (map_lift (mcp_rd is_global) d))"
    by (rule ext) (rule mcp.mcp_gamma_rd[OF activation_of_ne])
  show ?case
    using mcp.mcp_comp_sound[OF distinct_activation_of activation_of_ne, where sem = sem and ds = ds
      and p = p,
        of as]
    unfolding eq by simp
```

```
next
  case (EnterSingle p ci d)
  show ?case
    using single_entryD[OF mcp.single_entry_mcp_comp[OF activation_of_ne],
        of sem ds as is_global p
          "ls_channel (mcp_comp sem ds (activation_of as) is_global p) d" ci "(d, d)"]
    by (simp add: dg_pipeline.comp_entry_def)
next
  case (EmptyRd p s) show ?case by (rule mcp.mcp_emp_rd)
next
  case EmptyV show ?case by (rule mcp.sound_emptiness_mcp)
next
  case (SeedNe v ctx) show ?case by simp
next
  case (ClProved c d s) then show ?case by (intro mcp.mcp_classify_proved)
next
  case (ClRefuted c d s) then show ?case by (intro mcp.mcp_classify_refuted)
next
  case BotState show ?case by (rule mcp.act_gamma_bot[OF activation_of_ne])
next
  case (InitSound p) show ?case by (rule mcp_init_sound)
next
  case (SeedNeGlobal v ctx n) show ?case by simp
next
  case (PEntry p) show ?case by (simp add: elab_cfg_def)
next
  case (FinIntra p) show ?case by (simp add: elab_cfg_def compile_prog_finite)
next
  case (FinCalls p) show ?case by (simp add: elab_cfg_def compile_prog_finite)
next
  case (CallsUnique p) show ?case by (rule elab_cfg_calls_source_unique)
qed
```

At call-string contexts

The combination run with `k`-bounded call strings as contexts, rooted at the empty call string.

```
global_interpretation mcp_cs: dg_analysis
    "kenv_of ds" "λ_. init_stores (kenv_of ds)"
    "mcp_comp sem ds (activation_of as)" "mcp_emp (activation_of as)" mcp_rd
    "mcp_init ds (activation_of as)"
    Analysis_Buffer Analysis_Global Activation_Seed "λ_. cs_route k" "[]"
    "TD_side_rule_Interp_solve r"
    "TD_side_rule_Interp.solve_dom TYPE((var_id, cfg_node list) global_unknown)
       TYPE((mcp_st lifted, mcp_st lifted) dg_state) r"
    ⊥ "mcp_classify sem ds (activation_of as)" "mcp.act_gamma ds (activation_of as)"
    "mcp_empty_v (activation_of as)"
    "TD_side_rule_Interp_solve_c r"
    "λ_ c. dg_spec_of c" "λp d e. d" "λ_ d. d" "λ_ d. Bot" "λ_ p. []" "λp ci d e. d"
    "λp. elab_cfg p ds" "λ_. FunctionEntry prog_main_name"
  for sem ds as r k
  by (rule mcp_cs_routed)
```

At entry-state contexts

A call is keyed by what each active field gives the callee's formals on entry, as the entry-state analyses key a single domain (`nonrel_es_ctx`); a field no active analysis runs keys nothing, and the order field keys nothing.

```
type_synonym mcp_ctx =
  "(ivl kd list, (congruence kd list,
    ((int_never int_tuple kd list, (int_once int_tuple kd list, int_fixpoint int_tuple kd list)
      analysis_product) analysis_product,
    (sign kd list, (parity kd list, unit list) analysis_product) analysis_product)
    analysis_product) analysis_product) analysis_product"

definition mcp_route ::
    "analysis list ⇒ (var_id ⇒ bool) ⇒ cfg_node ⇒ mcp_ctx ⇒ mcp_st lifted
       ⇒ (var_id, exp) gcall_action ⇒ mcp_ctx" where
  "mcp_route as 𝒢 u ctx d ca =
     Product (if Interval ∈ set as then nonrel_es_ctx 𝒢 (lift_get slot1 d) ca else [])
     (Product (if Congruence ∈ set as then nonrel_es_ctx 𝒢 (lift_get slot2 d) ca else [])
     (Product (Product (if Ana_Int Refine_Never ∈ set as then nonrel_es_ctx 𝒢 (lift_get slot3n d) ca else [])
        (Product (if Ana_Int Refine_Once ∈ set as then nonrel_es_ctx 𝒢 (lift_get slot3o d) ca else [])
                 (if Ana_Int Refine_Fixpoint ∈ set as then nonrel_es_ctx 𝒢 (lift_get slot3f d) ca else [])))
     (Product (if Sign ∈ set as then nonrel_es_ctx 𝒢 (lift_get slot4 d) ca else [])
     (Product (if Parity ∈ set as then nonrel_es_ctx 𝒢 (lift_get slot5 d) ca else []) [])))))"
```

```
definition mcp_root_ctx :: mcp_ctx where
  "mcp_root_ctx = Product [] (Product [] (Product (Product [] (Product [] []))
     (Product [] (Product [] []))))"

global_interpretation mcp_es: dg_analysis
    "kenv_of ds" "λ_. init_stores (kenv_of ds)"
    "mcp_comp sem ds (activation_of as)" "mcp_emp (activation_of as)" mcp_rd
    "mcp_init ds (activation_of as)"
    Analysis_Buffer Analysis_Global Activation_Seed "mcp_route (activation_of as)" mcp_root_ctx
    "TD_side_rule_Interp_solve r"
    "TD_side_rule_Interp.solve_dom TYPE((var_id, mcp_ctx) global_unknown)
       TYPE((mcp_st lifted, mcp_st lifted) dg_state) r"
    ⊥ "mcp_classify sem ds (activation_of as)" "mcp.act_gamma ds (activation_of as)"
    "mcp_empty_v (activation_of as)"
    "TD_side_rule_Interp_solve_c r"
    "λ_ c. dg_spec_of c" "λp d e. d" "λ_ d. d" "λ_ d. Bot" "λ_ p. []" "λp ci d e. d"
    "λp. elab_cfg p ds" "λ_. FunctionEntry prog_main_name"
  for sem ds as r
  by (rule mcp_cs_routed)
```

End-to-end soundness

Under either context policy, every typed store reaching a node lies in the combined state the solved table holds there at some context.

```
context
  fixes sem :: sem_conf and ds :: kind_decls and as :: "analysis list" and r :: globals_rule and k
    :: nat
begin

lemma mcp_cs_node_sound:
  assumes wf: "wf_program_compile_input p"
    and solves: "mcp_cs.terminates sem ds as k r is_global p"
  shows "(reaching ds p v)
         ⊆ (⋃ctx. mcp.act_gamma ds (activation_of as) (mcp_cs.state_at sem ds as k r is_global p ctx v))"
  using mcp_cs.fun_route_result_node_sound
      [OF elab_cfg_def _ elab_wf_compile_input[OF wf] solves, of "cs_context k" v]
  by (simp add: cs_route_def cs_context_def)

lemma mcp_es_node_sound:
  assumes wf: "wf_program_compile_input p"
    and solves: "mcp_es.terminates sem ds as r is_global p"
  shows "(reaching ds p v)
         ⊆ (⋃ctx. mcp.act_gamma ds (activation_of as) (mcp_es.state_at sem ds as r is_global p ctx v))"
  using mcp_es.entry_state_result_node_sound
      [OF elab_cfg_def elab_wf_compile_input[OF wf] solves, of v]
  by simp

end

end
```

17.4 Dispatch_Carrier

```
theory Dispatch_Carrier
  imports "Voblint_Analysis_Order.Order_Analysis"
begin
```

What each domain's value needs to be shown and listed

A rendered state shows each analysis's part of it, and a report lists the contexts it was solved at. What that needs from each domain is domain content and stays here: how an order value is shown, and an injective key for every domain's value.

A report names an order fact's variables by their labels, so it prints the facts as an order value over names.

```
definition order_view_string :: "(vname × vname) list option ⇒ String.literal" where
  "order_view_string v =
     to_string (case v of None ⇒ (OBot :: vname order_rel) | Some ps ⇒ OFacts False (set ps))"
```

What one analysis shows of a state, as Goblint's report shows each component of its combined state on its own: a pointwise analysis a value per variable, an analysis whose state relates variables one value for the whole state.

```
datatype 'v field_state = Field_Store "(vname × 'v) list" | Field_Whole 'v

fun map_field_state :: "('v ⇒ 'w) ⇒ 'v field_state ⇒ 'w field_state" where
  "map_field_state f (Field_Store bs) = Field_Store (map (λ(x, v). (x, f v)) bs)"
| "map_field_state f (Field_Whole v) = Field_Whole (f v)"
```

Listing a set of contexts without reading its values' order

A solved table covers a *set* of contexts at each point, and a result handed to a caller lists them. Listing needs a linear order, but a domain's value type already spends its `ord` instance on the abstraction order, under which `[0,1]` and `[2,3]` are incomparable. `order_key` is a separate type with a derived linear order, and each domain's key maps its values into it injectively. Because the map is injective, listing by it drops no context — and because it reads the value's structure rather than its rendering, no printer decides what is listed.

```
datatype order_key = Key_Int int | Key_Node cfg_node | Key_List "order_key list"

derive linorder order_key

fun sign_key :: "sign ⇒ order_key" where
  "sign_key SBot = Key_Int 0"
| "sign_key SNeg = Key_Int 1"
| "sign_key SNonPos = Key_Int 2"
| "sign_key SZero = Key_Int 3"
| "sign_key SNonNeg = Key_Int 4"
| "sign_key SPos = Key_Int 5"
| "sign_key STop = Key_Int 6"

fun parity_key :: "parity ⇒ order_key" where
  "parity_key PBot = Key_Int 0"
| "parity_key PEven = Key_Int 1"
| "parity_key POdd = Key_Int 2"
| "parity_key PTop = Key_Int 3"

fun eint_key :: "eint ⇒ order_key" where
  "eint_key MinInf = Key_List [Key_Int 0]"
| "eint_key (Fin n) = Key_List [Key_Int 1, Key_Int n]"
| "eint_key PlusInf = Key_List [Key_Int 2]"

fun ivl_key :: "ivl ⇒ order_key" where
  "ivl_key (Ivl l u) = Key_List [eint_key l, eint_key u]"

definition congruence_key :: "congruence ⇒ order_key" where
  "congruence_key v =
     (case Rep_congruence v of
        None ⇒ Key_List []
      | Some (r, m) ⇒ Key_List [Key_Int r, Key_Int m])"


lemma sign_key_inject [simp]: "sign_key a = sign_key b ⟷ a = b"
  by (cases a; cases b) simp_all

lemma parity_key_inject [simp]: "parity_key a = parity_key b ⟷ a = b"
  by (cases a; cases b) simp_all
```

```
lemma eint_key_inject [simp]: "eint_key a = eint_key b ⟷ a = b"
  by (cases a; cases b) simp_all

lemma ivl_key_inject [simp]: "ivl_key a = ivl_key b ⟷ a = b"
  by (cases a; cases b) simp_all

lemma congruence_key_inject [simp]: "congruence_key a = congruence_key b ⟷ a = b"
  unfolding congruence_key_def
  by (auto simp flip: Rep_congruence_inject split: option.splits)


end
```

17.5 MCP_Split

```
theory MCP_Split
  imports "Voblint_CLI.MCP"
begin
```

The registered analyses with program globals on the shared channel

The keyed placement recombines a point's state with the solved globals and projects a state onto its local half or onto one global. On the combined state these operations are defined field by field, so a carrier that splits is a type class closed under the constructors the combined state is built from. The operations are told which globals the placement owns: an owned global is read from its own unknown, and every other variable, a global no unknown owns included, stays in the point's state. A pointwise field splits by the location each variable is stored at; the order field relates variables across the split and stays wholly local.

```
class split_key = infinite_key +
  fixes key_of_var :: "var_id ⇒ 'a"
  assumes inj_key_of_var: "inj key_of_var"

instantiation var_id :: split_key
begin

definition key_of_var_var_id :: "var_id ⇒ var_id" where "key_of_var_var_id x = x"

instance by standard (simp add: key_of_var_var_id_def inj_def)

end

lemma key_of_var_var_id [simp]: "(key_of_var :: var_id ⇒ var_id) x = x"
  by (simp add: key_of_var_var_id_def)
```

A carrier whose state splits into a chosen list of globals and the rest. `split_cmb gs d g` takes the globals in `gs` from `g` and everything else from `d`; `split_rl gs` forgets the globals in `gs`. `split_free R` is the global half a transfer reads in place of every global outside `R`: it claims nothing about those names and leaves the names in `R` to the values actually read.

```
class ownership_split = order_bot +
  fixes split_cmb :: "var_id list ⇒ 'a ⇒ 'a ⇒ 'a"
    and split_rl :: "var_id list ⇒ 'a ⇒ 'a"
    and split_global_at :: "var_id ⇒ 'a ⇒ 'a"
    and split_free :: "var_id list ⇒ 'a"
  assumes split_cmb_mono: "d ≤ d' ⟹ g ≤ g' ⟹ split_cmb gs d g ≤ split_cmb gs d' g'"
    and split_cmb_rl: "split_cmb gs (split_rl gs x) g = split_cmb gs x g"
    and split_rl_cmb: "split_rl gs (split_cmb gs d g) = split_rl gs d"
    and split_global_at_mono: "d ≤ d' ⟹ split_global_at n d ≤ split_global_at n d'"
    and split_free_ge: "n ∉ set R ⟹ split_global_at n d ≤ split_free R"
    and split_global_at_bot: "split_global_at n bot = bot"

instantiation gdefault_st :: (split_key, "{order_bot, order_top}") ownership_split
begin

definition split_cmb_gdefault_st ::
    "var_id list ⇒ ('a, 'b) gdefault_st ⇒ ('a, 'b) gdefault_st ⇒ ('a, 'b) gdefault_st" where
  "split_cmb_gdefault_st gs d g =
     fold (λx a. a⟨Global_Location (key_of_var x) := g⟨Global_Location (key_of_var x)⟩⟩) gs d"

definition split_rl_gdefault_st :: "var_id list ⇒ ('a, 'b) gdefault_st ⇒ ('a, 'b) gdefault_st" where
  "split_rl_gdefault_st gs d = fold (λx a. a⟨Global_Location (key_of_var x) := bot⟩) gs d"

definition split_global_at_gdefault_st :: "var_id ⇒ ('a, 'b) gdefault_st ⇒ ('a, 'b) gdefault_st"
  where
  "split_global_at_gdefault_st x d =
     bot⟨Global_Location (key_of_var x) := d⟨Global_Location (key_of_var x)⟩⟩"

definition split_free_gdefault_st :: "var_id list ⇒ ('a, 'b) gdefault_st" where
  "split_free_gdefault_st R = ⦇(bot, []), (top, map (λx. (key_of_var x, bot)) R)⦈"

lemma split_cmb_gdefault_st_get:
  "(split_cmb gs d g :: ('a, 'b) gdefault_st)⟨loc⟩ =
     (if loc ∈ (λx. Global_Location (key_of_var x)) ‘ set gs then g⟨loc⟩ else d⟨loc⟩)"
  unfolding split_cmb_gdefault_st_def by (induction gs arbitrary: d) auto

lemma split_rl_gdefault_st_get:
  "(split_rl gs d :: ('a, 'b) gdefault_st)⟨loc⟩ =
     (if loc ∈ (λx. Global_Location (key_of_var x)) ‘ set gs then bot else d⟨loc⟩)"
```

```
  unfolding split_rl_gdefault_st_def by (induction gs arbitrary: d) auto

lemma map_of_map_const_key:
  "map_of (map (λx. (f x, b)) R) k = (if k ∈ f ‘ set R then Some b else None)"
  by (induction R) auto

lemma split_free_gdefault_st_get:
  "(split_free R :: ('a, 'b) gdefault_st)⟨loc⟩ =
     (case loc of Global_Location x ⇒ if x ∈ key_of_var ‘ set R then bot else top | _ ⇒ bot)"
  by (cases loc) (auto simp: split_free_gdefault_st_def map_of_map_const_key)

instance
proof
  fix gs and d d' g g' :: "('a, 'b) gdefault_st"
  assume "d ≤ d'" "g ≤ g'"
  then show "split_cmb gs d g ≤ split_cmb gs d' g'"
    by (auto simp: le_default_st_iff split_cmb_gdefault_st_get)
next
  fix gs and x g :: "('a, 'b) gdefault_st"
  show "split_cmb gs (split_rl gs x) g = split_cmb gs x g"
    by (rule default_st_eqI) (simp add: split_cmb_gdefault_st_get split_rl_gdefault_st_get)
next
  fix gs and d g :: "('a, 'b) gdefault_st"
  show "split_rl gs (split_cmb gs d g) = split_rl gs d"
    by (rule default_st_eqI) (simp add: split_cmb_gdefault_st_get split_rl_gdefault_st_get)
next
  fix n and d d' :: "('a, 'b) gdefault_st"
  assume "d ≤ d'"
  then show "split_global_at n d ≤ split_global_at n d'"
    by (auto simp: le_default_st_iff split_global_at_gdefault_st_def)
next
  fix n :: var_id and R and d :: "('a, 'b) gdefault_st"
  assume "n ∉ set R"
  then have "key_of_var n ∉ key_of_var ‘ set R"
    using inj_key_of_var by (auto dest: injD)
  then show "split_global_at n d ≤ split_free R"
    by (auto simp: le_default_st_iff split_global_at_gdefault_st_def split_free_gdefault_st_get
        split: glocation.split)
next
  fix n :: var_id
  show "split_global_at n (bot :: ('a, 'b) gdefault_st) = bot"
    by (rule default_st_eqI) (simp add: split_global_at_gdefault_st_def)
qed

end

instantiation order_rel :: (type) ownership_split
begin

definition split_cmb_order_rel :: "var_id list ⇒ 'a order_rel ⇒ 'a order_rel ⇒ 'a order_rel" where
  "split_cmb_order_rel gs d g = d"
definition split_rl_order_rel :: "var_id list ⇒ 'a order_rel ⇒ 'a order_rel" where
  "split_rl_order_rel gs d = d"
definition split_global_at_order_rel :: "var_id ⇒ 'a order_rel ⇒ 'a order_rel" where
  "split_global_at_order_rel x d = bot"
definition split_free_order_rel :: "var_id list ⇒ 'a order_rel" where
  "split_free_order_rel R = bot"

instance
  by standard (simp_all add: split_cmb_order_rel_def split_rl_order_rel_def
      split_global_at_order_rel_def split_free_order_rel_def)

end

instantiation analysis_product :: (ownership_split, ownership_split) ownership_split
begin

definition split_cmb_analysis_product ::
  "var_id list ⇒ ('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product"
    where
  "split_cmb_analysis_product gs d g =
     Product (split_cmb gs (pleft d) (pleft g)) (split_cmb gs (pright d) (pright g))"

definition split_rl_analysis_product ::
  "var_id list ⇒ ('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product" where
  "split_rl_analysis_product gs d = Product (split_rl gs (pleft d)) (split_rl gs (pright d))"
```

```
definition split_global_at_analysis_product ::
  "var_id ⇒ ('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product" where
  "split_global_at_analysis_product x d =
     Product (split_global_at x (pleft d)) (split_global_at x (pright d))"

definition split_free_analysis_product :: "var_id list ⇒ ('a, 'b) analysis_product" where
  "split_free_analysis_product R = Product (split_free R) (split_free R)"

instance
proof
  fix gs and d d' g g' :: "('a, 'b) analysis_product"
  assume "d ≤ d'" "g ≤ g'"
  then show "split_cmb gs d g ≤ split_cmb gs d' g'"
    by (simp add: split_cmb_analysis_product_def less_eq_analysis_product_def split_cmb_mono)
next
  fix gs and x g :: "('a, 'b) analysis_product"
  show "split_cmb gs (split_rl gs x) g = split_cmb gs x g"
    by (simp add: split_cmb_analysis_product_def split_rl_analysis_product_def split_cmb_rl)
next
  fix gs and d g :: "('a, 'b) analysis_product"
  show "split_rl gs (split_cmb gs d g) = split_rl gs d"
    by (simp add: split_cmb_analysis_product_def split_rl_analysis_product_def split_rl_cmb)
next
  fix x :: var_id and d d' :: "('a, 'b) analysis_product"
  assume "d ≤ d'"
  then show "split_global_at x d ≤ split_global_at x d'"
    by (simp add: split_global_at_analysis_product_def less_eq_analysis_product_def
      split_global_at_mono)
next
  fix n :: var_id and R and d :: "('a, 'b) analysis_product"
  assume "n ∉ set R"
  then show "split_global_at n d ≤ split_free R"
    by (simp add: split_global_at_analysis_product_def split_free_analysis_product_def
        less_eq_analysis_product_def split_free_ge)
next
  fix n :: var_id
  show "split_global_at n (bot :: ('a, 'b) analysis_product) = bot"
    by (simp add: split_global_at_analysis_product_def bot_analysis_product_def
        split_global_at_bot)
qed

end
```

An unreachable point stays unreachable whatever global it is recombined with, and a reachable point recombined with a global nothing has published yet reads the global half at its bottom.

```
instantiation lifted :: ("{ownership_split, semilattice_sup}") ownership_split
begin

definition split_cmb_lifted :: "var_id list ⇒ 'a lifted ⇒ 'a lifted ⇒ 'a lifted" where
  "split_cmb_lifted gs d g = (case d of Bot ⇒ Bot
     | Lifted a ⇒ Lifted (split_cmb gs a (case g of Bot ⇒ bot | Lifted b ⇒ b)))"

definition split_rl_lifted :: "var_id list ⇒ 'a lifted ⇒ 'a lifted" where
  "split_rl_lifted gs = map_lift (split_rl gs)"

definition split_global_at_lifted :: "var_id ⇒ 'a lifted ⇒ 'a lifted" where
  "split_global_at_lifted x = map_lift (split_global_at x)"

definition split_free_lifted :: "var_id list ⇒ 'a lifted" where
  "split_free_lifted R = Lifted (split_free R)"

instance
proof
  fix gs and d d' g g' :: "'a lifted"
  assume "d ≤ d'" "g ≤ g'"
  then show "split_cmb gs d g ≤ split_cmb gs d' g'"
    by (cases d; cases d'; cases g; cases g')
       (auto simp: split_cmb_lifted_def intro: split_cmb_mono)
next
  fix gs and x g :: "'a lifted"
  show "split_cmb gs (split_rl gs x) g = split_cmb gs x g"
    by (cases x) (simp_all add: split_cmb_lifted_def split_rl_lifted_def split_cmb_rl)
next
  fix gs and d g :: "'a lifted"
  show "split_rl gs (split_cmb gs d g) = split_rl gs d"
```

```
    by (cases d) (simp_all add: split_cmb_lifted_def split_rl_lifted_def split_rl_cmb)
next
  fix x :: var_id and d d' :: "'a lifted"
  assume "d ≤ d'"
  then show "split_global_at x d ≤ split_global_at x d'"
    by (cases d; cases d') (simp_all add: split_global_at_lifted_def split_global_at_mono)
next
  fix n :: var_id and R and d :: "'a lifted"
  assume "n ∉ set R"
  then show "split_global_at n d ≤ split_free R"
    by (cases d) (simp_all add: split_global_at_lifted_def split_free_lifted_def split_free_ge)
next
  fix n :: var_id
  show "split_global_at n (bot :: 'a lifted) = bot"
    by (simp add: split_global_at_lifted_def)
qed

end

lemma split_cmb_Bot [simp]:
  "split_cmb gs (Bot :: 'a::{ownership_split, semilattice_sup} lifted) g = Bot"
  by (simp add: split_cmb_lifted_def)
```

Reading a keyed recombination back

A keyed point's state is its own state recombined with the solved environment over the owned globals. Read back, every owned global comes from that global's own unknown and every other variable from the point. The lemmas below establish this for one field and lift it through the fields of the combined state.

```
lemma view_of_default_st_get:
  "(view_of split_global_at xs E acc
      :: (var_id, 'a::{bounded_semilattice_sup_bot, order_top}) gdefault_st)⟨loc⟩
     = acc⟨loc⟩ ⊔ (case loc of Global_Location y ⇒ if y ∈ set xs then (E y)⟨loc⟩ else bot
                             | Local_Location y ⇒ bot)"
  unfolding view_of_def
  by (induction xs arbitrary: acc)
     (auto simp: split_global_at_gdefault_st_def ac_simps split: glocation.split)
```

The value a lifted state carries, `bot` when unreachable.

```
fun lift_val :: "'a::bot lifted ⇒ 'a" where
  "lift_val Bot = bot"
| "lift_val (Lifted a) = a"

lemma lift_val_sup:
  "lift_val (a ⊔ b) = lift_val a ⊔ lift_val (b :: 'a::bounded_semilattice_sup_bot lifted)"
  by (cases a; cases b) simp_all

lemma split_cmb_Lifted:
  "split_cmb gs (Lifted a) g
     = Lifted (split_cmb gs a (lift_val (g :: 'a::{ownership_split, semilattice_sup} lifted)))"
  by (cases g) (simp_all add: split_cmb_lifted_def)

lemma lift_val_view_of:
  "lift_val (view_of split_global_at xs E acc
       :: 'a::{ownership_split, bounded_semilattice_sup_bot} lifted)
     = view_of split_global_at xs (λy. lift_val (E y)) (lift_val acc)"
  unfolding view_of_def
proof (induction xs arbitrary: acc)
  case Nil then show ?case by simp
next
  case (Cons x xs)
  have "lift_val (split_global_at x (E x)) = split_global_at x (lift_val (E x))"
    by (cases "E x") (simp_all add: split_global_at_lifted_def split_global_at_bot)
  with Cons show ?case by (simp add: lift_val_sup)
qed
```

A map between carriers that commutes with the split operations, such as the projection onto one field of a product. Read-back facts proved for a field lift along it.

```
locale split_hom =
  fixes f :: "'a::{ownership_split, bounded_semilattice_sup_bot}
              ⇒ 'b::{ownership_split, bounded_semilattice_sup_bot}"
  assumes hom_cmb: "f (split_cmb gs a b) = split_cmb gs (f a) (f b)"
    and hom_rl: "f (split_rl gs a) = split_rl gs (f a)"
    and hom_at: "f (split_global_at x a) = split_global_at x (f a)"
```

```
    and hom_sup: "f (a ⊔ b) = f a ⊔ f b"
    and hom_bot: "f bot = bot"
begin

lemma hom_view_of:
  "f (view_of split_global_at xs E acc) = view_of split_global_at xs (λy. f (E y)) (f acc)"
  unfolding view_of_def by (induction xs arbitrary: acc) (simp_all add: hom_sup hom_at)

lemma lift_get_hom: "lift_get f x = f (lift_val x)"
  by (cases x) (simp_all add: hom_bot lift_get_def)

end
```

The read-back of one field of a keyed recombination: an owned global reads from the field of that global's unknown, every other variable from the point.

```
lemma keyed_field_read:
  fixes f :: "'a::{ownership_split, bounded_semilattice_sup_bot}
              ⇒ 'c::{ownership_split, bounded_semilattice_sup_bot} lifted"
    and E :: "var_id ⇒ 'a lifted"
  assumes hom: "split_hom f" and fa: "f a = Lifted st"
  shows "lift_get f (split_cmb gs (Lifted a) (full_view split_global_at xs E))
    = Lifted (split_cmb gs st (view_of split_global_at xs (λy. lift_val (lift_get f (E y))) bot))"
proof -
  interpret split_hom f by (rule hom)
  have "lift_get f (split_cmb gs (Lifted a) (full_view split_global_at xs E))
      = split_cmb gs (f a) (view_of split_global_at xs (λy. lift_get f (E y)) bot)"
    by (simp add: split_cmb_Lifted full_view_def hom_cmb hom_view_of hom_bot lift_val_view_of
        lift_get_hom)
  then show ?thesis
    using fa by (simp add: split_cmb_Lifted lift_val_view_of)
qed
```

A typed field of a keyed recombination describes a store when every owned global lies in what its unknown publishes and every other variable in what the point holds.

```
lemma keyed_field_gamma:
  fixes f :: "'a::{ownership_split, bounded_semilattice_sup_bot} ⇒ 'c::kind_int_domain tst lifted"
    and E :: "var_id ⇒ 'a lifted"
  assumes hom: "split_hom f" and fa: "f a = Lifted st"
  shows "s ∈ gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global)
            (lift_get f (split_cmb gs (Lifted a) (full_view split_global_at gs E))))
    ⟷ in_kinds κ s ∧ (∀y. s y ∈ gamma_kd (if is_global y ∧ y ∈ set gs
                            then (lift_val (lift_get f (E y)))⟨Global_Location y⟩
                            else st⟨nonrel_loc y⟩))"
  by (auto simp: keyed_field_read[OF hom fa] gamma_tabs_def default_st_to_fun_def location_of_def
      split_cmb_gdefault_st_get view_of_default_st_get image_iff)
```

The frame law for one field: a store the transfer's result describes, agreeing outside the written globals with a store the point described, is described by the result's own variables and written globals together with the environment elsewhere.

```
lemma keyed_field_mix:
  fixes f :: "'a::{ownership_split, bounded_semilattice_sup_bot} ⇒ 'c::kind_int_domain tst lifted"
  assumes hom: "split_hom f"
    and s: "s ∈ gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global)
              (lift_get f (split_cmb gs l0 (full_view split_global_at gs e))))"
    and s': "s' ∈ gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get f r))"
    and fr: "∀x. is_global x ⟶ x ∉ set W ⟶ s' x = s x"
  shows "s' ∈ gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global)
          (lift_get f (split_cmb gs (split_rl gs r)
             (full_view split_global_at gs (λx. if x ∈ set W then split_global_at x r else e x)))))"
proof -
  interpret split_hom f by (rule hom)
  obtain ri where r: "r = Lifted ri" using s' by (cases r) (simp_all add: lift_get_def)
  obtain stR where fr_i: "f ri = Lifted stR" using s' r
    by (cases "f ri") (simp_all add: lift_get_def)
  obtain a0 where l0: "l0 = Lifted a0" using s by (cases l0) (simp_all add: lift_get_def)
  obtain st0 where fa0: "f a0 = Lifted st0"
    using s l0 by (cases "f a0") (simp_all add: split_cmb_Lifted hom_cmb lift_get_def)
  have rl: "split_rl gs r = Lifted (split_rl gs ri)" by (simp add: r split_rl_lifted_def)
  have frl: "f (split_rl gs ri) = Lifted (split_rl gs stR)"
    by (simp add: hom_rl fr_i split_rl_lifted_def)
  have s'R: "in_kinds κ s'" "⋀y. s' y ∈ gamma_kd (stR⟨nonrel_loc y⟩)"
    using s' by (simp_all add: r fr_i lift_get_def gamma_tabs_def default_st_to_fun_def)
  have s0: "⋀y. s y ∈ gamma_kd (if is_global y ∧ y ∈ set gs
              then (lift_val (lift_get f (e y)))⟨Global_Location y⟩ else st0⟨nonrel_loc y⟩)"
```

```
    using s by (simp add: l0 keyed_field_gamma[OF hom fa0])
  show ?thesis
    unfolding rl keyed_field_gamma[OF hom fr1]
  proof (intro conjI allI)
    show "in_kinds κ s'" by (rule s'R(1))
  next
    fix y
    show "s' y ∈ gamma_kd (if is_global y ∧ y ∈ set gs
        then (lift_val (lift_get f (if y ∈ set W then split_global_at y r else e y)))⟨Global_Location y⟩
        else (split_rl gs stR)⟨nonrel_loc y⟩)"
    proof (cases "is_global y ∧ y ∈ set gs")
      case False
      then show ?thesis using s'R(2)[of y]
        by (auto simp: location_of_def split_rl_gdefault_st_get image_iff)
    next
      case owned: True
      show ?thesis
      proof (cases "y ∈ set W")
        case True
        then show ?thesis using owned s'R(2)[of y]
          by (simp add: location_of_def r split_global_at_lifted_def hom_at fr_i lift_get_def
              split_global_at_gdefault_st_def)
      next
        case False
        then show ?thesis using owned s0[of y] fr by simp
      qed
    qed
  qed
qed
```

A state is described by itself recombined with its own cuts.

```
lemma keyed_field_init:
  fixes f :: "'a::{ownership_split, bounded_semilattice_sup_bot} ⇒ 'c::kind_int_domain tst lifted"
  assumes hom: "split_hom f"
  shows "gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get f (Lifted i)))
    ⊆ gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get f (split_cmb gs (Lifted i)
        (full_view split_global_at gs (λx. split_global_at x (Lifted i))))))"
proof
  interpret split_hom f by (rule hom)
  fix s
  assume s:
    "s ∈ gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get f (Lifted i)))"
  then obtain st where fi: "f i = Lifted st" by (cases "f i") (simp_all add: lift_get_def)
  show "s ∈ gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get f (split_cmb gs (Lifted i)
      (full_view split_global_at gs (λx. split_global_at x (Lifted i))))))"
    using s unfolding keyed_field_gamma[OF hom fi]
    by (auto simp: fi lift_get_def gamma_tabs_def default_st_to_fun_def location_of_def
        split_global_at_lifted_def hom_at split_global_at_gdefault_st_def) (metis (full_types))
qed
```

Each numeric field of the combined state is reached through a slot that commutes with the split operations, because they are defined field by field.

```
lemma split_hom_pleft: "split_hom (pleft :: ('a::{ownership_split, bounded_semilattice_sup_bot},
    'b::{ownership_split, bounded_semilattice_sup_bot}) analysis_product ⇒ 'a)"
  by unfold_locales
     (simp_all add: split_cmb_analysis_product_def split_rl_analysis_product_def
       split_global_at_analysis_product_def sup_analysis_product_def bot_analysis_product_def)

lemma split_hom_pright: "split_hom (pright :: ('a::{ownership_split, bounded_semilattice_sup_bot},
    'b::{ownership_split, bounded_semilattice_sup_bot}) analysis_product ⇒ 'b)"
  by unfold_locales
     (simp_all add: split_cmb_analysis_product_def split_rl_analysis_product_def
       split_global_at_analysis_product_def sup_analysis_product_def bot_analysis_product_def)

lemma split_hom_comp: "split_hom f ⟹ split_hom g ⟹ split_hom (f ∘ g)"
  unfolding split_hom_def by simp

lemma split_hom_tslots:
  "split_hom (slot1 :: mcp_st ⇒ _)" "split_hom (slot2 :: mcp_st ⇒ _)"
  "split_hom (slot3n :: mcp_st ⇒ _)" "split_hom (slot3o :: mcp_st ⇒ _)"
  "split_hom (slot3f :: mcp_st ⇒ _)" "split_hom (slot4 :: mcp_st ⇒ _)"
  "split_hom (slot5 :: mcp_st ⇒ _)"
proof -
  note h = split_hom_comp split_hom_pleft split_hom_pright
  have eqs: "(slot1 :: mcp_st ⇒ _) = pleft" "(slot2 :: mcp_st ⇒ _) = pleft ∘ pright"
```

```
    "(slot3n :: mcp_st ⇒ _) = pleft ∘ pleft ∘ pright ∘ pright"
    "(slot3o :: mcp_st ⇒ _) = pleft ∘ pright ∘ pleft ∘ pright ∘ pright"
    "(slot3f :: mcp_st ⇒ _) = pright ∘ pright ∘ pleft ∘ pright ∘ pright"
    "(slot4 :: mcp_st ⇒ _) = pleft ∘ pright ∘ pright ∘ pright"
    "(slot5 :: mcp_st ⇒ _) = pleft ∘ pright ∘ pright ∘ pright ∘ pright"
    by (simp_all add: fun_eq_iff)
  show "split_hom (slot1 :: mcp_st ⇒ _)" "split_hom (slot2 :: mcp_st ⇒ _)"
    "split_hom (slot3n :: mcp_st ⇒ _)" "split_hom (slot3o :: mcp_st ⇒ _)"
    "split_hom (slot3f :: mcp_st ⇒ _)" "split_hom (slot4 :: mcp_st ⇒ _)"
    "split_hom (slot5 :: mcp_st ⇒ _)"
    unfolding eqs by (intro h)+
qed
```

The order field keeps its whole state at the point, so recombining leaves it alone.

```
lemma split_order_lifted [simp]:
  "split_cmb gs (d :: 'v order_rel lifted) g = d" "split_rl gs (d :: 'v order_rel lifted) = d"
  by (cases d; simp add: split_cmb_lifted_def split_cmb_order_rel_def split_rl_lifted_def
      split_rl_order_rel_def)+

lemma slot6_keyed:
  "lift_get slot6 (split_cmb gs (split_rl gs r) g) = lift_get slot6 (r :: mcp_st lifted)"
  by (cases r) (simp_all add: split_cmb_Lifted split_rl_lifted_def split_cmb_analysis_product_def
      split_rl_analysis_product_def lift_get_def split_rl_order_rel_def map_lift_def
      split: lifted.split)

lemma slot6_keyed_init: "lift_get slot6 (split_cmb gs (Lifted i) g) = slot6 (i :: mcp_st)"
  by (simp add: split_cmb_Lifted split_cmb_analysis_product_def lift_get_def)
```

Each analysis reads its own field of the combined state.

```
lemma val_gamma_slot:
  "gamma_lift (val_gamma κ Interval) (map_lift (mcp_rd is_global) x)
     = gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get slot1 x))"
  "gamma_lift (val_gamma κ Congruence) (map_lift (mcp_rd is_global) x)
     = gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get slot2 x))"
  "gamma_lift (val_gamma κ (Ana_Int Refine_Never)) (map_lift (mcp_rd is_global) x)
     = gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get slot3n x))"
  "gamma_lift (val_gamma κ (Ana_Int Refine_Once)) (map_lift (mcp_rd is_global) x)
     = gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get slot3o x))"
  "gamma_lift (val_gamma κ (Ana_Int Refine_Fixpoint)) (map_lift (mcp_rd is_global) x)
     = gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get slot3f x))"
  "gamma_lift (val_gamma κ Sign) (map_lift (mcp_rd is_global) x)
     = gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get slot4 x))"
  "gamma_lift (val_gamma κ Parity) (map_lift (mcp_rd is_global) x)
     = gamma_lift (gamma_tabs κ) (map_lift (default_st_to_fun is_global) (lift_get slot5 x))"
  "gamma_lift (val_gamma κ Order) (map_lift (mcp_rd is_global) x)
     = gamma_ord_lift κ (lift_get slot6 x)"
  by (cases x; simp add: mcp_rd_def lift_get_def)+

lemma part_gamma_keyed_mix:
  assumes s: "s ∈ gamma_lift (val_gamma κ a) (map_lift (mcp_rd is_global)
                (split_cmb gs l0 (full_view split_global_at gs e)))"
    and s': "s' ∈ gamma_lift (val_gamma κ a) (map_lift (mcp_rd is_global) r)"
    and fr: "∀x. is_global x ⟶ x ∉ set W ⟶ s' x = s x"
  shows "s' ∈ gamma_lift (val_gamma κ a) (map_lift (mcp_rd is_global) (split_cmb gs (split_rl gs r)
           (full_view split_global_at gs (λx. if x ∈ set W then split_global_at x r else e x))))"
  using s s'
  by (cases a rule: analysis_cases)
     (simp_all only: val_gamma_slot slot6_keyed,
      (rule keyed_field_mix[OF split_hom_tslots(1) _ _ fr]
        keyed_field_mix[OF split_hom_tslots(2) _ _ fr]
        keyed_field_mix[OF split_hom_tslots(3) _ _ fr]
        keyed_field_mix[OF split_hom_tslots(4) _ _ fr]
        keyed_field_mix[OF split_hom_tslots(5) _ _ fr]
        keyed_field_mix[OF split_hom_tslots(6) _ _ fr]
        keyed_field_mix[OF split_hom_tslots(7) _ _ fr]; assumption)+)

lemma part_gamma_keyed_init:
  "gamma_lift (val_gamma κ a) (map_lift (mcp_rd is_global) (Lifted i))
     ⊆ gamma_lift (val_gamma κ a) (map_lift (mcp_rd is_global) (split_cmb gs (Lifted i)
         (full_view split_global_at gs (λx. split_global_at x (Lifted i)))))"
  by (cases a rule: analysis_cases)
     (simp_all only: val_gamma_slot slot6_keyed_init,
      (rule keyed_field_init[OF split_hom_tslots(1)] keyed_field_init[OF split_hom_tslots(2)]
        keyed_field_init[OF split_hom_tslots(3)] keyed_field_init[OF split_hom_tslots(4)]
        keyed_field_init[OF split_hom_tslots(5)] keyed_field_init[OF split_hom_tslots(6)]
        keyed_field_init[OF split_hom_tslots(7)] | simp add: lift_get_def)+)
```

The registration

The keyed placement: each owned global at its own unknown `Analysis_Global x`, read and published by name. A run owns the program's declared globals. The combined analysis owes it what it owes the whole-state placement, plus the carrier laws above.

```
definition declared_globals :: "imp_prog ⇒ var_id list" where
  "declared_globals p = map GlobalId (declared_global_vars p)"

lemma mcp_cs_keyed:
  "dg_analysis
    (kenv_of ds) (λ_. init_stores (kenv_of ds))
    (mcp_comp sem ds (activation_of as)) (mcp_emp (activation_of as)) mcp_rd
    (mcp_init ds (activation_of as))
    Analysis_Buffer Analysis_Global Activation_Seed
    (TD_side_rule_Interp_solve r)
    (TD_side_rule_Interp.solve_dom TYPE((var_id, 'c) global_unknown)
       TYPE((mcp_st lifted, mcp_st lifted) dg_state) r)
    ⊥ (mcp_classify sem ds (activation_of as)) (mcp.act_gamma ds (activation_of as))
    (mcp_empty_v (activation_of as))
    (TD_side_rule_Interp_solve_c r)
    (λp c. keyed_split_spec is_global (split_cmb gs) (split_rl gs)
       split_global_at split_free c)
    (λp d e. split_cmb gs d (full_view split_global_at gs e)) (λ𝒢. split_rl gs) (λ𝒢 d. Bot)
    (λ𝒢 p. map (λx. (x, split_global_at x (Lifted (mcp_init ds (activation_of as))))) gs)
    (λp ci d e. split_cmb gs d (view_of split_global_at (call_global_reads is_global (ci_args ci))
       e (split_free (call_global_reads is_global (ci_args ci)))))
    (λp. elab_cfg p ds) (λ_. FunctionEntry prog_main_name)"
proof (rule dg_analysis_keyedI[OF td_certified_solver], goal_cases CompSound EnterSingle CmbMono
    CmbRl RlCmb CmbBot RgMono RgFree Mix InitView EmptyRd EmptyV SeedNe SeedNeGlobal ClProved
    ClRefuted BotState InitSound PEntry FinIntra FinCalls CallsUnique)
  case (CompSound p)
  have eq: "mcp_gamma (map (mcp.part_gamma ds is_global) (activation_of as))
     = (λd. gamma_lift (mcp.act_gamma ds (activation_of as)) (map_lift (mcp_rd is_global) d))"
    by (rule ext) (rule mcp.mcp_gamma_rd[OF activation_of_ne])
  show ?case
    using mcp.mcp_comp_sound[OF distinct_activation_of activation_of_ne, where sem = sem and ds = ds
       and p = p, of as]
    unfolding eq by simp
next
  case (EnterSingle p ci d)
  show ?case
    using single_entryD[OF mcp.single_entry_mcp_comp[OF activation_of_ne],
        of sem ds as is_global p
          "ls_channel (mcp_comp sem ds (activation_of as) is_global p) d" ci "(d, d)"]
    by (simp add: dg_pipeline.comp_entry_def)
next
  case (CmbMono d d' g g') then show ?case by (rule split_cmb_mono)
next
  case (CmbRl x g) show ?case by (rule split_cmb_rl)
next
  case (RlCmb d g) show ?case by (rule split_rl_cmb)
next
  case (CmbBot g) show ?case by simp
next
  case (RgMono x v v') then show ?case by (rule split_global_at_mono)
next
  case (RgFree x R v) then show ?case by (rule split_free_ge)
next
  case (Mix p l0 e r s s' W)
  note eq = mcp.mcp_gamma_rd[OF activation_of_ne, symmetric]
  show ?case
    using Mix unfolding eq mcp_gamma_def by (auto intro: part_gamma_keyed_mix)
next
  case (InitView p)
  note eq = mcp.mcp_gamma_rd[OF activation_of_ne, symmetric]
  show ?case
    unfolding eq mcp_gamma_def using part_gamma_keyed_init by fastforce
next
  case (EmptyRd p s) show ?case by (rule mcp.mcp_emp_rd)
next
  case EmptyV show ?case by (rule mcp.sound_emptiness_mcp)
next
  case (SeedNe v ctx) show ?case by simp
next
  case (SeedNeGlobal v ctx n) show ?case by simp
```

```
next
  case (ClProved c d s) then show ?case by (intro mcp.mcp_classify_proved)
next
  case (ClRefuted c d s) then show ?case by (intro mcp.mcp_classify_refuted)
next
  case BotState show ?case by (rule mcp.act_gamma_bot[OF activation_of_ne])
next
  case (InitSound p) show ?case by (rule mcp_init_sound)
next
  case (PEntry p) show ?case by (simp add: elab_cfg_def)
next
  case (FinIntra p) show ?case by (simp add: elab_cfg_def compile_prog_finite)
next
  case (FinCalls p) show ?case by (simp add: elab_cfg_def compile_prog_finite)
next
  case (CallsUnique p) show ?case by (rule elab_cfg_calls_source_unique)
qed
```

The two context policies again, with each owned global at its own unknown.

```
global_interpretation mcp_split: dg_analysis
    "kenv_of ds" "λ_. init_stores (kenv_of ds)"
    "mcp_comp sem ds (activation_of as)" "mcp_emp (activation_of as)" mcp_rd
    "mcp_init ds (activation_of as)"
    Analysis_Buffer Analysis_Global Activation_Seed "λ_. cs_route k" "[]"
    "TD_side_rule_Interp_solve r"
    "TD_side_rule_Interp.solve_dom TYPE((var_id, cfg_node list) global_unknown)
       TYPE((mcp_st lifted, mcp_st lifted) dg_state) r"
    ⊥ "mcp_classify sem ds (activation_of as)" "mcp.act_gamma ds (activation_of as)"
    "mcp_empty_v (activation_of as)" "TD_side_rule_Interp_solve_c r"
    "λp c. keyed_split_spec is_global (split_cmb gs) (split_rl gs)
       split_global_at split_free c"
    "λp d e. split_cmb gs d (full_view split_global_at gs e)" "λ𝒢. split_rl gs" "λ𝒢 d. Bot"
    "λ𝒢 p. map (λx. (x, split_global_at x (Lifted (mcp_init ds (activation_of as))))) gs"
    "λp ci d e. split_cmb gs d (view_of split_global_at (call_global_reads is_global (ci_args ci))
       e (split_free (call_global_reads is_global (ci_args ci))))"
    "λp. elab_cfg p ds" "λ_. FunctionEntry prog_main_name"
  for sem ds as r k gs
  by (rule mcp_cs_keyed)

global_interpretation mcp_split_es: dg_analysis
    "kenv_of ds" "λ_. init_stores (kenv_of ds)"
    "mcp_comp sem ds (activation_of as)" "mcp_emp (activation_of as)" mcp_rd
    "mcp_init ds (activation_of as)"
    Analysis_Buffer Analysis_Global Activation_Seed "mcp_route (activation_of as)" mcp_root_ctx
    "TD_side_rule_Interp_solve r"
    "TD_side_rule_Interp.solve_dom TYPE((var_id, mcp_ctx) global_unknown)
       TYPE((mcp_st lifted, mcp_st lifted) dg_state) r"
    ⊥ "mcp_classify sem ds (activation_of as)" "mcp.act_gamma ds (activation_of as)"
    "mcp_empty_v (activation_of as)" "TD_side_rule_Interp_solve_c r"
    "λp c. keyed_split_spec is_global (split_cmb gs) (split_rl gs)
       split_global_at split_free c"
    "λp d e. split_cmb gs d (full_view split_global_at gs e)" "λ𝒢. split_rl gs" "λ𝒢 d. Bot"
    "λ𝒢 p. map (λx. (x, split_global_at x (Lifted (mcp_init ds (activation_of as))))) gs"
    "λp ci d e. split_cmb gs d (view_of split_global_at (call_global_reads is_global (ci_args ci))
       e (split_free (call_global_reads is_global (ci_args ci))))"
    "λp. elab_cfg p ds" "λ_. FunctionEntry prog_main_name"
  for sem ds as r gs
  by (rule mcp_cs_keyed)
```

Every store that reaches a node lies in a state the solve files there, as for the whole-state placement (`mcp_cs_node_sound`).

```
lemma mcp_split_node_sound:
  assumes wf: "wf_program_compile_input p"
    and solves: "mcp_split.terminates sem ds as k r gs is_global p"
  shows "reaching ds p v ⊆ (⋃ctx. mcp.act_gamma ds (activation_of as)
           (mcp_split.state_at sem ds as k r gs is_global p ctx v))"
  using mcp_split.fun_route_result_node_sound
      [OF elab_cfg_def _ elab_wf_compile_input[OF wf] solves, of "cs_context k" v]
  by (simp add: cs_route_def cs_context_def)

lemma mcp_split_es_node_sound:
  assumes wf: "wf_program_compile_input p"
    and solves: "mcp_split_es.terminates sem ds as r gs is_global p"
  shows "reaching ds p v ⊆ (⋃ctx. mcp.act_gamma ds (activation_of as)
           (mcp_split_es.state_at sem ds as r gs is_global p ctx v))"
  using mcp_split_es.entry_state_result_node_sound
```

```
    [OF elab_cfg_def elab_wf_compile_input[OF wf] solves, of v]
  by simp

end
```

17.6 Analysis_Run

```
theory Analysis_Run
  imports
    Arithmetic_Diagnostics
    Dispatch_Carrier
    MCP_Split
    "HOL-Library.Code_Target_Numeral"
    "HOL-Library.Code_Abstract_Char"
begin
```

The report of one run

A solve is handed back as a report: every covered point-and-context state, the contexts each call enters, the check column and the arithmetic diagnostics. One solve stands behind all of them. The report holds semantic states, never their rendering; text, graphs and report pages are built from it by a projection outside the theorems.

The rows of a report

Points are CFG nodes and contexts are indices into the report's context list, so a consumer never has to infer which states belong together from displayed text. A row's state is the type parameter `'s`: the semantic state in a report, its rendering after a presentation has been applied.

A route lists every callee context a call enters from one caller context: an entry specification may offer several alternatives, and each may land in its own context. The empty list is a call the caller context does not take.

A state also lists, for every edge leaving its point, that edge's target and what the edge's own step makes of the state. Where the target is a join of several incoming edges – a loop head, the point after a branch – this is the only place the effect of one statement survives.

```
record ('e, 's) result_state =
  state_point :: pp
  state_context :: nat
  state_value :: "'s lifted"
  state_checks :: "('e × contextual_verdict) list"
  state_diagnostics :: "('e arithmetic_obligation × contextual_verdict) list"
  state_steps :: "(pp × 's lifted) list"

record call_route =
  route_point :: pp
  route_context :: nat
  route_callee :: pname
  route_targets :: "nat list"

record 'e result_check =
  check_point :: pp
  check_label :: check_label
  check_exp :: 'e
  check_verdict :: contextual_verdict
```

A global unknown is a program global kept at its own unknown, or the seed of a procedure entry, named by the procedure and the index of the context it was entered at; a seed without a context belongs to a procedure the solve never entered.

```
datatype 'n result_global_unknown =
    Global_Named 'n
  | Global_Seed pname "nat option"

record ('n, 's) result_global =
  global_unknown :: "'n result_global_unknown"
  global_state :: "'s lifted"

definition map_result_state :: "('s ⇒ 't) ⇒ ('e, 's) result_state ⇒ ('e, 't) result_state" where
  "map_result_state f st =
     (| state_point = state_point st,
       state_context = state_context st,
       state_value = map_lift f (state_value st),
       state_checks = state_checks st,
       state_diagnostics = state_diagnostics st,
       state_steps = map (λ(w, s). (w, map_lift f s)) (state_steps st) |)"

definition map_result_global :: "('s ⇒ 't) ⇒ ('n, 's) result_global ⇒ ('n, 't) result_global" where
  "map_result_global f g =
     (| global_unknown = global_unknown g,
       global_state = map_lift f (global_state g) |)"
```

The report

A context is kept as the policy's own value, tagged by policy, so one report type serves every policy. The report also keeps the configuration, which names the concretization its states are read through, and the program's variables, which a rendering lists. The configuration, the context tag, the variables, the expressions, the global names and the states are parameters, so one report and one builder serve every language an analysis runs on.

```
datatype 'm report_context =
    Report_Unit
  | Report_Entry 'm
  | Report_Call_String "pp list"

record ('k, 'x, 'vr, 'e, 'n, 's) analysis_report =
  report_config :: 'k
  report_vars :: "'vr list"
  report_cfg :: "('vr, 'e) gcfg"
  report_contexts :: "'x list"
  report_states :: "('e, 's) result_state list"
  report_routes :: "call_route list"
  report_checks :: "'e result_check list"
  report_globals :: "('n, 's) result_global list"
  report_diagnostics :: "'e arithmetic_diagnostic list"
```

One builder for every context policy

A report is built the same way whatever the context policy: list the contexts the table covers, file every covered `(point, context)` state under its context's index, ask the run which context each live call enters, and take the check and diagnostic columns off the same table. The policy contributes only how its contexts are ordered (`ctx_key`, injective) and tagged (`ctx_tag`).

Points are listed in graph order. A covered point outside the graph's node list is listed after them, so the rows hold every covered key without a separate argument about which points the solver visits.

```
fun callee_of_entry :: "cfg_node ⇒ pname" where
  "callee_of_entry (FunctionEntry p) = p"
| "callee_of_entry _ = STR ''''"

definition context_indices :: "(nat × 'c) list ⇒ 'c ⇒ nat list" where
  "context_indices indexed ctx = map fst (filter (λ(i, ctx'). ctx' = ctx) indexed)"
```

Contexts are listed in the order of an injective key, which picks each context back out of the key set by `the_elem`. HOL's own code equation for that matches only a literal one-element list, but a set built by an image or a filter is backed by a list that may repeat its element — the contexts of a context-free table are one `()` per point — and the generated code then fails with a match error. Deduplicating first is the same set.

```
lemma the_elem_set_remdups [code]:
  "the_elem (set xs) =
     (case remdups xs of
        [x] ⇒ x
      | _ ⇒ Code.abort (STR ''the_elem: not a singleton'') (λ_. the_elem (set xs)))"
proof (cases "remdups xs")
  case Nil
  then show ?thesis by simp
next
  case (Cons y ys)
  then show ?thesis
  proof (cases ys)
    case Nil
    with Cons have "set xs = {y}" by (metis set_remdups empty_set list.simps(15))
    with Cons Nil show ?thesis by simp
  next
    case (Cons z zs)
    with ‹remdups xs = y # ys› show ?thesis by simp
  qed
qed

definition ordered_by_key ::
  "('a ⇒ 'k::linorder) ⇒ 'a set ⇒ 'a list" where
  "ordered_by_key rank S =
    map
      (λk. the_elem (Set.filter (λx. rank x = k) S))
      (sorted_list_of_set (rank ‘ S))"
```

The check column: one row per compiled check edge, carrying the edge's label beside its node, its condition, and the verdict `classify_checks_verdicts` computes there. The label is what a consumer finds a source check's

row by.

```
definition result_checks_of ::
    "('vr::linorder, 'e::linorder) gcfg ⇒ ('ctx, 'a) solved_table
       ⇒ ('e ⇒ 'a ⇒ check_result) ⇒ 'e result_check list"
where
  "result_checks_of g r classify =
     map (λ(u, a, w).
           ⦇ check_point = u, check_label = ea_check_label a, check_exp = ea_check_cond a,
             check_verdict = point_verdict r classify u (ea_check_cond a) ⦈)
       (filter (λ(u, a, w). is_EA_Check a) (cfg_intra_list g))"
```

Which callee contexts a call enters from a caller context: the one the solve itself routed it to, or none when the entered state is unreachable, which is exactly when the solver's call tree skips the seed publication.

```
definition live_targets ::
    "(pp ⇒ 'c ⇒ ('vr, 'e) gcall_action ⇒ pname ⇒ 'c option)
       ⇒ pp ⇒ 'c ⇒ ('vr, 'e) gcall_action ⇒ pname ⇒ 'c list" where
  "live_targets succ u ctx ca q = (case succ u ctx ca q of None ⇒ [] | Some ctx' ⇒ [ctx'])"
```

The globals column lists the global unknowns of the constraint system: one unknown per global name the placement keeps apart, then for every procedure the seed of each context its entry was solved at. A procedure entered at no context is listed once, unreachable. Globals the placement keeps in every point's own state have no unknown of their own.

One row: the state the table filed at a point under a context, with the verdicts of the checks and arithmetic obligations there and the steps of the edges leaving it. The check, obligation and step lists come grouped by point, built once per report.

```
definition report_row ::
    "('c, 'n, 's) solved_run ⇒ (exp ⇒ 's ⇒ check_result)
       ⇒ (exp arithmetic_obligation ⇒ 's ⇒ check_result)
       ⇒ (pp, exp list) rbt ⇒ (pp, exp arithmetic_obligation list list) rbt
       ⇒ (pp, ((var_id, exp) gedge_action × pp) list) rbt ⇒ nat ⇒ 'c ⇒ pp
       ⇒ (exp, 's) result_state" where
  "report_row sr classify classify_ob conds obls edges i ctx v =
     (let state = lookup_table (run_table sr) v ctx
      in ⦇ state_point = v, state_context = i,
           state_value = state,
           state_checks =
             map (λc. (c, classify_point classify c state)) (group_lookup conds v),
           state_diagnostics =
             map (λobligation. (obligation, classify_point classify_ob obligation state))
               (concat (group_lookup obls v)),
           state_steps = map (λ(a, w). (w, run_step sr v ctx a)) (group_lookup edges v) ⦈)"

lemma report_row_simps [simp]:
  "state_point (report_row sr classify classify_ob conds obls edges i ctx v) = v"
  "state_context (report_row sr classify classify_ob conds obls edges i ctx v) = i"
  "state_value (report_row sr classify classify_ob conds obls edges i ctx v)
     = lookup_table (run_table sr) v ctx"
  by (simp_all add: report_row_def Let_def)
```

The builder reads the program only through what it is handed: its variables, its graph, the global names kept at their own unknowns, the arithmetic obligation sites with the classifier that decides one, and its procedures.

```
definition report_of ::
    "'k ⇒ var_id list ⇒ (var_id, exp) gcfg ⇒ 'n list
       ⇒ (pp × exp arithmetic_obligation list) list
       ⇒ (exp arithmetic_obligation ⇒ 's ⇒ check_result)
       ⇒ pname list ⇒ ('c ⇒ order_key) ⇒ ('c ⇒ 'x)
       ⇒ (exp ⇒ 's ⇒ check_result) ⇒ ('c, 'n, 's) solved_run
       ⇒ ('k, 'x, var_id, exp, 'n, 's) analysis_report" where
  "report_of config vars g globals sites classify_ob procs ctx_key ctx_tag classify sr =
     (let r = run_table sr;
          ctxs = ordered_by_key ctx_key (snd ‘ covered_keys r);
          indexed = enumerate 0 ctxs;
          nodes = cfg_node_list g
            @ sorted_list_of_set (fst ‘ covered_keys r - set (cfg_node_list g));
          intra = cfg_intra_list g;
          steps = group_by_key (λ(u, a, w). u) (λ(u, a, w). Some (a, w)) intra;
          checks = group_by_key (λ(u, a, w). u)
            (λ(u, a, w). if is_EA_Check a then Some (ea_check_cond a) else None) intra;
          obligations = group_by_key fst (Some ∘ snd) sites;
          state_at = report_row sr classify classify_ob checks obligations steps;
          route_at = (λu ca ce i ctx.
```

```
          (| route_point = u, route_context = i, route_callee = callee_of_entry ce,
            route_targets =
              (case lookup_table r u ctx of
                 Bot ⇒ []
               | Lifted _ ⇒
                   remdups (concat (map (context_indices indexed)
                     (live_targets (run_succ sr) u ctx ca (callee_of_entry ce))))) |);
        seeds_of = (λf.
          (case filter (λ(i, ctx). (FunctionEntry f, ctx) ∈ covered_keys r) indexed of
             [] ⇒ [(| global_unknown = Global_Seed f None, global_state = Bot |)]
           | entered ⇒
               map (λ(i, ctx). (| global_unknown = Global_Seed f (Some i),
                                  global_state = run_seed sr f ctx |))
                 entered))
    in (| report_config = config,
          report_vars = vars,
          report_cfg = g,
          report_contexts = map ctx_tag ctxs,
          report_states =
            concat (map (λ(i, ctx).
                           map (state_at i ctx)
                             (filter (λv. (v, ctx) ∈ covered_keys r) nodes))
                      indexed),
          report_routes =
            concat (map (λ(u, ca, ce, after).
                           map (λ(i, ctx). route_at u ca ce i ctx)
                             (filter (λ(i, ctx). (u, ctx) ∈ covered_keys r) indexed))
                      (cfg_calls_list g)),
          report_checks = result_checks_of g r classify,
          report_globals =
            map (λx. (| global_unknown = Global_Named x, global_state = run_shared sr x |))
              globals
            @ concat (map seeds_of procs),
          report_diagnostics = arithmetic_diagnostics sites r classify_ob |)"
```

The answer

Four answers: an invalid configuration asks for no run, a malformed program was never analysed, the logical `None` branch of the executable solve has no report, and every other run hands back its report.

```
datatype 'r analysis_answer =
    Invalid_Activation
  | Malformed_Program
  | No_Answer
  | Analysed 'r

end
```

17.7 Analysis_Render

```
theory Analysis_Render
  imports Analysis_Run
begin
```

The analysis graph

The report's states are filed by point *and* context, so the graph a reader sees of them has a node per such pair, where the CFG has one per point. A flow-insensitive program global adds a node of its own. The edges are the dependencies the equations have between those unknowns: a step within a context, a call entering the contexts the report routes it to, the callee's result resuming the caller, the call's continuation, and the reads and writes of each global, by the same footprints the keyed lifter reads and publishes (`edge_global_reads`, `edge_global_writes`, `call_global_reads`). It is a projection of the report, as the rendering below is, and no soundness theorem reads it.

```
datatype 'n unknown_node = Local_Node pp nat | Global_Node 'n

datatype ('vr, 'e) analysis_edge =
    Intra_Dep "('vr, 'e) gedge_action"
  | Enter_Dep "('vr, 'e) gcall_action"
  | Combine_Dep "('vr, 'e) gcall_action"
  | Continue_Dep "('vr, 'e) gcall_action"
  | Global_Read
  | Global_Write

record ('vr, 'e, 'n) analysis_graph =
  graph_nodes :: "'n unknown_node list"
  graph_edges :: "('n unknown_node × ('vr, 'e) analysis_edge × 'n unknown_node) list"
```

Which global names an edge or a call reads and writes depends on the language, so the graph takes these footprints as an argument.

```
record ('vr, 'e, 'n) footprints =
  fp_edge_reads :: "('n ⇒ bool) ⇒ ('vr, 'e) gedge_action ⇒ 'n list"
  fp_edge_writes :: "('n ⇒ bool) ⇒ ('vr, 'e) gedge_action ⇒ 'n list"
  fp_call_reads :: "('n ⇒ bool) ⇒ 'e list ⇒ 'n list"
  fp_call_writes :: "('n ⇒ bool) ⇒ ('vr, 'e) gcall_action ⇒ 'n list"
```

The program globals the report keeps at unknowns of their own.

```
definition report_global_names :: "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ 'n list" where
  "report_global_names res =
     concat (map (λg. case global_unknown g of Global_Named x ⇒ [x] | _ ⇒ []) (report_globals res))"

definition report_rows_of :: "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ (pp × nat) list" where
  "report_rows_of res = map (λst. (state_point st, state_context st)) (report_states res)"

definition report_route_targets ::
    "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ pp ⇒ nat ⇒ nat list" where
  "report_route_targets res u c =
     concat (map route_targets
       (filter (λr. route_point r = u ∧ route_context r = c) (report_routes res)))"

definition callee_of :: "pp ⇒ pname" where
  "callee_of entry = (case entry of FunctionEntry f ⇒ f | FunctionResult f ⇒ f | Statement _ ⇒ STR '''')"
```

What a step within one context adds: the step, and the globals it reads and writes.

```
definition intra_deps ::
    "('vr, 'e, 'n) footprints ⇒ ('n ⇒ bool) ⇒ (pp × nat) list ⇒ pp ⇒ nat
     ⇒ pp × ('vr, 'e) gedge_action × pp
     ⇒ ('n unknown_node × ('vr, 'e) analysis_edge × 'n unknown_node) list" where
  "intra_deps fp G rows u c e =
     (case e of (u', a, v) ⇒
        if u' = u ∧ (v, c) ∈ set rows
        then (Local_Node u c, Intra_Dep a, Local_Node v c)
             # map (λx. (Global_Node x, Global_Read, Local_Node v c)) (fp_edge_reads fp G a)
             @ map (λx. (Local_Node v c, Global_Write, Global_Node x)) (fp_edge_writes fp G a)
        else [])"
```

What a call adds: an entry into every context the report routes it to, the callee's result resuming the caller from each, and the continuation. The continuation's equation evaluates the call, so it reads the globals the arguments mention and writes a global destination or formal.

```
definition call_deps ::
    "('vr, 'e, 'n) footprints ⇒ ('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ ('n ⇒ bool)
     ⇒ (pp × nat) list ⇒ pp ⇒ nat ⇒ pp × ('vr, 'e) gcall_action × pp × pp
     ⇒ ('n unknown_node × ('vr, 'e) analysis_edge × 'n unknown_node) list" where
  "call_deps fp res G rows u c e =
     (case e of (u', ca, entry, after) ⇒
        if u' ≠ u then []
        else
          (let targets = report_route_targets res u c;
               result = FunctionResult (callee_of entry)
           in concat (map (λt.
                (if (entry, t) ∈ set rows then [(Local_Node u c, Enter_Dep ca, Local_Node entry t)] else [])
                @ (if (result, t) ∈ set rows ∧ (after, c) ∈ set rows
                   then [(Local_Node result t, Combine_Dep ca, Local_Node after c)] else []))
                targets)
              @ (if (after, c) ∈ set rows
                 then (Local_Node u c, Continue_Dep ca, Local_Node after c)
                      # map (λx. (Global_Node x, Global_Read, Local_Node after c))
                          (fp_call_reads fp G (ce_args ca))
                      @ map (λx. (Local_Node after c, Global_Write, Global_Node x))
                          (fp_call_writes fp G ca)
                 else [])))"

definition analysis_graph_of ::
    "('vr::linorder, 'e::linorder, 'n) footprints ⇒ ('k, 'x, 'vr, 'e, 'n, 's) analysis_report
       ⇒ ('vr, 'e, 'n) analysis_graph" where
  "analysis_graph_of fp res =
     (let g = report_cfg res;
          rows = report_rows_of res;
          names = report_global_names res;
          G = (λx. x ∈ set names)
      in ⦇ graph_nodes = map (λ(v, c). Local_Node v c) rows @ map Global_Node names,
           graph_edges =
             concat (map (λe. concat (map (λ(u, c). intra_deps fp G rows u c e) rows))
               (cfg_intra_list g))
             @ concat (map (λe. concat (map (λ(u, c). call_deps fp res G rows u c e) rows))
                 (cfg_calls_list g))
             @ concat (map (λ(u, c).
                 if u = cfg_entry g
                 then map (λx. (Local_Node u c, Global_Write, Global_Node x)) names else [])
               rows) ⦈)"
```

Rendering a report

A report holds semantic states. What a consumer displays is a projection of it: each active analysis's part of a state, as that analysis's own value, then as a string. The projection is defined here so that generated code computes it; no soundness theorem reads it, and it changes neither the check column nor the diagnostics.

The rendered result

A rendered state is shown one active analysis at a time, in activation order, each as its own `'v field_state`, as Goblint's report shows each component of its combined state. A context reads as the formal values its policy keys it by.

```
datatype 'v analysis_context =
    Context_Unit
  | Context_Entry "'v list"
  | Context_Call_String "pp list"

type_synonym ('d, 'v) analysis_view = "('d × 'v field_state) list"
```

The states a report holds at a point, joined over its contexts: what a reader who ignores contexts can say about the point. It may be coarser than the states it joins, but every store any of them describes, it describes too.

A read state maps names to values, and generated code cannot carry the order of such a function, only its join, which is pointwise. `vjoin` is that join alone, per type constructor; `join_val_sup` records that it is the lattice join, so the theorems about `report_point_join` speak about ⊔.

```
class join_val =
  fixes vjoin :: "'a ⇒ 'a ⇒ 'a"
```

A join that is the lattice join of a semilattice, for the proofs only.

```
class join_val_sup = join_val + semilattice_sup +
```

```
  assumes vjoin_sup: "vjoin x y = x ⊔ y"

instantiation "fun" :: (type, semilattice_sup) join_val
begin
definition vjoin_fun :: "('a ⇒ 'b) ⇒ ('a ⇒ 'b) ⇒ 'a ⇒ 'b" where
  "vjoin_fun f g = (λx. f x ⊔ g x)"
instance ..
end

instance "fun" :: (type, semilattice_sup) join_val_sup
  by standard (simp add: vjoin_fun_def sup_fun_def)

instantiation lifted :: (join_val) join_val
begin
fun vjoin_lifted :: "'a lifted ⇒ 'a lifted ⇒ 'a lifted" where
  "vjoin_lifted Bot y = y"
| "vjoin_lifted (Lifted x) Bot = Lifted x"
| "vjoin_lifted (Lifted x) (Lifted y) = Lifted (vjoin x y)"
instance ..
end

instance lifted :: (join_val_sup) join_val_sup
proof
  fix x y :: "'a lifted"
  show "vjoin x y = x ⊔ y" by (cases x; cases y) (simp_all add: vjoin_sup)
qed

instantiation analysis_product :: (join_val, join_val) join_val
begin
definition vjoin_analysis_product ::
    "('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product ⇒ ('a, 'b) analysis_product" where
  "vjoin_analysis_product p q = Product (vjoin (pleft p) (pleft q)) (vjoin (pright p) (pright q))"
instance ..
end

instance analysis_product :: (join_val_sup, join_val_sup) join_val_sup
  by standard (simp add: vjoin_analysis_product_def sup_analysis_product_def vjoin_sup)

definition report_point_join ::
    "('k, 'x, 'vr, 'e, 'n, 's::join_val) analysis_report ⇒ pp ⇒ 's lifted" where
  "report_point_join res v =
     fold vjoin (map state_value (filter (λst. state_point st = v) (report_states res))) Bot"
```

What a point's outgoing steps produce, joined over its contexts the same way. Every context lists the point's steps in the order of its outgoing edges, so the lists are joined position by position.

```
definition report_steps_join ::
    "('k, 'x, 'vr, 'e, 'n, 's::join_val) analysis_report ⇒ pp ⇒ (pp × 's lifted) list" where
  "report_steps_join res v =
     (case map state_steps (filter (λst. state_point st = v) (report_states res)) of
        [] ⇒ []
      | s # ss ⇒ fold (map2 (λ(w, a) (_, b). (w, vjoin a b))) ss s)"

record ('vr, 'e, 'n, 'd, 'v) run_result =
  res_cfg :: "('vr, 'e) gcfg"
  res_contexts :: "'v analysis_context list"
  res_states :: "('e, ('d, 'v) analysis_view) result_state list"
  res_joined ::
    "(pp × ('d, 'v) analysis_view lifted × (pp × ('d, 'v) analysis_view lifted) list) list"
  res_routes :: "call_route list"
  res_checks :: "'e result_check list"
  res_globals :: "('n, ('d, 'v) analysis_view) result_global list"
  res_diagnostics :: "'e arithmetic_diagnostic list"
  res_graph :: "('vr, 'e, 'n) analysis_graph"

definition map_analysis_view :: "('v ⇒ 'w) ⇒ ('d, 'v) analysis_view ⇒ ('d, 'w) analysis_view" where
  "map_analysis_view f = map (λ(a, s). (a, map_field_state f s))"
```

What a language shows of its report: each active analysis's part of a state, the values an entry-state context is keyed by, and the global footprints of its edges. The printer `show` is a parameter, composed in the adapter with whatever post-processing its output needs.

```
record ('k, 'm, 's, 'vr, 'e, 'n, 'd, 'a) report_renderer =
  rr_view :: "'k ⇒ 'vr list ⇒ 's ⇒ ('d, 'a) analysis_view"
  rr_context :: "'k ⇒ 'm ⇒ 'a list"
  rr_footprints :: "('vr, 'e, 'n) footprints"
```

```
fun render_context :: "('a ⇒ 'v) ⇒ ('m ⇒ 'a list) ⇒ 'm report_context ⇒ 'v analysis_context"
where
  "render_context show vals Report_Unit = Context_Unit"
| "render_context show vals (Report_Entry ctx) = Context_Entry (map show (vals ctx))"
| "render_context show vals (Report_Call_String us) = Context_Call_String us"

definition render_report ::
    "('k, 'm, 's::join_val, 'vr::linorder, 'e::linorder, 'n, 'd, 'a) report_renderer
       ⇒ ('a ⇒ 'v) ⇒ ('k, 'm report_context, 'vr, 'e, 'n, 's) analysis_report
       ⇒ ('vr, 'e, 'n, 'd, 'v) run_result" where
  "render_report rr show res =
     (let config = report_config res;
          state = map_analysis_view show ∘ rr_view rr config (report_vars res)
      in ⦇ res_cfg = report_cfg res,
           res_contexts = map (render_context show (rr_context rr config)) (report_contexts res),
           res_states = map (map_result_state state) (report_states res),
           res_joined = map (λv. (v, map_lift state (report_point_join res v),
                                  map (λ(w, s). (w, map_lift state s)) (report_steps_join res v)))
                         (cfg_node_list (report_cfg res)),
           res_routes = report_routes res,
           res_checks = report_checks res,
           res_globals = map (map_result_global state) (report_globals res),
           res_diagnostics = report_diagnostics res,
           res_graph = analysis_graph_of (rr_footprints rr) res ⦈)"

lemma render_report_columns [simp]:
  "res_cfg (render_report rr show res) = report_cfg res"
  "res_checks (render_report rr show res) = report_checks res"
  "res_diagnostics (render_report rr show res) = report_diagnostics res"
  "res_routes (render_report rr show res) = report_routes res"
  by (simp_all add: render_report_def Let_def)
```

What the analysis graph contains

The graph's nodes are exactly the report's unknowns, and a step between two reported points of one context carries the global reads and writes its footprint names.

```
lemma local_node_in_analysis_graph:
  "Local_Node v c ∈ set (graph_nodes (analysis_graph_of fp res))
     ⟷ (∃st ∈ set (report_states res). state_point st = v ∧ state_context st = c)"
  by (auto simp: analysis_graph_of_def report_rows_of_def Let_def)

lemma global_node_in_analysis_graph:
  "Global_Node x ∈ set (graph_nodes (analysis_graph_of fp res))
     ⟷ (∃gl ∈ set (report_globals res). global_unknown gl = Global_Named x)"
proof -
  have "x ∈ set (report_global_names res)
         ⟷ (∃gl ∈ set (report_globals res). global_unknown gl = Global_Named x)"
    by (force simp: report_global_names_def split: result_global_unknown.splits)
  then show ?thesis by (auto simp: analysis_graph_of_def Let_def)
qed
```

A step of a reported context contributes all its dependencies to the graph.

```
lemma intra_deps_in_analysis_graph:
  assumes "(u, c) ∈ set (report_rows_of res)" and "e ∈ set (cfg_intra_list (report_cfg res))"
  shows "set (intra_deps fp (λy. y ∈ set (report_global_names res)) (report_rows_of res) u c e)
          ⊆ set (graph_edges (analysis_graph_of fp res))"
  using assms unfolding analysis_graph_of_def Let_def
  by (auto intro!: bexI[where x = "(u, c)"] bexI[where x = e])
```

A step both of whose points the report holds in a context reads and writes, in the graph, the globals its footprint names.

```
lemma global_deps_in_analysis_graph:
  fixes res defines "G ≡ (λy. y ∈ set (report_global_names res))"
  assumes e: "(u, a, v) ∈ set (cfg_intra_list (report_cfg res))"
    and u: "(u, c) ∈ set (report_rows_of res)" and v: "(v, c) ∈ set (report_rows_of res)"
  shows "x ∈ set (fp_edge_reads fp G a)
          ⟹ (Global_Node x, Global_Read, Local_Node v c) ∈ set (graph_edges (analysis_graph_of fp res))"
    and "x ∈ set (fp_edge_writes fp G a)
          ⟹ (Local_Node v c, Global_Write, Global_Node x) ∈ set (graph_edges (analysis_graph_of fp res))"
proof -
  have deps: "set (intra_deps fp G (report_rows_of res) u c (u, a, v))
               ⊆ set (graph_edges (analysis_graph_of fp res))"
    unfolding G_def by (rule intra_deps_in_analysis_graph[OF u e])
  show "x ∈ set (fp_edge_reads fp G a)
```

```
        ⟹ (Global_Node x, Global_Read, Local_Node v c) ∈ set (graph_edges (analysis_graph_of fp res))"
    using deps v by (auto simp: intra_deps_def)
  show "x ∈ set (fp_edge_writes fp G a)
        ⟹ (Local_Node v c, Global_Write, Global_Node x) ∈ set (graph_edges (analysis_graph_of fp res))"
    using deps v by (auto simp: intra_deps_def)
qed
```

So the graph is read off the report alone: its local nodes are the reported rows (`local_node_in_analysis_graph`), its global nodes the keyed program globals (`global_node_in_analysis_graph`), and its global edges the footprints of the keyed lifter (`global_deps_in_analysis_graph`).

end

17.8 Analysis_Report

```
theory Analysis_Report
  imports Analysis_Render
begin
```

What a report claims

A report is read point by point: the states it holds at a point, one per context it was solved at, the verdicts of the checks it records there, and the arithmetic diagnostics it names there. The vocabulary below takes any report, whatever its expressions and states; only the evaluator that says what a condition means at a store is the language's.

The states a report holds

```
definition report_states_at :: "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ pp ⇒ 's lifted set"
  where
  "report_states_at res v = state_value ` {st ∈ set (report_states res). state_point st = v}"

lemma finite_report_states_at [simp]: "finite (report_states_at res v)"
  by (simp add: report_states_at_def)
```

Verdicts

`report_checks_at` selects the checks a report records at one program point.

```
definition report_checks_at ::
    "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ pp ⇒ 'e result_check set" where
  "report_checks_at res v = {c ∈ set (report_checks res). check_point c = v}"

lemma mem_report_checks_at [simp]:
  "c ∈ report_checks_at res v ⟷ c ∈ set (report_checks res) ∧ check_point c = v"
  by (simp add: report_checks_at_def)
```

What a decided verdict says about a store: `Check_Proved` that the condition holds, `Check_Refuted` that it fails, `Check_Unknown` nothing.

```
fun verdict_holds :: "check_result ⇒ exp ⇒ store ⇒ bool" where
  "verdict_holds Check_Proved e s = truthy (eval e s)"
| "verdict_holds Check_Refuted e s = (¬ truthy (eval e s))"
| "verdict_holds Check_Unknown e s = True"
```

The stores a check's verdict admits. A `Dead` check claims nothing about stores; its claim, that the point is unreached, is `DEAD` below.

```
definition check_stores :: "exp result_check ⇒ store set" where
  "check_stores c = (case check_verdict c of
                       Decided r ⇒ {s. verdict_holds r (check_exp c) s}
                     | Dead ⇒ UNIV)"
```

The verdict a report gives a condition at a point. `PROVED res v c` says the report proves check `c` at `v`: since a point may carry several checks, the condition names the one meant.

```
definition HAS_VERDICT ::
    "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ pp ⇒ 'e ⇒ check_result ⇒ bool" where
  "HAS_VERDICT res v e r ⟷
     (∃c ∈ report_checks_at res v. check_exp c = e ∧ check_verdict c = Decided r)"

abbreviation PROVED :: "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ pp ⇒ 'e ⇒ bool" where
  "PROVED res v e ≡ HAS_VERDICT res v e Check_Proved"

abbreviation REFUTED :: "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ pp ⇒ 'e ⇒ bool" where
  "REFUTED res v e ≡ HAS_VERDICT res v e Check_Refuted"

abbreviation UNKNOWN :: "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ pp ⇒ 'e ⇒ bool" where
  "UNKNOWN res v e ≡ HAS_VERDICT res v e Check_Unknown"
```

`DEAD` is a property of a point, not of a condition: every state the report holds there is `Bot`. The states are canonical — the table they come from collapses a state its emptiness test rejects to `Bot` — so `DEAD` reads that canonical form instead of running a test.

```
definition DEAD :: "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ pp ⇒ bool" where
  "DEAD res v ⟷ (∀d ∈ report_states_at res v. d = Bot)"
```

`SAFE` says the report names no arithmetic diagnostic at a point. What that guarantees depends on which obligations the report was built to decide: `run_voblint_SAFE` states it for the obligations of core VIMP's expressions, and says nothing about a hazard the report does not check.

```
abbreviation SAFE :: "('k, 'x, 'vr, 'e, 'n, 's) analysis_report ⇒ pp ⇒ bool" where
  "SAFE res v ≡ ∀d ∈ set (report_diagnostics res). diagnostic_point d ≠ v"
```

A report built from a table

Forgetting the labels, the check column is the contextual verdict report itself.

```
lemma result_checks_of_verdicts:
  "map (λchk. (check_point chk, check_exp chk, check_verdict chk)) (result_checks_of g r classify)
     = classify_checks_verdicts g r classify"
  unfolding result_checks_of_def classify_checks_verdicts_def classify_checks_ctx_def
    point_verdict_def
  by (simp add: comp_def case_prod_beta image_image)
```

`report_of` lists the contexts of the table in the order of an injective key, so it lists every one, and files every covered key as a row. Its rows at a point are therefore exactly the table's entries there.

```
lemma ordered_by_key_set:
  assumes fin: "finite S" and inj: "inj_on rank S"
  shows "set (ordered_by_key rank S) = S"
proof -
  have pick: "the_elem (Set.filter (λx. rank x = rank y) S) = y" if "y ∈ S" for y
  proof -
    have "Set.filter (λx. rank x = rank y) S = {y}"
      using that inj by (auto simp: inj_on_def)
    then show ?thesis by simp
  qed
  have "set (ordered_by_key rank S) = (λk. the_elem (Set.filter (λx. rank x = k) S)) ‘ rank ‘ S"
    unfolding ordered_by_key_def using fin by simp
  also have "... = S"
    using pick by (force simp: image_comp)
  finally show ?thesis .
qed

lemma row_in_rows:
  assumes "(i, ctx) ∈ set xs" and "v ∈ set ns" and "P v ctx"
  shows "f i ctx v ∈ set (concat (map (λ(i, ctx). map (f i ctx) (filter (λv. P v ctx) ns)) xs))"
  using assms by force

lemma report_states_at_report_of:
  assumes fin: "finite (covered_keys (run_table sr))" and inj: "inj ctx_key"
  shows "report_states_at (report_of config vars g globals sites cond procs ctx_key ctx_tag classify sr) v
         = lookup_table (run_table sr) v ‘ table_contexts (run_table sr) v"
proof -
  define r where "r = run_table sr"
  define ctxs where "ctxs = ordered_by_key ctx_key (snd ‘ covered_keys r)"
  define nodes where "nodes = cfg_node_list g
    @ sorted_list_of_set (fst ‘ covered_keys r - set (cfg_node_list g))"
  have ctxs: "set ctxs = snd ‘ covered_keys r"
    unfolding ctxs_def r_def
    by (rule ordered_by_key_set) (use fin inj in ‹auto intro: inj_on_subset›)
  have nodes: "w ∈ set nodes" if "(w, ctx) ∈ covered_keys r" for w ctx
  proof (cases "w ∈ set (cfg_node_list g)")
    case False
    with that fin show ?thesis unfolding nodes_def r_def by (force intro: rev_image_eqI)
  qed (simp add: nodes_def)
  have indexed: "∃i. (i, ctx) ∈ set (enumerate 0 ctxs)" if "ctx ∈ set ctxs" for ctx
  proof -
    from that obtain i where "i < length ctxs" and "ctxs ! i = ctx"
      by (auto simp: in_set_conv_nth)
    then have "(i, ctx) ∈ set (enumerate 0 ctxs)"
      by (metis add_0 length_enumerate nth_enumerate_eq nth_mem)
    then show ?thesis ..
  qed
  show ?thesis
  proof (intro equalityI subsetI)
    fix d
    assume "d ∈ report_states_at (report_of config vars g globals sites cond procs ctx_key ctx_tag classify sr) v"
    then show "d ∈ lookup_table (run_table sr) v ‘ table_contexts (run_table sr) v"
      unfolding report_states_at_def report_of_def Let_def by (auto simp: table_contexts_iff)
  next
    fix d
```

```
    assume "d ∈ lookup_table (run_table sr) v ‘ table_contexts (run_table sr) v"
    then obtain ctx where cov: "(v, ctx) ∈ covered_keys r"
      and d: "d = lookup_table r v ctx"
      by (auto simp: r_def table_contexts_iff)
    from cov ctxs have "ctx ∈ set ctxs" by force
    then obtain i where i: "(i, ctx) ∈ set (enumerate 0 ctxs)" using indexed by blast
    have w: "v ∈ set nodes" using nodes[OF cov] .
    show "d ∈ report_states_at (report_of config vars g globals sites cond procs ctx_key ctx_tag classify sr) v"
      unfolding report_states_at_def report_of_def Let_def r_def[symmetric] ctxs_def[symmetric]
        nodes_def[symmetric]
      apply (simp only: analysis_report.select_convs)
      apply (rule image_eqI[rotated], rule CollectI, rule conjI, rule row_in_rows[OF i w])
      using cov d by (simp_all add: r_def)
  qed
qed
```

A call string is listed in the order of the nodes it names.

```
lemma inj_call_string_key: "inj (λctx. Key_List (map Key_Node ctx))"
  by (rule injI) (simp add: inj_def)

end
```

17.9 Run_Report

```
theory Run_Report
  imports Analysis_Report
begin
```

The report of a run

A run is reported by `report_of`: the elaborated graph, the declared variables, the globals the placement keeps at unknowns of their own, the arithmetic obligations of the graph's expressions, and the program's procedures.

Arithmetic obligations of typed expressions

A division or remainder must not divide by zero. A signed addition, subtraction, multiplication or division must compute a result its kind represents: C leaves the behavior undefined otherwise (ISO/IEC 9899 6.5p5), and Goblint checks the same four operators (`SignedIntegerOverflowInArithmetic`). A unary minus reaches the typed language as a subtraction from zero, and a signed division overflows only on the kind's minimum divided by `-1`. A signed remainder's mathematical result always fits, but C leaves it undefined when the quotient does not fit (6.5.5p6), as for `INT_MIN % -1`: its obligation is that the truncated quotient fits. An unsigned operation wraps by definition and raises no obligation.

A conversion must convert a value its target kind represents. Elaboration makes every conversion a `TCast` node: an explicit cast, and the implicit conversions of an assigned value, an argument and a returned value to the kind of the place they are written to. A conversion whose source kind widens into its target cannot change a value of the source kind and raises no obligation. Goblint checks conversions to signed and unsigned kinds alike (`IntegerOverflowInExplicitCast`, ...`InImplicitCast`); so does Voblint, although C defines a conversion to an unsigned kind as wraparound and leaves one to a signed kind implementation-defined, not undefined. Voblint does not tell an explicit cast from an implicit conversion, since elaboration does not record it.

Every obligation of an expression is raised, however deeply it is nested and whatever the value of an enclosing logical operator, since the typed language evaluates both operands of one.

A literal elaborated at a kind is converted to it as well: `x = 300` for an 8-bit `x` elaborates to the literal at `x`'s kind, whose value wraps.

```
fun arith_obligations :: "exp ⇒ exp arithmetic_obligation list" where
  "arith_obligations (TN k n) =
     (if n ∈ ik_range k then [] else [Arithmetic_Obligation Conversion_Overflow (TN k n) (TN k n)])"
| "arith_obligations (TVar _ _) = []"
| "arith_obligations (TArith k op a b) =
     arith_obligations a @ arith_obligations b
     @ (if op = ADiv ∨ op = AMod
        then [Arithmetic_Obligation Division_By_Zero (TArith k op a b) b] else [])
     @ (if ¬ ik_signed k then []
        else if op = AMod
        then [Arithmetic_Obligation Remainder_Overflow (TArith k op a b) (TArith k ADiv a b)]
        else [Arithmetic_Obligation Signed_Overflow (TArith k op a b) (TArith k op a b)])"
| "arith_obligations (TCast k a) =
     arith_obligations a
     @ (if ik_widens (exp_kind a) k then []
        else [Arithmetic_Obligation Conversion_Overflow (TCast k a) a])"
| "arith_obligations (TCmp _ a b) = arith_obligations a @ arith_obligations b"
| "arith_obligations (TNot a) = arith_obligations a"
| "arith_obligations (TAnd a b) = arith_obligations a @ arith_obligations b"
| "arith_obligations (TOr a b) = arith_obligations a @ arith_obligations b"
```

A division obligation is safe when its divisor is not zero, compared at the divisor's kind.

```
definition arith_condition :: "exp arithmetic_obligation ⇒ exp" where
  "arith_condition ob =
     TCmp CNe (arithmetic_operand ob) (TN (exp_kind (arithmetic_operand ob)) 0)"
```

An overflow obligation is about the value an operation computes before its kind wraps it: the mathematical result of an arithmetic node, the converted value of a conversion, the quotient a remainder's operand names, and the minimum or maximum of a library result's two arguments. It is safe when that value lies in the operation's kind. The statement does not depend on the overflow policy: the policy decides how an analysis abstracts a wrapped result, never what the program computes.

```
fun unwrapped :: "exp ⇒ store ⇒ int" where
  "unwrapped (TN k n) t = n"
| "unwrapped (TArith k op a b) t = aop_eval op (eval a t) (eval b t)"
| "unwrapped (TCast k a) t = eval a t"
```

```
| "unwrapped e t = eval e t"

definition obligation_holds :: "exp arithmetic_obligation ⇒ store ⇒ bool" where
  "obligation_holds ob t ⟷
     (let k = exp_kind (arithmetic_operation ob); m = unwrapped (arithmetic_operation ob) t;
          n = eval (arithmetic_operand ob) t
      in case arithmetic_hazard ob of
           Division_By_Zero ⇒ n ≠ 0
         | Min_Conversion_Overflow ⇒ min m n ∈ ik_range k
         | Max_Conversion_Overflow ⇒ max m n ∈ ik_range k
         | Remainder_Overflow ⇒ unwrapped (arithmetic_operand ob) t ∈ ik_range k
         | _ ⇒ m ∈ ik_range k)"

abbreviation satisfies :: "store ⇒ exp arithmetic_obligation ⇒ bool" (infix "⊨" 50) where
  "s ⊨ ob ≡ obligation_holds ob s"
```

An overflow obligation is decided as Goblint decides it, from the interval of the unwrapped result: the operands' values are asked through `EvalInt`, so every active analysis contributes, and their interval components are combined by the operation over the integers. The obligation is safe when that interval lies in the kind, and certainly violated when it misses the kind. An answer that admits no value is read as unknown, as `answer_check` reads one.

A literal operand is its own exact interval, a truth value lies in `[0, 1]`, and any other operand lies in its kind's range when its kind is known (`src_kind`), whatever the active analyses answer for it: a domain without magnitudes, such as Parity, would otherwise leave even `10 / 3` or `x / 2` a possible overflow. The kind bounds an operand only at a store within the kinds (`in_kinds`), which every reaching store is. A conversion whose operand's interval lies in the target kind changes no value, so it keeps that interval: a `U8` argument of `min` promoted to `I32` stays within `[0, 255]`.

```
fun operand_ivl :: "kenv ⇒ channel ⇒ exp ⇒ ivl" where
  "operand_ivl κ ch (TN k n) = Ivl (Fin (ik_norm k n)) (Fin (ik_norm k n))"
| "operand_ivl κ ch (TCmp c a b) =
     of_answer_ik None (ch (EvalInt (TCmp c a b))) ⊓ Ivl (Fin 0) (Fin 1)"
| "operand_ivl κ ch (TNot a) =
     of_answer_ik None (ch (EvalInt (TNot a))) ⊓ Ivl (Fin 0) (Fin 1)"
| "operand_ivl κ ch (TAnd a b) =
     of_answer_ik None (ch (EvalInt (TAnd a b))) ⊓ Ivl (Fin 0) (Fin 1)"
| "operand_ivl κ ch (TOr a b) =
     of_answer_ik None (ch (EvalInt (TOr a b))) ⊓ Ivl (Fin 0) (Fin 1)"
| "operand_ivl κ ch (TCast k a) =
     (let i = operand_ivl κ ch a;
          j = of_answer_ik (src_kind κ (TCast k a)) (ch (EvalInt (TCast k a)))
      in if i ≤ ik_ivl k then i ⊓ j else j)"
| "operand_ivl κ ch e = of_answer_ik (src_kind κ e) (ch (EvalInt e))"

fun unwrapped_ivl :: "kenv ⇒ channel ⇒ exp ⇒ ivl" where
  "unwrapped_ivl κ ch (TN k n) = Ivl (Fin n) (Fin n)"
| "unwrapped_ivl κ ch (TArith k op a b) =
     ivl_aop op (operand_ivl κ ch a) (operand_ivl κ ch b)"
| "unwrapped_ivl κ ch (TCast k a) = operand_ivl κ ch a"
| "unwrapped_ivl κ ch e = operand_ivl κ ch e"

lemma unwrapped_ivl_sound:
  assumes "eval_query.channel_holds ch t" and kinds: "in_kinds κ t"
  shows "unwrapped e t ∈ gamma_ivl (unwrapped_ivl κ ch e)"
    and "eval e t ∈ gamma_ivl (operand_ivl κ ch e)"
proof -
  have answer: "eval a t ∈ gamma_ivl (of_answer_ik sk (ch (EvalInt a)))"
    if "⋀k. sk = Some k ⟹ eval a t ∈ ik_range k" for a sk
    using eval_query.channel_holdsD[OF assms(1), of "EvalInt a"]
      of_answer_ik_sound[where 'a = ivl, of sk "eval a t" "ch (EvalInt a)"] that
    by simp
  have truth: "eval a t ∈ gamma_ivl (Ivl (Fin 0) (Fin 1))"
    if
      "a ∈ {TCmp c x y | c x y. True} ∪ range TNot ∪ {TAnd x y | x y. True} ∪ {TOr x y | x y. True}"
    for a
    using that by (auto simp: less_eq_eint_def)
  have known: "eval a t ∈ gamma_ivl (of_answer_ik (src_kind κ a) (ch (EvalInt a)))" for a
    by (rule answer[OF src_kind_sound[OF kinds]])
  have other: "eval a t ∈ gamma_ivl (operand_ivl κ ch a)" if "∀k b. a ≠ TCast k b" for a
    using that answer[of None a] known[of a] truth[of a]
    by (cases a) (auto simp: less_eq_eint_def gamma_inf_ivl)
  have ans: "eval a t ∈ gamma_ivl (operand_ivl κ ch a)" for a
  proof (induction a)
    case (TCast k a)
```

```
    have "eval (TCast k a) t = eval a t"
      if "operand_ivl κ ch a ≤ ik_ivl k"
      using gamma_ivl_mono[OF that] TCast.IH by auto
    then show ?case
      using TCast.IH known[of "TCast k a"] by (auto simp: Let_def)
  qed (rule other, simp)+
  show "unwrapped e t ∈ gamma_ivl (unwrapped_ivl κ ch e)"
  proof (cases e)
    case (TN k n) then show ?thesis by (simp add: less_eq_eint_def)
  qed (simp_all only: unwrapped.simps unwrapped_ivl.simps ans ivl_aop_sound)
  show "eval e t ∈ gamma_ivl (operand_ivl κ ch e)" by (rule ans)
qed

definition fits_verdict :: "ikind ⇒ ivl ⇒ check_result" where
  "fits_verdict k i =
     (if is_bottom_ivl i then Check_Unknown
      else if i ≤ ik_ivl k then Check_Proved
      else if is_bottom_ivl (i ⊓ ik_ivl k) then Check_Refuted
      else Check_Unknown)"

lemma fits_verdict_sound:
  assumes "n ∈ gamma_ivl i"
  shows "fits_verdict k i = Check_Proved ⟹ n ∈ ik_range k"
    and "fits_verdict k i = Check_Refuted ⟹ n ∉ ik_range k"
proof -
  show "fits_verdict k i = Check_Proved ⟹ n ∈ ik_range k"
    using assms gamma_ivl_mono[of i "ik_ivl k"]
    by (auto simp: fits_verdict_def split: if_splits)
  show "fits_verdict k i = Check_Refuted ⟹ n ∉ ik_range k"
    using assms
    by (auto simp: fits_verdict_def is_bottom_ivl_correct gamma_inf_ivl split: if_splits)
qed
```

A library result's obligation is decided from the minimum or maximum of its arguments' intervals, a remainder's from the interval of its quotient.

```
definition obligation_classify ::
    "sem_conf ⇒ kind_decls ⇒ analysis list ⇒ exp arithmetic_obligation ⇒ mcp_val
       ⇒ check_result" where
  "obligation_classify sem ds as ob v =
     (let ch = mcp_ask sem ds as v; κ = kenv_of ds; e = arithmetic_operation ob;
          i = unwrapped_ivl κ ch e; j = operand_ivl κ ch (arithmetic_operand ob)
      in case arithmetic_hazard ob of
           Division_By_Zero ⇒ mcp_classify sem ds as (arith_condition ob) v
         | Min_Conversion_Overflow ⇒ fits_verdict (exp_kind e) (ivl_min i j)
         | Max_Conversion_Overflow ⇒ fits_verdict (exp_kind e) (ivl_max i j)
         | Remainder_Overflow ⇒
             fits_verdict (exp_kind e) (unwrapped_ivl κ ch (arithmetic_operand ob))
         | _ ⇒ fits_verdict (exp_kind e) i)"

lemma obligation_classify_sound:
  assumes t: "t ∈ mcp.act_gamma ds as v" and kinds: "in_kinds (kenv_of ds) t"
  shows "obligation_classify sem ds as ob v = Check_Proved ⟹ t ⊨ ob"
    and "obligation_classify sem ds as ob v = Check_Refuted ⟹ ¬ t ⊨ ob"
proof -
  have ch: "eval_query.channel_holds (mcp_ask sem ds as v) t"
    by (rule mcp.mcp_ask_sound[OF t])
  note ivl = unwrapped_ivl_sound[OF ch kinds]
  note fits = fits_verdict_sound[OF ivl(1)]
    and fits_min = fits_verdict_sound[OF ivl_min_sound[OF ivl]]
    and fits_max = fits_verdict_sound[OF ivl_max_sound[OF ivl]]
    and fits_rem = fits_verdict_sound[OF ivl(1)[of "arithmetic_operand ob"]]
  have ne: "truthy (eval (arith_condition ob) t) ⟷ eval (arithmetic_operand ob) t ≠ 0"
    by (simp add: arith_condition_def)
  show "obligation_classify sem ds as ob v = Check_Proved ⟹ t ⊨ ob"
    using fits(1) fits_min(1) fits_max(1) fits_rem(1)
      mcp.mcp_classify_proved[OF t] ne
    by (auto simp: obligation_classify_def obligation_holds_def Let_def
        split: arithmetic_hazard.splits)
  show "obligation_classify sem ds as ob v = Check_Refuted ⟹ ¬ t ⊨ ob"
    using fits(2) fits_min(2) fits_max(2) fits_rem(2)
      mcp.mcp_classify_refuted[OF t] ne
    by (auto simp: obligation_classify_def obligation_holds_def Let_def
        split: arithmetic_hazard.splits)
qed
```

Where the obligations are decided

An expression's obligations are decided at the source of the edge or call that evaluates it. One conversion has no expression of its own: a call's result, which the caller writes to its destination at the destination's kind. Its obligation is decided where the callee returns, on the returned expression, once for each kind a caller converts it to. A returned value whose kind widens into the destination's raises none. A library call's result is written to its destination as well. That of `min` or `max` is decided where the call is, unless both arguments' kinds widen into the destination's: the result is one of the arguments. That of the call a declaration without an initializer makes is any value of the destination's kind and raises none.

```
definition return_sites ::
    "kind_decls ⇒ imp_prog ⇒ (pp × exp arithmetic_obligation list) list" where
  "return_sites ds p =
     (let g = elab_cfg p ds
      in remdups (concat (map (λ(u, ca, ce, k). case ce_dst ca of
           None ⇒ []
         | Some x ⇒ concat (map (λ(v, a, w). case a of
              EA_Ret (Some e) q ⇒
                if q = callee_of_entry ce ∧ ¬ ik_widens (exp_kind e) (kenv_of ds x)
                then [(v, [Arithmetic_Obligation Conversion_Overflow (TCast (kenv_of ds x) e) e])]
                else []
            | _ ⇒ []) (cfg_intra_list g)))
         (cfg_calls_list g))))"

fun special_obligations :: "ikind ⇒ exp gspecial_call ⇒ exp arithmetic_obligation list" where
  "special_obligations k (Min a b) =
     (if ik_widens (exp_kind a) k ∧ ik_widens (exp_kind b) k then []
      else [Arithmetic_Obligation Min_Conversion_Overflow (TCast k a) b])"
| "special_obligations k (Max a b) =
     (if ik_widens (exp_kind a) k ∧ ik_widens (exp_kind b) k then []
      else [Arithmetic_Obligation Max_Conversion_Overflow (TCast k a) b])"
| "special_obligations k Nondet_Int = []"

definition special_sites ::
    "kind_decls ⇒ imp_prog ⇒ (pp × exp arithmetic_obligation list) list" where
  "special_sites ds p =
     [(v, special_obligations (kenv_of ds x) sc).
        (v, EA_Special sc x, w) ← cfg_intra_list (elab_cfg p ds),
        special_obligations (kenv_of ds x) sc ≠ []]"

definition prog_obligation_sites ::
  "kind_decls ⇒ imp_prog ⇒ (pp × exp arithmetic_obligation list) list" where
  "prog_obligation_sites ds p =
     obligation_sites arith_obligations (elab_cfg p ds) @ return_sites ds p @ special_sites ds p"
```

The obligations a report decides at a point, and what it means for a store to meet one: `s ⊨ ob`.

```
definition obligations_at :: "kind_decls ⇒ imp_prog ⇒ pp ⇒ exp arithmetic_obligation set" where
  "obligations_at ds p v = (⋃(u, obs) ∈ set (prog_obligation_sites ds p). if u = v then set obs else {})"

lemma mem_obligations_at:
  "ob ∈ obligations_at ds p v ⟷ (∃obs. (v, obs) ∈ set (prog_obligation_sites ds p) ∧ ob ∈ set obs)"
  by (force simp: obligations_at_def split: if_splits)
```

The variables a report lists: every variable an edge or a call of the elaborated graph mentions, and the return slot a `return e` writes, so the value a procedure returns can be shown.

```
fun special_vars :: "exp gspecial_call ⇒ var_id list" where
  "special_vars Nondet_Int = []"
| "special_vars (Min a b) = var_list a @ var_list b"
| "special_vars (Max a b) = var_list a @ var_list b"

fun edge_vars :: "(var_id, exp) gedge_action ⇒ var_id list" where
  "edge_vars (EA_Assign x e) = x # var_list e"
| "edge_vars (EA_Special sc x) = x # special_vars sc"
| "edge_vars (EA_Assume b) = var_list b"
| "edge_vars (EA_AssumeNot b) = var_list b"
| "edge_vars (EA_Ret e _) = (case e of Some e ⇒ ReturnVar # var_list e | None ⇒ [])"
| "edge_vars (EA_Check _ e) = var_list e"
| "edge_vars _ = []"

fun call_vars :: "(var_id, exp) gcall_action ⇒ var_id list" where
  "call_vars (CallEdge dst ps args) =
     (case dst of Some x ⇒ [x] | None ⇒ []) @ ps @ concat (map var_list args)"

definition cfg_vars :: "(var_id, exp) gcfg ⇒ var_id list" where
```

```
  "cfg_vars g =
     sorted_list_of_set (set (concat (map (λ(u, a, w). edge_vars a) (cfg_intra_list g))
                           @ concat (map (λ(u, ca, e, w). call_vars ca) (cfg_calls_list g))))"

type_synonym report =
  "(voblint_conf, mcp_ctx report_context,
    var_id, exp, var_id, mcp_val) analysis_report"
```

One builder for every context policy and placement: the policy contributes how its contexts are ordered and tagged, the placement which globals it keeps at their own unknowns.

```
definition prog_report_of ::
    "voblint_conf ⇒ kind_decls ⇒ imp_prog ⇒ var_id list ⇒ ('c ⇒ order_key)
       ⇒ ('c ⇒ mcp_ctx report_context) ⇒ ('c, var_id, mcp_val) solved_run
       ⇒ report" where
  "prog_report_of conf ds p gs ctx_key ctx_tag sr =
     report_of conf (cfg_vars (elab_cfg p ds)) (elab_cfg p ds) gs
       (prog_obligation_sites ds p) (obligation_classify (conf_sem conf) ds (activation_of (conf_analyses conf)))
       (prog_main_name # prog_procs p) ctx_key ctx_tag
       (mcp_classify (conf_sem conf) ds (activation_of (conf_analyses conf))) sr"
```

At entry-state contexts

An entry-state context is listed in the order of an injective key: each field's formal values, a kind-tagged value keyed by its tag, its kind and its value.

```
fun ikind_key :: "ikind ⇒ order_key" where
  "ikind_key I8 = Key_Int 0"
| "ikind_key U8 = Key_Int 1"
| "ikind_key I16 = Key_Int 2"
| "ikind_key U16 = Key_Int 3"
| "ikind_key I32 = Key_Int 4"
| "ikind_key U32 = Key_Int 5"
| "ikind_key I64 = Key_Int 6"
| "ikind_key U64 = Key_Int 7"

lemma ikind_key_inject [simp]: "ikind_key a = ikind_key b ⟷ a = b"
  by (cases a; cases b) simp_all

fun kd_key :: "('a ⇒ order_key) ⇒ 'a kd ⇒ order_key" where
  "kd_key f KBot = Key_List [Key_Int 0]"
| "kd_key f (KD k a) = Key_List [Key_Int 1, ikind_key k, f a]"
| "kd_key f KTop = Key_List [Key_Int 2]"

lemma kd_key_inject:
  assumes "⋀a b. f a = f b ⟹ a = b"
  shows "kd_key f x = kd_key f y ⟹ x = y"
  using assms by (cases x; cases y) auto

lemma map_kd_key_inject:
  assumes "⋀a b. f a = f b ⟹ a = b"
  shows "map (kd_key f) xs = map (kd_key f) ys ⟹ xs = ys"
  using kd_key_inject[OF assms] by (auto dest: list.inj_map_strong[rotated])

definition int_tuple_key :: "'m int_tuple ⇒ order_key" where
  "int_tuple_key a = Key_List [ivl_key (it_ivl a), congruence_key (it_cong a)]"

lemma int_tuple_key_inject [simp]: "int_tuple_key a = int_tuple_key b ⟷ a = b"
  unfolding int_tuple_key_def by (cases a; cases b) simp

fun mcp_ctx_key :: "mcp_ctx ⇒ order_key" where
  "mcp_ctx_key (Product i (Product c (Product (Product tn (Product to tf)) (Product s (Product q u))))) =
     Key_List [Key_List (map (kd_key ivl_key) i), Key_List (map (kd_key congruence_key) c),
               Key_List (map (kd_key int_tuple_key) tn), Key_List (map (kd_key int_tuple_key) to),
               Key_List (map (kd_key int_tuple_key) tf), Key_List (map (kd_key sign_key) s),
               Key_List (map (kd_key parity_key) q), Key_List (map (λ_. Key_List []) u)]"

lemma inj_mcp_ctx_key: "inj mcp_ctx_key"
proof (rule injI)
  fix a b :: mcp_ctx
  assume eq: "mcp_ctx_key a = mcp_ctx_key b"
  obtain i c tn to tf s q u
    where a:
      "a = Product i (Product c (Product (Product tn (Product to tf)) (Product s (Product q u))))"
    by (metis analysis_product.exhaust)
  obtain i' c' tn' to' tf' s' q' u'
```

```
    where b:
      "b = Product i' (Product c' (Product (Product tn' (Product to' tf')) (Product s' (Product q' u'))))"
    by (metis analysis_product.exhaust)
  from eq have keys:
    "map (kd_key ivl_key) i = map (kd_key ivl_key) i'"
    "map (kd_key congruence_key) c = map (kd_key congruence_key) c'"
    "map (kd_key int_tuple_key) tn = map (kd_key int_tuple_key) tn'"
    "map (kd_key int_tuple_key) to = map (kd_key int_tuple_key) to'"
    "map (kd_key int_tuple_key) tf = map (kd_key int_tuple_key) tf'"
    "map (kd_key sign_key) s = map (kd_key sign_key) s'"
    "map (kd_key parity_key) q = map (kd_key parity_key) q'"
    "map (λ_. Key_List []) u = map (λ_. Key_List []) u'"
    by (simp_all add: a b)
  have "length u = length u'" using keys(8) by (metis length_map)
  then have "u = u'" by (simp add: list_eq_iff_nth_eq)
  moreover have "i = i'" by (rule map_kd_key_inject[OF _ keys(1)]) simp
  moreover have "c = c'" by (rule map_kd_key_inject[OF _ keys(2)]) simp
  moreover have "tn = tn'" by (rule map_kd_key_inject[OF _ keys(3)]) simp
  moreover have "to = to'" by (rule map_kd_key_inject[OF _ keys(4)]) simp
  moreover have "tf = tf'" by (rule map_kd_key_inject[OF _ keys(5)]) simp
  moreover have "s = s'" by (rule map_kd_key_inject[OF _ keys(6)]) simp
  moreover have "q = q'" by (rule map_kd_key_inject[OF _ keys(7)]) simp
  ultimately show "a = b" by (simp add: a b)
qed
```

The run

A run solves the equations of the chosen context policy and placement and reports the answer; the logical `None` branch of the executable solve has no report. Under flow-insensitive placement each declared global lives at its own unknown.

```
definition cs_run :: "voblint_conf ⇒ nat ⇒ kind_decls ⇒ imp_prog ⇒ report option" where
  "cs_run conf k ds p =
     (let sem = conf_sem conf; as = conf_analyses conf; r = conf_globals conf;
          key = (λctx. Key_List (map Key_Node ctx));
          tag = (if k = 0 then (λ_. Report_Unit) else Report_Call_String);
          root = (cfg_exit (elab_cfg p ds), [])
      in case conf_program_globals conf of
           Program_Globals_Flow_Sensitive ⇒
             map_option (λsol. prog_report_of conf ds p [] key tag
                                (mcp_cs.solved_run_of sem ds as k is_global p sol))
               (TD_side_rule_Interp_solve_c r (mcp_cs.equations sem ds as k is_global p) root)
         | Program_Globals_Flow_Insensitive ⇒
             (let gs = declared_globals p
              in map_option (λsol. prog_report_of conf ds p gs key tag
                                    (mcp_split.solved_run_of sem ds as k gs is_global p sol))
                   (TD_side_rule_Interp_solve_c r (mcp_split.equations sem ds as k gs is_global p)
                      root)))"

definition es_run :: "voblint_conf ⇒ kind_decls ⇒ imp_prog ⇒ report option" where
  "es_run conf ds p =
     (let sem = conf_sem conf; as = conf_analyses conf; r = conf_globals conf;
          root = (cfg_exit (elab_cfg p ds), mcp_root_ctx)
      in case conf_program_globals conf of
           Program_Globals_Flow_Sensitive ⇒
             map_option (λsol. prog_report_of conf ds p [] mcp_ctx_key Report_Entry
                                (mcp_es.solved_run_of sem ds as is_global p sol))
               (TD_side_rule_Interp_solve_c r (mcp_es.equations sem ds as is_global p) root)
         | Program_Globals_Flow_Insensitive ⇒
             (let gs = declared_globals p
              in map_option (λsol. prog_report_of conf ds p gs mcp_ctx_key Report_Entry
                                    (mcp_split_es.solved_run_of sem ds as gs is_global p sol))
                   (TD_side_rule_Interp_solve_c r (mcp_split_es.equations sem ds as gs is_global p)
                      root)))"

definition report_run :: "voblint_conf ⇒ kind_decls ⇒ imp_prog ⇒ report option" where
  "report_run conf ds p =
     (case conf_context conf of
        Call_String k ⇒ cs_run conf k ds p
      | Entry_State ⇒ es_run conf ds p)"
```

What the builder keeps

Every state the solved table holds at a node, at any context, is listed among the report's states for that node.

```
lemma report_of_state_mem:
```

```
  assumes fin: "finite (covered_keys (run_table sr))" and inj: "inj ctx_key"
    and look: "lookup_table (run_table sr) v ctx = Lifted d"
  shows "Lifted d ∈ report_states_at
           (report_of config vars g globals sites cond procs ctx_key ctx_tag classify sr) v"
proof -
  have "lookup_table (run_table sr) v ctx
          ∈ lookup_table (run_table sr) v ‘ table_contexts (run_table sr) v"
    by (rule imageI[OF lookup_table_LiftedD[OF look]])
  then show ?thesis by (simp only: report_states_at_report_of[OF fin inj] look)
qed
```

Soundness

A report is sound for every store that reaches a node from a store whose globals are zero at their kinds. Its diagnostics are sound for the same stores. A point no diagnostic names meets every obligation of the expressions evaluated there at every store that reaches it; a refuted obligation fails at every such store. Both read the diagnostics off the solved table through `obligation_classify`, whichever context policy filed it.

```
lemma table_diagnostics_sound:
  fixes rt :: "('c, mcp_val) solved_table"
  assumes fin: "finite (table_contexts rt v)"
    and look: "lookup_table rt v ctx = Lifted st"
    and s: "s ∈ mcp.act_gamma ds as st" and kinds: "in_kinds (kenv_of ds) s"
  shows table_diagnostics_safe:
      "(v, obs) ∈ set sites ⟹ ob ∈ set obs
         ⟹ ∀d ∈ set (arithmetic_diagnostics sites rt (obligation_classify sem ds as)).
              diagnostic_point d ≠ v
         ⟹ s ⊨ ob"
    and table_diagnostics_refuted:
      "d ∈ set (arithmetic_diagnostics sites rt (obligation_classify sem ds as))
         ⟹ diagnostic_verdict d = Check_Refuted ⟹ diagnostic_point d = v
         ⟹ ¬ s ⊨ diagnostic_obligation d"
proof -
  let ?cl = "obligation_classify sem ds as"
  show "s ⊨ ob"
    if site: "(v, obs) ∈ set sites" and ob: "ob ∈ set obs"
      and absent: "∀d ∈ set (arithmetic_diagnostics sites rt ?cl). diagnostic_point d ≠ v"
  proof -
    have "point_verdict rt ?cl v ob = Lifted Check_Proved"
      using arithmetic_diagnostics_absent[OF site ob absent]
        point_verdict_not_dead[OF fin look, of ?cl ob]
      by auto
    then have "?cl ob st = Check_Proved"
      by (rule point_verdict_decided[OF _ _ look]) simp
    then show ?thesis by (rule obligation_classify_sound(1)[OF s kinds])
  qed
  show "¬ s ⊨ diagnostic_obligation d"
    if d: "d ∈ set (arithmetic_diagnostics sites rt ?cl)"
      and r: "diagnostic_verdict d = Check_Refuted" and v: "diagnostic_point d = v"
  proof -
    have "point_verdict rt ?cl v (diagnostic_obligation d) = Lifted Check_Refuted"
      using arithmetic_diagnostics_refuted[OF d r] v by simp
    then have "?cl (diagnostic_obligation d) st = Check_Refuted"
      by (rule point_verdict_decided[OF _ _ look]) simp
    then show ?thesis by (rule obligation_classify_sound(2)[OF s kinds])
  qed
qed
```

Everything a report guarantees, in one predicate: it lists a row for every check edge, and at every store that reaches a node the store lies in a state the report lists there, no check there is dead, a proved check holds and a refuted one fails, a point no diagnostic names meets every obligation decided there, and a refuted obligation fails.

```
definition report_sound :: "voblint_conf ⇒ kind_decls ⇒ imp_prog ⇒ report ⇒ bool" where
  "report_sound conf ds p res ⟷
     (∀v l e w. (v, EA_Check l e, w) ∈ intra (elab_cfg p ds) ⟶
        (∃chk ∈ set (report_checks res). check_point chk = v ∧ check_label chk = l ∧ check_exp chk = e))
   ∧ (∀v s. s ∈ reaching ds p v ⟶
        (∃d. Lifted d ∈ report_states_at res v ∧ s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d)
      ∧ (∀chk ∈ set (report_checks res). check_point chk = v ⟶
           check_verdict chk ≠ Dead
         ∧ (check_verdict chk = Decided Check_Proved ⟶ truthy (eval (check_exp chk) s))
         ∧ (check_verdict chk = Decided Check_Refuted ⟶ ¬ truthy (eval (check_exp chk) s)))
      ∧ (∀obs ob. (v, obs) ∈ set (prog_obligation_sites ds p) ⟶ ob ∈ set obs ⟶ SAFE res v ⟶ s ⊨ ob)
      ∧ (∀d ∈ set (report_diagnostics res). diagnostic_verdict d = Check_Refuted
```

```
          ⟶ diagnostic_point d = v ⟶ ¬ s ⊨ diagnostic_obligation d))"
```

A report built from a solved table is sound when the table covers every reaching store, whichever context policy and placement filed it. The table's states are listed and its verdicts reported, so each guarantee is read off the state a reaching store lies in. A well-formed program keeps every reaching store within its kinds, which the diagnostics read.

```
lemma prog_report_of_sound:
  fixes sr :: "('c, var_id, mcp_val) solved_run"
  assumes wf: "wf_program_compile_input p"
    and fin: "finite (covered_keys (run_table sr))" and inj: "inj ctx_key"
    and node: "⋀v. reaching ds p v ⊆ (⋃ctx. mcp.act_gamma ds (activation_of (conf_analyses conf))
                (case lookup_table (run_table sr) v ctx of Bot ⇒ ⊥ | Lifted st ⇒ st))"
  shows "report_sound conf ds p (prog_report_of conf ds p gs ctx_key ctx_tag sr)"
proof -
  let ?res = "prog_report_of conf ds p gs ctx_key ctx_tag sr"
  let ?rt = "run_table sr"
  let ?as = "activation_of (conf_analyses conf)" and ?sem = "conf_sem conf"
  let ?cl = "mcp_classify ?sem ds ?as"
  have checks: "report_checks ?res = result_checks_of (elab_cfg p ds) ?rt ?cl"
    and diags: "report_diagnostics ?res
                  = arithmetic_diagnostics (prog_obligation_sites ds p) ?rt (obligation_classify ?sem ds ?as)"
    by (simp_all add: prog_report_of_def report_of_def Let_def)
  have fin_g: "finite (intra (elab_cfg p ds))" by (simp add: elab_cfg_def compile_prog_finite)
  have rows: "∃chk ∈ set (report_checks ?res). check_point chk = v ∧ check_label chk = l
                ∧ check_exp chk = e"
    if "(v, EA_Check l e, w) ∈ intra (elab_cfg p ds)" for v l e w
    using that fin_g by (force simp: checks result_checks_of_def)
  have at: "(∃d. Lifted d ∈ report_states_at ?res v ∧ s ∈ mcp.act_gamma ds ?as d)
      ∧ (∀chk ∈ set (report_checks ?res). check_point chk = v ⟶
           check_verdict chk ≠ Dead
         ∧ (check_verdict chk = Decided Check_Proved ⟶ truthy (eval (check_exp chk) s))
         ∧ (check_verdict chk = Decided Check_Refuted ⟶ ¬ truthy (eval (check_exp chk) s)))
      ∧ (∀obs ob. (v, obs) ∈ set (prog_obligation_sites ds p) ⟶ ob ∈ set obs ⟶ SAFE ?res v ⟶ s ⊨ ob)
      ∧ (∀d ∈ set (report_diagnostics ?res). diagnostic_verdict d = Check_Refuted
           ⟶ diagnostic_point d = v ⟶ ¬ s ⊨ diagnostic_obligation d)"
    if s: "s ∈ reaching ds p v" for v s
  proof -
    obtain ctx st where look: "lookup_table ?rt v ctx = Lifted st"
      and st: "s ∈ mcp.act_gamma ds ?as st"
      using node[of v] s by (auto simp: mcp.act_gamma_bot split: lifted.splits)
    have fin_v: "finite (table_contexts ?rt v)" using fin unfolding table_contexts_def by simp
    have states: "Lifted st ∈ report_states_at ?res v"
      unfolding prog_report_of_def by (rule report_of_state_mem[OF fin inj look])
    have check: "check_verdict chk ≠ Dead
       ∧ (check_verdict chk = Decided Check_Proved ⟶ truthy (eval (check_exp chk) s))
       ∧ (check_verdict chk = Decided Check_Refuted ⟶ ¬ truthy (eval (check_exp chk) s))"
      if mem: "chk ∈ set (report_checks ?res)" and pt: "check_point chk = v" for chk
    proof -
      from mem obtain u and a :: "(var_id, exp) gedge_action" where chk0: "chk = (|check_point = u, check_label =
ea_check_label a,
          check_exp = ea_check_cond a, check_verdict = point_verdict ?rt ?cl u (ea_check_cond a)|)"
        unfolding checks result_checks_of_def by auto
      with pt have chk: "chk = (|check_point = v, check_label = ea_check_label a,
          check_exp = ea_check_cond a, check_verdict = point_verdict ?rt ?cl v (ea_check_cond a)|)"
        by simp
      have decided: "?cl (ea_check_cond a) st = r"
        if "point_verdict ?rt ?cl v (ea_check_cond a) = Decided r" and "r ≠ Check_Unknown" for r
        by (rule point_verdict_decided[OF that look])
      show ?thesis
        using point_verdict_not_dead[OF fin_v look, of ?cl "ea_check_cond a"]
          decided[of Check_Proved] decided[of Check_Refuted]
          mcp.mcp_classify_proved[OF st] mcp.mcp_classify_refuted[OF st]
        by (auto simp: chk)
    qed
    have kinds: "in_kinds (kenv_of ds) s"
      using node_collect_in_kinds[OF calls_in_kinds_elab_cfg[OF wf] init_stores_in_kinds] s
      by blast
    note diag = table_diagnostics_sound[OF fin_v look st kinds, where sem = ?sem]
    show ?thesis using states st check diag unfolding diags by blast
  qed
  show ?thesis unfolding report_sound_def using rows at by blast
qed
```

Every answered run is sound. The executable solve's answer is the run the typed theorems are about (`solve_c_run`), so its table covers every reaching store (`mcp_cs_node_sound` and its siblings for the other policy and placement).

```
lemma report_run_sound:
  assumes wf: "wf_program_compile_input p" and run: "report_run conf ds p = Some res"
  shows "report_sound conf ds p res"
proof -
  let ?sem = "conf_sem conf" and ?as = "conf_analyses conf" and ?r = "conf_globals conf"
  let ?gs = "declared_globals p"
  show ?thesis
  proof (cases "conf_context conf")
    case (Call_String k)
    let ?key = "λctx. Key_List (map Key_Node ctx)"
      and ?tag = "if k = 0 then (λ_. Report_Unit) else Report_Call_String"
    show ?thesis
    proof (cases "conf_program_globals conf")
      case Program_Globals_Flow_Sensitive
      from run obtain sol where
        sol: "TD_side_rule_Interp_solve_c ?r (mcp_cs.equations ?sem ds ?as k is_global p)
              (mcp_cs.root_query ds p) = Some sol"
        and res: "res = prog_report_of conf ds p [] ?key ?tag
                    (mcp_cs.solved_run_of ?sem ds ?as k is_global p sol)"
        by (auto simp: report_run_def cs_run_def Call_String
            Program_Globals_Flow_Sensitive Let_def dg_pipeline.root_query_def)
      note solved = mcp_cs.solve_c_run[OF sol]
      show ?thesis
        unfolding res
        by (rule prog_report_of_sound[OF wf])
           (use solved mcp_cs.vars_finite_of_terminates[OF solved(1)]
              mcp_cs_node_sound[OF wf solved(1), unfolded mcp_cs.state_at_unfold] in
            ‹simp_all add: dg_pipeline.result_def dg_pipeline.sol_vars_def inj_call_string_key›)
    next
      case Program_Globals_Flow_Insensitive
      from run obtain sol where
        sol: "TD_side_rule_Interp_solve_c ?r (mcp_split.equations ?sem ds ?as k ?gs is_global p)
              (mcp_split.root_query ds p) = Some sol"
        and res: "res = prog_report_of conf ds p ?gs ?key ?tag
                    (mcp_split.solved_run_of ?sem ds ?as k ?gs is_global p sol)"
        by (auto simp: report_run_def cs_run_def Call_String
            Program_Globals_Flow_Insensitive Let_def dg_pipeline.root_query_def)
      note solved = mcp_split.solve_c_run[OF sol]
      show ?thesis
        unfolding res
        by (rule prog_report_of_sound[OF wf])
           (use solved mcp_split.vars_finite_of_terminates[OF solved(1)]
              mcp_split_node_sound[OF wf solved(1), unfolded mcp_split.state_at_unfold] in
            ‹simp_all add: dg_pipeline.result_def dg_pipeline.sol_vars_def inj_call_string_key›)
    qed
  next
    case Entry_State
    show ?thesis
    proof (cases "conf_program_globals conf")
      case Program_Globals_Flow_Sensitive
      from run obtain sol where
        sol: "TD_side_rule_Interp_solve_c ?r (mcp_es.equations ?sem ds ?as is_global p)
              (mcp_es.root_query ds p) = Some sol"
        and res: "res = prog_report_of conf ds p [] mcp_ctx_key Report_Entry
                    (mcp_es.solved_run_of ?sem ds ?as is_global p sol)"
        by (auto simp: report_run_def es_run_def Entry_State
            Program_Globals_Flow_Sensitive Let_def dg_pipeline.root_query_def)
      note solved = mcp_es.solve_c_run[OF sol]
      show ?thesis
        unfolding res
        by (rule prog_report_of_sound[OF wf])
           (use solved mcp_es.vars_finite_of_terminates[OF solved(1)]
              mcp_es_node_sound[OF wf solved(1), unfolded mcp_es.state_at_unfold] in
            ‹simp_all add: dg_pipeline.result_def dg_pipeline.sol_vars_def inj_mcp_ctx_key›)
    next
      case Program_Globals_Flow_Insensitive
      from run obtain sol where
        sol: "TD_side_rule_Interp_solve_c ?r (mcp_split_es.equations ?sem ds ?as ?gs is_global p)
              (mcp_split_es.root_query ds p) = Some sol"
        and res: "res = prog_report_of conf ds p ?gs mcp_ctx_key Report_Entry
                    (mcp_split_es.solved_run_of ?sem ds ?as ?gs is_global p sol)"
        by (auto simp: report_run_def es_run_def Entry_State
            Program_Globals_Flow_Insensitive Let_def dg_pipeline.root_query_def)
      note solved = mcp_split_es.solve_c_run[OF sol]
      show ?thesis
        unfolding res
```

```
      by (rule prog_report_of_sound[OF wf])
         (use solved mcp_split_es.vars_finite_of_terminates[OF solved(1)]
            mcp_split_es_node_sound[OF wf solved(1), unfolded mcp_split_es.state_at_unfold] in
          ‹simp_all add: dg_pipeline.result_def dg_pipeline.sol_vars_def inj_mcp_ctx_key›)
    qed
  qed
qed
```

The guarantees one at a time, as the public theorems read them.

```
theorem report_check_sound:
  assumes wf: "wf_program_compile_input p"
    and run: "report_run conf ds p = Some res"
    and chk: "chk ∈ set (report_checks res)"
    and s: "s ∈ reaching ds p (check_point chk)"
  shows "check_verdict chk = Decided Check_Proved ⟹ truthy (eval (check_exp chk) s)"
    and "check_verdict chk = Decided Check_Refuted ⟹ ¬ truthy (eval (check_exp chk) s)"
  using report_run_sound[OF wf run] chk s unfolding report_sound_def by blast+

theorem report_state_sound:
  assumes wf: "wf_program_compile_input p"
    and run: "report_run conf ds p = Some res"
    and s: "s ∈ reaching ds p v"
  obtains d where "Lifted d ∈ report_states_at res v"
    and "s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d"
  using report_run_sound[OF wf run] s that unfolding report_sound_def by blast

theorem report_point_sound:
  assumes wf: "wf_program_compile_input p"
    and run: "report_run conf ds p = Some res"
    and s: "s ∈ reaching ds p v"
  shows report_obligation_safe:
      "(v, obs) ∈ set (prog_obligation_sites ds p) ⟹ ob ∈ set obs ⟹ SAFE res v ⟹ s ⊨ ob"
    and report_obligation_refuted:
      "d ∈ set (report_diagnostics res) ⟹ diagnostic_verdict d = Check_Refuted
         ⟹ diagnostic_point d = v ⟹ ¬ s ⊨ diagnostic_obligation d"
    and report_check_live:
      "chk ∈ set (report_checks res) ⟹ check_point chk = v ⟹ check_verdict chk ≠ Dead"
  using report_run_sound[OF wf run] s unfolding report_sound_def by blast+
```

A report lists a check row for every check edge of the elaborated graph, at the edge's source and with its label and condition.

```
lemma report_check_row:
  assumes wf: "wf_program_compile_input p"
    and run: "report_run conf ds p = Some res"
    and edge: "(v, EA_Check l e, w) ∈ intra (elab_cfg p ds)"
  shows "∃chk ∈ set (report_checks res). check_point chk = v ∧ check_label chk = l
           ∧ check_exp chk = e"
  using report_run_sound[OF wf run] edge unfolding report_sound_def by blast
```

Every obligation of an expression an edge or a call evaluates is one the report decides at the edge's or the call's source, every conversion of a call's result is one it decides where the callee returns, and every conversion of a library result one it decides at the library call.

```
lemma prog_obligation_site:
  assumes "(v, es) ∈ set (arithmetic_expression_sites (elab_cfg p ds))"
    and "e ∈ set es" and "ob ∈ set (arith_obligations e)"
  shows "∃obs. (v, obs) ∈ set (prog_obligation_sites ds p) ∧ ob ∈ set obs"
  using assms unfolding prog_obligation_sites_def obligation_sites_def by force

lemma prog_obligation_site_intra:
  assumes "(v, a, w) ∈ intra (elab_cfg p ds)"
    and "e ∈ set (edge_expressions a)" and "ob ∈ set (arith_obligations e)"
  shows "∃obs. (v, obs) ∈ set (prog_obligation_sites ds p) ∧ ob ∈ set obs"
  using assms
  by (intro prog_obligation_site[OF arithmetic_expression_sites_intra])
     (simp_all add: elab_cfg_def compile_prog_finite)

lemma prog_obligation_site_call:
  assumes "(v, ca, ce, k) ∈ calls (elab_cfg p ds)"
    and "e ∈ set (ce_args ca)" and "ob ∈ set (arith_obligations e)"
  shows "∃obs. (v, obs) ∈ set (prog_obligation_sites ds p) ∧ ob ∈ set obs"
  using assms
  by (intro prog_obligation_site[OF arithmetic_expression_sites_call])
     (simp_all add: elab_cfg_def compile_prog_finite)
```

```
lemma prog_obligation_site_return:
  assumes call: "(u, ca, ce, k) ∈ calls (elab_cfg p ds)" and dst: "ce_dst ca = Some x"
    and ret: "(v, EA_Ret (Some e) (callee_of_entry ce), w) ∈ intra (elab_cfg p ds)"
    and narrows: "¬ ik_widens (exp_kind e) (kenv_of ds x)"
  shows "(v, [Arithmetic_Obligation Conversion_Overflow (TCast (kenv_of ds x) e) e])
          ∈ set (prog_obligation_sites ds p)"
proof -
  have fin: "finite (intra (elab_cfg p ds))" "finite (calls (elab_cfg p ds))"
    by (simp_all add: elab_cfg_def compile_prog_finite)
  have "(u, ca, ce, k) ∈ set (cfg_calls_list (elab_cfg p ds))"
    and "(v, EA_Ret (Some e) (callee_of_entry ce), w) ∈ set (cfg_intra_list (elab_cfg p ds))"
    using call ret fin by simp_all
  then show ?thesis
    unfolding prog_obligation_sites_def return_sites_def Let_def
    using dst narrows by (force simp: image_iff)
qed

lemma prog_obligation_site_special:
  assumes "(v, EA_Special sc x, w) ∈ intra (elab_cfg p ds)"
    and "ob ∈ set (special_obligations (kenv_of ds x) sc)"
  shows "(v, special_obligations (kenv_of ds x) sc) ∈ set (prog_obligation_sites ds p)"
proof -
  have mem: "(v, EA_Special sc x, w) ∈ set (cfg_intra_list (elab_cfg p ds))"
    using assms(1) by (simp add: elab_cfg_def compile_prog_finite)
  have "special_obligations (kenv_of ds x) sc ≠ []" using assms(2) by auto
  then have "(v, special_obligations (kenv_of ds x) sc) ∈ set (special_sites ds p)"
    unfolding special_sites_def by (auto intro!: bexI[OF _ mem])
  then show ?thesis by (simp add: prog_obligation_sites_def)
qed
```

What an obligation of each hazard says about a store.

```
lemma obligation_holds_overflow [simp]:
  "t ⊨ Arithmetic_Obligation Signed_Overflow (TArith k op a b) x
     ⟷ aop_eval op (eval a t) (eval b t) ∈ ik_range k"
  by (simp add: obligation_holds_def Let_def)

lemma obligation_holds_conversion [simp]:
  "t ⊨ Arithmetic_Obligation Conversion_Overflow (TCast k a) x
     ⟷ eval a t ∈ ik_range k"
  "t ⊨ Arithmetic_Obligation Conversion_Overflow (TN k n) x ⟷ n ∈ ik_range k"
  by (simp_all add: obligation_holds_def Let_def)

lemma obligation_holds_remainder [simp]:
  "t ⊨ Arithmetic_Obligation Remainder_Overflow (TArith k AMod a b) (TArith k ADiv a b)
     ⟷ c_div (eval a t) (eval b t) ∈ ik_range k"
  by (simp add: obligation_holds_def Let_def)

lemma obligation_holds_min_max [simp]:
  "t ⊨ Arithmetic_Obligation Min_Conversion_Overflow (TCast k a) b
     ⟷ min (eval a t) (eval b t) ∈ ik_range k"
  "t ⊨ Arithmetic_Obligation Max_Conversion_Overflow (TCast k a) b
     ⟷ max (eval a t) (eval b t) ∈ ik_range k"
  by (simp_all add: obligation_holds_def Let_def)

lemma obligation_holds_division [simp]:
  "t ⊨ Arithmetic_Obligation Division_By_Zero e b ⟷ eval b t ≠ 0"
  by (simp add: obligation_holds_def Let_def)
```

An expression raises an obligation for each of its nodes that can fail, at any depth.

```
fun subexps :: "exp ⇒ exp set" where
  "subexps (TArith k op a b) = insert (TArith k op a b) (subexps a ∪ subexps b)"
| "subexps (TCast k a) = insert (TCast k a) (subexps a)"
| "subexps (TCmp c a b) = insert (TCmp c a b) (subexps a ∪ subexps b)"
| "subexps (TNot a) = insert (TNot a) (subexps a)"
| "subexps (TAnd a b) = insert (TAnd a b) (subexps a ∪ subexps b)"
| "subexps (TOr a b) = insert (TOr a b) (subexps a ∪ subexps b)"
| "subexps e = {e}"

lemma arith_obligations_overflow:
  "TArith k op a b ∈ subexps e ⟹ ik_signed k ⟹ op ≠ AMod
     ⟹ Arithmetic_Obligation Signed_Overflow (TArith k op a b) (TArith k op a b)
          ∈ set (arith_obligations e)"
  by (induction e) auto

lemma arith_obligations_remainder:
```

```
  "TArith k AMod a b ∈ subexps e ⟹ ik_signed k
     ⟹ Arithmetic_Obligation Remainder_Overflow (TArith k AMod a b) (TArith k ADiv a b)
          ∈ set (arith_obligations e)"
  by (induction e) auto

lemma arith_obligations_conversion:
  "TCast k a ∈ subexps e ⟹ ¬ ik_widens (exp_kind a) k
     ⟹ Arithmetic_Obligation Conversion_Overflow (TCast k a) a ∈ set (arith_obligations e)"
  by (induction e) auto
```

Rendering a report

A report is rendered by `render_report` with a renderer of its own: each active analysis shows a kind-tagged value per variable, the order analysis its facts, and an entry-state context the formal values of its active fields. An edge's footprint names the globals it reads and writes among those kept at their own unknowns: none under `Program_Globals_Flow_Sensitive`, the declared globals it touches under `Program_Globals_Flow_Insensitive`.

```
instantiation order_rel :: (type) join_val
begin
definition vjoin_order_rel :: "'v order_rel ⇒ 'v order_rel ⇒ 'v order_rel" where
  "vjoin_order_rel = sup"
instance ..
end
```

An Int value prints the same at every refinement mode, so the report drops the mode.

```
definition it_untag :: "'m int_tuple ⇒ unit int_tuple" where
  "it_untag a = IT (it_ivl a) (it_cong a)"

datatype report_value =
    Interval_Value "ivl kd"
  | Congruence_Value "congruence kd"
  | Int_Value "unit int_tuple kd"
  | Sign_Value "sign kd"
  | Parity_Value "parity kd"
  | Order_Value "(vname × vname) list option"

fun string_of_report_value :: "report_value ⇒ String.literal" where
  "string_of_report_value (Interval_Value v) = to_string v"
| "string_of_report_value (Congruence_Value v) = to_string v"
| "string_of_report_value (Int_Value v) = to_string v"
| "string_of_report_value (Sign_Value v) = to_string v"
| "string_of_report_value (Parity_Value v) = to_string v"
| "string_of_report_value (Order_Value v) = order_view_string v"
```

A variable is shown by its name, qualified by its procedure when it is local.

```
fun var_label :: "var_id ⇒ vname" where
  "var_label (GlobalId x) = x"
| "var_label (ScopedId f x) = f + STR ''::'' + x"
| "var_label (ReturnId f) = f + STR ''::return''"
| "var_label ReturnVar = STR ''return''"

definition field_view ::
    "('a kd ⇒ report_value) ⇒ var_id list ⇒ 'a tabs lifted ⇒ report_value field_state" where
  "field_view mk vars d =
     Field_Store (map (λx. (var_label x, mk (case d of Bot ⇒ KBot | Lifted σ ⇒ σ x))) vars)"
```

An order fact relates variables of one procedure's state, so it names each by its own name.

```
fun order_var_label :: "var_id ⇒ vname" where
  "order_var_label (ScopedId f x) = x"
| "order_var_label x = var_label x"

definition order_field_view :: "var_id order_rel lifted ⇒ report_value field_state" where
  "order_field_view d =
     Field_Whole (Order_Value (case d of
        Lifted (OFacts _ ps) ⇒
          Some (map (λ(a, b). (order_var_label a, order_var_label b)) (sorted_list_of_set ps))
      | _ ⇒ None))"

definition render ::
    "analysis list ⇒ var_id list ⇒ mcp_val ⇒ (analysis, report_value) analysis_view" where
  "render as vars v = map (λa. (a, case a of
      Interval ⇒ field_view Interval_Value vars (slot1 v)
    | Congruence ⇒ field_view Congruence_Value vars (slot2 v)
    | Ana_Int Refine_Never ⇒ field_view (Int_Value ∘ map_kd it_untag) vars (slot3n v)
```

```
    | Ana_Int Refine_Once ⇒ field_view (Int_Value ∘ map_kd it_untag) vars (slot3o v)
    | Ana_Int Refine_Fixpoint ⇒ field_view (Int_Value ∘ map_kd it_untag) vars (slot3f v)
    | Sign ⇒ field_view Sign_Value vars (slot4 v)
    | Parity ⇒ field_view Parity_Value vars (slot5 v)
    | Order ⇒ order_field_view (slot6 v))) as"

fun context_values :: "analysis list ⇒ mcp_ctx ⇒ report_value list" where
  "context_values as (Product i (Product c (Product (Product tn (Product to tf)) (Product s (Product q u))))) =
     concat (map (λa. case a of
         Interval ⇒ map Interval_Value i
       | Congruence ⇒ map Congruence_Value c
       | Ana_Int Refine_Never ⇒ map (Int_Value ∘ map_kd it_untag) tn
       | Ana_Int Refine_Once ⇒ map (Int_Value ∘ map_kd it_untag) to
       | Ana_Int Refine_Fixpoint ⇒ map (Int_Value ∘ map_kd it_untag) tf
       | Sign ⇒ map Sign_Value s
       | Parity ⇒ map Parity_Value q
       | Order ⇒ []) as)"
```

The globals an edge or a call reads and writes, among those kept at their own unknowns.

```
definition footprints :: "(var_id, exp, var_id) footprints" where
  "footprints =
     (| fp_edge_reads = edge_global_reads, fp_edge_writes = edge_global_writes,
       fp_call_reads = call_global_reads,
       fp_call_writes = (λG ca. case ca of CallEdge dst pars args ⇒
         global_names_in G (case_option [] (λx. [x]) dst @ pars)) |)"

definition renderer ::
    "(voblint_conf, mcp_ctx, mcp_val, var_id, exp, var_id,
      analysis, report_value) report_renderer" where
  "renderer =
     (| rr_view = (λconf. render (activation_of (conf_analyses conf))),
       rr_context = (λconf. context_values (activation_of (conf_analyses conf))),
       rr_footprints = footprints |)"

definition prog_render_report ::
    "(report_value ⇒ 'v) ⇒ report ⇒ (var_id, exp, var_id, analysis, 'v) run_result" where
  "prog_render_report = render_report renderer"

end
```

17.10 Run_Voblint

```
theory Run_Voblint
  imports Run_Report
begin
```

The public analysis operation

`run_voblint` analyzes a program under its kind declarations with the options of a `voblint_conf`: the activated analyses, a context policy, a global update rule, where program globals live, and the options the domains read, as Goblint's configuration tree selects them (`ana.activated`, `sem.int.signed_overflow`). It answers the report `report_of` builds from the solve. An activation is valid when it names at least one analysis and none twice.

```
definition valid_activation :: "analysis list ⇒ bool" where
  "valid_activation as ⟷ as ≠ [] ∧ distinct as"

definition run_voblint :: "voblint_conf ⇒ kind_decls ⇒ imp_prog ⇒ report analysis_answer" where
  "run_voblint conf ds p =
     (if ¬ valid_activation (conf_analyses conf) then Invalid_Activation
      else if ¬ wf_program_compile_input_exec p then Malformed_Program
      else case report_run conf ds p of
             None ⇒ No_Answer
           | Some res ⇒ Analysed res)"
```

What an answer guarantees

The headline theorem, about the report a caller receives. Its stores are the typed stores that reach a node from a store whose globals are zero at their kinds. Every such store lies in one of the states the report lists at the node, read through the concretization of the activated analyses; a proved check holds at it and a refuted one fails. Neither verdict says that the node is reached, and an unknown verdict says nothing. The theorem holds for every configuration. It assumes only that the run answered: termination of the solve is not a premise, because the executable solve answers only when it has terminated.

```
theorem run_voblint_sound:
  assumes run: "run_voblint conf ds p = Analysed res"
  shows run_voblint_states:
      "⋀v s. s ∈ reaching ds p v
         ⟹ ∃d. Lifted d ∈ report_states_at res v
              ∧ s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d"
    and run_voblint_proved:
      "⋀chk s. chk ∈ set (report_checks res) ⟹ check_verdict chk = Decided Check_Proved
         ⟹ s ∈ reaching ds p (check_point chk) ⟹ truthy (eval (check_exp chk) s)"
    and run_voblint_refuted:
      "⋀chk s. chk ∈ set (report_checks res) ⟹ check_verdict chk = Decided Check_Refuted
         ⟹ s ∈ reaching ds p (check_point chk) ⟹ ¬ truthy (eval (check_exp chk) s)"
proof -
  have exec: "wf_program_compile_input_exec p"
    and rep: "report_run conf ds p = Some res"
    using run by (auto simp: run_voblint_def split: if_splits option.splits)
  note wf = wf_program_compile_input_exec_sound[OF exec]
  show "⋀v s. s ∈ reaching ds p v
          ⟹ ∃d. Lifted d ∈ report_states_at res v
               ∧ s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d"
    by (erule report_state_sound[OF wf rep]) blast
  show "⋀chk s. chk ∈ set (report_checks res) ⟹ check_verdict chk = Decided Check_Proved
          ⟹ s ∈ reaching ds p (check_point chk) ⟹ truthy (eval (check_exp chk) s)"
    using report_check_sound(1)[OF wf rep] by blast
  show "⋀chk s. chk ∈ set (report_checks res) ⟹ check_verdict chk = Decided Check_Refuted
          ⟹ s ∈ reaching ds p (check_point chk) ⟹ ¬ truthy (eval (check_exp chk) s)"
    using report_check_sound(2)[OF wf rep] by blast
qed
```

An answer was given for a valid activation and a well-formed program, and its report records the configuration it was run with.

```
lemma run_voblint_AnalysedE:
  assumes "run_voblint conf ds p = Analysed res"
  obtains "valid_activation (conf_analyses conf)" and "wf_program_compile_input_exec p"
    and "report_run conf ds p = Some res"
  using assms unfolding run_voblint_def by (auto split: if_splits option.splits)

lemma report_run_shape:
  assumes "report_run conf ds p = Some res"
  shows "report_config res = conf" and "report_cfg res = elab_cfg p ds"
```

```
  using assms
  by (auto simp: report_run_def cs_run_def es_run_def prog_report_of_def
      report_of_def Let_def split: context_policy.splits program_globals.splits)
```

The report contract in one statement: an analysed answer was given for a valid activation and a well-formed program, and is a report of exactly that configuration and the program's elaborated graph. The theorems that follow read it.

```
theorem run_voblint_report_contract:
  assumes "run_voblint conf ds p = Analysed res"
  shows "valid_activation (conf_analyses conf)" and "wf_program_compile_input_exec p"
    and "report_config res = conf" and "report_cfg res = elab_cfg p ds"
  using assms by (auto elim!: run_voblint_AnalysedE dest: report_run_shape)
```

What the arithmetic diagnostics guarantee

A point the report's diagnostics do not name is safe: at every store that reaches it, every obligation of every expression an edge or a call there evaluates holds, every value returned there fits the kind each caller converts it to, and every `min` or `max` computed there fits the kind of its destination. No divisor is zero, no signed operation computes a result outside its kind, no signed remainder has a quotient outside its kind, and no conversion narrower than its source kind converts a value outside its target. A diagnostic says nothing on its own unless it is refuted: then its obligation fails at every store that reaches its point, so a run that reaches the point violates it. The run is one of core VIMP, which evaluates both operands of `&&` and `||`: a C program that short-circuits need not evaluate the operand whose obligation is refuted. Both hold under either overflow policy, which changes only how precisely the analyses bound a wrapped value.

```
theorem run_voblint_arithmetic_safe:
  assumes run: "run_voblint conf ds p = Analysed res"
    and absent: "SAFE res v"
    and s: "s ∈ reaching ds p v"
  shows run_voblint_arithmetic_safe_intra:
      "(v, a, w) ∈ intra (elab_cfg p ds) ⟹ e ∈ set (edge_expressions a)
         ⟹ ob ∈ set (arith_obligations e) ⟹ s ⊨ ob"
    and run_voblint_arithmetic_safe_call:
      "(v, ca, ce, k) ∈ calls (elab_cfg p ds) ⟹ e ∈ set (ce_args ca)
         ⟹ ob ∈ set (arith_obligations e) ⟹ s ⊨ ob"
    and run_voblint_arithmetic_safe_return:
      "(u, ca, ce, k) ∈ calls (elab_cfg p ds) ⟹ ce_dst ca = Some x
         ⟹ (v, EA_Ret (Some e) (callee_of_entry ce), w) ∈ intra (elab_cfg p ds)
         ⟹ ¬ ik_widens (exp_kind e) (kenv_of ds x)
         ⟹ eval e s ∈ ik_range (kenv_of ds x)"
    and run_voblint_arithmetic_safe_min:
      "(v, EA_Special (Min e1 e2) x, w) ∈ intra (elab_cfg p ds)
         ⟹ ¬ (ik_widens (exp_kind e1) (kenv_of ds x) ∧ ik_widens (exp_kind e2) (kenv_of ds x))
         ⟹ min (eval e1 s) (eval e2 s) ∈ ik_range (kenv_of ds x)"
    and run_voblint_arithmetic_safe_max:
      "(v, EA_Special (Max e1 e2) x, w) ∈ intra (elab_cfg p ds)
         ⟹ ¬ (ik_widens (exp_kind e1) (kenv_of ds x) ∧ ik_widens (exp_kind e2) (kenv_of ds x))
         ⟹ max (eval e1 s) (eval e2 s) ∈ ik_range (kenv_of ds x)"
proof -
  from run have wf: "wf_program_compile_input p"
    and rep: "report_run conf ds p = Some res"
    by (auto elim!: run_voblint_AnalysedE intro: wf_program_compile_input_exec_sound)
  note safe = report_obligation_safe[OF wf rep s _ _ absent]
  show "s ⊨ ob"
    if "(v, a, w) ∈ intra (elab_cfg p ds)" "e ∈ set (edge_expressions a)"
      "ob ∈ set (arith_obligations e)"
    using prog_obligation_site_intra[OF that] safe by blast
  show "s ⊨ ob"
    if "(v, ca, ce, k) ∈ calls (elab_cfg p ds)" "e ∈ set (ce_args ca)"
      "ob ∈ set (arith_obligations e)"
    using prog_obligation_site_call[OF that] safe by blast
  show "eval e s ∈ ik_range (kenv_of ds x)"
    if "(u, ca, ce, k) ∈ calls (elab_cfg p ds)" "ce_dst ca = Some x"
      "(v, EA_Ret (Some e) (callee_of_entry ce), w) ∈ intra (elab_cfg p ds)"
      "¬ ik_widens (exp_kind e) (kenv_of ds x)"
    using safe[OF prog_obligation_site_return[OF that]] by simp
  show "min (eval e1 s) (eval e2 s) ∈ ik_range (kenv_of ds x)"
    if edge: "(v, EA_Special (Min e1 e2) x, w) ∈ intra (elab_cfg p ds)"
      and narrows:
        "¬ (ik_widens (exp_kind e1) (kenv_of ds x) ∧ ik_widens (exp_kind e2) (kenv_of ds x))"
  proof -
    have ob: "Arithmetic_Obligation Min_Conversion_Overflow (TCast (kenv_of ds x) e1) e2
        ∈ set (special_obligations (kenv_of ds x) (Min e1 e2))"
```

```
      using narrows by simp
    show ?thesis using safe[OF prog_obligation_site_special[OF edge ob] ob] by simp
  qed
  show "max (eval e1 s) (eval e2 s) ∈ ik_range (kenv_of ds x)"
    if edge: "(v, EA_Special (Max e1 e2) x, w) ∈ intra (elab_cfg p ds)"
      and narrows:
        "¬ (ik_widens (exp_kind e1) (kenv_of ds x) ∧ ik_widens (exp_kind e2) (kenv_of ds x))"
  proof -
    have ob: "Arithmetic_Obligation Max_Conversion_Overflow (TCast (kenv_of ds x) e1) e2
        ∈ set (special_obligations (kenv_of ds x) (Max e1 e2))"
      using narrows by simp
    show ?thesis using safe[OF prog_obligation_site_special[OF edge ob] ob] by simp
  qed
qed

theorem run_voblint_arithmetic_refuted:
  assumes run: "run_voblint conf ds p = Analysed res"
    and d: "d ∈ set (report_diagnostics res)" and r: "diagnostic_verdict d = Check_Refuted"
    and s: "s ∈ reaching ds p (diagnostic_point d)"
  shows "¬ s ⊨ diagnostic_obligation d"
proof -
  from run have wf: "wf_program_compile_input p"
    and rep: "report_run conf ds p = Some res"
    by (auto elim!: run_voblint_AnalysedE intro: wf_program_compile_input_exec_sound)
  show ?thesis by (rule report_obligation_refuted[OF wf rep s d r refl])
qed
```

For signed overflow in particular: at a point no diagnostic names, every signed addition, subtraction, multiplication and division nested in an expression evaluated there computes, from the operand values of any reaching store, a mathematical result its kind represents. So no execution that reaches the point overflows there, whichever value wraparound would have produced.

```
corollary run_voblint_no_signed_overflow:
  assumes run: "run_voblint conf ds p = Analysed res"
    and absent: "SAFE res v"
    and s: "s ∈ reaching ds p v"
    and evaluated: "((v, a, w) ∈ intra (elab_cfg p ds) ∧ e ∈ set (edge_expressions a))
                  ∨ ((v, ca, ce, k) ∈ calls (elab_cfg p ds) ∧ e ∈ set (ce_args ca))"
    and sub: "TArith ik op x y ∈ subexps e" and signed: "ik_signed ik" and op: "op ≠ AMod"
  shows "aop_eval op (eval x s) (eval y s) ∈ ik_range ik"
  using evaluated arith_obligations_overflow[OF sub signed op]
    run_voblint_arithmetic_safe[OF run absent s]
  by (metis obligation_holds_overflow)
```

For signed remainders: at a point no diagnostic names, every signed remainder nested in an expression evaluated there divides, at any reaching store, with a truncated quotient its kind represents, so none is `INT_MIN % -1`.

```
corollary run_voblint_no_remainder_overflow:
  assumes run: "run_voblint conf ds p = Analysed res"
    and absent: "SAFE res v"
    and s: "s ∈ reaching ds p v"
    and evaluated: "((v, a, w) ∈ intra (elab_cfg p ds) ∧ e ∈ set (edge_expressions a))
                  ∨ ((v, ca, ce, k) ∈ calls (elab_cfg p ds) ∧ e ∈ set (ce_args ca))"
    and sub: "TArith ik AMod x y ∈ subexps e" and signed: "ik_signed ik"
  shows "c_div (eval x s) (eval y s) ∈ ik_range ik"
  using evaluated arith_obligations_remainder[OF sub signed]
    run_voblint_arithmetic_safe[OF run absent s]
  by (metis obligation_holds_remainder)
```

For conversions: at a point no diagnostic names, every conversion nested in an expression evaluated there whose source kind does not widen into its target converts a value the target represents, so no conversion changes the value it converts.

```
corollary run_voblint_no_conversion_overflow:
  assumes run: "run_voblint conf ds p = Analysed res"
    and absent: "SAFE res v"
    and s: "s ∈ reaching ds p v"
    and evaluated: "((v, a, w) ∈ intra (elab_cfg p ds) ∧ e ∈ set (edge_expressions a))
                  ∨ ((v, ca, ce, k) ∈ calls (elab_cfg p ds) ∧ e ∈ set (ce_args ca))"
    and sub: "TCast ik x ∈ subexps e" and narrows: "¬ ik_widens (exp_kind x) ik"
  shows "eval x s ∈ ik_range ik"
  using evaluated arith_obligations_conversion[OF sub narrows]
    run_voblint_arithmetic_safe[OF run absent s]
  by (metis obligation_holds_conversion(1))
```

The report's vocabulary

The guarantees above, read through the predicates every report shares: `PROVED`, `REFUTED`, `DEAD` and `SAFE`, and `s ⊨ ob` for a store meeting an obligation. `reaching ds p v` are the stores that reach `v`. A proved check holds at each of them and a refuted one fails; a dead point has none; at a safe point every obligation the report decides there holds. None of them says that `v` is reached.

`SAFE` is only as strong as the obligations it covers: those of every expression an edge or a call at `v` evaluates and of every value returned there that a caller converts to a narrower kind, and of every library `min` or `max` result written there to a kind that does not hold both arguments' kinds (`run_voblint_obligations_cover`). It says nothing about a hazard Voblint does not check.

```
theorem run_voblint_report_sound:
  assumes run: "run_voblint conf ds p = Analysed res" and s: "s ∈ reaching ds p v"
  shows run_voblint_PROVED: "PROVED res v c ⟹ truthy (eval c s)"
    and run_voblint_REFUTED: "REFUTED res v c ⟹ ¬ truthy (eval c s)"
    and run_voblint_DEAD: "DEAD res v ⟹ False"
    and run_voblint_SAFE: "SAFE res v ⟹ ob ∈ obligations_at ds p v ⟹ s ⊨ ob"
proof -
  show "PROVED res v c ⟹ truthy (eval c s)"
    using run_voblint_proved[OF run] s by (auto simp: HAS_VERDICT_def)
  show "REFUTED res v c ⟹ ¬ truthy (eval c s)"
    using run_voblint_refuted[OF run] s by (auto simp: HAS_VERDICT_def)
  show "DEAD res v ⟹ False"
    using run_voblint_states[OF run s] by (auto simp: DEAD_def)
  show "SAFE res v ⟹ ob ∈ obligations_at ds p v ⟹ s ⊨ ob"
  proof -
    assume safe: "SAFE res v" and ob: "ob ∈ obligations_at ds p v"
    from run have wf: "wf_program_compile_input p"
      and rep: "report_run conf ds p = Some res"
      by (auto elim!: run_voblint_AnalysedE intro: wf_program_compile_input_exec_sound)
    from ob obtain obs where "(v, obs) ∈ set (prog_obligation_sites ds p)" and "ob ∈ set obs"
      by (auto simp: mem_obligations_at)
    from report_obligation_safe[OF wf rep s this] safe show "s ⊨ ob" .
  qed
qed

corollary run_voblint_dead_unreached:
  "run_voblint conf ds p = Analysed res ⟹ DEAD res v ⟹ reaching ds p v = {}"
  using run_voblint_DEAD by blast
```

What an answer guarantees about a run of the program

The same guarantees, read at a run of the typed program itself rather than at a node of its graph. Run the program from a store whose globals are zero at their kinds and stop anywhere: the configuration reached is simulated at a node `v` of the elaborated graph, the store reaches `v`, the report holds a state at `v` describing it, `v` is not dead, every check the report decides at `v` has its verdict at the store, and if `v` is safe every obligation decided there holds of the store.

```
theorem run_voblint_source_sound:
  assumes run: "run_voblint conf ds p = Analysed res"
    and s0: "s0 ∈ init_stores (kenv_of ds)"
    and exec:
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →ₚ* (residual, s, frs)"
  shows "∃v stk. elab_ptable p ds, elab_cfg p ds, prog_main_name
                   ⊢ (residual, s, frs) ≈ (v, s, stk)
           ∧ s ∈ reaching ds p v
           ∧ (∃d. Lifted d ∈ report_states_at res v
                  ∧ s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d)
           ∧ ¬ DEAD res v
           ∧ (∀c. PROVED res v c ⟶ truthy (eval c s))
           ∧ (∀c. REFUTED res v c ⟶ ¬ truthy (eval c s))
           ∧ (SAFE res v ⟶ (∀ob ∈ obligations_at ds p v. s ⊨ ob))"
proof -
  from run have wf: "wf_program_compile_input p"
    by (auto elim!: run_voblint_AnalysedE intro: wf_program_compile_input_exec_sound)
  from source_reaches_node_collect[OF wf s0 exec] obtain v stk
    where sim: "elab_ptable p ds, elab_cfg p ds, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)"
      and s: "s ∈ reaching ds p v"
    by blast
  show ?thesis
    using sim s run_voblint_states[OF run s] run_voblint_report_sound[OF run s] by blast
qed
```

Read at a check: a run about to execute `Check l e` finds a check labelled `l` for `e` in the report, listed at a node

this very store reaches. Its verdict is not `Dead`, and a decided verdict holds of the store.

```
theorem run_voblint_check_sound:
  assumes run: "run_voblint conf ds p = Analysed res"
    and s0: "s0 ∈ init_stores (kenv_of ds)"
    and exec:
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →p* (residual, s, frs)"
    and chk: "next_check residual = Some (l, e)"
  shows "∃c ∈ set (report_checks res). check_label c = l ∧ check_exp c = e
           ∧ s ∈ reaching ds p (check_point c)
           ∧ check_verdict c ≠ Dead
           ∧ (check_verdict c = Decided Check_Proved ⟶ truthy (eval e s))
           ∧ (check_verdict c = Decided Check_Refuted ⟶ ¬ truthy (eval e s))"
proof -
  from run have wf: "wf_program_compile_input p"
    and rep: "report_run conf ds p = Some res"
    by (auto elim!: run_voblint_AnalysedE intro: wf_program_compile_input_exec_sound)
  from source_reaches_node_collect[OF wf s0 exec] obtain v stk
    where sim: "elab_ptable p ds, elab_cfg p ds, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)"
      and s: "s ∈ reaching ds p v"
    by blast
  from csim_next_check_edge[OF sim chk]
  obtain w where "(v, EA_Check l e, w) ∈ intra (elab_cfg p ds)"
    by blast
  from report_check_row[OF wf rep this] obtain c where c: "c ∈ set (report_checks res)"
    and pt: "check_point c = v" and lbl: "check_label c = l" and ex: "check_exp c = e"
    by blast
  have "check_verdict c ≠ Dead" by (rule report_check_live[OF wf rep s c pt])
  with c pt lbl ex s run_voblint_proved[OF run c] run_voblint_refuted[OF run c] show ?thesis
    by force
qed
```

Read by label, as a consumer finds a source check's row: when the report's labels are distinct, which a consumer can decide on the report, the row labelled `l` is the one for the check about to run, and its verdict holds of the store. That the parser wrote each check's own source position into its label stays outside the theorem.

```
corollary run_voblint_labelled_check_sound:
  assumes run: "run_voblint conf ds p = Analysed res"
    and s0: "s0 ∈ init_stores (kenv_of ds)"
    and exec:
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →p* (residual, s, frs)"
    and chk: "next_check residual = Some (l, e)"
    and distinct: "distinct (map check_label (report_checks res))"
  shows "∃c ∈ set (report_checks res). check_label c = l"
    and "∀c ∈ set (report_checks res). check_label c = l ⟶
           check_exp c = e
           ∧ check_verdict c ≠ Dead
           ∧ (check_verdict c = Decided Check_Proved ⟶ truthy (eval e s))
           ∧ (check_verdict c = Decided Check_Refuted ⟶ ¬ truthy (eval e s))"
proof -
  from run_voblint_check_sound[OF run s0 exec chk]
  obtain c0 where c0: "c0 ∈ set (report_checks res)" "check_label c0 = l" "check_exp c0 = e"
    and v0: "check_verdict c0 ≠ Dead"
      "check_verdict c0 = Decided Check_Proved ⟶ truthy (eval e s)"
      "check_verdict c0 = Decided Check_Refuted ⟶ ¬ truthy (eval e s)"
    by blast
  from c0 show "∃c ∈ set (report_checks res). check_label c = l" by blast
  have inj: "inj_on check_label (set (report_checks res))"
    using distinct by (simp add: distinct_map)
  show "∀c ∈ set (report_checks res). check_label c = l ⟶
           check_exp c = e
           ∧ check_verdict c ≠ Dead
           ∧ (check_verdict c = Decided Check_Proved ⟶ truthy (eval e s))
           ∧ (check_verdict c = Decided Check_Refuted ⟶ ¬ truthy (eval e s))"
  proof (intro ballI impI)
    fix c assume "c ∈ set (report_checks res)" and "check_label c = l"
    with c0 inj have "c = c0" by (auto dest: inj_onD)
    with c0 v0 show "check_exp c = e ∧ check_verdict c ≠ Dead
           ∧ (check_verdict c = Decided Check_Proved ⟶ truthy (eval e s))
           ∧ (check_verdict c = Decided Check_Refuted ⟶ ¬ truthy (eval e s))" by simp
  qed
qed
```

The whole chain for any context policy, given what the policy owes: every valid activation trace of the elaborated graph carries some context, and the stores filed under the policy's contexts at a node are exactly the stores that reach it. A run of the typed program is then simulated at a node, by a valid activation trace with a context,

whose store lies in that context's bucket; the buckets make up the stores reaching the node, the report holds a state there describing the store, and the report's checks hold of it.

```
theorem run_voblint_spine:
  fixes p :: imp_prog and ds :: kind_decls and adm c₀
  defines "g ≡ elab_cfg p ds" and "S ≡ init_stores (kenv_of ds)"
  assumes run: "run_voblint conf ds p = Analysed res"
    and s0: "s0 ∈ S"
    and exec:
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →p* (residual, s, frs)"
    and has_ctx: "⋀t. t ∈ 𝒯kenv_of ds,g,S
                  ⟹ ∃c. activation_context_rel (kenv_of ds) adm c₀ g t c"
    and buckets: "⋀v. (⋃c'. 𝒜kenv_of ds,adm,c₀,g,S v c')
                  = reaching ds p v"
  shows "∃v stk t c. elab_ptable p ds, g, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)
           ∧ activation_trace_repr (kenv_of ds) g S (v, s, stk) t
           ∧ activation_context_rel (kenv_of ds) adm c₀ g t c
           ∧ s ∈ 𝒜kenv_of ds,adm,c₀,g,S v c
           ∧ (⋃c'. 𝒜kenv_of ds,adm,c₀,g,S v c') = reaching ds p v
           ∧ (∃d. Lifted d ∈ report_states_at res v
                  ∧ s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d)
           ∧ (∀c. PROVED res v c ⟶ truthy (eval c s))
           ∧ (∀c. REFUTED res v c ⟶ ¬ truthy (eval c s))"
proof -
  from run have wf: "wf_program_compile_input p"
    by (auto elim!: run_voblint_AnalysedE intro: wf_program_compile_input_exec_sound)
  have cfg: "g = compile_prog (elab_ptable p ds) (prog_procs p)"
    unfolding g_def by (rule elab_cfg_def)
  from source_store_in_activation_collect
       [OF elab_wf_compile_input[OF wf] s0 exec has_ctx[unfolded cfg]]
  obtain v stk t c
    where sim: "elab_ptable p ds, g, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)"
      and rep: "activation_trace_repr (kenv_of ds) g S (v, s, stk) t"
      and rel: "activation_context_rel (kenv_of ds) adm c₀ g t c"
      and mem: "s ∈ 𝒜kenv_of ds,adm,c₀,g,S v c"
    unfolding cfg by blast
  have s: "s ∈ reaching ds p v" using mem buckets[of v] by blast
  show ?thesis
    using sim rep rel mem buckets[of v] run_voblint_states[OF run s]
      run_voblint_report_sound[OF run s]
    by blast
qed
```

Each context policy owes the spine's two facts. A call string is a function of the trace, so every trace carries one and the buckets cover the stores outright. An entry-state context is read off the solve, and the answer was given only because the solve terminated, which is what makes the admitted contexts cover every trace.

```
theorem run_voblint_call_string_chain:
  fixes p :: imp_prog and ds :: kind_decls and k :: nat
  defines "g ≡ elab_cfg p ds" and "S ≡ init_stores (kenv_of ds)"
    and "adm ≡ context_policy_of_fun (cs_context k)"
  assumes run: "run_voblint conf ds p = Analysed res"
    and cx: "conf_context conf = Call_String k"
    and s0: "s0 ∈ S"
    and exec:
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →p* (residual, s, frs)"
  shows "∃v stk t c. elab_ptable p ds, g, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)
           ∧ activation_trace_repr (kenv_of ds) g S (v, s, stk) t
           ∧ activation_context_rel (kenv_of ds) adm [] g t c
           ∧ s ∈ 𝒜kenv_of ds,adm,[],g,S v c
           ∧ (⋃c'. 𝒜kenv_of ds,adm,[],g,S v c') = reaching ds p v
           ∧ (∃d. Lifted d ∈ report_states_at res v
                  ∧ s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d)
           ∧ (∀c. PROVED res v c ⟶ truthy (eval c s))
           ∧ (∀c. REFUTED res v c ⟶ ¬ truthy (eval c s))"
proof -
  have has_ctx: "∃c. activation_context_rel (kenv_of ds) adm [] g t c"
    if "t ∈ 𝒯kenv_of ds,g,S" for t
    unfolding adm_def
    by (rule exI, subst activation_context_rel_of_fun_iff[OF that]) (rule refl)
  have buckets: "(⋃c'. 𝒜kenv_of ds,adm,[],g,S v c') = reaching ds p v"
    for v
    unfolding adm_def g_def S_def
    by (rule mcp_cs.fun_route_node_collect_eq_Union[where ctx_fun = "cs_context k", symmetric])
  show ?thesis
    unfolding g_def S_def
```

```
    by (rule run_voblint_spine[OF run s0[unfolded S_def] exec has_ctx[unfolded g_def S_def]
          buckets[unfolded g_def S_def]])
qed
```

An entry-state run admits the contexts its placement's registration reads off the solve.

```
theorem run_voblint_entry_state_chain:
  fixes p :: imp_prog and ds :: kind_decls and conf :: voblint_conf
  defines "g ≡ elab_cfg p ds" and "S ≡ init_stores (kenv_of ds)"
    and "adm ≡ (case conf_program_globals conf of
                 Program_Globals_Flow_Sensitive ⇒
                   mcp_es.entry_context_rel (conf_sem conf) ds (conf_analyses conf)
                     (conf_globals conf) p
               | Program_Globals_Flow_Insensitive ⇒
                   mcp_split_es.entry_context_rel (conf_sem conf) ds (conf_analyses conf)
                     (conf_globals conf) (declared_globals p) p)"
  assumes run: "run_voblint conf ds p = Analysed res"
    and cx: "conf_context conf = Entry_State"
    and s0: "s0 ∈ S"
    and exec:
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →_p* (residual, s, frs)"
  shows "∃v stk t c. elab_ptable p ds, g, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)
           ∧ activation_trace_repr (kenv_of ds) g S (v, s, stk) t
           ∧ activation_context_rel (kenv_of ds) adm mcp_root_ctx g t c
           ∧ s ∈ 𝒜kenv_of ds,adm,mcp_root_ctx,g,S v c
           ∧ (⋃c'. 𝒜kenv_of ds,adm,mcp_root_ctx,g,S v c')
               = reaching ds p v
           ∧ (∃d. Lifted d ∈ report_states_at res v
                ∧ s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d)
           ∧ (∀c. PROVED res v c ⟶ truthy (eval c s))
           ∧ (∀c. REFUTED res v c ⟶ ¬ truthy (eval c s))"
proof -
  from run have wf: "wf_program_compile_input p"
    and rep: "report_run conf ds p = Some res"
    by (auto elim!: run_voblint_AnalysedE intro: wf_program_compile_input_exec_sound)
  have chain: "(∀t. t ∈ 𝒯kenv_of ds,g,S
                  ⟶ (∃c. activation_context_rel (kenv_of ds) adm mcp_root_ctx g t c))
    ∧ (∀v. (⋃c'. 𝒜kenv_of ds,adm,mcp_root_ctx,g,S v c')
             = reaching ds p v)"
  proof (cases "conf_program_globals conf")
    case Program_Globals_Flow_Sensitive
    from rep cx Program_Globals_Flow_Sensitive obtain sol where
      "TD_side_rule_Interp_solve_c (conf_globals conf)
         (mcp_es.equations (conf_sem conf) ds (conf_analyses conf) is_global p)
         (mcp_es.root_query ds p) = Some sol"
      by (auto simp: report_run_def es_run_def Let_def dg_pipeline.root_query_def)
    note solves = mcp_es.solve_c_run(1)[OF this]
    show ?thesis
      using mcp_es.entry_state_has_context_of_terminates
          [OF elab_cfg_def elab_wf_compile_input[OF wf] solves]
        mcp_es.entry_state_node_collect_eq_Union_of_terminates
          [OF elab_cfg_def elab_wf_compile_input[OF wf] solves]
      unfolding adm_def g_def S_def Program_Globals_Flow_Sensitive by simp
  next
    case Program_Globals_Flow_Insensitive
    from rep cx Program_Globals_Flow_Insensitive obtain sol where
      "TD_side_rule_Interp_solve_c (conf_globals conf)
         (mcp_split_es.equations (conf_sem conf) ds (conf_analyses conf) (declared_globals p)
            is_global p)
         (mcp_split_es.root_query ds p) = Some sol"
      by (auto simp: report_run_def es_run_def Let_def dg_pipeline.root_query_def)
    note solves = mcp_split_es.solve_c_run(1)[OF this]
    show ?thesis
      using mcp_split_es.entry_state_has_context_of_terminates
          [OF elab_cfg_def elab_wf_compile_input[OF wf] solves]
        mcp_split_es.entry_state_node_collect_eq_Union_of_terminates
          [OF elab_cfg_def elab_wf_compile_input[OF wf] solves]
      unfolding adm_def g_def S_def Program_Globals_Flow_Insensitive by simp
  qed
  show ?thesis
    unfolding g_def S_def
    by (rule run_voblint_spine[OF run s0[unfolded S_def] exec
          conjunct1[OF chain, unfolded g_def S_def, rule_format]
          conjunct2[OF chain, unfolded g_def S_def, rule_format]])
qed
```

Every obligation of an expression evaluated at `v`, every narrowing conversion of a value returned at `v`, and every

narrowing conversion of a library result written at `v` is one the report decides there.

```
lemma run_voblint_obligations_cover:
  shows "(v, a, w) ∈ intra (elab_cfg p ds) ⟹ e ∈ set (edge_expressions a)
         ⟹ set (arith_obligations e) ⊆ obligations_at ds p v"
    and "(v, ca, ce, k) ∈ calls (elab_cfg p ds) ⟹ e ∈ set (ce_args ca)
         ⟹ set (arith_obligations e) ⊆ obligations_at ds p v"
    and "(u, ca, ce, k) ∈ calls (elab_cfg p ds) ⟹ ce_dst ca = Some x
         ⟹ (v, EA_Ret (Some e) (callee_of_entry ce), w) ∈ intra (elab_cfg p ds)
         ⟹ ¬ ik_widens (exp_kind e) (kenv_of ds x)
         ⟹ Arithmetic_Obligation Conversion_Overflow (TCast (kenv_of ds x) e) e
              ∈ obligations_at ds p v"
    and "(v, EA_Special sc x, w) ∈ intra (elab_cfg p ds)
         ⟹ set (special_obligations (kenv_of ds x) sc) ⊆ obligations_at ds p v"
     apply (auto simp: mem_obligations_at dest: prog_obligation_site_intra prog_obligation_site_call
       intro: prog_obligation_site_return)[3]
  using prog_obligation_site_special by (fastforce simp: mem_obligations_at)

end
```

17.11 Trace_Run

```
theory Trace_Run
  imports
    Run_Report
    "Voblint_Solver.Solver_Trace"
begin
```

The solver trace of one run

`Voblint_Solver.Solver_Trace` makes the executable solver report its steps. This theory adds the two things only a run knows: which context each call is routed to, and how to read the solver's unknowns and values back as the result reads them. Both are again alternative code equations, proved equal to the equations they replace, and `trace_event` is mapped to the OCaml hook here, where the exported program is assembled.

Routing

A route receives the calling unknown's node and context and the entry state, and returns the callee's context. A context policy whose route is a definition reports that triple from its own equation, by wrapping it.

```
datatype ('x, 'l, 'c) route_event = Ev_Route 'x 'l 'c
```

```
lemma trace_route:
  fixes route :: "pp ⇒ 'c ⇒ 'l ⇒ ('v, 'e) gcall_action ⇒ 'c"
    and route_rhs :: "pp ⇒ 'c ⇒ 'l ⇒ ('v, 'e) gcall_action ⇒ 'c"
  assumes "⋀u ctx d ca. route u ctx d ca = route_rhs u ctx d ca"
  shows "route u ctx d ca =
    (let c = route_rhs u ctx d ca; _ = trace_event STR ''route'' (λ_. Ev_Route (u, ctx) d c) in c)"
  using assms by (simp add: trace_event_def Let_def)
```

Reading unknowns and values back

The solver is generic in its unknowns and values. Where a run fixes them, it hands the trace the functions that read them as the result does: a context as the result shows it, a global unknown as the node-owned buffer, a program global, or the seed of a procedure entry in a context, a solver value's local part as a state, a global unknown's value as the globals it describes, and an entry state as a state. With shared program globals a local value holds its globals at bottom by construction, which the emptiness check would read as unreachable; it is shown without that check and without the globals, and a program global's unknown shows that global alone.

```
datatype 'c trace_key = Trace_Buffer | Trace_Global vname | Trace_Seed cfg_node 'c
```

The printers hand the trace each analysis by its name and each value as its rendering, so one trace reader serves every run, whatever the analyses' values.

```
type_synonym trace_view = "(String.literal, String.literal) analysis_view"
```

```
datatype ('c, 'g, 'd, 'l) trace_printers = Trace_Printers
  "'c ⇒ String.literal analysis_context"
  "'g ⇒ 'c trace_key"
  "'d ⇒ trace_view lifted"
  "vname option ⇒ 'd ⇒ trace_view lifted"
  "'l ⇒ trace_view lifted"
```

```
definition trace_view_of :: "('a ⇒ String.literal) ⇒ ('v ⇒ String.literal)
    ⇒ ('a, 'v) analysis_view ⇒ trace_view" where
  "trace_view_of name text = map (λ(a, f). (name a, map_field_state text f))"
```

```
fun analysis_name :: "analysis ⇒ String.literal" where
  "analysis_name Sign = STR ''sign''"
| "analysis_name Interval = STR ''interval''"
| "analysis_name (Ana_Int _) = STR ''int''"
| "analysis_name Parity = STR ''parity''"
| "analysis_name Congruence = STR ''congruence''"
| "analysis_name Order = STR ''order''"
```

A run names a global's unknown by the global's label.

```
fun key_of_global_unknown :: "(var_id, 'c) global_unknown ⇒ 'c trace_key" where
  "key_of_global_unknown Analysis_Buffer = Trace_Buffer"
| "key_of_global_unknown (Analysis_Global x) = Trace_Global (var_label x)"
| "key_of_global_unknown (Activation_Seed n c) = Trace_Seed n c"
```

A run's values are read as its report renders them, each variable by its label. The trace also shows the return slot, which carries a callee's result to its caller. With the declared globals at their own unknowns, a point's value is shown without them and a global's unknown shows that global alone.

```
definition trace_printers ::
    "program_globals ⇒ analysis list ⇒ kind_decls ⇒ imp_prog
       ⇒ ('c ⇒ String.literal analysis_context)
       ⇒ ('c, (var_id, 'c) global_unknown, (mcp_st lifted, mcp_st lifted) dg_state, mcp_st lifted)
           trace_printers" where
  "trace_printers pg as ds p ctx_view =
     (let raw = (λvs d. map_lift (trace_view_of analysis_name string_of_report_value
                                    ∘ render (activation_of as) vs)
                           (map_lift (mcp_rd is_global) d));
          vars = remdups (cfg_vars (elab_cfg p ds) @ [ReturnVar]);
          gs = declared_globals p;
          local = (case pg of
                     Program_Globals_Flow_Sensitive ⇒ raw vars
                   | Program_Globals_Flow_Insensitive ⇒ raw (filter (λx. x ∉ set gs) vars));
          globals = (λn. case n of None ⇒ gs | Some x ⇒ [GlobalId x])
      in Trace_Printers ctx_view key_of_global_unknown (λd. local (dg_local d))
           (λn d. raw (globals n) (dg_global d)) local)"
```

A run announces its printers before the solve, and the solve reports its steps. A call-string context of length zero is shown as no context, as the report shows it.

```
declare cs_run_def [code del] es_run_def [code del]

lemma cs_run_traced [code]:
  "cs_run conf k ds p =
     (let sem = conf_sem conf; as = conf_analyses conf; r = conf_globals conf;
          key = (λctx. Key_List (map Key_Node ctx));
          tag = (if k = 0 then (λ_. Report_Unit) else Report_Call_String);
          root = (cfg_exit (elab_cfg p ds), []);
          _ = trace_event STR ''run''
                (λ_. trace_printers (conf_program_globals conf) as ds p
                       (if k = 0 then (λ_ :: cfg_node list. Context_Unit) else Context_Call_String))
      in case conf_program_globals conf of
           Program_Globals_Flow_Sensitive ⇒
             map_option (λsol. prog_report_of conf ds p [] key tag
                                 (mcp_cs.solved_run_of sem ds as k is_global p sol))
               (solve_c_traced r (mcp_cs.equations sem ds as k is_global p) root)
         | Program_Globals_Flow_Insensitive ⇒
             (let gs = declared_globals p
              in map_option (λsol. prog_report_of conf ds p gs key tag
                                     (mcp_split.solved_run_of sem ds as k gs is_global p sol))
                   (solve_c_traced r (mcp_split.equations sem ds as k gs is_global p) root)))"
  by (simp add: cs_run_def solve_c_traced_eq trace_event_def Let_def
      split: program_globals.split)

lemma es_run_traced [code]:
  "es_run conf ds p =
     (let sem = conf_sem conf; as = conf_analyses conf; r = conf_globals conf;
          root = (cfg_exit (elab_cfg p ds), mcp_root_ctx);
          _ = trace_event STR ''run''
                (λ_. trace_printers (conf_program_globals conf) as ds p
                       (λctx. Context_Entry (map string_of_report_value
                                (context_values (activation_of as) ctx))))
      in case conf_program_globals conf of
           Program_Globals_Flow_Sensitive ⇒
             map_option (λsol. prog_report_of conf ds p [] mcp_ctx_key Report_Entry
                                 (mcp_es.solved_run_of sem ds as is_global p sol))
               (solve_c_traced r (mcp_es.equations sem ds as is_global p) root)
         | Program_Globals_Flow_Insensitive ⇒
             (let gs = declared_globals p
              in map_option (λsol. prog_report_of conf ds p gs mcp_ctx_key Report_Entry
                                     (mcp_split_es.solved_run_of sem ds as gs is_global p sol))
                   (solve_c_traced r (mcp_split_es.equations sem ds as gs is_global p) root)))"
  by (simp add: es_run_def solve_c_traced_eq trace_event_def Let_def
      split: program_globals.split)

declare mcp_route_def [code del]

lemmas mcp_route_traced [code] =
  trace_route[where route = "mcp_route as G", OF mcp_route_def] for as G
```

The hook

`trace_event` is `()` in the logic. The exported OCaml calls `Solver_trace_hook.emit` instead, which records the event only when tracing is on, forcing the suspension then and only then, and returns `()`. This mapping is trusted in the same way as every other target-language mapping: the hook must return, raise nothing and leave the solver's values alone. No mapping exists for the Eval target, so evaluation in proofs runs the equation.

code_printing constant `trace_event` ⇀ `(OCaml) "Solver'_trace'_hook.emit"`

end

17.12 Arithmetic_Diagnostics

```
theory Arithmetic_Diagnostics
  imports "Voblint_Framework.Contextual_Check_Report"
begin
```

Arithmetic obligations

Diagnostics inspect all expression operands, matching VIMP's total, pure expression model. They neither change execution nor assume short-circuit evaluation for logical operators. Lists retain repeated syntactically equal occurrences.

An obligation names an operation, the hazard it may run into and the operand the hazard concerns. A division or remainder must not divide by zero; its operand is the divisor. A signed arithmetic operation must compute a result its kind represents, as C requires (ISO/IEC 9899 6.5p5); its operand is the operation itself. A signed remainder must have a quotient its kind represents, since C leaves the remainder undefined when the quotient is (6.5.5p6); its operand is that quotient. A conversion must convert a value the target kind represents, the categories Goblint separates as arithmetic and cast overflow (`Checks.Category` at `0dc12d355`); its operand is the converted expression. The result of a library `min` or `max` is converted as well, but has no expression of its own: its obligation names the conversion of the first argument and, as operand, the second. Which hazards a language raises, and when one is safe, is the language's; the report treats them alike.

```
datatype arithmetic_hazard = Division_By_Zero | Signed_Overflow | Remainder_Overflow
  | Conversion_Overflow | Min_Conversion_Overflow | Max_Conversion_Overflow

datatype 'e arithmetic_obligation = Arithmetic_Obligation
  (arithmetic_hazard: arithmetic_hazard)
  (arithmetic_operation: 'e)
  (arithmetic_operand: 'e)
```

Sites and their solved classifications

`arithmetic_expression_sites` collects the expressions each CFG edge and call evaluates; `obligation_sites` turns them into the obligations the solved table then classifies, given how an expression of the language yields them.

Only suppress a negative guard edge when the same point has a matching positive guard edge. All other expression-bearing edges remain in the report.

```
definition arithmetic_expression_sites ::
    "('v::linorder, 'e::linorder) gcfg ⇒ (pp × 'e list) list" where
  "arithmetic_expression_sites g =
     (let edges = cfg_intra_list g
      in map (λ(u, a, v). (u, edge_expressions a))
           (filter (λ(u, a, v). case a of EA_AssumeNot e ⇒
             ¬ list_ex (λ(u', a', v'). u = u' ∧ a' = EA_Assume e) edges
             | _ ⇒ True) edges)) @
       map (λ(u, ca, ce, k). (u, ce_args ca)) (cfg_calls_list g)"

definition obligation_sites ::
    "('e ⇒ 'e arithmetic_obligation list) ⇒ ('v::linorder, 'e::linorder) gcfg
       ⇒ (pp × 'e arithmetic_obligation list) list" where
  "obligation_sites obls g =
     map (λ(v, es). (v, concat (map obls es))) (arithmetic_expression_sites g)"
```

A diagnostic names an obligation of a site list, read through a classifier of obligations. The site list and the classifier depend on the expression language and the analysis; the classification does not.

```
datatype 'e arithmetic_diagnostic = Arithmetic_Diagnostic
  (diagnostic_point: pp)
  (diagnostic_occurrence: nat)
  (diagnostic_obligation: "'e arithmetic_obligation")
  (diagnostic_verdict: check_result)

fun arithmetic_diagnostic_of ::
    "pp ⇒ nat ⇒ 'e arithmetic_obligation ⇒ contextual_verdict
       ⇒ 'e arithmetic_diagnostic list" where
  "arithmetic_diagnostic_of v i obligation (Lifted Check_Refuted) =
     [Arithmetic_Diagnostic v i obligation Check_Refuted]"
| "arithmetic_diagnostic_of v i obligation (Lifted Check_Unknown) =
     [Arithmetic_Diagnostic v i obligation Check_Unknown]"
| "arithmetic_diagnostic_of v i obligation _ = []"

definition arithmetic_diagnostics ::
```

```
    "(pp × 'e arithmetic_obligation list) list ⇒ ('ctx, 'a) solved_table
       ⇒ ('e arithmetic_obligation ⇒ 'a ⇒ check_result)
       ⇒ 'e arithmetic_diagnostic list" where
  "arithmetic_diagnostics sites r classify =
     concat (map (λ(v, obligations).
       concat (map (λ(i, obligation).
         arithmetic_diagnostic_of v i obligation (point_verdict r classify v obligation))
         (zip [0..<length obligations] obligations))) sites)"

lemma arithmetic_expression_sites_intra:
  assumes "finite (intra g)" and "(v, a, w) ∈ intra g"
  shows "(v, edge_expressions a) ∈ set (arithmetic_expression_sites g)"
proof -
  let ?edges = "cfg_intra_list g"
  have mem: "(v, a, w) ∈ set ?edges" using assms by simp
  have kept: "⋀b t. (v, b, t) ∈ set ?edges ⟹
      (case b of EA_AssumeNot e ⇒
        ¬ list_ex (λ(u', a', v'). v = u' ∧ a' = EA_Assume e) ?edges
      | _ ⇒ True) ⟹
      (v, edge_expressions b) ∈ set (arithmetic_expression_sites g)"
    unfolding arithmetic_expression_sites_def
    by (force simp: Let_def image_iff)
  show ?thesis
  proof (cases "∃e. a = EA_AssumeNot e")
    case False
    with mem show ?thesis by (intro kept[of a w]) (auto split: gedge_action.splits)
  next
    case True
    then obtain e where a: "a = EA_AssumeNot e" by blast
    show ?thesis
    proof (cases "list_ex (λ(u', a', v'). v = u' ∧ a' = EA_Assume e) ?edges")
      case False
      from kept[OF mem] False a show ?thesis by simp
    next
      case True
      then obtain t where pos: "(v, EA_Assume e, t) ∈ set ?edges"
        by (auto simp: list_ex_iff)
      from kept[OF pos] a show ?thesis by simp
    qed
  qed
qed

lemma arithmetic_expression_sites_call:
  assumes "finite (calls g)" and "(v, ca, ce, k) ∈ calls g"
  shows "(v, ce_args ca) ∈ set (arithmetic_expression_sites g)"
  using assms unfolding arithmetic_expression_sites_def
  by (force simp: image_iff)

lemma arithmetic_diagnostics_absent:
  assumes site: "(v, obligations) ∈ set sites"
    and ob: "obligation ∈ set obligations"
    and absent: "∀d ∈ set (arithmetic_diagnostics sites r classify). diagnostic_point d ≠ v"
  shows "point_verdict r classify v obligation = Bot ∨
    point_verdict r classify v obligation = Lifted Check_Proved"
proof -
  have "obligation ∈ set (map snd (zip [0..<length obligations] obligations))"
    using ob by simp
  then obtain pair where pair: "pair ∈ set (zip [0..<length obligations] obligations)"
    and snd: "snd pair = obligation"
    unfolding set_map by blast
  have indexed: "(fst pair, obligation) ∈ set (zip [0..<length obligations] obligations)"
    using pair snd by (cases pair) auto
  define i where "i = fst pair"
  have indexed: "(i, obligation) ∈ set (zip [0..<length obligations] obligations)"
    using indexed unfolding i_def .
  let ?verdict = "point_verdict r classify v obligation"
  have included: "set (arithmetic_diagnostic_of v i obligation ?verdict)
      ⊆ set (arithmetic_diagnostics sites r classify)"
    using site indexed
    unfolding arithmetic_diagnostics_def
    by (force simp only: set_concat set_map image_iff case_prod_beta)
  have point: "⋀d. d ∈ set (arithmetic_diagnostic_of v i obligation ?verdict) ⟹
     diagnostic_point d = v"
  proof -
    fix d
    assume d: "d ∈ set (arithmetic_diagnostic_of v i obligation ?verdict)"
```

```
    show "diagnostic_point d = v"
    proof (cases ?verdict)
      case Bot
      with d show ?thesis by simp
    next
      case (Lifted verdict)
      with d show ?thesis by (cases verdict) auto
    qed
  qed
  have empty: "arithmetic_diagnostic_of v i obligation ?verdict = []"
    using included point absent
    by (meson empty_subsetI set_empty2 subset_antisym subset_iff)
  show ?thesis
  proof (cases ?verdict)
    case Bot
    then show ?thesis by simp
  next
    case (Lifted verdict)
    with empty show ?thesis by (cases verdict) auto
  qed
qed
```

An error names a point whose obligation the solved table refutes.

```
lemma arithmetic_diagnostic_of_refuted:
  assumes "d ∈ set (arithmetic_diagnostic_of v i obligation verdict)"
    and "diagnostic_verdict d = Check_Refuted"
  shows "verdict = Lifted Check_Refuted ∧ diagnostic_point d = v
    ∧ diagnostic_obligation d = obligation"
  using assms by (cases "(v, i, obligation, verdict)" rule: arithmetic_diagnostic_of.cases) auto

lemma arithmetic_diagnostics_refuted:
  assumes "d ∈ set (arithmetic_diagnostics sites r classify)"
    and "diagnostic_verdict d = Check_Refuted"
  shows "point_verdict r classify (diagnostic_point d) (diagnostic_obligation d)
      = Lifted Check_Refuted"
  using assms unfolding arithmetic_diagnostics_def
  by (auto dest!: arithmetic_diagnostic_of_refuted)

end
```

Chapter 18

Voblint_Codegen

18.1 Voblint_Codegen

theory Voblint_Codegen
  imports
    "Voblint_CLI.Trace_Run"
    "Voblint_CLI.Run_Voblint"
begin

Code export surface

Top-level definitions normally supply their own code equations. Locale definitions need care: an inline `[code]` attribute is applied through interpretation and does not register the generic constant for direct calls with abstract locale parameters. Register those equations after the defining locale's `end`, in the theory that owns them, as the grouped `declare ... [code]` in `Voblint_Result.DG_Analysis` does. Keep this export theory focused on roots.

If a locale theorem still has assumptions, it is not an unconditional code equation. Discharge them through an interpretation or derive a code equation for the concrete caller. A successful attribute declaration alone does not establish executability: check its warnings, export the actual roots, and compile the OCaml.

Use `[code]` to retain a named function. `[code_unfold]` rewrites its uses during preprocessing and can duplicate its body in generated callers; it is not a substitute for a missing code equation.

This session owns executable exports; the examples session proves and demonstrates the exported definitions without materializing generated code.

One export block, deliberately. The external OCaml regression driver under `codegen/regression/ocaml/` needs a handful of CFG-inspection constants the CLI never calls; a second, narrower block would not make any surface narrower, because Isabelle emits the reachable transitive closure of whatever is named. Those constants are named here instead, so there is one generated artifact for one analysis.

This is still not the project's public API boundary, but it is not nothing either. Three things are decided separately here:

- the roots decide what the emitted OCaml *signature* exposes, and how much of it is transparent rather than abstract;
- their transitive closure decides what the *implementation* contains, which no shortening of the root list reduces;
- `module_name` decides the *packaging*.

So trimming a root narrows what a client can name and match on, while leaving the emitted code the same size. A supported external surface — one that could rename a constructor, or hide a representation behind an eliminator — belongs in the handwritten OCaml facade `cli/voblint.ml` rather than in the shape of this list.

The roots below are the *intended callable surface*: what handwritten OCaml under `cli/`, `codegen/regression/ocaml/` and `tests/property/` actually calls. Everything else in the emitted module is serializer-reachable implementation detail, still present and still callable — the signature narrows with the root list, the code does not. So the intent recorded here is not enforced. Handwritten OCaml names the export through the facade `cli/voblint.ml` (module `Voblint`), which re-exports this module's signature unchanged: the root list still decides what the facade can expose, and the facade is the one place a later, narrower surface would be stated.

Analysis entry goes through `run_voblint` alone, which checks the activation list and well-formedness and runs the requested analyses, update rule and context policy.

The last group of roots is there for signature visibility rather than for dispatch. A constant the serializer does not consider public is emitted but left out of the module signature, and a datatype it does not consider public stays abstract, which makes it unmatchable. So anything handwritten OCaml names — even only to take it apart — has to be a root: `Bot`/`Lifted`, which the CLI matches to tell a dead point from a live verdict, and `prog_table`/`prog_main`/`prog_procs`, which the OCaml program printer reads on a round trip.

Nothing calls these on the CLI's analysis path, so a reading of the list as "the callable surface" alone would drop them, and the resulting break shows up not here but in an OCaml consumer, as an unbound value or a match on an abstract type.

`module_name Generated` puts the whole reachable program into one OCaml module rather than one module per contributing Isabelle theory. The alternative — letting the serializer split by theory — does not survive contact with this program: OCaml's single-file output emits modules in dependency order and cannot express a cycle, and the theories here are mutually dependent at code level (the executable state is instantiated at the solver's

own widening/narrowing classes, and the CFG-specific solver instantiation needs `cfg_node` back). Even a split that Isabelle accepts can fail later in `ocamlfind ocamlopt`, on a type-class dictionary field that module-signature inference does not expose across a boundary the unsplit default never had.

So the generated internals are monolithic, and this says so directly instead of arriving there by remapping every contributing theory onto one name by hand. Modularity, if wanted, belongs in the handwritten facade over `Generated`, `cli/voblint.ml`, which today includes it whole.

Two further modules are emitted regardless: `Bit_Shifts` and `Str_Literal` are HOL's own runtime support, injected as literal target code rather than generated from constants here.

The list below is grouped by what each name is *for*, because the groups answer different questions and only one of them is an operation.

- **Run.** `run_voblint`, alone. Everything a caller can ask the analyser to do goes through it.
- **Result.** `analysis_answer`'s cases, which a caller must tell apart, the projection `prog_render_report` that turns an analysed report into displayable data, and the readers of that data: contexts, states, routes, checks, globals and diagnostics. The records reach OCaml abstract, readable only through their selectors, so a field added later cannot break a consumer that matched on field order. A caller maps `prog_render_report` over the answer with `map_analysis_answer`, passing the value printer it wants: the CLI composes `string_of_report_value` with its decoding of ASCII symbol tokens.
- **Ask.** The values naming a request: which domain, which rule merges side-effected globals, which context policy. A caller constructs these, so their constructors are public.
- **Program.** What the frontend builds an `imp_prog` out of: the command and expression constructors, the program builder, and the numeral and character conversions that bridge OCaml's integers to HOL's.
- **Inspect.** Structural readers with no consumer on the CLI's analysis path. They exist for the OCaml program printer and the CFG regressions, and are listed apart so that reading the surface as "what the CLI calls" does not quietly drop them.
- **Trace.** The solver's events and the readers a run hands the tracer (`Voblint_CLI.Trace_Run`). The CLI's trace renderer matches on them, so they must be public.

The CFG enumerations are generic in the variable and expression types and need their orders, which generated code would take as dictionary arguments. The CLI reads graphs of core VIMP only, so it gets the enumerations at that instance.

```
definition cfg_intra_edges ::
    "(var_id, exp) gcfg ⇒ (cfg_node × (var_id, exp) gedge_action × cfg_node) list" where
  "cfg_intra_edges = cfg_intra_list"

definition cfg_call_edges ::
    "(var_id, exp) gcfg ⇒ (cfg_node × (var_id, exp) gcall_action × cfg_node × cfg_node) list" where
  "cfg_call_edges = cfg_calls_list"

definition cfg_points :: "(var_id, exp) gcfg ⇒ cfg_node list" where
  "cfg_points = cfg_node_list"
```

The surface equality constructor keeps the OCaml name the parser and printer use, ahead of the derived comparator's `Eq`, which the code generator otherwise ranks first now that the surface type is `src_exp`.

```
code_identifier
  constant src_exp.Eq ⇀ (OCaml) "Generated.Eq"

export_code

  — Run: the typed analyzer
  run_voblint

  — Result: answers, the report's rendering, contexts, states, routes, checks, globals, diagnostics, the analysis graph
  Invalid_Activation Malformed_Program No_Answer Analysed map_analysis_answer
  prog_render_report string_of_report_value var_label
  TN TVar TArith TCast TCmp TNot TAnd TOr AAdd ASub AMul ADiv AMod CLt CLe CGt CGe CEq CNe
  res_cfg res_contexts res_states res_joined res_routes res_checks res_globals res_diagnostics
  res_graph graph_nodes graph_edges Local_Node Global_Node
  Intra_Dep Enter_Dep Combine_Dep Continue_Dep Global_Read Global_Write
  Context_Unit Context_Entry Context_Call_String
  state_point state_context state_value state_checks state_diagnostics state_steps
```

```
  Field_Store Field_Whole
  route_point route_context route_callee route_targets
  check_point check_label check_exp check_verdict
  global_unknown global_state Global_Named Global_Seed
  diagnostic_point diagnostic_occurrence diagnostic_obligation diagnostic_verdict
  arithmetic_hazard arithmetic_operation arithmetic_operand
  Division_By_Zero Signed_Overflow Remainder_Overflow Conversion_Overflow
  Min_Conversion_Overflow Max_Conversion_Overflow ik_signed
  Check_Proved Check_Refuted Check_Unknown
  Bot Lifted
  cfg_entry elab_cfg cfg_intra_edges cfg_call_edges cfg_points

  — Ask: the request's options, after Goblint's configuration tree
  voblint_conf_ext conf_analyses conf_context conf_globals conf_program_globals conf_sem
  Interval Congruence Ana_Int Order Sign Parity
  Call_String Entry_State
  Globals_Join Globals_Per_Origin Globals_Warrow Globals_Warrow_Per_Origin Globals_Bounded_Narrowing
  Program_Globals_Flow_Sensitive Program_Globals_Flow_Insensitive
  Refine_Fixpoint Refine_Once Refine_Never
  sem_conf_ext sem_int_signed_overflow default_sem_conf Assume_Top Assume_Wraparound

  — Program: what the frontend builds an input out of
  mk_program gproc_decl_ext
  SKIP Assign Seq gcom.If While Return Check gcom.Call
  N V Plus Minus Times Div Mod
  src_exp.Not And Or Less src_exp.Eq Cast

  — Typed programs: kind declarations per identity
  GlobalId ScopedId ReturnId I8 U8 I16 U16 I32 U32 I64 U64
  Statement FunctionEntry FunctionResult
  int_of_integer nat_of_integer integer_of_int integer_of_nat

  — Inspect: for the OCaml program printer and the CFG regressions, not the CLI
  declared_global_vars
  prog_table prog_main prog_procs prog_stmt_post_order
  EA_Nop EA_Assign EA_Special EA_Assume EA_AssumeNot EA_Body EA_Ret EA_Check
  CallEdge Nondet_Int

  — Trace: the events the solver reports and the readers a run hands the tracer
  Ev_Start Ev_Stop Ev_Query Ev_Query_Wpoint Ev_Iterate_From_Query Ev_Add_Infl Ev_Answer
  Ev_Query_Global Ev_Answer_Global Ev_Iterate Ev_Eq Ev_Rhs Ev_Still_Unstable Ev_Widen Ev_Sol
  Ev_Wpoint_Remove Ev_Wpoint_Clear Ev_Update Ev_Iterate_Changed Ev_Side Ev_Update_Global
    Ev_Destabilize
  Ev_Stable_Remove
  Ev_Route Trace_Printers Trace_Buffer Trace_Global Trace_Seed Inl Inr

  in OCaml module_name Generated file_prefix "Voblint_Generated"

end
```

Appendix A

End-to-end certificate

A.1 Example_Non_Vacuity

```
theory Example_Non_Vacuity
  imports "Voblint_CLI.Run_Voblint"
begin
```

The headline theorems are not vacuous

A theorem whose premises no configuration meets is true and says nothing. This theory shows, for the endpoints of `Voblint_CLI.Run_Voblint`, that their premises hold together on one concrete program, and that the conclusion is then informative: the verdict it constrains is `Check_Proved` or `Dead`, not `Check_Unknown`. It then shows five conditions to be load-bearing: the definite verdicts a report states, the transfer obligation of the congruence remainder, the context correlation of RETURN and the TOTAL obligation in `activation_coverage`, and the pairing in `entry_pairs_cover`. Dropping or weakening each admits an answer that misses a concrete execution. Non-vacuity is a property of the premises. It says nothing about whether `pstep` models the intended language.

The two-call program

`bump` is called with `5` and with `4`. Entry-state contexts keep the two activations apart, so both checks are proved; without contexts the entry of `bump` joins `[4, 5]` and neither is.

```
definition nv_prog :: imp_prog where
  "nv_prog = program {
     fun bump(n) { return n + 1; }
     fun main() {
       a = bump(5);
       b = bump(4);
       __voblint_check(a == 6);
       __voblint_check(b == 5);
     }
   }"

definition nv_conf :: voblint_conf where
  "nv_conf = (|conf_analyses = [Interval], conf_context = Entry_State,
     conf_globals = Globals_Warrow, conf_program_globals = Program_Globals_Flow_Sensitive,
     conf_sem = default_sem_conf|)"

lemma nv_report:
  "(case run_voblint nv_conf [] nv_prog of
      Analysed res ⇒
        map (λchk. (check_point chk, check_verdict chk)) (report_checks res)
          = [(Statement 4, Decided Check_Proved), (Statement 5, Decided Check_Proved)]
    | _ ⇒ False)"
  by eval

lemma nv_analysed:
  obtains res where
    "run_voblint nv_conf [] nv_prog = Analysed res"
    "map (λchk. (check_point chk, check_verdict chk)) (report_checks res)
       = [(Statement 4, Decided Check_Proved), (Statement 5, Decided Check_Proved)]"
  using nv_report by (cases "run_voblint nv_conf [] nv_prog") auto
```

The premises, discharged

Each premise of the source-level endpoints, proved for `nv_prog`: the zero store is initial, and the typed run reaches the first check (`nv_to_check`).

```
abbreviation nv_var :: "vname ⇒ var_id" where
  "nv_var x ≡ ScopedId (STR ''main'') x"

lemma nv_init: "(λ_. 0) ∈ init_stores (kenv_of [])"
  by (simp add: init_stores_def all_in_kinds_def kenv_of_def)

lemma nv_bump:
  "elab_ptable nv_prog [] (STR ''bump'')
     = Some (|formals = [ScopedId (STR ''bump'') (STR ''n'')],
              body = Return () (Some (TArith I32 AAdd (TVar I32 (ScopedId (STR ''bump'') (STR ''n'')))
                                     (TN I32 1)))|)"
  by eval

lemma nv_main:
  "main_body (elab_ptable nv_prog [])
     = Seq (Seq (Seq (gcom.Call () (Some (nv_var (STR ''a'')))) (STR ''bump'') [TN I32 5])
```

```
                (gcom.Call () (Some (nv_var (STR ''b''))) (STR ''bump'') [TN I32 4]))
            (Check () (0, 0) (TCmp CEq (TVar I32 (nv_var (STR ''a''))) (TN I32 6))))
        (Check () (0, 0) (TCmp CEq (TVar I32 (nv_var (STR ''b''))) (TN I32 5)))"
  by eval
```

One call of `bump`: enter, return `n + 1`, and resume the caller.

```
lemma nv_call:
  assumes k: "k ∈ ik_range I32" and k1: "k + 1 ∈ ik_range I32"
  shows
    "(kenv_of []),(elab_ptable nv_prog []) ⊢ (gcom.Call () (Some (nv_var x)) (STR ''bump'') [TN I32 k], t, []) →p*
(SKIP (), t(nv_var x := k + 1), [])"
proof -
  let ?κ = "kenv_of []" and ?Π = "elab_ptable nv_prog []"
  let ?n = "ScopedId (STR ''bump'') (STR ''n'')"
  let ?E = "TArith I32 AAdd (TVar I32 ?n) (TN I32 1)"
  let ?c = "bind_formals [?n] (map2 (λy. kind_conv ?κ (Some y)) [?n] [eval (TN I32 k) t])
             (enter_state is_global t)"
  let ?c' = "?c(ReturnVar := eval ?E ?c)"
  have s1:
    "?κ,?Π ⊢ (gcom.Call () (Some (nv_var x)) (STR ''bump'') [TN I32 k], t, []) →p (Seq (Return () (Some ?E)) (Restore
()), ?c, [Frame t (Some (nv_var x))])"
    using pstep_Call[where Π = ?Π and p = "STR ''bump''", OF nv_bump,
        of "[TN I32 k]" ?κ "()" "Some (nv_var x)" t "[]"]
    by simp
  have s2:
    "?κ,?Π ⊢ (Seq (Return () (Some ?E)) (Restore ()), ?c, [Frame t (Some (nv_var x))]) →p (Seq (Unwind ()) (Restore
()), ?c', [Frame t (Some (nv_var x))])"
    by (rule Seq2) (rule ReturnSome)
  have s3:
    "?κ,?Π ⊢ (Seq (Unwind ()) (Restore ()), ?c', [Frame t (Some (nv_var x))]) →p (SKIP (), combine_assign (Some (nv_var
x)) (kind_conv ?κ (Some (nv_var x)) (?c' ReturnVar)) (combine_env is_global t ?c'), [])"
    by (rule UnwindAct)
  have eq: "combine_assign (Some (nv_var x)) (kind_conv ?κ (Some (nv_var x)) (?c' ReturnVar))
             (combine_env is_global t ?c') = t(nv_var x := k + 1)"
    using k k1
    by (auto simp: fun_eq_iff combine_env_def enter_state_def kenv_of_def simp del: ik_range_iff)
  show ?thesis
    using s1 s2 s3 by (simp only: eq[symmetric]) (meson star.step star.refl)
qed

abbreviation nv_final :: store where
  "nv_final ≡ (λ_. 0)(nv_var (STR ''a'') := 6, nv_var (STR ''b'') := 5)"

abbreviation nv_checks :: "(unit, var_id, exp) gcom" where
  "nv_checks ≡ Seq (Check () (0, 0) (TCmp CEq (TVar I32 (nv_var (STR ''a''))) (TN I32 6)))
                 (Check () (0, 0) (TCmp CEq (TVar I32 (nv_var (STR ''b''))) (TN I32 5)))"
```

The typed run, stopped with the first check about to execute.

```
lemma nv_to_check:
  "(kenv_of []),(elab_ptable nv_prog []) ⊢ (main_body (elab_ptable nv_prog []), λ_. 0, []) →p* (nv_checks, nv_final,
[])"
proof -
  let ?κ = "kenv_of []" and ?Π = "elab_ptable nv_prog []"
  let ?C1 = "gcom.Call () (Some (nv_var (STR ''a''))) (STR ''bump'') [TN I32 5]"
  let ?C2 = "gcom.Call () (Some (nv_var (STR ''b''))) (STR ''bump'') [TN I32 4]"
  let ?K1 = "Check () (0, 0) (TCmp CEq (TVar I32 (nv_var (STR ''a''))) (TN I32 6))"
  let ?K2 = "Check () (0, 0) (TCmp CEq (TVar I32 (nv_var (STR ''b''))) (TN I32 5))"
  let ?t1 = "(λ_. 0)(nv_var (STR ''a'') := 6) :: store"
  have r: "(4::int) ∈ ik_range I32" "(5::int) ∈ ik_range I32" "(6::int) ∈ ik_range I32"
    by eval+
  have c1: "?κ,?Π ⊢ (?C1, λ_. 0, []) →p* (SKIP (), ?t1, [])"
    using nv_call[where k = 5 and x = "STR ''a''" and t = "λ_. 0", OF r(2)] r(3) by simp
  have c2: "?κ,?Π ⊢ (?C2, ?t1, []) →p* (SKIP (), nv_final, [])"
    using nv_call[where k = 4 and x = "STR ''b''" and t = ?t1, OF r(1)] r(2) by simp
  have calls: "?κ,?Π ⊢ (Seq ?C1 ?C2, λ_. 0, []) →p* (SKIP (), nv_final, [])"
    using psteps_Seq2[OF c1, of ?C2] c2 by (meson Seq1 star.step star_trans)
  have "?κ,?Π ⊢ (Seq (Seq ?C1 ?C2) ?K1, λ_. 0, []) →p* (?K1, nv_final, [])"
    using psteps_Seq2[OF calls, of ?K1] by (meson Seq1 star.step star.refl star_trans)
  then show ?thesis unfolding nv_main by (rule psteps_Seq2)
qed
```

The endpoints, instantiated

With the premises discharged, `run_voblint_source_sound` and its siblings apply to `nv_prog` and to `nv_dead_prog`, and the verdicts they constrain are `Check_Proved` and `Dead`.

```
theorem nv_source_certified:
  "∃res v stk.
     run_voblint nv_conf [] nv_prog = Analysed res
   ∧ map (λchk. (check_point chk, check_verdict chk)) (report_checks res)
       = [(Statement 4, Decided Check_Proved), (Statement 5, Decided Check_Proved)]
   ∧ elab_ptable nv_prog [], elab_cfg nv_prog [], prog_main_name
       ⊢ (nv_checks, nv_final, []) ≈ (v, nv_final, stk)
   ∧ nv_final ∈ reaching [] nv_prog v
   ∧ (∃d. Lifted d ∈ report_states_at res v
         ∧ nv_final ∈ mcp.act_gamma [] (activation_of (conf_analyses nv_conf)) d)
   ∧ ¬ DEAD res v"
proof -
  obtain res where ans: "run_voblint nv_conf [] nv_prog = Analysed res"
    and checks: "map (λchk. (check_point chk, check_verdict chk)) (report_checks res)
                   = [(Statement 4, Decided Check_Proved), (Statement 5, Decided Check_Proved)]"
    by (rule nv_analysed)
  from run_voblint_source_sound [OF ans nv_init nv_to_check]
  obtain v stk where
    sim: "elab_ptable nv_prog [], elab_cfg nv_prog [], prog_main_name
            ⊢ (nv_checks, nv_final, []) ≈ (v, nv_final, stk)"
    and reach: "nv_final ∈ reaching [] nv_prog v"
    and st: "∃d. Lifted d ∈ report_states_at res v
               ∧ nv_final ∈ mcp.act_gamma [] (activation_of (conf_analyses nv_conf)) d"
    and live: "¬ DEAD res v"
    by blast
  show ?thesis
    by (rule exI[of _ res], rule exI[of _ v], rule exI[of _ stk])
       (simp only: ans checks sim reach st live simp_thms)
qed
```

Read at the check: the store the run holds there reaches a listed check for `a == 6`, that check says `Check_Proved`, and the condition holds.

```
theorem nv_check_proved_sound:
  "∃res. run_voblint nv_conf [] nv_prog = Analysed res
     ∧ (∃chk ∈ set (report_checks res).
          check_exp chk = TCmp CEq (TVar I32 (nv_var (STR ''a''))) (TN I32 6)
          ∧ nv_final ∈ reaching [] nv_prog (check_point chk)
          ∧ check_verdict chk = Decided Check_Proved
          ∧ truthy (eval (TCmp CEq (TVar I32 (nv_var (STR ''a''))) (TN I32 6)) nv_final))"
proof -
  obtain res where ans: "run_voblint nv_conf [] nv_prog = Analysed res"
    and checks: "map (λchk. (check_point chk, check_verdict chk)) (report_checks res)
                   = [(Statement 4, Decided Check_Proved), (Statement 5, Decided Check_Proved)]"
    by (rule nv_analysed)
  have nc:
    "next_check nv_checks = Some ((0, 0), TCmp CEq (TVar I32 (nv_var (STR ''a''))) (TN I32 6))"
    by simp
  from run_voblint_check_sound [OF ans nv_init nv_to_check nc]
  obtain chk where listed: "chk ∈ set (report_checks res)"
    and re: "check_exp chk = TCmp CEq (TVar I32 (nv_var (STR ''a''))) (TN I32 6)"
    and mem: "nv_final ∈ reaching [] nv_prog (check_point chk)"
    and pr: "check_verdict chk = Decided Check_Proved
               ⟶ truthy (eval (TCmp CEq (TVar I32 (nv_var (STR ''a''))) (TN I32 6)) nv_final)"
    by blast
  have "(check_point chk, check_verdict chk)
          ∈ set [(Statement 4, Decided Check_Proved), (Statement 5, Decided Check_Proved)]"
    unfolding checks [symmetric] using listed by auto
  then have "check_verdict chk = Decided Check_Proved" by auto
  with ans listed re mem pr show ?thesis by blast
qed

definition nv_dead_prog :: imp_prog where
  "nv_dead_prog = program {
     fun main() {
       x = 1;
       __voblint_check(x == 0);
       if (x < 0) { __voblint_check(x == 5); } else { x = 2; }
     }
   }"

definition nv_dead_conf :: voblint_conf where
  "nv_dead_conf = ⦇conf_analyses = [Interval], conf_context = Call_String 0,
     conf_globals = Globals_Join, conf_program_globals = Program_Globals_Flow_Sensitive,
     conf_sem = default_sem_conf⦈"

lemma nv_dead_report:
```

```
  "(case run_voblint nv_dead_conf [] nv_dead_prog of
      Analysed res ⇒
        map (λchk. (check_point chk, check_verdict chk)) (report_checks res)
          = [(Statement 1, Decided Check_Refuted), (Statement 3, Dead)]
    | _ ⇒ False)"
  by eval

lemma nv_dead_analysed:
  obtains res where
    "run_voblint nv_dead_conf [] nv_dead_prog = Analysed res"
    "map (λchk. (check_point chk, check_verdict chk)) (report_checks res)
       = [(Statement 1, Decided Check_Refuted), (Statement 3, Dead)]"
  using nv_dead_report by (cases "run_voblint nv_dead_conf [] nv_dead_prog") auto

theorem nv_dead_unreached: "reaching [] nv_dead_prog (Statement 3) = {}"
proof -
  obtain res where ans: "run_voblint nv_dead_conf [] nv_dead_prog = Analysed res"
    and checks: "map (λchk. (check_point chk, check_verdict chk)) (report_checks res)
                   = [(Statement 1, Decided Check_Refuted), (Statement 3, Dead)]"
    by (rule nv_dead_analysed)
  obtain chk where mem: "chk ∈ set (report_checks res)" and pt: "check_point chk = Statement 3"
    and dead: "check_verdict chk = Dead"
    using checks by (cases "report_checks res" rule: remdups_adj.cases) auto
  from ans have wf: "wf_program_compile_input nv_dead_prog"
    and rep: "report_run nv_dead_conf [] nv_dead_prog = Some res"
    by (auto elim!: run_voblint_AnalysedE intro: wf_program_compile_input_exec_sound)
  show ?thesis
    using report_check_live[OF wf rep _ mem pt] dead by blast
qed
```

Load-bearing conditions

Each subsection below drops or weakens one condition and exhibits an answer that meets everything else yet misses a concrete execution.

Soundness alone is cheap

The guarantees constrain only definite verdicts, so a report that answers `Check_Unknown` at every check meets them at every store. The endpoint is informative because the verdicts `run_voblint` actually returns above are `Check_Proved` and `Dead`.

```
definition answer_all :: "contextual_verdict ⇒ report ⇒ report" where
  "answer_all vd res =
     res(|report_checks := map (λchk. chk(|check_verdict := vd|)) (report_checks res)|)"

lemma unknown_everywhere_sound:
  "∀chk ∈ set (report_checks (answer_all (Decided Check_Unknown) res)).
     (check_verdict chk = Decided Check_Proved ⟶ P chk)
     ∧ (check_verdict chk = Decided Check_Refuted ⟶ Q chk)"
  by (auto simp: answer_all_def)
```

An answer of `Check_Proved` everywhere is not sound: the store after `x = 1` reaches the check `x == 0`, which it falsifies.

```
lemma nv_dead_reaches_first_check:
  "(λ_. 0)(nv_var (STR ''x'') := 1) ∈ reaching [] nv_dead_prog (Statement 1)"
proof -
  have intra: "(FunctionEntry (STR ''main''), EA_Body (STR ''main''), Statement 0)
                 ∈ intra (elab_cfg nv_dead_prog [])"
    "(Statement 0, EA_Assign (nv_var (STR ''x'')) (TN I32 1), Statement 1)
                 ∈ intra (elab_cfg nv_dead_prog [])"
    by eval+
  have entry: "cfg_entry (elab_cfg nv_dead_prog []) = FunctionEntry (STR ''main'')"
    by eval
  have e0: "(λ_. 0) ∈ reaching [] nv_dead_prog (FunctionEntry (STR ''main''))"
    using node_collect_init [OF nv_init, of "kenv_of []" "elab_cfg nv_dead_prog []"]
    by (simp add: entry)
  have s0: "(λ_. 0) ∈ reaching [] nv_dead_prog (Statement 0)"
    by (rule node_collect_intra_step [OF e0 intra(1)]) simp
  show ?thesis
    by (rule node_collect_intra_step [OF s0 intra(2)]) (simp add: kenv_of_def)
qed

theorem proved_everywhere_unsound:
  assumes "run_voblint nv_dead_conf [] nv_dead_prog = Analysed res"
```

```
  shows "∃chk ∈ set (report_checks (answer_all (Decided Check_Proved) res)).
           check_verdict chk = Decided Check_Proved
           ∧ (λ_. 0)(nv_var (STR ''x'') := 1) ∈ reaching [] nv_dead_prog (check_point chk)
           ∧ ¬ truthy (eval (check_exp chk) ((λ_. 0)(nv_var (STR ''x'') := 1)))"
proof -
  have exps: "map (λchk. (check_point chk, check_exp chk)) (report_checks res)
      = [(Statement 1, TCmp CEq (TVar I32 (nv_var (STR ''x''))) (TN I32 0)),
         (Statement 3, TCmp CEq (TVar I32 (nv_var (STR ''x''))) (TN I32 5))]"
  proof -
    have "(case run_voblint nv_dead_conf [] nv_dead_prog of
             Analysed res ⇒ map (λchk. (check_point chk, check_exp chk)) (report_checks res)
                = [(Statement 1, TCmp CEq (TVar I32 (nv_var (STR ''x''))) (TN I32 0)),
                   (Statement 3, TCmp CEq (TVar I32 (nv_var (STR ''x''))) (TN I32 5))]
           | _ ⇒ False)"
      by eval
    then show ?thesis using assms by simp
  qed
  then obtain chk where mem: "chk ∈ set (report_checks res)" and pt: "check_point chk = Statement 1"
      and ex: "check_exp chk = TCmp CEq (TVar I32 (nv_var (STR ''x''))) (TN I32 0)"
    by (cases "report_checks res" rule: remdups_adj.cases) auto
  have mem': "chk(|check_verdict := Decided Check_Proved|)
                ∈ set (report_checks (answer_all (Decided Check_Proved) res))"
    using mem by (simp add: answer_all_def)
  show ?thesis
    using nv_dead_reaches_first_check pt ex
    by (intro bexI[OF _ mem']) simp
qed
```

The pre-fix Goblint congruence remainder

Before Goblint pull request 1161, the remainder of the odd class `1 + 2ℤ` by the constant `2` was the constant `1`. Any abstract remainder that answers so violates the obligation `congruence_mod_sound` discharges for the shipped one: `-5` is odd, and its truncating remainder by `2` is `-1`.

```
theorem prefix_congruence_mod_unsound:
  assumes const: "f (mk_congruence 1 2) (mk_congruence 2 0) = mk_congruence 1 0"
  shows "¬ (∀a b i j. i ∈ gamma_congruence a ⟶ j ∈ gamma_congruence b
                    ⟶ c_mod i j ∈ gamma_congruence (f a b))"
proof
  assume sound: "∀a b i j. i ∈ gamma_congruence a ⟶ j ∈ gamma_congruence b
                    ⟶ c_mod i j ∈ gamma_congruence (f a b)"
  have "c_mod (-5) 2 ∈ gamma_congruence (mk_congruence 1 0)"
    using sound [rule_format, of "-5" "mk_congruence 1 2" 2 "mk_congruence 2 0"]
    by (simp add: const gamma_congruence_def)
  then show False by (simp add: gamma_congruence_def c_mod_def c_div_def)
qed
```

The paired-coverage, RETURN and TOTAL counterexamples

Three weakenings of `activation_coverage` fail: RETURN read at the caller's context, TOTAL dropped, and entry coverage without pairing. They are shown on core VIMP, the language the analyzer runs. `ret_prog` calls `f` with `1` and then `5`; every variable has the default kind `I32`.

```
definition ret_prog :: imp_prog where
  "ret_prog = program {
     fun f(n) { return n; }
     fun main() { a = f(1); b = f(5); }
   }"

abbreviation ret_n :: var_id where
  "ret_n ≡ ScopedId (STR ''f'') (STR ''n'')"

abbreviation ret_κ :: kenv where
  "ret_κ ≡ kenv_of []"

abbreviation ret_g :: "(var_id, exp) gcfg" where
  "ret_g ≡ elab_cfg ret_prog []"

lemma ret_norm [simp]: "ik_norm I32 1 = 1" "ik_norm I32 5 = 5" "(1::int) ∈ ik_range I32"
  by eval+

lemma ret_intra:
  "intra ret_g =
     {(FunctionEntry (STR ''f''), EA_Body (STR ''f''), Statement 0),
      (Statement 0, EA_Ret (Some (TVar I32 ret_n)) (STR ''f''), FunctionResult (STR ''f'')),
```

```
    (Statement 1, EA_Ret None (STR ''f''), FunctionResult (STR ''f'')),
    (FunctionEntry (STR ''main''), EA_Body (STR ''main''), Statement 2),
    (Statement 4, EA_Ret None (STR ''main''), FunctionResult (STR ''main''))}"
  by eval

lemma ret_calls:
  "calls ret_g =
     {(Statement 2, CallEdge (Some (nv_var (STR ''a''))) [ret_n] [TN I32 1],
       FunctionEntry (STR ''f''), Statement 3),
      (Statement 3, CallEdge (Some (nv_var (STR ''b''))) [ret_n] [TN I32 5],
       FunctionEntry (STR ''f''), Statement 4)}"
  by eval

lemma ret_entry: "cfg_entry ret_g = FunctionEntry (STR ''main'')"
  by eval

lemma ret_enter:
  "call_enter ret_κ (CallEdge dst [ret_n] [e]) s
     = (enter_state is_global s)(ret_n := ik_norm I32 (eval e s))"
  by (simp add: call_enter_CallEdge kenv_of_def)

lemma ret_enter_zero [simp]: "enter_state is_global (λ_. 0) = (λ_. 0)"
  by (simp add: enter_state_def enter_frame_def fun_eq_iff)

lemma ret_combine:
  assumes "⋀v. is_global v ⟹ t v = 0"
  shows
    "combine_collect κ (Some x) (λ_. 0) t = (λ_. 0)(x := kind_conv κ (Some x) (t ReturnVar))"
  using assms by (auto simp: combine_collect_def fun_eq_iff)
```

The policy of the illustration: the callee context is the entry value of `n`.

```
definition ret_R :: "int call_context_policy" where
  "ret_R = context_policy_of_fun (λ_ _ es. es ret_n)"
```

The claim: `main` is covered in the initial context `0` up to its first call site and nowhere after it; `f` is covered everywhere, but only in the contexts its calls create.

```
definition ret_cover :: "cfg_node ⇒ int ⇒ store set" where
  "ret_cover v ctx =
     (if v ∈ {FunctionEntry (STR ''f''), Statement 0, FunctionResult (STR ''f'')}
      then (if ctx = 0 then {} else UNIV)
      else if ctx = 0 ∧ v ∈ {FunctionEntry (STR ''main''), Statement 2} then UNIV else {})"

lemma ret_weak_obligations:
  shows "∀s. s ∈ init_stores ret_κ ⟶ s ∈ ret_cover (cfg_entry ret_g) 0"
    and "∀u a v c s s'. (u, a, v) ∈ intra ret_g ⟶ s ∈ ret_cover u c
              ⟶ s' ∈ edge_step ret_κ a s ⟶ s' ∈ ret_cover v c"
    and "∀u dst pars args p cont c c' s.
              (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls ret_g
              ⟶ s ∈ ret_cover u c
              ⟶ c' ∈ ret_R u c (call_info_of (CallEdge dst pars args) p) s
                    (call_enter ret_κ (CallEdge dst pars args) s)
              ⟶ call_enter ret_κ (CallEdge dst pars args) s
                    ∈ ret_cover (FunctionEntry p) c'"
    and "∀cl dst pars args p cont c1 s t.
              (cl, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls ret_g
              ⟶ s ∈ ret_cover cl c1 ⟶ t ∈ ret_cover (FunctionResult p) c1
              ⟶ combine_collect ret_κ dst s t ∈ ret_cover cont c1"
    and "call_context_total_on ret_κ ret_cover ret_R ret_g"
  by (auto simp: ret_entry ret_intra ret_calls ret_cover_def ret_R_def ret_enter
                 call_context_total_on_def)

lemma ret_not_coverage:
  "¬ activation_coverage ret_κ ret_g
       (init_stores ret_κ) ret_cover ret_R 0"
proof
  assume "activation_coverage ret_κ ret_g
            (init_stores ret_κ) ret_cover ret_R 0"
  then interpret C: activation_coverage ret_κ
    ret_g "init_stores ret_κ" ret_cover ret_R 0 .
  have e: "(Statement 2, CallEdge (Some (nv_var (STR ''a''))) [ret_n] [TN I32 1],
             FunctionEntry (STR ''f''), Statement 3) ∈ calls ret_g"
    by (simp add: ret_calls)
  have adm: "admits_call_context ret_κ ret_g ret_R
               (Statement 2) 0 (STR ''f'') (λ_. 0)
               (call_enter ret_κ
```

```
                (CallEdge (Some (nv_var (STR ''a''))) [ret_n] [TN I32 1]) (λ_. 0)) 1"
    unfolding admits_call_context_def
    by (rule exI [of _ "Some (nv_var (STR ''a''))"], rule exI [of _ "[ret_n]"],
        rule exI [of _ "[TN I32 1]"], rule exI [of _ "Statement 3"])
       (simp add: e ret_R_def ret_enter)
  have "combine_collect ret_κ (Some (nv_var (STR ''a''))) (λ_. 0) (λ_. 0)
          ∈ ret_cover (Statement 3) 0"
    by (rule C.RETURN [OF e _ adm]) (simp_all add: ret_cover_def)
  then show False by (simp add: ret_cover_def)
qed

lemma ret_f_body:
  "star (cfg_intra_step ret_κ ret_g)
     (FunctionEntry (STR ''f''), (λ_. 0)(ret_n := k))
     (FunctionResult (STR ''f''), ((λ_. 0)(ret_n := k))(ReturnVar := k))"
proof (rule star_trans)
  show "star (cfg_intra_step ret_κ ret_g)
          (FunctionEntry (STR ''f''), (λ_. 0)(ret_n := k))
          (Statement 0, (λ_. 0)(ret_n := k))"
    by (rule intra_path_single [where a = "EA_Body (STR ''f'')"]) (auto simp: ret_intra)
next
  show "star (cfg_intra_step ret_κ ret_g)
          (Statement 0, (λ_. 0)(ret_n := k))
          (FunctionResult (STR ''f''), ((λ_. 0)(ret_n := k))(ReturnVar := k))"
    by (rule intra_path_single [where a = "EA_Ret (Some (TVar I32 ret_n)) (STR ''f'')"])
       (auto simp: ret_intra)
qed

lemma ret_globals_zero: "is_global v ⟹ (((λ_. 0)(ret_n := k))(ReturnVar := k)) v = 0"
  by (cases v) simp_all

lemma ret_reaches_second_call:
  "(λ_. 0)(nv_var (STR ''a'') := 1) ∈ C_ret_κ,ret_g,init_stores ret_κ (Statement 3)"
proof -
  have e0: "(λ_. 0) ∈ C_ret_κ,ret_g,init_stores ret_κ (FunctionEntry (STR ''main''))"
    using node_collect_init [OF nv_init, of ret_κ ret_g] by (simp add: ret_entry)
  have s2: "(λ_. 0) ∈ C_ret_κ,ret_g,init_stores ret_κ (Statement 2)"
    by (rule node_collect_intra_step [OF e0, where a = "EA_Body (STR ''main'')"])
       (auto simp: ret_intra)
  have ce: "(Statement 2, CallEdge (Some (nv_var (STR ''a''))) [ret_n] [TN I32 1],
             FunctionEntry (STR ''f''), Statement 3) ∈ calls ret_g"
    by (simp add: ret_calls)
  have bd: "star (cfg_intra_step ret_κ ret_g)
              (FunctionEntry (STR ''f''),
               call_enter ret_κ (CallEdge (Some (nv_var (STR ''a''))) [ret_n] [TN I32 1]) (λ_. 0))
              (FunctionResult (STR ''f''), ((λ_. 0)(ret_n := 1))(ReturnVar := 1))"
    using ret_f_body [where k = 1] by (simp add: ret_enter)
  show ?thesis
    using node_collect_call_return_step [OF s2 ce bd]
    by (simp add: ret_combine [OF ret_globals_zero] kenv_of_def)
qed
```

Reading the callee at the caller's own context: the claim meets INIT, INTRA, CALL, the weakened RETURN and TOTAL, yet it covers no store at the second call site in any context, although a concrete trace reaches it. So `b = f(5)` is declared unreachable.

```
theorem return_at_caller_context_unsound:
  "(λ_. 0)(nv_var (STR ''a'') := 1) ∈ C_ret_κ,ret_g,init_stores ret_κ (Statement 3)
   ∧ (⋃c. ret_cover (Statement 3) c) = {}"
  using ret_reaches_second_call by (simp add: ret_cover_def)
```

TOTAL is load-bearing

A resumed caller carries its own context whatever its callee carries, so the continuation of a call is collected in the caller's bucket even when the relation admits no callee context. Such a relation makes CALL and RETURN vacuous.

```
lemma resume_keeps_context:
  assumes s: "s ∈ A_κ,adm,c0,g,S u ctx"
    and ce: "(u, CallEdge dst pars args, FunctionEntry q, cont) ∈ calls g"
    and body: "star (cfg_intra_step κ g)
                 (FunctionEntry q, call_enter κ (CallEdge dst pars args) s)
                 (FunctionResult q, t)"
  shows "combine_collect κ dst s t ∈ A_κ,adm,c0,g,S cont ctx"
proof -
  from s obtain caller where cv: "caller ∈ T_κ,g,S"
```

```
    and cn: "sink_node caller = u" and ck: "activation_context_rel κ adm c0 g caller ctx"
    and cs: "sink_store caller = s"
    by (rule activation_collect_E)
  let ?entered = "Call caller [(FunctionEntry q, call_enter κ (CallEdge dst pars args) s)]"
  have ev: "?entered ∈ 𝒯κ,g,S"
    using valid_activation_trace.call [OF cv, where dst = dst and pars = pars and args = args
                          and p = q and cont = cont]
      ce cn cs by simp
  obtain callee where dv: "callee ∈ 𝒯κ,g,S"
    and dn: "sink_node callee = FunctionResult q" and ds: "sink_store callee = t"
    and dc: "caller_of callee = caller_of ?entered"
    by (rule valid_activation_trace_intra_path_extend [OF _ ev]) (use body in simp_all)
  let ?r = "Resume caller callee
             (path_of caller @ [(cont, combine_collect κ dst
                 (sink_store caller) (sink_store callee))])"
  have rv: "?r ∈ 𝒯κ,g,S"
    by (rule valid_activation_trace.ret [OF dv _ dn]) (use dc ce cn in simp_all)
  have "sink_store ?r ∈ 𝒜κ,adm,c0,g,S (sink_node ?r) ctx"
    by (rule activation_collect_I [OF rv refl]) (use ck in simp)
  then show ?thesis using cs ds by (simp add: sink_node_def sink_store_def)
qed
```

The relation that admits no context, and the claim that covers `main` up to its first call site in the initial context and nothing else.

```
definition tot_R :: "int call_context_policy" where
  "tot_R u ctx ci s es = {}"

definition tot_cover :: "cfg_node ⇒ int ⇒ store set" where
  "tot_cover v ctx =
     (if ctx = 0 ∧ v ∈ {FunctionEntry (STR ''main''), Statement 2} then UNIV else {})"

lemma tot_weak_obligations:
  shows "∀s. s ∈ init_stores ret_κ ⟶ s ∈ tot_cover (cfg_entry ret_g) 0"
    and "∀u a v c s s'. (u, a, v) ∈ intra ret_g ⟶ s ∈ tot_cover u c
             ⟶ s' ∈ edge_step ret_κ a s ⟶ s' ∈ tot_cover v c"
    and "∀u dst pars args p cont c c' s.
             (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls ret_g
             ⟶ s ∈ tot_cover u c
             ⟶ c' ∈ tot_R u c (call_info_of (CallEdge dst pars args) p) s
                   (call_enter ret_κ (CallEdge dst pars args) s)
             ⟶ call_enter ret_κ (CallEdge dst pars args) s
                   ∈ tot_cover (FunctionEntry p) c'"
    and "∀cl dst pars args p cont c1 c' p' s t es.
             (cl, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls ret_g
             ⟶ s ∈ tot_cover cl c1
             ⟶ admits_call_context ret_κ ret_g tot_R cl c1 p' s es c'
             ⟶ t ∈ tot_cover (FunctionResult p) c'
             ⟶ combine_collect ret_κ dst s t ∈ tot_cover cont c1"
    and "¬ call_context_total_on ret_κ tot_cover tot_R ret_g"
  by (auto simp: ret_entry ret_intra ret_calls tot_cover_def tot_R_def admits_call_context_def
                 call_context_total_on_def)
```

The claim meets INIT, INTRA, CALL and RETURN and fails only TOTAL. The run resumes at the second call site in the initial context, where the claim is empty, so the per-context conclusion of `activation_collect_sound` fails.

```
theorem total_dropped_unsound:
  "(λ_. 0)(nv_var (STR ''a'') := 1)
     ∈ 𝒜ret_κ,tot_R,0,ret_g,init_stores ret_κ (Statement 3) 0
   ∧ (λ_. 0)(nv_var (STR ''a'') := 1) ∉ tot_cover (Statement 3) 0"
proof
  let ?S = "init_stores ret_κ"
  let ?t0 = "Root [(FunctionEntry (STR ''main''), λ_. 0)]"
  have v0: "?t0 ∈ 𝒯ret_κ,ret_g,?S"
    using valid_activation_trace.root [where s = "λ_. 0" and S = ?S and g = ret_g]
      nv_init by (simp add: ret_entry)
  have v1: "Root [(FunctionEntry (STR ''main''), λ_. 0), (Statement 2, λ_. 0)]
             ∈ 𝒯ret_κ,ret_g,?S"
    using valid_activation_trace.intra
      [OF v0, where a = "EA_Body (STR ''main'')" and v = "Statement 2" and s' = "λ_. 0"]
    by (simp add: ret_intra sink_node_def sink_store_def)
  have s2: "(λ_. 0) ∈ 𝒜ret_κ,tot_R,0,ret_g,?S (Statement 2) 0"
  proof -
    have "sink_store (Root [(FunctionEntry (STR ''main''), λ_. 0), (Statement 2, λ_. 0)])
           ∈ 𝒜ret_κ,tot_R,0,ret_g,?S
```

```
              (sink_node (Root [(FunctionEntry (STR ''main''), λ_. 0), (Statement 2, λ_. 0)])) 0"
      by (rule activation_collect_I [OF v1]) (simp_all add: sink_node_def)
    then show ?thesis by (simp add: sink_node_def sink_store_def)
  qed
  have ce: "(Statement 2, CallEdge (Some (nv_var (STR ''a''))) [ret_n] [TN I32 1],
            FunctionEntry (STR ''f''), Statement 3) ∈ calls ret_g"
    by (simp add: ret_calls)
  have bd: "star (cfg_intra_step ret_κ ret_g)
             (FunctionEntry (STR ''f''),
              call_enter ret_κ (CallEdge (Some (nv_var (STR ''a''))) [ret_n] [TN I32 1]) (λ_. 0))
             (FunctionResult (STR ''f''), ((λ_. 0)(ret_n := 1))(ReturnVar := 1))"
    using ret_f_body [where k = 1] by (simp add: ret_enter)
  show "(λ_. 0)(nv_var (STR ''a'') := 1)
        ∈ A_{ret_κ,tot_R,0,ret_g,?S} (Statement 3) 0"
    using resume_keeps_context [OF s2 ce bd]
    by (simp add: ret_combine [OF ret_globals_zero] kenv_of_def)
  show "(λ_. 0)(nv_var (STR ''a'') := 1) ∉ tot_cover (Statement 3) 0"
    by (simp add: tot_cover_def)
qed
```

Entry coverage must be paired

`y = p(x)` with `fun p(a) { return a; }` and `x = 1`, all of kind `I32`. Abstract values are represented by their concretizations, which hold values of kind `I32` at `a`, so `entry_pairs_cover` is taken at `id`. The answer `[(x > 0, a < 0), (⊥, a > 0)]` covers the caller store with its first pair and the entered store with its second. Each pair's callee result, `a` published as the return value, is combined with that pair's own continuation; the union excludes the run's `y = 1`.

```
abbreviation up_x :: var_id where "up_x ≡ (nv_var (STR ''x''))"
abbreviation up_y :: var_id where "up_y ≡ (nv_var (STR ''y''))"
abbreviation up_a :: var_id where "up_a ≡ ScopedId (STR ''p'') (STR ''a'')"

definition unpaired_answer :: "(store set × store set) list" where
  "unpaired_answer =
     [({t. 0 < t up_x}, {t. t up_a ∈ ik_range I32 ∧ t up_a < 0}),
      ({}, {t. t up_a ∈ ik_range I32 ∧ 0 < t up_a})]"

definition pairwise_contribution :: "(store set × store set) list ⇒ store set" where
  "pairwise_contribution P =
     (⋃(q, e) ∈ set P.
        {combine_collect (ret_κ) (Some up_y) s0 (t(ReturnVar := t up_a))
         | s0 t. s0 ∈ q ∧ t ∈ e})"

theorem unpaired_entry_cover_unsound:
  defines "s ≡ (λ_. 0)(up_x := 1) :: store"
      and "s' ≡ (λ_. 0)(up_a := 1) :: store"
  shows "s' = call_enter (ret_κ) (CallEdge (Some up_y) [up_a] [TVar I32 up_x]) s"
    and "(∃(q, e) ∈ set unpaired_answer. s ∈ q)"
    and "(∃(q, e) ∈ set unpaired_answer. s' ∈ e)"
    and "¬ entry_pairs_cover id s s' unpaired_answer"
    and "∀u ∈ pairwise_contribution unpaired_answer. u up_y < 0"
    and "combine_collect (ret_κ) (Some up_y) s (s'(ReturnVar := s' up_a)) up_y = 1"
  by (auto simp: s_def s'_def unpaired_answer_def pairwise_contribution_def
                 entry_pairs_cover_def call_enter_CallEdge enter_state_def enter_frame_def
                 combine_collect_def kenv_of_def fun_eq_iff)

end
```